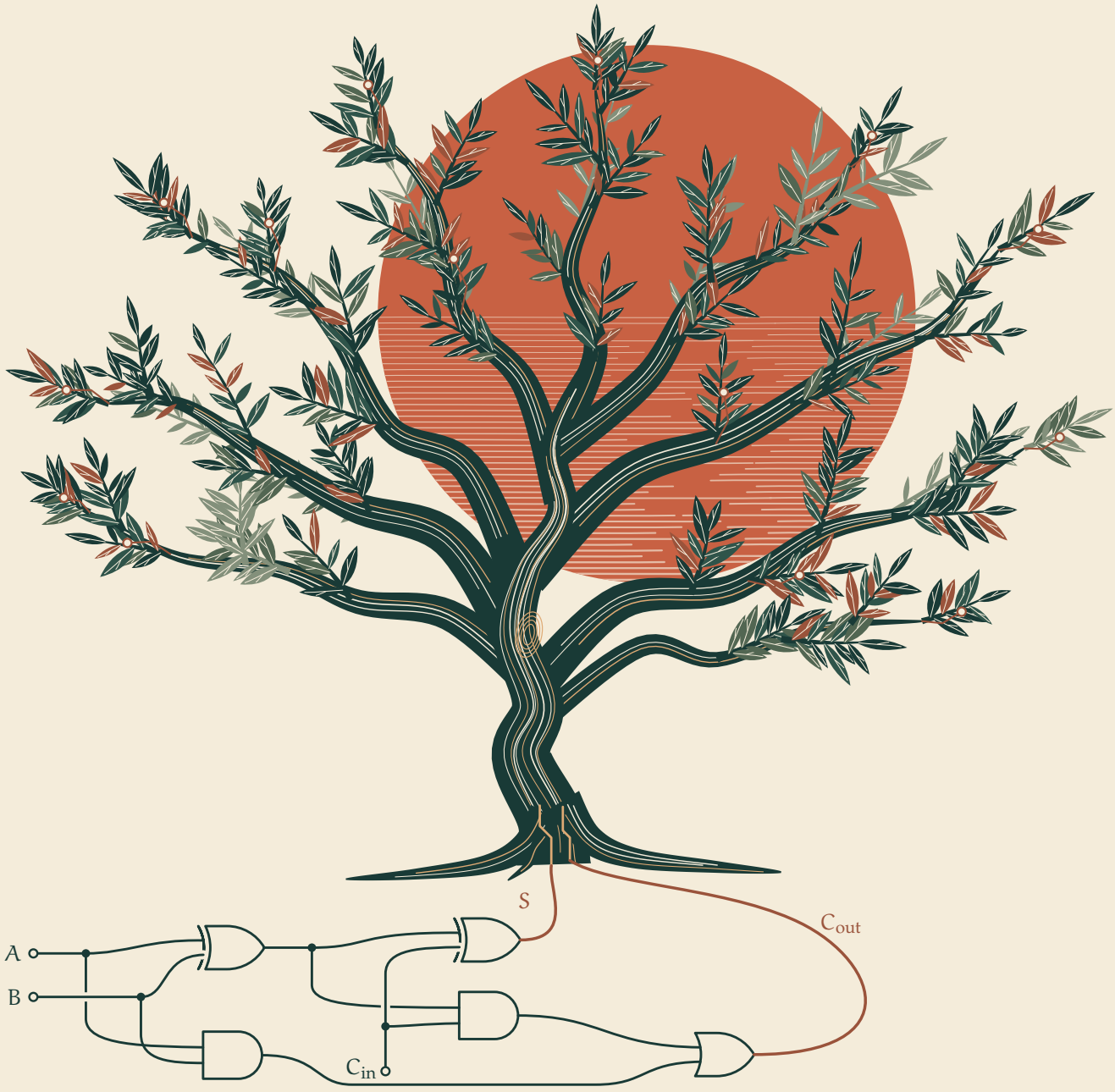
Marzio De Corato · Danilo Bruschi

Architecture *of* Computer Systems



From information representation to processors, operating systems, storage, and the physical layer

Revision 2.22.0
6 November 2026

Department of Computer Science “Giovanni Degli Antoni”

Università degli Studi di Milano

About this edition

Architecture of Computer Systems. Aldina edition 2.22.0, 6 November 2026.

The typography is based on ClassicThesis by André Miede and Ivo Pletikosić and ArsClassica by Lorenzo Pantieri. The Aldina design uses Palatino text, Iwona headings, Euler mathematics and Bera Mono code, with bordeaux and brass accents. Ivory panels distinguish worked examples and solutions.

The license declarations supplied with the original text are retained: original text and figures are CC BY-NC-SA 4.0; code examples are MIT-licensed. Third-party quotations, standards, software, fonts and package files remain under their respective terms. The source package's NOTICE records attribution and license references.

PREFACE

A computer system connects mathematical representations, physical devices and programs that share finite resources. Understanding its behaviour requires following those connections: a type constrains the interpretation of a bit pattern, an instruction transforms machine state, a memory access passes through a mapping, and an operating-system operation changes what an execution may do next.

This book develops those mechanisms through explanations, diagrams, calculations and code. Its six Parts form a continuous sequence. Information representation establishes the meaning of bytes and fixed-width operations. Machine-level programming exposes registers, addresses and binary interfaces. Processor organisation explains how logic and state realise an instruction set. Input/output connects the processor to events, interconnects and storage. Processes and concurrency introduce scheduling, shared-state invariants and liveness. Memory and filesystems develop locality, translation, persistence and protection.

The reader should be comfortable with elementary algebra and simple programs containing variables, conditions and loops. C notation is used to describe objects and interfaces; x86-64 assembly makes address calculations and state transitions explicit. Language, ABI, operating-system and hardware assumptions are stated where they determine a result. Y86-64 and several array-based translation examples use deliberately specified models whose complete rules appear in the text.

READING AND SOLVING

The numbered chapters develop the definitions needed by their examples and exercises. Examples occur beside the mechanism they explain. Worked problems combine those ideas, while end-of-chapter exercises provide space for an independent attempt before the solutions. Every exercise includes a worked answer, using a numerical derivation,

a register or memory trace, an execution schedule, an invariant or code, according to the problem.

For a calculation, record the quantities, their units and the assumptions before substituting values. For machine code, establish the initial registers and readable memory, then trace each instruction's effects. For concurrent execution, identify the shared state and the property that must remain true. A result should agree with the stated input and also survive a relevant boundary check.

Internal references connect related explanations. Appendix B provides compact technical tables and command references. Appendix C gives complete build-and-run examples for C, assembly and threads, including the conventions needed to turn calculation fragments into callable code. The exercises can also be followed on paper from their stated data.

MODELS AND SOURCES

A model makes selected behaviour explicit. An ideal pipeline, a serial memory-cost equation and a simulated page-table array each answer a particular question under stated assumptions. Their conclusions should be interpreted within those assumptions. The text identifies the additional mechanisms that matter when connecting a model to an implemented system.

Bibliographic references identify architectural specifications, language and operating-system interfaces, foundational papers and broader treatments. Bryant and O'Hallaron connect program behaviour to the machine and operating system; Tanenbaum and Bos develop operating-system abstractions; Hennessy and Patterson provide quantitative architectural analysis [1, 59, 50]. The definitions and methods required for the exercises are developed within this volume.

CONTENTS

Preface iii
1 Information, Representation, and Program Execution 1
1.1 The Computer System and the Life of a Program 2
Concept map: the layered system 15
1.2 Binary and Hexadecimal Representation 24
Concept map: number systems and encodings 36
1.3 Encoding Non-Numerical Data 45
Concept map: encoding as agreement 56
1.4 Signed Integers and Two's Complement 65
Concept map: signed integers 76
1.5 Boolean Circuits and Bit-Level Computation 86
Concept map: gates and bit operations 98
1.6 Fractional Numbers and IEEE 754 Floating Point 107
Concept map: floating point 120
1.7 Integrated Case Studies: From Bytes to Behaviour 129
2 Machine-Level Programming and the x86-64 Interface 149
2.1 Machine State, Registers, and Memory 150
Concept map: machine state & memory 167
2.2 Integer Arithmetic and Condition Codes 175
Concept map: integer arithmetic 188
2.3 Addresses, Arrays, and Data Layout 197
2.4 Bitwise Operations, Shifts, and Masks 215
Concept map: bit manipulation 231
2.5 Linux System Calls and the User–Kernel Boundary 239
Concept map: system calls 260
2.6 Procedures, Control Flow, and Reverse Engineering 268
Concept map: ABI & control flow 289
2.7 Stack Frames, Memory Safety, and Control-Flow Reuse 296
Concept map: stack & exploitation 313
Integrated map of machine-level programming 321
3 Processor Organisation, Timing, and Pipelining 323

3.1 Combinational Logic: From Truth Tables to the ALU 324
Concept map: combinational logic 341
3.2 Sequential Logic: State, Registers, and Memory 347
Concept map: sequential logic 365
3.3 The Y86-64 Instruction Set and Execution Stages 373
Concept map: Y86-64 encoding & stages 388
3.4 The Sequential Processor 395
Concept map: single-cycle CPU 410
3.5 Pipelining: Overlap, Throughput, and Hazards 417
A compact map of pipeline reasoning 430
3.6 The Five-Stage Pipelined Processor 438
Concept map: the five-stage pipeline 457
Integrated map of processor construction 464
4 Exceptional Control Flow, Input/Output, and Storage 465
4.1 Exceptional Control Flow and Protected Entry 466
Concept map 489
4.2 System Calls: Controlled Entry to the Kernel 498
4.3 Device Controllers, Polling, and Interrupts 516
4.4 Buses, Transactions, and Interconnection 541
4.5 Magnetic Storage and Access-Time Analysis 562
4.6 RAID and Solid-State Storage 582
4.7 Device Classes and Direct Memory Access 604
5 Processes, Threads, and Concurrent Execution 629
5.1 Processes, States, and Context Switching 630
5.2 Process Creation, Program Replacement, and Termination 646
5.3 CPU Scheduling and Time Sharing 665
5.4 Threads, Concurrency, and Parallel Execution 682
5.5 Shared State, Critical Sections, and Semaphores 699
5.6 Counting Semaphores and Classical Synchronisation Problems 718
5.7 Deadlock, Starvation, and Livelock 739
6 Memory, Filesystems, and Protection 759
6.1 Memory Hierarchies and Locality 760
6.2 Virtual Memory and Demand Paging 777
6.3 Address Translation in Code 796
6.4 TLBs and Multilevel Page Tables 819
6.5 Files, Names, and Allocation 839
6.6 Consistency and Crash Recovery 856
6.7 Protection, Encryption, and Auditing 873
A Consulting Architectural Specifications 889
A.1 Four source layers 890
A.2 Intel manual map 890

A.3 AMD manual map 891
A.4 System V AMD64 psABI map 891
A.5 Linux source map 891
A.6 A repeatable reading method 892
A.7 Six guided source-reading exercises 893
B Technical Reference 897
B.1 Laboratory and Command Reference 897
B.2 Linux Assembly Toolchain Reference 899
B.3 x86-64 Register and Convention Reference 901
B.4 Y86-64 Quick Reference 902
B.5 Datapath and Pipeline Control Reference 904
B.6 Exception and System-Call Reference 906
B.7 I/O and Storage Calculation Reference 908
B.8 Integrated I/O Traces 910
B.9 Process and Concurrency Quick Reference 917
B.10 Compact Glossary 918
C Building and Running the Examples 935
C.1 From a calculation fragment to a function 935
C.2 A checked translation function 936
C.3 A caller and its expected results 937
C.4 Checking structure layout 938
C.5 Threads, inspection and model traces 939
References and Further Reading 941

1

INFORMATION, REPRESENTATION, AND PROGRAM EXECUTION

A running program is the final stage of a chain of representations. Physical signals encode bits; bit patterns encode numbers, characters, addresses and instructions; files preserve those patterns; processes give them an execution context. Each layer supplies its own objects and rules, and errors arise when a pattern is interpreted under the wrong contract.

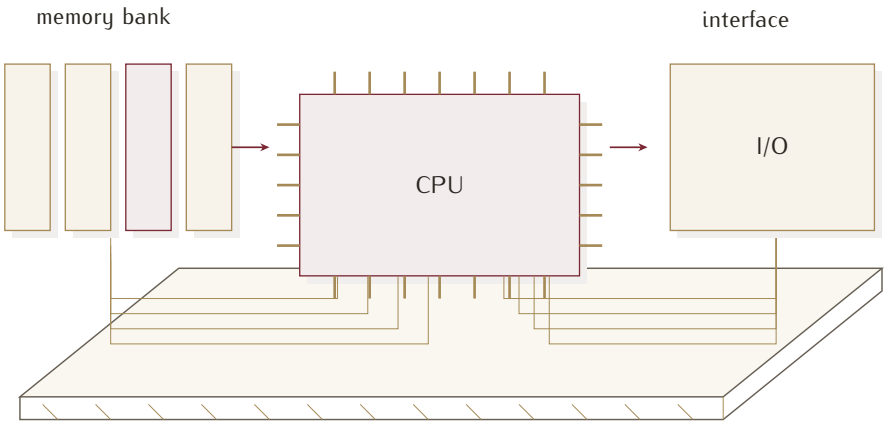
The discussion follows that chain from source code to process state, then develops the finite representations on which it depends. Binary and hexadecimal are used as working notations. Character encodings are related to communication history and to the Unicode model [5, 6]. Signed arithmetic is separated from the rules of the C language, while IEEE 754 illustrates how a fixed format trades range, precision and exceptional values [3, 7, 17]. Boolean operations connect stored representations to the circuits that transform them.

One model unifies the Part: a representation selects admissible bit patterns and assigns them meaning; an operation maps one admissible state to another; a language may impose rules stricter than the hardware. Keeping those levels distinct makes overflow, endianness, aliasing, parsing and rounding easier to analyse.

1.1 THE COMPUTER SYSTEM AND THE LIFE OF A PROGRAM

A source file sits on storage as a sequence of bytes. After compilation and a launch request, a processor eventually executes its instructions and a result appears. Between those observations, several components must agree: tools construct the executable, the operating system prepares an execution context, and hardware fetches instructions and moves data. Following one small program through that journey gives each component a concrete job and gives each failure a place to investigate.

A program unfolds through a physical system



Signals cross interfaces between components.

Figure 1.1: *The board view introduces the processor, memory bank and device interface as cooperating physical components. The later figures follow how code, addresses and I/O requests move through their logical contracts.*

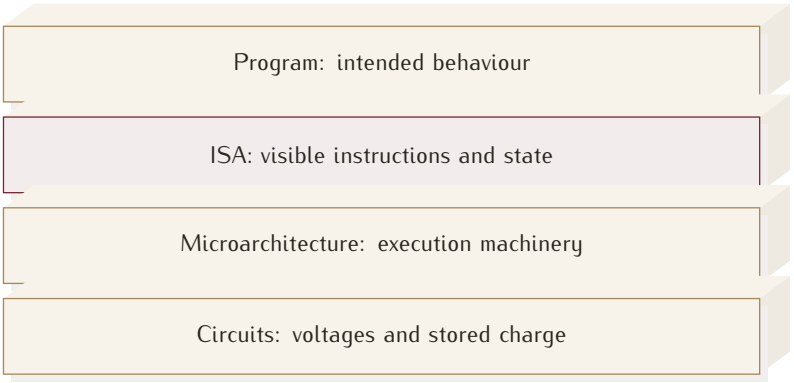
1.1.1 One program, several layers of meaning

Consider a file `hello.c` containing a call to `puts`. To the programmer it is C source text. To the filesystem it is a named sequence of bytes. To the compiler it is input governed by the C language. After translation it becomes instructions and data governed by an instruction-set architecture and an executable format. Once launched, it belongs to a

process whose memory and access to devices are mediated by the operating system. The same artefact is therefore never explained completely by one layer alone.

This layered view matters because abstractions hide detail without making the detail disappear. A C variable is easier to reason about than a collection of register transfers, yet its type still determines representation and arithmetic rules. A file is easier to use than a storage controller, yet reading it still moves data through caches, memory and I/O mechanisms. A process appears to execute independently, yet the kernel must multiplex finite processor and memory resources among many activities. Later parts of the book descend through these layers one at a time; this chapter establishes the interfaces that connect them.

One instruction rests on several contracts



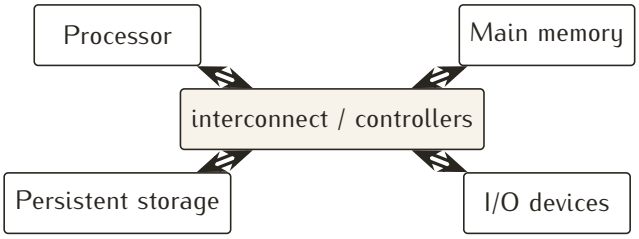
Each layer gives a different explanation of the same execution.

Figure 1.2: *Follow one instruction downward: its meaning is specified by the ISA, its execution is organised by the processor, and its physical implementation uses circuits. A change below a contract can preserve the behaviour above it.*

Three distinctions will recur throughout that descent. An *instruction-set architecture* (ISA) specifies the machine-visible instruction and state interface; a *microarchitecture* is one internal design that implements that ISA; and an *application binary interface* (ABI) specifies conventions that let separately compiled software cooperate on a particular platform. The operating system adds privileged mechanisms and policies above those contracts. A processor generation can change its pipeline while preserving the ISA, a compiler can change optimisation while preserving the ABI, and a kernel can replace internal data structures while preserving the system-call interface. Stable interfaces are what let the surrounding layers evolve independently.

1.1.2 The physical machine beneath the abstraction

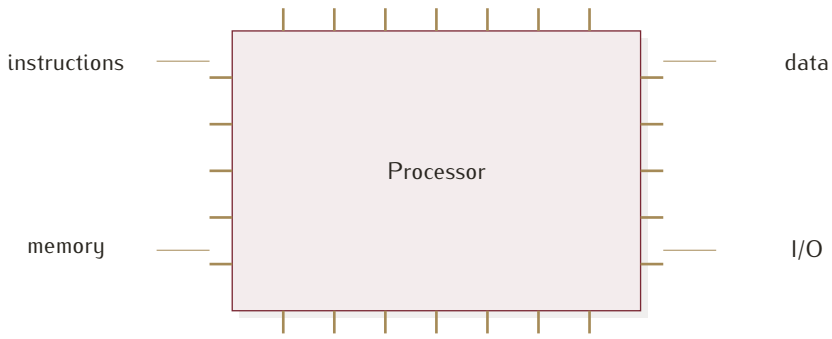
At the coarsest level, a computer system contains processors, main memory, persistent storage and I/O devices connected by interconnects and controllers. The processor transforms architectural state by executing instructions. Main memory holds code and data that must be accessed with comparatively low latency while programs run. Persistent storage retains information without power. Devices connect the system to displays, networks, sensors and other external state. These categories are useful abstractions; real platforms divide responsibility among packages, controllers, memory channels and point-to-point links in ways that are more complex than a single shared bus.



Power, reset and clock before the first instruction

Voltage regulators bring supply rails into their allowed ranges, reset logic holds state elements in a defined condition, and oscillators and phase-locked loops establish the clocks required by the platform. Power-good, reset and clock edges are physical signals with finite rise time and jitter rather than mathematical events.

A small machine with distinct responsibilities



The processor exchanges requests and data through interfaces.

Figure 1.3: *The chip outline makes the physical component visible while the four labels identify logical traffic. Instruction and data paths may share lower-level resources; the drawing separates their roles.*

Before reset release, software cannot rely on the ordinary architectural execution contract. Some boot failures and unstable systems therefore originate below the ISA, in power integrity, reset sequencing or timing rather than in program semantics [43, 42].

Performance introduces another hierarchy. Registers are small and close to execution units; caches keep recently useful data near the processor; DRAM offers much larger capacity at higher latency; persistent storage is larger again and generally slower. The exact ratios change across machines, but the principle is stable: moving data is not free, and a system gains performance when locality allows a small fast level to satisfy many accesses that would otherwise reach a slower one [25]. Later chapters treat caches, buses and storage in detail; here the hierarchy explains why two programs performing the same abstract computation can have very different execution times.

1.1.3 From source bytes to an executable file

The word *compilation* often hides several transformations. For an ordinary C program, the preprocessor first expands directives and headers into a translation unit; the compiler converts that program into assembly or another machine-level intermediate representation; the assembler encodes instructions and data in a relocatable object file; and the linker combines objects and libraries, resolves symbols and relocations, and constructs an executable image. The familiar sequence

$$\texttt{hello.c} \longrightarrow \texttt{hello.i} \longrightarrow \texttt{hello.s} \longrightarrow \texttt{hello.o} \longrightarrow \texttt{hello}$$

records a chain of representations, each consumed by a different tool.

A relocatable object may already contain genuine machine instructions and still not be directly executable. References can remain unresolved, final placement may be unknown, and the loader-facing segment structure and entry point may not yet exist. The linker resolves the references required at link time and constructs the executable structure; dynamic dependencies can remain for the runtime linker. Conversely, a finished executable is still only a file: nothing in its bytes is "running" until the operating system uses the executable format to construct a process image and transfers control into it.

It is useful to separate three moments. *Translation time* determines instructions, object metadata and symbol relationships. *Load time* establishes mappings, resolves or prepares dynamic dependencies and creates an execution environment. *Run time* is when processor, memory hierarchy, libraries, kernel and devices interact while the computation proceeds. Confusing these phases leads to category errors such as looking for a run-time stack inside a link-time symbol table or assuming that the address seen in a relocatable object must be the address used by a running process.

From an ELF file to the first user instruction

On a typical Linux/x86-64 system, the executable is described by ELF metadata. Program headers identify loadable segments and their requested access permissions, while

the System V AMD64 ABI specifies process-startup conventions. The Linux loader interprets this software format, creates the required virtual mappings and initial user stack, and selects an entry point. If a `PT_INTERP` entry requests a dynamic linker, control reaches that loader before the program's ordinary C startup sequence. The processor itself does not understand ELF; it understands page translations, permissions and architectural control transfers. Software interprets the executable format, while hardware enforces the memory and privilege mechanisms on which the loader relies [31, 30, 35, 26].

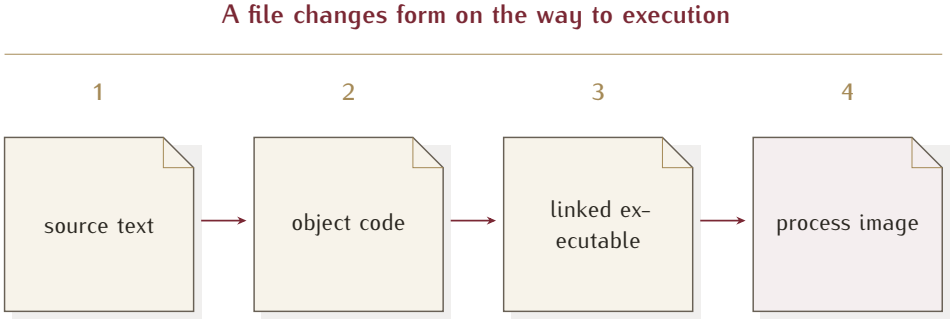


Figure 1.4: *Source bytes express a program in a language. Object files contain code and unresolved relationships. Linking forms an executable; loading establishes a runnable memory image. The arrows represent transformations, not elapsed-time proportions.*

A reproducible inspection of a small executable can compare `readelf -hW`, `readelf -lW`, `objdump -d -Mintel` and a system-call tracer. The useful question is which component owns each transition: linker, kernel loader, dynamic linker, C run-time startup or the program itself.

The source file at the beginning of this chain is already binary data in the literal sense that the filesystem stores bytes. An editor decodes those bytes as text; the compiler then interprets that text according to the source-language rules. Inspecting `hello.c` with `xxd` makes this boundary visible and prepares the representation questions of Chapters 1.2 and 1.3.

1.1.4 Names, files and the shell

A path is a name used to reach an object in a filesystem namespace. An absolute Unix path begins at the root, as in `/home/alice/project/hello.c`; a relative path is interpreted from the process's current working directory. The path is not the file object itself. Directories associate names with underlying objects, which is why renaming a file can change a directory entry without rewriting its contents, and why multiple hard links can name the same underlying file.

In the pathname exercises in this chapter, assume that every nonfinal component names an existing searchable directory, that no symbolic link changes the traversal, and that the namespace remains unchanged. The calculation determines the resulting pathname; the final object may still be absent. Actual lookup checks each component and can fail before reaching the final name. Shell expansion is a separate step: an unquoted `~` at the beginning of a shell word can expand to a home-directory pathname before any filesystem lookup.

The filesystem namespace also exposes device interfaces and live kernel information, as illustrated by `/dev` and `/proc`. Reading such an object can invoke a driver or generate a view of kernel state. The pathname and byte-oriented I/O interface remain familiar even when the contents have no persistent disk representation.

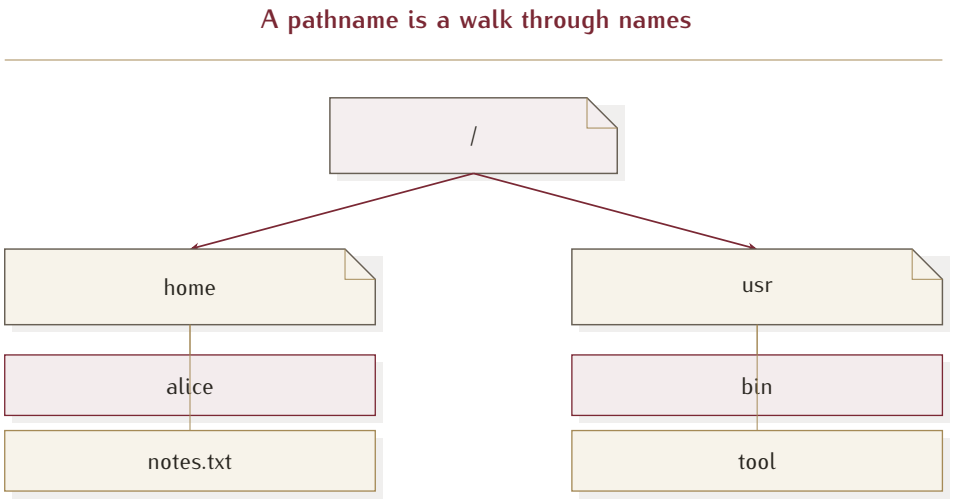


Figure 1.5: *A directory associates names with filesystem objects. In the illustrated paths, /home/alice/notes.txt and /usr/bin/tool are reached through successive directories. The spatial placement of names says nothing about disk locations.*

Path handling is therefore both a convenience and a security boundary. Untrusted components such as `..` or symbolic links can cause a textual path to resolve outside an intended directory, and a separate "check then open" sequence can race with filesystem changes. Later security discussions return to directory file descriptors and atomic kernel operations; for now the important point is that a path names an object through mutable namespace state rather than being the object itself.

The shell is an ordinary user-space program that turns command syntax into requests for operating-system services. When a user types `./hello`, the shell resolves a path and asks the kernel to execute the resulting file. Redirection changes which open file is attached to a program's standard input or output; a pipeline creates kernel-

managed communication channels and starts several processes whose byte streams are connected. Thus

```
cat access.log | grep ERROR | wc -l
```

looks like one command line but describes several processes and two pipes. The programs can remain largely unaware of where their input comes from because the operating system presents byte-oriented descriptors at the interface.

1.1.5 From executable file to process: a preview

A program is a stored description of computation; a process is an execution context created and managed by the operating system. The distinction will be developed fully in Part 5, so the present goal is only to establish the vocabulary. A process owns or refers to resources such as an address space and open files, and it has execution state that allows the kernel to stop and later resume its work.

Similarly, *virtual memory* is introduced here only as the abstraction through which a process uses addresses. It gives software a protected address-space view without requiring the program to reason directly about physical DRAM placement. The page tables, translation caches and policies that make that abstraction possible belong to later system layers; this opening chapter needs only the distinction between the address a program uses and the physical resources managed underneath it.

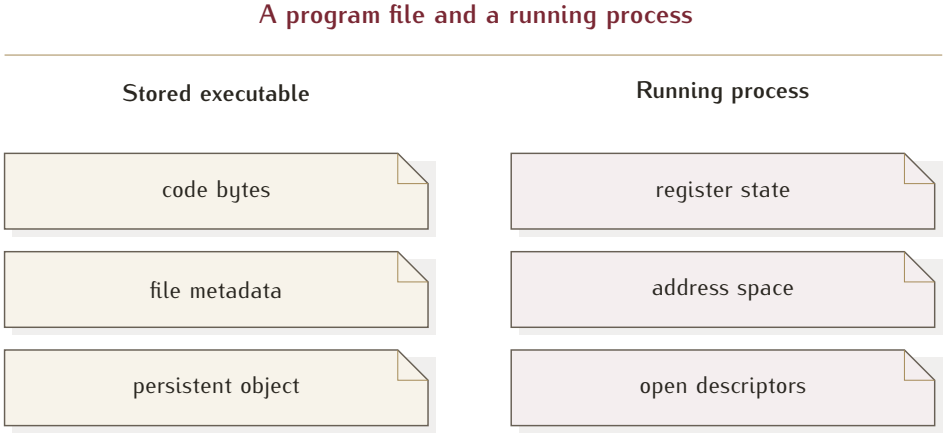


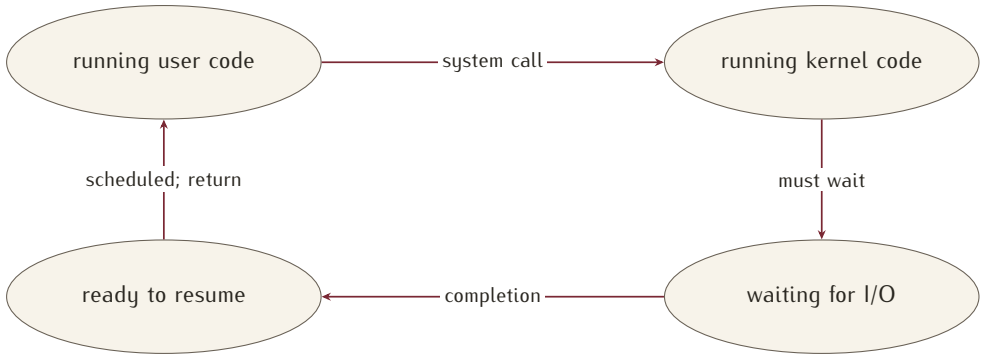
Figure 1.6: *A stored executable supplies material for execution. Each process has its own execution state and resource relationships. Shared executable code does not require identical register values or identical open descriptors.*

Multiprogramming means that several processes can exist and make progress over the same interval even when there are fewer hardware execution contexts than runnable activities. A process that could execute if given a logical processor is *ready*; one currently

assigned such a context is *running*; one waiting for an event such as I/O completion is *blocked*. These terms are deliberately introductory. Part 5 develops scheduling and context switching, including systems with simultaneous multithreading where one physical core can expose more than one logical processor. The scheduler chooses among runnable work; it does not create parallelism merely by changing labels in a state diagram.

The operating system's role can now be stated more precisely. It provides mechanisms – controlled privilege transitions, address translation, timers, I/O interfaces – and applies policies that decide how those mechanisms are used, such as which runnable task executes next or whether a process may open a file. Separating mechanism from policy is a recurring design principle because a stable mechanism can support several policies without changing the hardware contract.

A request can change where execution waits



Simplified blocking path; the return passes through kernel code.

Figure 1.7: *A blocked request suspends the calling execution while other work can run. Completion makes it runnable; scheduling and the return path restore user execution. The final arrow abbreviates those two distinct steps.*

Abstractions and resource management

A computer system can be read from two complementary directions. From the top down, the operating system presents stable abstractions such as processes, address spaces, files and streams. From the bottom up, it allocates processors, memory, storage and devices among competing activities. Tanenbaum and Bos use these two views – extended machine and resource manager – as the organising idea for operating systems [59].

The distinction clarifies why an abstraction is more than a convenient name. A process hides scheduling and protection state; a file hides device commands, block placement and recovery rules; virtual memory hides the physical placement and temporary absence of pages. Each abstraction is implemented by mechanisms that also enforce

sharing. CPU time is multiplexed in time, while memory and storage are largely multiplexed in space.

> **Caution**
>
> Each abstraction has costs and failure modes. A file operation can block, a virtual-memory access can fault, and a process can be descheduled. Its interface defines how the program observes and handles those events.

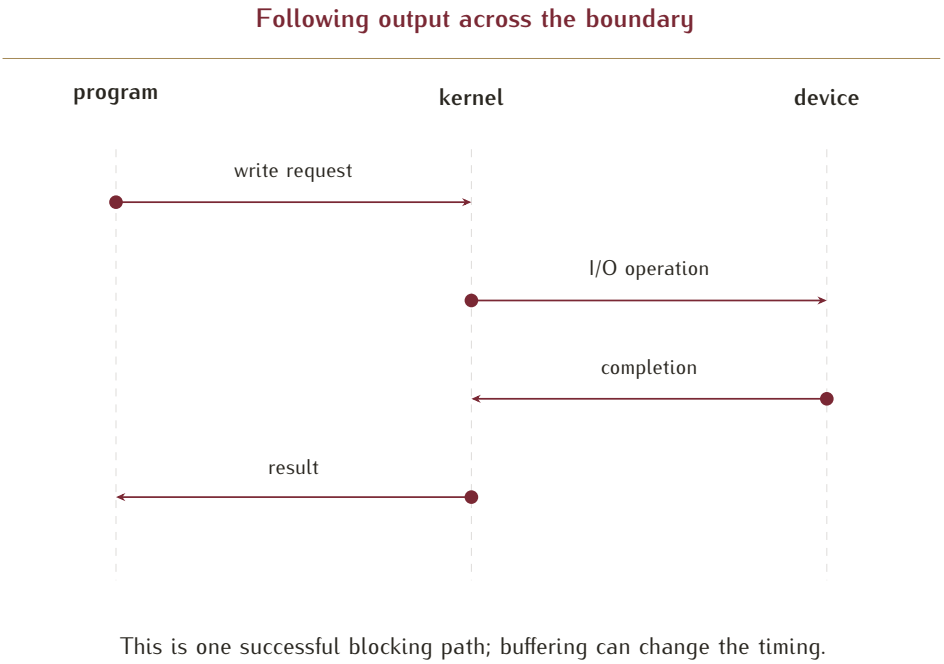


Figure 1.8: *The request and the result cross the user/kernel interface. Device completion belongs to a lower interface and need not coincide with visible output. A buffered write can return before the eventual physical transfer.*

> **Exercise 1.1.1 — Two readings of one event**
>
> A program reads 64 KiB from a file. Describe the event first as an abstraction seen by the program, then as resource management involving CPU time, page-cache memory, storage bandwidth and device completion. Identify which details are part of the contract and which are implementation choices.

Solution 1.1.1

The program requests up to 65,536 bytes from a file descriptor into a buffer. A successful call reports the number actually transferred; that number may be smaller than the request, for example at end of file. The caller must use the reported count and handle errors. Sequential reads also advance the open file description's offset by the transferred count. These are interface obligations visible to the program.
For a cache hit, the kernel can obtain the bytes from the page cache, check the destination mapping and copy the data into the process buffer. CPU instructions, memory bandwidth and cache capacity are consumed; a device request may be unnecessary. For a cache miss, the kernel arranges storage I/O and can block the thread while the device obtains the missing blocks. Device completion makes progress possible; the scheduler determines when the thread runs again. Read-ahead may fetch additional bytes for later requests.
The number of device commands, cache replacement policy, physical block placement and delay before rescheduling are implementation choices. A 64 KiB request therefore specifies a logical transfer, without fixing the number of physical reads or guaranteeing a single uninterrupted use of the CPU.

1.1.6 Tracing `hello` across the system

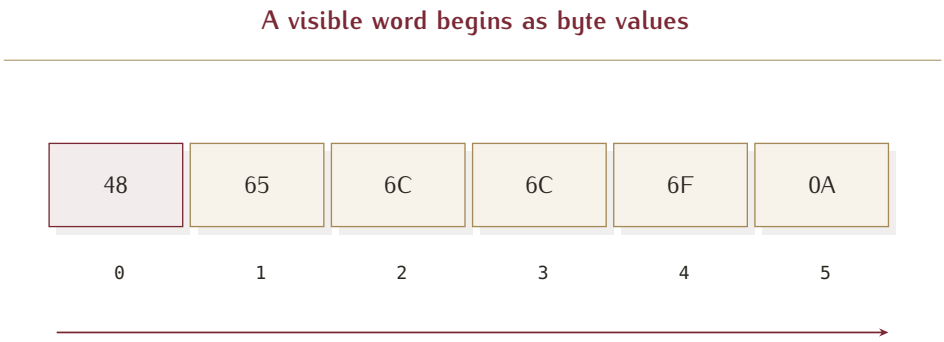


Figure 1.9: *The six hexadecimal values encode five letters and a newline. A write of six bytes transmits this entire sequence; a C string terminator, if stored after it, would be a separate byte.*

The small program used throughout this chapter can now be followed without collapsing its layers. Its source characters are encoded bytes in a file. Translation tools convert those bytes into object code and then an executable. The shell names the executable and requests a kernel service. The loader interprets executable metadata and constructs an address space. User-mode startup code eventually calls `main`; machine instructions

manipulate registers and memory; a library or system call carries the output bytes toward a terminal; and the operating system coordinates access to the underlying device. The single visible line `hello` is the end of a chain of representations and control transfers.

Laboratory 1: From source to process

Use the program below to follow translation from source to process. Give commands for each artefact, classify the artefacts and explain the expected output. For process inspection, replace the return with `sleep(30); return 0;` and add `unistd.h`; explain which information a live `/proc/PID/maps` supplies.

```c
#include <stdio.h>
int main(void) { puts("hello"); return 0; }
```

Worked solution

Laboratory 1: solution

Save as `hello.c`, then run:

```
gcc -E hello.c -o hello.i
gcc -S -O0 hello.i -o hello.s
gcc -c hello.s -o hello.o
gcc hello.o -o hello
file hello.c hello.i hello.s hello.o hello
./hello
```

The preprocessor consumes text C and produces expanded text C. The compiler consumes that translation unit and emits assembly text. The assembler emits a binary relocatable object containing instructions, symbols and relocation information. The linker combines objects and libraries into an executable file; the loader uses its program headers to construct an execution image. Only the linked executable is the directly runnable artefact in this sequence. Running it prints `hello` and a newline, then exits successfully. For the sleeping variant, `./hello &` starts it in the background; the shell's `$!` is its PID. Use `ps -p "$!"` and `cat /proc/"$!"/maps` while it is alive. Maps reports mapped address intervals, permissions and backing objects; exact addresses and library paths depend on the run. It describes a process instance, whereas the executable remains a file.

Return to the laboratory

This first trace is intentionally broader than it is deep. Filesystem naming, executable loading, system calls, process state and virtual addresses have been introduced only far enough to locate them in the system. Later chapters develop each mechanism; the recurring `hello` example will make it possible to descend through those layers without inventing a new world every time.

From mechanical calculation to the executable process

Timeline

Year	Machine, standard or event	Architectural significance
1642	**Pascaline**	Mechanical carry automates addition.
1948	**Manchester Baby**	A program executes from writable electronic memory.
1964	**IBM System/360**	One documented ISA spans a compatible machine family.

Origins and early engineering constraints

The history of a modern program begins before electronic computers and before the word *software* existed. Blaise Pascal built his first adding machine in 1642 to assist his father's tax calculations; Gottfried Wilhelm Leibniz demonstrated a stepped-drum calculator in the 1670s that could automate repeated addition for multiplication. In both machines the procedure was embodied in gears, carries and the operator's hands. Joseph-Marie Jacquard's loom, placed in industrial service at the beginning of the nineteenth century, introduced a different engineering idea: punched cards could select a repeatable sequence of actions without rebuilding the mechanism. Charles Babbage adopted that separation in the Analytical Engine, conceived in 1834. His plans distinguished a "store" for numbers from a "mill" for operations and used cards for operations, variables and control. Ada Lovelace's 1843 notes described how such a machine could manipulate symbols according to rules, but neither Babbage nor Lovelace saw a complete Analytical Engine operate. The design matters because it separated the description of a computation from the mechanism that carried it out, while its failure to become a working service warns against confusing an architectural proposal with an implemented computer [86, 67, 68].

Punched media first achieved large-scale practical success in data processing rather than general computation. Herman Hollerith's electromechanical tabulating system accelerated the 1890 United States census by representing facts about each person as holes in a card. The cards encoded data records. Their use created an industrial ecosystem of punches, sorters, tabulators and standard record layouts.

During the 1930s and early 1940s, programmable calculators such as Konrad Zuse's Z3 used punched film to supply an instruction sequence. The Z3, completed in 1941, used about 2,600 relays and binary floating-point arithmetic, yet its program was external to the working memory. This distinction is essential: automatic sequence control is not the same as a stored-program computer [88, 67].

The first large electronic machines accelerated arithmetic without immediately solving programming. ENIAC was completed in 1945 and publicly unveiled in February

1946. Its vacuum-tube arithmetic was electronic and decimal, but the original programming method required operators to configure switches and connect function tables and accumulators with cables. A new calculation could therefore involve days of physical setup. John von Neumann's June 1945 *First Draft of a Report on the EDVAC*, building on work by the Moore School group, described instructions represented numerically in the same high-speed memory used for data.

That proposal did not by itself build the first stored-program machine, and the familiar label "von Neumann architecture" compresses the contributions of many engineers, including J. Presper Eckert, John Mauchly, Herman Goldstine and others. It nevertheless stated the decisive contract: an instruction can be fetched, decoded and modified because it is a memory word rather than a wiring pattern [87, 22, 69].

The contract became operational in Manchester and Cambridge. On 21 June 1948, the University of Manchester Small-Scale Experimental Machine, designed by Frederic C. Williams and Tom Kilburn, executed Tom Kilburn's 17-instruction highest-factor program from Williams-Kilburn tube memory. The machine was deliberately small: it existed chiefly to prove that a cathode-ray-tube store could hold both instructions and data.

On 6 May 1949, Maurice Wilkes's EDSAC at the University of Cambridge printed a table of squares. EDSAC then developed into a practical computing service, with "initial orders" loaded at startup and a library of subroutines that users could reuse. These dates mark two different achievements: Manchester demonstrated the stored-program principle; Cambridge turned that principle into a regular scientific service [89, 91, 70].

Once programs lived in memory, a second stack of machinery appeared above the processor. Assemblers replaced numeric opcodes with symbolic mnemonics and labels. Linkers combined independently assembled routines and repaired address references. Grace Hopper's A-0 system, developed for the UNIVAC in 1951–1952, treated reusable routines as named resources that a loader could collect.

John Backus's IBM team delivered the first FORTRAN compiler for the IBM 704 in 1957, demonstrating that a high-level notation could produce code acceptable to scientific users who cared intensely about performance. Batch monitors such as GM-NAA I/O, placed in service in 1956 for the IBM 704, automated job succession. The Compatible Time-Sharing System, first demonstrated at MIT in 1961, allowed several users to interact with one machine. In each step, work previously performed by an operator became a documented software service [117, 110, 68].

Standardisation and present-day architecture

Architecture and operating systems then made programs portable across machines and protected from one another. IBM announced System/360 on 7 April 1964 as a compatible family spanning a wide performance range. The same architectural program could run on different implementations because the instruction set, registers, data formats and interruption behaviour were defined independently of a particular circuit.

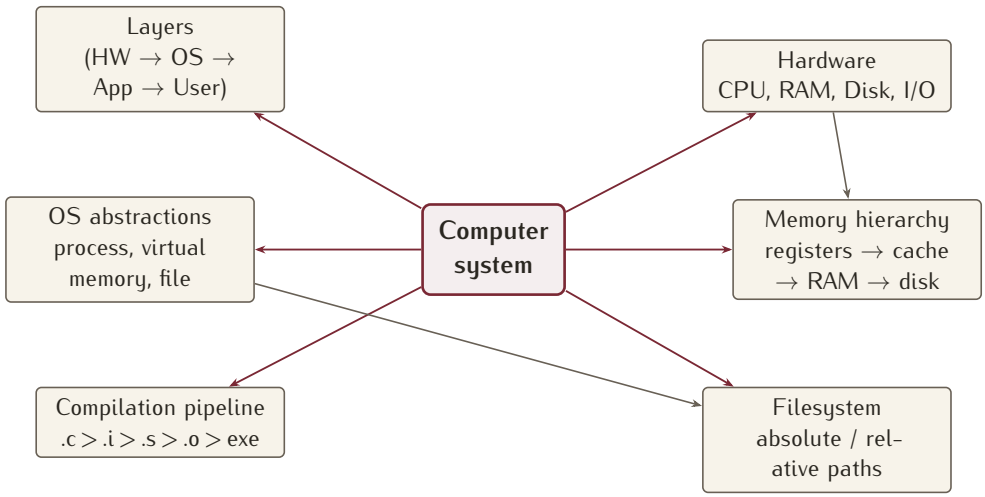
UNIX began at Bell Labs in 1969; the 1974 paper by Dennis Ritchie and Ken Thompson described a process, file and system-call model that could be implemented on more

than one computer. C made that portability practical while preserving explicit access to addresses and machine-sized objects. The result was a stable boundary at which hardware detail could be named and controlled [63, 74, 19, 18].

These developments established the interfaces used by the earlier program trace: executable formats describe stored code, ABIs specify binary cooperation, and operating systems construct protected execution contexts. Each interface allows the implementation on either side to evolve while preserving the documented behaviour [31, 30, 26, 27, 35].

Boot loaders, executable formats and process startup still preserve the old separation between stored program, operator-controlled loading and automatic execution.

Concept map: the layered system



Reading the map. A computer system is understood along two axes: the static *structure* (layers, hardware blocks, the memory hierarchy that ranks them by speed/size) and the dynamic *life of a program* (source travels through the compilation pipeline, then runs under OS abstractions, reaching files through the filesystem). The two axes meet at the hardware: the memory hierarchy is where the OS's virtual-memory abstraction is ultimately realised.

1.1.7 Worked problems

Layer trace

Question. When a text editor saves a document through an ordinary buffered file write, which components carry the request to storage?
The editor supplies the new bytes to a library routine. The library invokes a kernel interface such as `write`. The kernel checks the descriptor and buffer, and

Worked example (continued)

the filesystem updates the corresponding cached file data. A driver later translates the required storage operations into controller commands; the controller transfers data to the device.

$$\text{editor} \longrightarrow \text{library} \longrightarrow \text{kernel/filesystem} \longrightarrow \text{driver} \longrightarrow \text{controller/device}.$$

These arrows describe responsibility for the request. The return from a buffered write can precede durable device completion: acceptance into a cache and survival after power loss are separate events. A read follows the same interfaces, with the requested data travelling back towards the application.

Path resolution

Question. Starting from `/home/lee/project`, resolve `../data/./raw.bin` under this chapter's pathname assumptions.
Process the components in order. The initial `..` reaches `/home/lee`. The name `data` selects `/home/lee/data`; `.` leaves that directory unchanged. Finally, `raw.bin` selects `/home/lee/data/raw.bin`.
The intermediate directory after every component provides a check on the calculation. Starting instead from `/srv/app/logs`, the path `../tmp/./run.log` passes through `/srv/app`, then `/srv/app/tmp`, and ends at `/srv/app/tmp/run.log`. Actual lookup also requires the directory components and permissions assumed by the model.

Build diagnosis

Question. In a fresh build, a source file produces an object file. The linker then reports an undefined external symbol and produces no executable. Which stages have completed?
The new object file demonstrates that preprocessing, compilation and assembly completed. Its relocation or symbol information still refers to a definition that must be supplied by another object or library. The linker cannot find that definition, so symbol resolution prevents completion of the executable. Check the files supplied to the linker and the spelling and linkage of the missing name.
Loading and execution occur after a successful link. A compiler syntax diagnostic would locate the failure earlier, before the object file is produced. Inspect timestamps or use a clean output directory when diagnosing a real build, because an object left by an earlier successful run provides no evidence about the current attempt.

Shell composition

Question. What does `sort names.txt | uniq -c` connect, and how does it count repeated names?
The shell creates a pipe and connects `sort`'s standard output to `uniq`'s standard input. Sorting places equal lines next to one another. The `-c` option then makes `uniq` print the size of each adjacent group along with its line.
For a complete example, run:

```
printf 'b\na\na\n' | sort | uniq -c
```

Sorting produces `a`, `a`, `b`. The first group has two lines and the second has one, so the output contains count `2` with name `a`, followed by count `1` with name `b`. Leading padding depends on the command's formatting. Plain `uniq` would emit each group once without its count; omitting `sort` would leave separated repetitions in different groups.

Exercise 1.1.2 — Classify the objects

Place a GPU driver, a spreadsheet, an SSD controller, and the user in the abstraction stack.

Worked solution

Classify each object by the work it performs. The SSD controller is physical circuitry, so it belongs to hardware. The GPU driver is executable software that manages a device through operating-system interfaces. The spreadsheet is an application that uses those services. The user supplies goals and input through applications. Reading the stack upward therefore gives controller, device/system software, application and user. The fact that a driver controls hardware does not change the driver's own representation as software.

Exercise 1.1.3 — Resolve a path

Current directory: /srv/project/lab. Resolve ../../tmp/x.

Worked solution

Start at `/srv/project/lab`. The first `..` removes `lab`, leaving `/srv/project`. The second removes `project`, leaving `/srv`. Append `tmp` and then `x`. The resolved path is `/srv/tmp/x`. The two parent components must both be processed before appending the remaining names. This is a lexical path calculation under the stated directory model; existence and permissions are separate checks.

Exercise 1.1.4 — Choose the command

Which shell mechanism appends output without replacing the file?

Worked solution

Use `>>`. The shell opens the destination for appending and connects the command's output descriptor to it. Existing bytes remain, and subsequent writes are appended. For example, `printf 'new\n' >> log.txt` appends a line. A single `>` normally opens the destination with truncation, which would discard its previous contents. The redirection is performed by the shell before the command writes its output.

Exercise 1.1.5 — Locate the fault

main.c becomes main.o, but the executable is not produced because foo is unresolved. Which stage failed?

Worked solution

The failure is at linking. The object file `main.o` already exists, so preprocessing, compilation and assembly have completed far enough to produce that object. Its unresolved reference to `foo` must be matched to a definition in another object or library. The linker cannot find a suitable definition and therefore cannot finish the executable. Check that the object defining `foo` or the required library is included, and that the symbol name and linkage agree.

Summary

This chapter follows one chain: intention → source text → translated bytes → executable file → process state → physical transitions. Every later topic selects one link and studies it more closely. Files and paths describe where representations are stored. Compilation explains how one representation becomes another. Processes and virtual memory explain how the operating system presents a private machine to each running program.

1.1.8 Exercises

Exercise 1.1.6 — Layer diagnosis

A student says: "The compiler is part of the CPU because it produces machine instructions." Identify the error and place the compiler, assembler, linker, loader and CPU in the correct layers.

Go to the solution →

Exercise 1.1.7 — Follow the paths

The working directory is `/home/ada/project/lab`. Evaluate the absolute path denoted by each expression: (a) `../notes.txt`; (b) `./src/../data/x.bin`; (c) `/tmp/../var/log`; (d) `../../ada/project`.

Go to the solution →

Exercise 1.1.8 — Process states

A process issues a disk read, the disk is slow, and another ready process is available. Describe the state transitions for both processes until the read completes.

Go to the solution →

Exercise 1.1.9 — Translation artefacts

Classify `main.c`, `main.i`, `main.s`, `main.o` and the linked executable according to: human-readable text, contains native machine code, has unresolved references, directly executable.

Go to the solution →

Exercise 1.1.10 — Pipeline semantics

Explain why `grep ERROR server.log | wc -l` does not require `grep` to know that its output is consumed by `wc`.

Go to the solution →

Exercise 1.1.11 — Hard and symbolic links

A file has two hard-link names, `a` and `b`, and a symbolic link `s` whose stored target is `a`. What remains usable after `a` is removed? What changes after `b` is removed as well?

Go to the solution →

Exercise 1.1.12 — Observing the compiler driver

Save `int main(void){return 0;}` as `empty.c`, then inspect `gcc -v empty.c -o empty`. Identify the compiler, assembler and linker roles in the driver output. Explain why their executable paths can vary while the sequence of transformations remains the same.

Worked solution

Exercise 1.1.13 — A process-launch timeline

Draw a timeline showing a shell process, a child process and the kernel during program launch and termination.
Worked solution

Exercise 1.1.14 — Information generated by a virtual filesystem

Explain why a file in `/proc` can report current process information even though it is not stored persistently on disk.
Worked solution

Exercise 1.1.15 — Pathnames and command execution

Compare `ls file`, `ls ./file` and `./file`. Which are pathnames, and when is execution attempted?
Worked solution

1.1.9 Solutions

The Computer System and the Life of a Program

Solution 1.1.6

The compiler, assembler and linker are system-software programs. They execute as ordinary processes under the operating system. The loader is partly a kernel service and partly, for dynamically linked programs, user-space runtime machinery cooperating with the kernel. The CPU is hardware. Producing instructions does not make the producer hardware, just as a text editor producing C source is not part of the C language.

← Return to the exercise

Solution 1.1.7

(a) Climb from `lab` to `project`: `/home/ada/project/notes.txt`. (b) `.` keeps the same directory, `src/..` returns to `lab`: `/home/ada/project/lab/data/x.bin`. (c) Starting from root, `tmp/..` returns to root: `/var/log`. (d) From `lab`, one `..` reaches `project`, the second reaches `ada`; appending `ada/project` gives `/home/ada/ada/project`. The final result is syntactically valid even though it may not exist.

← Return to the exercise

Solution 1.1.8

The reading process moves from running to blocked because it cannot continue without data. The scheduler dispatches a ready process, which moves from ready to running. When the device signals completion, the blocked process becomes ready; it does not necessarily run immediately. At a later scheduling decision, it may move from ready to running, finish the kernel read path and then return to user code after the system call.

← Return to the exercise

Solution 1.1.9

`main.c` and `main.i` are C text; neither contains native machine code. `main.s` is assembly text; it symbolically describes machine operations but is not itself machine code. `main.o` is binary and contains machine code plus metadata and usually unresolved/relocatable references; it is not directly executable. The linked executable is binary, contains machine code and sufficient metadata for the loader, although dynamic-library references may still be resolved during loading or first use.

← Return to the exercise

Solution 1.1.10

The shell creates a pipe and arranges file descriptors before executing the programs. `grep` writes to its standard-output descriptor exactly as it would write to a terminal or file. The kernel routes those bytes into the pipe. `wc` reads from its standard-input descriptor. The uniform file-descriptor interface decouples the programs and enables composition.

← Return to the exercise

Solution 1.1.11

After removing `a`, `b` still names the underlying file, so its data remains accessible. The symbolic link `s` is dangling because its stored path `a` no longer resolves. After removing `b`, no hard-link name remains; once no open descriptor, memory mapping or other live kernel reference retains the file, the filesystem may reclaim its storage. `s` remains a symbolic-link object but still points to a nonexistent path.

← Return to the exercise

Solution 1.1.12

Run `gcc -v` on a small source file and read the commands printed by the driver. On a typical GNU toolchain, `cc1` performs C compilation, `as` assembles the generated assembly, and `collect2` invokes the system linker, usually `ld`. Report the sequence your installation displays, since executable paths and tool names can differ across platforms.

Return to the exercise

Solution 1.1.13

Use a simple foreground command implemented with `fork`, `execve` and `waitpid`. Read the following timeline from top to bottom; each row describes the next relevant event.

Shell	Child	Kernel
Parse command; call `fork`	Created from the shell	Create the child execution context
Call `waitpid`; block	Set up redirections; call `execve`	Install descriptors and replace the child's image
Waiting	Run startup and `main`	Schedule execution and service requests
Waiting	Finish; request termination	Record exit status; make the waiter runnable
Resume `waitpid`; collect status	No longer executing	Reap the terminated child

Worked solution (continued)

The parent can enter `waitpid` before or after the child reaches any particular step. If termination has already been recorded, the wait can return without blocking. Redirection is shown in the child so that the interactive shell keeps its own descriptors. Other shell implementations may use a spawn interface; the creation, image replacement and status-collection responsibilities still apply.
Return to the exercise

Solution 1.1.14

`/proc` is a pseudo-filesystem. Its directory entries and file contents are generated by kernel code from current process and kernel data structures when they are opened or read. The pathname behaves like a file interface, but no persistent disk block needs to contain the reported text.
Return to the exercise

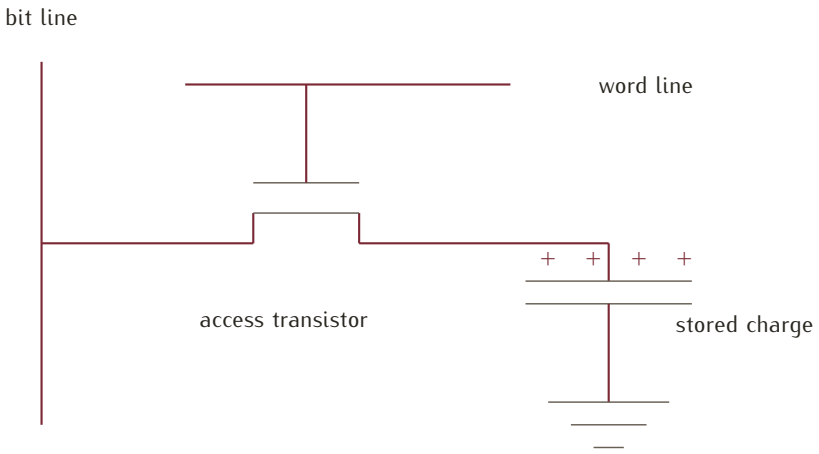
Solution 1.1.15

Assume that `ls` denotes an external command and is not replaced by an alias or shell function. In `ls file`, the shell searches `PATH` for the command `ls`; `file` is a relative pathname passed as an argument. In `ls ./file`, `ls` is still searched, while `./file` explicitly names the current-directory entry. In `./file`, the command itself contains a slash, so the shell attempts to execute exactly that pathname; execution succeeds only if permissions and executable format or interpreter rules permit it.
Return to the exercise

1.2 BINARY AND HEXADECIMAL REPRESENTATION

A debugger displays the byte `0xD8`. Expanding it gives `11011000`: eight positions, each carrying a power-of-two weight. The same expansion exposes a sign bit, a field inside a protocol, or part of an address, depending on the agreed interpretation. Binary and hexadecimal provide two readable views of this finite state. Keeping the width visible makes conversion, byte order and wraparound follow from the representation.

A stored bit has a physical carrier



A word line selects the access device; a bit line exchanges the cell state.

Figure 1.10: *This schematic DRAM cell connects a storage capacitor to a bit line through an access transistor controlled by a word line. The drawing introduces the physical state behind a bit; voltage thresholds and sensing determine its digital interpretation.*

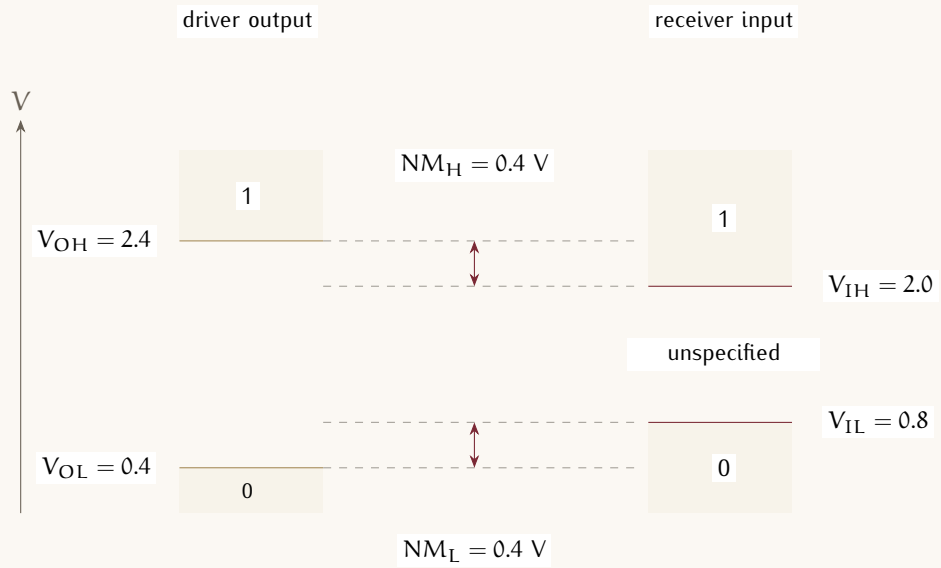
1.2.1 From physical states to binary symbols

A bit belongs to the abstract set $\{0, 1\}$. A physical implementation stores charge, voltage, current or another state that a receiver interprets according to an electrical interface.

A bit occupies a range of voltages

A receiver guarantees logic 0 below its specified low-input limit V_{IL} and logic 1 above its high-input limit V_{IH}. Between them, the logic value is unspecified. A driver guarantees a maximum low-output voltage V_{OL} and a minimum high-output voltage V_{OH} under stated conditions. Their differences give the static noise margins [140]:

$$NM_L = V_{IL} - V_{OL}, \qquad NM_H = V_{OH} - V_{IH}.$$



Worked check. Use hypothetical limits $V_{OL} = 0.4$ V, $V_{IL} = 0.8$ V, $V_{IH} = 2.0$ V and $V_{OH} = 2.4$ V. Both margins are 0.4 V. A +0.3 V disturbance on the worst permitted low output produces 0.7 V, still within the low-input guarantee. A +0.5 V disturbance produces 0.9 V, outside that guarantee. This last result establishes loss of a guarantee; it does not identify which bit a particular receiver will produce. The four values belong to an illustrative interface. Device data sheets specify the real limits across supply, temperature and loading conditions. Restoring signals towards valid ranges allows many physical components to cooperate using the abstraction of exact bits.

For a deliberately simplified exercise, suppose an ideal comparator uses a threshold of 1.0 V and applies the rule "above the threshold → 1, at or below it → 0". The sequence 0.4, 1.4, 0.9, 1.8 V therefore becomes `0101`. Real inputs are more complicated because low, high and indeterminate regions replace the single ideal threshold, but the exercise makes the abstraction boundary visible: continuous electrical quantities are being interpreted as discrete symbols.

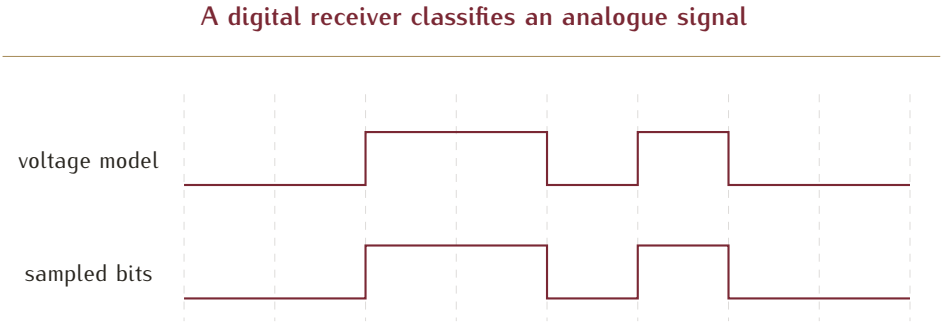


Figure 1.11: *The square waveform depicts the logical abstraction after classification into valid levels. Real inputs have voltage ranges, transition times and an unspecified region. The earlier noise-margin diagram supplies the electrical conditions behind this idealisation.*

The same boundary explains why increasing a link's nominal clock does not automatically increase reliable bandwidth. Physical signals need finite time to move between valid regions, and faster edges are more sensitive to reflections, crosstalk, attenuation and timing uncertainty. Modern links therefore use controlled impedance, differential signalling, clock recovery and error detection precisely so that the higher-level model can continue to speak in exact bits.

1.2.2 Positional notation and the role of hexadecimal

A base-b numeral with digits $d_{n-1} \dots d_1 d_0$ denotes

$$\sum_{i=0}^{n-1} d_i b^i, \qquad 0 \leqslant d_i < b.$$

Decimal is the familiar case $b = 10$; binary uses $b = 2$ and digits 0–1; hexadecimal uses $b = 16$ and digits 0–9 followed by A–F. Once this positional rule is fixed, conversion expresses the same mathematical value with different powers of a base.

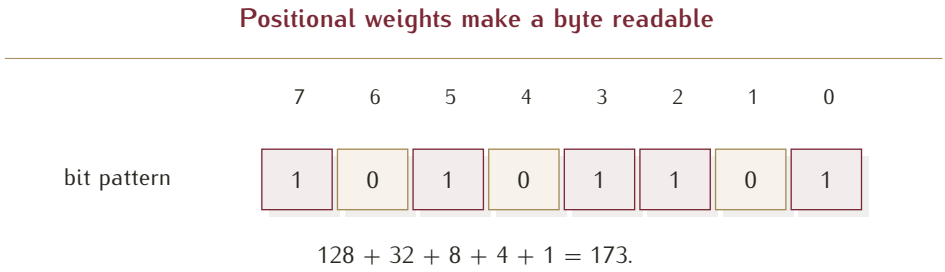


Figure 1.12: *Read bit positions from seven on the left to zero on the right. Each set bit contributes its power-of-two weight. The five highlighted cells therefore represent the unsigned value 173.*

For example,

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10},$$

while

$$\texttt{0x2F} = 2 \cdot 16 + 15 = 47_{10}.$$

Repeated division by the target base extracts the digits of a non-negative integer from least significant to most significant: if $n = bq + r$, then r is the next digit and q is the value still to be represented. Converting 173 to binary therefore yields remainders that, read in reverse order, form 10101101_2; the independent check $128 + 32 + 8 + 4 + 1 = 173$ confirms the result. The same method in base 16 converts 1000_{10} to `0x3E8`. In the reverse direction, Horner's method reads digits from left to right, starting with $v = 0$ and repeatedly computing $v \leftarrow bv + d$. For `0x3E8`, the intermediate values are 3, $16 \cdot 3 + 14 = 62$, and $16 \cdot 62 + 8 = 1000$.

Hexadecimal becomes especially useful because $16 = 2^4$. Each hex digit names exactly one nibble, so conversion between binary and hexadecimal is local:

$$1101\ 0110_2 = \texttt{0xD6}.$$

No arithmetic is required after the bits have been grouped. This exact alignment is why hexadecimal dominates debuggers, memory dumps, masks and machine-code listings: it shortens a bit pattern while preserving its internal boundaries. Leading zeros may therefore matter even when they do not alter a mathematical value. The number 47 can be written 101111_2, but an eight-bit object holding that value is $00101111_2 = \texttt{0x2F}$; the width tells us what storage object we are looking at.

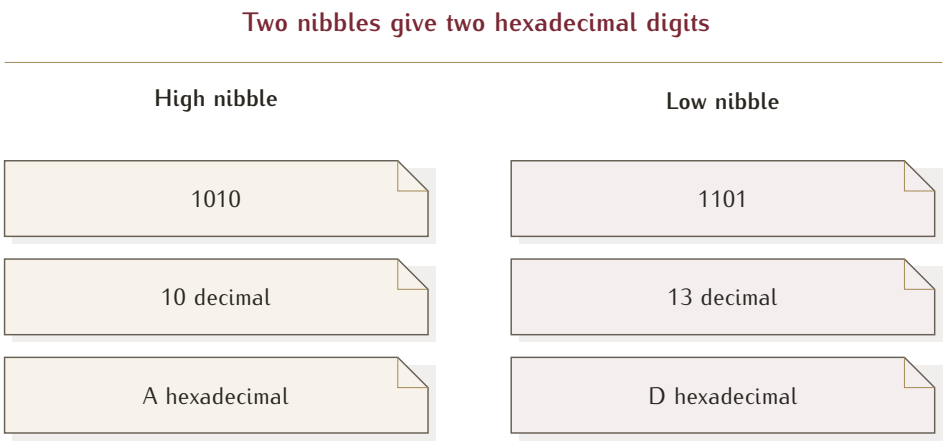


Figure 1.13: *Hexadecimal groups the byte into two four-bit pieces. Grouping preserves the bit pattern and its width. It does not determine whether a later operation interprets the byte as signed, unsigned, text or flags.*

Width before value

With w bits there are exactly 2^w distinct patterns. An unsigned interpretation maps them to 0 through $2^w - 1$, but another contract may map the same patterns to signed integers, characters, instructions or flags. Thus the byte `0xFF` is 255 under an unsigned interpretation, -1 under eight-bit two's-complement interpretation, and merely one component of a larger object if the surrounding format says so. The bits do not announce their own meaning.

This observation also fixes the basic units used throughout the book. A nibble contains four bits and a byte eight. In x86 terminology a *word* is 16 bits, a doubleword 32 bits and a quadword 64 bits, although the word *word* is used more loosely in general architecture discussions. Decimal and binary prefixes must likewise be separated: $1\,\mathrm{kB} = 10^3$ bytes, whereas $1\,\mathrm{KiB} = 2^{10}$ bytes. Storage vendors commonly use decimal units; address-space and memory calculations often use powers of two.

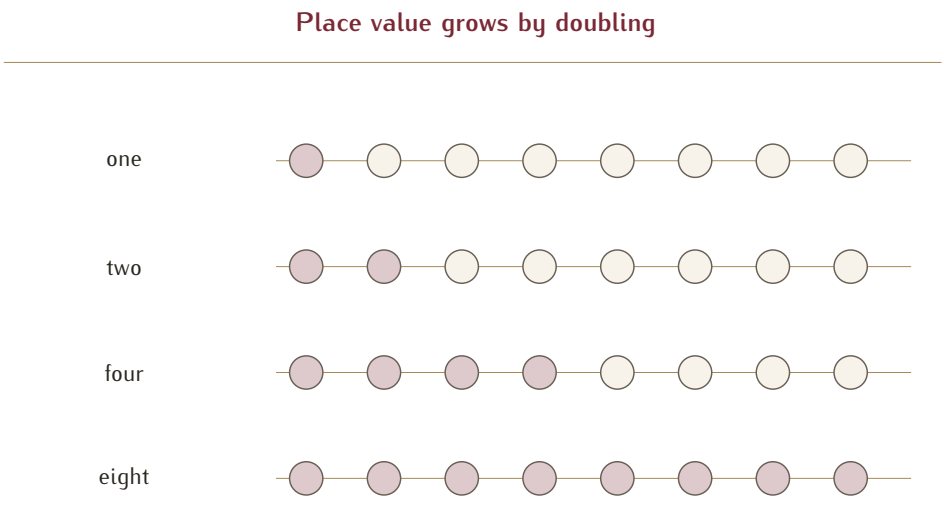


Figure 1.14: *The beads give a physical picture of the first four binary weights. Moving one position left doubles the contribution of a set bit. This pattern continues to 16, 32, 64 and 128 within a byte.*

1.2.3 Byte-addressed memory, width and endianness

The systems studied here expose memory as byte-addressable: an address identifies one byte, and a multibyte object occupies consecutive addresses. If a 32-bit integer begins at address a, its four bytes occupy a through $a+3$. What remains to be specified is how the significance of the integer is mapped onto those addresses.

On a little-endian system such as x86-64, the least significant byte occupies the lowest address. The value `0x12345678`, stored as one 32-bit object at address `0x1000`, therefore appears byte by byte as

address	`0x1000`	`0x1001`	`0x1002`	`0x1003`
byte	`0x78`	`0x56`	`0x34`	`0x12`

whereas a big-endian representation would place `0x12` at the lowest address. The numerical value has not been reversed; only the order of its component bytes in memory differs. Nor does endianness reverse human-readable hexadecimal notation or the bytes of an ordinary ASCII string: the characters in `ABCD` still occupy increasing addresses as `0x41 42 43 44`, because each character in that example is already a one-byte field.

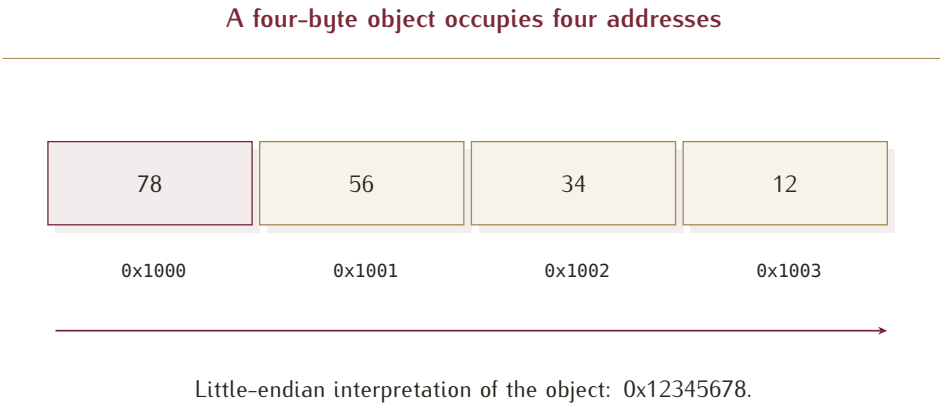


Figure 1.15: *Each address names one byte. Little-endian storage places the least significant byte at the lowest address. The four occupied addresses are consecutive even though the conventional written numeral orders its digits differently.*

Byte order matters whenever a representation is exposed as bytes – in file formats, network protocols, serialisation, debugger views and manual binary analysis. Within a correctly typed integer operation the compiler and ISA normally hide the layout, but the issue returns as soon as code inspects or exchanges the object representation.

Width imposes a second constraint. A mathematical integer is unbounded; a w-bit object has only 2^w patterns. At the bit level, retaining only w result bits is equivalent to reduction modulo 2^w. C unsigned arithmetic is modular at the width of the arithmetic type. Integer promotions must be applied first: on the stated platform, two `uint8_t` operands are promoted to `int`, so $250 + 10$ evaluates to 260; converting that result back to `uint8_t` produces 4. Signed overflow is a different language-level question and must not be inferred merely from the fact that the hardware adder emits low-order result bits; Chapter 1.4 treats that distinction carefully.

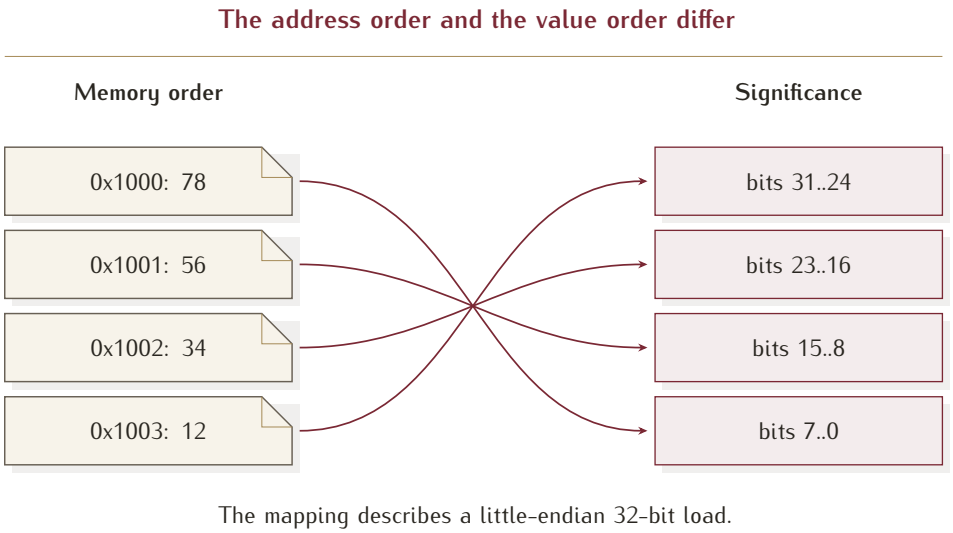


Figure 1.16: *Trace the lowest memory address to the least significant eight bits of the loaded value. Endianness changes this correspondence; it does not reverse the bits inside each byte.*

Hexadecimal wraparound

In one byte, `0xFA+0x0A=0x104`. The storage object can retain only the low two hex digits, so the byte becomes `0x04`; as unsigned arithmetic, this is $260 \bmod 256 = 4$. The discarded carry is still useful status information to the hardware, but it is not another bit in the one-byte result.

A byte-addressed system with a w-bit address field has at most 2^w distinct byte addresses if every bit pattern is implemented and usable. A 32-bit byte address therefore describes a theoretical 4 GiB address space. A 64-bit pointer type does not imply that every one of the 2^{64} bit patterns is implemented as a usable virtual or physical address; architecture and operating-system rules can expose fewer address bits.

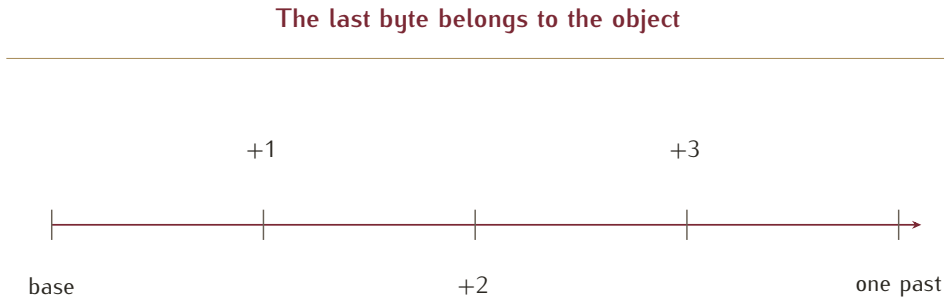


Figure 1.17: *A four-byte interval starts at its base and ends three bytes later. The one-past address is a useful boundary marker, but it is not a fifth byte of the object.*

1.2.4 Reading bytes without inventing meaning

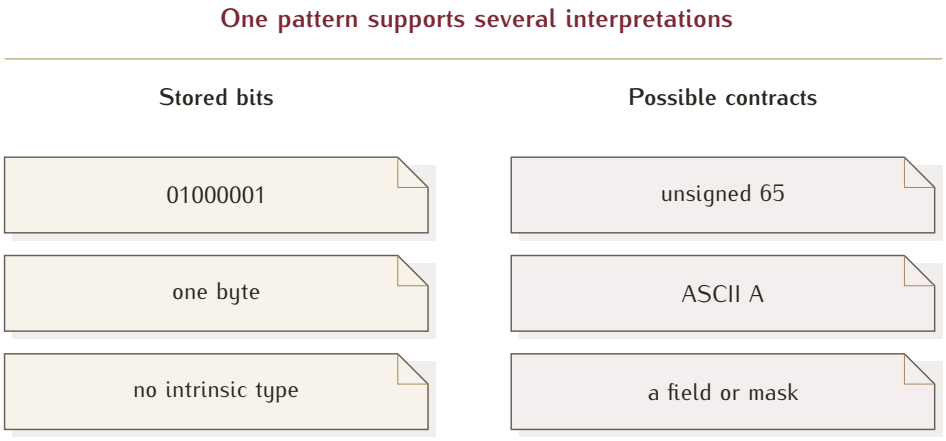


Figure 1.18: *The byte itself contains no tag saying which meaning to use. A debugger, parser or instruction selects an interpretation from the surrounding contract. A visually familiar character is only one possible reading.*

A memory dump records bytes at addresses. Their types must be established from the surrounding program or format. Consider

```
00001000: 48 65 6c 6c 6f 00 78 56 34 12
```

The first six bytes can be interpreted as the NUL-terminated ASCII string `Hello`; the last four can be interpreted as the little-endian 32-bit integer `0x12345678`. Neither interpretation is forced by the dump itself. An analyst must know, or infer and then verify, the object's boundaries and representation contract.

A disciplined reading therefore proceeds in a fixed order: identify the starting address and available length; group bytes according to a candidate field width; apply byte order only to multibyte scalar fields; decode each field under the appropriate representation; and finally check the interpretation against known ranges, terminators, signatures or other invariants. This order prevents three common mistakes: dropping width information, applying endianness to printed numerals or character order, and assuming that every bit pattern is intrinsically a number.

Text offers an immediate example of the last point. ASCII-compatible UTF-8 encodes the letter `A` as the single byte `0x41`, while U+00E8 requires two UTF-8 bytes. Consequently the string "Aè" occupies three bytes even though it contains two Unicode code points. Chapter 1.3 develops the distinction between characters, code points and encoded byte sequences; here the lesson is simply that byte count and user-visible text length belong to different contracts.

Laboratory 2: Inspecting object bytes

On a little-endian x86-64 system, predict the byte output of the program and repeat the reasoning for values `0x01020304` and `0x000000FF`. Explain which part is a C guarantee.

```c
#include <stdint.h>
#include <stdio.h>
int main(void) {
    uint32_t x = 0x12345678u;
    unsigned char *p = (unsigned char *)&x;
    for (size_t i=0; i<sizeof x; ++i) printf("%02x ", (unsigned)p[i]);
    putchar('\n');
}
```

Worked solution

Laboratory 2: solution

The three outputs, with low-address bytes first, are respectively `78 56 34 12`, `04 03 02 01` and `ff 00 00 00`. On the stated platform, `uint32_t` occupies four eight-bit bytes and little-endian order places the least significant byte first. C permits inspecting an object's representation through an unsigned-character pointer. The byte order follows the platform, rather than that pointer rule. An external format must declare its own order explicitly. Compile with `gcc -std=c11 -Wall -Wextra`

Worked solution (continued)

`bytes.c -o bytes`; changing only the initialiser produces each specified observation.

Return to the laboratory

The same method will be used throughout the rest of Part I. Bits become useful information only after width, boundaries and interpretation have been supplied; once those contracts are explicit, hexadecimal notation and memory dumps become precise evidence rather than visual puzzles.

Why binary became the engineering language of digital systems

Timeline

Year	Machine, standard or event	Architectural significance
1703	**Leibniz binary arithmetic**	Binary is described as a general positional system.
1937	**Shannon switching theory**	Boolean algebra is applied to relay circuits.
1964	**System/360 documentation**	Binary storage and hexadecimal notation become routine engineering language.

Origins and early engineering constraints

Base two was a mathematical possibility long before it became an engineering convention. In 1703, Gottfried Wilhelm Leibniz published an account of arithmetic using only the symbols 0 and 1. George Boole's *Laws of Thought* of 1854 supplied an algebra for propositions, but neither result forced builders to use binary machinery. Mechanical calculators were naturally decimal because toothed wheels could hold ten angular positions and users expected decimal input and output. Even the electronic ENIAC, completed in 1945, represented each decimal digit by a ten-state ring counter. The history explains why a two-state abstraction eventually made large electronic systems easier to construct and verify [85, 20, 87].

Konrad Zuse made one early practical commitment to binary. His Z1, completed in 1938, used mechanical switching elements; the relay-based Z3, completed in 1941, used a 22-bit binary floating-point format. Zuse's choice reduced arithmetic to combinations of two-state elements, but his machines were not the direct ancestor of every later design.

In the United States, Britain and Germany, engineers were independently exploring relays, vacuum tubes, telephone switching and pulse circuits. What made binary attractive across those projects was tolerance: a physical device need not hold an exact mathematical zero or one. It need only remain inside one of two voltage, current, magnetic or mechanical regions with enough separation that later stages can restore the intended state [88, 67].

Claude Shannon supplied the decisive design language. In his 1937 MIT master's thesis, published in 1938 as "A Symbolic Analysis of Relay and Switching Circuits," Shannon showed that series and parallel relay contacts could be represented by Boolean conjunction, disjunction and negation. Circuit synthesis could therefore begin with an algebraic specification and end with a network of switches. The work established a systematic correspondence between logical expressions and switching networks, allowing circuit behaviour to be derived and simplified algebraically. A bit became simultaneously a binary digit, a truth value and a physically regenerable signal [21].

The earliest electronic computers still differed sharply in word organisation. The Manchester Baby of 1948 stored 32-bit words; EDSAC used 18-bit storage slots with 17 usable bits in a short word; the IBM 704, introduced in 1954, used 36-bit words; the CDC 6600, delivered in 1964, used 60-bit words. There was no natural law selecting 8, 32 or 64 bits. Word length followed memory technology, instruction encoding, numerical precision, cost and intended workload.

The unit called a "byte" was likewise not initially fixed at eight bits. IBM's System/360, announced on 7 April 1964, made the eight-bit byte commercially decisive by using it across a compatible family intended for both character processing and scientific arithmetic. Its 32-bit words contained four addressable bytes, and hexadecimal notation represented each byte as two digits [89, 91, 110, 119, 63, 74].

Octal and hexadecimal arose as human interfaces to binary rather than rival machine bases. Three bits map exactly to one octal digit, which suited machines whose words and instruction fields were organised in groups of three. Four bits map exactly to one hexadecimal digit, which suits byte-oriented systems: the byte `11101010` becomes `EA` without a decimal conversion. In the System/360 documentation, hexadecimal exposes byte boundaries, masks and addresses. The same remains true when reading an ELF dump or x86 instruction stream. Decimal answers "how many?"; hexadecimal often answers "which bits and which bytes?" [67, 26, 27]

Standardisation and present-day architecture

Modern hardware reinforces the distinction between binary architecture and analogue implementation. A CMOS gate receives continuously varying voltages, and timing depends on capacitance, transistor drive, temperature and process variation. A DRAM cell stores charge that leaks and must be refreshed. NAND flash may distinguish several threshold-voltage windows in one physical cell; high-speed serial links can encode more than one bit per transmitted symbol. Error-correcting codes, sense amplifiers, timing margins and training logic reconstruct reliable digital values from those analogue phenomena. The architectural manual does not expose the exact charge on a cell because the digital interface promises a value after those recovery mechanisms have done their work [67, 26, 27].

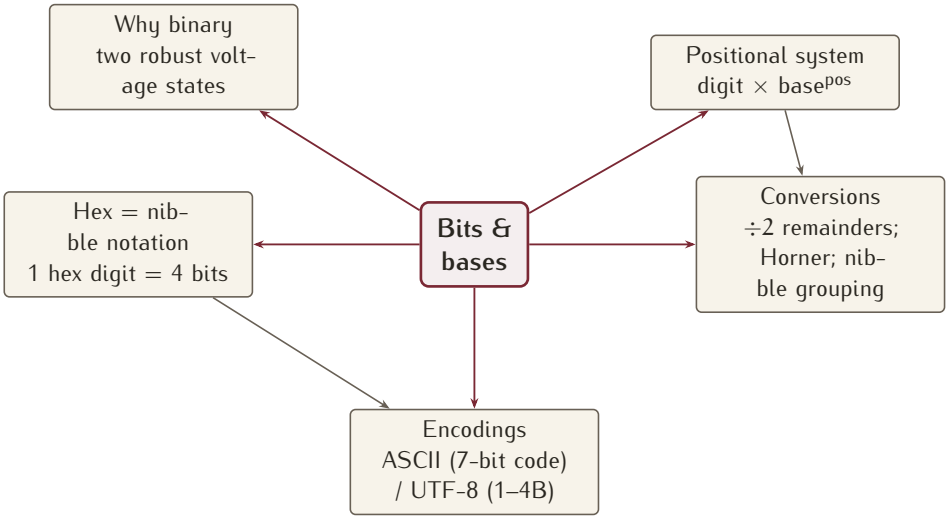
The logical unit has also widened. A scalar instruction may operate on 8, 16, 32 or 64 bits; a vector instruction may treat a 512-bit register as sixty-four bytes, sixteen 32-bit lanes or eight 64-bit lanes. Caches move whole lines, typically much wider than a

source-language object. Memory controllers schedule bursts and channels. Nevertheless, each layer preserves a binary contract at its boundary. Width and grouping change; the meaning of a bit position remains exact [67, 26, 27].

Binary became widespread because two-state signals can be restored, Boolean algebra describes their composition, and binary fields can retain stable meanings across manufacturing generations. The manuals studied in this book are the institutional form of that success: they state which bit controls which operation, which combinations are reserved, how wide each operand is and what happens when arithmetic exceeds that width. Without those statements, a pattern of zeros and ones is only a physical state; with them, it becomes architecture [67, 26, 27].

Word widths, radix notation and hexadecimal dumps remain the working language that connects source-level values to registers, memory and protocols.

Concept map: number systems and encodings



Reading the map. Digital circuits favour a small number of well-separated signal regions because noise margins and restoration are easier to engineer. Binary signalling offers a robust and economical implementation. Other physical encodings are possible. Positional notation then gives a systematic interpretation of the resulting bit strings; hexadecimal is convenient because each digit names exactly four bits. Text encodings add a further convention that maps byte sequences to characters.

1.2.5 Worked problems

Capacity

Question. How many patterns and which unsigned range are available in a seven-bit field?
Each position has two choices. Multiplying across the seven independent positions gives $2^7 = 128$ patterns. Unsigned interpretation assigns them consecutive values starting at zero, so the range is 0 through 127.
At the endpoints, `0000000` represents zero and `1111111` represents $64+32+16+8+4+2+1 = 127$. Counting both endpoints gives $127-0+1 = 128$ values, consistent with the pattern count. Two more bits multiply the capacity by four: nine bits give 512 patterns and the range 0 through 511.

Decimal to binary

Question. Express the decimal integer 45 in binary.
The largest power of two that fits is 32, leaving 13. The weight 16 does not fit, while 8 leaves 5, 4 leaves 1, 2 does not fit, and 1 leaves zero. Thus the coefficients of $32, 16, 8, 4, 2, 1$ are $1, 0, 1, 1, 0, 1$:

$$45 = 32+8+4+1 = 101101_2.$$

Expanding the selected weights recovers 45 and checks the answer. Applying the same procedure to 73 selects 64, 8 and 1, giving 1001001_2.

Binary to hexadecimal

Question. Convert 110101101011_2 to hexadecimal.
Group the bits from the right into nibbles. Each group occupies the same four binary positions as one hexadecimal digit:

$$\underbrace{1101}_{\mathrm{D}}\ \underbrace{0110}_{6}\ \underbrace{1011}_{\mathrm{B}} \quad\Longrightarrow\quad \texttt{0xD6B}.$$

The group values are $8+4+1 = 13$, $4+2 = 6$ and $8+2+1 = 11$. Expanding D, 6 and B back to four bits reproduces the original pattern. The same grouping turns 101111000010_2 into `0xBC2`.

UTF-8 storage

Question. How many UTF-8 bytes encode U+0041, U+20AC and U+1F600 in that order?
U+0041 lies in the one-byte ASCII range U+0000–U+007F. U+20AC lies in the three-byte range, while U+1F600 requires four bytes. The complete encodings are:

$$\underbrace{\texttt{0x41}}_{\text{U+0041}}\quad\underbrace{\texttt{0xE2 82 AC}}_{\text{U+20AC}}\quad\underbrace{\texttt{0xF0 9F 98 80}}_{\text{U+1F600}}.$$

Their lengths add to $1+3+4=8$ bytes for three code points. A terminating C NUL, if requested, occupies a ninth byte. For comparison, U+0062, U+00E9 and U+03A9 use $1+2+2=5$ bytes. Chapter 1.3 derives these encodings from their bit fields and distinguishes code points from user-perceived characters.

Exercise 1.2.1 — Minimum width

How many bits are needed for 300 unsigned values?

Worked solution

With w bits there are 2^w distinct patterns. Encoding 300 distinct values requires $2^w \geqslant 300$. Eight bits supply only $2^8 = 256$ patterns. Nine bits supply $2^9 = 512$, so the minimum is $\boxed{9}$ bits. For consecutive unsigned values beginning at zero, 300 values occupy 0 through 299; a 9-bit representation has room for that entire interval.

Exercise 1.2.2 — Hex conversion

Convert 0x7A to binary.

Worked solution

Each hexadecimal digit represents four bits. The digit 7 is $4+2+1$, hence `0111`; A represents 10, which is $8+2$, hence `1010`. Concatenate the groups in the original order: `0x7A` $= 0111\,1010_2$. Checking in decimal gives $7\cdot 16+10=122$ and $64+32+16+8+2=122$. The leading zero preserves the complete byte width.

Exercise 1.2.3 — Positional value

Evaluate 100101_2.

Worked solution

Starting at the right, the bit positions have weights $1, 2, 4, 8, 16, 32$. In 100101_2, the set bits occupy weights 32, 4 and 1. Thus $1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = \boxed{37}$. Dividing 37 by powers of two recovers $32 + 4 + 1$, which checks the interpretation and the position ordering.

Exercise 1.2.4 — Byte count

How many bytes are required by ten ASCII characters in UTF-8, excluding a C terminator?

Worked solution

An ASCII character uses one UTF-8 byte because its code point lies in the single-byte range. Ten such characters therefore require $10 \cdot 1 = \boxed{10}$ bytes. The question excludes a C string terminator. If the same characters were stored as a terminated C string, one additional zero byte would make the allocation at least 11 bytes. UTF-8 text containing non-ASCII characters requires counting the encoded bytes for those characters separately.

Summary

The governing idea is finite representation. A w-bit object offers exactly 2^w patterns. Positional weights explain how one interpretation maps those patterns to numbers. Hexadecimal packages the bits into groups that the eye can scan. Endianness then specifies the order in which the bytes of a multi-byte object appear in memory. These are modest rules, but they support every disassembly, packet trace and binary file examined later.

1.2.6 Exercises

Exercise 1.2.5 — General positional value

Evaluate 21012_3 in decimal. Then state whether it is a valid base-3 numeral and justify the answer.

Go to the solution →

Exercise 1.2.6 — Decimal, binary and hexadecimal

Convert 723_{10} to binary and hexadecimal. Show a route that allows the two answers to cross-check one another.

Go to the solution →

Exercise 1.2.7 — Counting patterns

How many distinct bit patterns exist in 5 bytes? How many hex digits are required to print one complete 5-byte pattern without abbreviation?

Go to the solution →

Exercise 1.2.8 — Binary and decimal storage units

A drive is advertised as 500 GB, where GB means 10^9 bytes. Approximately how many GiB does the operating system report before formatting overhead?

Go to the solution →

Exercise 1.2.9 — Endian reconstruction

At consecutive addresses a little-endian machine stores the bytes `0x2A 00 01 80`. Interpret them as one 32-bit unsigned integer and as two 16-bit unsigned integers.

Go to the solution →

Exercise 1.2.10 — One-byte wraparound

Compute the one-byte unsigned result of $230 + 70$. Give the decimal result, the resulting bits and a test that detects the overflow using only the wrapped sum and one operand.

Go to the solution →

Exercise 1.2.11 — Interpret a dump

A dump begins with `0x4D 5A 00 00 78 56 34 12`. Give two plausible interpretations and explain what additional context would choose between them.

Go to the solution →

Exercise 1.2.12 — A twenty-bit unsigned range

Derive the largest unsigned value representable with 20 bits and express it in hexadecimal.

Worked solution

Exercise 1.2.13 — Nibble decoding and population count

Convert `0xDEADBEEF` to binary by nibble grouping and count the 1 bits.

Worked solution

Exercise 1.2.14 — Byte strings and endianness

Explain why a byte dump of `"AB"` is independent of machine endianness.
Worked solution

Exercise 1.2.15 — Comparing two byte dumps

Use a four-byte file containing `0x41 42 00 FF`. Predict how `od -An -tx1` and `xxd -g 1` present its bytes, address column and character column. Explain which differences affect only display.
Worked solution

1.2.7 Solutions

Binary and Hexadecimal Representation

Solution 1.2.5

A base-3 digit must be 0, 1 or 2. Every digit of 21012_3 satisfies that condition, so the numeral is valid. From right to left the positional weights are $1, 3, 9, 27, 81$. Therefore

$$21012_3 = 2 \cdot 81 + 1 \cdot 27 + 0 \cdot 9 + 1 \cdot 3 + 2 \cdot 1 = 194.$$

As a check, repeated division gives remainders $2, 1, 0, 1, 2$; reading them upward reconstructs 21012_3. Validity is determined by the permitted digits before evaluating the weighted sum.

← Return to the exercise

Solution 1.2.6

Repeated division by 16 gives $723 = 16 \cdot 45 + 3$, $45 = 16 \cdot 2 + 13$, $2 = 16 \cdot 0 + 2$, hence $2D3_{16}$. Expanding each hex digit to four bits gives $0010\,1101\,0011_2$, or 1011010011_2 without leading zeros. Checking by powers of two: $512 + 128 + 64 + 16 + 2 + 1 = 723$.

← Return to the exercise

Solution 1.2.7

Five bytes contain 40 bits, so there are 2^{40} patterns. One hex digit represents 4 bits, so exactly 10 hex digits are required. Leading zeros must be retained when printing the full fixed-width pattern.

← Return to the exercise

Solution 1.2.8

The drive contains 500×10^9 bytes. One GiB is $2^{30} = 1,073,741,824$ bytes. The quotient is approximately 465.66 GiB. The drive has not lost capacity; two different units are being used.

← Return to the exercise

Solution 1.2.9

As one 32-bit value, reverse the byte significance: `0x8001002A`. As two 16-bit objects, the first pair `0x2A 00` represents `0x002A`=42, and the second pair `0x01 80` represents

Worked solution (continued)

`0x8001`=32769. Object boundaries matter: one cannot reverse all four bytes when the intended layout contains two separate fields.

← Return to the exercise

Solution 1.2.10

The mathematical sum is $230 + 70 = 300$. Retaining one byte reduces it modulo 256: $300 - 256 = 44 = 32 + 8 + 4$, giving 00101100_2 or `0x2C`.
For operands $0 \leqslant a, b < 256$, write the wrapped result as s. Without overflow, $s = a + b \geqslant a$. With overflow, $s = a + b - 256 < a$ because $b < 256$. Thus $s < a$ detects the discarded carry using just the result and one operand; $s < b$ is an equivalent test. Here $44 < 230$ confirms overflow. In C, apply the test to the narrowed byte result, since the addition of two byte operands is normally promoted to `int`.

← Return to the exercise

Solution 1.2.11

The first bytes `0x4D 5A` are ASCII `MZ`; they are also the little-endian 16-bit integer `0x5A4D`. The final four bytes can be the little-endian integer `0x12345678`. File-format documentation, a declared C structure, an object boundary, or a program's use of the bytes would select an interpretation. The bytes alone do not carry type labels.

← Return to the exercise

Solution 1.2.12

Set every bit to one to maximise the value. The positional weights sum to

$$1 + 2 + \cdots + 2^{19} = 2^{20} - 1 = 1{,}048{,}575.$$

Twenty bits divide into five four-bit groups. Each group is `1111`, whose hexadecimal digit is F, so the fixed-width answer is `0xFFFFF`. Adding one would require a twenty-first bit.

Return to the exercise

Solution 1.2.13

Nibble grouping gives `D E A D B E E F` = `1101 1110 1010 1101 1011 1110 1110 1111`. The one counts are $3 + 3 + 2 + 3 + 3 + 3 + 3 + 4 = 24$.

Return to the exercise

Solution 1.2.14

A string is an ordered sequence of bytes. ASCII `"AB"` occupies `41 42` followed by a zero terminator in a C string. Endianness only changes the byte order used to serialise a multibyte scalar value; it does not reverse the order of separate character bytes.
Return to the exercise

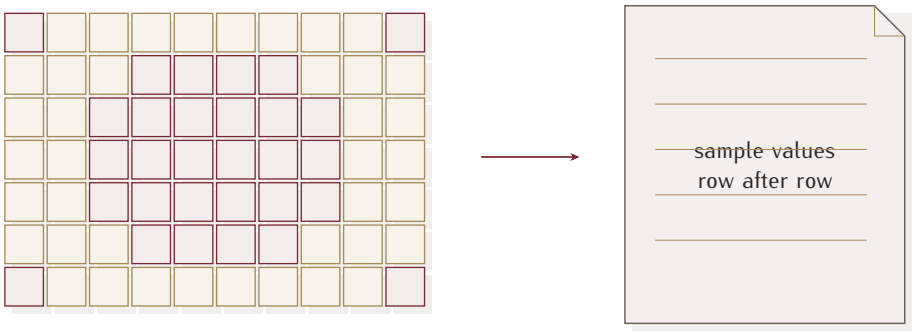
Solution 1.2.15

Both commands display the byte sequence `41 42 00 ff` in that order. With `od -An -tx1`, `-An` suppresses the address column and `-tx1` requests one-byte hexadecimal units. With `xxd -g 1`, the line begins with an offset such as `00000000:`, groups each byte separately, and includes a character column. The printable bytes appear as `AB`; NUL and `0xFF` are shown with nonprinting placeholders. Padding, offset width and hexadecimal letter case are display conventions. Neither command changes the file or reorders its bytes.
Return to the exercise

1.3 ENCODING NON-NUMERICAL DATA

A program reads a file and receives bytes. To display a character, draw a pixel or interpret a header, it must know how those bytes were encoded. A producer and a consumer agree on that representation before the data can be used reliably. Text makes the agreement especially visible: a character, its code point and its encoded bytes occupy different places in the explanation. The same questions will guide the interpretation of images, sound and structured records.

From a drawn pattern to encoded samples



A raster records a grid; the file chooses how that grid becomes bytes.

Figure 1.19: *A raster represents a spatial pattern with discrete samples. Serialisation must specify an order and a sample representation. The page on the right represents the serialised image record.*

1.3.1 Encoding as a contract between meaning and bytes

An encoding assigns a representation to members of some abstract set. If the abstract objects are characters, the representation may be a sequence of bytes; if they are pixels, the representation may contain several numeric channels per sample; if they are instructions, selected bit fields identify operations and operands. The important distinction is between the *object being represented* and the *representation chosen for storage or transmission*.

The consequence is that the same bits can support different interpretations. The byte `0x41` is the unsigned integer 65 under one contract, the ASCII letter `A` under another, and merely the low byte of a larger integer under a third. None of those meanings is physically embedded in the byte. A program obtains one by choosing a type, format, instruction or protocol rule and then operating consistently with that rule.

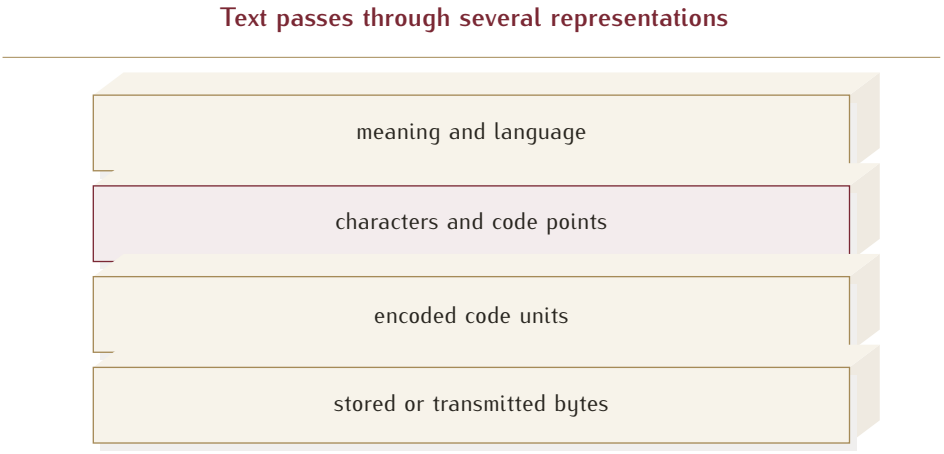


Figure 1.20: *The layers explain why a character count, a byte count and a glyph count can differ. The stored representation supports a later rendering process; a byte sequence is not itself a drawing of a letter.*

ASCII provides a compact historical example. It assigns codes 0–127 to control characters, punctuation, digits and Latin letters; because every code fits in seven bits, an ASCII character occupies one byte in common byte-oriented storage. Thus the string `ABC` is stored in an ASCII-compatible representation as `0x41 42 43`. The characters are abstract symbols, the code values are integers assigned by the character set, and the bytes are the concrete serialised representation. Keeping those levels separate becomes essential once the repertoire extends beyond ASCII.

1.3.2 Unicode, code points and UTF-8

Unicode provides a much larger coded character repertoire. A Unicode *code point*, conventionally written U+XXXX, is a position in the range U+0000–U+10FFFF. Some positions identify assigned characters, such as U+0041 LATIN CAPITAL LETTER A and U+00E8 LATIN SMALL LETTER E WITH GRAVE; others are unassigned or reserved for special purposes. The code point alone does not determine a byte sequence until an encoding form and, where relevant, a byte-serialisation convention are chosen. Unicode defines encoding forms including UTF-8, UTF-16 and UTF-32; this chapter uses UTF-8 because it is byte-oriented and ASCII-compatible.

UTF-8 encodes each Unicode *scalar value* in one to four bytes. Scalar values exclude the surrogate code points U+D800–U+DFFF: their ranges are U+0000–U+D7FF and U+E000–U+10FFFF. Surrogates therefore cannot appear as independently encoded UTF-8 values. Code points U+0000 through U+007F use one byte whose high bit is zero, so ordinary ASCII text retains exactly the same byte representation. Higher code points are split across a leading byte and one or more continuation bytes. The leading pattern identifies the sequence length while continuation bytes begin with the bit pattern `10`; the remaining bits carry the code-point payload.

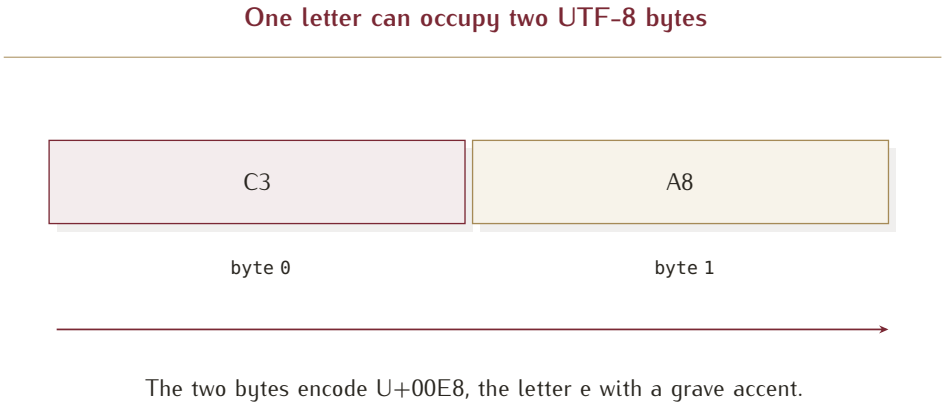


Figure 1.21: *UTF-8 encodes U+00E8 in two bytes. The leading-byte pattern begins the sequence and the continuation-byte pattern completes it. Counting these bytes gives two; counting the decoded code point gives one.*

The payload capacities determine the length ranges. Read each row together with the scalar-value restriction:

Scalar-value range	Bit template
U+0000–U+007F	`0xxxxxxx`
U+0080–U+07FF	`110xxxxx 10xxxxxx`
U+0800–U+FFFF, excluding surrogates	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000–U+10FFFF	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Use the shortest row that can represent the scalar value. For example, a two-byte form for U+0041 would be overlong and invalid. After collecting the payload, a decoder checks the row's lower bound, excludes surrogates and rejects values above U+10FFFF. These rules also exclude leading bytes C0, C1 and F5–FF, and constrain the second byte after E0, ED, F0 and F4. A truncated sequence fails before payload reconstruction completes [6].

For U+00E8, hexadecimal value `0x00E8`, UTF-8 produces `0xC3 A8`. The text "Aè" therefore contains two code points but occupies three bytes: `0x41 C3 A8`. This sim-

ple example is enough to invalidate the common assumption that character count and byte count are interchangeable.

Encoding U+00E8

U+ooE8 lies in the range encoded by the two-byte UTF-8 form `110xxxxx 10xxxxxx`. Splitting the code-point bits across those payload positions gives `11000011 10101000`, or `0xC3 A8`. Decoding performs the inverse operation: validate the leading and continuation patterns, collect the payload bits, and reconstruct the code point.

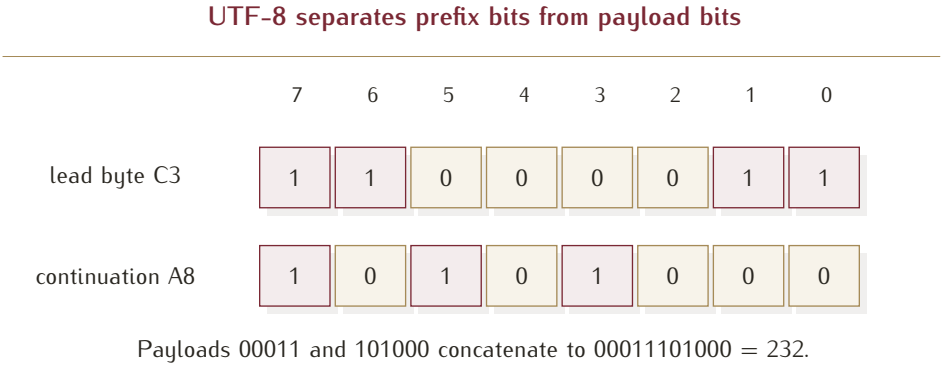


Figure 1.22: *For this two-byte sequence, remove the leading 110 and the continuation prefix 10. Concatenating the remaining five and six payload bits gives decimal 232, the value of U+ooE8.*

A robust decoder must do more than concatenate payload bits. It must reject malformed continuation structure, overlong encodings and values outside the Unicode scalar-value rules. Text processing then introduces another distinction: one code point is not necessarily one user-perceived character. Combining marks, emoji sequences and other composition rules can make several code points form one grapheme cluster. Consequently byte count, code-point count and displayed-character count answer different questions.

C strings expose the byte-level side of the same model. A conventional C string is an array of `char` terminated by a zero byte. Functions such as `strlen` count bytes before the terminator, not Unicode code points or grapheme clusters. For pure ASCII those quantities happen to coincide; for UTF-8 text they generally do not. Code that allocates or indexes by "number of characters" must therefore state which level it means.

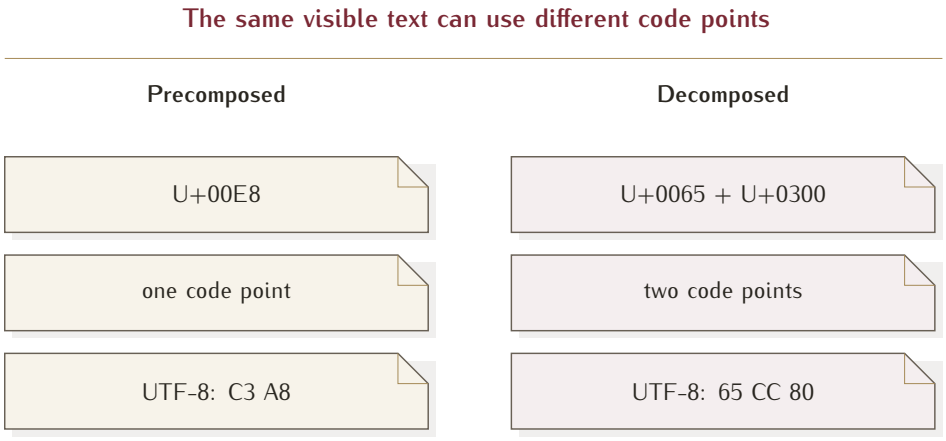


Figure 1.23: *The precomposed letter and the base letter plus combining accent can represent canonically equivalent text. Their byte sequences differ. Applications that compare text must choose and apply an appropriate normalisation policy.*

1.3.3 Structured numerical data: images and sound

Text is not special in requiring a contract. An image format must specify dimensions, channel order, numeric representation, row layout and often colour-space information before a byte offset can be interpreted as a pixel. In a simple uncompressed RGB image with three one-byte channels per pixel, a row-major pixel at coordinates (x, y) in an image of width W begins at

$$3(yW + x)$$

bytes from the start of the pixel array, provided the format has no row padding and uses that channel order. A real format may impose alignment, stride or metadata that changes the calculation; the formula is valid only under the stated layout.

A pixel often begins as analogue charge

In a camera sensor, incoming photons generate charge in photosites. Analogue circuitry amplifies and conditions the signal before an analogue-to-digital converter quantises it. Colour filter arrays, calibration and later image processing then transform those samples into the channel values exposed to software.

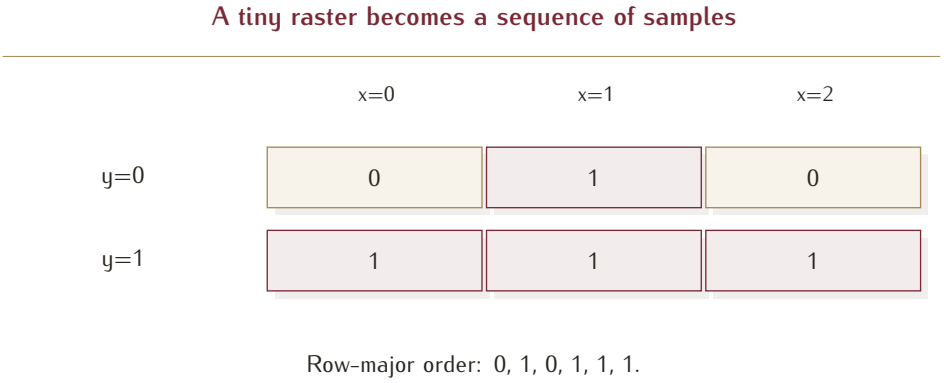


Figure 1.24: *The matrix depicts a two-row, three-column monochrome raster. Row-major serialisation takes all samples in the first row before those in the second. A format must still specify bit packing, row padding and orientation.*

Pixel bytes are already several abstraction layers above the physical measurement. Bit depth limits the set of digital codes. Sensor noise, analogue gain, quantisation and colour processing influence which code the file records [43].

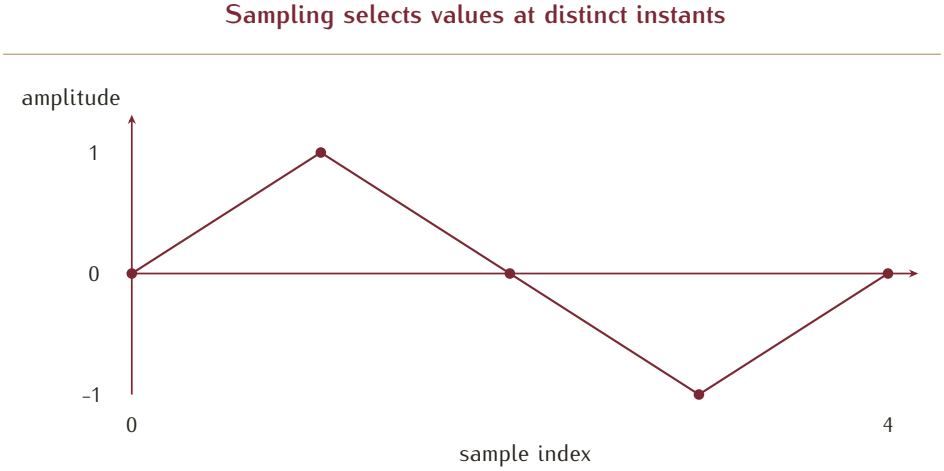


Figure 1.25: *An audio stream stores samples at specified instants and quantises their amplitudes. The drawn connections do not assert the exact waveform between samples. Reconstruction requires assumptions about sampling and bandwidth.*

Sound follows the same pattern across time rather than space. Pulse-code modulation samples an analogue signal at a specified rate and quantises each sample to a fixed

numerical representation. A stereo recording sampled at 48,000 samples per second with 16-bit samples contains, before container overhead or compression,

$$48{,}000 \times 2 \times 2 = 192{,}000$$

bytes per second. The byte count depends on sample rate, channel count and stored sample width. Interpreting the amplitudes additionally requires the sample representation, including signedness and byte order.

1.3.4 Binary formats, validation and context

A binary file format combines several representations into a larger grammar. A header may begin with a signature, followed by lengths, flags, offsets, text fields and numerical payloads whose widths and byte orders differ. The decoder must therefore reason at two levels simultaneously: individual fields have representation rules, while the file as a whole has structural invariants that say where those fields may occur and how they relate.

Consider an invented format whose first four bytes form an ASCII signature, followed by a little-endian 16-bit payload length and then that many bytes of UTF-8 text. Reading the length correctly is not enough. A parser must first prove that the two length bytes exist, decode them using the specified byte order, verify that the declared payload fits inside the supplied buffer, and only then ask a UTF-8 decoder to validate the payload. Performing those steps in a different order can turn a mere format error into an out-of-bounds access.

Representation validation is a security boundary. Lengths, offsets, dimensions and counts obtained from untrusted input must be checked before they control allocation or indexing; text must satisfy the selected encoding; reserved bit patterns must obey the format's versioning rules. A byte sequence can be structurally present yet semantically invalid, and a parser that silently invents missing assumptions creates exactly the ambiguity attackers exploit.

Compression introduces another representation layer between the stored bytes and the object that an application recovers. A compressed image still denotes an image; a compressed text stream still denotes text. The compressed bytes cannot be interpreted as raw pixels or characters until the compression format has been decoded. Lossless compression promises exact recovery of the encoded data, whereas lossy compression deliberately replaces the original with an approximation according to a domain-specific model.

Physical communication adds still another layer below the bytes. Storage and network channels can corrupt symbols, so practical systems use parity, checksums, cyclic redundancy checks or error-correcting codes to detect and sometimes correct faults before higher-level decoders trust the data.

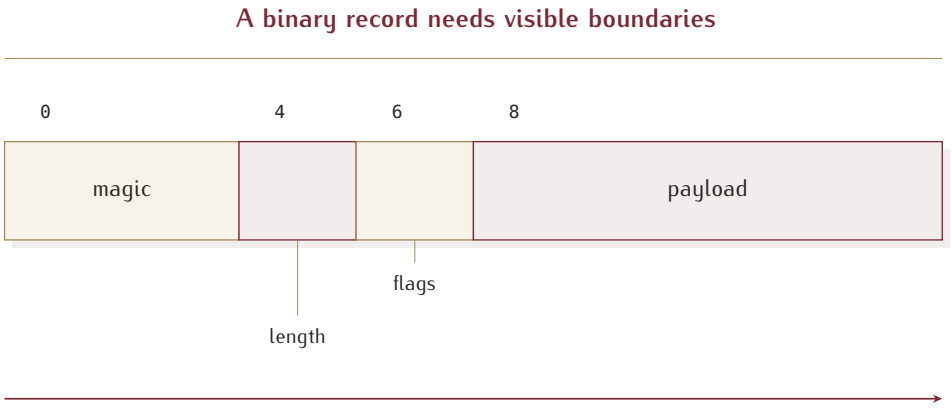


Figure 1.26: *The header fields partition the first eight bytes of this invented record. A length value is useful only after the parser establishes that the field is present and interprets its byte order. Payload validation follows the format contract.*

Reliable bytes require error detection and correction

A file or packet is presented to software as an exact sequence of bytes.

Real channels suffer noise, retention faults, interference and timing errors. Redundant check information allows hardware or protocol logic to detect discrepancies and, with suitable codes, reconstruct some corrupted data.

These mechanisms support the byte abstraction within their detection and correction capabilities. A correctable error can be repaired; larger error patterns may be detected without repair or may escape detection. Format and authorisation checks separately determine whether the recovered bytes describe a permitted record.

1.3.5 A disciplined decoding workflow

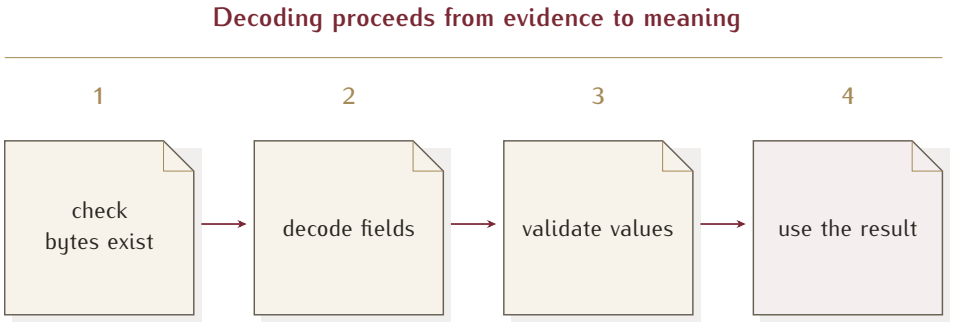


Figure 1.27: *The sequence separates structural safety, representation rules and semantic policy. A successfully decoded integer can still be outside the allowed range, and a plausible header does not establish that the whole payload exists.*

The recurring method is therefore to begin with provenance and boundaries, then apply the correct representation contract, and only afterwards use the resulting value. For text, identify the encoding before counting characters. For an image, establish dimensions, stride and channel representation before computing an offset. For a binary record, prove that every field lies inside the available buffer before decoding it. At each step, ask what is observed directly in the bytes and what is being inferred from an external rule.

Laboratory 3: UTF-8 layers

Use text consisting of A, U+00E8 and U+20AC. Give the UTF-8 bytes, the C `strlen` result and the code-point count. Change the second byte of the euro sequence to `0x41` and show a strict-decoder test.

Worked solution

Laboratory 3: solution

A encodes as `0x41`, U+00E8 as `0xC3 A8`, and U+20AC as `0xE2 82 AC`. The six bytes are `41 c3 a8 e2 82 ac`; a C string adds a terminating zero excluded by `strlen`, so the length is 6. There are three code points and, for this particular text, three displayed characters. A strict test is:

```
python3 - <<'PYCODE'
b = bytes.fromhex('41 c3 a8 e2 82 ac')
print(len(b), len(b.decode('utf-8', errors='strict')))
bad = bytes.fromhex('41 c3 a8 e2 41 ac')
```

Worked solution (continued)

```
try:
    bad.decode('utf-8', errors='strict')
except UnicodeDecodeError:
    print('invalid continuation')
PYCODE
```

It prints `6 3`, then `invalid continuation`. Byte `0x41` lacks the `10xxxxxx` prefix required inside the three-byte euro sequence.
Return to the laboratory

An encoding allows independent components to exchange information when they agree on the mapping and validate its boundaries. The same discipline applies throughout the book: instruction decoders, executable loaders, network protocols and filesystems all turn bytes into behaviour by applying contracts that exist outside the bytes themselves.

From telegraph codes to Unicode text processing

Timeline

Year	Machine, standard or event	Architectural significance
1874	**Baudot code**	Fixed-length signalling codes support telegraphy.
1963	**ASCII**	A standard seven-bit interchange code is published.
1991	**Unicode 1.0**	A universal coded character repertoire begins to replace incompatible code pages.

Origins and early engineering constraints

Character encoding grew from the need to transmit messages over communications channels. Samuel Morse's commercial telegraph system of the 1840s represented letters by temporal patterns of short and long signals interpreted by trained operators. Emile Baudot's printing telegraph, patented in 1874, used fixed-length five-unit codes that could be handled mechanically. Because five bits provide only 32 patterns, later teleprinter systems used shift codes to alternate between letters and figures. The International Telegraph Alphabet No. 2, standardised internationally in 1932 from the Baudot-Murray lineage, still reflected the economics of narrow electromechanical channels: a code was a compact instruction to a particular terminal, and several patterns were controls rather than printable symbols [68, 67].

Punched-card data developed a separate lineage. Herman Hollerith's 1890 census system encoded categories by hole positions in a card; later IBM cards used zone and digit punches to represent letters, numbers and control information. A character was

therefore not yet an abstract object independent of its medium. It could be a teleprinter state transition, a hole pattern, a printer-chain position or a vendor-specific numerical code. Early computers inherited those local conventions because they were connected to existing card readers, punches and printers [67, 78, 5].

The push toward interchange produced ASCII. The American Standards Association's X3.2 committee began work in 1960; the first American Standard Code for Information Interchange appeared in 1963 and a major revision followed in 1967. ASCII used seven bits: 128 code positions containing letters, digits, punctuation and control characters such as carriage return, line feed and escape. The ordering supports predictable conversions and comparisons. Digits are consecutive, uppercase and lowercase Latin letters form contiguous ranges, and the low control area reflects telecommunication practice. RFC 20, published in October 1969, specified ASCII for network interchange and records the code table used by the early Internet community [61].

IBM followed a different compatibility path when it announced System/360 in 1964. EBCDIC, the Extended Binary-Coded Decimal Interchange Code, grew from IBM's punched-card and Binary Coded Decimal traditions and occupied an eight-bit byte. ASCII and EBCDIC contain many of the same basic characters but assign different byte values and collating orders. Neither byte pattern identifies itself. The byte `0xC1`, for example, is the letter A in common EBCDIC code pages but not in ASCII. This is why transferring a file without recording its encoding can preserve every byte and still corrupt the text [63, 74, 114].

Seven-bit ASCII left the high half of an eight-bit byte available, but no global authority initially assigned those 128 additional values. ISO 646 national variants instead replaced selected positions within the seven-bit repertoire. Later eight-bit code pages used additional byte values for accented letters, Greek, Cyrillic and other scripts, with incompatible assignments. A Western European program might assume ISO-8859-1; a DOS program might assume code page 437 or 850; a Macintosh used another mapping. East Asian systems required multibyte schemes whose interpretation depended on state or byte ranges. The consequences went beyond visual inconvenience. Filenames, sort order, regular expressions, database keys and program source could all change meaning at a boundary [67, 78, 5].

Unicode changed the unit being standardised. Work began in the late 1980s among engineers including Joe Becker, Lee Collins and Mark Davis; Unicode 1.0 was published in 1991. Instead of defining one more byte table, Unicode assigned numbers and properties to abstract characters drawn from many writing systems. ISO/IEC 10646 developed in parallel, and the repertoires were aligned. A Unicode code point is an integer in a coded space. The chosen encoding form determines how that integer is serialised into storage units [78, 5].

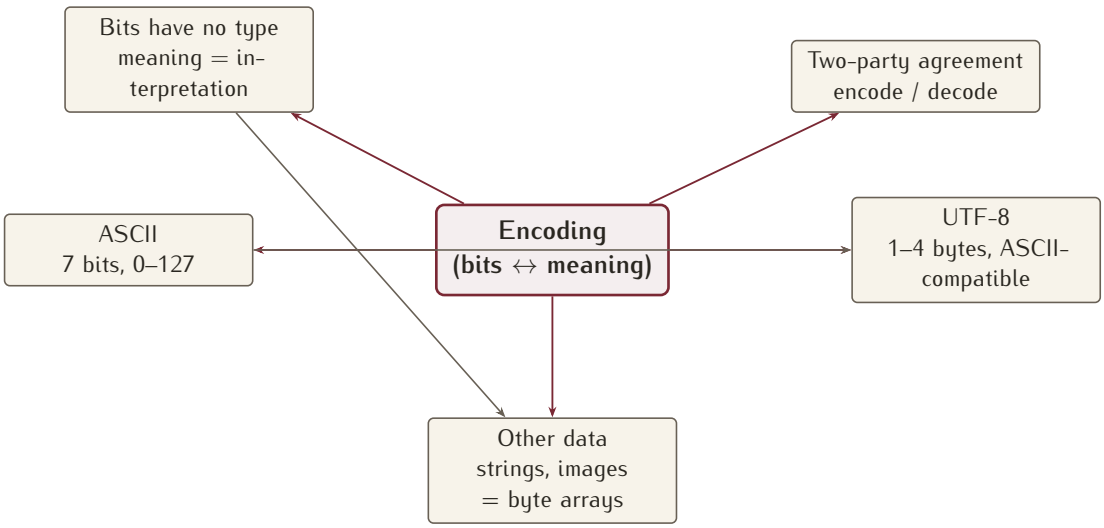
UTF-8 was designed by Ken Thompson and Rob Pike in September 1992 for the Plan 9 operating system and described publicly in 1993. Its modern form uses one byte for ASCII characters and self-synchronising sequences of two, three or four bytes for

other Unicode scalar values; the original design allowed longer sequences. Preserving ASCII byte values was decisive for C and UNIX: the null byte, slash, newline, command names and existing source text retained their representation. RFC 3629, published in 2003, fixed the modern UTF-8 range to the Unicode codespace and excluded overlong and otherwise invalid sequences [10, 6].

Unicode did not make a code point equal to a visible character. The letter perceived by a reader may be one precomposed code point or a base letter followed by combining marks. Emoji may use variation selectors, skin-tone modifiers and zero-width joiners. Unicode Standard Annex No. 29 defines grapheme-cluster boundaries; normalisation forms define canonical or compatibility transformations; the bidirectional algorithm orders mixed left-to-right and right-to-left text. Fonts and shaping engines map character sequences to glyphs according to script-specific rules. A program that counts bytes, code points and user-perceived characters is therefore answering three different questions [67, 78, 5].

Unicode Standard Version 17.0.0, the edition referenced here, was released in September 2025. Its scale should not obscure the engineering hierarchy. Files and networks carry bytes. UTF-8 interprets those bytes as code points. Unicode properties and algorithms organise code points. A shaping engine and font choose glyphs. A language community supplies semantics. Security-sensitive software must additionally consider confusable characters, invisible controls, normalisation and parser differences. The historical progression from telegraph code to Unicode gradually separated transport, coded identity, textual structure and visual rendering into contracts that can be tested independently [5, 78].

Concept map: encoding as agreement



Reading the map. The root idea, that bits acquire meaning only through an agreed interpretation, is the same principle that returns as signed/unsigned in 1.4 and as data/address/instruction in Part II. ASCII and UTF-8 are two concrete agreements; strings and images are byte arrays whose structure is supplied by other formats and encodings.

1.3.6 Worked problems

Same bytes, different meanings

Question. Give several interpretations of the unchanged byte `0x41`.
Under an unsigned integer interpretation, $4 \cdot 16 + 1 = 65$. ASCII assigns that value to the letter A. In an eight-bit image channel, it can be an intensity code; its physical or displayed intensity depends on the colour representation. An instruction decoder may consume the byte as an opcode, prefix or operand field according to the ISA and its position in the instruction stream.
Changing the declared format changes the decoding rule while leaving the byte intact. Likewise, `0xFF` can represent unsigned 255, signed eight-bit -1, or the maximum code in an unsigned eight-bit colour channel. The consuming operation and object boundaries determine which interpretation applies.

C string length

Question. For `char s[] = "CAT"`, compute `sizeof s` and `strlen(s)`. Would `strlen` be safe on any four-byte buffer?
The initializer supplies four array elements: C, A, T and NUL. Each `char` occupies one C byte, so `sizeof s` is 4. The length scan stops at the first NUL and counts the three preceding bytes, giving `strlen(s) == 3`.
A four-byte buffer containing C, A, T, X has no terminator within its bounds. Passing that buffer to `strlen` can make the scan read outside the object, so its size alone is insufficient to establish a valid C string. For `char t[] = "DOGS"`, the initializer supplies five elements and the corresponding results are 5 and 4.

UTF-8 parsing

The three bytes are `0xE2`, `0x82`, and `0xAC`. A complete UTF-8 check must first verify the byte-prefix structure and then verify that the decoded code point is legal and minimally encoded.
Question. Do these bytes form a valid UTF-8 encoding, and which parts of the validation establish that conclusion?
Reasoning.

Worked example (continued)

1. **Check the structural prefixes.** `0xE2` begins with `1110`, announcing a three-byte sequence. `0x82` and `0xAC` both begin with `10`, so they have the required continuation-byte form.
2. **Recover the payload bits.** Remove the prefix bits and concatenate the payload: `0010 000010 101100`, which is code point U+20AC.
3. **Apply the semantic validity checks.** U+20AC is above U+07FF, so the encoding is not overlong; it is not in the surrogate range U+D800–U+DFFF; and it does not exceed U+10FFFF.

Conclusion. `0xE2 82 AC` is the valid UTF-8 encoding of U+20AC, the euro sign.
Prefix checking alone establishes only that the bytes have the shape of a three-byte sequence. Unicode validity additionally requires a minimally encoded scalar value that is neither a surrogate nor outside the Unicode range.
Encode U+20AC from the code point: place its bits into the template `1110xxxx 10xxxxxx 10xxxxxx`. The reconstructed bytes are again `0xE2 82 AC`.
The sequence `0xED A0 80` has correct-looking prefixes but decodes to U+D800, a surrogate. A complete validator must therefore reject it even though the structural prefix check passes.

Raster offset

Question. In packed RGB with width 20, three bytes per pixel and no row padding, find the offset of zero-based pixel (row 3, column 4).
Three complete rows precede the target, occupying $3 \cdot 20 \cdot 3 = 180$ bytes. Four pixels precede it within its row, adding $4 \cdot 3 = 12$ bytes. Its first channel is therefore at offset $180 + 12 = 192$, followed by the other channels at 193 and 194.
Combining the two contributions gives the general expression $(yW + x) \cdot 3$. For width 32 and pixel $(2, 5)$, it gives $(2 \cdot 32 + 5) \cdot 3 = 207$. With a padded row stride S, the corresponding expression would be $yS + 3x$; only the within-row term remains unchanged.

Exercise 1.3.1 — Interpretation

Why can the same four bytes be a float or an integer?

Worked solution

A four-byte pattern supplies 32 bits. An integer interpretation assigns positional weights and, for a signed type, a sign convention. Binary32 instead divides those bits into sign, exponent and fraction fields. For example, bits `0x3F800000` represent unsigned integer 1065353216 and binary32 value 1.0. The instruction, declared type

Worked solution (continued)

or file-format contract selects which decoding to apply. The stored bytes remain identical in both interpretations.

Exercise 1.3.2 — UTF-8 structure

How many continuation bytes follow a leading byte `11110xxx`?

Worked solution

The prefix `11110xxx` identifies the first byte of a four-byte UTF-8 sequence. One of those four bytes is already the leading byte, leaving $4 - 1 = \boxed{3}$ continuation bytes. Each continuation byte has prefix `10xxxxxx`. This determines the structural length; validity also requires the remaining bytes and payload to form a permitted Unicode scalar value.

Exercise 1.3.3 — Image size

Compute raw storage for 64x32 RGB with 8 bits per channel and no padding.

Worked solution

There are $64 \cdot 32 = 2048$ pixels. Each RGB pixel has three 8-bit channels, so it occupies $3 \cdot 8 = 24$ bits, or 3 bytes. With no row padding or file header, storage is $2048 \cdot 3 = \boxed{6144}$ bytes. Equivalently, one row uses $64 \cdot 3 = 192$ bytes and 32 rows use $192 \cdot 32 = 6144$. Headers, compression and padding would require separate accounting.

Exercise 1.3.4 — Monochrome rounding

How many bytes store 17 packed one-bit pixels?

Worked solution

A byte holds eight one-bit pixels. Two bytes hold 16 pixels, leaving one of the 17 pixels without storage. A third byte is required, giving $\lceil 17/8 \rceil = \boxed{3}$ bytes. Of the 24 allocated bits, 17 carry pixel values and seven are unused padding bits. Integer division rounded down would return two and under-allocate the buffer.

Summary

The same method applies beyond text. Identify the abstract objects, specify the byte-level representation, define boundaries and lengths, and state the context needed for decoding. When a parser fails, the fault usually lies in one of those agreements: the

wrong encoding, the wrong byte order, a corrupted length, an omitted delimiter or a field read under the wrong type.

1.3.7 Exercises

Exercise 1.3.5 — Same byte, different meanings

Interpret `0x31` as (a) an unsigned integer, (b) an ASCII character, and (c) an 8-bit mask. Explain why none is intrinsically more correct.

Go to the solution →

Exercise 1.3.6 — UTF-8 sequence length

Without decoding the code point, determine the number of bytes in a UTF-8 character whose first byte is (a) `0x24`, (b) `0xC2`, (c) `0xE1`, (d) `0xF0`, (e) `0x80`.

Go to the solution →

Exercise 1.3.7 — Decode UTF-8

Decode `0xC2 A3` to a Unicode code point.

Go to the solution →

Exercise 1.3.8 — C string accounting

An array is declared as `char s[] = {0x48, 0x69, 0, 0x21, 0};` its five elements contain bytes `0x48 69 00 21 00`. What are `sizeof` the array, `strlen` when passed as a string, and the bytes that remain physically in the array after the first terminator?

Go to the solution →

Exercise 1.3.9 — Image size with row padding

A 24-bit RGB image has width 5 and height 4. Each row is padded to a multiple of 4 bytes. Compute the payload bytes per row, stride and total pixel-array size.

Go to the solution →

Exercise 1.3.10 — Audio calculation

Compute the raw size of 30 seconds of mono audio sampled at 48 kHz with 24-bit samples. Give bytes and approximate MiB.

Go to the solution →

Exercise 1.3.11 — Invented file format

A format begins with four ASCII bytes `DATA`, followed by a big-endian 16-bit payload length and then the payload. Decode `0x44 41 54 41 00 03 41 42 43`.

Go to the solution →

Exercise 1.3.12 — Encoding a Greek character

Encode U+03BB (Greek small lambda) in UTF-8 by filling the bit template.

Worked solution

Exercise 1.3.13 — Visual similarity and byte equality

Find two Unicode strings that look similar but have different byte sequences; explain why byte equality and visual equality are different tests.

Worked solution

Exercise 1.3.14 — An image with sixteen-bit channels

Calculate raw storage for a 3840×2160 RGBA image with 16 bits per channel.

Worked solution

Exercise 1.3.15 — Designing a binary header

Design a ten-byte header containing a signature, version, flags and payload length. State byte order and valid ranges explicitly.

Worked solution

1.3.8 Solutions

Encoding Non-Numerical Data

Solution 1.3.5

As an unsigned integer it is 49. In ASCII it is the character `1`. As a mask it has bits 5, 4 and 0 set because `0x31` is 00110001_2. Meaning comes from the consuming operation or agreed format; the stored byte carries no type tag.

← Return to the exercise

Solution 1.3.6

The leading prefixes are: `0x24=0xxxxxxx`, one byte; `0xC2=110xxxxx`, two bytes; `0xE1=1110xxxx`, three bytes; `0xF0=11110xxx`, four bytes. `0x80=10xxxxxx` is a continuation byte and cannot validly begin a character by itself.

← Return to the exercise

Solution 1.3.7

The two bytes are `11000010 10100011`. The leading prefix `110` requires one continuation byte, whose prefix is correctly `10`. Removing these prefixes leaves payload groups `00010` and `100011`. Concatenation gives $00010100011_2 = 128 + 32 + 2 + 1 = 163 =$ `0x00A3`.
The result lies in the two-byte range U+0080–U+07FF, so the encoding is minimal and represents a Unicode scalar value. The answer is U+00A3, the pound sign.

← Return to the exercise

Solution 1.3.8

The declaration contains five `char` elements, so `sizeof s` is 5. `strlen` is 2 because it stops after `0x48 69` at the first zero. Bytes `0x21 00` remain stored after that terminator, but ordinary C string functions do not treat them as part of the first string.

← Return to the exercise

Solution 1.3.9

Five pixels at 3 bytes each require 15 payload bytes per row. The next multiple of 4 is 16, so stride is 16 bytes. Four rows occupy $4 \cdot 16 = 64$ bytes. The logical pixel data is only 60 bytes; four padding bytes are present across the rows.

← Return to the exercise

Solution 1.3.10

Mono audio has one channel. A rate of 48 kHz means 48,000 samples per second, so 30 seconds contains $48{,}000 \cdot 30 = 1{,}440{,}000$ samples. Each 24-bit sample occupies $24/8 = 3$ bytes. The raw data therefore occupies

$$1{,}440{,}000 \cdot 3 = 4{,}320{,}000 \text{ bytes}, \qquad \frac{4{,}320{,}000}{1{,}048{,}576} \approx 4.12 \text{ MiB}.$$

This count assumes packed samples with no per-sample padding, compression or container header.

← Return to the exercise

Solution 1.3.11

The first four bytes are the required signature `DATA`. The big-endian length field `0x00 03` is 3. The next three payload bytes are `0x41 42 43`, ASCII `ABC`. The total file length is 4+2+3=9 bytes, consistent with the supplied sequence.

← Return to the exercise

Solution 1.3.12

U+03BB is hexadecimal `03BB`. Splitting its bits across the two-byte UTF-8 template `110xxxxx 10xxxxxx` gives high five bits `01110` and low six bits `111011`. The result is `11001110 10111011`, or `CE BB`.

Return to the exercise

Solution 1.3.13

One example is precomposed é, U+00E9, encoded as `C3 A9`, versus e followed by U+0301 COMBINING ACUTE ACCENT, encoded as `65 CC 81`. They can render alike but have different code-point and byte sequences. Byte equality tests the stored representation. NFC normalisation maps these canonically equivalent sequences to the same precomposed form. Normalisation does not in general equate every pair of visually similar characters; appearance also depends on the font and shaping rules.

Return to the exercise

Solution 1.3.14

There are $3840 \times 2160 = 8,294,400$ pixels. Four channels at 16 bits each use 64 bits, or 8 bytes, per pixel. Raw storage is $8,294,400 \times 8 = 66,355,200$ bytes, about 63.28 MiB, excluding headers, row padding and compression.

Return to the exercise

Solution 1.3.15

Define bytes 0–3 as the fixed ASCII signature `ACS1`; byte 4 must equal 1. Byte 5 is a flags field: bit 0 requests a checksum and bit 1 marks UTF-8 text, so allowed values are 0–3 and bits 2–7 must be zero. Bytes 6–9 hold an unsigned 32-bit payload length L in big-endian order, with representable range $0 \leqslant L \leqslant 2^{32} - 1$. This version permits exactly one record and no trailing bytes.

The parser first requires at least ten bytes, then checks the signature, version and flag mask. It decodes the four length bytes using unsigned arithmetic, and compares L with the available size minus ten; subtraction occurs only after the minimum-size check. Equality is required. It also enforces an application resource limit before allocation and validates UTF-8 when bit 1 is set. If bit 0 is set, the payload ends with a four-byte CRC-32/ISO-HDLC value in big-endian order; L must be at least four and includes those checksum bytes. The CRC covers only the preceding payload bytes, and a text flag applies only to that preceding text.

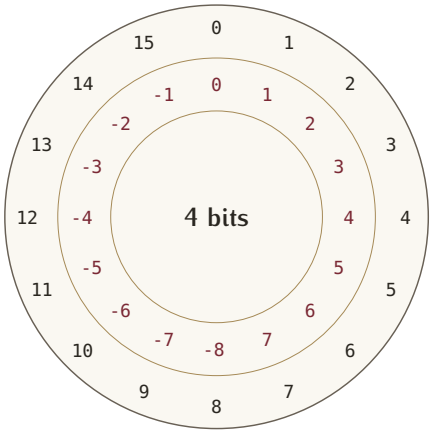
For example, `0x41 43 53 31 01 02 00 00 00 03 41 42 43` is a thirteen-byte record with version 1, UTF-8 flag, no checksum, and three payload bytes encoding `ABC`. The size check is $13 - 10 = 3 = L$.

Return to the exercise

1.4 SIGNED INTEGERS AND TWO'S COMPLEMENT

An eight-bit adder receives `0xFF` and `0x01`. Its low eight result bits are zero. Read the operands as unsigned integers and the mathematical sum is 256; read them as two's-complement integers and the sum is $-1 + 1 = 0$. The bit-level operation is shared, while the interpretation changes the conclusion about overflow. This example will anchor the signed representation, its range and the separate rules imposed by a programming language.

Two names for the same sixteen patterns



Outer values are unsigned; inner values use four-bit two's complement.

Figure 1.28: *Each angular position corresponds to one four-bit pattern. Positions zero through seven have equal signed and unsigned values; positions eight through fifteen represent minus eight through minus one under the signed interpretation.*

1.4.1 Two's complement as a fixed-width interpretation

For a w-bit object, two's complement assigns weights

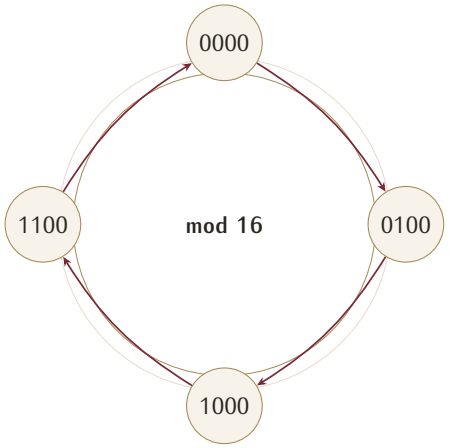
$$-2^{w-1},\ 2^{w-2},\ \ldots,\ 2^{1},\ 2^{0}$$

to the bits. Equivalently, if the most significant bit is zero, the pattern has the same value as an unsigned integer; if that bit is one, subtract 2^w from the unsigned value. On eight bits, for example, `0x85` is 133 unsigned but

$$133 - 256 = -123$$

under the signed two's-complement interpretation.

Four-bit patterns wrap around a finite ring



These landmarks are four increments apart on the sixteen-pattern ring.

Figure 1.29: *The ring represents arithmetic modulo 16, with only four landmark patterns drawn. Unsigned and two's-complement interpretations name the same patterns differently; the bit-level wraparound remains the same.*

The representable range is therefore asymmetric:

$$-2^{w-1} \leqslant x \leqslant 2^{w-1} - 1.$$

With eight bits the range is -128 through 127. The asymmetry matters: there is no positive counterpart of -128 in the same width, which is why negating the minimum signed value needs special care in both proofs and programs.

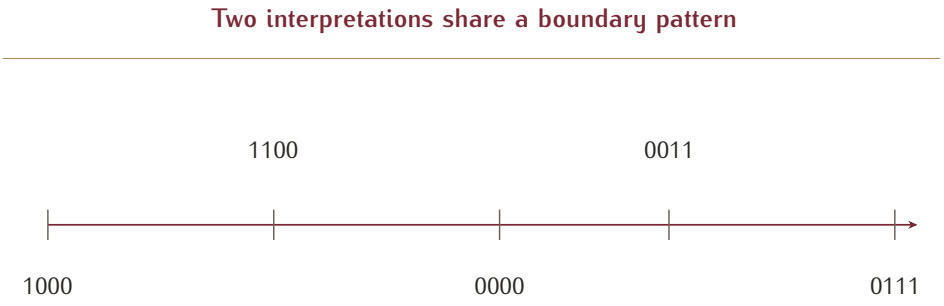


Figure 1.30: *The most significant bit has weight minus eight. The remaining bits contribute positive weights. The pattern 1000 is therefore minus eight, and there is no positive eight in this signed four-bit range.*

To obtain the two's-complement bit pattern for $-x$ within a fixed width, invert the bits of x and add one. This follows from modular arithmetic because $-x$ is represented by $2^w - x$ modulo 2^w. Thus eight-bit 5 is `0x05=00000101`, while inverting and adding one gives `11111011=0xFB`, whose signed interpretation is -5.

Reading a signed byte

The eight-bit pattern `0xF6` has unsigned value 246. Since the high bit is one, its signed value is $246 - 256 = -10$. The same byte has therefore not changed; only the interpretation has. This distinction becomes critical when C converts between signed and unsigned types or when a debugger prints raw memory.

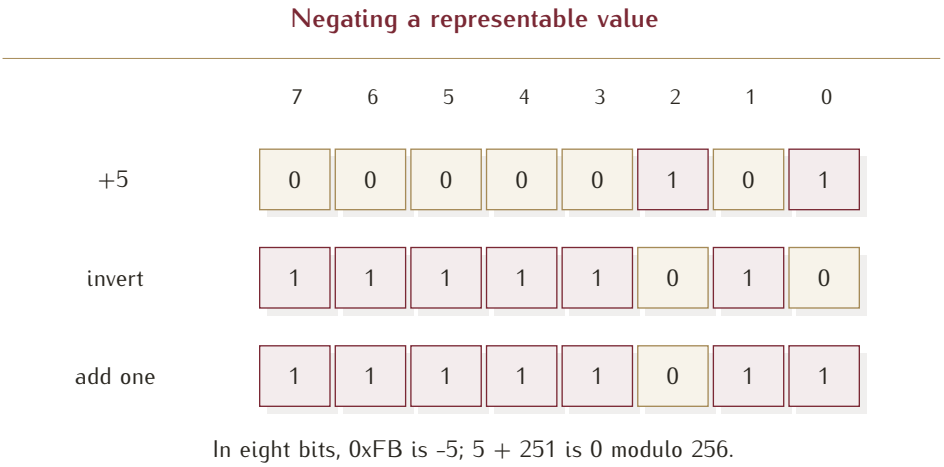


Figure 1.31: *Bit inversion followed by addition of one constructs the additive inverse modulo 256. Interpreting 11111011 as signed gives minus five. The minimum signed value needs special care because its positive counterpart is outside the range.*

Figure 1.32 keeps the stored object fixed while changing the interpretation. The sign convention and the selected field determine which weights participate in the calculation.

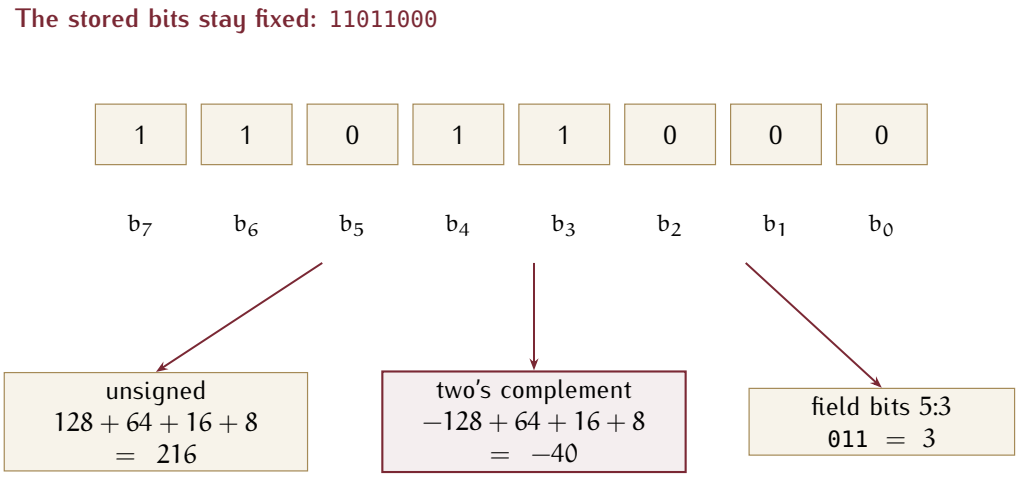


Figure 1.32: *Three interpretations of the same eight-bit object. Signedness changes the weight of the highest bit; field extraction selects positions and discards the others. State the interpretation before calculating a value.*

Changing one position in three interpretations

Clear bit 7 of `0xD8`, obtaining `0x58`. The unsigned value decreases from 216 to 88. The two's-complement value changes from −40 to 88, an increase of 128. Bits 5 through 3 remain `011`, so their extracted value stays 3. Each result follows from the weights shown in Figure 1.32.

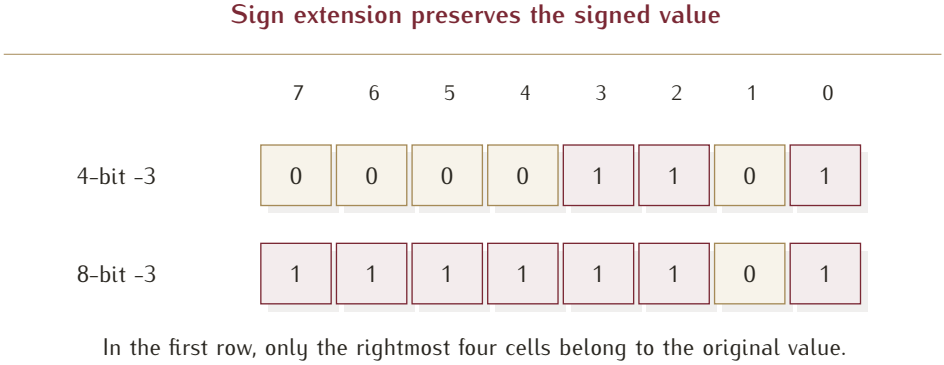


Figure 1.33: *The original four-bit pattern is 1101. Extending its sign fills the new upper four bits with ones, giving 11111101. Zero extension would instead produce positive thirteen.*

1.4.2 Addition, subtraction and the meaning of overflow

A fixed-width adder computes low-order result bits regardless of whether software later interprets those bits as signed or unsigned. If the mathematical sum exceeds the available width, carry information leaves the most significant position and the stored result wraps modulo 2^w. Unsigned C arithmetic deliberately uses this modular model. Signed overflow asks a different question: whether the mathematical result lies outside the signed representable range.

For unsigned addition, the carry out of the most significant bit is the natural indication that the mathematical result did not fit. For signed two's-complement addition, carry alone is not the criterion. Overflow occurs when two operands with the same sign produce a result with the opposite sign. Thus on eight bits

$$100 + 40 = 140$$

produces the bit pattern `0x8C`, whose signed interpretation is −116; because two positive operands produced a negative signed result, signed overflow occurred even though the low eight result bits are exactly what the adder is designed to emit.

Overflow flags are produced by real carry paths

Unsigned carry and signed overflow are different status conditions associated with the same addition.

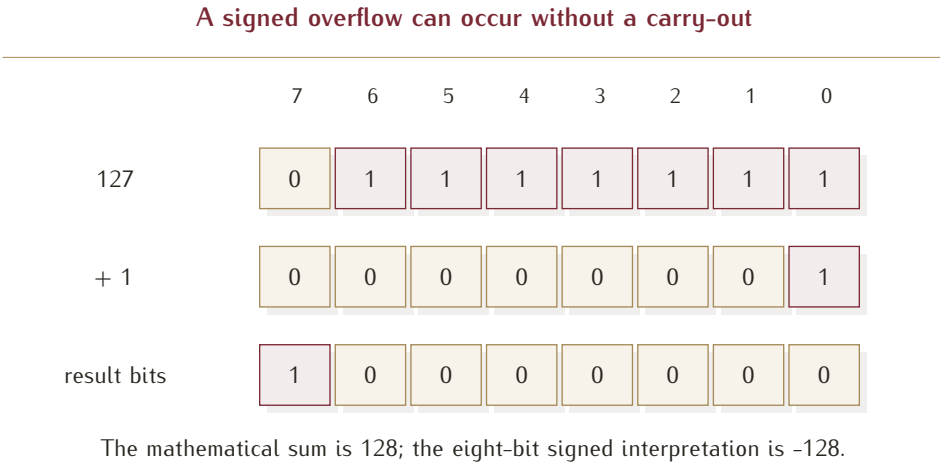


Figure 1.34: *Two positive signed operands produced a negative-looking result pattern because the mathematical sum exceeded 127. The unsigned result 128 is representable, so signed overflow and unsigned carry answer different questions.*

A hardware adder propagates or predicts carry between bit positions. Carry out of the most significant position supports unsigned reasoning, while signed overflow can be detected from the relationship between the carry into and out of the sign position or, equivalently, from operand and result signs.

One arithmetic operation can therefore update several architectural flags at once. Later conditional branches choose which flag interpretation is relevant rather than asking the adder to perform a different numerical operation [26, 27].

Subtraction can reuse the same adder because

$$x - y = x + (\sim y + 1)$$

within the fixed width. This is one reason two's complement became so attractive: addition and subtraction need not be implemented as unrelated numerical systems. Comparisons also reuse arithmetic status, although signed and unsigned comparisons inspect different flag relationships because the same bit patterns represent different ordered sets.

Sign extension preserves a signed value when a two's-complement object is widened: replicate the original sign bit into the newly added high positions. The eight-bit pattern `0xFB` (-5) therefore becomes `0xFFFB` on sixteen bits. Zero extension would instead produce `0x00FB`=251 and would preserve the unsigned interpretation rather than the signed one.

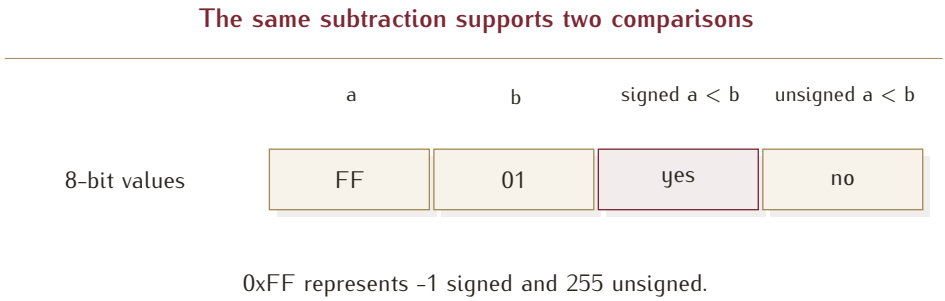


Figure 1.35: *The operands are the same eight-bit patterns in both columns. Signed comparison reports minus one below one; unsigned comparison reports 255 above one. The chosen condition must match the intended interpretation.*

The hardware cost of sign extension

Datapaths can implement this by wiring or replication logic, but the widened value still consumes routing, register and execution resources at its new width. Modern processors often incorporate extension into load or arithmetic operations rather than treating it as a separate instruction.

The mathematical rule is simple, yet the chosen operand width influences instruction encoding, register dependencies and data movement. Correct representation and efficient implementation are related but distinct questions.

1.4.3 The C language contract: representation, operations and conversions

For standard signed integer types, C23 specifies two's-complement representation. The representation requirement leaves the separate arithmetic rule intact: signed overflow has undefined behaviour. If a signed operation produces a mathematical result outside the type's representable range, the C language still does not grant ordinary wraparound semantics. A compiler may therefore optimise under the assumption that a well-defined execution has not performed such an overflow, even though the underlying machine adder naturally produces low-order result bits.

This separation between language and hardware is essential when reading generated code. An x86 `add` instruction updates a finite-width register or memory destination and status flags; a C expression that compiled into that instruction is governed first by C's rules. The fact that the hardware has a deterministic wrapped bit pattern does not retroactively define a source-language operation that had undefined behaviour.

Conversions add another layer. Converting a signed value to the corresponding unsigned width maps it into the unsigned range modulo 2^w. Under C23, converting an unsigned value to a signed type that cannot represent it gives an implementation-defined result or raises an implementation-defined signal; when code needs an exact

bit pattern, fixed-width unsigned types and explicit operations usually make the intended contract clearer.

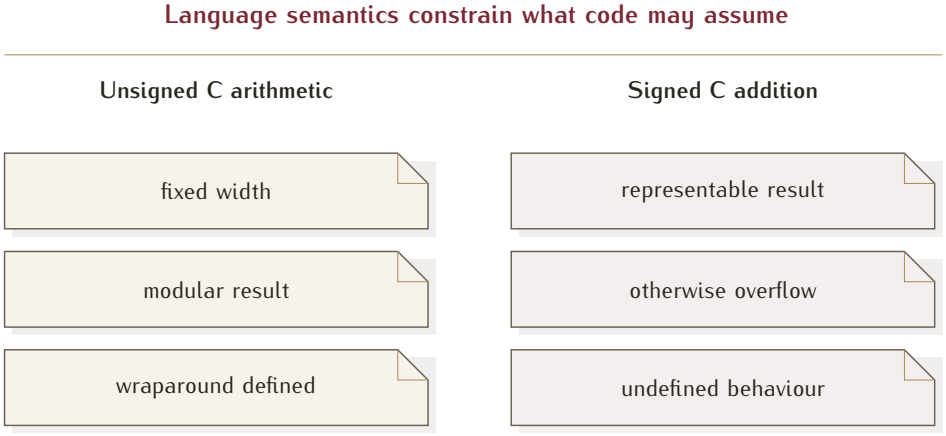


Figure 1.36: *The hardware result pattern and the C language contract are separate facts. Optimisation is governed by language semantics. Signed arithmetic therefore needs representability checks or a documented checked operation when overflow is possible.*

Integer promotions and the usual arithmetic conversions can also change the width and signedness before an operation occurs. A small unsigned type may first be promoted to `int`; mixed operands are resolved using integer conversion rank and representable ranges. When `int` and `unsigned int` have the same rank, the signed operand is converted to `unsigned int`. Comparisons that appear obvious at the source level can therefore surprise a programmer who reasons only from the declared spellings of the variables.

Why a mixed comparison can surprise

Suppose a negative `int` is compared with an `unsigned int` of the same rank. The usual arithmetic conversions convert the negative value to unsigned before the comparison, producing a large unsigned value modulo 2^w. Determine the converted operand types before reasoning about the comparison.

1.4.4 Checking arithmetic before it fails

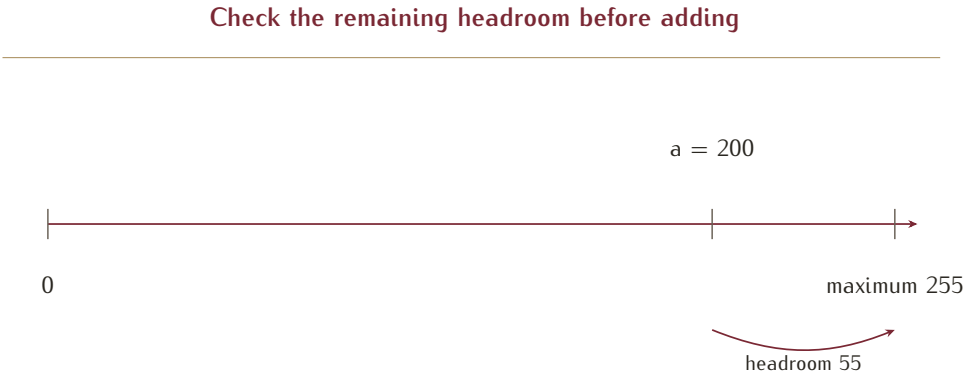


Figure 1.37: *Subtracting the current value from the unsigned maximum yields the remaining capacity. For a equal to 200, adding 55 reaches 255 and adding 56 would wrap. The comparison is made before evaluating the potentially wrapping sum.*

Because evaluating an overflowing signed expression can already be outside the source-language contract, safe code often checks a bound *before* performing the operation. For positive multiplication, a condition of the form

$$a > \frac{\mathrm{MAX}}{b}$$

when $b > 0$ can establish that ab would exceed the representable maximum without first computing the dangerous product. Unsigned arithmetic can also be checked against application bounds even though modular wrap itself is defined; defined arithmetic is not automatically correct arithmetic for sizes, offsets or resource counts.

The minimum signed value deserves special treatment. Since -2^{w-1} has no positive counterpart, expressions such as `-x` or `abs(x)` require care when `x` may be the minimum. The same asymmetry appears in division when the minimum value is divided by -1: the mathematical positive result is one larger than the signed maximum.

Bit-level identities require an explicit width and arithmetic model. In unsigned modular arithmetic, `x & -x` isolates the least significant set bit. For nonzero x, let that bit have index k: the k lower bits are zero. Complementing and adding one restores those low zeros, leaves bit k set and complements every higher bit. The AND therefore retains exactly 2^k. For $x = 0$, the result is zero. In C, use an unsigned arithmetic type such as `unsigned int`; narrower operands may undergo integer promotion before negation. For example, $x = \texttt{0x58} = 01011000_2$ has modular negative `0xA8`, and their AND is `0x08`.

Integer bugs often become memory-safety bugs only later. A wrapped or misconverted length can cause an allocation to be smaller than intended, after which a copy based on the original logical length writes beyond the object. Security-sensitive code therefore validates ranges and multiplication/addition bounds at the representation boundary, before the value is trusted as a size or offset.

Laboratory 4: Signed and unsigned semantics

For a 32-bit two's-complement model, interpret `0xFFFFFFFF` as unsigned and signed, compute unsigned maximum plus one, and explain the C function `int f(int x) { return x+1>x; }`. Give a defined C experiment.
Worked solution

Laboratory 4: solution

The bit pattern denotes unsigned $2^{32}-1 = 4{,}294{,}967{,}295$ and signed -1. Unsigned arithmetic wraps modulo 2^{32}, so maximum plus one is zero. In C, signed overflow is undefined; for every input where `x+1` is defined, the comparison in `f` is true. An optimiser may therefore return true without an addition. A defined experiment is:

```c
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>
int main(void) {
    uint32_t u = UINT32_MAX;
    int32_t s = -1;
    uint32_t wrapped = u + UINT32_C(1);
    printf("%" PRIu32 " %" PRId32 " %" PRIu32 "\n",u,s,wrapped);
}
```

It prints `4294967295 -1 0` on the stated exact-width platform. Inspect `f` at different optimisation levels as a code-generation exercise; do not execute signed overflow to infer a portable result.
Return to the laboratory

The durable method is to keep three questions separate. First, what bit pattern does the fixed-width machine operation produce? Second, what value does a chosen signed or unsigned representation assign to that pattern? Third, what operations does the source language permit the program to perform on that type? Two's complement makes those layers fit together elegantly, but it does not erase the boundaries between them.

How signed integers converged on two's complement

Timeline

Year	Machine, standard or event	Architectural significance
1954	**IBM 704**	Sign-magnitude arithmetic remains commercially important.
1964	**IBM System/360**	Two's complement is adopted across a major compatible family.
2024	**C23 publication**	Standard signed integers are required to use two's-complement representation.

Origins and early engineering constraints

A negative integer has no unique physical representation. A machine designer must choose how a fixed set of bits encodes sign and magnitude and how the arithmetic unit interprets the encoding. Early computers used several incompatible answers. Sign-and-magnitude placed the sign in one bit and the absolute value in the rest, closely matching written notation. One's complement represented a negative value by inverting every bit of the positive value. Two's complement inverted the bits and added one. All three were technically workable, and the eventual dominance of two's complement followed from datapath economy and software compatibility rather than mathematical necessity [26, 27, 3].

Sign-and-magnitude was common in early scientific machines. The IBM 704, announced in 1954 and delivered from 1955, used 36-bit words and provided fixed- and floating-point arithmetic. Its fixed-point representation treated the high-order bit as a sign, leaving a positive and a negative zero. Addition required logic to compare signs and magnitudes before deciding whether to add or subtract. The format was intelligible to programmers working from mathematical notation, but it complicated a general-purpose adder and made equality and zero testing special cases [110, 67].

One's complement reduced subtraction to addition of a complemented operand, but it also retained two zeros. When an addition produced a carry out of the most significant bit, the carry had to be added back into the low bit, the "end-around carry." The Control Data 6600, designed by Seymour Cray and delivered in 1964, used 60-bit one's-complement words. Its architecture demonstrates that one's complement was not a primitive mistake: it could support an exceptionally fast and influential computer. It did, however, require programmers, compilers and comparison logic to understand both zero encodings and the end-around rule [119, 120].

The shared-adder construction explained above gave two's complement a practical advantage: one zero encoding and one addition datapath support both signed and unsigned operands. This reduced special-case control in general-purpose arithmetic units [26, 27, 3].

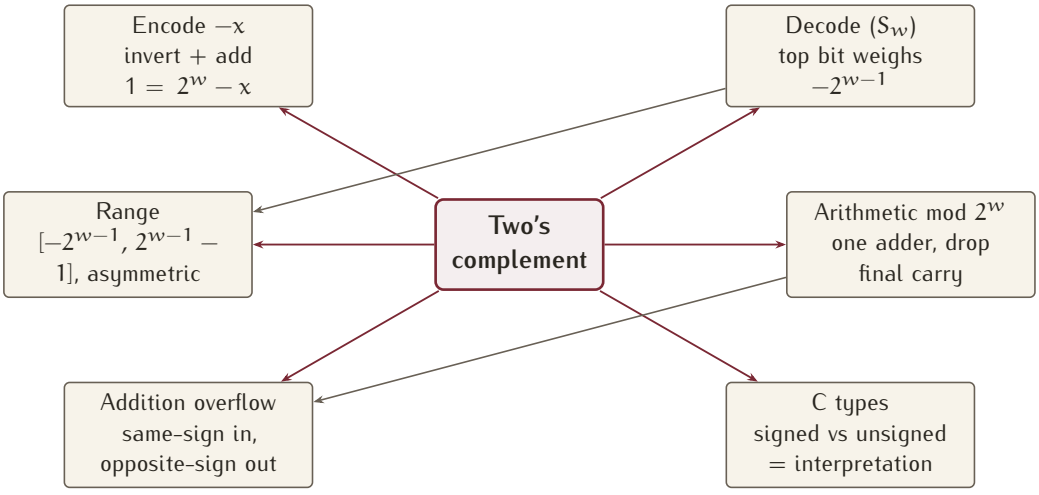
The Manchester stored-program machines used subtraction as a central operation. The 1948 Baby had a hardware subtractor and Tom Kilburn's first program found factors by repeated subtraction. Its 32-bit memory words illustrate the close relationship between representation and available hardware: the instruction set was shaped around the operation that the datapath implemented directly. During the 1950s and 1960s, two's complement spread through minicomputers and general-purpose architectures.

IBM System/360, announced in 1964, defined two's-complement binary integers alongside packed and zoned decimal formats. DEC's PDP-5 documentation of 1964 explicitly described 12- and 24-bit two's-complement operands. By making the representation part of an architectural family rather than a single circuit, these machines turned it into a software contract [89, 63, 109, 118].

Language standards took longer to converge than hardware. Traditional C retained all three signed representations. C23, published as ISO/IEC 9899:2024, finally required two's complement while preserving the signed-overflow rules discussed earlier [2, 3].

Modern processors add operations that deliberately choose other arithmetic meanings. Saturating SIMD instructions clamp results at a minimum or maximum, which is useful in audio, image and machine-learning code. Widening multiplication preserves more product bits. Checked-arithmetic libraries report overflow instead of discarding it. Cryptographic code often relies on unsigned modular arithmetic, while memory sizes and pointer offsets demand explicit bounds checks. A hexadecimal dump still does not disclose which interpretation is correct [26, 27, 3].

Concept map: signed integers



Reading the map. One design choice propagates through the whole representation: encode $-x$ as $2^w - x$. It fixes both the negative weight of the top bit and the asymmetric range. It also lets one modulo-2^w adder serve both signed and unsigned arithmetic; only the interpretation of the resulting bits, and the appropriate overflow flag, changes.

1.4.5 Worked problems

Encode a negative

Question. Encode -37 in eight-bit two's complement.
The width fixes the modulus at $2^8 = 256$. The unsigned value of the required pattern is $256 - 37 = 219$, which expands as $128 + 64 + 16 + 8 + 2 + 1$. Thus the pattern is $11011011_2 = \texttt{0xDB}$.
An independent route starts from $37 = 00100101_2$, inverts the bits to `11011010`, and adds one. Both routes yield `11011011`; decoding subtracts 256 and recovers -37. For -18, the same calculation gives $256 - 18 = 238 = 11101110_2$.

Decode a pattern

Question. Decode 11100110_2 as an eight-bit two's-complement integer.
The unsigned sum of its set-bit weights is $128 + 64 + 32 + 4 + 2 = 230$. The high bit is one, so signed decoding subtracts the modulus: $230 - 256 = -26$. Equivalently, assigning the high bit weight -128 gives $-128 + 64 + 32 + 4 + 2 = -26$ directly.
Re-encoding -26 produces $256 - 26 = 230$, restoring the original pattern. Applying the same rule to `10010110` gives $150 - 256 = -106$.

Modular addition

Question. Add unsigned eight-bit values 250 and 20, retaining an eight-bit result.
The exact sum is 270. Divide it into one complete modulus and a remainder: $270 = 1 \cdot 256 + 14$. The low byte is therefore $14 = \texttt{0x0E}$ and the carry-out is one, indicating unsigned overflow.
Signed interpretation of the same operands gives $-6 + 20 = 14$, which fits the signed range. Thus the result bits are shared while the two overflow tests answer different questions. For unsigned $200 + 100$, the analogous decomposition is $300 = 256 + 44$, giving low byte `0x2C` with carry one.

Signed overflow

Question. Add 100 and 60 as signed eight-bit values. Find the returned pattern and overflow status.
The mathematical sum is 160, beyond the signed maximum 127. Its low eight bits are $10100000_2 = \texttt{0xA0}$, whose signed interpretation is $160 - 256 = -96$. The same-sign positive inputs have produced a negative interpreted result, so signed overflow is set. Unsigned carry is clear because 160 fits in eight unsigned bits.

Worked example (continued)

For comparison, $-90 + (-60) = -150$ lies below -128. Adding 256 gives low-bit value 106, or `0x6A`; the positive interpreted result again signals signed overflow for the same-sign negative operands.

Exercise 1.4.1 — Range

Give the signed range on 12 bits.

Worked solution

In a signed 12-bit two's-complement representation, the high bit has weight -2^{11} and the remaining bits can contribute from 0 through $2^{11} - 1$. The minimum pattern is a leading 1 followed by eleven zeros, giving -2048. The maximum is a leading 0 followed by eleven ones, giving 2047. Thus the range is $\boxed{[-2048, 2047]}$, containing $4096 = 2^{12}$ values.

Exercise 1.4.2 — Encode

Encode -9 on 8 bits.

Worked solution

The positive magnitude is $9 = 00001001_2$. Invert every bit to obtain `11110110`, then add one to obtain `11110111`. Its unsigned interpretation is 247; interpreting the same byte as two's complement gives $247 - 256 = -9$. The result is therefore $\boxed{11110111_2}$. Adding the patterns for $+9$ and -9 produces zero modulo 256, providing an independent check.

Exercise 1.4.3 — Wrap

Compute 5-12 modulo 256.

Worked solution

The mathematical difference is $5 - 12 = -7$. A one-byte unsigned result is reduced modulo $2^8 = 256$, so add 256 to obtain 249. Since $249 = 128 + 64 + 32 + 16 + 8 + 1$, its bits are $\boxed{11111001}$. Interpreting these same bits as signed two's complement gives $249 - 256 = -7$. The question asks for the modulo result, whose unsigned decimal value is 249.

Exercise 1.4.4 — Flags

Does a final carry necessarily imply signed overflow?

Worked solution

Carry-out and signed overflow test different ranges. In eight bits, `0xFF+0x01` produces `0x100`: the stored byte is zero and there is a carry. Its signed interpretation is $-1+1=0$, which is representable, so signed overflow is clear. Conversely, `0x7F+0x01` stores `0x80` without carry, while signed $127+1$ exceeds the maximum. A final carry therefore cannot determine the signed-overflow flag.

Summary

Two's complement becomes easy once the width is always visible. For a hardware or bit-vector calculation, write the width, compute modulo 2^w, then interpret the resulting pattern under the required signedness. For C expressions, first apply promotions and conversions, and check whether the operation is defined. Overflow occurs when the mathematical result lies outside the range selected by the operation's signedness and width. Comparisons, casts and security-sensitive size calculations all depend on this discipline.

1.4.6 Exercises

Exercise 1.4.5 — Decode by positional weights

Decode the 8-bit two's-complement pattern 10110110_2 using the negative high-bit weight formula.

Go to the solution →

Exercise 1.4.6 — Encode and verify

Encode -93 as an 8-bit two's-complement value and verify it by adding the representation of $+93$.

Go to the solution →

Exercise 1.4.7 — Carry versus signed overflow

For each 8-bit addition, state the stored result, carry-out condition and signed-overflow condition: (a) `0xFF+0x01`; (b) `0x7F+0x01`; (c) `0x80+0x80`.

Go to the solution →

Exercise 1.4.8 — Sign extension

Sign-extend 10010110_2 from 8 to 16 bits and give the decimal value. Then zero-extend the same byte and give that value.

Go to the solution →

Exercise 1.4.9 — Mixed comparison

Assume 32-bit `int` and `unsigned int`. Evaluate the truth of `-2 < 1u` and explain the conversion.

Go to the solution →

Exercise 1.4.10 — Precondition for multiplication

Write a precondition that allows unsigned `a * b` to be computed without overflow when `a` and `b` have type `size_t`.

Go to the solution →

Exercise 1.4.11 — Narrowing

The 16-bit unsigned value `0x12F4` is converted to an 8-bit unsigned type. What value and bit pattern result? Explain mathematically.

Go to the solution →

Exercise 1.4.12 — Security analysis

On the stated platform, where `uint32_t` is `unsigned int`, a program reads unsigned 32-bit `width` and `height`, computes `uint32_t bytes = width * height * 4`, allocates `bytes`, then decodes `width * height` pixels. Explain the defect and propose a safe sequence of checks.

Go to the solution →

Exercise 1.4.13 — The five-bit signed number circle

Draw the 5-bit two's-complement number circle and mark where positive and negative overflow occur.

Worked solution

Exercise 1.4.14 — Why sign extension preserves value

Prove that sign extension preserves the two's-complement value.

Worked solution

Exercise 1.4.15 — Carry and signed overflow

Find a case where unsigned overflow occurs without signed overflow and a case where signed overflow occurs without unsigned carry.
Worked solution

Exercise 1.4.16 — A reverse loop that handles zero length

Rewrite a reverse loop over an array so that it is correct for `size_t` indices including an empty array.
Worked solution

1.4.7 Solutions

Signed Integers and Two's Complement

Solution 1.4.5

The high bit contributes -128. The remaining set bits have weights 32, 16, 4 and 2. Thus the value is $-128 + 32 + 16 + 4 + 2 = -74$. As a cross-check, invert to 01001001, add one to get $01001010 = 74$, then attach the minus sign.

← Return to the exercise

Solution 1.4.6

$93 = 01011101_2$. Invert: 10100010; add one: 10100011. Verification: $01011101 + 10100011 = 1\,00000000$. Discarding the carry yields zero, so 10100011 is the additive inverse modulo 256.

← Return to the exercise

Solution 1.4.7

(a) Stored `0x00`, carry yes; signed interpretation $-1 + 1 = 0$, so no signed overflow. (b) Stored `0x80`, no carry; signed $127 + 1$ exceeds the maximum, so signed overflow yes. (c) Sum is `0x100`, stored `0x00`, carry yes; signed $-128 + (-128)$ is below the minimum and the result is nonnegative, so signed overflow yes.

← Return to the exercise

Solution 1.4.8

The byte represents $-128 + 16 + 4 + 2 = -106$. Sign extension repeats the high bit: $11111111\,10010110_2$ = `0xFF96`, still -106. Zero extension gives $00000000\,10010110_2 =$ `0x0096` $= 150$.

← Return to the exercise

Solution 1.4.9

The operands have types `int` and `unsigned int`, which have equal conversion rank. The usual arithmetic conversions therefore convert the signed operand to `unsigned int`. The bit pattern for -2 becomes $2^{32} - 2$, which is greater than 1. The comparison is false.

← Return to the exercise

Solution 1.4.10

If `b == 0`, the product is zero and safe. Otherwise require `a <= SIZE_MAX / b`. An equivalent symmetric check exchanges the operands. The division is performed before multiplication and therefore cannot be affected by the would-be product overflow.

← Return to the exercise

Solution 1.4.11

Narrowing retains the low eight bits, `0xF4`, which is 244 unsigned. Mathematically, `0x12F4` $= 4852$ and $4852 \bmod 256 = 244$.

← Return to the exercise

Solution 1.4.12

The expression is evaluated with 32-bit unsigned arithmetic on the stated platform. Either multiplication can wrap modulo 2^{32} before assignment. Merely assigning to a wider destination would still lose the high bits. For example, $65536 \cdot 65536$ wraps to zero, so an empty allocation could be followed by an attempted decode of over four billion pixels.
Widen the operands before multiplying and check the final byte count before narrowing:

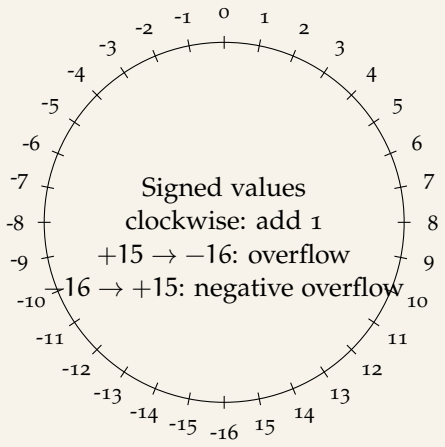
```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
bool rgba_bytes(uint32_t width, uint32_t height, size_t *out) {
    if (out == NULL) return false;
    uint64_t pixels = (uint64_t)width * (uint64_t)height;
    if (pixels > SIZE_MAX / 4) return false;
    *out = (size_t)pixels * 4;
    return true;
}
```

The largest product of two 32-bit unsigned inputs is $(2^{32} - 1)^2 < 2^{64}$, so the first multiplication fits. The comparison proves that both conversion to `size_t` and multiplication by four are safe. This helper accepts zero dimensions and returns zero bytes; the caller must handle an empty image without dereferencing storage. Before allocation, enforce format-specific dimension limits and verify the input payload length. After allocation, check for failure before decoding. On a 64-bit `size_t` platform, 65536×65536 correctly requires 17,179,869,184 bytes; a resource limit can reject that impractical size.

← Return to the exercise

Solution 1.4.13

A 5-bit pattern circle contains 32 positions. Unsigned values run 0–31; signed two's-complement values run 0–15 then -16– -1 for patterns `10000` through `11111`. Positive signed overflow crosses from `01111` (+15) to `10000` (-16). Negative signed overflow crosses below `10000` (-16) and wraps into the nonnegative half, for example -16-1 produces pattern `01111`.



Return to the exercise

Solution 1.4.14

For a nonnegative w-bit value, sign extension adds leading zeros and leaves the unsigned and signed value unchanged. For a negative pattern with unsigned value U, its signed value is $U - 2^w$. Extending by k one bits gives unsigned value $U + (2^{w+k} - 2^w)$; interpreting that as signed subtracts 2^{w+k} and yields $U - 2^w$, the original value.

Return to the exercise

Solution 1.4.15

Unsigned overflow without signed overflow: in 8 bits, `0xFF+0x01` produces `0x00` with carry, while signed -1+1 is 0 and has no signed overflow. Signed overflow without unsigned carry: `0x7F+0x01` produces `0x80` with no carry out but overflows from +127 to the negative half.

Return to the exercise

Solution 1.4.16

Start one position beyond the last valid index, and decrement only while a position remains:

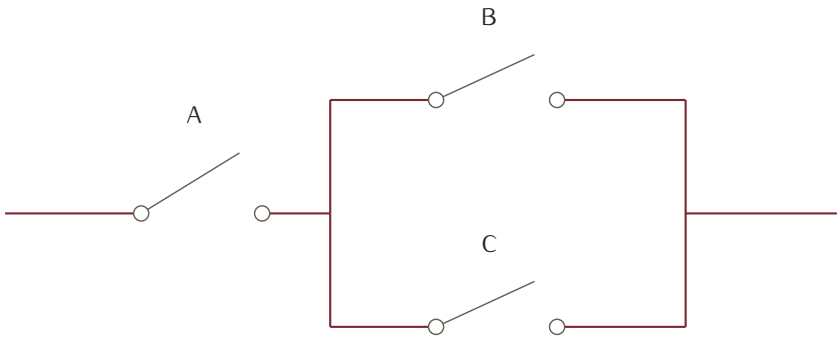
```
for (size_t i = n; i != 0; ) {
    --i;
    use(a[i]);
}
```

For $n = 3$, the body accesses indices 2, 1 and 0. For $n = 0$, the condition fails before the decrement or array access. The invariant is $0 \leqslant i \leqslant n$ at the loop test; after a successful test, decrement produces an index below n.
The compact form `for (size_t i=n; i-- > 0; ) use(a[i]);` also visits those indices. Its final failed test decrements zero and wraps `i` to `SIZE_MAX`; that defined unsigned wrap is harmless because the body is skipped. The first form avoids the wrap and makes the bound proof explicit.
Return to the exercise

1.5 BOOLEAN CIRCUITS AND BIT-LEVEL COMPUTATION

Suppose a device should activate when at least two of three inputs are high. A truth table can list every allowed input combination; an equation can express the rule; a circuit can implement it. These are three views of one specification. We will move between them, then apply the same operations to the independent bit positions of a machine word. A mask becomes a precise statement about which positions may change.

Boolean algebra can describe a conducting path



A conducting path requires A and at least one of B or C.

Figure 1.38: *The contacts form A in series with the parallel pair B and C. A path conducts exactly when A and at least one parallel contact are closed. Open contact positions are drawn to reveal the topology; the picture does not assert an input state.*

1.5.1 Boolean functions and elementary gates

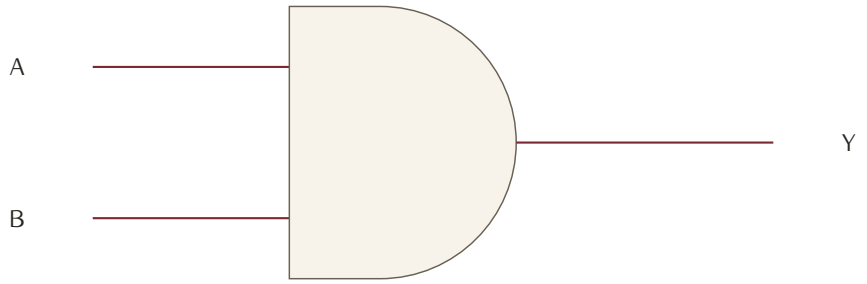
A Boolean variable takes one of two values, conventionally 0 and 1. A Boolean function maps one or more such inputs to an output in the same set. The elementary operations are NOT, AND and OR:

$$\neg A, \qquad A \wedge B, \qquad A \vee B.$$

NOT inverts one input; AND is true only when both inputs are true; OR is true when at least one input is true. XOR, written $A \oplus B$, is true when the inputs differ. A truth table lists the function exhaustively, so for two inputs it has four rows and leaves no ambiguity about the required output.

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$	$\neg A$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

An AND gate requires both inputs



Y = 1 exactly when A = 1 and B = 1.

Figure 1.39: *The gate symbol summarises a Boolean function. Its output is one for the single input combination in which both inputs are one. Physical propagation delay is omitted from this functional drawing.*

At the circuit level, transistor networks implement these Boolean functions. CMOS logic, for example, arranges transistor networks so that ranges of input voltage produce output voltages recognised as logical low or high. The Boolean model deliberately hides analogue transition time, capacitance and current once the input and output contracts are satisfied.

The CMOS inverter is an analogue circuit used digitally

A NOT gate maps 0 to 1 and 1 to 0.

A CMOS inverter uses complementary transistor networks whose conductance changes continuously with input voltage. Around the switching region both devices can conduct partially; the output transition has finite rise and fall times and must charge or discharge capacitance.

Boolean reasoning is valid because the circuit is engineered so that stable input regions produce restored output regions. Delay and energy remain physical costs even though the logical function itself is exact [41, 42].

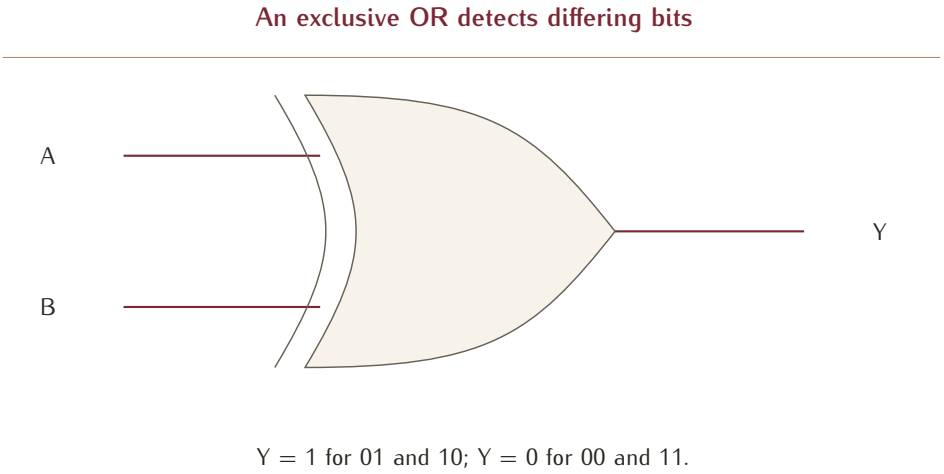


Figure 1.40: *Exclusive OR tests whether two bits differ. Applied position by position to words, it supports parity, bit toggling and reversible combination. The extra curved line distinguishes the XOR symbol from OR.*

NAND and NOR are especially important because either can form a functionally complete basis: any Boolean function can be constructed using only NAND gates or only NOR gates. That fact matters both historically and practically, but it should not be confused with a claim that every modern implementation literally replaces all logic with identical NAND cells. Synthesis tools choose among available cells according to delay, area, power and wiring constraints while preserving the same Boolean behaviour.

1.5.2 From truth tables to expressions and circuits

A truth table can be converted mechanically into a Boolean expression. One direct method takes every row whose output is 1, writes a product term that matches that row, and ORs the terms together. The resulting sum-of-products expression is correct by construction, although it may contain redundant logic. Boolean algebra then supplies identities such as

$$A \wedge 1 = A, \qquad A \vee 0 = A, \qquad A \wedge A = A,$$

$$\neg(A \wedge B) = \neg A \vee \neg B, \qquad \neg(A \vee B) = \neg A \wedge \neg B,$$

which can simplify the expression without changing its truth table.

For example, the majority-of-three function is true when at least two of A, B, C are true. Its simplified expression is

$$AB \vee AC \vee BC.$$

A circuit can implement exactly those three pairwise AND terms followed by an OR network. The derivation reveals why the circuit is correct: each product term identifies one sufficient pair, while the OR combines the alternatives.

A truth table enumerates every input case

	A	B	A AND B	A XOR B
case 0	0	0	0	0
case 1	0	1	0	1
case 2	1	0	0	1
case 3	1	1	1	0

Two Boolean inputs create four possible rows.

Figure 1.41: *The table makes the domain finite and explicit. Each row supplies one obligation for a circuit implementation. AND and XOR agree on some rows and differ on others, so a single test vector cannot establish equivalence.*

Canonical forms are useful because they provide a systematic bridge from specification to implementation, but they are not always minimal. Karnaugh maps can simplify small functions visually; exact algorithmic minimisation such as Quine–McCluskey is useful for modest problem sizes but grows rapidly with the number of variables; industrial synthesis relies heavily on heuristic transformations and technology mapping for large designs. The important principle is that the optimised circuit must preserve the original Boolean function.

1.5.3 Bit vectors, masks and the C operators

A machine register contains a vector of bits, so the same Boolean operation can be applied independently at every bit position. If

$$A = 10110100_2, \qquad B = 11000110_2,$$

then `A & B` computes eight AND operations in parallel, `A | B` computes eight OR operations, and `A ^ B` computes eight XOR operations. This parallel bit-vector interpretation is what makes masks so useful: selected positions can be tested, cleared, set or toggled without disturbing the others.

Suppose a permission byte uses bit 0 for read, bit 1 for write and bit 2 for execute. The mask `1u << 1` selects the write bit. Then

```
bool can_write = (p & (1u << 1)) != 0;
```

first performs a bitwise AND and then asks the logical question whether the masked result is nonzero. The two stages should not be collapsed conceptually: bitwise operators transform representations; logical operators interpret operands as truth values. The binary operators && and || short-circuit; unary ! negates the truth value of its single operand.

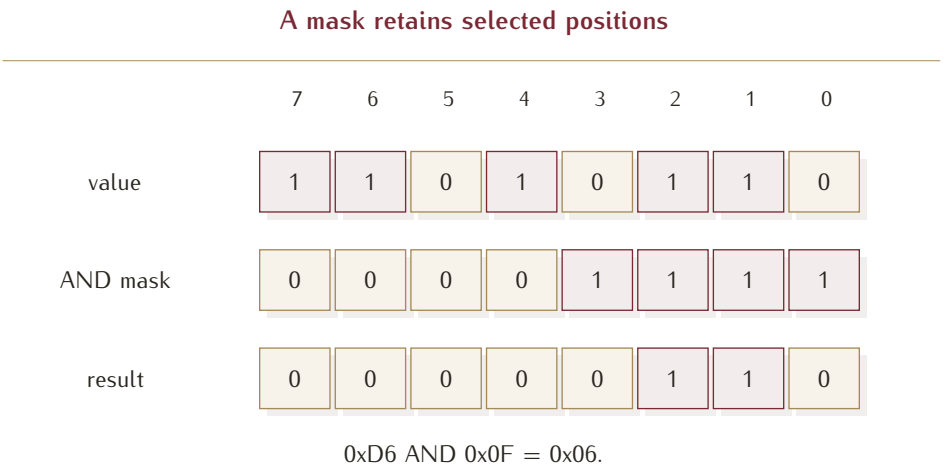


Figure 1.42: *A one in the mask preserves the corresponding input bit; a zero forces that result bit to zero. The low-nibble mask therefore extracts the final four positions without moving them.*

Bitwise AND is not logical AND

For integers `2` and `1`, the expression `2 && 1` is true because both operands are nonzero, whereas `2 & 1` is zero because the bit patterns `10` and `01` share no set position. The source syntax differs by one character, but the operations answer different questions.

A mask can also be viewed as the characteristic function of a set of bit positions. AND with the mask keeps members of that set; AND with its complement clears them; OR sets them; XOR toggles them. This set-like view often makes packed fields easier to reason about than memorising isolated idioms.

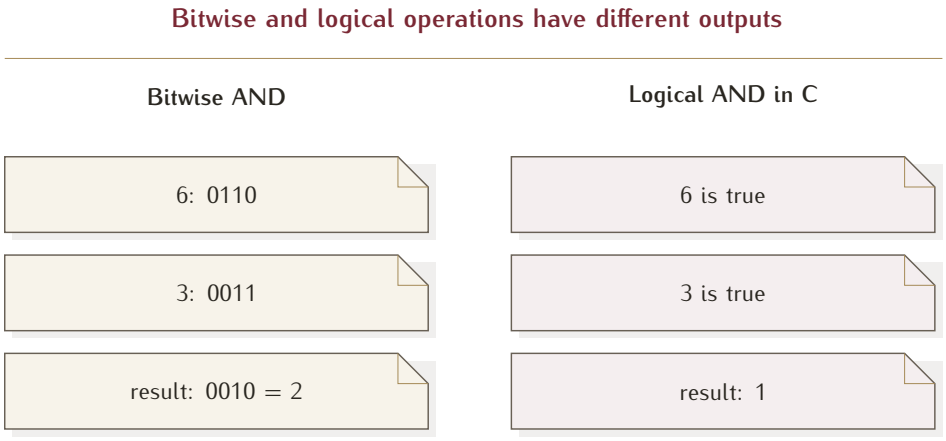


Figure 1.43: *The same integer operands produce two different results because the operators ask different questions. Logical AND also specifies short-circuit evaluation in C, while bitwise AND evaluates both operands.*

Shifts move bit positions in a structured way. For an operand whose promoted type is unsigned with width w, left shift by k multiplies by 2^k modulo 2^w, while right shift divides by 2^k and discards the remainder. Both require $0 \leqslant k < w$. In C, however, shift validity depends on operand type, promoted width and shift count; signed shifts require additional care. The x86 instruction set has its own hardware behaviour for shift counts and arithmetic versus logical right shifts. Later chapters therefore keep C semantics and instruction semantics separate rather than assuming that a source-level operator inherits every detail of one processor.

1.5.4 Timing, propagation and glitches

A combinational circuit has no architectural memory, but it still has physical delay. When an input changes, signals propagate through gates and wires before the output settles. A path containing several dependent gates has a propagation delay determined by the slowest relevant route, often called the critical path. Circuit optimisation considers gate count, the depth of dependent gates and the load on each node.

Unequal path delays can produce a temporary output pulse even when the initial and final Boolean values are both correct. A truth table describes stable input/output combinations. During a transition, unequal propagation delays can produce a brief *glitch*.

Unequal delays create glitches

A combinational network computes a Boolean function of its current inputs.

Different paths contain different gate and wire delays. When several inputs or intermediate signals change, one path can reach an output before another, briefly producing a value different from the settled result for the new input combination.

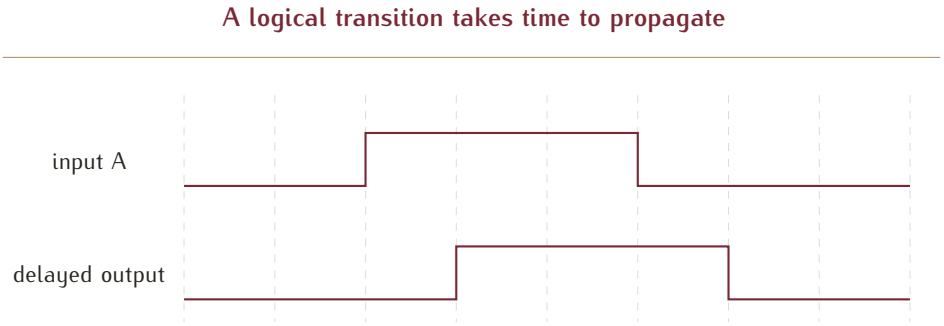


Figure 1.44: *The delayed waveform shifts each transition by one displayed interval. Combinational logic can be functionally correct while its output is temporarily unsettled. Sampling must allow enough time for the relevant paths to settle.*

Synchronous systems normally allow combinational logic time to settle before a receiving register samples it. Asynchronous interfaces, clocks and control pulses require stricter hazard-aware design because a transient may otherwise be interpreted as a real event [42, 41].

Fan-out and wiring contribute to the same timing problem. One logical output may drive many physical inputs, increasing capacitance; long interconnects can dominate delay in modern chips. A gate symbol specifies a logical function; its implementation determines delay, loading and energy cost. Logic synthesis and physical design preserve the Boolean specification while choosing cells, buffering and placement that meet timing and power constraints.

1.5.5 Arithmetic from Boolean building blocks

Binary addition shows how a numerical operation emerges from Boolean functions. A half adder takes input bits A and B and produces

$$S = A \oplus B, \qquad C = A \wedge B,$$

where S is the sum bit and C the carry. A full adder adds a carry-in C_{in} as well:

$$S = A \oplus B \oplus C_{in},$$

$$C_{out} = AB \vee AC_{in} \vee BC_{in}.$$

Connecting full adders across bit positions yields a multi-bit adder. A simple ripple-carry design lets each carry wait for the previous position, which makes the critical path grow with width; faster architectures predict or combine carry information more aggressively. The carry and overflow rules from Chapter 1.4 correspond to signals generated by this same arithmetic structure.

One full-adder row

For $A = 1$, $B = 1$ and $C_{in} = 0$, the mathematical one-bit sum is 2. The circuit therefore emits $S = 0$ and $C_{out} = 1$. For $A = 1$, $B = 0$ and $C_{in} = 1$, the same output pair appears for a different input combination. The truth table encodes those cases exactly; the Boolean equations are a compact implementation of the table.

An adder combines arithmetic and Boolean results

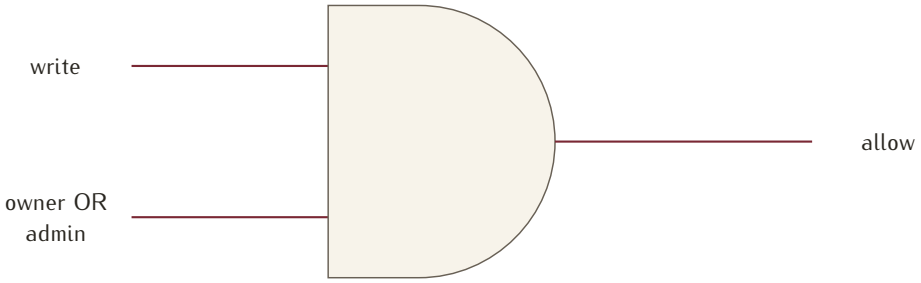


A + B + carry-in = S + 2 times carry-out.

Figure 1.45: *A full adder represents the sum of three one-bit inputs with a result bit and a carry bit. The equation states the numerical invariant that each stage must preserve when full adders are connected.*

1.5.6 Boolean reasoning in systems code

Permission policy can be drawn as a function



All input names denote decoded Boolean propositions.

Figure 1.46: *The write condition is necessary, and either ownership or administrator status supplies the other required condition. Packed permission bits must be extracted and validated before they are used as these Boolean inputs.*

Low-level code repeatedly uses Boolean structure to express policy and to manipulate packed state. Permission fields, device registers, instruction encodings and protocol headers all assign meanings to individual bits or small fields. The safe workflow is to define the field contract, construct masks with explicit unsigned widths, perform the intended bitwise transformation, and then convert the result into a logical proposition only when a decision is required.

Bit manipulation becomes dangerous when width, shift count or validation is implicit. An unchecked shift can exceed the source-language domain; a mask copied from a wider register can accidentally preserve reserved bits; a security check can be weakened if bitwise and logical operators are confused. Reserved fields should be validated explicitly rather than silently ignored when the surrounding protocol says they must be zero.

Laboratory 5: A Boolean specification

Specify a three-input alarm that is active when at least two inputs are one. Derive an expression, verify all rows in scalar and packed form, and describe its gate depth assuming each AND and OR is a two-input gate.
Worked solution

Laboratory 5: solution

The expression is $xy \vee xz \vee yz$. It covers pairwise simultaneous assertions, including row 111. A complete executable check is:

```
python3 - <<'PYCODE'
mask = 0
for x in range(2):
    for y in range(2):
        for z in range(2):
            i = 4*x+2*y+z
            a = (x and y) or (x and z) or (y and z)
            b = int(x+y+z >= 2)
            assert int(a) == b
            mask |= b << i
            print(x,y,z,b)
X,Y,Z = 0xf0,0xcc,0xaa
packed = (X & Y) | (X & Z) | (Y & Z)
assert packed == mask == 0xe8
print(hex(mask))
PYCODE
```

Outputs by rows ooo through 111 are $0,0,0,1,0,1,1,1$. Three parallel AND gates feed a two-level OR tree. A longest input-to-output path crosses one AND and two OR gates, giving three gate delays under the equal-delay model.
Return to the laboratory

A Boolean specification can be implemented in software or hardware. Each implementation must preserve the specified relation. A truth table states what relation must hold; algebra allows equivalent forms; C bitwise operators apply those relations to stored bit vectors; and transistor networks realise the function with finite delay and energy. Keeping those levels connected prepares the transition from representation to the combinational and sequential datapaths studied later in the book.

From Boolean algebra to integrated arithmetic logic

Timeline

Year	Machine, standard or event	Architectural significance
1854	**Boole's algebra**	Logical relations receive an algebraic calculus.
1938	**Shannon's paper**	Switching circuits are designed with Boolean algebra.
1970	**74181 ALU**	Arithmetic and logic are packaged as a reusable integrated function.

1938: calculating a circuit before wiring it

Telephone switching posed a practical design problem: a network of contacts had to connect a path under specified conditions, using a manageable number of components. Shannon's published analysis gave such networks an algebraic description. Series and parallel connections could be transformed using logical identities, so an engineer could compare circuits through their behaviour [21].

For a concrete example, suppose a path should conduct when A is closed and at least one of B or C is closed. Its condition is $AB + AC = A(B + C)$. The expression on the left suggests two parallel branches, each carrying an A contact. The expression on the right factors A into a shared series contact followed by the parallel pair B, C. Both implement the same truth table, while their contact arrangements differ.

Boole's algebra supplied the symbolic foundation; Shannon's work connected that reasoning to switching-network analysis and synthesis [20, 21]. The method survives in logic optimisation: preserve the required input-output relation while changing the implementation to meet physical constraints.

Relay computers made the correspondence physically visible. Konrad Zuse's Z3 of 1941 used roughly 2,600 electromechanical relays. The Harvard Mark I, placed in operation in 1944, combined electromechanical storage and control. Relays provided clean open and closed states but moved mechanically, limiting speed and eventually wearing. Vacuum tubes removed moving contacts. ENIAC, completed in 1945, used about 18,000 tubes and performed thousands of additions per second, but its size, heat and maintenance burden showed the cost of implementing each logical function with discrete active devices and extensive wiring [88, 87, 67].

The transistor changed the physical switch. On 23 December 1947, John Bardeen and Walter Brattain demonstrated the point-contact transistor at Bell Laboratories; William Shockley soon developed the junction-transistor concept. Transistors were smaller, required less power and could ultimately be manufactured in large numbers. Early tran-

sistor computers still assembled individual devices on boards, so interconnection and reliability remained limiting factors. The next step was to fabricate several devices as one circuit [51, 52, 100].

Jack Kilby demonstrated a working integrated circuit at Texas Instruments on 12 September 1958 using a piece of germanium with components connected by fine wires. In 1959 Robert Noyce at Fairchild Semiconductor proposed a silicon monolithic circuit that used Jean Hoerni's planar process and metal interconnections on the chip. Kilby's experiment established integration; Noyce's structure supplied a manufacturable route. The distinction is important because a laboratory proof and a scalable production technology are not the same historical event [100, 101].

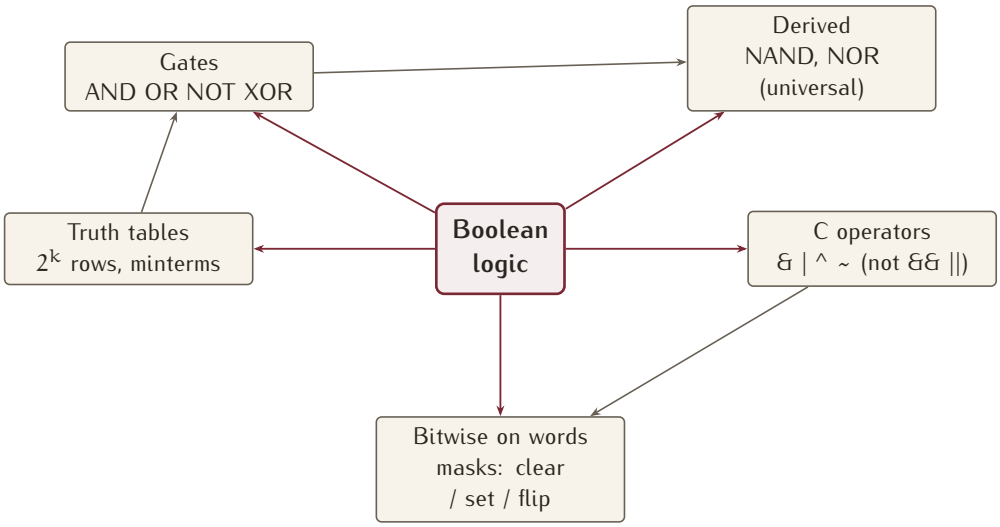
During the 1960s, transistor-transistor logic families made gates and functional blocks available as catalogued components. Designers could buy packages containing NAND gates, multiplexers, counters and adders and connect them on printed-circuit boards. The 74181 arithmetic-logic unit, documented by Texas Instruments in the early 1970s, accepted two four-bit operands and control inputs and produced arithmetic or logical functions plus carry-related signals. Several 74181 slices could be cascaded into a wider datapath. Its data sheet is a compact lesson in the relation between a truth table, active-low signals, carry look-ahead and a reusable physical part [102].

Large-scale integration moved the datapath onto one chip. Intel announced the 4004 on 15 November 1971. Developed by Federico Faggin, Marcian "Ted" Hoff, Stan Mazor and Masatoshi Shima for the Busicom calculator project, it implemented a four-bit CPU in about 2,300 transistors. The 4004 changed the economics of placing a programmable control unit inside a product. Eight-bit and sixteen-bit microprocessors followed, and the processor became a manufactured component rather than a cabinet assembled from boards [103, 71].

Moore's 1965 observation described the rising component density achieved by the integrated-circuit industry. Metal-oxide-semiconductor technology and especially CMOS allowed density to grow while reducing static power, but scaling created new constraints. Wire delay, dynamic switching energy, leakage, clock distribution, memory latency and verification effort became as important as the Boolean expression itself. A gate-level diagram still states logical dependence; it does not by itself determine timing, power or layout [23].

Modern design is written principally at register-transfer level in languages such as Verilog, SystemVerilog or VHDL. Synthesis tools map operators and Boolean conditions to libraries of characterised cells. Static timing analysis checks whether paths meet a clock period. Place-and-route tools account for physical wires. Formal equivalence tools compare transformed netlists with the intended logic. Process technologies have moved from planar MOSFETs to FinFET and gate-all-around structures, while advanced packages connect several chiplets. Yet an x86 integer unit, a vector lane, an AES block and a cache-coherence controller are still compositions of Boolean and sequential functions [51, 52, 100].

Concept map: gates and bit operations



Reading the map. Gates are the atoms; NAND/NOR show one atom suffices. Truth tables both *define* gates and, via minterms, let any function be synthesised from them (the road to 3.1's ALU). The same gates surface in software as C's bitwise operators, whose everyday use is masking and whose confusion with the logical operators is the chapter's main hazard.

1.5.7 Worked problems

Truth table

A three-input control signal is defined by $F = (A \wedge B) \vee C$. A truth table is the safest way to prove what the circuit does for every possible input.
Question. Complete all eight rows of the truth table. For which input combinations is $F = 1$?
Reasoning.

1. **List every input combination.** With three Boolean inputs there are $2^3 = 8$ rows: 000, 001, 010, 011, 100, 101, 110, and 111.
2. **Evaluate the inner operation first.** Compute the intermediate column $A \wedge B$. It is one only in rows 110 and 111.
3. **Combine with** C. Form $(A \wedge B) \vee C$. Whenever $C = 1$, the output is one; when $C = 0$, the output is one only if $A = B = 1$.

Worked example (continued)

Inputs			Derived	
A	B	C	$A \wedge B$	F
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Conclusion. $F = 1$ for 001, 011, 101, 110, and 111. It is zero for 000, 010, and 100. Boolean operators are compositional: the output of the AND gate becomes one input of the OR gate. Evaluating the inner expression first prevents a common mistake in which the whole formula is guessed from only one or two rows.
Check the two structural cases. If $C = 1$, the final OR must force $F = 1$ in all four such rows. If $C = 0$, the formula reduces to $A \wedge B$, so only 110 can produce one. For $G = (A \oplus B) \wedge C$, construct the same intermediate column for $A \oplus B$. Only 011 and 101 produce $G = 1$.

Clear and set

Question. Clear bit 3 and set bit 6 of unsigned byte `x`, preserving every other bit. The clear mask has a zero at bit 3 and ones elsewhere. AND applies that mask; OR then supplies a one at bit 6:

```
x = (x & ~(1u << 3)) | (1u << 6);
```

For `0x8B`, the byte-wide clear mask is `0xF7`, giving `0x8B` & `0xF7` = `0x83`. The set mask `0x40` then gives `0xC3`. The other six positions retain their original values because AND used ones and OR used zeros at those positions. To clear bit 1 and set bit 7, substitute the masks `1u << 1` and `1u << 7`.

Extract field

Question. Read bits 5 through 3 of `0xD8` as an unsigned field.

Worked example (continued)

The byte is `11011000`; positions 5, 4 and 3 contain `011`. Move the field down by three positions and keep its three bits:

$$(\texttt{0xD8} \gg 3)\ \&\ 7 = \texttt{0x1B}\ \&\ \texttt{0x07} = 3.$$

The shift places original bit 3 at bit zero. The mask removes the higher positions that moved with it. The direct slice `011` independently confirms the answer. For `0xB4`, bits 6 through 4 similarly give $(\texttt{0xB4} \gg 4)\ \&\ 7 = 3$.

Replace a nibble

Question. Replace the low nibble of `0xA7` with `0xC`, preserving the high nibble.
First clear the destination field: $\texttt{0xA7}\ \&\ \texttt{0xF0} = \texttt{0xA0}$. Limit the new value to four bits with `0x0F`, then combine the disjoint fields:

$$(\texttt{0xA7}\ \&\ \texttt{0xF0})\ |\ (\texttt{0xC}\ \&\ \texttt{0x0F}) = \texttt{0xAC}.$$

OR alone would retain old one-bits: $\texttt{0xA7}\ |\ \texttt{0x0C} = \texttt{0xAF}$, which is the wrong field value. Clearing first also gives $(\texttt{0x5E}\ \&\ \texttt{0xF0})\ |\ \texttt{0x02} = \texttt{0x52}$ when replacing E by 2.

Exercise 1.5.1 — Gate identification

Which gate has output vector 0110 for inputs 00,01,10,11?

Worked solution

The requested outputs are 0 for equal inputs 00 and 11, and 1 for different inputs 01 and 10. This is exclusive OR: $f = A \oplus B$. Ordinary OR would also produce 1 for 11, and AND would produce 1 only for 11. Comparing all four rows establishes the gate; a match on one input pair alone is insufficient.

Exercise 1.5.2 — Count rows

How many rows has a truth table with six inputs?

Worked solution

Each of the six independent inputs has two possible values. The number of complete input assignments is $2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^6 = \boxed{64}$. A full truth table contains one row for every assignment, from `000000` through `111111`. The number of output columns does not change the number of input combinations.

Exercise 1.5.3 — Test a bit

Give an expression that returns nonzero when bit 5 of `x` is set.

Worked solution

Create a one-bit mask with `1u << 5`. Its value is $2^5 = 32 =$ `0x20`. The expression `x & (1u << 5)` clears every other bit and retains bit 5. It returns zero when that bit is clear and 32 when it is set. If a Boolean 0/1 result is required, compare the expression with zero. The unsigned operand gives a well-defined mask under the stated bit width.

Exercise 1.5.4 — Align down

Align an `unsigned int x` down to a 16-byte boundary.

Worked solution

A multiple of 16 has its lowest four bits clear. The mask `0xF` selects those four bits; complementing it gives ones in the higher positions and zeros in the low four. Thus `x & ~0xFu` preserves the higher bits and rounds downward. For `0x123B`, the result is `0x1230`. The removed displacement is 11, which is between 0 and 15; the result is divisible by 16. Use a mask of the same unsigned width as `x`.

Summary

A reusable route leads from requirement to implementation: state the desired bit relation, write the truth table, derive an expression, simplify it, and consider the physical path that realises it. The same route will reappear in the ALU, the control logic of a processor, permission fields, instruction encodings and network protocols.

1.5.8 Exercises

Exercise 1.5.5 — Truth table from expression

Construct the truth table for $f(x, y) = \neg x \vee (x \wedge y)$ and simplify the expression.

Go to the solution →

Exercise 1.5.6 — Sum of products

A three-input function is 1 exactly on rows 000, 011 and 101. Write a canonical sum-of-products expression.

Go to the solution →

Exercise 1.5.7 — NAND construction

Using only NAND gates, construct NOT and AND. Count NAND gates for each.

Go to the solution →

Exercise 1.5.8 — Full-adder output

For inputs $x = 1$, $y = 1$, $c_{in} = 1$, compute s and c_{out} from the full-adder equations.

Go to the solution →

Exercise 1.5.9 — Mask manipulation

An 8-bit register has value `0xA6`. Set bits 0 and 3, clear bits 5 and 7, then toggle bit 1. Give every intermediate hexadecimal value.

Go to the solution →

Exercise 1.5.10 — Logical versus bitwise

With `x=4` and `y=2`, evaluate `x & y`, `x && y`, `x | y`, and `x || y`.

Go to the solution →

Exercise 1.5.11 — Extract and insert a field

A 32-bit unsigned word stores a 5-bit field in positions 11 through 7. Give an expression to extract the field and an expression to replace it with value `v` while preserving all other bits.

Go to the solution →

Exercise 1.5.12 — Critical path

Two circuits compute the same function. Circuit A has a longest path of five gates, each with 2 ns delay. Circuit B has seven gates total but a longest path of three gates, also with 2 ns delay per gate. Ignore wire delay in this estimate. Estimate worst-case propagation delay and explain which is faster.

Go to the solution →

Exercise 1.5.13 — Truth-table proofs of De Morgan laws

Prove both De Morgan laws by truth table.

Worked solution

Exercise 1.5.14 — An even-parity circuit

Derive a circuit that outputs 1 exactly when three input bits contain an even number of ones.
Worked solution

Exercise 1.5.15 — Constructing OR from NOR

Implement OR using only NOR and count the gates.
Worked solution

Exercise 1.5.16 — A portable constant rotation

Write portable unsigned C expressions to rotate a 32-bit value left by 7 positions, taking care with shift counts.
Worked solution

1.5.9 Solutions

Boolean Circuits and Bit-Level Computation

Solution 1.5.5

Rows $(x, y) = (0,0), (0,1), (1,0), (1,1)$ produce 1, 1, 0, 1. Distribute using $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$: $\neg x \vee (x \wedge y) = (\neg x \vee x) \wedge (\neg x \vee y) = 1 \wedge (\neg x \vee y) = \neg x \vee y$.

← Return to the exercise

Solution 1.5.6

The minterms are $\neg x \wedge \neg y \wedge \neg z$, $\neg x \wedge y \wedge z$, and $x \wedge \neg y \wedge z$. OR them:

$$f = (\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge \neg y \wedge z).$$

Each term is true on one listed row and false on all others.

← Return to the exercise

Solution 1.5.7

NOT uses one gate by tying inputs together: $\neg x = x \uparrow x$. AND first computes $t = x \uparrow y = \neg(x \wedge y)$ and then inverts t with $t \uparrow t$. Thus AND uses two NAND gates.

← Return to the exercise

Solution 1.5.8

$s = 1 \oplus 1 \oplus 1 = 1$. The carry is $(1 \wedge 1) \vee (1 \wedge (1 \oplus 1)) = 1 \vee 0 = 1$. The three input ones sum to decimal 3, binary 11: carry 1 and sum 1.

← Return to the exercise

Solution 1.5.9

Start `0xA6`=10100110_2. Set mask `0x09`: `0xA6|0x09=0xAF`. Clear mask for bits 5 and 7 is $\sim$ `0xA0` within 8 bits, namely `0x5F`; `0xAF&0x5F=0x0F`. Toggle bit 1 with `0x02`: `0x0F^0x02=0x0D`.

← Return to the exercise

Solution 1.5.10

Binary values are `100` and `010`. Bitwise AND is 0; logical AND is 1 because both are nonzero. Bitwise OR is `110`=6; logical OR is 1.

← Return to the exercise

Solution 1.5.11

The field mask at the low end is `0x1F`. Extraction is `(x >> 7) & 0x1Fu`. To replace, clear the old field with `x & ~(0x1Fu << 7)`, mask `v` to five bits, shift it, and OR:

```
x = (x & ~(0x1Fu << 7)) | ((v & 0x1Fu) << 7);
```

← Return to the exercise

Solution 1.5.12

Circuit A's critical path is about $5 \cdot 2 = 10$ ns. Circuit B's is about $3 \cdot 2 = 6$ ns. Total gate count does not determine latency when independent gates operate in parallel; the longest dependency path does.

← Return to the exercise

Solution 1.5.13

Evaluate both sides for each of the four assignments:

x	y	$\neg(x \wedge y)$	$\neg x \vee \neg y$	$\neg(x \vee y)$	$\neg x \wedge \neg y$
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

Columns three and four agree in every row, proving the first law. Columns five and six agree in every row, proving the second. Since the table exhausts the Boolean input domain, no further input case remains.
Return to the exercise

Solution 1.5.14

The allowed rows are 000, 011, 101 and 110: they contain zero or two ones. A canonical expression is

$$\bar{x}\bar{y}\bar{z} + \bar{x}yz + x\bar{y}z + xy\bar{z}.$$

XOR records parity: $x \oplus y$ is one after an odd number of assertions among the first two inputs, and XOR with z updates that parity. Invert the result to obtain even parity:

$$q = x \oplus y, \qquad r = q \oplus z, \qquad F = \neg r.$$

Worked solution (continued)

The circuit therefore connects a first two-input XOR to a second XOR with z, followed by an inverter. It uses two XOR gates and one NOT gate. Rows with one or three ones produce $r = 1$ and $F = 0$; the four allowed rows produce $r = 0$ and $F = 1$.

Return to the exercise

Solution 1.5.15

First compute $t = x \,\mathrm{NOR}\, y = \neg(x \vee y)$. Then feed t to both inputs of a second NOR: $t \,\mathrm{NOR}\, t = \neg t = x \vee y$. Two NOR gates are sufficient.

Return to the exercise

Solution 1.5.16

A fully portable version, on implementations providing `uint32_t`, performs shifts in `uintmax_t` so that integer promotions cannot turn a narrow unsigned operand into signed arithmetic:

```
#include <stdint.h>
uint32_t rotl7(uint32_t x) {
    uintmax_t u = x;
    return (uint32_t)((u << 7) | (u >> 25));
}
```

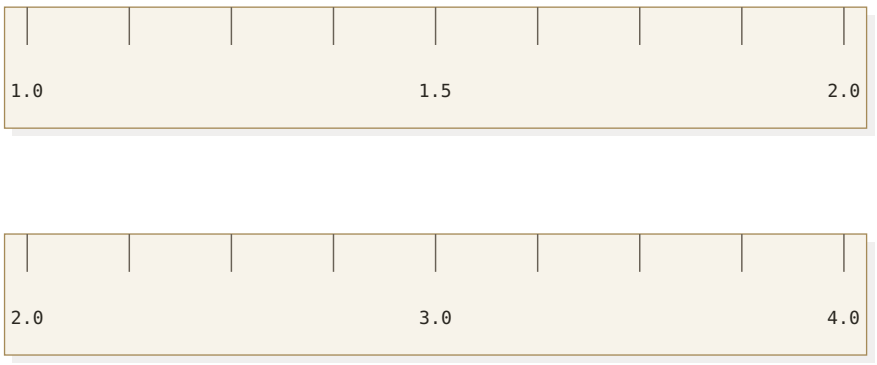
Both shift counts are below the width of `uintmax_t`, which is at least 32. The left shift moves the low 25 bits into positions 7–31; the right shift moves the high seven bits into positions 0–6. Conversion to `uint32_t` discards any higher bits. For `0x80000001`, the result is `0x000000C0`. On the book's platform, where `uint32_t` is an unsigned type unaffected by promotions, the shorter expression `(x << 7) | (x >> 25)` has the same result.

Return to the exercise

1.6 FRACTIONAL NUMBERS AND IEEE 754 FLOATING POINT

A computation must handle both a very small measurement and a very large accumulated total. A fixed number of bits must describe their significant digits and their scale. Floating-point formats divide the representation between these tasks, so the spacing between neighbouring representable values changes with magnitude. Tracing a value through encoding, rounding and arithmetic reveals where information is preserved and where it must be approximated.

A floating scale changes the distance between neighbours



A toy four-significant-bit format: spacing doubles in the next binade.

Figure 1.47: *The two rulers depict adjacent binades in a toy four-significant-bit format. The first has step one eighth, and the second has step one quarter. Their equal drawn widths compare equal counts of representable intervals, not equal real-number lengths.*

1.6.1 From binary fractions to scale and precision

Binary positional notation extends naturally to the right of the radix point. If

$$0.b_1 b_2 \ldots b_k$$

is a finite binary fraction, then its value is

$$\sum_{i=1}^{k} b_i 2^{-i},$$

so the positions after the point carry weights $1/2, 1/4, 1/8, \ldots$. Thus 1011.101_2 represents

$$8 + 2 + 1 + \frac{1}{2} + \frac{1}{8} = 11.625_{10}.$$

The same positional rule also tells us which rational numbers can be represented exactly. After a fraction has been reduced to lowest terms, its binary expansion terminates exactly when its denominator is a power of two. Values such as $3/8$ or $13/32$ therefore have finite encodings, whereas $1/10$ does not, because its denominator contains a factor of five. Its binary expansion repeats indefinitely, just as $1/3$ repeats in decimal.

A terminating and a repeating fraction

Repeated multiplication by two extracts the bits of a fractional value. For 0.6875_{10},

$$0.6875 \cdot 2 = 1.375 \Rightarrow 1, \quad 0.375 \cdot 2 = 0.75 \Rightarrow 0,$$
$$0.75 \cdot 2 = 1.5 \Rightarrow 1, \qquad 0.5 \cdot 2 = 1.0 \Rightarrow 1,$$

which gives 0.1011_2. Performing the same procedure for 0.1_{10} eventually revisits a previous remainder, so the bits repeat. The algorithm and the denominator test describe the same fact from two directions: one generates the representation, while the other tells us in advance whether that generation can terminate.

This distinction matters because decimal source text is not itself the value stored by a binary floating-point variable. When a program writes `0.1`, conversion chooses a nearby representable binary value; arithmetic then operates on those stored values and rounds each result according to the selected rules. In binary64, the rounded sum of the stored approximations to `0.1` and `0.2` differs from the binary64 value obtained by converting decimal `0.3`, which is why sufficiently precise decimal formatting commonly reveals `0.30000000000000004`. Nothing random has happened. The behaviour follows from finite radix representation and rounding.

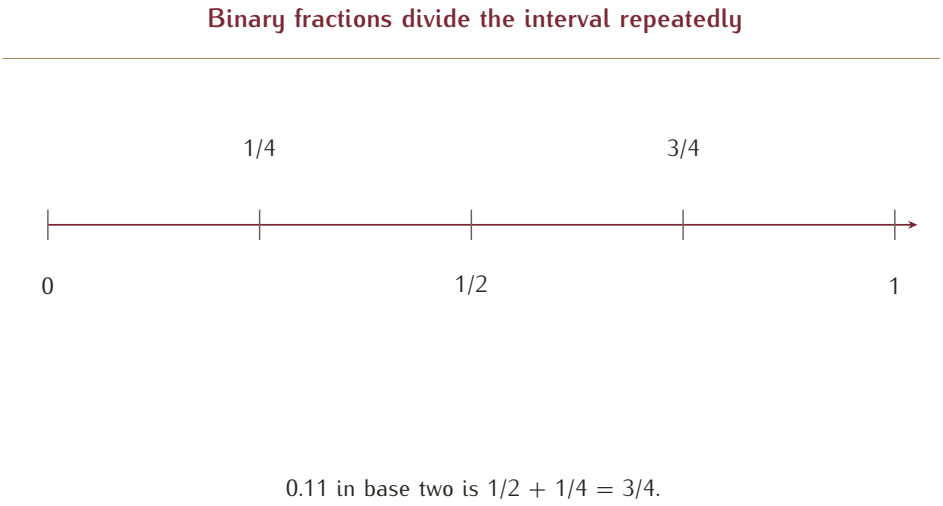


Figure 1.48: *Positions to the right of the binary point have weights one half, one quarter and so on. Fractions whose reduced denominator is a power of two terminate in binary; other fractions generally require rounding.*

The representation must cover fractions across widely different magnitudes. Binary scientific notation supplies the required separation of scale and precision by writing the magnitude of every finite nonzero value, before finite-format limits are considered, in the form

$$1.f_1 f_2 \cdots \times 2^e.$$

For example,

$$10110.101_2 = 1.0110101_2 \times 2^4, \qquad 0.001011_2 = 1.011_2 \times 2^{-3}.$$

Normalisation fixes the leading digit at one, so an ordinary binary format need not store that digit explicitly. The bits that remain describe the significant part of the value, while the exponent records how far the radix point has moved. This is the conceptual step that turns a finite binary fraction into a floating-point format.

1.6.2 IEEE 754 as a representation contract

IEEE 754 standardises that idea by assigning fields and reserved patterns to the sign, exponent and fraction. The two binary formats most often encountered in C on the target systems are binary32, commonly used for `float`, and binary64, commonly used for `double`:

Format	Total bits	Sign	Exponent	Fraction
binary32	32	1	8	23
binary64	64	1	11	52

For a normal finite value the fields denote

$$(-1)^{s}(1+f)2^{E-\mathrm{Bias}},$$

where E is the unsigned exponent field, f is the stored fraction interpreted as a binary fraction, and the biases are 127 for binary32 and 1023 for binary64. Because the leading one of a normalised significand is implicit, binary32 stores 23 fraction bits but carries 24 bits of significand precision; binary64 similarly carries 53 bits of precision.

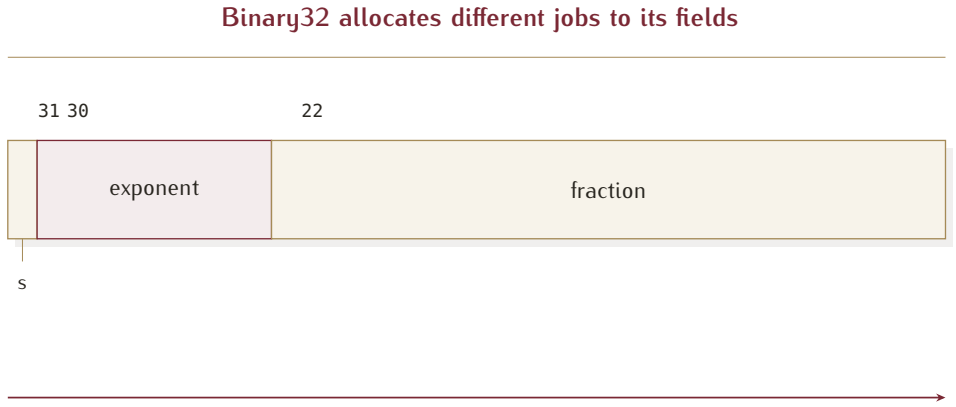
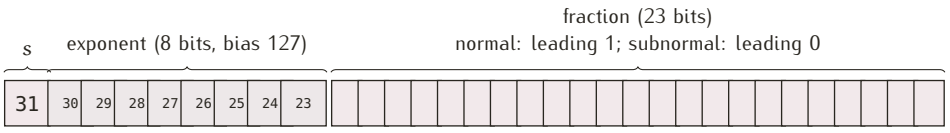


Figure 1.49: *Binary32 stores one sign bit, eight exponent bits and 23 explicit fraction bits. For normal values, an implicit leading one gives 24 bits of significand precision. Zero, subnormal, infinity and NaN encodings use special exponent cases.*



The exponent extremes are reserved because the ordinary normal-value formula is not sufficient to describe zero, gradual underflow or exceptional results. If the exponent field is neither all zero nor all one, the value is normal and uses the implicit leading one. An all-zero exponent with a non-zero fraction denotes a *subnormal*; there the leading bit is zero and the value is

$$(-1)^{s}f\,2^{1-\mathrm{Bias}},$$

which fills the gap between the smallest normal value and zero with progressively reduced precision. An all-zero exponent and fraction denotes signed zero. An all-one exponent with a zero fraction denotes signed infinity, while an all-one exponent with a non-zero fraction denotes NaN.

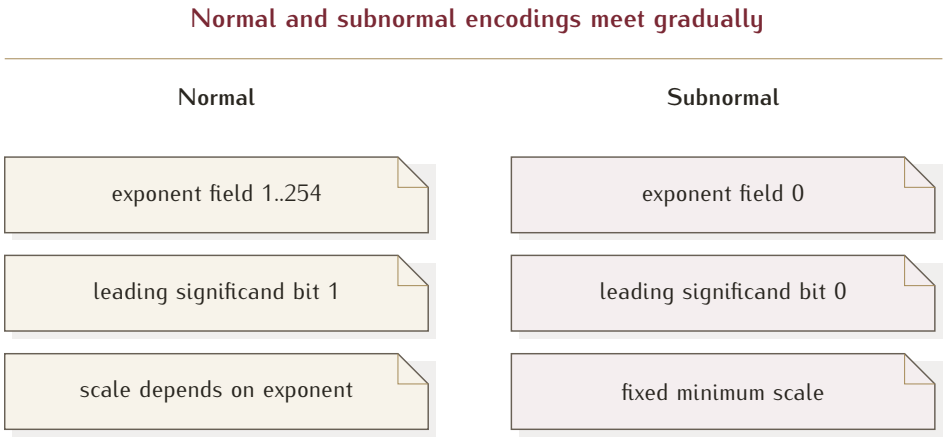


Figure 1.50: *Subnormal values extend the representable range towards zero with a fixed exponent scale. Their leading bit is zero, so effective significant precision decreases as the stored fraction becomes smaller.*

Decoding a binary32 value

For the word `0x41280000`, the sign bit is zero and the exponent field is `0x82=130`, so the unbiased exponent is $130 - 127 = 3$. The fraction begins `0101` followed by zeros, hence $f = 1/4 + 1/16 = 0.3125$ and the significand is 1.3125. The represented value is therefore

$$1.3125 \times 2^3 = 10.5.$$

Encoding works in the opposite direction. Since $0.15625 = 0.00101_2 = 1.01_2 \times 2^{-3}$, binary32 stores sign 0, exponent $-3 + 127 = 124 = 01111100_2$, and fraction `010000...0`, producing `0x3E200000`.

Infinities and NaNs allow arithmetic to represent overflow, division by zero and invalid operations within a common vocabulary instead of forcing every exceptional operation to stop the machine. Signed zero similarly retains directional information that can matter in functions with discontinuities or branch cuts, while subnormals provide gradual rather than abrupt underflow near zero. The categories define both the bit layout and the arithmetic semantics of the format [7, 17].

IEEE arithmetic also records exception conditions: invalid operation, division by zero, overflow, underflow and inexact. Under ordinary nontrapping handling, an invalid operation such as 0/0 returns a NaN, and a finite nonzero value divided by zero returns a signed infinity. Overflow can return infinity or the largest finite value according to the rounding direction. Underflow concerns tiny results with lost accuracy; an exactly representable subnormal by itself need not raise the underflow flag. These flags describe what happened during operations, whereas inspecting a stored word identifies its value category [7, 17].

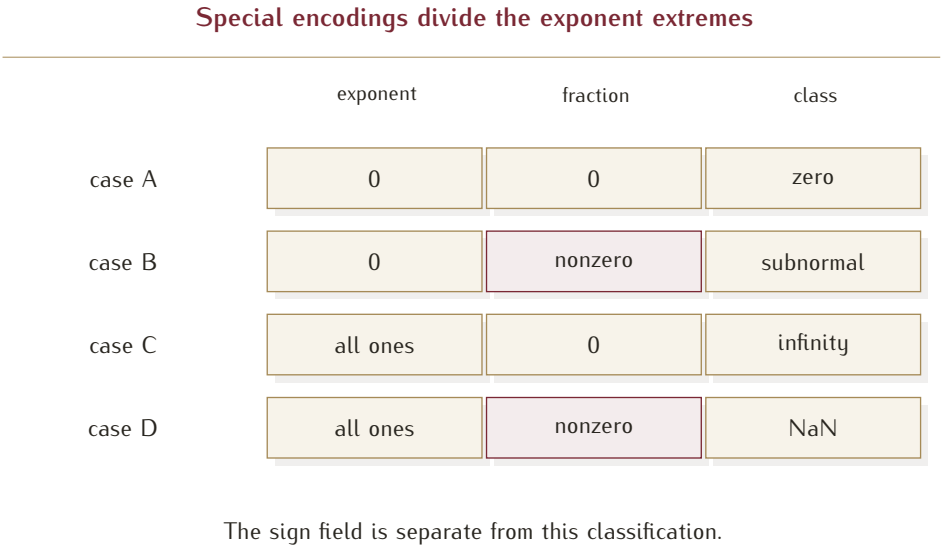


Figure 1.51: *Inspecting exponent and fraction together identifies the special classes. An all-ones exponent does not by itself imply infinity because a nonzero fraction denotes NaN. Signed zero retains a sign bit even though its numerical value is zero.*

1.6.3 Rounding, spacing and the behaviour of programs

Most exact real results lie between two representable floating-point values, so a format needs a rule for choosing one of them. IEEE 754 provides several rounding directions; the usual default is round to nearest, ties to even. The nearest representable value is selected, and an exact halfway case goes to the candidate whose least significant retained bit is even, avoiding a systematic directional bias over repeated ties. Hardware does not need to retain an arbitrarily long exact significand to make that decision: guard, round and sticky information records enough about the discarded low-order bits to determine the correctly rounded neighbour.

For round-to-nearest, denote the least significant retained bit by L, the first discarded bit by G (guard), the next by R (round), and the OR of all remaining discarded bits by S (sticky). Increment the retained significand exactly when

$$G \wedge (R \vee S \vee L) = 1.$$

If $G = 0$, the discarded part is below half a spacing. If $G = 1$ and either R or S is one, it is above half. With $G = 1, R = S = 0$, the value is exactly halfway: retaining an even low bit, or incrementing an odd one, implements ties-to-even. For a three-bit significand, 1.001_2 rounds to 1.00_2, while 1.011_2 rounds to 1.10_2. Directed rounding instead chooses towards zero, towards $+\infty$ or towards $-\infty$ as requested [7, 17].

Floating-point values are consequently not spaced uniformly on the real line. Within one exponent interval, adjacent normal values are evenly spaced, but when the exponent grows by one the spacing doubles. Binary32 values in $[1, 2)$ differ by 2^{-23}; in $[2^{20}, 2^{21})$ the spacing is $2^{-3} = 0.125$. In $[2^{24}, 2^{25})$, consecutive representable values are two integer units apart, so the exact sum $2^{24} + 1$ lies halfway between two binary32 values and rounds back to 2^{24} under ties-to-even. A positive addend can disappear even though the arithmetic unit has behaved exactly as specified.

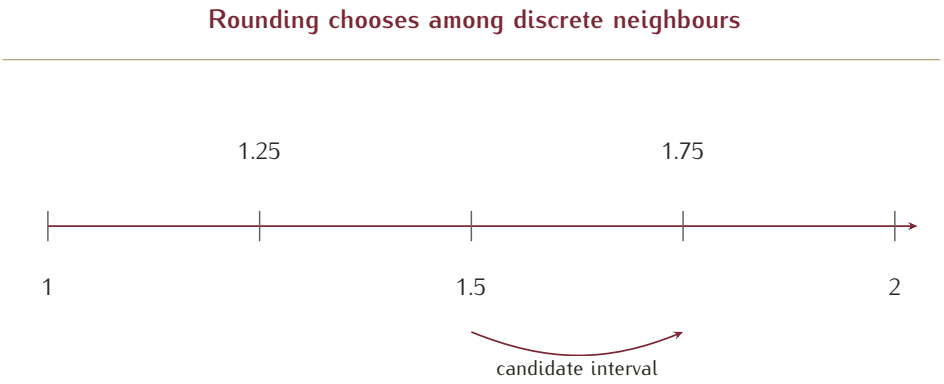


Figure 1.52: *The toy format represents multiples of one quarter between one and two. A real value between two neighbours must be rounded according to the selected rule. The drawing uses a small precision so the spacing is visible.*

This changing spacing explains why floating-point arithmetic does not preserve every algebraic identity of real arithmetic. Intermediate results round, and the order in which they are formed therefore matters. Addition need not be associative, distributivity can fail, and subtracting nearby rounded values can magnify relative error through cancellation.

Order and cancellation matter

Let a be so large, in the chosen format, that adding 1 to $-a$ leaves $-a$ unchanged after rounding. Then

$$(a + (-a)) + 1 = 1,$$

whereas

$$a + ((-a) + 1) = a + (-a) = 0.$$

The two expressions are identical over the reals but not under finite intermediate rounding. A related problem appears in $\sqrt{x+1} - \sqrt{x}$ for large x, where subtraction discards many common leading digits. Rationalising the expression,

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}},$$

avoids the cancellation and usually produces a much more accurate result.

Exact equality is therefore neither universally wrong nor universally right. It is appropriate when the program expects exactly the same representable value by construction, for example when checking a sentinel or comparing an unchanged copy. It is a poor generic test for two mathematically equivalent calculations that may have rounded differently. A fixed absolute epsilon is equally unreliable across scales; when an approximate comparison is part of the specification, a common form is

$$|a - b| \leqslant \varepsilon_{abs} + \varepsilon_{rel} \max(|a|, |b|),$$

with tolerances derived from the application's error model rather than chosen as ritual constants.

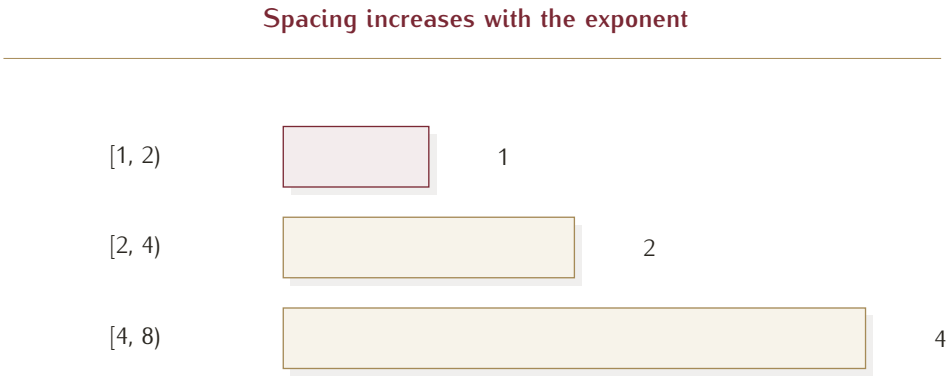


Figure 1.53: *Moving to the next binade doubles the gap between adjacent normal values. The significand has the same number of bits, while the exponent changes the scale. The bars compare the gap between successive representable values in each binade.*

NaN requires a separate mental model because it is unordered. With ordinary non-trapping IEEE comparison semantics, ordered comparisons and equality with NaN are false, while inequality is true, so a range check such as `if (x < min || x > max)` can accidentally accept a NaN. When non-finite inputs are not meaningful, finiteness must be validated explicitly.

Long computations introduce a second source of difficulty: individually small rounding errors can accumulate. Adding a tiny term to a much larger running total may repeatedly absorb that term, whereas pairwise summation or compensated techniques preserve more low-order information. A simplified Kahan-style accumulator keeps a correction term for information lost by the previous addition:

```
double sum = 0.0, c = 0.0;
for (size_t i = 0; i < n; ++i) {
    double y = a[i] - c;
    double t = sum + y;
    c = (t - sum) - y;
    sum = t;
}
```

The method does not make floating-point arithmetic exact; it changes the algorithm so that less of the discarded information is allowed to accumulate into the final error.

Numerical edge cases can cross security boundaries when floating-point values control array sizes, timeouts, quotas or geometry. NaN can bypass incomplete comparisons, infinity can survive scaling, and conversion to an integer can fail when the value lies outside the destination range. Inputs used in memory or control calculations should

therefore be checked for finiteness, documented range and suitability for the intended conversion before privilege or memory-safety decisions depend on them.

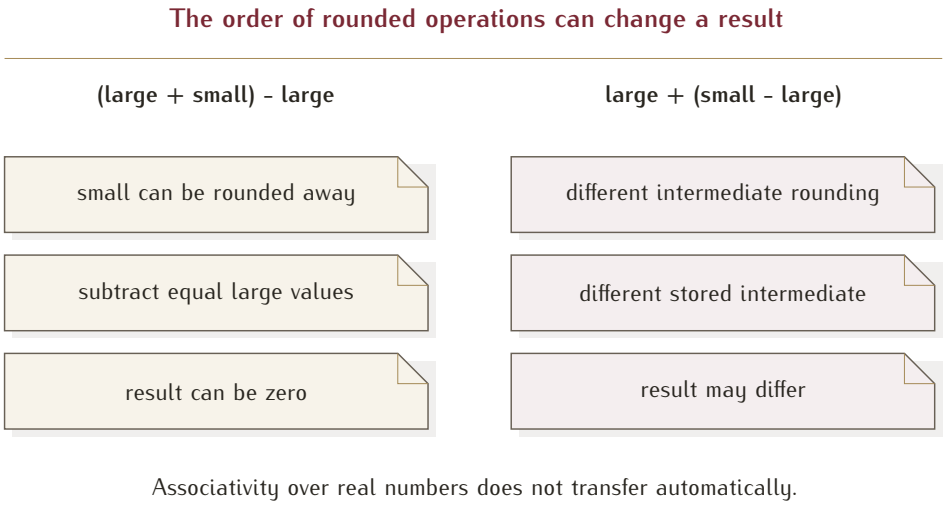


Figure 1.54: *Each intermediate operation rounds to a representable value. Reassociating an expression changes those intermediate rounding points. The numerical examples in the chapter give explicit operands and evaluate the actual results.*

The same caution applies to domain modelling. Binary floating point is appropriate when approximate real quantities and bounded relative error are acceptable; it is not automatically the right representation for contracts whose rules are stated in decimal units. Financial software may instead use scaled integers, fixed-point or decimal arithmetic, but even "integer cents" is only correct when the domain's scale, rounding, tax, interest and exchange-rate rules make that representation sufficient. Representation has to match the contract.

1.6.4 From the IEEE contract to the floating-point datapath

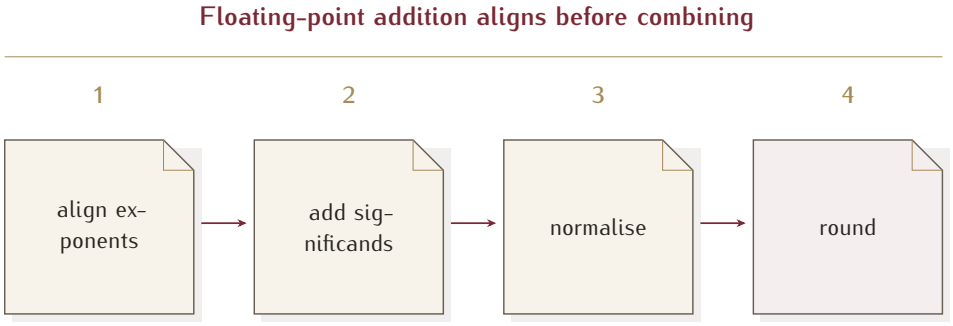
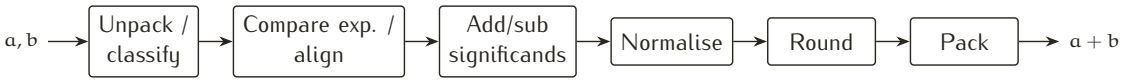


Figure 1.55: *An adder first expresses the operands at a common scale. After combining their significands, it normalises and rounds the result. Discarded low-order information can affect rounding, even when it does not remain in the stored fraction.*

IEEE 754 specifies externally visible formats and arithmetic behaviour. Processor designers choose the circuitry that implements those requirements. On x86-64, scalar binary32 and binary64 operations are normally carried by SSE-family or later vector instructions in XMM register views, while legacy x87 remains relevant for older binaries, some `long double` implementations and specialised code. The active control state also matters: SSE-family floating-point uses `MXCSR` for rounding control, exception status and masks, while x87 has separate control and status words. Changing denormal handling or rounding mode alters numerical behaviour and is therefore not a harmless performance switch [26, 27, 30].

A floating-point addition illustrates why the single arithmetic operator hides substantial machinery. Before the significands can be added, their exponents must be compared and the smaller-exponent operand shifted into alignment; after the arithmetic, the result may need normalisation, rounding and exceptional-case classification. A simplified datapath therefore follows the sequence



Actual implementations pipeline and overlap these functions in different ways, which is why latency and throughput depend on microarchitecture and instruction form even though the architecturally visible rounded result obeys the same format contract. The distinction mirrors a theme that will recur throughout the book: the ISA fixes what software may observe, while the implementation remains free to organise the internal work differently.

Multiplication additionally forms partial products; division and square root often use iterative algorithms. These different structures help explain operation-dependent latency, throughput and energy use [50, 41].

Laboratory 6: Floating-point representation and rounding

Under binary32 round-to-nearest, inspect the words for $+0, -0, 1, 0.1, 10.5, +\infty$ and one NaN. Use `memcpy`, then compare naive and compensated addition of 0.1 over ten million binary64 additions. Give a non-associative binary32 example and explain what is predictable across runs.
Worked solution

Laboratory 6: solution

This program exposes the representations and supplies both sums:

```
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <inttypes.h>
int main(void) {
    float a[]={0.0f,-0.0f,1.0f,0.1f,10.5f,INFINITY,NAN};
    for (unsigned i=0;i<7;i++) {
        uint32_t u; memcpy(&u,&a[i],sizeof u);
        printf("%08" PRIx32 "\n",u);
    }
    double s=0,k=0,c=0;
    for (int i=0;i<10000000;i++) {
        s += 0.1;
        double y=0.1-c, t=k+y;
        c=(t-k)-y; k=t;
    }
    printf("naive %.17g compensated %.17g\n",s,k);
    volatile float x=16777216.0f,y=1.0f,z=-16777216.0f;
    float left=(x+y)+z, right=x+(y+z);
    printf("%g %g\n",left,right);
}
```

Compile on the stated platform with `gcc -std=c11 -O2 -fno-fast-math fp.c -o fp`. The first six words are `00000000`, `80000000`, `3f800000`, `3dcccccd`, `41280000`, `7f800000`. NaN has exponent 255 and a nonzero fraction; its payload is implementation-dependent. For 0.1, sign=0, stored exponent=123 and fraction=`0x4CCCCD`; for 10.5 they are 0,130,`0x280000`. The exact decimal target is 1,000,000. With binary64 intermediates and round-to-nearest, the program prints `naive 999999.99983897537 compensated 1000000`. The naive error relative to the

Worked solution (continued)

decimal target is about -1.61×10^{-4}; compensation recovers the target when rounded to binary64 in this experiment. This is not a guarantee of exactness for arbitrary inputs. The results repeat when the input bits, operation order, evaluation precision and floating-point control state remain fixed. Different optimisation rules, excess precision or rounding modes can change them. The final outputs are 0 and 1 under binary32 ties-to-even: $2^{24} + 1$ rounds to 2^{24} on the left, whereas $1 - 2^{24}$ is exactly representable on the right. Adding 2^{24} to that exact difference gives 1.

Return to the laboratory

From machine-specific scaling to IEEE floating-point arithmetic

Timeline

Year	Machine, standard or event	Architectural significance
1941	Z3	Binary floating-point arithmetic operates in a programmable machine.
1964	System/360	A family standardises floating formats across implementations.
1985	IEEE 754	Interchange formats, rounding and exceptional values receive a common standard.

Origins and standardisation

Early machines used incompatible widths, exponent bases, normalisation rules and rounding methods, so a numerical program could be syntactically portable while changing low-order results or exceptional behaviour from one system to another. Konrad Zuse's Z3 used binary floating point; IBM's scientific machines made floating arithmetic central to commercial computing; System/360 later standardised hexadecimal floating point across a family of implementations. Intel's 8087, introduced in 1980, appeared while the IEEE P754 committee was working towards a common arithmetic, and implementers and numerical analysts together shaped a standard that was both mathematically disciplined and commercially feasible [88, 110, 63, 105, 77].

IEEE Std 754-1985 standardised binary interchange formats, rounding modes, signed zeros, infinities, subnormals, NaNs and exception flags. Later revisions added and clarified decimal formats, fused operations and other requirements. The standard made approximation inspectable by specifying which representable result follows from a mathematical operation and a rounding mode [7, 77, 17].

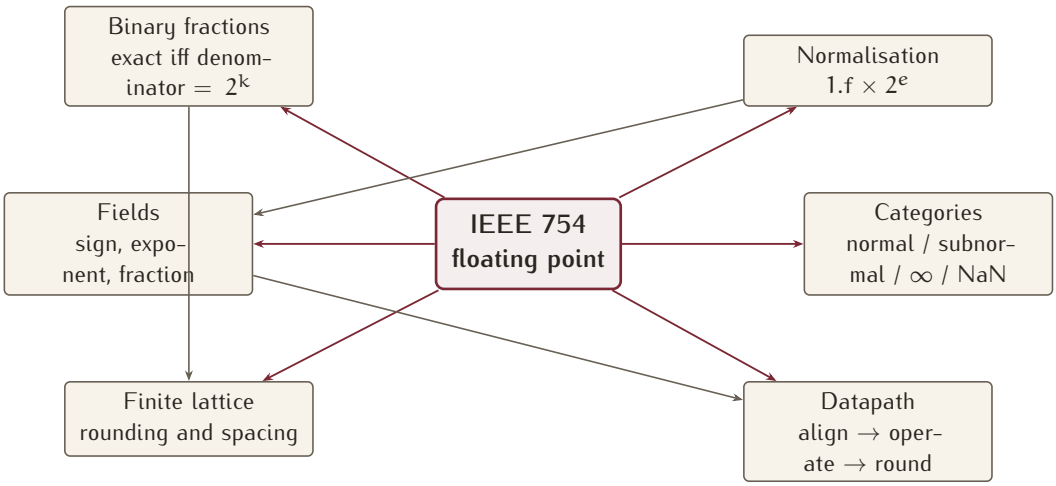
The history also warns against reducing numerical failures to the slogan "floating point is inaccurate". The Patriot incident involved accumulated timing error in a finite-precision computation, whereas Ariane 5 Flight 501 failed after an unhandled conversion

overflow from floating point to a narrower integer. The relevant engineering questions are which representation was used, which range was assumed, how error propagated and what happened when the assumption failed [65, 66].

Present-day architecture

Modern processors and accelerators support several precisions. Binary64 remains common in scientific work, binary32 often balances storage and accuracy, and reduced-precision formats are widely used where throughput and energy dominate, commonly with wider accumulation. x87, SSE, AVX and fused multiply-add also show that the same source expression can pass through different intermediate precisions or rounding structures unless the language and compilation environment constrain those choices. IEEE 754's lasting achievement is therefore a common grammar for analysing approximation across changing hardware [7, 17, 26].

Concept map: floating point



1.6.5 Worked problems

Binary fraction

Question. Convert 101.011_2 to decimal.
The integer positions contribute $1 \cdot 4 + 0 \cdot 2 + 1 = 5$. The fractional positions contribute $0 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} + 1 \cdot \frac{1}{8} = 0.375$. Adding the two parts gives 5.375 exactly.
The reverse conversion recovers $5 = 101_2$ and $0.375 = 3/8 = 0.011_2$. As another example, $11.101_2 = 3 + \frac{1}{2} + \frac{1}{8} = 3.625$.

Terminating criterion

Question. Does 3/20 have a finite binary expansion?
Any binary fraction ending after k places is an integer divided by 2^k. After cancellation, its denominator therefore contains only factors of two. The fraction 3/20 is already reduced, and $20 = 2^2 \cdot 5$ retains a factor of five, so no finite binary expansion can represent it.
Repeated multiplication by two makes the repetition explicit: the fractional remainders are $0.3, 0.6, 0.2, 0.4, 0.8, 0.6, \ldots$. The repeated remainder 0.6 repeats the later bits indefinitely. The same denominator test rejects 7/40, while 5/32 terminates because $32 = 2^5$.

Normalisation

Question. Normalise 11010.01_2.
Move the radix point four places left, leaving exactly one leading one. The new significand is 1.101001_2; multiplying by 2^4 restores the original scale:

$$11010.01_2 = 1.101001_2 \times 2^4.$$

Moving the point four places right checks every digit. Similarly, $101.101_2 = 1.01101_2 \times 2^2$, because that input requires two leftward moves.

Decode binary32

Question. Decode the binary32 word `0x40000000`.
The sign bit is zero, the eight-bit exponent field is $128 =$ `0x80`, and the 23 fraction bits are zero. Since the exponent is between 1 and 254, the normal-value rule applies. Restore the implicit leading one and subtract the bias:

$$(-1)^0(1+0)2^{128-127} = 2.$$

Encoding 2 reverses those steps and returns `0x40000000`. Reducing the exponent field to 127 gives `0x3F800000`, which represents 1.

Stable comparison

An application compares measurements $x = 1{,}000{,}000.0$ and $y = 1{,}000{,}000.4$ using an explicitly permitted tolerance. The acceptance criterion reflects the application's accuracy requirement; floating-point rounding alone would not justify treating this difference as negligible.

Worked example (continued)

Question. With atol $= 0.01$ and rtol $= 10^{-6}$, does

$$|x - y| \leqslant \text{atol} + \text{rtol} \max(|x|, |y|)$$

classify the two values as close?
Reasoning.

1. **Measure the observed difference.** $|x - y| = |1,000,000.0 - 1,000,000.4| = 0.4$.
2. **Build the allowed tolerance.** The larger magnitude is $1,000,000.4$, so the relative contribution is $10^{-6} \times 1,000,000.4 \approx 1.0000004$. Adding the absolute term gives approximately 1.0100004.
3. **Compare difference and allowance.** Since $0.4 \leqslant 1.0100004$, the test accepts the pair as close.

Conclusion. The comparison returns true for the stated tolerances.
The absolute term protects comparisons near zero, where a purely relative tolerance can become unrealistically small. The relative term scales the allowance with magnitude, which is necessary when values are around one million rather than around one.
Tighten the relative tolerance to 10^{-8} while keeping atol $= 0.01$. The allowance becomes about 0.020000004, which is smaller than 0.4, so the decision correctly changes to false.
For $x = 0.0002$, $y = 0.0008$, atol $= 0.001$, and rtol $= 10^{-6}$, the difference is 0.0006 and the allowance is slightly above 0.001; the absolute term now determines the result.

Exercise 1.6.1 — Weight

What is the weight of the fifth bit right of the binary point?

Worked solution

Positions to the right of a binary point have weights $2^{-1}, 2^{-2}, 2^{-3}, \ldots$. The fifth position therefore has weight $2^{-5} = 1/32 = \boxed{0.03125}$. A lone 1 in that position denotes 0.00001_2. Multiplying this value by $2^5 = 32$ gives 1, which checks the positional interpretation.

Exercise 1.6.2 — Bias

Which biased exponent encodes e=-3 in binary32?

Worked solution

For a normal binary32 number, the stored exponent is $E = e + 127$. Substituting $e = -3$ gives $E = -3 + 127 = \boxed{124}$. In eight bits this is `01111100`, or `0x7C`. Decoding reverses the bias: $124 - 127 = -3$. The reserved exponent fields 0 and 255 have separate meanings and do not use this normal-number calculation.

Exercise 1.6.3 — Exactness

Is $5/16$ finite in binary?

Worked solution

The fraction $5/16$ is already reduced and its denominator is $16 = 2^4$. It therefore has a terminating binary expansion. Write $5 = 4 + 1$ and divide by 16: $5/16 = 1/4 + 1/16 = 0.0101_2$. Four fractional positions are sufficient. This also checks the value, since $0.0101_2 = 2^{-2} + 2^{-4} = 0.3125$.

Exercise 1.6.4 — Cancellation

Why can subtracting nearly equal values lose useful digits?

Worked solution

Nearly equal operands share most leading digits. Subtraction removes that common part, leaving a small difference whose relative error can be large compared with errors already present in the operands. For example, if two measurements near 1000 each have uncertainty 0.01, a computed difference of 0.02 has uncertainty of comparable size. The subtraction may be exactly rounded while the difference remains inaccurate because the inputs were approximations. Examine input error and the scale of the result when diagnosing cancellation.

Summary

The right question is seldom "Is floating point exact?" The useful questions are: which values are representable, how large is the local spacing, where does rounding occur, and how sensitive is the problem to small perturbations? Once those questions are explicit, many apparent anomalies become ordinary consequences of a finite approximation system.

1.6.6 Exercises

Exercise 1.6.5 — Fractional conversion

Convert 13.375_{10} to binary and normalised binary scientific notation.

Go to the solution →

Exercise 1.6.6 — Exact or approximate

State whether each rational number has a finite binary expansion: $7/16$, $3/20$, $11/32$, $5/12$.

Go to the solution →

Exercise 1.6.7 — Decode a simple binary32 word

Decode `0xBF400000` as IEEE 754 binary32.

Go to the solution →

Exercise 1.6.8 — Encode a power of two

Give the binary32 sign, exponent field and fraction field for 32.0.

Go to the solution →

Exercise 1.6.9 — Classify special fields

Classify binary32 words with fields (a) $s = 0, E = 0, F = 0$; (b) $s = 1, E = 255, F = 0$; (c) $s = 0, E = 255, F \neq 0$; (d) $s = 0, E = 0, F \neq 0$.

Go to the solution →

Exercise 1.6.10 — Spacing near a power of two

Binary32 has 24 bits of significand precision. What is the spacing between adjacent values in $[2^{10}, 2^{11})$?

Go to the solution →

Exercise 1.6.11 — Non-associativity scenario

Explain qualitatively why, in binary32, $(10^{20} + (-10^{20})) + 3$ can differ from $10^{20} + ((-10^{20}) + 3)$.

Go to the solution →

Exercise 1.6.12 — Robust range validation

A program accepts a floating value that must lie in $[0, 100]$ and must be finite. Why is `if (x < 0 || x > 100) reject;` incomplete, and what condition is required?

Go to the solution →

Exercise 1.6.13 — Encoding a negative binary32 value

Encode -6.25 as binary32 and give the hexadecimal word.
Worked solution

Exercise 1.6.14 — The next number after one

Find the next binary32 value after 1.0 and express the difference as a power of two.
Worked solution

Exercise 1.6.15 — Reducing cancellation

Construct an example of cancellation and rewrite the formula to reduce it.
Worked solution

Exercise 1.6.16 — Comparing error measures

Compare absolute, relative and ulp-based error measures for values near zero and values near one million.
Worked solution

1.6.7 Solutions

Fractional Numbers and IEEE 754 Floating Point

Solution 1.6.5

$13_{10} = 1101_2$. The fraction $0.375 = 3/8 = 0.011_2$. Thus $13.375 = 1101.011_2$. Moving the radix point three places left gives $1.101011_2 \times 2^3$.

← Return to the exercise

Solution 1.6.6

Reduce denominators and inspect prime factors. $7/16$ and $11/32$ are exact because denominators are powers of two. $3/20$ has denominator $2^2 \cdot 5$, and $5/12$ reduces no further and has denominator $2^2 \cdot 3$; both repeat in binary.

← Return to the exercise

Solution 1.6.7

Sign bit is 1, so the value is negative. Exponent field is `0x7E=126`, giving exponent -1. Fraction field `0x400000` represents $1/2$, so the significand is 1.5. Value is $-1.5 \cdot 2^{-1} = -0.75$.

← Return to the exercise

Solution 1.6.8

$32 = 1.0 \times 2^5$. Sign is 0. Biased exponent is $5 + 127 = 132 = 10000100_2$. The fraction is all zeros because the significand is exactly 1.0. The complete hexadecimal word is `0x42000000`.

← Return to the exercise

Solution 1.6.9

(a) positive zero; (b) negative infinity; (c) NaN; (d) positive subnormal number. The ordinary normal formula applies to none of these cases.

← Return to the exercise

Solution 1.6.10

With 24 significant bits, the least significant place in this exponent interval has weight $2^{10-(24-1)} = 2^{-13}$. Thus adjacent normal values are separated by $2^{-13} = 1/8192$.

← Return to the exercise

Solution 1.6.11

The first grouping adds two represented large opposite values and can produce zero, then adds 3, giving 3. In the second grouping, adding 3 to -10^{20} does not change the rounded large value because spacing at that magnitude is far larger than 3. The final addition then cancels to zero. Intermediate rounding breaks associativity.

← Return to the exercise

Solution 1.6.12

If x is NaN, both ordered comparisons are false, so the input passes. Test finiteness explicitly and then the range, for example reject when `!isfinite(x) || x < 0.0 || x > 100.0`. Infinity is caught by the ordered bounds, but an explicit finiteness requirement documents the contract and handles NaN.

← Return to the exercise

Solution 1.6.13

$6.25 = 110.01_2 = 1.1001_2 \times 2^2$. Binary32 therefore has sign 1, biased exponent $2 + 127 = 129 = 10000001_2$, and fraction `1001000...`. The word is `1 10000001 10010000000000000000000`, hexadecimal `0xC0C80000`.

Return to the exercise

Solution 1.6.14

At 1.0 the exponent is zero after normalisation and the binary32 significand has 23 stored fraction bits. The next representable value is $1 + 2^{-23}$, encoded as `0x3F800001`.

Return to the exercise

Solution 1.6.15

Take $x = 10^8$ and evaluate in binary64 with ties-to-even. The product $x^2 = 10^{16}$ is exactly representable, but $x^2 + 1$ rounds back to 10^{16} because the spacing there is 2. The computed square root is 10^8, so direct subtraction returns zero. Rationalising gives

$$\sqrt{x^2+1} - x = \frac{1}{\sqrt{x^2+1} + x}.$$

At this input the rewritten computation produces approximately 5×10^{-9}, close to the exact positive difference. For larger inputs, forming x^2 can itself overflow;

Worked solution (continued)

this rewrite addresses cancellation only within a range where the intermediate operations remain finite.

Return to the exercise

Solution 1.6.16

For reference value r and approximation a, absolute error is $|a - r|$ and relative error is $|a - r|/|r|$ when $r \neq 0$. To define a local ulp error here, divide absolute error by the upward spacing at the reference value. In binary32:

Reference and approximation	Absolute	Relative	Local ulps
$r = 2^{-149},\ a = 2^{-148}$	2^{-149}	1	1
$r = 10^6,\ a = 10^6 + 2^{-4}$	0.0625	6.25×10^{-8}	1

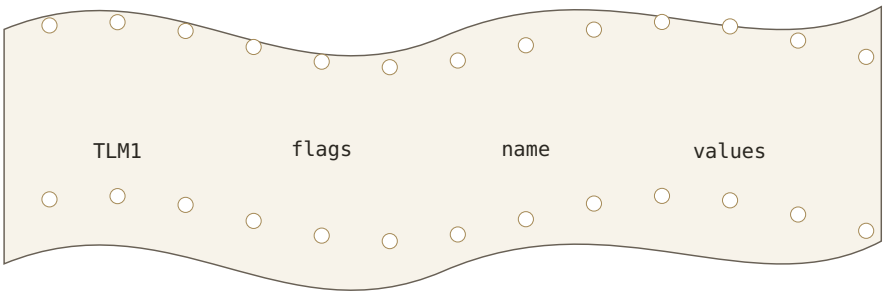
Near zero, adjacent subnormals are separated by 2^{-149}, so one representable step can be a 100% relative error. At exactly zero, relative error is undefined. Near one million, the exponent is 19 and spacing is $2^{19-23} = 2^{-4}$; one step has much smaller relative significance. Absolute error uses the problem's units, relative error compares scale, and ulps compare representable spacing. At exponent boundaries the upward and downward gaps differ, so a precise ulp convention matters.

Return to the exercise

1.7 INTEGRATED CASE STUDIES: FROM BYTES TO BEHAVIOUR

A parser receives a byte buffer and begins to read a record. Before each access, it must establish that the bytes exist; after reading them, it must interpret their order, width and encoding. Those decisions can affect the next length calculation and the memory allocated for the result. The cases in this chapter follow that chain from the first byte to an observable outcome, bringing the representation rules together in one execution.

One record carries several representation contracts



The ribbon is a visual metaphor for the record's ordered bytes.

Figure 1.56: *The telemetry record combines a signature, bit fields, text and numerical quantities. Its field boundaries and individual encoding rules give meaning to the sequence. The later proportional byte diagram supplies the exact widths.*

1.7.1 A representation workflow for real binary data

When a program receives bytes from a file, network connection, device or another process, interpretation should proceed in an order that prevents later assumptions from outrunning earlier evidence. First establish provenance and the total available length. Next identify field boundaries and prove that the bytes required by each field are present. Only then apply serialisation rules such as byte order, followed by representation rules

for signed integers, floating-point values, text or flags. Semantic validation comes after structural decoding: reserved values, ranges, dimensions, text validity and cross-field consistency all belong here. Allocation, indexing and further computation should occur only after those earlier claims have been established.

This order prevents premature use of unvalidated data. If a length field is trusted before the parser proves that the field itself exists, or if an unchecked dimension controls an allocation before multiplication bounds have been validated, a representation error can become a memory-safety error. The workflow deliberately turns every use of a decoded value into the consequence of a previously established invariant.

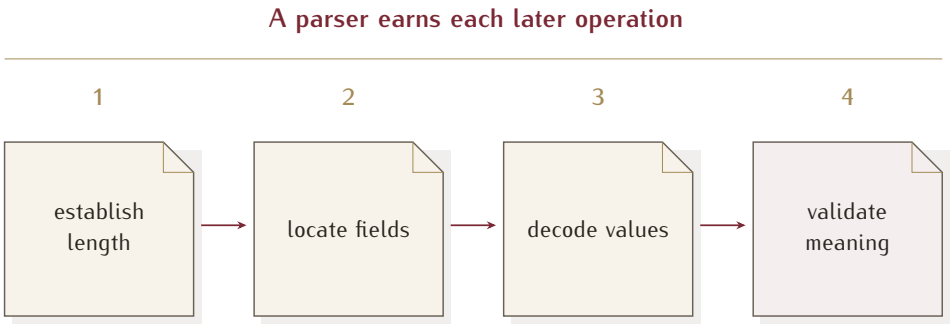


Figure 1.57: *Each transition adds evidence: available bytes, valid boundaries, a representation interpretation and acceptable meaning. Allocation and indexing belong after the checks that justify their operands.*

1.7.2 One telemetry record, five representation contracts

Consider an invented format named TLM1 containing exactly one telemetry record:

Offset	Width	Field	Rule
0	4	magic	ASCII bytes `TLM1`
4	1	flags	bit 0 valid, bit 1 alarm, bits 2–7 reserved
5	1	name length n	unsigned, 0–16
6	n	name	strict UTF-8, no NUL byte
$6+n$	2	temperature	signed 16-bit, big-endian, hundredths of a degree
$8+n$	4	voltage	IEEE 754 binary32, little-endian, finite and nonnegative

The valid bit describes the measurement status supplied by the producer; either value is permitted in a structurally valid record. Consumers decide whether to

use a measurement whose valid bit is clear. The alarm bit independently reports the producer’s alarm state.

A candidate record contains

```
0x54 4C 4D 31 03 03 41 C3 A8 FF 85 00 00 20 41.
```

The first four bytes decode under ASCII as `TLM1`, so the format signature is present. The flags byte `0x03` is 00000011_2: the valid and alarm bits are set while every reserved bit is clear. The next byte declares a three-byte name, which means the parser must prove that at least three more bytes exist before attempting to decode text. Those bytes, `0x41 C3 A8`, form valid UTF-8 for `A` followed by U+00E8. The field therefore occupies three bytes while containing two Unicode code points, precisely the distinction introduced in Chapter 1.3.

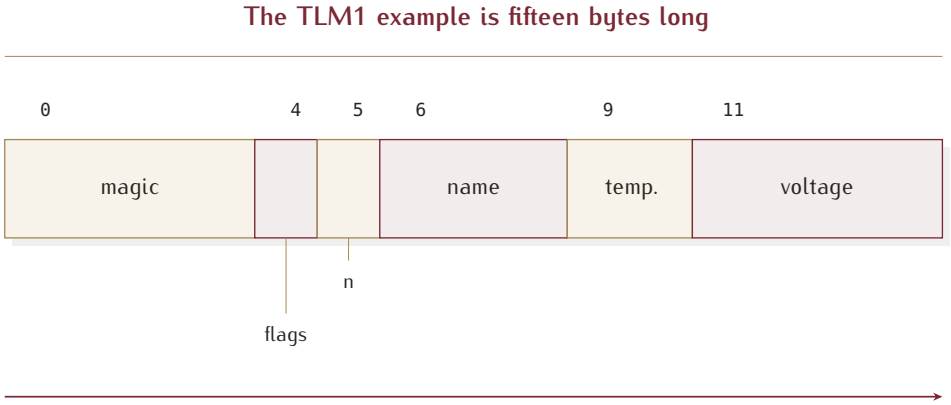


Figure 1.58: *The widths are proportional to the actual byte counts in the worked record. The name occupies offsets six through eight, temperature occupies nine and ten, and voltage occupies eleven through fourteen.*

The following two bytes, `0xFF 85`, are explicitly big-endian. They first form the 16-bit pattern `0xFF85`; only after byte order has been applied should the signed interpretation be used. Its unsigned value is 65413, so two’s-complement decoding gives $65413 - 65536 = -123$. Because the format defines hundredths of a degree, the semantic value is -1.23 degrees.

The final four bytes, `0x00 00 20 41`, use a different byte order. Little-endian assembly gives the binary32 word `0x41200000`; sign 0, biased exponent 130 and fraction field corresponding to 0.25 yield $(1.25)2^3 = 10.0$. Structural decoding is still not the final step: the format additionally requires the voltage to be finite and nonnegative. Here both conditions hold.

The record satisfies the format and reports name “Aè”, valid+alarm flags, temperature −1.23 and voltage 10.0. This decoding combines five contracts: ASCII for the signature, a Boolean mask for the flags, UTF-8 for the name, a big-endian scaled signed integer for temperature, and little-endian IEEE 754 for voltage. The field boundaries determine where each rule applies.

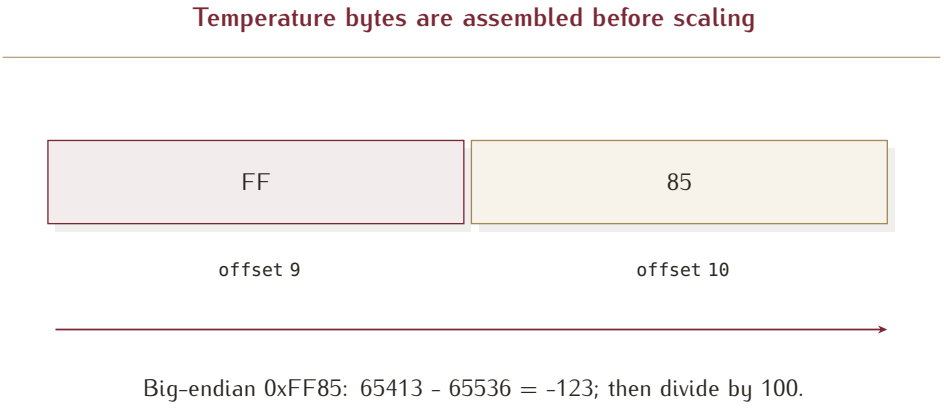


Figure 1.59: *The signed interpretation is applied to the assembled 16-bit pattern. Scaling by the format's hundredths-of-a-degree unit then gives minus 1.23 degrees. Reversing the bytes would produce a different pattern.*

A safe implementation first requires six bytes before reading the length byte. It then enforces $n \leqslant 16$ and proves that the complete record occupies $6 + n + 2 + 4 = 12 + n$ bytes before reading the name, temperature or voltage. TLM1 contains exactly one record, so the supplied length must equal $12 + n$: shorter input is truncated and longer input contains forbidden trailing data. For the three-byte name above, the required length is 15 bytes; buffers of 13 or 14 bytes are still truncated. The parser must also reject reserved flag bits, malformed UTF-8, a NUL byte in the name, and any voltage that is non-finite or negative. Hand-decoding the bytes correctly is not evidence that an unchecked parser is safe. Bounds and semantic invariants belong to the implementation contract as much as endianness does.

A telemetry record begins with a transducer

The record contains integers, fixed-point quantities and floating-point values.

A sensor first converts temperature, pressure, light or motion into voltage, current, resistance or charge. Analogue conditioning filters and scales that signal before an analogue-to-digital converter samples and quantises it; calibration then maps converter codes into engineering units before software serialises them into bytes.

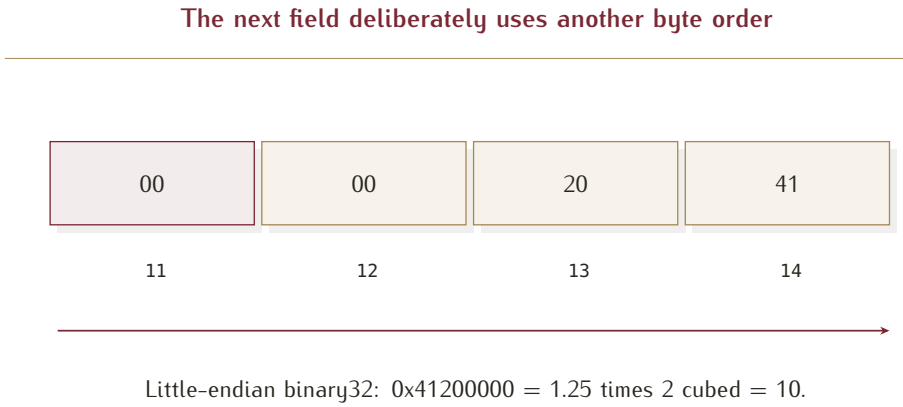


Figure 1.60: *The voltage field has its own representation contract. Its four bytes form a binary32 value with biased exponent 130 and significand 1.25. The parser must additionally reject values that violate the format's finite, nonnegative requirement.*

Representation error is only one component of measurement uncertainty. Sensor tolerance, noise, reference accuracy, sampling and quantisation all precede the file format, so a useful format preserves units and calibration assumptions rather than merely allocating enough bits [43].

1.7.3 From decoded values to safe resource decisions

Representation correctness becomes especially important when a field controls memory or indexing. Suppose an image format supplies 32-bit unsigned width and height followed by raw RGBA pixels. The tempting calculation

```
size_t bytes = width * height * 4;
unsigned char *image = malloc(bytes);
```

can fail before the assignment to `size_t`: if both operands are 32-bit values, their multiplication can occur at that width. Even a mathematically representable result may also violate an application policy by requesting unreasonable dimensions or more data than the file actually contains.

A safer calculation widens intentionally, imposes format limits and proves each multiplication before evaluating it:

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

bool rgba_size(uint32_t width, uint32_t height, size_t *out) {
 if (out == NULL) return false;
 if (width == 0 || height == 0) return false;
 if (width > 20000 || height > 20000) return false;
```

```
  size_t w = width;
  size_t h = height;
  if (h > SIZE_MAX / w) return false;
  size_t pixels = w * h;
  if (pixels > SIZE_MAX / 4) return false;
  *out = pixels * 4;
  return true;
}
```

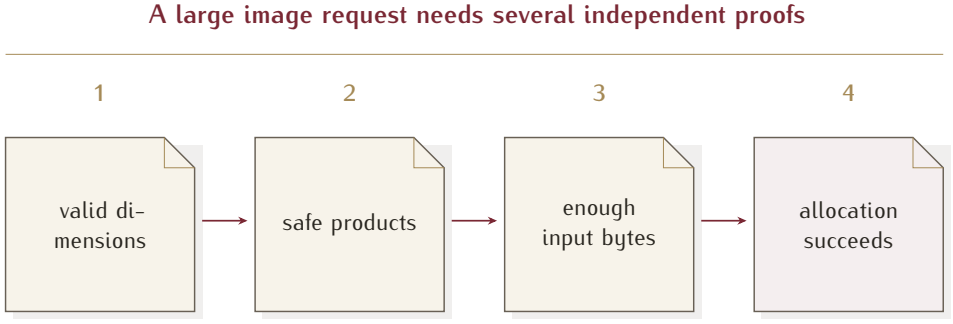


Figure 1.61: *A dimension limit expresses policy, multiplication checks establish representability, and a length check establishes available encoded data. Allocation success establishes storage for the output. These are separate obligations in the RGBA example.*

Each check establishes a different proposition. Arithmetic checks prove representability; the dimension limit states resource policy; a file-length check proves that the encoded bytes exist; and the return from `malloc` establishes whether memory was actually obtained. Treating one of these as evidence for the others is a category error. A value can fit in `size_t` and still be unreasonable, while a valid allocation size does not prove that the input buffer contains the promised pixels.

1.7.4 Boolean policy as another representation layer

Suppose one byte encodes permissions: bit 0 read, bit 1 write, bit 2 execute, bit 3 owner, bit 4 administrator, with bits 5–7 reserved. If modification is permitted only when write is set and either owner or administrator is set, the policy is

$$\text{allow} = W \wedge (O \vee A).$$

The packed representation and the Boolean policy are separate layers. Masks first extract the relevant bits; logical operators then combine the resulting propositions:

```
  const unsigned WRITE = 1u << 1;
```

```
const unsigned OWNER = 1u << 3;
const unsigned ADMIN = 1u << 4;
const unsigned RESERVED = 0xE0u;

bool valid = (p & RESERVED) == 0;
bool allow = valid && (p & WRITE) != 0 &&
             ((p & OWNER) != 0 || (p & ADMIN) != 0);
```

Requiring `valid` in the acceptance condition ensures that reserved-bit violations deny access. A future format version may assign those positions new semantics; silently accepting unknown set bits can make two implementations disagree about what the same byte authorises.

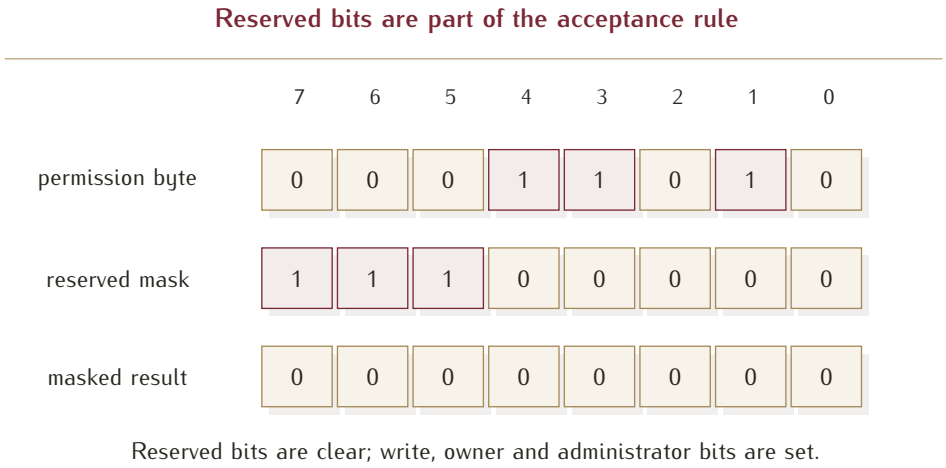


Figure 1.62: *In this illustration the reserved-bit check succeeds. The remaining permissions satisfy write AND (owner OR administrator). A different byte with any high reserved bit set must fail the format-validity check before policy is accepted.*

1.7.5 Numerical results still depend on representation choices

The same synthesis principle applies to arithmetic. A measured quantity stored as binary32 may decode perfectly and still be unsuitable for an algorithm that requires a finite range or reproducible decimal rounding. A financial quantity represented as binary64 may have ample magnitude but fail to represent decimal cents exactly. Conversely, a signed 64-bit count of cents can represent a bounded price domain exactly while still requiring careful rules for interest, exchange rates and division. Choosing a representation requires checking the application's operations, invariants and representable range together.

When two implementations produce different numerical results, the investigation should begin by asking whether they used the same input bytes, the same type widths, the same rounding rules and the same operation order. Chapter 1.6 showed that finite

floating-point arithmetic is not real arithmetic with tiny random errors added afterwards; the rounded representation participates in every operation. This chapter adds the systems question: which component chose that representation, and at what boundary could a different choice have entered?

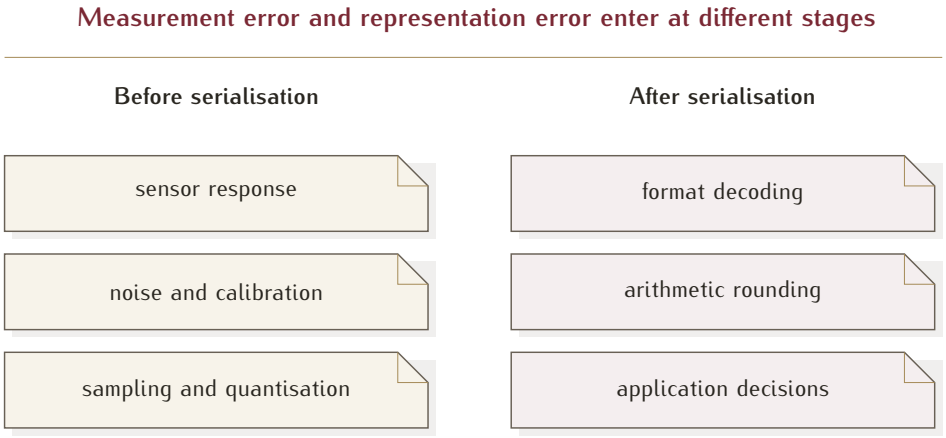


Figure 1.63: *The transducer and conversion chain determine the quality of the measured quantity before the record is written. Later software adds its own representation and arithmetic constraints. The format should preserve the units and assumptions needed to interpret the result.*

1.7.6 Tracing a command from shell syntax to bytes

The representation story also reaches beyond file parsing. When a shell executes a program that reads TLM1 data, several contracts meet before the first field is decoded. The shell interprets command syntax and paths; the kernel resolves the executable and creates or replaces a process image; the loader maps code and data; machine instructions manipulate addresses and registers; system calls obtain bytes from a file descriptor; only then does the parser apply the TLM1 format. An observed decoding failure can therefore originate in the wrong file being opened, a short read, a malformed record or a semantic validation failure. Evidence must locate the layer before the explanation can be trusted.

This is the habit that Part I is intended to establish. A systems programmer should be able to move from physical or stored state to byte pattern, from byte pattern to typed value, and from typed value to program behaviour without collapsing those levels into one another. Once that discipline is in place, the following parts can introduce assembly, datapaths, I/O and concurrency while continuing to ask the same question: which contract gives the observed bits their meaning here?

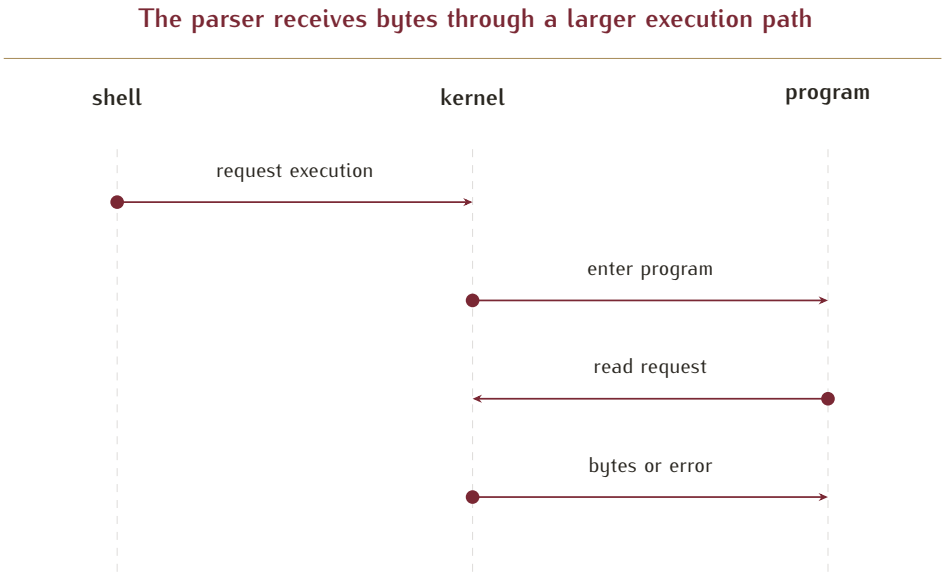


Figure 1.64: *A wrong pathname, failed execution, short read or invalid record can all prevent the intended interpretation. The trace locates the boundary at which evidence changes, helping distinguish an I/O problem from a parsing problem.*

1.7.7 Capstone laboratory

Laboratory 7: TLM1 codec and tests

Implement and test a complete TLM1 encoder and decoder for the contract developed in this chapter. Use explicit byte lengths and endian conventions. Include invalid structural, representation and semantic cases.
Worked solution

Laboratory 7: solution

The following Python implementation returns both semantic fields and their exact byte representations. Temperature is the signed 16-bit scaled integer specified by the format.

```
import math, struct

def encode(name, flags, temperature, voltage):
    if not isinstance(name, str): raise ValueError('name type')
    if type(flags) is not int: raise ValueError('flags type')
```

Worked solution (continued)

```
    if type(temperature) is not int:
        raise ValueError('temperature type')
    if type(voltage) not in (int, float):
        raise ValueError('voltage type')
    raw = name.encode('utf-8', errors='strict')
    if len(raw)>16 or b'\0' in raw: raise ValueError('name')
    if flags not in range(4): raise ValueError('flags')
    if not -32768 <= temperature <= 32767:
        raise ValueError('temperature')
    try: finite = math.isfinite(voltage)
    except OverflowError: raise ValueError('voltage range')
    if not finite or voltage < 0: raise ValueError('voltage')
    try: v = struct.pack('<f', voltage)
    except (OverflowError, struct.error):
        raise ValueError('binary32 overflow')
    if not math.isfinite(struct.unpack('<f',v)[0]):
        raise ValueError('binary32 overflow')
    return (b'TLM1'+bytes([flags,len(raw)])+raw
            +temperature.to_bytes(2,'big',signed=True)+v)

def decode(data):
    if len(data)<6 or data[:4]!=b'TLM1':
        raise ValueError('prefix')
    flags,n=data[4],data[5]
    if n>16 or len(data)!=12+n: raise ValueError('length')
    if flags & 0xfc: raise ValueError('reserved flags')
    name_bytes=data[6:6+n]
    if b'\0' in name_bytes: raise ValueError('NUL')
    name=name_bytes.decode('utf-8',errors='strict')
    tbytes=data[6+n:8+n]; vbytes=data[8+n:12+n]
    t=int.from_bytes(tbytes,'big',signed=True)
    v=struct.unpack('<f',vbytes)[0]
    if not math.isfinite(v) or v<0: raise ValueError('voltage')
    return (name,flags,t,v), (name_bytes,tbytes,vbytes)

sample=encode('A\u00e8',3,-123,10.0)
assert sample.hex()=='544c4d31030341c3a8ff8500002041'
print(decode(sample))
for name in ['', 'x'*16, '\u00e8']:
    for t in [-32768,32767]:
        assert decode(encode(name,0,t,0))[0]==(name,0,t,0.0)
bad=[sample[:i] for i in range(len(sample))]
bad += [sample+b'\0', sample[:4]+b'\x04'+sample[5:]]
bad += [b'BAD!'+sample[4:], sample[:5]+b'\x11'+sample[6:]]
bad += [sample[:6]+b'A\0B'+sample[9:],
```

Worked solution (continued)

```
        sample[:6]+b'A\xffB'+sample[9:]]
for v in [float('nan'),float('inf'),-1.0]:
    bad.append(sample[:-4]+struct.pack('<f',v))
for data in bad:
    try: decode(data)
    except (ValueError,UnicodeDecodeError): pass
    else: raise AssertionError('invalid input accepted')
for fields in [('ok',0,0,1e300), ('ok',0,32768,1.0),
               ('ok',4,0,1.0), ('x'*17,0,0,1.0)]:
    try: encode(*fields)
    except ValueError: pass
    else: raise AssertionError('invalid fields accepted')
v = 0.1
rounded = struct.unpack('<f', struct.pack('<f',v))[0]
assert decode(encode('ok',0,0,v))[0][3] == rounded
```

The worked sample has 15 bytes. Lengths 13 and 14 are rejected; 15 decodes to name Aè, flags 3, scaled temperature −123, voltage 10.0. Truncation, a wrong magic prefix or extra bytes are structural failures. Malformed UTF-8 is a representation failure. Reserved flags, forbidden NUL, out-of-range temperature and invalid voltage violate field semantics. The encoder also checks the rounded binary32 representation, so a finite host value that overflows binary32 is rejected.
Return to the laboratory

A length-delimited measurement

A separate format begins with a 16-bit little-endian body length. The body contains one flag byte, with only bit 0 permitted, followed by a finite binary32 measurement in little-endian order. Decode `0x05 00 01 00 00 80 3F`.
The length field is 5, so the complete record requires $2 + 5 = 7$ bytes, exactly the supplied size. The flag byte is `0x01`, which has no reserved bits set. The remaining four bytes form `0x3F800000`, representing 1.0. The body length also matches its fixed layout: one flag byte plus four measurement bytes. Changing the declared length to 4 would fail this structural check before any floating-point interpretation was needed.

From operator logbooks to whole-system observability

Timeline

Year	Machine, standard or event	Architectural significance
1969	UNIX process tools	Execution is exposed through composable system interfaces.
1986	GNU GDB	A portable symbolic debugger extends source-level inspection.
1991–1993	strace ports	System-call observation spreads from SunOS to Linux.

Debugging originally took place at the boundary between calculation and machinery. Operators of relay and vacuum-tube computers entered programs through switches, cables, cards or paper tape and watched lamps that exposed registers or control states. A wrong result could come from an incorrect formula, a transcription error, a miswired panel, a failed relay or a weak vacuum tube. The diagnostic record was often an operator's logbook rather than a software trace [11, 12, 130].

The famous Harvard Mark II moth belongs in this context. On 9 September 1947, members of the Harvard Computation Laboratory team found a moth in a relay and taped it into the logbook with the note "first actual case of bug being found." Grace Hopper worked with the Mark II team. The word *bug*, however, had already described technical defects in the nineteenth century; the Smithsonian specifically records Thomas Edison's use in the 1870s. The episode is historically useful when told accurately: it did not invent debugging, and it demonstrates that early diagnosis crossed the boundary between program and physical machine [123].

Stored-program computers made internal state both more complex and easier to preserve. A front panel could stop the machine and expose an accumulator or memory address. Printed memory dumps and later magnetic "core dumps" recorded a process image after failure. The term survived after magnetic-core memory disappeared because the diagnostic abstraction was more durable than the technology that named it. Symbolic assemblers then associated addresses with labels, while compilers created a larger gap between source-level variables and generated instructions. Debugging required metadata that could reconnect those levels [11, 12, 130].

By the 1960s, operating systems provided controlled facilities for examining another process. UNIX began on a PDP-7 in 1969 and was subsequently moved to the PDP-11. The first edition of its manual appeared in November 1971; Ritchie and Thompson described the system in their 1974 paper. UNIX exposed the `ptrace` mechanism, used by debuggers to stop a process, read or write registers and memory, and continue or single-step it.

The UNIX tools `adb` and later `sdb` reflected two different views: instruction-level and source-level diagnosis. Richard Stallman began GNU GDB in 1986 as a portable de-

bugger for the GNU system. Current GDB documentation describes breakpoints, watchpoints, stack frames, symbols, remote targets and architecture-specific register sets; those services depend on object-file debug information and kernel process-control interfaces rather than magical access to source variables [19, 11, 12].

Object formats institutionalised this evidence. The Executable and Linkable Format was introduced with UNIX System V Release 4 in 1989, and DWARF grew alongside it for source-level debugging. The 1992 DWARF Version 1 specification preserved compatibility with the format documented for System V Release 4 in April 1990. A compiler can emit DWARF records describing source files, line tables, types, variable locations and call-frame information. An ELF file separates executable sections, symbols, relocations and debugging data.

Optimisation complicates the mapping: a variable may live in a register only for part of its lifetime, be split among locations, be computed on demand or be removed entirely. The debugger reports “optimized out” when the available machine state and debugging metadata cannot recover the requested source-level value [31, 12].

System-call tracing added another layer. On UNIX-like systems, tools such as `strace` report the kernel interface used by a process: system-call numbers, arguments, return values and signals. Disassembly shows what instructions execute; system-call tracing shows where user code requests kernel service. Performance monitoring adds a third view. Hardware counters count events such as retired instructions, cycles, cache misses and branch mispredictions, while sampling tools relate selected events to addresses and symbols. These counters are microarchitecture-specific and can be affected by multiplexing, skid and speculative execution, so their interpretation must be checked against processor documentation [26, 27].

Linux has accumulated several tracing frameworks. Ftrace grew from function tracing work merged into Linux 2.6.27 in 2008 and now provides function, event and graph tracers through tracefs. The `perf` subsystem, integrated around Linux 2.6.31 in 2009, combines hardware performance counters with kernel and userspace sampling.

Extended BPF entered the Linux kernel in 2014, evolving from the original Berkeley Packet Filter into a verified in-kernel execution environment; Linux documentation now treats BPF programs, maps and helpers as infrastructure for networking, tracing and security. Kprobes, uprobes and tracepoints expose different stability and overhead trade-offs. A textbook diagram labelled “trace the program” therefore hides a choice of observation point and a corresponding perturbation of the system [130, 131, 35].

Dynamic analysis became partly preventive. Valgrind, AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer instrument execution or generated code to detect invalid memory access, undefined operations and data races. Record-and-replay systems capture enough nondeterministic input to reproduce a failure. Virtual machines and emulators can expose instruction traces that physical hardware would make prohibitively expensive. Formal verification takes a different route by proving selected

properties without waiting for a failing execution. None of these methods sees the whole system; each establishes evidence under a model [11, 12, 130].

The modern observability problem is harder because execution is layered and concurrent. A latency spike may involve compiler layout, branch prediction, cache coherence, page faults, scheduler migration, an interrupt, a DMA completion or a remote service. Logs alone can preserve chronology but omit causal state. Counters quantify events but may not identify the offending request. Traces reveal paths but can generate enormous data and perturb timing.

The disciplined method remains the one implicit in the 1947 logbook: record the observed state, identify the contract that should have held, and follow the chain of transformations until the discrepancy is localised. The difference is that today's chain crosses source language, ABI, instruction set, microarchitecture, kernel and device. Observability tests the machine model against recorded evidence [11, 12, 130].

1.7.8 Capstone exercises

Exercise 1.7.1 — Decode a modified record

Decode `0x54 4C 4D 31 01 01 5A 00 64 00 00 80 3F` under TLM1.

Go to the solution →

Exercise 1.7.2 — Detect truncation

A TLM1 record declares name length 10 but the entire supplied buffer contains only 12 bytes. Prove that the record cannot contain all mandatory fields.

Go to the solution →

Exercise 1.7.3 — Authorisation truth table

Construct the truth table for $W \wedge (O \vee A)$ and identify the rows that grant access.

Go to the solution →

Exercise 1.7.4 — Choose a representation

A ticketing system stores prices with exactly two decimal places and a maximum value of 10 million currency units. Compare binary64 with a signed 64-bit integer count of cents.

Go to the solution →

Exercise 1.7.5 — Endianness and signed decoding

A temperature field contains bytes `0xFE D4`. Decode it as a signed 16-bit big-endian value in hundredths of a degree, then show the incorrect result obtained by treating the bytes as little-endian.

Go to the solution →

Exercise 1.7.6 — Prove a bounds check

A parser has a buffer of length `L`, a current offset `o` and a field length `n`. Explain why testing `n <= L - o` is safe only after proving `o <= L`, and give a subtraction-free alternative.

Go to the solution →

Exercise 1.7.7 — Bytes, code points and grapheme clusters

The UTF-8 bytes `0x65 CC 81` encode a decomposed accented letter. State the number of bytes and Unicode code points, and explain why a user may still perceive one character.

Go to the solution →

Exercise 1.7.8 — Reserved bits and forward compatibility

A version-1 format defines bits 0–2 and requires bits 3–7 to be zero. Why should a version-1 parser reject a record with bit 6 set rather than silently ignore it?

Go to the solution →

Exercise 1.7.9 — Floating-point semantic validation

A binary32 field decodes to positive infinity. The field is structurally well formed and nonnegative. Why can a parser still reject it, and which test expresses the missing invariant?

Go to the solution →

Exercise 1.7.10 — Exact length versus trailing data

The declared TLM1 record ends at byte 15, but a supplied file has 19 bytes. State the policy required by TLM1 as defined here, then describe a different explicitly specified container policy that could permit the remaining bytes.

Go to the solution →

Exercise 1.7.11 — Cross-layer diagnosis of `EACCES`

A raw `openat` syscall returns `-13`. Separate what can be concluded at the syscall ABI layer from what still has to be investigated at the filesystem and policy layers.

Go to the solution →

Exercise 1.7.12 — Property-based parser tests

Propose four properties for generated inputs to the TLM1 parser, including at least one property about truncation and one about successful round trips.

Go to the solution →

Summary

These cases combine parsing, arithmetic, representation and resource checks. A complete trace records the bytes consumed, the values obtained and the conditions required before each operation. That record allows a faulty assumption to be located at the step where it changes the result.

1.7.9 Solutions

Integrated Case Studies: From Bytes to Behaviour

Solution 1.7.1

Magic is `TLM1`. Flags `0x01` mean valid only. Name length 1 and byte `0x5A` give `Z`. Temperature bytes `0x00 64` are big-endian signed 100, so 1.00 degree. Voltage bytes `0x00 00 80 3F` are little-endian binary32 `0x3F800000`=1.0. The record is valid.

← Return to the exercise

Solution 1.7.2

The fixed prefix through the length byte occupies 6 bytes. The name requires 10, reaching offset 16. Temperature then needs 2 and voltage 4, for a minimum total of $6 + 10 + 2 + 4 = 22$ bytes. A 12-byte buffer is structurally truncated; no decoding of later fields should be attempted.

← Return to the exercise

Solution 1.7.3

Evaluate the parenthesised disjunction before applying write permission:

W	O	A	$O \vee A$	$W \wedge (O \vee A)$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

The granting rows, in WOA order, are 101, 110 and 111. Every row with $W = 0$ denies access. With $W = 1$, either ownership or administrator status supplies the required second condition. The separate reserved-bit validation in the packed implementation must also succeed.

← Return to the exercise

Solution 1.7.4

Binary64 can approximate the values and has ample range, but most decimal cents are not exact binary fractions. A signed 64-bit count of cents represents every permitted price exactly and supports exact addition when the sum remains in range; the maximum 10 million units is one billion cents, far inside range. Integer cents better match the invariant, while multiplication by rates or currency conversion may still require carefully rounded rational or decimal operations.

← Return to the exercise

Solution 1.7.5

Big-endian gives `0xFED4`=65236 unsigned and $65236 - 65536 = -300$, hence -3.00 degrees. Reversing the byte order gives `0xD4FE`=54526 unsigned and -11010, hence -110.10 degrees. Byte order must be applied before signed interpretation.

← Return to the exercise

Solution 1.7.6

Assume that L, o and n all have type `size_t`. If $o > L$, unsigned subtraction `L-o` wraps to a large value and can make the test pass. First require `o <= L`, then test `n <= L-o`. Short-circuit evaluation makes `o <= L && n <= L-o` a safe acceptance test. A subtraction-free alternative uses the defined modulo arithmetic of `size_t` and checks for wrap before accepting the end offset:

```
size_t end = o + n;
bool fits = end >= o && end <= L;
```

For unsigned operands of the same width, a wrapped sum is smaller than o. Therefore `end >= o` proves that addition did not wrap; `end <= L` then proves that the complete half-open interval lies within the buffer. No pointer is formed or dereferenced until both conditions hold. This argument does not apply to overflowing signed addition, which is undefined in C. The empty interval $o = L, n = 0$ is accepted; $o > L, n = 0$ is rejected.

← Return to the exercise

Solution 1.7.7

There are three bytes and two code points: U+0065 LATIN SMALL LETTER E and U+0301 COMBINING ACUTE ACCENT. Unicode text segmentation may present them as one grapheme cluster. Buffer length, code-point count and user-perceived characters are different quantities.

← Return to the exercise

Solution 1.7.8

The bit may acquire meaning in a later version. Silently accepting it lets a version-1 implementation process data under incomplete semantics and can bypass validation. Rejecting unknown reserved bits preserves the version contract; a future parser can opt into the new meaning explicitly.

← Return to the exercise

Solution 1.7.9

Representation validity does not imply domain validity. If the format requires a finite measurement, infinity violates the semantic contract. The parser must test finiteness, for example with `isfinite(value)`, before applying the nonnegative range check.

← Return to the exercise

Solution 1.7.10

A strict single-record format rejects any trailing bytes, which detects concatenation and hidden data. This is the required policy for TLM1 as defined in this chapter, so the 19-byte file must be rejected. A different, explicitly specified container may permit an extension area or another record and must define how those bytes are framed and validated. Accepting trailing bytes without such a rule is ambiguous and can create parser differentials between components.

← Return to the exercise

Solution 1.7.11

At the ABI layer, `rax` contains `-EACCES`; a library wrapper would return -1 and set `errno` to 13. The result does not identify the cause by itself. Path permissions, directory search permission, mount options, credentials and security-module policy remain possible explanations.

← Return to the exercise

Solution 1.7.12

Useful properties cover both valid and invalid inputs. Every prefix shorter than the required record length must be rejected without an out-of-bounds read, and any reserved flag bit must cause rejection. Encoding and then parsing a valid record must preserve the name, flags and scaled temperature. Voltage must equal the input rounded to binary32, since a host binary64 value such as 0.1 generally changes during encoding. For every accepted wire record, decoding and re-encoding should reproduce its bytes, including the sign of zero. Mutating a length byte must never

Worked solution (continued)

permit allocation or access beyond the supplied buffer. Additional properties can constrain UTF-8 validity and finite voltage values.

← Return to the exercise

2

MACHINE-LEVEL PROGRAMMING AND THE X86-64 INTERFACE

Machine code exposes the state hidden by a high-level language. Registers, memory operands, flags and the instruction pointer make every step explicit. Trace how values move, how control changes and how binary conventions allow separately compiled code to cooperate.

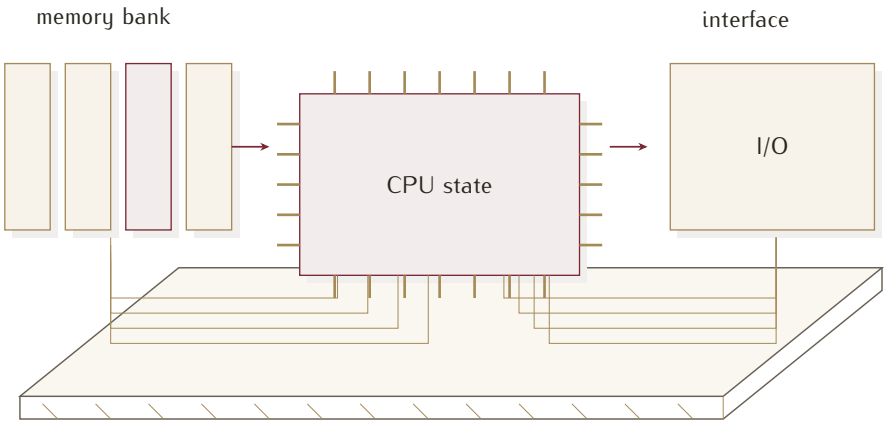
The x86-64 examples generally use Intel syntax, with AT&T listings identified where they appear. The instruction and calling conventions are checked against the architecture manuals and the System V AMD64 ABI [26, 27, 30]. Four layers remain separate throughout. Instruction effects belong to the ISA; argument registers and stack alignment belong to an ABI; compiler choices depend on toolchain and optimisation; Linux system-call numbers and entry rules belong to an operating-system interface [32, 33]. A disassembly can expose all four at once, but they should not be conflated.

Every worked trace begins from a stated machine state. Operand width and effective addresses are established before flags, branches or high-level intent are inferred. The same discipline supports debugging, reverse engineering and defensive analysis of stack corruption: first prove what the instructions do, then explain what the surrounding program means.

2.1 MACHINE STATE, REGISTERS, AND MEMORY

Stop a program just before an instruction executes. The debugger shows registers, flags, memory and an instruction pointer. Now take one step and compare the two snapshots. Machine-level reasoning grows from that comparison: identify what the instruction reads, calculate its result at the declared width, and record exactly which parts of the state change. Unchanged state is equally useful evidence when reconstructing a longer computation.

Instructions transform visible machine state



Signals cross interfaces between components.

Figure 2.1: *Registers and memory provide state for instruction execution. The board traces represent communication paths rather than a specific commercial processor layout. The chapter distinguishes architecturally visible effects from internal implementation.*

Most application development takes place above the machine interface: registers are hidden, addresses appear through symbolic objects and the compiler normally manages byte order and instruction selection. The abstraction becomes insufficient when a program faults, when a mitigation must be assessed, when a crash dump is inspected or when disassembly is the only available evidence. Understanding the programmer-visible machine is therefore essential for debugging, reverse engineering and defensive analysis of memory corruption.

The foundation consists of register vocabulary, byte-addressed memory, little-endian layout, operand widths and the distinction between an address and the value stored there. Without that foundation, even short disassemblies remain opaque; with it, common data movement, arithmetic and control-flow patterns become analysable.

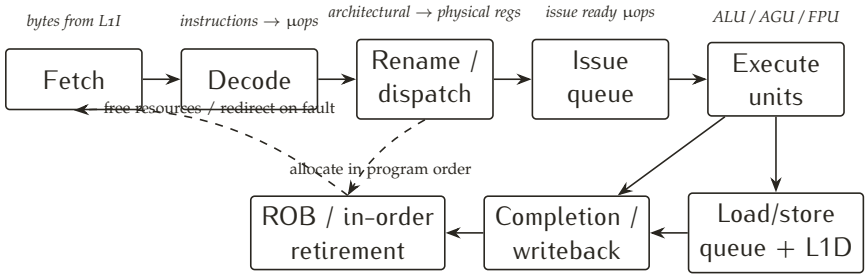
2.1.1 Architectural state and execution

A sequential architectural model describes execution as a repeated sequence:

1. it *fetches* an instruction from memory at the address held in the x86-64 *instruction pointer*, `rip`;
2. it *decodes* the bytes, determining the opcode and the operands;
3. it *executes* the operation, updating registers and flags according to the instruction, and sometimes writing memory;
4. it advances `rip` by the instruction's length and returns to step 1.

Exceptions to this cycle (jumps, calls, interrupts, syscalls) modify `rip` non-sequentially, but the principle holds: everything the CPU "knows how to do" reduces to elementary operations on registers and memory.

An implementation may overlap these activities through a pipeline; the number and arrangement of stages depend on the processor. A simplified out-of-order core can be sketched as follows. Fetch and decode form the front end; rename, scheduling, execution, memory handling and retirement extend beyond it:



Fetch pulls instruction bytes, normally from the L1 instruction cache. *Decode* turns them into one or more internal micro-operations (μops). *Rename/dispatch* maps architectural registers onto physical registers and allocates bookkeeping entries. The issue logic selects ready μops for execution, potentially out of program order. Integer, address-generation and floating-point units produce results; loads and stores also pass through ordering and data-cache structures. Completion broadcasts results to dependent work. *Retirement* then confirms completed instructions in program order, makes stores eligible for architectural commitment and updates the committed rename state. Retired stores can remain in a store buffer before becoming globally visible. It does not generally copy every value into a separate bank of "architectural registers": the precise mechanism is microarchitecture-specific.

The baseline x86-64 model used here has sixteen 64-bit *general-purpose* registers: `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp`, `rsp`, `r8`–`r15`. All can hold numbers or addresses interchangeably; there is no type tag in a register's bits.

The *instruction set architecture* defines the state and effects visible to software; it does not prescribe the internal pipeline, physical register file or scheduling policy. Integer instructions name immediates, registers and, where the opcode permits it, memory operands. A common effective-address calculation is

$$A = d + B + sI, \qquad s \in \{1, 2, 4, 8\},$$

where the displacement d, base B and index I are optional according to the encoding. The expression computes an address; only an instruction that requests a load or store dereferences it. Most ordinary two-operand integer instructions allow at most one explicit memory operand, and not every opcode accepts every addressing form [26, 27]. The `lea` instruction exposes the address calculation itself without reading memory.

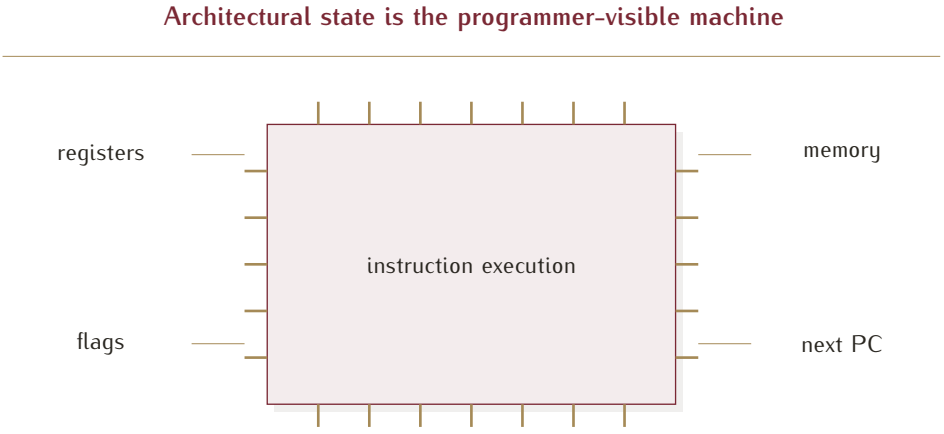
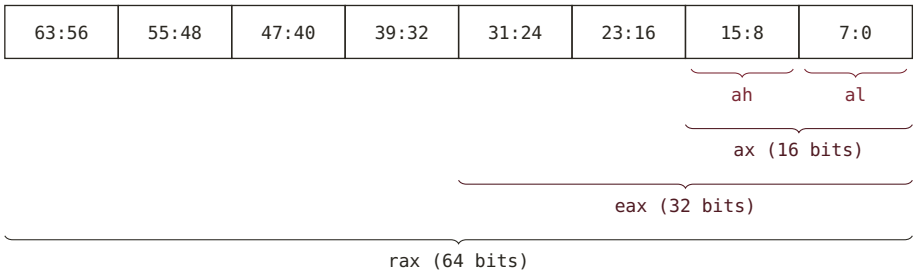


Figure 2.2: *The drawing separates the state visible to software from the machinery that implements it. A trace records changes to the relevant registers, memory bytes, flags and instruction pointer after each instruction.*

Each legacy 64-bit general-purpose register exposes low 32-, 16- and 8-bit names. For `rax`: `eax`, `ax`, and `al`; the legacy high-byte name `ah` addresses bits 8–15 but cannot be encoded in an instruction that uses a REX prefix. The same restriction applies to `bh`, `ch` and `dh`.



One x86-64 rule governs every later subregister trace:

> *Every write to a 32-bit register (*`eax`*,* `ebx`*, . . .)* implicitly zeroes *the upper 32 bits of the corresponding 64-bit register. Writes to 16- or 8-bit sub-registers leave the upper bits unchanged.*

So `mov eax, 5` puts `0x0000000000000005` in `rax`; `mov al, 5` sets only the low byte and leaves the rest as it was. Remember this asymmetry.

A register file is a small, heavily wired memory

Registers form a named array of fixed-width values with rapid access.

A physical register file contains storage cells, decoders, write drivers and extensive selection wiring. Depending on the circuit style, reads may use multiplexed paths or bit lines and sensing circuitry. Multiple simultaneous read and write ports require additional access devices, wires and selection logic. Those ports consume area and energy and become harder to scale as the number and width of registers increase.

Registers are fast partly because the structure is relatively small and close to execution units. Large out-of-order cores may bank, replicate or distribute physical-register storage because one ideal unrestricted multiported array would be too slow or power hungry [50, 42].

2.1.2 Registers, addresses, and memory

From a user-space program's view, memory is a byte-addressed virtual address space whose addresses are carried in 64-bit registers. Current x86-64 implementations support a smaller canonical virtual-address width, and the architecture constrains the unused high bits. Only a small fraction of the nominal space is mapped at any instant. The operating system records which ranges are readable, writable or executable and what backs them.

An access to an unmapped or disallowed address raises a page-fault exception. The kernel may resolve the event, as in demand paging or copy-on-write, or deliver a signal such as `SIGSEGV`; termination occurs only if the condition remains unhandled. Each process sees a private but sparse virtual address space, and page tables translate resident virtual pages to physical frames. The operating system separately tracks nonresident backing storage and can populate a frame after a page fault.

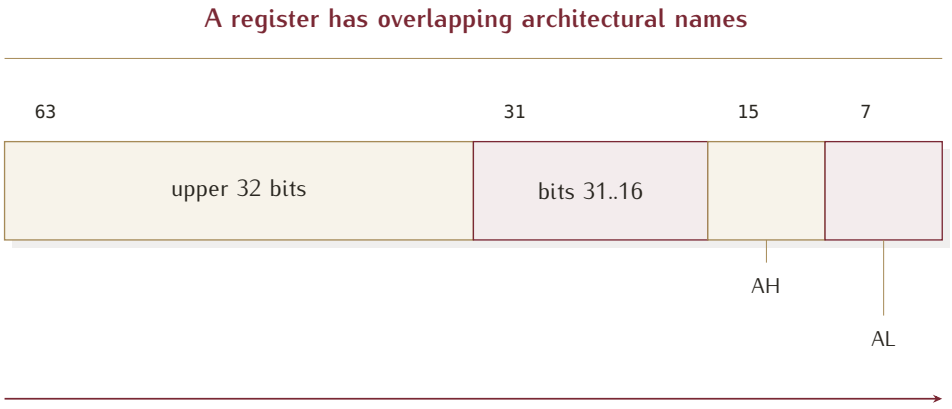


Figure 2.3: *The field widths are proportional to their bit counts. A write to EAX also clears the upper 32 bits of RAX. Writes to AX or AL ordinarily preserve the remaining bits of the containing register.*

A 64-bit register can hold any bit pattern, including one that would be invalid as a memory address. When an instruction accesses memory, the address must satisfy the processor's canonical-address rules and the requested bytes must be accessible. The ordinary integer accesses considered here use 1, 2, 4 or 8 bytes, addressed by their lowest byte. A noncanonical memory address normally raises a general-protection exception, or a stack-segment exception for certain stack accesses, rather than an ordinary page fault.

Writing the qword `0x1122334455667788` to memory, in what order do the bytes go? On x86-64 the convention is *little-endian*: the least significant byte (`0x88`) goes to the lowest address.

address	A	A+1	A+2	A+3	A+4	A+5	A+6	A+7
byte	88	77	66	55	44	33	22	11

The same eight bytes read as a dword at A are `0x55667788`; as two words, `0x7788` and `0x5566`. The load width determines which bytes contribute to the value.

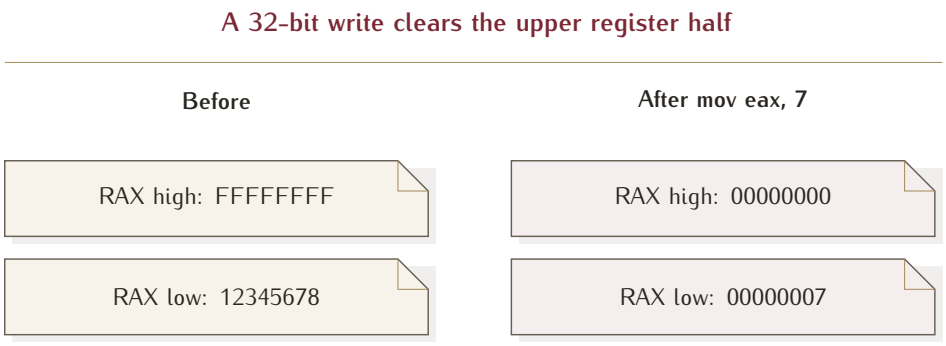


Figure 2.4: *The 32-bit destination write replaces EAX and zeroes the upper half of RAX. This behaviour differs from writing a low 8-bit or 16-bit subregister, which leaves other bits unchanged.*

The key idea of pointer arithmetic is the distinction between an address *as data* (a 64-bit number) and an address *as a direction* ("go to that cell and read/write"). In an Intel-syntax `mov`, brackets distinguish a memory operand from a register operand: `mov rax, rdi` copies the register value, while `mov rax, [rdi]` reads memory at that address. The brackets also describe the address expression used by `lea`, which calculates a value without accessing memory. The opcode therefore determines whether an address expression is dereferenced.

Figure 2.5 uses one four-byte object to connect addresses, stored bytes and register values. It can be checked directly against a debugger's byte dump.

A shorter load from the same bytes

With the bytes in Figure 2.5, a little-endian 16-bit load starting at `0x1001` combines `0x56` with `0x34` at weight 2^8, giving `0x3456`. The bytes at `0x1000` and `0x1003` lie outside this access. A different destination-register extension rule changes the upper register bits, while the two bytes read from memory remain the same.

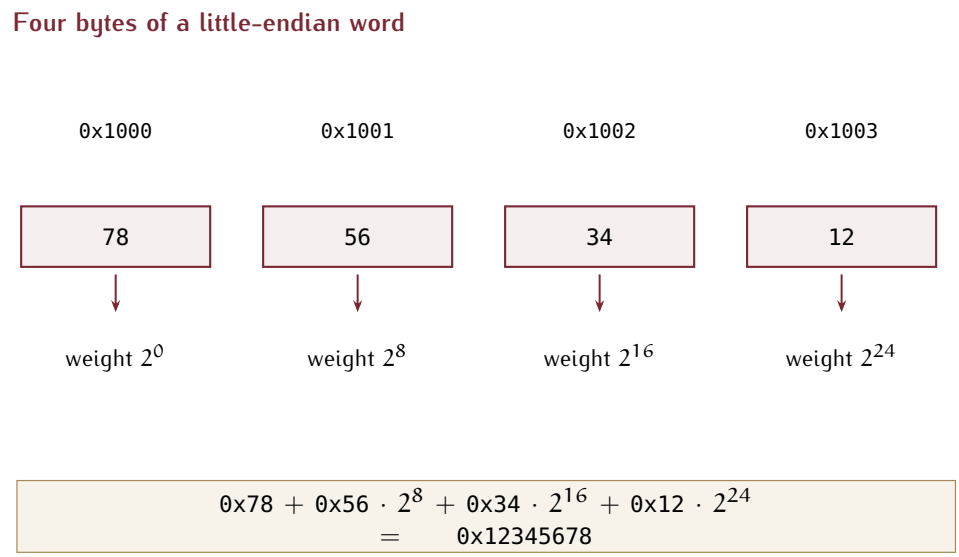


Figure 2.5: *Addresses increase from left to right. A 32-bit load at* `0x1000` *assigns the lowest-addressed byte the least significant weight. A one-byte load at* `0x1002` *reads* `0x34`*; its extension into a register is a separate instruction property.*



Figure 2.6: *The arrows describe dereferencing in a model where both ranges are readable. Copying RAX copies its numeric value. Loading through RAX instead reads bytes at the location that value names.*

2.1.3 Reading short instruction sequences

Worked example

Problem. End with `rcx` = `0xCAFEBABE` and `r8` = `0x2A`. No syscall.
Solution. The `mov r64, imm64` form can encode a full 64-bit immediate and is ten bytes for all sixteen baseline general-purpose registers; shorter encodings exist when the value fits a sign-extended or zero-extended 32-bit form. The compact form exploits zero-extension:

```
mov ecx, 0xcafebabe; 32-bit immediate; rcx is zero-extended
mov r8d, 0x2a; same for r8 (REX prefix needed)
```

Writing `ecx` is identical in result to writing `rcx` zero-extended, but five bytes instead of ten.

Caution

GNU assembler normally encodes `mov rcx, 0` in seven bytes. Writing `mov ecx, 0` takes five bytes and preserves flags. If flags may change, `xor ecx, ecx` takes two bytes and is recognised as a dependency-breaking zero idiom on many modern implementations.

Worked example

Problem. `rsi` holds a valid address. Copy the same numeric value into `rbx` without reading the memory it points to.
Solution.

```
mov rbx, rsi; register-register copy, no memory access
```

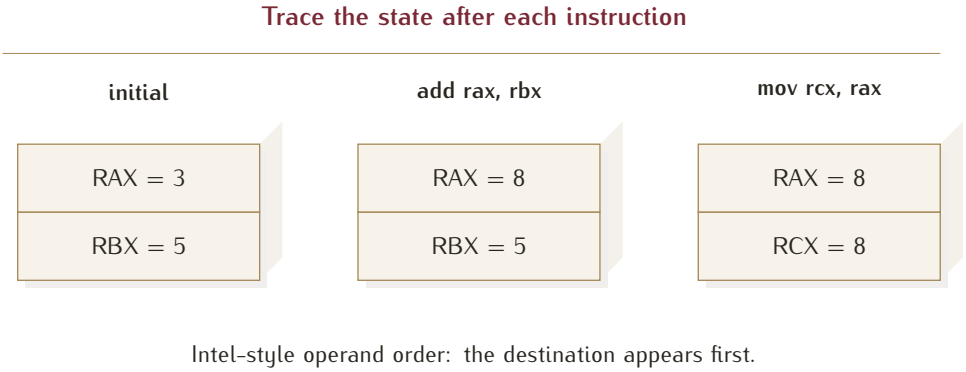


Figure 2.7: *The add replaces RAX with the sum and leaves RBX unchanged. The subsequent mov copies the resulting value into RCX. The snapshots show only registers needed for this trace; add also updates arithmetic flags.*

The wrong form, `mov rbx, [rsi]`, dereferences `rsi` and copies eight bytes of memory.

`lea` calculates an effective address and writes the numeric result to a register. `lea rbx, [rsi+rcx*4+0x20]` computes `rsi`+4·`rcx`+`0x20` in one instruction, touching no memory and not altering the flags, which is why it is often used as a "multiply-and-offset" even when no address is involved.

Worked example

Problem. `rdx` points to a writable 8-byte buffer. Store `0x0011223344556677` so the byte at `rdx` is `0x77`, at `rdx`+1 is `0x66`, and so on.

Solution.

```
mov rax, 0x0011223344556677
mov [rdx], rax; 8-byte store, little-endian
```

Caution

A typical beginner error is `mov [rdx], 0x0011223344556677`: x86-64 has no general `mov r/m64, imm64` encoding. The 64-bit memory form carries an imm32 that is sign-extended. An assembler may reject the out-of-range constant or diagnose truncation; inspect diagnostics and the emitted encoding. Split into two dword stores if you must use immediates.

Worked example

Problem. `rsi` points to a byte b. Load it into `rcx` so `rcx` equals b, with the other 56 bits zero.
Solution.

```
movzx rcx, byte ptr [rsi]; load a byte, zero the high bits
```

Plain `mov cl, [rsi]` would leave the upper 56 bits untouched (possibly garbage). `movsx` instead sign-extends: use `movzx` for unsigned counts/indices, `movsx` for signed values.

Worked example

Problem. `rsi` holds a 64-bit value. Extract byte number 3 (bits 24–31), zero-extended, into `rax`.
Solution.

```
mov rax, rsi
shr rax, 24; bits 24..31 move to 0..7
movzx rax, al; keep the low byte, zero-extend
```

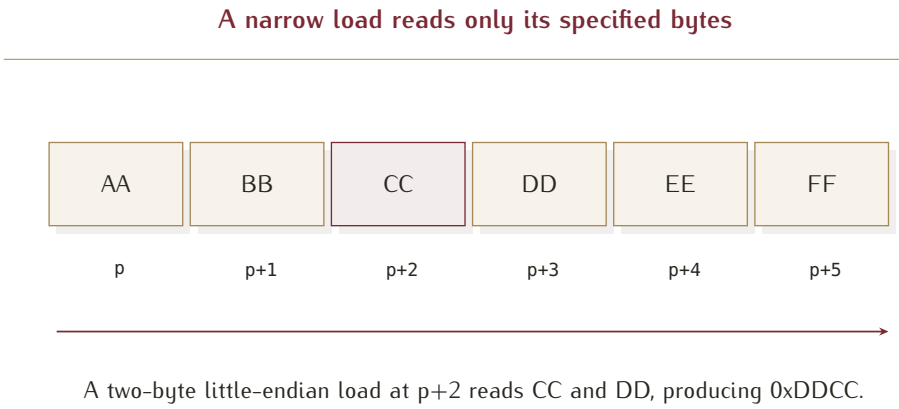


Figure 2.8: *The load width determines the byte interval. Bytes before or after that interval do not contribute to the result. The example assumes the entire two-byte interval is readable.*

Caution

The memory operand in `mov al, byte ptr [rsi+3]` uses the contents of RSI as an address and reads at that address plus three. The value in RSI here represents packed data. This instruction could therefore read unrelated memory or fault. "Byte 3 of a register value" $\neq$ "byte at address register+3".

Worked example

Problem. Exchange `r12` and `r13` without `xchg` and without altering other general-purpose registers. Flags may change.
Solution.

```
xor r12, r13; r12 = r12 xor r13
xor r13, r12; r13 = original r12
xor r12, r13; r12 = original r13
```

This works because XOR is its own inverse: $(x \oplus y) \oplus y = x$.

The XOR swap has a subtle failure: if the two operands were the *same* register, the first XOR zeroes it and it cannot be recovered. `xchg` or a stack temporary is immune to that degenerate case.

Worked example

Problem. `rdi` points to a qword p; p is itself an address pointing to a qword v. Leave v in `rax`.
Solution.

```
mov rax, [rdi]; rax = p (first dereference)
mov rax, [rax]; rax = v (second dereference)
```

The model is "reassign `rax` at each level".

Worked example

Problem. `rdi` holds a base pointer. Leave `rdi+0x40` in `rax` without dereferencing.
Solution.

```
lea rax, [rdi+0x40]; rax = rdi + 0x40, no memory read
```

The general `lea` form is

$$\texttt{lea}\ \text{dst},\ [\text{base} + \text{index} \cdot \text{scale} + \text{offset}], \quad \text{scale} \in \{1, 2, 4, 8\}.$$

When inputs must survive a destructive op, copy first:

```
mov rax, rdi; save original rdi
mov rbx, rsi; save original rsi
lea rdx, [rdi+rsi]; rdx = rdi + rsi, sources untouched
```

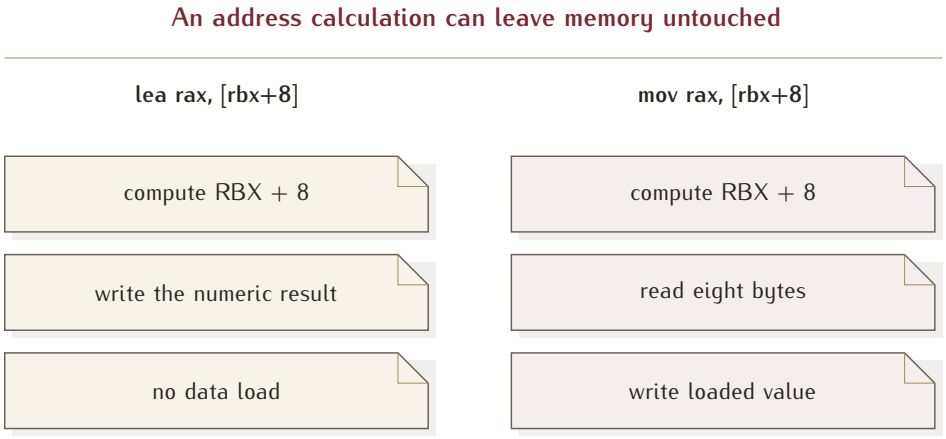


Figure 2.9: *LEA computes an effective-address expression without dereferencing it. MOV with a memory source uses the computed address to access memory. This distinction matters for both correctness and possible memory faults.*

A load can cross several physical storage structures

A load reads bytes from one address space.

A recent store may satisfy the load through a store-buffer forwarding path without reading a cache array. Otherwise the request may hit in an SRAM cache, proceed to a larger on-chip cache, or reach an off-chip DRAM array. Each structure uses different cells, wire lengths and sensing circuits. A miss launches messages across interconnects and may require a DRAM row activation before the requested column is returned.

The same instruction can take very different time and energy depending on where the value is found and which paths it traverses. The uniform address-space abstraction is essential for programming, but performance reasoning requires the underlying hierarchy [47, 50].

2.1.4 Architectural state, implementation, and performance

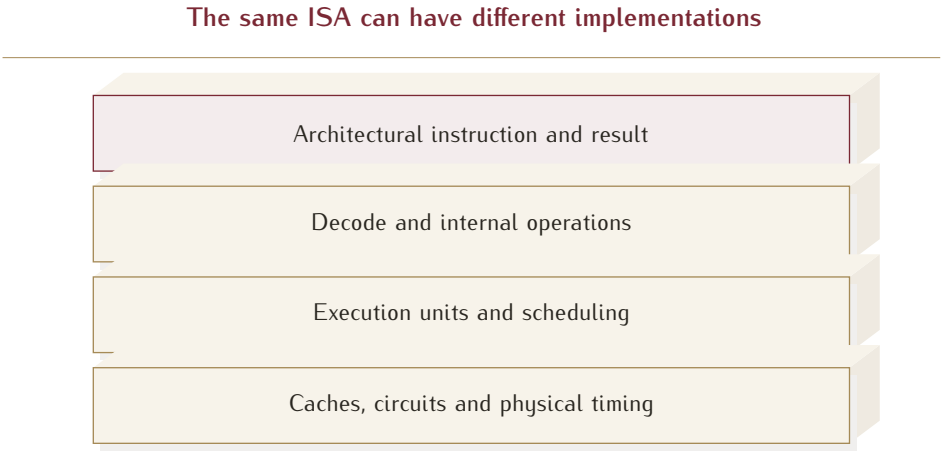


Figure 2.10: *The layers explain why instruction semantics alone do not specify a universal latency or throughput. Different processors can preserve the same visible result while organising internal work differently.*

The short traces above are useful because they operate entirely at the architectural boundary: a register has a committed value, `rip` identifies the next architectural instruction, and a memory operand denotes bytes in the process's address space. That boundary is stable even when the implementation underneath it is aggressively speculative. A modern out-of-order core may rename an architectural destination such as `rax` onto one of many physical registers, execute independent instructions before older ones have retired, and satisfy a load from a store-forwarding path or a cache rather than from DRAM. None of those choices changes the state transition promised by the ISA. If an older instruction faults or a branch prediction proves wrong, younger speculative effects must be prevented from becoming committed architectural state [26, 27, 50].

The same separation explains why assembly can expose an address without revealing where the corresponding bytes are physically located. A user-space address is virtual; translation and protection determine whether an access is permitted, while the memory hierarchy determines where a valid value is found at a particular moment. The instruction `mov rax,[rsi]` therefore has a simple architectural meaning even though its latency may depend on store forwarding, cache hits, DRAM service or, in the exceptional case, a page fault handled by the operating system.

To calculate CPU execution time on one processor at a fixed clock frequency, use

$$T_{CPU} = N_{inst} \times CPI \times T_{clock},$$

where instruction count is determined by the executed path and compiler choices, while CPI summarises the average cost per retired instruction for that workload on that implementation. A shorter clock period does not by itself guarantee a shorter program, because cache misses, dependencies, branch recovery and execution-resource pressure can raise CPI. Conversely, one instruction may have a long latency while the core still sustains high throughput by overlapping independent work. These distinctions become central in Part III; here they serve one purpose: they keep the programmer-visible machine separate from the mechanisms used to realise it.

A disciplined trace should therefore record only facts justified at the level being analysed. At the ISA level, state which operands are values, which are addresses, which registers or memory bytes change, and how `rip` advances. When performance matters, add explicit assumptions about the hierarchy or microarchitecture rather than smuggling them into the architectural trace. That discipline makes a debugger, disassembler and optimisation manual complementary sources rather than competing descriptions of the same machine.

A debugger displays committed architectural state at a stopping point, without exposing every speculative physical-register value. Its register view must be interpreted using the ISA rules established above.

A complete instruction-state update

At a breakpoint, `rip=0x401120`, `rax=7` and `rbx=3`. The next three bytes, `0x48 01 D8`, encode `add rax, rbx` in Intel syntax. Execution sets `rax` to 10 and `rip` to `0x401123`, while `rbx` remains 3. The arithmetic flags become CF=0, OF=0, SF=0, ZF=0 and AF=0. PF=1 because the low result byte `0x0A` contains two set bits, an even count. Other flags remain unchanged. These are instruction effects; the registers' possible roles as source-language variables require separate debugging information.

An unaligned access may cross physical boundaries

An instruction reads or writes a fixed number of consecutive bytes.

If those bytes cross a cache-line, page, bank or internal transfer boundary, hardware may split the request into multiple operations. Two cache lines or translations may be involved, and a store may require separate masked updates and coherence transactions for the affected regions. Whether an access is atomic depends on the ISA guarantees, access size, alignment, memory type and the boundaries crossed; it cannot be inferred merely from the fact that the instruction is permitted.

x86-64 generally permits unaligned ordinary data accesses, but permitted does not mean physically identical in cost or universally atomic. Alignment affects latency, internal traffic and the possibility of crossing a protection or page boundary [26, 28, 47].

Worked example: comparing two processors

Processor A executes 1.2×10^9 instructions at an average CPI of 1.5 and a frequency of 3.0 GHz. Processor B executes 1.0×10^9 instructions at CPI 1.8 and 3.6 GHz.

$$T_A = \frac{1.2 \times 10^9 \cdot 1.5}{3.0 \times 10^9} = 0.60 \text{ s}, \qquad T_B = \frac{1.0 \times 10^9 \cdot 1.8}{3.6 \times 10^9} = 0.50 \text{ s}.$$

B wins even though its CPI is worse, because it executes fewer instructions and has a shorter clock period. A benchmark result depends on the program, its compiled instruction stream and the processor that runs it.

Caution

CPI averages the cycles consumed over the instruction stream being measured. Cache misses, branch mispredictions, dependencies and contention change the observed value. The same executable can have different CPI on different inputs.

A transfer has two distinct costs. *Latency* is the delay before useful data arrives. *Bandwidth* is the sustained amount of data transferred per unit time. A small dependent load is dominated by latency; a long sequential copy can approach bandwidth limits.

Fast storage is expensive and small; large storage is cheaper and slower. Systems therefore use a hierarchy: registers, several cache levels, DRAM and persistent storage. Caches work because programs exhibit *temporal locality* (recently used data tends to be reused) and *spatial locality* (nearby addresses tend to be used together).

Worked example: why contiguous traversal matters

Assume cold caches and a 64-byte-aligned array with 64-byte lines, each holding eight qwords. Reading 1024 consecutive qwords touches 128 lines. Performing 1024 reads separated by 64 bytes requires a larger array and touches 1024 lines, using one eighth of each line. The number of loads is equal, but the second pattern requires eight times as many distinct line fills under this model.

Memory safety failures become easier to analyse when addresses are treated as ordinary data plus access permissions. A crash at `mov rax,[rdi]` can arise because `rdi` is unmapped, lacks read permission, is non-canonical, or crosses into an inaccessible page. A pointer to an object whose lifetime has ended is also a software error, but the hardware may still permit the access while the page remains mapped. The instruction is simple; the validity of the address is the real question.

For short fragments, draw a table with one row per instruction and columns for every register or memory cell that can change. Record values in hexadecimal and annotate widths. This avoids mentally mixing old and new values.

Worked example: subregister effects

Initial state: `rax=0x1122334455667788`, `rbx=0xaabbccddeeff0011`.

```
mov al, bl
mov eax, 0x12345678
add rax, 1
```

After the first instruction, only the low byte changes, so `rax=0x1122334455667711`. The second instruction writes `eax`, automatically clearing the high 32 bits: `rax=0x0000000012345678`. The final value is `0x0000000012345679`.

Laboratory 8: Processor inventory and code

Interpret a model inventory containing 8 logical CPUs, 4 cores, two threads per core, 32 KiB L1 data cache per core and a 64-byte line. Then derive a valid optimised System V x86-64 implementation of `uint64_t combine(const uint64_t *p){return p[0]+p[1];}`. State what must be measured on another machine.
Worked solution

Laboratory 8: solution

The 8 logical CPUs are $4 \cdot 2$ hardware execution contexts, not eight independent L1 data caches under the stated sharing model. A 32 KiB cache has $32768/64 = 512$ lines of data capacity before metadata. An ABI-valid function is:

```
mov rax, [rdi]
add rax, [rdi + 8]
ret
```

The pointer arrives in `rdi`. The first load reads element 0, the memory operand of `add` reads element 1, and the sum returns in `rax`. No store to the source array is required. At low optimisation the compiler may spill arguments or intermediates to a stack frame. To inspect a real host use `lscpu`, `/proc/cpuinfo`, and cache `level`, `type`, `size`, `shared_cpu_list`, `coherency_line_size` files under sysfs. Record the measured inventory separately from this supplied example; a virtual machine may expose a synthetic topology.
Return to the laboratory

From accumulators to the x86-64 architectural state

Timeline

Year	Machine, standard or event	Architectural significance
1949	EDSAC	A small accumulator-centred state supports stored programs.
1978	Intel 8086	A segmented 16-bit programmer-visible state begins the x86 line.
2003	AMD64 deployment	General registers and virtual addresses expand while compatibility is retained.

The register file of a modern processor is the result of several distinct design traditions. Early calculators often had one privileged arithmetic location. In mechanical machines it might be a wheel assembly; in electronic machines it became an accumulator. ENIAC had twenty decimal accumulators, each holding a signed ten-digit number and participating directly in arithmetic. EDSAC, which entered operation in 1949, used an accumulator associated with a single-address instruction format. When an instruction names only one memory operand, the other operand and the result are implicitly tied to the accumulator. This saves instruction bits but forces many intermediate values through one architectural location [87, 91].

Stored-program machines soon added specialised registers. A program counter identified the next instruction. An instruction register held the current operation. Index registers modified addresses so loops could traverse arrays without rewriting instructions. The Manchester Mark 1, operational in 1949, included two B-lines, an early and influential form of index register. Stack pointers, base registers and status registers appeared as operating systems and high-level languages demanded more structured state. Each register reduced recurring instruction sequences or enabled a new software abstraction [90].

General-register architectures relaxed the accumulator bottleneck. Several registers could hold operands, addresses and intermediate results, allowing compilers to keep active values close to the arithmetic units. IBM System/360, announced in 1964, exposed sixteen 32-bit general registers and four 64-bit floating-point registers in its original architecture. The general registers could serve as accumulators, base registers and index registers. This regularity supported compilers and made one architecture viable across a family of machines. It also established the idea that register state is part of a stable software contract even when each model implements it differently [63, 109].

The x86 line began with more visible specialisation. Intel's 8086, introduced in 1978, exposed AX, BX, CX and DX with historical roles in arithmetic, base addressing, counting and data; SP, BP, SI and DI supported stacks and indexed data; segment registers participated in address formation. The architecture allowed many uses beyond the mnemonic names, but instruction encodings and implicit operands preserved the distinctions. AX could be split into AH and AL, and several instructions assumed accumulator or count registers. Compatibility made these conventions durable [104, 26].

The 80386, introduced in 1985, extended the main integer registers to 32 bits and added the E prefix: EAX, EBX and so on. It also introduced a 32-bit instruction pointer, EIP, and 32-bit paging. Floating-point and SIMD extensions followed their own line of development. The x87 interface used an eight-entry floating-point register stack. MMX later exposed packed-integer registers that alias the x87 architectural register state. SSE introduced XMM registers; AVX extended them with YMM state, and AVX-512 added ZMM state and mask registers. Architectural state grew in width and acquired new register classes for media, scientific and cryptographic workloads [106, 26].

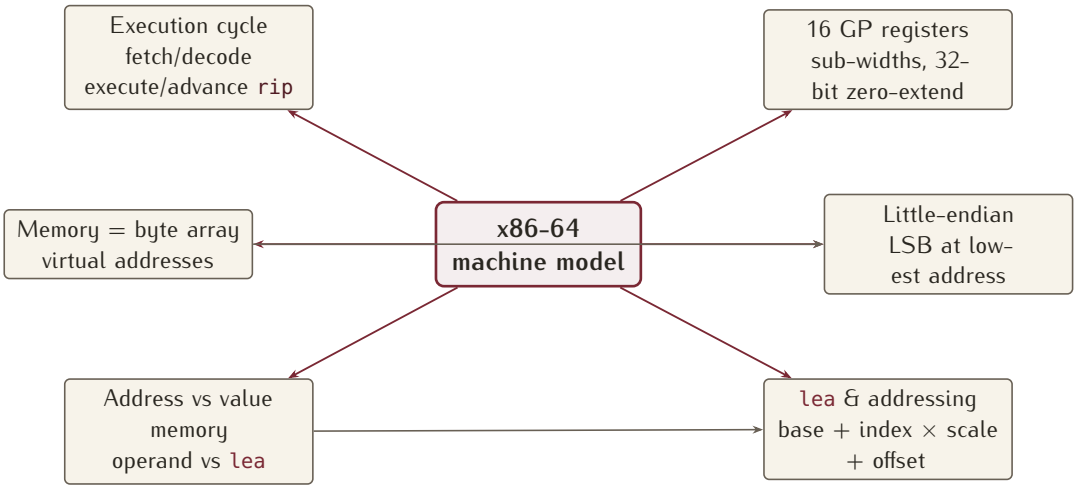
AMD designed the 64-bit extension that became the contemporary desktop and server x86 architecture. AMD published the AMD64 design around 2000 and introduced Opteron processors in April 2003, followed by Athlon 64 in September 2003. Long mode widened the classic registers to RAX through RSP, added R8 through R15 and used a largely flat virtual-address model. The extension deliberately preserved 32-bit compatibility. Intel later implemented the compatible architecture as Intel 64. The addition of eight general registers was as important to compiled code as the wider integers because it reduced spills and made a register-based calling convention practical [107, 27, 26].

The subregister rules described earlier preserve compatibility while reducing some partial-register dependencies: a 32-bit write supplies a complete new 64-bit value, whereas a smaller write preserves part of the previous value. Their asymmetry reflects the constraints of extending an existing encoding [26, 27].

Operating systems enlarge the effective machine state further. A thread context includes general registers, instruction pointer, flags, segment-related state, vector state, control state and metadata. Linux does not save every possible component on every trivial transition; modern x86 provides mechanisms such as XSAVE to manage extended state components. Debug registers, performance counters and model-specific registers belong to privileged or specialised interfaces and are not ordinary process registers. A debugger reconstructs a view from kernel interfaces and saved frames [26, 27, 50].

A disciplined reading of machine state should therefore use three levels. The ISA manual defines architectural registers and instruction effects. The System V AMD64 ABI assigns software roles, such as argument and callee-saved registers. The microarchitecture decides renaming, execution ports and physical implementation. Confusing these levels produces errors: a register can be caller-saved by ABI without being destroyed by every instruction, and a false dependency caused by reuse of a register name can be removed by renaming, while a true dependence on a produced value remains. The historical path from accumulator to AMD64 makes those distinctions visible [26, 27, 30, 28, 29].

Concept map: machine state & memory



Reading the map. Five concepts make machine code legible: the fetch–execute cycle, the register file (with its zero-extension quirk), memory as a flat byte array under virtual addressing, the little-endian byte order, and the address-vs-value distinction that `lea` and `[ ]` make concrete.

2.1.5 Worked problems

CPU time

Question. A program executes 2×10^9 instructions at an average CPI of 1.5 on a 3 GHz processor. How much CPU time does it need?
The instruction count gives $2 \times 10^9 \cdot 1.5 = 3 \times 10^9$ clock cycles. At 3×10^9 cycles per second,

$$T = \frac{3 \times 10^9 \text{ cycles}}{3 \times 10^9 \text{ cycles/s}} = 1 \text{ s}.$$

The units cancel to seconds. This is time spent executing on the processor under the stated clock assumption; time blocked on input or descheduled by the operating system would require additional information.
For 1.2×10^9 instructions at CPI 2 and 2.4 GHz, the same calculation gives 2.4×10^9 cycles and again one second.

Machine comparison

Question. For equal instruction counts, which is faster: a 4 GHz processor with CPI 2 or a 3 GHz processor with CPI 1?

Worked example (continued)

Writing $T = N\,CPI/f$ shows that the common instruction count N cancels in the ratio. The average time per instruction is

$$t_A = \frac{2}{4 \times 10^9} = 0.5 \text{ ns}, \qquad t_B = \frac{1}{3 \times 10^9} \approx 0.333 \text{ ns}.$$

Processor B completes the run in two thirds of A's time, a speedup of $T_A/T_B = 1.5$. Its lower CPI more than compensates for its longer clock period.
As a boundary check, 5 GHz at CPI 2.5 and 2 GHz at CPI 1 both give 0.5 ns per instruction, so equal instruction counts produce equal CPU times.

Locality

Question. Which accesses in a sequential array scan show spatial locality, and which repeated accesses show temporal locality?
Successive reads of `a[0]`, `a[1]` and `a[2]` visit neighbouring addresses. They exhibit spatial locality: a cache line fetched for one element may also contain the following elements. Repeated loads of a memory-resident loop counter visit the same address and exhibit temporal locality. A counter kept entirely in a register generates no data-cache access.
One trace can exhibit both. In `a[8], a[9], a[10], a[8]`, the first three accesses are spatially close, and the final access reuses the location read first. Repeatedly scanning a small array can exploit both forms of locality while its lines remain cached.

Bus width

Question. How many bytes can a 64-bit data bus transfer in one beat before overhead is considered?
Dividing by eight bits per byte gives $64/8 = 8$ bytes per beat. Multiplying back, $8 \times 8 = 64$ bits, checks the conversion. A 128-bit bus similarly carries 16 bytes per beat.
Bandwidth also requires the beat rate. At r beats per second, the ideal bandwidth of the 64-bit bus is $8r$ bytes/s; protocol overhead and idle cycles reduce the useful rate. Bus width alone therefore specifies a transfer quantity, while the beat rate supplies its time scale.

Exercise 2.1.1 — Clock period

What is the period of a 2.5 GHz clock?

Worked solution

Frequency counts cycles per second, so period is its reciprocal: $T = 1/f$. With $f = 2.5 \cdot 10^9$ Hz, $T = (1/2.5) \cdot 10^{-9}$ s $= \boxed{0.4 \text{ ns}}$. Multiplying $2.5 \cdot 10^9$ s^{-1} by $0.4 \cdot 10^{-9}$ s gives one cycle. The conversion uses 1 ns $= 10^{-9}$ s.

Exercise 2.1.2 — Cycle count

Compute cycles for 10^6 instructions at CPI 1.8.

Worked solution

CPI is the average number of cycles per retired instruction. Multiply instruction count by CPI: 10^6 instructions $\cdot$ 1.8 cycles/instruction $= \boxed{1.8 \cdot 10^6 \text{ cycles}}$. The instruction units cancel. Execution time would require one further division by clock frequency; the present question asks only for cycles. An average CPI may be fractional even though each observed run takes an integral number of cycles.

Exercise 2.1.3 — Address space

How many byte locations can a 32-bit address bus name?

Worked solution

A 32-bit address has 2^{32} distinct patterns. Under byte addressing, each pattern names one byte, giving $2^{32} = 4,294,967,296$ byte locations, or 4 GiB of addressable space. The addresses run from zero through $2^{32} - 1$. Addressable space is a naming capacity; the installed or mapped memory may occupy only part of it.

Exercise 2.1.4 — Classify locality

A program accesses `a[i]` and then `a[i+1]`. Classify the locality and explain whether both accesses must hit the same cache line.

Worked solution

Consecutive array elements occupy nearby addresses. Moving from `a[i]` to `a[i+1]` therefore exhibits spatial locality. A cache line brought in for the first access may already contain the second element. This differs from temporal locality, which concerns reusing the same location after an earlier access. If the two elements straddle a line boundary, a second line can still be needed despite the nearby addresses.

Summary

Machine-level reading is disciplined bookkeeping. Write the initial state, identify operand widths, compute effective addresses, apply the instruction semantics and record the new state. High-level meaning can be proposed only after the transitions are secure. This order prevents familiar mistakes such as assigning a permanent source variable to a register or assuming that every memory access has a declared type.

2.1.6 Exercises

Exercise 2.1.5 — Register tracing

Initial state: `rax=0xffffffff00000000`, `rbx=0x1122334455667788`. Trace:

```
mov ax, bx
mov eax, 0x89abcdef
mov al, 0x10
```

Give the final `rax` after each instruction.

Go to the solution →

Exercise 2.1.6 — Endian reconstruction

Memory from address `0x1000` contains bytes `78 56 34 12 ef cd ab 90`. What values are loaded by `mov eax,[0x1000]`, `movzx eax,byte ptr [0x1003]`, and `mov rax,[0x1000]`?

Go to the solution →

Exercise 2.1.7 — Address or value?

Assume `rdi=0x2000` and the readable qword at address `0x2000` contains `0x3000`. State the results of `mov rax,rdi`, `lea rax,[rdi+8]`, `mov rax,[rdi]`, and two successive dereferences.

Go to the solution →

Exercise 2.1.8 — Performance equation

A program executes 8×10^8 instructions. On CPU A, CPI is 1.25 at 2.5 GHz. On CPU B, a different compiler produces 6.5×10^8 instructions with CPI 1.6 at 3.2 GHz. Which run is faster and by what factor?

Go to the solution →

Exercise 2.1.9 — Memory locality

A 64-byte cache line holds sixteen 4-byte integers. Assume a line-aligned array and cold caches. Compare scanning all 4096 elements with reading only elements 0,16,32,.... How many useful integers per fetched line does each pattern obtain?

Go to the solution →

Exercise 2.1.10 — Find the fault

A function receives a pointer in `rdi` and should return the pointed byte as a zero-extended `uint64_t`. It uses `mov al,[rdi]; ret`. Explain the bug and repair it.

Go to the solution →

Exercise 2.1.11 — Design a state table

Write a four-column trace table for `rax`, `rbx`, `rdi` and memory[`0x4000`] for a sequence that swaps `rax`/`rbx`, stores the old `rax` at `0x4000`, and leaves the address `0x4008` in `rdi`. Give one correct instruction sequence.

Go to the solution →

2.1.7 Solutions

Machine State, Registers, and Memory

Solution 2.1.5

After `mov ax,bx`, the low 16 bits become `0x7788`, so `rax=0xffffffff00007788`. Writing `eax` clears the high 32 bits: `0x0000000089abcdef`. Writing `al` changes only the low byte: `0x0000000089abcd10`.

← Return to the exercise

Solution 2.1.6

Little-endian storage places the least significant byte first. The 32-bit load is `0x12345678`; the single byte at `0x1003` is `0x12`; the 64-bit load is `0x90abcdef12345678`.

← Return to the exercise

Solution 2.1.7

`mov rax,rdi` copies the value `0x2000`. `lea rax,[rdi+8]` computes `0x2008`; neither instruction reads memory. `mov rax,[rdi]` reads the qword at `0x2000` and therefore produces `0x3000`.
Following that load with `mov rax,[rax]` dereferences the newly obtained address `0x3000`. If eight readable bytes exist there, the instruction returns their qword value. The question gives no contents for those bytes, so a numeric second result cannot be determined. An inaccessible second address instead causes a fault. Each dereference thus needs its own valid address and memory contents.

← Return to the exercise

Solution 2.1.8

CPU time is instruction count times cycles per instruction divided by cycles per second. A needs $(8 \cdot 10^8)(1.25) = 10^9$ cycles, giving $T_A = 10^9/(2.5 \cdot 10^9) = 0.400$ s. B needs $(6.5 \cdot 10^8)(1.6) = 1.04 \cdot 10^9$ cycles, giving $T_B = 1.04/3.2 = 0.325$ s. B finishes first. Its speedup relative to A is $T_A/T_B = 0.400/0.325 \approx 1.231$. B takes more cycles, yet its shorter cycle period gives the shorter elapsed CPU time.

← Return to the exercise

Solution 2.1.9

The sequential scan uses all 16 integers from each line. The strided scan touches one integer per line, using only 1/16 of each fetched line. Both visit 256 lines under the stated assumptions, but they perform 4096 and 256 element reads respectively.

Worked solution (continued)

Both scans touch the same number of lines in this example. Their useful byte counts per line differ.

← Return to the exercise

Solution 2.1.10

`mov al,[rdi]` replaces only the low eight bits of `rax`. For example, if `rax` initially contains `0x1234000000000000` and the byte is `0x7F`, the function returns `0x123400000000007F` instead of 127.

```
movzx eax, byte ptr [rdi]
ret
```

`movzx` extends the byte with zeros through bit 31 of `eax`. Writing `eax` also clears bits 32–63 of `rax`, so the complete return value lies in $[0, 255]$. Clearing `eax` before the original byte load is another valid repair, although it uses an extra instruction.

← Return to the exercise

Solution 2.1.11

Let the initial register values be `rax`=a, `rbx`=b, `rdi`=d, and let the writable qword at `0x4000` contain m. A sequence using no extra temporary register is:

```
xchg rax, rbx
mov qword ptr [0x4000], rbx
mov edi, 0x4008
```

The four requested state columns evolve as follows; the caption beside each row identifies the completed instruction.

After	`rax`	`rbx`	`rdi`	memory[`0x4000`]
Initial	a	b	d	m
`xchg`	b	a	d	m
Store	b	a	d	a
`mov edi`	b	a	`0x4008`	a

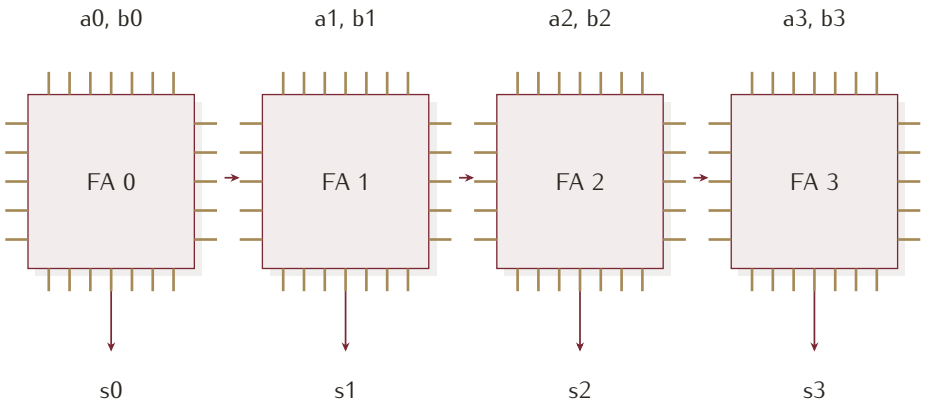
After the swap, the old `rax` is already in `rbx`, so the store uses that register. The final 32-bit write zero-extends `0x4008` into `rdi`. All three instructions preserve flags.

← Return to the exercise

2.2 INTEGER ARITHMETIC AND CONDITION CODES

An addition leaves a value in a register and updates condition codes. A following branch can use those flags to decide whether a loop continues or a check succeeds. To reconstruct that decision, follow the operands, their width and the instruction that most recently wrote each relevant flag. The arithmetic result and the recorded conditions then tell one consistent story.

An arithmetic word is built from bit-level work



Each full adder passes carry to the next higher bit.

Figure 2.11: *The full-adder chain makes carry dependence visible across four positions. Each stage combines its operand bits with the incoming carry and produces a sum bit and an outgoing carry. The chain illustrates arithmetic structure before instruction-level use.*

2.2.1 Width, signedness, and fixed-width arithmetic

Arithmetic traces build on the register and memory model of Chapter 2.1. Three questions determine each instruction: *how many bits* to operate on; is the operation *signed or unsigned*; *which registers are touched implicitly*.

How many bits? Instructions come in four widths (8, 16, 32, 64), affecting both the result (addition mod 2^{32} differs from mod 2^{64}) and the machine-code size.

Signed or unsigned? `div` and `idiv` are different instructions. For multiplication, the low half of the product (mod 2^n) is the same for signed and unsigned, but the high half and the overflow tests differ. Likewise `jg`/`jl` (signed) differ from `ja`/`jb` (unsigned).

Which registers implicitly? `mul rsi` writes `rdx`:`rax`; `div rsi` reads `rdx`:`rax` before dividing. An unprepared `rdx` changes the intended dividend and can cause an incorrect quotient or a divide-error exception.

The two's-complement representation from Part I gives signed and unsigned interpretations to the same bits. For addition, CF records carry beyond the destination width, while OF records a mathematical signed sum outside the signed range. For subtraction, CF records unsigned borrow. These conditions are independent and must be evaluated for the selected width.

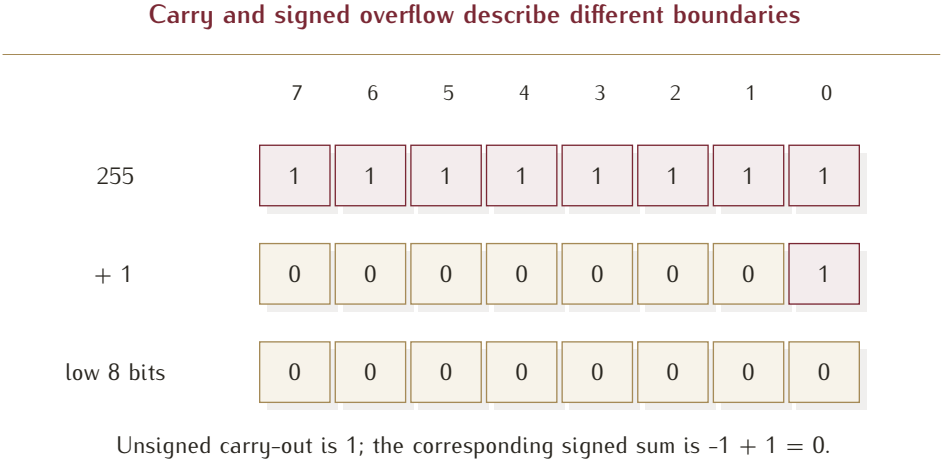


Figure 2.12: *The eight-bit unsigned sum exceeds 255 and produces a carry. The same patterns interpreted as signed operands add to zero without signed overflow. The low result bits alone do not identify which boundary was crossed.*

Worked example

Problem. Add `0xABCDEF` to `rsi`, leaving the result in `rsi`.
Solution.

```
add rsi, 0xabcdef
```

The immediate fits in 24 bits, but `add reg64, imm32` uses a 32-bit immediate (sign-extended). `lea rsi, [rsi+0xabcdef]` produces the same value in `rsi` while preserving flags.

Use `add` when flags are wanted or irrelevant; use `lea` for pointer arithmetic when the existing flags must survive into a later conditional. If the constant is not representable as a sign-extended 32-bit value, load it into a register first (`mov rax, imm64`; `add rsi, rax`).

2.2.2 Three-register expression: $a \cdot b + c$

Worked example

Problem. Compute `rax` $=$ `rdi`$\cdot$`rsi`$+$`rdx` (mod 2^{64}).
Solution.

```
mov rax, rdi
imul rax, rsi; rax = rdi * rsi (mod 2^64)
add rax, rdx
```

`imul reg, reg` is "signed" by name, but the low 64 bits of the product are identical for signed and unsigned; the difference matters only for the high bits and for CF/OF.

Caution

`imul rax, rsi` means "`rax` $=$ `rax`$\cdot$`rsi`", not "`rdi`$\cdot$`rsi`": the destination is also a source. Copy the source into `rax` first, or use the three-operand form `imul rax, rdi, 7`.

Multiplication and division use different circuit organisations

A multiplier forms partial products and reduces them with an adder tree before a final carry-propagate addition. Division units may use digit-recurrence algorithms with dependent steps, or reciprocal-based methods followed by refinement and correction; the choice depends on operand type and implementation. These structures differ in area, pipeline depth and how readily independent operations can overlap.

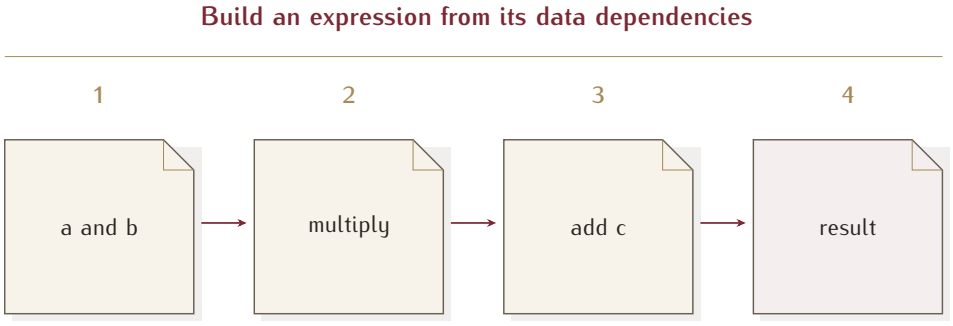


Figure 2.13: *The product must be available before the dependent addition can use it. The implementation also needs a width and overflow contract. The small example fits ordinary signed and unsigned machine widths.*

Multiplication can have high throughput once pipelined even when its latency exceeds addition; division is commonly less frequent and harder to pipeline for one result per cycle. Instruction cost therefore depends on the implemented unit and dependency chain rather than on the mathematical symbol [50, 51].

2.2.3 Multiply, divide, and effective-address arithmetic

Worked example

Problem. Compute `rax` = `rdi` / `rsi` (unsigned integer division), with `rsi` nonzero.
Solution. `div` takes the dividend from `rdx`:`rax` (a virtual 128-bit register), divides by the operand, and puts the quotient in `rax`, the remainder in `rdx`:

```
mov rax, rdi
xor edx, edx; clear the high dividend (zeroes all of rdx)
div rsi; rax = rdi/rsi, rdx = rdi%rsi
```

Omitting `xor edx, edx` makes `div` treat `rdx`:`rax` as a huge 128-bit dividend, giving a wrong quotient or a divide error. For *signed* division use `cqo` (sign-extend `rax` into `rdx`:`rax`) then `idiv`.

Several arithmetic instructions use fixed register pairs rather than naming every operand explicitly. The 64-bit one-operand forms of `imul` and `mul` place the full 128-bit product in `rdx`:`rax`; `idiv` and `div` consume a 128-bit dividend from that pair. Before signed division, `cqo` (written `cqto` in GNU AT&T syntax) sign-extends `rax` into `rdx`:`rax`. When the divisor is known at compile time, an optimising compiler often replaces division with cheaper arithmetic: powers of two permit shifts with the correction required by signed rounding, while other constants can be handled by a precomputed multiply-and-shift sequence. The multiplication idioms below use only the arithmetic already available in the `lea` addressing expression.

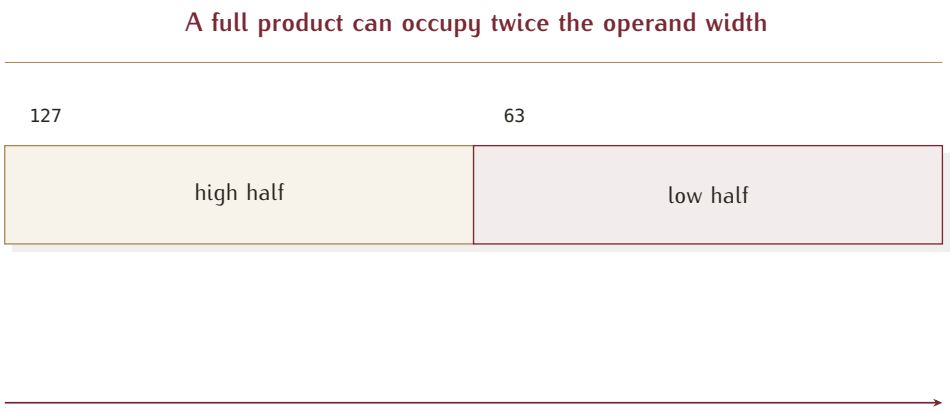


Figure 2.14: *Keeping only the low half discards information whenever the full product needs more than 64 bits. On x86-64, the one-operand unsigned multiply uses RDX:RAX for the full result, with RDX holding the high half.*

Worked example

Problem. Compute `rax` $=$ `rdi` $\cdot 11$.
Solution. The direct form is one instruction: `imul rax, rdi, 11`. Without `imul`, factor $11 = 1 + 2 \cdot 5$:

```
lea rax, [rdi+rdi*4]; rax = rdi*5
lea rax, [rdi+rax*2]; rax = rdi + (rdi*5)*2 = rdi*11
```

`lea [base + base*scale]` with scale $\in \{1, 2, 4, 8\}$ yields implicit $\times 2, 3, 5, 9$. Two chained `lea`s reach $\times 6, 11, 17, 25, \ldots$ The two `lea`s above are data-dependent. Their latency, micro-op count and issue throughput depend on the exact addressing form and microarchitecture, so no fixed cycle count is portable. Compilers use such sequences when the multiplier is a known constant and the target cost model makes them profitable.

Worked example

Problem. Copy the 32-bit `esi` into `rax` with the upper 32 bits zero.
Solution.

```
mov eax, esi; 32-bit write zero-extends to rax
```

For *sign*-extension instead, the 32-bit-write rule is not enough; use `movsxd rax, esi`.

Worked example

Problem. Compute `rcx` $= -$`rdi` (mod 2^{64}).
Solution. In two's complement, $-x = \mathrm{NOT}(x) + 1$, which is `neg`:

```
mov rcx, rdi
neg rcx; rcx = -rdi
```

Edge case: the bit pattern for -2^{63} is unchanged by `neg`, and the instruction sets OF because the mathematical result 2^{63} is not representable in signed 64-bit. This is the architectural result of the x86 instruction. In C, evaluating unary minus on the minimum signed value has undefined behaviour; hardware wraparound must not be used to infer the language rule.

2.2.4 Combining scales: $7a + 3b - 4$

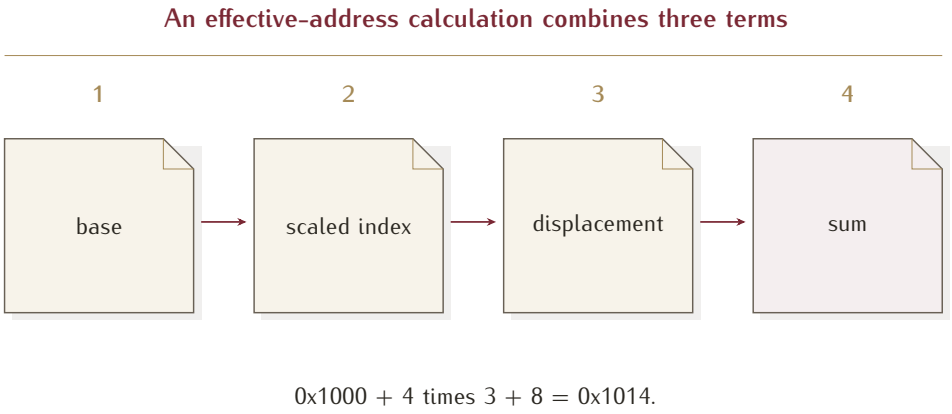


Figure 2.15: *The effective-address expression combines a base, a permitted scale times an index, and a displacement. The example computes a numeric address; whether memory is read depends on the instruction that uses the expression.*

Worked example

Problem. Compute `rax` $= 7\cdot$`rdi`$+3\cdot$`rsi`-4.
Solution.

```
lea rcx, [rdi+rdi*2]; rcx = rdi*3
lea rax, [rdi+rcx*2]; rax = rdi + 2*(rdi*3) = rdi*7
lea rcx, [rsi+rsi*2]; rcx = rsi*3
add rax, rcx; rax = 7*rdi + 3*rsi
sub rax, 4; rax = 7*rdi + 3*rsi - 4
```

This sequence uses five instructions and no explicit multiplication. A version with two `imul` instructions is another valid implementation. Compare dependency depth, code size and execution resources on the target processor before choosing between them; instruction count alone does not establish a performance advantage.

Worked example

Problem. Compute `rax` = `rdi`/`rsi` and `rcx` = `rdi` % `rsi` (unsigned).
Solution.

```
mov rax, rdi
xor edx, edx
div rsi; rax = quotient, rdx = remainder
mov rcx, rdx; rcx = remainder
```

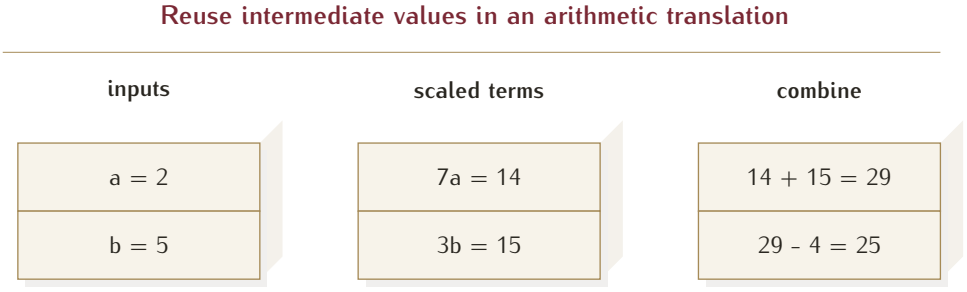


Figure 2.16: *The snapshots expose each intermediate quantity. A register allocation can reuse storage after an old value is no longer needed, provided it preserves operands that later instructions still require.*

Caution

The architecture fixes `div`'s quotient in `rax` and remainder in `rdx`. If a task requires different destinations, divide first and then reposition the results with `xchg`/`mov`. Moving the `xor` after `div` would erase the remainder.

2.2.5 Reading the instruction contract

An instruction-reference entry identifies operand forms, implicit registers, defined flags and possible faults. For 64-bit division, a zero divisor or a quotient outside the destination range raises `#DE`; after a successful `div` or `idiv`, arithmetic condition flags are

undefined. Signed division of the minimum 64-bit integer by -1 therefore faults because the quotient would be 2^{63}.

The one-operand `imul` produces a double-width signed product. Its two- and three-operand forms retain the low destination-width bits. CF and OF indicate whether the full signed product fits that width; the other arithmetic flags are undefined. For unsigned overflow of a truncated product, those signed flags are insufficient: inspect the high half of an unsigned full product or establish a bound before multiplication [26, 27].

Signed subtraction overflow without unsigned borrow

A 64-bit subtraction computes `0x8000000000000000`-1, leaving `0x7FFFFFFFFFFFFFFF`. Under unsigned interpretation, $2^{63} \geqslant 1$, so no borrow is needed and CF=0. Under signed interpretation, $-2^{63} - 1$ lies below the minimum, so OF=1. The stored result has SF=0 and ZF=0. This example demonstrates why CF and OF cannot be used interchangeably.

Instruction cost and physical work are different

Replacing multiplication by shifts and additions appears to reduce operation complexity.

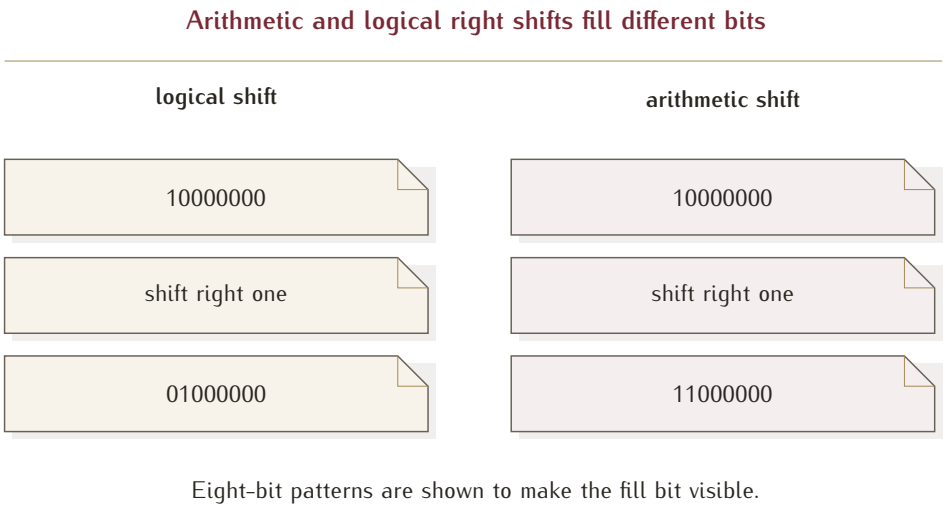


Figure 2.17: *A logical right shift introduces zero at the top. An arithmetic right shift replicates the sign bit. For this signed negative operand the arithmetic shift preserves the direction of the signed scale change.*

Energy depends on which transistors and wires switch, whether the multiply unit is already pipelined, how many instructions are fetched and decoded, and whether extra intermediate values travel through the register file and bypass network. A longer sequence of simple instructions can toggle more total capacitance than one specialised instruction.

Strength reduction should be judged from generated code and measured on the target processor. Algebraic simplicity, latency, throughput and energy are related but not interchangeable [50, 28].

2.2.6 Condition codes and width

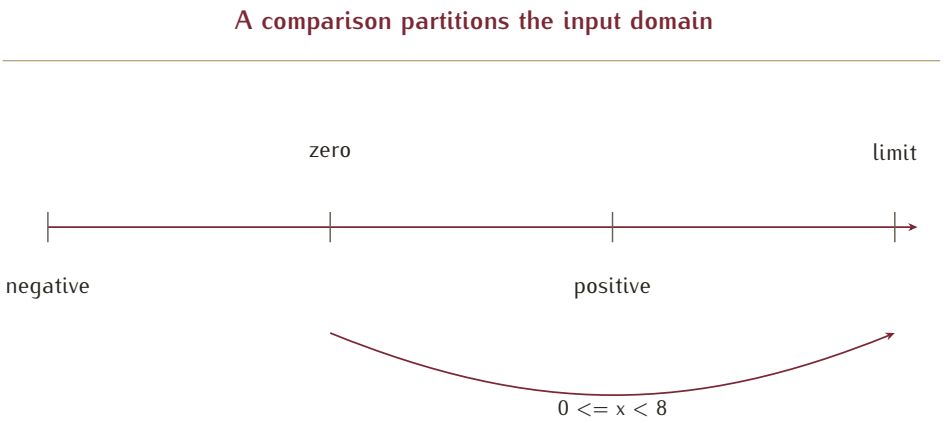


Figure 2.18: *The two bounds establish a half-open interval. Assembly may implement the checks separately or combine them under a justified unsigned transformation. The endpoints determine which branch must be taken.*

Each condition bit reflects the most recent instruction that defined that bit; another instruction may preserve it or leave it undefined. For ordinary addition and subtraction, the four principal conditions are:

- ZF: the result is zero;
- SF: the most significant result bit is one;
- CF: an unsigned carry or borrow occurred;
- OF: the signed result fell outside the representable range.

The same bit pattern can be interpreted as signed or unsigned, so no single flag means “less than”. Unsigned comparisons use CF and ZF; signed comparisons combine SF, OF and ZF. In particular, unsigned less-than tests CF=1, while signed less-than tests SF $\neq$ OF. Equality tests ZF=1 under either interpretation.

Two further arithmetic flags describe smaller pieces of the result. PF is set when the low result byte contains an even number of one bits. AF records carry or borrow across the boundary between bits 3 and 4. They have specialised uses; ordinary integer order comparisons rely on CF, OF, SF and ZF [26, 27].

Worked example: one subtraction, two interpretations

Let `eax=0xffffffff` and execute `cmp eax,1`. The hardware computes `0xffffffff-1=0xfffffffe` for flag purposes. Interpreted unsigned, $2^{32}-1 > 1$, so `ja` is taken. Interpreted signed, $-1 < 1$, so `jl` is taken. The operands are unchanged; the branch mnemonic selects the interpretation.

Caution

Do not infer signedness from `add`, `sub` or `cmp`; these instructions perform the same bit-level operation for both interpretations. Signedness appears in extension instructions, division, high-half multiplication and the choice of conditional jump.

Addition, subtraction and the low product half are computed modulo 2^w for a w-bit destination. A result that fits in 64 bits may overflow at 32 bits. Conversely, writing a 32-bit result to a general-purpose register also clears the high half, which can be useful or destructive depending on intent.

Worked example: carry without signed overflow

For 8-bit arithmetic, `0xff+1=0x00`. CF is set because an unsigned carry left the byte. OF is clear because $-1+1=0$ is a valid signed result. By contrast, `0x7f+1=0x80` clears CF but sets OF: $127+1$ cannot be represented as an 8-bit signed integer.

Read the flags required by the chosen condition

	relation	interpretation	condition
below	a < b	unsigned	CF = 1
less	a < b	signed	SF differs OF
equal	a = b	either	ZF = 1

These conditions follow a comparison implemented as a - b.

Figure 2.19: *CMP sets flags from a subtraction without storing its result. The subsequent branch or set instruction chooses the signed or unsigned interpretation. The flag formula must match the subtraction operand order.*

Worked example: signed division

Compute $-37/5$ using 64-bit signed division.

```
mov rax, -37
cqo; sign-extend rax into rdx:rax
mov rcx, 5
idiv rcx; rax = -7, rdx = -2
```

The quotient truncates toward zero. A nonzero remainder has the same sign as the dividend; here $-37 = (-7) \cdot 5 + (-2)$ and $|-2| < 5$.

Worked example: unsigned division needs a clean high half

```
mov rax, 100
xor edx, edx; high 64 bits of dividend = 0
mov rcx, 7
div rcx; rax = 14, rdx = 2
```

Omitting the clear would divide the 128-bit number `rdx:rax`, not 100. A nonzero high half can also make the quotient too large, raising a divide exception.

The effective-address form of `lea` combines a base, a scaled index and a displacement while preserving flags.

Worked example: derive $13x + 5$

```
lea rax, [rdi + rdi*4]; 5x
lea rax, [rax + rdi*8]; 13x
add rax, 5
```

The first effective address is $x + 4x$; the second is $5x + 8x$. A compiler may choose a different sequence based on latency, code size and available execution ports.

2.2.7 A complete arithmetic translation

Consider a C function with mixed signedness:

```
long score(long a, unsigned long b, long c) {
 long t = a * 7 + c;
 return (b < 10) ? t: -t;
}
```

One plausible translation is:

```
lea rax, [rdi + rdi*2]; 3a
lea rax, [rdi + rax*2]; 7a
add rax, rdx; t
mov rcx, rax
neg rcx; -t
cmp rsi, 10; unsigned b
cmovae rax, rcx; if b >= 10, choose -t
ret
```

The comparison must be unsigned because `b` is unsigned. The translation preserves the C result for every input where the source operations are defined. The product and sum must fit `long`; when `b` $\geqslant$ 10, negating `t` must also fit. The hardware may compute the unused negation speculatively in the other case because integer `neg` itself does not trap on signed overflow.

Integer overflow can become a memory-safety bug when an allocation size or bounds check is computed at insufficient width. For unsigned operands, a check such as `if (n * 8 <= size)` can accept an excessive length if `n*8` wraps before comparison. Low-level review must track widths through every instruction, especially after narrowing casts and 32-bit register writes.

Laboratory 9: Flags and comparisons

Write System V x86-64 fragments returning signed `a<b`, unsigned `a<b`, and signed `a<0` as integer 0/1. Then calculate CF and OF for 64-bit maximum signed plus one and unsigned maximum plus one.
Worked solution

Laboratory 9: solution

For signed less-than use `cmp rdi,rsi; setl al; movzx eax,al; ret`. For unsigned less-than replace `setl` by `setb`. For a signed comparison with zero use `test rdi,rdi; sets al; movzx eax,al; ret`. Zero extension makes the entire integer return a clean 0 or 1. Adding one to `0x7FFFFFFFFFFFFFFF` produces `0x8000000000000000`: CF=0 and OF=1. Adding one to `0xFFFFFFFFFFFFFFFF` produces zero: CF=1 and OF=0. GDB's `info registers eflags` after an actual `add` can inspect the same hardware flags. A compiler may choose different equivalent instructions; its condition must still match the operand interpretation.
Return to the laboratory

From mechanical carry to flags, vectors and checked arithmetic

Timeline

Year	Machine, standard or event	Architectural significance
1642	**Pascaline carry train**	Mechanical arithmetic makes carry propagation visible.
1964	**System/360 condition code**	Arithmetic outcomes become architectural branch inputs.
1978	**8086 FLAGS**	Carry, zero, sign and overflow are exposed in x86 program state.

Arithmetic hardware inherited its hardest elementary problem from mechanical calculators: carry. In Blaise Pascal's 1642 calculating machine, a wheel passing from nine to zero had to advance the next decimal wheel. Mechanical carry chains were slow and delicate. Electronic binary adders replaced wheels with switching networks, but the dependency remained: the high bit of a sum may depend on a carry generated many positions below. The history of integer arithmetic is therefore a history of representations, carry organisation and the information preserved about a result [26, 27, 50].

ENIAC, completed in 1945 and publicly unveiled in February 1946, used decimal accumulators and represented each digit through a ten-state ring. Addition propagated decimal carries electronically. Binary machines simplified each digit to two states and used half-adders and full-adders. A ripple-carry adder connects the carry output of one bit to the next; it is compact but delay grows with width. Carry-lookahead derives generate and propagate signals so groups can determine carries in parallel. Larger processors use hierarchical, prefix or carry-select structures chosen for timing, area and power. The architectural instruction `ADD` hides these circuits, but the distinction between latency and width comes from them [26, 27, 50].

Signed representation also shaped hardware. Early machines used sign-magnitude or ones' complement because those encodings resembled human notation or simplified certain circuits. They created two zero encodings and complicated addition. Two's complement lets the same fixed-width adder implement signed and unsigned addition; subtraction becomes addition of an inverted operand plus one. The Manchester Baby of 1948 and Manchester Mark 1 of 1949 used two's-complement integers. IBM System/360, announced in 1964, used two's complement for binary integers while also supporting decimal arithmetic for commercial workloads. Two's complement became dominant because its algebra aligns naturally with arithmetic modulo 2^w [90, 63, 109].

Condition codes preserved selected facts about an operation. System/360 in 1964 placed a two-bit condition code in the Program Status Word, while the Intel 8086 of 1978 exposed separate carry, zero, sign and overflow flags in its FLAGS register. Different architectures made different choices, but familiar flags include zero, sign, carry and signed overflow. Carry records an unsigned result outside the fixed-width range; overflow records a signed result outside the two's-complement range.

Multiplication and division required more specialised algorithms. Andrew Booth described his signed multiplication algorithm in 1951, reducing work for runs of equal bits. Hardware multipliers later used carry-save trees and techniques associated with Wallace or Dadda reduction. Division used restoring and non-restoring methods, SRT digit recurrence and, in some processors, reciprocal approximations refined by multiplication. In the 64-bit x86 forms, one-operand `MUL` and `IMUL` produce a double-width result in the RDX:RAX pair; `DIV` and `IDIV` divide a double-width dividend and fault when the divisor is zero or the quotient does not fit [26, 27].

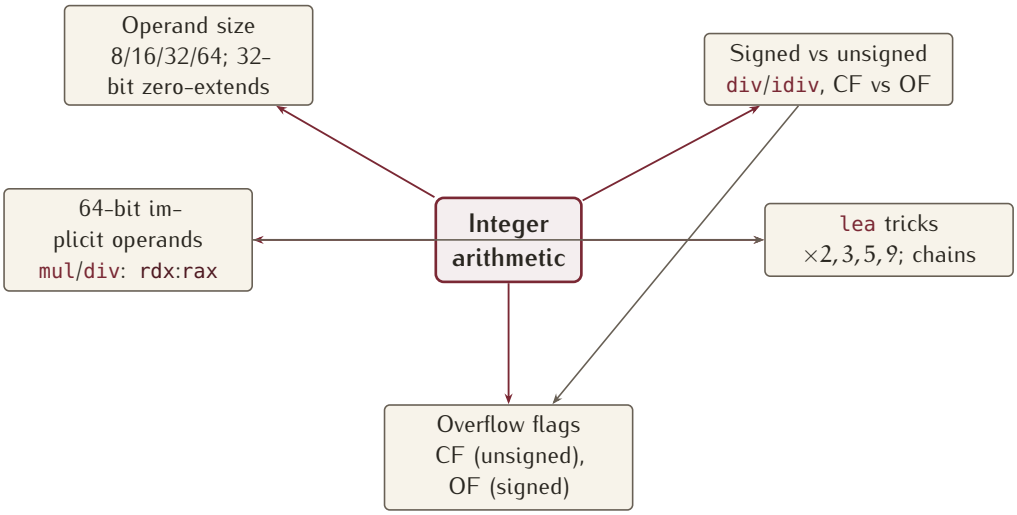
Architectures also discovered that address arithmetic and ordinary arithmetic benefit from different flag behaviour. x86 `LEA` performs effective-address calculation without changing condition codes, so compilers use it for expressions such as $3x + y$. Increment and decrement historically preserve carry, an irregularity inherited from multi-precision loops. Modern compilers therefore choose instructions partly by dependency effects on flags. On out-of-order cores, flags may be renamed internally, but an instruction that consumes them still creates an architectural dependency [28, 29].

Packed arithmetic changed the scale. Intel introduced MMX in 1997 for small integer elements packed into 64 bits. SSE2, introduced with the Pentium 4 in 2000, added integer operations in 128-bit XMM registers; AVX2, first shipped in 2013, widened common integer vectors to 256 bits; AVX-512, first shipped in selected Xeon Phi products in 2016, added wider operations and mask registers on selected processors. One instruction can add many byte, word, doubleword or quadword lanes. Some vector instructions wrap modulo the lane width, while saturating instructions clamp to a minimum or maximum. The programmer must choose the intended arithmetic model explicitly [26, 27, 50].

Checked arithmetic has moved partly into languages and compilers. The 1989 ANSI C standard and its successors treated unsigned arithmetic as modulo 2^w and signed overflow as undefined behaviour; C23, published as ISO/IEC 9899:2024, standardises two's-complement representation but does not make ordinary signed overflow wrap. Compilers expose built-ins that return an overflow indication and can map them to flags or wider operations. Security-sensitive size calculations use these checks to prevent allocation and bounds errors. Hardware carry chains therefore reappear at the software level as APIs for proving that a computation stayed within range [3].

Current CPUs may fuse comparisons with conditional branches, execute several integer operations per cycle and use dedicated vector units, yet the programmer-visible distinctions remain. Carry belongs to unsigned arithmetic, overflow to signed arithmetic, zero and sign to the truncated result, and division exceptions to the architectural quotient rule. Exact flag definitions must be taken from the Intel or AMD manual because instructions differ in which flags they define, preserve or leave undefined [26, 27, 50].

Concept map: integer arithmetic



Reading the map. Every arithmetic instruction is fixed by three choices: width, signedness, and which registers it implicitly uses. Signedness selects `div` vs `idiv` and which overflow flag (CF/OF) is meaningful; the implicit `rdx`:`rax` pair drives multiply/divide; and `lea`, by reusing the addressing arithmetic, replaces small multiplications.

2.2.8 Worked problems

Linear combination

Question. Write branch-free x86-64 code returning $5x + 3y$, with x in `rdi` and y in `rsi`.

The scale factors of `lea` provide $5x = x + 4x$ and then add $2y$. One further addition supplies the remaining y:

```
lea rax, [rdi+rdi*4]  # 5x
lea rax, [rax+rsi*2]  # 5x + 2y
add rax, rsi          # 5x + 3y
ret
```

All results have modulo-2^{64} semantics. With $x = 2$ and $y = 3$, `rax` progresses through 10, 16 and 19, agreeing with $5(2) + 3(3)$.

For $3x + 5y$, use `lea rax,[rdi+rdi*2]` and `lea rcx,[rsi+rsi*4]`, then `add rax,rcx`. The same method decomposes each coefficient into a base term and a supported scale.

Unsigned less-than

Question. Return 1 exactly when `rdi` is less than `rsi` under unsigned interpretation.

```
cmp rdi, rsi
setb al
movzx eax, al
ret
```

`cmp` sets flags from the subtraction `rdi`−`rsi` while preserving both operands. An unsigned borrow occurs exactly when the first operand is smaller. `setb` reads CF and writes that condition as a byte; `movzx` makes it a full-width 0 or 1.
With operands 0 and $2^{64}-1$, subtraction needs a borrow and the result is 1. With equal operands CF=0, so the result is 0. Replacing `setb` with `setae` computes the complementary unsigned condition, greater-than-or-equal.

Maximum

Question. Return the unsigned maximum of `rdi` and `rsi` without a branch.

```
cmp rdi, rsi
mov rax, rdi
cmovb rax, rsi
ret
```

The comparison sets CF when `rdi` is smaller. `mov` places the first operand in `rax` and preserves those flags; `cmovb` replaces it with the second operand precisely when CF=1.
For $(4,9)$ the move selects 9. For $(9,4)$ the provisional 9 remains. Equal operands leave their common value unchanged. The unsigned minimum follows the same construction with `cmova`, which replaces the provisional first value when it is greater than the second.

Absolute difference

An assembly routine receives two unsigned integers in `rdi` and `rsi`. For the test case, `rdi`=13 and `rsi`=29.
Question. Write a complete x86-64 sequence that returns $|13-29|$ in `rax` without allowing unsigned wraparound to become the answer.
Reasoning.

1. **Compare before subtracting.** The comparison shows that 13 is below 29 under unsigned interpretation.
2. **Choose the safe order.** The larger value, 29, must be the minuend and 13 the subtrahend.

Worked example (continued)

3. **Encode both paths.**

```
        cmp rdi, rsi
        jae .rdi_is_larger
        mov rax, rsi
        sub rax, rdi
        ret
    .rdi_is_larger:
        mov rax, rdi
        sub rax, rsi
        ret
```

For the given inputs the first path computes $29 - 13 = 16$.

Conclusion. The routine returns 16 in `rax`.
Unsigned subtraction is modular. If the smaller operand were placed first, the hardware would wrap through 2^{64} and produce a huge value rather than a distance. Comparing first guarantees a nonnegative mathematical subtraction on either path.
Swap the inputs to 29 and 13. The other path executes, but the result remains 16. This symmetry is a strong check for an absolute-difference routine.
With inputs 50 and 8, the greater-or-equal path computes $50 - 8 = 42$; no other part of the algorithm changes.

Exercise 2.2.1 — Copy

Return the second argument.

Worked solution

Under the System V integer-argument convention used here, the second argument arrives in `rsi` and the return value belongs in `rax`. Copy the complete 64-bit value:

```
mov rax, rsi
ret
```

This is a callable-function form: `ret` requires a valid return address on the stack. For the calculation alone, trace the instructions up to the return; to execute the function, call it through a valid ABI-conforming caller as shown in Appendix C. For an input `rsi=0x123456789ABCDEF0`, `rax` must contain the identical 64-bit pattern.

Exercise 2.2.2 — Signed division

What prepares the high half before idiv?

Worked solution

For a 64-bit signed division, `idiv` reads the double-width dividend from `rdx:rax`. When the intended dividend is the signed value already in `rax`, `cqo` copies its sign into every bit of `rdx`. A positive dividend yields `rdx`=0; a negative dividend yields all ones. This prepares the correct sign-extended 128-bit representation. Clearing `rdx` would change a negative dividend into a different large value. The divisor must be nonzero and the quotient must fit.

Exercise 2.2.3 — Equality flag

Which setcc follows cmp for equality?

Worked solution

After `cmp`, equality means that the subtraction used for comparison produced zero, so ZF is set. `sete` and `setz` are synonymous and store 1 when ZF=1, otherwise 0. For a full-width Boolean result, use `sete al` followed by `movzx eax,al`. A byte-only write to `al` leaves the other bits of `rax` unchanged, so returning `rax` directly after only `sete` could expose stale upper bits.

Exercise 2.2.4 — Shift

What does shl rax,3 do when no high bits are lost?

Worked solution

Shifting left by three positions moves each set bit from weight 2^i to 2^{i+3}, multiplying its contribution by $2^3 = 8$. With no high bits discarded, `shl rax,3` therefore computes exactly $8x$. For $x = 7$, the result is 56. In general the 64-bit result is $8x \bmod 2^{64}$; the no-loss precondition is $x \leqslant (2^{64} - 1)/8$ for unsigned values.

Summary

A reliable arithmetic trace records width before value. Compute the low w bits, then evaluate the flags from their definitions. When reading compiler output, ask which later instruction consumes those flags. When optimising, distinguish an algebraic identity over unbounded integers from one valid for the finite machine representation and language semantics.

2.2.9 Exercises

Exercise 2.2.5 — Carry and overflow

For 8-bit addition, determine result, CF and OF for: (a) `0xff+1`; (b) `0x7f+1`; (c) `0x80+0x80`.

Go to the solution →

Exercise 2.2.6 — Signed versus unsigned branch

After `cmp eax,ebx`, `eax=0xffffffff` and `ebx=1`. Which of `ja`, `jb`, `jg`, `jl` are taken?

Go to the solution →

Exercise 2.2.7 — Prepare a dividend

Write instructions to divide unsigned `rdi` by 10, returning quotient in `rax` and remainder in `rdx`.

Go to the solution →

Exercise 2.2.8 — Exact signed division

Trace `mov rax,-17; cqo; mov rcx,4; idiv rcx`. State quotient and remainder and verify the identity.

Go to the solution →

Exercise 2.2.9 — LEA synthesis

Construct $\texttt{rax} = 11 \cdot \texttt{rdi} + 3 \cdot \texttt{rsi} - 7$ without `imul`. A scratch register is available. Use no more than four arithmetic/address instructions.

Go to the solution →

Exercise 2.2.10 — Width bug in a bounds check

A length in `rdi` is multiplied by 8 with `imul edi,edi,8`, then compared as 64-bit with `rsi`. Explain why very large lengths can pass an incorrect check.

Go to the solution →

Exercise 2.2.11 — Reconstruct an expression

Translate the following into algebra and then C-like pseudocode:

```
lea rax, [rdi+rdi*4]
lea rax, [rsi+rax*2]
sub rax, rdx
```

Go to the solution →

2.2.10 Solutions

Integer Arithmetic and Condition Codes

Solution 2.2.5

First compute the full unsigned sum and retain its low byte. CF records whether the sum reaches $2^8 = 256$. Then interpret the input bytes as signed values and test their mathematical sum against $[-128, 127]$.

Case	Unsigned sum	Low byte	CF	OF
(a) `0xFF+1`	256	`0x00`	1	0
(b) `0x7F+1`	128	`0x80`	0	1
(c) `0x80+0x80`	256	`0x00`	1	1

For (a), the signed sum is $-1 + 1 = 0$, which fits. For (b), $127 + 1 = 128$ exceeds the signed maximum. For (c), $-128 + (-128) = -256$ falls below the signed minimum. Thus the last two cases set OF even though their carry outcomes differ.

← Return to the exercise

Solution 2.2.6

`cmp` computes `0xFFFFFFFF`-1 =`0xFFFFFFFE` for its flags. The operands stay unchanged. There is no unsigned borrow, so CF=0; the result is nonzero, so ZF=0. The result sign bit is one, so SF=1. The signed subtraction $-1 - 1 = -2$ fits in 32 bits, so OF=0.

`ja` tests CF=0 and ZF=0 and is taken; `jb` tests CF=1 and is not taken. `jg` requires ZF=0 and SF=OF, so it is not taken; `jl` requires SF≠OF and is taken. These outcomes agree with $2^{32} - 1 > 1$ unsigned and $-1 < 1$ signed.

← Return to the exercise

Solution 2.2.7

```
mov rax, rdi
xor edx, edx
mov ecx, 10
div rcx
```

The first two instructions construct the unsigned 128-bit dividend $0 :$ `rdi` in `rdx:rax`. Writing `ecx` zero-extends 10 into `rcx`, making a nonzero 64-bit divisor. Division leaves $q = \lfloor n/10 \rfloor$ in `rax` and $r = n - 10q$ in `rdx`, with $0 \leqslant r < 10$.

Worked solution (continued)

For $n = 123$, the outputs are $q = 12$ and $r = 3$, and $123 = 10(12) + 3$. For any 64-bit unsigned input the quotient fits in 64 bits, so the prepared operation cannot incur a quotient-overflow fault.

← Return to the exercise

Solution 2.2.8

`cqo` sign-extends the negative value in `rax` into the full signed dividend `rdx:rax`. Thus `idiv rcx` divides -17 by $+4$. Truncation toward zero gives quotient -4 in `rax`; the remainder is $-17-(-4)(4) = -1$ in `rdx`. The identity $-17 = (-4)\cdot 4 + (-1)$ holds, the remainder has the dividend's sign, and its magnitude 1 is smaller than the divisor's magnitude 4.

← Return to the exercise

Solution 2.2.9

One solution uses `rcx` as scratch:

```
lea rax, [rdi+rdi*4]; 5a
lea rax, [rdi+rax*2]; 11a
lea rcx, [rsi+rsi*2]; 3b
lea rax, [rax+rcx-7]; 11a+3b-7
```

The final displacement is part of the address calculation, so the constant does not require a separate subtraction. This sequence also illustrates why the availability of a scratch register is part of the optimisation problem.

← Return to the exercise

Solution 2.2.10

The multiplication reads only `edi`, computes low 32-bit product bits and clears the high half of `rdi`. For example, an original length 2^{32} becomes zero before multiplication, so the later 64-bit comparison can accept zero bytes for a huge logical request.
For unsigned length n and byte capacity S, test $n \leqslant \lfloor S/8 \rfloor$ before computing the byte count:

```
mov rax, rsi
shr rax, 3
cmp rdi, rax
ja .reject
lea rax, [rdi*8]  # accepted byte count
```

Worked solution (continued)

The comparison proves both that multiplication by eight fits in 64 bits and that the result does not exceed the supplied capacity. A 64-bit multiply alone would still need an unsigned-overflow check; the signed overflow flags from truncated `imul` do not generally test unsigned product overflow.

← Return to the exercise

Solution 2.2.11

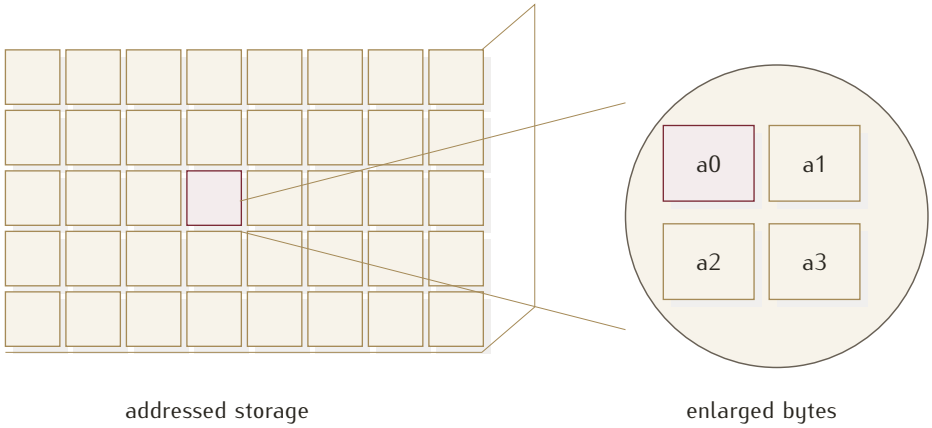
First `rax`=5a. Then `rax`=b+10a. Finally `rax`=10a+b-c. On the stated platform, declare `a`, `b` and `c` as `uint64_t`; then `return 10*a + b - c;` has the required modulo-2^{64} semantics. Signed C operands would require separate range assumptions.

← Return to the exercise

2.3 ADDRESSES, ARRAYS, AND DATA LAYOUT

The expression `a[i]` names an element; the load instruction needs a byte address and a width. Between the two lies a calculation involving the base address, element size and index. Records add field offsets, and linked objects add another memory read before the next address is known. Following these calculations makes compiled data layout visible and exposes the bounds and lifetime conditions required for each access.

Zooming from an addressable region to individual bytes



The magnified region has a byte order and a finite extent.

Figure 2.20: *The magnified cells represent a finite region within a larger addressed store. Element size, alignment and layout determine which byte positions belong to an object. The enlargement is conceptual rather than a scale drawing of memory hardware.*

2.3.1 Addresses, values and access widths

In a byte-addressed model, address a names one byte. A qword load beginning at a reads eight consecutive bytes. The address identifies the beginning of that interval; the instruction determines its width and how the loaded bits enter a register. All examples in this chapter use Intel assembly syntax and ordinary x86-64 user-space memory.

Reading a byte and reading a qword

Let `rdi=0x1000`, with bytes `80 01 00 00 00 00 00 00` beginning there. The instructions below are independent alternatives.

```
movzx eax, byte ptr [rdi]
movsx rax, byte ptr [rdi]
mov rax, qword ptr [rdi]
```

The first produces 128: it reads one byte and zero-extends it, and the write to `eax` clears the upper half of `rax`. The second interprets that byte as signed and produces −128, represented by `0xFFFFFFFFFFFFFF80`. The third reads eight bytes and produces `0x180`=384 under little-endian interpretation. Each result follows from the same starting address and a different access or extension rule.

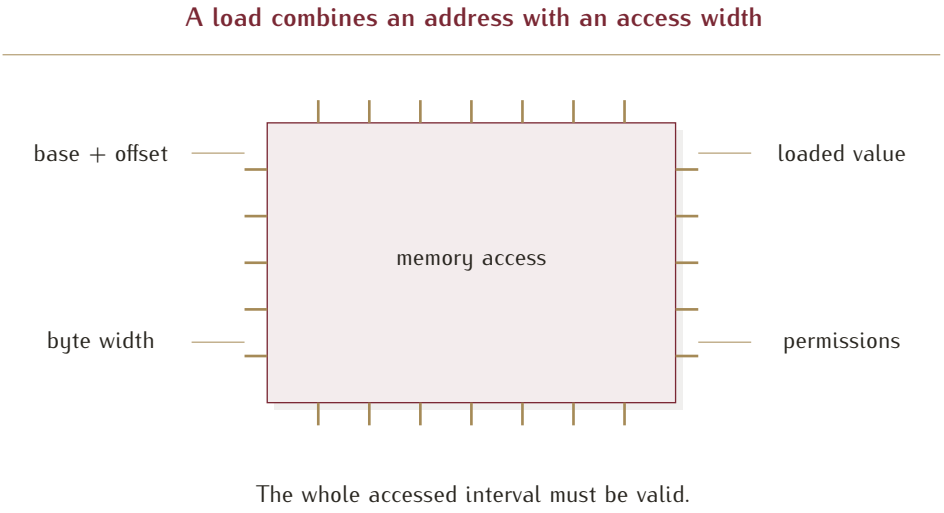


Figure 2.21: *An address alone is insufficient to describe a load. Its width determines the complete interval, and the access must satisfy the relevant mapping and object rules. A wider load can cross a boundary that a one-byte load would not.*

A memory operand with no explicit size can obtain its width from the other operand: `mov rax,[rdi]` reads eight bytes because the destination is 64 bits. A store of an immediate often needs a size annotation, as in `mov byte ptr [rdi],0`. Access width also determines how many bytes must be readable or writable.

2.3.2 Forming an effective address

The common x86-64 memory form is

$$[\text{base} + \text{index} \cdot \text{scale} + \text{displacement}], \qquad \text{scale} \in \{1, 2, 4, 8\}.$$

The base and index are register values; the displacement is a constant encoded in the instruction. The address calculation is identical whether the operand is used for a load or a store. In `lea`, the calculated value is written to a register without reading memory at that address.

One address expression, two instruction effects

Take `rdi=0x1000`, `rsi=3` and an eight-byte value 77 at address `0x1038`.

```
lea rax, [rdi + rsi*8 + 32]
mov rdx, [rdi + rsi*8 + 32]
```

The effective address is $\texttt{0x1000} + 3 \cdot 8 + 32 = \texttt{0x1038}$. The first instruction places `0x1038` in `rax`. The second places 77 in `rdx`. The load requires a readable eight-byte interval; `lea` performs the arithmetic alone.

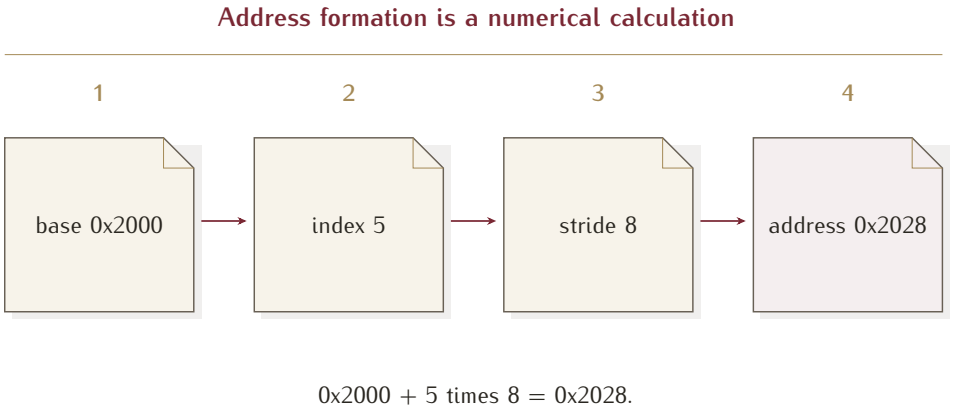


Figure 2.22: *The stride converts an element index into a byte displacement. The final arithmetic is necessary for addressing, while object bounds and lifetime establish whether an access through that address is valid.*

The index cannot be `rsp` in this addressing form. A separate encoding-extension bit allows `r12` to be an index. RIP-relative addressing and several special absolute-address forms use other encodings. These details matter when decoding instruction bytes; the ordinary base/index formula is sufficient for the array calculations that follow [26, 27].

2.3.3 Contiguous arrays and matrices

For an array starting at byte address A_0, with elements of size s, element i starts at

$$A_i = A_0 + is.$$

An eight-byte array therefore uses scale 8, a four-byte array scale 4, and a byte array scale 1. The calculation assumes a valid element index. An address expression by itself performs no bounds check.

An element size outside the scale field

Records occupy 24 bytes each, their base is in `rdi`, and the index is in `rsi`. To read an eight-byte field at offset 8:

```
lea rax, [rsi + rsi*2]
mov rax, [rdi + rax*8 + 8]
```

The first instruction computes $3i$; the memory operand adds $8(3i) + 8$ to the base. For $i = 2$ and base `0x2000`, the field is at $\texttt{0x2000} + 48 + 8 = \texttt{0x2038}$. A scale of 24 is obtained through two arithmetic steps.

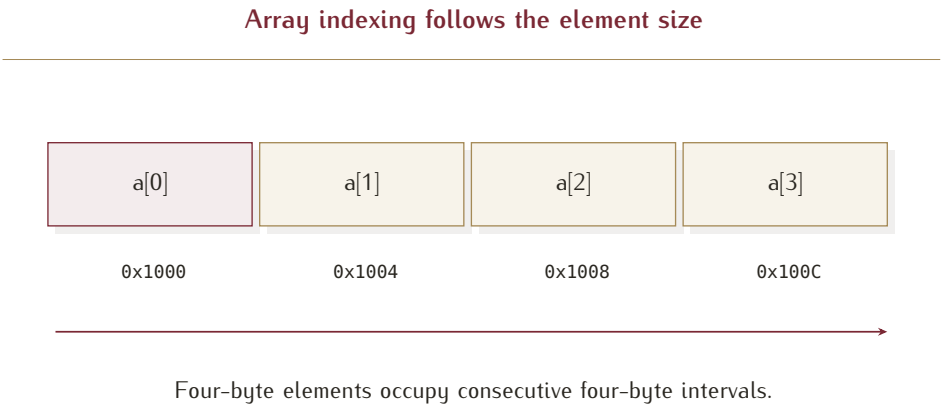


Figure 2.23: *The element address is base plus index times four in this example. Adjacent labels identify element starts, not adjacent byte addresses. The valid index range depends on the array length.*

A row-major matrix `T a[R][C]` stores each row's C elements consecutively. To reach row i, skip iC elements; to reach its column j, skip another j. Thus

$$\text{addr}(a[i][j]) = A_0 + (iC + j)\,\text{sizeof}(T).$$

Both indices must be within their respective dimensions. Flattening the arithmetic does not remove the source object's row boundaries.

A five-column matrix

Let `rdi` point to `int a[4][5]`, with four-byte integers, row index in `rsi` and column index in `rdx`. A signed 64-bit result can be obtained with:

```
lea rax, [rsi + rsi*4]
add rax, rdx
movsxd rax, dword ptr [rdi + rax*4]
```

At $(i, j) = (2, 3)$, the element index is $2 \cdot 5 + 3 = 13$ and the byte displacement is 52. The sign-extending load preserves the value of a negative `int`. A load into `eax` would instead zero-extend its 32-bit representation.

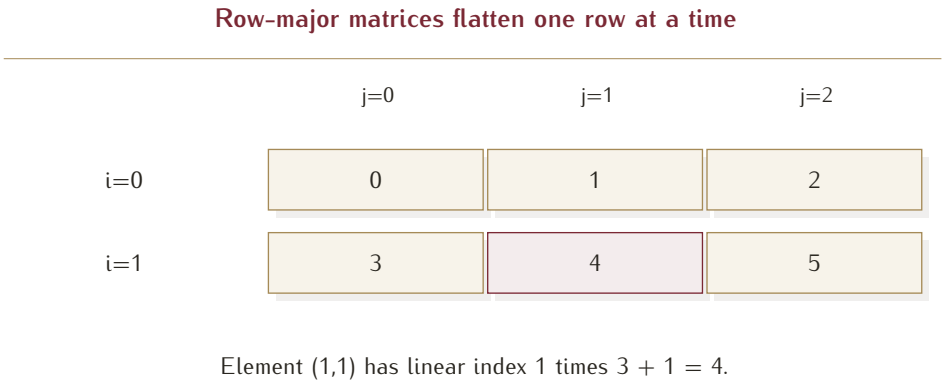


Figure 2.24: *The cells show linear element indices in a two-by-three row-major array. Multiplying the row by the number of columns skips complete rows; adding the column selects the element within its row.*

An array of pointers has a different layout. For `int *rows[R]`, first read the pointer in slot `i`, then access element `j` through that pointer. The rows can occupy unrelated allocations and can have different lengths. With eight-byte pointers and four-byte integers:

```
mov rcx, [rdi + rsi*8]
movsxd rax, dword ptr [rcx + rdx*4]
```

The second address depends on the first load's value. Replacing this pair with one flat matrix formula would assume a contiguity guarantee that the pointer array does not provide.

2.3.4 Structures, alignment and padding

An ABI specifies the sizes and alignments used to lay out objects on a platform. A member is placed after earlier members, with padding when required for its alignment. The complete structure is rounded to its aggregate alignment so that every element of

an array of structures can satisfy the same rules. The following examples assume the System V AMD64 ABI: `char`, `short`, `int` and pointers have sizes and natural alignments 1, 2, 4 and 8 bytes respectively [30].

Deriving a complete record layout

```
struct record {
    char tag;
    int count;
    short code;
    void *next;
};
```

Member	Offset	Size	Placement reason
`tag`	0	1	First available byte
`count`	4	4	Next multiple of 4
`code`	8	2	Next multiple of 2
`next`	16	8	Next multiple of 8

Bytes 1–3 and 10–15 are padding. The last member ends at offset 24, already a multiple of alignment 8, so the total size is 24 bytes. Ten such records occupy 240 bytes. The address of record `i`'s `count` is $A_0 + 24i + 4$.

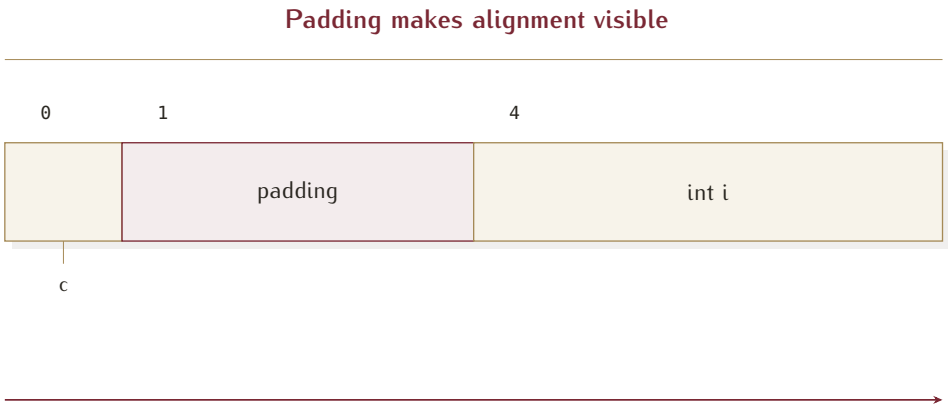


Figure 2.25: *This layout assumes a one-byte char and a four-byte int requiring four-byte alignment. The three padding bytes are not a separate source-language member. Other ABIs can specify different sizes or alignment rules.*

The C operators `sizeof` and `offsetof` provide the actual implementation's size and offsets. Their results should agree with a derivation based on that implementation's ABI. Serialised files and network messages need their own explicit byte layout: copying a native structure can carry padding and platform-dependent representations into the external format.

Laboratory 10: Structure layout and indirection

Assume System V x86-64 sizes/alignment: char 1, int 4, pointer 8. Compare `A{char tag; int count; A *next;}` with `B{char tag; B *next; int count;}`. Give offsets, sizes, array stride and code for reading `p[i].next->count` from an A array.
Worked solution

Laboratory 10: solution

A places tag at 0, count at 4 and next at 8, with total size 16 and alignment 8. B places tag at 0, next at 8 and count at 16, with total size rounded to 24. Reordering B as pointer, int, char gives offsets 0,8,12 and size 16. An array stride equals the structure size. For A, with `rdi`=p and `rsi`=i, a signed 64-bit result is:

```
mov rax, rsi
shl rax, 4
mov rax, [rdi + rax + 8]
movsxd rax, dword ptr [rax + 4]
```

The first shift multiplies by 16. Displacement 8 selects next; the second load uses that pointer and displacement 4 selects count. A valid index and live non-null next object are preconditions. `sizeof`, `_Alignof` and `offsetof` in a compiled C program verify the ABI-specific layout; `-Wpadded` requests explanations of inserted padding.
Return to the laboratory

Alignment also interacts with transfer boundaries. An eight-byte access beginning at offset 60 within a 64-byte cache line spans two lines. At offset 56 it stays within one line. An access aligned to its own power-of-two size cannot cross a conventionally aligned larger power-of-two boundary whose size is a multiple of the access size. This arithmetic does not establish that a processor uses one internal memory operation or that the operation is atomic. Instruction requirements, memory type and architectural atomicity rules remain relevant [28, 47].

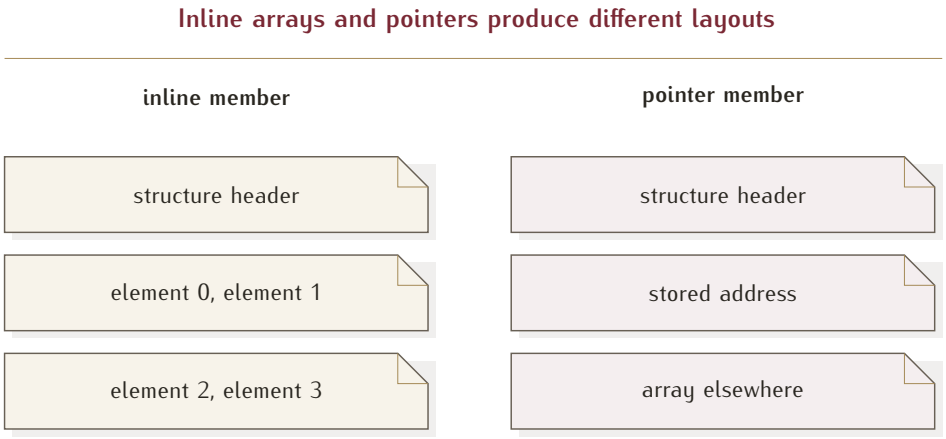


Figure 2.26: *An inline array contributes its elements directly to the enclosing object. A pointer member stores a reference to another allocation. Copying a structure with a pointer member does not automatically copy the pointed-to array.*

C grows around the objects a system must represent

Ritchie's account of C's development describes a language shaped by systems work on small machines. Extending earlier word-oriented languages to character data, typed pointers and structures required decisions about how a declaration corresponded to actual storage [145].

An array embedded in a structure made the issue concrete. Its elements had to occupy the structure's bytes directly. C's treatment of an array expression as a pointer to its first element, in the contexts where that conversion applies, avoided requiring a separately stored pointer inside every array object. The distinction remains visible in the different results of `sizeof array` and `sizeof pointer`.

Portability became a stronger driver as Unix and its tools moved to other machines. That work exposed assumptions about sizes, representations and interfaces [145]. For the layouts in this chapter, the practical consequence is to state the ABI, obtain offsets from the compiler when appropriate, and separate a language-level object from a specified external byte format. These checks preserve the intended data relationship when the machine changes.

2.3.5 Object validity and aliasing

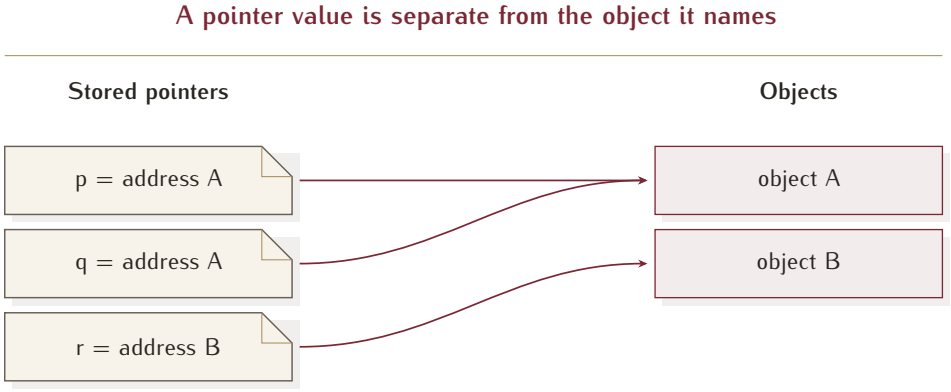


Figure 2.27: *Two pointer objects can contain equal addresses and refer to the same target. Modifying the target through one reference can affect what is later observed through the other, subject to the language's access and lifetime rules.*

A valid source-level access requires a live object, an allowed index or field, suitable alignment, an initialised value when read, and an access type permitted by the language. Page permissions provide a different boundary. An address may lie on a mapped readable page while pointing beyond the intended array or into a freed allocation.

Laboratory 11: Bounds and diagnostics

For `int a[4]={10,20,30,40}`, derive the legal load at index 3 and explain why loading at the one-past address for index 4 is invalid. Assume base `0x1000` and four-byte int. Explain what warnings, AddressSanitizer and a debugger can establish.

Worked solution

Laboratory 11: solution

Index 3 uses $\texttt{0x1000} + 3 \cdot 4 = \texttt{0x100C}$ and reads 40. Index 4 computes `0x1010`, the one-past address, which is permitted as a pointer boundary but cannot be dereferenced as an array element. A constant out-of-bounds index may trigger a compiler diagnostic. Instrumentation can detect many executed invalid accesses, while a debugger can show the computed address and surrounding bytes. Neither a numerically mapped address nor an absent diagnostic proves object-boundary validity. A controlled check can compile `int main(void){int a[4]={10,20,30,40}; volatile int i=4; return a[i];}` with `-fsanitize=address,undefined -g`; the

Worked solution (continued)

source-level bounds violation is established independently of any particular diagnostic wording.

Return to the laboratory

Two pointers alias when accesses through them can refer to overlapping storage. The compiler must preserve the effects of that overlap whenever the language permits it.

A store that changes a later load

```
int update(int *a, int *b) {
    *a = 1;
    *b = 2;
    return *a;
}
```

For distinct pointed integers, the function returns 1. If both pointers designate the same integer, the second store replaces the first and the return value is 2. A compiler must account for both permitted cases. A suitable `restrict` contract could exclude this overlap for the access pattern, allowing the return value to remain in a register; callers would then have to satisfy that stronger contract.

Aliasing rules also constrain the types used to access an object. A cast from `float *` to `uint32_t *` does not by itself permit reading a float object through the resulting integer pointer, even when both types occupy four bytes. C permits character-type access to an object's representation; `memcpy` provides another way to copy those bytes into an existing object of a suitable destination type. On the four-byte IEEE binary32 platform used here, copying a float's bytes into a `uint32_t` lets the program inspect its representation without making the incompatible typed access. The size and representation assumptions still have to hold [3].

In the flat user-space examples, address generation produces a virtual address. Translation and permissions determine which physical bytes can be accessed, while caches determine where a usable copy may be found. Chapters 6.2–6.4 develop those mechanisms. The source-level bounds and lifetime obligations established here apply throughout the translation process.

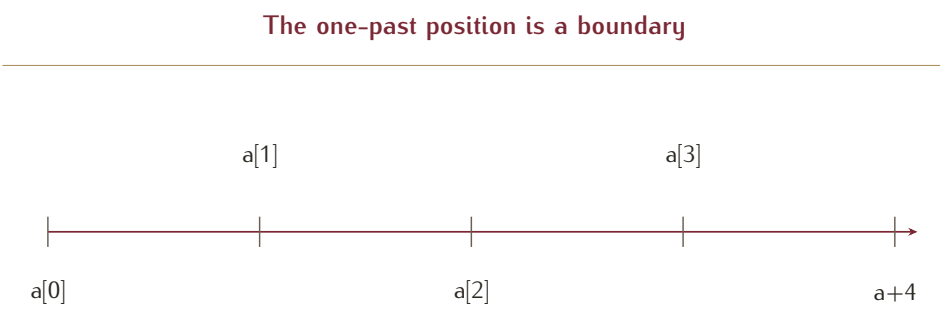


Figure 2.28: *The one-past pointer supports range endpoints and traversal under the language rules. It does not designate an element that may be dereferenced. The last valid element is one stride before that boundary.*

2.3.6 Encoding and recognising address calculations

The ModR/M byte in an x86 instruction distinguishes register operands from several memory forms. Its `reg` field can name a register or extend the opcode, depending on the instruction. A scaled-index address additionally uses a SIB byte containing scale, index and base fields. Prefixes and special field values extend or alter their interpretation. The index pattern `100` with REX.X=0 means no index; with REX.X=1 it selects `r12`. When ModR/M has `mod=00`, SIB base pattern `101` means no base register and requires a 32-bit displacement; this special case is not a displacement-free address [26].

Instruction length therefore depends on the complete addressing form. A decoder consumes prefixes and opcode, followed by any required ModR/M, SIB, displacement and immediate fields, before identifying the next instruction. The assembly expression and its encoded bytes describe the same address calculation at different levels of detail.

When examining disassembly, first establish which register provides the base and which operations establish its value. Collect access widths, constant offsets and scaled indices. Repeated four-byte accesses at offsets 4 and 8 may suggest integer fields; an eight-byte load followed by another dereference suggests a pointer. Verify those hypotheses against the control flow and the surrounding accesses. Registers can be reused for unrelated objects, and a bit pattern's type cannot be proved from its width alone.

2.3.7 Worked problems

Array address

Question. What effective address selects element `i` of a 32-bit array whose base is in `rdi` and index is in `rsi`?
Each element occupies $32/8 = 4$ bytes. Multiplying the element index by four gives its byte displacement, so the operand is `[rdi+rsi*4]`. For `i` $= 3$, the displacement is 12 bytes, reaching the fourth element. A load such as `mov eax,[rdi+rsi*4]` then reads its four bytes.
For 64-bit elements, each occupies eight bytes and the operand becomes `[rdi+rsi*8]`; a qword load uses a 64-bit destination. The scale determines the element spacing, while the instruction's operand width determines the number of bytes actually read.

Unsigned byte load

Question. Which instruction loads the byte at address `rdi` into `eax` as an unsigned value from 0 through 255?

```
movzx eax, byte ptr [rdi]
```

The load obtains eight bits; `movzx` fills bits 8–31 with zeros. The write to `eax` also clears the upper half of `rax`. For a source byte `0xFF`, the complete result is therefore 255.
The signed alternative depends on the desired result width. `movsx eax,byte ptr [rdi]` maps `0xFF` to `0xFFFFFFFF`, the 32-bit representation of -1, while still clearing the upper half of `rax`. `movsx rax,byte ptr [rdi]` extends through all 64 bits and produces `0xFFFFFFFFFFFFFFFF`, the 64-bit representation of -1.

Swap memory

Exchange two readable and writable qwords at `rdi` and `rsi`. The objects are disjoint or identical; partially overlapping qwords are excluded. `rax` and `rcx` may change.

```
mov rax, [rdi]
mov rcx, [rsi]
mov [rdi], rcx
mov [rsi], rax
```

With initial values 11 and 29, the loads save RAX=11 and RCX=29 before either store. The first store places 29 at the first location; the second places 11 at the second. Identical pointers leave the qword unchanged. For dwords, use EAX and

Worked example (continued)

ECX for all four operations. This is a single-threaded exchange; the sequence as a whole is not atomic with respect to another observer.

Page crossing

Question. Does an eight-byte access beginning at address 4092 cross a 4096-byte page boundary?
The last accessed byte is $4092 + 8 - 1 = 4099$. Page 0 covers addresses 0–4095 and page 1 begins at 4096, so the access takes four bytes from each page. Equivalently, integer division gives different page numbers:

$$\left\lfloor \frac{4092}{4096} \right\rfloor = 0, \qquad \left\lfloor \frac{4099}{4096} \right\rfloor = 1.$$

Both page mappings must permit the requested access. A four-byte access starting at 4092 ends at 4095 and stays within page 0; moving its start to 4093 makes its last byte 4096 and creates a crossing.

Exercise 2.3.1 — End pointer

Base B, n=12, element size 8. Give the one-past-end address.

Worked solution

The array contains 12 elements of 8 bytes, so it occupies $12 \cdot 8 = 96$ bytes. Element i begins at $B + 8i$. The last element has index 11 and starts at $B + 88$; its final byte is at $B + 95$. The one-past-end address is therefore $\boxed{B + 96}$. The difference $(B + 96) - B = 96$ agrees with the total storage. This address marks the upper bound of the array. Dereferencing it would access storage outside the array.

Exercise 2.3.2 — Width

Which instruction loads a signed 32-bit integer into 64 bits?

Worked solution

The source is a signed 32-bit integer, so its sign bit must be copied into the upper 32 bits of the destination. Use:

```
movsxd rax, dword ptr [address]
```

Worked solution (continued)

The operand size `dword` selects four bytes. For source bits `0xFFFFFFFE`, the signed value is -2 and the destination becomes `0xFFFFFFFFFFFFFFFE`. For source `0x00000002`, the destination becomes `0x0000000000000002`. Loading through `eax` would clear the upper half and would turn the first example into the positive value $2^{32} - 2$.

Exercise 2.3.3 — Index scale

Which scale is used for 16-bit elements?

Worked solution

A 16-bit element occupies $16/8 = 2$ bytes. Adjacent elements therefore begin two byte addresses apart, giving $\mathrm{address}(i) = B + 2i$. The scale is $\boxed{2}$. For base `0x1000`, indices 0, 1 and 2 select `0x1000`, `0x1002` and `0x1004`. In `[base+index*2]`, the scale converts an element index into a byte displacement. The index itself remains an element count.

Exercise 2.3.4 — Little-endian

Where is the least significant byte of a qword stored?

Worked solution

Little-endian ordering puts the least significant byte at the lowest address. If a qword value is `0x1122334455667788` and begins at B, the bytes at B through $B + 7$ are `0x88 77 66 55 44 33 22 11`. The byte at B has weight 2^0 in the reconstruction; the byte at $B + 7$ has weight 2^{56}. Multiplying each byte by its positional weight and adding reconstructs the original qword.

Summary

Reconstruction is the central skill. Given an address expression, identify the likely base object, scale, index and field displacement. Given a source layout, derive the byte formula and compare it with the assembly. This two-way movement turns apparently cryptic operands into evidence about data structure and compiler intent.

2.3.8 Exercises

Exercise 2.3.5 — Effective address

With `rdi=0x1000`, `rcx=5`, compute the address in `[rdi+rcx*8+0x18]`.

Go to the solution →

Exercise 2.3.6 — Array translation

Write one instruction to load `uint16_t a[i]` as an unsigned 64-bit value, with base in `rdi` and index in `rsi`.

Go to the solution →

Exercise 2.3.7 — Matrix address

For `int a[7][9]`, derive the byte offset of `a[i][j]` and sketch an x86-64 sequence.

Go to the solution →

Exercise 2.3.8 — Structure layout

Predict offsets and total size for `struct S{char a; double b; int c; char d;}` under the System V AMD64 ABI, with ordinary unpacked layout and an eight-byte size and alignment for `double`.

Go to the solution →

Exercise 2.3.9 — Improve layout

Reorder the fields of the previous structure to reduce size without changing field types. Give the new size.

Go to the solution →

Exercise 2.3.10 — Pointer-array access

With `rdi` pointing to an array of pointers and `rsi`=i, load the 32-bit signed integer at `*p[i]` into `rax`.

Go to the solution →

Exercise 2.3.11 — Audit an index calculation

A loop computes `mov eax,[rdi+rcx*4]` and stops when `rcx==rsi`. What assumptions are required to prove the access safe?

Go to the solution →

2.3.9 Solutions

Addresses, Arrays, and Data Layout

Solution 2.3.5

The index is 5 elements and each element contributes 8 bytes, giving $5 \cdot 8 = 40 =$ `0x28`. The fixed displacement is `0x18`, or 24 bytes. Adding the two displacements gives $40 + 24 = 64 =$ `0x40`, so the effective address is `0x1000` + `0x40` = `0x1040`. A load reads the operand-sized object beginning there. An `lea` with the same address expression places the number `0x1040` in a register and performs no data-memory load. Operand width must be stated separately from address calculation.

← Return to the exercise

Solution 2.3.6

Each `uint16_t` occupies two bytes. The element address is therefore `rdi+rsi*2`; an unsigned result requires zero extension:

```
movzx rax, word ptr [rdi+rsi*2]
```

Suppose `rdi=0x1000`, `rsi=3` and bytes `0x34 F2` are stored at `0x1006` and `0x1007`. The address calculation gives `0x1006`; little-endian decoding gives `0xF234`; zero extension gives `rax=0x000000000000F234`. The two source bytes are unchanged. Using sign extension would interpret the high source bit as a sign and produce a different value.

← Return to the exercise

Solution 2.3.7

Assume 4-byte `int`, row-major storage, `rdi`=base, `rsi`=i and `rdx`=j, with $0 \leqslant i < 7$ and $0 \leqslant j < 9$. Each preceding row contributes nine elements, so the flattened element index is $9i + j$. Converting to bytes gives $4(9i + j)$.

```
lea rax, [rsi+rsi*8]   ; 9*i, in elements
add rax, rdx           ; 9*i+j, in elements
movsxd rax, dword ptr [rdi+rax*4]
```

For $i = 2, j = 3$, the index is $18 + 3 = 21$ and the displacement is $21 \cdot 4 = 84 =$ `0x54`. Thus the load reads four bytes at $base +$ `0x54` and sign-extends the `int`. A complete row occupies 36 bytes, providing the independent check $2 \cdot 36 + 3 \cdot 4 = 84$. The routine assumes that the array and indices satisfy the stated bounds.

← Return to the exercise

Solution 2.3.8

Use the stated ABI assumptions: `char` has size/alignment 1, `double` 8 and `int` 4. Place each member at the next address divisible by its alignment.

Member	Offset	Size	Following gap
`a`	0	1	bytes 1–7
`b`	8	8	none
`c`	16	4	none
`d`	20	1	bytes 21–23

The largest member alignment is 8, so the complete structure must have a size divisible by 8. Rounding 21 upward gives $\boxed{24}$ bytes. Tail padding ensures that the `double` in the next element of an array of structures is also aligned. These offsets depend on the stated ABI and ordinary unpacked layout.

← Return to the exercise

Solution 2.3.9

Place members in descending alignment: `double b; int c; char a; char d;`. The double occupies bytes 0–7, the int bytes 8–11, and the chars bytes 12 and 13. Two bytes of tail padding round the size from 14 to the next multiple of 8, giving $\boxed{16}$ bytes. All members keep their types and values. The saving is $24 - 16 = 8$ bytes per structure. Such a change requires updating any external binary format, ABI interface or code that depends on the original member offsets.

← Return to the exercise

Solution 2.3.10

The pointer array has 8-byte elements on the stated 64-bit platform. Its element `i` is at `rdi+rsi*8`. Loading that element yields the address of the target integer; a second load reads the integer itself:

```
mov rcx, [rdi+rsi*8]
movsxd rax, dword ptr [rcx]
```

For `rdi=0x1000` and `i` = 2, read the pointer at `0x1010`. If it contains `0x3000` and the dword there is `0xFFFFFFFB`, the second instruction produces `rax=0xFFFFFFFFFFFFFFFB`, the signed value −5. The first load is 8 bytes; the second is 4 bytes. Both source objects must be valid readable storage. `rcx` is a temporary register.

← Return to the exercise

Solution 2.3.11

Let n be the trusted element count in `rsi`. Establish the loop invariant $0 \leqslant i \leqslant n$, with `rcx` holding i. Before every load, test whether $i = n$ and exit if so. At an executed load, $0 \leqslant i < n$ implies that the four bytes lie in $[base, base + 4n)$. After processing the element, increment i by one. The next iteration restores the invariant.

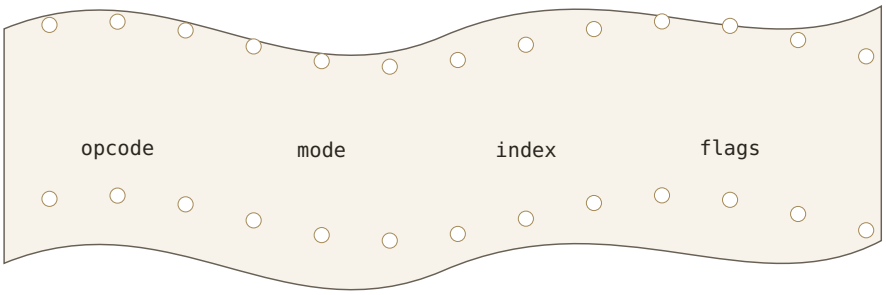
The proof also needs a valid allocation of at least $4n$ bytes, a representable size/address calculation, a correctly initialised index and no concurrent invalidation of the object. With $n = 0$, the pre-test exits without a load. A post-test would already have accessed element 0. With $i > n$, an equality-only test fails to catch the bad state, so the initial invariant is essential; an unsigned upper-bound test can reject such an invalid index explicitly.

← Return to the exercise

2.4 BITWISE OPERATIONS, SHIFTS, AND MASKS

A status word arrives with several independent fields packed into it. The program needs one field, must preserve another, and must clear a third before inserting a replacement. Each step can be drawn as a transformation of named bit positions. Masks and shifts become easier to derive when the intended destination of every retained bit is visible throughout the calculation.

A packed word is a deliberately partitioned object



Field names on the ribbon stand for disjoint bit intervals.

Figure 2.29: *Packing assigns positions to independent pieces of information. A mask or shift operates on that partition, so the exact widths and boundaries must be established before combining or extracting fields. The ribbon does not prescribe a hardware format.*

2.4.1 Masks and shifts as machine operations

A 64-bit object can be interpreted as an integer, but systems programs also treat it as a vector of individual bits and fields. Masks select or update fields; shifts align them; XOR toggles chosen positions. These operations support packed data structures, instruction encoding, device registers and later security exercises. Encoding restrictions and the control-data analysis in Chapter 2.7 also depend on precise bit manipulation.

The four bitwise operations apply independently to each bit position of a multi-bit word: `and`, `or`, `xor`, `not`. All modify the destination. Except for `not`, they update SF/ZF/PF, clear OF/CF and leave AF undefined. `test` computes an AND but discards the result, keeping only the flags; it is used to check for zero (`test rax, rax`) or a specific bit (`test rax, 0x100`).

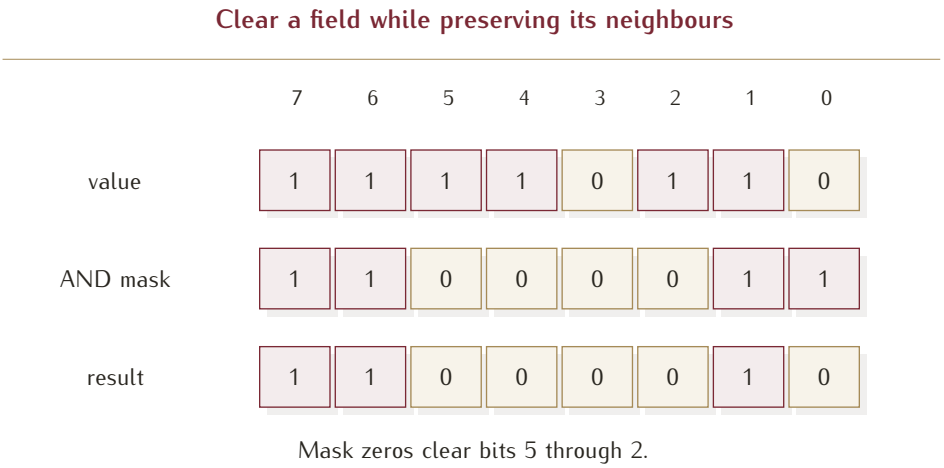


Figure 2.30: *The mask preserves positions seven, six, one and zero while clearing the middle four-bit field. A later OR can install a replacement field after that value has been limited to the field width.*

AND with a mask keeps the selected bits and drops the rest: `and rax, 0xff` keeps the low byte. *OR with a mask* forces the selected bits to 1: `or rax, 0x4` sets bit 2. *XOR with a mask* flips the selected bits: `xor rax, 0xff` inverts the low 8 bits, and `xor reg, reg` is the zero idiom. *NOT* inverts all bits; it produces the same destination bits as `xor rax, -1`, uses a shorter encoding for this operand, and preserves flags.

At the C level, bitwise operators (`&`, `|`, `^`, `~`) transform each bit position, whereas logical operators (`&&`, `||`, `!`) produce integer truth values 0 or 1. Logical AND and OR evaluate the right operand only when the left operand has not already determined the result; bitwise AND and OR evaluate both operands. A logical right shift inserts zeros; an arithmetic right shift copies the sign bit. When the promoted left operand has unsigned type, C `>>` is a logical shift. For signed negative operands, the result is implementation-defined, so portable C cannot assume that `>>` performs an arithmetic shift [2, 3]. A negative shift count or a count at least the width of the promoted left operand gives undefined behaviour, even though x86 masks the hardware count. At machine level, `test` and `cmp` update condition codes without retaining an arithmetic result.

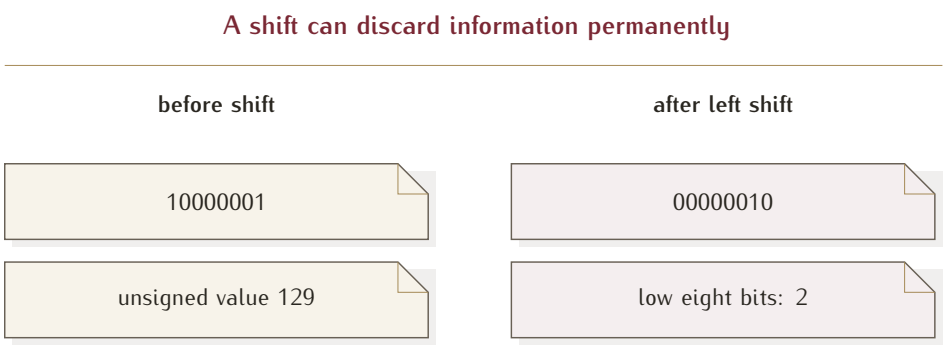


Figure 2.31: *The left shift moves the low one into position one and drops the previous top bit from the eight-bit result. Shifting back cannot reconstruct that discarded information.*

Three shift families: `shl`/`shr` (logical, fill zeros), `sar` (arithmetic right, replicate the sign), `rol`/`ror` (rotate). For unsigned values use `shr`; for signed division by 2^k use `sar`, which rounds toward $-\infty$:

```
sar rax, 2; rax = rax / 4, rounded toward -infinity
```

This differs from `idiv` for negative non-multiples: $-9/4$ is -3 via `sar` (toward $-\infty$) but -2 via `idiv` (toward zero). The C compiler inserts corrections when lowering a signed division by a power of two.

For the scalar shifts described here, x86 masks the count: a 64-bit operand uses the low 6 count bits, while 32-, 16- and 8-bit operands use the low 5. After masking, an 8- or 16-bit shift count can still be at least the operand width. For example, a count of 8 remains 8 for a byte shift; it does not become zero. The instruction specification defines the resulting bits and flags.

> **Caution**
>
> In C, shifting by $\geqslant$ the operand width is *undefined behaviour* and is not guaranteed to match x86. AArch64 register shifts, for instance, use only the low count bits for the width. For portability, validate the count against the width of the promoted left operand.

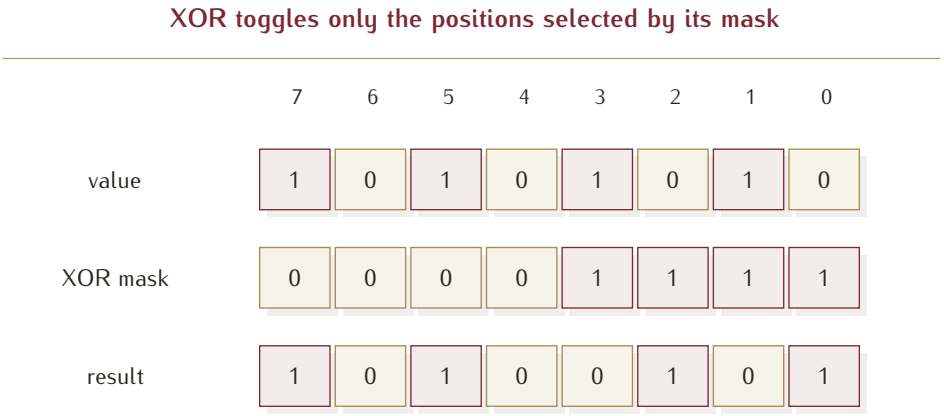


Figure 2.32: *An XOR mask bit of zero leaves the corresponding value bit unchanged. A mask bit of one reverses it. Applying the same mask twice restores the original pattern.*

For `shl`, `shr` and `sar`, a masked count of zero preserves every flag. With a nonzero count, SF, ZF and PF describe the result, and AF is undefined. CF receives the last bit shifted out, except that `shl`/`shr` leave CF undefined when the masked count is at least the operand width. OF has a defined new value only for a count of one: `shl` uses the XOR of the result's top bit and CF; `shr` copies the original top bit; `sar` clears OF. For larger counts OF is undefined. A branch after a shift must therefore use a flag that the actual count defines [26].

2.4.2 A single 1 bit at position k

Worked example

Problem. Leave $2^{12} =$ `0x1000` in `rax` (one bit set at position 12).
Solution. Three equivalent forms: `mov eax, 0x1000` (direct); `mov eax, 1`; `shl eax, 12` (makes $1 \ll 12$ explicit); or the shellcode form `xor eax, eax`; `inc eax`; `shl eax, 12` (no high immediate byte). For a dynamic k already validated to satisfy $0 \leqslant k < 64$, use a 64-bit destination:

```
mov ecx, edi; k in edi
mov eax, 1
shl rax, cl; rax = 1 << k, with 0 <= k < 64
```

The classic shift instructions take the count in `cl` or an immediate (BMI2 `shlx`/`shrx` accept a general register; here we stay on the base forms).

Worked example

Problem. `rdi` holds a 64-bit value. Extract bits 20–27 (8 consecutive bits), zero-extended, into `rax`.
Solution. The universal field-extraction pattern: shift the field down to position 0, then mask to keep only its bits.

```
mov rax, rdi
shr rax, 20; bits 20..27 now at 0..7
and eax, 0xff; mask to 8 bits
```

`and eax, 0xff` (32-bit) saves a byte over `and rax, 0xff` and zero-extends. For an n-bit field, mask with $2^n - 1$ (e.g. `0xFFFFFF` for 24 bits).

Worked example

Problem. `rdi` holds a value. Set bit 5, clear bit 10, flip bit 20; result in `rax`.
Solution.

```
mov rax, rdi
or rax, (1 << 5); set bit 5
and rax, ~(1 << 10); clear bit 10 (complemented mask)
xor rax, (1 << 20); flip bit 20
```

Constant expressions like `1 << 5` are evaluated by the assembler. `~(1 << 10)` is `0xFFFFFFFFFFFFFBFF`: all ones except bit 10.

Caution

`and rax, imm32` sign-extends its encoded 32-bit immediate to 64 bits. In GNU assembler Intel syntax, write `and rax, -4096` (equivalently, `and rax, 0xfffffffffffff000`) to preserve the high bits and clear the low 12. The positive literal `0xffff000` is not representable as a sign-extended 32-bit immediate with that positive 64-bit value; GNU `as` rejects `and rax, 0xffff000`. Distinguish the bits in the encoding from the source-level integer accepted by the assembler. A mask with bit 31 *clear*, such as `and rax, 0x0000ffff`, has zero high bits after sign extension and therefore clears the upper 32 bits of the destination.

A variable shift is a wide multiplexer network

A shift moves every bit left or right by a selected count.

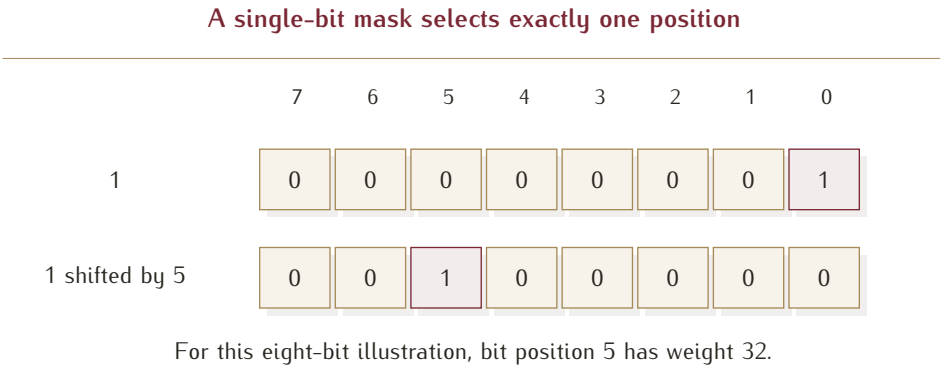


Figure 2.33: *Shifting one left by five creates a mask for position five when the operation is valid at the chosen width. Source-language shift rules and machine count handling must be considered separately.*

A barrel shifter uses several layers of multiplexers. One layer conditionally shifts by 1, another by 2, then 4, 8 and so on. The control bits select a route through this network, allowing any shift amount in roughly logarithmic depth at the cost of substantial wiring.

Wide variable shifters consume area and can lie on arithmetic critical paths, especially when combined with normalisation or address generation [51, 41].

Worked example

Problem. `rdi` holds a value. Leave the number of 1 bits in `rax`.

Solution. When the processor advertises the POPCNT feature through CPUID, use `popcnt rax, rdi`. Otherwise count bits with a loop or the following parallel reduction.

The "magic constants" popcount. View 64 bits as pairs, sum horizontally, then nibbles, then bytes:

```
#include <stdint.h>
uint64_t popcnt(uint64_t x) {
 x = (x & 0x5555555555555555) + ((x >> 1) & 0x5555555555555555);
 x = (x & 0x3333333333333333) + ((x >> 2) & 0x3333333333333333);
 x = (x & 0x0f0f0f0f0f0f0f0f) + ((x >> 4) & 0x0f0f0f0f0f0f0f0f);
 return (x * 0x0101010101010101) >> 56;
}
```

The final multiply by `0x0101...01` sums all bytes into the top byte; the $\gg 56$ isolates it. The groups contain counts bounded by 2, 4 and 8, so no carry leaks into the adjacent group during these stages. The byte sum is at most 64 and fits in the selected top byte. This implementation needs no POPCNT instruction; its generated instruction count and performance depend on the compiler and target.

Worked example

Problem. `rdi`, `rsi` hold two 64-bit values. Leave in `rax` a value whose bit `i` is 1 iff bit `i` of `rdi` and `rsi` differ.
Solution.

```
mov rax, rdi
xor rax, rsi; rax = rdi XOR rsi
```

The XOR already sets ZF according to its result: ZF=1 exactly when the two source values are equal. A subsequent `test rax,rax` would repeat that zero test. When only equality matters, `cmp rdi,rsi` establishes the same equality condition while preserving both operands.

2.4.3 Packing fields and constructing constrained values

A "bad byte" is a byte excluded by a particular input and parsing path. The NUL byte (`0x00`) terminates ordinary C strings and therefore truncates input handled by functions that rely on NUL termination. A newline (`0x0A`) is retained by `fgets` when it is read, unless the size limit or end-of-file is reached first; it becomes forbidden only when the surrounding protocol treats that newline as a record terminator.

Likewise, `strtok` has no intrinsic "space rule": it splits at bytes contained in the delimiter string supplied by the caller, which may include spaces, tabs, commas or another set. URL decoders, shells and network protocols add further transformations. The correct procedure is therefore to document the complete channel contract (read function, size bound, terminators, delimiter set, decoding steps and copies) before classifying any byte as forbidden [4].

The importance of bad bytes becomes clear in the presence of *buffer overflows* and code injection, and the corresponding defences (stack canaries, non-executable stack, ASLR). The NUL-termination convention was introduced in Chapter 1.3. An unchecked copy can violate a destination bound independently of the terminator convention; tracing both length and termination rules explains which bytes reach the destination.

If the target contains NUL bytes but no `0xFF` byte, its complement contains no NUL. It can then be loaded and complemented at runtime:

```
; target = 0x0000002e74787461 ("atxt.\0", little-endian)
mov rax, 0xffffffd18b878b9e; bitwise complement of target
not rax
push rax
```

The complement `0xFFFFFFD18B878B9E` has no zero byte; `not` restores the original, and `push` places it on the stack. Relative to loading the target directly with the same immediate width, this construction adds one three-byte `not` instruction.

If the complement still contains a forbidden byte, choose two full-width constants whose bytes are allowed and whose XOR equals the target:

```
mov rax, const_1; all bytes allowed
mov rbx, const_2; all bytes allowed
xor rax, rbx; rax = const_1 XOR const_2 = target
push rax
```

Design constraint: find $c_1 \oplus c_2 =$ target with neither c_i containing a badbyte. When only NUL is forbidden, choose each byte of the first constant to differ from both zero and the corresponding target byte; their XOR then gives a nonzero byte of the second constant. Tighter allowed sets may require a search.

Assemble the exact sequence and inspect all emitted instruction bytes, including opcodes, prefixes and displacements. If a forbidden byte occurs, revise the construction and inspect the new encoding. A permitted immediate alone does not establish that the complete sequence satisfies the channel constraint.

Population count is a reduction tree

Population count returns the number of 1 bits in a word.

Hardware can group small sets of bits, count each group and combine the partial counts through an adder tree. The structure resembles a compressor tree used in multiplication. Wider words need more stages and more routing, although vector units can perform many counts in parallel.

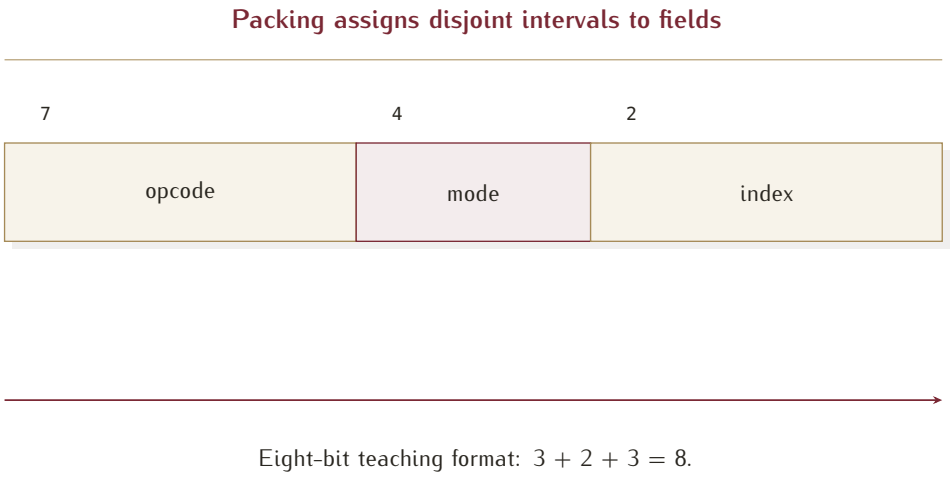


Figure 2.34: *The bit intervals partition an eight-bit word from most significant to least significant. Each source value must fit its allotted width before shifting and combining. Disjoint fields can be joined by bitwise OR.*

A dedicated instruction can replace a long software loop because specialised combinational hardware performs many additions concurrently. Parallel physical structure accounts for the gain [51, 50].

Worked example

Problem. Leave `0x0041424344454647` in `rax` without the machine code containing byte `0x00`.
Solution. Load a NUL-free immediate, then shift it into place:

```
mov rax, 0x4142434445464748; no NUL in the machine code
shr rax, 8; shift right by 8 bits
```

`0x4142434445464748` contains no `0x00`; `mov rax, imm64` encodes as `0x48 B8` followed by the 8 immediate bytes, with no NUL. After `shr rax, 8` (encoded `0x48 C1 E8 08`, no NUL), `rax` holds `0x0041424344454647`.

Caution

Check the assembled bytes against the stated input path. A test that supplies a length-delimited buffer does not reproduce a path that later applies a NUL-terminated copy; the same byte sequence can reach the destination differently.

Worked example

Problem. `rdi` holds a flag word (bit 0 "read", bit 1 "written", bit 2 "exec"…). Set bits 0 and 2, leave bit 1, clear bit 7; result in `rax`.
Solution.

```
mov rax, rdi
or rax, 0x05; set bit 0 and bit 2
and rax, -129; clear bit 7 (mask 0xffffffffffffff7f)
```

To clear only bit 7 while preserving every other bit, use a full-width mask (`0xFFFFFFFFFFFFFF7F`, i.e. the signed immediate -129), not `and rax, 0xff7f`, which would also zero everything above bit 15.

2.4.4 Deriving field masks

A reliable bit-manipulation solution starts with a layout. For each field, record its least significant bit l, width k and mask

$$M = (2^k - 1)2^l, \qquad 0 \leqslant l, \quad 1 \leqslant k, \quad l + k \leqslant w.$$

Extraction is $(x \,\&\, M) \gg l$. Replacement clears the field and ORs in the masked new value:

$$x' = (x \& \sim M) \mid ((v \ll l) \,\&\, M).$$

Masking `v` is essential; otherwise high bits can overwrite neighbouring fields. These formulas describe mathematical bit vectors. A C implementation must also validate shift counts and handle a full-width field separately, since `1 << w` is not a valid shift at width w.

Worked example: replace a 5-bit field

Bits 12–16 of `rax` hold a field. Replace it with the low five bits of `rdi` while preserving every other bit.

```
mov rcx, rdi
and rcx, 0x1f
shl rcx, 12
mov rdx, 0x1f000
not rdx
and rax, rdx
or rax, rcx
```

The register construction makes the complement explicit. Here the mask `0xFFFFFFFFFFFE0FFF` also fits a sign-extended 32-bit immediate, so `and rax,-126977` can replace the three instructions that construct and apply the clearing mask.

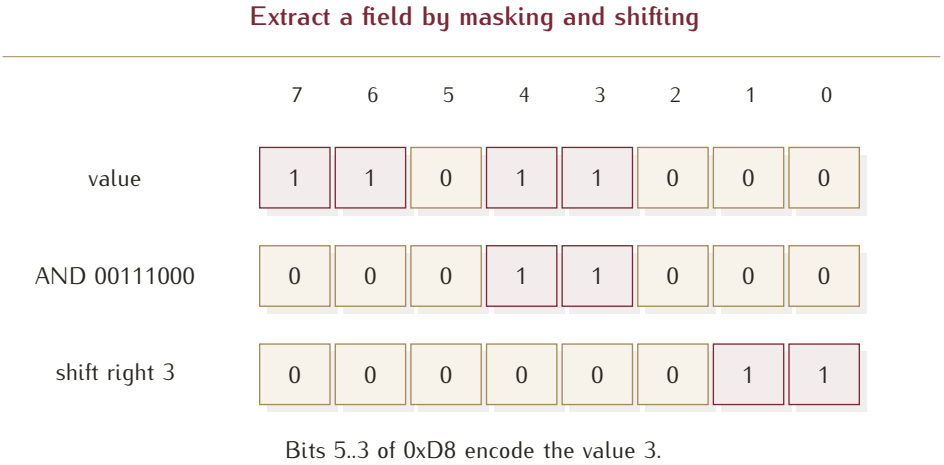


Figure 2.35: *The mask isolates the selected field at its original position. The logical right shift then moves it to the low end, where its unsigned numeric value is three.*

Worked example: the rounding difference

Arithmetic shift right maps the signed value -3 to -2, whereas signed integer division gives $-3/2 = -1$ by truncating toward zero. For a signed 64-bit input in `rdi`, add a bias of one only when the input is negative, then shift:

```
mov rax, rdi
shr rax, 63; bias = 1 for a negative input
add rax, rdi
sar rax, 1
```

For -3, the bias produces -2 before the shift and the result is -1. For -4, it produces -3 before the shift and again the result is the truncated quotient, -2. Nonnegative inputs receive a zero bias. Adding one to a negative input fits in the signed range, including at the minimum value.

`test x,m` answers whether selected bits of x are all zero without changing x. The bit-test family `bt`, `bts`, `btr` and `btc` copies one selected bit to CF and can set, reset or complement it. These instructions are useful, but ordinary masks are often clearer and easier for compilers to schedule.

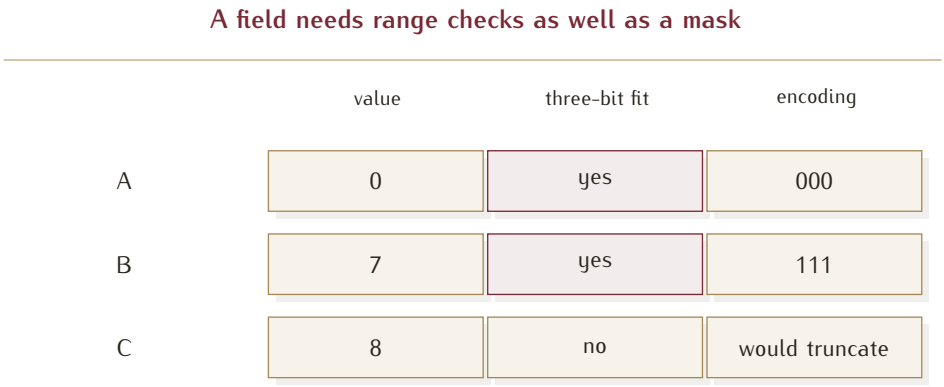


Figure 2.36: *Masking an out-of-range value can produce a valid-looking but different field. A format that requires rejection must check the value before packing rather than silently discard its high bits.*

Worked example: return one or zero for bit 13

```
bt rdi, 13
setc al
```

Worked example (continued)

```
movzx eax, al
ret
```

The result is normalised to 0 or 1. A shift-and-mask sequence is equally valid:

```
mov rax, rdi
shr rax, 13
and eax, 1
ret
```

2.4.5 Packing an instruction-like word

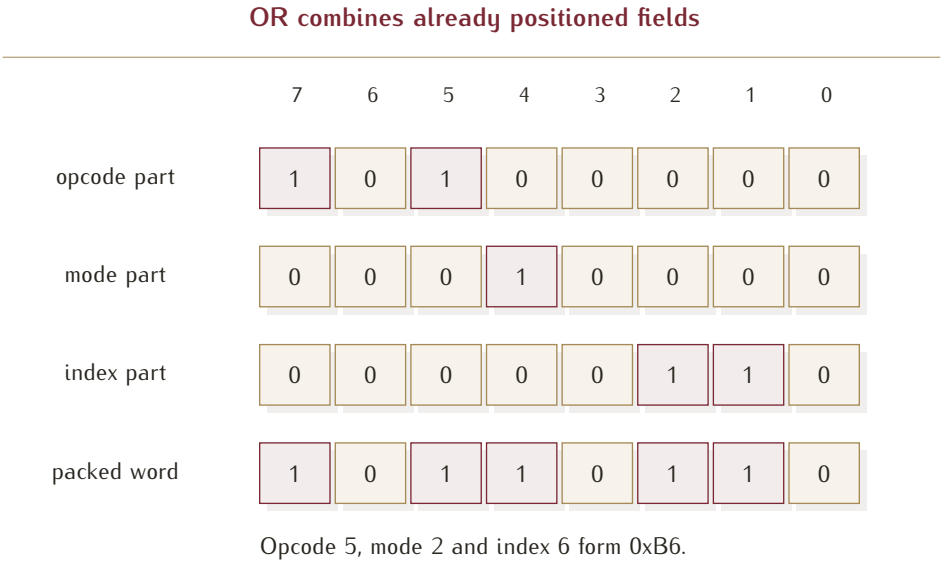


Figure 2.37: *The three fields occupy nonoverlapping positions in the teaching format. OR preserves all of them without carries between fields. Decoding must use the same widths and positions.*

Consider a 32-bit word with opcode in bits 31–26, register A in 25–21, register B in 20–16 and a 16-bit immediate in 15–0. Packing is a sequence of masks, shifts and ORs; unpacking reverses it.

Worked example: pack four fields

Inputs: opcode in `edi`, A in `esi`, B in `edx`, immediate in `ecx`.

```
and edi, 0x3f
shl edi, 26
and esi, 0x1f
```

Worked example (continued)

```
shl esi, 21
or edi, esi
and edx, 0x1f
shl edx, 16
or edi, edx
and ecx, 0xffff
or edi, ecx
mov eax, edi
```

Every input is masked before insertion. This makes the routine correct even when callers pass values with extra high bits.

Bit fields

Bit packing saves storage and bandwidth, which mattered greatly in early machines and still matters in instruction encodings, device registers, network headers and compressed data. The trade-off is complexity: fields must be documented, masking must be correct, and concurrent updates to a shared packed word may require atomic instructions. [26, 3]

Laboratory 12: Masks and instruction encoding

Implement unsigned 64-bit field extraction at bits 7–11, replacement of bits 20–27 by the low eight bits of y, and rotate-left by 13. Give assembly and an independent randomized reference check.
Worked solution

Laboratory 12: solution

Inputs are `rdi`=x, `rsi`=y where needed, output `rax`. Save the following complete GNU assembler source as `fields.S`:

```
.intel_syntax noprefix
.text
.globl extract, replace, rotate
extract:
mov rax, rdi
shr rax, 7
and eax, 31
ret
replace:
mov rax, rdi
and rax, -267386881    # clear 0x0ff00000
movzx esi, sil
shl rsi, 20
```

Worked solution (continued)

```
or rax, rsi
ret
rotate:
mov rax, rdi
rol rax, 13
ret
.section .note.GNU-stack,"",@progbits
```

The clear mask is the 64-bit complement of $255 \ll 20$. Replacement first clears the old field, then inserts the masked new field. A reference harness can evaluate the rules without depending on those instructions:

```
import ctypes, pathlib, random, subprocess, tempfile
U = ctypes.c_uint64
M = (1 << 64) - 1
rng = random.Random(217)
with tempfile.TemporaryDirectory() as work:
    library = pathlib.Path(work) / 'fields.so'
    subprocess.run(['gcc', '-shared', '-fPIC', 'fields.S',
                    '-o', str(library)], check=True)
    lib = ctypes.CDLL(str(library))
    lib.extract.argtypes = lib.rotate.argtypes = [U]
    lib.replace.argtypes = [U, U]
    for name in ('extract', 'replace', 'rotate'):
        getattr(lib, name).restype = U
    cases = [(0, 0), (M, M), (1, M), (1 << 63, 0)]
    cases += [(rng.getrandbits(64), rng.getrandbits(64))
              for _ in range(10000)]
    for x, y in cases:
        e = (x // 128) % 32
        r = sum((((y >> (i-20)) & 1) if 20 <= i < 28
                 else ((x >> i) & 1)) << i
                for i in range(64))
        t = sum(((x >> i) & 1) << ((i+13) % 64)
                for i in range(64))
        assert lib.extract(x) == e
        assert lib.replace(x, y) == r
        assert lib.rotate(x) == t
print('30012 assembly/reference comparisons passed')
```

Run the Python script from the directory containing `fields.S`. It builds and calls the actual functions, using arithmetic and per-bit references, and checks 10,004 inputs for each operation. For example, `shr rax,7` encodes as `48 c1 e8 07`: REX.W, group opcode, ModR/M selecting SHR on RAX, immediate 7. Changing RAX

Worked solution (continued)

to RCX changes ModR/M to `e9`. There is no displacement in this register-only instruction.

Return to the laboratory

From packed machine words to modern bit-manipulation instructions

Timeline

Year	Machine, standard or event	Architectural significance
1949	**Packed machine words**	Scarce memory encourages fields and masks inside words.
1978	**8086 shifts and rotates**	Bit movement becomes a standard integer facility.
2013	**BMI1/BMI2 deployment**	Dedicated extraction, deposit and bit-count support expands.

Bit operations arose because the first stored-program machines had small memories, fixed word formats and peripherals whose state had to be represented compactly. A word was simultaneously a number and a container of named fields. On the Manchester Mark I, operational in 1949, data occupied 40-bit words and two 20-bit instructions could share a word, while modifier registers, the machine's "B-lines", altered address fields. The IBM 704, announced in 1954, used 36-bit words whose instruction formats divided the word into an operation field, a decrement, a tag and an address. To program such machines was already to reason about selected groups of bits rather than about undifferentiated integers [90, 110].

Shifting was first an arithmetic mechanism as much as an instruction. In a bit-serial arithmetic unit, one adder is reused while registers move one position at a time; multiplication and division naturally become sequences of shifts and conditional additions or subtractions. Even in parallel machines, a left shift gives the low-order part of multiplication by two, while a logical right shift performs unsigned division by two with truncation.

A signed arithmetic right shift is a different contract: it replicates the sign bit, and its rounding behaviour must be stated by the architecture. The distinction is visible in the 1964 *IBM System/360 Principles of Operation*, which separately documents logical, arithmetic and double-length shifts, their count rules and their condition-code effects [109].

Masks were equally fundamental. AND keeps the positions selected by a mask and clears the others; OR sets selected positions; XOR toggles them; complement reverses them. System/360, announced on 7 April 1964, provided logical operations on registers and storage, "test under mask" instructions and a two-bit condition code in the Program Status Word. A device-status word or control word could therefore contain many inde-

pendent predicates, and software could branch according to whether a selected field was zero, all one or mixed. The manual defines the Boolean result, the resulting condition code and the bits preserved by the operation [74, 109].

Packing was also driven by character and decimal data. Before the eight-bit byte became dominant, machines commonly used five-, six- or seven-bit character encodings and packed several characters into one word. IBM commercial systems represented decimal digits in binary-coded decimal, and System/360 retained packed-decimal instructions alongside binary arithmetic. In packed decimal, two digits share a byte and the final half-byte carries the sign. Financial applications valued exact decimal representation, and descendants of those instructions remain architecturally visible in IBM mainframes. The eight-bit byte standardised by System/360 made field boundaries more convenient while retaining useful sub-byte fields [74, 109].

The x86 line preserved and enlarged this vocabulary. The Intel 8086, introduced in 1978, defined logical shifts, arithmetic shifts and rotates, with the count supplied either by one or by register CL. Rotates circulate discarded bits back into the operand; rotate-through-carry treats the carry flag as an additional bit. Later IA-32 and Intel 64 extensions added bit test and modification, bit scan, population count and leading- or trailing-zero count.

The exact behaviour differs by instruction: some counts are masked, some instructions leave particular flags undefined, and operand-size rules can change the result. The instruction entry in the Intel or AMD architecture manual specifies the operand widths, count rules and flags [104, 26, 27].

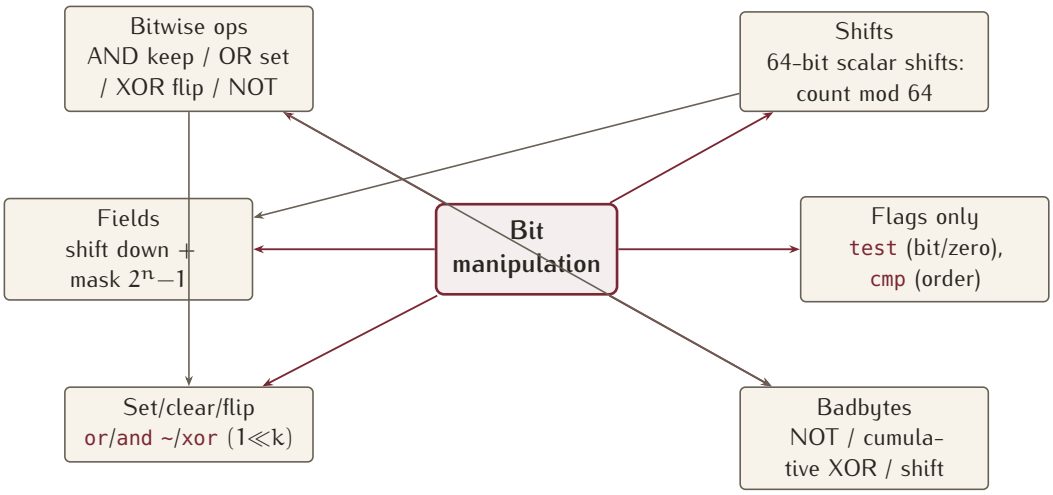
Population count illustrates the movement from software idiom to hardware primitive. The operation returns the number of one bits in a word. It is useful for Hamming distances, error-correcting codes, cryptography, chess bitboards and set intersections. For decades programmers used table lookups or parallel arithmetic identities. AMD included `POPCNT` in the Advanced Bit Manipulation extension implemented in the Barcelona generation of 2007; Intel implemented the instruction with the Nehalem generation introduced in 2008. Current vector extensions can count bits in multiple packed elements. An operation that once required a carefully staged sequence is now a decoded architectural instruction, but its latency, throughput and vector availability remain microarchitecture-dependent [27, 106, 28].

A second wave of x86 instructions arrived with BMI1 and BMI2 in the 2010s. Instructions such as `ANDN`, `BEXTR`, `BZHI`, `PDEP`, `PEXT`, `SHLX`, `SHRX` and `SARX` make common masks and variable shifts explicit. `PEXT` gathers selected source bits into contiguous low-order positions; `PDEP` scatters low-order source bits into positions selected by a mask. These instructions can express compact data-structure operations, but they are not automatically faster on every processor. Compiler target selection and vendor optimisation manuals must be consulted because the same architectural instruction can have very different execution costs across implementations [26, 27, 28, 29].

Bit vectors eventually became a standard representation for sets. If bit i denotes membership of element i, union is OR, intersection is AND, and symmetric difference is XOR. Operating systems use this idea for CPU-affinity masks, signal sets, permissions and readiness maps. Linux kernel code contains bitmap and bit-operation APIs because concurrency changes the contract: an ordinary read-modify-write sequence can lose another core's update, whereas an atomic bit operation has ordering and indivisibility properties defined by the architecture and kernel API. The mathematical identity remains simple; the implementation is correct only when width, alignment and synchronisation are explicit [35, 40].

Security engineering gave masks a further role. Constant-time cryptographic routines try to avoid secret-dependent branches and secret-indexed memory accesses. A selection can be written as $(a \,\&\, m) \mid (b \,\&\, \sim m)$ with m all zero or all one. That expression is not, by itself, proof of constant-time execution: the compiler may transform it, the target ISA may lower it differently, and microarchitectural effects must be audited. Pointer-tagging schemes likewise reuse selected address bits only under a documented architectural and operating-system contract. Intel Linear Address Masking and analogous mechanisms are not permission to ignore canonical-address and ABI rules [26, 27].

Concept map: bit manipulation



Reading the map. The four bitwise ops (with the shift family) generate every higher idiom: extracting a field is shift+mask; set/clear/flip is OR/AND-NOT/XOR with $(1 \ll k)$; `test`/`cmp` expose only flags; and bad-byte avoidance reuses NOT/XOR/shift to reshape a constant's machine-code bytes.

2.4.6 Worked problems

Keep a byte

Question. Which mask keeps only the low byte of `rax` and clears every higher bit?
The low byte occupies bits 0–7. Its mask is $2^8 - 1 = 255 =$ `0xFF`, so `and rax,0xff` preserves exactly those positions. For `0x1234`, the high byte intersects zeros in the mask and disappears, leaving `0x34`. Applying the mask again preserves `0x34`, since all remaining one bits already lie inside the selected field.
To retain the low sixteen bits, widen the mask to $2^{16} - 1 =$ `0xFFFF` and use `and rax,0xffff`.

Swap nibbles

Question. How can the high and low nibbles of an eight-bit unsigned value be exchanged?
Isolate the low nibble with `0x0F` and move it left by four. Isolate the high nibble with `0xF0` and move it right by four. The two moved fields occupy disjoint positions, so OR combines them:

$$y = ((x \,\&\, \texttt{0x0F}) \ll 4) \mid ((x \,\&\, \texttt{0xF0}) \gg 4).$$

For $x =$ `0xA7`, the two contributions are `0x70` and `0x0A`, giving `0x7A`. Repeating the operation moves each nibble back and restores `0xA7`. The same check gives `0x3C`→`0xC3`→`0x3C`.

Field insertion

Question. Insert the low four bits of an unsigned value v into bits 7–4 of x, preserving every other bit.
The destination mask is $M = \texttt{0x0F} \ll 4 = \texttt{0xF0}$. First clear that region with $x\& \sim M$. Then retain the low four bits of v and shift them left by four:

$$x' = (x\& \sim \texttt{0xF0}) \mid ((v \,\&\, \texttt{0x0F}) \ll 4).$$

Use unsigned operands and a complement taken at the width of x. With $x =$ `0xABCD` and $v = 5$, clearing produces `0xAB0D`, insertion contributes `0x0050`, and the result is `0xAB5D`. Extracting bits 7–4 recovers 5; all bits outside `0xF0` remain unchanged.
For a three-bit field at bits 5–3, the corresponding mask is $7 \ll 3 =$ `0x38`, and the inserted contribution is $(v \,\&\, 7) \ll 3$.

Round up

Question. Round the nonnegative integer 37 upward to a multiple of 16.
A multiple of 16 has four zero low bits. Adding 15 gives $37 + 15 = 52 = \texttt{0x34}$. Clearing the low four bits selects `0x30`, or 48:

$$(37 + 15)\& \sim 15 = 48.$$

The result is a multiple of 16, is at least 37, and exceeds 37 by less than 16. These three properties establish that it is the least acceptable multiple.
For power-of-two alignment a, the general unsigned expression is $(x + a - 1)\& \sim (a - 1)$, provided $a > 0$ and the addition fits the chosen width. Checking $x \leqslant \mathrm{MAX} - (a - 1)$ before the addition prevents wraparound. With $x = 65$ and $a = 32$, adding 31 gives 96, already aligned, so the result is 96.

Exercise 2.4.1 — Clear bit

For `unsigned int x`, give a mask expression to clear bit 9.

Worked solution

Form the mask `1u << 9`, which has only bit 9 set. Complement it to obtain zeros at bit 9 and ones everywhere else, then compute `x & ~(1u << 9)`. AND with one preserves each other bit; AND with zero clears bit 9. For `x=0x2A5`, the mask is `0x200` and the result is `0x0A5`. Use an unsigned mask whose width matches `x`.

Exercise 2.4.2 — Extract field

Extract width 5 at offset 11.

Worked solution

The requested field occupies bit positions 15 through 11. Shift right by 11 so that those positions become 4 through 0, then retain five bits with $2^5 - 1 = 31 = \texttt{0x1F}$. The expression is `(x >> 11) & 31u`. For a field value 19, the contribution `19u << 11` becomes 19 after the shift; the mask removes any original bits above position 15. Unsigned `x` makes the right shift logical.

Exercise 2.4.3 — All bits

How test that every bit in mask m is set?

Worked solution

Compute `x & m`. A one bit in `m` selects a required bit, while a zero bit excludes that position from the test. Every selected bit is set exactly when the result equals the mask: `(x & m) == m`. Testing only for a nonzero intersection would establish that at least one required bit is set. For `m=0x0A`, `x=0x02` fails the all-bits test because bit 3 is missing. A zero mask is satisfied vacuously.

Exercise 2.4.4 — Power of two

Is 48 a power of two?

Worked solution

The value 48 is $32 + 16 = 110000_2$, so it contains two set bits. A positive power of two contains exactly one set bit. The standard test confirms the result: $48 \,\&\, 47 = 32$, which is nonzero. Thus 48 is not a power of two. For an unsigned value, the complete test also excludes zero: `x != 0 && (x & (x-1)) == 0`.

Summary

Before writing a mask, draw the word and label the field boundaries. Derive the mask from the width, then shift. When reading code, reverse the process: identify the mask's runs of ones, account for the shift and state the extracted range. This visual discipline catches more errors than memorising isolated hexadecimal constants.

2.4.7 Exercises

Exercise 2.4.5 — Set, clear and toggle

Give expressions and x86-64 instructions to set bit 9, clear bit 9 and toggle bit 9 of `rax`.

Go to the solution →

Exercise 2.4.6 — Extract a field

Bits 20–27 hold an unsigned field. Extract it into `rax` from `rdi`.

Go to the solution →

Exercise 2.4.7 — Replace a field

Replace bits 8–15 of `rax` with the low byte of `rdi`, preserving all other bits.

Go to the solution →

Exercise 2.4.8 — Shift semantics

Explain results of `shr eax,1` and `sar eax,1` for `eax=0xfffffffd`.

Go to the solution →

Exercise 2.4.9 — Test a compound condition

Branch if either bit 2 or bit 7 of `rdi` is set, without changing `rdi`.

Go to the solution →

Exercise 2.4.10 — Pack fields

Pack a 4-bit version in bits 31–28, a 12-bit length in 27–16 and a 16-bit identifier in 15–0. State the masks and formula.

Go to the solution →

Exercise 2.4.11 — Explain a byte restriction strategy

A submission contract forbids zero bytes in machine code. The required 64-bit constant contains zero bytes. Give two general construction strategies and explain how to verify them.

Go to the solution →

2.4.8 Solutions

Bitwise Operations, Shifts, and Masks

Solution 2.4.5

The selected mask is $2^9 = 512 = \texttt{0x200}$. Set with $x | 512$ using `or rax,512`; clear with $x \,\&\, \sim 512$ using `and rax,-513`; toggle with $x \oplus 512$ using `xor rax,512`. The negative clearing immediate sign-extends to `0xFFFFFFFFFFFFFDFF`, preserving every other bit. `btr rax,9` and `btc rax,9` are alternative clear/toggle operations with different flag effects.

← Return to the exercise

Solution 2.4.6

Shift the source right by 20 so that source bits 20–27 occupy result bits 0–7, then clear every higher bit:

```
mov rax, rdi
shr rax, 20
and eax, 0xff
```

The mask is $2^8 - 1 = 255$. Writing `eax` also clears the upper half of `rax`, so the result is an unsigned value in $[0, 255]$. For source `0xAB00000`, the shift produces `0xAB` and the final result is 171. Source bits outside the selected field cannot affect the result.

← Return to the exercise

Solution 2.4.7

The destination field has mask $M = \texttt{0xFF00}$. Isolate the source byte, move it to bits 8–15, clear the old field, and insert the new bits:

```
mov rcx, rdi
and ecx, 0xff
shl rcx, 8
and rax, -65281   # ~0xff00 at width 64
or rax, rcx
```

The temporary preserves `rdi`. The clearing immediate sign-extends to `0xFFFFFFFFFFFF00FF`; every bit outside the destination field therefore survives the AND. The shifted temporary contains zeros outside that field, so OR preserves those same surrounding bits.
For initial `rax=0x1122334455667788` and low source byte `0xAB`, clearing yields `0x1122334455660088`; insertion yields `0x112233445566AB88`.

← Return to the exercise

Solution 2.4.8

The initial 32-bit pattern `0xFFFFFFFD` is $2^{32}-3$ unsigned and -3 signed. `shr eax,1` inserts a zero at bit 31, giving `0x7FFFFFFE`, the unsigned quotient $\lfloor (2^{32}-3)/2 \rfloor = 2147483646$.
`sar eax,1` instead copies the old sign bit into bit 31, giving `0xFFFFFFFE`. This represents -2 in signed 32-bit arithmetic and equals $\lfloor -3/2 \rfloor$. Both instructions write `eax`, so both clear the upper 32 bits of `rax`. After `sar eax,1`, a signed 64-bit use of -2 would need a separate sign extension.

← Return to the exercise

Solution 2.4.9

The selected positions have mask $2^2 + 2^7 = 4 + 128 = 132 = \texttt{0x84}$:

```
test rdi, 0x84
jne target
```

`test` forms the AND for flags and leaves `rdi` unchanged. Its result is zero only when both selected bits are zero. Otherwise ZF=0 and `jne` takes the branch. Values 4, 128 and 132 all take the branch; value 8 leaves both selected bits clear and falls through.

← Return to the exercise

Solution 2.4.10

For unsigned 32-bit operands on the stated platform, use `((version&0xfu)<<28) | ((length&0xfffu)<<16) | (id&0xffffu)`. The masks retain exactly 4, 12 and 16 bits. For version 2, length `0x345` and identifier `0x6789`, the contributions are `0x20000000`, `0x03450000` and `0x00006789`; their OR is `0x23456789`. These masks truncate excess input bits; a format that forbids out-of-range values must reject them before packing.

← Return to the exercise

Solution 2.4.11

Two constructions can produce the example target `0x0041424344454647`. The first loads a value with one extra low byte and shifts it away:

```
mov rax, 0x4142434445464748
shr rax, 8
ret
```

The shift introduces the required zero high byte at runtime. The complete encoding is `48 b8 48 47 46 45 44 43 42 41 48 c1 e8 08 c3`, which contains no zero byte.

Worked solution (continued)

The second constructs the target as the XOR of two full-width constants:

```
mov rax, 0x0101010101010101
mov rcx, 0x0140434245444746
xor rax, rcx
ret
```

The second constant is the first XOR the target. XORing them again cancels the first constant and recovers the target. Both immediates have nonzero bytes; the remaining opcode bytes are also nonzero. For an arbitrary target byte t, select c different from both zero and t, then use $d = c \oplus t$. This ensures that both c and d are nonzero.

To verify either sequence, place it after `.intel_syntax noprefix` and `.text` in `construct.S`, then assemble and extract only its code section:

```
as --64 -o construct.o construct.S
objcopy -O binary --only-section=.text construct.o construct.bin
objdump -d -M intel construct.o
```

Inspect `construct.bin` with `xxd` and check its bytes programmatically:

```
from pathlib import Path
code = Path('construct.bin').read_bytes()
assert code and 0 not in code
```

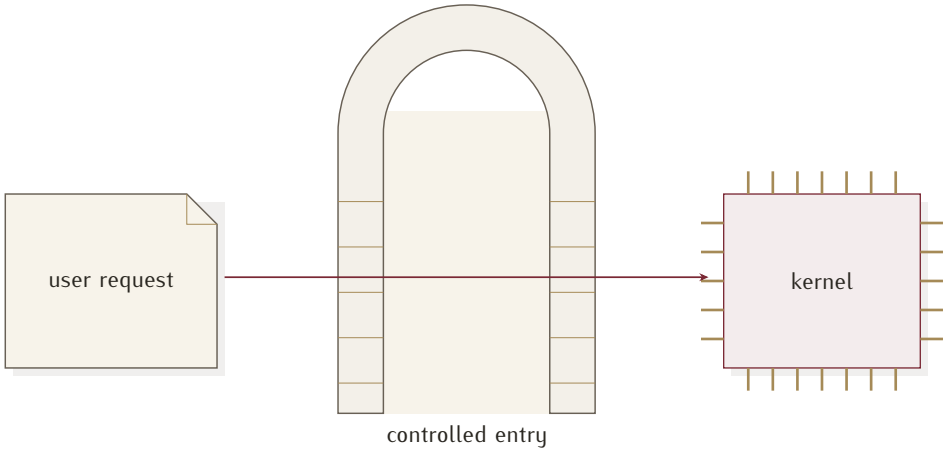
The disassembly checks instruction boundaries and immediate values. The byte test covers opcodes, prefixes and immediates together. Finally, trace or call the sequence through a valid caller and confirm the returned `rax`; a permitted encoding and a correct computed value are separate requirements.

← Return to the exercise

2.5 LINUX SYSTEM CALLS AND THE USER–KERNEL BOUNDARY

A program has prepared a buffer and asks the kernel to write its bytes. It places a request number and arguments in the system-call registers, crosses the protected entry boundary, and receives either a result or an error. Following this request connects the assembly interface to the kernel's authority and validation. It also explains why a successful call can report fewer bytes than the program originally requested.

A controlled crossing into kernel authority



The gate depicts an interface check, not a physical wall.

Figure 2.38: *The arch is a visual metaphor for the protected system-call interface. The request crosses according to an ABI, and the kernel validates its values and references before using privileged mechanisms. The picture does not imply a physical wall.*

2.5.1 From a user request to the kernel boundary

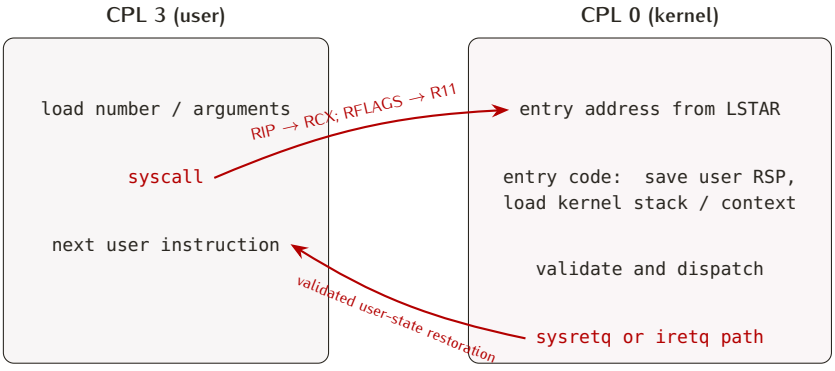
So far the discussion has concentrated on registers and memory in one process. Operations such as opening a file, creating a process or requesting ordinary kernel-managed I/O require an operating-system service, exposed through a *system call*. From user space, a syscall is a controlled request to the kernel. On x86-64 the instruction transfers control to kernel-selected code at a more privileged level; entry software then establishes the

kernel stack and, where configured, the kernel address-space context before dispatching the request. A validated return path later restores user execution. Privilege does not remove every protection rule: page mappings, control registers and kernel policy still constrain what the executing code can access [26, 33].

x86 defines four privilege levels (*rings*) 0–3; mainstream Linux uses primarily CPL 0 for the kernel and CPL 3 for user processes. At CPL 0 the kernel may execute privileged instructions and configure address translation, interrupt routing and device access, but it remains subject to the current page tables, architectural state, virtualisation and platform controls. A normal CPL 3 process cannot execute privileged instructions, access kernel-only mappings or perform unpermitted port I/O; it requests kernel services through controlled interfaces such as system calls.

A system call is a deliberate synchronous request across a protected boundary. The discussion begins with the caller-visible ABI: register assignment, raw results, short counts and file descriptors. Chapter 4.2 treats the protected entry sequence and version-specific kernel control flow. Operating-systems texts sometimes use the broad term trap for a service request. Native x86-64 `syscall` enters through its dedicated MSR-configured mechanism. The C library translates a raw negative error into -1 plus thread-local `errno`; file descriptors provide a common handle for regular files, terminals, pipes, sockets and many devices.

On x86-64, `syscall` enters at an address configured in model-specific registers, saves the user return RIP in `rcx` and RFLAGS in `r11`, and applies configured flag masking. Linux may return with `sysretq` when safe or with `iretq` when a more general restoration path is required.



The implementation walkthroughs here and in Chapter 4.2 use the Linux v7.1 release tree. The syscall ABI is a user-visible compatibility contract; the names and arrangement of internal entry and dispatch helpers are implementation details and can change in later kernels.

The syscall calling convention (distinct from the C function ABI of 2.6):

Register	Meaning
`rax`	syscall number
`rdi`	arg 1
`rsi`	arg 2
`rdx`	arg 3
`r10`	arg 4
`r8`	arg 5
`r9`	arg 6
`rax`	return value (overwrites the number)

The fourth argument uses `r10` because the hardware transition reserves `rcx` for the saved user instruction pointer. An ordinary function call uses `rcx` for that argument.

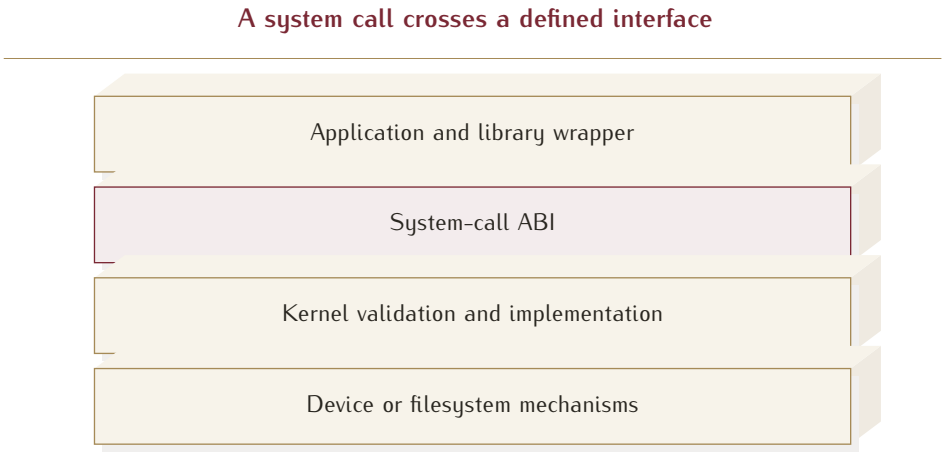


Figure 2.39: *The wrapper presents a language-level interface while the system-call ABI defines the machine-level request. Kernel mechanisms implement the operation after interpreting and validating that request.*

`syscall` vs `int 0x80`. Before `syscall`, Linux used the software interrupt `int 0x80` (number in `eax`, args in `ebx`/`ecx`/`edx`/`esi`/`edi`/`ebp`), which follows the legacy interrupt-gate compatibility path and a different 32-bit ABI. `syscall` is the native long-mode mechanism designed for system-call entry. When the kernel enables IA-32 compatibility, `int 0x80` invokes the 32-bit syscall ABI even from 64-bit user code. Numbers differ (`write` is 4 there and 1 in the native ABI), and pointer arguments are limited to the compatibility width. Its presence identifies the requested interface, not necessarily the execution mode of the whole program.

#	Name	Action
0	read	read from a file descriptor
1	write	write to a file descriptor
2	open	open a file, return an fd
3	close	close an fd
9	mmap	map a memory region
10	mprotect	change a region's permissions
59	execve	replace the process with a new program
60	exit	terminate the (calling) thread
231	exit_group	terminate all threads of the process

Most shellcode uses a small subset: `read`, `write`, `open`, `execve`, `mmap`, `mprotect`, `exit`.

Privilege is enforced by control paths

User and kernel modes have different permission to execute instructions and access mappings.

Mode bits, page-translation results and instruction-decode signals feed permission checks. If a requested operation is not allowed, control logic prevents the state update and selects an exception path. The same transistors and interconnect carry user and kernel data; protection comes from checked control conditions and preserved state.

A privilege boundary is only as strong as all of the hardware paths that enforce it. Speculative and transient effects matter because a blocked architectural update can still have caused physical activity in caches or predictors [26, 50].

2.5.2 The Linux x86-64 syscall ABI

On return, `rax` contains the raw result instead of the syscall number. Linux reserves $[-4095, -1]$ for errors: `rax`$= -$errno, such as -2 for `ENOENT`, -13 for `EACCES`, and -14 for `EFAULT`. The generic test is `cmp rax,-4095` followed by unsigned `jae .error`. For calls such as `read`, `write` and `open`, successful results are nonnegative, so `test rax,rax; js .error` is also sufficient.

The "negative = error" convention is the raw Linux kernel interface. A conventional C-library wrapper converts a kernel error return to -1 and sets `errno` to the corresponding positive error number. Assembly that executes `syscall` directly observes the unconverted negative result. The libc function named `syscall()` is still a wrapper: it returns -1 and sets `errno` on a kernel error.

Linux x86-64 system calls assign register roles

	role	register
number	operation	RAX
argument 1	first value	RDI
argument 2	second value	RSI
argument 3	third value	RDX
argument 4	fourth value	R10

Arguments 5 and 6 use R8 and R9; RCX and R11 are clobbered.

Figure 2.40: *The first four argument positions shown here expose the difference from the ordinary function-call convention: argument four uses R10. The complete contract includes the remaining arguments and the documented clobbers.*

1. load args into `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` (as many as needed);
2. load the syscall number into `rax`;
3. execute `syscall`;
4. check `rax` for the return value.

The hardware `syscall` transition clobbers `rcx` and `r11`; Linux returns the service result in `rax` and preserves the other general-purpose registers. Save live values held in `rax`, `rcx` or `r11`, and account separately for memory or process state that the requested service can change.

Worked example

Problem. Terminate the process with exit code 7.
Solution.

```
mov rdi, 7; exit code
mov rax, 231; exit_group
syscall
```

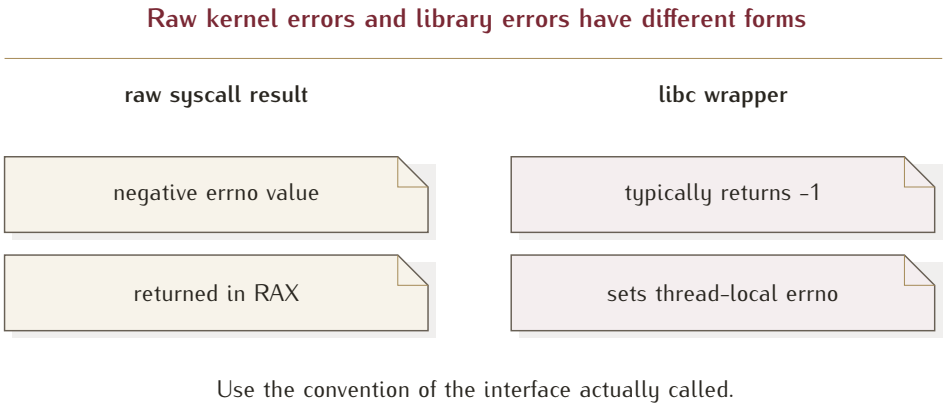


Figure 2.41: *The wrapper translates the kernel's error representation into the library contract. A direct syscall sequence must inspect its raw result correctly; assuming that every failure is exactly minus one loses information.*

`exit_group` (231) ends all threads; `exit` (60) only the current one (equivalent in single-threaded programs). A NUL-free shellcode form:

```
xor edi, edi
mov dil, 7; exit code 7
xor eax, eax
mov al, 231; exit_group
syscall
```

2.5.3 File-descriptor I/O

Worked example

Problem. Prepare and issue a system-call request to write the string "Ciao\n" (5 bytes) to stdout.

Solution. `write` (1) takes fd, buffer address, byte count; stdout is fd 1. Build the buffer on the stack, choosing the constant so the bytes land at offset 0 from `rsp` (little-endian: the first character is the low byte):

```
; "Ciao\n" = 0x43 0x69 0x61 0x6f 0x0a; qword = 0x0000000A6F616943
mov rax, 0x0000000a6f616943; 'C' low byte, '\n' at byte 4
push rax; [rsp]='C'... [rsp+4]='\n'
mov rdi, 1; fd = stdout
mov rsi, rsp; buffer
mov rdx, 5; length
mov rax, 1; write
syscall
```

A single `write` can return fewer bytes than requested or fail with an interrupt/error. This compact example demonstrates the ABI only; robust code checks `rax`, retries where appropriate (for example after `EINTR`), and loops until the intended byte count is written or a terminal condition occurs.

Caution

The SysV red zone permits temporary user-space storage within 128 bytes below `rsp`; signal and interrupt delivery must preserve that area. Such values must not remain live across an ordinary function call, whose stack use can overlap them. Explicitly allocating the buffer by adjusting `rsp` makes its lifetime and bounds easier to track and also applies to environments that provide no red zone.

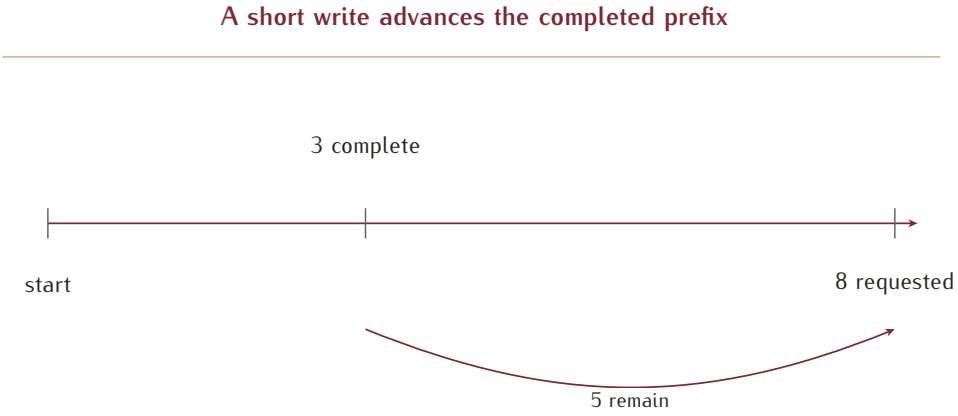


Figure 2.42: *A positive short result reports progress rather than total completion. A loop advances the buffer pointer and reduces the remaining count by the returned amount. Error, interruption and zero-progress cases require their own handling.*

Worked example

Problem. Request a read-only open of `/etc/passwd`. On success, leave the descriptor in `rdi`; preserve a negative raw error in `rax` on failure.

Solution. `open` (2): path address, flags, mode. `O_RDONLY` = 0. The path `/etc/passwd\0` is 12 bytes; split into two qwords and push the *high* half first (the stack grows downward, so the low half ends at the lower address where `rdi` points):

```
; "/etc/passwd\0": bytes 0-7 = 0x7361702F6374652F, bytes 8-15 = 0
    ↪ x0000000000647773
```

Worked example (continued)

```
mov rax, 0x0000000000647773; bytes 8-15: "swd\0\0\0\0\0"
push rax
mov rax, 0x7361702F6374652F; bytes 0-7: "/etc/pas"
push rax
mov rdi, rsp; path
xor rsi, rsi; O_RDONLY
xor rdx, rdx; mode 0
mov rax, 2; open
syscall
test rax, rax
js .open_failed
mov rdi, rax; successful descriptor
.open_failed:
```

Caution

Modern code prefers `openat` (257) with `rdi` = `AT_FDCWD` (−100) and `rsi` = path. `open` (2) still exists on Linux x86-64, but seccomp policies or non-Linux systems may change what is available.

Worked example

Problem. Request up to 100 bytes from the descriptor in `rdi` into a buffer; leave the actual count or raw error in `rax`.
Solution.

```
sub rsp, 128; buffer space (with margin)
mov rsi, rsp; buffer
mov rdx, 100; max bytes
mov rax, 0; read
syscall; rax = bytes read (<= 100), or -errno
```

Common errors: forgetting to set `rax` (the syscall number); allocating less than the requested count (a stack buffer overflow, Section 2.7); confusing argument positions: both ABIs use `rdx` for argument 3, while argument 4 uses `rcx` for functions and `r10` for syscalls.

Worked example

Problem. Open `/tmp/test`, read up to 200 bytes, write them to stdout, in one sequence.
Solution.

Worked example (continued)

```
# Entry fragment: open /tmp/test and transfer one read of up to 200 bytes.
mov rax, 0x74
push rax
mov rax, 0x7365742f706d742f
push rax
mov rdi, rsp
xor esi, esi
xor edx, edx
mov eax, 2
syscall
test rax, rax
js .exit_error
mov r12, rax                # owned input descriptor
sub rsp, 256
.read_again:
mov rdi, r12
mov rsi, rsp
mov edx, 200
xor eax, eax
syscall
cmp rax, -4                 # EINTR: no completed read
je .read_again
test rax, rax
js .close_error
mov rdx, rax                # only bytes actually read
mov rsi, rsp
.write_more:
test rdx, rdx
je .close_success
mov edi, 1
mov eax, 1
syscall
cmp rax, -4
je .write_more
test rax, rax
jle .close_error            # error or no progress
add rsi, rax
sub rdx, rax
jmp .write_more
.close_success:
xor r13d, r13d
jmp .close_input
.close_error:
mov r13d, 1
.close_input:
```

Worked example (continued)

```
mov rdi, r12
mov eax, 3
syscall
# On Linux, do not retry close after an error: the fd may be reused.
test rax, rax
js .exit_error
mov edi, r13d
jmp .exit
.exit_error:
mov edi, 1
.exit:
mov eax, 231
syscall
```

The successful open creates one owned descriptor, kept in `r12`. A read interrupted before transferring data is retried; EOF produces a zero count and skips the write loop. Each positive write advances the buffer and reduces the remaining count. Read or write failure reaches cleanup, and both successful and failed transfers close the input before process exit. Standard output is borrowed from the caller and remains open. An unhandled `SIGPIPE` can terminate a process whose output pipe has no reader; in that case the kernel closes its descriptors during termination.

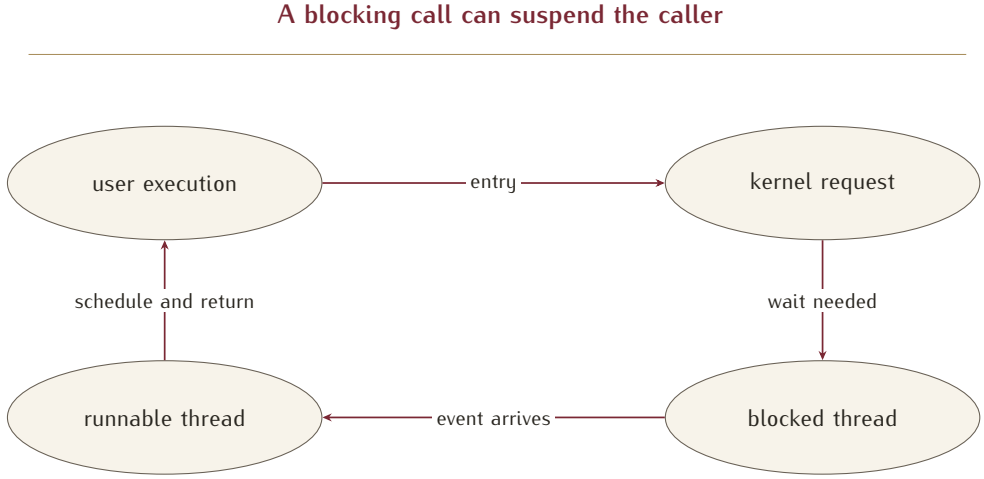


Figure 2.43: *The state transition applies to the thread that made the call. The scheduler can select other runnable work while the request waits. Event completion and actual resumption occur at different times.*

The Unix I/O sequence is governed by a small set of precise rules: `open` returns the smallest unused descriptor; `read`/`write` report the actual count, which may be short; callers loop when their contract requires more data. Descriptors 0/1/2 conventionally serve as stdin/stdout/stderr, although they can be closed or redirected. The open→read→write→exit sequence is a canonical I/O pattern, and `execve` replacing the process image is process control.

2.5.4 Mappings and process replacement

Worked example

Problem. Allocate 4096 bytes (one page) of readable, writable, executable memory.
Solution. `mmap` (9), six arguments: addr (0 = kernel chooses), size, prot (`R`=1, `W`=2, `X`=4, OR-ed), flags (`MAP_PRIVATE`=2, `MAP_ANONYMOUS`=0x20), fd (−1 anonymous), offset (0).

```
xor rdi, rdi; addr = NULL
mov rsi, 4096; size
mov rdx, 7; PROT_READ|WRITE|EXEC
mov r10, 0x22; MAP_PRIVATE|MAP_ANONYMOUS
mov r8, -1; fd
xor r9, r9; offset
mov rax, 9; mmap
syscall; rax = mapped address, or -errno
```

An RWX page lets you write bytes and then execute them. The NX bit controls executability per page; the cleaner pattern maps RW, writes code, then switches to RX with `mprotect` (10), and a `W^X` policy may forbid simultaneously-writable-and-executable pages.

Worked example

Problem. Replace the current process with a small laboratory program named `./hello` and pass an explicit argument vector.
Solution. `execve` (59) receives a pathname, a NULL-terminated array of argument pointers and an environment-pointer array. The following block is complete GNU assembler source in Intel syntax. The pathname has static storage; the pointer array is built on the stack immediately before the system call.

```
.intel_syntax noprefix
.section .rodata
path:
    .asciz "./hello"
```

Worked example (continued)

```
.section .text
.global _start
.type _start, @function
_start:
    lea rdi, [rip + path]  # pathname
    xor eax, eax
    push rax               # argv[1] = NULL
    push rdi               # argv[0] = pathname
    mov rsi, rsp           # argv
    lea rdx, [rsp + 8]     # envp points to an empty vector
    mov eax, 59            # __NR_execve
    syscall

    # Reached only if execve failed.  Return a fixed non-zero status.
    mov edi, 127
    mov eax, 60            # __NR_exit
    syscall
.size _start, .-_start
.section .note.GNU-stack,"",@progbits
```

On success `execve` does not return because the process image has been replaced. On failure the raw syscall leaves −errno in `rax`; production code may preserve or translate that error before terminating. The listing uses RIP-relative addressing, which is the normal GAS Intel form for a nearby static object.

A system call redirects an active pipeline

The `syscall` instruction transfers control to a kernel entry point.

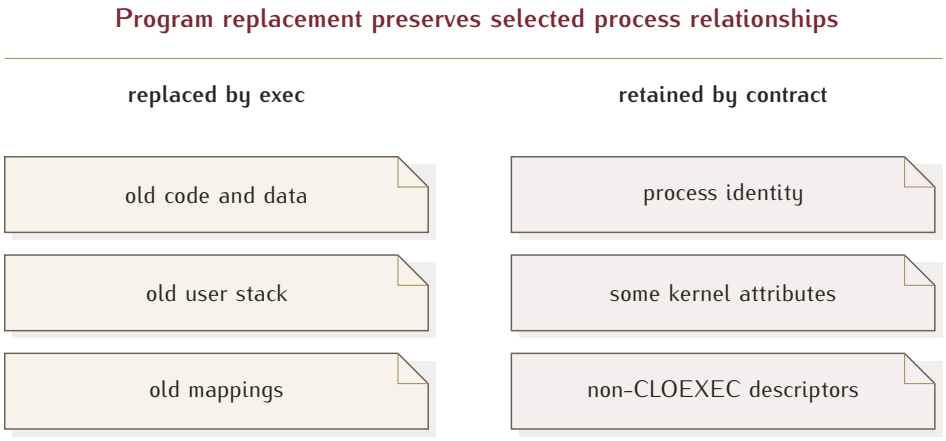


Figure 2.44: *Successful program replacement creates a new program image within the existing process. Descriptors marked close-on-exec are removed. The program cannot keep using arbitrary pointers into the old image.*

The front end redirects instruction fetch, privilege-related architectural state changes according to the ISA, and younger work is prevented from becoming committed state. The same cache and translation machinery continues to serve accesses. Depending on the operating system and mitigations, entry code may retain the current page-table context or switch to another one, as in some kernel page-table isolation designs. Software entry code then saves any additional state required by its convention.

Crossing the protection boundary can disturb prediction and translation state. The full service cost includes control-flow redirection, mitigation choices and kernel work [26, 33, 28].

2.5.5 Descriptor identity and shared open state

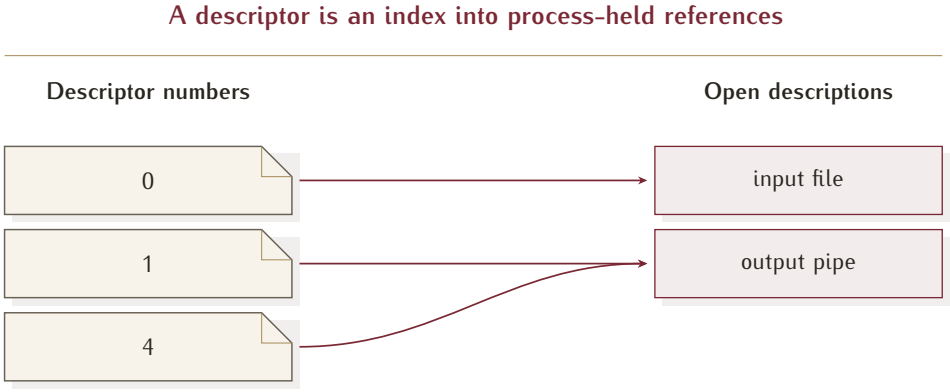


Figure 2.45: *The descriptor value selects an entry in the process's descriptor table. Duplicated descriptors can refer to one open file description and share its offset and status. The integer itself is not a memory address of the file.*

The file-descriptor interface adds another level of indirection. A file descriptor is a small integer in a process-local descriptor table; it refers to an *open file description*, the POSIX object that carries shared open-state such as the current file offset and status flags. Two descriptors can therefore be distinct integers yet refer to the same open file description, which is why a read through one duplicated descriptor can advance the offset observed through the other. Regular files, pipes, terminals, sockets and many devices can all appear behind descriptors even though their kernel implementations differ.

Operations on descriptors must also be read as protocols rather than as one-shot transfers. A successful `read` may return fewer bytes than requested, including zero at end of file; a `write` may make only partial progress; interruption and resource exhaustion can require retry or error handling. Robust code advances a buffer pointer by the number of bytes actually transferred and preserves the number still outstanding. Completion follows from the returned counts and the application's required total; the ABI specifies how each individual request is passed.

Memory mappings and program replacement expose different kernel services through the same boundary. `mmap` changes the process's virtual address-space mappings rather than copying bytes through a caller-provided buffer, while `execve` replaces the current program image without creating a new process identity. Chapter 4.2 follows the privileged entry path in detail; the concern here is the programmer-visible contract: which registers carry the request, which process resources are named, and which state remains meaningful after the service returns—if it returns at all.

Following a request into the kernel

The hardware transition leaves `rsp` unchanged and pushes no frame. Entry software supplies the stack switch and register-save image. To follow the implementation, inspect these source locations in the referenced Linux tree:

1. `arch/x86/entry/entry_64.S`: `entry_SYSCALL_64` saves the user stack pointer, changes to a kernel stack, constructs a `struct pt_regs` image and calls `do_syscall_64`.
2. `arch/x86/entry/syscall_64.c`: `do_syscall_64` performs entry bookkeeping, and `do_syscall_x64` checks the number before the generated `x64_sys_call` dispatcher selects a handler.
3. `arch/x86/entry/syscalls/syscall_64.tbl`: the table assigns numbers and entry symbols.
4. For `write`, continue into `fs/read_write.c`; the generated `__x64_sys_*` wrapper and VFS helpers eventually reach the selected file object's write operation.

Exact helper names can change between kernel versions; the architectural entry point, table, generated wrapper and subsystem implementation are the stable layers to identify [33, 34, 35].

Linux attempts a fast `sysretq` return when the restored context satisfies its safety conditions; otherwise it uses an `iretq`-based path. The source comments are valuable here: they explain why canonical-address checks, tracing state and processor errata can force the slower path.

Interpreting a short read

A request `read(fd,buf,4096)` returns 700. Exactly bytes `buf[0]` through `buf[699]` have been supplied by this call. To fill the original buffer, the next request uses `buf+700` and count $4096 - 700 = 3396$. If the application needed only one available chunk, it can process those 700 bytes immediately. A later zero return on a regular file or byte-stream EOF ends the fill attempt; a short positive count alone does not establish EOF.

Write() can finish before the physical transfer

A positive return from `write` reports how many bytes the kernel interface accepted for that call.

The kernel may copy or map data into buffers, update queues and notify a device. A storage controller, network interface or display engine can then move data over an interconnect and convert it into medium-specific signalling. Buffering separates program progress from the later device transfer, and partial writes or errors remain possible according to the interface contract.

A successful return does not generally prove that magnetic domains, flash cells or externally received symbols have already changed. Durability, delivery and ex-

ternal visibility require the guarantees of the relevant storage or communication protocol [4, 38, 47].

Worked example: detect an error from write

```
mov eax, 1; __NR_write
mov edi, 1; stdout
lea rsi, [rip+msg]
mov edx, msg_len
syscall
test rax, rax
js .error              ; negative raw return
```

Testing only for `-1` is wrong at the raw interface: different failures return different negative numbers.

Worked example: write all bytes

Inputs: descriptor in `edi`, readable buffer in `rsi`, length in `rdx`. Return 0 on completion and a negative raw error otherwise. This assembly interface also returns the first unaccepted byte address in `rsi` and the remaining count in `rdx`; callers must retain those outputs if they intend to resume.

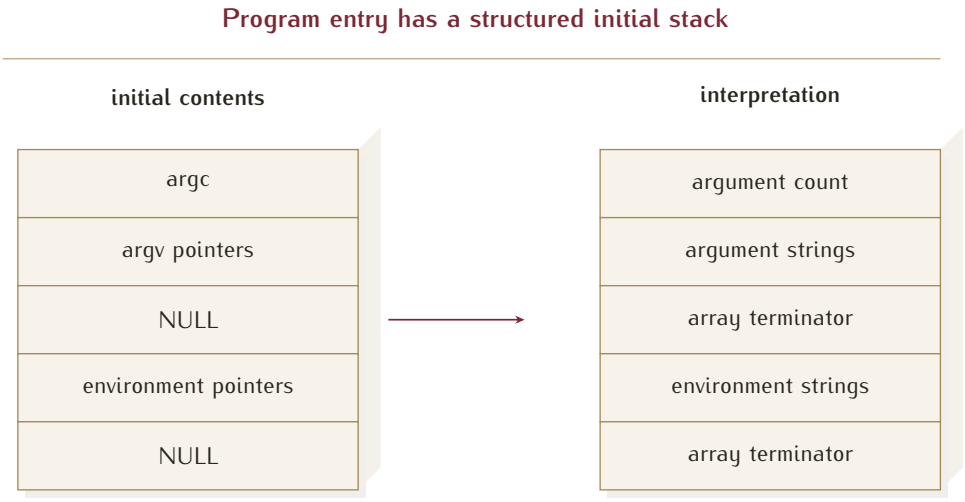
```
write_all_raw:
.write_more:
 test rdx, rdx
 je .done
 mov eax, 1
 syscall
 test rax, rax
 js .ret_error
 je .zero_progress      ; no progress: map to -EIO in this teaching wrapper
 add rsi, rax
 sub rdx, rax
 jmp .write_more
.done:
 xor eax, eax
 ret
.zero_progress:
 mov rax, -5            ; sign-extended -EIO in the 64-bit return
.ret_error:
 ret
```

The function returns `-EINTR` and `-EAGAIN` to its caller. Earlier loop iterations may already have written a prefix, so a retry must use the returned `rsi`/`rdx` pair. Reusing

Worked example (continued)

the original buffer and length could duplicate that prefix. Zero progress with bytes remaining maps to `-EIO`, preventing an infinite loop. The error must be written to `rax`: writing `-5` only to `eax` would zero-extend it to the positive value `0x00000000FFFFFFFB`.

2.5.6 From process entry to process exit



Schematic prefix of the Linux process-entry stack; auxiliary data follows.

Figure 2.46: *The initial stack provides arguments and environment through arrays of pointers. The drawing shows the ordering of these categories rather than exact addresses or every ABI field. Startup code adapts this entry state to language-level execution.*

An executable entered at `_start` does not receive C arguments in the function-call registers. The initial stack contains `argc`, pointers to argument strings, environment pointers and auxiliary-vector entries. A C runtime entry stub decodes this state, initialises libraries, calls `main`, runs termination handlers and finally requests process exit.

Worked example: a minimal position-independent hello program

GNU assembler source using Intel syntax:

```
.intel_syntax noprefix
.section .rodata
msg:
 .ascii "hello from a raw syscall\n"
.set msg_len, . - msg
```

Worked example (continued)

```
.section .text
.globl _start
.type _start, @function
_start:
 mov edi, 1
 lea rsi, [rip + msg]
 mov edx, msg_len
.write_more:
 mov eax, 1
 syscall
 cmp rax, -4             # retry EINTR
 je .write_more
 test rax, rax
 jle .failed             # error or zero progress
 add rsi, rax
 sub rdx, rax
 jne .write_more
 xor edi, edi            # complete output: status 0
 jmp .exit
.failed:
 mov edi, 1
.exit:
 mov eax, 60             # this program has one thread
 syscall
.size _start, . - _start
.section .note.GNU-stack,"",@progbits
```

Build a static position-independent executable with `gcc -nostdlib -static-pie hello.s -o hello`. The RIP-relative `lea` computes the address of `msg` without dereferencing it. The loop advances through any partial output, retries interrupted writes and returns status 0 only after the full message is accepted. Other errors or zero progress produce status 1, subject to the process's signal policy.

Worked example: copying a file safely

The following function copies bytes between already-open blocking stream or regular-file descriptors. It reports failure if an operation cannot complete. It borrows both descriptors, so their owner closes them after the function returns. Its fixed buffer bounds each read, and the inner loop consumes only bytes returned by that read:

```
#include <errno.h>
#include <unistd.h>
int copy_bytes(int in, int out) {
```

Worked example (continued)

```
      char buf[4096];
      for (;;) {
          ssize_t n = read(in, buf, sizeof buf);
          if (n < 0 && errno == EINTR) continue;
          if (n < 0) return -1;
          if (n == 0) return 0;
          ssize_t done = 0;
          while (done < n) {
              ssize_t k = write(out, buf + done, (size_t)(n - done));
              if (k < 0 && errno == EINTR) continue;
              if (k < 0) return -1;
              if (k == 0) { errno = EIO; return -1; }
              done += k;
          }
      }
  }
```

The invariant is $0 \leqslant \text{done} \leqslant n \leqslant 4096$. Each successful write strictly increases `done`; completion starts the next read. Other errors are returned with `errno` intact, possibly after a prefix has been copied. The function has no resumable state: after a failed output, some bytes already consumed from the input may remain only in the local buffer. Retrying the entire function could lose that suffix. Supporting nonblocking descriptors requires preserving the buffer and its progress between readiness events. Like ordinary `write`, the function is subject to the process's `SIGPIPE` policy.

Unix I/O

Unix made files, terminals, pipes and many devices accessible through a small set of descriptor operations. A common byte-stream interface allowed redirection and composition while each object retained its own behaviour. The shell can connect a program's standard output to a file or pipe because the program writes descriptor 1 without needing to know what object lies behind it. [19, 4]

The kernel boundary validates pointers only as user addresses with the requested permissions. It cannot infer higher-level intent. Passing a writable pointer of the wrong size, reusing a descriptor after closing it, or ignoring a short read remains a user-space bug. Libc wrappers generally cannot infer object capacity either. Both interfaces require the caller to provide valid bounds.

Laboratory 13: Partial I/O

Write a complete C program that reads at most 16 bytes from standard input and echoes exactly those returned, handling interruptions and short writes. State outcomes for empty input, `abc`, and twenty input bytes.
Worked solution

Laboratory 13: solution

The buffer's valid interval is determined by the returned count. This implementation makes one successful read, then completes that read's output:

```
#include <unistd.h>
#include <errno.h>
int main(void) {
    char b[16]; ssize_t n;
    do { n=read(0,b,sizeof b); } while(n<0 && errno==EINTR);
    if(n<0) return 1;
    ssize_t off=0;
    while(off<n) {
        ssize_t k=write(1,b+off,(size_t)(n-off));
        if(k<0 && errno==EINTR) continue;
        if(k<=0) return 1;
        off+=k;
    }
    return 0;
}
```

An empty file gives n=0 and no output. A three-byte regular file containing `abc` gives three output bytes in the ordinary complete-read case. A twenty-byte regular file permits the initial 16 bytes to be returned; the remaining bytes are outside this program's one-read task. Other streams can return fewer than requested. `strace -e trace=read,write,exit,exit_group ./echo16` exposes the actual counts. Passing 16 unconditionally to write after a short read would output bytes that the read did not initialise.
Return to the laboratory

From supervisor transfers to the x86-64 Linux system-call ABI

Timeline

Year	Machine, standard or event	Architectural significance
1950s	Supervisor calls	User jobs request privileged monitor services through controlled entry.
1969	UNIX system calls	Processes receive a compact file/process service interface.
2003	x86-64 SYSCALL ABI	Fast hardware entry and a register contract support modern Linux.

The programmer-visible system call is the descendant of monitor and supervisor requests developed when computers became shared facilities. A 1950s batch monitor loaded jobs and centralised operations such as input/output. Hardware protection then made it possible to keep the monitor resident while preventing a user job from overwriting it. The missing piece was a legal transfer by which the less privileged program could ask for a service [35, 33, 32].

IBM System/360 supplied the `SVC` instruction for this purpose in 1964. Executing it caused a supervisor-call interruption; an immediate field identified a service convention chosen by the operating system. The processor saved an old Program Status Word and loaded a new one, changing control state under architectural rules. Atlas used extracodes interpreted by its supervisor. Multics developed protected procedure calls within a ring-based system. These mechanisms established the division still visible today: hardware supplies a protected entry primitive, while the operating system defines service numbers, argument rules and object semantics [109, 94, 60].

UNIX reduced the service vocabulary to a small set of composable abstractions. In the system described by Ritchie and Thompson in 1974, a process read and wrote file descriptors that could represent files, terminals or pipes. `fork` created a process and `exec` replaced its program image. The interface was exposed through C functions, but a subset crossed into the kernel through machine-dependent trap code. Linux adopted the Unix model from 1991 onward and has preserved old system calls where binary compatibility requires them while adding new variants for flags, large files, namespaces, time widths and security [19, 35].

On 32-bit x86 Linux, `int $0x80` became the classic entry. The system-call number was in EAX, and arguments were passed in a Linux-specific register order. Because a general interrupt gate performs several protected-mode actions, vendors introduced faster entry instructions. Intel's `SYSENTER/SYSEXIT` appeared with Pentium II-era processors; AMD64 defined `SYSCALL/SYSRET`. In 64-bit Linux, `SYSCALL` is the normal native entry, although compatibility paths remain for older ABIs [26, 27].

The machine ABI and the C declaration specify different parts of the interface. A C declaration such as `ssize_t write(int, const void *, size_t)` determines types and source-level use. The libc wrapper applies the calling convention for an ordinary

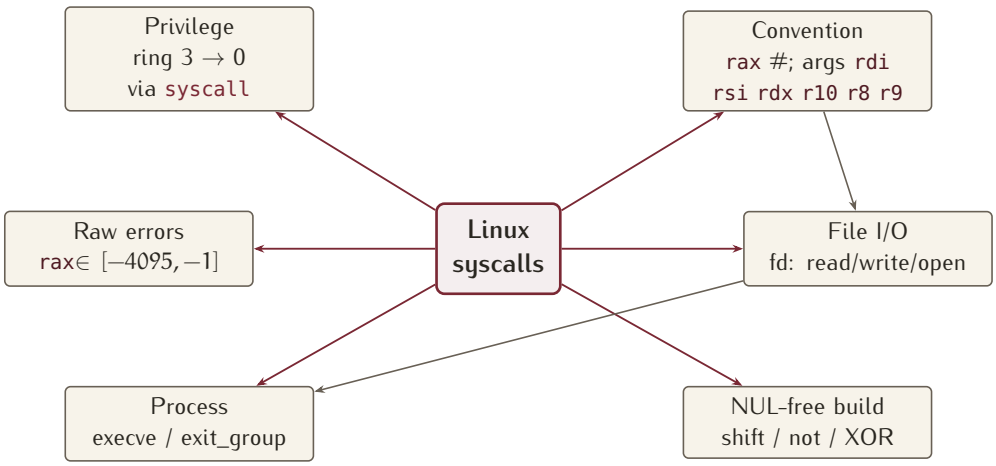
function, arranges the kernel register convention and handles errors. Direct assembly bypasses the wrapper and must obey the kernel ABI exactly. It must also handle short reads and writes, interruption and resource lifetime; merely issuing the instruction is not a complete file-copy program [35, 32, 33].

Each call has history embedded in its shape. `open` was supplemented by `openat` and `openat2` to express directory-relative resolution and extensible policy. Time-related calls gained 64-bit forms on 32-bit architectures to pass the 2038 boundary. `mmap` reflects virtual-memory systems in which files and anonymous pages enter the address space. `clone` exposes Linux process and thread creation options, while POSIX threads normally reach it through a library. New interfaces tend to include a size or flags field so future kernels can extend them without inventing another incompatible layout [35, 32, 33].

The kernel treats every pointer as untrusted. A user address may be unmapped, cross a page boundary, race with another thread or refer to memory whose access is not permitted. Linux uses user-access helpers and fault-handling conventions rather than blindly dereferencing it as a kernel pointer. File descriptors are looked up in the calling process's table; credentials, namespaces and Linux Security Modules influence permissions. A syscall therefore follows a validation and object-lookup protocol before reaching the relevant subsystem [35, 32, 33].

Some operations avoid or amortise entry. The vDSO lets selected queries execute in userspace using kernel-maintained data. `io_uring`, introduced in 2019, submits and completes operations through shared rings, reducing repeated crossings and allowing asynchronous execution. The kernel validates queue state and performs the requested effects across the same privilege boundary. Seccomp can filter calls by number and arguments; containers change namespace views but use the same kernel entry [35, 32, 33].

Concept map: system calls



Reading the map. A syscall is a controlled ring 3→0 transition with its own register convention; the result comes back in `rax` (the reserved negative range reports errors).

On top of that sit the file-I/O syscalls (read/write/open over descriptors) and process control (execve/exit), while Chapter 2.4 explains additional encoding constraints when a particular input channel imposes them.

2.5.7 Worked problems

Write stdout

Question. Which Linux x86-64 syscall registers are required to request a write of a buffer to standard output?
Set `rax`=1 for `write`, `rdi`=1 for standard output, `rsi` to the buffer address and `rdx` to its byte count, then execute `syscall`. For a five-byte object named `msg`:

```
mov eax, 1
mov edi, 1
lea rsi, [rip + msg]
mov edx, 5
syscall
```

The object must supply five readable bytes for the duration of the call. On return, `rax`=5 establishes completion; a positive smaller value identifies a written prefix, and a negative value reports a raw error. The request count describes the intended transfer, and the returned count establishes what happened.

Read then echo

Question. Read up to N bytes from standard input and echo exactly those returned, handling interrupted reads and partial writes.
The following fragment starts with `rsi` pointing to a writable N-byte buffer and `rdx`=N. The labels `.done` and `.error` hand control back to the enclosing program. The output must remain readable throughout the write loop.

```
.read_again:
xor edi, edi
xor eax, eax
syscall
cmp rax, -4          # EINTR
je .read_again
test rax, rax
js .error
je .done             # no returned bytes
mov rdx, rax         # actual initialized count
mov edi, 1
.write_more:
mov eax, 1
syscall
```

Worked example (continued)

```
cmp rax, -4
je .write_more
test rax, rax
jle .error          # error or zero progress
add rsi, rax
sub rdx, rax
jne .write_more
jmp .done
```

If a request for 100 bytes returns 7, the first write requests 7. If that write accepts 3, the next begins at `buffer+3` with count 4. At each step, the accepted prefix plus the remaining suffix equals the seven bytes supplied by the read. An error stops the transfer after any progress already made. The process's `SIGPIPE` policy also applies if standard output is a pipe.

Exit status

Question. How is a Linux x86-64 process exit status passed to the kernel and later observed by a parent or shell?

For process-wide termination, set `rax=231` for `exit_group` and pass the requested status in `rdi`. Raw `exit`, number 60, terminates only the calling thread. Neither returns on successful termination.

The parent obtains an encoded wait status that can describe either normal termination or a signal. After `WIFEXITED(status)` confirms normal termination, `WEXITSTATUS(status)` extracts the low eight bits of the requested value. A conventional shell exposes that value as the command status.

For an argument of 300, the low eight bits are $300 \bmod 256 = 44$, so the parent extracts 44 and the shell reports 44. The original argument, encoded wait word and extracted exit code are three representations of this termination event.

Partial result

Question. A write request for 100 bytes returns 40. What happened, and how does the caller complete the transfer?

The positive return establishes that 40 bytes were accepted. The remaining suffix has length $100 - 40 = 60$ and begins at `buffer+40`. The next request must use that pointer and count. Repeating the original request would send the first 40 bytes again.

Continue updating the pointer and count after each positive return until the remaining count reaches zero. Negative returns follow the error policy, and a zero return with bytes remaining must terminate or otherwise make progress possi-

Worked example (continued)

ble. With a request of 512 bytes and a return of 500, the same calculation gives `buffer+500` and 12 bytes remaining.

Exercise 2.5.1 — Argument register

Which register carries the fourth syscall argument?

Worked solution

The fourth Linux x86-64 syscall argument is passed in `r10`. The complete order is `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, with the syscall number in `rax`. Ordinary function calls use a different fourth-argument register. A wrapper adapting a fourth argument received in `rcx` must move it to `r10` before `syscall`, which uses `rcx` for its own control-transfer state.

Exercise 2.5.2 — EOF

What does read returning zero mean?

Worked solution

For a positive-size `read` request, a successful return of zero means that no bytes were read because the stream is at end of file in the usual file/stream model. The caller must stop treating the buffer as newly filled. A positive return counts bytes actually read; a negative raw syscall result denotes an error. A request with count zero can also return zero without testing for EOF, so the positive-count assumption is essential when using this return value as an EOF test.

Exercise 2.5.3 — Write count

Which register supplies write count?

Worked solution

The count argument for Linux x86-64 `write(fd,buf,count)` is the third syscall argument, so it belongs in `rdx`. The descriptor belongs in `rdi`, the buffer address in `rsi`, and syscall number 1 in `rax`. If the request is to write five bytes, set `rdx`=5. After the syscall, inspect `rax` to learn how many bytes were actually accepted; the requested count alone does not establish completion.

Exercise 2.5.4 — Short write

May write return less than count without a negative error?

Worked solution

A successful `write` may return a positive value smaller than the requested count. Suppose 10 bytes were requested and the return is 4. Four bytes have been accepted, so advance the buffer address by 4 and reduce the remaining count to 6 before retrying. Preserve a separate error policy for negative results and prevent an endless loop on zero progress. Retrying the original buffer and full count would duplicate the prefix already written.

Summary

Trace a system call on two planes. The register plane explains how the request is encoded. The operating-system plane explains validation, blocking, resource ownership and return. Code that records only the first plane may be syntactically correct yet semantically fragile, especially around errors, partial reads and pointer lifetimes.

2.5.8 Exercises

Exercise 2.5.5 — Syscall registers

List the Linux x86-64 registers for syscall number and first six arguments. Which two registers are clobbered by `syscall` beyond the return register?

Go to the solution →

Exercise 2.5.6 — Write a message

Give the four register assignments and instruction needed to write 5 bytes from address `msg` to standard error.

Go to the solution →

Exercise 2.5.7 — Raw error check

Why is `cmp rax,-1; je error` insufficient after a raw syscall? Give the correct generic test.

Go to the solution →

Exercise 2.5.8 — Echo the returned count

A `read` requests 64 bytes and returns 17. Which value must be placed in `rdx` for the following `write`, and why?

Go to the solution →

Exercise 2.5.9 — Short write loop

A write of 100 bytes returns 40. State the next buffer pointer and remaining count.

Go to the solution →

Exercise 2.5.10 — Syscall ABI versus function ABI

A four-argument C function receives its fourth integer argument in which register? A four-argument syscall uses which register? Explain the difference.

Go to the solution →

Exercise 2.5.11 — Diagnose a position bug

A GNU assembler Intel-syntax non-PIE example uses `movabs rsi,offset msg` to load the address of a local symbol and works, but a position-independent build either needs a text relocation or fails under the chosen link model. Diagnose why the absolute address is the problem, give a RIP-relative replacement, and explain what changes in the relocation/linking requirements.

Go to the solution →

2.5.9 Solutions

Linux System Calls and the User-Kernel Boundary

Solution 2.5.5

Number in `rax`; arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. The transition clobbers `rcx` and `r11`; the result is in `rax`.

← Return to the exercise

Solution 2.5.6

Linux x86-64 uses syscall number 1 for `write`. Descriptor 2 selects standard error; the second argument is the address of the bytes and the third is their count:

```
mov eax, 1
mov edi, 2
lea rsi, [rip + msg]
mov edx, 5
syscall
```

This requests five bytes beginning at `msg`. The memory must contain at least those five readable bytes. The return in `rax` must be checked: a positive short result requires another write for the remaining suffix, and a negative raw result reports an error.

← Return to the exercise

Solution 2.5.7

Linux raw errors occupy $[-4095, -1]$, so testing only -1 misses errors such as -2 and -13. Use `cmp rax,-4095; jae error`: under unsigned comparison, precisely those 4095 highest 64-bit patterns fall in the reserved error range. For a syscall whose successful values are nonnegative, a sign test also suffices. The reserved-range test is the generic Linux rule.

← Return to the exercise

Solution 2.5.8

Use 17, copied from the read return value. Only those bytes were initialised by this read; writing 64 could expose stale buffer contents and violates the I/O contract.

← Return to the exercise

Solution 2.5.9

Let the initial buffer address be B. The accepted prefix occupies $[B, B + 40)$, so the remaining suffix starts at $B + 40$ and contains $100 - 40 = 60$ bytes:

```
add rsi, 40
mov edx, 60
```

More generally, after a positive count in `rax`, use `add rsi,rax` and `sub rdx,rax`. The invariant is that accepted bytes plus remaining bytes equal the original count. Repeat for the suffix until the remainder is zero; check negative returns before subtracting them and handle zero progress to avoid an endless loop.

← Return to the exercise

Solution 2.5.10

The System V function ABI uses `rcx`; the Linux syscall ABI uses `r10`. `syscall` itself overwrites `rcx`, so the kernel entry convention cannot keep argument four there.

← Return to the exercise

Solution 2.5.11

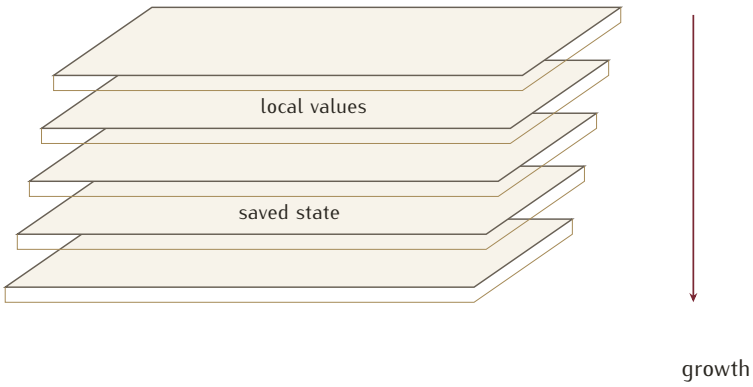
The problematic instruction embeds or requests an absolute address for `msg`; moving the image therefore requires that address to be fixed up, and some position-independent link models deliberately reject such text relocations. A typical replacement is `lea rsi,[rip+msg]`, which forms the address from the instruction pointer plus a link-time displacement. The linker still resolves that displacement, but the loaded image no longer needs an absolute code address patched merely because ASLR chose a different base. The precise relocation type depends on the assembler and on symbol visibility, so RIP-relative addressing is the architectural technique rather than a promise that no relocation record ever exists.

← Return to the exercise

2.6 PROCEDURES, CONTROL FLOW, AND REVERSE ENGINEERING

An instruction transfers control to a function, and execution later returns to the instruction after the call. During that interval, arguments must remain identifiable, preserved registers must be restored and temporary storage must stay valid for its intended lifetime. A calling convention coordinates those obligations. Following one call and its return provides a method for reading compiled functions whose original source is unavailable.

A procedure leaves a recoverable continuation



Layered sheets represent a conventionally organised stack.

Figure 2.47: *The layered sheets represent one conventional arrangement of stack storage. Calls, prologues and returns manipulate distinct parts of that arrangement. Exact addresses and preserved registers come from the selected ABI and compiled function.*

2.6.1 The function-call contract

Syscalls follow a convention (six argument registers, `syscall`, `rax` for the return). C functions on the platform used here follow a different contract, the *System V AMD64 ABI [30]*. A calling convention fixes: where arguments go (registers or stack); who cleans the stack; which registers the callee may clobber (*caller-saved*) and which it must

preserve (*callee-saved*); where the return value goes; and how variadic functions work. A shared convention is the prerequisite for separately compiled code, dynamic libraries, and debugging to interoperate.

ABI vs API. The *API* is the language-level interface: function names and argument types (`int open(const char*, int, ...)`). The *ABI* is the machine-level contract: how arguments reach registers/stack, how the return value is encoded, what happens to registers across a call. Two programs can share an API but differ in ABI (x86-64 vs ARM64, or System V vs Microsoft x64). A binary library requires a compatible ABI or an explicit adaptation layer.

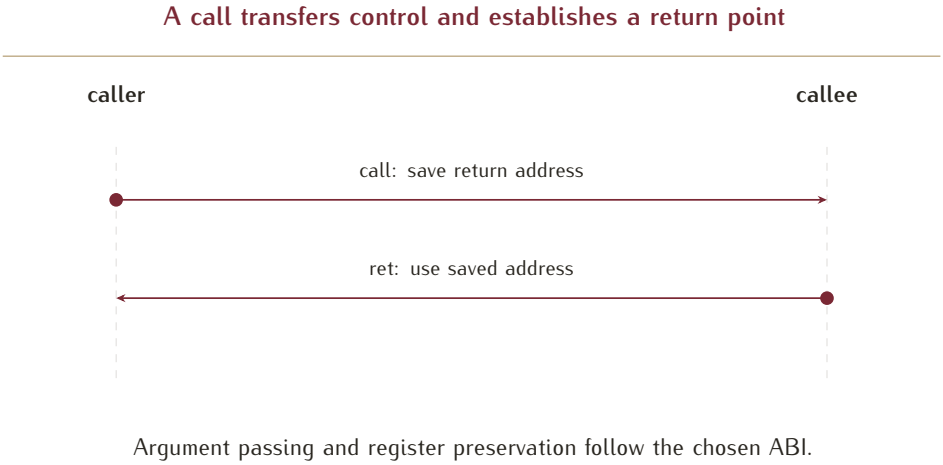


Figure 2.48: *The control transfer and its saved return point are only part of the procedure contract. Caller and callee must also agree about argument locations, result locations, stack alignment and preserved registers.*

For the simple scalar signatures used below, integer and pointer arguments of at most 64 bits use `rdi`, `rsi`, `rdx`, `rcx`, `r8` and `r9` in order. Scalar `float` and `double` arguments use `xmm0`–`xmm7` through a separate register count. Additional arguments use the stack; for a sequence of ordinary 64-bit integers, the seventh begins at `[rsp+8]` on entry. Scalar integer results use `rax` or its required subregister, and `float`/`double` results use `xmm0`. Read each integer argument and result at its declared width; unused high register bits do not generally supply an extension of a narrower value.

Caller-saved general-purpose registers are `rax`, `rcx`, `rdx`, `rsi`, `rdi` and `r8`–`r11`. The callee preserves `rbx`, `rbp` and `r12`–`r15`, and restores the stack pointer before returning. For these scalar calls, `rsp` is 16-byte aligned before `call`; after its eight-byte return-address push, the callee enters with `rsp`$\bmod 16 = 8$. Wider vector or over-aligned stack arguments can require stronger alignment. The 128-byte user-space red zone is temporary storage below `rsp`, usable only for values that do not need to survive an ordinary nested call. Aggregate and extended floating-point cases require the classification rules discussed later.

Register preservation divides responsibility

	class	responsibility
caller-saved	temporary values	caller saves if needed
callee-saved	preserved values	callee restores if used

Specific register sets belong to a particular ABI.

Figure 2.49: *A caller cannot assume that a caller-saved value survives an arbitrary call. A callee that changes a callee-saved register must restore it before returning. This division permits separate compilation.*

2.6.2 Stack frames and control transfer

`call target` pushes the address of the next instruction, then jumps to `target`. `ret` pops a value and jumps to it. So `call` records "where to return" and jumps; `ret` retrieves it and jumps back. If the stack is corrupted and the return address overwritten, `ret` jumps elsewhere, which is the basis of the buffer-overflow exploit (2.7).

```
foo:
 push rbp; save caller's frame pointer
 mov rbp, rsp; new frame pointer
 sub rsp, 0x20; 32 bytes of locals;... body; rdi/rsi args; rax return...
 leave; mov rsp, rbp; pop rbp
 ret
```

With a frame pointer, locals are commonly addressed at negative offsets from `rbp`, such as `[rbp-8]` and `[rbp-16]`. With `-fomit-frame-pointer`, the compiler omits this stable base and addresses locals relative to `rsp`, freeing one register. The two forms are normally distinguishable in disassembly.

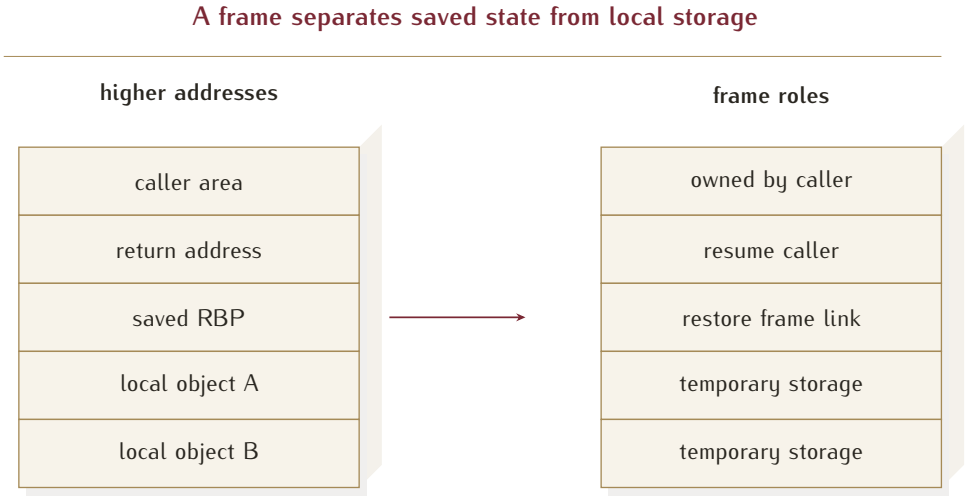


Figure 2.50: *The schematic illustrates a frame-pointer-based prologue. A source local does not have a universal offset, and a compiler can eliminate or rearrange storage. The disassembly and ABI determine the actual frame.*

Following the return through the same frame

In Figure 2.51, suppose the body leaves 42 in `rax` and reaches `leave; ret`. Which locations does the epilogue read, and where does `rsp` finish?

Solution. `leave` first assigns `rsp=rbp=0x7FF0`. Its pop reads the saved frame pointer there, restores `rbp`, and advances `rsp` to `0x7FF8`. The following `ret` reads the return address from `0x7FF8` and advances `rsp` to `0x8000`. Control resumes after the call and the scalar return value remains 42 in `rax`. Neither instruction erases the local bytes; their former storage has simply left this active frame.

The stack is usually ordinary cached memory

A call pushes a return address and a function uses a stack frame.

Stack bytes are normally stored in the same cache hierarchy as other data. Because stack accesses cluster near the current stack pointer, recently used frames often remain in L1 data cache. Store buffers and forwarding paths can supply a just-written stack value before it has reached the cache array.

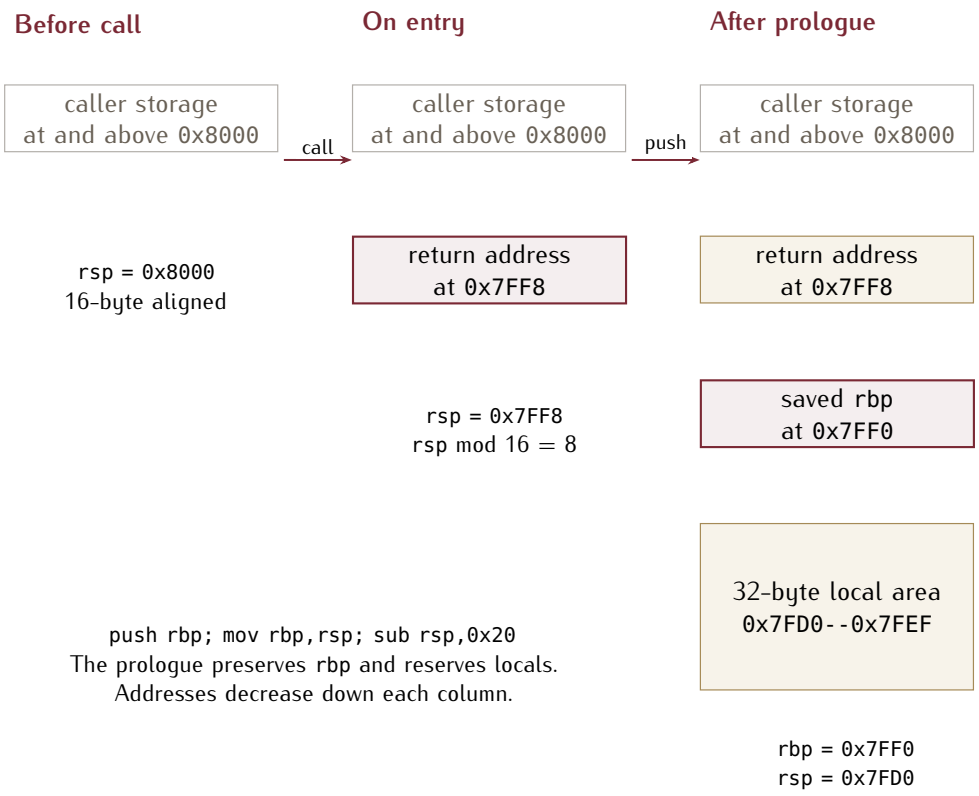


Figure 2.51: *Three snapshots of the same scalar System V AMD64 call. Each labelled return-address or saved-register slot is eight bytes. The local area is 32 bytes; its drawn height is schematic. The call instruction saves only its return address. The prologue performs the remaining operations explicitly.*

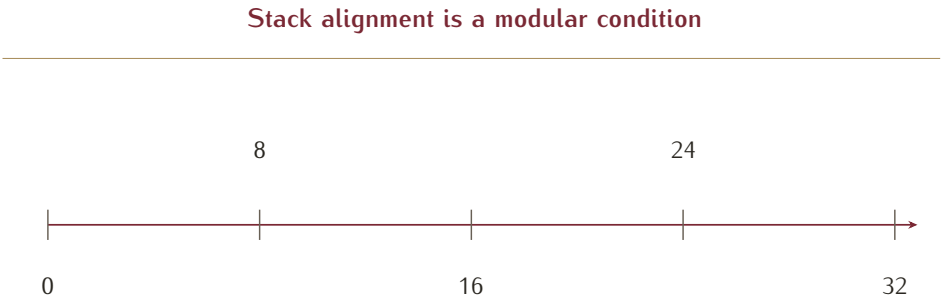


Figure 2.52: *The alignment requirement concerns the address modulo 16, independent of the stack's absolute location. A call that pushes an eight-byte return address changes that remainder. Prologue adjustments must account for the entry convention.*

Stack-operation cost depends on cache residency, dependencies and address-generation resources. Deep recursion can eventually exceed cache or mapped-stack capacity, turning a regular access pattern into misses or faults [28, 47].

Returns are predicted with a finite hardware stack

`ret` obtains its destination from the software stack.

Many processors also maintain a small return-address prediction structure, commonly described as a return stack buffer. A call records a predicted return location; a return consumes it so instruction fetch can continue before the architectural stack load resolves. Deep, unusual or unbalanced call patterns can exceed or confuse this finite structure.

Correctness still comes from the architectural stack, but performance and some speculative-security behaviour depend on a separate hidden predictor. One software abstraction can therefore be represented by multiple physical structures with different roles [28, 50].

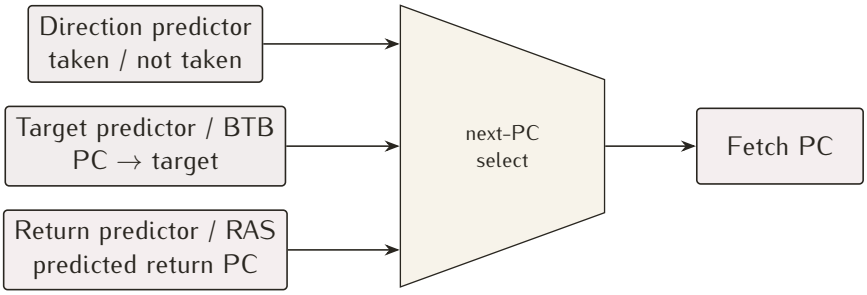
2.6.3 Control flow inside a function

Machine-level control flow uses three families: unconditional jumps such as `jmp`; conditional jumps selected from the signed or unsigned families; and procedure transfers through `call` and `ret`. The instructions `cmp` and `test` set flags without changing their operands. Common mnemonics after `cmp a, b` are listed below.

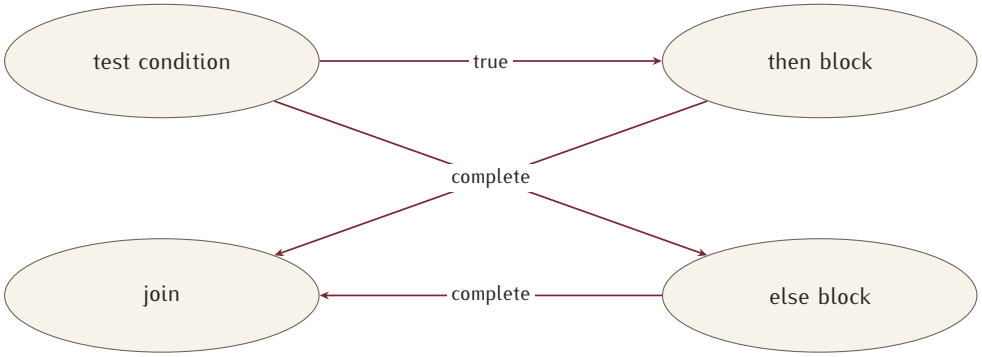
Mnemonic	Meaning	Jumps if
`je`/`jz`	equal	$a = b$
`jne`/`jnz`	not equal	$a \neq b$
`jg`	greater (signed)	$a > b$
`jge`	$\geqslant$ (signed)	$a \geqslant b$
`jl`	less (signed)	$a < b$
`jle`	$\leqslant$ (signed)	$a \leqslant b$
`ja`	above (unsigned)	$a > b$
`jae`	$\geqslant$ (unsigned)	$a \geqslant b$
`jb`	below (unsigned)	$a < b$
`jbe`	$\leqslant$ (unsigned)	$a \leqslant b$

The classic confusion is `jg` vs `ja`: both mean "greater", but `jg` is signed ($-1 < 1$) and `ja` unsigned (-1 is `0xFF...FF`, the largest). Using one for the other misbehaves near the sign bit.

Branch predictor and Return Address Stack. Branches are the natural brake on a pipeline: the CPU cannot know what to fetch after a `jne` until the condition is computed. Several cooperating prediction structures hide this:



A conditional branch selects a successor



The two paths rejoin with state produced by the selected block.

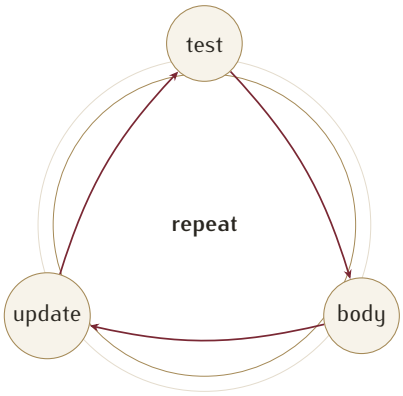
Figure 2.53: *Control flow records which instructions may execute next. Reconstructing a condition requires following the flags or value tested by the branch. Both incoming paths matter when reasoning about values at the join.*

The current fetch context indexes prediction structures; the exact indexing and update points are implementation-specific. A direction predictor estimates whether a conditional branch will be taken. A target predictor such as a BTB supplies likely destinations for branches that transfer control away from the fall-through address. A return predictor commonly uses a small hardware stack updated speculatively around calls and returns. These structures are implementation details: their capacities, update rules, recovery behaviour and penalties are not fixed by the ISA. A wrong prediction discards younger speculative work and redirects fetch; the cost depends on pipeline depth and on when the branch resolves. The ROP chains of 2.7 rely on the architectural rule that `ret` obtains its target from the software-visible stack. Return prediction influences speculative fetching before that target is confirmed.

Branch prediction uses physical tables during instruction fetch

A conditional jump chooses one of two next instruction addresses.

A loop revisits code while its data changes



The exit edge leaves the cycle when the test fails.

Figure 2.54: *The cycle shows the repeated path of a simple test-before-body loop. Each traversal can use different register and memory values. An execution trace must distinguish repeated dynamic instances of the same static instruction.*

To keep an active front end supplied, many processors consult several tables containing branch history, target information and confidence-like state. These structures are commonly SRAM-like arrays and associated logic that are read and updated frequently, although not necessarily on every clock cycle or in every power state. Prediction chooses a fetch path before the condition is known; a later mismatch cancels younger work and redirects fetch.

Code layout and branch regularity influence predictor state, wasted switching and pipeline occupancy. These implementation effects occur while preserving the branch instruction's architectural result. Predictor state can also become part of a timing side channel [50, 28].

`cmov` conditionally updates a register destination from a register or memory source (`cmove rax, rdi` copies if ZF= 1). Compare the two idioms for "if a > b then c = x else c = y":

```
mov rcx, rsi; c = y (default)
cmp rax, rbx
cmovg rcx, rdi; if a > b, c = x
```

A conditional move can avoid a control-flow dependency, but it executes its data-producing path and introduces a data dependency. It is advantageous only when those costs are lower than the expected branch behaviour on the target.

A memory-source `cmov` can read its source even when the condition is false, so it cannot guard an invalid address. For constant-time cryptography, `cmov` helps avoid a secret-dependent branch (whose timing could leak the secret), but it is not sufficient alone: memory accesses must also be secret-independent, and the generated code must be checked, since the compiler may reintroduce a branch. Conditional moves can avoid some branch-misprediction costs, although constant-time code imposes broader requirements.

Worked example

Problem. `rdi` holds a signed 64-bit value. Leave 7 in `rax` if that value is greater than zero, otherwise -7.
Solution. With branches: `test rdi, rdi; jle .neg; mov rax, 7; jmp .end; .neg: mov rax, -7; .end:`. Branchless with `cmov`:

```
mov rax, 7
mov rcx, -7
test rdi, rdi
cmovle rax, rcx
```

Worked example

Problem. Compute $\sum_{i=0}^{n-1} i$; n in `rdi`, result in `rax`.
Solution.

```
xor rax, rax          ; sum = 0
xor rcx, rcx          ; i = 0
.loop:
cmp rcx, rdi
jae .end
add rax, rcx
inc rcx
jmp .loop
.end:
```

Closed form. The mathematical sum is $n(n-1)/2$. To reproduce the loop's low 64-bit result for every unsigned n, preserve the full product before dividing:

```
mov rax, rdi
dec rax
mul rdi                 # full unsigned product in rdx:rax
shrd rax, rdx, 1        # low 64 bits after dividing by two
```

For $n = 0$, multiplication by zero gives zero despite the preceding decrement wrapping. For positive n, one of $n, n-1$ is even, so the right shift performs exact integer division. This constant-length sequence avoids the loop while preserving its modulo-2^{64} result.

Caution

Truncating the product before the shift can lose a bit that belongs in the final sum. For $n = 2^{32} + 1$, the full product is $2^{64} + 2^{32}$ and the sum is $2^{63} + 2^{31}$, which fits in unsigned 64 bits. A low-half `imul` followed by `shr` would instead produce only 2^{31}. If the contract requires an unbounded mathematical sum to fit rather than a modular result, also test the high half after division: the result fits exactly when the original product's `rdx` is below 2.

2.6.4 Reading compiled functions

Worked example

Problem. What does `foo` compute for a nonnegative signed argument `x` in `rdi`? Assume enough stack space for all recursive calls, and state when the mathematical result fits in 64 bits.

```
.intel_syntax noprefix
.text
.globl foo
.type foo, @function
foo:
 push rbp
 mov rbp, rsp
 sub rsp, 16           # reserve an aligned local area
 mov [rbp-8], rdi      # x = argument
 mov rax, [rbp-8]
 cmp rax, 0
 jne .else             # if x != 0, recurse
 xor eax, eax          # base case: return 0
 jmp .end
.else:
 mov rax, [rbp-8]
 sub rax, 1
 mov rdi, rax          # argument = x - 1
 call foo              # rax = foo(x - 1)
 mov rdi, [rbp-8]      # reload the preserved x
 add rax, rdi          # rax = foo(x - 1) + x
.end:
 leave
 ret
.size foo, .-foo
```

Worked example (continued)

Solution. Reading block by block: the prologue reserves 16 bytes, so the local at `[rbp-8]` is not overwritten by the return address pushed by the recursive `call`; the same allocation also preserves the required stack alignment before that call. For the stated precondition $x \geqslant 0$, the function returns 0 when $x = 0$ and otherwise returns $\text{foo}(x-1)+x$. By induction, the mathematical sum is $\sum_{i=0}^{x} i = x(x+1)/2$. The assembly retains its low 64 bits. The unsigned mathematical value fits exactly when $x(x+1)/2 \leqslant 2^{64}-1$; a signed 64-bit interpretation requires the stronger bound $x(x+1)/2 \leqslant 2^{63}-1$. These products are mathematical expressions used to state the bounds, so an implementation must avoid overflowing while testing them. Negative inputs fall outside the contract. The recursive depth is $x+1$, making stack capacity a separate limit even for inputs whose sum fits.

General technique: identify prologue/epilogue, track `[rbp-N]` (locals) and `[rbp+N]` (stack args), read the conditional jumps to draw the control-flow graph, derive each branch, recompose. Small functions go by hand; larger ones warrant a decompiler (Ghidra, IDA, Binary Ninja). The reconstruction reverses the usual translation from a high-level procedure to machine code.

Worked example

Problem. `int sum3(int a, int b, int c)` returns $a+b+c$. Call it with 10, 20, 30; leave the result in `rbx`. Assume the surrounding caller has already aligned `rsp` and permits this fragment to change `rbx`.

Solution.

```
mov edi, 10; int args use 32-bit sub-registers
mov esi, 20
mov edx, 30
call sum3; eax = sum3(10, 20, 30)
movsxd rbx, eax; sign-extend the int result into rbx
```

For more than six ordinary integer arguments, the remaining values occupy the caller-allocated stack argument area, with the earlier stack argument nearest the return address. A caller can construct this layout with stores or reverse-order pushes, adding padding where needed for pre-call alignment. The caller reclaims that area after the call.

After a call, caller-saved registers can contain new values, including the documented result registers. A caller that needs an old value must preserve it before the call. An ABI-conforming callee restores every callee-saved register that it changes.

Worked example

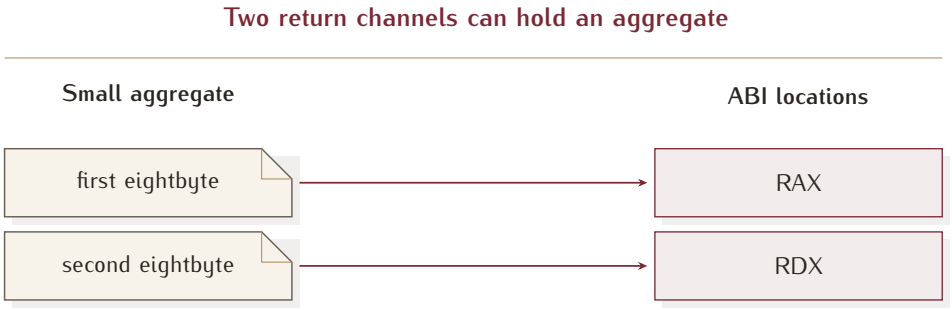
Problem. Write a callable function that invokes `f(x)` twice with the same original argument `x` from `rdi`. Return the second result.

Solution. `rdi` is caller-saved, so move it into a callee-saved register:

```
push rbx; we must preserve rbx for our caller
mov rbx, rdi; survives across calls
call f
mov rdi, rbx
call f
pop rbx
ret
```

"push `rbx` on entry, pop on exit" is the callee-saved contract for the function we are writing.

2.6.5 Aggregate arguments and variadic calls



Illustrative two-INTEGER-class return under the System V AMD64 ABI.

Figure 2.55: *For an aggregate classified into two INTEGER eightbytes and eligible for register return, RAX and RDX carry the result. Other classifications and sizes use different rules, so field size alone is not a universal predictor.*

The scalar register sequence is a special case of the ABI's classification algorithm. Register preservation remains a software obligation: the caller saves live caller-saved values, and the callee restores incoming callee-saved values that it changes. The hardware call instruction does neither task automatically.

Each argument or return object is divided into eightbytes and assigned classes such as INTEGER, SSE, SSEUP, X87 or MEMORY. Small structures may therefore travel partly or wholly in registers. An object classified MEMORY is passed on the stack; a large return value is commonly produced in caller-provided storage, whose hidden address

occupies the first integer-argument position. This is why adding a field to a structure can change a function's binary interface even when its source-level name is unchanged.

For example, an otherwise ordinary `struct pair {long n; double x;}` has one INTEGER eightbyte and one SSE eightbyte. When it is the sole argument, its two fields use `rdi` and `xmm0`; a return of that type uses `rax` and `xmm0`. A three-`long` structure instead needs memory under these ordinary aggregate rules. Register availability and the full classification algorithm must be considered for more complex signatures.

For a variadic call, `al` carries an upper bound on the number of vector registers used for arguments. C's default argument promotions also apply to the unnamed arguments: a `float` becomes `double`, and narrower integer types undergo integer promotion. To call `printf("%g\n",3.5)`, pass the format address in `rdi`, the binary64 value 3.5 in `xmm0`, and set `eax`=1 before the call. A call with only integer arguments can set `eax`=0. In both cases the caller must align the stack and preserve any live caller-saved values. The direction flag must also be clear on function entry and return, so ordinary string operations can assume forward direction [30].

Preserving a value across a call

A function receives x in `rdi` and must return $x + f(x)$ modulo 2^{64}. It uses `rbx` to keep x across the nested call:

```
push rbx
mov rbx, rdi
call f
add rax, rbx
pop rbx
ret
```

On entry $\texttt{rsp} \bmod 16 = 8$; the push restores pre-call alignment. Function `f` must preserve `rbx`, so the original x remains available after it returns. The pop restores this function's caller's `rbx`. If $x = 7$ and $f(7) = 10$, the result is 17.

Triangulating source, disassembly and execution

A disassembly describes the instructions in one compiled artifact. Optimisation can remove variables, merge blocks, invert conditions, use conditional moves, unroll loops or keep values entirely in registers [1]. Reverse engineering is therefore strongest when three views agree.

1. The source or specification states the intended computation.
2. The disassembly states the instructions and control-flow graph of a particular binary.
3. A debugger or trace reports selected states from one execution.

Each view answers a different question. Source expresses intended operations, disassembly exposes ABI details and executed instructions, and a debugger checks selected

transitions. Establishing behaviour for all valid inputs additionally requires reasoning about the possible paths and their invariants.

Laboratory 14: Reconstructing a loop

Analyse the following System V x86-64 loop and reconstruct its C-level contract. Identify arguments, result, back edge and call-preserved registers. Explain how optimisation affects debugger variable locations.

```
xor eax, eax
xor ecx, ecx
.Lloop:
cmp rcx, rsi
jae .Lend
add rax, [rdi + rcx*8]
inc rcx
jmp .Lloop
.Lend:
ret
```

Worked solution

Laboratory 14: solution

RDI points to a qword array and RSI gives an unsigned element count. RAX accumulates the sum modulo 2^{64}; RCX is the index. The loop exits when $i \geqslant n$ and its back edge jumps after incrementing i. Equivalent code is `uint64_t sum(const uint64_t *p, size_t n){uint64_t s=0; for(size_t i=0;i<n;i++) s+=p[i]; return s;}`. With $[4,7,2]$ and n=3, successive sums are 4,11,13, and the return is 13. All modified registers are caller-saved; no callee-saved register needs restoration. An optimiser can keep i and s entirely in registers, merge them into other expressions, or eliminate their source-level identity. Debug locations must therefore be interpreted at the current instruction rather than assuming a permanent stack slot.

Return to the laboratory

Caution

Debug information relates machine locations to source constructs over particular instruction ranges. At high optimisation levels, stepping may appear to move nonlinearly through source lines because instructions have been reordered or one instruction contributes to several source constructs.

Exercise 2.6.1 — Three-pass reconstruction

Assume the System V AMD64 ABI, a signed 64-bit argument $-10^6 \leqslant x \leqslant 10^6$ in `rdi`, and a signed result in `rax`. First mark the basic blocks, then annotate data flow and ABI obligations, and finally give a C description. Check the boundary between branches and an input on each path.

```
describe:
    test rdi, rdi
    js negative
    lea rax, [rdi + rdi*2]
    add rax, 4
    ret
negative:
    mov rax, rdi
    neg rax
    ret
```

Solution 2.6.1

The entry block tests the sign bit of `rdi`. Its conditional jump leads to the negative-input block; fall-through leads to the nonnegative-input block. Each path ends in `ret`, so there is no merge block before return.
On the nonnegative path, `lea` computes $x + 2x = 3x$ without loading memory, and `add` produces $3x + 4$ in `rax`. On the negative path, the copy and negation produce $-x$. The System V AMD64 integer argument and result registers are respected. Only caller-saved registers and flags change; no stack storage or nested call is used, and the return instruction consumes the caller's saved return address.
For the stated input interval, both formulas fit in a signed 64-bit integer. A matching C function is:

```
long describe(long x) {
    return x < 0 ? -x : 3*x + 4;
}
```

The boundary $x = 0$ selects the nonnegative path and returns 4; $x = -1$ selects the other path and returns 1. Further checks give $f(3) = 13$, $f(-2) = 2$, $f(-1000000) = 1000000$ and $f(1000000) = 3000004$. Extending the claim to every 64-bit input would require a different specification because C signed overflow and machine wraparound have different contracts.

A C source file is preprocessed, compiled to assembly or an internal representation, assembled into a relocatable object and linked into an executable or shared object. These artefacts answer different questions:

- compiler assembly shows one compiler's chosen instructions and symbolic labels;
- an object file contains encoded instructions plus symbols and relocation records;
- an executable contains linked virtual addresses, segments and imported interfaces;
- disassembly reconstructs instruction mnemonics from bytes but cannot restore lost source names and types with certainty.

Worked example: stack alignment across a call

A scalar-call callee enters with `rsp=0x8008`. Pushing `rbp` changes it to `0x8000`; reserving 32 local bytes changes it to `0x7FE0`, aligned to 16. A nested `call` stores its return address at `0x7FD8`. After that callee returns, `rsp` is again `0x7FE0`. Releasing the locals and popping `rbp` restores `0x8008`; the final `ret` consumes this function's own return address.

At `-O0`, compilers commonly retain a simple mapping between source variables and stack slots to support debugging. At higher levels, variables may live only in registers, expressions may be combined, branches may become conditional moves, and whole functions may be inlined or removed.

Worked example: branch versus conditional move

Source:

```
long max2(long a, long b) { return a > b ? a: b; }
```

A compact translation is:

```
mov rax, rsi
cmp rdi, rsi
cmovg rax, rdi
ret
```

The comparison is signed. `cmovg` avoids a control-flow branch but still depends on the flags. A compiler may prefer a branch when it predicts one outcome strongly or when one arm contains expensive work.

2.6.6 A reverse-engineering workflow

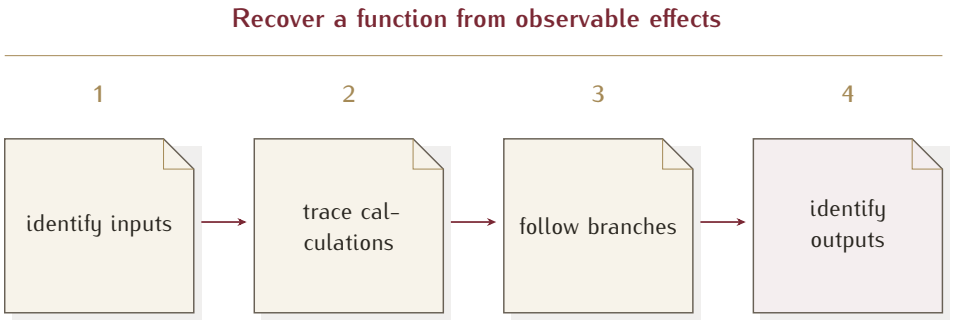


Figure 2.56: *The reconstruction begins with the calling convention and ends with a candidate specification. Local arithmetic alone is insufficient: branches, memory writes and exceptional preconditions can change the function's behaviour.*

A disciplined reconstruction proceeds through the following steps:

1. identify function boundaries and entry assumptions;
2. mark arguments, return values and preserved registers from the ABI;
3. split the code into basic blocks at the entry, jump targets and instructions following control transfers;
4. write the condition for every outgoing edge from the exact flag-producing instruction;
5. name values by role only after observing how they are used;
6. reconstruct loops from back edges and induction-variable updates;
7. validate the hypothesis with boundary inputs in a debugger.

Worked example: fix an ABI violation

Incorrect function:

```
helper:
 mov rbx, rdi
 lea rax, [rbx+1]
 ret
```

It changes `rbx`. A conforming version either uses a caller-saved temporary or preserves the register:

```
helper:
 push rbx
 mov rbx, rdi
```

Worked example (continued)

```
lea rax, [rbx+1]
pop rbx
ret
```

The push also changes alignment; a function that makes another call must account for that before the nested call.

Disassembly and debugging

Machine-code inspection predates modern security work. Programmers needed to diagnose crashes, inspect compiler output and maintain software when source was unavailable. Symbolic debuggers and object formats gradually added names, line mappings and unwind information. Reverse engineering uses the same mechanisms but must remain explicit about uncertainty: decoding recovers instruction meanings at selected byte boundaries, while distinguishing code from data and inferring source-level intent require additional evidence. [11, 12]

An indirect call or jump is safe only if its target is constrained. Function pointers, virtual dispatch tables and return addresses all influence control flow. Modern defences add control-flow integrity, shadow stacks or pointer authentication on some platforms, but the first line of defence remains preventing memory corruption and validating indirect targets.

Laboratory 15: Source, objects and executable code

Use the complete C program below to compare `-O0` and `-O2` object code. Give the expected semantic result and explain calls, relocations and inlining that may vary.

```
#include <stddef.h>
long max2(long a,long b) { return a>b?a:b; }
long sum(const long *a,size_t n) {
    long s=0; for(size_t i=0;i<n;i++) s+=a[i]; return s;
}
long work(const long *a,size_t n) { return max2(sum(a,n),10); }
int main(void) { long a[]={2,3,4}; return work(a,3)!=10; }
```

Worked solution

Laboratory 15: solution

The array sum is 9, max2 selects 10, and main returns zero. Signed additions are within range for this test. Build objects with `gcc -O0 -g -c demo.c -o demo0.o` and `gcc -O2 -g -c demo.c -o demo2.o`; inspect them with `objdump -dr -Mintel` and `readelf -s -r`. A low-optimisation version commonly contains separate calls and stack locations; an optimiser may inline sum or max2 and fold the known

Worked solution (continued)

main result. Relocations describe addresses or symbols still needing link-time resolution. A call that was inlined has no runtime call site to identify. To check the complete program, build `gcc -O2 demo.c -o demo` and run `./demo`; its exit status is zero. The reconstructed semantics remain sum-then-maximum even when the final instruction sequence no longer resembles the source's function boundaries.
Return to the laboratory

From the Wheeler jump to standard calling conventions

Timeline

Year	Machine, standard or event	Architectural significance
1949	**Wheeler jump**	A subroutine convention synthesises call and return.
1989	**System V ABI**	Binary calling conventions are standardised for separately built software.
2003	**AMD64 psABI**	Registers, stack alignment and aggregate passing are defined for x86-64.

Subroutines became practical only when a machine could return to a dynamically chosen caller. Early stored-program computers had jumps, but a call instruction and stack were not guaranteed. Programmers stored a return address in a known location or modified a jump instruction in the subroutine. David Wheeler developed an influential EDSAC technique around 1949-1951 in which the caller arranged a return link and the subroutine modified its final jump. The "Wheeler jump" made reusable closed subroutines possible on hardware that had no modern call stack [79].

Libraries quickly followed. EDSAC users prepared "initial orders" and subroutine tapes for numerical operations. Maurice Wilkes, David Wheeler and Stanley Gill published *The Preparation of Programs for an Electronic Digital Computer* in 1951, one of the first programming books. Reuse created conventions: a routine had to know where arguments were located, which storage it could destroy and how to return. These were calling conventions before the term ABI became common [30, 31, 12].

Nested and recursive calls required a more general storage discipline. A stack lets each invocation allocate a frame containing a return address, saved registers, local variables and temporary values. The Burroughs B5000, introduced in 1961, was designed around a stack architecture and high-level languages such as ALGOL. Other architectures retained general registers but used a memory stack through convention and dedicated instructions. ALGOL 60's nested procedures and recursive semantics helped make activation records a central compiler concept [30, 31, 12].

Different manufacturers and compilers created incompatible conventions. Some passed arguments in memory, some on a stack, and some in registers. FORTRAN, first delivered by IBM in 1957, ALGOL 60, C as developed at Bell Labs from 1972, and Pascal from 1970 had different requirements for arrays, strings, hidden results and name decoration. Separate compilation worked only when object formats, linkers and calling rules agreed. The application binary interface emerged as the larger contract joining instruction set, object format, data layout, calling convention, dynamic linking and operating-system process conventions [30, 31, 12].

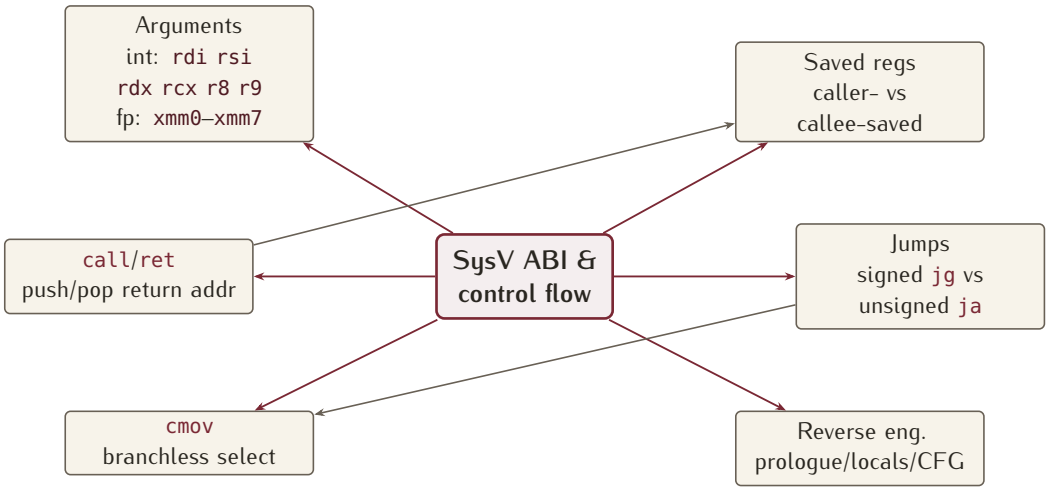
The x86 `CALL` instruction pushes a return address and transfers control; `RET` pops an address. On 16- and 32-bit systems, many C conventions placed arguments on the stack. Names such as `cdecl`, `stdcall` and `fastcall` distinguished cleanup and register use, especially on Microsoft platforms. Unix systems converged through System V ABI supplements associated with System V Release 4 in 1989. Prepared for AMD64 deployment, the x86-64 System V ABI placed the first integer or pointer arguments to RDI, RSI, RDX, RCX, R8 and R9 and the first floating arguments to XMM registers, with further arguments in memory [30].

Stack alignment became important for vector operations and data layout. Under the System V AMD64 ABI, the stack has a defined alignment relation at function entry, and callers arrange alignment before a call. A 128-byte red zone below RSP may be used by leaf functions in ordinary user space because asynchronous signal handling is specified not to overwrite it; kernel code and some environments disable this assumption. Variadic functions require extra metadata because the callee cannot infer argument classes from one fixed prototype. Aggregate arguments and return values are classified into ABI-defined classes rather than passed by one simple “small structs use registers” rule [30].

Unwinding and debugging added another layer. A frame pointer once provided a linked list of frames, but optimising compilers often reuse RBP as a general register. DWARF, whose 1992 Version 1 specification documented an existing System V format, uses call-frame information to describe how to reconstruct the caller’s registers at each instruction range. Exceptions in C++ and other languages use unwind tables and personality routines to destroy objects and transfer control. A function prologue is therefore not fixed to `push rbp; mov rbp, rsp`; it is one possible implementation under a larger unwind and ABI contract [30, 31, 12].

Tail-call optimisation shows that a source-level call need not create another frame. If a function’s final action is to return another call’s result and ABI conditions permit, the compiler may restore its state and jump to the target. The callee then returns directly to the original caller. Inlining removes the call boundary entirely. Conversely, security instrumentation may add stack checks, control-flow records or profiling hooks. Reverse engineering must infer semantics from actual control flow rather than assume one canonical prologue and epilogue [30, 31, 12].

Concept map: ABI & control flow



Reading the map. The ABI fixes where arguments and return values live and which registers survive a call; `call`/`ret` track return addresses on the stack, while software enforces the register-save conventions; conditional jumps (signed vs unsigned) and the branchless `cmov` implement decisions; and reverse engineering runs the whole thing backwards from a disassembly.

2.6.7 Worked problems

Toolchain option

Question. Which GCC option stops after generating assembly text, and why does lowercase `-s` have a different effect?
Use `gcc -S calc.c -o calc.s`. Uppercase `-S` ends the driver pipeline after compilation, leaving readable assembly in `calc.s`. Lowercase `-s` asks the linker to strip symbol and relocation information from its output; it still proceeds through the normal linking stage unless another option stops the pipeline.
The output type checks the distinction. `-S` produces assembly text, `-c` produces a relocatable object, and the ordinary linked build produces an executable. Case matters because each spelling names a different option.

Disassembly clue

Question. What does `cmp rdi,rsi` followed by `jge` reveal about the comparison?
`cmp` sets flags from `rdi−rsi`. `jge` takes the branch when SF=OF, the condition for signed greater-than-or-equal. The jump therefore interprets these operands as signed values for this decision.

Worked example (continued)

For -1 compared with 1, the result is -2, so SF=1 and OF=0; the branch is not taken. Replacing `jge` with `jae` would test CF=0. That unsigned comparison regards the all-one representation of -1 as the largest 64-bit value and takes the branch. The condition reveals the interpretation at this use, while the original source declaration still requires supporting evidence.

Array clue

Question. What layout information follows from `mov eax,[rdi+rsi*4]`, and which source-level details remain uncertain?
The instruction reads four bytes because its destination is `eax`. Its address changes by four when `rsi` increases by one. These facts are consistent with `rdi` as an array base and `rsi` as an index into four-byte elements.
The machine code carries no C type name. The bytes might represent an integer, floating-point bits copied through an integer register, or a field inside a larger layout. Check how the index is formed and how the loaded value is subsequently used before assigning a type. Similarly, a scale of eight establishes eight-byte spacing per index increment, which can occur with integers, pointers or other records.

Optimisation

Question. Why might a source-level temporary have no stack slot in an optimised executable?
Consider `long f(long x){long t=x+1; return t*2;}`, restricted to inputs for which the signed operations fit. The value of `t` contributes to the returned expression $2x + 2$. A compiler may compute that expression directly with `lea rax,[rdi+rdi+2]; ret`, so no memory object is needed for the temporary.
Alternatively, a value may live in a register, be replaced by a known constant, or disappear when it has no observable effect. To follow the computation, trace the input's contribution to the result and inspect debug location ranges when available. A source variable's role can survive even when it has no distinct storage location.

Exercise 2.6.2 — Preprocess only

Which GCC option stops after preprocessing?

Worked solution

Use `gcc -E source.c`. The driver performs preprocessing, including macro expansion and header inclusion, and writes the resulting translation-unit text to standard output unless an output file is specified. Compilation, assembly and linking have not yet occurred. For example, adding `-o source.i` saves that preprocessed text. This stage is useful for checking which declarations and macro expansions the compiler will receive.

Exercise 2.6.3 — Object only

Which option compiles and assembles without linking?

Worked solution

Use `gcc -c source.c -o source.o`. The driver preprocesses and compiles the source, then assembles the generated code into a relocatable object file. It stops before linking. Undefined external symbols may therefore remain in `source.o`; they are resolved when the required objects and libraries are linked. The presence of the object file does not mean that a runnable executable has been created.

Exercise 2.6.4 — Return register

Where is an integer return value found?

Worked solution

For the integer return cases studied here, the result is in the accumulator register at the width required by the ABI: for example `eax` for a 32-bit int and `rax` for a 64-bit integer or pointer. A write to `eax` clears the upper 32 bits of `rax`. The caller must interpret the documented return type; reading arbitrary upper bits of a narrower return as a signed 64-bit value can misinterpret the result. Aggregate and floating-point returns have separate conventions.

Exercise 2.6.5 — Symbol tool

Which tool commonly lists ELF symbols compactly?

Worked solution

Use `nm object.o` to list symbols and their classifications compactly. An undefined-symbol marker indicates a reference that must be supplied elsewhere at link time. `readelf -s object.o` displays the ELF symbol table with fields such as value, size,

Worked solution (continued)

binding and section index. Inspecting these tables distinguishes missing definitions from code-generation errors. Stripped files or dynamically exported symbols can require the corresponding dynamic-symbol options.

Summary

Read a function in three passes. First establish its frame and register obligations. Then draw the control-flow graph. Finally track values through the graph and assign cautious semantic names. Delaying names prevents an early guess from distorting later evidence. The result should explain every instruction, including apparently incidental alignment or spill operations.

2.6.8 Exercises

Exercise 2.6.6 — Argument locations

For `long f(long a,long b,long c,long d,long e,long f,long g)`, locate all arguments on entry under System V AMD64.

Go to the solution →

Exercise 2.6.7 — Preserved register

A function changes `r12` and returns without restoring it. What rule is violated and how can it be fixed?

Go to the solution →

Exercise 2.6.8 — CALL state

If `rsp=0x8000` immediately before a direct call at address `0x401000` whose next instruction is at `0x401005`, what happens to `rsp` and memory?

Go to the solution →

Exercise 2.6.9 — Reconstruct a conditional

Interpret:

```
mov rax, rdi
cmp rdi, rsi
cmovl rax, rsi
ret
```

Go to the solution →

Exercise 2.6.10 — Loop reconstruction

Let `edi` hold an unsigned 32-bit base x, and let `esi` hold an integer count $0 \leqslant n \leqslant 31$. Reconstruct this post-tested loop, including its result when $n = 0$:

```
mov eax, 1
.loop:
imul eax, edi
dec esi
jg .loop
ret
```

Go to the solution →

Exercise 2.6.11 — Object versus executable disassembly

Why can a call target appear as zero or a placeholder in a relocatable object disassembly, while the executable shows a concrete address?

Go to the solution →

Exercise 2.6.12 — Optimisation evidence

A source variable has no stack slot in `-O2` output. Does this prove the compiler ignored it? Explain how to track it.

Go to the solution →

2.6.9 Solutions

Procedures, Control Flow, and Reverse Engineering

Solution 2.6.6

The first six are in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. The seventh is at `[rsp+8]` because `[rsp]` holds the return address.

← Return to the exercise

Solution 2.6.7

`r12` is callee-saved under System V AMD64. The caller may retain a live value there across the call, so replacing it permanently violates the function's binary contract. One repair is:

```
push r12
# function body may use r12
pop r12
ret
```

Every return path must restore the saved value. On ordinary scalar-call entry, $\texttt{rsp} \bmod 16 = 8$; this single push makes it aligned for a nested call, provided subsequent stack adjustments preserve the required relation. Using a caller-saved temporary avoids the save/restore pair when its value need not survive nested calls.

← Return to the exercise

Solution 2.6.8

The near call reserves eight stack bytes, so `rsp` changes from `0x8000` to `0x7FF8`. It stores the next instruction address, `0x0000000000401005`, in that qword. Little-endian byte order at `0x7FF8` through `0x7FFF` is `05 10 40 00 00 00 00 00`. Execution then continues at the call target.

If the callee restores `rsp` to `0x7FF8` before `ret`, the return reads that qword, sets `rip=0x401005` and restores `rsp=0x8000`. The saved address points after the call, ensuring that returning continues the caller instead of invoking the callee again.

← Return to the exercise

Solution 2.6.9

The signed condition is `rdi<rsi`. If true, `rax` becomes `rsi`; otherwise it stays `rdi`. The function returns the signed maximum of the two inputs.

← Return to the exercise

Solution 2.6.10

The test follows the multiply and decrement, so the body executes at least once. On the stated platform a matching C fragment is:

```
uint32_t result = 1;
do {
    result *= x;
    --n;                /* n has type int, initially 0..31 */
} while (n > 0);
return result;
```

For $n > 0$, the function returns $x^n \bmod 2^{32}$. For $n = 0$, it executes one multiplication, changes the count to -1, and returns x. The required value was $x^0 = 1$. For example, $x = 3, n = 2$ gives successive products 3 and 9; $x = 3, n = 0$ gives 3. A pre-test before entering the body is needed for the usual zero-exponent contract. The count restriction ensures the signed decrement does not overflow; the product deliberately retains only its low 32 bits.

← Return to the exercise

Solution 2.6.11

The assembler can encode the call instruction before knowing its final target address. It places a provisional displacement in the operand and records a relocation identifying the referenced symbol and the required calculation. For a relative call, the linker ultimately needs the distance from the instruction following the call to the target.

For example, a target at `0x401100` and next instruction at `0x401005` require displacement `0xFB`, or 251 bytes. Before layout is known, the object can only record the unresolved relationship. `objdump -dr` shows the relocation beside the encoded operand; the linked executable shows the resolved destination or a linker-created dispatch stub such as a PLT entry.

← Return to the exercise

Solution 2.6.12

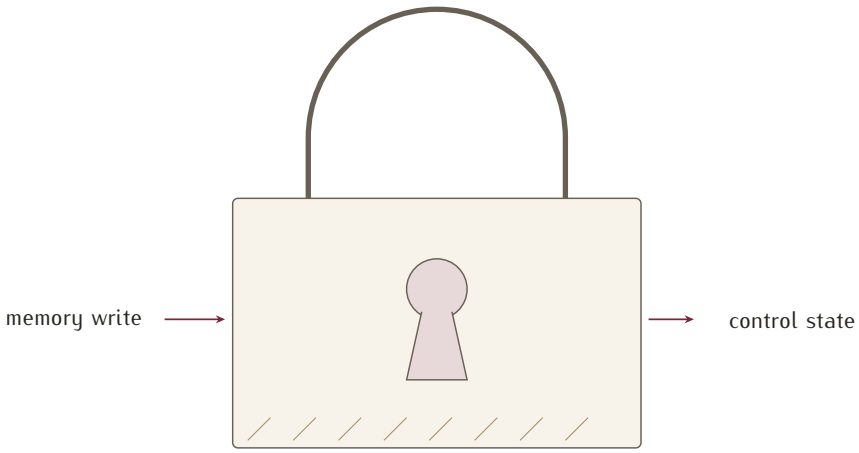
No. The value may remain in a register, be folded into another expression, exist only along one path, or be eliminated because only its effect matters. Track data dependencies from inputs to outputs and memory effects rather than searching for a one-to-one variable location.

← Return to the exercise

2.7 STACK FRAMES, MEMORY SAFETY, AND CONTROL-FLOW REUSE

A function stores a local array on its stack. Nearby bytes may hold saved registers and an address used to return to the caller. Following the actual addresses explains how an out-of-bounds write can reach another object and how changed saved state affects later execution. The frame layout comes from a programming convention; its protection depends on the checks and hardware mechanisms active in the stated example.

Control data needs both valid access and protected use



The lock represents a specified protection rule, not universal immunity.

Figure 2.57: *Saved control values can influence later instruction addresses. The chapter examines object boundaries and the scope of mechanisms that protect those values or their use. Each protection must be understood through its actual enforcement rule.*

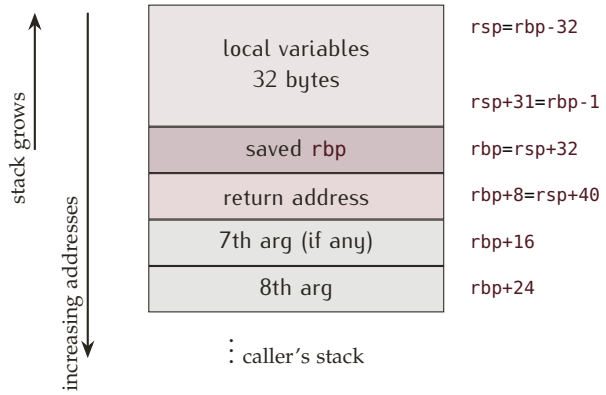
Unless a protection is explicitly enabled in a trace, assume the stated stack memory is accessible and the named code addresses are valid, with no active shadow-stack check. These preconditions describe the machine state required by the example.

2.7.1 Stack frames as conventions

On the System V AMD64 platform, the active stack occupies a region addressed through `rsp` and conventionally grows toward lower addresses. Procedure code uses it for re-

turn addresses, saved registers and local storage. This proximity matters for memory safety: an out-of-bounds write into a local array may reach saved control data and turn a data error into a control-flow error. The discussion first reconstructs ordinary stack operation, then examines classic stack corruption and the defensive purpose of canaries, non-executable pages, address randomisation and control-flow protections.

After the classic prologue with 32 bytes of locals (lower addresses at the top):



Locals are at `[rbp-N]` (or `[rsp+N]`); the return address at `[rbp+8]`.

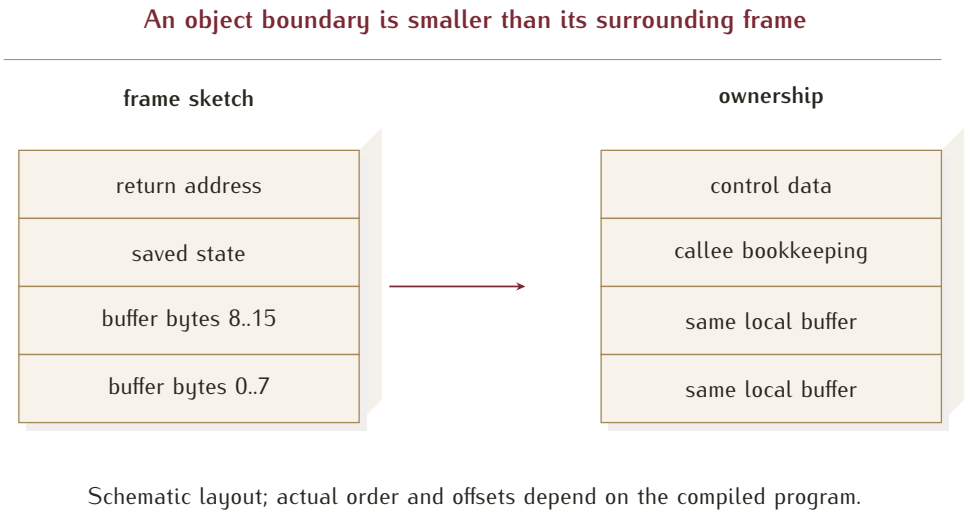


Figure 2.58: *The local buffer owns only its allotted byte interval. Other bytes in the frame serve different purposes. A contiguous address space does not grant the buffer permission to use its neighbours.*

Caution

In this particular frame layout, a sequential write beyond the 32-byte local area reaches the saved `rbp` and then the saved return address. The epilogue would subsequently restore corrupted state. The overwrite has crossed from data into saved control-flow state. Exploitability also depends on address knowledge, page permissions and mitigations.

2.7.2 From an out-of-bounds write to control data

Aleph One's 1996 article gave a widely influential account of exploiting a stack buffer overflow [13]. In the classic construction, an unchecked input overwrites both a local buffer and the saved return address; the return transfers control to injected instructions placed in writable memory. Contemporary systems attack each premise of that construction through compiler instrumentation, page permissions and address randomisation. The historical mechanism remains worth studying because it connects object bounds, stack layout and control flow, but any concrete analysis must be tied to the protections and binary actually in use.

1. **Non-executable data pages.** Page permissions can prevent instructions from being fetched from the stack or heap. An injected byte sequence then causes a protection fault rather than executing.
2. **Stack canaries.** Compiler instrumentation places a guard value between selected local objects and saved control state, then verifies the value before returning. StackGuard was an early systematic implementation of this idea [14].
3. **Address-space layout randomisation.** Randomised placement makes code and data addresses less predictable. Position-independent executables extend this property to the main program image.
4. **Control-flow protections.** Software CFI, protected shadow stacks and pointer-authentication mechanisms restrict the targets or integrity of indirect transfers. Their guarantees differ, so the relevant architecture and toolchain must be named.

Defences change the attack model. Non-executable data pages motivated code-reuse techniques, in which execution is redirected through instructions already present in executable mappings. Address randomisation makes reliable control transfer depend on address knowledge, while canaries try to detect a class of sequential stack overwrites before a return. Control-flow protections impose additional constraints on admissible targets. These mechanisms are complementary: each removes or weakens a different assumption, and none repairs the original out-of-bounds write.

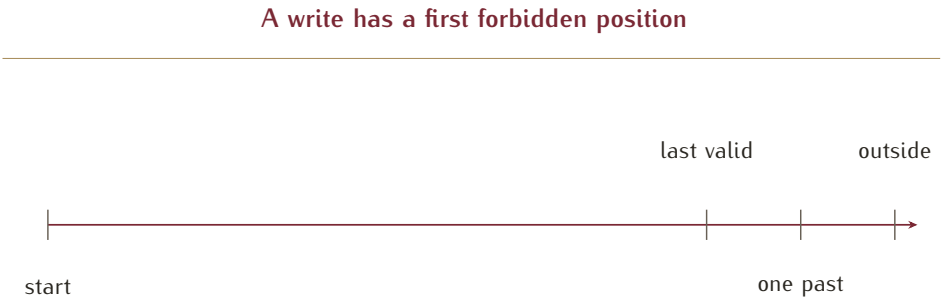


Figure 2.59: *The ninth byte begins at index eight, immediately after the object. Whether a later byte reaches saved state depends on the actual layout. The out-of-bounds violation already exists at the first forbidden position.*

Worked example

Problem. A function `f` is called with three arguments in `rdi`, `rsi`, `rdx`. Store the original `rdi` into the local at `[rbp-8]`.

Solution.

```
push rbp; prologue
mov rbp, rsp
sub rsp, 0x10; 16 bytes of locals (16-aligned)
mov [rbp-8], rdi; save original rdi
leave
ret
```

When the frame pointer is omitted, the compiler addresses local objects relative to `rsp`. The offsets remain correct because the compiler tracks every stack-pointer adjustment; they are not inherently fragile. A dedicated `rbp` frame pointer can nevertheless simplify debugging, unwinding and hand analysis at the cost of one general-purpose register.

2.7.3 Mitigations and their limits

A schematic protected function using the common Linux x86-64 glibc guard layout is shown below. The precise sequence and which functions receive protection depend on compiler options and object layout:

```
foo:
 push rbp
 mov rbp, rsp
 sub rsp, 0x20
 mov rax, qword ptr fs:[0x28]; reference canary from TLS
 mov [rbp-8], rax      ; store canary below saved frame state
 ; ... body, buffers at [rbp-0x10], [rbp-0x18] ...
 mov rax, [rbp-8]
 xor rax, qword ptr fs:[0x28] ; compare with original
 jz .ok
 call __stack_chk_fail ; canary changed: abort
.ok:
 leave
 ret
```

In this common glibc arrangement, the guard has a random component and a zero low byte. Reproducing the complete guard through a NUL-terminated input can therefore terminate that input before later control bytes are supplied. The zero byte itself is not an automatic trap, and an overwrite that leaves it unchanged may escape this particular check. A linear overflow toward the return address normally crosses the canary; if the checked canary value is modified, the epilogue detects the mismatch before returning.

A defensive assessment must account for three known failure conditions. Another vulnerability, such as a format string, may *leak* the canary so that an overwrite can reproduce it unchanged. A *structural bypass* may instead reach a different target, such as a function pointer, without crossing the canary. Fork-based servers may also inherit one canary value, making *byte-by-byte forcing* relevant under specific restart and observation conditions.

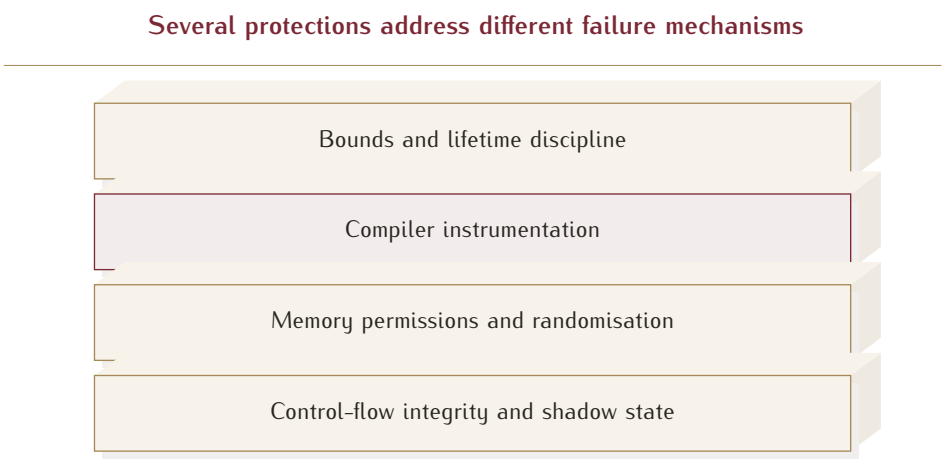


Figure 2.60: *The drawing organises protections by where they act. Bounds checks address invalid accesses; permission and control-flow mechanisms constrain particular consequences. Correctness still depends on understanding what each mechanism actually guarantees.*

A canary ultimately needs an unpredictable seed

A stack canary is a secret value checked before return.

The operating system normally derives random values from a cryptographic random-number subsystem that was seeded and continually managed earlier. That subsystem can combine timing observations, device events and hardware random-number facilities. Hardware entropy sources may sample noise, oscillator jitter or metastable circuits, but raw physical output requires health testing and conditioning before it is suitable as a seed. A process can therefore receive unpredictable bytes without sampling a physical source directly at each creation.

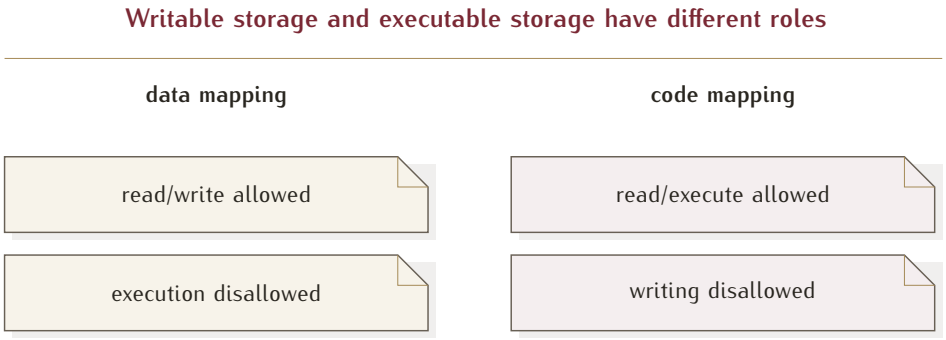


Figure 2.61: *Separating write permission from execute permission limits how a writable data region can be used. It does not establish that all writes stay within their source-language objects or that all control transfers are valid.*

A canary that is present but predictable offers much weaker protection. The property depends on software placement, safe distribution of the value and the quality of the random subsystem and its underlying seed material [14, 8, 26].

2.7.4 Control-flow reuse

Return-oriented programming reuses short instruction sequences already present in executable mappings. A sequence that ends in `ret` can be treated as a *gadget*: the return obtains the address of the next sequence from the stack. A chain is therefore analysed as a series of machine-state transformations, each with explicit preconditions, register effects and stack consumption [64]. Whether a useful chain exists depends on the binary, loaded libraries, available addresses and active mitigations. Automated tools can enumerate candidate sequences, but they do not remove the need to verify their actual effects.

The following traces use known local code addresses to analyse changes in registers and stack state. The first reaches the process-termination wrapper with status 42; the second requests a raw three-byte write and then terminates. Each trace depends on the stated stack layout, usable code mappings and protection assumptions.

The trace must record four invariants: the stack pointer advances by the expected amount; every selected sequence has only the stated side effects; the destination interface receives its arguments in the required registers; and the final transfer terminates the laboratory process. If any address depends on relocation or randomisation, that dependency must be stated rather than silently assumed.

Worked example

Problem. A local trace supplies addresses of `pop_rdi` (`pop rdi; ret`), `plain_ret` (`ret`) and the libc `_exit` wrapper. Immediately before the corrupted return, `rsp` $\bmod 16 = 8$. Arrange stack words to reach `_exit(42)` with the scalar-call ABI alignment. Let N be the byte offset from the input buffer to the saved return-address slot.

Solution. The following words begin at that slot:

Offset from saved return slot	Contents
0	address of `pop_rdi`
8	42
16	address of `plain_ret`
24	address of `_exit`
32	unused continuation slot

The first return consumes 8 bytes; the pop consumes another 8 and loads 42 into `rdi`; the gadget's return consumes the third word; the plain return consumes the fourth. Entry to `_exit` is therefore at the original `rsp`+32, still congruent to 8 modulo 16. The fifth slot occupies the position where an ordinary call's return address would reside; `_exit` does not return. Without the extra plain return, entry would have the wrong alignment. Prefix padding has length N, which includes any intervening locals, padding and saved frame pointer, beyond the declared array itself.

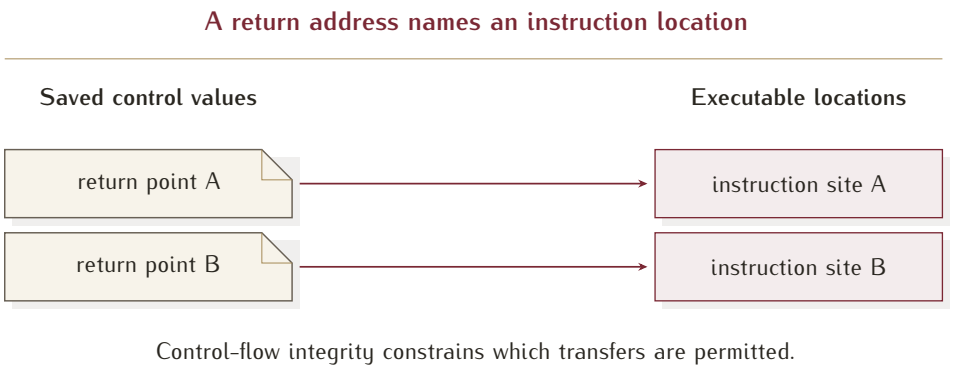


Figure 2.62: *A return instruction consumes saved control state as an instruction address. Reasoning about control-flow reuse begins with that relationship. Protection mechanisms can check the relationship against a separate policy or protected record.*

On the stated glibc/Linux platform, the `_exit` wrapper requests `exit_group` and terminates all process threads with the supplied status.

```
[ padding ]
[ pop_rdi ]; [ 1 ]; fd = stdout
[ pop_rsi ]; [ addr of "PWN" ]; buffer
[ pop_rdx ]; [ 3 ]; length
[ pop_rax ]; [ 1 ]; write
[ syscall_ret ]
[ pop_rdi ]; [ 0 ]; exit code
[ pop_rax ]; [ 231 ]; exit_group
[ syscall_ret ]
```

The chain occupies $14 \cdot 8 = 112$ bytes, so its total length with prefix padding is $N + 112$. If a 64-byte buffer is immediately followed by an eight-byte saved frame pointer and then the return slot, $N = 72$ and the total is 184 bytes. Extra padding or a canary would change that calculation. This straight-line trace ignores the write result: it requests three bytes, but a short write or error would require additional control flow to guarantee that all three were accepted.

Automated analysis tools can enumerate short instruction sequences and model their side effects. A manual trace verifies stack consumption, register effects and the assumptions needed for a proposed control-flow path. These checks connect an automated result to the memory and calling-convention model.

2.7.5 Stack optimisation and ABI details

Worked example

Problem. Write `long add_one(long x)` returning $x + 1$ without disturbing the stack, for inputs with $x <$ LONG_MAX.

Solution. The result fits directly in a register, so this leaf function needs no local stack storage:

```
add_one:
 lea rax, [rdi+1]
 ret
```

The red zone helps slightly more complex leaf functions that need temporaries but call no one: SysV guarantees those 128 bytes are untouched by signal/interrupt handlers. (On Windows x64 there is no red zone.)

Worked example

Problem. `wrapper(long x)` returns `twice(x+1)` where `twice` is a separate function. Assume $x + 1$ is representable as `long`; optimise with a tail call.

Solution.

```
wrapper:
 add rdi, 1
 jmp twice; tail call: no additional saved return address
```

The jump leaves the original caller's return address on top of the stack. Thus `twice` enters with the ordinary callee alignment and returns directly to `wrapper`'s caller. A conventional nested call would instead need an eight-byte alignment adjustment before `call twice`, followed by its removal and `ret`. The tail jump is valid when the wrapper has restored its preserved registers, released any local frame, arranged the target arguments and has no remaining work after the call. ABI, debugging and instrumentation requirements can constrain this transformation.

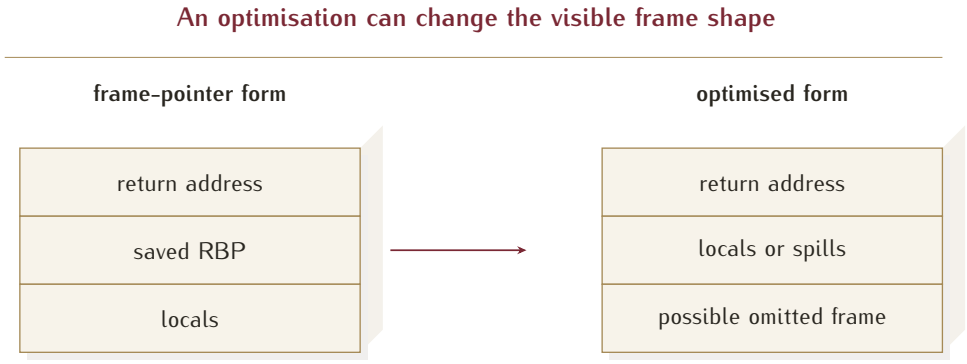


Figure 2.63: *Optimisation may omit a frame pointer, reuse stack slots or keep values in registers. A diagram of one compiled function must therefore state its assumptions and be checked against that function's instructions.*

NX is an instruction-translation permission check

A page marked non-executable cannot supply instructions.

Page-table entries carry execute-permission information that is checked during instruction translation. A cached translation can carry the relevant permission in an instruction TLB or related structure; on a miss, the page walk obtains it from the translation hierarchy. If execute permission fails, the processor reports an architectural page fault instead of allowing the bytes to become an executable instruction stream.

NX does not change the electrical nature of the stored bytes. The same memory contents can be readable as data yet forbidden as instructions, and permission changes require the architecture's prescribed translation-cache invalidation or coordination [26, 27].

2.7.6 Shadow stacks and complementary protections

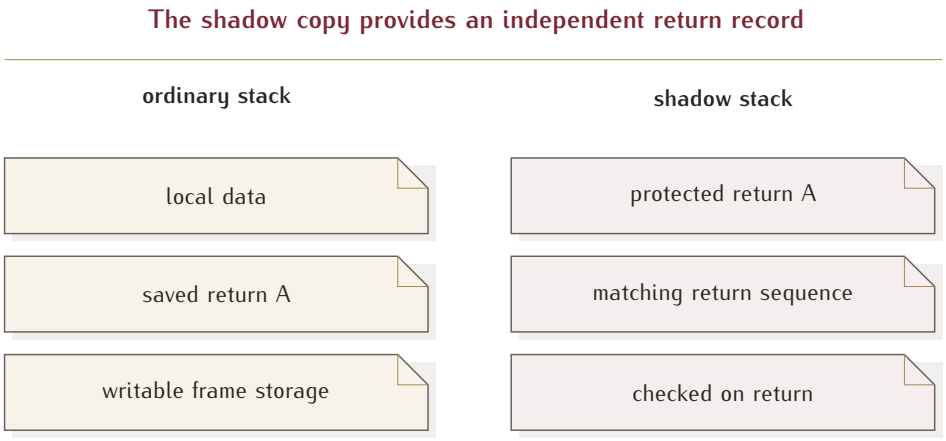


Figure 2.64: *A shadow stack preserves a separately protected record of return addresses. A check compares the ordinary return against that record. The mechanism protects return integrity within its specified scope; it is not a general object-bounds check.*

The frame shape depends on optimisation: red-zone storage, omitted frame pointers, tail calls and register allocation can all alter which bytes exist on the stack. A concrete assessment therefore follows the generated code and its actual object layout.

On supporting processors, Control-flow Enforcement Technology can maintain a protected shadow copy of return addresses. A `call` updates both the normal and shadow stacks; a return checks that the copies agree and raises a control-protection exception on mismatch. Linux support additionally requires kernel configuration and enabled user-space components; hardware capability alone does not activate the feature [26, 37].

Layer	Stops primarily	Does not by itself stop
Canary	many linear frame overwrites	disclosure, non-linear corruption
NX	execution from data pages	code reuse
ASLR	reliable fixed addresses	leaks, brute force in weak settings
Shadow stack	forged return addresses	arbitrary non-return control transfers

The correct defensive question is therefore "which state is protected, by which component, and at what point is the check enforced?"

The complete overwritten interval

Assume a 32-byte array occupies `[rbp-32,rbp)`, saved `rbp` occupies `[rbp,rbp+8)`, and the saved return address occupies `[rbp+8,rbp+16)`. No canary is present in this supplied layout. An 80-byte sequential copy starting at the array writes offsets 0–79 from `rbp-32`. The first 32 bytes are in bounds; the remaining 48 are outside the array. Offsets 32–39 overwrite saved `rbp`, offsets 40–47 overwrite the return address, and offsets 48–79 reach further caller storage. Thus the write reaches control data, but predicting its later use still requires the epilogue, written values and active protections.

Memory safety can be violated by physical disturbance

The ordinary architectural memory model assumes that bytes change through permitted writes or device transfers.

In susceptible DRAM devices, repeatedly activating particular rows can increase the probability of charge loss or disturbance in other cells. If refresh and mitigation are insufficient, a neighbouring or otherwise coupled bit may change without a conventional store to that address. The exact vulnerable relationships and mechanisms depend on the DRAM design and generation; the broad class is known as Rowhammer.

Logical isolation assumes that storage preserves each bit unless a permitted operation changes it. Disturbance attacks show that memory safety also depends on device characteristics, controller policies, refresh and mitigation mechanisms [9, 50].

Worked example: deriving frame offsets

A function begins with:

```
push rbp
mov rbp, rsp
sub rsp, 0x30
```

and accesses a byte buffer at `[rbp-0x20]`. The saved `rbp` is at `[rbp]`, and the return address at `[rbp+8]`. The distance from the start of the buffer to the saved return address is `0x28` bytes. The distance follows from these instructions; a different allocation or object placement requires a new calculation.

Fortified interfaces and sanitizers add checks on selected lengths, bounds and object lifetimes. These mechanisms complement the control-data protections above and can detect faults before a return address is involved.

Caution

A canary protects the state covered by its placement and check. Assessing exploitability additionally requires the bug, controllable data, memory permissions, information disclosure, target environment and available control-flow paths.

Worked example: tracing a local return chain

Suppose a local executable contains two sequences:

```
G1: pop rdi; ret
G2: mov rax, rdi; ret
```

If `rsp` points to the words `G1`, `0x1234`, `G2`, `safe_exit`, then the first `ret` reaches G1, `pop rdi` loads `0x1234`, the next `ret` reaches G2, and G2 copies the value to `rax`. G2 then consumes the final word and transfers to `safe_exit`; this label must denote a valid terminal instruction sequence with its own stated entry requirements. The trace has consumed four qwords, or 32 bytes, and left `rdi=rax=0x1234`.

Defensive review begins by removing the invalid write. Use length-aware interfaces, derive destination capacity from the actual object, reject integer-overflowed sizes, keep untrusted lengths unsigned only when that improves the proof, and test with sanitizers. Mitigations are valuable layers for when prevention fails.

2.7.7 A memory-safety audit checklist

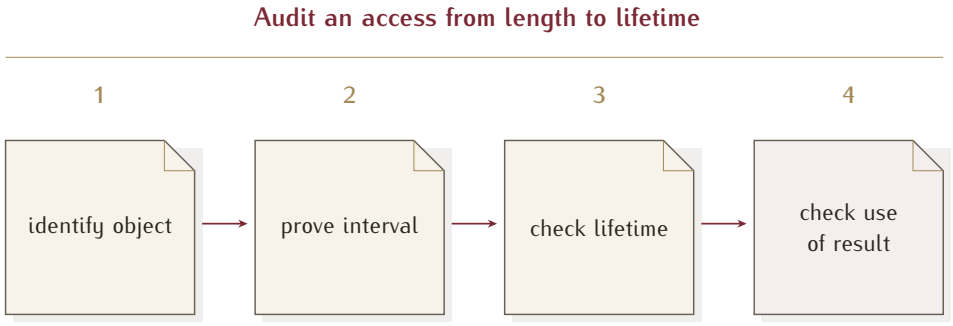


Figure 2.65: *The audit follows the actual object and requested byte interval. It also checks whether the object remains alive and whether the resulting value controls later indexing or branching. Page permissions alone do not answer those questions.*

For every memory-writing instruction or library call, ask:

1. What object does the destination pointer designate, and what is its current lifetime?
2. What is the destination capacity in bytes?
3. How is the write length computed, and can that arithmetic wrap or narrow?
4. Can source and destination overlap?
5. Does a terminating zero byte require extra capacity?
6. Which error paths leave partially initialised state or stale pointers?

At assembly level, add two more questions: what width is written, and which address expression proves the access remains in bounds?

Stack-smashing defences

Stack-based overflows were known long before modern Internet software, but widespread exploitation made them a central systems-security problem. Over time, compilers added canaries and safer diagnostics, operating systems enforced non-executable data and randomised mappings, and processors added stronger control-flow mechanisms. The continuing lesson is that compatibility keeps unsafe C interfaces alive, so defences must combine language practice, compiler instrumentation, operating-system policy and hardware support. [13, 14, 26]

> **Laboratory 16: Bounds and mitigations**
>
> Analyse a local copying routine with a 16-byte destination and a NUL-terminated source. Give a complete bounds-checked replacement. Explain the defect for 20 non-NUL input bytes and what stack canaries, NX, ASLR and AddressSanitizer can contribute.
> Worked solution

Laboratory 16: solution

The oversized string needs $20 + 1 = 21$ destination bytes, so copying it unconditionally overruns the 16-byte object by five bytes. A replacement checks capacity before copying:

```
#include <stddef.h>
#include <string.h>
int copy_text(char *dst,size_t cap,const char *src) {
    size_t n=strlen(src);
    if(n>=cap) return 0;
    memcpy(dst,src,n+1);
    return 1;
}
```

The contract requires a valid terminated source, a writable destination of cap bytes and nonoverlapping objects. For cap=16, lengths 0 through 15 succeed; length 16

Worked solution (continued)

or greater is rejected before any destination write. A canary can detect selected overwrites before return; NX restricts executable pages; ASLR changes address placement; AddressSanitizer instruments many invalid accesses. Those controls have different coverage, while the explicit bound removes this copy's capacity violation under the contract. Inspecting generated code may reveal the corresponding canary checks, stack permissions and instrumentation without constructing a control-flow exploit.

Return to the laboratory

Protection domains and hardening

Stack integrity is one layer of a wider protection argument. An operating system associates an executing security domain with rights over objects: memory mappings, files, devices, processes and privileged operations. Access-control lists organise rights by object; capability systems organise them by holder. In either form, least privilege asks that a domain receive only the authority needed for its task [59].

A useful review separates three questions: what memory error exists, what control or data influence it provides, and which independent boundary prevents that influence from reaching a protected object. This avoids treating one mitigation as a universal security property.

Exercise 2.7.1 — Domain and mitigation review

For a network parser running as an unprivileged service, list its necessary objects and rights. Then evaluate a stack canary, NX, ASLR, a read-only configuration mapping and a restricted service account. State which failure each control can limit and which failures remain.

Solution 2.7.1

Assume the service parses network requests, reads a fixed configuration and writes an application log. Its necessary objects are a listening socket, accepted connection sockets, the configuration data and a controlled log destination. Rights can be limited to accept/read/write on the relevant sockets, read access to configuration and append or a logging interface for diagnostics. The process also needs its executable and runtime dependencies during startup. Administrative configuration changes, arbitrary file writes and process tracing are outside this model.

stack canary. A checked canary can detect an overwrite that crosses its protected stack location before the function returns. It does not detect every out-of-bounds write, protect heap metadata or prevent corruption of data that lies before

Worked solution (continued)

	the canary. Disclosure of the canary can weaken this defence.
nx.	Non-executable data pages prevent direct execution of injected bytes from those pages. Existing executable code can still be invoked through corrupted control data, and data-only corruption remains possible.
aslr.	Randomised placement makes fixed address guesses less reliable. Information disclosure, limited randomness or repeated attempts can weaken that protection. The memory error itself remains.
read-only configuration mapping.	Page permissions reject ordinary writes through that mapping. The underlying file also needs suitable access rights: a read-only mapping does not by itself revoke a separate writable file descriptor or another writable mapping. Parsing errors and corruption of unrelated writable state remain possible.
restricted service account.	A dedicated account with narrowly chosen object permissions limits what a compromised service can access. Objects intentionally available to that account remain exposed, and resource exhaustion or flaws in more privileged components require additional controls.

The controls address different failure paths. The parser still needs explicit length and format validation, checked arithmetic, bounded allocation and defined ownership of buffers. A review should test malformed, truncated and oversized inputs as well as verify the process's effective object rights.

From activation records to adversarial control-flow reuse

Timeline

Year	Machine, standard or event	Architectural significance
1961	**ALGOL activation records**	Nested procedures motivate systematic stack frames.
1996	**Published stack-corruption analysis**	A widely read account connects unchecked copies to saved return addresses.
2016	**Intel CET publication**	Shadow stacks and indirect-branch tracking target control-flow reuse.

The stack was created to make nested computation manageable. ALGOL 60 made recursive procedures a language requirement, and machines such as the Burroughs B5000 introduced in 1961 treated activation records as a central architectural object supporting procedure execution. An activation record places a function's return address, saved state and local storage in one disciplined region. On many C implementations, local arrays and the stored return address therefore occupy nearby memory. The C language historically provided operations that trusted programmers to respect bounds. When input exceeded an array, the machine performed ordinary writes beyond the object and could corrupt control data [1, 134].

Buffer overruns existed long before the public Internet. They became a prominent security problem when network-facing programs ran with privilege and accepted hostile input. The 1988 Morris worm exploited several weaknesses, including a buffer overflow in the fingerd service on some systems. In 1996 the Phrack article "Smashing the Stack for Fun and Profit," published under the name Aleph One, gave a systematic explanation of overwriting a saved return address to execute injected code. The article became a widely used pedagogical account of an already known attack technique [13, 14].

The classic attack assumed executable stack memory and predictable addresses. Defences attacked those assumptions. Non-executable memory marks data pages as non-executable using page-table permissions; the x86 NX bit was specified for AMD64 and shipped with Athlon 64 processors in 2003, then adopted by Intel as Execute Disable in 2004. PaX introduced a practical form of Address Space Layout Randomisation for Linux in 2001; ASLR changes the locations of stacks, libraries, executables and mappings. Stack canaries place a value between vulnerable local data and control state and check it before return. StackGuard, presented by Crispin Cowan and colleagues at the 1998 USENIX Security Symposium, was an important compiler-based canary design [14, 134].

The original memory error can remain after these defences are enabled. With injected data excluded from execution, an attacker may try to reuse existing instructions. Return-to-libc replaces the saved return address with a library function and arranges arguments. Return-Oriented Programming, analysed systematically by Hovav Shacham in 2007 for x86, chains short instruction sequences ending in `RET`. Jump- and call-oriented variants

use other indirect transfers. These techniques turn the program's existing code into a programmable substrate and show why code/data separation alone is insufficient [64].

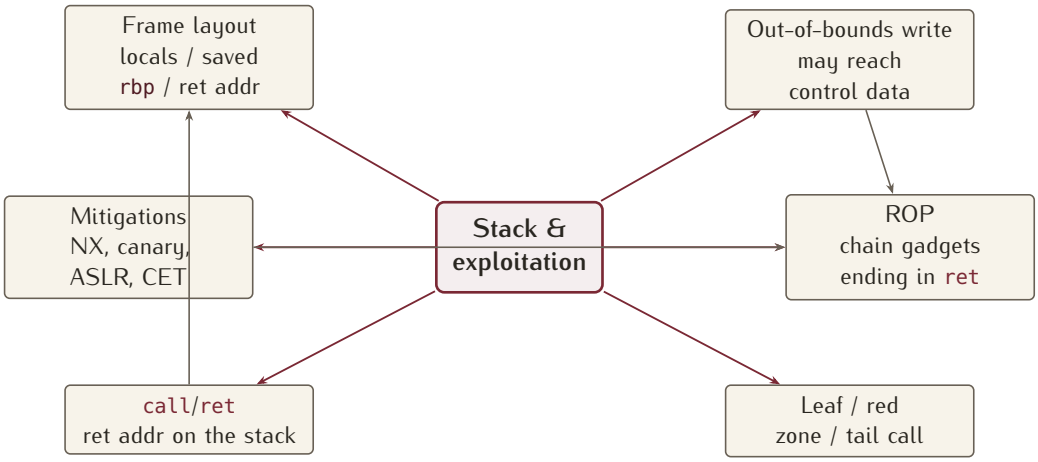
Modern exploits also target non-control data, heap objects, function pointers and virtual tables. Use-after-free errors can replace an object's contents after its lifetime ends. Integer overflow can cause undersized allocation before a copy. Information leaks can reveal randomised addresses. Memory safety is therefore broader than stack smashing, and mitigations must be viewed as layers rather than proofs of safety [1, 134, 108].

The ABI influences both attack shape and defence. System V AMD64 passes many arguments in registers, so a ROP chain may need gadgets that load RDI, RSI or RDX. Stack alignment must be respected before calling some library code. The red zone and omitted frame pointers change layouts. Position-independent executables and shared libraries interact with ASLR. Reverse engineering must use the actual binary, compiler options and runtime mappings rather than a generic diagram [30].

Hardware control-flow enforcement adds another layer. Intel published the Control-flow Enforcement Technology specification in 2016 and began shipping client support in 2020. CET defines a shadow stack for return addresses and Indirect Branch Tracking for indirect call and jump targets. The processor keeps protected return information separate from the ordinary data stack and compares it on return. Linux 6.6, released in October 2023, introduced userspace shadow-stack support on x86; use requires suitable configuration and runtime enablement through cooperating system and application components. CET does not prevent arbitrary data corruption, and software and binaries must be built and enabled appropriately; it specifically narrows classes of control-flow hijacking [108, 37].

Compilers now combine stack protectors, position-independent code, fortified library checks, control-flow protection and sanitizers. AddressSanitizer adds red zones and metadata to detect invalid accesses during testing; UndefinedBehaviorSanitizer instruments selected language violations. Memory-safe languages reduce whole categories of bugs by enforcing bounds and lifetimes, although foreign-function interfaces and unsafe blocks remain relevant. Kernel self-protection adds read-only data, strict executable permissions and randomisation [134].

Concept map: stack & exploitation



Reading the map. Because `call`/`ret` keep the return address on the stack next to the locals, a buffer overflow can overwrite it and hijack control. Mitigations (NX, canary, ASLR, CET) each block one step; ROP defeats NX by reusing `ret`-terminated gadgets, which makes return-address integrity a separate protection requirement.

2.7.8 Worked problems

Push/pop trace

Question. If `rsp=0x1000` and `rax=0x55`, what changes after `push rax`?
A 64-bit push first reserves eight bytes, giving $\texttt{0x1000} - 8 = \texttt{0x0FF8}$. It then stores the qword at that new address. In little-endian order, bytes `0x0FF8` through `0x0FFF` become `55 00 00 00 00 00 00 00`; `rax` remains `0x55`.
A subsequent `pop rcx` would read the qword into `rcx` and restore `rsp=0x1000`, leaving the stored bytes in memory. Similarly, a push beginning at `0x2008` stores at the new top `0x2000`. The pointer movement determines which slot each operation uses.

Swap top two

Question. Exchange the top two qwords A and B using pops and pushes. The stack interval is readable and writable, and `rax`/`rcx` may change.

```
pop rax
pop rcx
push rax
push rcx
```

Let the initial stack pointer be S, with $[S] = A$ and $[S+8] = B$. The first pop saves A and advances `rsp` to $S+8$. The second saves B and advances it to $S+16$.

Worked example (continued)

Pushing A writes it at $S + 8$; pushing B writes it at S. Thus `rsp` returns to S, with B on top and A in the following slot.
Both original values are saved before either slot is overwritten. During the sequence those slots hold data for this operation; an intervening return must not consume one as a control-flow address.

Frame local

Question. After `push rbp; mov rbp,rsp; sub rsp,32`, where can an eight-byte local fit relative to `rbp`?
The reserved local interval is $[\texttt{rbp} - 32, \texttt{rbp})$. A qword at `[rbp-8]` occupies offsets -8 through -1, entirely inside that interval. The saved caller frame pointer begins at offset 0, so the two objects do not overlap. A second qword can occupy offsets -16 through -9 using `[rbp-16]`.
Once established, `rbp` keeps these local offsets stable while temporary adjustments move `rsp`. The allocation provides four nonoverlapping qword slots; the compiler or handwritten routine still decides which objects use them.

Call alignment

Question. A caller has `rsp=0x1000` and pushes one qword of scratch data. What padding is needed before an ordinary scalar System V AMD64 call, and where is `rsp` at callee entry?
The scratch push leaves `rsp=0x0FF8`, whose remainder modulo 16 is 8. Subtracting another eight bytes gives `0x0FF0`, the required aligned call-site value. The call then pushes its eight-byte return address, so the callee enters at `0x0FE8`, with remainder 8. Adding eight to that entry value recovers the aligned call-site address.
After the callee returns, `rsp` is `0x0FF0`; removing the padding and popping the scratch value restores `0x1000`. Two qword pushes from the initial address would already leave `0x0FF0` and need no extra padding. If a qword carries a stack argument, place alignment padding before that argument so the argument remains at its ABI-defined offset from the return address.

Exercise 2.7.2 — Stack growth

Does push increase or decrease RSP?

Worked solution

For the ordinary 64-bit push studied here, the stack grows toward lower addresses. The instruction subtracts 8 from `rsp`, then stores the qword at the new top. If old `rsp=0x2000`, the new value is `0x1FF8`, and the stored bytes occupy `0x1FF8` through `0x1FFF`. The arithmetic on `rsp` must be distinguished from the value being stored.

Exercise 2.7.3 — Discard top

How can the top qword be discarded without using its value?

Worked solution

To discard one 8-byte stack slot, execute `add rsp,8`, provided the task permits changing `rsp` this way. If the old top is `0x1FF8`, the new top becomes `0x2000`. The bytes remain physically in memory; they are simply outside the current active stack extent. No register receives the discarded value. A later return still needs the correct return-address slot at the resulting stack top.

Exercise 2.7.4 — Return address

Where does call place the return address?

Worked solution

A near 64-bit `call` pushes the address of the following instruction. It decreases `rsp` by 8, stores that return address at the new top, and transfers control to the target. If old `rsp=0x2000`, the return address is stored at `0x1FF8`. A matching `ret` loads the address from that slot and advances `rsp` by 8. The saved return address identifies the caller's continuation immediately after the call.

Exercise 2.7.5 — Frame teardown

What does leave conceptually perform?

Worked solution

With a conventional frame pointer, `leave` restores the stack pointer from `rbp` and then pops the caller's saved frame pointer. Conceptually it performs `mov rsp,rbp` followed by `pop rbp`. The first operation discards local stack allocation; the second restores the older frame pointer and moves `rsp` to the return-address slot. A following `ret` then consumes that address. The explanation assumes the matching frame prologue established this layout.

Summary

A defensible analysis of stack corruption identifies the exact write primitive, the reachable objects, the integrity checks performed by the program and the control transfer eventually consumed. The same state-based method supports vulnerability assessment, mitigation testing and secure code review. The analysis remains tied to the observed binary, input bounds and active protection mechanisms.

2.7.9 Exercises

Exercise 2.7.6 — Frame offsets

After `push rbp; mov rbp,rsp; sub rsp,0x40`, where are the saved `rbp`, return address and a local at `[rbp-0x18]` relative to `rbp`?

Go to the solution →

Exercise 2.7.7 — Distance to control data

A byte buffer begins at `[rbp-0x30]`. How many bytes separate its first byte from the saved return address at `[rbp+8]`?

Go to the solution →

Exercise 2.7.8 — Match mitigation to purpose

Match canary, NX, ASLR and PIE to: randomise library/stack positions; detect sequential frame overwrite; make data non-executable; allow the main executable to be randomised.

Go to the solution →

Exercise 2.7.9 — Limits of canary detection

Explain two memory-corruption cases a stack canary may not stop.

Go to the solution →

Exercise 2.7.10 — Trace RET

If `rsp=0x7000` and memory`[0x7000]=0x401234`, state the effect of `ret`.

Go to the solution →

Exercise 2.7.11 — Safer copy design

A destination array has capacity 32 bytes and must contain a terminated string. What maximum source length can be copied, and what invariant should the code enforce?

Go to the solution →

Exercise 2.7.12 — Defensive binary audit

You find `lea rdi,[rbp-0x20]` followed by a call to an unbounded input routine. List the evidence still needed before judging exploitability.

Go to the solution →

2.7.10 Solutions

Stack Frames, Memory Safety, and Control-Flow Reuse

Solution 2.7.6

Saved `rbp` is at offset 0, return address at +8, and the local starts at `-0x18`. The allocated frame extends down to `rbp-0x40`.

← Return to the exercise

Solution 2.7.7

Subtract the buffer start from the return-slot start:

$$(\texttt{rbp} + 8) - (\texttt{rbp} - \texttt{0x30}) = 8 + 48 = 56 = \texttt{0x38} \text{ bytes.}$$

The frame pointer cancels, so the distance is independent of the frame's absolute address. Offset 56 from the buffer start reaches the first byte of the saved return address. The declared buffer size and any padding or canary inside this interval must be determined separately from the actual object layout.

← Return to the exercise

Solution 2.7.8

ASLR randomises mappings; a canary detects many frame overwrites; NX prevents instruction fetch from pages marked non-executable; PIE makes the main executable relocatable for ASLR.

← Return to the exercise

Solution 2.7.9

A write that corrupts adjacent data but does not cross the canary can still change program logic. Heap corruption and use-after-free occur outside the protected stack frame. A canary also acts only when the function reaches its checked epilogue.

← Return to the exercise

Solution 2.7.10

The instruction pointer becomes `0x401234` and `rsp` becomes `0x7008`. This simple pop-into-control-flow behaviour is why overwriting return-address stack data is dangerous.

← Return to the exercise

Solution 2.7.11

If the source contains n content bytes, its terminated representation requires $n + 1$ bytes. A capacity of 32 therefore permits at most $n = 31$ content bytes, with the terminator stored at index 31.
Enforce `n < 32` before copying or calculating a size that includes the terminator. On acceptance, copy only the n content bytes and write `'\0'` at `dst[n]`; equivalently, copy $n + 1$ bytes from a valid terminated source after the bound has been established. At $n = 31$, the highest written index is 31. At $n = 32$, reject before writing, since termination would require index 32 outside the object. The source must be readable for the chosen copy length, and an operation such as `memcpy` also requires nonoverlapping ranges.

← Return to the exercise

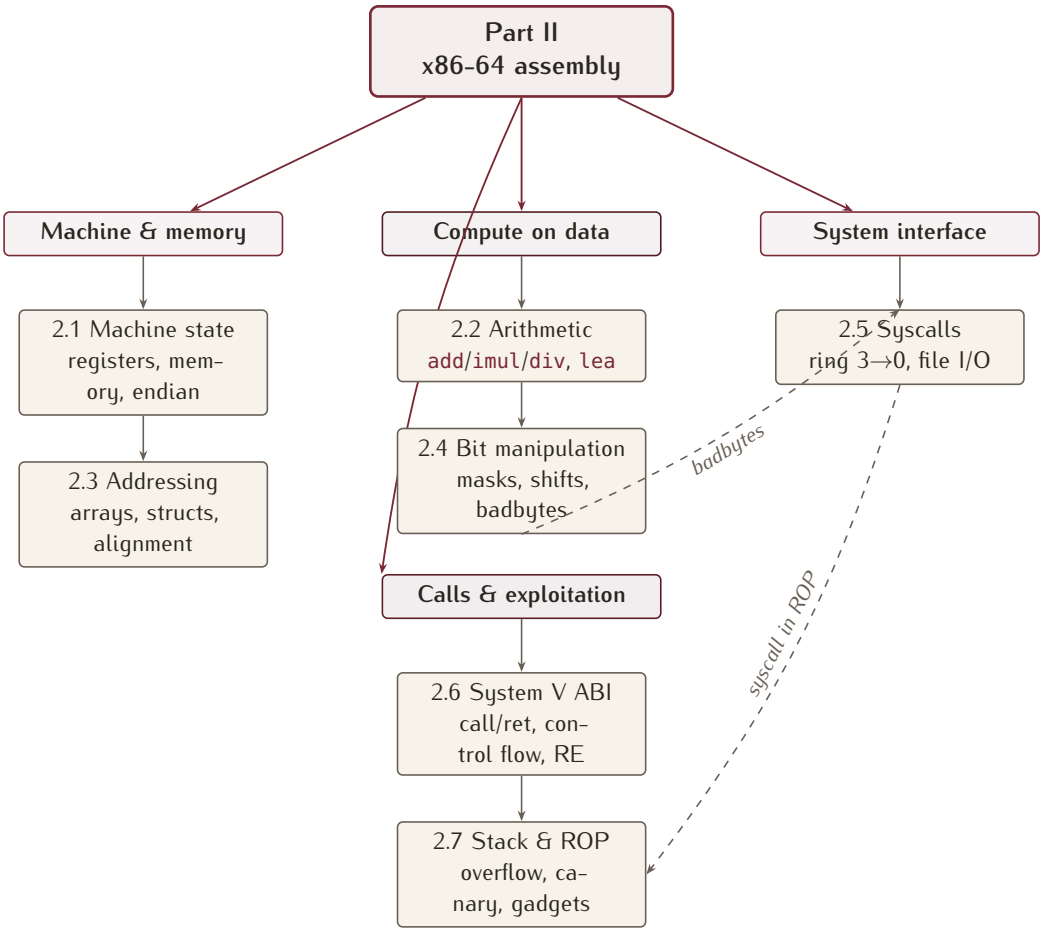
Solution 2.7.12

Determine the actual destination size, call semantics, maximum attacker-controlled length, stack layout, canary presence, saved-control offsets, NX/PIE/ASLR status, available information leaks, process privileges and whether the faulty path returns. The unbounded interface requires a proof that every feasible input fits the actual destination. The observed call pattern alone does not establish a reachable overflow or its exploitability.

← Return to the exercise

Integrated map of machine-level programming

Part II follows one arc: from *what the machine is* to *what it computes* to *how it talks to the system* to *how that machinery is called and abused*. The four coloured groups below are those four stages.



The throughline. Chapters 2.1 and 2.3 establish registers, byte-addressed memory and effective addresses. Chapters 2.2 and 2.4 operate on that state through arithmetic and bit manipulation. Chapter 2.5 crosses the user–kernel boundary, while Chapter 2.6 explains how functions preserve enough structure to be called and reconstructed. This chapter shows why that same structure must be protected: corrupted stack control data can redirect execution, and a system call may become the final effect of a reuse chain.

3

PROCESSOR ORGANISATION, TIMING, AND PIPELINING

An instruction-set manual defines the behaviour visible to software. Circuit organisation and timing depend on the implementation. The missing bridge begins with Boolean functions, adds state through feedback and clocks, and then organises those elements into datapaths that implement an architectural contract.

Y86-64, SEQ and PIPE provide a deliberately small teaching system introduced by Randal Bryant and David O'Hallaron [1]. Their small instruction set makes the entire datapath and its control equations available for analysis. Instructions are first specified as state transitions. A sequential datapath realises those transitions directly; a pipelined datapath overlaps them and must resolve timing, data and control dependencies. Comparisons with production processors use broader architecture texts [49, 50, 51].

Correctness remains observational: after committed instructions, the implementation must expose the architectural state prescribed by the ISA. Timing adds a second obligation: every clock period must cover the longest admissible combinational path, storage overhead and design margin. Throughput matters only after both obligations are met.

3.1 COMBINATIONAL LOGIC: FROM TRUTH TABLES TO THE ALU

Three input bits settle at the entrance of a circuit. A short time later, its output must indicate whether a majority of those bits is one. The truth table defines the required answer; gates, selectors and adders provide ways to realise it. We will trace signals through these blocks and examine how equivalent logical descriptions can lead to different physical paths and delays.

Selection and arithmetic meet in a datapath block

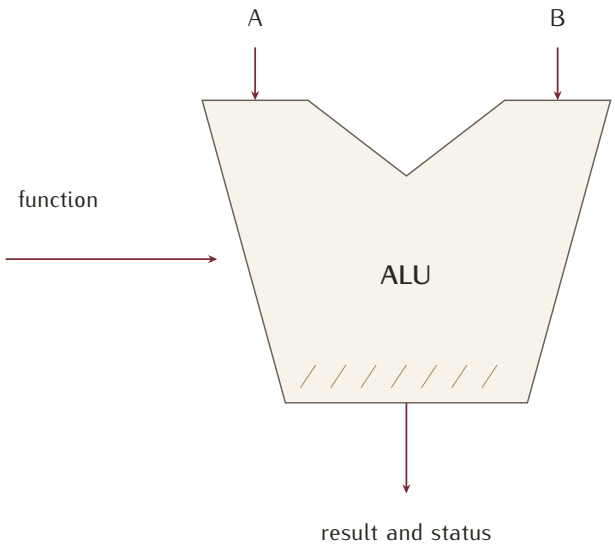


Figure 3.1: *The ALU symbol gathers two operands and a function selection into a result and status computation. Its internal Boolean implementation can share circuitry among operations. The following drawings expand selection, addition and timing.*

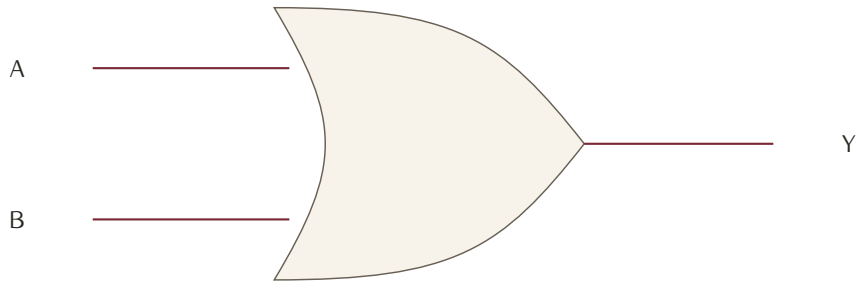
3.1.1 From Boolean specification to datapath blocks

Part I introduced gates, truth tables and bitwise operators as abstract Boolean objects. The next step is to compose them into the arithmetic core of a processor: multiplexers, adders and the arithmetic-logic unit. The guiding question is how increasingly complex blocks can be built from elementary components without introducing stored state.

A *combinational circuit* has no stored logical state: once propagation has settled, the same input values produce the same output values. Its Boolean specification is therefore a function of the current inputs, although a physical implementation still has delay and transient behaviour. A *sequential* circuit differs because it contains state and its output can depend on earlier inputs (Section 3.2). On their own, combinational circuits suffice for arithmetic, choosing among alternatives, decoding instructions: everything that needs no memory.

The hardware-design model used here is deliberately structural. It distinguishes *combinational circuits*, networks of gates with no feedback that compute a Boolean function, from the memory elements of 3.2, and it introduces *HCL* (Hardware Control Language) to describe them. Three HCL constructs map onto this design: Boolean expressions over gates, the *multiplexer* written as a `[ ]` case expression, and *word-level* circuits such as buses and word multiplexers from which the ALU is assembled. The resulting ALU is "several combinational blocks plus a selector", the model used below.

A combinational block implements a total Boolean rule



The stable output is one when at least one input is one.

Figure 3.2: *An OR gate maps the current input combination to its Boolean output. The functional drawing contains no stored state. Propagation timing is a separate property of the physical implementation.*

A gate is a pull-up and pull-down network

AND, OR, NAND and NOR are primitive Boolean functions.

Static CMOS realises a gate with complementary transistor networks. The pull-down network creates a conducting path to ground for input combinations that require output 0; the pull-up network creates the complementary path to the supply for output 1. Series devices increase resistance, while parallel branches increase capacitance and layout area. NAND and NOR map naturally to single complementary stages; ordinary non-inverting AND and OR cells can add an inverter to the corresponding NAND or NOR stage.

Logically equivalent expressions can have different transistor counts, delays and energy. Synthesis tools therefore optimise a Boolean function against a physical cell library rather than treating every gate symbol as identical [41, 42].

A k-input function is fully described by its truth table (2^k rows). The standard synthesis is the *sum-of-products* normal form: for each row where the output is 1 (a *minterm*), write an AND of the inputs (complemented where they are 0); OR all the products.

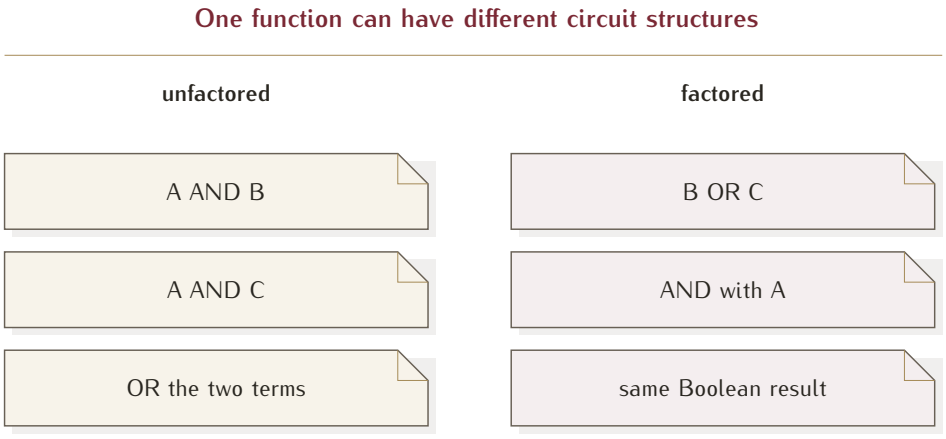


Figure 3.3: *The two structures implement the same truth table. Their gate counts, fan-out and path delays can differ. Boolean equivalence permits the transformation; implementation goals determine whether it is desirable.*

Worked example

Problem. Each of x, y, z is either 0 or 1 and occupies a complete register. Compute the *majority* function: output 1 if at least two inputs are 1.

Solution. The function is 1 on rows 011, 101, 110, 111, so

$$\text{maj} = (\bar{x}\,y\,z) \vee (x\,\bar{y}\,z) \vee (x\,y\,\bar{z}) \vee (x\,y\,z) = (x\,y) \vee (x\,z) \vee (y\,z),$$

where the simplified form follows by grouping each two-one row with 111. For example, $\bar{x}yz + xyz = yz(\bar{x} + x) = yz$; the other pairs give xz and xy. Reusing the 111 term in these groups preserves the function because Boolean OR is idempotent. In x86-64:

```
mov rax, rdi; x
and rax, rsi; x AND y
mov rcx, rdi
and rcx, rdx; x AND z
```

Worked example (continued)

```
or rax, rcx
mov rcx, rsi
and rcx, rdx; y AND z
or rax, rcx; rax = maj
and rax, 1
```

Normal form and minimisation. The sum of all selected minterms is canonical for a fixed variable order. A simplified sum-of-products need not be canonical. Reducing gate count can save area; delay also depends on logic depth, fan-out and wiring. Simplification uses absorption laws and minimisation methods: Karnaugh maps are convenient for small manual cases; Quine–McCluskey provides exact algorithmic minimisation but scales poorly, so large designs normally rely on heuristic logic synthesis. The dual is *product-of-sums* (group the 0 rows, AND of ORs). Both compute the same function with different circuit structures; which is preferable depends on the implementation technology.

3.1.2 Selection with multiplexers

A *multiplexer* (mux) routes one of several data inputs to a single output, selected by control bits. A 2-to-1 mux has data D_0, D_1 and select S: output D_0 if $S = 0$, D_1 if $S = 1$, i.e. $Q = \bar{S}\,D_0 \vee S\,D_1$.

Worked example

Problem. `rdi` $= D_0$, `rsi` $= D_1$, `rdx` = select, with select in $\{0, 1\}$. Leave D_0 in `rax` if select $= 0$, else D_1.

Solution. **Branchless, with a mask** (using `neg` to splat a bit across the word: $-0 = 0$, $-1 =$ all-ones):

```
mov rax, rdi; D0
mov rcx, rdx
and rcx, 1
neg rcx; 0 or 0xFFFF...FF
mov r8, rsi
xor r8, rdi; D1 XOR D0
and r8, rcx; keep iff select=1
xor rax, r8; select=0 -> D0; select=1 -> D1
```

With `cmov` (more readable, using the low select bit): `test rdx, 1`; `mov rax, rdi`; `cmovnz rax, rsi`.

Both instruction sequences select a complete word without a conditional branch. Timing properties require analysis of the full generated code, its memory accesses and the target processor; branch avoidance alone is insufficient.

The same construction extends naturally to four inputs. For $D_0 \dots D_3$ and two select bits $s_1 s_0$:

$$Q = \bar{s_1}\bar{s_0}D_0 \vee \bar{s_1}s_0D_1 \vee s_1\bar{s_0}D_2 \vee s_1s_0D_3,$$

i.e. interpret $s_1 s_0$ as a number $0 \dots 3$ and select that input. A software implementation can select from an array ordered D_0, D_1, D_2, D_3 using a checked index in $0 \dots 3$, or use a cascade of `cmov`. An indexed memory access introduces address-dependent behaviour absent from the register-only examples.

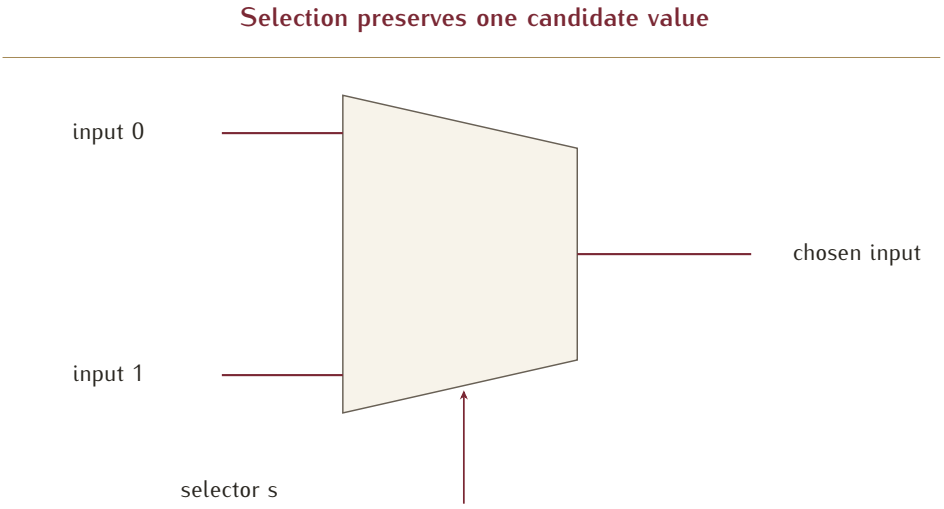


Figure 3.4: *The multiplexer chooses one input under control of the selector. A word-wide multiplexer repeats the same choice for each bit, allowing the processor to route complete values between alternative sources.*

In HCL, a word-valued case expression lists conditions in priority order:

```
word Q = [
    S == 0 : D0;
    1      : D1;
];
```

The first true condition selects its value. The final condition `1` supplies a default, so every input is covered. For the one-bit selector S, this describes the same mux as the

gate equation. HCL describes a combinational selection network; the priority of the conditions specifies its function [1].

A multiplexer is a controlled signal path

A multiplexer chooses one input according to a selector.

The circuit can be built from gates or transmission devices that connect one data path while isolating the others. A wide word multiplexer repeats this structure for every bit and distributes the selector to many cells. Each extra input increases data and control wiring and often adds logic depth.

Datapaths with many forwarding or operation choices can become dominated by multiplexer delay. Adding a logical option to a processor is therefore also a physical routing and timing decision [42, 51].

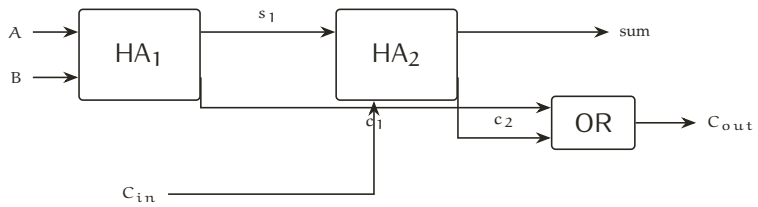
3.1.3 Arithmetic and ALU construction

The *half adder* sums two bits: $\text{sum} = A \oplus B$, $\text{carry} = A \wedge B$. The *full adder* adds a carry-in:

$$\text{sum} = A \oplus B \oplus C_{in}, \qquad C_{out} = (A \wedge B) \vee (C_{in} \wedge (A \oplus B)).$$

The sum is 1 when an odd number of inputs is 1 (three-input XOR); the carry-out is 1 when at least two of three are 1, the *majority* function above.

A full adder is cleanly built from two half adders plus one OR gate:



The first half adder forms $s_1 = A \oplus B$ and $c_1 = A \wedge B$; the second adds C_{in} to s_1, giving the final sum and c_2; the OR of c_1, c_2 is the carry-out.

Worked example

Problem. Bits A, B, C_{in}, each 0 or 1, occupy `rdi`, `rsi`, `rdx`. This is a circuit-emulation fragment with an explicit output convention, rather than a callable System V function. Leave the sum bit in `rax`, the carry-out in `rbx`.

Solution.

```
mov rax, rdi
xor rax, rsi
xor rax, rdx
and rax, 1; sum = A XOR B XOR Cin
mov rbx, rdi
```

Worked example (continued)

```
and rbx, rsi; A AND B
mov rcx, rdi
xor rcx, rsi
and rcx, rdx; Cin AND (A XOR B)
or rbx, rcx
and rbx, 1; cout
```

A full adder has eight input combinations

	A,B,Cin	sum	carry
0	000	0	0
1	001 / 010 / 100	1	0
2	011 / 101 / 110	0	1
3	111	1	1

Rows group combinations by the number of input ones.

Figure 3.5: *The sum bit gives the parity of the three input bits, and the carry bit records whether at least two are one. The grouped table accounts for all eight combinations through the invariant A + B + Cin = sum + 2 carry.*

Chaining n full adders through the carry gives an n-bit *ripple-carry adder*.

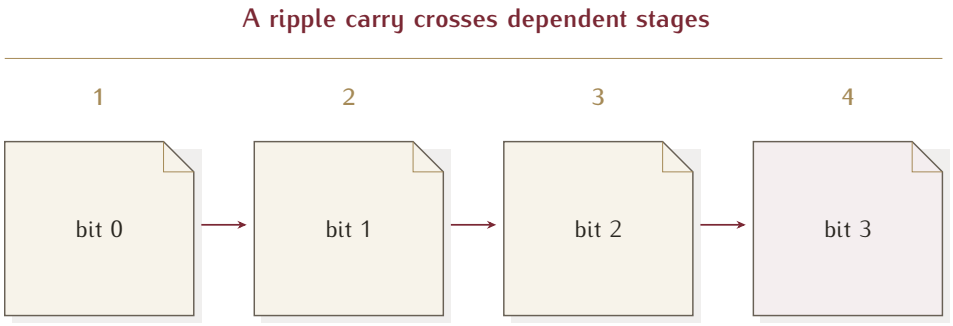


Figure 3.6: *Each full-adder stage uses the carry produced by the previous stage. The dependence explains why a ripple-carry design can have a delay that grows with word width, even though all operand bits arrive together.*

A ripple-carry adder has a simple regular layout, but its worst-case delay grows linearly with the word width. Each bit may have to wait for the carry from the previous bit. Define, for bit i,

$$p_i = a_i \oplus b_i, \qquad g_i = a_i b_i,$$

where p_i means "this position propagates an incoming carry" and g_i means "this position generates a carry". In the following Boolean equations, juxtaposition denotes AND and $+$ denotes OR. Then

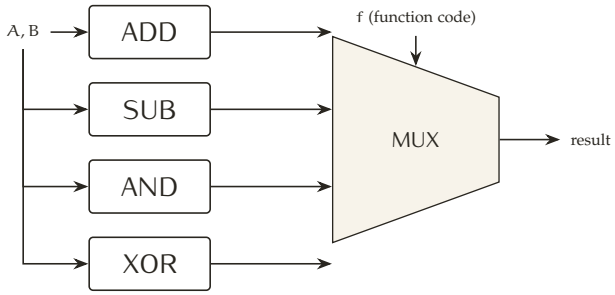
$$c_{i+1} = g_i + p_i c_i.$$

Expanding gives, for example,

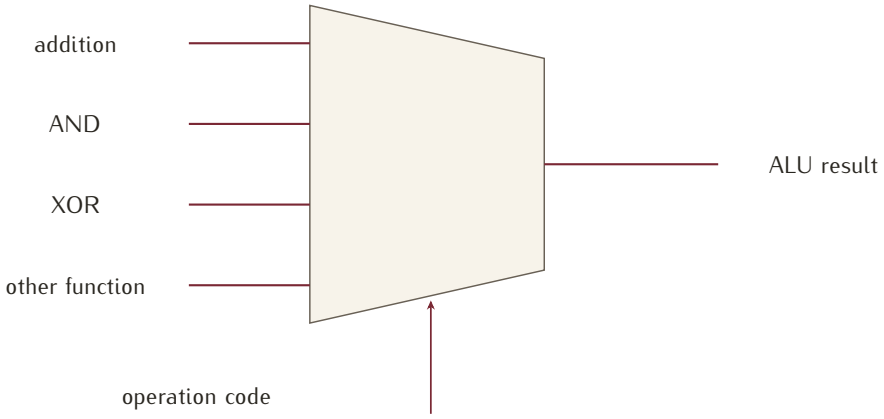
$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0.$$

Hierarchical carry-lookahead and parallel-prefix networks combine generate and propagate information in a tree. With bounded-fan-in gates, suitable tree structures achieve logarithmic logical depth. Their physical delay also includes fan-out and interconnect. Ripple, prefix and carry-select designs offer different area, energy and timing costs; the ISA does not prescribe one adder structure.

The *Arithmetic Logic Unit* (ALU) is the CPU's central arithmetic block. For operands A, B and a function code f, it produces add/sub/and/or/xor/shift... Here A = `rdi`, B = `rsi`, so *sub* means $A - B$. In the teaching model, several combinational blocks produce candidate results and a multiplexer selects one by f:



An ALU exposes alternative candidate functions



The selected result must agree with the instruction's function.

Figure 3.7: *This functional view separates the candidate operations from the selection logic. A real ALU may share circuitry among operations. The picture defines the routing requirement without claiming one particular transistor implementation.*

The ALU used throughout this chapter has the following function-code contract:

Control	Operation	Typical use
00	$A + B$	address calculation, addition
01	$A - B$	subtraction, comparison
10	$A \wedge B$	masking
11	$A \oplus B$	toggling, equality preparation

The implementation can compute all candidate results in parallel and select one with a multiplexer, or reuse shared gates to save area. Both implementations satisfy the same function table, with different physical costs and delays.

Worked example

Problem. `rdi`, `rsi` operands; the low 2 bits of `rdx` select: 0 add, 1 sub, 2 and, 3 xor. Result in `rax`.

Solution. Compute all four, then select with a `cmov` cascade (mirroring the mux):

```
mov r10, rdx
and r10, 3
mov rax, rdi
add rax, rsi; default: A + B
mov r8, rdi
sub r8, rsi; A - B
mov r9, rdi
and r9, rsi; A AND B
mov r11, rdi
xor r11, rsi; A XOR B
cmp r10, 1
cmove rax, r8
cmp r10, 2
cmove rax, r9
cmp r10, 3
cmove rax, r11
```

The sequence emulates the selected result. Its final x86 flags come from the last selector comparison, so a complete ALU emulator would need to compute the selected operation's status bits separately.

3.1.4 From combinational blocks to processor control

A processor datapath is built from combinational blocks connected to sequential state. Combinational logic computes outputs from current inputs; sequential elements store values across clock edges. The ALU, multiplexers, decoders and adders in this chapter are therefore the building blocks that later become fetch logic, address computation, branch-condition evaluation and writeback selection.

A mux expresses conditional selection without sequential control flow. An adder expresses arithmetic on fixed-width words. A decoder turns a compact code into one-hot control signals. These blocks are used repeatedly in a CPU: selecting a register input, choosing the ALU operand, deciding the next PC, selecting a writeback destination, and enabling memory access.

A control signal is a Boolean or small integer value computed from instruction fields and status information. For example, a signal named `need_regids` might be true when the current instruction has a register byte. A signal named `mem_read` might be true for load, pop and return instructions. These signals are Boolean functions of the decoded instruction.

Worked example

Let `icode` identify an instruction class in the simplified Y86-64 model developed in Chapter 3.3. A load (`mrmovq`), stack pop (`popq`) and return (`ret`) each read memory. Their shared control signal is

$$\text{memRead} = (\text{icode} = \text{mrmovq}) \vee (\text{icode} = \text{popq}) \vee (\text{icode} = \text{ret}).$$

In hardware this becomes comparison logic followed by OR gates.

3.1.5 Timing, cost, and physical implementation

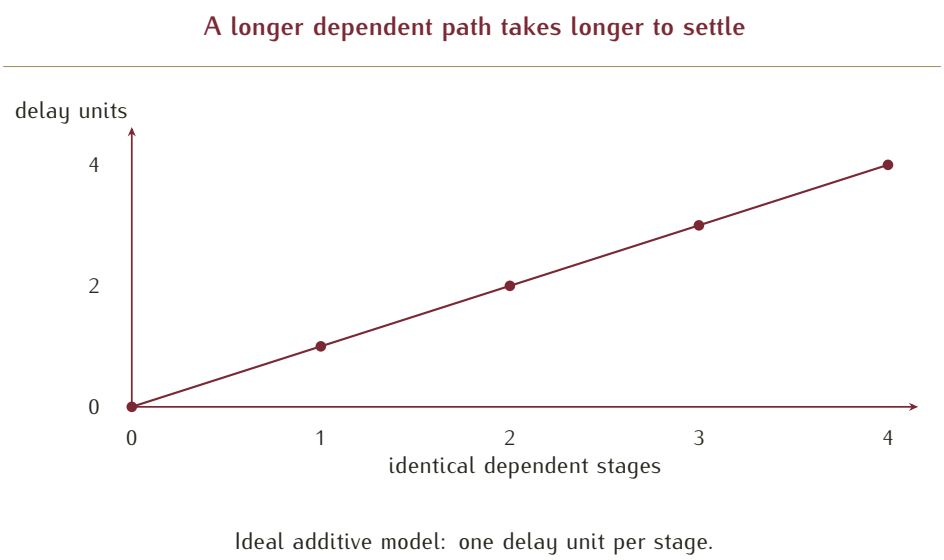


Figure 3.8: *The graph illustrates accumulated delay along a chain of dependent gates under equal-delay assumptions. Real gate delays depend on technology, loading and transition direction; parallel paths contribute through the slowest relevant path.*

A combinational circuit has a propagation delay. The output becomes valid only after signals have passed through a chain of gates. The longest such chain is the critical path. In processor design, the critical path helps determine the minimum clock period for a sequential implementation.

Logic minimisation is useful. Processor design also requires placing the right operations in the right stage so that the critical path is manageable and the control logic remains understandable.

> **Caution**
>
> Do not confuse the algebraic expression for a function with its timing. Two circuits can compute the same Boolean function but have different delays because their gate structures differ.

A single gate acts on one bit, so building circuits that act on whole words is a matter of replicating gates across bit positions, though not always in the same way. The simplest case is pure parallelism: a 64-bit AND is just sixty-four AND gates arranged side by side, each acting on its own bit position. A word-level multiplexer is built the same way, one bit-level mux per position, all driven by a shared select signal so that the whole word is switched at once. An adder is the interesting exception, because the bits are not independent: it is a network that must propagate carry information from one position to the next, which is what makes addition more than sixty-four copies of a one-bit operation.

An ALU computes a selected arithmetic or logical operation and may also compute status information. For addition and subtraction, overflow and sign information are meaningful. For logical operations, zero and sign may still be meaningful, while overflow is normally cleared or ignored depending on the ISA.

In Y86-64, condition codes are set by OPq instructions and later read by conditional moves and jumps. Therefore the ALU both computes values and prepares information for control flow.

> **Caution**
>
> A diagram of an ALU with several operations should be read as a mux after parallel or selectable computations. The operator code is a control input that chooses which result becomes the ALU output.

At scale, interconnect can dominate gate delay

An idealised schematic can treat a wire as a connection with zero delay and no energy cost.

On-chip metal has resistance and capacitance distributed along its length. Long global wires need repeaters, wider metal layers or additional pipeline stages. Nearby switching lines can also change one another's delay through capacitive coupling.

Modern processor organisation is strongly shaped by distance. Clusters, banks and hierarchical networks keep common communication local because moving a signal across the die can cost more time and energy than computing a simple function [42, 50].

3.1.6 Alternative implementations and status flags

Worked example: exactly two of three inputs

Let $E(x, y, z)$ be 1 exactly when two inputs are 1. The marked rows are 011, 101 and 110, hence

$$E = \bar{x}yz + x\bar{y}z + xy\bar{z}.$$

This expression cannot absorb the 111 row, because that row must remain 0. A useful check is to substitute $x = y = z = 1$: every term contains one complemented variable, so the result is 0. In assembly, one implementation can compute the population count $x + y + z$ and compare it with 2; another can evaluate the three minterms. The first is often clearer at word level, while the second mirrors the gate-level specification.

NAND and NOR are *functionally complete*: either gate alone can express every Boolean function. For example, $\neg x = x \,\mathrm{NAND}\, x$, and an AND is a NAND followed by self-NAND inversion. Universality does not imply that every implementation should use one gate type. Standard-cell libraries contain many cells because delay, area, fan-out and power all matter.

A multiplexer offers another universal construction. A 2^n-to-1 multiplexer whose select lines are the n inputs can implement any n-input single-output function by tying each data input to the corresponding truth-table value. This is the hardware analogue of storing the table as a bit mask and indexing it with the input tuple.

Worked example: a truth table as an eight-bit constant

Number the rows by $i = 4x + 2y + z$. Store the output of row i in bit i of a constant T. Then

$$f(x, y, z) = (T \gg i) \,\&\, 1.$$

For majority, the 1-rows are $3, 5, 6, 7$, so $T = 2^3 + 2^5 + 2^6 + 2^7 = \texttt{0xe8}$. The method also reveals a lookup table as a combinational circuit: the address selects one stored bit.

A combinational network has no stored state: after its inputs change, signals propagate until the outputs settle. The model deliberately ignores transistor details and keeps the timing fact that matters at the architectural level: a clocked state element must not sample the result before the worst relevant path has stabilised.

A single adder can perform subtraction by using two's complement:

$$x - y = x + (\neg y) + 1.$$

Hardware therefore complements the second operand when the function code requests subtraction and sets the initial carry-in to 1.

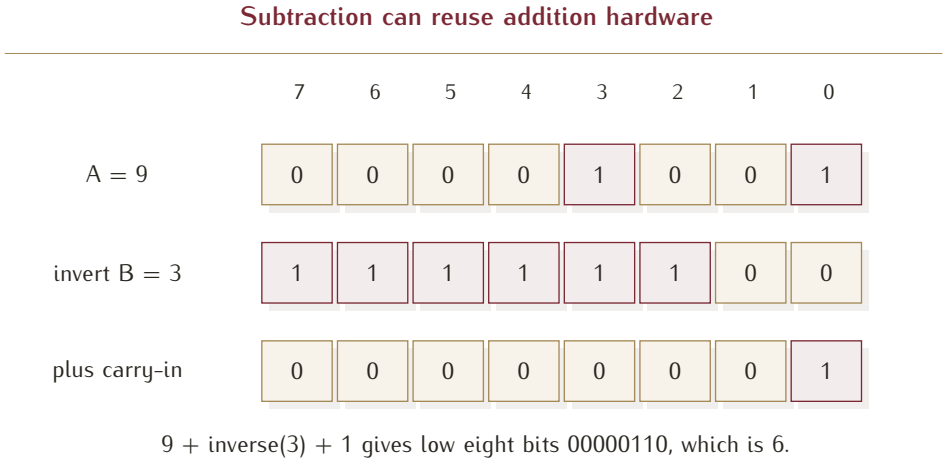


Figure 3.9: *Complementing B and supplying an initial carry of one transforms an adder into a two's-complement subtractor. The result-width and flag interpretation remain part of the arithmetic contract.*

The same result drives condition-code logic. For a w-bit subtraction $r = x - y$:

- ZF is 1 when every result bit is 0;
- SF is the most significant result bit;
- signed overflow occurs when x and y have opposite signs and the result sign differs from x;
- for $x + \neg y + 1$, an unsigned borrow occurs exactly when the final carry-out is 0; x86 records this borrow in CF.

The comparison operation does not need to preserve r as a general-purpose result. It needs only the flags derived from that result.

Worked example: signed less-than from flags

For two's-complement subtraction, $x < y$ in signed arithmetic exactly when $\mathrm{SF} \oplus \mathrm{OF} = 1$. If no overflow occurs, the sign of $x - y$ is trustworthy. If overflow occurs, the sign bit is inverted relative to the mathematical result, and XOR with OF repairs it. Equality is tested separately with ZF.

A combinational output does not change mathematically "at once". Every gate has propagation delay, and different paths can have different lengths. During an input transition, a logically stable output can briefly glitch because one path arrives before another. Synchronous systems tolerate many such transients by sampling only after the

circuit has settled. Asynchronous interfaces and clock signals require more care because a transient may itself be interpreted as an event.

Worked example: estimating a critical path

Suppose a mux input passes through two AND gates, one OR gate and the final selection gate. If their worst-case delays are 35 ps, 35 ps, 45 ps and 30 ps, the path delay is 145 ps. A parallel path with only 90 ps does not determine the clock period. The longest sensitizable path does. Adding one “small” gate to the long path can reduce the maximum frequency even when the gate count changes little.

Laboratory 17: An exhaustive Boolean test

Verify “exactly two of x,y,z” with a Boolean expression, arithmetic test and lookup mask. Then determine whether reversing variable order can reveal a bug for this particular symmetric function, and supply an asymmetric test if needed.
Worked solution

Laboratory 17: solution

Use $(x y \bar{z}) \vee (x \bar{y} z) \vee (\bar{x} y z)$. With $i = 4x + 2y + z$, true rows are 3,5,6, giving mask `0x68`. Verification is:

```
for x in range(2):
    for y in range(2):
        for z in range(2):
            i=4*x+2*y+z
            a=(x and y and not z) or (x and not y and z) or (not x and y and
↪   z)
            assert int(a)==int(x+y+z==2)==((0x68>>i)&1)
```

Reversing x and z leaves this symmetric function unchanged, so this test cannot expose that index-order error. For an asymmetric function use $f = x \wedge \bar{z}$, whose true rows are 4 and 6 and mask is `0x50`. At $(x, y, z) = (1, 0, 0)$, correct index 4 returns one; reversed index 1 returns zero. An asymmetric specification is therefore needed to test this variable-order convention.
Return to the laboratory

From relay logic to the integrated arithmetic logic unit

Timeline

Year	Machine, standard or event	Architectural significance
1937	**Relay logic design**	Boolean equations guide switching-network construction.
1947	**Transistor**	A solid-state switching device replaces bulky valves and relays.
1970	**74181**	An integrated ALU makes word-level combinational logic modular.

The arithmetic logic unit emerged through advances in describing calculation as networks of elementary decisions and manufacturing those networks reliably. Mechanical calculators by Blaise Pascal in the 1640s and Gottfried Wilhelm Leibniz in the 1670s embodied addition and carry in wheels and gears. They already contained the functional idea of an adder, but the mechanism was inseparable from decimal notation and mechanical motion. In the nineteenth century, George Boole's *An Investigation of the Laws of Thought* (1854) supplied an algebra for propositions. His algebra later supplied a language for describing devices with two stable conditions [20, 67].

The switching-network method introduced in Chapter 1.5 now scales to datapath blocks. An ALU contains several candidate functions and selection logic; their Boolean descriptions determine the result, while their physical implementation determines path delay and load. The historical problem changes from finding an equivalent contact network to building a useful arithmetic unit from available components [21].

Relay computers such as Konrad Zuse's Z3, completed in 1941, and the Harvard Mark I, operational in 1944, demonstrated automatic digital calculation, but relays were slow and mechanically finite-lived. Vacuum tubes removed moving contacts and allowed electronic switching. ENIAC, completed in 1945, used about 18,000 tubes and organised arithmetic around twenty decimal accumulators plus specialised multiplication, division and square-root units. Its decimal representation reminds us that electronic switching did not automatically imply binary arithmetic. Binary became dominant because two-state circuits were easier to compose and because control and arithmetic could share the same Boolean design methods [88, 87].

The point-contact transistor, demonstrated by John Bardeen and Walter Brattain at Bell Laboratories in December 1947 within William Shockley's research group, reduced power and size while improving reliability. Early transistor computers still assembled individual devices on boards. Integration changed the scale. Jack Kilby demonstrated a small integrated circuit at Texas Instruments on 12 September 1958; Robert Noyce's 1959 monolithic silicon concept combined Jean Hoerni's planar process with metal interconnections suitable for manufacture. Their work made it possible to replicate complete logic functions as components rather than wire every transistor individually [100, 101].

The integrated-circuit industry soon offered standard logic families and medium-scale functional blocks. The Fairchild 903 and Texas Instruments 7400 series packaged gates; adders, multiplexers, decoders and registers followed. The 74181, introduced by Texas Instruments in 1970, is a famous four-bit arithmetic logic unit slice. It performed a set of arithmetic and logical functions selected by control inputs and provided carry-lookahead signals. Designers could cascade slices to build wider datapaths. Texas Instruments advertised the SN54/74181 in April 1970 as a four-bit arithmetic and logic block, alongside a carry-lookahead component for combining slices. Its package exposed data inputs, function selection, result lines and carry-related outputs, giving the ALU diagram a concrete physical form [102, 146].

Carry handling drove much of the engineering. A ripple-carry adder is structurally simple, but each stage waits for the previous carry. Carry-lookahead circuits derive generate and propagate conditions so groups can predict carries in parallel. Larger machines used hierarchical schemes, carry-save structures for multiplication and dedicated decimal correction where required. In every technology, the slowest combinational route constrains the clock period or the time at which a result can be trusted [51, 52, 28].

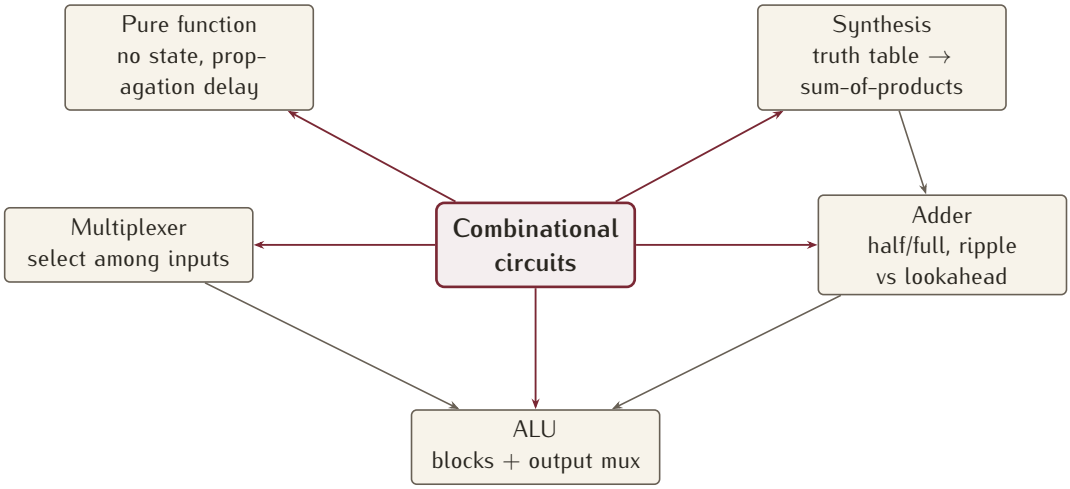
The first commercial microprocessors integrated control and arithmetic on one chip. Intel's 4004, introduced in 1971, was a four-bit CPU developed for the Busicom calculator project. The 8008 and 8080 widened and generalised the design; the 8086 of 1978 began the x86 line. As transistor budgets grew, the ALU ceased to be one indivisible block. Modern cores contain several integer execution units, address-generation units, vector pipelines, floating-point units, cryptographic units and matrix engines. Instructions are decoded into internal operations that may be scheduled to different units, and architectural flag state may be managed through internal renaming [71, 104, 28, 29].

Vector extensions show how the old truth-table model scales. Intel MMX, SSE, AVX and AVX-512 and AMD's corresponding implementations apply one instruction to packed elements. A vector adder still consists of Boolean and arithmetic circuits, but it contains lanes, bypass networks, mask handling, exception rules and physical-register connections. Intel AMX and comparable matrix engines add two-dimensional tile storage and specialised data paths. The architectural instruction names only the promised transformation; the microarchitecture may distribute it across many circuits and cycles [51, 52, 28].

Teaching an ALU as a multiplexer over add, subtract, AND, OR and XOR is therefore historically justified but intentionally compressed. It captures the combinational contract: for given inputs and a control code, a result and conditions are produced. It does not imply that a current processor contains one central four-function device. To connect the model to engineering practice, one should read Shannon for the logic method, study a historical MSI ALU such as the 74181, then compare the programmer-visible operations in the Intel or AMD manuals with execution-unit descriptions in optimisation manuals [21, 26, 27, 28, 29].

Truth tables and Boolean minimisation remain visible in RTL, synthesis reports and the functional units of a modern processor.

Concept map: combinational logic



Reading the map. A truth table synthesises (sum-of-products) into gate networks; two recurring networks are the multiplexer (selection) and the adder (arithmetic). The ALU combines them: arithmetic/logic blocks all evaluate, and an output mux selects by the function code, with no stored state and a worst-case settling time determined by the implementation.

3.1.7 Worked problems

Four-bit add

A four-bit adder reports both the retained low four bits and the carry that does not fit.

Question. When 9 and 11 are added by a four-bit adder, what low four-bit result and carry-out are produced?

The exact sum is $9 + 11 = 20 = 10100_2$. Split it at the four-bit boundary: the retained low word is $0100_2 = 4$ and the carry-out is 1. Recombining gives $1 \cdot 16 + 4 = 20$. In contrast, $7 + 6 = 13 = 01101_2$ has low result 1101_2 and carry-out 0.

ALU selector

A small ALU uses opcode 00 for addition, 01 for subtraction, 10 for AND and 11 for XOR. Its operands are A = `0xCC` and B = `0x0F`.

Question. What output does opcode 10 select?

Worked example (continued)

The code selects AND. Written at equal width, the operands are 11001100_2 and 00001111_2. Keeping a one only where both inputs have one gives

$$11001100_2 \wedge 00001111_2 = 00001100_2 = \texttt{0x0C}.$$

Only bits 2 and 3 survive. The other operation blocks may also produce candidate results, but the selector routes the AND result to the output. Changing only the opcode to 11 instead selects $11001100_2 \oplus 00001111_2 = 11000011_2 = \texttt{0xC3}$.

Exercise 3.1.1 — Majority

When is majority(A,B,C) equal to 1?

Worked solution

The output is one when at least two inputs are one. The four accepted rows are 011, 101, 110 and 111. The Boolean expression $AB \vee AC \vee BC$ detects whether any pair is simultaneously true. For 101, the term AC is one; for 100, all three pair terms are zero. This checks both a positive case and a case just below the threshold.

Exercise 3.1.2 — NAND

Express NAND using AND NOT.

Worked solution

NAND first combines the inputs with AND and then complements the result: $Y = \neg(A \wedge B)$. AND produces one only on row 11, so NAND produces zero only on that row. In order 00, 01, 10, 11, the NAND outputs are 1, 1, 1, 0. Complementing each input separately and then ANDing them would describe a different function.

Exercise 3.1.3 — Mux expression

Write the 2:1 mux expression.

Worked solution

With one-bit selector s, the expression is $Y = (\neg s \wedge a) \vee (s \wedge b)$. For $s = 0$, the first term equals a and the second is zero, yielding $Y = a$. For $s = 1$, the first term is zero and the second equals b, yielding $Y = b$. In software, normalize the inputs to one bit or use full-width masks when selecting complete machine words; a one-bit selector by itself does not mask every bit of a word.

Exercise 3.1.4 — Borrow

In 4-bit unsigned subtraction 3-5, what is the low result?

Worked solution

The mathematical difference is $3-5=-2$. Four result bits retain the value modulo 16, so $-2+16=\boxed{14}=1110_2$. Unsigned 3 is smaller than 5, which requires a borrow. As a check, $14+5=19$ and $19 \bmod 16=3$, recovering the first operand. The stored pattern and the borrow condition describe separate parts of the subtraction result.

Summary

To understand a block, move in both directions. From specification to circuit, derive Boolean relations and compose known components. From circuit to specification, identify paths, controls and output cases. Then add timing: which input change must traverse the longest chain before the output becomes trustworthy?

3.1.8 Exercises

Exercise 3.1.5 — Canonical synthesis

Construct the sum-of-products expression for a three-input function that is 1 on rows $001, 010, 100, 111$. Is this parity or majority?

Go to the solution →

Exercise 3.1.6 — Multiplexer construction

Use x and y as the select lines of a 4-to-1 mux to implement $f=x\oplus y\oplus z$. What must be connected to D_0, D_1, D_2, D_3?

Go to the solution →

Exercise 3.1.7 — Lookup mask

With index $i=4x+2y+z$ and bit i representing row i, compute the hexadecimal truth-table mask for "exactly two of three".

Go to the solution →

Exercise 3.1.8 — Full-adder trace

For $A=1$, $B=1$, $C_{in}=1$, compute the half-adder intermediate values and the final sum and carry.

Go to the solution →

Exercise 3.1.9 — Carry lookahead

Given $p_0 = 1, p_1 = 1, p_2 = 0$, $g_0 = 0, g_1 = 0, g_2 = 1$ and $c_0 = 1$, compute c_1, c_2, c_3.

Go to the solution →

Exercise 3.1.10 — Signed comparison

A subtraction produces $ZF = 0$, $SF = 1$, $OF = 1$. Evaluate signed less-than, less-or-equal and greater-than.

Go to the solution →

Exercise 3.1.11 — Critical path

Two candidate implementations have path delays: A uses 5 gates at 28 ps each; B uses 3 gates at 42 ps each plus a 20 ps mux. Which has the shorter worst path?

Go to the solution →

Exercise 3.1.12 — Design task

Describe a word-level ALU operation that outputs the smaller of signed operands A and B using a subtractor, flags and a mux.

Go to the solution →

3.1.9 Solutions

Combinational Logic: From Truth Tables to the ALU

Solution 3.1.5

Each selected row contributes a minterm that is one only on that row:

$$001 \to \bar{x}\bar{y}z, \quad 010 \to \bar{x}y\bar{z}, \quad 100 \to x\bar{y}\bar{z}, \quad 111 \to xyz.$$

ORing those terms gives

$$f = \bar{x}\bar{y}z \vee \bar{x}y\bar{z} \vee x\bar{y}\bar{z} \vee xyz = x \oplus y \oplus z.$$

The accepted rows contain either one or three set inputs, so this is odd parity. Majority accepts two or three set inputs instead. For example, row 100 gives parity 1 and majority 0, whereas row 110 gives parity 0 and majority 1.

← Return to the exercise

Solution 3.1.6

For $xy = 00$, $f = z$, so $D_0 = z$. For 01 and 10, parity is $\bar{z}$, so $D_1 = D_2 = \bar{z}$. For 11, two ones cancel and $f = z$, so $D_3 = z$.

← Return to the exercise

Solution 3.1.7

Exactly two inputs are one in rows 011, 101 and 110. With index $i = 4x + 2y + z$, their indices are 3, 5 and 6. Set these positions in the lookup mask: $M = 2^3 + 2^5 + 2^6 = 8 + 32 + 64 = 104 = \texttt{0x68}$. The binary mask is 01101000_2, with row zero at the least significant bit. For any row, $(M \gg i)\,\&\,1$ now returns one precisely for those three indices.

← Return to the exercise

Solution 3.1.8

First half adder: $s_1 = A \oplus B = 0$, $c_1 = AB = 1$. Second: sum $= s_1 \oplus C_{in} = 1$, $c_2 = s_1 C_{in} = 0$. Thus $C_{out} = c_1 \vee c_2 = 1$. The three input bits represent decimal 3, encoded as carry-sum 11_2.

← Return to the exercise

Solution 3.1.9

Apply $c_{i+1} = g_i \vee (p_i \wedge c_i)$ from the least significant position upward. First $c_1 = 0 \vee (1 \wedge 1) = 1$. Then $c_2 = 0 \vee (1 \wedge 1) = 1$. Finally $c_3 = 1 \vee (0 \wedge 1) = 1$. The first two

Worked solution (continued)

positions propagate the incoming carry; the third generates a carry independently of its input. Substitution into the expanded lookahead expression gives the same final carry because its g_2 term alone is already one.

← Return to the exercise

Solution 3.1.10

$SF \oplus OF = 0$, so less-than is false. Less-or-equal is false because both that term and ZF are 0. Greater-than is true because $\neg(SF \oplus OF) \wedge \neg ZF = 1$.

← Return to the exercise

Solution 3.1.11

A takes $5 \cdot 28 = 140$ ps. B takes $3 \cdot 42 + 20 = 146$ ps. Fewer gates does not necessarily mean less delay; A is 6 ps faster under the given model.

← Return to the exercise

Solution 3.1.12

Subtract at the operand width to obtain $r = A - B$ and its sign and overflow flags. Form $s = SF \oplus OF$, which is one exactly when $A < B$ under signed interpretation. Connect $D_0 = B$ and $D_1 = A$ to a word-wide mux controlled by s:

$$\min(A, B) = (\neg s \wedge B) \vee (s \wedge A),$$

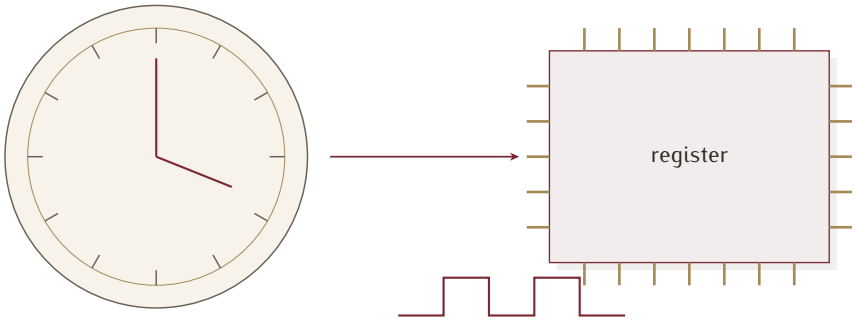
where each one-bit selection term is applied to every bit of its chosen word. Equal operands give $s = 0$ and select B, the common value.
For 8-bit $A = -128$ and $B = 1$, subtraction wraps to 127, so SF=0 and OF=1. Their XOR is 1 and the mux correctly selects $A = -128$. Using the result's sign bit alone would select the wrong operand in this overflow case.

← Return to the exercise

3.2 SEQUENTIAL LOGIC: STATE, REGISTERS, AND MEMORY

A counter must remember its previous value to produce the next one. Gates alone describe how to compute an answer from present inputs; a state element retains the information that the next computation needs. Following a clock edge through a register makes that retention precise and reveals the timing conditions under which the new state can be captured reliably.

The clock edge organises changes of stored state



An active edge separates the old stored state from the next state.

Figure 3.10: *The clock face supplies an intuitive cue for repeated sampling events; the waveform below identifies the digital signal. The register captures only under its specified clock and timing conditions, with physical setup and hold requirements.*

3.2.1 From combinational logic to stored state

Combinational circuits compute outputs but have no memory: same inputs, same outputs, with no trace of what happened an instant earlier. A CPU cannot be built without memory: the program counter must hold the current instruction's address, registers

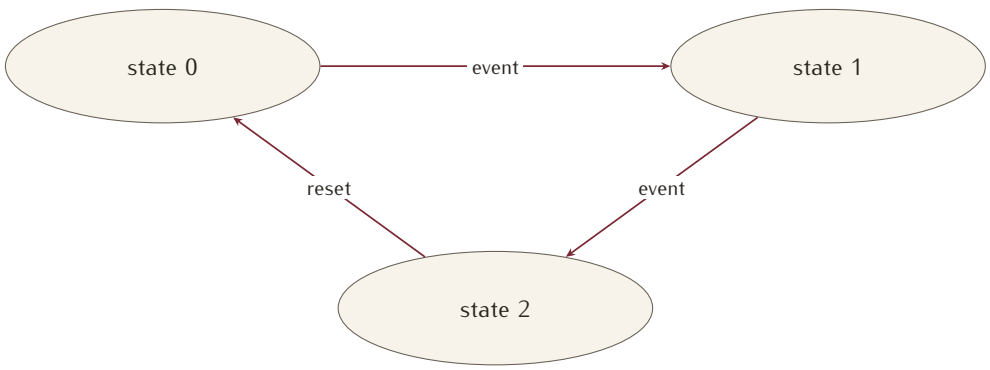
must retain values between instructions, the condition codes must keep the last comparison's result. We need *sequential circuits*, whose output depends on the current inputs and on internal state, that is, on prior history.

Sequential storage is built from the bottom up: the SR latch exposes the basic feedback mechanism and its forbidden input, the D latch removes that input ambiguity while remaining level-sensitive, and the edge-triggered D flip-flop provides the state element used in registers and pipeline registers. Small state elements use flip-flops; larger structures (caches, internal memories) use SRAM cells or dedicated techniques.

The key idea is *feedback*: a gate's output is routed back to a gate's input, forming a loop that can *hold* a value: under the hold condition, feedback preserves the output after the initiating input has been removed. Mathematically, a combinational circuit is $Q = f(I)$; a sequential one is $Q_{new} = f(I, Q_{old})$, the past appearing in the circuit's structure.

Memory and clocking supply the stateful counterpart to the combinational logic of Section 3.1. A clocked register loads its input on the rising edge and otherwise retains its value; the register file and data memory expose larger addressable state. The distinction is fundamental: combinational logic responds continuously to its inputs, while the edge-triggered state elements in this processor model capture values at defined edges. The teaching model uses a register file with two read ports and two write ports, together with a controlled data memory; these elements become the state of the SEQ processor in Sections 3.3–3.4.

Stored state makes the next result depend on history



The current state and the input determine the next state.

Figure 3.11: *A sequential circuit retains state between events. The arrows encode a small illustrative transition system. Combinational next-state logic and storage elements together implement that temporal behaviour.*

Stored state comes from powered positive feedback

A latch remembers one bit.

Two inverting stages connected in a loop have two stable operating points: one node high and the other low, or the reverse. While power is present, feedback drives small disturbances back toward one stable state. Writing the latch temporarily overpowers or redirects this feedback so the circuit settles into the other state.

Static memory retains a powered electrical state without the periodic refresh required by DRAM. The same feedback that preserves a bit also creates the possibility of metastability near the boundary between the stable states [41, 42].

3.2.2 Latches, flip-flops, and the clock

The *SR latch* (built from two cross-coupled NOR gates) has inputs S (set), R (reset) and output Q: S=1, R=0 sets Q=1; S=0, R=1 resets Q=0; S=R=0 holds (the memory mechanism); S=R=1 is *forbidden*.

Inputs and previous state		Next state
S	R	Q_old	Q_new
0	0	0	0
0	0	1	1
0	1	*	0
1	0	*	1
1	1	*	forbidden

Worked example

Problem. `rdi` = S, `rsi` = R, `rdx` = Q_{old} (all three values are 0 or 1, and S=R=1 never occurs). Compute Q_{new} in `rax`.

Solution. Closed form $Q_{new} = S \vee (\bar{R} \wedge Q_{old})$:

```
mov rax, rsi
xor rax, 1; NOT R
and rax, rdx; (NOT R) AND Q_old
or rax, rdi; S OR ((NOT R) AND Q_old)
and rax, 1
```

Caution

S=R=1 is forbidden by structure: both NOR gates drive Q and $\bar{Q}$ to 0 simultaneously, although the latch's two outputs should be complementary. When S, R then return to 0, the final value depends on which input changes first and on gate

Caution (continued)

micro-delays, a *race condition* with possible metastability. A D latch prevents this input combination in its stable Boolean model; its physical timing requirements still apply.

The *D latch* removes the forbidden case with a single data input D and an enable EN: when EN=1 the output follows the data ($Q = D$, "transparent"); when EN=0 it holds. Internally it drives an SR latch with $S = EN \wedge D$ and $R = EN \wedge \bar{D}$. For stable inputs in the ideal Boolean model these signals are mutually exclusive; they are not logical complements when EN=0, because both are then zero. Its equation $Q_{new} = EN \cdot D + \overline{EN} \cdot Q_{old}$ is a 2-to-1 mux with EN as the selector:

```
test rsi, rsi
mov rax, rdx; assume Q stays Q_old
cmovnz rax, rdi; if EN != 0, take D
and rax, 1
```

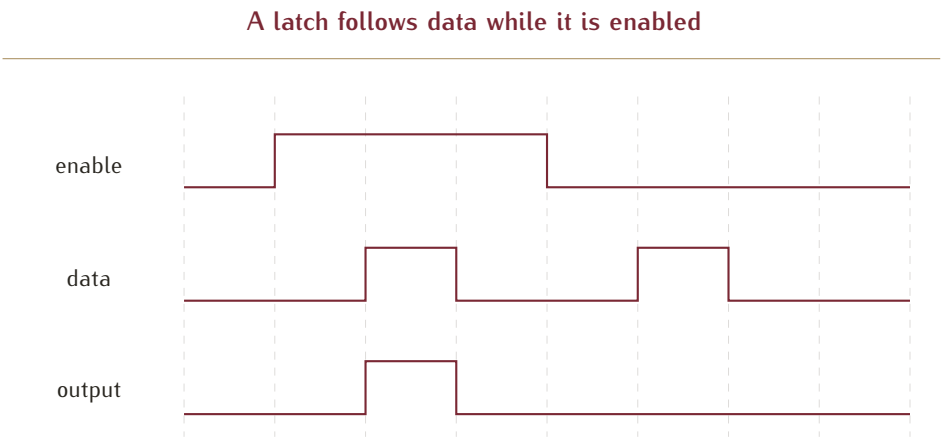
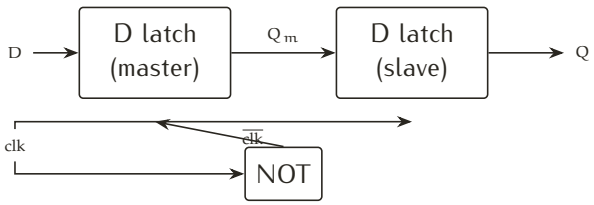


Figure 3.12: *During the enabled interval, the output follows the data in this idealised model. When the latch closes, it retains the last accepted value. Changes outside that transparent interval do not propagate to the stored output.*

While $EN = 1$, changes in D propagate through the transparent latch. Cascading several latches enabled during the same interval can let a value pass through multiple stages in one phase. Edge-triggered flip-flops provide a single capture boundary for the synchronous model. Their setup and hold requirements determine whether that capture is reliable; carefully designed multiphase latch systems provide another valid timing discipline.

An *edge-triggered D flip-flop* samples D only at a clock transition (rising edge for positive-edge devices), holding constant between edges regardless of how D changes. One implementation uses two D latches in series with complementary clocks (*master–slave*): while the master is transparent the slave holds, and they swap on the other half-cycle.

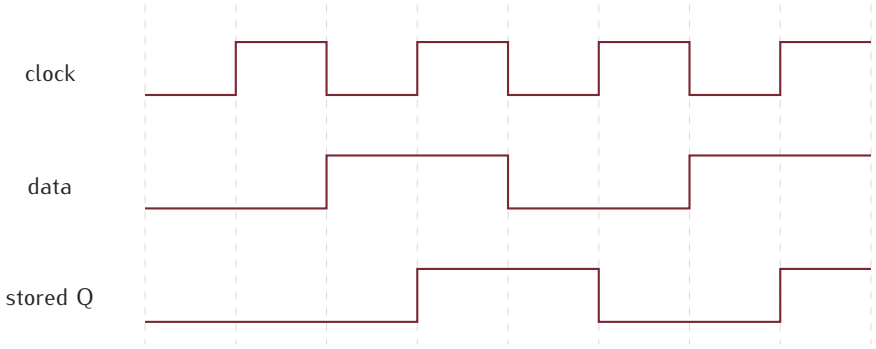


Worked example

Problem. `rdi` = D, `rsi` = previous clock, `rdx` = current clock, `rcx` = old Q; all four values are normalised to 0 or 1. Model a positive-edge flip-flop: sample D only on a rising edge (clk_{old}=0, clk_{new}=1); else hold.
Solution. The rising edge is $\overline{clk_{old}} \wedge clk_{new}$:

```
mov r8, rsi
xor r8, 1
and r8, rdx; (NOT clk_old) AND clk_new = rising edge
test r8, r8
mov rax, rcx; assume Q_old
cmovnz rax, rdi; on the edge, take D
and rax, 1
```

An edge-triggered register samples at selected transitions



Ideal positive-edge sampling; data is assumed stable around each edge.

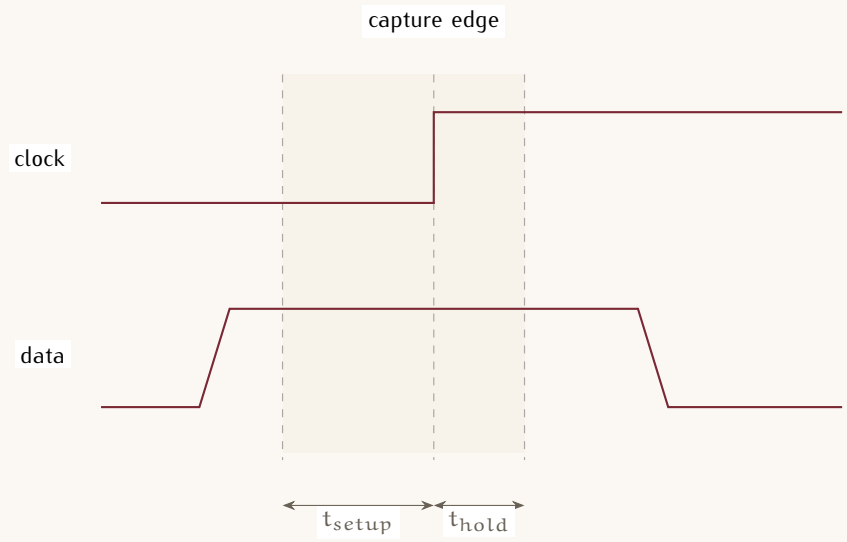
Figure 3.13: *The output changes at positive clock edges according to the sampled data. This abstraction presumes the required setup and hold conditions. The plot illustrates when logical transitions occur.*

A negative-edge device detects $\mathrm{clk}_{old} \wedge \overline{\mathrm{clk}_{new}}$ instead; same structure, different edge detector.

The clock edge has a surrounding timing window

An edge-triggered state element requires its input to be stable for a specified setup interval before the capture edge and a hold interval afterwards. A transition inside that window can leave the internal circuit slow to settle. Metastability concerns this physical settling behaviour; Boolean truth tables describe only the intended stable states [139].

Physical mechanism (continued)



Data is already stable before the capture edge and remains stable afterwards.

The shaded interval shows where the data must remain stable. The drawing is schematic, with a single rising capture edge and no numerical time scale. A synchroniser, often two cascaded flip-flops for a single-bit control crossing clock domains, gives a potentially metastable signal time to settle before later logic uses it, reducing the probability of an observed failure. Common models make the probability of remaining unresolved decay approximately exponentially with additional settling time. A finite settling interval does not provide an absolute guarantee [139].

Worked timing check. For a synchronous register-to-register path, assume clock-to-output delay 0.08 ns, maximum combinational delay 0.62 ns, setup time 0.10 ns and an adverse clock-uncertainty allowance of 0.05 ns. The setup constraint gives

$$T \geqslant 0.08 + 0.62 + 0.10 + 0.05 = 0.85 \text{ ns},$$

corresponding to at most approximately $1/0.85$ GHz $= 1.18$ GHz under these assumptions. Hold timing is a separate minimum-delay constraint. Increasing the period cannot by itself repair a path whose data changes too soon after the capture edge [42].

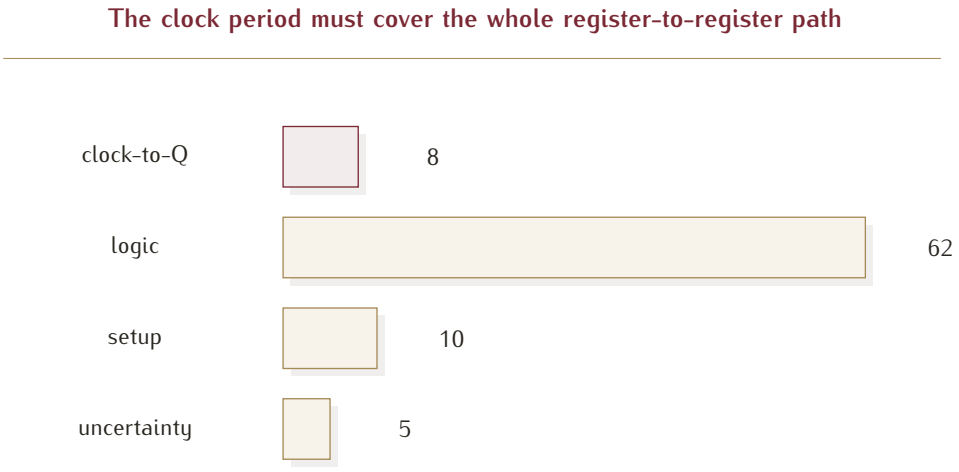


Figure 3.14: *The bar lengths use the numerical model from the physical timing discussion. The setup constraint includes all four contributions, giving 0.85 ns. Hold analysis requires a separate minimum-delay check.*

3.2.3 Registers, register files, and memory

In the edge-triggered model used here, an n-bit *register* can be represented as n flip-flops in parallel, sharing a clock and a LOAD enable: on the edge, LOAD=1 loads all n bits at once, LOAD=0 holds.

Worked example

Problem. `rdi` = new input, `rsi` = LOAD in $\{0, 1\}$, `rdx` = old state. If LOAD=1 load the low 8 bits of `rdi`, else hold `rdx`. Result in `rax`.

Solution.

```
mov rax, rdi
and rax, 0xFF; low 8 bits of the new input
mov r8, rdx
and r8, 0xFF; low 8 bits of the old state
test rsi, rsi; flags for LOAD must reach CMOV unchanged
cmovz rax, r8; if LOAD=0, keep the old value
```

The mask `0xFF` generalises to $2^n - 1$ (n=64 needs no mask).

Both inputs are masked because x86 registers are 64-bit even when the modelled register is narrower: `cmov` picks one, and dirty high bits would leak into the result. In

hardware this cannot happen, because n flip-flops have n outputs; only the software emulation needs the width discipline.

In the Y86-64 teaching processor, the architectural registers are represented by a *register file* with two logical read ports selected by srcA and srcB during Decode and two logical write ports: (dstE, valE) for an ALU result and (dstM, valM) for a loaded value. The source IDs can name explicit instruction operands or implicit registers such as the stack pointer. For example, `popq rA` writes the incremented stack pointer through the E port and the loaded word through the M port. If both ports target `rsp`, the M value takes priority in the Y86-64 model. Physical implementations can realise this interface in different ways. In Y86-64 there are 15 real program registers; the ID `0xF` is the special "no register" code.

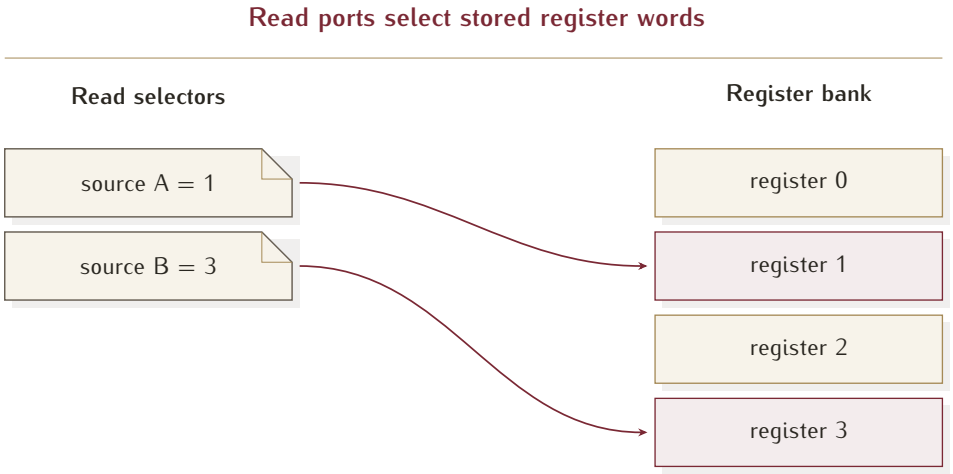
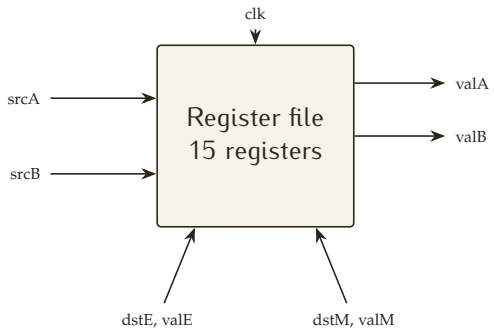


Figure 3.15: *A register file exposes indexed words through its read ports. The selector identifies a register, while the data output supplies that register's contents. The number of ports constrains simultaneous access patterns.*



Worked example

Problem. `rdi` points to the 15 real Y86-64 registers, stored as 8-byte cells; `rsi`, `rdx` are valid read-port IDs in 0..`0xE`. Read into `rax` = valA, `rbx` = valB.
Solution.

```
mov rax, [rdi + rsi*8]; valA = regfile[srcA]
mov rbx, [rdi + rdx*8]; valB = regfile[srcB]
```

For either write port, assume the destination ID is in 0..15. The code `0xF` (15), usually named RNONE, means "no write/no register":

```
cmp rcx, 0xF
je skip
mov [rdi + rcx*8], r8
skip:
```

Why two read ports. `addq rA, rB` reads two operands and writes one. With a single read port, reading rA and rB would require two access intervals or another source of the second value; two read ports fetch both in parallel. The cost is hardware: additional ports require more access circuitry, decoding and wiring. Superscalar and out-of-order implementations need multiple simultaneous operand reads and result writes, but physical designs use techniques such as banking, replication and register renaming; exact port counts and organisation are microarchitecture-specific. Extra bandwidth increases area, power and timing pressure.

More register-file ports multiply wires and energy

A register file can provide several operands and accept several results in one cycle.

Each independent port needs access transistors, word lines, bit lines, decoders and drivers. Adding ports enlarges every cell and increases capacitive loading throughout the array. Beyond modest sizes, designers bank, replicate or distribute storage rather than build one unrestricted structure.

Superscalar width is constrained by the physical cost of supplying operands and collecting results. Execution units can be replicated more easily than a central multiported storage and bypass fabric [50, 42].

An SRAM read is a small analogue measurement

Reading an SRAM cell returns its stored bit.

In a conventional differential SRAM read, bit lines are first precharged. Activating a word line connects the selected cell and creates a small voltage difference between complementary bit lines. A sense amplifier detects and rapidly enlarges that difference into a full logic level. During a write, stronger drivers force the bit lines so that the cell's feedback changes state.

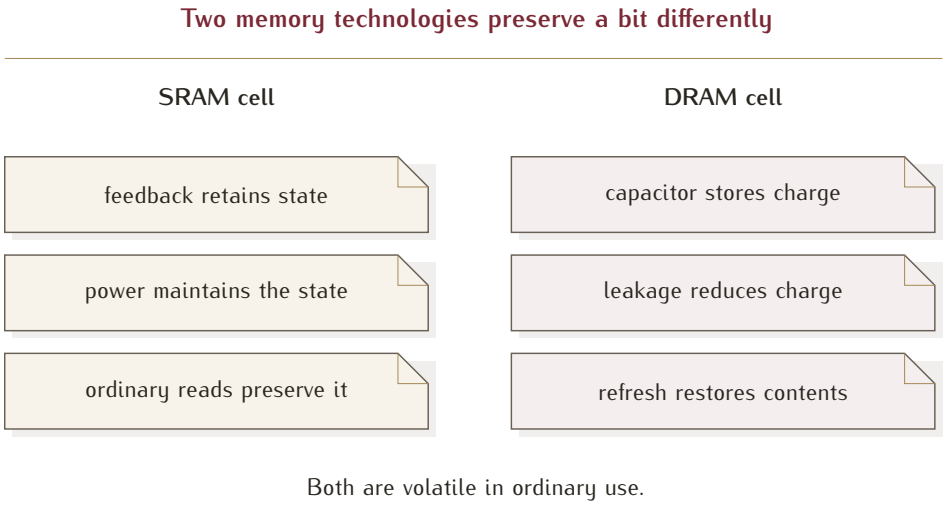


Figure 3.16: *The storage mechanism influences density, timing and maintenance. SRAM uses a stable feedback state; DRAM uses stored charge that must be restored. The comparison describes conventional cells, with device details treated separately.*

Cache access time includes decoding, word-line travel, bit-line discharge and sensing. Larger arrays have longer, more capacitive lines, which is one reason large caches are slower and often divided into banks [41, 47].

Beyond the register file, the CPU accesses a larger, slower *data memory*. The idealised model exposes two controls: READ (return the qword at address A) and WRITE (store DataIn at A).

```
; READ: rdi=A, rsi=READ, rdx=default -> rax
test rsi, rsi
mov rax, rdx
jz done
mov rax, [rdi]
done:
```

In the simplified single-instruction Y86 memory-stage interface, the control logic requests a read, a write, or neither for that instruction. Real cache and memory systems can accept concurrent loads, stores, writebacks and coherence traffic through multiple ports, banks, queues or cycles; the mutually exclusive signals belong to this teaching interface.

3.2.4 The synchronous abstraction and its timing limits

Sequential circuits introduce time. A register holds a value during a clock cycle and updates at a clock edge. This gives the designer a clean abstraction: within a cycle, combinational logic reads the state captured at the preceding boundary and computes

candidate new values; at the next clock boundary, the synchronous model treats the selected state updates as one transition, even though a physical clock network has finite skew and device-level timing constraints.

A register in a feedback path separates the value used for the current computation from the value captured for the next cycle. Combinational feedback without an appropriate storage and timing design may oscillate or have several stable outcomes, as the latch discussion illustrates. The synchronous model instead specifies each state transition at a clock boundary and checks the physical timing needed to realise it.

Worked example

If a register file writes `rax` at the clock edge and an instruction in the next cycle reads `rax`, the new value is observed. In a pipeline, two different instructions may try to read and write related registers in nearby cycles; this is the source of data hazards.

A well-designed sequential system maintains invariants across clock edges. A program counter should contain the address of an instruction or a well-defined exceptional state. In the simple in-order model, the architectural register file holds the values produced by completed instructions. Pipeline registers should hold fields belonging to one dynamic instruction. These invariants make reasoning possible.

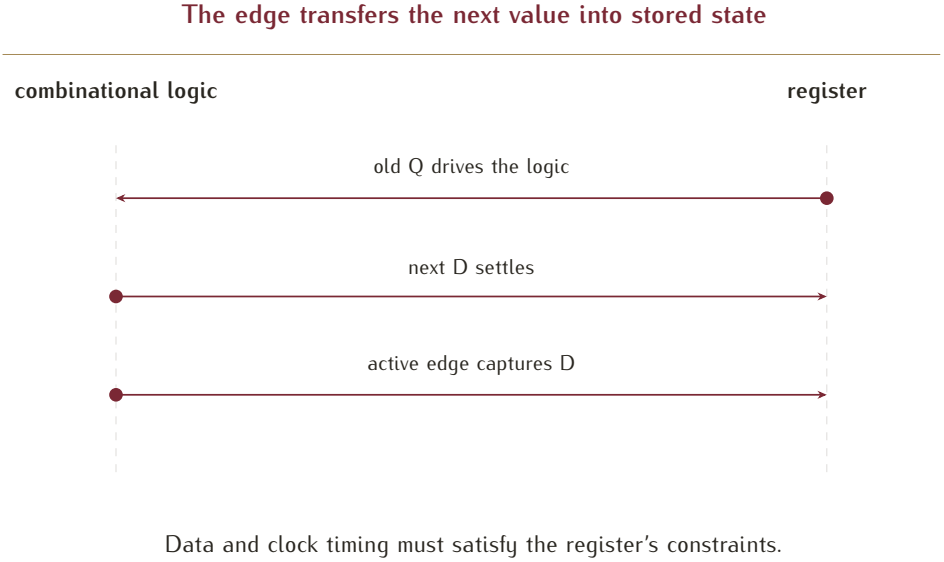


Figure 3.17: *The old state drives a combinational computation whose result becomes the next input. An active clock edge commits that value into the register. The cycle boundary separates these two stored-state snapshots.*

The distinction between architectural state and internal state will matter later. Architectural state is visible to software: registers, memory, PC and condition codes. Internal pipeline registers are implementation details; they help the processor run faster but should not change the meaning of the program.

> **Caution**
>
> A latch is level-sensitive, while a flip-flop is edge-triggered. Mixing these concepts can lead to wrong timing diagrams. The SEQ and PIPE models use edge-triggered processor state.

The clock is a distributed analogue waveform

The ideal synchronous model assigns one common clock event to all participating registers.

A clock tree or mesh drives enormous capacitance across the chip. Buffer stages and carefully matched routes reduce skew, but process variation, temperature, supply noise and local loading still alter arrival times. Jitter moves edges from cycle to cycle, while skew creates different arrival times at different registers.

Timing analysis reserves margin for clock uncertainty in addition to combinational delay and setup time. The clock network can consume a large fraction of dynamic power because its ungated portions switch on every active cycle even when little useful work occurs [42, 41].

For a path from a launch register to a capture register, define $\delta = t_{capture} - t_{launch}$: positive skew means that the capture clock arrives later. Ignoring jitter for the moment, the constraints are

$$T_{clk} \geqslant t_{cq,max} + t_{comb,max} + t_{su} - \delta, \qquad t_{cq,min} + t_{comb,min} \geqslant t_{h} + \delta.$$

A later capture edge helps setup and hurts hold. With uncertain skew, each check uses its own adverse bound plus a nonnegative uncertainty allowance. Increasing the period can repair a setup violation; hold depends on the shortest same-edge path and requires delay or clock-distribution changes.

> **Worked example**
>
> If a register output feeds an adder and then another register, setup requires the clock period to cover source clock-to-Q, the adder's maximum delay, destination setup time and clock uncertainty/skew. A separate minimum-delay analysis checks hold time; a very short path can violate hold even when the clock period is long.

3.2.5 From stored state to controlled machines

A sequential circuit can be described mathematically by two functions:

$$S_{t+1} = F(S_t, X_t), \qquad Y_t = G(S_t, X_t),$$

where S_t is the current state, X_t the external input, and Y_t the output. A combinational circuit is the special case with no state variable. This formulation is useful because it turns a circuit with feedback into a transition system that can be reasoned about one clock edge at a time.

Worked example: a two-bit saturating counter

Let the state be $00, 01, 10, 11$. Input $u = 1$ increments unless the state is already 11; $u = 0$ decrements unless it is already 00. Its complete transition table is

Current state	$u = 0$	$u = 1$
00	00	01
01	00	10
10	01	11
11	10	11

Such counters are used conceptually in branch predictors: the state remembers a short history and resists changing prediction after a single contrary outcome.

Worked example: maximum frequency

A register has $t_{cq} = 22$ ps and $t_{setup} = 18$ ps. The longest combinational path is 310 ps, and the design reserves 20 ps for skew and margin. Then $T_{clk} \geqslant 370$ ps, so $f_{max} \leqslant 1/(370\text{ ps}) \approx 2.70$ GHz. Reporting only the 310 ps logic delay would overstate the attainable frequency.

Synchronising each bit of a changing bus independently can produce a word that never existed at the source. Multi-bit transfer instead requires a stable-data handshake, an asynchronous FIFO or another protocol with a defined coherence guarantee. Gray coding is useful for counters whose successive states change one bit; it does not by itself transfer arbitrary payload words.

The Y86-64 register file reads source values combinationally and accepts its E and M writes at the active edge. The special identifier RNONE suppresses the corresponding register access. The pipeline must also specify what a read observes when an older instruction writes the same register.

Worked example: same-cycle read and write

Suppose Write-back writes 9 to `rax` while Decode reads `rax`. Which value should Decode see? A design must define the convention. A register file may internally forward the write value to the read output, or the pipeline can provide an external bypass. Leaving the case undefined makes dependent instructions unreliable.

A memory stores many words but presents a small interface: address, data input, data output, read enable and write enable. An address decoder selects one row; sense circuitry reads stored bits; write drivers change the selected cells. At the architectural level, the processor sees a function from addresses to bytes. At the circuit level, capacity and access time depend on geometry, banking and technology.

The instruction memory in a simple teaching processor is commonly treated as read-only combinational memory. The data memory may be modelled as reading or writing during the Memory stage. Real caches and DRAM have variable latency, but the fixed-latency abstraction keeps the first processor design understandable.

A finite-state machine combines a state register with next-state logic and output logic. A traffic light, protocol controller or multi-cycle processor controller can all be represented this way. Two naming styles are common: Moore outputs depend only on state, while Mealy outputs also depend directly on current inputs. The distinction affects latency and the possibility of output glitches.

A state table turns behaviour into implementation data

	current	input	next
A	idle	start	busy
B	busy	done	idle
C	busy	wait	busy

Other input combinations need explicitly defined behaviour.

Figure 3.18: *Each row specifies one transition obligation. A complete design must define all relevant state/input combinations, including reset and invalid-state handling. The table can then drive next-state logic and tests.*

Worked example: detecting the sequence 101

Use states for the longest useful suffix seen so far: S_0 (none), S_1 (seen 1), S_2 (seen 10). From S_2, input 1 completes the pattern. If overlapping matches are allowed, the next state after detection is S_1 because the final 1 can begin another match. For input 0, the transitions are $S_0 \to S_0$, $S_1 \to S_2$, $S_2 \to S_0$, with output 0. For input 1, all three states transition to S_1; the output is 1 only from S_2. These six cases define the complete Mealy machine.

A processor must start from a known architectural state. Reset logic establishes a valid PC, status and control state. Some storage bits may be architecturally unspecified, but control registers cannot begin arbitrarily. Reset can be synchronous, taking effect on a clock edge, or asynchronous, forcing state independently of the clock. Asynchronous assertion is useful for safety; deassertion must be synchronised to avoid timing problems.

The stored-program state machine

The stored-program computer can be viewed as a large sequential circuit. Its state includes the program counter, registers and memory; the instruction bits select the transition performed at the next step. This perspective connects early automata theory, digital logic and modern instruction-set semantics: executing a program is repeatedly applying a state-transition function. [22, 52]

Laboratory 18: A pattern detector

Use a Mealy detector with states A (no prefix), B (suffix 1), C (suffix 10). Emit one when input 1 arrives in C. Trace input `11010101` with overlap enabled and disabled, showing the transition rule and first differing output.
Worked solution

Laboratory 18: solution

For input 0, the transitions are A→A, B→C and C→A, all with output 0. For input 1, A→B and B→B produce 0. The remaining case, C with input 1, detects the pattern and produces 1. Its next state is B when overlap is allowed, since the final 1 can start another match; without overlap, it returns to A.
Starting at A, the complete trace is:

Worked solution (continued)

		Overlap		No overlap	
Bit	Input	New state	Output	New state	Output
1	1	B	0	B	0
2	1	B	0	B	0
3	0	C	0	C	0
4	1	B	1	A	1
5	0	C	0	A	0
6	1	B	1	B	0
7	0	C	0	C	0
8	1	B	1	A	1

The output sequences are `00010101` and `00010001`. Both detect at bit 4, but their next states differ at that point. The first different output occurs at bit 6, when the overlapping detector recognises the pattern beginning at bit 4.

Return to the laboratory

From delay lines and cathode-ray tubes to semiconductor memory hierarchies

Timeline

Year	Machine, standard or event	Architectural significance
1948	**Williams tube**	Electronic random-access state enables early stored programs.
1953	**Magnetic-core memory**	Reliable nonvolatile cores become the dominant main memory.
1970	**Commercial DRAM**	Dense semiconductor memory establishes the modern memory path.

The earliest electronic computers could calculate faster than they could remember. Designers needed a device that preserved many binary states, allowed them to be read and rewritten, and operated at electronic rather than mechanical speed. No single technology satisfied all requirements. The resulting history explains why a textbook separates latches and registers from main memory even though both store bits [72, 51, 50].

Mercury acoustic delay lines came from wartime radar work. A pulse was converted into a sound wave, travelled through a column of mercury and was detected, amplified and re-injected. Data existed as a circulating sequence, so a word could be accessed only when its pulses reached the output. EDSAC, which ran its first program at Cambridge on 6 May 1949, used mercury delay-line storage. The memory was sequential internally even though instructions referred to numbered locations. Timing and recirculation were part of normal operation [91, 95].

Freddie Williams and Tom Kilburn developed a different approach in 1946-1947. A cathode-ray tube stored charge patterns on its screen; a pickup plate sensed the charge, and refresh circuitry rewrote fading information. The Manchester Small-Scale Experimental Machine, or "Baby," ran a stored program on 21 June 1948 and was built largely to prove that the Williams-Kilburn tube could serve as random-access electronic memory. The technology was fast but sensitive to adjustment and environmental conditions. It was used in several early machines before being displaced by more robust alternatives [89, 96].

Magnetic core memory became the durable solution of the 1950s and 1960s. Small ferrite rings could be magnetised in either of two directions. Wires threaded through arrays selected a core by coincident currents; reading typically destroyed the stored state and required a rewrite. An Wang patented pulse-transfer and core ideas, while Jay Forrester and the MIT Whirlwind team developed practical coincident-current arrays. Whirlwind used core memory in 1953, and the technology spread rapidly because it was non-volatile, relatively reliable and genuinely random access. It remained dominant into the 1970s [97, 72].

Registers followed a different physical route. Vacuum-tube and transistor bistable circuits stored one bit in cross-coupled feedback. The SR latch demonstrates the principle: two gates sustain a state after the initiating signal disappears. The D latch and edge-triggered flip-flop add control so a new value is accepted at a defined level or clock transition. These circuits are expensive in devices per bit but fast and directly connected to the datapath. Core memory is dense and addressable as an array; flip-flop registers are small, parallel and placed where immediate state is needed. The distinction between small, directly accessed registers and larger addressable memory survives in semiconductor processors and memories [51, 50, 72].

Semiconductor main memory became practical through two inventions. Static RAM cells use feedback, usually six transistors in a modern CMOS implementation, and retain data while powered without refresh. Dynamic RAM stores charge on a capacitor and must be periodically refreshed. Robert Dennard of IBM conceived the one-transistor DRAM cell in 1966 and received a patent in 1968. Intel's 1103, introduced in 1970, used a three-transistor dynamic cell and became the first commercially successful DRAM to compete strongly with magnetic core. It belongs to the early commercial development of DRAM alongside the one-transistor design pioneered by Dennard. IBM System/370 Model 145, announced in 1970, was among the first major systems to use semiconductor main memory [116, 73, 113].

DRAM changed capacity but introduced timing protocols. A chip is organised into rows and columns; activating a row moves charge information into sense amplifiers, after which columns are accessed. Precharge prepares the bank for another row. Modern Double Data Rate SDRAM transfers on both clock edges and uses banks, bank groups, burst transfers and command scheduling. The familiar statement that memory is an array indexed by an address is the architectural view. At the device level,

row-buffer locality, refresh, timing constraints and channel scheduling shape latency and bandwidth [51, 50, 72].

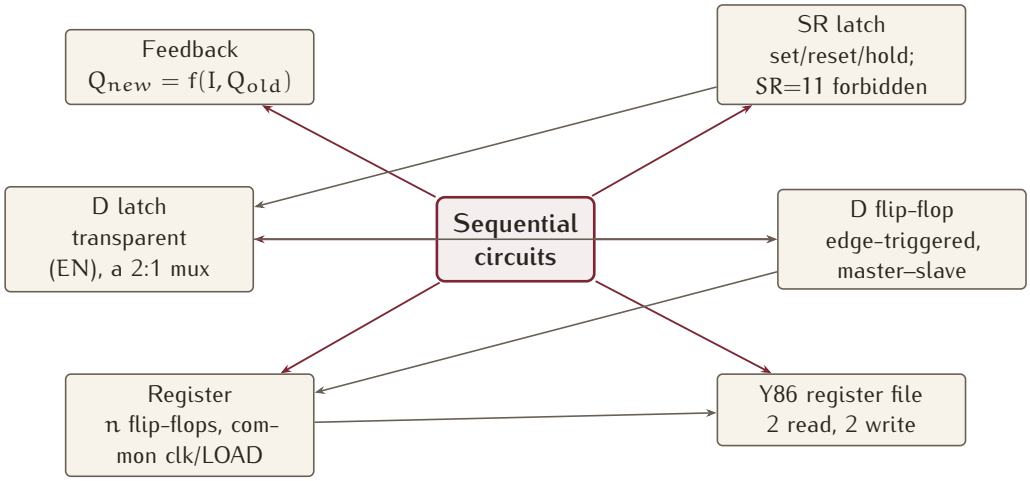
The processor-memory speed gap led to caches. Small SRAM arrays near execution units hold recently used blocks from slower DRAM. Early machines had specialised buffers; commercial mainframes and supercomputers developed instruction lookahead and cache-like structures during the 1960s. By the 1980s and 1990s, on-chip caches became standard in microprocessors. Current cores usually have private L1 instruction and data caches, larger private or shared mid-level caches and coherence mechanisms connecting cores. The ordinary unit of transfer and coherence in this model is a cache line, which can contain several scalar objects [51, 50, 72].

High Bandwidth Memory and chiplet packaging are the current extension. HBM stacks DRAM dies and connects them through-silicon vias to a wide interface near a processor or accelerator. It supplies far more parallel bandwidth than a narrow DIMM channel, but capacity, cost, thermals and placement differ. CXL memory devices add another tier with explicit fabric and coherence semantics. Persistent-memory technologies have also appeared and changed, showing that the hierarchy is not fixed [51, 50, 72].

The clocked abstraction remains essential. A flip-flop model assumes setup and hold constraints, clock-to-output delay and bounded combinational delay between registers. Real chips must also control clock skew, metastability, voltage variation and power. Memory arrays add sense margins, retention and refresh. The path from delay lines to HBM shows how several storage mechanisms coexist in a hierarchy whose levels trade area, latency, bandwidth, persistence and cost. Primary study should combine the documented Manchester, Cambridge, MIT and IBM histories with current processor and memory-interface manuals [89, 91, 96, 97, 116].

Clocks, registers and finite-state control remain the language of RTL even when physical implementation is deeply optimised.

Concept map: sequential logic



Reading the map. Feedback gives a circuit memory. An SR latch exposes set, reset and hold through feedback. A D latch combines feedback with controlled transparency; an edge-triggered D flip-flop captures data at a clock transition. n flip-flops can make a register. Addressed storage arrays and ports provide the register-file and memory interfaces, with cell technologies selected for their different capacity and access requirements. These blocks hold the state read and written by the processor of Sections 3.3–3.4.

3.2.6 Worked problems

D latch

A level-sensitive D latch starts with $Q = 0$. Four successive settled intervals have $(EN, D) = (0,1)$, $(1,1)$, $(0,0)$ and $(1,0)$.
Question. What is Q after each interval?
With enable low, the latch retains its previous output; with enable high, it follows the current data. Applying that rule gives:

Interval	EN	D	New Q	Reason
1	0	1	0	Hold the initial state
2	1	1	1	Follow D
3	0	0	1	Hold the previous 1
4	1	0	0	Follow D

The outputs are $0, 1, 1, 0$. During each enabled interval, further changes in D would also propagate after the latch's delay. Changing the final pair to $(1,1)$ changes only the last result to 1. The closed intervals still preserve the value captured before them, assuming the latch's timing requirements are met.

Priority at the active edge

A register has synchronous clear with priority over load. Its update rule is $Q_{next} = 0$ if clear is 1, otherwise $Q_{next} = D$ if load is 1, otherwise $Q_{next} = Q$. For $Q = 5, D = 9$, controls $(clear, load) = (1,1)$ capture 0, $(0,1)$ capture 9, and $(0,0)$ preserve 5. The simultaneous case distinguishes clear priority from load priority; all three decisions take effect at the designated clock event.

Masked register

A byte register updates selected positions using

$$Q_{next} = (Q_{old} \wedge \neg mask) \vee (D \wedge mask).$$

Worked example (continued)

Take $Q_{old} = \texttt{0xA5}$, $D = \texttt{0x3C}$ and $mask = \texttt{0x0F}$.
Question. What value is captured, and which input supplies each nibble?
Complementing the mask at byte width gives `0xF0`. The retained contribution is $\texttt{0xA5} \wedge \texttt{0xF0} = \texttt{0xA0}$; the new contribution is $\texttt{0x3C} \wedge \texttt{0x0F} = \texttt{0x0C}$. Their OR is `0xAC`. The high nibble A comes from the old state because the mask has zeros there. The low nibble C comes from D because the mask has ones there. Each bit therefore has exactly one selected source. Reversing the mask to `0xF0` takes the high nibble from D and the low nibble from the old state, giving `0x35`.

Exercise 3.2.1 — Active low

When is reset_n asserted?

Worked solution

The suffix `_n` denotes an active-low signal. Reset is asserted when `reset_n=0` and deasserted when it is 1. If the reset value is zero, assertion forces or schedules $Q = 0$ according to the circuit's synchronous/asynchronous reset specification. The active level tells us which value requests reset; the timing specification tells us when the state responds.

Exercise 3.2.2 — Hold

What happens when a load-enabled register has load=0?

Worked solution

When `load=0`, a load-enabled register keeps its current value at the clock event: $Q_{next} = Q$. For example, if $Q = 5$ and the data input changes to 9 while load remains zero, the next state is still 5. With load one, the next state becomes the data input, here 9. This statement assumes that no higher-priority reset or other control is asserted.

Exercise 3.2.3 — Metastability

What timing violation can lead to metastability?

Worked solution

Metastability can occur when the input changes inside the sampling element's setup/hold window around the active clock edge. Setup time requires stability before the edge; hold time requires stability afterward. Violating either can delay

Worked solution (continued)

the internal decision between logic levels. A synchronizer gives the first stage time to settle before another stage uses its output. It reduces the probability of failure without providing a finite absolute upper bound on resolution time.

Exercise 3.2.4 — Parallel load

What does parallel load mean?

Worked solution

Parallel load means that the register's bits sample their corresponding data inputs at the same designated clock event. A four-bit register receiving `1010` with load asserted captures the complete word in that event. This contrasts with a serial shift, where successive bits enter over successive events. Physical propagation delays still exist after the clock; simultaneous logical capture does not mean zero delay or identical analog switching times.

Summary

The decisive move is from function to recurrence. Combinational logic answers "what follows from these inputs?" Sequential logic answers "what follows from these inputs and this remembered state?" Once the state vector and update rule are explicit, diagrams that once looked animated become ordinary equations evaluated cycle by cycle.

3.2.7 Exercises

Exercise 3.2.5 — SR latch states

For an active-high NOR SR latch, state the action of inputs $(S, R) = (0, 0), (1, 0), (0, 1), (1, 1)$.

Go to the solution →

Exercise 3.2.6 — Next-state equation

A one-bit state toggles when input $T = 1$ and holds when $T = 0$. Give Q_{next} as a Boolean expression.

Go to the solution →

Exercise 3.2.7 — Clock period

$t_{cq} = 30$ ps, maximum combinational delay 420 ps, setup 25 ps, an adverse setup-skew allowance of 15 ps and margin 10 ps. Compute the minimum period and maximum frequency.

Go to the solution →

Exercise 3.2.8 — Hold check

$t_{cq}^{min} = 18$ ps, $t_{comb}^{min} = 12$ ps, hold time 22 ps and capture-clock skew $\delta = +11$ ps (capture arrives later). Is the path safe?

Go to the solution →

Exercise 3.2.9 — Sequence detector trace

Trace the overlapping 101 detector with states S_0, S_1, S_2 for input 10101. List the state after each bit and detection cycles.

Go to the solution →

Exercise 3.2.10 — Register-file conflict

A register file reads `rax` while the same edge writes a new value to `rax`. Give two valid implementation conventions and explain why the pipeline must know which is used.

Go to the solution →

Exercise 3.2.11 — Synchronisation

Why is placing each bit of an asynchronous 8-bit bus through an independent two-flop synchroniser unsafe?

Go to the solution →

Exercise 3.2.12 — FSM design

Design states for a vending controller accepting 1- and 2-unit coins and dispensing at a total of 3. Reject a coin that would exceed 3, preserving the existing credit; an interval with no coin preserves the state.

Go to the solution →

3.2.8 Solutions

Sequential Logic: State, Registers, and Memory

Solution 3.2.5

$(0,0)$ holds the previous state; $(1,0)$ sets $Q = 1$; $(0,1)$ resets $Q = 0$; $(1,1)$ forces both outputs low and is forbidden because the complementary-output invariant is lost and release can be unpredictable.

← Return to the exercise

Solution 3.2.6

There are four combinations of state and input: $(Q, T) = (0,0)$ gives 0, $(1,0)$ gives 1, $(0,1)$ gives 1, and $(1,1)$ gives 0. This is the XOR truth table, hence $Q_{next} = Q \oplus T$. A flip-flop captures that next-state value at the active clock edge. For example, from $Q = 0$ with successive inputs $1, 1, 0$, the stored states after the edges are $1, 0, 0$.

← Return to the exercise

Solution 3.2.7

The receiving register must see stable data before its setup deadline. Under the stated conservative budget, the clock must cover launch delay, the maximum data-path delay, setup, skew and margin: $T_{clk} \geqslant 30 + 420 + 25 + 15 + 10 = 500$ ps. Since 500 ps is $5 \cdot 10^{-10}$ s, $f_{max} = 1/T_{clk} = 2 \cdot 10^{9}$ Hz, or 2.0 GHz. This setup calculation uses the specified adverse skew allowance; a separate minimum-delay check is required for hold timing.

← Return to the exercise

Solution 3.2.8

The earliest new data can arrive is $t_{cq}^{min} + t_{comb}^{min} = 18 + 12 = 30$ ps after launch. The capture clock arrives 11 ps later and requires the old input to remain stable for another 22 ps, giving a deadline of $11 + 22 = 33$ ps.
The path therefore changes data 3 ps too early: its hold slack is $30 - 33 = -3$ ps. Add at least 3 ps of guaranteed minimum data-path delay under this model, or reduce the adverse capture skew. Practical timing closure also includes margins. Increasing the clock period alone leaves this same-edge timing relationship unchanged.

← Return to the exercise

Solution 3.2.9

Starting S_0: input $1 \to S_1$; $0 \to S_2$; 1 detects and returns to S_1; $0 \to S_2$; 1 detects and returns to S_1. Detections occur on bits 3 and 5.

← Return to the exercise

Solution 3.2.10

The read can return the old stored value, requiring an external bypass, or the register file can be write-first and return the incoming new value. Either is valid, but dependent-instruction correctness depends on matching the forwarding/control design to the convention.

← Return to the exercise

Solution 3.2.11

Each bit's synchroniser can resolve a transition on a different destination-clock cycle. Suppose the source bus changes from `01111111` to `10000000`. If the high bit arrives first while the lower bits retain their old values, the receiver can observe `11111111`, a word that was never transmitted as a stable source value.
For arbitrary words, hold the data stable while a request/acknowledge handshake transfers ownership, or use an asynchronous FIFO designed for the two clock domains. Gray-coded counters suit a narrower purpose: successive counter states change one bit, and their crossing still needs the specified timing and synchronisation constraints. Independent bit synchronisers alone provide no coherent-word guarantee.

← Return to the exercise

Solution 3.2.12

Use S_0, S_1, S_2 to represent credit 0, 1 and 2. Each accepted input event is one coin of value 1 or 2; input 0 represents no coin. The complete reachable-state table is:

State	Coin	Next state	Dispense	Coin action
S_0	0	S_0	0	None
S_0	1	S_1	0	Accept
S_0	2	S_2	0	Accept
S_1	0	S_1	0	None
S_1	1	S_2	0	Accept
S_1	2	S_0	1	Accept
S_2	0	S_2	0	None
S_2	1	S_0	1	Accept
S_2	2	S_2	0	Return coin

Worked solution (continued)

The two dispensing transitions reach exactly three units and clear the credit for the next purchase. A two-unit coin offered at credit 2 is returned, leaving credit 2 intact. For example, inputs $2, 2, 1$ give credit 2, reject the second coin while preserving credit 2, then dispense and return to credit 0.

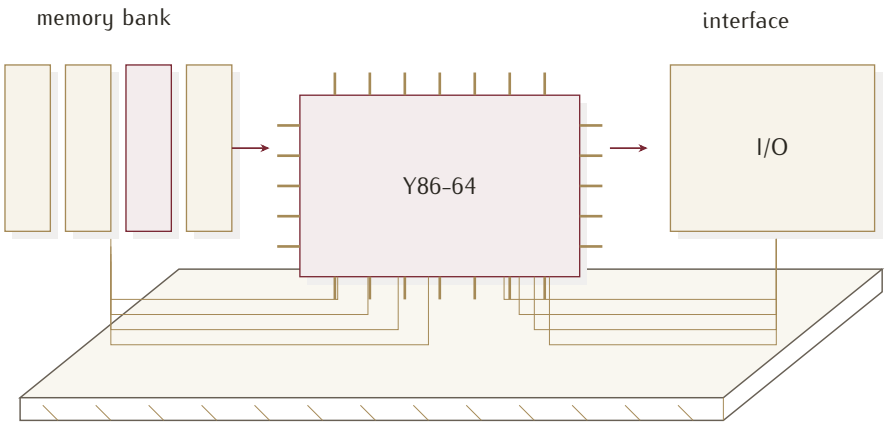
A possible binary encoding is $S_0 = 00, S_1 = 01, S_2 = 10$. The unused code 11 requires a specified fault policy in an implementation, such as suppressing dispense, raising a fault indication and resetting to S_0. Reset establishes S_0 before normal transactions begin.

← Return to the exercise

3.3 THE Y86-64 INSTRUCTION SET AND EXECUTION STAGES

A processor fetches a short instruction encoding, identifies its operands and eventually changes architectural state. Y86-64 keeps this journey small enough to follow completely. For each instruction, we can identify the bytes consumed, the values read and the state written. That account defines the ISA's required behaviour for both a sequential and a pipelined implementation.

An ISA specifies behaviour before choosing a datapath



The drawing introduces a teaching ISA, not a commercial chip layout.

Figure 3.19: *Y86-64 defines instruction encodings and state changes small enough to trace completely. The conceptual processor and interfaces provide a physical setting, while the ISA remains a behavioural contract independent of the later implementation.*

3.3.1 From logic to an instruction-set contract

The preceding two sections built the parts: gates, multiplexers, adders and an ALU as combinational elements; latches, flip-flops, registers and memory as sequential elements. What remains is to assemble a complete CPU. That needs a *contract* specifying what the CPU must do and how it receives instructions: the *instruction set architecture* (ISA).

The case study uses Y86-64, a compact educational ISA. It preserves the ideas needed for processor construction (program registers, memory operations, conditional control flow and status reporting) while deliberately reducing the instruction set and regularising the encoding. The small specification makes it possible to derive the implementation of every instruction class and inspect each state transition. The section defines the byte layouts first, derives the execution equations next, and then traces complete instructions.

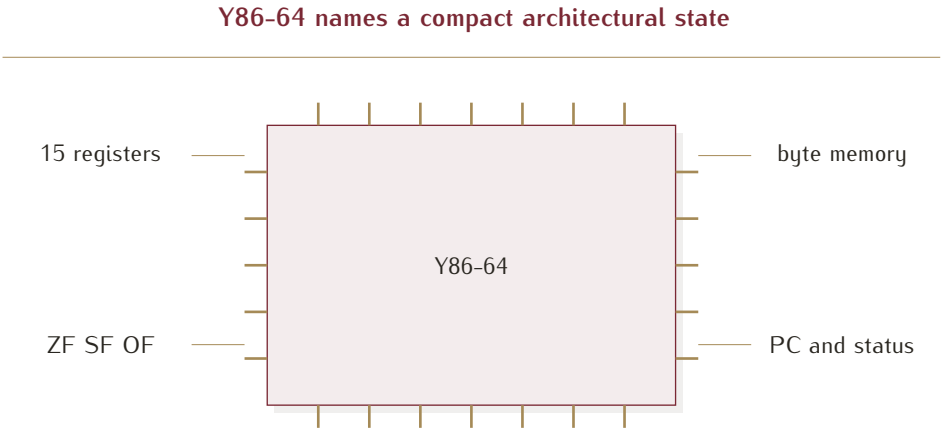
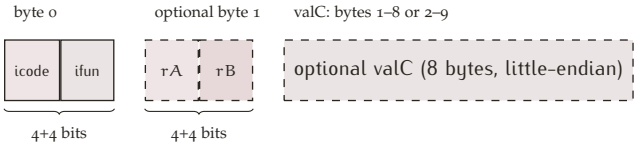


Figure 3.20: *The fifteen program registers exclude a register with identifier F, which denotes RNONE in the encoding. Condition codes, memory, PC and status complete the visible state needed by the teaching processor.*

The architectural state consists of fifteen program registers, three condition codes, the program counter, byte-addressed memory and a status value. The first instruction byte contains an instruction class and function field; selected classes add a register byte and an eight-byte constant. For implementation, each instruction is described by the values produced during Fetch, Decode, Execute, Memory, Write back and PC update. Signal names such as `valP`, `valC`, `valE`, `valM` and `Cnd` identify intermediate values in this implementation model.

3.3.2 Encoding and instruction boundaries

Each Y86-64 instruction is 1 to 10 bytes. The *first byte* packs two 4-bit fields: the high nibble is the *icode* (instruction code: 0 halt, 1 nop, 2 rrmovq, 3 irmovq, 4 rmmovq, 5 mrmovq, 6 OPq, 7 jXX, 8 call, 9 ret, A pushq, B popq); the low nibble is the *ifun* (sub-function). For instruction classes that require a register-specifier byte, the *second byte* packs rA (high) and rB (low), with `0xF` meaning "no register". An 8-byte *immediate* (valC), if present, follows in little-endian.



Instruction	Length	Composition
halt, nop, ret	1	byte 0 only
rrmovq/cmovXX, OPq, pushq, popq	2	byte 0 + register byte
jXX, call	9	byte 0 + 8-byte immediate
irmovq, rmmovq, mrmovq	10	byte 0 + register byte + 8-byte immediate

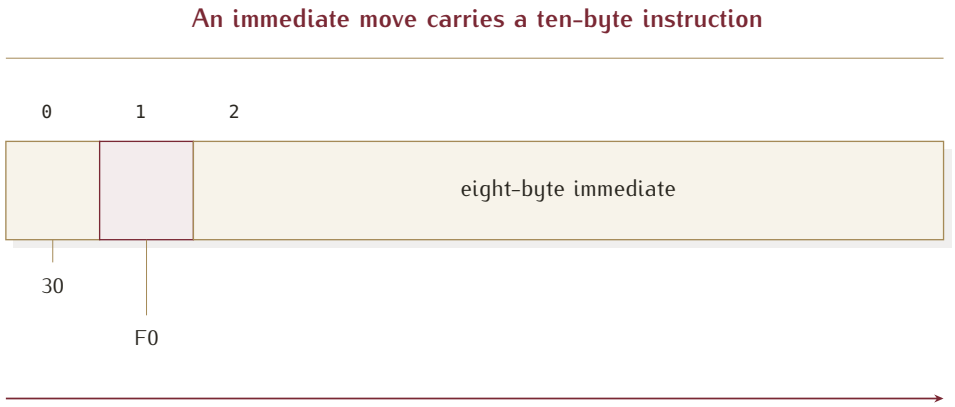


Figure 3.21: *The hexadecimal first byte identifies irmovq. The register byte uses F for no source register and 0 for RAX. The eight-byte immediate is stored little-endian, and the next sequential PC is ten bytes later.*

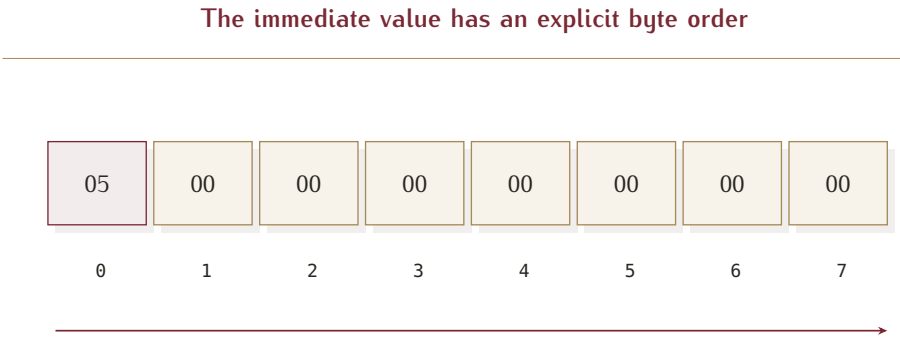


Figure 3.22: *The first immediate byte contributes the least significant eight bits. The other seven bytes are zero in this example. Measure these offsets from the first byte of valC.*

Extracting the two nibbles is a bit operation: shift right 4 for the high nibble, AND `0xF` for the low.

Worked example

Problem. Byte 0 in `rdi`. Put icode in `rax`, ifun in `rbx`.
Solution.

```
mov rax, rdi
shr rax, 4; high nibble = icode
and rax, 0xF
mov rbx, rdi
and rbx, 0xF; low nibble = ifun
```

The register byte (rA high, rB low) decodes with identical code, a deliberate Y86-64 choice: the regular "high nibble / low nibble" layout keeps the hardware decoder simple.

Y86-64 vs x86-64. Y86-64 keeps 15 program registers (`rax`–`r14`, omitting `r15`; register ID `0xF` means no register) with only 64-bit width, ~12 instruction families (x86-64 has hundreds), and *fixed lengths per icode* decodable from the first byte. x86-64's encoding is far more complex (optional prefixes, ModR/M, SIB, displacement, variable immediates, up to 15 bytes). Y86-64 also has no MMU, protected modes, SIMD, or floating point: it is built to teach architecture, with the `yas` assembler and `yis` simulator to run programs.

The register numbers used by these encodings are:

ID	Register	ID	Register	ID	Register
0	rax	5	rbp	A	r10
1	rcx	6	rsi	B	r11
2	rdx	7	rdi	C	r12
3	rbx	8	r8	D	r13
4	rsp	9	r9	E	r14

Register identifier F is RNONE. Hexadecimal letters in this table denote identifiers, not register contents.

Instruction decoding is pattern recognition in wires and gates

The decoder maps an instruction bit pattern to control signals.

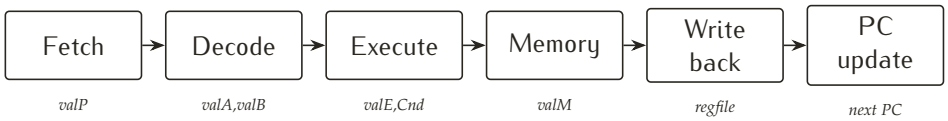
Comparators, programmable-logic-like structures, tables and specialised predecoders recognise opcode fields and generate internal operations. Regular fixed fields allow local parallel decoding; variable-length encodings require boundary detection and additional routing. Frequently used results may be cached in decoded form.

Instruction-set encoding affects front-end area, energy and achievable fetch width. Y86-64's regular format makes its decoding rules explicit and illustrates how regular encodings simplify physical decoding [50, 28].

3.3.3 Instruction semantics as staged dataflow

The semantics of an instruction can be organised into six stages. In SEQ these describe dependent work within one clock cycle:

1. *Fetch*: read the instruction bytes from instruction memory at the PC; decode icode/ifun, read the register byte (rA, rB) and immediate valC if present; compute valP = PC + length (the next sequential address).
2. *Decode*: select source identifiers and read valA, valB. Stack operations select `rsp` even when that register is implicit in the encoding.
3. *Execute*: the ALU computes valE. OPq instructions also update the condition codes; conditional moves and jumps read those codes to compute Cnd. For OPq, valE = valB OP valA; for memory access, valE = valB + valC (effective address).
4. *Memory*: read/write data memory at the selected address: valE for loads, stores, push and call; the old stack pointer in valA for pop and return. Only memory instructions (rmmovq, mrmovq, pushq, popq, call, ret) use this stage.
5. *Write back*: write results to the register file (typically valE → rB for arithmetic, valM → rA for loads).
6. *PC update*: set the PC. Normally PC ← valP; for a taken jump PC ← valC; for call PC ← valC; for ret PC ← valM.



The six stages describe the combinational work of the datapath; the CPU's sequential state is the PC, the register file, the data memory, and the condition-code register (ZF, SF, OF).

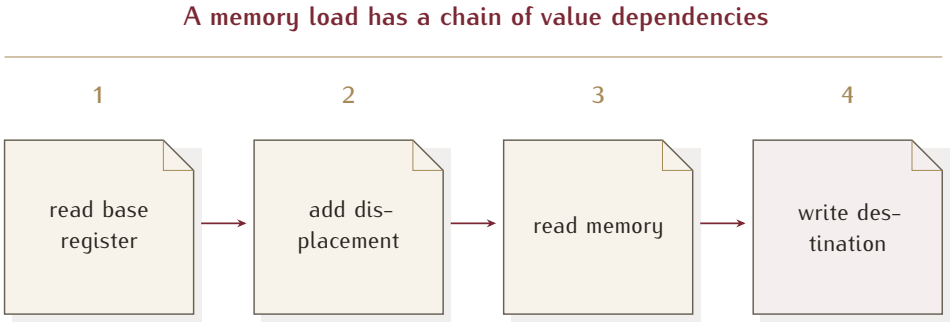


Figure 3.23: *For mrmovq, the register value and displacement form the effective address. Memory supplies the loaded value, which is then written to the destination register. The drawing follows dependencies rather than assigning clock cycles.*

$valP = \text{PC} + \text{length}$, where length depends only on icode.

Worked example

Problem. `rdi` = PC, `rsi` = a valid icode in 0..`0xB`. Compute $valP$ in `rax`: lengths 1 (halt/nop/ret), 2 (rrmovq/OPq/pushq/popq), 9 (jXX/call), 10 (irmovq/rmmovq/mrmovq).

Solution.

```
mov rax, 1; default 1 (halt, nop, ret)
mov r8, 2
cmp rsi, 2
cmove rax, r8; rrmovq
cmp rsi, 6
cmove rax, r8; OPq
cmp rsi, 0xA
cmove rax, r8; pushq
cmp rsi, 0xB
cmove rax, r8; popq
mov r8, 9
cmp rsi, 7
```

Worked example (continued)

```
cmove rax, r8; jXX
cmp rsi, 8
cmove rax, r8; call
mov r8, 10
cmp rsi, 3
cmove rax, r8; irmovq
cmp rsi, 4
cmove rax, r8; rmmovq
cmp rsi, 5
cmove rax, r8; mrmovq
add rax, rdi; valP = PC + length
```

In hardware, a decoder selects the length from icode and an adder forms the sequential successor. Their propagation delays contribute to the fetch path. A pipeline also needs a rule for choosing a fetch address while an earlier branch or return is still being resolved.

For `OPq rA, rB`, Execute computes $valE = valB$ OP $valA$ modulo 2^{64}. It sets ZF exactly when the result is zero and SF to result bit 63. For addition, OF is 1 when equal-sign operands produce the opposite result sign; for subtraction $valB - valA$, it is 1 when different-sign operands produce a result whose sign differs from $valB$. AND and XOR set $OF = 0$. Other base Y86-64 instructions preserve all three condition codes.

For OPq, `ifun=0,1,2,3` selects `addq`, `subq`, `andq` and `xorq`, respectively [1]. The instruction decoder and ALU control share this table. The following x86-64 sequence computes the selected result and its zero indicator; the other Y86 condition codes would require their own computations.

Worked example

Problem. `rdi` $= valA$, `rsi` $= valB$, low 2 bits of `rdx` = ifun (0 add, 1 sub, 2 and, 3 xor). Leave $valE$ in `rax`, ZF in `rbx`.
Solution.

```
mov r10, rdx
and r10, 3
mov rax, rsi
add rax, rdi; default valB + valA
mov r8, rsi
sub r8, rdi
mov r9, rsi
and r9, rdi
mov r11, rsi
xor r11, rdi
```

Worked example (continued)

```
cmp r10, 1
cmove rax, r8
cmp r10, 2
cmove rax, r9
cmp r10, 3
cmove rax, r11
xor rbx, rbx
test rax, rax
setz bl; ZF = (valE == 0)
```

`setz` (and `setne`, `setg`…) sets one byte to 0/1 from the flags, the idiomatic flag-to-value conversion. It leaves the high bits untouched. Here `xor rbx, rbx` clears them before `test` establishes the flags being converted; placing XOR between TEST and SETZ would destroy those flags.

`je dst` jumps to `dst` if ZF= 1, else falls through. In PC update this is PC $\leftarrow$ valC if Cnd=1, else valP, a 2-to-1 mux:

```
; rdi=Cnd, rsi=valC, rdx=valP -> rax
test rdi, rdi
mov rax, rdx; default: fall-through valP
cmovnz rax, rsi; if Cnd != 0, take valC
```

Y86-64 records ZF, SF and OF. Conditional transfers combine them in the usual signed relations: `je` tests ZF, `jl` tests SF $\neq$ OF, `jle` tests ZF $\vee$ (SF $\neq$ OF), and `jg` tests $\overline{\text{ZF}} \wedge$ (SF=OF). x86-64 additionally records CF for unsigned comparisons; the teaching ISA omits it to keep the datapath small. At the end of SEQ, a multiplexer selects the next PC from valP, valC or valM, according to the instruction class and branch outcome.

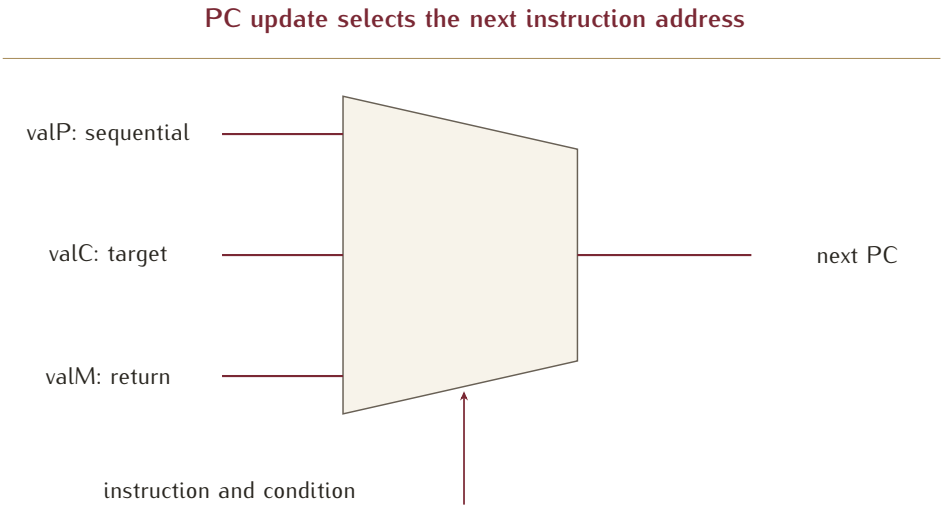


Figure 3.24: *Most instructions continue at valP. Taken jumps and calls use the encoded target, while ret uses the value read from the stack. The selection implements the ISA's control-flow semantics.*

Worked example

The instruction `irmovq V, rB` uses `rA=RNONE` and `rB` as the destination register. The special code `0xF` does not identify a general register; it marks the unused register field. This allows the same register-byte format to be reused across instruction classes.

Fetched instruction bytes arrive in blocks

The architectural decode consumes the bytes of one instruction.

Instruction caches deliver aligned blocks or words, not semantic instruction objects. Selection logic chooses the needed bytes. In a variable-length instruction set, additional boundary logic may join data across fetch blocks and predict where later instructions begin; fixed-length encodings avoid much of that boundary problem. The selected bytes then fan out into parallel decoding logic.

Variable-length instructions that cross fetch boundaries can require extra buffering or latency. The architectural stream is sequential, while the physical front end transfers fixed chunks and may speculate about instruction boundaries [28, 50].

The names `valA`, `valB`, `valC`, `valE`, `valM` and `valP` are a disciplined way to track data through the datapath. They prevent vague descriptions such as "the value goes to the ALU" by specifying which value is read, computed, loaded or selected.

The same signal names appear in both SEQ and PIPE. In SEQ they are temporary values within one instruction's cycle. In PIPE they are carried between stages in pipeline registers. This continuity is why the Y86-64 presentation scales from a simple processor to a pipelined one.

> **Caution**
>
> A register identifier and a register value are different objects. `srcA` is a register number; `valA` is the value read from that register. Confusing identifiers with values leads to wrong datapath descriptions.

3.3.4 Instruction classes, status, and control

The clearest way to organise the Y86-64 instruction set is by the kind of machine state each class disturbs. Data-movement instructions transfer register or memory values; their effective addresses may require arithmetic. ALU instructions produce a result and update condition codes. Jumps select a new PC, while calls and returns also access the stack and update its pointer. Two further groups sit slightly apart: stack instructions couple a memory access to an update of `rsp`, so they touch two pieces of state at once, and the halt and invalid-instruction cases exist mainly to drive the processor status code. Grouping the instructions this way pays off later, because instructions in the same class exercise the same parts of the datapath.

register-register. `rrmovq` and conditional moves use register values and write registers.

immediate. `irmovq` supplies a constant `valC` and writes a register.

memory. `rmmovq` and `mrmovq` compute an effective address and access data memory.

control. `jXX`, `call` and `ret` determine the next PC through different sources.

stack. `pushq` and `popq` update `rsp` and access memory.

Y86-64 records normal operation as `AOK`, halt as `HLT`, an invalid instruction as `INS`, and an invalid address as `ADR`. These statuses allow the processor model to define what happens when execution cannot continue normally. A complete implementation must propagate status information so that wrong-path or later instructions do not hide the real cause of termination.

> **Caution**
>
> Ignoring status values can make a simulator appear correct on ordinary programs while failing on invalid encodings or bad addresses. Correctness includes the defined behaviour at the boundary of valid execution.

Exception status must remain associated with the instruction

Each instruction eventually has a status such as normal completion, invalid opcode or address fault.

Detection logic creates status information at different places in the datapath. Pipeline registers, queue entries or other bookkeeping carry that information with the instruction so that the commitment mechanism can suppress writes and select recovery. Age and priority rules ensure that the architecturally relevant oldest exception is exposed before younger effects become visible.

Instruction status records why execution stops

	status	meaning
AOK	normal	continue
HLT	halt instruction	stopped
ADR	bad address	address error
INS	bad instruction	encoding error

Status is part of the architectural result.

Figure 3.25: *The processor must distinguish successful progression from specified stop conditions. Invalid instruction encodings and invalid addresses produce different status values in the model; they cannot be treated as ordinary register results.*

Precise exceptions require physical bookkeeping. Status must stay attached to the correct in-flight operation until the machine can present a consistent architectural state [50, 51].

3.3.5 Complete instruction traces

Worked example: decoding bytes

Assume the bytes at the PC are

```
30 f2 78 56 34 12 00 00 00 00.
```

Worked example (continued)

Byte `30` means `irmovq`. The register byte `f2` means no source register and destination register id `2`, `rdx`. The constant is little-endian `0x0000000012345678`. Thus the instruction is `irmovq $0x12345678, %rdx`, and `valP=PC+10`.

Consider four representative instructions.

`opq ra,rb:`	read both registers; compute `valB OP valA`; write `valE` to `rB`; update condition codes; advance to `valP`.
`mrmovq d(rb),ra:`	read base `rB`; compute effective address `valB+valC`; read memory into `valM`; write it to `rA`.
`call dest:`	decrement stack pointer; store `valP` at the new stack address; set PC to `valC`.
`ret:`	read the return address from memory at the old stack pointer; increment stack pointer; set PC to `valM`.

These descriptions already suggest the required multiplexers and control signals.

For `pushq`, the stack update follows the same RSP − 8 calculation as a call, while the stored qword comes from the selected source register. The memory address and new RSP must therefore agree.

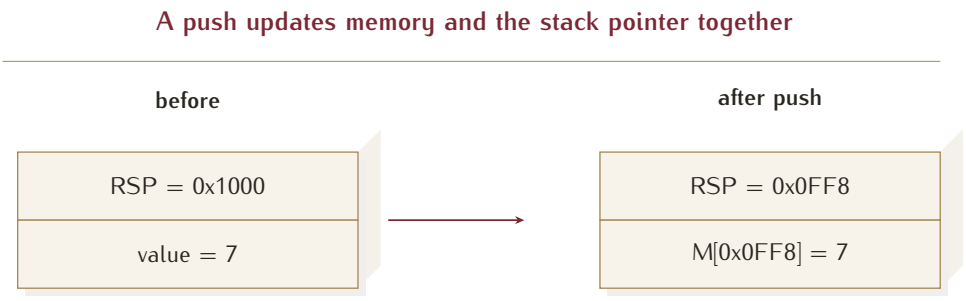


Figure 3.26: *The push computes the decremented stack pointer and stores the value at that new address. Both effects belong to the same architectural instruction. The trace assumes the target memory range is valid.*

Worked example: stage values for a store

Suppose `rmmovq %rax, 24(%rbx)` executes with `rax=0x55`, `rbx=0x1000`. Decode reads `valA=0x55` and `valB=0x1000`. Execute computes `valE=0x1000+24=0x1018`. Memory writes qword `0x55` at address `0x1018`. There is no register destination; the next PC is `valP`.

The instruction traces assume valid encodings and accessible memory. A processor that stops on a fault must preserve the effects of earlier instructions and suppress younger state changes. In a sequential design, checking the memory access before committing the instruction prevents a failed load from writing an invalid result. In a pipeline, status must accompany the instruction until the machine can stop at the corresponding architectural boundary.

Y86-64 uses ZF, SF and OF. A conditional move either writes its destination or turns the destination into "none". A conditional jump selects its target or fall-through PC. Both use the same Boolean condition function over flags; the difference is where the condition controls the datapath. For instruction classes 2 and 7, function 0 is unconditional; functions 1 through 6 select le, l, e, ne, ge and g in that order. Base instruction classes other than OPq and conditional transfers use function 0.

Condition	Meaning	Boolean test
`le`	signed $\leqslant$	$(\mathrm{SF} \oplus \mathrm{OF}) \vee \mathrm{ZF}$
`l`	signed $<$	$\mathrm{SF} \oplus \mathrm{OF}$
`e`	$=$	ZF
`ne`	$\neq$	$\neg\mathrm{ZF}$
`ge`	signed $\geqslant$	$\neg(\mathrm{SF} \oplus \mathrm{OF})$
`g`	signed $>$	$\neg(\mathrm{SF} \oplus \mathrm{OF}) \wedge \neg\mathrm{ZF}$

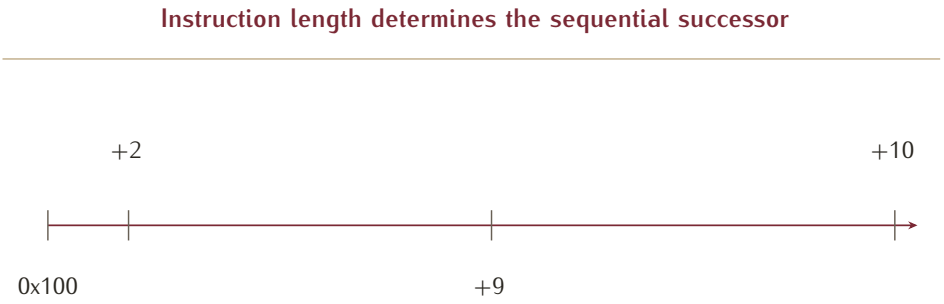


Figure 3.27: *Starting at address 0x100, these three consecutive lengths place boundaries at 0x102, 0x10B and 0x115. The actual successor can instead be a control-flow target, depending on the instruction semantics.*

3.3.6 Hand assembly as a consistency check

A useful exercise is to encode short Y86-64 programs manually. Work in two passes. In pass one, assign addresses by summing instruction lengths and record label positions. In pass two, emit opcode bytes, register bytes and little-endian constants, replacing label references with their final addresses. This is the same fundamental problem solved by a real assembler, though a production tool also handles sections, symbols and relocation.

Laboratory 19: Decoding Y86-64 bytes

Decode and execute bytes `30 f0 05 00 00 00 00 00 00 00 30 f3 07 00 00 00 00 00 00 00 60 03 00` in canonical Y86-64, starting at PC 0. Give instruction boundaries and final register values.
Worked solution

Laboratory 19: solution

At PC 0, `30 f0` selects irmovq with destination RAX; the following eight bytes encode 5, so the instruction is `irmovq $5,%rax` of length 10. At PC 10, `30 f3` similarly loads 7 into RBX and advances to 20. At PC 20, `60 03` is `addq %rax,%rbx`, length two. At PC 22, byte `00` halts. Final RAX is 5 and RBX is $7 + 5 = 12$; the add sets ZF=0, SF=0 and OF=0. The byte at address 22 is the terminating instruction and produces status HLT. All other program registers retain their initial values; execution performs no data-memory writes.
Return to the laboratory

From stored instructions to durable instruction-set architectures

Timeline

Year	Machine, standard or event	Architectural significance
1945	**Stored-program proposal**	Instructions and data share an encoded memory representation.
1964	**System/360 ISA**	Architecture is specified independently of implementation scale.
2003	**x86-64 era**	Long-lived ISA compatibility coexists with radically different microarchitectures.

Stored-program machines made the instruction format a central part of the interface between a program and its hardware. ENIAC was electronic and programmable, but substantial reprogramming initially meant setting switches and reconnecting cables. The 1945 *First Draft of a Report on the EDVAC* described instructions and numerical data stored in the same memory. The Manchester Baby demonstrated the stored-program principle

on 21 June 1948; EDSAC began useful operation in May 1949. Once a program was a sequence of words in memory, designers had to define exactly how each word selected an operation, operand and next instruction [22, 89, 91, 69].

Early machines treated this definition as a property of one installation. Word lengths, numeric formats and operation codes varied widely. EDSAC used a 17-bit short word and single-letter order codes represented on paper tape. The IAS family used 40-bit words containing two 20-bit instructions. Scientific and commercial machines often had different arithmetic and data models. Software was rewritten when a customer moved to another computer, and even models from one manufacturer could be incompatible [49, 50, 26].

> **1964: preserving a program while changing the machine**
>
> IBM's System/360 project confronted the cost of incompatible product lines. A family covering different performance and price points needed a common programmer-visible contract. Its 1964 architectural specification described registers, instructions, data formats and interruption behaviour independently of a particular model's circuitry [63, 74, 109].
>
> The decision made software compatibility an engineering constraint. A smaller implementation could take more internal steps to perform an operation than a larger one, while preserving the architectural result for programs within the supported configuration. Required facilities, capacity and system environment still mattered when moving software.
>
> Read the distinction through one addition: its operands, result width and condition-code effects belong to the architecture. The number of internal cycles, the organisation of the arithmetic unit and the use of microcode belong to the implementation. A binary program depends on the first contract; a performance prediction additionally depends on the second.
>
> Y86 applies the same separation on a smaller scale. The instruction rules remain fixed while the following chapters construct two different execution organisations.

Microprocessors carried that constraint into mass-market computing. Intel introduced the 8086 in 1978 as a 16-bit processor with segmented addressing and variable-length instructions. The 80286 added protected mode; the 80386 extended registers and addressing to 32 bits; later processors added caches, superscalar execution and vectors while retaining the ability to run much older code. AMD's x86-64 extension, commercially introduced with Opteron in April 2003 and Athlon 64 in September 2003, added 64-bit general registers, more registers and a long mode that preserved a compatibility submode. Intel subsequently implemented the compatible architecture under the Intel 64 name. The x86 encoding consequently retains conventions from several generations, reflecting the cost of changing an established binary interface [104, 106, 107, 26, 27].

RISC research challenged another assumption: that a useful architecture should provide many elaborate instructions. IBM's 801 project in the 1970s and university work at Berkeley and Stanford showed that a simpler load/store instruction set, many registers and compiler-friendly operations could support efficient pipelines. David Patterson and David Ditzel's 1980 "Case for the Reduced Instruction Set Computer" gave the movement its influential name. MIPS, SPARC, ARM and later Power architectures adopted different versions of the idea. These designs placed particular emphasis on regular encoding, explicit loads and stores, register use and pipeline control, distributing work among hardware, microcode and the compiler [53, 54].

Current architectures continue both traditions. x86 processors translate complex variable-length instructions into internal micro-operations. AArch64 instructions use fixed 32-bit encodings and a load/store model. RISC-V, first developed at the University of California, Berkeley from 2010, defines a small base ISA and optional standard extensions under an open specification. GPUs expose wide parallel execution models and specialised memory spaces. Compatibility, tooling, power and ecosystem often matter more than the aesthetic purity of an instruction format [49, 50, 26].

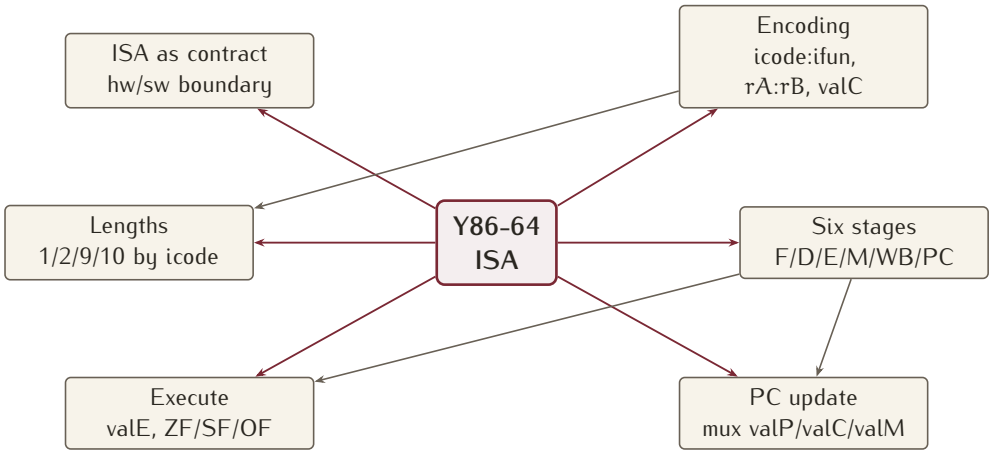
Bryant and O'Hallaron designed Y86-64 as an educational ISA with a complete encoding and reference sequential and pipelined implementations. Its selected registers, operations and condition codes make the entire datapath manageable to trace. The resemblance to x86-64 helps connect the models, while Y86-64 has its own byte encoding and requires its own assembler and simulator [1].

This distinction between ISA and implementation is the main historical result. The ISA states what software may observe: register effects, memory effects, exceptions and instruction encodings. It does not require a one-to-one physical unit for each stage in a diagram. A current out-of-order core may fetch many bytes, decode instructions into micro-operations, rename registers, execute speculatively and retire in order, while appearing to obey the same sequential architecture as an earlier scalar processor. Conversely, two implementations of one ISA can have radically different performance and security behaviour [49, 50, 26].

To read an ISA as an engineer, one should compare generations rather than memorise isolated mnemonics: inspect an early stored-program order code, read the System/360 architectural contract, follow the 8086-to-AMD64 compatibility line, then examine a RISC specification and the Y86-64 teaching model. These comparisons explain how instruction fields acquired their roles and why older conventions survive: programs, operating systems and ABIs turn architectural decisions into long-lived public commitments [22, 63, 104, 107, 1].

Instruction-set contracts still separate software compatibility from the freedom to redesign the underlying microarchitecture.

Concept map: Y86-64 encoding & stages



Reading the map. The ISA contract fixes the encoding (icode:ifun, register byte, immediate), from which instruction lengths follow. Execution unfolds in six stages; Execute produces $valE$ and the flags, and PC update is the mux that selects the next address ($valP/valC/valM$), the skeleton the SEQ datapath of 3.4 fills in.

3.3.7 Worked problems

Opcode split

The instruction byte `0x63` is binary 0110 0011. Shifting right by four gives `icode=6`; masking with `0xF` gives `ifun=3`. Class 6 is OPq, whose function 3 selects XOR, so this is the first byte of a valid `xorq` instruction. The following register byte must still identify its operands.
Recombining the fields gives $(6 \ll 4) \vee 3 = \texttt{0x63}$, which checks the bit extraction. Validity is a separate check against the function table: `0x64` would split into $(6, 4)$, whose function is undefined for OPq.

Return address and stack update

A `ret` at PC $=$ `0x100` executes with RSP $=$ `0x800` and the qword at `0x800` equal to `0x250`. Fetch computes $valP =$ `0x101`. Decode supplies the old stack pointer as both $valA$ and $valB$. Execute forms $valE = valB + 8 =$ `0x808`, while Memory reads at $valA =$ `0x800` and produces $valM =$ `0x250`. Write-back stores `0x808` in RSP, and PC update selects `0x250`. The computed fall-through address is unused. Reading at `0x808` would retrieve the following stack word and violate the return semantics.

Exercise 3.3.1 — Jump length

How long is a jXX with 8-byte destination?

Worked solution

The specified Y86-64 jump contains one instruction byte, which combines opcode and function, followed by an 8-byte destination constant. Its total length is $1 + 8 = \boxed{9}$ bytes. Starting at address PC, the sequential next address is PC + 9. A taken jump selects the decoded destination instead. The instruction has no register-specifier byte in this format.

Exercise 3.3.2 — Low nibble

How obtain ifun from byte b?

Worked solution

The function code occupies the low four bits of the instruction byte. Mask with `0x0F`: `ifun = b & 0xF`. For byte `0x63`, the result is `0x03`; shifting the same byte right by four gives opcode `0x06`. Masking preserves the low nibble and clears the opcode bits, so the two fields can be interpreted independently.

Exercise 3.3.3 — No register

Which nibble value means RNONE?

Worked solution

The register-specifier nibble `0xF`, binary 1111, denotes `RNONE` in the Y86-64 format used here. It marks an absent register operand or destination. It must be handled as a sentinel by the decoder, because reading or writing an actual register with that identifier would invent state outside the declared register file. For an immediate move, the unused source-register nibble uses this value.

Exercise 3.3.4 — ZF

When is ZF set?

Worked solution

ZF is set when the result of an operation that updates the condition codes is zero at the operation's width. For example, subtracting equal operands produces all zero result bits and sets ZF. A nonzero result clears it. An instruction that does not write condition codes leaves the prior ZF unchanged, so interpreting ZF requires identifying the most recent instruction that actually updated it.

Summary

For each instruction, build a small ledger: bytes consumed, registers read, ALU function, memory action, destination registers, condition-code action and next PC. Once that ledger is correct, both sequential and pipelined implementations become exercises in scheduling the same obligations.

3.3.8 Exercises

Exercise 3.3.5 — Instruction length

Give the lengths of `nop`, `addq %rax,%rbx`, `irmovq $5,%rcx`, `call 0x100` and `ret`.

Go to the solution →

Exercise 3.3.6 — Decode a register byte

For an `OPq` instruction whose register byte is `23`, identify `rA` and `rB` using the standard Y86-64 numbering.

Go to the solution →

Exercise 3.3.7 — Little-endian constant

Decode the eight bytes `f0 de bc 9a 78 56 34 12` as a 64-bit constant.

Go to the solution →

Exercise 3.3.8 — Stage values for `mrmovq`

`mrmovq -8(%rbp),%rax` executes with `rbp=0x2000` and memory qword at `0x1ff8=0x77`. Give `valB,valC,valE,valM,dstM`.

Go to the solution →

Exercise 3.3.9 — Call semantics

Before `call 0x400`, PC is `0x100`, the instruction length is 9, and `rsp=0x800`. Give the new stack pointer, stored return address and next PC.

Go to the solution →

Exercise 3.3.10 — Conditional move

A `cmovle %rax,%rbx` sees $ZF = 0, SF = 0, OF = 1$. Does it write `rbx`?

Go to the solution →

Exercise 3.3.11 — Invalid encoding

A strict decoder requires the documented RNONE source field for `irmovq`. What should it report if this field contains a real register ID, and what extra check is needed beyond testing the instruction class?

Go to the solution →

Exercise 3.3.12 — Two-pass assembly

A program starts at 0: `irmovq $1,%rax; jmp done; nop; done: halt`. Find the address encoded in the jump.

Go to the solution →

3.3.9 Solutions

The Y86-64 Instruction Set and Execution Stages

Solution 3.3.5

In Y86-64, every instruction starts with one byte containing `icode` and `ifun`. `nop` and `ret` need nothing further, so each occupies one byte. `addq` adds a register-specifier byte, giving two bytes. `irmovq` adds that register byte and an eight-byte immediate, giving $1+1+8=10$ bytes. `call` has an eight-byte destination and no register byte, giving $1+8=9$ bytes. In the requested order the lengths are therefore $1, 2, 10, 9, 1$.

← Return to the exercise

Solution 3.3.6

Write the register byte as `0x23`: its high nibble is $\mathrm{rA}=2$ and its low nibble is $\mathrm{rB}=3$. The register table maps these to `rdx` and `rbx`. Decode reads $\mathrm{valA}=\mathrm{R}[\mathrm{rdx}]$ and $\mathrm{valB}=\mathrm{R}[\mathrm{rbx}]$; Execute computes valB OP valA, and Write-back places that result in `rbx`. The first instruction byte supplies the function, so the register byte alone does not determine whether the operation is ADD, SUB, AND or XOR.

← Return to the exercise

Solution 3.3.7

Little-endian storage places the least significant byte at the lowest address. The first byte therefore contributes `0xF0`, the next `0xDE`$\cdot 2^8$, and so on. To write the ordinary hexadecimal numeral, read the bytes from highest address to lowest: `12 34 56 78 9a bc de f0`. The decoded constant is `0x123456789ABCDEF0`. Endianness reverses the order of bytes in the numeral; the bit order within each byte stays unchanged.

← Return to the exercise

Solution 3.3.8

Decode obtains the base register value `valB=0x2000` and signed displacement `valC=-8`. Execute adds them: `valE=0x2000-8=0x1ff8`. Memory reads the qword at that effective address, giving `valM=0x77`. Write-back routes this memory result to `dstM=rax`. Thus the destination receives the stored value `0x77`; `0x1FF8` is the address used to retrieve it. The base register retains its original value.

← Return to the exercise

Solution 3.3.9

Fetch computes the address after the nine-byte call: $\mathtt{valP} = \mathtt{0x100} + 9 = \mathtt{0x109}$. Decode reads the old stack pointer `0x800`. Execute reserves one qword by subtracting eight, giving $\mathtt{valE} = \mathtt{0x7F8}$. Memory stores the return address `0x109` there, as bytes `09 01 00 00 00 00 00 00` at addresses `0x7F8` through `0x7FF`. Write-back sets RSP to `0x7F8`, and PC update selects the encoded destination `0x400`. A later return can read the saved qword and resume at the instruction following the call.

← Return to the exercise

Solution 3.3.10

The signed less-or-equal test is $\mathtt{Cnd} = \mathrm{ZF} \vee (\mathrm{SF} \oplus \mathrm{OF})$. Substituting the given flags gives $0 \vee (0 \oplus 1) = 1$. The move therefore writes the current RAX value to RBX. Overflow changes the interpretation of the result sign, which is why testing SF alone would give the wrong decision. The conditional move preserves the three condition codes.

← Return to the exercise

Solution 3.3.11

The strict decoder reports INS because the register byte violates the stipulated format. For example, `30 03` has a valid irmovq instruction class but uses source ID 0 in place of F. Validation must therefore check the function code and register-field constraints as well as the class. The compact reference HCL tests class membership in its `instr_valid` expression; that expression alone is not a complete strict validator for every malformed byte sequence. Acceptance by a permissive simulator does not establish conformity to the documented encoding.

← Return to the exercise

Solution 3.3.12

Pass one assigns addresses from the instruction lengths: `irmovq` begins at 0 and occupies bytes 0–9; `jmp` begins at 10 and occupies bytes 10–18; `nop` occupies byte 19; `halt`, labelled `done`, is at byte 20, or `0x14`.
Pass two emits the jump opcode `0x70`, followed by its eight-byte destination in little-endian order:

```
70 14 00 00 00 00 00 00 00.
```

The encoded value is the absolute target 20. Execution jumps over the nop; the sequential successor of the jump would have been 19.

← Return to the exercise

3.4 THE SEQUENTIAL PROCESSOR

Consider one instruction from the moment its address enters the datapath to the moment its results are stored. The sequential processor gives that instruction the machine's complete execution path before beginning the next. Every selector and control equation can therefore be checked against a single architectural transition. This design becomes the reference for deciding whether later overlap still produces the required result.

A single instruction uses a connected computation path

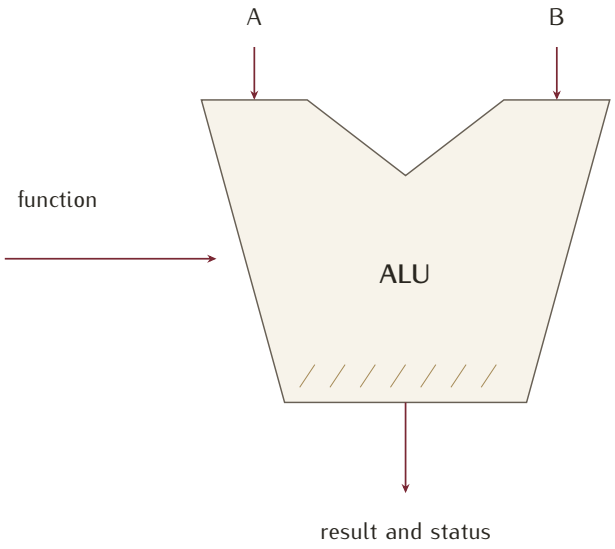


Figure 3.28: *SEQ connects decoding, selection, arithmetic, memory and state updates within one clocked instruction. The ALU is one shared part of that path. Its input selectors and output destinations depend on the current instruction.*

3.4.1 From ISA stages to one clocked machine

The previous section introduced the six execution stages of a Y86-64 instruction. The next question is how they *coordinate*. The answer depends on where state boundaries are placed: a single-cycle datapath propagates values through the stages between clock edges, while a multicycle design stores intermediate results.

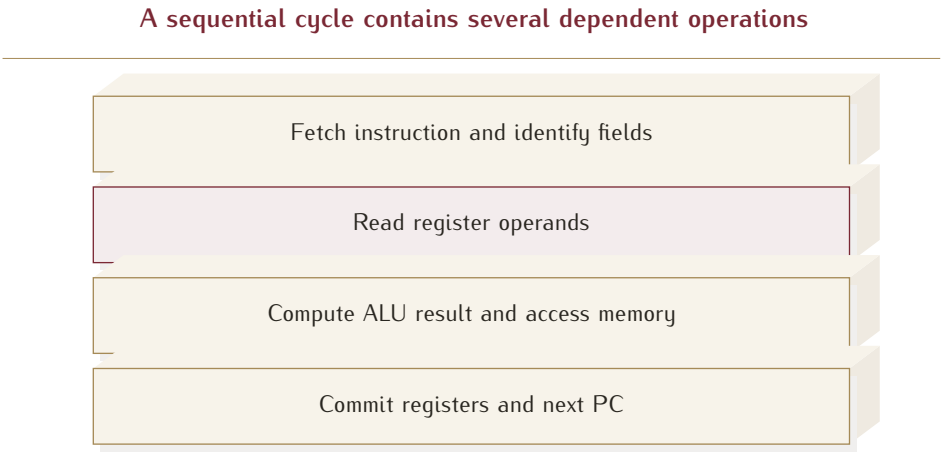


Figure 3.29: *The stage names describe work that must be performed for an instruction. A single-cycle implementation places the relevant combinational work between architectural-state updates, so the longest supported path constrains the clock.*

The first implementation is a *single-cycle* architecture in which every instruction traverses all six stages within one clock period. The model is simple, but its timing cost is severe: the clock must accommodate the longest combinational path of the most expensive instruction. Multi-cycle and pipelined designs address that limitation in different ways.

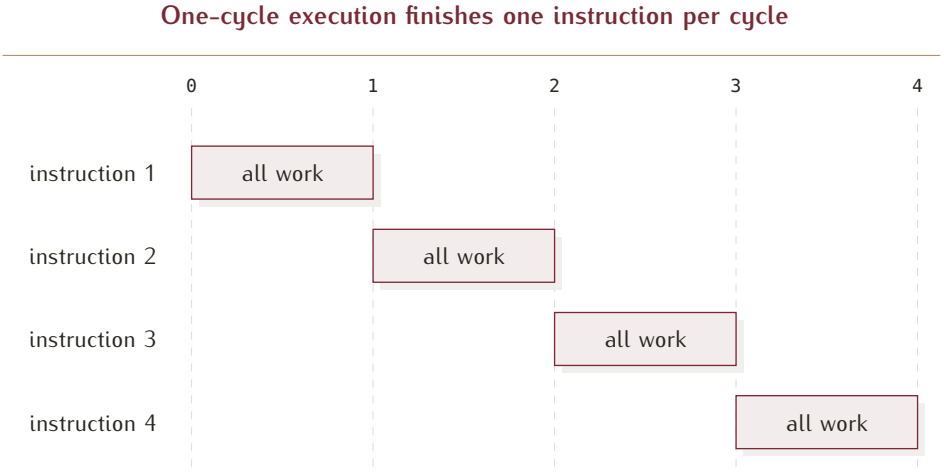


Figure 3.30: *The timeline represents a single-cycle sequential machine. Instructions do not overlap architecturally in this model. Shorter instruction paths still occupy the same clock period as the longest path.*

3.4.2 The SEQ single-cycle datapath

In a single-cycle CPU, every instruction completes all its stages within one clock cycle:

- All six-stage logic (fetch, decode, execute, memory, write-back, PC update) is *combinational*: gates, ALU, muxes, the PC adder, decoders.
- The architectural state in this teaching model is held by the *PC*, the *register file*, the *data memory*, and the *condition-code register*; the datapath between state boundaries is modelled combinationally.
- Between edges, combinational logic propagates: the current PC feeds the instruction decoder, the register file produces valA and valB, the ALU computes valE, and so on, down to the new PC and new register values.

At the active clock boundary the synchronous abstraction treats enabled state changes as one architectural transition. Physical flip-flops and clock networks do not switch at an infinitely precise common instant, which is why setup, hold and skew constraints still matter.

In the SEQ timing model, each enabled state element captures its new value at the active edge. The instruction is therefore one combinational sweep from the old state to the new. The resulting performance problem motivates *pipelining*: the clock period is bounded below by the worst-case path delay. The distinction between *latency* (time for one instruction) and *throughput* (instructions per unit time) is the lens for the single-cycle versus pipelined comparison below.

Worked example

Problem. At cycle end, the ALU result is written to the destination and ZF is updated. `rdi`, `rsi` are the operands of an `add`. Leave the new destination in `rax` and the new ZF in `rbx`.
Solution.

```
mov rax, rsi
add rax, rdi; valE = valB + valA
xor rbx, rbx
test rax, rax
setz bl; ZF = (valE == 0)
```

In hardware this is one cycle: the ALU produces valE and ZF in combinational time, and on the edge both are written (to the register file's write port and the flag register).

Worked example

Problem. Simulate `sub` then `je`. `rdi`, `rsi` operands; `rdx` jump target, `rcx` = valP. After `sub`, `je` jumps only if ZF=1. Leave the new PC in `rax`.
Solution.

Worked example (continued)

```
; cycle 1: sub
mov r8, rsi
sub r8, rdi; valE = valB - valA
xor r9, r9
test r8, r8
setz r9b; ZF; cycle 2: je
test r9, r9
mov rax, rcx; default: fall-through valP
cmovnz rax, rdx; if ZF != 0, take the target
```

Between cycles, ZF is held by the sequential flag register; `je` reads it to choose the PC, the same pattern as real x86-64, where arithmetic updates the flags and the next jump consumes them.

Y86-64 rmmovq and mrmovq instructions use base+displacement: the datum is at $valB + valC$. Execute computes $valE = valB + valC$; Memory uses $valE$ to read (mrmovq) or write (rmmovq).

Worked example

Problem. `rdi` $= valB$ (base), `rsi` $= valC$ (displacement), `rdx` $= valA$. Store $valA$ at address $valE$.

Solution.

```
mov rax, rdi
add rax, rsi; valE = valB + valC
mov [rax], rdx; mem[valE] = valA
```

The ALU adder that forms $valE$ is the *same* one used for OPq arithmetic: a 64-bit add, whether the operands are two arithmetic values or a base/displacement pair. The instruction class selects the inputs, result destination and flag-write control. OPq can update the condition codes; address calculation for a store preserves them.

The single-cycle critical path charges real capacitance

The clock period must exceed the sum of component delays on the longest path.

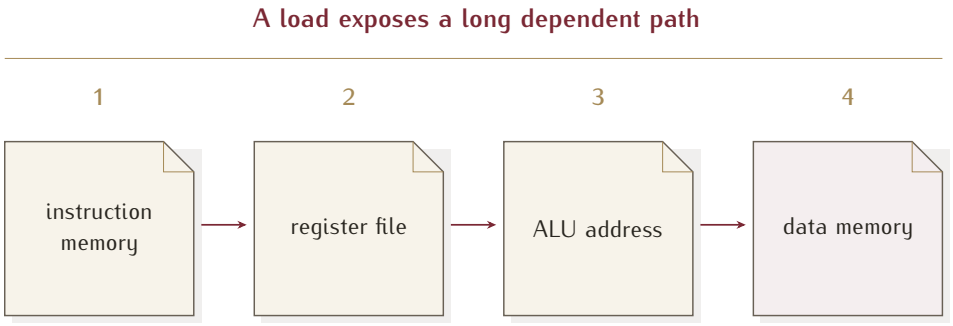


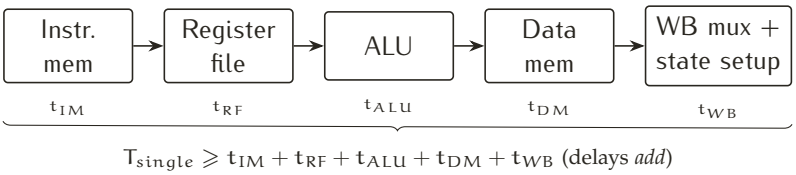
Figure 3.31: *The next operation in this path requires a result from the previous one. In a single-cycle design, the clock must accommodate the complete path plus state-element timing requirements. The boxes are conceptual components, not equal-delay stages.*

Each logic stage drives transistor gates and wires that behave approximately as capacitive loads through finite resistance. A first-order local model uses $\tau \approx R_{eq}C_{load}$, although real libraries use characterised nonlinear delay tables. The complete path also includes clock-to-output and setup requirements.

The worst instruction determines the minimum safe period in a single-cycle design because all relevant voltages must settle within valid margins before the next capturing edge [41, 42].

3.4.3 Timing, state, and architectural commitment

Here is the structural problem of single-cycle. The clock period must be long enough for *every* instruction to complete its combinational path within one cycle, so it is fixed by the *worst* instruction. That is typically `mrmovq` (load), which traverses almost the whole datapath:



Worked example

Problem. The delays (ps) of Instruction Memory, Register File, ALU, Data Memory, and write-back/setup are in `rdi`, `rsi`, `rdx`, `rcx`, `r8`. Compute the single-cycle period (their sum) in `rax`.

Solution.

Worked example (continued)

```
mov rax, rdi
add rax, rsi; + RF
add rax, rdx; + ALU
add rax, rcx; + DM
add rax, r8; + WB
```

Caution

The sum is a *lower* bound: the real period must be at least this, plus any clock-to-output, clock uncertainty and variation allowance not already included. Destination setup is included in t_{WB} here and must not be counted twice. If other paths have different delays, the period must also cover their maximum. In practice one computes delays over all instructions, takes the maximum, and adds a safety margin.

The long `mrmovq` path crosses stages that simpler instructions leave idle. A multi-cycle design places internal sequential registers between stages and assigns each stage a clock cycle. The period is then set by the slowest individual stage rather than by the sum of all stage delays, while simpler instructions can complete in fewer cycles. These registers do not yet form a pipeline in the sense developed in the next section, because only one instruction is in flight.

Worked example

Problem. Three separated stages; the cycle is the slowest stage plus the register cost. Delays in `rdi`, `rsi`, `rdx`; register cost in `rcx`. All values and their final sum are nonnegative and fit in a signed 64-bit register. Compute the multi-cycle period in `rax`.

Solution. $T_{multi} = \max(t_1, t_2, t_3) + t_{reg}$:

```
mov rax, rdi
cmp rsi, rax
cmovg rax, rsi; max(t1, t2)
cmp rdx, rax
cmovg rax, rdx; max(.., t3)
add rax, rcx; + register cost
```

Single-cycle, multi-cycle, pipelined. In *single-cycle*, each instruction finishes in one (long) cycle; simple instructions waste most of it. In *multi-cycle*, stages are separated by registers, each cycle is short (max stage delay + register overhead), and simple instructions use fewer cycles, but only one instruction is in flight. In *pipelined*, the stages are

likewise separated, but as an instruction moves from stage 1 to stage 2 a *new* one enters stage 1: in a full n-stage scalar pipeline up to n instructions are in flight. With no stalls, one instruction completes per short cycle. The next sections develop this.

Timing changes with process, voltage and temperature

A simple timing model assigns each component a delay bound.

Transistor speed varies with manufacturing process, supply voltage and temperature. Lower voltage generally reduces drive strength. Temperature changes carrier mobility, threshold behaviour, resistance and leakage; the net delay trend can depend on technology and operating voltage rather than being universally monotonic. Local voltage droop can temporarily slow a path. Designers therefore verify timing across multiple PVT corners and include variation margins.

A qualified frequency-and-voltage operating point covers the specified range of manufacturing variation and operating conditions. Overclocking or undervolting reduces those margins and can produce intermittent, data-dependent failures [44, 42].

Splitting a path changes both timing and energy

A multi-cycle design divides an instruction across shorter clocked steps.

Shorter combinational paths can permit a faster clock. For a fixed throughput target, they may also create an opportunity to lower supply voltage, although that benefit is not automatic. Each added state boundary requires registers, clock wiring and control, and intermediate values are stored rather than flowing directly through one path.

The energy per completed instruction depends on voltage, clock activity, register overhead, unit utilisation and the control required to gate unused structures [41, 50].

3.4.4 Control equations, tracing, and verification

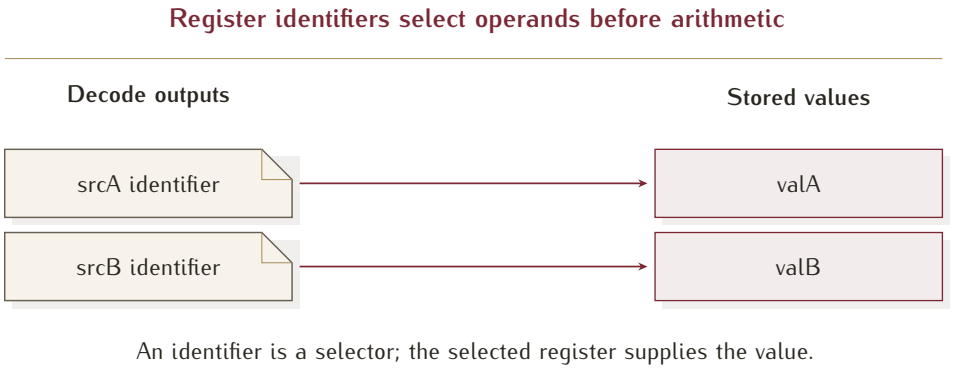


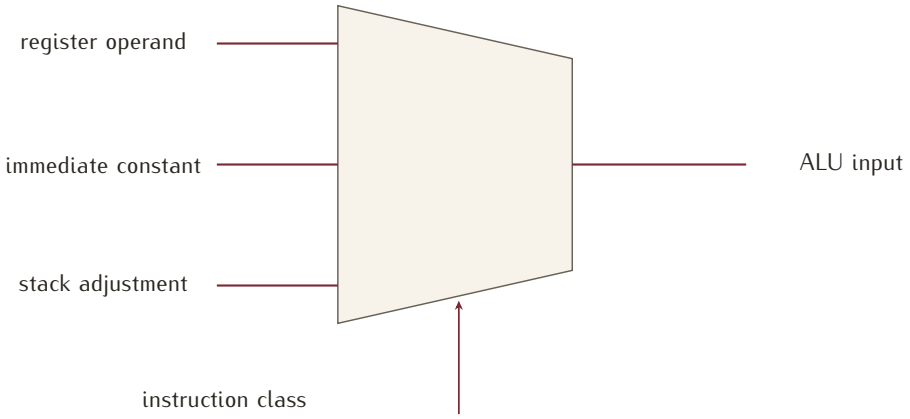
Figure 3.32: *The distinction prevents confusing a small register number with the data contained in that register. Decoder control chooses source identifiers, then the register file produces the operand values used by later stages.*

To derive SEQ, collect the instructions that need each source and express the selection as a case function. For example, the ALU's A input can be:

$$\texttt{aluA} = \begin{cases} \texttt{valA}, & \texttt{rrmovq/OPq}, \\ \texttt{valC}, & \texttt{irmovq/rmmovq/mrmovq}, \\ -8, & \texttt{call/pushq}, \\ +8, & \texttt{ret/popq}, \\ 0, & \text{otherwise (unused ALU result).} \end{cases}$$

The B input is `valB` for OPq, loads, stores and the four stack-using classes call, ret, pushq and popq. For `rrmovq/cmovXX` and `irmovq`, it is zero, so adding passes the A input through unchanged. An unused ALU computation can also select zero. OPq selects its arithmetic or logical function through ifun; all other useful ALU results use addition. A multiplexer implements each input selection.

The ALU input depends on the instruction



Separate selection logic supplies each ALU operand.

Figure 3.33: *Arithmetic, address formation and stack updates reuse the ALU with different sources. Control equations select the correct input values for each instruction family. Reuse saves hardware while increasing the importance of correct control.*

The register selectors complete this control specification. A dash in the following table means RNONE; it disables the corresponding architectural read requirement or write destination.

Instruction	srcA	srcB	dstE	dstM
halt, nop, jXX	–	–	–	–
rrmovq/cmovXX	rA	–	rB if Cnd	–
irmovq	–	–	rB	–
rmmovq	rA	rB	–	–
mrmovq	–	rB	–	rA
OPq	rA	rB	rB	–
call	–	rsp	rsp	–
ret	rsp	rsp	rsp	–
pushq	rA	rsp	rsp	–
popq	rsp	rsp	rsp	rA

For unconditional rrmovq, Cnd is always true. For a failed conditional move, dstE becomes RNONE. The E write port carries valE and the M write port carries valM. If `popq %rsp` selects RSP on both ports, the memory-result write has priority: final RSP is the popped qword. This priority is part of the register-file behaviour required by the instruction semantics.

Control equations need a safe default. A destination register should default to `RNONE`, a memory-write signal to false, and a status to normal unless a specific error is detected. Safe defaults make unused paths inert. An accidental register identifier or asserted write enable can corrupt architectural state even when the data value seems irrelevant.

Worked example: deriving `srcA`

`srcA=rA` for `rrmovq`, `rmmovq`, `OPq` and `pushq`. It is `rsp` for `popq` and `ret`. Other instructions do not need an A source, so the default is `RNONE`. This single equation covers ordinary register operands, the stack's implicit operand and the absent case.

Control equations encode instruction cases

	class	memory action	result source
OPq	arithmetic	none	valE
mrmovq	load	read	valM
rmmovq	store	write	no register result

The table summarises selected effects; the complete equations include all classes.

Figure 3.34: *The row structure makes differences between instruction families explicit. A load uses the memory result, a store changes memory, and an arithmetic instruction uses the ALU result. Control must also handle destinations, PC and status.*

Consider:

```
irmovq $0x100, %rsp
irmovq $7, %rax
pushq %rax
popq %rbx
halt
```

After the first instruction, `rsp=0x100`. The second writes 7 to `rax`. `pushq` computes `valE=0x100-8=0xf8`, writes 7 to memory at `0xf8`, and writes `0xf8` to `rsp`. `popq` reads memory at the old `valA=0xf8`, computes `valE=0x100`, writes `valM=7` to `rbx`, and restores `rsp`. Two register writes are required in the same instruction: one for the updated stack pointer and one for the popped value.

In a single-cycle design, every instruction occupies one clock period, so the period must accommodate the slowest instruction. A register-to-register move may use instruction memory, register file, a simple ALU pass and write-back. A load additionally uses data memory. A return uses data memory and then routes the loaded value to the PC. Short instructions finish early but cannot start the next instruction until the common edge.

Worked example: critical-path estimate

Assume instruction memory 180 ps, register read 90 ps, ALU 120 ps, data memory 220 ps, register setup 25 ps and multiplexers/control 65 ps in total. For this numerical model, launch clock-to-Q is included in the first block delay and additional clock uncertainty is zero. A load path takes approximately

Worked example (continued)

$180 + 90 + 120 + 220 + 65 + 25 = 700$ ps. If this is the longest path, the maximum clock is about 1.43 GHz. An `OPq` that omits data memory still consumes the full 700 ps cycle.

The combinational network may compute many candidate values during a cycle, but they become architecturally visible only at the clock edge. This distinction is *commitment*. For an invalid instruction or address fault, the processor must arrange that no incorrect register, memory or condition-code update commits.

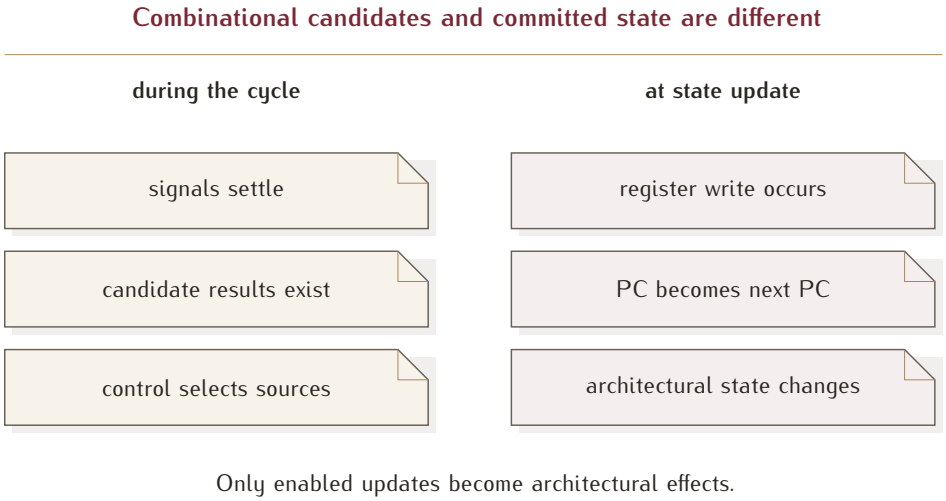


Figure 3.35: *A computed signal may be unused for the current instruction. The write enables and destination controls decide which candidates alter state. Correct values on unused wires do not compensate for an incorrect enabled write.*

Memory writes are particularly important because they are side effects. A clean model qualifies the write enable with a normal status. Condition codes should be updated only by valid arithmetic instructions. The model stops on HLT, INS or ADR according to its simulator convention; it has no operating-system exception-handler vector. A different processor may redirect control to a handler, but that requires an explicit architectural rule.

HCL expresses Boolean and word-level combinational functions. A case expression lists guarded values; conceptually all conditions and candidate values exist in hardware in parallel, and a multiplexer selects the result. The order can encode priority, but it does not mean that one case is "executed" after another.

Worked example: next-PC priority

A simplified next-PC rule can be written conceptually as:

$$\texttt{newPC} = \begin{cases} \texttt{valC}, & \texttt{taken call or jump}, \\ \texttt{valM}, & \texttt{ret}, \\ \texttt{valP}, & \text{otherwise}. \end{cases}$$

The exact priority must reflect the instruction classes, which are mutually exclusive in SEQ. In a pipeline, signals from different instructions can coexist, so priority becomes more subtle.

A processor can be checked with invariants at each boundary:

- `valP` equals the address immediately after the fetched instruction;
- `valA` and `valB` correspond to `srcA` and `srcB`;
- `valE` equals the instruction's specified arithmetic or address result;
- a memory write uses the specified address and data;
- only `dstE` and `dstM` can change the register file;
- the chosen next PC matches the instruction semantics.

A failing final state can then be localised to the first broken invariant rather than debugged as one monolithic circuit.

3.4.5 Beyond single-cycle SEQ

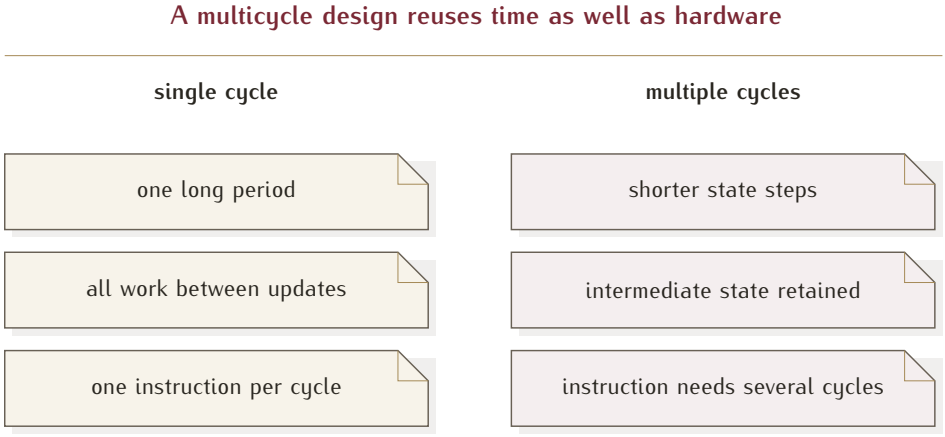


Figure 3.36: *Breaking the work across cycles introduces intermediate control and storage requirements. It can shorten the period or permit hardware reuse, but one instruction then spans several cycles. A comparison needs both timing terms.*

A multi-cycle processor reuses hardware across several shorter cycles. One instruction may fetch in one cycle, decode in another, execute later and access memory in a fourth. Temporary registers preserve intermediate values. The clock period can be shorter and one ALU can serve several roles, but CPI exceeds one and a controller must track the current phase. Pipelining also divides work into stages, but overlaps different instructions instead of completing one before starting the next.

Hardwired and microprogrammed control

Control can be generated directly by Boolean logic or by a microprogram that sequences simpler internal operations. Microprogramming made complex control sequences easier to organise. A writable or patchable control store also permits selected updates; a fixed ROM requires a hardware change. Hardwired control offers direct paths for regular operations. Contemporary processors often combine decoded micro-operations, specialised sequencers and hardwired fast paths. [15, 49]

The sequential datapath is obtained mechanically from the instruction table. Every required intermediate value becomes a signal; every alternative source becomes a multiplexer input; and every architectural update is placed after the combinational logic that computes it. Timing then determines the minimum clock period.

Laboratory 20: SEQ state transitions

Trace `irmovq $5,%rax; irmovq $3,%rbx; subq %rbx,%rax; je target`, starting at PC 0 with target 100. Give each next PC, the subtraction operands/result and branch outcome. Repeat with first constant 3.
Worked solution

Laboratory 20: solution

The two immediates have lengths ten, so their PCs are 0 and 10 with valP 10 and 20. Each uses no source register, ALU inputs valC and zero, and writes valE to RAX or RBX. The subtraction at PC 20 has valP 22, srcA=RBX, srcB=RAX, valA=3, valB=5, ALU result $valB - valA = 2$, dstE=RAX, and flags ZF=0,SF=0,OF=0. The jump at PC 22 has length nine and valP 31; it reads ZF, finds the equality condition false and sets next PC=31. With first constant 3, subtraction instead yields zero and ZF=1, so next PC=100. The jump itself does not rewrite condition codes. No instruction in this trace performs a data-memory read or write.
Return to the laboratory

From timed control sequences to microcode and multicycle datapaths

Timeline

Year	Machine, standard or event	Architectural significance
1951	**Wilkes microprogramming**	Control sequences are represented in a control store.
1964	**System/360 implementations**	Different internal control strategies realise one ISA.
1978	**8086 execution unit**	Multicycle sequencing economises hardware in a microprocessor.

Every stored-program processor must convert one public instruction into a sequence of internal actions. The instruction may name a register addition, but the machine must still fetch the opcode, select registers, drive an adder, capture the result and update condition state. In the first electronic machines these actions were coordinated by pulse generators, counters, delay elements, diode matrices and hand-wired Boolean logic. Arithmetic was often serial: one adder processed successive bit positions while registers shifted. A “single instruction” was therefore physically a timed sequence long before textbooks introduced a multicycle datapath [49, 50].

The EDSAC, which ran its first program at Cambridge on 6 May 1949, illustrates hard-wired sequencing. Its order code was small and closely matched the available circuits. Timing pulses stepped the machine through memory access, decoding and arithmetic.

The resulting control could be fast for a simple design, but adding a complicated instruction meant modifying a web of logic. As instruction sets and product lines grew, designers needed a more systematic way to specify the sequence of control signals [91, 70].

Maurice V. Wilkes proposed the decisive abstraction in 1951 in "The Best Way to Design an Automatic Calculating Machine." He described control signals selected from a matrix under a sequence register: one row represented one internal step, and a second selection mechanism determined the following row. The proposal turned control design into a stored sequence of microinstructions. A microinstruction could select sources, an ALU operation, a destination, a memory action and the next microaddress. Instead of deriving every control line directly from the architectural opcode, the processor could execute a small control program [15, 93].

Wilkes's group implemented the concept in EDSAC 2, which became operational in 1958. Its microprogram control matrix used ferrite cores [92]. The term *microprogramming* did not imply that users could rewrite the machine at will; it meant that the public order code was interpreted by a lower-level sequence held in a specialised store. The distinction remains essential: the Instruction Set Architecture defines what software observes, while microinstructions are implementation details whose format, timing and even existence can change between processors [93].

IBM made microprogramming commercially consequential with System/360, announced on 7 April 1964. The family promised one compatible architecture across models spanning a roughly fiftyfold performance range. That promise could not be met by building every model with the same datapath. Smaller systems reused narrower hardware over more cycles, while larger systems used more parallel implementation. The System/360 Model 30 stored microprograms in Card Capacitor Read-Only Storage. Interchangeable punched cards altered the capacitive pattern read by the control circuitry and could support emulation of older IBM 1401 instructions. Architecture had become a stable customer contract; microcode became one means of implementing and preserving it [74, 109, 121].

This history explains the textbook multicycle processor. Fetch may use the memory interface in one cycle; decode and register read follow; the ALU is then reused to calculate an address or arithmetic result; memory access and write-back occur later. Intermediate registers hold the instruction, memory data, ALU result and operands between cycles. A finite-state machine or microsequencer asserts a different control word at each step. Reuse lowers hardware cost, but instruction latency varies: a register move needs fewer states than a load, branch or multiply [49, 50, 28].

The 8086, introduced in 1978, brought this combination of techniques to a mass-market microprocessor. It used a control ROM and a microinstruction sequencer to implement much of its irregular instruction set. Intel's user manuals specified the public contract through opcodes, registers, flags and bus cycles. The internal sequence was hidden so that future processors could change it without invalidating binaries [103, 104, 106].

The 1980s RISC movement revived hardwired control for a different reason. Projects such as Berkeley RISC-I, first described in 1981, and the Stanford MIPS work favoured regular instruction formats, a load/store organisation and simple operations that could be decoded quickly and pipelined. The slogan "RISC versus CISC" is easily overstated. Microprogramming is a control technique; RISC is an architectural design strategy; pipelining is a method of overlap. A RISC core may contain firmware-like sequencers for rare events, and a CISC core may hardwire common operations [49, 50, 28].

The Intel Pentium Pro, introduced in 1995, made this combination explicit in mainstream x86. Its front end translated x86 instructions into simpler internal micro-operations, which were renamed, scheduled and retired by an out-of-order engine. Current Intel and AMD processors still use fast decoders for common instructions, micro-op caches on many designs and sequenced microcode for complex or exceptional cases. An instruction such as a simple integer add may become one micro-op; a string operation or system instruction may invoke a longer sequence. The architectural manuals specify the result and exceptions that each implementation must preserve [106, 26, 27, 28, 29].

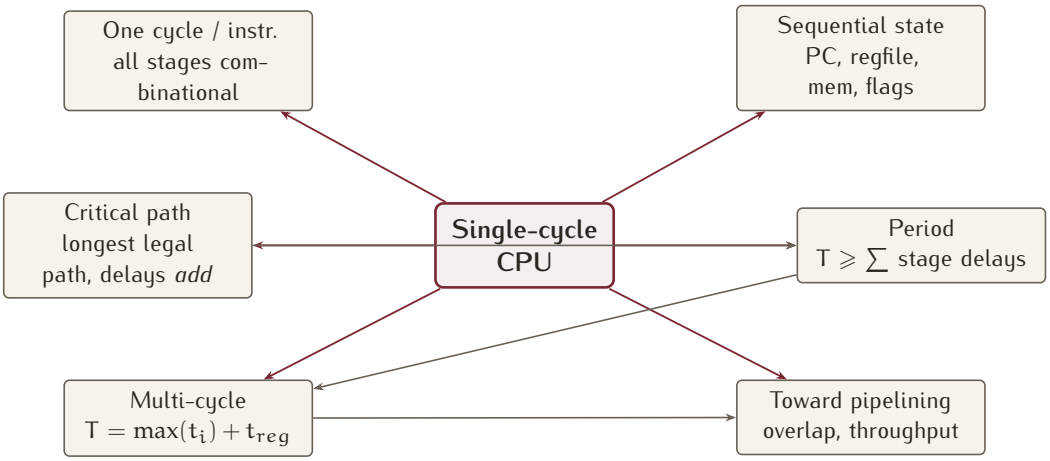
Microcode also became a field-update mechanism. Vendors can deliver signed updates at boot to correct documented errata or change selected behaviours. The updates distributed after the 2018 disclosure of Spectre and related transient-execution problems made this role unusually visible: new control bits and modified prediction or speculation behaviour were exposed to operating systems. Such an update is powerful but bounded. It can alter sequences and control decisions designed to be patchable; it cannot add arbitrary new datapaths, caches or physical wires [49, 50, 28].

The single-cycle processor provides an analytical baseline for separating instruction semantics from implementation timing. It completes the visible effect of every instruction in one clock period, so the period must accommodate the longest combinational path. A multicycle design inserts state boundaries and reuses units. A pipelined design overlaps steps from different instructions.

A modern out-of-order core translates, queues and speculates while retiring in architectural order. These designs address the control-sequencing problem that Wilkes approached through microprogramming: finite hardware must produce the required internal actions for each instruction. The documentary path runs from the 1951 paper and EDSAC 2 through the 1964 System/360 manuals to present Intel and AMD architecture and optimisation manuals [15, 93, 109, 26, 27].

A sequential reference machine remains useful as the architectural specification against which more aggressive implementations are checked.

Concept map: single-cycle CPU



Reading the map. A single-cycle CPU performs every stage combinationally within one clock period and retains only a small sequential state. The delays of the slowest instruction accumulate along the critical path, so the period is long. Dividing the work across cycles shortens the individual steps; adding pipeline registers also permits overlap and raises throughput, which motivates Chapters 3.5 and 3.6.

3.4.6 Worked problems

Stack datapath: push and return paths

For a push with RSP = `0x1000` and value `0xAA`, Decode supplies the old stack pointer and the data. Execute computes `0x1000` − 8 = `0x0FF8`. Memory writes the qword `0xAA` at `0x0FF8`, and Write-back sets RSP = `0x0FF8`.
Now consider a separate return with RSP = `0x0FF8` and the qword there equal to `0x400120`. Memory reads that old stack location while the ALU computes `0x0FF8` + 8 = `0x1000`. The resulting state is PC = `0x400120` and RSP = `0x1000`. Memory and ALU results reach different destinations.

Critical path and slack

Candidate combinational path delays are 600, 820 and 710 ps. Their maximum is 820 ps, so a period below 820 ps cannot satisfy this model even before register and clock overhead. With a 900 ps period and no further overhead, the longest path has 900 − 820 = 80 ps of slack. If an additional 40 ps allowance is required, the slack becomes 40 ps. Improving the 600 ps path alone leaves the limit unchanged; reducing the longest path to 760 ps makes the minimum period 760 + 40 = 800 ps under that allowance.

Exercise 3.4.1 — Page cross

As an address-range calculation for a system with 4 KiB pages, does an 8-byte access at 0x1FFD cross a page boundary? The Y86-64 model itself has no paging unit.

Worked solution

An 8-byte access includes its first byte and the following seven bytes. Its interval is `0x1FFD` through $\texttt{0x1FFD} + 7 = \texttt{0x2004}$. A 4 KiB page ends at `0x1FFF`; the next begins at `0x2000`. The access therefore touches three bytes in the first page and five in the second. The page numbers of the endpoints are 1 and 2, proving the crossing.

Exercise 3.4.2 — Stack address

Where does pop read?

Worked solution

A pop reads the qword at the old stack pointer, then advances the stack pointer by eight bytes in the ordinary case. With old `rsp=0x1FF8`, it reads bytes `0x1FF8` through `0x1FFF` and the resulting stack top is `0x2000`. Reading from the incremented pointer would skip the actual top item. For Y86-64 popq with destination rsp, the loaded value has priority over the separately computed increment, so the final RSP is the popped qword.

Exercise 3.4.3 — Clock

Use sum or maximum of candidate instruction paths?

Worked solution

Choose the maximum of the candidate instruction-path delays. In a single-cycle design every instruction receives one clock period, so that period must accommodate the slowest required path. If candidate delays are 300, 470 and 390 ps, the period must be at least 470 ps, plus any overhead not already included. Summing them would incorrectly assume one instruction traverses all mutually exclusive paths sequentially.

Exercise 3.4.4 — Reset

What architectural state must be initialised?

Worked solution

Reset must establish a defined starting PC and every architectural/control state required by the processor specification, such as status and condition codes. The fetch unit must know where execution begins, and controls must prevent invalid memory writes before valid instructions arrive. Registers whose initial values are unspecified may remain unspecified if the program initializes them before use. The correct reset set follows the declared architecture and the implementation's valid-state protocol.

Summary

SEQ gives a reference implementation against which later designs can be judged. Every control signal should be justified by an ISA obligation. Every state update should occur exactly once. Every illegal instruction or memory condition should produce the specified status. This clarity will be invaluable when several instructions overlap.

3.4.7 Exercises

Exercise 3.4.5 — ALU inputs

For `rmmovq rA,D(rB)`, identify `aluA`, `aluB` and the ALU function.

Go to the solution →

Exercise 3.4.6 — Stack update

For `pushq rA`, what are `aluA`, `aluB`, `dstE`, memory address and memory data?

Go to the solution →

Exercise 3.4.7 — Pop's two destinations

Explain why `popq %rax` uses both `dstE` and `dstM`.

Go to the solution →

Exercise 3.4.8 — Timing estimate

Instruction memory 160 ps, register read 70 ps, ALU 100 ps, data memory 190 ps, mux/control 50 ps and destination setup 20 ps. Include launch clock-to-Q in the instruction-memory budget and assume zero additional clock uncertainty. Estimate the load path and maximum frequency.

Go to the solution →

Exercise 3.4.9 — Suppressing a write

How can SEQ implement a failed conditional move without adding a special write-enable port to the register file?

Go to the solution →

Exercise 3.4.10 — Exception ordering

A load encounters an invalid address but its destination register is otherwise selected. Which control must change?

Go to the solution →

Exercise 3.4.11 — Derive `mem_addr`

State the memory-address source for `rmmovq`, `mrmovq`, `call`, `pushq`, `ret` and `popq`.

Go to the solution →

Exercise 3.4.12 — Multi-cycle comparison

A single-cycle load path is 700 ps. A five-phase multi-cycle design has a 190 ps clock and needs five cycles for a load, two for a move. Compare individual load latency and possible benefit.

Go to the solution →

3.4.8 Solutions

The Sequential Processor

Solution 3.4.5

For the store `rmmovq rA,D(rB)`, decode reads the source data into `valA` and the base address into `valB`; fetch supplies displacement `valC`. The ALU takes `aluA=valC`, `aluB=valB` and addition, producing `valE=valB+valC`. Memory then writes `valA` at address `valE`. For R[rB] = `0x3000` and D = 16, the address is `0x3010`, independently of the value being stored.

← Return to the exercise

Solution 3.4.6

A push allocates eight bytes before writing. Decode supplies `valB=R[rsp]` and `valA=R[rA]`. Execute selects `aluA=-8`, `aluB=valB`, so `valE=R[rsp]-8`. Write-back uses `dstE=rsp`. Memory writes `valA` at the new address `valE`. For an old stack pointer `0x2000`, both the write address and new stack pointer are `0x1FF8`. This also handles `pushq rsp`: the data is the old register value read during decode.

← Return to the exercise

Solution 3.4.7

Decode reads the old RSP as both valA and valB. Memory uses valA to read the top qword into valM, while the ALU computes $valE = valB + 8$. The two results have different destinations: `dstE=rsp` receives valE, and `dstM=rax` receives valM. For old RSP `0x800` and memory value 42, the instruction ends with RSP `0x808` and RAX 42. A single destination would lose either the stack update or the loaded value.

← Return to the exercise

Solution 3.4.8

The load traverses instruction memory, register read, address calculation, data memory, mux/control and destination setup. Adding the stated serial delays gives $T = 160 + 70 + 100 + 190 + 50 + 20 = 590$ ps. In seconds this is $590 \cdot 10^{-12}$, so the corresponding frequency limit is $1/T \approx 1.695 \cdot 10^{9}$ Hz, or 1.69 GHz. This is the load-path estimate; another instruction path would set the overall clock if its delay were larger.

← Return to the exercise

Solution 3.4.9

Make the E destination depend on the condition: `dstE = Cnd ? rB : RNONE`. When Cnd is false, the register file recognises RNONE and performs no write on that port. The ALU may still compute the source value; the absent destination prevents it from changing state. For true Cnd, rB is selected and receives valE as usual. Conditional moves have no M-port destination and preserve the condition codes, so this one destination selection suppresses their only register effect.

← Return to the exercise

Solution 3.4.10

The failed memory access produces status ADR. Before the state-update edge, the load's M-port write must be disabled, either by replacing dstM with RNONE or by qualifying its write enable with a successful-access signal. For example, loading into RAX must leave the old RAX intact when the memory unit reports an error, regardless of the bits present on valM. An ordinary mrmovq has no memory write or condition-code update; the general commitment rule also disables any such effects for instructions that request them. Execution stops with the fault status.

← Return to the exercise

Solution 3.4.11

For `rmmovq` and `mrmovq`, memory uses $\mathrm{valE} = \mathrm{valB} + \mathrm{valC}$, the base-plus-displacement address. For `call` and `pushq`, it uses $\mathrm{valE} = \mathrm{RSP}_{\mathrm{old}} - 8$, the newly allocated stack slot. For `ret` and `popq`, it uses $\mathrm{valA} = \mathrm{RSP}_{\mathrm{old}}$, the existing top of the stack. Their ALU computes $\mathrm{RSP}_{\mathrm{old}} + 8$ in parallel, but using that incremented value as the memory address would skip the qword to be consumed. Thus the memory-address mux selects valE for the first four classes and valA for the last two.

← Return to the exercise

Solution 3.4.12

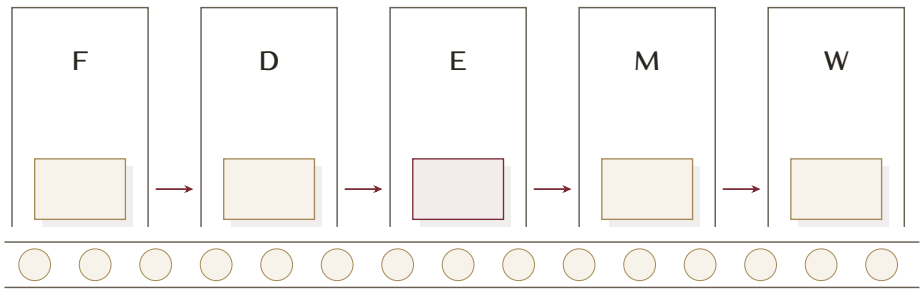
The load latency becomes $5 \cdot 190 = 950$ ps, worse than 700 ps. A move takes $2 \cdot 190 = 380$ ps instead of 700 ps. For a mix with load fraction p and move fraction $1 - p$, the multi-cycle average is $950p + 380(1 - p) = 380 + 570p$ ps. It beats 700 ps when $p < 320/570 \approx 0.561$. This calculation assumes only those two instruction classes and excludes other stalls; hardware reuse may provide an additional area benefit.

← Return to the exercise

3.5 PIPELINING: OVERLAP, THROUGHPUT, AND HAZARDS

An addition has reached the arithmetic unit. Behind it, another instruction is being decoded, while a third is fetched from memory. These activities use different parts of the processor and can overlap. The pipeline's task is to keep each instruction's operands, control signals and eventual effects together as they advance.

Several instructions can be in progress at once



Separate instructions occupy successive stages at the same time.

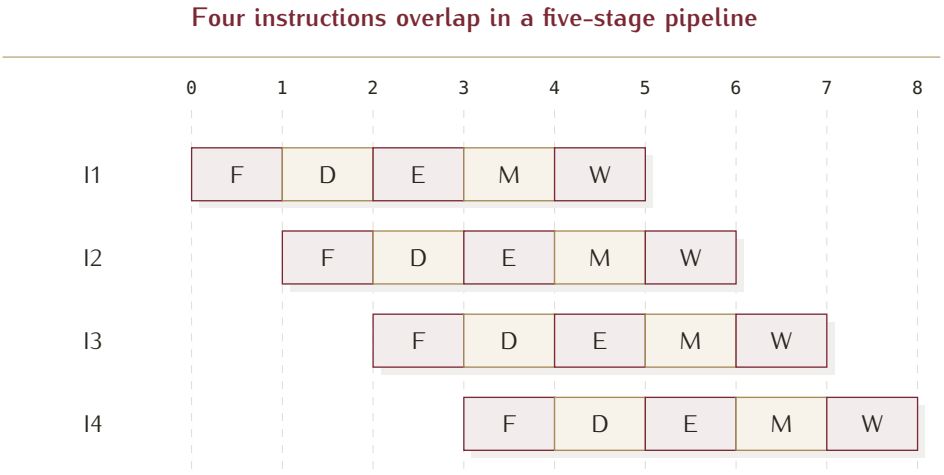
Figure 3.37: *The conveyor is an analogy for staged overlap. Separate instructions occupy different stages simultaneously, while one instruction moves through successive stages. Actual pipeline control can hold, bypass or discard work when dependencies require it.*

Now let the second instruction need the addition's result. The useful overlap has acquired a deadline: the result must arrive before the consumer uses it. Following that value from its producer to its consumer explains forwarding, waiting and bubbles. The same method will extend to unresolved branches and resources requested by two instructions at once.

3.5.1 Overlap changes throughput before it changes latency

The single-cycle datapath of Chapter 3.4 places the complete instruction path inside one clock period, so its clock must be long enough for the slowest instruction. Dividing the path into stages permits a shorter period, but a merely multi-cycle design still lets instructions use those stages largely in sequence. Pipelining goes further: while one instruction is executing an ALU operation, a younger instruction may be decoded and another fetched. Pipeline registers preserve the intermediate values and control information at each boundary so that these simultaneous activities continue to belong to different dynamic instructions.

A three-stage production line illustrates the distinction between *latency* and *throughput*. If stages A, B and C each require five minutes and one item must traverse the entire line before the next begins, each item takes fifteen minutes and the line completes only twelve items in three hours. If the stages overlap, the first item still needs fifteen minutes to emerge, but after the line fills a new item can complete every five minutes. With overlap, the operation still traverses all three stages, while the line completes a stream of operations at three times its previous steady-state rate.



No stalls: four instructions complete in eight clock periods.

Figure 3.38: *Each row follows one instruction while each vertical slice shows simultaneous work in different stages. With five stages and four instructions, ideal completion takes 5 + 4 - 1 = 8 cycles.*

That distinction matters whenever performance is reported. A five-stage pipeline may approach one completed instruction per cycle in steady state even though an individual instruction occupies the machine for roughly five stage intervals. Pipeline-register overhead can even make its latency slightly worse than the corresponding unpipelined combinational path. A long instruction stream benefits from throughput; a single isolated instruction sees latency.

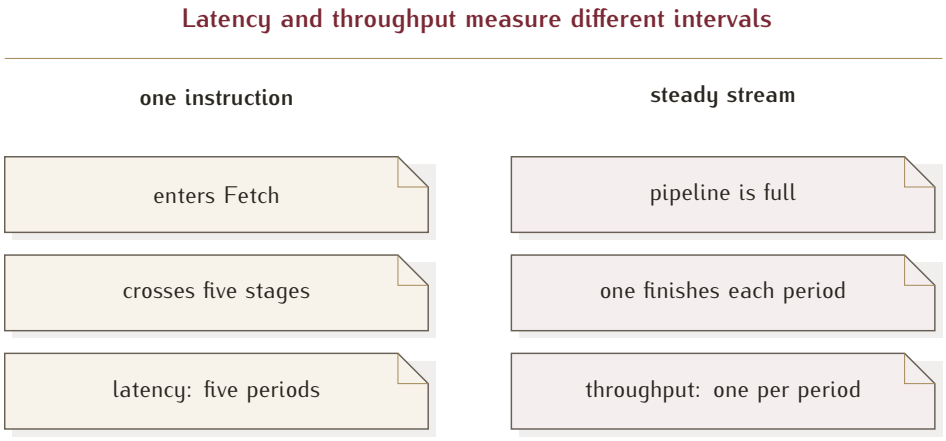


Figure 3.39: *Pipelining can improve the rate at which results complete while an individual instruction still traverses several stages. Fill and drain intervals reduce the average rate over a short instruction sequence.*

The clock period is determined by the slowest stage together with the cost of crossing a pipeline boundary:

$$T_{pipe} = \max(t_1, t_2, \ldots, t_n) + t_{reg}.$$

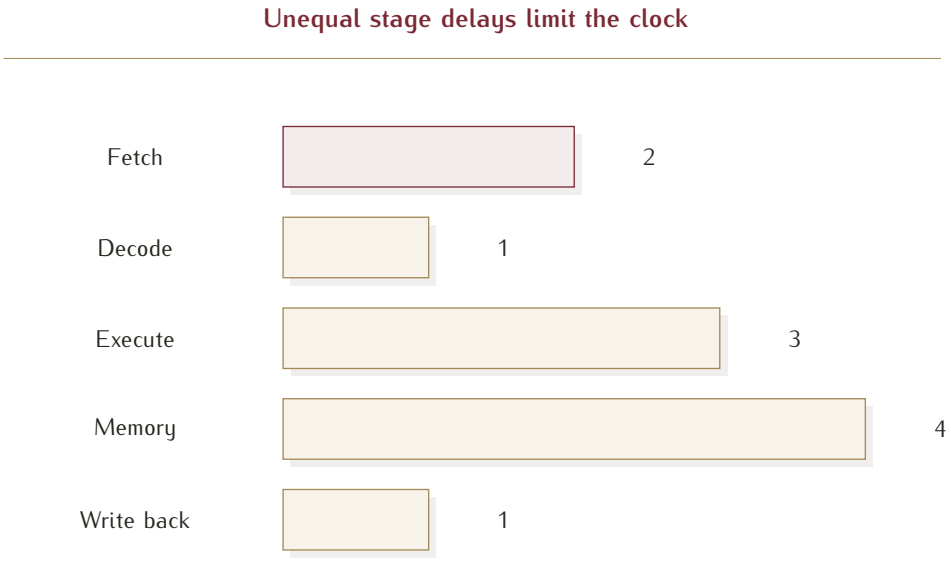


Figure 3.40: *The pipeline period must cover the slowest stage plus pipeline-register and timing overhead. Summing all stage delays describes a different quantity. Balancing stages can improve the achievable period.*

The maximum replaces the sum that constrained a single-cycle path, but the register term prevents arbitrary speedup through ever finer subdivision. Clock-to-output delay, setup time, skew allowances and control multiplexing all consume part of the cycle; if one stage remains much slower than the others, it still sets the period for the entire machine.

Stage balance and register overhead

Suppose three stages have logic delays 4 ns, 7 ns and 5 ns, and every pipeline boundary adds 1 ns. The clock period cannot be shorter than $7 + 1 = 8$ ns. One operation therefore has a latency of roughly three cycles, while a long independent stream can approach one result every 8 ns after filling. Splitting either shorter stage leaves the 7 ns bottleneck intact. Under the same overhead model the period remains 8 ns, while each operation traverses an additional stage and the clock drives another register bank.

A pipeline register has non-zero overhead

A register cleanly separates two processing stages. The boundary adds clock-to-output delay, setup and hold constraints, clock-tree load and the multiplexing needed for stalls,

bubbles or flushes. It must carry control and status information as well as data. Increasing pipeline depth cannot raise frequency indefinitely. As useful logic per stage shrinks, register and clock overhead consume a larger fraction of each cycle and the cost of recovering from control mistakes grows [50, 42].

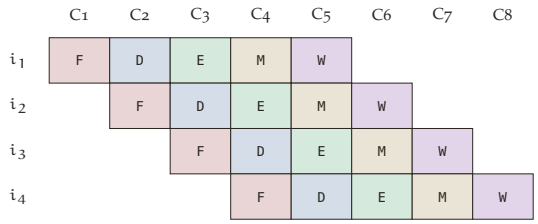
For N independent instructions in an ideal k-stage pipeline, the first instruction requires k cycles to emerge and each later instruction can complete one cycle after the previous one, so the total is

$$N + k - 1$$

cycles. The corresponding ideal speedup approaches k only as N becomes large and only if the stages are well balanced, pipeline-register overhead is small and no hazards introduce extra cycles. Any quoted speedup is therefore conditional on a machine and workload model rather than a permanent property of an ISA [24, 50].

3.5.2 Timing diagrams make overlap visible

A timing diagram places dynamic instructions on rows and clock cycles on columns, with each cell showing the stage occupied during that cycle. For the familiar five-stage teaching pipeline—Fetch, Decode, Execute, Memory and Write Back—instruction i enters Fetch at cycle i in the ideal case. At cycle c its stage number is consequently $c - i + 1$, provided that value lies between one and five.



At cycle 5, the first instruction is in Write Back, the second in Memory, the third in Execute, the fourth in Decode and a fifth instruction can be in Fetch. The diagram records when a value is produced, when another instruction needs it and whether the assumed overlap is legal. It must therefore track *dynamic* instructions. A source instruction inside a loop may appear many times in the diagram because each iteration creates a distinct dynamic execution.

Locating an instruction in an ideal pipeline

If instruction i enters Fetch at cycle i, then its stage at cycle c is $c - i + 1$. The x86-64 fragment below uses RDI for instruction index i and RSI for cycle c, both in $1 \ldots 2^{63} - 1$. It returns the stage code in RAX, or zero outside the pipeline window:

```
mov rax, rsi
sub rax, rdi
```

Worked example (continued)

```
inc rax              ; stage = cycle - i + 1
cmp rax, 1
jl out_of_range
cmp rax, 5
jg out_of_range
jmp done
out_of_range:
xor rax, rax
done:
```

The equation follows from the declared pipeline model. Once stalls or flushes occur, the relation between instruction number and cycle is no longer this regular, and the timing diagram must record the actual control decisions.

3.5.3 Hazards are failures of naive overlap

Overlap is useful only while it remains observationally equivalent to the sequential execution defined by the ISA. As soon as a younger instruction needs a value that an older one has not made available, Fetch needs a next PC that has not yet been resolved, or two stages require a resource that cannot serve them simultaneously, the naive schedule would expose internal timing to the program. These failures are called *data*, *control* and *structural* hazards. Designers combine forwarding, stalls, bubbles, prediction and resource duplication to handle these hazards. The chosen mechanisms must preserve the architectural contract while retaining useful overlap.

A Read-After-Write dependency provides the simplest example:

```
addq %rax, %rbx      # produces a new rbx
subq %rbx, %rcx      # consumes that rbx
```

If the consumer reads the register file before the producer has written its result, sequential semantics would be violated. One correct but inefficient response is to delay the consumer with bubbles or explicit `nop`s until the value is available. If the producer makes the value available at a declared boundary in cycle p and the consumer needs it at the corresponding boundary in cycle q, the required delay in that simplified model is

$$\max(0, p - q).$$

A forwarding implementation routes the newly produced value from the producing stage or a later pipeline register to the consumer, bypassing the register-file write and read. That removes many RAW stalls, although a classic load-use pair can still require a bubble because the loaded data emerges only after the immediately following instruction needs its Execute input. Chapter 3.6 turns these principles into concrete PIPE control equations.

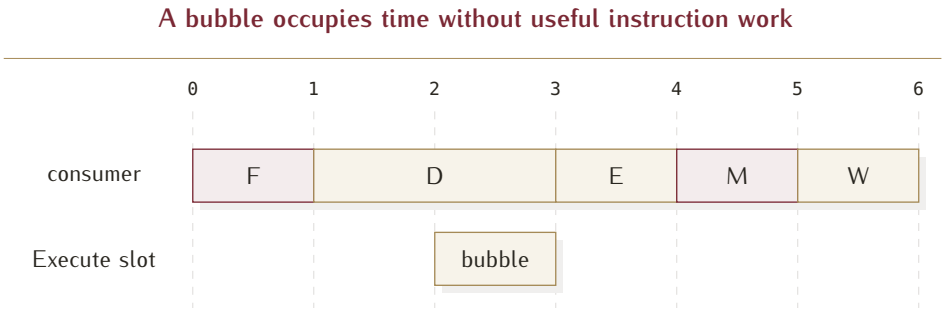


Figure 3.41: *Holding the consumer while injecting a bubble prevents it from using an unavailable operand. The bubble must move through the appropriate later stages without performing architectural writes.*

Figure 3.42 compares two producers while keeping the consumer and stage structure fixed. The one-cycle difference comes from where the value is produced.

Removing the wait with useful independent work

Consider the following Y86-64 sequence, with accessible memory and distinct destination registers:

```
mrmovq 0(%rdi), %rax
addq %rax, %rbx
irmovq $1, %rcx
```

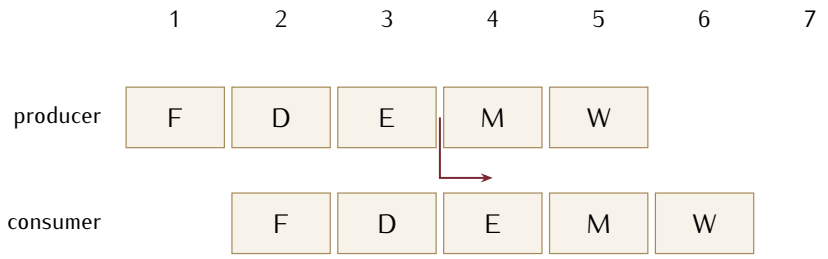
The add needs the loaded RAX immediately. Move the independent immediate assignment between the producer and consumer:

```
mrmovq 0(%rdi), %rax
irmovq $1, %rcx
addq %rax, %rbx
```

If the load is fetched in cycle 1, it reaches Memory in cycle 4. In the reordered sequence, the immediate assignment occupies Execute in cycle 4 and the add reaches Execute in cycle 5, when the loaded value can be forwarded. All three instructions complete by cycle 7 in this five-stage model; the original adjacent load-use pair needs one bubble and finishes in cycle 8.

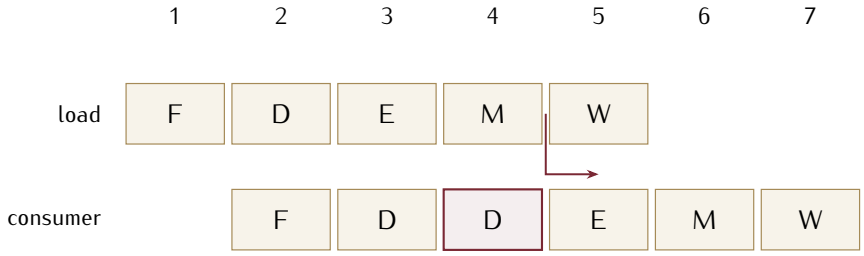
The RCX assignment preserves the other instructions' operands and condition codes and performs no memory access. The load remains first, so a load fault precedes both later instructions. The final register and flag values therefore agree. Longer memory latency or extra dependencies require a new schedule.

ALU producer: the value reaches the next Execute stage



Result produced at the end of cycle 3; consumed in Execute in cycle 4.

Load producer: one cycle of waiting makes the value available



The load completes Memory at the end of cycle 4. Execute waits until cycle 5.

Figure 3.42: *Cycle numbers describe a five-stage in-order model with operand forwarding into Execute and single-cycle memory. The repeated D cell holds the consumer; control inserts a bubble into Execute during cycle 4. This abstract schedule explains the dependency. Chapter 3.6 derives the Y86 PIPE control and forwarding paths that implement it.*

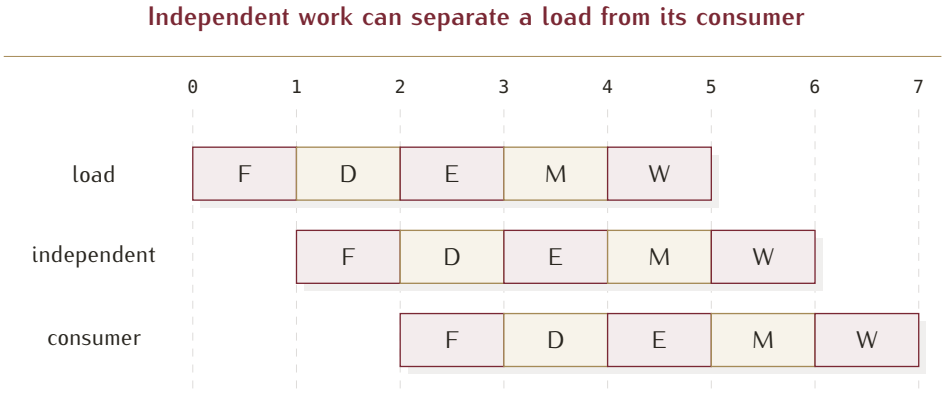


Figure 3.43: *With the stated five-stage timing and forwarding, the independent instruction supplies a useful separation. Such movement is valid only if it preserves dependencies, memory effects and required exception behaviour.*

Forwarding before architectural write-back

Suppose an ALU instruction computes its result at the end of Execute but does not update the architectural register file until Write Back. A dependent younger instruction need not necessarily wait for that architectural update: if a bypass path can deliver the Execute result directly to its input, the value is *available* before it is architecturally committed. A load is different in the classic five-stage pipeline because its data is obtained only after the Memory stage, which can be too late for the next instruction's Execute inputs. The production and consumption boundaries, together with the available bypass paths, determine whether forwarding suffices.

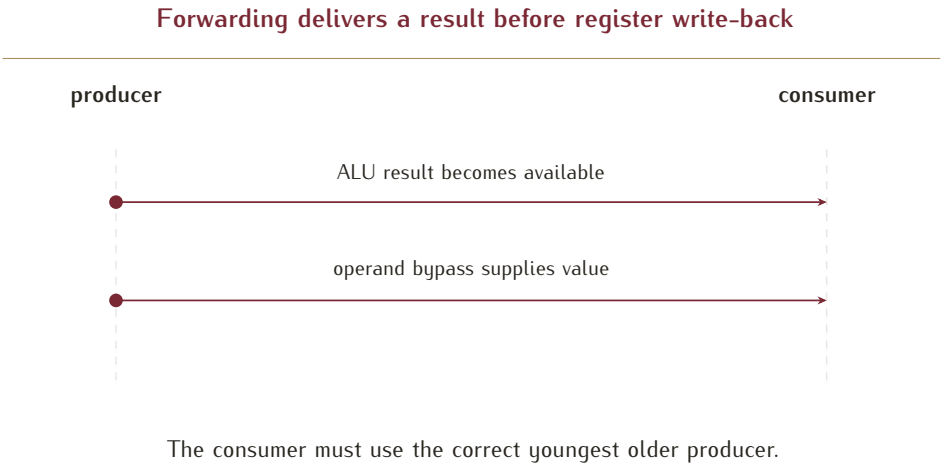


Figure 3.44: *The bypass route makes an already computed value available at the consumer's input. It does not erase the data dependence; it changes where the value is obtained and when it can be used.*

Software scheduling provides another way to respect dependencies. Where the ISA and compiler model permit it, software scheduling can move an independent instruction into a cycle that would otherwise be a bubble; early RISC designs exposed this responsibility more directly, whereas later implementations usually detect hazards in hardware. The compiler can exploit independence that is visible statically, but it cannot in general predict every run-time cache miss, branch outcome or dynamically changing dependency, so software scheduling and hardware control handle different parts of the timing problem.

A control dependency has the same underlying form but concerns the next instruction address. Fetch needs a PC every cycle, whereas a branch condition may not be known until Execute and a return target may arrive later still. The processor can wait, predict, or fetch speculatively and later discard wrong-path instructions; whichever policy it chooses, younger instructions that should not have executed must be prevented from changing architectural state. Feedback from later stages to earlier ones is therefore a recurring cost of pipelining, whether it carries a forwarded operand or a corrected PC.

Register dependencies have three distinct directions. In a RAW (read-after-write) dependency, an older instruction produces the value a younger one reads. In a WAR (write-after-read) anti-dependency, the older instruction must read the old value before a younger instruction overwrites its register. In a WAW (write-after-write) output dependency, two instructions write the same register, and the younger value must remain at the end. For example, `rrmovq %rax,%rbx; irmovq $7,%rax` has a WAR relation on RAX, while `irmovq $5,%rax; irmovq $7,%rax` has a WAW relation. The simple in-order pipeline reads operands in Decode and writes results in Write Back in program order, so its register timing preserves WAR and WAW automatically. RAW still needs

the availability analysis above. Designs that reorder reads or writes need additional mechanisms to preserve these relations.

Structural hazards arise when the schedule asks one physical resource to perform incompatible actions at the same time. A teaching pipeline with separate instruction and data memories avoids a conflict between Fetch and a load/store Memory stage; a design with one single-ported memory would need arbitration or a stall. The existence of a structural hazard thus depends on both the instruction sequence and the implemented resource organisation.

A *stall* prevents selected pipeline registers from advancing; a *bubble* injects a harmless state into a stage; *forwarding* supplies a value through a bypass path; and a *flush* invalidates younger work that must not become architecturally visible. The vocabulary names different control actions, all of which serve the same higher-level requirement: overlap may change timing, but not the program's defined result.

3.5.4 From ideal throughput to measured CPI

The ideal steady-state pipeline approaches one completed instruction per cycle, but real execution pays for every cycle in which useful completion is prevented. CPI therefore provides a convenient accounting model:

$$\mathrm{CPI} \approx \mathrm{CPI}_{base} + \text{data stall penalty} + \text{control penalty} + \text{structural penalty}.$$

This equation describes the teaching pipeline under the stated additive-penalty assumptions. Modern out-of-order machines can overlap several effects and require a model that accounts for that overlap. If a branch class occurs with frequency f_b, is mispredicted with probability p_m and costs P cycles when wrong, its average contribution is approximately $f_b p_m P$ cycles per instruction. Similar expected-cost reasoning applies to load-use stalls and other recurring hazards.

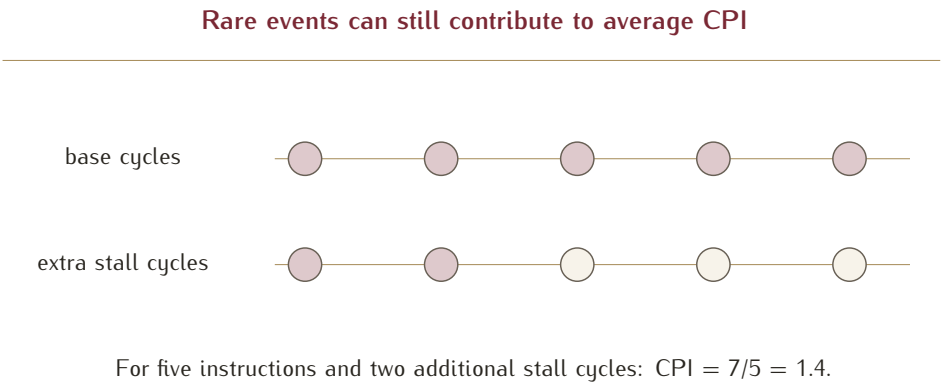


Figure 3.45: *The beads separate useful instruction-count cycles from additional lost cycles in this simplified steady-state accounting. A finite sequence that includes pipeline fill and drain needs those costs as well.*

Pipeline depth therefore represents a trade-off rather than a monotonic path to speed. More stages may reduce combinational delay per stage and raise clock frequency, yet they also increase clocked state, control complexity and the amount of speculative work lost after a branch mistake or exception. High-frequency designs of the early 2000s made this trade-off especially visible; contemporary processors balance pipeline depth with branch prediction, out-of-order execution, power limits and instruction-level parallelism [50].

The energy cost of another pipeline boundary

Each bank of pipeline registers adds nodes that must be charged and discharged, along with load on the clock network. A simplified switching-power model is

$$P_{sw} \approx \alpha C_{eff} V^2 f,$$

where α counts average charging events per clock cycle, C_{eff} is the switched capacitance, V the voltage swing and f the clock frequency. This component is part of the full power budget, which also includes leakage and other current paths [138].
Worked check. For hypothetical $\alpha = 0.2$, $C_{eff} = 100$ pF, $V = 1$ V and $f = 1$ GHz, switching power is 0.02 W, or 20 mW. Holding the other values fixed and raising voltage to 1.2 V gives $20 \cdot 1.2^2 = 28.8$ mW: a 44% increase. The numbers estimate switching power for the specified capacitive load.
Clock gating can reduce unnecessary events, subject to correct enable timing. Pipeline depth therefore has both a timing cost and an energy cost: shorter combinational paths can require more clocked state. The throughput model must be considered together with these physical costs [41, 50].

Laboratory 21: Pipeline timing

Four instructions traverse F,D,E,M,W. Register writes occur early enough for D to read them in the same cycle. Instruction I2 uses I1's result. Give timing with ALU forwarding and no stalls, with no forwarding for an ALU producer, and with forwarding for a load producer. I3 and I4 are independent.

Worked solution

Laboratory 21: solution

The rows below list the occupied stage cycles; repeated D means the instruction waits in decode.

Case	I1	I2	I3	I4
ALU forwarding	F1 D2 E3 M4 W5	F2 D3 E4 M5 W6	F3 D4 E5 M6 W7	F4 D5 E6 M7 W8
No forwarding	F1 D2 E3 M4 W5	F2 D3–5 E6 M7 W8	F3–5 D6 E7 M8 W9	F6 D7 E8 M9 W10
Load forwarding	F1 D2 E3 M4 W5	F2 D3–4 E5 M6 W7	F3–4 D5 E6 M7 W8	F5 D6 E7 M8 W9

The ideal sequence finishes in eight cycles. Without forwarding, I2 waits until I1 writes in cycle 5, so two bubbles extend completion to cycle 10. ALU forwarding supplies I1's E-stage result to I2 in time for E4, restoring the ideal eight cycles. A load produces its value after M4, so the consumer enters E5 after one bubble and completion is cycle 9. The same-cycle write/read convention is essential to the no-forwarding count.

Return to the laboratory

From instruction overlap to the disciplined pipeline

Timeline

Year	Machine, standard or event	Architectural significance
1961	**IBM Stretch**	Instruction overlap is used to raise throughput.
1964	**CDC 6600**	Functional-unit concurrency makes hazards an explicit scheduling problem.
1980s	**RISC pipelines**	Regular instruction formats expose stage boundaries and forwarding clearly.

From overlap to controlled overlap

Early high-performance machines such as Stretch and Atlas overlapped instruction-processing and memory activities in less regular ways, while the CDC 6600 used a scoreboard to prevent functional-unit concurrency from violating dependencies. Tomasulo's

1967 algorithm went further by using tags and reservation stations to allow execution around unavailable operands, foreshadowing mechanisms still recognisable in modern out-of-order cores [76, 94, 119, 120, 122].

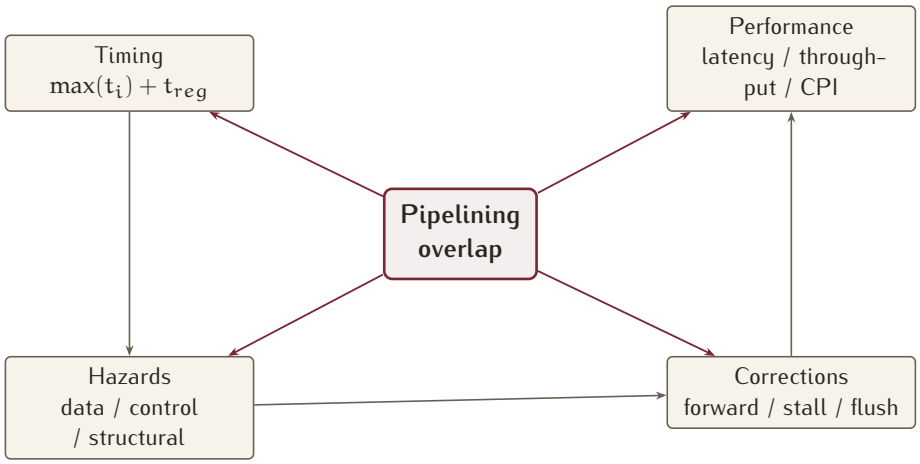
The regular pipelines associated with RISC research made the reasoning especially visible. Fixed-length instructions, load/store memory access and comparatively simple decoding allowed stage boundaries and hazards to be expressed cleanly, although commercial processors soon added interlocks and more sophisticated control. The familiar five-stage pipeline distils these dependencies into a regular teaching model; implementations partition their work in many other ways [53, 54, 49].

Architectural behaviour and implementation timing

Later x86 processors retained the same public ISA while changing their internal overlap radically, demonstrating why architectural manuals do not provide one permanent cycle count for an instruction. Timing belongs to a particular microarchitecture, dependency pattern and memory state; architectural correctness belongs to the ISA contract. The five-stage model remains valuable because it makes the proof obligations visible: where a value is produced, when it is needed, whether it can be forwarded, and how younger work is cancelled when control or exceptions invalidate it [106, 26, 50].

Modern cores use deeper pipelines, queues, prediction and out-of-order execution, but the central problem remains the same: overlap must not change the architectural result.

A compact map of pipeline reasoning



Timing permits overlap, hazard control preserves sequential semantics, and control penalties affect measured throughput. Chapter 3.6 now has a precise task: derive the forwarding paths and control equations for the Y86-64 PIPE datapath.

3.5.5 Worked problems

Finite batches and steady-state speedup

Twenty instructions in an empty five-stage pipeline complete in $20 + 5 - 1 = 24$ cycles. With a 240 ps clock this is $24 \cdot 240 = 5760$ ps. A single-cycle implementation with a 900 ps clock takes $20 \cdot 900 = 18000$ ps, so the finite-batch speedup is $18000/5760 = 3.125$. As the instruction count grows, startup becomes negligible and the ratio tends to $900/240 = 3.75$, assuming no stalls. The stage count alone determines neither ratio.

Dependency and availability

An older add produces RAX and the next instruction reads that result. This is a RAW dependency even if the value is already available. In the five-stage model an ALU bypass can supply the result to the consumer's Execute input without a stall; without that path, the consumer must wait for the register-file write. Changing the consumer to read an independently defined RBX removes this particular dependency. Availability and forwarding determine whether a dependency creates a timing hazard.

CPI with a finite instruction count

For 100 instructions and 15 additional, non-overlapping stall cycles, ignoring startup and drain gives $(100 + 15)/100 = 1.15$ CPI. In an initially empty five-stage pipeline, including the four extra completion cycles gives $(100 + 4 + 15)/100 = 1.19$ CPI. These are two accounting conventions for the same stated pipeline; a performance comparison must use the same convention on both sides.

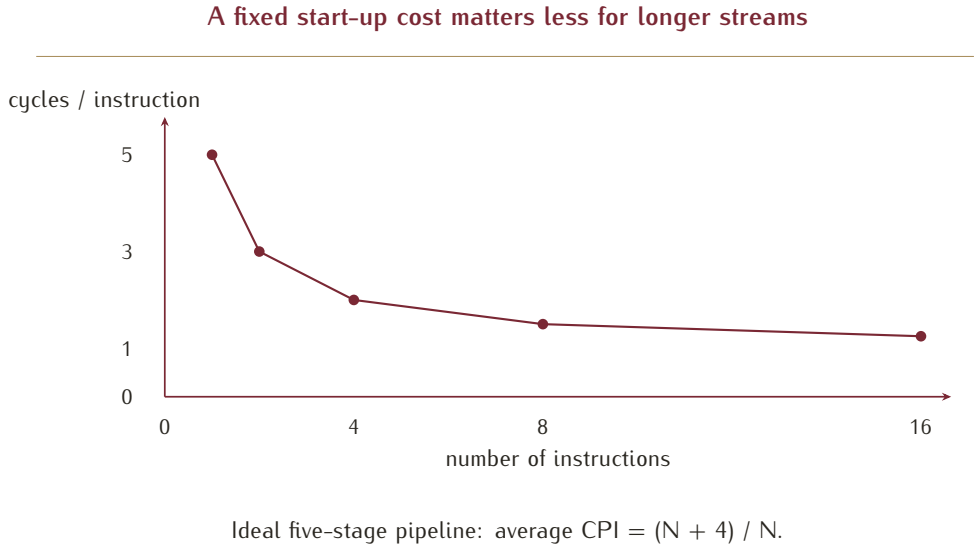


Figure 3.46: *The plotted averages include filling and draining the pipeline. As the stream grows, the four-cycle overhead is spread over more instructions. Hazards and variable-latency operations add further costs.*

Exercise 3.5.1 — Fill

How many fill cycles in a seven-stage pipeline?

Worked solution

For a seven-stage pipeline admitting one instruction per cycle, the first instruction reaches the final stage in cycle 7. Six earlier cycles pass without a completed instruction, so the startup overhead relative to one completion per admitted instruction is $7 - 1 = \boxed{6}$ cycles. For n instructions under ideal steady flow, total completion time is $n + 6$ cycles. This separates the seven-cycle latency of the first instruction from the six extra cycles in the batch formula.

Exercise 3.5.2 — Occupancy

Can more than one instruction occupy the same stage in a scalar pipeline cycle?

Worked solution

A scalar stage accepts at most one instruction in a cycle under the stated model. Different instructions can occupy different stages simultaneously. For example, in a five-stage pipeline, one instruction may be in W while others occupy M, E, D and

Worked solution (continued)

F. Two instructions needing the same unreplicated stage resource in the same cycle create a structural conflict and require scheduling, stalling or a wider design.

Exercise 3.5.3 — Latency

Does a shorter clock necessarily reduce instruction latency?

Worked solution

Instruction latency is the number of stages traversed multiplied by clock period, including any stalls. A five-stage pipeline with 200 ps cycles has ideal latency 1000 ps. A ten-stage design with 120 ps cycles has latency 1200 ps even though its clock is faster. Its ideal completion rate may improve once filled. Stage count, pipeline-register overhead and control penalties must therefore be included when comparing latency.

Exercise 3.5.4 — Dependency

Can a WAR anti-dependency cause an ordering hazard in the simple in-order five-stage pipeline?

Worked solution

A WAR anti-dependency requires an older read to precede a younger write of the same location. A WAR hazard would arise if that younger write happened first. In the simple in-order five-stage design, reads occur in the earlier decode stage and writes occur in the later write-back stage, preserving that order. RAW is the usual data hazard: a younger reader can arrive before an older producer's result is ready. Out-of-order reads or writes would require revisiting the WAR analysis.

Summary

Pipeline reasoning is a scheduling problem constrained by semantics. Draw instructions against cycles, mark when each value is produced and when it is needed, then decide whether forwarding, stalling or flushing preserves the sequential meaning. The diagram exposes each required ordering and makes the proposed overlap checkable.

3.5.6 Exercises

Exercise 3.5.5 — Pipeline completion time

A five-stage pipeline executes 12 independent instructions with no stalls. How many cycles are required from empty to empty?

Go to the solution →

Exercise 3.5.6 — Throughput

The pipeline clock is 250 ps. What steady-state throughput does one completed instruction per cycle represent?

Go to the solution →

Exercise 3.5.7 — Unbalanced stages

Stage delays are 90, 140, 110, 180 and 80 ps; register overhead is 20 ps. Find the clock period and the fraction of that period occupied by the 80 ps combinational delay. This fraction measures the share of the period allocated to combinational propagation. Switching activity would require a separate observation.

Go to the solution →

Exercise 3.5.8 — Classify hazards

Classify: (a) load followed by use of the loaded register; (b) Fetch and load sharing one memory port; (c) conditional jump before its condition is known.

Go to the solution →

Exercise 3.5.9 — Forwarding or stall?

In the classic five-stage model, with loads available only after M and arithmetic operands needed in E, state whether full forwarding avoids a stall for each adjacent pair: (a) `addq` then dependent `subq`; (b) `mrmovq` then dependent `addq`; (c) `irmovq` then dependent store address.

Go to the solution →

Exercise 3.5.10 — CPI estimate

Loads are 25% of instructions; 20% of loads cause one stall. Branches are 18%, 8% are mispredicted with a two-cycle penalty. Estimate CPI.

Go to the solution →

Exercise 3.5.11 — Software scheduling

A scheduler moves a store from before a load to the slot between that load and its consumer. Why can this be unsafe even when their address registers differ? Give a concrete aliasing example.

Go to the solution →

Exercise 3.5.12 — Pipeline split decision

A 220 ps stage is split into 105 and 115 ps stages. Register overhead is 25 ps. The original pipeline clock was 245 ps, limited by that stage, and every other logic stage is at most 115 ps. What is the new clock and asymptotic frequency improvement?

Go to the solution →

3.5.7 Solutions

Pipelining: Overlap, Throughput, and Hazards

Solution 3.5.5

The first instruction needs five cycles to traverse the five stages. Thereafter one instruction completes per cycle, so the remaining eleven instructions need eleven further cycles. Total time is $5 + 11 = 16$ cycles, equivalently $k + n - 1 = 5 + 12 - 1$. Instruction i enters in cycle i and completes in cycle $i + 4$; instruction 12 therefore completes in cycle 16. This includes both pipeline filling and draining.

← Return to the exercise

Solution 3.5.6

A completion every 250 ps means one completion per $250 \cdot 10^{-12}$ seconds. Throughput is therefore $1/(250 \cdot 10^{-12}) = 4 \cdot 10^{9}$ instructions/s, or 4 GIPS. This is the steady-state ideal rate with no stalls and one completion per cycle. An individual instruction still occupies several stages, so its latency is several clock periods. For a finite run, filling and draining reduce the average completion rate.

← Return to the exercise

Solution 3.5.7

The largest logic delay is 180 ps. Adding the 20 ps boundary overhead gives $T = 180 + 20 = 200$ ps. The 80 ps stage uses $80/200 = 0.40$, or 40%, of that period for combinational propagation. Under the common overhead model, its remaining budget consists of 20 ps overhead and $200 - 80 - 20 = 100$ ps slack. These are timing allowances; they do not imply that the circuit switches for exactly 40% of each cycle.

← Return to the exercise

Solution 3.5.8

(a) The consumer reads a register whose required value comes from an earlier load: this is a read-after-write data hazard. Forwarding or a stall must preserve that dependency. (b) Fetch and a data load need the same single memory port in the same cycle: this is a structural hazard, resolved by scheduling or additional resources. (c) The next instruction address depends on an unresolved branch: this is a control hazard, handled by waiting or prediction followed by recovery if necessary.

← Return to the exercise

Solution 3.5.9

Number the producer's Fetch as cycle 1. Without a stall, it reaches E3 and M4, while its adjacent consumer reaches E4.

(a) The add produces its ALU result by the end of E3. Forwarding delivers it to the subtract's E4 inputs, so the pair needs no stall.

(b) The load obtains its data only after M4, too late for the add's E4 computation. Hold the consumer for one cycle; it then executes in E5 using the forwarded load value.

(c) The immediate move produces its value in E3. Forwarding supplies that value as the store's base operand for address calculation in E4, so no stall is needed.

The decisive difference is the stage at which the producer supplies its result. All three pairs retain their data dependencies.

← Return to the exercise

Solution 3.5.10

Start with ideal CPI 1. Of all instructions, the fraction causing a load-use stall is $0.25 \cdot 0.20 = 0.05$, adding 0.05 cycles/instruction. Mispredicted branches form fraction $0.18 \cdot 0.08 = 0.0144$; each costs two cycles, adding 0.0288. With additive penalties, CPI $= 1 + 0.05 + 0.0288 = 1.0788$. For 10,000 instructions this corresponds to 500 load stalls and 288 branch-penalty cycles beyond 10,000 ideal cycles, excluding filling and draining.

← Return to the exercise

Solution 3.5.11

Suppose registers `p` and `q` both hold address A, initially containing 1. The original order `store 9,[p]; load r,[q]; use r` loads 9. After moving the store, `load r,[q]; store 9,[p]; use r` loads 1. Different register names did not imply different addresses. Reordering can also change the order of faults or observable accesses, so the scheduler needs an appropriate memory-dependence and exception model before moving the store.

← Return to the exercise

Solution 3.5.12

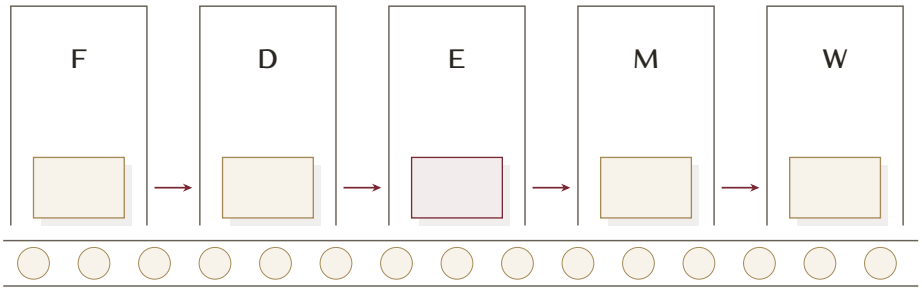
The new limiting substage is 115 ps, so the new period is 140 ps if no other stage is slower. Frequency improves by $245/140 = 1.75\times$, while the instruction traverses one additional stage. Its latency in seconds depends on both the new stage count and shorter period; control penalties can reduce the realised throughput gain.

← Return to the exercise

3.6 THE FIVE-STAGE PIPELINED PROCESSOR

At one clock edge, several instructions are waiting at different pipeline boundaries. A load is about to supply a value, a younger instruction may need that value, and an older instruction may already carry an exception. The control logic must decide what advances, what waits and what is cancelled. Tracing these decisions together turns the five-stage datapath into an executable machine with explicit priorities.

Every stage carries an instruction and its control



The staged view must be supplemented with stalls, forwarding and precise updates.

Figure 3.47: *The five stations represent Fetch, Decode, Execute, Memory and Write back. Their overlap is useful only when control preserves the sequential ISA behaviour. The chapter's timing figures show where waiting and redirection alter the ideal motion.*

3.6.1 From overlap to a controlled pipeline

Combinational and sequential logic, Y86-64 stage semantics, the single-cycle reference model and the principles of overlap now combine in one five-stage pipelined processor.

The five stages are Fetch, Decode, Execute, Memory and Write Back. The sequential model's PC update becomes a distributed decision: Fetch predicts an address, while later branch and return information can redirect it. Each stage has its combinational logic; between stages sits a *pipeline register*, a bank of flip-flops holding the intermediate

values from one cycle to the next. Without stalls or flushes, each instruction advances one stage per cycle. Pipeline fields preserve its operands, destinations and status; Fetch predicts a successor before older control transfers have finished.

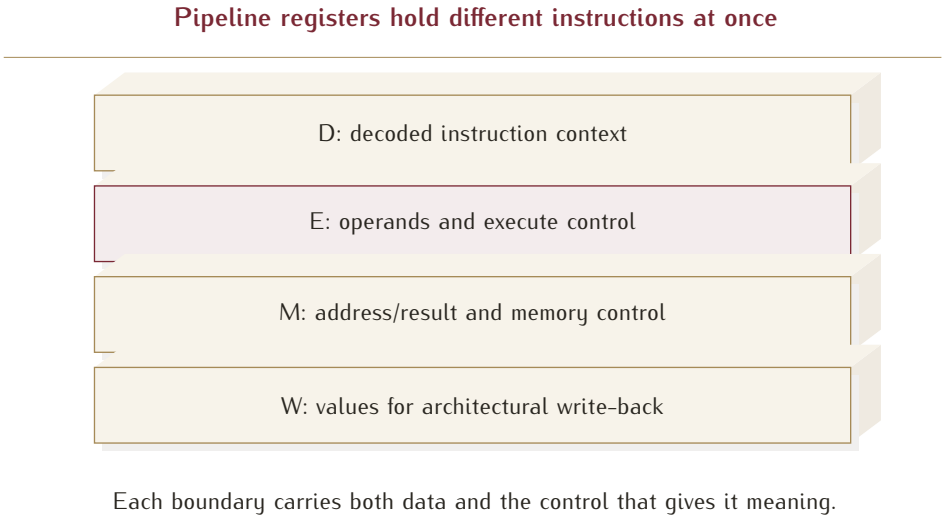
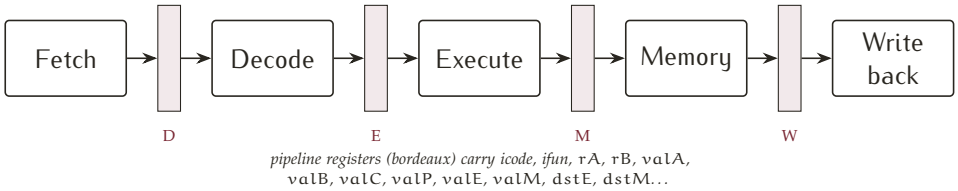


Figure 3.48: *The processor must keep control fields aligned with their instruction's data as both move between stages. A correct value paired with another instruction's destination identifier can still corrupt architectural state.*

Forwarding, stalling and cancellation now become concrete datapath controls. Their derivation requires tracking both the value carried by each field and the dynamic instruction that owns it.

3.6.2 Pipeline state and fetch control

1. **Fetch (F=1)**: read the instruction; decode icode/ifun, register IDs, and valC; compute the *predicted PC* for the next cycle.
2. **Decode (D=2)**: read valA, valB from the register file.
3. **Execute (E=3)**: the ALU produces valE and the condition codes; for jumps, evaluate Cnd.
4. **Memory (M=4)**: read/write data memory at the selected address: M_valE for loads, stores, pushes and calls; the old stack pointer in M_valA for pops and returns.
5. **Write back (W=5)**: write W_valE to W_dstE and/or W_valM to W_dstM, subject to status and RNONE controls.



Fields are named by the register they sit in: `D_valP` is *valP* between F and D; `E_valA` is *valA* between D and E; `M_valE` between E and M; `W_valM` between M and W.

Worked example

Problem. The low two bits of `r8` select which value to observe: 0 `D_valP` (`rdi`), 1 `E_valA` (`rsi`), 2 `M_valE` (`rdx`), 3 `W_valM` (`rcx`). Leave it in `rax`.

Solution. A 4-to-1 mux (same technique as 3.1):

```
mov r10, r8
and r10, 3
mov rax, rdi; default D_valP
cmp r10, 1
cmove rax, rsi; E_valA
cmp r10, 2
cmove rax, rdx; M_valE
cmp r10, 3
cmove rax, rcx; W_valM
```

The lowercase prefix in `e_valE` identifies a combinational Execute output. Uppercase prefixes identify stored pipeline-register fields. At an active edge it can become `M_valE`; at the following edge it can become `W_valE`. In one cycle these names can belong to three different instructions. A particular instruction's value moves through the fields as that instruction advances. Carrying a 64-bit result across boundaries costs storage and wiring even when a stage only passes it onward.

Forwarding is a global wiring problem

Forwarding selects a recent result instead of waiting for register write-back.

Result buses must reach multiplexers near many consumer inputs, and comparators decide which source tag matches. As the number of stages, issue slots and execution units grows, the number and length of possible bypass paths grow rapidly. Wide global buses add capacitance and can become critical paths.

A simple forwarding arrow in a pipeline diagram can represent one of the most difficult structures to scale physically. Designers cluster units or restrict forwarding to control wire cost and timing [50, 42].

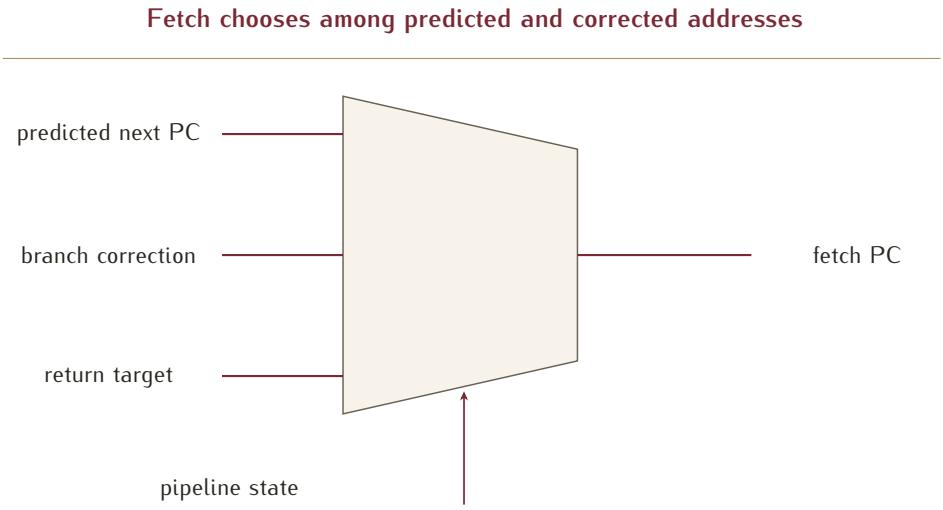


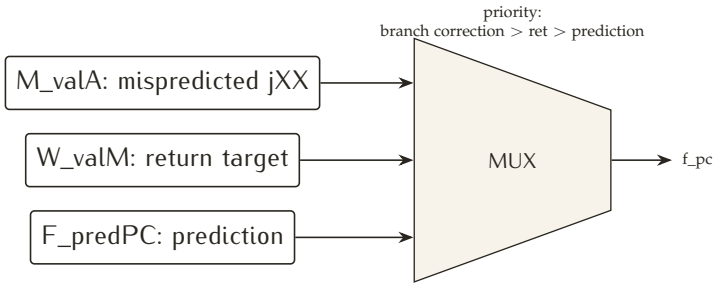
Figure 3.49: *A predicted address keeps fetching active before the final control-flow result is known. Later information can redirect Fetch. The choice and its priority are explicit parts of PIPE control.*

PIPE separates two related decisions. First, Fetch predicts the PC to store in `F_predPC`: for `jXX` and `call`, it predicts the target `valC`; for ordinary instructions it predicts the fall-through `valP`. A `ret` cannot obtain its target in Fetch because the return address is read from memory later.

At the start of a later cycle, the *current fetch PC* is selected from information that has matured in the pipeline. In this PIPE design the priority is:

1. if a conditional jump in the Memory stage was mispredicted, use `M_valA`, the saved fall-through address;
2. else if a `ret` in Write back has produced its return address, use `W_valM`;
3. otherwise use `F_predPC`.

Exception status travels through separate fields and suppresses architectural updates. This teaching processor stops on an exception; its PC mux implements branch correction, return and prediction.



Worked example

Problem. Express the PIPE fetch-PC choice and next prediction as HCL case expressions.

Solution.

```
word f_pc = [
 M_icode == IJXX && !M_Cnd: M_valA;
 W_icode == IRET: W_valM;
 1: F_predPC;
];
word f_predPC = [
 f_icode in {IJXX, ICALL}: f_valC;
 1: f_valP;
];
```

Prediction keeps Fetch supplied before a branch condition or return address is known. Correction redirects Fetch when later stages reveal the true path. Status propagation, stalling and bubbling ensure that a faulting instruction and younger instructions cannot commit inappropriate state.

3.6.3 Data hazards and forwarding

In the corresponding no-forwarding model, the producer writes the register file in Write back (5th cycle from its Fetch); the consumer reads in Decode (2nd cycle from its Fetch). For the consumer to read the new value, its Decode must not precede the producer's Write back. Assuming write-in-first-half / read-in-second-half, Decode in the *same* cycle as Write back suffices, so an otherwise adjacent producer and consumer require two inserted nops or two equivalent stall cycles.

Worked example

Problem. The producer emits in Write back at cycle `rdi`+4; the consumer would read in Decode at cycle `rsi`+1. The producer and consumer fetch indices satisfy

Worked example (continued)

$1 \leqslant \texttt{rdi} < \texttt{rsi} \leqslant 2^{63} - 1$ in an otherwise unstalled schedule. Compute how many cycles the consumer must be delayed.

Solution. The consumer must Decode at cycle $\geqslant \texttt{rdi} + 4$, versus the natural $\texttt{rsi} + 1$: if $\texttt{rsi} + 1 < \texttt{rdi} + 4$, need $\texttt{rdi} - \texttt{rsi} + 3$ nops, else 0.

```
mov rax, rdi
sub rax, rsi
add rax, 3; rdi - rsi + 3
xor rcx, rcx
test rax, rax
cmovs rax, rcx; if negative, 0
```

The forwarding network. With *forwarding*, a value from a later stage is bypassed to the consumer without waiting for Write back. In this PIPE model, forwarding selects the operands captured from Decode into E, including values computed during the current Execute or Memory stage. Other five-stage designs place their bypass muxes at Execute inputs. Each placement needs its own timing analysis; the equations below use Decode selection. Full forwarding resolves most RAW dependencies with no stall. The classic exception is the *load-use hazard*: an instruction using a value just loaded from memory. The load does not have it ready in Execute (memory is read in Memory, the next cycle), so the immediate consumer must stall one cycle, a residual RAW data hazard in the simplified five-stage PIPE model. Real processors can stall for many additional reasons, including cache/TLB misses, queue pressure, serialisation, resource conflicts and memory ordering.

Wrong-path work consumes real energy

Instructions after a mispredicted branch are flushed and have no architectural effect.

Before cancellation, they may have been fetched, decoded, renamed, issued and partially executed. Those transitions charge capacitances and occupy queues. They may also alter replacement, predictor or cache state even though register and memory results are not committed.

Cancellation suppresses architectural results while leaving some earlier physical activity intact. Better prediction avoids wasted electrical activity, and residual microarchitectural effects underpin transient-execution side channels [50, 28].

3.6.4 Control hazards, stalls, and bubbles

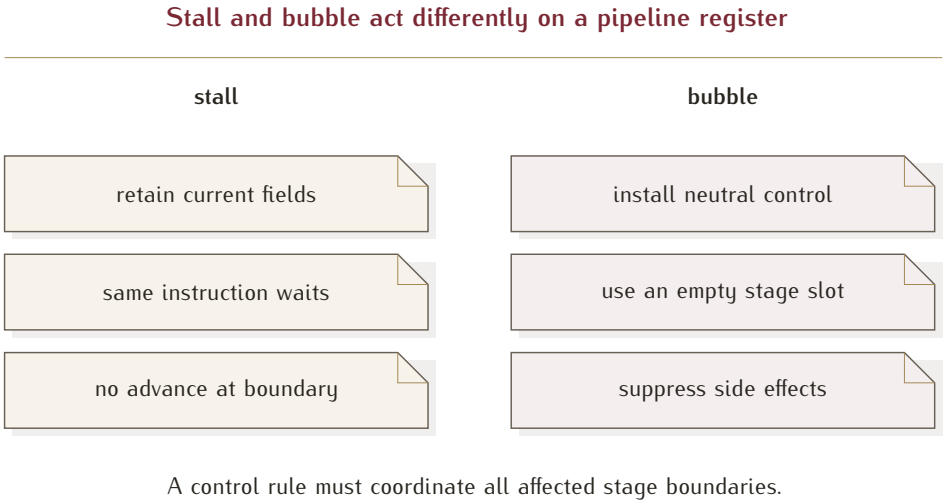


Figure 3.50: *A stall preserves the current instruction state at a boundary. A bubble inserts a harmless empty slot. Their combined use prevents duplication, loss or premature execution when an instruction must wait.*

Branch prediction chooses a PC before the branch condition is known. If the prediction is wrong, instructions fetched along the wrong path must not modify architectural state. The pipeline therefore inserts bubbles or squashes wrong-path instructions and redirects fetch to the correct PC.

> **Caution**
>
> Prediction and correction are different decisions. A predicted PC is chosen early to keep Fetch busy. The correction PC is chosen later when more information is available, such as the actual branch condition or the return address read from memory.

Correctness requires the same permitted architectural effects and stopping state as sequential ISA execution. Register, memory and condition-code updates must respect program order at every observable boundary, including a fault; a matching final answer alone would miss an invalid intermediate store.

A cache hit still has physical delay

A load produces its value at the end of the memory stage.

Translation lookup, cache indexing, tag comparison and data-array access are arranged to overlap where the implementation permits. Banks, bit lines, sensing circuits

and byte-selection logic still need finite time to resolve. If a translation or tag check fails, the early data-array activity cannot be used as the architectural result.

The load-use hazard follows from the data-production time and the chosen stage boundaries. Faster caches spend area and energy on parallel lookup, banking, prediction and short physical routes [47, 50].

Worked example

```
mrmovq 0(%rbx), %rax
addq %rax, %rcx
```

The `addq` needs `rax` as an ALU input. The `mrmovq` produces the value only after the memory read. A bubble gives the load time to reach the point where the value can be forwarded.

The PIPE hazard controls treat load-use cases, branch mispredictions and returns as the main sources of stalls and bubbles.

A return instruction is difficult because its target address is stored on the stack. Fetch cannot know the correct next PC until the return reaches the stage that reads memory. The pipeline must therefore prevent wrong-path instructions from committing and redirect Fetch once the return address is available.

Calls are easier than returns in this respect. The call target is the immediate value `valC`, available during Fetch. The return target is data-dependent because it is read from memory.

Caution

The pipeline control rules preserve correctness across overlapping instructions. A single missing stall or bubble can allow an instruction from the wrong path to update a register or memory location.

If an instruction raises an exception, older instructions are allowed to complete in program order, while the faulting instruction's normal state updates and all younger updates must be suppressed. Status values travel with instructions until the exception can be reported at a precise architectural boundary. This preserves the sequential illusion: the visible state corresponds to all instructions before the fault and none after it.

3.6.5 Hazard priority and precise state

At a clock edge, each pipeline register either captures the next instruction's fields, retains its current instruction, or receives an inert bubble. The forwarding and hazard rules determine which of these transitions preserves the instruction stream.

Decode needs the newest value for each source register. Several older instructions may name the same destination, so priority must follow program order: the nearest older producer wins. A conceptual selection for `d_valA` is:

1. if a call or jump needs `D_valP`, use it;
2. for a real source register, if `d_srcA` matches the effective destination `e_dstE` in Execute, use `e_valE`; a failed conditional move sets that destination to RNONE;
3. else match Memory's `M_dstM` and use `m_valM`;
4. else match Memory's `M_dstE` and use `M_valE`;
5. else match Write-back's `W_dstM` before `W_dstE`;
6. otherwise read the register file.

The order follows availability and age. Within a single pop instruction, the M result takes priority over the E result when both destinations are `rsp`. A stale Write-back value must never override a newer Execute value. Decode applies the same producer priority to `d_valB`, using `d_srcB`; only the special selection of D_valP for calls and jumps belongs exclusively to the A path. RNONE denotes an unused source and imposes no operand dependency.

Worked example: two consecutive writers

```
irmovq $1, %rax
irmovq $2, %rax
addq %rax, %rbx
```

When the add decodes, both older instructions may still be in the pipeline. The correct operand is 2 from the younger of the two producers. Forwarding priority is required for correct operand selection.

Forwarding priority follows program order

	candidate	age	example value
Execute	nearest older	youngest match	30
Memory	older	next match	20
Write back	oldest here	last match	10

The consumer needs the value from the youngest matching older producer.

Figure 3.51: *If several in-flight instructions write the same register, the consumer must see the most recent preceding write in program order. Forwarding selection therefore gives priority to the youngest matching producer.*

A load in Execute has destination `E_dstM`; its data will not appear until the end of Memory. If the instruction in Decode needs that register as `d_srcA` or `d_srcB`, the control logic must:

- stall F, preserving the current predicted PC;
- stall D, preserving the consumer instruction;
- bubble E, so no duplicate instruction advances.

After one cycle, the load value can be forwarded from Memory and the consumer proceeds.

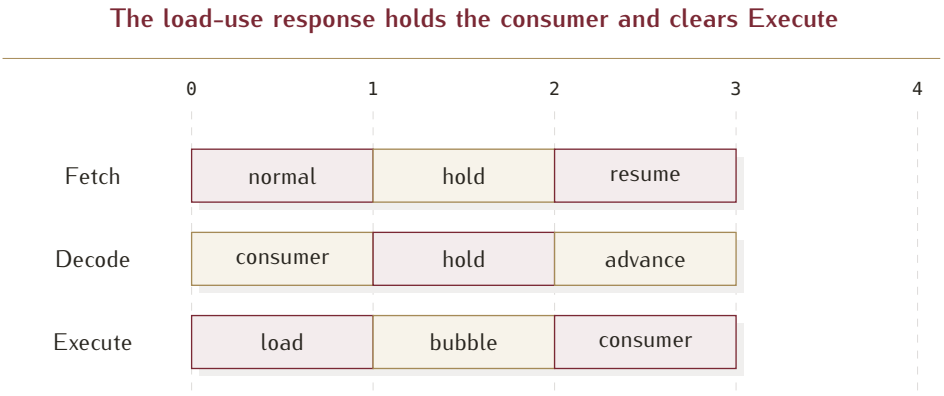


Figure 3.52: *The older load progresses so its value can become available. Fetch and Decode are held while Execute receives a bubble. The next enabled advance admits the consumer with its required operand.*

A conditional jump resolves in Execute. If the prediction was taken but `e_Cnd=0`, the instructions currently in Decode and Fetch belong to the wrong path. At the next edge the processor bubbles D and E and advances the branch into M. In that next cycle, the fetch-PC mux uses the saved fall-through address `M_valA`. Older instructions continue; the branch itself completes normally.

Worked example: why two bubbles are required

When a branch is in Execute, one younger instruction is in Decode and another is in Fetch. Both were selected using the predicted PC. Correcting only one would allow the other wrong-path instruction to advance and possibly commit. The number of squashed stages reflects how late the branch resolves.

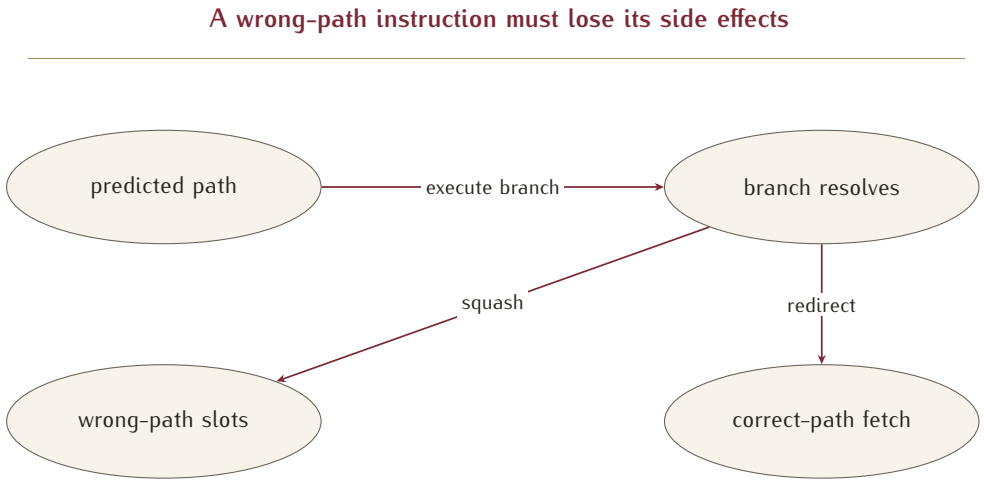


Figure 3.53: *The branch outcome distinguishes instructions that belong to the sequential execution from speculative work on the wrong path. The control logic redirects fetching and neutralises the younger invalid work.*

A `ret` obtains its target from data memory, so the correct next PC is unavailable until Memory. A simple control policy stalls Fetch while a return is in Decode, Execute or Memory and injects bubbles behind it. This costs cycles but preserves correctness. More advanced processors predict return addresses with a return-address stack.

Control conditions can occur together. The HCL equations therefore encode priority. For example, Fetch stalls for either load-use or return, while Decode stalls for load-use but is bubbled for a misprediction or a return when no load-use conflict dominates. Execute bubbles for load-use or misprediction. A contradictory request to both stall and bubble the same register must be resolved explicitly.

Event	F register	D register	E register
Load-use	stall	stall	bubble
Mispredicted branch	redirect normally	bubble	bubble
Return in D/E/M	stall	bubble unless load-use stalls	normal

Let L denote a load-use conflict, B a mispredicted branch in E, and R a return in D, E or M. Specifically, L requires an E-stage mrmovq or popq whose dstM matches a required Decode source; B requires a jump with false e_Cnd. The control equations are [147]:

$$F_{stall} = L \vee R, \qquad D_{stall} = L,$$
$$D_{bubble} = B \vee (\neg L \wedge R), \quad E_{bubble} = L \vee B.$$

The factor ¬L preserves a waiting return in D. Load-use and branch misprediction are mutually exclusive for one E-stage instruction, so D never receives both commands. F has no bubble input in this policy, and E never stalls.

An exception is precise when all older instructions have committed and no younger instruction has changed architectural state. PIPE achieves this by carrying status through the pipeline, suppressing side effects for exceptional instructions and preventing younger state updates as soon as an older exceptional status could invalidate those updates. Memory writes and condition-code updates require special gating because they occur before Write-back.

Let X mean that m_stat or W_stat is ADR, INS or HLT. Then $M_{bubble} = X$, and W stalls once W_stat is terminal; M never stalls and W never bubbles. A valid OPq updates flags only while ¬X. The M bubble removes younger work while the terminal instruction advances into W, where its status is retained [147].

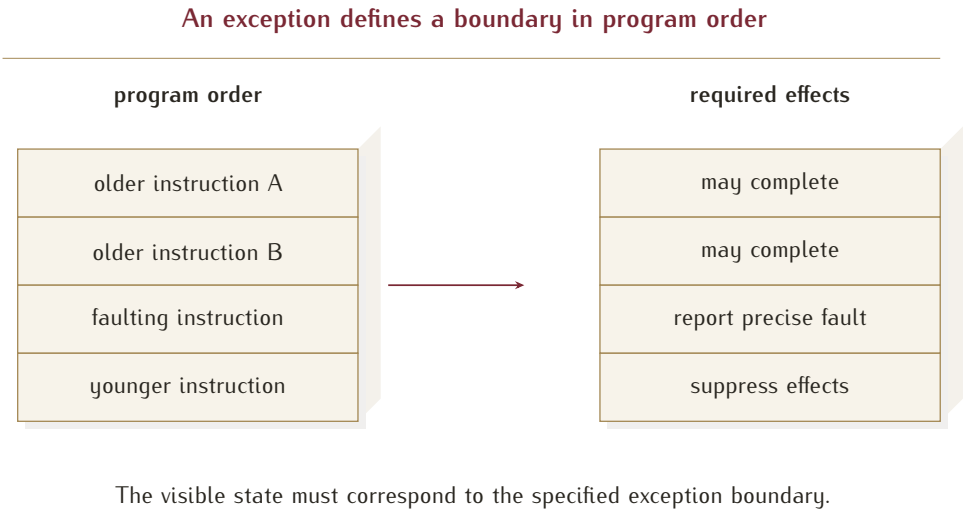


Figure 3.54: *Precise exception behaviour preserves the effects of older instructions while preventing forbidden effects from the faulting and younger instructions. The detailed rule for the faulting instruction belongs to the ISA and exception class.*

Worked example: younger arithmetic after a fault

Suppose a load generates `ADR` in Memory while a younger `addq` is in Execute. The add may have computed a result combinationally, but it must not update condition codes or later write a register after the fault is selected as the architectural outcome. The result can exist on an internal wire while its flag and register writes remain disabled.

3.6.6 Tracing and performance accounting

For each cycle, use one row with columns F, D, E, M and W. Annotate the instruction, its mnemonic and the stored fields needed by later stages. Then perform four steps:

1. start from the current uppercase register contents and evaluate the combinational blocks in dependency order;
2. resolve forwarding from the available Execute, Memory and Write-back outputs, then evaluate hazard conditions;
3. decide stall, bubble or normal load for every pipeline register;
4. write the next cycle's uppercase contents.

Never update columns left-to-right in place. All stages operate from the old state and update together at the edge.

A timing trace checks control as well as data

cycle	held state	inserted slot
t	hazard detected	consumer in D	load in E
t+1	F and D held	consumer still D	bubble in E
t+2	advance allowed	consumer leaves D	consumer in E

This table describes the same simplified load-use response as the timeline.

Figure 3.55: *Tracing both retained state and newly inserted state prevents a common bookkeeping error: counting a stalled instruction twice. Each dynamic instruction should advance, wait or be squashed according to one coordinated decision.*

A pipeline trace can be checked with

$$\begin{aligned}\text{cycles} = {} & N + 4 + \text{stall cycles} \\ & + \text{misprediction bubbles} + \text{return penalty} + \cdots\end{aligned}$$

for a five-stage pipeline starting and ending empty. Overlapping penalties require care: a priority rule may cause two apparent hazards to consume fewer cycles than their separate sums.

Pipeline control is a timing argument written as Boolean logic. Forwarding is legal only when the producing value already exists; a stall is required when it does not; a bubble removes work that must not affect architectural state. Exception status follows the same rule: an older fault must reach commitment before younger effects become visible.

Laboratory 22: Three pipeline hazards

Use the standard five-stage teaching PIPE with predict-taken conditional jumps, full forwarding and return fetch stalls. At L0: load into RBX; L1: add RBX to RCX; L2: jne T; L3: ret. The add yields zero, so jne is actually not taken. T begins a disposable wrong-path instruction. Trace the load-use stall, branch correction and ret. Memory is valid; the return target is R.
Worked solution

Laboratory 22: solution

The load follows F1,D2,E3,M4,W5. The add follows F2,D3–4,E5,M6,W7: one decode hold allows forwarding of the loaded value from M4. The branch follows F3–4,D5,E6,M7,W8. The add's zero result sets ZF=1, so jne is not taken. The predicted target is fetched in cycle 5; its younger work is squashed when the branch resolves, and corrected fall-through ret is fetched in cycle 7 from the M-stage correction path. Ret then follows D8,E9,M10,W11. Fetch is held while ret occupies D,E,M; the saved return address is available for fetching R in cycle 11. Thus load-use accounts for one bubble, misprediction for two lost fetch slots, and ret for three fetch holds in this model. Replacing the load by an immediate producer removes the one load-use hold and shifts later events one cycle earlier; the branch still yields zero if the replacement value is chosen to preserve that arithmetic result. Changing the producer without preserving its value would not guarantee the same branch outcome.
Return to the laboratory

3.6.7 Measuring and explaining performance

Performance analysis should connect a measured rate to a dependency structure. Identify the resource or dependency that limits useful work [1].

For a loop processing n elements, a simple empirical model is

$$T(n) = T_0 + n\,\mathrm{CPE},$$

where T_0 is fixed overhead and CPE is cycles per element over the range where the model fits. A fitted CPE is a measurement summary; it is not automatically the latency of one instruction.

latency. Minimum time from an input becoming available to the dependent result becoming available.

throughput. Sustainable completion rate when independent work is available.

critical path. Longest dependency chain that constrains completion.

resource bound. Limit imposed by issue width, execution ports, load/store bandwidth or another shared facility.

Loop unrolling reduces loop-control overhead but does not by itself shorten a recurrence. Multiple independent accumulators can expose instruction-level parallelism by replacing one long dependency chain with several shorter chains, provided the compiler and machine have enough registers and execution resources.

Worked example

Problem. A reduction performs one dependent floating-point addition per element. Assuming that changing the floating-point evaluation order is allowed, why may four-way unrolling with four accumulators improve throughput even though the total number of additions is almost unchanged?

Solution. Each accumulator carries only every fourth dependency. Independent additions can overlap, so the limiting rate can move from the add latency toward the execution unit's throughput or the memory subsystem's delivery rate. This reassociation can change rounding: for binary64 values $10^{16}, 1, -10^{16}, 1$ under round-to-nearest, left-to-right accumulation gives 1, while grouping $(10^{16} + (-10^{16})) + (1 + 1)$ gives 2. A compiler or programmer needs a numerical contract that permits the changed evaluation order; preserving the mathematical sum alone does not preserve every floating-point result.

Laboratory 23: A measured-rate model

A controlled benchmark produces the following repeated times in microseconds: n=1000: 21,20,22; n=2000: 40,41,42; n=4000: 80,82,81. Compute medians, ranges, and the slope/intercept fitted through the first and last medians. State what these synthetic data establish.

Worked solution

Laboratory 23: solution

Medians are 21,41,81 microseconds; each group has range 2 microseconds. Using the first and last points, slope is $(81 - 21)/(4000 - 1000) = 0.02$ microseconds per element, or 20 ns/element. Intercept is $21 - 0.02 \cdot 1000 = 1$ microsecond. The model predicts $1 + 0.02 \cdot 2000 = 41$, matching the middle median. This supplies a consistent linear region for the given data; it does not identify the bottleneck or report a measurement from the reader's machine. A real report records compiler/options, processor, clock policy, placement, cache state, input generation and an observable use of the result before attributing the slope to a particular instruction or resource.

Return to the laboratory

From a five-stage pipeline to rename, speculation and retirement

The five-stage processor explains overlap and hazards with a small number of named registers. Modern Intel and AMD cores extend the same problem with a wider front end, out-of-order scheduling and in-order retirement.

A useful conceptual sequence is: fetch and branch prediction; decode; register rename; dispatch into queues; issue when operands and execution units are ready; execute; complete; retire in program order. Loads additionally pass through address generation, translation and the cache hierarchy. Stores are normally prevented from becoming globally visible until they are safe to commit.

The fixed in-order PIPE schedule already preserves read-before-write and write-before-write ordering, so WAR and WAW name conflicts do not require stalls here. More flexible out-of-order scheduling can expose these conflicts. Register renaming can remove these *false* dependencies by assigning different physical destinations. It cannot remove a true read-after-write dependency: a consumer still needs the producer's value.

Execution may finish out of order, but architectural state must appear as though instructions completed in program order. The retirement mechanism commits completed instructions from the oldest forward. If the oldest instruction faults, younger speculative work is discarded, producing a precise architectural exception. This is the industrial counterpart of propagating status and suppressing younger writes in the teaching PIPE design [28, 29].

A predicted branch chooses speculative fetch addresses. A misprediction wastes work and requires recovery; forwarding shortens the wait for a required value. Correctness is defined by the ISA, while throughput depends on queue sizes, port availability, cache behaviour and prediction accuracy.

Hardware performance counters exposed through tools such as `perf` can estimate retired instructions, cycles, branch misses and cache events. "Retired" is the key word: it distinguishes committed architectural work from speculative internal activity. Counter names and exact meanings are model-specific and should be checked against the processor's performance-monitoring documentation.

Commit separates temporary electrical activity from public state

Precise execution exposes results in program order and discards younger work after a fault.

More advanced processors store speculative values in physical registers, queues and reorder structures. Completion can occur out of order, while retirement logic updates the architectural mapping in order. Recovery restores rename maps and invalidates younger entries, but the earlier switching activity cannot be undone.

The architectural state is a selected interpretation of a much larger physical state. Correctness requires controlling which internal results become public; security also asks what can be inferred from the internal effects that remain [50, 28].

From scoreboarding to speculative out-of-order retirement

Timeline

Year	Machine, standard or event	Architectural significance
1964	**CDC scoreboard**	Hardware tracks readiness and functional-unit conflicts dynamically.
1967	**Tomasulo algorithm**	Tags and reservation stations enable execution as operands become ready.
1995	**Pentium Pro**	Speculation and in-order retirement reach mainstream x86 processors.

A simple pipeline lets several instructions occupy different stages, but it still tends to move them in a fixed order. A slow instruction can stall independent work behind it and reduce throughput. Out-of-order execution separates three events that an elementary diagram merges: instructions are encountered in program order, execution begins when operands and resources are ready, and architectural results become visible in an order that preserves the sequential contract [50, 28].

The CDC 6600, first delivered in 1964, supplied an influential early mechanism. Its scoreboard tracked the availability of functional units, source operands and destination registers. An instruction could issue when structural and dependency conditions permitted, wait while an operand was produced and write its result when doing so would not violate a dependency. The scoreboard centralised the knowledge. It could tolerate different functional-unit latencies, but false dependencies associated with architectural register names still restricted parallelism [119, 120].

IBM's System/360 Model 91, announced in 1966 and operational in 1967, introduced a more distributed method for floating-point execution. Robert Tomasulo's 1967 paper described reservation stations that held operations near the functional units. If an operand was unavailable, the station stored a tag identifying the producing operation. The Common Data Bus broadcast a completed value together with its tag, waking every matching consumer. Register renaming emerged naturally: a younger write could receive a new tag rather than waiting for an older use of the same architectural register name [122].

Dynamic scheduling alone does not guarantee precise exceptions. Suppose a younger divide completes before an older load faults. If the divide writes architectural state immediately, the machine has exposed an effect from beyond the faulting instruction. Designers therefore developed history buffers, future files and reorder structures that allow execution to finish speculatively while delaying public commitment. At retirement, completed instructions update architectural state in program order. If the oldest instruction has faulted, younger results are discarded and the saved state corresponds to a sequential execution stopped at the fault [50, 28, 29].

This principle entered mass-market x86 with the Intel Pentium Pro, introduced in November 1995. Its front end decoded x86 instructions into simpler micro-operations,

renamed register results, issued ready operations to an out-of-order core and retired them in order. The implementation could execute around cache misses and long-latency operations without changing the x86 programming model. AMD's K5 and later Athlon designs followed different internal organisations but the same broad separation between architectural instructions and dynamically scheduled internal work [106, 28].

Register renaming removes false dependencies. A Read-After-Write edge is real: the consumer needs the producer's value. Write-After-Read and Write-After-Write conflicts arise only because several logical values reuse one architectural name. Mapping successive writes to different physical registers lets the machine execute them independently; the retirement map determines which physical register currently represents the architectural one. Flags can be renamed too, even when the ISA presents them as one status register. The rename map, free list and recovery checkpoints are therefore as important as the arithmetic units [50, 28, 29].

Branches create the control supply problem. A wide core cannot wait until every branch resolves before fetching later instructions, so it predicts direction and target and begins work on the predicted path. The reorder structure gives recovery: on a misprediction, younger operations and their speculative mappings are invalidated, and fetch restarts at the correct address. Better predictors increase useful work, but every additional speculative instruction consumes queue entries, physical registers and energy. Measure the benefit by the rate at which useful independent work reaches retirement [50, 28, 29].

Loads and stores create a harder ordering problem because their addresses may not be known at issue time. Store buffers let a retired or nearly retired store wait before it becomes globally visible. Load and store queues detect whether a younger load has bypassed an older store to the same address and trigger replay if a prediction was wrong. The cache-coherence protocol orders ownership among cores, while the ISA memory model defines which inter-thread outcomes software may observe. x86 is relatively strong, but it still permits Store-to-Load reordering for different addresses. Correct concurrent C and C++ programs require atomic operations and language memory orders rather than assumptions based on source order [26, 27, 40, 3].

AMD64 reached the market with Opteron in April 2003, followed by Athlon 64 on 23 September 2003. It enlarged the architectural register set and address space but did not expose the physical registers, queues or speculative state used internally. This is an important historical separation: a new ISA mode can change registers, encodings and exception rules while multiple microarchitectures implement those rules with different widths, queue sizes and predictors. The AMD64 and Intel 64 manuals define the sequential machine; optimisation manuals describe tendencies of particular implementations [107, 27, 26, 28, 29].

The security boundary of speculation became public in January 2018 with Spectre and Meltdown. Faulting or wrong-path instructions could leave measurable traces in caches and other microarchitectural structures despite their results being excluded from

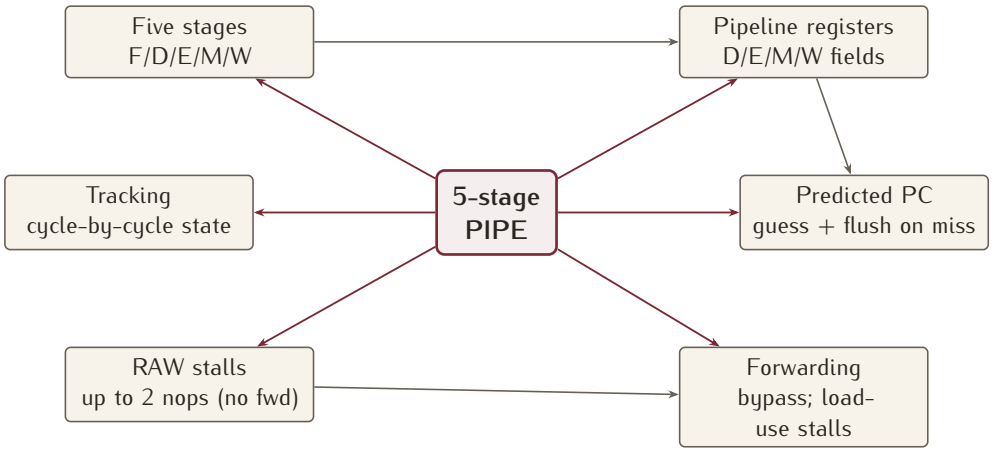
architectural commitment. Those traces could disclose data through timing. Mitigations introduced barriers, predictor controls, kernel page-table isolation and microcode facilities, often at measurable cost. Secure design now distinguishes architectural state, speculative microarchitectural state and the information that timing can disclose [50, 28, 29].

Current Intel and AMD cores combine several decoders or micro-op caches, register renaming, central or distributed schedulers, multiple integer and vector execution ports, load/store queues, multi-level caches and in-order retirement. Simultaneous multithreading lets architectural threads share many of those resources. Power, temperature and firmware policy can vary effective frequency and resource availability. As a result, there is no context-free answer to "how many cycles does this instruction take": dependency latency, reciprocal throughput, port pressure, cache state and speculation all matter [28, 29].

The five-stage pipeline supplies several concepts that reappear in out-of-order execution. Pipeline registers become queues and reservation stations; fixed forwarding paths become tag-based wakeup and bypass networks; a single register file becomes architectural-to-physical mapping; a flush becomes checkpoint recovery; write-back is separated from retirement. The historical path (CDC 6600 in 1964, System/360 Model 91 and Tomasulo in 1966–1967, Pentium Pro in 1995, AMD64 in 2003 and the transient-execution crisis of 2018) names the mechanisms that converted instruction overlap into the speculative cores used today [119, 122, 106, 107, 26, 27].

Modern out-of-order cores generalise the same separation of front end, execution, memory ordering and retirement introduced by the five-stage model.

Concept map: the five-stage pipeline



Reading the map. Five stages separated by pipeline registers carry every instruction; tracking must include stalls, bubbles and flushes. Fetch uses prediction, correction and return stalls; forwarding resolves dependencies when the producing value is available, and a load-use interlock handles the remaining one-cycle timing gap.

3.6.8 Worked problems

Independent register controls

Suppose D contains a consumer waiting for a load in E. With `D_stall=1`, all fields of D remain unchanged at the next edge. With `E_bubble=1`, the E register receives an inert entry while the load advances to M. Thus the consumer is preserved exactly once, and the load can produce the missing value. Stalling D says nothing by itself about E; the two control signals govern different storage boundaries.

Bubble

At the start of cycle c, an older instruction occupies M, a conditional branch occupies E and a younger wrong-path `irmovq` occupies D. Execute finds the taken prediction wrong. During this cycle, D and Fetch may still compute candidate values, but the next edge must cancel both younger instructions.
The older instruction advances from M to W. The branch advances from E to M with its saved fall-through address. E receives a bubble instead of the irmovq, and D receives a bubble instead of the newly fetched instruction. In cycle $c + 1$, Fetch selects the branch's saved fall-through from M_valA.
The reference bubble carries NOP control, RNONE destinations and SBUB status, so it produces no register, memory or flag update as it moves through the machine. The older instruction retains its legitimate effects, and the corrected stream starts behind the branch.

Tracing an immediate through pipeline state

An `irmovq` fetched in cycle 3 produces `f_valC`. At the next edge that constant becomes `D_valC`, used in Decode during cycle 4. The following edge copies it into `E_valC`, available to Execute in cycle 5. The reference design does not need a separate `d_valC` computation: the constant passes from D to E. With no stalls, the instruction then occupies M in cycle 6 and W in cycle 7. A stall changes the transition times while preserving ownership of the fields.

PC correction in the reference Y86-64 PIPE model

A conditional jump at `0x100` occupies nine bytes, so its fall-through address is $\texttt{0x100} + 9 = \texttt{0x109}$. Its target is `0x140`. The predict-taken policy fetches an instruction there and, assuming a two-byte target instruction, another at `0x142`.
When the branch reaches Execute, its condition is false. The required successor is therefore `0x109`. At the next edge, D and E receive bubbles to cancel the instructions at `0x142` and `0x140`, respectively; the branch advances to M carrying

Worked example (continued)

the saved `0x109` in M_valA. Fetch then selects that address through the branch-correction input. Both wrong-path instructions lose their state-changing controls before reaching any architectural write.
The same arithmetic applies to a jump at `0x120`: its fall-through is `0x129`. Recovery uses the saved successor of the branch, independent of how far the predicted target lies from it.

Exercise 3.6.1 — Hold vs bubble

What is the difference?

Worked solution

A hold keeps the current contents of a pipeline register for another cycle. A bubble installs an invalid or harmless entry so that no architectural effect is committed from that slot. During a load-use stall, a design can hold the waiting instruction in decode while inserting a bubble into execute. Holding both without the intended control policy could repeat an operation; replacing the waiting decode instruction with a bubble could lose it. The controls refer to specific pipeline boundaries.

Exercise 3.6.2 — Write back cycle

Fetched in cycle 10, when does a five-stage instruction reach W?

Worked solution

With no stalls and one stage per cycle, the instruction occupies F in cycle 10, D in 11, E in 12, M in 13 and W in $\boxed{14}$. The final-stage cycle is the fetch cycle plus four stage transitions. A stall adds its delay to the later stage times. The answer assumes the cycle convention used by this five-stage timeline.

Exercise 3.6.3 — Valid bit

Why is a valid bit useful?

Worked solution

A valid bit records whether a pipeline entry contains a real instruction. After reset or a flush, an entry can contain arbitrary leftover data while valid is zero. Control logic must suppress register writes, memory writes and other architectural effects for invalid entries. A bubble can therefore move through the stages without behaving like a real instruction. The reference PIPE instead carries an explicit

Worked solution (continued)

SBUB status together with inert control fields. A valid bit is another representation of this distinction; either design must gate side effects consistently.

Exercise 3.6.4 — Priority

Can simultaneous stall and bubble be resolved without a rule?

Worked solution

A pipeline register cannot both retain its old instruction and replace it with a bubble at the same edge. The controller must either prove that the requests are mutually exclusive or define which action has priority. State that rule in the next-state equation and apply it to every simultaneous request. Otherwise identical input conditions could be interpreted as two different next states, making the processor specification incomplete.

Summary

Derive each PIPE control condition from the invariant it protects. A stall protects value availability; a flush, control order; status propagation, precise state; forwarding, operand freshness. Once the protected property is clear, the Boolean control condition is easier to derive and harder to misuse.

3.6.9 Exercises

Exercise 3.6.5 — Uppercase and lowercase signals

Explain the difference between `E_valA` and `e_valE` in one cycle.

Go to the solution →

Exercise 3.6.6 — Forwarding priority

An instruction in Decode needs `rax`. Execute has computed the ALU result 5 for the effective destination `rax`; Memory contains an older instruction that will write 3; Write-back contains an even older write of 1. Which value is selected?

Go to the solution →

Exercise 3.6.7 — Load-use controls

State the actions on F, D and E for a detected load-use hazard.

Go to the solution →

Exercise 3.6.8 — Branch correction

A conditional branch in Execute is found not taken after a taken prediction. Which younger instructions are invalidated and where does Fetch resume?

Go to the solution →

Exercise 3.6.9 — Return penalty

Why can the return PC not normally be forwarded from Execute?

Go to the solution →

Exercise 3.6.10 — Precise exception

A faulting load is in M, an older add is in W, and a younger store is in E. Which may commit?

Go to the solution →

Exercise 3.6.11 — Cycle count

Ten instructions traverse a five-stage pipeline. There is one load-use stall and one branch misprediction costing two bubbles. Ignoring overlap, how many cycles from empty to empty?

Go to the solution →

Exercise 3.6.12 — Combined hazard reasoning

A `ret` is in Decode while the instruction in Execute is a load whose destination is used by that return's stack-related source. Which control event should dominate?

Go to the solution →

3.6.10 Solutions

The Five-Stage Pipelined Processor

Solution 3.6.5

`E_valA` is an operand retained in the E pipeline register at the start of the cycle. `e_valE` is the ALU result computed during that cycle from the selected E-stage inputs. For an add with $E_valA = 3$ and $E_valB = 7$, the combinational result is $e_valE = 10$. If M accepts the instruction at the next edge, its field M_valE becomes 10. The uppercase prefix identifies stored state; the lowercase prefix identifies the current stage output.

← Return to the exercise

Solution 3.6.6

Select 5 from Execute. Among the three older writers, the E-stage instruction is nearest to the consumer in program order, so its value supersedes the values 3 and 1. The forwarding mux checks the effective E destination first and finds the RAX match. It therefore selects e_valE without consulting the lower-priority matches. Using 3 or 1 would give the consumer a value that sequential execution had already overwritten.

← Return to the exercise

Solution 3.6.7

Stall F to retain the predicted PC, stall D to retain the waiting consumer, and bubble E to prevent that consumer from advancing with a stale operand. At the same edge the load advances from E to M. During the next cycle, Memory produces the value, forwarding supplies it to Decode, and the consumer can be captured into E at the following edge. The load advances exactly once and the consumer executes exactly once.

← Return to the exercise

Solution 3.6.8

The instructions currently in D and the newly fetched path are wrong; D and E are bubbled according to the control design. In the following cycle the branch is in M and Fetch selects its saved fall-through address from `M_valA`.

← Return to the exercise

Solution 3.6.9

For ret, Execute computes the updated stack pointer $\mathrm{valE} = \mathrm{RSP}_{\mathtt{old}} + 8$. The return address comes from a separate memory read at $\mathrm{RSP}_{\mathtt{old}}$ during M. Forwarding valE would therefore fetch from the updated stack location rather than from the saved return address. Reference PIPE waits while ret occupies D, E and M, then selects W_valM when ret reaches W. A return-address predictor could supply an earlier guess, but that would add a separate prediction and recovery mechanism.

← Return to the exercise

Solution 3.6.10

The older add in W may write its result. The load in M produces ADR and leaves its destination unchanged. The younger store may calculate its address in E, but its memory write must be suppressed: terminal m_stat causes M to receive a bubble instead of the store at the next edge. The load advances to W carrying ADR, and W then retains the terminal status. The architectural state includes the older add and excludes both the failed load's result and the younger store.

← Return to the exercise

Solution 3.6.11

Ten instructions in an initially empty five-stage pipeline take $10 + 5 - 1 = 14$ cycles without hazards. The load-use dependency adds one stall cycle. The misprediction adds two lost cycles. Under the explicit assumption that these penalties do not overlap, total time is $14 + 1 + 2 = 17$ cycles. A cycle diagram would place three additional empty stage opportunities into the ideal completion schedule. Counting the pipeline fill again after each isolated bubble would overestimate the time.

← Return to the exercise

Solution 3.6.12

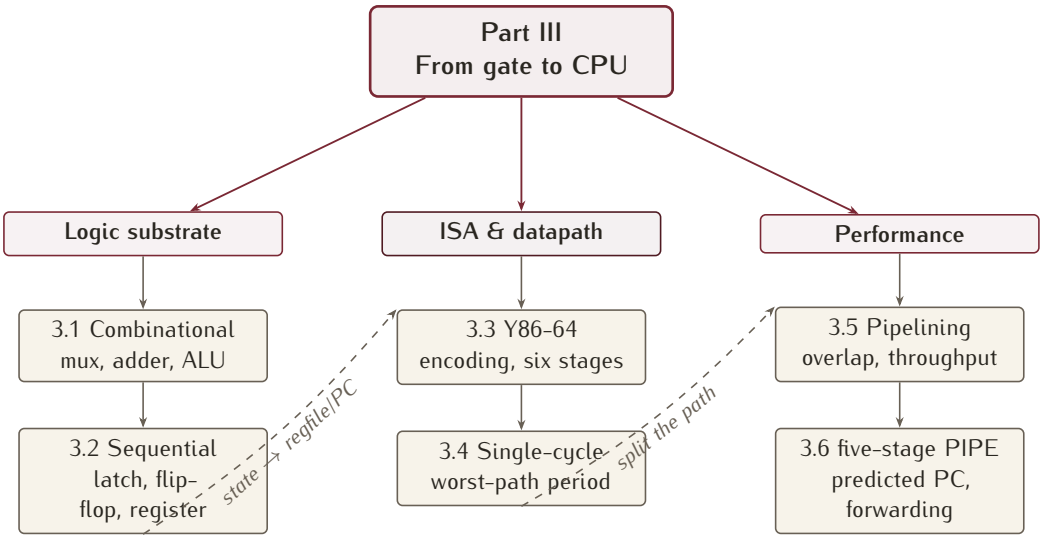
Consider `mrmovq 0(%rax),%rsp; ret`. With the load in E and ret in D, both the load-use condition L and return condition R are true. The equations give $\mathrm{F}_{\mathtt{stall}} = 1$, $\mathrm{D}_{\mathtt{stall}} = 1$, $\mathrm{D}_{\mathtt{bubble}} = 0$ and $\mathrm{E}_{\mathtt{bubble}} = 1$. Thus the return is preserved while the load advances to M.

In the next cycle the load's memory result supplies the return's required RSP value. With the load-use conflict cleared, ret advances into E; D receives a bubble behind it and Fetch remains held by the return condition. Ret then reads the saved target using the newly loaded stack pointer. Bubbling D in the first cycle would have discarded the return before it could use its operand.

← Return to the exercise

Integrated map of processor construction

Part III develops a continuous path from gates to a working pipelined processor. The three coloured groups are the three rungs: the logic substrate, the instruction-level model, and the performance-driven organisation.



The throughline. Chapters 3.1 and 3.2 supply computation and state: multiplexers, adders and the ALU transform values, while latches, flip-flops and registers preserve them. Chapters 3.3 and 3.4 organise those elements around the Y86-64 ISA and expose the long critical path of a single-cycle implementation. Chapters 3.5 and 3.6 overlap instructions, adding prediction, forwarding, stalls and flushes while preserving sequential meaning. The memory elements introduced at the start of the Part reappear as the pipeline registers that make overlap possible.

4

EXCEPTIONAL CONTROL FLOW, INPUT/OUTPUT, AND STORAGE

A running system must respond to events that do not fit ordinary sequential execution. Devices complete work asynchronously, programs request privileged services, translations fail, and storage introduces delays hidden by source-level operations. Each event transfers responsibility while preserving enough state to resume, report failure or terminate safely.

Exceptions and system calls are developed from the Intel and AMD manuals and Linux entry documentation [26, 27, 33, 34]. The view then moves outward: processor to controller, controller to interconnect, and programmed I/O to direct memory access. DMA is treated as a shared-memory protocol whose correctness depends on ownership, address translation, ordering and completion [38, 39, 40].

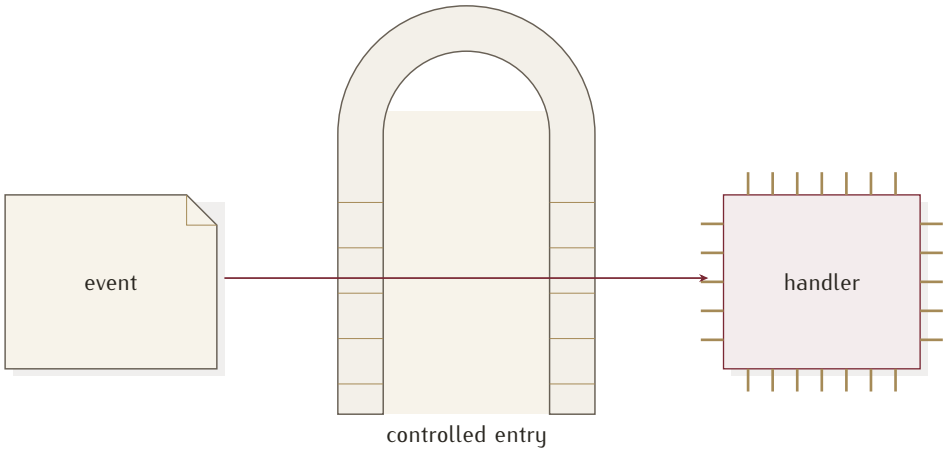
Storage requires separate physical and logical models. Magnetic disks expose positioning and rotation; RAID combines devices under explicit failure assumptions; flash controllers hide pages, erase blocks, wear and garbage collection behind a block interface. NVMe is presented as a queue-based host-controller protocol rather than as a synonym for flash memory [55, 56]. Quantitative examples separate fixed latency, transfer time, queueing and protocol overhead, with complementary operating-system context from standard systems texts [57, 58, 59].

This Part closes by following an I/O request back toward the processor. Once a device has transferred data and reported completion, the kernel may move a waiting execution from blocked to runnable; at that point the device protocol is finished, but a new systems question appears. Several runnable executions may now compete for the available logical processors. That question leads directly to process state, context switching and scheduling in Part 5.

4.1 EXCEPTIONAL CONTROL FLOW AND PROTECTED ENTRY

An instruction tries to access a protected page while a device can also request attention. Either event may redirect execution, but its cause determines which state must be saved and where execution can resume. Following the boundary between the interrupted work and its handler explains faults, traps, interrupts and severe failures. The processor's privilege rules determine how that transfer is made safely.

An exceptional event enters through a protected path



The entry path determines saved state, authority and the return convention.

Figure 4.1: *An event changes the instruction stream through an architecturally defined mechanism. The gate symbol emphasises controlled entry and preserved context. Faults, traps and interrupts still require their distinct cause and resumption rules.*

4.1.1 Breaking the sequential flow

Until now the processor has been described as a machine that repeats a single loop: fetch the instruction pointed to by the program counter, decode it, execute it, advance the program counter, repeat. Two constructs already break the purely linear order of

this loop from *inside* the program: conditional jumps and procedure calls. Both, however, are decided by the program itself and are visible in its own instruction stream. Exceptional control flow includes responses to instruction conditions and external events: division by zero, a missing memory page, a timer tick or a key press. Operating systems also provide explicit service-entry instructions through which a program deliberately requests privileged work.

Processors provide architecturally defined entry mechanisms for events that cannot be handled by ordinary sequential execution. Hardware and privileged software cooperate to preserve the necessary state and redirect execution to a handler. A useful classification distinguishes asynchronous interrupts from synchronous traps, potentially recoverable faults and non-restartable aborts; the categories differ in cause and in the point from which execution may continue.

The word *exception* is overloaded. In C++, Java or Python it names a language construct managed by the runtime. Here it names the hardware/OS event that sits far below the language. The two are related (a runtime may translate a hardware fault into a language exception, as some managed runtimes do for null references; ordinary C++ null dereference instead has undefined behaviour) but they live at different levels of abstraction.

Interrupt requests enter asynchronously relative to a core clock

An interrupt request can arrive independently of the instruction currently executing; architectural delivery waits for an allowed boundary.

A legacy source can assert a pin or controller input, while a message-signalled interrupt is encoded as a transaction on an interconnect. In either case, the request reaches logic whose timing is not generally aligned with the target core's clock. Receivers buffer, synchronise or otherwise transfer the event into the relevant clock domain and record it as pending. Synchronisers provide resolution time and containment, reducing the probability that unresolved metastability reaches normal control logic to the system's reliability target.

Interrupt latency includes detection or packet reception, domain crossing, routing and arbitration before architectural entry begins. Edge-triggered, level-triggered and message-signalled schemes therefore use different capture and acknowledgement mechanisms [26, 43].

4.1.2 Interrupts, faults, traps and aborts

The first distinction is between an *external interrupt* and a *processor exception*. An external interrupt is asynchronous with respect to the instruction stream: a timer, controller or other source becomes pending independently of the instruction currently being executed, and the processor recognises the request at an architecturally permitted boundary. A processor exception reports an architecturally defined condition involving execution or processor state; a machine check may report an asynchronous hardware error. On

x86, the *fault*, *trap* and *abort* classes describe the reported architectural state and the availability of a reliable restart point.

CLASS	RELATIONSHIP TO EXECUTION	REPORTED BOUNDARY / RECOVERY MODEL
External interrupt	asynchronous to the current instruction stream	recognised between architectural instructions; return resumes the interrupted flow
Fault	synchronous processor exception	saved state normally identifies the faulting instruction so that it can be retried after the cause is repaired
Trap	synchronous processor exception	reported after the trapping instruction; saved state normally identifies the following instruction
Abort	severe processor exception	a precise restart point may be unavailable, so reliable restart is not assumed

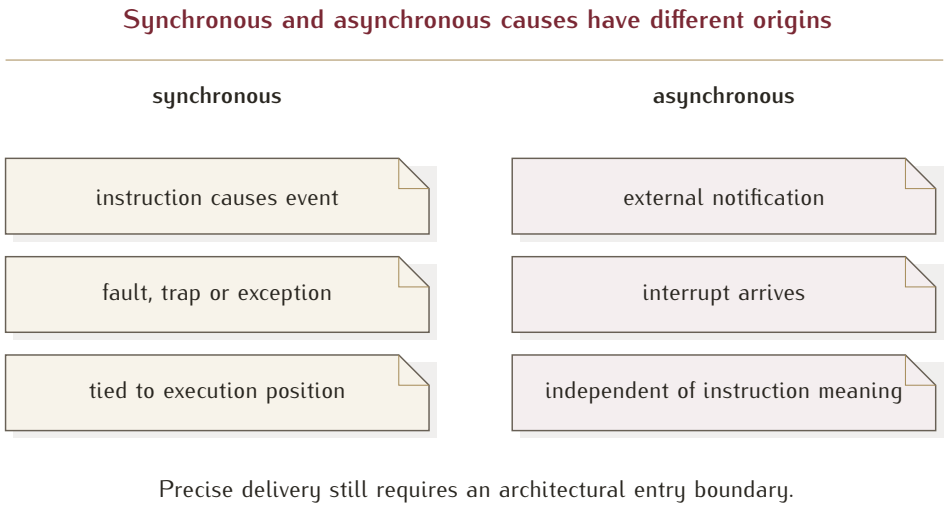


Figure 4.2: *An instruction-triggered event can be explained from that instruction and its operands. An interrupt originates independently, such as from a device or timer. The architecture defines how each event is reported and delivered.*

A page fault is the canonical restartable fault: if the operating system can establish the required mapping, the instruction that triggered the translation failure can be attempted again. A debug single-step event is a canonical trap because it is reported after

the instruction being observed. Machine-check exceptions illustrate the abort class, although the machine-check architecture can report conditions of differing severity and platform error-correction mechanisms may repair some memory errors before software needs to treat them as fatal.

A system call belongs to a different classification question. At the operating-system level it is a deliberate request for a privileged service; on x86-64, however, the native `syscall` instruction uses a dedicated protected-entry mechanism rather than raising an IDT trap exception. Legacy software-interrupt mechanisms such as `int` do use the IDT. Keeping these levels separate avoids turning "trap" into a generic synonym for every intentional transition to the kernel.

Worked example

Problem. Classify each event at the architectural level: (a) integer `div` with a zero divisor; (b) a periodic timer request; (c) native x86-64 `syscall`; (d) a reference to a non-present page that the operating system can supply; (e) an unrecoverable machine-check condition.

Solution. (a) *fault*: x86 reports `#DE` with a saved instruction pointer identifying the faulting instruction. (b) *external interrupt*: it is asynchronous to the current instruction stream and is recognised at an architectural boundary. (c) neither fault, trap nor abort in the Intel exception taxonomy: `syscall` is a dedicated synchronous protected-entry instruction. (d) *fault*: after the page is made present, the instruction can be retried. (e) *abort-class exception*: reliable restart is not assumed for the severe condition stated in the problem.

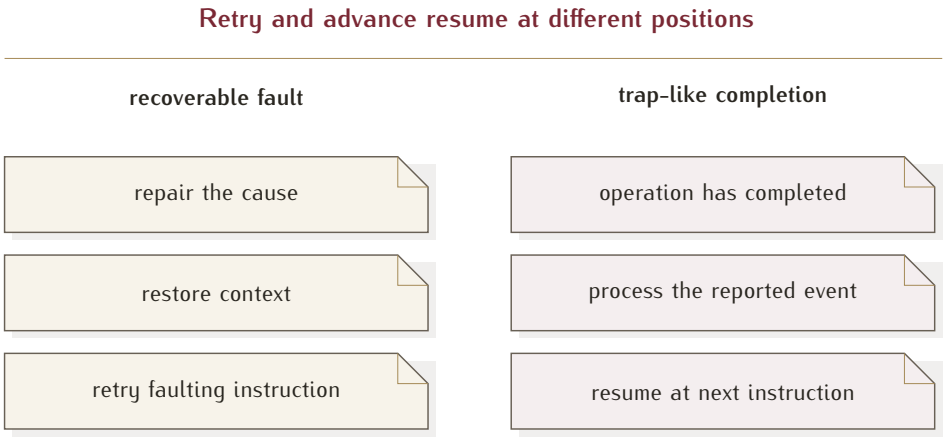
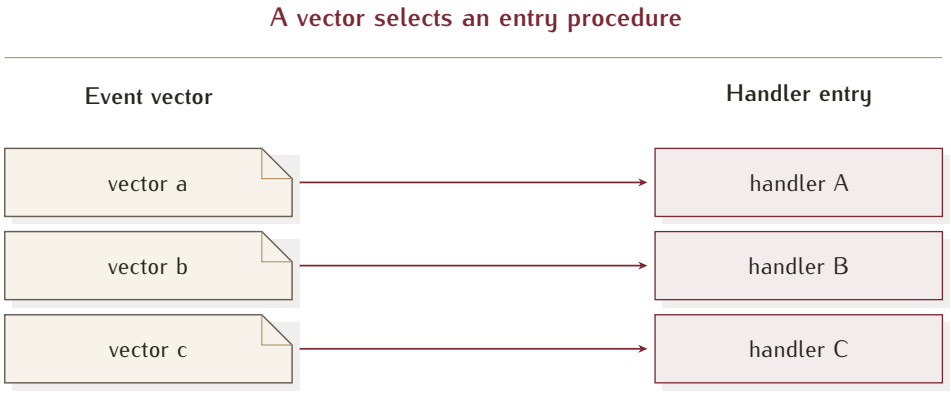


Figure 4.3: *The saved return point must be interpreted according to the event class. A recoverable page fault commonly requires retry, while a completed trap-like operation resumes after the triggering instruction. Handlers must follow the architecture's specified semantics.*

The saved execution point is therefore part of the contract. A restartable fault normally reports the faulting instruction; a trap normally reports the instruction following the one that caused the trap; an external interrupt preserves the architectural point at which the interrupted flow can continue. Exact saved-`rip` semantics remain vector-specific, which is why an operating system must follow the architectural definition rather than infer behaviour from an informal label alone.

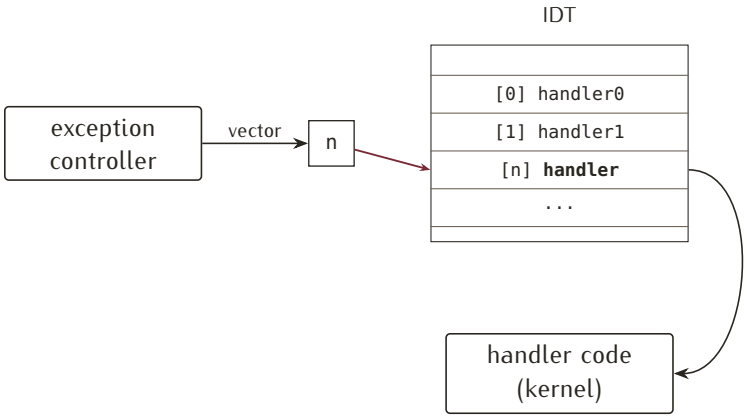
4.1.3 Vectors, privilege and protected entry



The protected dispatch structure associates vectors with entry metadata.

Figure 4.4: *The vector is an identifier used by the entry mechanism. The selected entry provides the handler location and associated architectural information. A caller cannot safely substitute an arbitrary user address for a protected entry.*

An IDT-delivered event selects a small non-negative integer called its *vector*. Architectural events have assigned vectors; the operating system and interrupt controllers configure device vectors. Devices may share a vector or use several vectors for different queues. When an exception occurs, the processor uses this number as an index into a table that stores, for every vector, the address of the corresponding handler. On x86 this table is the *Interrupt Descriptor Table* (IDT); the generic name is *interrupt vector table*.



Each entry is a fixed-size *descriptor*. In legacy 32-bit mode a gate descriptor is 8 bytes; in 64-bit (long) mode it is 16 bytes. The address of the descriptor for vector n is therefore

$$\text{entry_addr} = \text{IDT_base} + n \times \text{descriptor_size}.$$

Because the vector is an 8-bit index, the table has at most $2^8 = 256$ entries. On x86 the first 32 vectors (0 to 31) are reserved for architectural events, including exceptions and NMI (`#DE`=0, `#UD`=6, `#GP`=13, `#PF`=14, and so on); vectors 32 and above are available for external device interrupts.

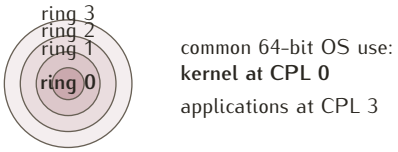
Worked example

Problem. An IDT begins at `0x8000` and uses 16-byte descriptors. What is the address of the descriptor for the page-fault vector (`#PF`, number 14)?
Solution. $0x8000 + 14 \times 16 = 0x8000 + 0xE0 = 0x80E0$.

Worked example

Problem. How large is a full 256-entry IDT in long mode?
Solution. $256 \times 16 = 4096$ bytes, the size of one 4 KiB page. A table occupies one such page only if its placement is suitably aligned.

If any program could execute any instruction and touch any device, a single bug or a single malicious process would compromise the whole machine. To prevent this the x86 architecture defines four *protection rings*, numbered 0 (most privileged) to 3 (least privileged). The number of the ring in which the processor is currently executing is the *Current Privilege Level* (CPL). Common contemporary x86 operating systems, including Linux, normally use two of the four rings: the kernel runs in ring 0 (*kernel* or *supervisor* mode, CPL = 0) and user applications run in ring 3 (*user* mode, CPL = 3).



Writes to control registers, `lidt` and `wrmsr` require CPL 0 in the protected environment considered here. Interrupt-flag controls `cli` and `sti` have their own CPL/IOPL and mode-dependent rules; ordinary Linux user code is not authorised to change IF through them. Port-I/O instructions require a more precise statement: x86 permits `in` and `out` when the current privilege level is allowed by IOPL or by the per-port I/O-permission bitmap in the task-state segment. A normal Linux process at CPL 3 is not granted such permission, so an unpermitted access raises `#GP`; the architecture nevertheless supports selective delegation. Memory-mapped device regions are governed by page mappings and kernel policy rather than by the `in`/`out` rule [26, 27].

User and kernel mode describe the usual operating-system division of authority. Hardware enforces that division through several distinct checks. On x86-64 the current privilege level belongs to the execution context, while instruction checks, descriptor rules, page-table permissions, I/O controls and interrupt-entry rules enforce different

parts of the protection model [26, 27]. Kernel code normally executes at CPL 0 and applications at CPL 3. Controlled entry mechanisms (system calls, exceptions and interrupts) change control flow and, when required, the privilege context and stack. A checked return restores the less-privileged context. This division implements controlled access and least privilege. Kernel accesses remain subject to the translation and protection rules enabled for supervisor execution [60].

Numerically, more privilege corresponds to a *smaller* number. Privilege checks are mechanism-specific. Segment access, gate invocation, page permissions, I/O permission level and the task's I/O bitmap use related but different rules; there is no single CPL-versus-DPL inequality that correctly describes them all. In normal Linux user mode, privileged instructions and unpermitted port I/O fault, while the kernel configures the relevant controls.

Worked example

Problem. A process runs with CPL = 3 and executes `in al, 0x60` to read the keyboard data port directly. What happens?

Solution. In an ordinary Linux process, IOPL and the TSS I/O-permission bitmap do not authorise port `0x60`. The processor therefore raises `#GP`; Linux handles vector 13 and normally delivers a fatal signal to the process. The outcome follows from the configured permissions; another configuration could explicitly authorise selected user-mode I/O. Controlled user-space device access can instead be arranged through kernel-mediated mappings and frameworks such as VFIO, which pair device assignment with IOMMU isolation.

Protected entry is a coordinated state transfer

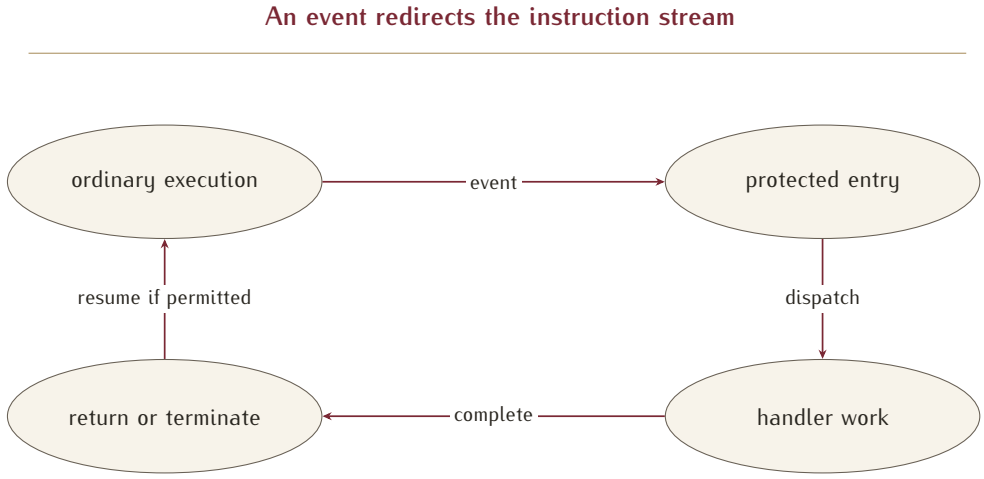


Figure 4.5: *Protected entry preserves the state required by the architecture and dispatches an appropriate handler. The handler classifies the cause and chooses a permitted outcome. The final return is conditional on the event and policy.*

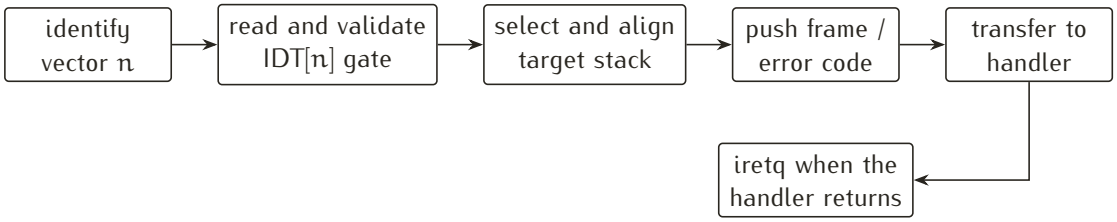
The processor saves selected state and jumps to a handler.

Detection signals inhibit or redirect normal control, choose a descriptor or vector, update privilege-related state and initiate memory operations for saved context where required. These actions use the same datapath, register and translation structures as ordinary execution, but under specialised control sequencing.

Entry latency and ordering are consequences of real internal dependencies. The architecture specifies the visible result; the implementation schedules the necessary physical transfers while preventing younger instructions from committing [26, 50].

4.1.4 Precise entry in a pipelined machine

Exception entry is vector- and context-dependent. The processor completes or abandons the current instruction according to the exception class, selects and validates the IDT gate, and then chooses the current stack, a privilege-level stack from the TSS, or an IST stack as required. In 64-bit mode it aligns the selected stack pointer to a 16-byte boundary and unconditionally saves the interrupted `SS:RSP`, followed by `RFLAGS`, `CS` and `RIP`. A privilege change affects where the new stack pointer comes from and causes the new `SS` to become null; it does not determine whether the old `SS:RSP` fields are present. Some vectors also push an error code. The handler returns with `iretq` when appropriate. Not every event changes CPL or supplies an error code [26, 27].



In 64-bit mode the fixed architectural return frame contains five 8-byte stack entries: old `SS`, old `RSP`, `RFLAGS`, `CS` and `RIP`. It therefore occupies 40 bytes for a vector without an error code and 48 bytes for a vector that supplies one. This count is independent of whether CPL changes. Stack selection and alignment are separate operations: before pushing the fields, hardware rounds the selected `RSP` down to a 16-byte boundary, so the total movement of the active stack pointer can exceed 40 or 48 bytes by the discarded alignment offset. Kernel entry code may then add a software-defined frame.

Worked example

Problem. On a page fault in long mode, how many bytes does the hardware push onto the kernel stack before entering the handler?
Solution. A page fault supplies an error code, so its fixed long-mode hardware frame is 48 bytes: old `SS`, old `RSP`, `RFLAGS`, `CS`, `RIP` and the error code. The same 48-byte field count applies whether the fault changes CPL or remains in kernel mode. A user-to-kernel fault normally selects a kernel stack first; a same-CPL fault normally continues on the current stack unless IST is configured. Any bytes discarded while aligning the selected `RSP` to 16 are additional to the 48-byte field count, and software entry code saves further state separately.

A pipelined processor keeps several instructions in flight at once. When an instruction raises a fault, architecturally younger instructions must not be allowed to commit: they were processed on the assumption that no earlier exception would occur. The recovery machinery therefore squashes the younger work and redirects control while preserving a precise architectural boundary. In the simple single-issue teaching pipeline introduced in Part III, with at most one instruction resident in each of N stages, at most $N-1$ younger instructions can occupy the remaining stages. This numerical bound does not apply to superscalar or out-of-order processors, whose queues and reorder buffers may contain many more younger instructions.

Caution

A flush coordinates operations on the pipeline registers: the younger stages are turned into bubbles (no-ops), pending writes are cancelled, and the program counter is redirected to the handler. A branch misprediction and an exception share only a high-level *squash-and-redirect* pattern. A misprediction discards spec-

Caution (continued)

ulative work and resumes along the program's correct path without creating an architectural event. A precise exception must additionally preserve an architecturally coherent retirement boundary, order competing faults, construct the specified entry state and transfer control to a privileged handler when required.

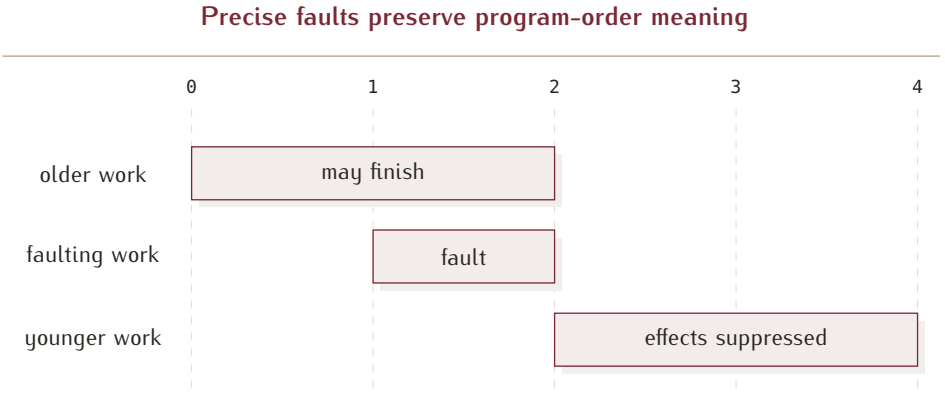


Figure 4.6: *A precise fault reports a state consistent with the defined instruction boundary. Older work may complete, while forbidden effects from the faulting instruction and younger work are prevented. The ISA determines the exact restart semantics.*

The distinction between *branch* and *exception* matters for where control resumes. After a resolved branch, execution continues along the program's own logic. In the Linux x86-64 configuration considered here, an exception from user mode enters a kernel handler selected through the configured vectoring mechanism and executes at CPL 0 before a controlled return, commonly through `iretq` when that restoration path is required. The architecture also permits events that remain at the current privilege level, so an exception is not universally synonymous with a transition to CPL 0. What branch recovery and exception recovery share is the need to discard younger speculative work.

Physical cost of flushed pipeline work

Flushed instructions are excluded from the committed architectural history.

Control bits invalidate pipeline entries and block their writes, but energy already spent fetching, decoding and executing them has become heat. Their requests may also have touched caches, predictors or interconnect queues depending on the implementation.

Frequent exceptions, interrupts or control mispredictions reduce useful work per joule as well as throughput. Architectural cancellation cannot recover consumed electrical energy [50, 41].

4.1.5 Linux entry code and saved context

The low-level x86 entry code must turn several hardware frame shapes into a consistent software representation. Start with `arch/x86/entry/entry_64.S`, the IDT-entry macros and `Documentation/arch/x86/entry_64.rst`. The code saves general registers, handles user/kernel `GS` state, performs entry bookkeeping and eventually invokes a C-level handler. On exit it chooses an appropriate return path and restores the interrupted context [33, 34, 35].

Linux uses `GS`-relative addressing for per-CPU kernel data while user space may have its own `GS` base. Entry code must know whether the current base is the user or kernel one before accessing per-CPU state. Nested exceptions and unusual entry paths make a simplistic "always execute `swapgs`" rule unsafe; this is why the kernel documentation calls the logic subtle.

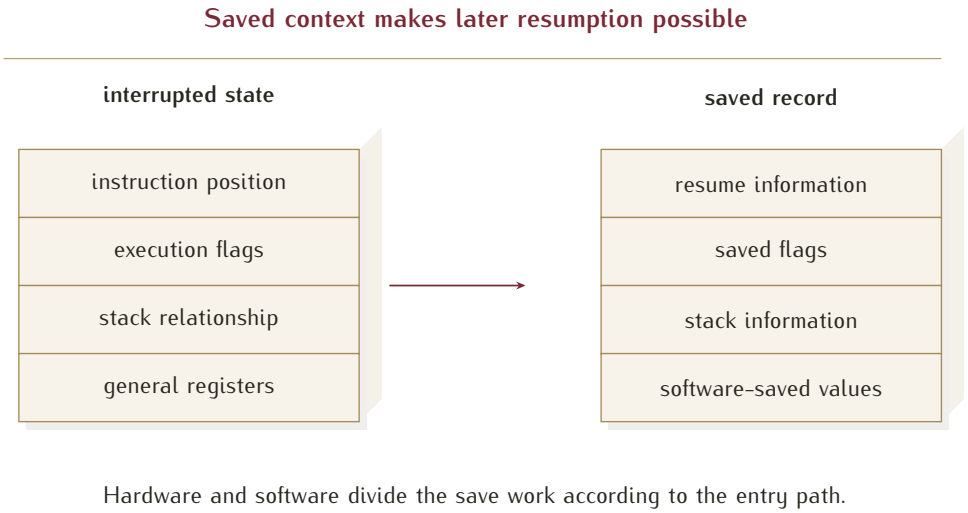


Figure 4.7: *The exact saved frame depends on architecture, privilege transition and entry mechanism. Software saves additional registers when required. Treating every entry path as one universal hardware-pushed frame would lose these distinctions.*

A hardware exception or interrupt enters through an IDT gate. A 64-bit `syscall` enters through model-specific registers and does not create the same hardware stack frame. A normal `call` remains at the same privilege level and pushes only a return address. All three change `rip`; almost everything else about their contract differs.

4.1.6 What protected entry must preserve

An exception is useful only if the machine can establish a well-defined boundary between the interrupted computation and the handler. The architectural state includes the program counter, general-purpose registers, status flags, segment or privilege information and any control state needed to interpret addresses. Reorder buffers and

speculative queues must be managed so that software-visible state corresponds to a legal instruction boundary.

A useful distinction is between *minimal hardware state* and *complete software context*. Hardware saves enough information to enter a handler safely and later execute a return-from-exception instruction. The handler prologue saves any additional registers it intends to use. The operating system saves still more state when the event may lead to a process switch. Confusing these layers produces two opposite errors: assuming that hardware saved every register, or saving so much state on every event that the common path becomes needlessly expensive.

Worked example: minimal frame versus process context

A timer interrupt arrives while a user process is executing. The processor transfers to a kernel stack and records a return address and flags. Entry code preserves the interrupted general-register context in its kernel frame. If the scheduler selects another task, the context-switch path also preserves the kernel continuation and switches the task-dependent execution and address-space state. The saved context is distributed across the entry frame, kernel stack and task structures; it need not all reside in one control-block object. If the scheduler had returned immediately to the same process, the third step would not have been necessary.

The IDT descriptor also supplies a gate type and descriptor privilege level (DPL). DPL restricts explicit software invocation of a gate; hardware delivery of an exception or interrupt does not apply that caller check. An interrupt gate clears IF in the handler's active flags, while a trap gate preserves it. These gate names do not determine whether the delivered exception is classified as a fault or trap.

4.1.7 Recovery, nesting and priority

A page fault is the standard example of a fault that may be repaired. The memory-management unit rejects a translation or protection check and reports the virtual address and a compact error code. The kernel inspects the process's virtual-memory map. Several outcomes are possible:

- the page is valid but not resident, so the kernel allocates a frame or reads data from storage and retries the instruction;
- the access is part of a legitimate copy-on-write operation, so a private writable copy is created;
- the address violates the process's permissions or is not mapped, so the process receives a fatal signal;
- the fault occurred while the kernel was accessing a user pointer, in which case a protected recovery path converts the hardware event into an error return.

Hardware reports the condition through a vector and associated state. Privileged software chooses the recovery policy from the process's mappings, permissions and execution context.

Virtual memory combines address translation, caching and protection. A *TLB miss* means that a translation is absent from the translation cache; it does not, by itself, mean that a page is absent from memory or that an exception must occur. On x86-64, hardware walks the page tables and can refill the TLB without entering the kernel when the mapping is present and permits the access. A non-present entry, a protection violation or another architecturally defined paging error causes a page-fault exception instead. The kernel then determines whether execution can resume, for example after demand allocation, file-page retrieval or copy-on-write, or whether the process must receive a fault notification. Some other architectures use software-managed TLB refill; that is a different machine contract. Section 6.2 separates these paths from a data-cache miss.

A sequential processor can identify the fault within the current instruction; recovery still follows that instruction's specified fault semantics. A pipelined or out-of-order processor may already have younger instructions in flight, and some may have computed results. For the simple instructions in the teaching model, a *precise fault* presents software with this boundary:

1. every instruction older than the faulting instruction has completed;
2. the faulting instruction has not changed architectural state;
3. no younger instruction has changed architectural state;
4. the saved program counter identifies the correct restart or continuation point.

The implementation therefore records exception status as instructions move through the machine, suppresses writes by the faulting and younger instructions, drains or commits older instructions, then redirects fetch to the handler. In an in-order five-stage pipeline this often appears as stalls and bubbles. In an out-of-order core, retirement order provides the commit boundary.

Some real instructions define restartable partial progress. For example, a fault during x86 `rep movsb` can leave earlier byte copies completed, RSI and RDI advanced, and RCX reduced; saved RIP still identifies the repeated instruction so its remaining iterations can resume. Precision means conformity to this architectural restart state, rather than a universal promise that every effect of the faulting instruction is undone [26].

Worked example: exception and younger store

Instruction I_4 causes a protection fault in its memory stage. A younger instruction I_5 has already calculated the address of a store. Precise state requires that I_5 must not make its stored value architecturally visible. Speculative cache or coherence activity may nevertheless have occurred. The processor may have performed speculative address calculation, but the store cannot become architecturally visible before I_4 is known to complete successfully.

Further events can occur while a handler is running. An interrupt may arrive while another handler is active; a page fault may occur while the kernel copies data; a non-maskable event may pre-empt code that has disabled ordinary interrupts. Systems therefore assign priorities, define which interrupts can be masked and keep critical sections short.

Disabling maskable interrupts controls local delivery on one processor. Other processors continue executing and may access shared data concurrently. Shared kernel data still requires locks or other synchronisation. Excessive interrupt masking increases latency and can cause devices to overflow their buffers.

Security consequence: the entry path is an attack surface

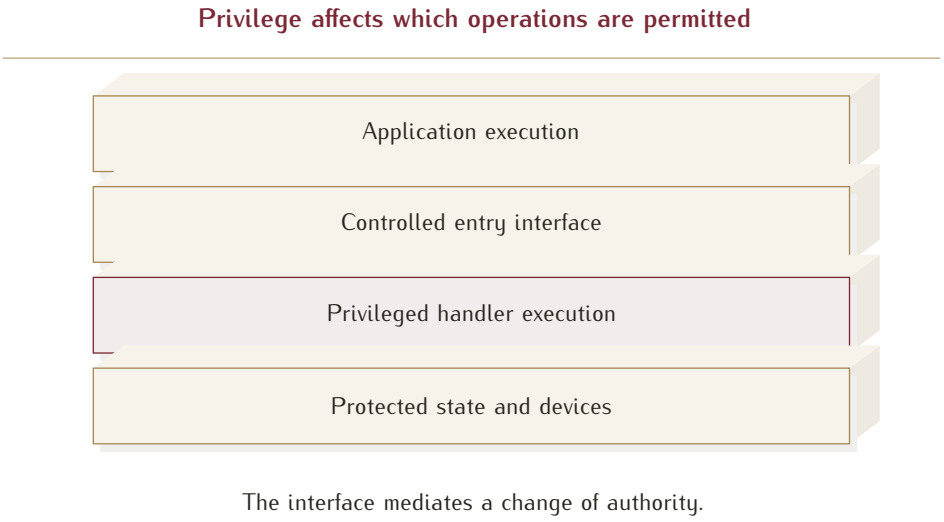


Figure 4.8: *The handler executes with authority that ordinary user code lacks. Its entry checks and handling of user-controlled data are therefore part of the protection boundary. Privileged execution does not make supplied pointers automatically trustworthy.*

An exception or system-call entry can cross a privilege boundary. The CPU and kernel must validate the gate, stack, saved state and return path. Historically, subtle mistakes in descriptor privilege, speculative execution, return-state validation or user-pointer handling have allowed privilege escalation or information leakage. A robust design treats every value originating in user mode as untrusted, even when hardware performed the transition correctly.

Protection moved into hardware

Early computers often ran one program with unrestricted access. Multiprogramming created the need to isolate programs and recover from events without resetting the

machine. Descriptor tables, privilege levels and interrupt controllers are hardware answers to that need. Modern operating systems use only a subset of the protection-ring structure, but the central separation between user and supervisor execution remains fundamental. [60, 59]

Laboratory 24: Faults and signal delivery

Compare Linux x86-64 synchronous events produced by `ud2`, integer `idiv` with zero divisor, and a load from an unmapped page. Give the architectural exception, typical user-visible signal and a minimal terminating signal handler. Explain why the tests should use an explicit instruction contract.
Worked solution

Laboratory 24: solution

`ud2` raises invalid opcode, normally delivered as SIGILL. Integer divide by zero raises divide error, normally SIGFPE despite its name. An unmapped user load raises page fault and is normally delivered as SIGSEGV when the mapping cannot be repaired. Each fault saves a restart address for the offending instruction. The C language gives no portable result for null dereference or signed division by zero, so explicit assembly instruction tests avoid asking the compiler to preserve undefined source behaviour. The following Linux x86-64 program installs a terminating handler and selects one explicit instruction test:

```
#define _GNU_SOURCE
#include <signal.h>
#include <stdint.h>
#include <sys/mman.h>
#include <unistd.h>

static void handler(int sig) {
    static const char msg[] = "caught synchronous signal\n";
    (void)write(2, msg, sizeof msg - 1);
    _exit(128 + sig);
}

int main(int argc, char **argv) {
    struct sigaction sa = {0};
    if (argc != 2) return 2;
    sa.sa_handler = handler;
    if (sigemptyset(&sa.sa_mask) == -1) return 2;
    if (sigaction(SIGILL, &sa, 0) == -1 ||
        sigaction(SIGFPE, &sa, 0) == -1 ||
        sigaction(SIGSEGV, &sa, 0) == -1) return 2;
```

Worked solution (continued)

```
    if (argv[1][0] == 'u') {
        __asm__ volatile("ud2");
    } else if (argv[1][0] == 'd') {
        __asm__ volatile("mov $1, %%eax; xor %%edx, %%edx;"
                         "xor %%ecx, %%ecx; idiv %%ecx"
                         : : : "rax", "rdx", "rcx", "cc");
    } else if (argv[1][0] == 'p') {
        long page = sysconf(_SC_PAGESIZE);
        if (page <= 0) return 2;
        void *p = mmap(0, (size_t)page, PROT_READ | PROT_WRITE,
                       MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
        if (p == MAP_FAILED) return 2;
        uintptr_t address = (uintptr_t)p;
        if (munmap(p, (size_t)page) == -1) return 2;
        /* No intervening mapping or other thread reuses this address. */
        __asm__ volatile("movb (%0), %%al"
                         : : "r"(address) : "rax", "memory");
    } else {
        return 2;
    }
    return 3; /* The selected fault should terminate in the handler. */
}
```

Compile with `gcc -std=gnu11 -O2 -Wall -Wextra fault-demo.c -o fault-demo`. Run `./fault-demo u`, `./fault-demo d` and `./fault-demo p` separately. On Linux x86-64 the handler's exit statuses are 132, 136 and 139, corresponding to 128+SIGILL, SIGFPE and SIGSEGV. The message is sent to standard error; a failed or partial diagnostic write does not prevent termination. Both `write` and `_exit` are async-signal-safe.

The page test saves the numeric address before unmapping and uses an assembly load, so the C compiler is never asked to evaluate a dereference through an invalid C object pointer. The single-threaded test makes no intervening mapping at that address. Under GDB, inspect the instruction and signal at the first stop, then allow signal delivery to observe the handler. Returning to an unchanged faulting instruction would normally repeat the fault.

Return to the laboratory

Exceptional control flow above the ISA

Exceptional control flow continues above the processor's trap mechanism. A Unix-like process can create another process, replace its program image, wait for state changes and receive signals. These interfaces are software contracts built on protected entry; they are not additional x86 exception classes [1].

`fork`. Creates a child process with a logically separate execution context and an address space initially derived from the parent, commonly implemented with copy-on-write mappings.

`execve`. Replaces the calling process image with a new program. It does not create a second process and does not return on success.

`waitpid`. Allows a process to observe and reap selected child state changes.

signal. A kernel-mediated notification delivered according to the process signal disposition and mask. Signals may be generated synchronously by the current thread's faulting instruction or asynchronously by other events; delivery follows the signal disposition, thread mask and kernel rules.

A caught signal can enter a handler that later returns to the interrupted context, so shared state between the handler and main code requires special discipline. Most library routines are not async-signal-safe: they may hold internal locks or manipulate state that the interrupted code was already updating. A minimal handler commonly sets a `volatile sig_atomic_t` flag or uses an async-signal-safe operation to write to a prearranged descriptor, leaving substantial work to normal control flow.

Caution

A signal is not a reliable counter unless the interface explicitly provides queued real-time semantics and the program uses them correctly. Traditional signals of one number may coalesce, so "one signal per event" is an unsafe assumption.

Exercise 4.1.1 — Control-flow classification

For each transition – page fault, timer interrupt, system call, `fork`, successful `execve`, signal delivery and function return – identify the responsible contract, whether a new process is created, whether the previous program image survives and where control resumes.

Solution 4.1.1

Assume ordinary successful cases except where the event itself is a fault. Distinguish a change of execution context from creation or replacement of a process.

page fault. The ISA reports a translation or protection condition to the kernel. No process is created and the program image remains. After a recoverable fault, execution retries the faulting instruction; an unrecoverable fault can cause a signal and termination instead.

Worked solution (continued)

timer interrupt.	Hardware and kernel interrupt handling transfer control to an interrupt handler. No process or image is created. The scheduler may select another thread, but the interrupted thread can later resume its saved execution state.
system call.	The operating-system ABI defines a requested service. An ordinary returning call resumes user code after the entry instruction with a result or error. Whether a process is created or an image replaced depends on the requested operation.
`fork`**.**	The process-creation interface creates a child with a separate identity and logically separate address space. Both executions retain the existing program image and continue after the call; the parent receives the child's PID and the child receives zero.
successful `execve`**.**	The process interface replaces the calling process's program image while retaining its PID. Control begins in the new image's entry machinery. The old call does not return and no additional process is created.
signal delivery.	The kernel applies the process's signal disposition. For a caught signal, it arranges entry to a handler in the same process and image; normal handler return restores the saved user context, subject to the interrupted operation's restart rules. Default actions may instead stop, continue or terminate the process.
function return.	The language calling convention and machine return instruction restore control to the caller. They create neither a process nor a new program image. The return address and calling convention determine the continuation and result.

Processes, threads and scheduling

An interrupt or trap transfers control to the kernel, but the operating-system question is what happens to the current execution context next. A process may be running, ready to run or blocked for an event; a thread is the schedulable execution stream that shares selected process resources with peer threads. The kernel records enough state to suspend one thread and resume another [59].

Scheduling is a policy layered on mechanisms such as timer interrupts, context switching, run queues and blocking wakeups. Different workloads value different objectives: throughput and turnaround in batch work, response time in interactive work, and

bounded lateness in real-time work. No single ordering rule optimises all objectives, and a context switch itself consumes time and perturbs caches and translations.

A blocked thread is ineligible for scheduling until the awaited event occurs. A ready thread is eligible and waits for a processor. Busy waiting deliberately remains runnable while testing a condition; blocking gives the processor to other work and requires a wakeup path.

Exercise 4.1.2 — State-transition trace

Trace a thread that calls `read`, blocks for device completion, is awakened by an interrupt and later resumes. Mark user/kernel mode, running/ready/blocked state, the scheduling decision and the point at which the system call can return.

Solution 4.1.2

Assume the requested bytes are absent from memory, device I/O is needed, and completion succeeds. The thread begins running in user mode. The library wrapper executes the system-call entry sequence; the same thread is now running kernel code. The kernel checks the request, submits the device operation and puts the thread on the appropriate wait queue.

Once the thread is blocked, it cannot be selected for execution until the awaited condition changes. The scheduler chooses another ready thread or the idle context. The original thread has a saved kernel continuation inside the read path; it is not executing while blocked.

Device completion causes an interrupt on a processor. The interrupt handler executes kernel code, records completion and makes the waiting thread ready. The original thread may still wait for CPU time. The interrupt's return does not itself imply that the waiting thread has resumed.

When a scheduling decision selects that thread, its state changes from ready to running and execution continues in the kernel read path. The kernel performs any remaining checks and data copying, supplies the byte count, and returns through the system-call exit path. Only then does the thread run user code after the read call. The state sequence is therefore running/user, running/kernel, blocked, ready, running/kernel, running/user.

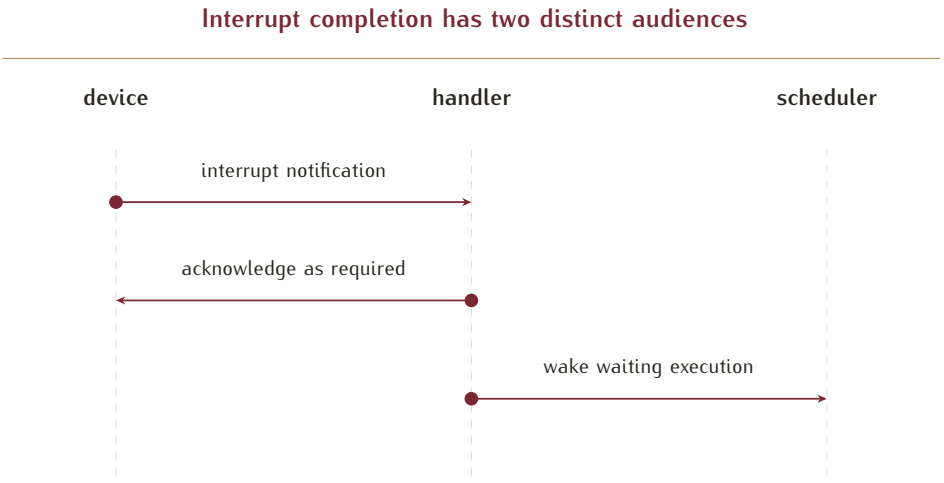


Figure 4.9: *The device-facing action manages the interrupt or completion source. The scheduler-facing action changes the waiting execution's eligibility. Exact acknowledgement order and requirements are device- and controller-specific.*

From operator intervention to protected exceptional control flow

Timeline

Year	Machine, standard or event	Architectural significance
1950s	**Hardware interrupts**	Machines suspend instruction flow without operator rewiring.
1962	**Manchester Atlas**	Privilege and protected memory support a resident supervisor.
1985	**Intel 80386**	Paged protection, rings and structured exception entry mature in x86.

In early automatic calculators, the human operator handled many failures that later machines would report through structured architectural events. A jammed card reader, a failed relay or an arithmetic anomaly stopped the work, lit a lamp or forced the operator to inspect the machine. That model became untenable as electronic computers grew faster than their peripherals. On ENIAC, completed in 1945 and publicly unveiled in 1946, arithmetic ran electronically while program setup and much input/output still depended on switches, cables and operators. By the beginning of the 1950s, machines such as UNIVAC I and EDSAC were already expected to continue a sequence of jobs and to react to devices whose completion could not be predicted instruction by instruction.

Designers needed a way to interrupt a computation, preserve its state, perform the required service and then resume the interrupted program [87, 91, 67].

The first practical answer was an interrupt mechanism tied to a small set of external conditions. The processor completed an instruction, saved a return location in a fixed place and transferred control to a service routine selected by hardware. This mechanism looks modest from a modern viewpoint, yet it introduced three enduring ideas. The event is not an ordinary branch chosen by the current program. The machine records an architectural return state, then transfers control to code that the interrupted program did not select. Early systems differed widely in vocabulary.

Manufacturers spoke of interruptions, traps, extracodes, automatic subroutines and supervisory transfers. The IBM System/360, announced on 7 April 1964, gave the subject a particularly systematic form. Its architecture distinguished classes such as external, input/output, program, supervisor-call and machine-check interruptions, and associated each class with old and new Program Status Words. The PSW contained the instruction address, condition code, interrupt masks and control information. An interruption therefore became a precisely documented change of architectural state rather than an informal jump to a repair routine [63, 109].

Protection developed at the same time because an interrupt handler is useful only if ordinary code cannot overwrite it or imitate its authority. The Manchester Atlas, operational in 1962, combined virtual memory, a resident supervisor and a protected instruction mechanism. Its "extracodes" let a program request operations implemented by the supervisor while the hardware restricted sensitive actions.

IBM's System/360 used a supervisor state bit in the PSW and defined privileged instructions that generated a program interruption if attempted in problem state. These designs established exceptions as controlled transfers across an authority boundary. The operating system must be able to inspect and modify the machine state of a less privileged program, while the program must not be able to seize the operating system's state [94, 63, 109].

The x86 lineage made this boundary progressively more explicit. The 8086 of 1978 had a real-mode interrupt vector table at physical address zero: 256 entries, each containing a segment and offset. Software could execute `INT n`; hardware faults and external pins could select vectors as well. There was no strong memory protection in the modern sense. The 80286, introduced in 1982, added protected mode, descriptor tables and four privilege levels called rings. The 80386, introduced in 1985, extended the model to 32-bit offsets and paging.

An interrupt or exception could now enter through an Interrupt Descriptor Table gate that named a target code segment, specified a Descriptor Privilege Level and caused a stack switch when privilege decreased numerically, for example from ring 3 to ring 0. The processor saved a defined frame, including the old stack state when crossing rings, before transferring control. These mechanisms remain recognisable in Intel 64

and AMD64, although long mode changes segmentation and provides additional entry instructions [104, 106, 26, 27].

Modern manuals distinguish faults, traps and aborts by their restart semantics. A fault is reported with a saved instruction pointer that normally identifies the faulting instruction, so a page fault handler can make the missing page accessible and retry. A trap is reported after the instruction, as with a debug single-step event. An abort signals a severe condition for which reliable restart may not be possible. External interrupts are asynchronous with respect to the instruction stream. Most processor exceptions arise synchronously from an instruction; machine checks can also report errors asynchronously, without identifying one causative instruction.

The restart categories determine which address is saved, which partial effects are permitted and how a handler can continue execution. Intel documents the vector assignments, error-code formats, gate semantics and return operation; AMD documents the corresponding AMD64 rules. The manuals must be read alongside the operating-system code because the processor defines entry mechanics, while the kernel defines policy [26, 27].

Linux on x86-64 turns that hardware contract into a disciplined entry path. Assembly code in `arch/x86/entry/entry_64.S` establishes a kernel-visible register frame and reaches common C entry code. The kernel entry documentation explains why bookkeeping such as lock dependency state, context tracking, tracing and interrupt-state transitions cannot be inserted arbitrarily. A page fault eventually reaches memory-management code; a general-protection fault follows another path; an external interrupt is acknowledged and dispatched through the interrupt subsystem.

On return, Linux must decide whether the fast architectural path is legal or whether a conservative return is needed because the saved state contains unusual segment, flag or tracing conditions. The visible arrow in a textbook diagram therefore compresses a carefully ordered protocol involving CPU state, per-CPU kernel state, scheduler accounting and security checks [33, 34, 35].

Speculation has added a final historical layer. Architecturally, an exception is precise when all older instructions have committed and no younger instruction has changed visible state. Out-of-order processors achieve that illusion with structures such as reorder buffers and retirement logic: execution may occur in a different order internally, but faults are presented in program order.

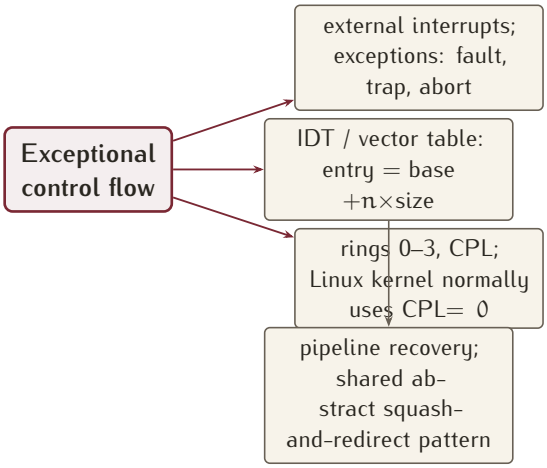
Microarchitectural side channels showed that squashed speculative work can nevertheless leave measurable traces in caches and predictors. Since 2018, operating systems and processors have therefore had to reason about both architectural exceptions and transient execution. Protected entry must therefore be analysed both through its architectural state transitions and through the microarchitectural traces of transient work [26, 27, 33].

The historical line is therefore concrete. Operator intervention gave way to hardware interruption, and interruption became a saved-state protocol. Privileged state and

memory protection turned that protocol into an authority boundary. Precise retirement later preserved the sequential abstraction on speculative processors, while modern kernels implemented the boundary with extensive entry, validation and return code. For exact study, the appropriate chain is IBM's *System/360 Principles of Operation*, the Intel and AMD architecture manuals, and the current Linux x86 entry documentation and source tree [109, 26, 27, 33, 34, 35].

Vectoring, privilege checks and saved machine state remain visible in contemporary exception-entry code and hardware tables.

Concept map



Concept check

1. Which class re-executes the faulting instruction?
2. Compute the IDT entry address for vector 0x80 with base 0 and 16-byte descriptors.
3. With CPL = 3, may a process execute `lidt`?
4. For a divide error in 64-bit mode, how many bytes are pushed by the hardware on a same-stack entry, and how does the count change for a CPL 3 to CPL 0 stack switch?
5. What abstract squash-and-redirect pattern do branch recovery and exception entry share, and which additional obligations make a precise exception different?

Solutions to the review questions

1. A fault normally records a return position that allows the faulting instruction to be retried after the handler repairs the condition. A trap normally reports a completed instruction and resumes after it. Recovery also depends on the particular exception and the handler policy.
2. The descriptor begins at $\texttt{base} + \textit{vector} \cdot 16 = 0 + \texttt{0x80} \cdot 16 = \texttt{0x800}$, or 2048 in decimal. The vector selects a descriptor; it is not itself the handler address.

Worked solution (continued)

3. No. In the stated protected x86 environment, `lidt` requires privilege level 0. Execution at CPL 3 causes a protection exception rather than installing a new IDT.
4. An ordinary long-mode IDT entry saves five eight-byte slots for SS, RSP, RFLAGS, CS and RIP: $5 \cdot 8 = 40$ bytes. A divide error has no hardware error code. The five-slot frame also applies on a CPL 3 to CPL 0 stack switch; the selected destination stack changes. Alignment adjustments before the pushes are separate from these 40 bytes. An exception with a hardware error code adds another eight-byte slot.
5. Both paths discard younger work and redirect instruction fetch to a selected address. Precise exception entry must additionally expose the architecturally correct committed state, preserve the return context, select privilege and stack state as required, and identify the exception. A branch misprediction repairs a speculative fetch choice while retaining ordinary program control.

4.1.8 Worked problems

Long-mode frame and stack alignment

Suppose a same-CPL interrupt in 64-bit mode uses the current stack, with RSP=`0x1008`, and has no error code. Hardware first rounds the selected stack pointer down to `0x1000`, consuming eight bytes of alignment space. It then pushes old SS, old RSP, RFLAGS, CS and RIP as five eight-byte slots. The final RSP is

$$\texttt{0x1000} - 40 = \texttt{0x0FD8}.$$

The total movement is $8 + 40 = 48$ bytes. The saved old RSP still records `0x1008`, allowing the return mechanism to restore the original stack position.
For a page fault with the same selected stack pointer, the error code adds eight bytes. The fixed frame is then 48 bytes, final RSP is `0x0FD0`, and the total movement is $8 + 48 = 56$ bytes. A privilege transition or IST selection changes the source of the selected stack pointer; the long-mode SS:RSP fields remain present in either case.

Gate type and active IF

Assume IF was 1 before entry. An interrupt gate saves the old flags in the return frame, then clears active IF so ordinary maskable interrupts are disabled in the handler. A trap gate leaves active IF at 1. If IF was initially 0, both leave active IF

Worked example (continued)

at 0; that input cannot distinguish the gate types. NMI follows separate rules. The saved flags and the handler's active flags are therefore distinct values.

Exercise 4.1.3 — Table size

How many bytes for all 256 16-byte IDT entries?

Worked solution

There are 256 vectors and each long-mode IDT descriptor occupies 16 bytes. Total storage is $256 \cdot 16 = 4096$ bytes, exactly 4 KiB. Descriptor v starts at $base + 16v$. The last descriptor starts at $base + 4080$ and occupies the final 16 bytes through $base + 4095$. For this full table, the corresponding limit is 4095 because the limit counts from zero.

Exercise 4.1.4 — NMI

Does IF=0 block NMI?

Worked solution

IF controls delivery of ordinary maskable external interrupts. NMI is the non-maskable interrupt class and is not disabled merely by clearing IF. Thus IF=0 does not by itself block NMI. NMI has separate architectural delivery and blocking rules, which must be considered when reasoning about nested events. Distinguishing interrupt classes is essential before applying a flag-based masking rule.

Exercise 4.1.5 — Error code

Do all exceptions push an error code?

Worked solution

Only specified exception classes push an error code. A handler must know whether the entry path supplies one before locating the saved return frame. For an event without a hardware error code, the entry stub may add a synthetic value to normalize its software stack layout. That synthetic slot belongs to the stub's convention. It must be removed consistently before the architectural return instruction consumes the frame.

Exercise 4.1.6 — Return

Which instruction restores the architectural return frame?

Worked solution

In the long-mode model used in this chapter, `iretq` consumes the architectural return frame and restores the saved instruction pointer, code selector, flags and stack state, subject to architectural validity checks. Any handler-pushed general registers and any error-code slot must first be removed according to the entry stub's layout. A plain `ret` reads a function return address and cannot restore the complete interrupt return state.

Exercise 4.1.7 — Frame bytes versus alignment

A no-error-code event uses a selected RSP ending in 0x8. How many bytes are fixed frame fields, and how many bytes does the active RSP move in total?

Worked solution

The five fixed long-mode return-frame fields occupy $5 \cdot 8 = 40$ bytes. The selected stack pointer initially ends in `0x8`; aligning it downward to a 16-byte boundary first moves it by 8 bytes. With no error code, the total downward movement is therefore $8 + 40 = \boxed{48}$ bytes. Only 40 of those bytes are fixed saved fields. The other eight are alignment space, which must not be counted as another saved register.

Summary

Exceptional control flow is ordinary system behaviour viewed at the right scale. The critical questions are which event occurred, what state was saved, which privilege and stack are active, which younger effects were cancelled, and where execution may resume. Recording those facts makes the transfer to the handler and the eventual continuation fully traceable.

4.1.9 Exercises

Exercise 4.1.8 — Exception taxonomy

Classify architecturally: (a) a timer request, (b) native x86-64 `syscall`, (c) a not-present page that can be loaded, (d) an unrecoverable machine-check condition. State explicitly whether each item is an external interrupt, a fault/trap/abort exception, or a different protected-entry mechanism.

Go to the solution →

Exercise 4.1.9 — IDT arithmetic

An IDT begins at `0x4000`. Descriptors are 16 bytes. Find the descriptor addresses for vectors 6, 13 and 32.

Go to the solution →

Exercise 4.1.10 — Return address

Why does a recoverable page fault save the faulting instruction address, while a trap saves the following address?

Go to the solution →

Exercise 4.1.11 — Precise state

Instructions I_1 through I_5 are in order. I_3 is an ordinary scalar instruction with no architecturally defined partial-progress behaviour and faults before completion. State which instructions may have committed when the handler observes a precise exception.

Go to the solution →

Exercise 4.1.12 — Privilege check

Consider an IDT-based software-interrupt service gate, distinct from native SYSCALL. It is callable by software only when the architecture's privilege rule permits the caller. Explain why setting a system-call gate callable from user mode does not let user code choose an arbitrary kernel address.

Go to the solution →

Exercise 4.1.13 — Nested interrupt reasoning

A handler disables maskable interrupts for 40 microseconds. A periodic device needs service at least every 25 microseconds or its one-entry buffer overflows. What failure is possible?

Go to the solution →

Exercise 4.1.14 — Stack-frame accounting

In 64-bit mode, compare two events starting from a selected stack pointer ending in `0x8`: (a) an interrupt without an error code, and (b) a page fault with an error code. The software stub then saves nine 64-bit general registers. Give the fixed hardware-frame bytes, the alignment loss, and the total stack movement before local variables for each case.

Go to the solution →

Exercise 4.1.15 — Design question

Give an ordered plan for handling a fault detected in the memory stage of a five-stage in-order pipeline while preserving precise state.

Go to the solution →

4.1.10 Solutions

Exceptional Control Flow and Protected Entry

Solution 4.1.8

(a) External interrupt: asynchronous to the instruction stream. (b) Dedicated protected-entry instruction: native x86-64 `syscall` is not classified by Intel as a fault, trap or abort exception. (c) Fault: synchronous and potentially restartable once the page becomes present. (d) Abort-class machine-check exception: under the stated unrecoverable condition, reliable restart is not assumed.

← Return to the exercise

Solution 4.1.9

An IDT entry is selected by vector number, so the byte displacement is $16v$. For vector 6, displacement $96 =$ `0x60` gives `0x4060`. For vector 13, displacement $208 =$ `0xD0` gives `0x40D0`. For vector 32, displacement $512 =$ `0x200` gives `0x4200`. These are descriptor addresses. The processor then decodes the selected descriptor to obtain the handler entry and associated control information.

← Return to the exercise

Solution 4.1.10

After the page is made resident, the memory reference must execute again; saving the following address would skip it. A trap is reported after the triggering instruction completes; for a single-step trap, that instruction may be an ordinary arithmetic operation. The saved continuation normally follows it, subject to the vector's exact rules.

← Return to the exercise

Solution 4.1.11

I_1 and I_2 must have completed architecturally before this fault is exposed. I_3 must not have committed its architectural effects, and neither I_4 nor I_5 may have committed. Speculative internal results may exist but must be discarded or remain invisible.

← Return to the exercise

Solution 4.1.12

User code supplies the vector number to `int`, and the processor locates that vector's descriptor in the protected IDT. For explicit software invocation, the gate DPL must permit the caller: CPL must be numerically no greater than DPL. Thus a CPL 3 caller can use a DPL 3 service gate, while a DPL 0 gate rejects that invocation.

Worked solution (continued)

The descriptor supplies the fixed handler offset and code-segment selector. The caller cannot substitute another kernel address, modify the IDT, or choose the destination privilege independently. After the processor validates the gate and establishes the required entry state, the kernel handler validates the requested service and arguments. Granting permission to invoke one gate therefore grants access to that defined entry path.

← Return to the exercise

Solution 4.1.13

Let interrupts be disabled at $t = 0$ and re-enabled at $t = 40\,\mu s$. An event arriving just after $t = 0$ occupies the device's one-entry buffer. A second periodic event can arrive just after $t = 25\,\mu s$, while the first remains unserviced. The device must then discard or overwrite an event, or signal overflow, according to its interface. The nearly $40\,\mu s$ masking interval exceeds the $25\,\mu s$ service limit by almost $15\,\mu s$, even before handler-entry overhead. Shorter masked regions or sufficient additional buffering are needed to meet this device's requirement.

← Return to the exercise

Solution 4.1.14

Hardware first rounds the selected RSP down by 8 bytes. Case (a) has five fixed qwords, so $40 + 8 = 48$ bytes of hardware-side stack movement; adding nine saved registers gives $48 + 72 = 120$ bytes. Case (b) adds the page-fault error code, so $48 + 8 = 56$ hardware-side bytes; with the same software saves the total is $56 + 72 = 128$ bytes. The fixed architectural frames are 40 and 48 bytes respectively; alignment is deliberately reported separately.

← Return to the exercise

Solution 4.1.15

Assume an ordinary faulting instruction with no permitted partial progress, valid handler-entry state and a pipeline extended to enter handlers.

1. Record the fault code and instruction address with the instruction in M. Suppress its normal register or memory result.
2. Disable the younger E-stage instruction's flag update immediately and prevent all younger register and memory writes. Replace younger pipeline entries with bubbles.
3. Allow the older W-stage instruction to finish, so the visible state contains every instruction before the fault and no later effects.

Worked solution (continued)

4. Select and validate the handler entry, privilege and destination stack required by the architecture. Save the specified restart state and error information.
5. Redirect Fetch to the handler and resume normal issue from that entry. A recoverable fault can later return to the saved faulting instruction.

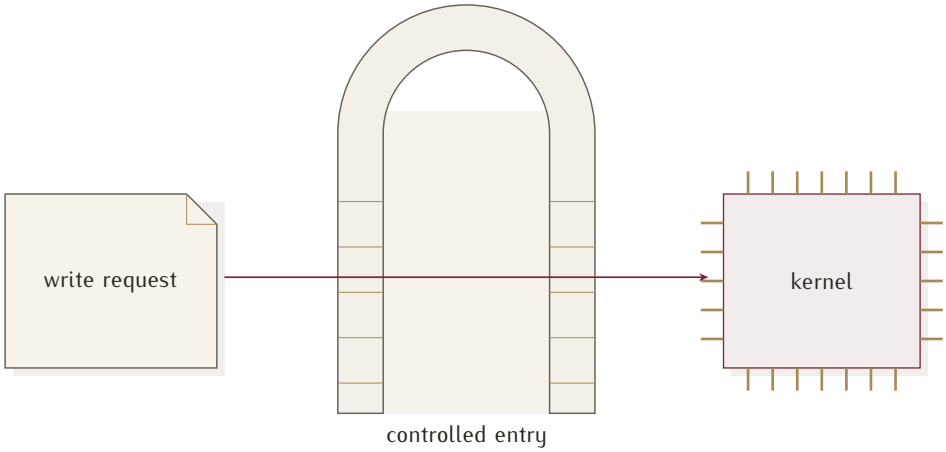
Base Y86-64 stops on ADR or INS; the handler-selection and saved-frame machinery in the last two steps is an additional architectural mechanism.

← Return to the exercise

4.2 SYSTEM CALLS: CONTROLLED ENTRY TO THE KERNEL

The application has chosen a file, a buffer and a length. The kernel now has to interpret those user-supplied arguments under the caller's permissions and the current process state. This chapter follows the protected entry and the work behind it: establish a kernel context, validate the request, perform the permitted operation and return a result. The register-level interface from Chapter 2.5 becomes part of a complete service path.

A request becomes privileged work through a defined interface



Arguments identify an operation; validation establishes what may be performed.

Figure 4.10: *The system call carries a request across the execution boundary. Kernel work then resolves the referenced object, checks the operation and handles progress or failure. The interface separates permission to request an action from its successful completion.*

4.2.1 From a user request to protected kernel execution

Chapter 2.5 described the caller-visible Linux x86-64 system-call ABI. Here we follow the same contract from the other side. Native long-mode code places the system-call number in `rax`, supplies up to six integer or pointer arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9`, and receives the raw result in `rax`. The fourth argument differs from the ordinary System V function convention because the architectural `syscall` instruction

itself overwrites `rcx` with the user return instruction pointer and `r11` with saved flags. That register assignment is part of the Linux x86-64 system-call ABI; the implementation that eventually services the request is a separate matter.

The request passes through four interfaces with different responsibilities. A source program may call the POSIX-like API `write(fd,buf,n)` through a C library wrapper that obeys the ordinary function-calling convention. The wrapper then marshals the request according to the system-call ABI and executes the architecture-specific entry instruction. Only after protected entry does kernel code validate the request and invoke the relevant service implementation. Their responsibilities are:

Layer	Example	Contract
Application API	`write(fd,buf,n)`	types and documented behaviour
Library ABI	ordinary call to libc	function registers, stack and preserved state
System-call ABI	`rax`=number; arguments in fixed registers	user/kernel entry convention
Kernel service	file/VFS/device implementation	validation, policy and resource management

A binary that issues raw Linux system calls is therefore tied to that kernel ABI even if its source-level intent resembles a portable API. Conversely, a portable source program can be rebuilt for another operating system whose service numbers, argument registers and entry instructions are different.

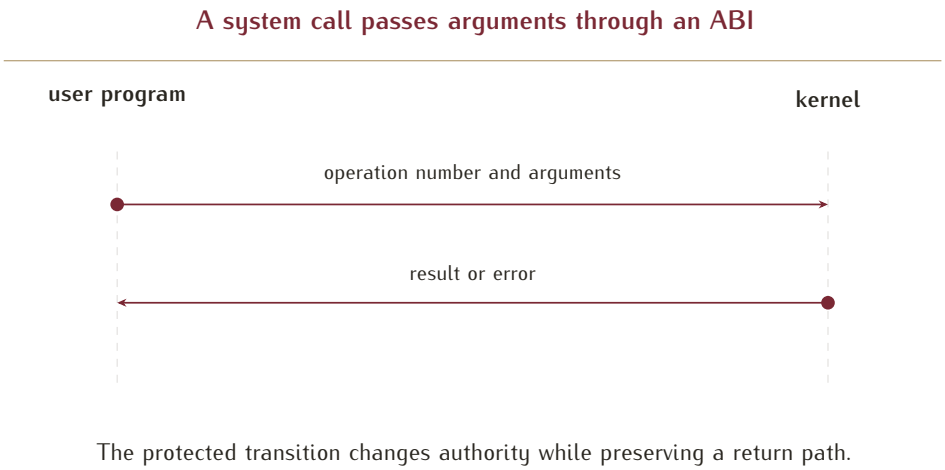


Figure 4.11: *The interface carries values with defined machine locations and meanings. Kernel code must interpret them under the system-call contract and validate any referenced user memory. Return values use the documented ABI convention.*

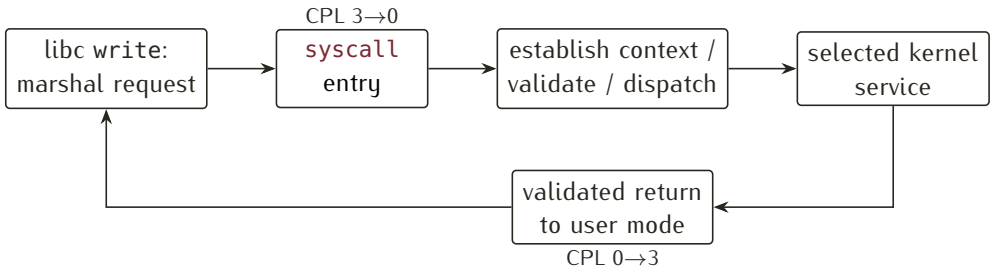
The word *trap* requires care here. Operating-systems literature often uses it informally for a deliberate synchronous request into the kernel, but native x86-64 `syscall` is not an Intel trap exception and does not enter through an IDT trap gate. Processor exceptions and external interrupts use the architecture's exception/interrupt machinery; native `syscall` obtains its entry state from model-specific registers and follows a dedicated transition. Both mechanisms control a transfer of authority, with distinct entry-state rules.

User and kernel execution share the same silicon

The processor changes from user execution to kernel execution. Privilege and control state change the permission checks, mappings and legal behaviours applied by the same execution hardware; data may remain in the same caches and physical structures while its architectural accessibility changes. Protection constrains access through machinery shared by user and kernel execution. This is one reason timing and cache side channels can cross a boundary even when direct architectural access is forbidden [26, 50].

4.2.2 Following `write` across the boundary

Consider `write(1,buf,n)`. At the API level the call asks to transfer up to n bytes from the caller's memory to the object referenced by descriptor 1. The wrapper places the service number in `rax`, the descriptor in `rdi`, the user address in `rsi` and the count in `rdx`, then executes `syscall`. The processor redirects control to the configured kernel entry point and changes privilege state; architecture-specific entry code must then establish the trusted execution context that ordinary C kernel code expects before dispatch can proceed [26, 33, 34].



Hardware entry and software context

Native 64-bit `syscall` puts the address following the entry instruction in RCX and saves RFLAGS in R11. It selects the kernel entry through configured model-specific registers and applies the configured flag mask. Hardware does not push an IDT return frame or load a kernel RSP. Entry software must preserve the user stack pointer, establish a trusted kernel stack and save the wider context before using ordinary kernel calling conventions. RCX holds the return address. Software saves any additional register state required by its entry convention.

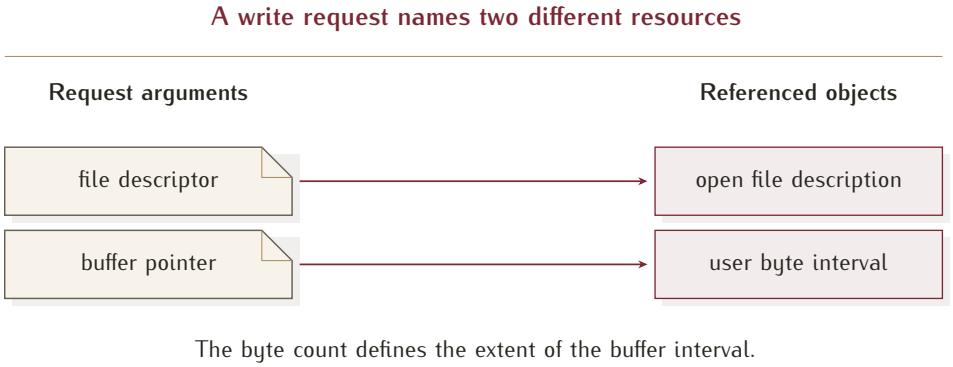


Figure 4.12: *The descriptor resolves through process-owned references to an open object. The pointer and length identify source bytes in user memory. Validating one does not establish that the other is usable.*

In the Linux source snapshot followed by this book, the native x86-64 path reaches common entry code, validates the system-call number and dispatches through generated x86-64 syscall machinery; an unsupported number yields `-ENOSYS`. The exact source symbols and generated tables are implementation details of a particular kernel revision, whereas the caller-visible register convention is the ABI boundary. Keeping that distinction explicit lets the reader use source code without mistaking one release's dispatcher for part of the architectural contract [33, 34, 35].

Once the service has been selected, descriptor 1 is resolved in the calling process's descriptor table. It is conventionally standard output, but the integer does not mean "terminal": the descriptor may refer, through an open file description, to a terminal, regular file, pipe, socket or another object. The kernel checks the descriptor and access mode, treats `buf` as an untrusted user address, validates the range, and follows the object-specific path needed to transfer data. Depending on the object and interface, data may be copied immediately, buffered, pinned for later transfer or transformed into another subsystem request. The service contract is realised through the operations supported by the referenced object.

Direct Linux x86-64 `write`

Assume the label `message` identifies the six readable bytes `hello\n`. This fragment uses GNU assembler AT&T syntax:

```
mov $1, %rax          # __NR_write
mov $1, %rdi          # descriptor 1
lea message(%rip), %rsi
mov $6, %rdx
syscall
# rax: byte count, or a negative -errno value
```

Worked example (continued)

The four instructions before `syscall` are not themselves privileged. They merely construct the request in registers. Privilege changes only when the architectural entry instruction executes, and even then the kernel has not yet accepted the request: validation remains necessary because every argument came from an untrusted process.

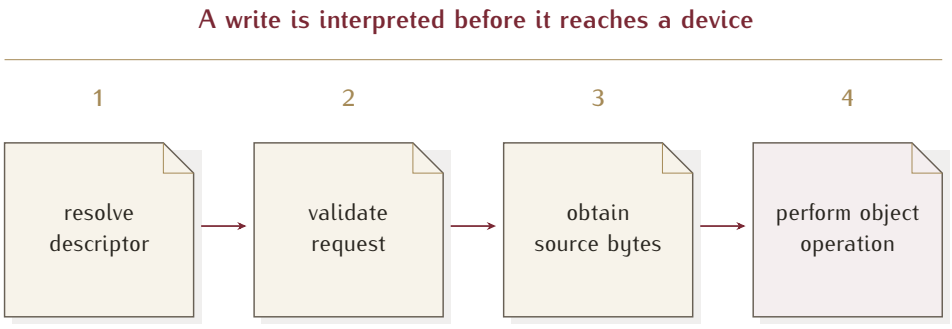


Figure 4.13: *The generic request is specialised by the object behind the descriptor. Buffering, blocking and partial progress depend on that object and the operation's contract. A successful return does not universally imply durable media storage.*

This path also explains why a library routine and a system call should not be treated as synonyms. Functions such as `strlen` and typical implementations of `memcpy` can operate entirely on user-accessible memory. `snprintf` can likewise format into a user buffer without crossing the boundary. By contrast, `write` and `open` normally use wrappers around kernel services. A higher-level routine such as `printf` may buffer output and therefore perform no system call at the point where the source call occurs, or it may cause one or more writes later. Source-level function calls and kernel-entry events consequently have no one-to-one correspondence.

4.2.3 Validation, return values and partial progress

Protected entry establishes kernel authority; request validation determines which operation that authority may perform for the caller. A user pointer is just a number supplied by an unprivileged process, so kernel code must access it according to the operating system's user-memory rules, reject invalid ranges and account for arithmetic overflow when a length is combined with an address. Service numbers require bounds checks, flags must be validated, file descriptors must be resolved in the caller's own descriptor table,

and sizes may need limits before privileged memory is allocated. The general principle is that the kernel executes with kernel privilege but with the caller's authorised rights.

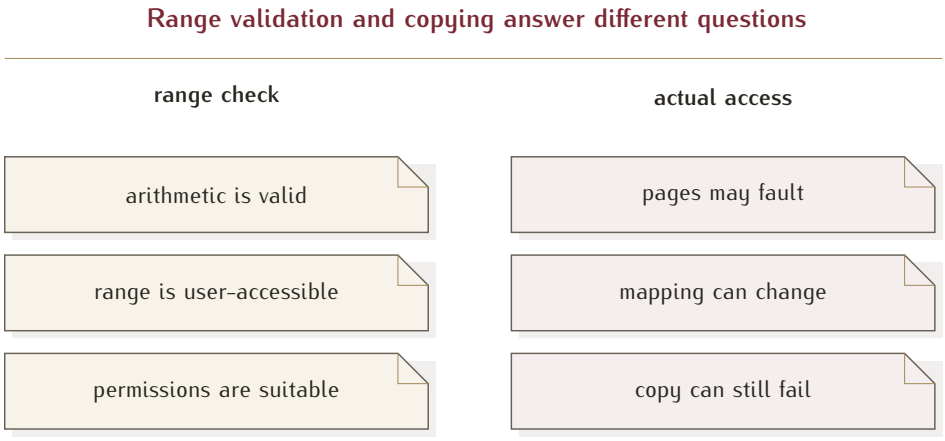


Figure 4.14: *A preliminary check does not eliminate all later access failures. Robust kernel paths use access mechanisms that cope with faults and concurrent changes under the operating system's rules.*

Security consequence: avoid the confused deputy

A privileged service becomes a confused deputy when it uses its own authority on behalf of a caller without checking what the caller is entitled to request. Per-process descriptors, capability-style handles and explicit permission checks bind a request to delegated authority. The system-call boundary is therefore both an ABI boundary and a policy-enforcement point.

The raw Linux ABI also differs from the C library interface when an error is returned. At the kernel boundary, an error occupies the reserved range −4095 through −1 in `rax`. A libc wrapper recognises that range, returns −1 according to the C API and stores the positive error number in thread-local `errno`. The value of `errno` does not establish that the most recent call failed: a successful function need not clear stale error state.

Successful I/O does not always mean that the requested byte count was completed. A read or write may make partial progress because of pipe capacity, non-blocking mode, signal delivery, device behaviour or another interface-specific condition. The caller must account separately for completed bytes, interruption and failure.

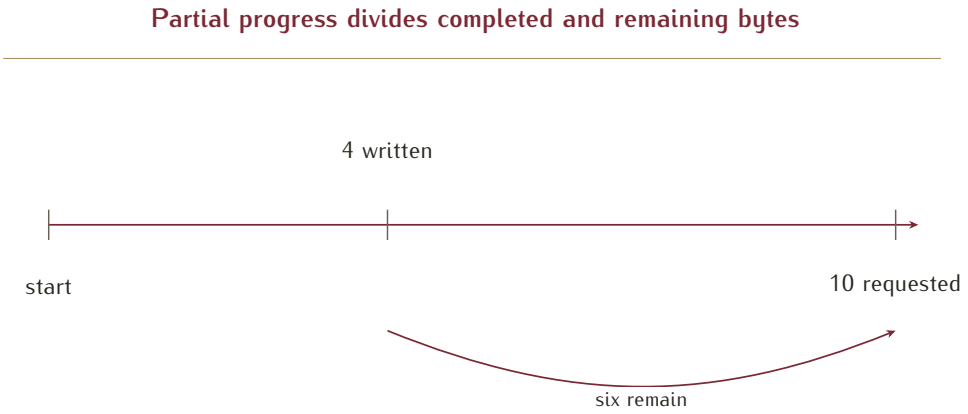


Figure 4.15: *After a positive result of four, the caller has evidence for that completed prefix. A retry loop must advance both its pointer and its accounting, avoiding retransmission of the already completed bytes.*

A robust output loop

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <limits.h>
#include <unistd.h>

ssize_t write_all(int fd, const void *buf, size_t n) {
    if (n > (size_t)SSIZE_MAX) {
        errno = EOVERFLOW;
        return -1;
    }
    const unsigned char *p = buf;
    size_t done = 0;

    while (done < n) {
        ssize_t rc = write(fd, p + done, n - done);
        if (rc > 0) {
            done += (size_t)rc;
            continue;
        }
        if (rc < 0 && errno == EINTR)
            continue;
        if (rc == 0) errno = EIO; /* No progress with bytes remaining. */
        return -1;
```

Worked example (continued)

```
        }
        return (ssize_t)done;
    }
```

The caller supplies a readable `n`-byte buffer for the duration of the call. The function rejects counts that cannot fit its `ssize_t` return, accumulates partial writes and retries EINTR. Zero progress with bytes remaining becomes EIO; EAGAIN and other errors return immediately. A failure may follow a written prefix: the function neither rolls that prefix back nor reports its length, so blindly retrying the original buffer can duplicate data. If a broken pipe generates SIGPIPE with its default disposition, the process can terminate before an EPIPE return is observed. The application's signal policy is therefore part of this output interface.

Return values require deliberate classification

	outcome	meaning	caller action
positive	progress made	that many bytes	advance
zero	no progress	contract dependent	handle explicitly
error	failure indication	inspect error	decide retry or stop

This classification is for count-returning I/O interfaces.

Figure 4.16: *A caller must distinguish partial progress, zero progress and an error indication. Retrying every outcome unconditionally can loop forever or duplicate work. The precise rule depends on the interface and error code.*

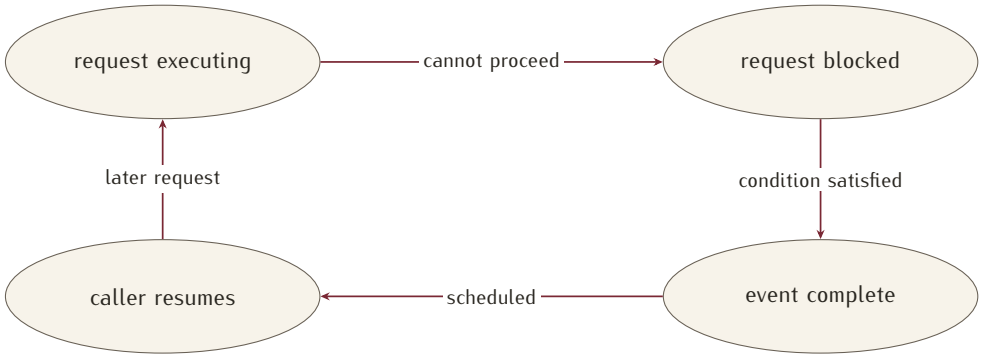
For large buffers, the physical cost of moving data may dominate the fixed cost of the privilege transition. Copying touches cache lines, consumes bandwidth and may move the same payload through several levels of the memory hierarchy; zero-copy or shared-buffer interfaces attempt to change mappings or ownership instead of reproducing every byte, although they introduce their own lifetime and synchronisation constraints. An I/O performance analysis separates entry overhead, software processing, device waiting and data movement [38, 50].

4.2.4 When a system call does not return immediately

A system call may finish quickly, but it may also block because the requested condition is not yet true. A positive-count read from an empty blocking pipe with at least one writer still open is a simple example: the kernel cannot manufacture data, so the calling task becomes waiting rather than continuing to occupy a logical processor. Another runnable task may execute, and when a writer later supplies data the blocked task becomes runnable again. Eventually it resumes inside the kernel, completes the pending operation and only then returns to user mode. What looked in source code like one ordinary function call can therefore contain a scheduler decision and one or more context switches.

Signals make the return path less uniform still. Depending on the interface, the point at which interruption occurs and the installed signal-action flags, an operation may be restarted or may return an interruption error that user code must handle. A successful entry therefore does not imply one fixed exit instruction or even a return to the same user-space instruction stream: a system call can block, restart, deliver a signal, replace the process image through `exec`, or terminate the process. Architecture-specific return code uses a fast path such as `sysretq` only when its safety conditions are satisfied; otherwise a more general return path is required.

Waiting releases the processor to other runnable work



An event can make the caller ready before it actually runs.

Figure 4.17: *The blocked state records a dependency on an event or resource. Completion removes that dependency, while the scheduler chooses when the caller gets processor time. The transition preserves the execution context needed to continue.*

The service path therefore includes scheduling as well as request processing. The blocked thread retains its kernel continuation while another runnable thread uses the

processor. Process creation, program replacement and waiting use the same protected-entry machinery to change the set of available execution contexts.

4.2.5 Mechanism, policy and operating-system structure



Figure 4.18: *The layers identify responsibilities within the path. Architectural entry supplies controlled transfer; the operating system then applies validation and policy before carrying out the requested object-specific action.*

The system-call path also illustrates a broader distinction that will recur throughout the operating-system material. Hardware entry, context saving, descriptor lookup and queue submission are *mechanisms*: they provide controlled ways to perform actions. Choosing who may open an object, which task should run next or how requests should be ordered for fairness is *policy*. Separating mechanism from policy makes alternatives easier to compare and limits how much implementation must change when requirements change.

The location of a service is itself a design choice. Monolithic kernels keep many services in the privileged address space, microkernels move more functionality into isolated processes and communicate through messages, while other designs draw different boundaries. Isolation can reduce fault propagation but increase communication and transition costs; placing more code in the kernel can make common paths efficient while enlarging the trusted computing base. Kernel structure must satisfy the system's failure, performance and maintenance requirements [59].

Laboratory 25: Library calls and kernel events

Predict which calls in this complete program may involve kernel work, then explain why a trace can differ with redirected output.

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv) {
    const char *s = argc > 1 ? argv[1] : "abc";
    if (printf("length=%zu\n", strlen(s)) < 0) return 1;
    if (fflush(stdout) == EOF) return 1;

    const char marker[] = "X\n";
    size_t done = 0;
    while (done < sizeof marker - 1) {
        ssize_t n = write(1, marker + done,
                          sizeof marker - 1 - done);
        if (n > 0) done += (size_t)n;
        else if (n < 0 && errno == EINTR) continue;
        else return 1;
    }

    if (printf("pid=%ld\n", (long)getpid()) < 0) return 1;
    char *p = malloc(100);
    if (!p) return 1;
    p[0] = 'a';
    int rc = printf("%c\n", p[0]);
    free(p);
    if (rc < 0 || fflush(stdout) == EOF) return 1;
    return 0;
}
```

Worked solution

Laboratory 25: solution

For successful execution with the default input, the output lines are `length=3`, `X`, `pid=` followed by the process ID, and `a`, in that order. The explicit `fflush(stdout)` before the descriptor write coordinates the two handles of the same output object, as required when switching from buffered stream output to direct descriptor output [4]. The loop completes the marker despite short writes or EINTR; other failures terminate the program.

Worked solution (continued)

`strlen` computes the length in user memory. `write` requests kernel output. In glibc since 2.25, `getpid` invokes the system call rather than using the older PID cache [32]. `printf` formats in user space and writes when its buffer is flushed. `malloc` may reuse allocator space or request more memory; `free` likewise need not enter the kernel.
Save the program as `demo.c`, then run:

```
gcc -std=c11 -O0 -Wall -Wextra demo.c -o demo
strace -o terminal.trace -e trace=write,getpid,brk,mmap,munmap ./demo
strace -o file.trace -e trace=write,getpid,brk,mmap,munmap ./demo > output.
    ↪ txt
```

On a terminal, the final two newline-terminated printf calls normally flush separately. With stdout redirected to a regular file, they can share one buffered write at the final fflush. The first line is flushed explicitly in both cases, so it remains before X. Compare the write payloads and counts in the two traces; startup may also contribute memory-management calls. Rebuilding with optimisation can remove allocation work whose only observable result is unchanged, so the trace describes a particular executable and libc configuration.
Return to the laboratory

From monitor requests to the Linux system-call path

Timeline

Year	Machine, standard or event	Architectural significance
1950s	**Monitor request**	A job invokes a protected service instead of controlling devices directly.
1969	**UNIX**	UNIX begins developing its file and process interface.
2003	**x86-64 Linux entry**	`syscall`, ABI registers and kernel entry code form a layered path.

Why protected service requests appeared

Early shared computers needed resident supervisory software to sequence jobs and operate devices, but a user program could not safely be given the same authority as that supervisor. Designated supervisor transfers solved the problem by letting ordinary code request a controlled change of execution state without directly acquiring arbitrary control over devices or memory protection. IBM System/360's `SVC`, time-sharing supervisors and Multics protection mechanisms all expressed versions of the same require-

ment: unprivileged computation needs services whose implementation must remain protected from the requester [109, 63, 117, 94, 60].

UNIX made the interface compact and composable. File descriptors gave ordinary files, terminals and pipes a common byte-oriented handle, while `fork` and `exec` separated process creation from program replacement. Linux inherited that programming model while defining its own system-call ABI and kernel implementations [19, 35].

From general interrupt machinery to dedicated fast entry

Historically, 32-bit x86 Linux commonly used `int $0x80`, entering through the architecture's general interrupt mechanism. Later processors introduced faster dedicated entry instructions, including `SYSENTER` and AMD64 `SYSCALL/SYSRET`. In native x86-64 Linux, the system-call number occupies `RAX` and the first six integer or pointer arguments use `RDI`, `RSI`, `RDX`, `R10`, `R8` and `R9`; `R10` is used for the fourth argument because the architectural transition consumes `RCX` and `R11` for return state [26, 27, 30, 32].

Modern optimisation has changed how often software crosses this boundary without removing its purpose. A vDSO can satisfy selected queries from kernel-maintained user mappings, while interfaces such as `io_uring` batch submission and completion to reduce repeated transitions. Validation and privileged effects nevertheless remain kernel responsibilities. The resulting service path connects API semantics, ABI register placement, architectural entry, kernel validation and object-specific implementation [33, 35, 32].

The architecture provides a controlled entry; the operating system defines the service contract and policy that make that entry useful.

4.2.6 Worked problems

A request and its error result

For raw Linux x86-64 `write(1,p,12)`, place 1 in RAX, 1 in RDI, p in RSI and 12 in RDX. Suppose the kernel reports EFAULT, whose Linux x86-64 value is 14. The raw return is −14, represented by `0xFFFFFFFFFFFFFFF2`. A conventional libc wrapper recognises the reserved error range, stores 14 in thread-local `errno`, and returns −1. A successful return of 12 instead reports twelve bytes transferred and gives no reason to inspect stale `errno`.

Partial read

A positive return means that many bytes are valid, even when it is fewer than requested.
Question. A read requests 4096 bytes and returns 700. How many buffer bytes are valid, and is the short result automatically an error?

Worked example (continued)

Exactly bytes 0 through 699 contain data returned by this call; bytes 700 through 4095 are outside the returned prefix and must not be treated as new input. The return is successful partial progress. The next action depends on the protocol: process those 700 bytes, or continue an exact-length read using a pointer advanced by 700 and a remaining count of 3396. For a positive-count read from an ordinary file or stream, a later zero return indicates EOF; a zero-count request also returns zero and cannot establish EOF.

Exercise 4.2.1 — Fourth arg

Register for the fourth argument?

Worked solution

The fourth Linux x86-64 syscall argument is in `r10`. The first three are `rdi`, `rsi` and `rdx`; the fifth and sixth are `r8` and `r9`. This register assignment belongs to the syscall ABI. A C-function wrapper receiving its fourth integer argument in `rcx` must adapt it before issuing `syscall`.

Exercise 4.2.2 — EOF

Return value indicating EOF from read?

Worked solution

For a positive-count read of an ordinary file or stream, zero is the successful EOF result: no bytes were returned. A positive value is the actual byte count, and a negative raw syscall result indicates an error. A zero-count request can also return zero, so the requested count must be known before interpreting zero as EOF. Only the returned number of bytes should be processed.

Exercise 4.2.3 — Dispatch

What must be checked before using a user-supplied syscall number in a table-based dispatcher?

Worked solution

In the stated table-based design, check that the number lies within the table bounds, then dispatch to an implemented service or an unsupported-service stub according to that entry. Apply the required entry policy as well. Only then use it as an index. An unchecked user-controlled index could select unrelated memory as a

Worked solution (continued)

function pointer. The generated native Linux x86-64 dispatcher discussed earlier need not implement dispatch as this function-pointer table. A rejected number produces the defined unsupported-call error path. Argument validation remains the responsibility of the selected service and its supporting code.

Exercise 4.2.4 — Wrapper

Does strlen require a syscall for an ordinary in-memory string?

Worked solution

For an ordinary accessible string, `strlen` reads user-space memory until the first zero byte and counts the preceding bytes. It can perform that work entirely in user space, so it requires no syscall merely to compute the length. For bytes `0x41 42 00`, it returns 2. A memory fault during those reads is an exceptional event; it does not turn the library's normal length computation into a syscall.

Summary

Follow a call across four boundaries: source API to wrapper, wrapper to register convention, processor entry to kernel handler, and kernel result back to user-level semantics. Errors often arise at the seams: a pointer valid in user space but unsafe to dereference directly, a short transfer mistaken for completion, or a resource lifetime assumed to outlast its descriptor.

4.2.7 Exercises

Exercise 4.2.5 — Register placement

Place the number and arguments for `write(fd,buf,count)` in the Linux x86-64 raw system-call ABI.

Go to the solution →

Exercise 4.2.6 — Fourth argument

An ordinary function receives its fourth integer argument in `rcx`. Why must a raw system-call wrapper move it to `r10`?

Go to the solution →

Exercise 4.2.7 — Wrapper error translation

A raw call returns −13 in `rax`. Describe the conventional C wrapper result.

Go to the solution →

Exercise 4.2.8 — Short write

A program asks to write 1000 bytes. The first call returns 640, the second 360. What pointer and length should be used for the second call?

Go to the solution →

Exercise 4.2.9 — Library versus service

Classify each operation as normally pure user-space or kernel-mediated: `strlen`, opening a file, formatting an integer into an existing buffer, obtaining the process ID.

Go to the solution →

Exercise 4.2.10 — Bounds validation

A generated dispatcher recognises service numbers 0 through 449. What must happen for service number 450? Why is checking only for a negative number insufficient when the number is treated as unsigned?

Go to the solution →

Exercise 4.2.11 — Blocking path

A process requests a positive number of bytes from an empty blocking pipe while at least one writer remains open. Explain why the eventual return from the same call may occur after other processes have executed.

Go to the solution →

Exercise 4.2.12 — Security design

A service copies `len` bytes from user pointer `p`. List four independent checks or precautions.

Go to the solution →

4.2.8 Solutions

System Calls: Controlled Entry to the Kernel

Solution 4.2.5

The raw Linux x86-64 interface places syscall number 1 in `rax`, descriptor `fd` in `rdi`, buffer address `buf` in `rsi`, and byte count `count` in `rdx`, then executes `syscall`. The return value replaces `rax`. A nonnegative result counts bytes written; a negative raw result reports an error number with a minus sign. The caller must also account for `rcx` and `r11` being clobbered by the instruction.

← Return to the exercise

Solution 4.2.6

The ordinary function convention delivers argument four in RCX, but `syscall` overwrites RCX with the user continuation address before the kernel reads the request. The Linux syscall convention therefore places argument four in R10. A wrapper uses `mov r10, rcx` before entry; the kernel then reads R10 as that argument while RCX serves its architectural return-state role. The wrapper must also account for R11 being overwritten with saved flags.

← Return to the exercise

Solution 4.2.7

The wrapper recognises a negative kernel error, sets `errno` to 13 and returns -1 to the caller. On the stated Linux x86-64 system, 13 is EACCES.

← Return to the exercise

Solution 4.2.8

The first call reports that bytes 0 through 639 have been written. Advance a byte pointer from `buf` to `(const unsigned char *)buf + 640` and reduce the request to $1000 - 640 = 360$ bytes. The second call then writes bytes 640 through 999, and $640 + 360 = 1000$ completes the original request. Using the original pointer with the remaining count of 360 would repeat bytes 0 through 359 and leave the intended final 360 bytes unwritten.

← Return to the exercise

Solution 4.2.9

`strlen` and local formatting are pure user-space computations. Opening a file and obtaining a kernel process identifier require services, although a library wrapper may present them as ordinary functions.

← Return to the exercise

Solution 4.2.10

The accepted set is $0 \leqslant n < 450$, so 450 must take the unsupported-service path, conventionally raw `-ENOSYS` on Linux. For an unsigned service number, the condition `n < 0` is always false. It also cannot detect large positive values or a negative source value converted to unsigned. An upper-bound test such as `n >= 450` rejects all values outside the stated range before any service-specific dispatch. A table implementation must apply this check before indexing; generated branch or switch dispatch must provide the same rejection semantics.

← Return to the exercise

Solution 4.2.11

The kernel puts the caller on a wait queue and schedules another runnable task. When a writer supplies data, the reader is awakened, later scheduled, finishes the kernel operation and returns to user mode. The source call spans one or more context switches.

← Return to the exercise

Solution 4.2.12

Four independent requirements are:

1. Bound len by the service limit and the capacity of the kernel destination. A valid user buffer can still be too large for the requested operation.
2. Check address arithmetic without wrapping. First require $p \leqslant U_{max}$. For a nonzero length, the last-byte calculation must satisfy $len - 1 \leqslant U_{max} - p$ before forming $p + len - 1$, where U_{max} is the largest permitted user address.
3. Require the entire source range to lie in permitted user address space and apply the service's object and access permissions.
4. Use the operating system's protected user-copy primitive and handle its failure or incomplete-copy result. A preliminary range check cannot guarantee that every later access succeeds: mappings or contents can change concurrently.

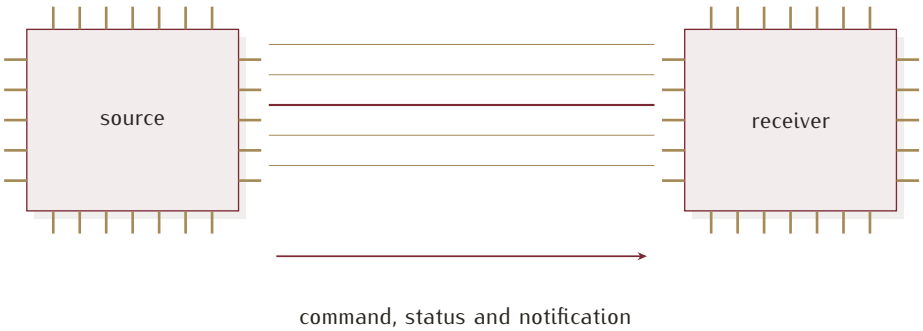
For structured requests, copy control fields into kernel-owned storage and validate the captured values before using them. Re-reading a user length after checking it could invalidate the bound. The exact copying and consistency protocol belongs to the service contract.

← Return to the exercise

4.3 DEVICE CONTROLLERS, POLLING, AND INTERRUPTS

The processor has asked a device to do work. While the device progresses, the processor can repeatedly inspect status or arrange to be notified when attention is needed. Both choices depend on a controller that exposes a precise interface. Following a request from command to completion reveals the roles of status bits, buffering, acknowledgements and error handling.

A controller exposes signals that software can interpret



The interface specifies what each signal means and when it may change.

Figure 4.19: *A controller interface translates software-visible operations into device work. The parallel traces distinguish conceptual signals; the actual transport can instead use mapped registers, messages or descriptor queues with specified semantics.*

4.3.1 Controller interfaces and notification

A keystroke reaches software through a device controller. Software reaches the device through a controller interface, using port-mapped I/O or loads and stores to memory-mapped registers and buffers according to the platform. Between the controller and the CPU sit two common strategies for noticing that a datum is ready: *polling* and *interrupts*. The discussion builds the path outward from the controller to software notification and service.

An I/O device exposes a *controller* or endpoint, often integrated into the device itself, that speaks the device's electrical protocol on one side and exposes a set of registers to the CPU on the other. Conceptually the controller offers three kinds of register:

- a *status* register, whose bits report the device's state (for example, "a datum is ready" or "the device is busy");
- a *command* (or control) register, into which the CPU writes to tell the device what to do;
- a *data* register, through which the actual datum is exchanged.

The set of commands the controller accepts, their formats and codes, together form the controller's *programming interface*. Programming a device means writing to its command register and reading its status and data registers in the right order.

Worked example

Problem. In a legacy PS/2 controller configured to deliver keyboard data, with no pending command response or auxiliary-device byte, which ports deliver the next scan-code byte?

Solution. The status/command port is `0x64` and the data port is `0x60`. To obtain the next byte the CPU first *polls* the status register at `0x64`, checking the "output buffer full" bit (bit 0); when it is set, the CPU reads a byte from data port `0x60`. A real driver also checks status error bits and routes auxiliary-device or command-response bytes appropriately. A key transition can require several scan-code bytes.

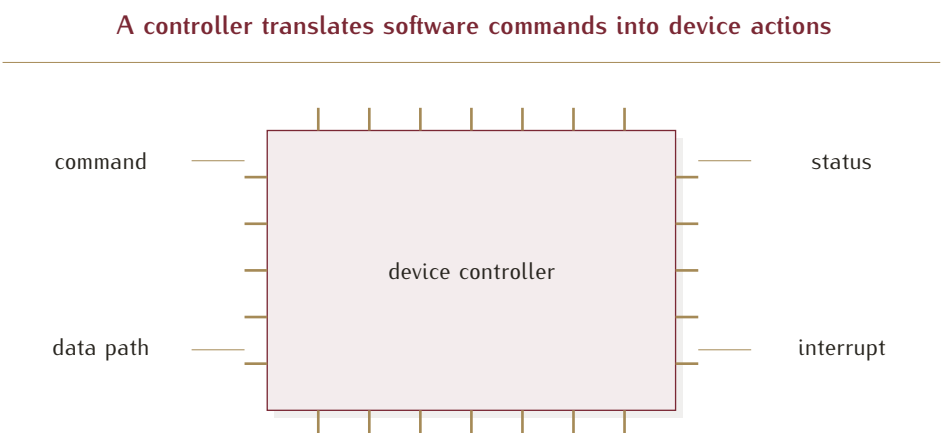
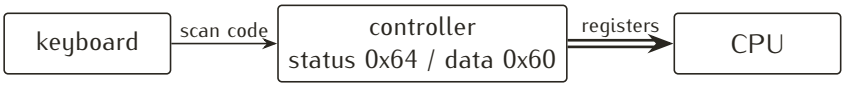


Figure 4.20: *Software interacts with controller-defined registers, queues or descriptors. The controller performs device-specific work and exposes progress through status or completion information. Its specification defines access widths, side effects and ordering.*



Device registers can cross voltage and clock domains

A load or store accesses a memory-mapped device register.

The processor request becomes an interconnect transaction that can reach a controller operating at another voltage or clock. Level shifters translate valid voltage ranges. Clock-domain-crossing circuits and handshaking protocols transfer events or multi-bit data safely, usually adding latency; they do not preserve the exact source-domain timing. The controller may then drive pins, serial links or analogue front ends.

MMIO accesses traverse device-specific bridges and control paths. Their latency, ordering and side effects follow the mapped register interface [42, 49, 50].

4.3.2 Polling and interrupts

The simplest way to wait for a datum is to loop, reading the status register over and over until the "ready" bit appears. This tight-loop form of *polling* is *busy waiting*. Under the simplified keyboard-data conditions above, the receive sequence is: (1) wait until the status bit says a datum is available; (2) read the datum from the data port.

Busy waiting is simple: the code is a short loop with no extra machinery. The cost is that the executing hardware context remains occupied while it spins. Polling can nevertheless be the right design for very short waits, very high event rates, latency-sensitive queues or a dedicated core; many systems use hybrid interrupt/polling schemes. The

trade-off depends on event rate, latency target and available CPU time. If a key arrives once a second while the loop executes millions of tests, nearly all of the polling work observes no new event.

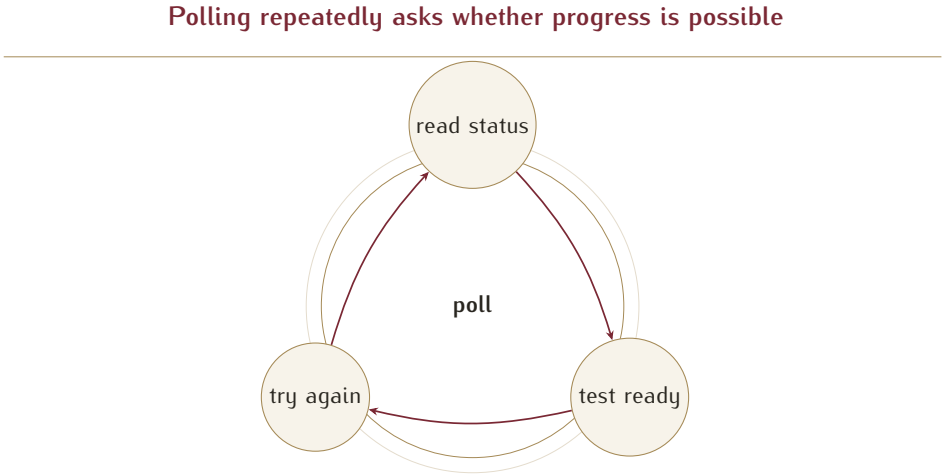


Figure 4.21: *The repeated reads consume processor and interconnect work while waiting. Polling can have low notification latency under suitable conditions, but the cost depends on the interval, workload and device interface.*

Worked example

Problem. A polling loop tests the status bit once per iteration; the device becomes ready exactly at iteration k. How many iterations run in total?

Solution. The loop runs until the bit is set, so it executes k iterations, the last of which finds the datum ready. The first $k-1$ iterations consume processor time without observing a ready device.

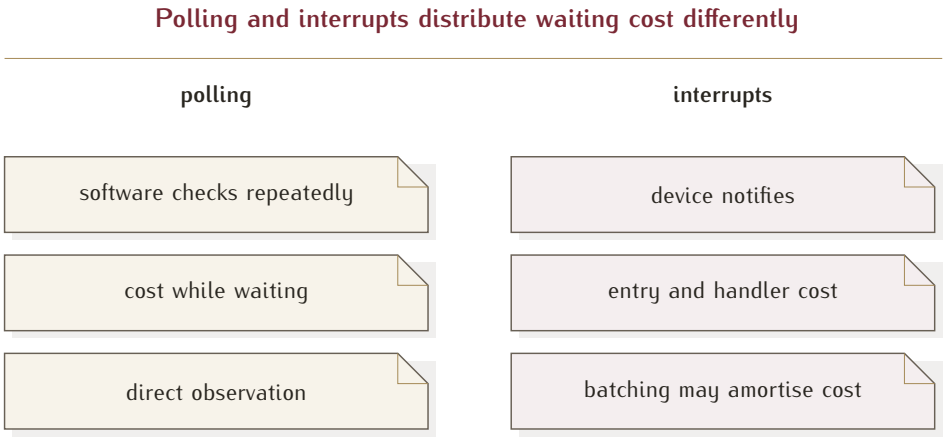


Figure 4.22: *The two strategies change when the processor spends effort discovering completion. Systems can combine them, such as interrupt-triggered batches followed by polling while the device is busy.*

A variant reduces the waste: the CPU polls only at intervals and does other work in between. This lowers the fraction of time spent checking, but it also increases the latency between the arrival of the datum and its handling. A longer checking interval trades lower checking cost for a larger detection delay.

Polling consumes energy while status remains unchanged

A polling loop repeatedly reads a status bit until it changes.

Every iteration fetches and executes instructions and accesses a cache-resident status location or a device register. If the status is remote MMIO, each read can generate interconnect traffic. The loop therefore consumes dynamic power and execution capacity even when successive observations report no change.

Interrupts and low-power wait mechanisms can allow parts of the processor to stop switching until an event occurs. Polling can still be preferable for very short waits or high-rate devices, but its latency benefit is purchased with energy and occupied resources [50, 41].

A device interrupt is asynchronous with respect to the target instruction stream: its arrival can occur independently of the instruction currently executing. The architecture recognises an unmasked pending interrupt at an architecturally permitted boundary, records the state required by the entry mechanism and transfers control through the interrupt-vector machinery. The handler and operating system may save additional context. A return-from-interrupt path later restores an appropriate user or kernel context; whether the interrupted computation resumes, is rescheduled or is terminated depends on the event and on operating-system policy.

Legacy devices signalled an *interrupt request* on a physical IRQ line, which an interrupt controller routed and prioritised before delivery to a processor. The PC/AT arrangement used two cascaded 8259 PICs, providing fifteen usable external IRQ inputs; each controller was programmed with a vector base, so the delivered vector was derived from that base and the input number. Modern PCI devices commonly use message-signalled interrupts instead of dedicated lines, and APIC-based systems route those messages to selected logical processors [26].

Worked example

Problem. A PIC is programmed with base vector 32 and the keyboard raises IRQ 1. What interrupt vector reaches the CPU?
Solution. $32 + 1 = 33$. The CPU indexes the IDT at entry 33 and runs the keyboard handler.

4.3.3 Routing, vectoring and acknowledgement

The x86 *APIC* (Advanced Programmable Interrupt Controller) architecture separates processor-local delivery from routing of external sources. The *I/O APIC* is a hardware controller that receives the interrupts raised by the I/O devices and routes them; a *Local APIC* is associated with each logical processor and accepts and routes interrupt messages for that execution context. The I/O APIC supplies redirection entries for pin-based sources, while local APICs also handle local and inter-processor interrupts. MSI/MSI-X messages can reach the local-APIC delivery path without traversing an I/O APIC; interrupt-remapping hardware may validate and translate them. The exact topology is platform-specific.

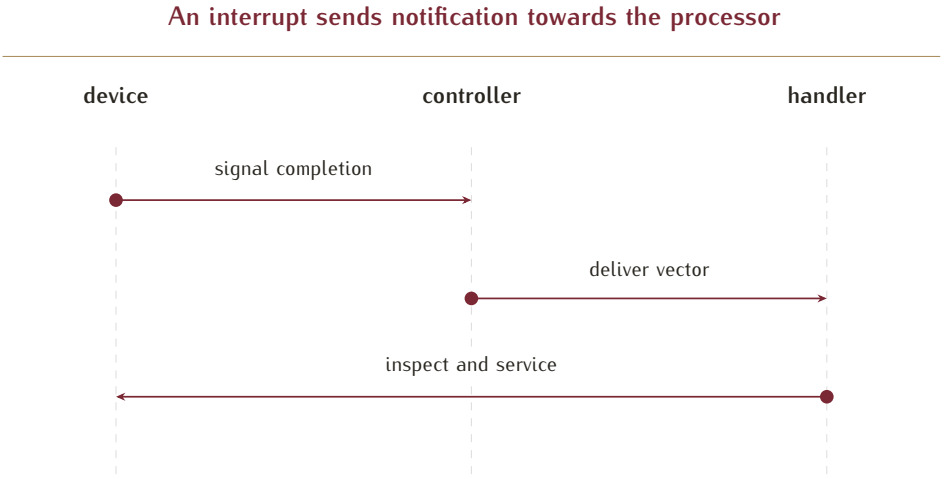
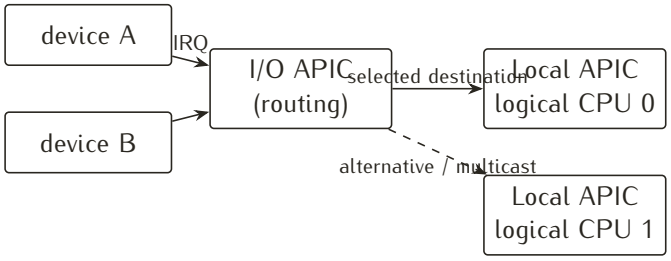


Figure 4.23: *The interrupt is a notification path. It can indicate that device state needs attention, but it is not itself the bulk data transfer. Routing and acknowledgement depend on the interrupt-controller and device contracts.*



Worked example

Problem. Order the steps of a legacy interrupt-driven keyboard read whose pin-based request is routed through an I/O APIC.

Solution.

1. The key press causes the controller to request an interrupt.
2. The I/O APIC routes the request to a selected core's Local APIC.
3. At an eligible instruction boundary, the processor recognises the pending interrupt.
4. The architecture saves the required context and selects the handler through the IDT.
5. The handler reads and acknowledges the device state, passes the event to the operating system and eventually restores an admissible return context.

An interrupt controller routes messages through a hierarchy

The APIC selects a target core and an interrupt vector.

Controller logic arbitrates pending sources, records priority and sends a message through on-chip or platform interconnect. The target core samples and accepts the request only when architectural and priority conditions allow it. Legacy pin- or line-based requests still exist, while many modern devices use encoded message-signalled interrupts; in both cases, buffering and routing remain physical resources.

Interrupt delivery time varies with masking, priority, interconnect contention and core state. The vector-table lookup is only one stage of the complete path [26, 50].

Two sources of exceptional control flow must be distinguished. Processor exceptions such as faults and traps are synchronous with an instruction and were examined in Sections 4.1 and 4.2. External interrupts are raised asynchronously by devices and are examined here. A divide error and a page fault are exceptions; a key press and the completion of a disk write are interrupts. Processor exceptions and external interrupts are dispatched through IDT vectors on x86. Native long-mode system calls, although conceptually deliberate exceptional control transfers, use the separate `syscall` entry mechanism. Origins and saved-state rules must therefore be reasoned about per mechanism.

From a device event to a local-APIC vector and a Linux IRQ handler

The statement "the device raises an interrupt" compresses routing, prioritisation, vector delivery and software dispatch into one sentence.

A pin-based interrupt can be routed through an I/O APIC, whose redirection entry selects destination and vector information. PCI and PCI Express devices commonly use MSI or MSI-X instead: the device issues a specially formatted memory-write transaction whose address and data encode an interrupt message. The local APIC of a target processor accepts the request, applies masking and priority rules, and presents a vector to the core [26, 27].

When the core accepts the interrupt, it indexes the IDT with the vector and enters the corresponding protected handler. Interrupt-gate entry clears IF. Maskable interrupts remain disabled until an authorised instruction re-enables them or a valid return restores an enabled saved IF. The handler eventually signals end-of-interrupt to the local APIC so that further requests of the relevant priority can be delivered.

The x86 APIC implementation is under `arch/x86/kernel/apic/`; generic interrupt descriptors, actions and flow handlers are under `kernel/irq/`. PCI MSI setup lives in the PCI and IRQ subsystems. A device driver normally requests an IRQ and supplies a top-half handler; substantial deferred work may be moved to threaded handlers, tasklets, workqueues or NAPI depending on the subsystem [35].

It must determine why the device interrupted, acknowledge or mask the device as required, protect shared state against concurrent execution, and avoid unbounded work in hard-interrupt context. The APIC delivers a vector; it does not understand the device's status registers or complete the driver's protocol.

The complete chain is therefore: device condition → interrupt message or pin → routing controller → local APIC → IDT vector → architecture entry code → generic IRQ layer → device driver.

4.3.4 Device registers, cost and driver structure

A controller exposes state through registers, but architectures differ in how those registers are addressed. In *port-mapped I/O*, a separate I/O address space is accessed with instructions such as `in` and `out`, subject to IOPL and I/O-bitmap permission checks on x86. In *memory-mapped I/O* (MMIO), controller registers occupy addresses in the physical address map and are accessed with load/store-like operations. MMIO simplifies instruction-set design and supports large register spaces, but ordinary memory optimisations cannot be applied blindly.

A compiler must not remove or merge accesses whose side effects matter. The processor must respect the ordering rules required by the device. Operating systems therefore map device regions with special attributes and use architecture-specific accessors such as Linux `readl`/`writel`, plus the barriers required by the mapping and device protocol. C `volatile` alone does not supply device ordering, DMA coherence or completion guarantees. The mapped address identifies a device operation whose width, ordering and side effects must follow the device contract.

A device register can have access side effects

	register kind	read effect	write effect
ordinary state	value	observe	replace
read-to-clear	event flags	clear flags	device specific
write-one-clear	event flags	observe	clear selected ones

These are distinct illustrative register contracts.

Figure 4.24: *A driver must use the exact semantics specified for each register. A read-modify-write sequence suitable for ordinary storage can be wrong for a side-effect register, especially when events arrive concurrently.*

A status register can contain independent flags

	7	6	5	4	3	2	1	0
status example	0	0	0	0	0	1	0	1
READY mask	0	0	0	0	0	0	0	1
ERROR mask	0	0	0	0	0	1	0	0

In this invented register, both READY and ERROR are set.

Figure 4.25: *Masks isolate flags according to the controller's documented layout. The example is an invented register, so its bit meanings must not be transferred to a real device. A real status read may also have specified side effects.*

Worked example: status, data and acknowledgement

A receive controller has `STATUS` bit 0 = ready, a `DATA` register and a write-one-to-clear `ACK` bit. In this model, reading DATA leaves the entry intact, while writing ACK releases it for replacement. A safe handler reads status, reads data, then writes 1 to `ACK`. If it acknowledges before reading a shallow FIFO, a new datum may replace the old one and the original value can be lost. Device documentation defines the required ordering.

For a separate illustrative write-one-to-clear status register, let READY be bit 0 and ERROR bit 2, with current value `0x05`. Writing `0x01` clears READY and preserves ERROR, leaving `0x04`, assuming no new event. Reading `0x05`, clearing bit 0 in software and writing back `0x04` would do the opposite: the one in bit 2 clears ERROR while READY stays set. Each set bit in the written mask acknowledges its corresponding flag.

Let a status test consume c processor cycles and occur every P seconds on a processor with frequency f. The fraction of one core spent polling is approximately

$$U_{\mathrm{poll}} = \frac{c/f}{P} = \frac{c}{fP}.$$

This check-only utilisation assumes a feasible period $P \geqslant c/f$ and excludes processing and timer costs. Tighter polling can reduce detection latency but can approach full utilisation. Periodic polling reduces utilisation; if arrivals are uniformly distributed within the interval, the average extra detection delay is about $P/2$ and the worst case is nearly P.

Worked example: periodic polling

A status check costs 180 cycles on a 3 GHz core and runs every 100 µs. The CPU time per check is $180/(3 \cdot 10^9) = 60$ ns. Utilisation is 60 ns/100 µs $= 0.0006$, or 0.06% of one core. The average detection delay contributed by the interval is approximately 50 µs.

Sustained traffic can make polling efficient. For a very fast device under sustained load, the interrupt rate may exceed the cost of a tight poll loop, and dedicated cores can poll queues with predictable latency. The decision depends on arrival rate, service cost, power budget and whether the CPU has other useful work.

An interrupt incurs entry/exit and dispatch costs, including context saving and cache effects; it may also change privilege. A simple cost model treats this overhead as fixed, although its measured value varies with machine and context. If each event carries very little work, interrupt overhead can dominate. Controllers therefore support batching or *interrupt moderation*: they generate one interrupt after several completions or after a timer expires. Batching raises throughput but adds latency to the first event in the group.

If events arrive at rate λ and each interrupt costs C_i seconds, one-interrupt-per-event consumes approximately λC_i core-seconds per second. Grouping b events per interrupt reduces the fixed component to $\lambda C_i / b$, while the per-event processing cost remains.

Worked example: interrupt coalescing

A network interface receives 200,000 packets/s. Entry and exit cost 2 µs per interrupt, excluding packet processing. One interrupt per packet would require 0.4 core-seconds per second, or 40% of a core. Coalescing 16 packets per interrupt reduces the fixed share to about 2.5% of a core, at the price of waiting to form a batch or expire a timer.

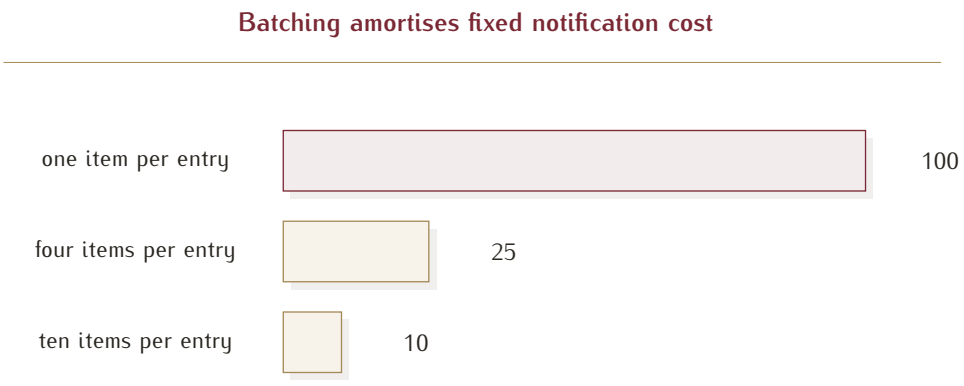


Figure 4.26: *The bars show only the fixed notification cost per item under the stated assumption. Processing each item still has its own cost, and waiting to build a batch can increase latency.*

The interrupt handler should do only the urgent work: identify the source, capture completion information, acknowledge or mask the device, and schedule further processing. Lengthy parsing, copying or protocol work is deferred so that interrupts are not blocked for too long. Operating systems implement this split with mechanisms often described as top halves and bottom halves, deferred procedure calls, tasklets or work queues.

The driver also manages concurrency. The process that issued an I/O request, the interrupt handler and another CPU may all access the same descriptor queue. Locks, atomic operations and memory ordering are needed, but operations permitted in process context may be forbidden in interrupt context because ordinary hard-interrupt context cannot sleep. A threaded handler runs under different context rules, while spinlock and other atomic sections still prohibit sleeping.

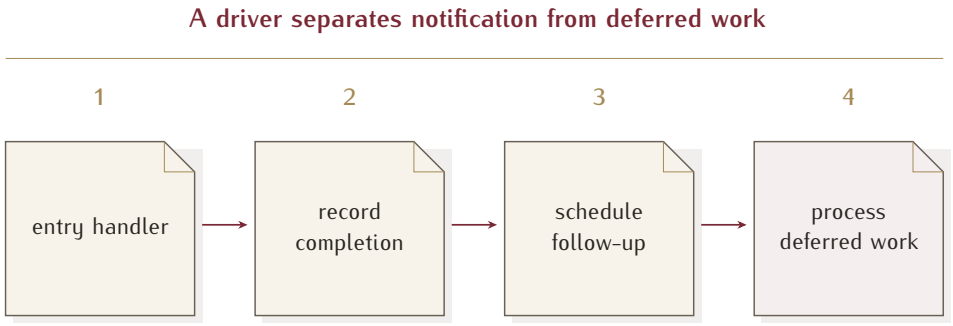


Figure 4.27: *The immediate handler performs work that is safe and necessary in its context. Longer or blocking work may be deferred to a suitable context. Correct ownership and ordering keep the two portions coordinated.*

Interrupt affinity controls which core handles a vector. Good affinity can keep device data, driver code and the consuming application on compatible cores, improving cache locality. Poor affinity can create cross-core traffic or overload one core while others are idle.

Worked example: vector allocation

A four-queue network interface receives four MSI-X vectors. The operating system maps queue q to vector $80 + q$ and routes each vector to the core that consumes that queue. The IDT still provides the final vector-to-entry mapping, while the interrupt controller and device tables determine how the event reaches that vector.

An edge-triggered interrupt represents a transition; if software masks the line and another edge occurs without being latched, the event can be lost. A level-triggered interrupt remains asserted while the condition holds, so the handler must remove the cause before returning or the interrupt will fire again. Controllers and drivers define precise acknowledge and end-of-interrupt sequences.

A driver coordinates source identification or masking, bounded completion processing, device acknowledgement, controller end-of-interrupt and re-enabling delivery. Their order follows the device and interrupt-controller contract. A continuously busy queue requires a service budget or deferred processing so that draining it cannot monopolise hard-interrupt context. Reading a status register may itself clear bits, while other devices require explicit write-one-to-clear operations.

Security consequence: interrupts carry untrusted device state

A compromised or malfunctioning device can generate excessive interrupts, malformed completion records or unexpected DMA results. Drivers run with high privilege and parse hardware-controlled data, making them part of the trusted computing base. Rate limiting, bounds checks, reset paths and IOMMU isolation reduce the damage caused by hostile peripherals.

From the 8259 PIC to per-core routing

The original PC interrupt controller offered a small fixed set of lines and priority rules suited to a uniprocessor. Multiprocessors and high-rate devices required programmable routing, many vectors and per-core delivery. APIC and message-signalled interrupts extend the same core idea, identifying an event and entering through a vector, to systems with many devices and processors. [26]

Laboratory 26: Polling and interrupt timing

Model event arrivals at times 1,4,9 with a queue of capacity four. Handling is instantaneous; one polling check costs one CPU-work unit and drains all queued events. Process arrivals before a simultaneous check. Compare checks every unit at times 1–10, checks at 5 and 10, and interrupts that each respond two time units after arrival with cost two work units. Give event response delays, means, maxima, work and drops.

Worked solution

Laboratory 26: solution

Process arrivals before a check at the same timestamp. Continuous discrete polling services the events at 1,4,9, so delays are $0, 0, 0$, mean and maximum zero, with ten checks and ten work units. Periodic polling services the first two events at 5 and the last at 10, giving delays $4, 1, 1$, mean 2 and maximum 4, with two work units. Interrupt delivery services at 3,6,11, giving delays $2, 2, 2$, mean and maximum 2, with six work units. No queue ever holds more than two waiting events, so there are no drops. For N isolated events, interrupt work is 2N; periodic polling over the fixed interval still costs two units. Increasing arrivals can change capacity and batching requirements, so these fixed-latency results apply only to the declared model. The final interrupt service occurs at time 11, so its observation window extends one unit beyond the last polling check; the stated work counts cover each complete schedule.

Return to the laboratory

The I/O software stack

Controllers expose device-specific registers and completion events, while applications expect stable operations such as read, write and control. Operating systems bridge the gap with layers: interrupt handlers acknowledge urgent events, device drivers translate requests into controller operations, device-independent code supplies naming, buffering, protection and error conventions, and user-space libraries adapt the interface for programs [59].

Layering explains why an interrupt is rarely the whole I/O operation. A top half may record status and schedule deferred work; a driver may complete several requests from one device notification; a buffered write may return before persistence; and an error may be retried or translated before it reaches the caller. A common interface can address different devices while preserving their documented latency, ordering and failure behaviour.

Error handling should occur at the lowest layer that has enough information to act without hiding a contract-relevant failure. A controller may correct a transient link error, a driver may retry a command, and a filesystem may reconstruct metadata. The application must still receive failures that change the meaning of its operation.

> **Exercise 4.3.1 — Follow one request through the layers**
>
> For a blocking read from an SSD-backed file, draw the path from library call to system call, filesystem or page cache, block layer, driver, submission queue, controller, completion and wakeup. Mark where buffering, retry, scheduling and error translation may occur.

Solution 4.3.1

Assume a regular file, an ordinary buffered blocking read, a cache miss and successful SSD completion. The library wrapper places the arguments in the system-call ABI's registers and enters the kernel. The kernel resolves the descriptor and checks the request. The filesystem and page-cache path locate the file range; a cache hit could satisfy this request without contacting the SSD.

Worked solution (continued)

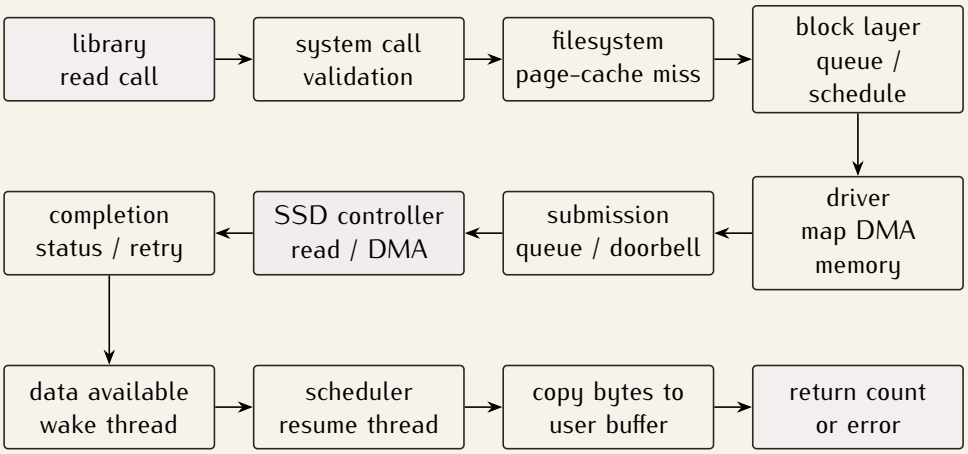


Figure 4.28: *The assumed page-cache miss follows the request through submission and completion. DMA fills prepared kernel memory; the resumed read path supplies the user buffer. Errors can end the path early, and a cache hit bypasses the storage request.*

For the assumed miss, file offsets are mapped to storage blocks and requests pass through the block layer. Requests may be merged, queued or scheduled. The driver maps the destination memory for DMA, constructs the controller's submission-queue entries and notifies the device. The calling thread can block on the missing data while other work runs.

The controller fetches the command, reads the storage data and transfers it into the prepared memory. It records completion; an interrupt or polling path lets the driver observe that record. The driver checks status and completes the block request. Upper layers mark the data available, wake the waiting thread and eventually copy the requested bytes into its user buffer. Scheduling determines when the awakened thread actually resumes. The system call returns a byte count or an error, which the library exposes through its language interface.

Buffering occurs in the page cache and may also occur inside the device. Scheduling occurs both for CPU execution and for storage requests. A recoverable device failure may trigger driver or block-layer retry according to policy; a permanent failure is translated into an operating-system I/O error and then the library's error convention. Neither retry nor readahead is guaranteed for every request. A short successful read must be distinguished from an error.

From I/O polling to vectored, message-signalled and hybrid interrupt handling

Timeline

Year	Machine, standard or event	Architectural significance
1954	DYSEAC external I/O	Computation and peripheral transfer proceed concurrently.
1964	System/360 interruptions	Vectored status and channel completion are architecturally defined.
2003	MSI-X	Devices signal many queue-specific vectors with messages rather than pins.

The earliest electronic computers often handled input/output by direct program control. A routine issued a command to a reader, printer or tape unit, examined a status indication and repeated the test until the device became ready. This busy-waiting model was tolerable when one job owned the machine and arithmetic was not dramatically faster than the peripheral. It became untenable once vacuum-tube processors could perform thousands of operations while electromechanical equipment completed a single action. Designers needed to let computation continue during device work and regain the processor’s attention when service became necessary [124, 109, 26].

One of the earliest well-documented solutions appeared in the National Bureau of Standards DYSEAC. Alan L. Leiner’s system specification was issued in 1952; the machine was completed for the U.S. Army Signal Corps in 1954. NBS historical material records that SEAC could interrupt a running DYSEAC computation and transfer data without destroying the interrupted work. Developed for real-time interaction among machines and external equipment, the mechanism established an enduring pattern: an asynchronous request, preservation of a return point and controlled redirection of execution [124].

Interrupt systems then acquired priority and vectoring. The IBM 704, delivered from 1955, included trap facilities used for program monitoring and floating-point exceptions. The TX-2 at MIT Lincoln Laboratory, operational in 1958, provided multiple priority levels for real-time work. By the time IBM announced System/360 on 7 April 1964, interruption had become a formally specified part of the architecture.

System/360 distinguished input/output, external, program, supervisor-call and machine-check interruption classes. For each class the processor stored the old Program Status Word and loaded a new one from architecturally fixed locations. The PSW held the instruction address, condition code, masks and control state, so entry and return were documented state transitions rather than an informal jump [74, 109].

System/360 also separated event notification from bulk transfer. A channel program described a sequence of I/O commands executed by channel hardware while the central processor continued ordinary instructions. The eventual input/output interruption reported completion or an exceptional condition after the channel had already performed substantial work. This division survives in modern controllers: the device engine moves

data and updates a completion record; the interrupt tells software that policy or cleanup is now required. The interrupt provides notification and transfers control to software; the channel or device engine performs the data movement [109, 111, 99].

The personal-computer lineage adopted a cheaper, more constrained design. Intel introduced the 8086 in 1978 with a table of 256 four-byte interrupt vectors beginning at physical address zero. Hardware pins, processor-detected exceptions and the software `INT` instruction all selected entries from that table. The IBM Personal Computer, unveiled on 12 August 1981, paired an 8088 with an Intel 8259A Programmable Interrupt Controller. One 8259A accepted eight interrupt-request lines, prioritised them, allowed masking and supplied the vector number during interrupt acknowledgement; the IBM PC/AT of 1984 cascaded two controllers to provide more lines. This arrangement shaped the legacy IRQ numbers and sharing problems that remained visible in PC operating systems for decades [104, 126, 26].

Multiprocessors required a different routing model. Intel's Advanced Programmable Interrupt Controller architecture placed a local APIC at each processor and one or more I/O APICs near external interrupt sources. The discrete 82489DX supported early x86 multiprocessor designs; later processors integrated the local APIC. A local APIC accepts external interrupts, timer events and interprocessor interrupts. The latter are used for scheduler rescheduling requests, cross-CPU function calls and translation-lookaside-buffer shootdowns. Destination selection, priority and masking are now per-CPU problems, and the Intel manual specifies the register and delivery semantics [106, 26].

Conventional PCI introduced Message Signalled Interrupts so a device could report an event by issuing a specially formatted memory-write transaction instead of asserting a shared electrical line. PCI Local Bus 2.2 standardised MSI in 1998; the PCI-SIG MSI-X change, dated 10 June 2003, added a table of independently maskable vectors.

PCI Express, developed in the early 2000s, made message-based signalling a natural part of a packetised fabric. A multiqueue network adapter or NVMe controller can bind queue pairs to different MSI-X vectors and distribute them across CPUs. The word "interrupt" now names a routed transaction that may pass through PCIe switches, an interrupt-remapping unit and a local APIC before the processor enters a handler [127, 81, 26].

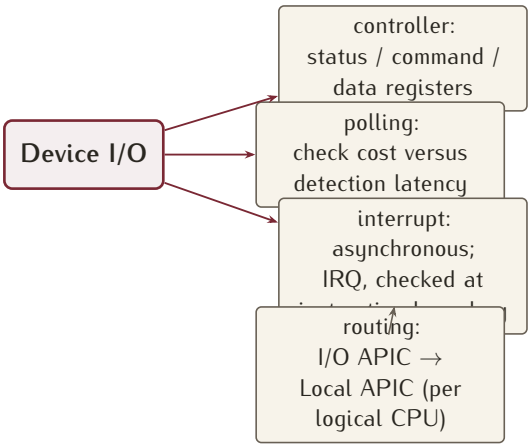
High event rates then forced operating systems to combine interrupts with polling. Jamal Hadi Salim, Robert Olsson and Alexey Kuznetsov described the Linux "new API" networking design in 2001. In current Linux documentation the acronym NAPI no longer expands to a particular phrase, but the mechanism remains clear: a device interrupt schedules a NAPI instance; the kernel then polls a bounded budget of receive and transmit completions, normally suppressing further interrupts while work remains. Low traffic receives prompt notification; heavy traffic is processed in batches, avoiding thousands of entries per second and reducing receive livelock. Current drivers may also expose busy polling for applications with stringent latency goals [128, 129].

A Linux hard-interrupt handler is deliberately short. Architecture entry code records the processor state, the generic IRQ layer identifies the descriptor and invokes the registered handler, and the driver acknowledges or masks the source. Work that must sleep is deferred to a suitable thread or workqueue. Bounded non-sleeping work may instead use softirqs or ordinary NAPI polling. These contexts have different blocking and locking rules. Hard-interrupt context imposes strict preemption, locking and blocking constraints, and an incorrectly ordered acknowledgement can lose events or create an interrupt storm. The kernel source and documentation form part of the contract alongside the device and processor manuals [35, 34, 129].

Virtualisation introduced interrupt remapping. A device assigned directly to a guest must not be able to inject an arbitrary vector into the host or another guest. Intel VT-d and AMD I/O-virtualisation hardware validate device-originated messages against remapping tables and permitted destinations. Linux coordinates those tables with its IOMMU, VFIO and IRQ subsystems. The apparent one-line diagram "device → CPU" can thus conceal a completion queue, a PCIe write, a requester identity, an interrupt-remapping lookup, APIC delivery, architecture entry code and deferred driver work [132, 133, 26, 27].

Polling and interrupts developed into complementary mechanisms. DYSEAC showed that asynchronous interruption could preserve useful computation. System/360 made interruption and channel completion precise architectural events, and the 8259A adapted them to the economical single-processor PC. APICs distributed interrupts across multiprocessors; MSI and MSI-X converted wires into transactions; and NAPI deliberately returned to bounded polling under load. The design choice is still governed by event rate, latency, transition cost and isolation. A rigorous account must name the controller, vectoring rule, saved state, acknowledgement protocol and deferred-work mechanism rather than saying only that "the device interrupts the CPU" [124, 109, 26, 127, 129].

Interrupt controllers, masking, acknowledgement and deferred work remain the practical vocabulary of device service today.



Concept check

1. Which controller register is polled to see if a key is ready?
2. State one advantage and one disadvantage of busy waiting.
3. Is an interrupt synchronous or asynchronous?
4. With PIC base 32 and IRQ 4, what vector reaches the CPU?
5. Which APIC is local to a logical processor, and which routes traditional external device IRQs?

Solutions to the review questions

1. Poll the controller status register and test the bit indicating that input data is available. In a legacy keyboard-controller example this is the output-buffer-full indication. Read the data register only after the status allows it; status and data have different roles.
2. Busy waiting can detect a short event with little dispatch overhead because the processor is already checking. It consumes execution time while the condition is false and can delay other work. Its cost therefore depends on event frequency, checking cost and what else could use the processor.
3. A device interrupt is asynchronous with respect to the ordinary instruction sequence: the device can complete independently of the instruction currently executing. A synchronous exception is tied to an instruction or its attempted execution.
4. The remapped vector is $32 + 4 = 36 = \texttt{0x24}$. The controller input number identifies the source; the configured base determines the vector range delivered to the processor.
5. A local APIC belongs to a logical processor in the model. An I/O APIC routes traditional external device interrupt lines to selected processor destinations. Message-signalled interrupts can instead convey interrupt requests as interconnect transactions.

4.3.5 Worked problems

Polling work and elapsed wait

A device becomes ready at 2 ms. Polls occur at $10, 20, \ldots, 2000\,\mu s$, and readiness is observed before a simultaneous poll. There are exactly $2000/10 = 200$ checks. At 80 ns of CPU work per check, they consume $200 \cdot 80 = 16000$ ns $= 16\,\mu s$, or $16/2000 = 0.8\%$ of one processor over the interval. The remaining time is available to other work under this periodic-polling model. Checks every $20\,\mu s$ reduce the

Worked example (continued)

count to 100 and work to 8 µs, while increasing the detection delay for arrivals between checks.

Pending events: level versus edge

A level-triggered device keeps its request asserted while a completion remains unread. Sending controller EOI without removing or masking the device condition leaves that level active, so service can be requested again. The device acknowledgement and controller EOI therefore serve different purposes.
For an edge-triggered source, suppose a complete pulse occurs during a masked interval. If hardware records no pending edge, the signal is already inactive when delivery is enabled and the event is lost. If the controller latches it, one pending event can be delivered later. A single pending bit still cannot count arbitrarily many edges; the driver's queue or device status must retain enough information to recover all work.

Exercise 4.3.2 — Registers

Which register is normally examined before reading data?

Worked solution

Read the controller's status register to determine whether data is ready and whether an error is reported. The data register supplies the payload once the protocol permits the transfer. For example, a ready bit of zero can require further waiting; ready one permits the prescribed data read. Some registers have read-to-clear or other side effects, so the device's register protocol determines the exact order.

Exercise 4.3.3 — Polling trade-off

Main disadvantage?

Worked solution

Polling repeatedly executes status checks while waiting. If each check costs c cycles and checks occur at rate r, the processor spends approximately cr cycles per second on the checks, even when the device has no new data. Faster polling can reduce detection latency while increasing this cost. The useful trade-off depends on event frequency, response requirements and the cost of interrupt-based notification.

Exercise 4.3.4 — APIC

Which APIC is associated with each logical processor?

Worked solution

The local APIC is associated with a logical processor in the x86 interrupt-delivery model. It participates in accepting and prioritizing interrupts for that processor and in interprocessor interrupt delivery. The I/O APIC handles routing of external device interrupt sources. The word "local" identifies the processor-side role; it does not imply that all external-device routing is performed there.

Exercise 4.3.5 — EOI

Why send end-of-interrupt?

Worked solution

An end-of-interrupt notification tells the interrupt controller that service for the interrupt has reached the required completion point. The controller can then update its in-service state and permit further delivery according to its priority rules. The device's own pending condition may also need clearing through a separate register operation. Acknowledging only the controller while leaving a level-triggered device request asserted can lead to another immediate interrupt.

Summary

Choose the notification mechanism from the workload and response requirements. Estimate event rate, acceptable latency, per-poll cost, per-interrupt cost and opportunities for batching. Then trace ownership: who writes command state, who observes completion, who acknowledges the event and who wakes the waiting thread? These steps specify how the device and software exchange responsibility for each request.

4.3.6 Exercises

Exercise 4.3.6 — Controller roles

Name the purpose of status, command and data registers.

Go to the solution →

Exercise 4.3.7 — Poll utilisation

A check costs 240 cycles on a 2.4 GHz core and runs every 50 microseconds. Compute core utilisation.

Go to the solution →

Exercise 4.3.8 — Polling latency

A device is polled every 8 ms. Under uniform arrivals, estimate mean and worst additional detection delay.

Go to the solution →

Exercise 4.3.9 — Interrupt overhead

A device generates 80,000 events/s and fixed interrupt overhead is 1.5 microseconds. What fraction of a core is spent on fixed overhead? What if eight events are batched per interrupt?

Go to the solution →

Exercise 4.3.10 — Level-triggered event

Why can a level-triggered interrupt immediately reappear after return if the handler sends end-of-interrupt but does not clear the device condition?

Go to the solution →

Exercise 4.3.11 — Affinity

A queue is consumed mainly by core 2, but its interrupt is routed to core 7. Give two possible costs.

Go to the solution →

Exercise 4.3.12 — MMIO ordering

A driver fills a descriptor in RAM and then writes a doorbell register. Why must descriptor writes become visible before the doorbell?

Go to the solution →

Exercise 4.3.13 — Policy choice

Choose polling, interrupts or a hybrid for a queue that is idle most of the day but receives intense 5-ms bursts. Defend the choice.

Go to the solution →

4.3.7 Solutions

Device Controllers, Polling, and Interrupts

Solution 4.3.6

Status reports readiness and errors; command/control tells the controller what operation to perform; data carries the payload or points to where the payload is stored, depending on the interface.

← Return to the exercise

Solution 4.3.7

One cycle lasts $1/(2.4 \cdot 10^9)$ seconds, so a 240-cycle check occupies $240/(2.4 \cdot 10^9) = 10^{-7}$ s $= 100$ ns. The polling period is $50\,\mu s = 50{,}000$ ns. Utilisation is the busy fraction $100/50{,}000 = 0.002 = 0.2\%$. Equivalently, 20,000 checks per second consume 4.8 million cycles out of 2.4 billion available. This accounts for the check itself; extra handling work would add to the load.

← Return to the exercise

Solution 4.3.8

Let t be the arrival's offset after the previous poll, uniformly distributed from 0 to 8 ms. The next poll occurs after $8 - t$ ms. The mean is therefore $(8 + 0)/2 = 4$ ms. An event arriving just after a poll waits almost 8 ms, while one arriving just before the next poll waits almost zero. These values exclude poll execution and event-handling time and assume every poll finds all pending events.

← Return to the exercise

Solution 4.3.9

At one interrupt per event, fixed work is

$$80{,}000\,s^{-1} \cdot 1.5 \cdot 10^{-6}\,s = 0.12,$$

or 12% of one core. Full batches of eight reduce the interrupt rate to $80{,}000/8 = 10{,}000$ per second. Fixed work becomes $10{,}000 \cdot 1.5 \cdot 10^{-6} = 0.015$, or 1.5%. The reduction is 10.5 percentage points, an eightfold reduction of this fixed component. Processing the 80,000 events still costs additional work, and waiting for each batch can increase response latency.

← Return to the exercise

Solution 4.3.10

The request line remains asserted while the condition holds. Once the controller is ready to deliver again, it observes the same active level and raises the interrupt. The handler must remove or mask the source according to the device protocol.

← Return to the exercise

Solution 4.3.11

First, core 7 may update completion metadata or queue locks that core 2 then reads or acquires. The associated cache-line ownership transfers add coherence traffic and can stall either core. Second, core 7 may need to wake or notify work on core 2, adding cross-core signalling and scheduling delay. Routing service to core 2 can reduce both costs if that core has capacity; an overloaded core may instead justify distributing the work.

← Return to the exercise

Solution 4.3.12

The doorbell tells the device to fetch the descriptor. If stores are reordered or still buffered, the device may read stale or partial fields. Use the DMA mapping/synchronisation protocol required for the descriptor memory, a publication barrier where required, and the prescribed MMIO accessor. A barrier orders visibility; it does not by itself clean a non-coherent cache or prove that a posted doorbell write has reached the device.

← Return to the exercise

Solution 4.3.13

Use interrupts while idle, then bounded polling during a burst. The first arrival interrupts a processor, which records the event and schedules queue processing. While the queue remains busy, the processing path drains batches up to a budget and arranges another poll if work remains. This reduces repeated interrupt-entry overhead without monopolising the processor indefinitely.

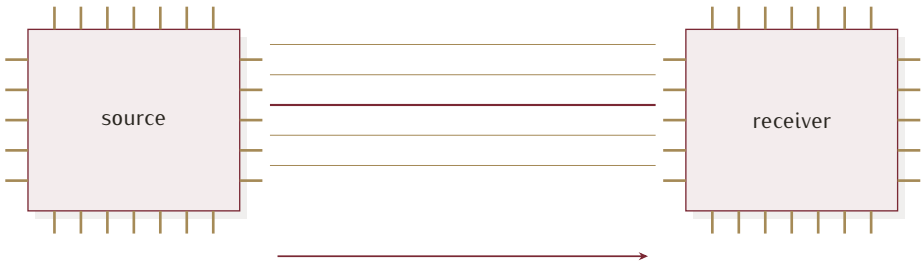
When the queue becomes empty, re-arm notification according to the device's race-safe protocol. A completion arriving between the empty observation and re-enabling interrupts must remain latched or be detected by the prescribed recheck; otherwise the queue could sleep with work pending. The design combines low idle cost with efficient burst handling. Budget and re-arm details must be chosen from measured load and the controller's contract.

← Return to the exercise

4.4 BUSES, TRANSACTIONS, AND INTERCONNECTION

A processor issues a read, yet the requested bytes may cross controllers and links before a response returns. Every participant needs to agree on the destination, operation, data and completion conditions. The interconnect supplies that agreement through transactions carried over physical connections. Separating the transaction from its transport explains bandwidth limits, arbitration and the cost of sharing a link.

Communication consumes space as well as time



information travels across a physical path

The interface specifies what each signal means and when it may change.

Figure 4.29: *A request must travel between components and obey a signalling protocol. The physical path has propagation delay, while the protocol determines acceptance, ordering and throughput. The drawn conductors are schematic rather than a prescribed bus standard.*

4.4.1 An interconnect is a protocol

A shared parallel bus is one way to connect a processor, memory and device controllers. Address, data and control lines coordinate access to a common medium. Switched point-to-point fabrics provide another organisation, allowing independent links to carry traffic concurrently. This chapter starts with the shared-bus model and then follows the transaction abstraction into packet-based systems.

A non-multiplexed parallel bus can have a hundred or more signal lines, each with a role. The lines fall into three functional groups:

- the *data* lines, which carry the bytes being transferred; their number is the bus *width*;
- the *address* lines, which carry the address of the memory cell or device register involved;
- the *control* lines, which carry the timing and the type of the operation (read or write, ready, acknowledge).

For this non-multiplexed signal model, the line count is the sum of the three groups; power, ground and any extra physical signalling conductors are excluded. The number of *data* lines determines how many bytes move in one transfer: a binary bus with w data lines and w divisible by eight carries up to $w/8$ payload bytes per data beat. The number of *address* lines determines how much memory can be named: with a address lines the bus can address 2^a distinct cells.

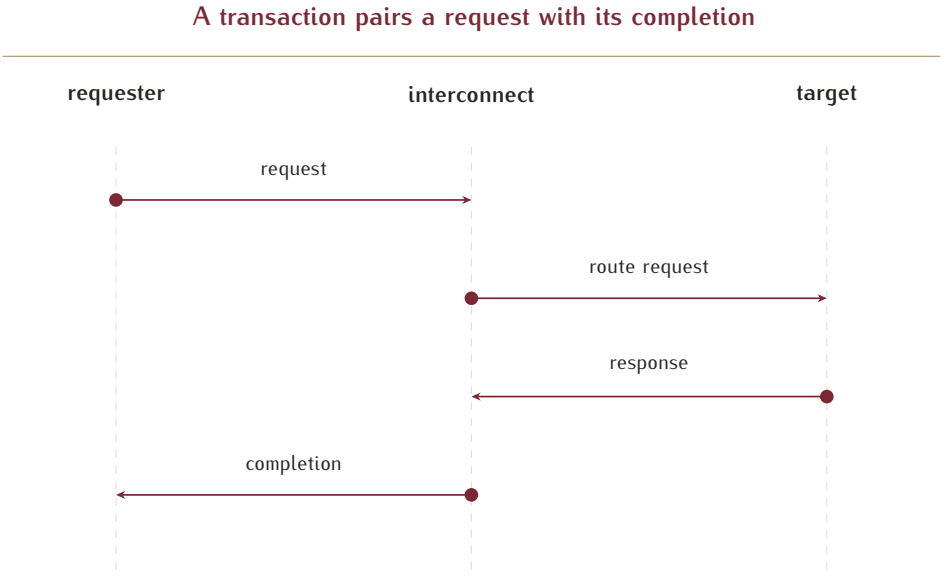
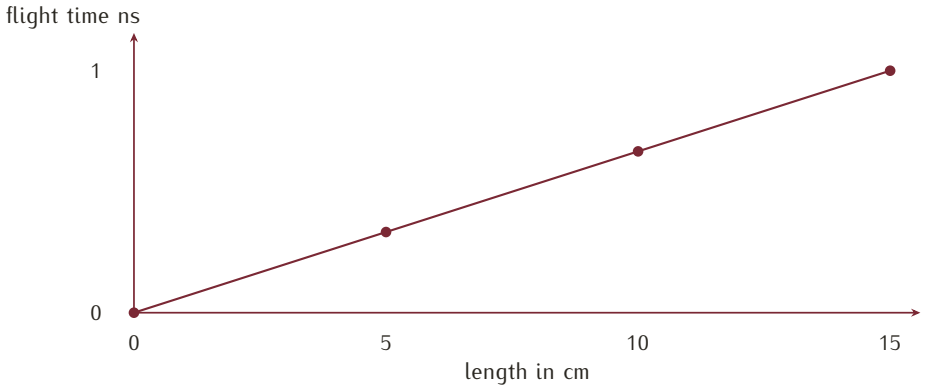


Figure 4.30: *The transaction identifies an operation and a target and eventually supplies a completion condition. Different protocols can carry the phases on separate channels or overlap transactions. Their ordering rules define what that overlap means.*

Worked example

Problem. A bus has 64 data lines, 32 address lines and 8 control lines. How many bytes does it transfer per operation, how many cells can it address, and how many lines does it have in total?
Solution. Bytes per transfer: $64/8 = 8$. Addressable cells: 2^{32}. Total lines: $64 + 32 + 8 = 104$.

Flight time grows with path length



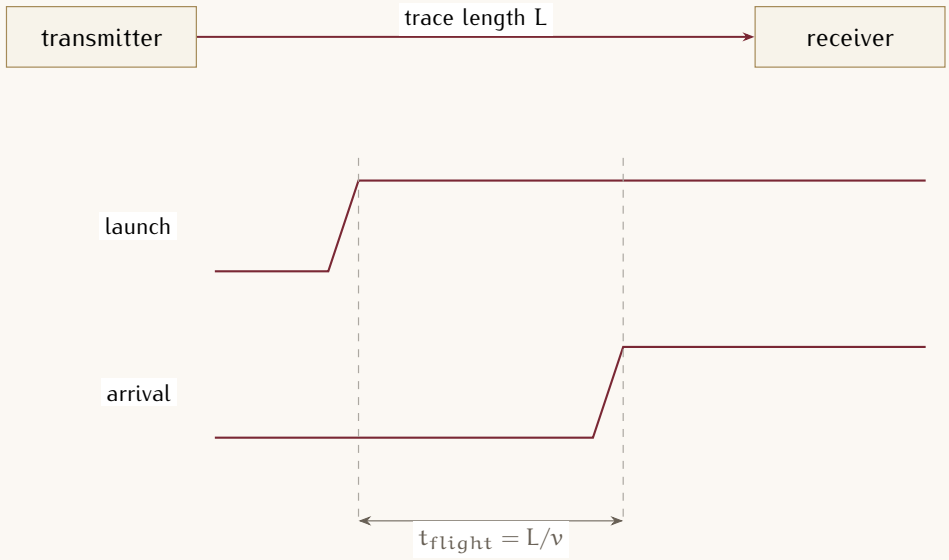
Assumed propagation velocity: 1.5 times 10 to the 8 metres per second.

Figure 4.31: *The linear graph follows t = L/v for a fixed propagation velocity. It excludes driver, receiver and protocol delay. The physical time of flight remains even when the signalling protocol permits many bits to be in transit.*

A signal takes time to cross the board

A changing voltage launches an electromagnetic disturbance along a conductor and its return path. The signal arrives after a finite flight time. When that flight time matters relative to the edge's rise time, reflections and impedance changes require a transmission-line model [45].

Physical mechanism (continued)



Worked check. Assume a 15 cm trace and propagation velocity $1.5 \cdot 10^8$ m/s in its medium. Then

$$t_{flight} = \frac{0.15}{1.5 \cdot 10^8} = 10^{-9}\ \text{s} = 1\ \text{ns}.$$

At a hypothetical 3 GHz clock, one period is approximately 0.333 ns, so the one-way flight alone spans three periods. Driver, receiver and protocol delays must be added when calculating a transaction's latency.
A stream can still have several bits in flight simultaneously. Propagation delay constrains when the first response can arrive; the signalling protocol and symbol rate determine how frequently later data can follow. This is the physical counterpart of the distinction between latency and throughput in the pipeline chapter. The assumed propagation velocity applies to this example. A real board's material and geometry determine its value.

4.4.2 Width, clocking and topology

A bus is characterised chiefly by its *width* (data lines, hence bytes per transfer), its *clock frequency* (cycles per second on a synchronous bus), and its *bandwidth* (bytes per second). For a synchronous bus that performs one transfer per clock cycle,

$$\text{bandwidth} \;=\; \text{width}_{\text{bytes}} \times \text{clock}.$$

A bus whose transfers are paced by a shared clock is *synchronous*; one that coordinates by handshake without a common clock is *asynchronous*. On a *multiplexed* bus the same phys-

ical lines carry the address at one moment and the data at another, saving pins while requiring separate address and data phases; the cycle cost depends on the protocol.

Worked example

Problem. A synchronous bus is 8 bytes wide and runs at 200 MHz, one transfer per cycle. What is its peak bandwidth?
Solution. 8 bytes $\times$ 200 $\times$ 10^6 Hz $=$ 1600 $\times$ 10^6 bytes/s $=$ 1600 MB/s.

Worked example

Problem. How many clock cycles are needed to transfer 1024 bytes on a bus 8 bytes wide, one transfer per cycle?
Solution. 1024/8 $=$ 128 cycles.

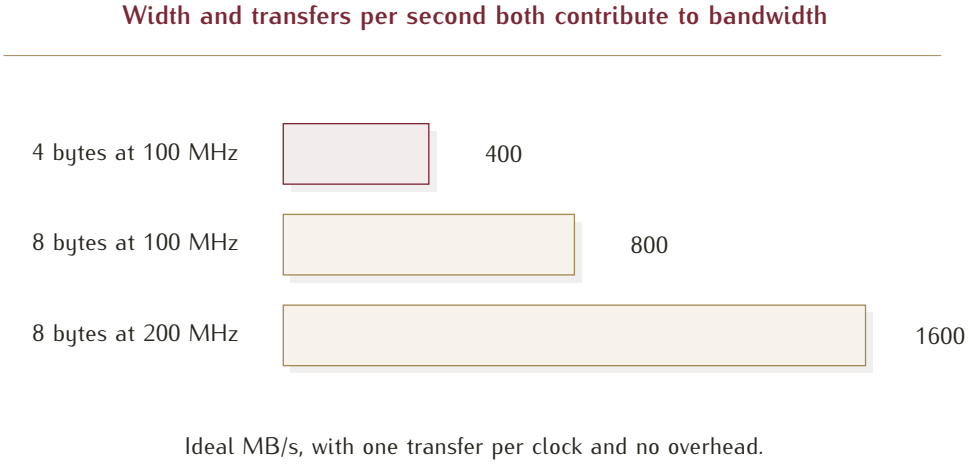
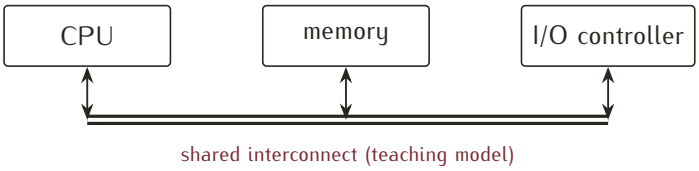


Figure 4.32: *The calculation multiplies bytes per transfer by transfers per second. These are theoretical interface rates under the stated assumptions. Protocol overhead, turnarounds and limited target service rate can reduce delivered bandwidth.*

Bandwidth measures bytes transferred per unit time. To find how long a transfer *takes*, divide the number of bytes by the bandwidth: time $=$ bytes/bandwidth. Keeping the units straight (bytes, bytes/s, seconds) is most of the work in these problems.

Traditional buses can be classified by what they connect. A teaching *system bus* joins processors, memory and I/O controllers; a *backplane bus* joins modules or slots; a *peripheral bus* connects devices. Modern systems often replace one shared electrical bus with point-to-point links, switches and packet fabrics, but the same questions remain: who may initiate a transfer, how is a destination named, and how are data and completion reported?



Differential links use a pair to improve signal integrity

A serial link transmits a stream of bits or encoded symbols.

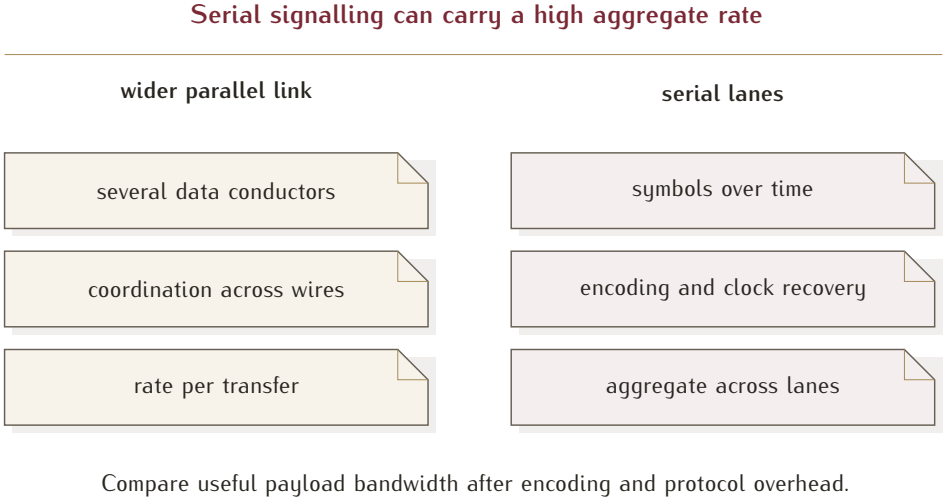


Figure 4.33: *Conductor count alone does not determine useful throughput. A serial lane can transmit at a high symbol rate, and several lanes can operate together. The effective payload rate also depends on the encoding and protocol.*

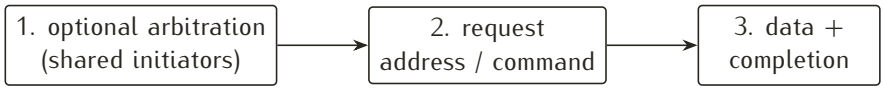
A differential transmitter drives complementary voltage or current changes on two conductors around a controlled common-mode level, and the receiver responds mainly to their difference. Disturbance coupled similarly onto both lines is partly rejected, subject to pair symmetry and receiver limits. Serialisers, deserialisers and clock-data-recovery circuits convert between parallel on-chip words and the high-rate physical stream.

PCI Express and many other high-speed links use differential pairs because maintaining timing across a wide parallel board bus becomes difficult at high data rates. The logical packet is carried by an analogue channel whose loss, skew and jitter must be controlled and equalised [45, 46].

4.4.3 Transactions and protocol semantics

In the following shared-bus teaching model, a transfer has three logical steps. If several agents can initiate work, arbitration first grants ownership to one requester; a fixed-owner or point-to-point link may not need per-transaction arbitration. The initiator then

presents an address or destination plus a command, after which data and completion information move according to the protocol. Modern fabrics may pipeline, split or overlap these steps and commonly use *requester/completer* rather than master/slave terminology.

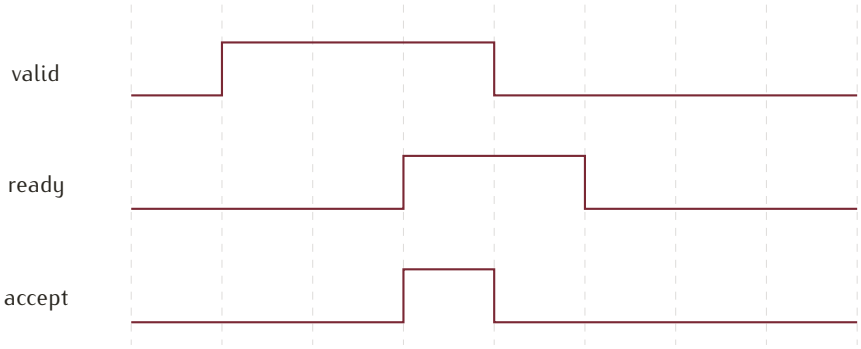


Worked example

Problem. Two devices want the bus at the same time. What component resolves the contention, and what is the name of the device that wins?
Solution. The arbiter grants one requester ownership for the transaction. The winning requester, traditionally called the bus master, drives the address and command. The addressed component acts as the completer.

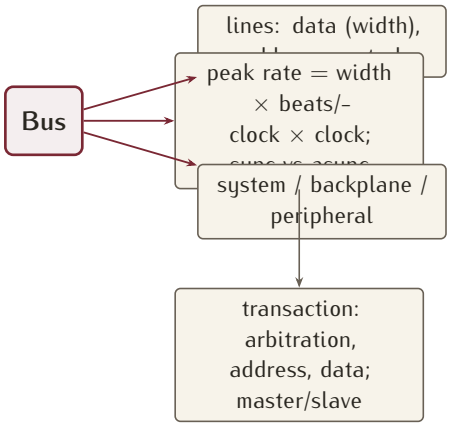
A memory read is a transaction between a requester and a responding memory device. The CPU presents the address and read command; the memory returns the requested word; and the protocol signals completion. A burst or sequence of exchanges can transfer a cache line; a storage controller can use related memory transactions while moving a disk block.

A handshake identifies when a transfer is accepted



Illustrative synchronous rule: transfer at an edge with valid AND ready.

Figure 4.34: *The sender keeps valid data available until the receiver can accept it. Acceptance requires both conditions at the specified sampling edge. The protocol also defines how long the data must remain stable.*



From the teaching bus to MMIO and packet-based interconnects

A shared address/data/control bus is a useful transaction model, but contemporary systems often realise the same logical roles through point-to-point links, switched fabrics and on-chip networks.

A transaction still identifies a requester, a target, an address or route, a command, data and completion status. Arbitration and flow control still decide when traffic can proceed. What changes is the physical representation: PCI Express carries packets over serial links; an integrated memory controller participates in an on-chip fabric; cache-coherent interconnects exchange coherence messages rather than exposing one global set of wires.

Firmware and the operating system assign address ranges to device registers. A CPU load or store to such a range is translated into an MMIO transaction rather than a DRAM access. Device mappings require special ordering and caching attributes. Ordinary compiler ordering is not sufficient: drivers use architecture-aware accessor functions and barriers so that writes reach the device in the required order.

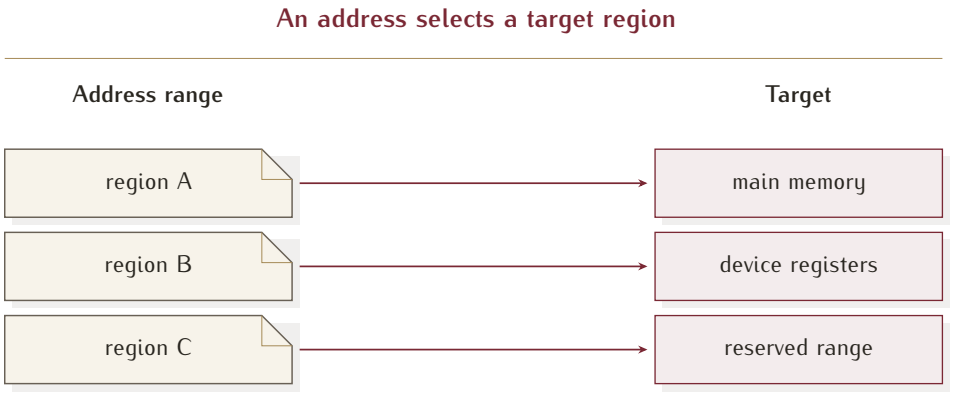


Figure 4.35: *Address decoding routes a request according to the system map. Different regions can have different cacheability, permissions and side effects. A numeric address does not universally imply ordinary RAM.*

PCI enumeration and resource assignment are implemented under `drivers/pci/`. Drivers map device resources and use interfaces such as `ioremap`, `readl` and `writel` rather than treating an MMIO address as an ordinary C pointer. These interfaces encode portability and ordering requirements that a raw dereference would hide [35, 40].

A statement such as “write the command register” may compile to one store instruction, but completion can involve store buffering, an uncacheable or device-ordered mapping, interconnect routing, a bridge, and finally a register inside the controller. A later read may be used to flush posted writes when the device protocol requires proof that the command has arrived.

Worked example

A link delivers 10 GB/s of payload after a fixed request latency of 2 microseconds. For one isolated 4 KiB transfer, the data phase takes $4096/(10^{10})$ seconds, or $0.4096\,\mu s$. The complete request takes $2 + 0.4096 = 2.4096\,\mu s$. About 83 percent of the elapsed time passes before the payload begins to arrive. Larger transfers spread the same fixed delay over more bytes.

An eye diagram visualises voltage and timing margin

The receiver samples each symbol at the intended time and obtains the intended value.

Waveform segments aligned to the symbol period can be overlaid to form an eye diagram. The vertical opening indicates available amplitude margin at a sampling phase, while the horizontal opening reflects timing spread from jitter and inter-symbol interference. Equalisation and transmitter emphasis reshape the channel response and can enlarge the opening observed at the receiver.

Link quality is not captured by clock frequency alone. An eye diagram is a useful summary, but it does not by itself prove a target bit-error rate unless the measurement method and statistical assumptions are specified [46, 45].

Worked example: conceptual MMIO read

The CPU issues a load from an address mapped to a device register. A bridge recognises the address range and forwards a read request. The controller returns the register value or an error. Only after the response reaches the core can the dependent instruction use the value. A cacheable DRAM load may be satisfied by a cache; a device-register read is normally ordered and routed to the controller.

4.4.4 Bandwidth, coordination and arbitration

For a parallel bus of width w bits transferring t data beats per clock at frequency f, the peak payload rate is

$$B_{peak} = \frac{w}{8} t f.$$

This is an upper bound. Address phases, turnarounds, acknowledgements, protocol headers, idle cycles and contention reduce useful bandwidth. If a fraction η of transmitted bytes are payload and the link is busy for fraction u, then

$$B_{useful} = B_{raw} \eta u.$$

Worked example: burst efficiency

A 64-bit bus runs at 200 MHz and transfers one data word per cycle. Raw bandwidth is $8 \cdot 200$ MB/s $= 1.6$ GB/s. Each burst carries eight data words but consumes two additional cycles for command and turnaround. Its protocol efficiency is $8/(8+2) = 0.8$, giving at most 1.28 GB/s before other contention.

A synchronous bus uses a shared clock to define when signals are valid. Its timing is simple, but clock skew and propagation delay limit physical size and frequency. An asynchronous bus uses request/acknowledge handshakes. Each transfer adapts to the responding device, but the protocol requires more transitions and control logic.

Modern high-speed interconnects are often source-synchronous or packetised rather than a single classic shared bus. The conceptual questions remain: how does the receiver know that data are valid, how are errors detected, and how is flow control applied when buffers fill?

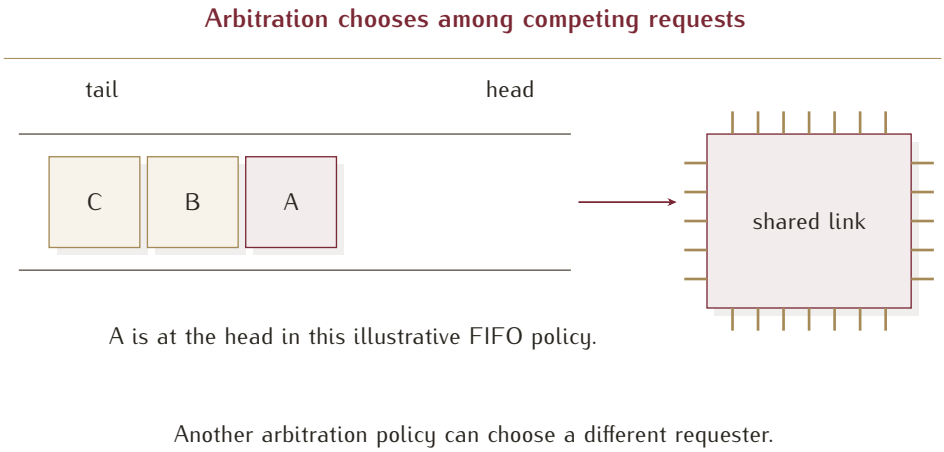


Figure 4.36: *When only one request can use a shared resource, arbitration determines the next grant. Fairness and latency depend on the policy, while the interface controls the permitted transaction once the grant is made.*

When several initiators can start transactions, an arbiter selects one. Fixed priority is simple but can starve low-priority masters. Round-robin improves fairness. Weighted or deadline-aware policies allocate service according to bandwidth or latency requirements. Arbitration can be centralised or distributed.

A transfer may use *cycle stealing*, where a DMA controller obtains individual bus opportunities between CPU accesses, or *burst mode*, where it retains ownership for a block. Bursts amortise arbitration and command overhead but can increase the waiting time of other masters.

Worked example: round-robin service

Three masters A, B and C continuously request a bus. A round-robin arbiter grants A, then B, then C. If each grant carries a 256-byte burst on a 2 GB/s link, one burst occupies 128 ns excluding overhead. Each master receives one third of the long-run bandwidth and waits at most two other grants under this simplified saturated model.

4.4.5 Hierarchy and modern point-to-point fabrics

A computer contains a hierarchy of interconnects. A memory fabric connects cores, caches and memory controllers; peripheral links connect device controllers; bridges translate transactions between protocols and clock domains. Address decoding determines which target owns a range. Configuration mechanisms assign MMIO windows and interrupt resources.

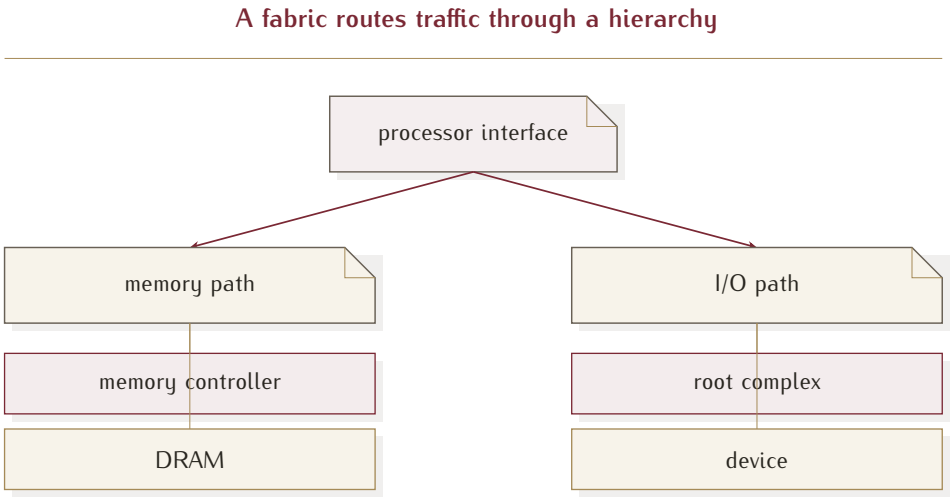


Figure 4.37: *The hierarchy explains why different transactions can traverse different components before reaching their targets. Real platforms may include switches, multiple channels and coherent links; the drawing makes no universal topology claim.*

Bridges buffer transactions and may post writes: the CPU-side request appears complete after the bridge accepts it, before the target has physically acted. When the device protocol requires proof of arrival, software can use a prescribed readback or completion mechanism. An ordinary memory-ordering barrier alone need not drain posted writes to the target. The programmer-visible ordering contract is therefore as important as raw bandwidth.

> **Device ordering is not ordinary memory ordering**
>
> Compilers, cores, caches and bridges can all reorder or combine accesses when the architecture permits. Device drivers use architecture-specific accessors and barriers to establish the order required by a controller protocol. Marking a pointer `volatile` constrains compiler optimisation but does not by itself provide every hardware ordering guarantee.

A traditional parallel bus shares wires among participants. PCI Express replaces that physical structure with point-to-point serial links and switched packet routing. Each lane has separate transmit and receive differential pairs and can carry traffic in both directions simultaneously; multiple lanes form a wider link. Transactions are encoded as packets, and switches forward them toward endpoints.

Calling PCIe a “bus” is still useful at the architectural level because it provides discovery, address spaces and transactions. Physically, however, it behaves more like a

network. Packet headers, encoding and flow-control traffic mean that signalling rate is not identical to payload bandwidth.

Security consequence: bus mastery extends trust

A bus master can access memory without executing CPU instructions. Without isolation, a malicious peripheral can read secrets or overwrite kernel data. The IOMMU introduced in Section 4.7 constrains device-visible address spaces, just as the MMU constrains processes.

Why buses became hierarchies

Early systems could place CPU, memory and peripherals on one shared backplane. As processor and memory rates increased, one set of wires could no longer satisfy every participant. Systems evolved toward specialised links, bridges, switched fabrics and coherent on-chip networks. The word "bus" survived because the transaction and arbitration concepts remain useful even when the physical topology changes. [49, 50]

Laboratory 27: Arbitration and burst efficiency

Three masters are continuously backlogged. Each grant costs one arbitration cycle plus B payload cycles. Compare fixed priority with round robin for B=1,4,16. All requests require one burst; measure from a round boundary and define waiting as time until arbitration for the request begins. Give total payload efficiency, each master's waiting/response time within a round, and starvation behaviour.
Worked solution

Laboratory 27: solution

Each grant costs one arbitration cycle followed by B payload cycles. Both policies keep the bus busy under the stated load, so the payload fraction is $B/(B+1)$. Number the requesters A, B and C in round-robin order. A begins arbitration at time zero. Each subsequent requester waits one more grant.

B	Efficiency	Wait A, B, C (cycles)	Response A, B, C (cycles)
1	$1/2 = 50\%$	0, 2, 4	2, 4, 6
4	$4/5 = 80\%$	0, 5, 10	5, 10, 15
16	$16/17 \approx 94.12\%$	0, 17, 34	17, 34, 51

Response includes the request's own grant. Under strict fixed priority, suppose A has highest priority and always has another burst ready. Its initial wait is zero and its response is $B+1$ cycles. B and C can wait indefinitely. The aggregate payload fractions in the table still apply because A alone fills the bus. Round robin gives each requester one third of the grants and bounds the wait from this round boundary by $2(B+1)$ cycles.

Worked solution (continued)

Return to the laboratory

From shared backplanes to packetised coherent fabrics

Timeline

Year	Machine, standard or event	Architectural significance
1975	S-100 bus	A shared backplane exposes address, data and control signals.
1992	PCI	Configuration and bus mastering are standardised for personal computers.
2002	PCI Express 1.0	Point-to-point packet links replace the shared parallel bus electrically.

Early computers did not contain a “bus” in the later standardised sense. Their arithmetic units, memories and peripherals were joined by wiring designed for one machine. Modularity created the need for a common electrical and logical interface. Mainframes developed channels for input/output, while minicomputers put processor, memory and controller boards into a backplane. DEC’s Unibus, introduced with the PDP-11 in 1970, used a common address space and a shared set of lines for memory and devices. The S-100 bus grew from the 1975 Altair 8800 and, despite weak initial standardisation, enabled an ecosystem of third-party boards. The important historical change was commercial as well as electrical: a machine could be expanded by components designed outside the original manufacturer [67, 126, 81].

The IBM PC of 1981 extended this ecosystem through the expansion interface later called ISA. Its openness and the availability of technical documentation helped compatible boards and clone systems proliferate. Yet ISA preserved the limits of a shared parallel bus: devices contended for common wires, configuration depended on jumpers or firmware conventions, and timing margins tightened as frequency increased. IBM’s proprietary Micro Channel Architecture and the industry EISA response show that bus design also allocates market power. Compatibility, licensing and software support can decide the fate of an interconnect as decisively as signal integrity [67, 126, 81, 135].

PCI, specified by Intel in the early 1990s and later administered by PCI-SIG, introduced a cleaner processor-independent local-bus architecture, plug-and-play configuration space and bus mastering. A device could initiate transactions rather than waiting for the CPU to move each word. Bridges created a hierarchy, and software enumerated buses, devices and functions. PCI-X increased clock rates and transaction capability for servers, but both remained multi-drop parallel buses. At high rates, simultaneously switching many traces became difficult because skew, reflections and electrical loading had to remain within a shrinking fraction of a clock period [67, 126, 81, 135].

PCI Express 1.0, released in 2002, changed the physical structure while preserving much of the PCI software model. It replaced the shared parallel medium with point-to-point serial links joined by switches. A link contains one or more lanes, each with separate transmit and receive differential pairs. Transactions are packetised; link-layer sequence numbers, credits and replay mechanisms provide reliable delivery over each hop. The specification revision history distinguishes the initial 2002 release from version 1.0a in April 2003 [80]. Software still discovers configuration space and Base Address Registers, so an operating system can treat a radically different fabric as the successor to PCI rather than a wholly unrelated device model [67, 126, 81, 135].

Each generation increased signalling rate and refined encoding. Early PCIe generations used 8b/10b encoding; PCIe 3.0 moved to 128b/130b. PCIe 6.0 adopted PAM4 signalling, fixed-size flow-control units and forward error correction. PCIe 7.0 version 1.0, released by PCI-SIG on 11 June 2025, doubles the raw rate to 128.0 GT/s per lane. A sixteen-lane link has a very large aggregate transfer capability, but the raw GT/s figure is not identical to application payload bandwidth because encoding, flit, protocol and packet overheads remain. Retimers, equalisation and board design are now integral to what older textbooks might have drawn as one line labelled “bus” [81].

Ordering also became more subtle. A shared bus gives a natural global observation order for many operations. A switched packet fabric must specify which transactions may pass one another, how posted writes complete, when a read acts as an ordering point and how caches interact with memory-mapped I/O. Device drivers use architecture- and kernel-provided accessors rather than assuming that a C load or store has the required device ordering. Linux memory-barrier documentation separates compiler ordering, CPU memory ordering and I/O ordering because each is a different contract [40, 35].

Compute Express Link adds another step. CXL was announced in 2019 as an industry interconnect using the PCIe physical layer while defining protocols for I/O, cache-coherent accelerator access and host-managed device memory. CXL 4.0, publicly released in late 2025 with an evaluation specification dated February 2026, raises operation to 128 GT/s, supports bundled ports and extends memory reliability, availability and serviceability features. The target is no longer merely a peripheral that exposes registers and DMA queues. A device may be a memory expander, accelerator or switch in a fabric where processors and devices share data under explicit coherence and pooling rules [82, 135].

Device memory can differ from local DRAM in latency, bandwidth, persistence and failure containment. A load to a pooled memory region consequently raises two separate questions. Which component owns the data, and what service can the interconnect provide when another requester needs it? [67, 126, 81, 135].

The line from Unibus and S-100 to PCIe and CXL is a sequence of answers to the same four questions: who may initiate a transfer, how is the target named, what ordering is promised, and how are failures contained? The form changed from shared

copper traces to high-speed serial packets and coherent fabrics, but the need for a precise contract only increased. Current facts should be taken from PCI-SIG and CXL specifications; historical software-visible behaviour should be checked against the corresponding architecture and operating-system documentation rather than inferred from a simplified block diagram [81, 82, 135, 40].

PCIe and coherent fabrics still implement addressed transactions, arbitration, ordering and flow control inherited from earlier interconnect problems.

Concept check

1. A bus has 32 data lines: how many bytes per transfer?
2. Give the raw bandwidth of a 4-byte bus at 400 MHz with one transfer per clock.
3. Name the three phases of a transaction, in order.
4. With 24 address lines, how many cells can be addressed?
5. Which component decides who becomes bus master?

Solutions to the review questions

1. $32/8 = 4$ bytes are carried by one full-width data transfer. A transaction can require several such transfers plus control overhead.
2. With one full-width transfer per clock, $4 \cdot 400 \cdot 10^6 = 1.6 \cdot 10^9$ bytes/s, or 1.6 GB/s. This is the raw rate; wait states, arbitration and protocol overhead reduce useful throughput.
3. In the stated shared-bus model, arbitration first selects an initiator, address/-control identifies the operation and destination, and data transfer completes the exchange. A protocol can pipeline or overlap phases, but each transaction still needs their corresponding decisions.
4. $2^{24} = 16{,}777{,}216$ distinct addresses are available. If a cell is one byte, this is 16 MiB. The address count alone does not specify the cell width.
5. The arbitration mechanism chooses which requester receives ownership. It can be centralised or distributed and must apply the stated priority or fairness policy. A requester becomes master after receiving the corresponding grant.

4.4.6 Worked problems

Rate, encoding and payload efficiency

A 32-bit binary data bus provides 4 bytes per beat. At 250 million beats/s its raw data rate is $4 \cdot 250 \cdot 10^6 = 10^9$ bytes/s, or 1 GB/s. A 64-bit bus at 125 million beats/s has the same raw rate. If each transaction carries 64 payload bytes and 16 overhead-byte equivalents, useful efficiency is $64/(64+16) = 0.8$. A continuously

Worked example (continued)

busy 1 GB/s raw channel then carries 0.8 GB/s of payload under that overhead model.

Clock frequency and DDR transfer rate

An interface transfers on both edges of a 400 MHz data clock. It therefore provides $2 \cdot 400 = 800$ million beats/s, or 800 MT/s. For a 64-bit data path this corresponds to $800 \cdot 10^6 \cdot 8 = 6.4$ GB/s before protocol overhead. Clock frequency, beat rate and byte rate have different units and describe different quantities.

Queue waiting and response time

Three complete transfers, each requiring 5 µs, are queued ahead of a new request on a serial server. With no request already partially serviced and no overhead, the new request waits $3 \cdot 5 = 15$ µs before service begins. If its own service requires 4 µs, its response time is $15 + 4 = 19$ µs. With a transfer already in progress, only that transfer's remaining service time contributes to the wait.

Exercise 4.4.1 — Address space

Twenty address lines, byte addressed?

Worked solution

Twenty binary address lines encode $2^{20} = 1,048,576$ distinct addresses. With byte addressing, this names 1,048,576 bytes, or $\boxed{1\ \text{MiB}}$. The largest address is $2^{20} - 1$. If a different design addressed words rather than bytes, the storage capacity in bytes would require multiplying by the word size; the stated byte-addressing assumption fixes the answer here.

Exercise 4.4.2 — Beats

How many 8-byte beats for 100 bytes?

Worked solution

Each beat carries up to 8 bytes. Twelve complete beats transfer $12 \cdot 8 = 96$ bytes, leaving 4 bytes. One more beat is needed, giving $\lceil 100/8 \rceil = \boxed{13}$ beats. The final beat contains four useful bytes and four byte positions that must be masked or ignored according to the bus protocol. Rounding down to 12 would omit part of the transfer.

Exercise 4.4.3 — Arbitration

Purpose of arbiter?

Worked solution

The arbiter chooses which requesting agent is granted use of the shared bus. When two devices request simultaneously, its policy determines the next master and may also determine fairness or priority. After a grant, the selected master performs the transaction under the protocol. Arbitration chooses ownership of the shared transport; the address and data phases still specify the actual operation.

Exercise 4.4.4 — Handshake

What closes an asynchronous request?

Worked solution

An asynchronous transaction ends through the acknowledgement or completion event defined by the handshake protocol. The requester keeps the required signals valid until the receiver has accepted them; both sides then return the handshake to its idle state. Unlike a fixed-cycle assumption, this permits variable response delay. Completion and error indications must be distinguished if the protocol provides both.

Summary

When evaluating an interconnect, avoid a single headline number. State payload size, direction, concurrency, protocol overhead and contention. Then distinguish peak theoretical bandwidth from sustained useful bandwidth. This same decomposition will explain why storage and DMA performance depend on more than device media speed.

4.4.7 Exercises

Exercise 4.4.5 — Raw bandwidth

A 32-bit bus runs at 125 MHz and transfers one word per cycle. Compute peak bandwidth.

Go to the solution →

Exercise 4.4.6 — Protocol efficiency

A burst transfers 16 data beats and uses 3 non-data cycles. What fraction of cycles carry payload?

Go to the solution →

Exercise 4.4.7 — Transaction ordering

List a plausible sequence for a simple bus read.

Go to the solution →

Exercise 4.4.8 — Fixed-priority starvation

Why can a low-priority master starve? Give one remedy.

Go to the solution →

Exercise 4.4.9 — Burst trade-off

A master doubles its burst length. State one likely throughput benefit and one latency cost.

Go to the solution →

Exercise 4.4.10 — Posted write

Why might software perform a read after programming a critical device register?

Go to the solution →

Exercise 4.4.11 — Useful bandwidth

A 4 GB/s link has 85% payload efficiency and is busy 70% of the time. Estimate useful payload rate.

Go to the solution →

Exercise 4.4.12 — Topology reasoning

Explain why PCIe can preserve bus-like memory transactions even though its physical topology is point-to-point and switched.

Go to the solution →

4.4.8 Solutions

Buses, Transactions, and Interconnection

Solution 4.4.5

Each 32-bit transfer carries $32/8 = 4$ bytes. At one transfer per cycle and $125 \cdot 10^6$ cycles/s, peak bandwidth is $4 \cdot 125 \cdot 10^6 = 500 \cdot 10^6$ bytes/s, or 500 MB/s in decimal units. The corresponding binary rate is approximately 476.84 MiB/s. Address phases, idle cycles and protocol overhead would reduce useful payload bandwidth below this raw limit.

← Return to the exercise

Solution 4.4.6

A complete burst occupies $16 + 3 = 19$ cycles. Sixteen carry useful data and three carry overhead. Payload efficiency is therefore $16/19 \approx 0.8421$, or 84.2%. For a link with raw rate B, this pattern delivers approximately $0.8421B$ when bursts run continuously. Dividing by 16 alone would omit the time spent on the non-data cycles.

← Return to the exercise

Solution 4.4.7

Request/arbitration, grant, address and read command, target decode, target response, data transfer, completion/release. Protocols may overlap or split these phases, but each function must exist.

← Return to the exercise

Solution 4.4.8

If higher-priority masters request continuously, the arbiter always selects them. Round-robin with finite grant lengths guarantees eventual service to every continuously pending requester. Ageing or quotas can also provide fairness if their rules enforce it. A maximum burst length alone is insufficient under strict priority: the highest-priority master could simply win again after every burst.

← Return to the exercise

Solution 4.4.9

Command and arbitration overhead are amortised over more data, raising payload efficiency. Other masters wait longer for the bus, increasing their response and worst-case latency.

← Return to the exercise

Solution 4.4.10

A posted write may be sitting in a bridge. A subsequent read from the device forces ordering and waits for a response, providing evidence that earlier writes reached the target, when the architecture and device specification define that behaviour.

← Return to the exercise

Solution 4.4.11

First account for payload efficiency: during busy periods the link delivers $4 \cdot 0.85 = 3.4$ GB/s of useful data. Busy periods occupy only 70% of total time, so the long-run rate is $3.4 \cdot 0.70 = 2.38$ GB/s. Over one second this model transfers 2.38 GB of payload. The factors describe separate losses, so their product applies; adding the percentages would combine incompatible denominators.

← Return to the exercise

Solution 4.4.12

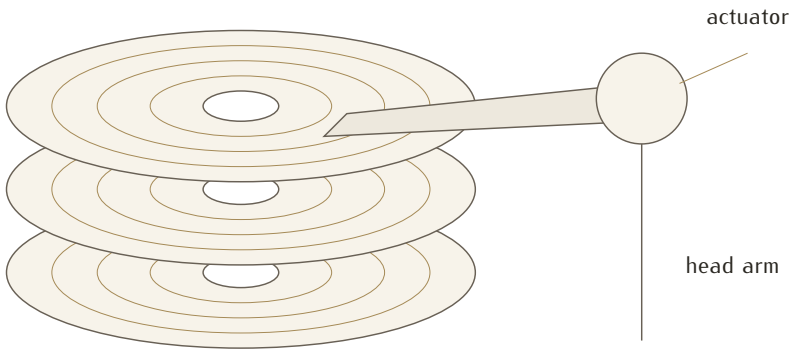
Endpoints still issue reads, writes and configuration transactions in defined address spaces. The fabric encodes them as packets and switches route them. The architectural transaction model is bus-like; the transport is network-like.

← Return to the exercise

4.5 MAGNETIC STORAGE AND ACCESS-TIME ANALYSIS

A disk receives a request for a block that lies away from the head's current position. The actuator moves, the platter continues to rotate, and the requested region eventually passes under the head. Data transfer can then begin. Following these motions separates positioning from transfer and gives each term in the access-time equation a physical meaning.

A rotating store moves data past a sensing head



The arm changes track; rotation brings the requested sector beneath it.

Figure 4.38: *The stack of platters and head arm makes the two positioning motions visible. The actuator selects a track and rotation supplies angular motion. Actual drives hide detailed placement behind logical block interfaces.*

4.5.1 Geometry and mechanical latency

Main memory, which holds the code and data of running programs, commonly responds in tens of nanoseconds. A mechanical disk may require several milliseconds for a random access. The resulting gap is often five to six orders of magnitude, although the exact ratio depends on the memory subsystem and drive. The geometric model explains how positioning contributes to access time.

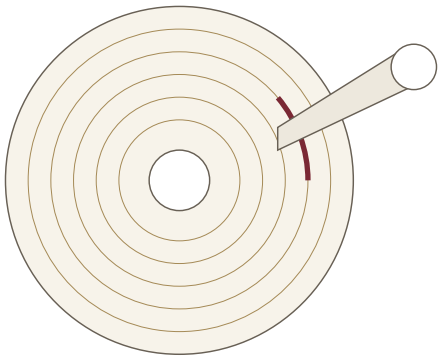
In the classical idealised cylinder/head/sector (CHS) model, a stack has s active recording surfaces; one must not infer mechanically that every physical platter contributes two usable surfaces. Each active surface is divided into concentric tracks, and a cylinder is the modelled set of one track at a common radius on every active surface. Tracks are divided into fixed-size sectors for the exercise. Historically this geometry was exposed through cylinder/head/sector addressing. Modern host software instead issues logical block addresses, and the controller maps those blocks to an evolving physical layout.



one sector on one track (shaded area)
one track (annular band)

tracks and sector regions on one surface

A track and a sector describe different directions



track: radial choice

sector: angular region

head: sensing position

The drawing highlights one angular region on one track.

Figure 4.39: *Moving the head radially changes the track. Rotation changes which angular region lies beneath it. The simplified geometry supplies intuition for seek and rotational delay without claiming modern logical blocks expose physical geometry directly.*

Because an outer track has a greater circumference than an inner track, a drive can place more sectors on outer radii while keeping a similar linear bit density. Firmware groups neighbouring tracks into zones with a common sectors-per-track value. Uniform sectors per track is an assumption of the capacity exercises. Zoned drives require a separate calculation for each zone.

For uniform geometry, capacity is the product of the following factors:

$$\text{capacity} = (\text{surfaces}) \times (\text{tracks/surface}) \times (\text{sectors/track}) \times (\text{bytes/sector}).$$

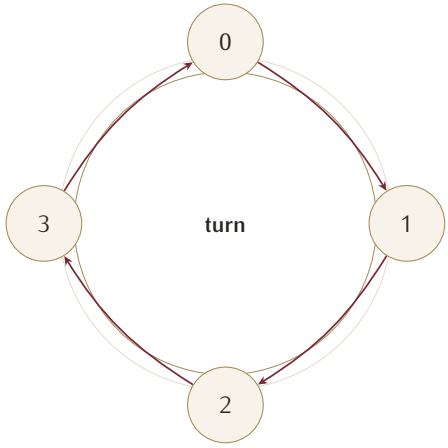
Worked example

Problem. A disk has 4 surfaces, 2000 tracks per surface, 500 sectors per track and 512 bytes per sector. What is its capacity?
Solution. $4 \times 2000 \times 500 \times 512 = 2\,048\,000\,000$ bytes, 2.048 decimal GB (about 1.907 GiB).

Worked example

Problem. In the classroom model, a disk has 2 platters and both faces of each platter are active. How many recording surfaces, and how many tracks make up one cylinder?
Solution. Surfaces: $2 \times 2 = 4$. A cylinder contains one track per surface, so 4 tracks.

Rotation carries successive regions past the head



Four schematic angular landmarks; the actual sector count can be much larger.

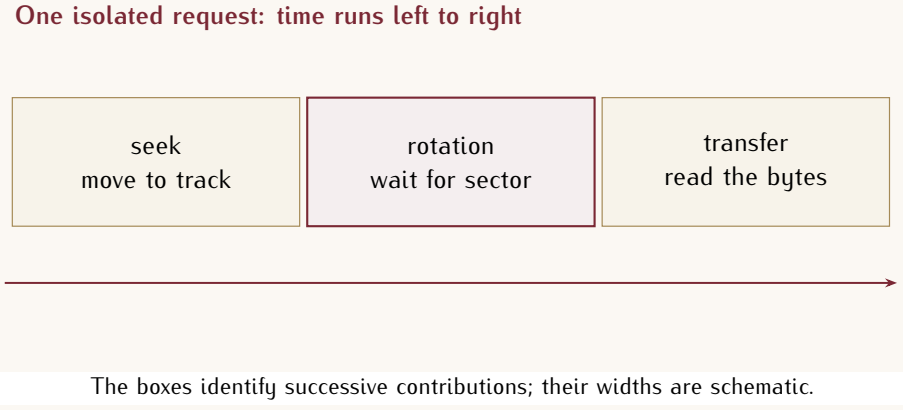
Figure 4.40: *After the head reaches a track, the desired region can still require a rotational wait. Under a uniform independent phase assumption, the mean wait is half a revolution.*

Positioning the head before decoding the data

The actuator brings the head to the required track; rotation brings the requested region beneath it. A magnetoresistive sensor responds to recorded magnetic transitions. The read channel amplifies and processes the resulting waveform, recovers

Physical mechanism (continued)

timing and decodes the protected sequence of bits. Servo information helps keep the head on track [47].



The boxes identify successive contributions; their widths are schematic.

Worked check. At 7200 revolutions per minute, a revolution takes 60/7200 s, or approximately 8.333 ms. If the requested angular position is uniform relative to the head after seek completion, the mean rotational wait is half a revolution: approximately 4.167 ms. With a stipulated 5 ms seek, positioning alone therefore averages about 9.167 ms, before transfer and controller overhead.
The uniform-phase assumption belongs to this estimate. A sequence of adjacent requests, queue reordering or prior knowledge of rotational position changes the observed distribution. The diagram separates physical contributions so that a reported throughput can be interpreted alongside the request pattern.

For an isolated request with an empty device queue, separate the service time into four terms:

$$T_{\text{service}} = T_{\text{radial}} + T_{\text{angular}} + T_{\text{data}} + T_{\text{controller}}.$$

The radial term covers actuator motion and settling; the angular term is the wait for the requested position to reach the head; the data term covers passage of the requested bytes through the channel; and the controller term collects command and interface overhead. Queueing delay is excluded and must be added when the device is busy.

The *rotational latency* depends on spindle speed and the sector's angular position when positioning ends. A full rotation takes 60/RPM seconds. Under a uniform, independent angular-position assumption, the average wait is half a rotation:

$$T_{\text{rot}}^{\text{avg}} = \frac{1}{2} \cdot \frac{60}{\text{RPM}} \quad \text{(seconds)}.$$

The *transfer time* for one sector is the time for the disk to sweep past one sector out of the track:

$$T_{\text{transfer}} = \frac{1}{\text{sectors/track}} \cdot \frac{60}{\text{RPM}} \quad \text{(seconds)}.$$

Worked example

Problem. A disk spins at 7200 RPM. What is the average rotational latency in milliseconds?
Solution. One rotation is 60/7200 s, or about 8.33 ms; half of it is 4.17 ms.

Worked example

Problem. A disk at 10000 RPM has 500 sectors per track. What is the transfer time of one sector?
Solution. One rotation is 60/10000 s, or 6 ms; one sector is $6/500 = 0.012$ ms $= 12$ microseconds.

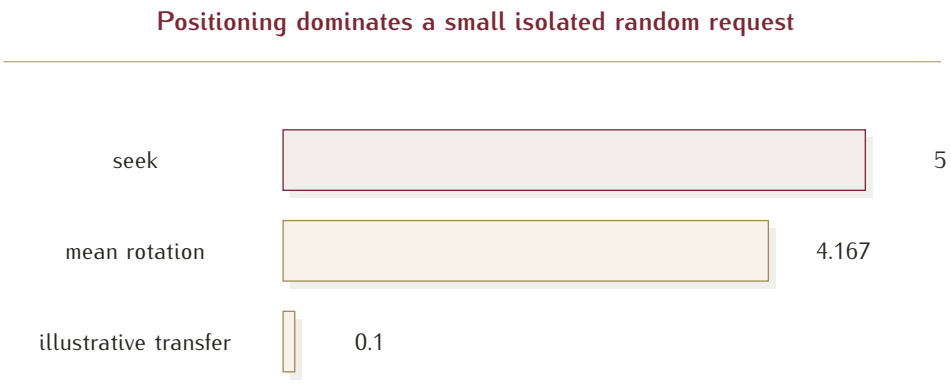


Figure 4.41: *The bars separate positioning and transfer in the declared example. Controller overhead and queueing delay are omitted. A different workload or device changes the contributions, especially for large sequential requests.*

The transfer term is usually small compared with seek and rotational latency. Mechanical positioning therefore dominates random-access performance. Sequential access is much faster because many neighbouring sectors can share one seek and one initial rotational wait.

Worked example

Problem. In a simplified model, a disk reports a mean radial positioning time of 4.2 ms, rotates at 10,000 RPM, and places 750 equal sectors on the current track. Ignore queueing and use 0.15 ms for controller overhead. Estimate the mean service time for one sector.
Solution. One revolution takes $60/10{,}000 = 0.006$ s $= 6$ ms, so the expected angular wait is 3 ms. One sector takes $6/750 = 0.008$ ms to pass the head. Therefore $T_{\text{service}} = 4.2 + 3 + 0.008 + 0.15 = 7.358$ ms, approximately 7.36 ms.

The average radial positioning time is a measured device characteristic. Approximating it as a fraction of a quoted full-stroke seek is only a rough classroom model and is not a universal physical law.

Worked example

Problem. Reading k consecutive sectors on the same track: after the average rotational latency, the k sectors stream past. Write the time.
Solution. $T = T_{\text{rot}}^{\text{avg}} + k \cdot T_{\text{transfer}}$. This expression starts with the head already settled on the track and excludes controller overhead. Only transfer scales with k; an initial seek must be added if the head starts elsewhere.

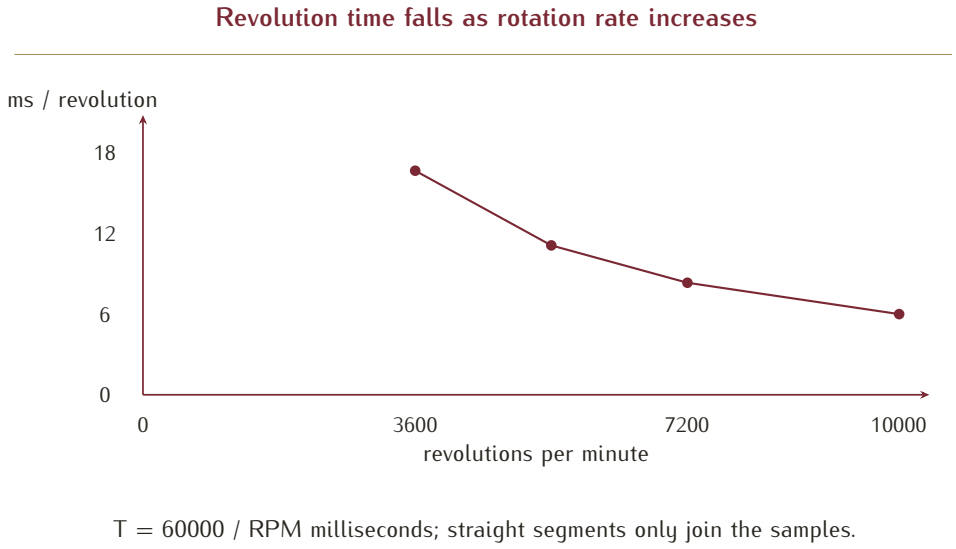
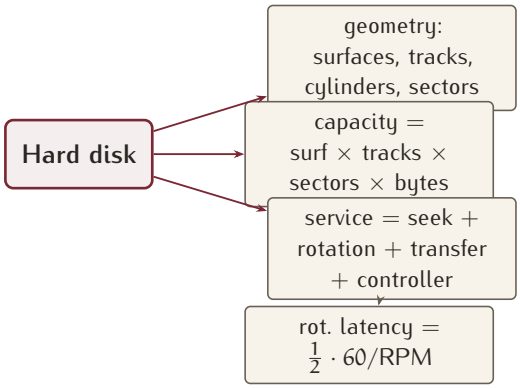


Figure 4.42: *The sampled points follow the reciprocal relationship between rotation rate and revolution time. The mean rotational wait is half this time only under the stated uniform-phase assumption.*

A track delivers (sectors/track)(bytes/sector)(RPM/60) bytes per second while it streams under the head. With small random requests, the head spends much of its time seeking and waiting for rotation. Caching frequently used blocks in RAM avoids that positioning work on subsequent hits.



4.5.2 Logical blocks and quantitative performance

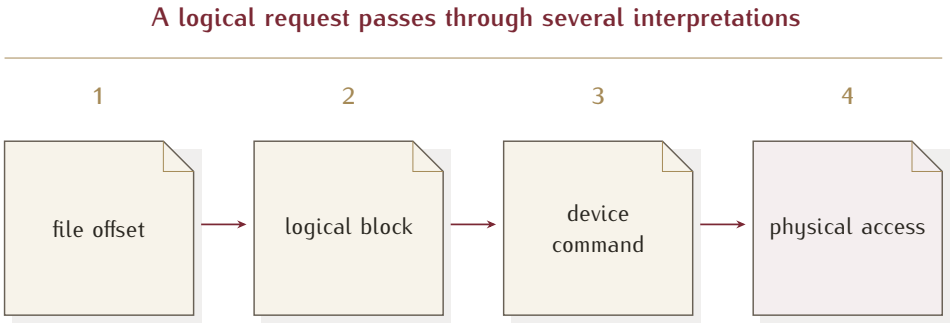


Figure 4.43: *Filesystem allocation maps file positions to logical storage blocks. The device translates those requests into internal operations. A host-visible logical block number therefore need not reveal a track or sector geometry.*

Transfer time for n bytes at sustained media rate B is n/B. This gives a convenient form of the service-time model for requests spanning several sectors.

Worked example: a random 4 KiB read

A 7200 RPM disk has average seek time 8 ms and sustained transfer rate 160 MB/s. One revolution takes 60/7200 seconds, or approximately 8.33 ms, so average rota-

Worked example (continued)

tion is 4.17 ms. Transferring 4096 bytes takes about 25.6 µs. Ignoring controller overhead,

$$T \approx 8 + 4.1667 + 0.0256 = 12.1923 \text{ ms}.$$

Positioning takes about $12.1667/0.0256 = 475$ times as long as transferring the 4 KiB payload.

For long sequential requests, seek and rotation are paid once and the device approaches its transfer bandwidth. For independent random requests, each operation pays positioning cost, so one outstanding request with no application think time yields approximately $1/E[T_{service}]$ operations per second. Parallel queues and controller reordering require a different model.

In the example above, a single outstanding random read yields roughly 82 IOPS. Its stated sequential rate is 160 MB/s. "Disk speed" is therefore incomplete without workload shape, request size, queue depth and read/write mix.

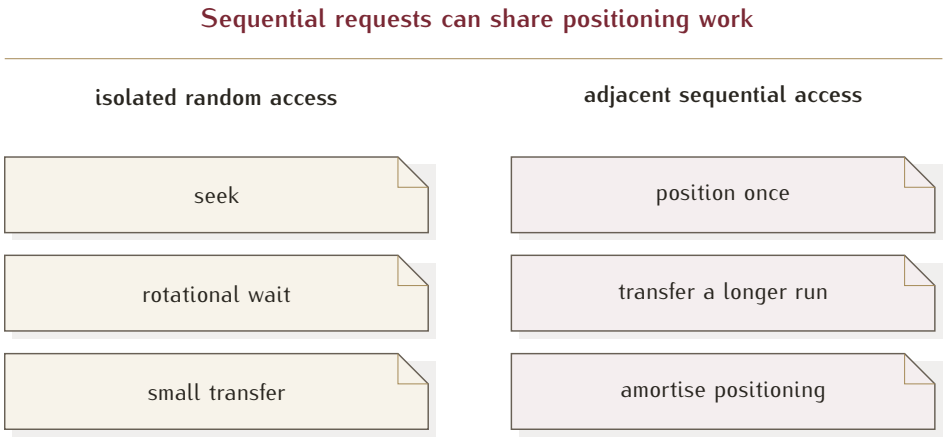


Figure 4.44: *The same device can deliver very different workloads at very different rates. A long contiguous transfer spreads positioning cost over many bytes, while isolated random requests repeatedly pay it.*

Worked example: amortising positioning

Using the preceding disk, read 8 MiB sequentially. Transfer time is about 8 MiB/160 MB/s ≈ 52.4 ms. Adding one average seek and rotation gives roughly

Worked example (continued)

64.6 ms. Splitting the same data into 2048 independent random 4 KiB reads would require about $2048 \cdot 12.2$ ms, around 25 seconds without queue optimisation.

4.5.3 Queues, scheduling and caching

When several requests are pending, the operating system or drive can reorder them to reduce head movement. First-come, first-served preserves arrival order and avoids starvation when requests complete, but may travel inefficiently. Shortest-seek-time-first favours the nearest request but can starve distant ones. SCAN moves the head in one direction serving requests along the path, then reverses, resembling an elevator.

Modern devices obscure exact geometry and implement internal queues, so software schedulers use deadlines, workload classes and device knowledge rather than literal cylinder numbers alone. The design objective balances throughput, fairness and tail latency.

Worked example: FCFS versus a nearest-request policy

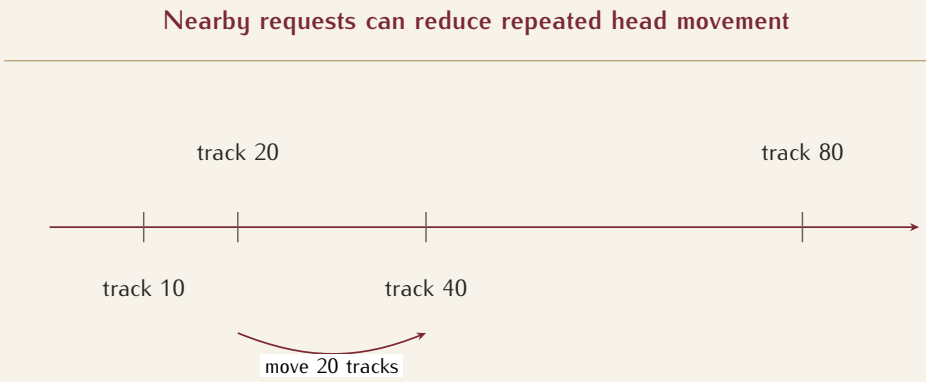


Figure 4.45: *Distance can help explain a scheduling decision in a simplified rotating-disk model. Real seek time is not universally linear in distance, and a complete scheduler also considers rotation, priorities and fairness.*

The head starts at track 50 and requests are at 10, 55, 58 and 90. FCFS in listed order moves $40 + 45 + 3 + 32 = 120$ tracks. Serving nearest requests gives 55, 58, 90, 10 for $5 + 3 + 32 + 80 = 120$ tracks here, so no gain. If the order were 90,10,58,55, FCFS would move $40 + 80 + 48 + 3 = 171$ tracks, while a local ordering can be

Worked example (continued)

much shorter. For this second order, nearest-first saves $171 - 120 = 51$ tracks of movement.

The page cache stores recently used file data in memory. Sequential detection can trigger read-ahead, issuing future reads before the application asks for them. Writes are often buffered and acknowledged before media persistence, then flushed later. This improves throughput and allows merging, but it creates a durability distinction between "copied into memory" and "safely stored".

Applications that require ordering and persistence use interfaces such as `fsync` and carefully designed update protocols. Drive write caches need power-loss protection or explicit flush commands to honour those guarantees. Reliability cannot be inferred from a successful `write` alone.

If requests arrive near the device's service capacity, waiting time grows rapidly. A mean access time of 10 ms does not imply a stable 100-request/s service at 100-request/s arrival rate; there is no headroom for variation. Queue depth can improve throughput by exposing reordering opportunities, yet it also increases the time an individual request waits.

A practical analysis reports percentiles alongside averages. Interactive systems care about the slowest requests, while batch workloads may prioritise aggregate bandwidth.

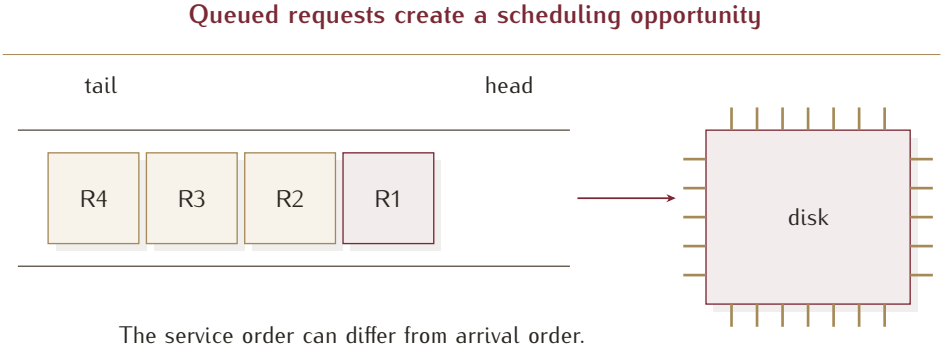


Figure 4.46: *A scheduler may choose a nearby or otherwise advantageous request while others wait. The request's total response time includes queueing as well as its eventual service time.*

Security consequence: persistence and deletion differ

Deleting a file usually removes metadata references; it does not immediately overwrite every data block or cached copy. Snapshots, remapped sectors and storage-controller

caches further complicate erasure. Encryption with well-managed keys is often more reliable than trying to prove that every historical physical location has been overwritten.

Laboratory 28: Storage measurement interpretation

A synthetic report reads 64 MiB sequentially in 0.08 s on a first run and 0.01 s on a repeat. A separate random test performs 1000 reads of 4096 bytes in 0.20 s. Calculate the rates and explain what additional evidence is needed to identify device performance.
Worked solution

Laboratory 28: solution

Sequential rates are $64/0.08 = 800$ MiB/s and $64/0.01 = 6400$ MiB/s. The random test transfers $1000 \cdot 4096 = 4{,}096{,}000$ bytes, or 3.90625 MiB, giving 19.53125 MiB/s and $1000/0.20 = 5000$ operations/s. The repeat's higher rate can reflect cached data served from RAM. The report must state cache state, request concurrency, buffering, file placement and device details before attributing any number to the storage medium. These are calculations from supplied synthetic timings, not benchmark results from a particular host. Repeated runs and dispersion are necessary to assess stability.
Return to the laboratory

Filesystems and crash consistency

A storage device exports blocks; a filesystem turns them into named objects, directories, metadata and allocation state. Implementation choices include how a file maps logical offsets to blocks, how directories map names to identifiers, how free space is tracked and how cached changes reach persistent media [59].

Crash consistency is an ordering problem. One logical update may require several writes – for example, data, allocation metadata, an inode and a directory entry. A crash between them can expose an impossible combination unless the filesystem constrains the order or records recovery information. Journaling writes a description or protected form of an update before making the corresponding home-location changes; log-structured designs organise new state as sequential log records and later reclaim obsolete segments.

Neither technique makes every completed API call durable. Durability depends on the interface contract, explicit synchronisation, controller caches and the storage device's ordering guarantees. Correct reasoning states the crash point and the promised persistence boundary.

Exercise 4.5.1 — Crash-point analysis

A program creates a file, writes one block, renames it and reports success. List the metadata and data updates that may be involved. Choose three crash points and state which outcomes are acceptable under a clearly stated durability contract. Explain what journaling can repair and what still requires explicit synchronisation.

Solution 4.5.1

Assume a filesystem and storage stack that provide separate operations to synchronise file contents and directory changes, honour completion of those operations across power loss, and support same-directory rename that is atomic both during execution and after crash recovery. Assume any initial destination state is already durable. The application promises that after it reports success, the final name refers to the complete new block. Before success, the old state or the complete new state is acceptable; a success report followed by missing or partial new contents violates the promise.

Creating a temporary file can update its inode, allocation information and parent directory. Writing one block can allocate a data block and change its contents, length, timestamps and allocation metadata. Renaming changes directory entries and may update link counts or remove an old destination. Their exact on-disk organisation depends on the filesystem.

A protocol satisfying the stated contract is: create a temporary file in the destination directory; write the whole block, checking short writes and errors; synchronise that file successfully; rename it to the final name; synchronise the directory successfully; then report success. If a step fails, success must not be reported.

1. A crash during the write occurs before the file synchronisation. The temporary file may be absent, incomplete or complete. The application has not promised a durable new final file. Recovery may discard an abandoned temporary file.
2. A crash after file synchronisation but before rename can leave a complete durable temporary file and the old final name, if one existed. This satisfies the pre-success contract. Recovery can decide whether to complete or discard the pending operation.
3. A crash after rename but before directory synchronisation can expose the old or new namespace state after recovery, according to the filesystem's guarantees. Success is still withheld. After the directory synchronisation has completed, the stated assumptions require the final name and complete block to survive.

Journaling can replay or roll back metadata transactions to restore a structurally consistent filesystem. Metadata consistency alone does not establish that the application's payload and chosen final name are durable. The file and directory synchronisation points connect that application promise to the storage contract. Cross-

Worked solution (continued)

directory rename, replacement of an existing file and filesystems with weaker guarantees require an explicitly extended protocol.

From magnetic drums to heat-assisted magnetic recording

Timeline

Year	Machine, standard or event	Architectural significance
1956	IBM 350	Commercial random-access disk storage uses moving heads and platters.
1973	IBM 3340 Winchester	Sealed head-disk assemblies improve density and reliability.
2023	Commercial HAMR	Heat-assisted recording extends magnetic areal density.

Magnetic secondary storage began with devices whose geometry was visible to programmers. Gustav Tauschek patented a magnetic drum in 1932. A drum stored bits along tracks on a rotating cylinder, normally with fixed heads. Once electronic computers adopted drums in the late 1940s and 1950s, access time depended on rotation: the requested word had to arrive under its head. On machines such as the IBM 650, announced in 1953, programmers could use "optimum programming" to place the next instruction near the position expected when the current instruction finished. Programmers could consequently schedule instructions by choosing their locations on the rotating drum [112, 98].

Magnetic tape solved a different problem. UNIVAC I and early IBM installations used reels to store much more data than electronic memory could hold. Tape was inexpensive and efficient for long sequential transfers, but reaching a record required passing earlier records. IBM's vacuum-column tape drives in the early 1950s allowed tape to start and stop without subjecting an entire reel to the same acceleration. The result helped replace enormous punched-card workflows, yet it did not provide direct access in the modern disk sense [115, 98].

The decisive disk milestone was IBM's RAMAC project in San Jose under Reynold B. Johnson. The IBM 305 RAMAC, announced in 1956, used the IBM 350 disk storage unit. Fifty 24-inch disks held roughly five million decimal characters, commonly described as about 3.75 MB. A pair of heads moved vertically and radially to select a surface and track. The capacity is tiny by present standards, but the conceptual change was enormous: records could be reached directly and rewritten without sorting a card deck or scanning a tape. IBM's own historical account identifies RAMAC as the first computer to use a random-access disk drive [75, 98].

Subsequent designs made the mechanical model familiar. The IBM 1301 of 1961 used air-bearing “flying” heads. Removable disk packs allowed data to move between drives. The IBM 3340, introduced in 1973 under the code name Winchester, incorporated low-load landing heads and protected head-disk modules that influenced later drives; the term “Winchester” was subsequently applied broadly, sometimes imprecisely, to sealed hard disks. Voice-coil actuators, servo information and embedded control improved positioning. By the 1980s, 5.25-inch and then 3.5-inch drives brought disks into personal computers, while interfaces such as ST-506, SCSI, IDE/ATA and later SATA moved increasing amounts of control logic into the drive [98, 75, 84].

The standard access-time equation comes directly from this history. A random request waits for the actuator to seek the target track, for rotation to bring the sector under the head, and for transfer. Average rotational latency is half a revolution only under a uniform-arrival assumption; seek time is not a simple linear function of track distance; command queueing and caching alter observed latency.

Logical Block Addressing hides cylinders, heads and sectors from ordinary software, but it does not abolish mechanics. Firmware maps logical requests to physical locations, reorders queues, manages defective sectors and uses caches. A performance model should state whether it describes bare media geometry, drive-level service time or end-to-end queueing [98, 75, 84].

Recording physics repeatedly approached density limits and changed technique. Longitudinal recording magnetised regions in the plane of the platter. Perpendicular magnetic recording, introduced commercially in the mid-2000s, oriented magnetisation through the plane and improved thermal stability at smaller bit sizes. Shingled Magnetic Recording overlaps adjacent tracks like roof shingles; it raises density but constrains random rewriting, so host-managed or device-managed translation is required. Helium-filled enclosures reduce aerodynamic drag and permit more platters. These developments produced a mechatronic system with signal processing, error correction, cache, firmware and sometimes explicit write-placement rules [98, 75, 84].

Heat-Assisted Magnetic Recording addresses the conflict between very small stable grains and the magnetic field available from a write head. A near-field optical transducer briefly heats a microscopic region so it can be written, after which it cools into a thermally stable state. HAMR had a long research and qualification period because the head must repeatedly create local heating while maintaining nanometre-scale positioning and long service life.

On 3 March 2026 Seagate reported its Mozaic 4+ HAMR platform qualified and in production with two hyperscale providers, supporting products up to 44 TB. Seagate’s figures describe the particular platform and qualification reported in that announcement [84].

Hard disks have not been made obsolete by flash. SSDs dominate workloads that value low random-access latency and high IOPS, but HDDs retain a cost-per-capacity advantage in large archival and cloud tiers. Their performance is increasingly shaped by

parallel queues, host scheduling, erasure coding and distributed storage rather than by one isolated seek. The mechanical service-time formula remains useful when combined with the queueing, caching and persistence behaviour of the storage stack [98, 75, 84].

Applications see logical blocks, while firmware hides zones, defects and physical geometry behind an LBA interface.

Concept check

1. A disk uses both faces of each of its 3 platters. How many recording surfaces does it have?
2. Average rotational latency at 15000 RPM?
3. Write the capacity formula.
4. Transfer time of one sector at 7200 RPM with 500 sectors/track.
5. Why is sequential access so much faster than random access?

Solutions to the review questions

1. With both faces of every platter used, three platters provide $2 \cdot 3 = 6$ recording surfaces. A model with unused faces must state a smaller number explicitly.
2. One revolution takes $60/15000 = 0.004$ s, or 4 ms. For uniformly distributed angular arrival, the average rotational wait is half a revolution: 2 ms.
3. For a uniform geometry, capacity is $\mathit{surfaces} \cdot \mathit{tracks\ per\ surface} \cdot \mathit{sectors\ per\ track} \cdot \mathit{bytes\ per\ sector}$. All factors must describe the same model. Zoned recording requires summing the capacities of zones with different sector counts.
4. At 7200 RPM, a revolution takes $60/7200 = 1/120$ s. With 500 equally sized sectors per track, one sector occupies $1/(120 \cdot 500)$ s, or about 16.67 µs of transfer time. Seek and rotational waiting are additional costs.
5. A sequential run can amortise positioning over many adjacent sectors and sustain transfer after the head reaches the stream. Random requests repeatedly incur seek and rotational delays. The size and ordering of requests determine how much of that positioning cost can be shared.

4.5.4 Worked problems

Block allocation and logical offsets

Assume a file uses whole 4096-byte data blocks, with no compression, holes or tail packing. A 10,000-byte file requires $\lceil 10000/4096 \rceil = 3$ blocks: two hold 8192 bytes, and the third holds the remaining 1808 bytes. The allocated capacity is 12,288

Worked example (continued)

bytes; the last block has $4096 - 1808 = 2288$ unused bytes. Filesystem metadata consumes additional space outside this calculation.
For a separate device exposing 512-byte logical blocks numbered from zero, LBA 250 begins at byte offset $250 \cdot 512 = 128000$. Dividing that offset by 512 returns 250 exactly. Logical block numbering identifies the device address; the firmware determines its physical location.

Exercise 4.5.2 — Surfaces

Four double-sided platters provide how many surfaces?

Worked solution

Each double-sided platter supplies two recording surfaces. Four such platters therefore provide $4 \cdot 2 = \boxed{8}$ surfaces. This count assumes both sides are usable recording surfaces, as stated. Tracks on corresponding radii across the surfaces form a cylinder in the geometric model. Surface count alone does not determine capacity; sectors per track and the number of tracks are also needed.

Exercise 4.5.3 — Period

10,000 RPM period?

Worked solution

At 10,000 revolutions per minute, one revolution takes $60/10000 = 0.006$ seconds, or $\boxed{6\text{ ms}}$. Under a uniform random arrival angle, the average rotational wait is half a revolution, 3 ms. The question asks for the full period, so 6 ms is the answer. Seek and transfer time are separate contributions to a complete disk access.

Exercise 4.5.4 — Rounding

How many 4 KiB blocks for 9000 bytes?

Worked solution

One block holds 4096 bytes. Two blocks hold 8192 bytes, leaving $9000 - 8192 = 808$ bytes for a third block. Thus the file requires $\lceil 9000/4096 \rceil = \boxed{3}$ data blocks. Their allocated capacity is 12,288 bytes, leaving 3288 unused bytes in the last block under this simple allocation model. Metadata blocks are excluded unless requested separately.

Exercise 4.5.5 — Locality

Why are adjacent blocks faster in sequence?

Worked solution

Once the head is positioned and the desired region rotates underneath it, adjacent blocks can often be streamed with little additional positioning. A sequence of unrelated requests may require another seek and rotational wait for each access. The transfer still consumes time proportional to the data size. Sequential locality improves the fraction of access time devoted to payload transfer, subject to the actual layout and controller scheduling.

Summary

Storage analysis should begin with the access pattern. Count random positioning events, estimate rotational waits and separate them from streaming transfer. A large bandwidth figure cannot rescue a workload dominated by thousands of independent seeks. Conversely, a sequential workload can exploit the medium far more effectively than its average random-access latency suggests.

4.5.5 Exercises

Exercise 4.5.6 — Rotation

Compute one revolution and average rotational latency for a 10,000 RPM disk.

Go to the solution →

Exercise 4.5.7 — Complete access

Average seek is 6 ms, rotation is 3 ms, controller overhead is 0.2 ms, and 64 KiB transfer at 200 MB/s takes how long? Find total.

Go to the solution →

Exercise 4.5.8 — Approximate IOPS

If average random access is 12.5 ms and one request is outstanding, estimate IOPS.

Go to the solution →

Exercise 4.5.9 — Sequential amortisation

A 100 MiB sequential read uses a 150 MB/s transfer rate and pays 11 ms positioning once. Estimate total time.

Go to the solution →

Exercise 4.5.10 — Scheduling distance

Head starts at 40. Requests are 10, 35, 90, 42. Compute FCFS movement and one nearest-first order.

Go to the solution →

Exercise 4.5.11 — Queue saturation

A device can sustainably serve 100 requests/s. Why is an arrival rate of 99 requests/s risky despite being below capacity?

Go to the solution →

Exercise 4.5.12 — Durability

Why does successful return from `write` not necessarily imply persistence on magnetic media?

Go to the solution →

Exercise 4.5.13 — Measurement design

Design a test that distinguishes storage performance from page-cache performance.

Go to the solution →

4.5.6 Solutions

Magnetic Storage and Access-Time Analysis

Solution 4.5.6

At 10,000 revolutions per minute, one revolution lasts $60/10{,}000 = 0.006$ s $= 6$ ms. A request arriving at an independent uniformly distributed rotational phase waits from almost zero to almost one full revolution, with mean half a revolution: $6/2 = 3$ ms. The average assumes a random target sector and excludes seek, controller and transfer time.

← Return to the exercise

Solution 4.5.7

Convert 64 KiB to $64 \cdot 1024 = 65{,}536$ bytes. At decimal 200 MB/s, transfer time is $65{,}536/(200 \cdot 10^6) = 0.00032768$ s $= 0.32768$ ms. Add the serial components: $6 + 3 + 0.2 + 0.32768 = 9.52768$ ms, approximately 9.528 ms. All terms are now in milliseconds. The mixed KiB and MB units require the explicit conversion before division.

← Return to the exercise

Solution 4.5.8

With one outstanding request, each completed operation consumes its entire mean access time before the next can finish. Convert 12.5 ms to 0.0125 s, then take the reciprocal: IOPS $= 1/0.0125 = 80$ operations/s. Multiplying $80 \cdot 0.0125$ gives one second, which checks the units. Concurrent requests, caching or request reordering would require a different throughput model.

← Return to the exercise

Solution 4.5.9

100 MiB $= 100 \cdot 2^{20} = 104{,}857{,}600$ bytes. At $150 \cdot 10^6$ bytes/s, transfer takes $104857600/(150 \cdot 10^6) = 0.6990507$ s. Adding the one-time positioning cost of 0.011 s gives 0.7100507 s, about 710 ms.

← Return to the exercise

Solution 4.5.10

FCFS follows $40 \to 10 \to 35 \to 90 \to 42$, travelling $30 + 25 + 55 + 48 = 158$ tracks. Nearest-first initially chooses 42, two tracks away, then 35, seven away. From 35, request 10 is closer than 90, so the final order is $42, 35, 10, 90$, travelling $2 + 7 + 25 +$

Worked solution (continued)

$80 = 114$ tracks. The saving is 44 tracks. New arrivals could continually postpone a distant request under nearest-first scheduling.

← Return to the exercise

Solution 4.5.11

The offered load is $99/100 = 0.99$, leaving capacity for one extra request per second on average. If a burst puts ten requests ahead of a caller, clearing the excess while new requests keep arriving at 99/s takes about ten seconds in a constant-rate approximation. Individual service can still take only 10 ms. Variable arrival and service times therefore produce long waits even while the mean arrival rate stays below capacity.

← Return to the exercise

Solution 4.5.12

The bytes may only be in the kernel page cache or a volatile drive cache. Persistence requires the documented flush/synchronisation protocol and hardware that honours it.

← Return to the exercise

Solution 4.5.13

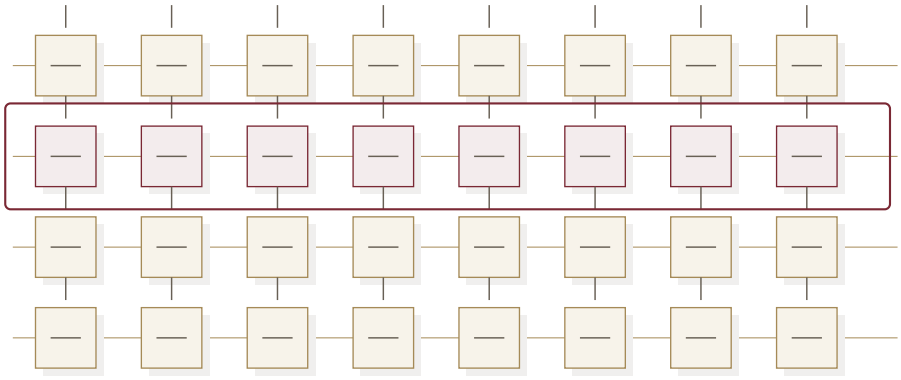
Create a sufficiently large, fully allocated test file and record its size, the filesystem and the device. Fix the request size, offsets, queue depth and read order. Time a first buffered read and repeat the same read with the same settings. Record bytes read, elapsed time and physical device-read counters on an otherwise quiet system. A repeated read with high application throughput and little new device traffic indicates page-cache service. To measure the storage path, use a supported direct-I/O read with the required offset, buffer and length alignment, or a controlled working set that exceeds the cache. Check that physical read traffic occurs. Drive caches and request reordering still affect the storage measurement. Repeat the trials and report dispersion separately for cold, warm and direct reads.

← Return to the exercise

4.6 RAID AND SOLID-STATE STORAGE

A storage update may place a second copy on another drive, recompute parity or write a new flash page behind an unchanged logical address. Each choice changes which failures can be recovered and what work a later request requires. We will follow the placement of data through RAID layouts and flash translation, keeping the surviving copies, controller metadata and physical media operations visible.

Pages and erase blocks arise from the organisation of cells



One row is highlighted as a page; erase operations cover a larger block.

Figure 4.47: *The repeated cell symbols show an organised array with one row highlighted as a page. The group represents a larger erase unit schematically. Page and block sizes, cell design and programming restrictions are device-specific.*

4.6.1 Redundancy, failure models and RAID

A storage-device failure can destroy information accumulated over years. Adding devices does not remove the risk: as the population grows, so does the chance that at least one component will fail during a given interval. Two responses address different parts of the problem. RAID combines devices and stores redundant information so that selected failures need not interrupt access. Solid-state storage removes mechanical positioning but introduces the constraints and ageing mechanisms of flash memory.

The property we want is *availability*: the ability of the system to keep providing its service even after a component fails. Redundant RAID layouts can raise availability by introducing *redundancy*, storing enough extra information that the data on a failed disk can be reconstructed from the disks that remain. If a failure is within the level's tolerance, the array can often remain available in a degraded state and later reconstruct data onto a replacement. Availability and rebuild success are not guaranteed: another failure, an unrecoverable read error, controller/software faults or operator mistakes can still cause outage or loss. RAID is not a backup.

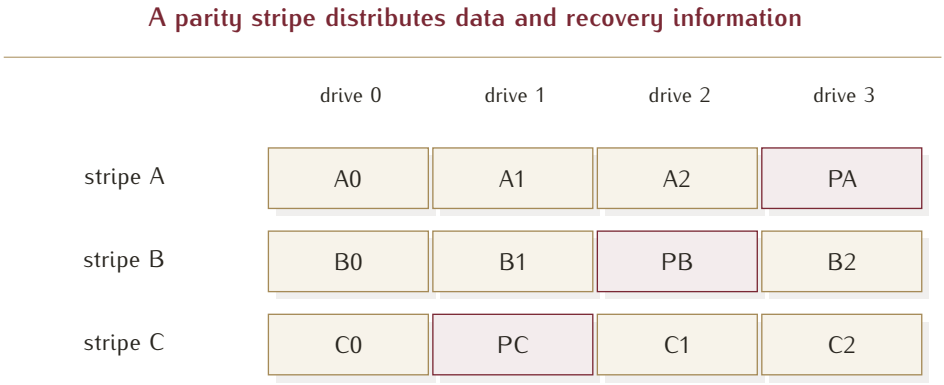


Figure 4.48: *Each stripe contains data blocks and one parity block on different drives. Rotating parity avoids dedicating one drive to all parity positions. The layout tolerates one missing block per stripe within its stated failure model.*

Mirrors and parity blocks reserve physical space for recovery. An array designer can trade some usable capacity for tolerance of particular failure patterns. The layout also determines the additional reads and writes required when one data block changes.

The terminology follows the original RAID taxonomy and subsequent standard usage [55]. Write N for the number of equal-capacity disks and C for each capacity. The RAID 1 case below is a two-way mirror; RAID 10 uses an even N arranged as at least two mirrored pairs. Capacities exclude array metadata and filesystem overhead.

- *RAID 0* (striping) spreads data across all disks for speed. Usable capacity $N \cdot C$; minimum 2 disks; tolerates *no* failures. Any lost disk loses data.
- *RAID 1* (mirroring) keeps two identical copies. With two disks the usable capacity is C; minimum 2 disks; tolerates 1 failure.
- *RAID 5* adds one disk's worth of *distributed parity*. Usable capacity $(N - 1) \cdot C$; minimum 3 disks; tolerates 1 failure.
- *RAID 6* adds *double* parity. Usable capacity $(N - 2) \cdot C$; minimum 4 disks; tolerates 2 failures.

- *RAID 10* combines mirroring and striping. Usable capacity $(N/2) \cdot C$; minimum 4 disks; tolerates at least 1 failure.

LEVEL	USABLE	MIN DISKS	TOLERATES	IDEA
RAID 0	$N \cdot C$	2	0	striping
RAID 1	C	2	1	mirroring
RAID 5	$(N-1)C$	3	1	distributed parity
RAID 6	$(N-2)C$	4	2	double parity
RAID 10	$(N/2)C$	4	$\geqslant 1$	mirror + stripe

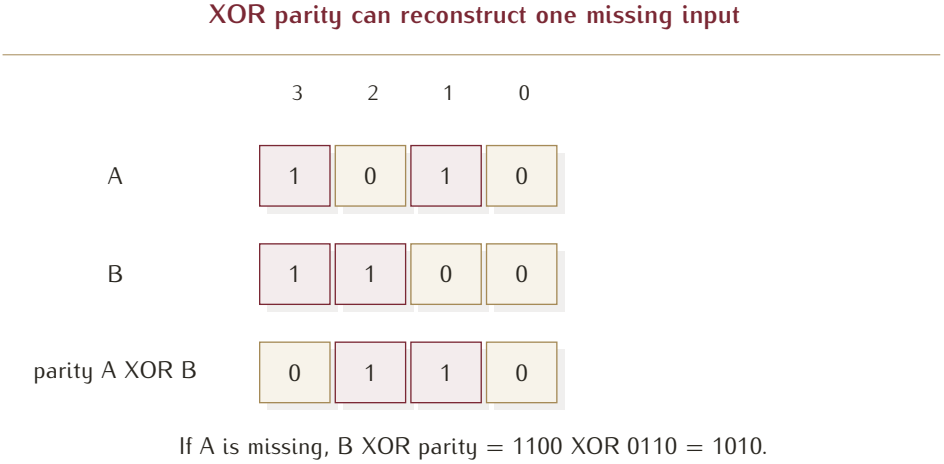


Figure 4.49: *The parity relation is reversible when exactly one required input is unknown. It does not supply independent information sufficient to recover two arbitrary missing inputs from a single XOR parity block.*

Worked example

Problem. A RAID 5 array has 5 disks of 4 TB each. What is the usable capacity, and how many failures does it tolerate?
Solution. Usable $= (5-1) \times 4 = 16$ TB; it tolerates one disk failure. One disk's worth of capacity (4 TB) is spent on parity, distributed across all five disks.

Worked example

Problem. You need 8 TB of usable capacity with RAID 1 (a two-disk mirror). What size must each disk be?
Solution. In RAID 1 the usable capacity equals one disk, so each disk must be 8 TB.

In RAID 5 the parity is *distributed* across all the disks rather than concentrated on one dedicated disk (as in the older RAID 4). The redundancy overhead is one disk out of N, that is a fraction 1/N of the raw capacity; with more disks the relative overhead shrinks, but the array then tolerates the same single failure over more spindles.

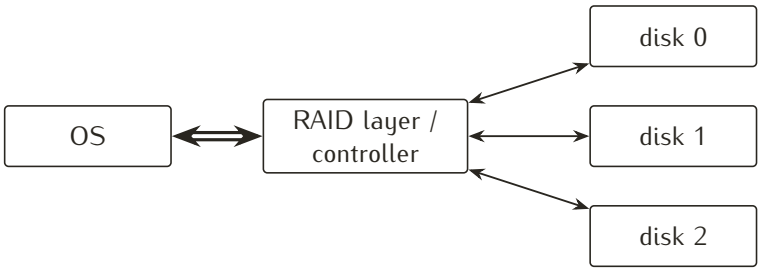
When a RAID layer receives a logical read or write, it maps the request onto member devices according to the selected layout: striping distributes blocks, mirroring keeps additional copies, and parity layouts encode redundant information [55]. The operating system normally sees one logical block device, but usable capacity, failure tolerance and performance depend on the level. RAID 0 provides capacity and parallelism without redundancy and can increase the probability that one member failure loses the array; RAID 1 and parity layouts tolerate specified failure patterns at a cost in space and write work. The mapping may be implemented in host software, firmware, a hardware controller or a distributed storage layer.

Usable capacity depends on the redundancy scheme

four equal drives	other four-drive layouts
raw capacity: 4C	RAID 6: 2C
RAID 0: 4C	two mirrored pairs: 2C
RAID 5: 3C	capacity excludes metadata

All drives have capacity C; layout and implementation overhead are simplified.

Figure 4.50: *Parity and mirroring reserve capacity to tolerate specified failures. Equal usable capacity does not imply equal failure tolerance, read behaviour or rebuild behaviour. The exact layout remains part of the comparison.*



4.6.2 Parity writes, degraded mode and rebuild

RAID combines drives to improve performance, availability or both. It does not automatically protect against accidental deletion, software corruption, malware, controller bugs, fire or loss of the entire array. The first design step is to state which component failures must be tolerated and how quickly service must recover.

Worked example: capacity and tolerated failures

Six 4 TB drives provide 24 TB raw. RAID 5 offers 20 TB and tolerates one drive failure. RAID 6 offers 16 TB and tolerates any two drive failures. RAID 10 offers 12 TB; it can survive multiple failures only when no mirrored pair loses both members. “Two failures” is therefore guaranteed for RAID 6 but topology-dependent for RAID 10.

For a full-stripe write, new parity can be computed from the incoming data and written once. A small update to one block in RAID 5 commonly uses read-modify-write:

1. read the old data block;
2. read the old parity block;
3. compute the parity difference;
4. write the new data;
5. write the new parity.

Thus one logical small write can require four physical operations. RAID 6 has two independent parity syndromes and a larger update cost. Controllers mitigate this with write-back caches, stripe aggregation and scheduling.

Worked example: XOR parity update

Old parity is $P = D_0 \oplus D_1 \oplus D_2$. Only D_1 changes to D_1'. The new parity is

$$P' = P \oplus D_1 \oplus D_1'.$$

The controller need not reread every unchanged data block; it needs old parity, old changed data and new changed data.

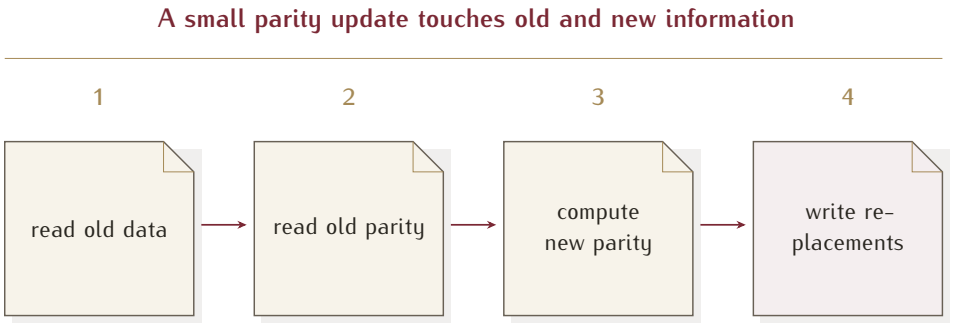


Figure 4.51: *The update changes parity consistently with the replaced data block. Its durability ordering matters: a crash between partial writes can leave data and parity inconsistent unless the system provides an appropriate recovery mechanism.*

After a drive failure, missing data are reconstructed from surviving members. Every read of a missing RAID 5 block may require reading all corresponding surviving blocks. Rebuild scans a large fraction of the array, consumes bandwidth and stresses drives. During that interval redundancy is reduced, and an additional failure may destroy the array.

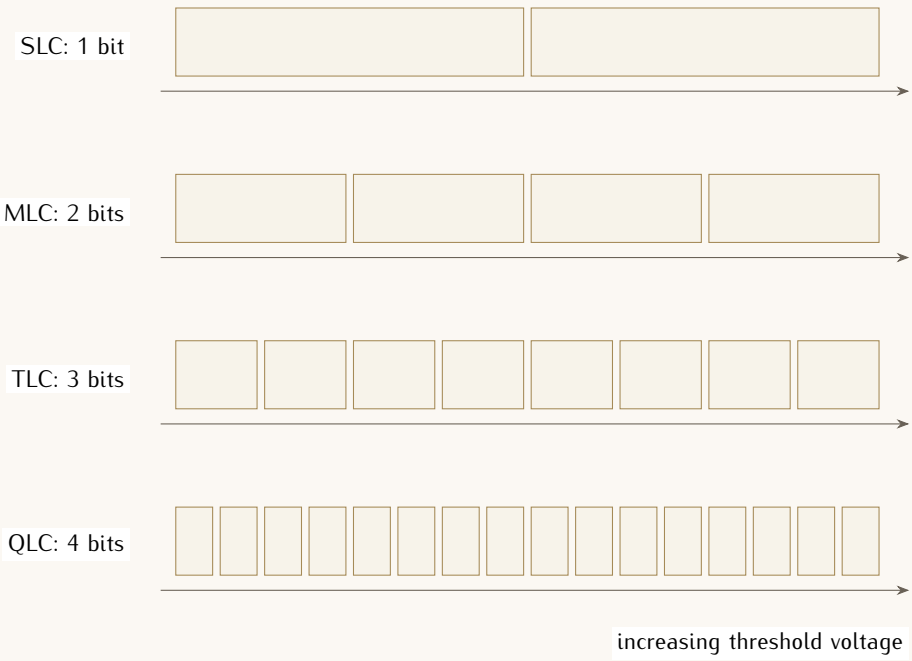
Rebuild time depends on foreground workload, seeks, controller limits, error recovery and throttling as well as capacity and sequential bandwidth. Large drives make the vulnerability window longer, one reason double-parity layouts are common for large arrays.

Reading an analogue threshold as digital information

Charge stored in a floating-gate or charge-trap structure changes a transistor's threshold voltage. The read circuitry tests conduction at reference voltages and decodes the result. A cell carrying b bits needs 2^b distinguishable states: 2, 4, 8 and 16 for SLC, two-bit MLC, TLC and QLC respectively [142, 143].

Physical mechanism (continued)

Distinguishable threshold-voltage ranges



Read each row as a division of a voltage range into distinguishable states. The rectangles are schematic regions, with equal spacing chosen for clarity. Actual cells have distributions that spread and shift; their shapes, spacing and bit assignments depend on the technology and controller.

Worked check. In an invented 4 V window with equally spaced state centres, compare 2 and 16 centres spanning the endpoints. The separations are $4/(2-1) = 4$ V and $4/(16-1) \approx 0.267$ V. A shift of 0.10 V occupies 2.5% and 37.5% of those separations respectively. The calculation illustrates shrinking decision margins; it is not a flash specification or an error-rate prediction.

Programming and erasing move charge using strong electric fields. Array organisation groups erase operations across a block. Those physical operations, together with threshold tracking and error correction, explain why density, endurance and access time must be considered jointly [143, 48, 44].

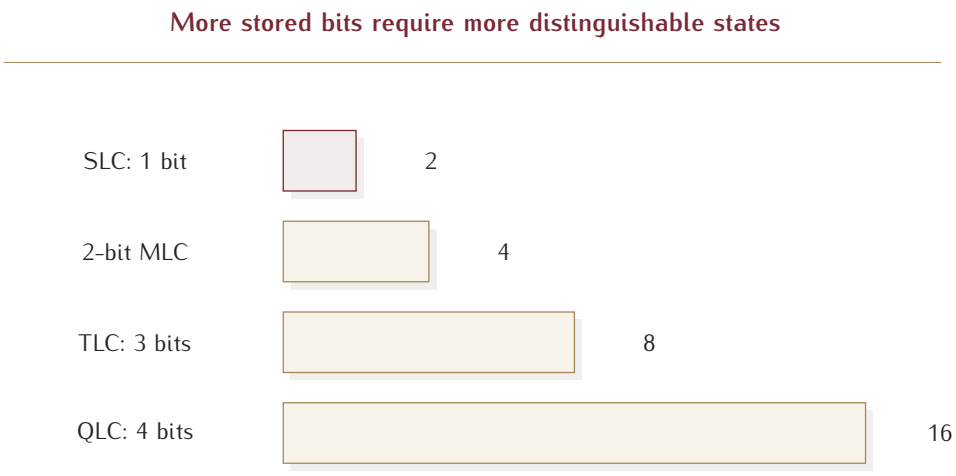


Figure 4.52: *The count doubles with each additional stored bit. Distinguishing more threshold states within a usable range places tighter demands on sensing and control. Actual distributions and state assignments depend on the flash technology.*

4.6.3 Flash media and translation

The following model describes a NAND-flash SSD and has no moving parts, so there is no seek and no rotational latency. NAND flash is internally programmed in pages and erased in larger erase blocks, but page and block sizes vary substantially by flash generation and device. These internal units are distinct from the host-visible logical block size (for example 512-byte or 4-KiB LBAs) exposed by the SSD interface. The model treats a page as the read/program unit and an erase block as the erase unit; real NAND may permit restricted partial-page operations. A block's size in bytes is therefore

$$\text{block} = (\text{pages per block}) \times (\text{page size}).$$

Worked example

Problem. An SSD has blocks of 64 pages, each page 4 KiB. What is the block size?
Solution. 64×4 KiB $= 256$ KiB, or 262,144 bytes.

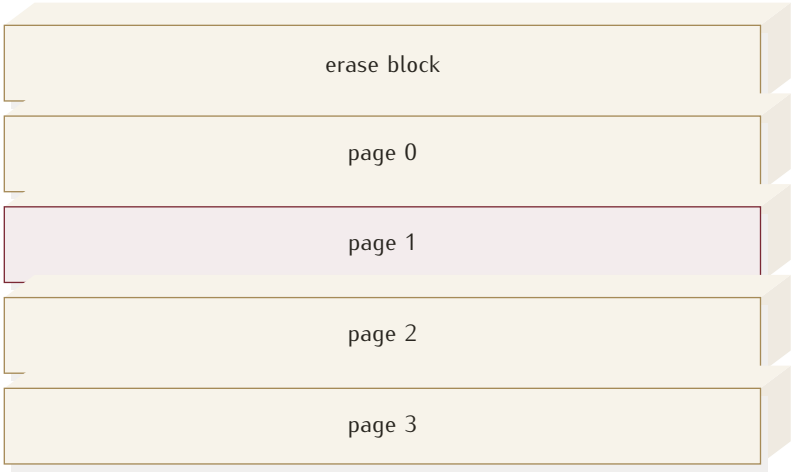
Flash cells cannot be rewritten an unlimited number of times: each cell endures only a finite number of program/erase cycles before it wears out. To keep the whole drive from failing where the writes happen to concentrate, the controller applies *wear leveling*: it remaps writes across available physical erase blocks and may relocate long-lived data so wear is distributed more effectively. It cannot make wear perfectly even, and endurance

also depends on spare area, write amplification, workload, retention and controller policy. Vendors commonly specify endurance with measures such as total bytes written over a stated warranty period; elapsed time still matters to retention and warranty.

Worked example

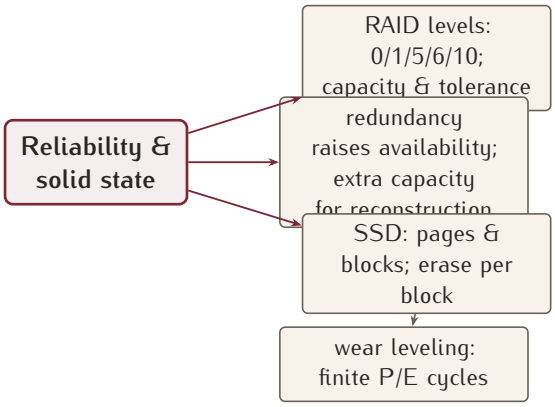
Problem. Which SSD operation works on a page, and which requires erasing an entire block?
Solution. Reading and programming (writing) act on a page; erasing acts on a whole block. This asymmetry is why in-place updates are expensive and why the controller must manage free blocks and wear.

Flash distinguishes read/program pages from erase blocks



Schematic block with four pages; real sizes are device-specific.

Figure 4.53: *The page and block units belong to different operations. Programming a new page does not erase the whole block, but later reclaiming the block requires preserving any still-live pages before erasure.*



An NVMe read through the Linux storage stack

Solid-state storage removes seek and rotational delay, but a read still crosses several software and hardware queues before bytes reach a process.

A user `read` enters the VFS. If the requested data is already in the page cache, the kernel may copy it without issuing device I/O. On a cache miss, filesystem code maps the file offset to storage and submits block I/O. The multi-queue block layer schedules or merges requests as appropriate and passes them to the NVMe host driver. Direct I/O and memory-mapped access follow related but not identical paths.

For the conventional NVMe-over-PCIe path considered here, submission and completion queues reside in host memory. The driver writes command entries, updates a memory-mapped doorbell register, and the controller fetches commands and transfers data by DMA. Completion entries are written to memory; an interrupt or polling path tells the driver to consume them. Queue pairs permit parallelism across CPUs and reduce the single-lock bottlenecks associated with older host interfaces [56].

Start from `fs/read_write.c` and the relevant filesystem. Continue through `block/` and the `blk-mq` implementation, then inspect `drivers/nvme/host/`, especially the PCI transport for queue creation, doorbell writes, DMA setup and completion handling. The exact call graph depends on filesystem, caching mode, I/O scheduler and kernel version [35].

The CPU prepares descriptors, updates queue state and handles completion. The storage controller performs the bulk transfer between device and memory through DMA. A final copy from the page cache to a user buffer may still consume CPU time. Memory mapping and direct I/O change the copying path. Both still require ownership, translation and completion tracking.

SSD latency combines flash-media access with software dispatch, queue residence, PCIe transport, controller firmware, DMA and interrupt or polling overhead.

Flash endurance and retention are physical ageing

High-field program and erase operations gradually damage the insulating material and trap charge. Stored charge also leaks or redistributes over time, with temperature accel-

erating several degradation mechanisms. The controller estimates cell condition, adjusts read thresholds and applies error correction.

Wear levelling, spare blocks, scrubbing and over-provisioning compensate for a medium whose physical reliability changes with use and age [48, 44].

4.6.4 The FTL, endurance and state-dependent latency

The Flash Translation Layer (FTL) maps logical block addresses to physical pages. Updating a logical block writes a fresh physical page and marks the old page invalid.

Figure 4.54 follows a single logical page through an update. The host continues to use the same logical name while the controller changes which physical page supplies its contents.

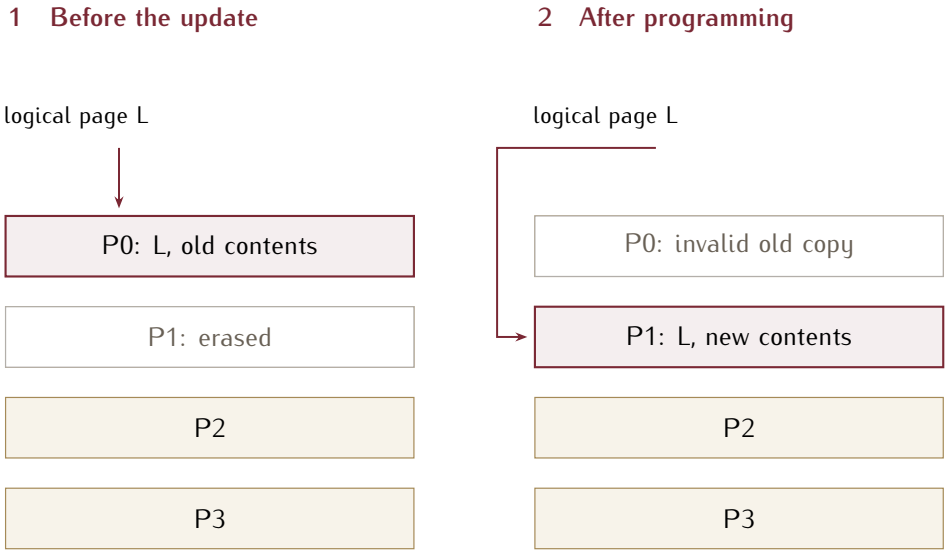


Figure 4.54: *A simplified out-of-place update within a block that has a free erased page. Later garbage collection must preserve every remaining live page before erasing the whole block. Recovery of the mapping metadata itself requires a crash-consistent controller protocol.*

Following the old copy after an update

After the mapping in Figure 4.54 switches from P0 to P1, a logical read of L returns the new contents from P1. P0 still contains old physical information until its block is erased. If P2 and P3 hold live data, garbage collection must first copy those pages and update their mappings safely. Erasing the block immediately would destroy data unrelated to the update of L.

Garbage collection selects blocks containing invalid pages, copies their remaining valid pages elsewhere, erases the block and returns it to the free pool. The extra internal traffic creates *write amplification*:

$$WA = \frac{\text{physical bytes programmed}}{\text{host bytes written}}.$$

Lower free space, fragmented updates and weak locality can increase WA.

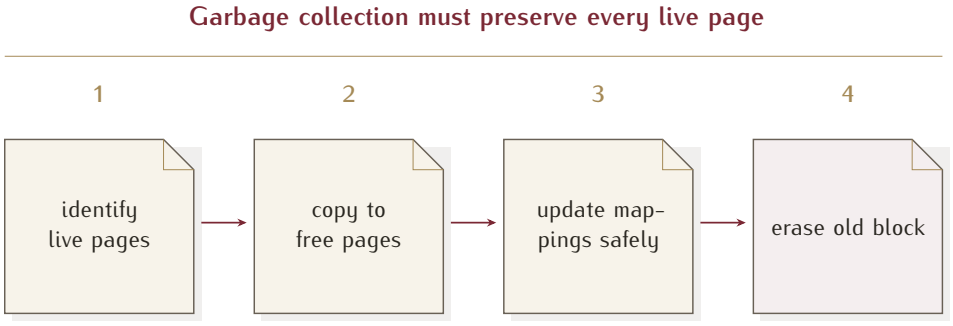


Figure 4.55: *Reclamation operates on a block that can contain both obsolete and still-valid pages. The order must preserve every live logical page and recoverable mapping information before the erase operation makes the block reusable.*

Worked example: garbage collection amplification

An erase block contains 256 pages. Before it can be erased, 64 still-valid pages must be copied. The host update that triggered collection writes 64 new pages. Physical programming is $64 + 64 = 128$ pages for 64 host pages, so this simplified event has $WA = 2$.

When a file system frees logical blocks, the SSD cannot infer that their old data are no longer needed unless the operating system communicates this information. TRIM/discard lets the FTL mark mappings invalid earlier, improving future garbage collection. Over-provisioned flash gives the controller spare blocks for relocation and replacement.

Dynamic wear levelling moves active data; static wear levelling may also relocate long-lived cold data so that its nearly unused blocks can share the wear burden.

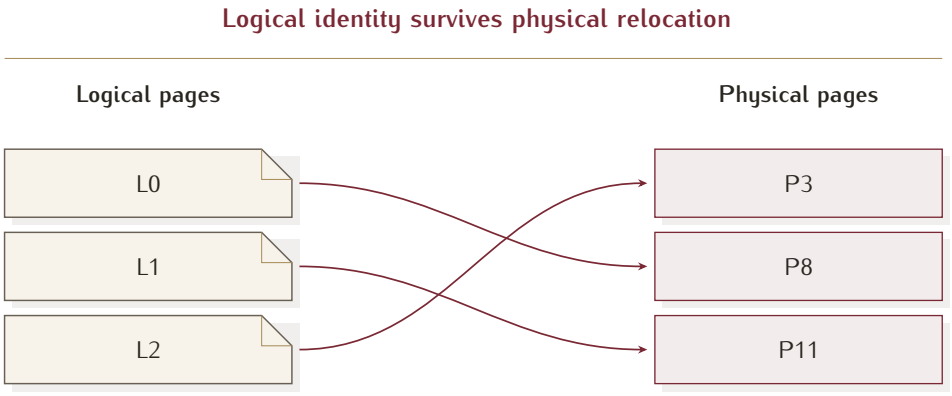


Figure 4.56: *The translation layer decouples host-visible logical addresses from physical flash placement. Remapping supports out-of-place updates and wear management, while mapping metadata itself requires consistency and recovery.*

An SSD has no mechanical seek, but its latency is not constant. Queueing, parallel flash channels, garbage collection, error correction, thermal limits, SLC-style write caches and firmware policies all matter. A fresh drive with free blocks may absorb writes quickly; sustained writes after the cache fills can be much slower. Benchmarks must specify workload duration and preconditioning.

Security consequence: remapping complicates overwriting

Because the FTL remaps logical addresses, repeatedly overwriting a logical block does not guarantee that every old physical page was erased. Secure-erase or cryptographic-erase commands are designed for the device's translation layer. Cryptographic erasure depends on strong encryption having protected the data from the outset and on destroying every relevant key copy. Unencrypted historical pages, recovery keys and backups need separate treatment.

> **Laboratory 29: Rotating parity and reconstruction**
>
> Define a rotating-parity RAID 5 layout with D drives: stripe s has parity disk s mod D, and data fill the other disks in ascending order. Implement mapping from logical block L; compare L=5 for D=3,4,5, small-write cost and full-stripe cost.
>
> Worked solution

Laboratory 29: solution

The following function implements the declared layout:

```
def locate(L,D):
    assert L>=0 and D>=3
    stripe,slot=divmod(L,D-1)
    parity=stripe%D
    disks=[d for d in range(D) if d!=parity]
    return stripe,disks[slot],parity
for D in [3,4,5]: print(D,locate(5,D))
```

The results (stripe,data,parity) are (2,1,2), (1,3,1) and (1,2,1). A read-modify-write of one data block reads old data and old parity, then writes new data and new parity: four block operations. A complete stripe write computes parity from its D-1 new data blocks and writes D blocks, with no old-data reads in this model. If the requested data disk fails, reconstruct its block by XOR-ing the parity and the remaining D-2 data blocks in that stripe, requiring D-1 reads. If the failed drive held parity, surviving data can be read directly.

Return to the laboratory

From disk redundancy to flash translation and NVMe

Timeline

Year	Machine, standard or event	Architectural significance
1987	NAND flash	A dense erase-block medium creates new translation and endurance problems.
1988	RAID paper	Redundancy and striping are classified independently of drive medium.
2011	NVMe 1.0	Memory-resident queues expose solid-state parallelism to hosts.

Disk arrays began as a response to a scaling mismatch. During the 1980s, small disks improved rapidly in cost and performance, but using many of them increased the probability that some component would fail. In 1988 David Patterson, Garth Gibson and Randy Katz at the University of California, Berkeley, published “A Case for Redundant Arrays of Inexpensive Disks.” The paper compared organisations that became known as RAID levels and argued that arrays could obtain capacity and parallelism from multiple drives while using redundancy to survive failures. The acronym’s later expansion often substitutes “independent” for “inexpensive,” but the historical paper used the latter [55].

RAID numbers identify placement rules. RAID 4 gathered parity updates on one dedicated disk. RAID 5 distributed that load across members. RAID 6 adds an independent recovery relation, allowing two missing members to be reconstructed. A controller

writing several related blocks also needs a way to recover from power loss during the update, such as protected write-back storage or a persistent update journal [55, 56, 98].

RAID also taught a limitation that remains important: redundancy is not backup. Mirroring reproduces deletion and corruption; parity does not provide an independent historical copy; rebuilds expose the array to further failures and latent read errors. Enterprise systems consequently combine drive redundancy with end-to-end checksums, scrubbing, snapshots, replication and backups. Modern distributed storage may use erasure codes across nodes or failure domains rather than a classic hardware RAID controller, but it is answering the same question about how much extra information is needed to reconstruct lost data [55, 56, 98].

Flash memory has a different physical history. Semiconductor read-only memories and EPROMs preceded it. Fujio Masuoka and colleagues at Toshiba presented NOR flash in 1984 and NAND flash in 1987. NAND later became a basis of high-density storage. A flash cell stores charge in a floating-gate or charge-trap structure. Reads can be page-granular, programming occurs in pages, and erasure affects a larger block. Cells wear with program/erase cycles, and a page generally cannot be overwritten in place without erasing its block. These constraints make a raw flash chip fundamentally different from a magnetic disk even when both are exported as logical blocks [98].

The Flash Translation Layer hides this mismatch. It maps host logical block addresses to changing physical pages, performs garbage collection, spreads wear, manages bad blocks and coordinates error correction. Overprovisioned capacity gives the controller working space. Write amplification arises when a small logical update forces valid pages to be copied before a block can be erased. Performance depends on queue depth, spare area, workload locality, internal parallelism and garbage-collection state. A benchmark taken on an empty drive for a short interval does not describe steady-state service under sustained writes [55, 56, 98].

Early SSDs often used interfaces designed for disks. Serial ATA 1.0 was released in August 2001, and Intel's AHCI 1.0 specification followed in 2004. The AHCI software model supplies up to 32 command slots per port and preserved assumptions suited to rotating media. NVMe was developed to expose non-volatile memory over PCI Express with many submission and completion queues, each capable of large depth and naturally mapped to parallel CPU cores [136].

The NVM Express 1.0 base specification was released on 1 March 2011. Rather than write command registers for each operation, host and controller communicate through queue entries in memory; doorbell registers announce new work, and completions are posted to memory and optionally signalled by MSI-X interrupts. This design reduces lock contention and lets software exploit controller parallelism [55, 56, 98].

The standard continued to broaden. NVMe over Fabrics 1.0, released in June 2016, extended the command model beyond PCIe with initial RDMA and Fibre Channel transport bindings. The TCP transport binding followed in November 2018 [137]. The Zoned Namespace command set, standardised in 2020 and incorporated into NVMe 2.0 in 2021,

lets a device expose sequential-write zones so host software can reduce translation and garbage-collection work. The specification set announced on 5 August 2025 included Base Specification revision 2.3, the reference edition used here. The standards catalogue subsequently listed Base Specification 2.4, ratified on 31 July 2026. It includes evolutionary support for multiple command sets and management capabilities rather than defining one monolithic "SSD protocol" [56, 83].

Current drives may store multiple bits per cell: SLC stores one, while MLC, TLC and QLC distinguish progressively more threshold ranges. Density rises, but voltage margins, program latency and endurance become more demanding, increasing reliance on coding and controller algorithms. Some devices use an SLC-like write cache and later fold data into denser NAND. The operating system sees a fast block namespace, yet service time may change sharply when caches fill or background management begins. The host protocol and media-management layer therefore contribute separate terms to an SSD performance model [55, 56, 98].

RAID is a data-placement and redundancy policy; flash is a storage medium; NVMe is a host-controller interface. An NVMe SSD can be one member of a RAID set, and a distributed system can apply erasure coding above many NVMe devices. Their roles remain distinct even when all three appear in the same storage stack [55, 56, 83, 35].

Redundancy, flash translation and queueing remain the mechanisms that turn imperfect media into reliable high-throughput storage.

Concept check

1. Usable capacity of RAID 6 with 6 disks of 2 TB?
2. Minimum disks for conventional RAID 5 and for RAID 10 formed from at least two mirrored pairs?
3. How many failures does RAID 0 tolerate?
4. Erase-block size of an SSD with 128 pages of 4 KiB?
5. What problem does wear leveling solve?

Solutions to the review questions

1. RAID 6 reserves two disks worth of capacity for dual parity in this equal-disk model. Usable capacity is $(6-2)\cdot 2 = 8$ TB. The calculation uses decimal TB consistently.
2. Conventional RAID 5 needs at least three disks. A RAID 10 layout formed by striping across at least two mirrored pairs needs four disks. The pair structure also determines which combinations of failures remain recoverable.
3. RAID 0 has no redundancy, so it guarantees tolerance of zero disk failures. Losing a member loses the stripes stored on that member and can make the volume unusable.

Worked solution (continued)

4. An erase block containing 128 pages of 4 KiB has size $128 \cdot 4096 = 524{,}288$ bytes, or 512 KiB. This erase granularity is distinct from the page programming granularity.
5. Flash cells support a limited number of program/erase cycles. Wear levelling distributes erase activity so frequently updated logical addresses do not exhaust a small fixed set of physical blocks while other blocks remain lightly used. The mapping and relocation policy implement that distribution.

4.6.5 Worked problems

Availability and repair

Assume alternating service and repair intervals, with mean operating time 1000 h and mean repair time 2 h. Using MTBF here for the mean operating interval, the long-run availability is

$$A = \frac{1000}{1000 + 2} \approx 0.998004 = 99.8004\%.$$

The unavailable fraction is $2/1002 \approx 0.001996$, and the two fractions sum to one. This average describes the repairable model; it does not specify the duration of an individual outage or the probability of losing data. With mean operating and repair times of 5000 h and 5 h, availability becomes $5000/5005 \approx 99.9001\%$.

Partial-page updates

Consider 16 KiB pages and 2 MiB erase blocks: each erase block contains $2048/16 = 128$ pages. Suppose a 4 KiB host update changes part of a logical page and the controller has no cached copy of its other 12 KiB. Under a whole-page programming model, it reads the unchanged bytes, merges the update and programs a fresh 16 KiB page. The old physical page becomes invalid. This update alone programs four times the host payload; later collection may add further copying of valid pages before the old block can be erased. A cache that combines several updates can change this ratio.

Exercise 4.6.1 — RAID 0

How many disk failures can it tolerate?

Worked solution

RAID 0 stripes data across disks without storing redundant information. If any member fails, some stripes lose their only copy, so the array cannot reconstruct the full data set. Its guaranteed disk-failure tolerance is therefore $\boxed{0}$. Striping can increase capacity and throughput, but the count of surviving disks does not create a reconstruction method.

Exercise 4.6.2 — RAID 5 minimum

Minimum disks?

Worked solution

RAID 5 needs at least three disks in the standard arrangement: data from a stripe occupy at least two members and one disk-equivalent holds parity, with the parity position distributed across stripes. With three equal-capacity disks, usable capacity is two disk capacities and one member failure can be reconstructed. Two disks correspond to other redundancy arrangements and do not form the standard RAID 5 configuration described here.

Exercise 4.6.3 — Unavailability

If A=0.999, what is unavailability?

Worked solution

Availability and unavailability are complementary fractions over the same observation model. Thus $U = 1 - A = 1 - 0.999 = 0.001$. Multiplying by 100 converts the fraction to $\boxed{0.1\%}$. Over an illustrative 1000 time units, this corresponds to one unavailable unit under the stated average. The calculation alone does not specify how outages are distributed or predict a particular failure event.

Exercise 4.6.4 — Wear levelling

What problem does it address?

Worked solution

Flash erase blocks tolerate a finite number of program/erase cycles. A frequently rewritten logical region could repeatedly consume the endurance of the same physical blocks. Wear levelling changes the mapping or placement so that erase activity is spread more evenly. It therefore reduces uneven early exhaustion. It does not

Worked solution (continued)

remove the finite-endurance limit, and its data movements can contribute to write amplification.

Summary

Ask what failure model the layout tolerates, what happens during degraded operation, how long rebuild takes and which common causes defeat independence assumptions. For SSDs, add the mapping and cleaning work hidden behind a logical write. The controller's abstraction is useful, yet serious reasoning must occasionally look beneath it.

4.6.6 Exercises

Exercise 4.6.5 — Capacity

Five 8 TB drives are arranged as RAID 5 and RAID 6. Compute usable capacity.

Go to the solution →

Exercise 4.6.6 — RAID 10 failures

Four drives form mirrored pairs (A,B) and (C,D). Which two-drive failures are survivable?

Go to the solution →

Exercise 4.6.7 — RAID 5 write penalty

List the four physical operations in a read-modify-write update.

Go to the solution →

Exercise 4.6.8 — Rebuild estimate

A 12 TB member is rebuilt at an effective 150 MB/s. Estimate the ideal time, ignoring overhead.

Go to the solution →

Exercise 4.6.9 — Flash update

Why does updating an already-programmed SSD page generally require writing elsewhere or erasing its containing block first?

Go to the solution →

Exercise 4.6.10 — Write amplification

A workload sends 1 GiB of host writes while the SSD programs 1.7 GiB internally. Compute write amplification.

Go to the solution →

Exercise 4.6.11 — TRIM

How can TRIM reduce future garbage-collection work?

Go to the solution →

Exercise 4.6.12 — Design decision

A service requires continued operation after any two drive failures and keeps separate backups for deletion and corruption. Choose among RAID 5, RAID 6 and RAID 10, and justify.

Go to the solution →

4.6.7 Solutions

RAID and Solid-State Storage

Solution 4.6.5

RAID 5 reserves the equivalent of one drive for distributed parity, leaving four data-drive equivalents: $(5-1)\cdot 8 = 32$ TB. RAID 6 reserves two equivalents, leaving $(5-2)\cdot 8 = 24$ TB. Parity is distributed across the drives; the capacity calculation does not imply dedicated parity disks. These are nominal usable capacities before filesystem and metadata overhead, assuming all drives contribute the stated equal capacity.

← Return to the exercise

Solution 4.6.6

The four survivable pairs are (A, C), (A, D), (B, C) and (B, D): each leaves one member of each mirror available. The other two possible pairs, (A, B) and (C, D), destroy both copies in one mirror. Its striped data can then no longer be reconstructed from the remaining pair.

← Return to the exercise

Solution 4.6.7

Read old data D, read old parity P, compute $P' = P \oplus D \oplus D'$, then write new data D' and new parity P'. There are two block reads and two block writes. XOR calculation adds processor work. A crash between the writes can leave inconsistent data and parity, so a practical array also needs a crash-recovery mechanism.

← Return to the exercise

Solution 4.6.8

$12 \cdot 10^{12}/(150 \cdot 10^{6}) = 80{,}000$ s, about 22.2 hours. Real rebuilds can be substantially longer because of foreground work and errors.

← Return to the exercise

Solution 4.6.9

A programmed NAND cell occupies a threshold-voltage range. Page programming can move the cell through allowed states in one direction. A general replacement pattern may need a threshold to move back, and the reset operation erases an entire block. The controller normally programs a fresh page, safely updates the logical mapping and marks the old page obsolete. Later, garbage collection preserves any other live pages in that block before erasing it.

← Return to the exercise

Solution 4.6.10

Write amplification compares bytes programmed internally with bytes accepted from the host. Both quantities use GiB, so the units cancel: $WA = 1.7/1 = 1.7$. For each host byte, the flash receives 1.7 bytes of programming on average, including internal movement and maintenance in this accounting interval. The additional 0.7 GiB represents 70% more programmed data than the host supplied. A value of one would indicate no extra programming under this metric.

← Return to the exercise

Solution 4.6.11

After a filesystem frees an extent, it can send a discard covering the corresponding logical blocks. The translation layer can then treat their old physical pages as obsolete. For example, if an erase block contains eight pages and three formerly live pages are discarded, reclamation can avoid copying those three pages. The gain depends on whether the controller has already reclaimed the block and on its discard policy. The command alone supplies no secure-erasure guarantee.

← Return to the exercise

Solution 4.6.12

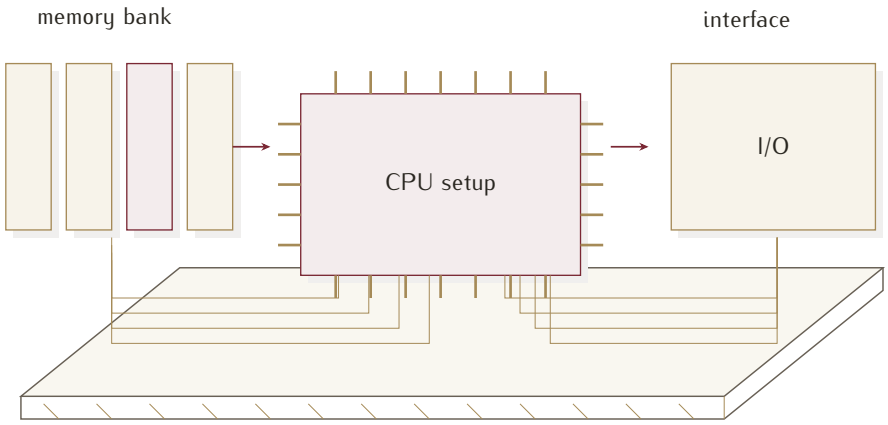
RAID 6 directly guarantees tolerance of any two member failures, so it satisfies the stated availability requirement. RAID 5 tolerates only one. RAID 10 may survive two, but not when both belong to the same mirror. Performance needs could motivate other layouts, but the strict failure condition selects double parity among these choices.

← Return to the exercise

4.7 DEVICE CLASSES AND DIRECT MEMORY ACCESS

A device has a buffer of data to move into memory. The CPU could copy each piece itself, or it could describe a transfer for a device or DMA engine to perform. Delegating the movement creates a coordination problem: the addresses must be valid for the device, the buffer must remain available, and completion must be observed before the CPU uses the result. These obligations connect device classes to memory protection and cache visibility.

A device can move a buffer without a CPU copy of every word



The physical data path is delegated; software still controls mappings and ownership.

Figure 4.57: *DMA changes which component initiates memory transfers. The board view makes the processor, memory and device interface distinct so their setup, movement and completion responsibilities can be followed separately.*

4.7.1 Programmed I/O and the case for DMA

Polling and interrupts describe how software learns that a device needs attention. A separate choice determines who moves the payload. In programmed I/O the CPU executes the transfer instructions. With direct memory access (DMA), software describes the transfer and a device or controller issues the memory transactions.

Direct memory access allows a device or controller to transfer data between an I/O endpoint and main memory without requiring the CPU to execute a load-and-store loop for every word. Completion may be reported by an interrupt, interrupt coalescing, polling or another queue-completion mechanism. In the simplified shared-bus model, DMA introduces another bus master, which is why an arbiter is needed to decide, at each moment, whether the CPU or the DMA controller drives the bus. The payoff is that the CPU is freed to compute while the transfer proceeds.

Unix-like kernels provide character and block device interfaces alongside specialised subsystems such as networking and graphics. A *character-device* interface generally presents a stream or device-specific sequence of records without the random block-addressing contract of storage. A *block device* presents independently addressable logical blocks and supports the kernel's block-I/O model. Calls can still transfer many bytes, and drivers/controllers may buffer, merge or split requests; the class describes the interface, not one-byte versus one-block hardware transactions.

CLASS	UNIT	EXAMPLES
character	stream or records	keyboard, mouse, serial port
block	fixed-size block	disk, SSD, CD-ROM

SSDs expose a block-storage interface, whereas network interfaces and GPUs use specialised kernel subsystems and device-specific queues; on Linux, many GPU control interfaces appear as character devices, not block devices. All can use DMA for bulk transfer, but DMA usage does not determine whether the OS interface is character, block, network or another class.

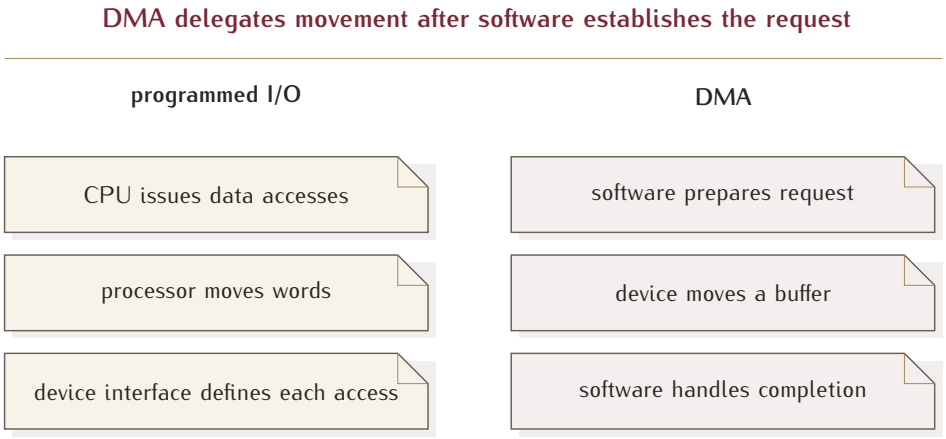


Figure 4.58: *DMA reduces the need for the processor to perform each data-word transfer. The software still configures valid memory, permissions, lengths and notification and later determines when the buffer can be reused.*

Worked example

Problem. Classify: keyboard, disk, serial port, GPU framebuffer.
Solution. Keyboard and serial interfaces are typically exposed through character/input subsystems; the disk is a block device. A GPU is not correctly classified as a block device: it is normally controlled through specialised driver interfaces, often character-device nodes plus memory mappings and command queues. Classification follows the kernel interface: reading a fixed-size record from a character device does not turn it into a block device.

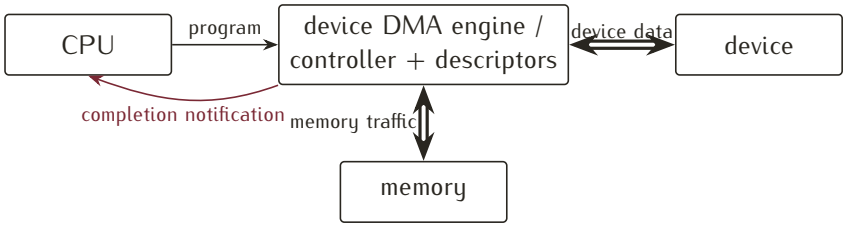
Consider transferring 1 GB from disk to memory without DMA. The CPU must issue one copy for every word: if a word is 8 bytes, that is $1,000,000,000/8 = 125$ million copy operations, each costing several cycles. Those instructions consume execution time even if the scheduler interleaves other work; DMA delegates that payload movement.

Worked example

Problem. Without DMA, how many CPU cycles does it take to copy 1 GB (10^9 bytes) a word at a time, if a word is 8 bytes and each copy costs 3 cycles?
Solution. Words $= 10^9/8 = 1.25 \times 10^8$; cycles $= 1.25 \times 10^8 \times 3 = 3.75 \times 10^8$. DMA removes the per-word copy loop from the CPU. The processor still pays setup, synchronisation and completion costs, whose size depends on the controller, driver and transfer length.

4.7.2 DMA as a delegated memory transaction

A DMA engine may be integrated into a device, an I/O controller or a dedicated subsystem. Its purpose is to perform bus-mastering transfers without a CPU load/store instruction for every word. Software programs registers or publishes descriptors that describe source and destination buffers, transfer length, direction and control/status information. The exact fields depend on the device and interface. Setup, mapping, cache synchronisation and completion handling consume CPU work even though a device performs the payload transfer.



DMA creates an independent transaction stream

A device or controller issues read and write transactions over an interconnect, buffers data and obeys flow-control rules. On a coherent platform, hardware may participate in cache-coherence transactions; on a non-coherent platform, software may need explicit cache maintenance. An IOMMU, when enabled and configured for the device, translates and checks device-visible addresses. Large transfers create sustained activity in links, memory channels and power-delivery networks.

DMA removes per-word instruction overhead but not the physical cost of moving bytes. Bandwidth, contention, coherence policy, address translation, power and thermal limits determine the achieved transfer rate [38, 39, 50].

The number of notifications depends on how the driver groups completed work. A programmed-I/O device can raise one interrupt for an entire FIFO, and a DMA queue can be polled without any interrupt. CPU occupancy therefore includes the cost of the chosen notification policy as well as the cost of setting up or performing the transfer.

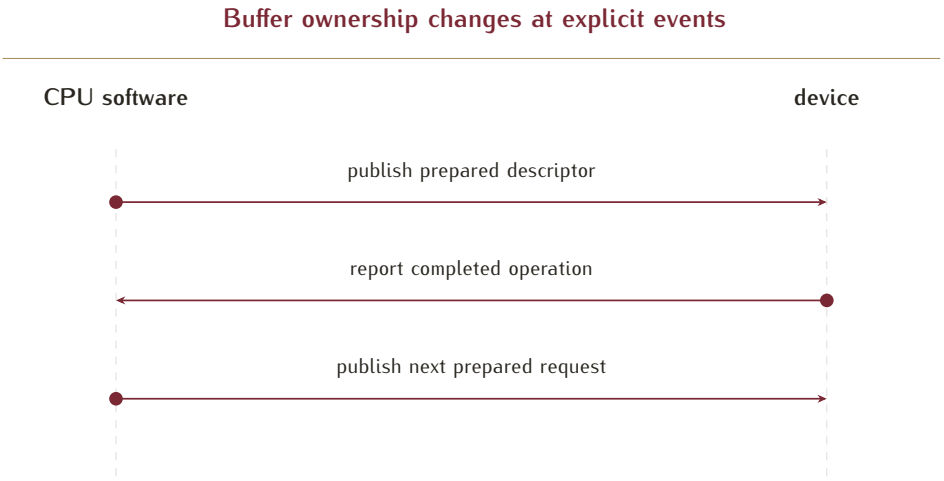


Figure 4.59: *A buffer must remain valid while the device can access it. Ownership transitions coordinate modifications and reclamation. Completion observation may need memory-ordering or cache-maintenance steps specified by the platform.*

Worked example

Problem. Transferring 4096 words: how many completion interrupts does DMA generate, and how many would interrupt-per-word programmed I/O generate?
Solution. Under the stated interrupt-per-word PIO policy, 4096 words generate 4096 interrupts. The DMA interrupt count needs an additional policy: one interrupt per complete transaction gives one; polling gives zero. A PIO device with a FIFO can interrupt once for a batch, giving a smaller count under that policy.

In the shared-bus model, DMA adds a second master, so the CPU and the DMA controller can both want the bus at once. A hardware *arbiter* decides who drives it. In *cycle stealing* the DMA takes single bus cycles from the CPU, one transfer at a time; in *burst mode* it seizes the bus and moves a whole block before releasing it. Modern fabrics arbitrate separately at links, switches and memory controllers; several non-conflicting transactions may proceed concurrently.

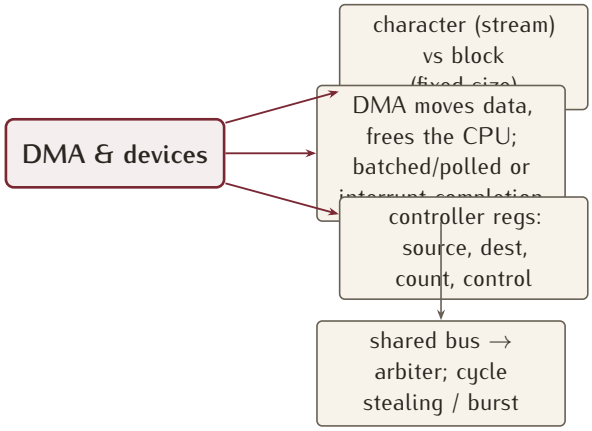
Worked example

Problem. Order the steps of reading a disk sector with DMA.
Solution.

1. Software submits a read command and supplies a mapped destination buffer.

Worked example (continued)

2. The controller obtains the requested data and transfers it into memory by DMA.
3. The controller records completion; notification may use an interrupt, interrupt coalescing, polling or a hybrid policy.
4. Software observes completion, performs any required synchronisation and then consumes the data in memory.



4.7.3 Interfaces, descriptors and ownership

Let setup cost be S_p cycles and per-byte CPU cost be c_p. For n bytes,

$$C_{PIO} = S_p + nc_p.$$

For the comparison below, DMA has a larger fixed setup/completion cost S_d and a smaller amortised CPU bookkeeping cost per byte c_d:

$$C_{DMA} = S_d + nc_d, \qquad c_d \ll c_p.$$

Assuming $S_d > S_p$ and $c_p > c_d$, equal CPU cost occurs at

$$n^* = \frac{S_d - S_p}{c_p - c_d}.$$

Worked example: DMA crossover

PIO setup costs 80 cycles and copying costs 3 cycles/byte. DMA setup and completion cost 1400 cycles with 0.05 CPU cycles/byte for bookkeeping. Then

$$n^* = \frac{1400 - 80}{3 - 0.05} \approx 447.46 \text{ bytes.}$$

Worked example (continued)

For integer byte counts, PIO consumes fewer cycles through 447 bytes; DMA consumes fewer from 448 bytes onward under this model. Real systems also consider latency, cache effects and whether setup can be batched.

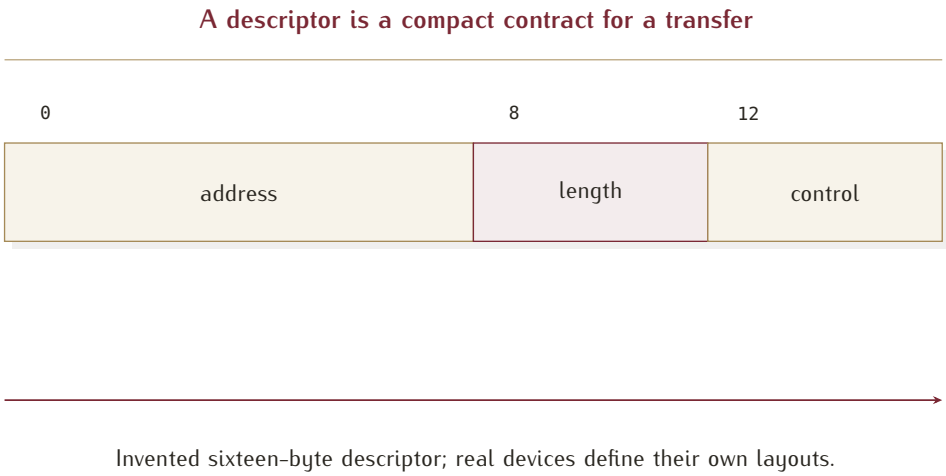


Figure 4.60: *The fields identify the device-visible buffer interval and operation attributes. A real descriptor format specifies widths, alignment, byte order and ownership bits. The illustrated layout is a teaching example only.*

A typical DMA operation follows these steps:

1. the driver allocates or identifies a memory buffer suitable for device access;
2. the operating system creates a device-visible mapping and obtains a DMA address;
3. the driver writes source/destination addresses, length and flags into registers or a descriptor;
4. it publishes the descriptor with the required memory ordering and rings a device doorbell;
5. the device becomes a bus master and transfers data;
6. the device writes completion status and raises an interrupt, or the driver polls a completion queue;
7. the driver synchronises the buffer, removes mappings when needed and wakes the requester.

DMA moves data without a CPU instruction for every byte, but the CPU still sets up, protects and completes the operation.

A logical buffer may occupy non-contiguous physical pages. Scatter–gather DMA describes the transfer as a list of address-length segments. Network and storage de-

vices commonly use circular descriptor rings: the driver produces work descriptors, the device consumes them, and the device produces completion information.

Producer and consumer indices require careful ownership rules. The driver must initialise the descriptor and flush or order its writes before updating the visible tail pointer. The device must make result data visible before publishing the completion marker that software relies on. Missing barriers can expose partially initialised descriptors even when the source code appears ordered.

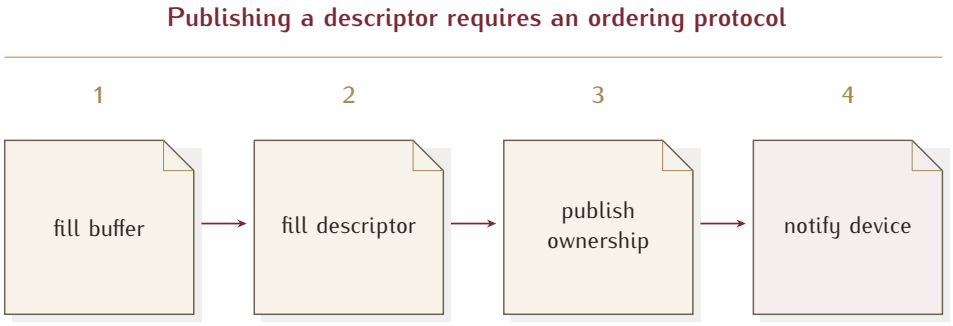
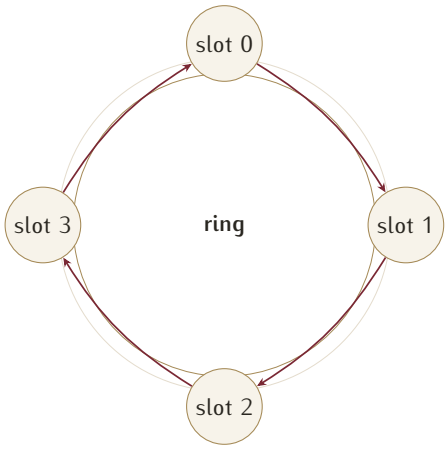


Figure 4.61: *The device must not observe a ready descriptor before its required contents and buffer data are available. Software follows the platform's DMA API and ordering rules to establish that visibility before notification.*

A ring reuses descriptor positions over time



Producer and consumer positions advance under the ring's ownership protocol.

Figure 4.62: *The finite set of slots is reused as requests complete. Index wraparound does not mean an old descriptor is automatically free: the protocol must distinguish occupied and available positions.*

Worked example: segment count

A 10 KiB user buffer begins 1 KiB into a 4 KiB page and is otherwise physically fragmented by page. The first segment covers 3 KiB, the next covers 4 KiB, and the final covers 3 KiB: three physical page fragments. They require three descriptors if the mapping keeps each fragment separate and no hardware limit requires further splitting. The DMA API may merge them into fewer device-visible segments through an IOMMU; a driver must use the mapped segment count and addresses returned by the API [39].

4.7.4 Coherence, ordering and device-visible addresses

A DMA engine accesses memory independently of the CPU caches. In a coherent system, hardware ensures that the device and processors observe compatible values. On a non-coherent system, the DMA API may clean dirty CPU cache lines before device access and invalidate lines before CPU access to device-written data. Mapping and synchronisation in the declared direction handle both ownership transitions, including preventing an old dirty CPU line from later overwriting a receive buffer. Direction matters: "to device", "from device" and bidirectional mappings require different maintenance.

Even on coherent systems, ordering is separate from coherence. Coherence coordinates access to each location across participating agents. It does not by itself order

stores to different locations; the publication barrier and doorbell protocol must make descriptor contents visible before the device fetches them.

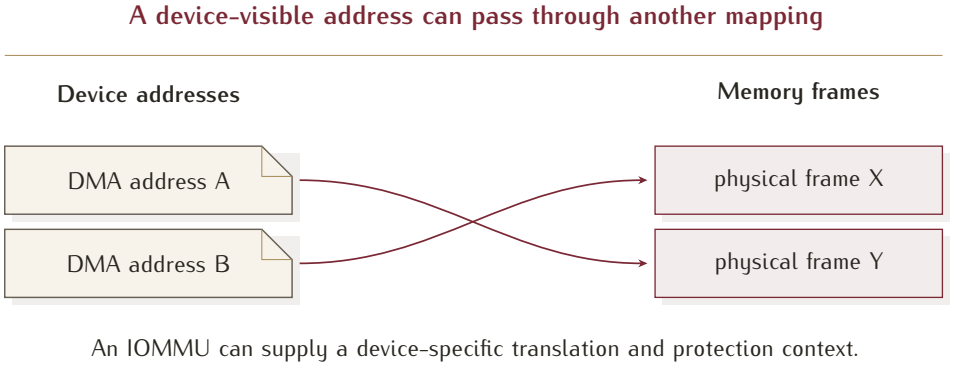


Figure 4.63: *Software must give the device an address valid in the device's address domain. A CPU virtual pointer is not generally interchangeable with that address. Mapping lifetime must cover the entire device operation.*

Before an IOMMU can be understood, three address identities must be kept separate. Software normally accesses a DMA buffer through a CPU virtual address. The processor's page tables translate that virtual address to one or more CPU physical pages. The driver then asks the DMA subsystem for a device-visible DMA address, which may equal a physical address on a simple platform but may instead be an I/O virtual address translated by an IOMMU. The device must be programmed with the DMA address returned by the mapping API, not with an arbitrary CPU pointer or an assumed physical address. [38, 39]

An IOMMU translates device-visible addresses to physical memory and checks permissions. When isolation is enabled, the operating system can restrict each protection domain to authorised mappings. With suitable device grouping, topology and mapping lifetime, this blocks transactions into unrelated host or guest memory.

Translation introduces its own page tables, I/O TLBs and invalidation requirements. Long-lived pinned mappings reduce setup cost but expand the memory a device can access. Short-lived mappings improve least privilege but add overhead. The correct policy depends on device rate and threat model.

Read the DMA API documentation first, then inspect `kernel/dma/` and the architecture-specific mapping implementation. IOMMU drivers are under `drivers/iommu/`; a concrete device driver shows where mappings are created, placed into descriptors, completed and released [35].

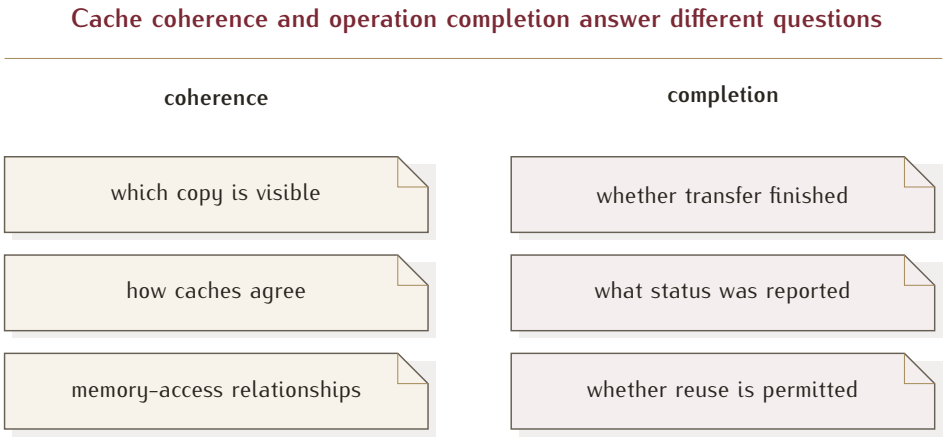


Figure 4.64: *Coherent memory access does not tell software that a device has finished its work. Conversely, a completion signal does not by itself specify all necessary visibility operations on a noncoherent platform.*

Security consequence: DMA can bypass CPU privilege checks

The CPU's page tables do not constrain an unconstrained bus master. Before IOMMUs were common, a malicious peripheral could overwrite kernel memory directly. Device assignment, hot-plug interfaces and high-speed external buses require explicit DMA isolation in addition to CPU user/kernel separation.

4.7.5 Completion and the return to scheduling

DMA does not dictate how completion is detected. A low-rate transfer may interrupt on every completion. A high-rate queue may be polled for a period after the first interrupt, combining low idle overhead with efficient batches under load. Storage and network stacks often use such hybrid strategies.

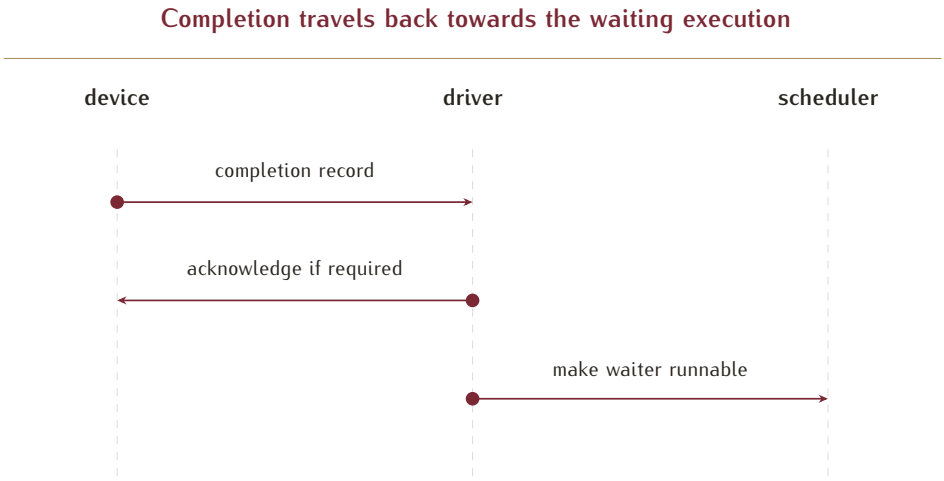


Figure 4.65: *The driver interprets the completion record and reconciles buffer ownership before waking a waiter. Notification, visibility and scheduling are related steps with distinct responsibilities.*

The optimal policy depends on service-time variability and latency targets. An interrupt saves power while idle; polling avoids repeated entry overhead during a burst. Adaptive systems switch modes based on recent queue activity.

Laboratory 30: Descriptor ownership and wraparound

Model an eight-slot descriptor ring with unbounded producer p and consumer c counters. A slot is published only after its length and address are initialised. Give full/empty conditions, a correct publication order, a concrete partial-publication failure and a wraparound trace.
Worked solution

Laboratory 30: solution

The ring is empty when $p = c$ and full when $p - c = 8$. Slot indices are p mod 8 and c mod 8; the unbounded counters retain generation information. A producer first ensures space, writes address and length, then publishes ownership with release ordering. The consumer observes ownership with acquire ordering, reads the fields, completes the work and releases the slot before c advances. If ownership is published first, a consumer may see "ready" with an old length of 1024 bytes even though the new buffer is only 64 bytes. It can then transfer 960 bytes beyond the intended extent. A mutex around the shared operations is an alterna-

Worked solution (continued)

tive model. After eight posts from p=c=0, p=8,c=0 is full although both indices are zero. Consuming three gives p=8,c=3; posting three more reuses slots 0,1,2 and gives p=11,c=3, full again. Equality of indices alone cannot distinguish empty from full. Real finite counters require an explicit wrap-safe range rule and DMA ownership also needs the platform's device-memory ordering and cache-maintenance contract.

Return to the laboratory

Exercise 4.7.1 — Descriptor ownership protocol

Specify a transmit ring with producer and consumer indices. State the ownership of each descriptor, the order in which address, length and ready status become visible, how completion is detected, and when the buffer may be reused. Add the IOMMU and cache-coherence assumptions explicitly.

Solution 4.7.1

Use a ring of N descriptors and monotonically increasing producer and completion counters p and c. The slot for a new request is $p \bmod N$; the ring is full when $p - c = N$. Counters are wide enough that wraparound cannot occur during this example. Each descriptor has an address, length and a generation-aware ownership field, so an old completion cannot be confused with reuse of the same slot.

Initially the CPU owns every free descriptor and buffer. For one transmission:

1. The CPU obtains a buffer, fills its bytes, and maps it for the device's DMA access. Assume an IOMMU restricts the device to the intended DMA mappings. The descriptor contains the DMA address returned by this mapping operation, which may differ from both a CPU virtual address and a physical address.
2. While the descriptor is CPU-owned, the CPU writes its address, length and generation. It completes the platform's required DMA synchronisation and ordering operations, then publishes device ownership. The publication must become visible after all fields and payload bytes it protects. The producer counter advances, and any required device doorbell is written with the device's documented ordering.
3. The device observes ownership, reads the stable fields and payload, performs the transfer, and publishes a completion for that generation after it has finished accessing the buffer. An interrupt may notify the CPU; polling is another way to discover the same completion record.
4. The CPU observes completion using the platform's required DMA read ordering, checks status and generation, and reclaims the descriptor. It can then un-

Worked solution (continued)

map or reuse the buffer and advance c. Reuse before confirmed completion would allow the device to read a mixture of old and new data.

Assume coherent descriptor and payload memory for the first implementation. Coherence still requires the ownership publication and completion ordering above. On a non-coherent system, the CPU must additionally perform the DMA API's cache synchronisation before handing data to the device and before consuming device-written status. C atomics alone do not define device or MMIO ordering. The IOMMU limits reachable memory; it does not establish descriptor ownership, replace cache maintenance or prove that a transfer has completed.

From channel processors to DMA, IOMMUs and coherent accelerator fabrics

Timeline

Year	Machine, standard or event	Architectural significance
1958	IBM 709 channels	Independent I/O programs transfer data while the CPU computes.
1981	PC DMA controller	The Intel 8237 supplies economical delegated transfers with strict limits.
2019	CXL 1.0	Coherent accelerator and memory protocols extend PCIe-based systems.

Independent transfer engines addressed a structural imbalance: processors became too valuable to spend one instruction on every character or word transferred by a peripheral. In programmed I/O the CPU tests status, moves a unit of data and repeats. That model may be adequate for a slow console, but magnetic tape, drums and later networks could occupy the processor for long intervals. Designers created independent engines that could generate memory addresses and transfer data while the main instruction stream continued [124, 38, 132].

The National Bureau of Standards DYSEAC, specified by Alan L. Leiner in 1952 and completed in 1954, is an early documented case of external equipment transferring information into the machine while computation proceeded. NBS records describe SEAC interrupting DYSEAC and sending data into DYSEAC files without disrupting the current calculation. The architectural idea later called DMA was already present: memory traffic originated outside the central instruction sequence and required an explicit protocol for arbitration, completion and state preservation [124].

IBM then made delegated I/O a major system feature. The IBM 709, announced in 1957 and first installed in 1958, used the IBM 766 Data Synchronizer. Its independently programmed channels could control tape, card and printer equipment and transfer data between those units and magnetic-core storage while the CPU performed computation.

The 1958 *IBM 709 Reference Manual* documents the interface, and contemporary IBM material emphasised simultaneous I/O and processing. This was more capable than a later single-chip DMA controller: it executed a small I/O program and managed several devices. It established the enduring distinction between a central processor that requests an operation and an I/O engine that carries it out [111, 99].

IBM System/360, announced on 7 April 1964, generalised the model through selector and multiplexor channels. Software constructed Channel Command Words describing commands, storage addresses, byte counts and chaining. The Start I/O instruction initiated execution; channel and control-unit status was eventually reported through an input/output interruption. Because the channel fetched commands and moved data independently, the operating system had to protect the command list, keep target storage resident and interpret partial-completion status. The modern descriptor ring inherits the separation among command description, autonomous transfer and completion notification, with its own ownership and ordering rules [74, 109].

Microprocessor systems implemented a cheaper form. Intel's 8237A programmable DMA controller, available by the late 1970s and used in the IBM PC of 1981, provided four channels. A device asserted a DMA request; the controller obtained the bus, generated addresses and transfer counts, and returned control when the transfer ended. The original PC design exposed severe limits: eight-bit channels, 64 KiB boundary constraints and a narrow address space. The PC/AT of 1984 added another controller and sixteen-bit channels but retained compatibility quirks. These details matter because "DMA" never meant unrestricted access to arbitrary memory; each controller had explicit width, count, arbitration and boundary rules [125, 126].

PCI, standardised by the PCI Special Interest Group from 1992, moved toward bus mastering. A capable endpoint could become transaction initiator and issue reads or writes directly, rather than relying on one motherboard DMA chip. Ethernet adapters, SCSI controllers and graphics devices became autonomous processors with descriptor engines and local firmware. PCI Express 1.0, released in July 2002, retained the load/store transaction model over a serial packet fabric. Modern network and NVMe devices fetch submission descriptors, move payloads by DMA, post completion entries and signal through MSI-X. The external box labelled "controller" may contain several DMA engines operating on many queues in parallel [80, 81, 127, 56].

Autonomy created an addressing problem. A CPU uses virtual addresses translated by processor page tables. A device places a bus or I/O address in a transaction. On some simple platforms the bus address equals a physical address, but Linux explicitly warns that this is not a portable assumption: bridges and IOMMUs may create arbitrary mappings, and a device may support fewer address bits than the CPU. The DMA API requires a driver to map a buffer, use the returned DMA address, synchronise it according to the coherent or non-coherent contract, and unmap it at the end of the ownership interval [38, 39].

Input-Output Memory Management Units added translation and protection. IBM mainframes had long used channel-side address translation and protection mechanisms, but commodity x86 virtualisation made the requirement urgent. AMD published its I/O Virtualization Technology specification in the mid-2000s; Intel published the first VT-d architecture specification in 2006. An IOMMU translates an I/O Virtual Address to physical pages, associates requests with a device identity, caches translations and can block an access outside the assigned domain. This lets a guest or userspace driver operate a passed-through device without granting that device all of host memory. It also requires invalidation, fault handling and careful device grouping [27, 26, 132].

Linux expresses this protection through the generic IOMMU subsystem, VFIO and IOMMUFD. VFIO provides an IOMMU-aware framework for secure userspace device access; IOMMUFD, merged in Linux 6.2 in February 2023, exposes file-descriptor objects for I/O address spaces and device attachment. The current kernel documentation stresses that an I/O address space can be shared by several devices and that userspace mappings are copied into hardware domains. The interface is not a licence for arbitrary physical access: isolation depends on firmware topology, interrupt remapping, driver binding and correct lifetime management [132, 133].

DMA also exposes cache and ordering rules. On a coherent platform, hardware may keep CPU caches and device accesses mutually visible, but descriptor publication still needs ordering: software must finish writing a descriptor before ringing a doorbell, and it must observe a completion before consuming dependent data. On a non-coherent platform, the driver may additionally clean or invalidate cache lines. Linux distinguishes coherent allocations, streaming mappings and explicit synchronisation operations because an ordinary C pointer does not encode device ownership or interconnect ordering [38, 39, 40].

Scatter-gather lists solved the requirement for physically contiguous buffers. A user buffer can span many physical pages; the kernel maps those pages and supplies a list of segments or converts them into device descriptors. Queue-based controllers then consume producer and consumer indices. Remote Direct Memory Access extended registered, protected transfers across a network, while GPUs and accelerators added multiple copy engines and peer-to-peer paths. Setup costs remain real: pages may be pinned, IOMMU caches must be invalidated, registration must be revoked, and small transfers may still be cheaper to copy on the CPU [124, 38, 132].

Compute Express Link added another stage. CXL 1.0 was announced in 2019 as a cache-coherent interconnect built on PCI Express physical infrastructure. CXL 2.0 in 2020 added switching and memory pooling; CXL 3.0 in 2022 expanded fabrics and memory sharing; CXL 4.0, released by the consortium in late 2025 with its evaluation specification dated February 2026, doubles the link rate to 128 GT/s and extends fabric and reliability capabilities. CXL does not abolish DMA. It adds coherent cache and memory protocols so selected accelerators and memory devices can participate in system-wide addressability under new consistency, discovery and security rules [82, 135, 81].

The evolution can be read as a sequence of increasingly explicit contracts. DYSEAC demonstrated external transfer during computation. The IBM 709 Data Synchronizer and System/360 channels then executed delegated I/O programs, while the 8237 reduced the idea to an economical PC peripheral. PCI bus masters made endpoints autonomous; IOMMUs translated and confined their transactions; Linux APIs exposed mapping and lifetime rules; and CXL added coherent accelerator and memory semantics. Every generation must answer the same questions. Who owns the buffer? Which address space does the device see? What ordering is guaranteed, how is completion reported, and what prevents a faulty device from corrupting unrelated memory? [124, 111, 109, 38, 132, 135]

DMA still transfers ownership of memory transactions to devices, while IOMMUs and kernel frameworks constrain which memory those devices may reach.

A completion notification closes the device-side transaction but not necessarily the software story. Suppose the request belonged to a process that blocked while waiting for the data. Once the driver records successful completion, the kernel can make that execution runnable again. The buffer now contains the requested data, yet the process still cannot continue until some logical processor is assigned to it. Process state, context switching and scheduling determine when the newly runnable work actually resumes.

Concept check

1. Is a serial port a character or a block device?
2. With DMA, who moves the data during the transfer?
3. How can software learn that a DMA transfer has completed, and why is the answer not necessarily one interrupt per block?
4. In the simple teaching controller, which source, destination, count and control fields are programmed, and how do modern descriptor rings generalise them?
5. Why is an arbiter needed once DMA is present?

Solutions to the review questions

1. A serial port is normally represented as a character device, exposing an ordered byte stream. A block interface instead addresses fixed-size storage blocks with block-oriented operations.
2. The DMA-capable device or DMA engine issues the data movement transactions after software has configured the operation and made the relevant memory available. The CPU establishes and supervises the operation without copying each data item itself.
3. Software can poll a status or completion record, receive an interrupt, or combine both approaches. A controller can batch completions or coalesce interrupts, so

Worked solution (continued)

one notification may cover several completed blocks or descriptors. The completion record identifies the work actually finished.

4. A simple controller needs source and destination addresses, a transfer count and control information such as direction and start/enable state. A descriptor ring stores these fields for multiple operations and adds ownership/status information. Producer and consumer positions identify which descriptors are ready and which have completed.
5. The processor and DMA initiators can request the same interconnect or memory resources. Arbitration coordinates those requests, prevents conflicting ownership where the protocol requires it, and implements priority or fairness. The resulting sharing also affects throughput and latency.

4.7.6 Worked problems

Transfer time and CPU occupancy

An 8 MiB DMA transfer at 400 MiB/s needs $8/400 = 0.02$ s, or 20 ms, for its data phase. If setup consumes 12 µs and completion processing consumes 8 µs of CPU time, the CPU spends 20 µs on those two phases. Assuming serial phases and no contention or notification delay, elapsed time is 20.020 ms. The CPU can perform other work during much of the 20 ms data phase; memory-bandwidth contention may still affect that work.

Descriptor ring

A DMA engine uses a four-entry descriptor ring. Producer and consumer indices both begin at 0, and an explicit occupancy count begins at 0.
Question. Trace the ring while the driver submits A, B, and C; the device completes A and B; then the driver submits D and E. Where do the indices and occupancy finish?
Reasoning.

1. **Submit A, B, and C.** They occupy entries 0, 1, and 2. The producer advances modulo four to 3; consumer remains 0; occupancy becomes 3.
2. **Complete A and B.** The consumer advances from 0 to 2 and occupancy falls from 3 to 1. Entry 2, containing C, is still owned by the device.
3. **Submit D and E with wraparound.** D is written at entry 3, moving producer to 0 and occupancy to 2. E is written at entry 0, moving producer to 1 and occupancy to 3.

Worked example (continued)

Event	Producer	Consumer	Occupancy
Initial	0	0	0
Submit A,B,C	3	0	3
Complete A,B	3	2	1
Submit D	0	2	2
Submit E	1	2	3

The final state is producer 1, consumer 2, occupancy 3. The live descriptors are E at entry 0, C at entry 2, and D at entry 3.
Producer marks the next free slot for software, consumer marks the next completion to reclaim, and both advance modulo the ring size. The occupancy count removes the usual ambiguity in which equal indices could mean either empty or full.
Count live entries directly: C was not completed, and D and E were added later, so exactly three descriptors must remain. The final occupancy agrees.
If only A completes before D and E are submitted, occupancy progresses from 3 to 2 to 3 to 4. The ring is then full and another submission must wait.

Exercise 4.7.2 — Bus master

Who initiates memory bus transactions during DMA?

Worked solution

After setup and any required arbitration, the DMA-capable controller or engine initiates the memory transactions for the payload. The CPU prepares descriptors, addresses, lengths and permissions, then starts or queues the operation. The engine transfers data and reports completion through the configured mechanism. Address translation and coherence rules still apply where the platform requires them.

Exercise 4.7.3 — Interrupt count

Must one block cause one interrupt?

Worked solution

A block transfer does not necessarily cause exactly one interrupt. A controller may group several completions before interrupting, or the driver may inspect a

Worked solution (continued)

completion queue by polling. For example, an interrupt after eight completed blocks gives one notification for the batch. Reducing notifications can save CPU overhead while increasing notification delay; the completion protocol must still identify every finished transfer.

Exercise 4.7.4 — Driver abstraction

Why use a driver?

Worked solution

A driver translates an operating-system request into device-specific commands and interprets the resulting status and errors. It also manages access, buffers, queues and completion. Applications can then use a common interface across different controllers. For example, the same logical read request may be implemented through very different hardware command formats without requiring application code to construct those commands.

Exercise 4.7.5 — PIO vs DMA

Which moves each payload word with CPU instructions?

Worked solution

Programmed I/O uses CPU instructions to move each payload word through device registers or mapped locations. For a 4096-byte payload moved four bytes at a time, that requires 1024 payload transfers executed by the CPU, in addition to control work. DMA assigns the payload movement to an engine after setup. Polling versus interrupt notification is a separate choice from which agent moves the payload.

Summary

A correct DMA trace names the buffer, address domain, owner and ordering point. A performance trace adds setup cost, transfer size, contention and completion policy. Real driver design needs both: speed cannot compensate for a stale descriptor or incorrect mapping.

4.7.7 Exercises

Exercise 4.7.6 — Device class

Classify a terminal and a disk under the character/block interface distinction.

Go to the solution →

Exercise 4.7.7 — PIO cost

PIO costs 2.5 CPU cycles per byte plus 100 setup cycles. Find the CPU cost for 4 KiB.

Go to the solution →

Exercise 4.7.8 — DMA crossover

PIO is $50 + 4n$ cycles and DMA is $1250 + 0.1n$ cycles. Find the crossover size.

Go to the solution →

Exercise 4.7.9 — Scatter–gather

Assume the device-visible mapping preserves three separate segments of 2 KiB, 4 KiB and 1 KiB, with no further splitting. How many descriptors are required if each describes one segment? What is total length?

Go to the solution →

Exercise 4.7.10 — Publish ordering

State the required order when posting a descriptor ring entry.

Go to the solution →

Exercise 4.7.11 — Cache direction

On a non-coherent system, what cache action is needed before a device reads CPU-produced data and before the CPU reads device-produced data?

Go to the solution →

Exercise 4.7.12 — IOMMU protection

What attack is prevented by mapping only a receive buffer into a device's IOMMU domain?

Go to the solution →

Exercise 4.7.13 — Hybrid completion

Describe a completion policy for a queue that alternates between long idle periods and saturated bursts.

Go to the solution →

4.7.8 Solutions

Device Classes and Direct Memory Access

Solution 4.7.6

A terminal is character-oriented: it is a stream with terminal semantics. A disk is block-oriented: independently addressable blocks support caching and file systems.

← Return to the exercise

Solution 4.7.7

The transfer contains $4 \cdot 1024 = 4096$ bytes. Per-byte work costs $4096 \cdot 2.5 = 10{,}240$ cycles. Adding the one-time setup cost gives $10{,}240 + 100 = 10{,}340$ CPU cycles. At a specified core frequency f, the corresponding CPU time would be $10{,}340/f$ seconds. Setup is paid once for this request, while the transfer term scales with its byte count.

← Return to the exercise

Solution 4.7.8

Set equal: $50 + 4n = 1250 + 0.1n$, so $3.9n = 1200$ and $n \approx 307.7$ bytes. For whole-byte requests, 308 bytes is the first size where DMA is cheaper: PIO costs 1282 cycles and DMA costs 1280.8. At 307 bytes PIO costs 1278 cycles, less than DMA's 1280.7.

← Return to the exercise

Solution 4.7.9

Each descriptor describes one mapped device-visible region. The buffer has three separate regions, so it needs three descriptors, with lengths 2048, 4096 and 1024 bytes. Their sum is $2 + 4 + 1 = 7$ KiB $= 7168$ bytes. A scatter–gather engine follows the descriptors in the requested logical order; the physical gaps between their addresses do not contribute to transferred length.

← Return to the exercise

Solution 4.7.10

Start with a CPU-owned free slot. Fill in the mapped DMA address, length and operation flags. Make the payload and descriptor fields visible under the platform's DMA synchronisation rules. Publish ownership or the producer position only after those writes, then ring the doorbell with the prescribed MMIO accessor. A device that sees ownership may now fetch the descriptor. On completion, the CPU first

Worked solution (continued)

observes the device's completion marker and applies the required read ordering or synchronisation before consuming status or reclaiming the slot.

← Return to the exercise

Solution 4.7.11

Clean/write back dirty CPU cache lines before device reads. Invalidate or synchronise relevant cache lines before the CPU consumes data written by the device. Drivers use the platform's DMA mapping and synchronisation APIs with the correct transfer direction; those APIs implement the required architecture-specific operations. The CPU must also relinquish ownership before device writes so a dirty cache line cannot later overwrite the received data.

← Return to the exercise

Solution 4.7.12

Give the receive engine write permission only for the intended mapping in its enforced IOMMU domain. A write into that buffer succeeds. A descriptor aimed at an unrelated kernel page has no authorised translation, so the IOMMU blocks the transaction and may report a fault. The protection is at the mapping granularity. The driver must still validate lengths and ownership inside mapped pages, and must revoke mappings after the device has finished.

← Return to the exercise

Solution 4.7.13

Enable an interrupt while idle. On the first completion, mask or moderate further interrupts and poll/drain a bounded batch. Continue polling while work remains; after a quiet threshold, re-enable interrupts using the device's race-safe arm-and-check protocol. If a completion arrives between the last empty check and interrupt rearming, software must detect it or the device must latch an interrupt; otherwise work can remain stranded. The polling budget preserves service for other work.

← Return to the exercise

5

PROCESSES, THREADS, AND CONCURRENT EXECUTION

A computer system rarely runs one activity from beginning to end without interruption. Interactive programs, background services and kernel work all compete for processors and devices. The operating system turns a program stored on disk into a *process*: a protected execution context with an address space, processor state, open resources and an identity. On a single processor, several processes make progress by taking turns; on a multicore machine, some of them may truly execute at the same instant.

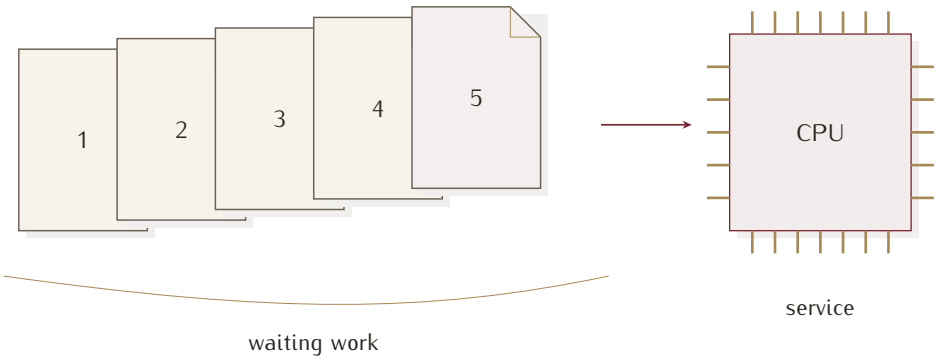
This Part develops the process abstraction, process creation and termination, CPU scheduling, threads, race conditions, semaphores, classical synchronisation problems and deadlock. The discussion separates three questions that are often confused: *who may run*, *who may access shared state*, and *whether a set of waiting activities can ever make progress*. The first is a scheduling question, the second a synchronisation question, and the third a liveness question [57, 59, 58].

The examples use Unix/Linux terminology where it clarifies the mechanisms, but the principles are broader than one operating system. Exact interfaces and edge cases must be checked against the relevant standards and current manual pages [32].

5.1 PROCESSES, STATES, AND CONTEXT SWITCHING

A storage request completes while another computation is using the processor. The waiting thread can become runnable, and the scheduler must decide when it should resume. To pause one execution and later continue it, the kernel retains execution state together with identity, resources and scheduling information. Following this handover introduces the process abstraction and the thread-level execution contexts that a real kernel schedules [57, 59, 58].

An execution alternates between service and waiting



Queue order and service policy determine who progresses next.

Figure 5.1: *The waiting sheets represent ready executions competing for processor service. A blocked execution belongs to a different waiting relationship until its dependency is satisfied. The scheduler chooses which eligible continuation runs next.*

A program and a process are therefore related but not interchangeable ideas. A program is a passive representation of instructions and data—for example an executable file—whereas a process is an active execution context with an identity, an address space, architectural state and references to operating-system resources. Starting the same executable twice creates two processes whose code may initially be identical but whose registers, stacks, heap state, open-resource state and scheduling histories can diverge immediately. Conversely, a single process need not execute the same program for its en-

tire lifetime: as the next chapter will show, Unix-like systems can preserve the process identity while replacing its program image with `exec`.

5.1.1 The process as an execution and protection abstraction

At the hardware level, instructions execute on schedulable hardware contexts. On a simple processor there may be one such context; on a multicore machine there are several physical cores, and a core that implements simultaneous multithreading may expose more than one *logical processor* or hardware thread. The distinction matters. The scheduler assigns a software thread to an available logical processor. Several such contexts may share the execution resources of one physical core.

For the state traces in this chapter, assume one software thread per process. In a multithreaded process, each thread has its own registers, stack and scheduling state while sharing the process address space and many resources.

The process abstraction lets the kernel multiplex those execution contexts among activities that may be unrelated or mutually untrusted. Each process receives an address-space view, credentials and resource references that are controlled independently of another process's view; at the same time, the kernel gains a unit to which it can attach scheduling policy, accounting, signals, termination and protection decisions. The abstraction therefore serves two purposes at once: it describes an execution that can be stopped and resumed, and it defines a protection domain within which ordinary program state is interpreted.

	Program	Process
Nature	passive description	active protected execution
Typical representation	executable file	address space + kernel metadata
Architectural state	none	PC/IP, registers, flags, stack pointer
Resources	requested by code	allocated/referenced by the kernel
Multiplicity	one file may be shared	many independent instances may coexist

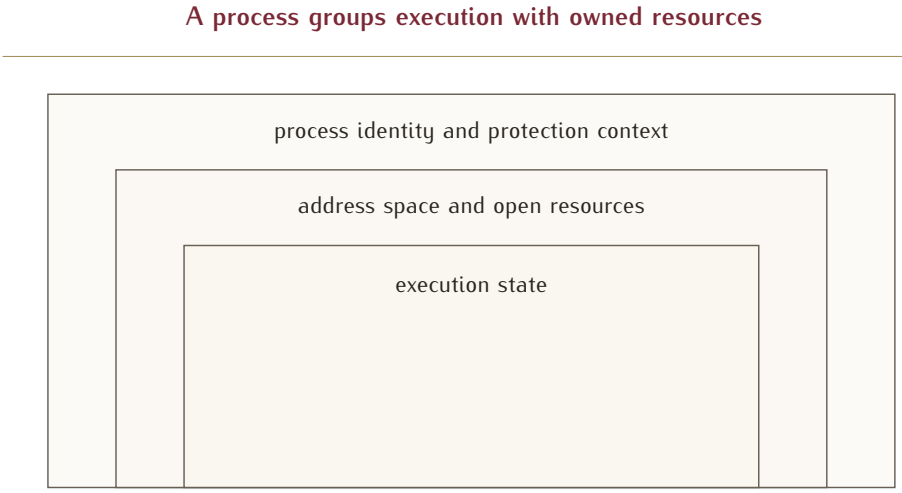


Figure 5.2: *The nested regions organise related state; they are not literal contiguous memory areas. The process supplies a resource and protection context, while each schedulable execution requires its own saved registers and stack.*

This view also clarifies three terms that are often blurred together. *Multiprogramming* keeps enough independent work available that the processor need not sit idle merely because one activity is waiting for I/O. *Multitasking* uses scheduling and, commonly, preemption to let several activities make progress over the same interval, which is essential for interactive response. *Parallelism* is narrower: two software execution streams are parallel only when they actually advance at the same time on distinct hardware execution contexts. *Concurrency* is broader than either implementation choice; concurrent activities have overlapping lifetimes and can interact even when a single logical processor interleaves them rather than running them simultaneously.

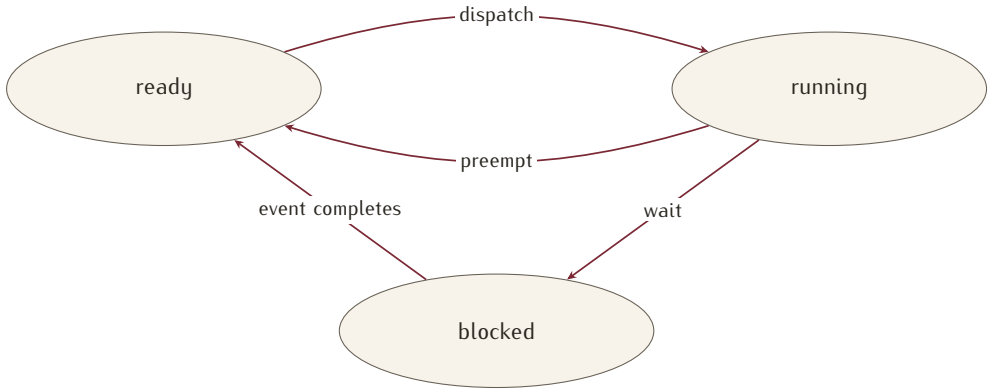
The operating system can therefore create the appearance that many processes are running "at once" without requiring one processor for each process. Each single-threaded process follows its own instruction sequence, while the kernel decides where the pauses occur and prevents one process from freely reading or overwriting another’s protected state.

5.1.2 From running to ready to blocked

A scheduler does not need to consider every existing process at every instant. It needs to distinguish activities that can use a logical processor from those that cannot. A compact five-state model makes this distinction explicit. A newly created process begins in a conceptual *new* state while the system establishes the resources required to admit it. Once eligible for execution it is *ready* (often called *runnable*); after the scheduler selects

it and the dispatcher installs its execution context, it is *running*. If the process reaches an operation that cannot complete until some event occurs—for example a disk request, a positive-length read from an empty blocking pipe whose write end remains open, a timer, or a contended synchronisation object—it becomes *blocked* or *waiting*. When it finishes or is forcibly terminated, it enters a terminal condition in which ordinary execution no longer continues. Real kernels subdivide these states for implementation and policy reasons, but the compact model exposes the transitions that matter for reasoning [59, 57].

Ready, running and blocked have distinct meanings



Completion moves blocked work to ready, not directly to running.

Figure 5.3: *Ready work is eligible for a processor. Running work currently has one. Blocked work still depends on an event or resource. The scheduler's dispatch supplies the separate transition from eligibility to execution.*

The distinction between *ready* and *blocked* is the most important one in the diagram. A ready process lacks only a processor: if the dispatcher gave it a logical processor now, it could execute immediately. A blocked process lacks some event on which its next progress depends, so assigning it a processor before that event occurs would merely waste the processor. A disk-completion handler can record the completed operation and move the waiting process from blocked to ready, but becoming ready does not imply that the process executes immediately. It has re-entered the population from which scheduling policy may choose.

Consider three processes. Process A is currently computing, process B issued a disk read and is waiting for the completion, and process C has just been admitted. Before the interrupt, A is running, B is blocked and C is ready. When the disk interrupt arrives, B becomes ready. Whether A is then preempted depends on policy: a non-preemptive system may let A continue until it blocks or exits, whereas a preemptive scheduler may compare priorities, elapsed quantum or other policy state before decid-

ing what runs next. The state transition and the scheduling decision are related, but they are not the same operation.

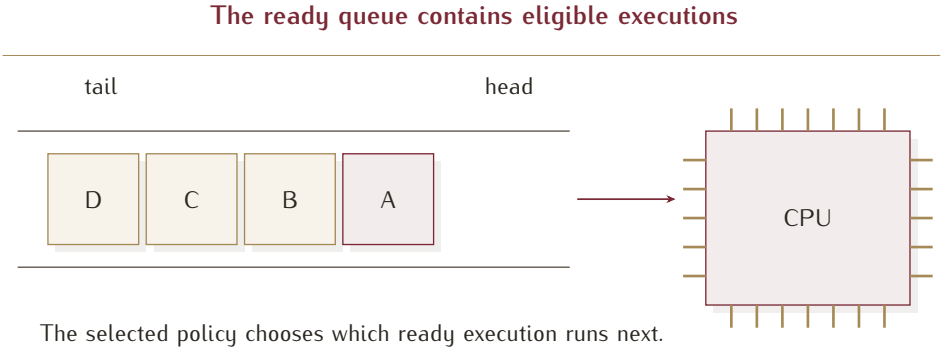


Figure 5.4: *The queue represents scheduling eligibility rather than every existing process. A waiting operation must first become ready through its completion condition before ordinary ready-queue selection can dispatch it.*

A useful vocabulary follows from this separation. The *scheduler* chooses which runnable execution should receive a processor; the *dispatcher* performs the machinery needed to make that choice effective by switching execution context. Some texts use the terms less rigidly, and real kernels distribute the work across several functions, but keeping policy and mechanism conceptually distinct prevents a common confusion: deciding *who* should run is not the same as installing the state that lets that execution resume.

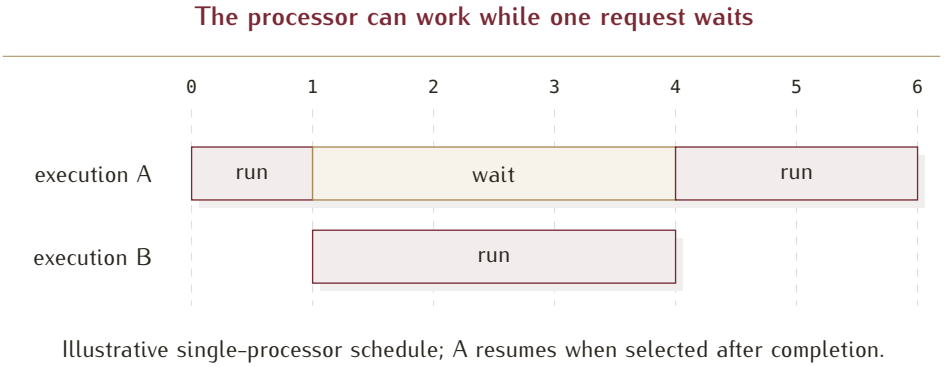


Figure 5.5: *The waiting interval is not a second running execution. Another ready execution can use the processor during that interval, preserving useful work while A depends on I/O or another event.*

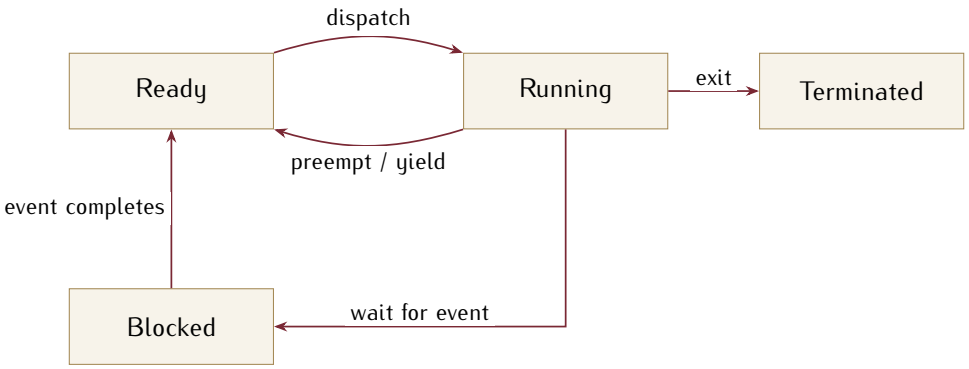


Figure 5.6: *A process-state model for one execution context. Wake-up makes a blocked process ready; it does not guarantee immediate dispatch. Creation and admission are omitted so that the scheduling transitions remain visible.*

5.1.3 Context switching: preserving the illusion of continuity

Suppose A is running and the scheduler chooses B. Resuming B requires its saved architectural context, including its next instruction address, general-purpose registers, stack pointer, status flags and, when relevant, floating-point, vector or other architecture-specific state. Enough of A's context must therefore be preserved so that a later return to A restores the machine state expected by A's instruction stream; enough of B's saved context must then be installed so that execution continues from the point at which B previously stopped. This transition is a *context switch*.

The exact save/restore path is architecture- and kernel-dependent. Some state may already have been saved by an interrupt or exception entry sequence, some may be saved lazily or eagerly by software, and a thread switch within one address space may require less memory-context work than a switch between processes. The invariant is more general than any particular stack frame: once a process is resumed, the architectural state visible to its program must be consistent with a legal continuation of that process. A process-control structure connects an execution to the saved machine state and resource information required to manage it.

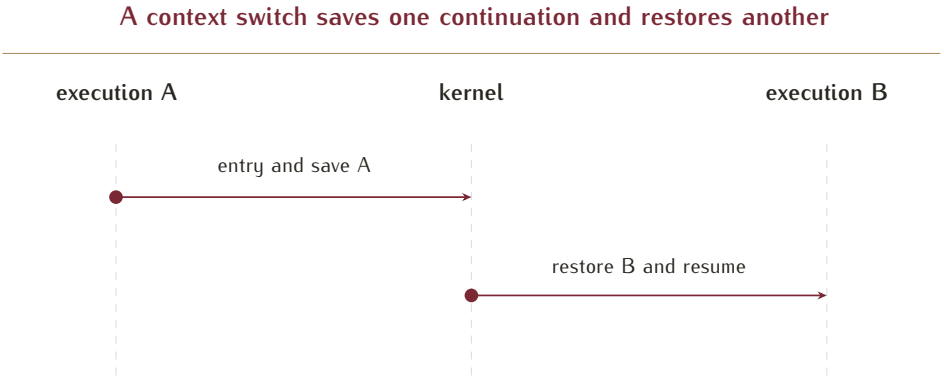


Figure 5.7: *The kernel records enough of A's state for a later continuation, selects B and restores B's context. Switching between threads of the same process can preserve the address space while still changing execution state.*

Several events can create an opportunity or need for a context switch. A timer interrupt may end a time quantum; a running process may invoke a system call that blocks; a page fault may require an operation that cannot complete immediately; a process may explicitly yield; an interrupt may wake a higher-priority task; or the current process may terminate. Termination or an actual blocking transition requires relinquishing the current execution context, possibly to an idle task if no ordinary task is runnable. The arrival of an interrupt, syscall or fault alone does not require a task switch. The kernel may handle an interrupt and return to the same process, a system call may complete without blocking, and a fault may be resolved without selecting another task. The important question is whether policy and runnable-state changes make another execution the next one to run.

A switch also has more than one cost. The *direct* cost is the work performed by the kernel: entering privileged code when necessary, saving and restoring state, updating scheduling structures and changing the active memory context. The *indirect* cost can be larger and more workload-dependent. The incoming execution may find that useful cache lines have been displaced, branch-prediction state reflects another control-flow history, or translation structures no longer contain the mappings it needs. Some architectures and operating systems can tag translation entries or preserve selected state across switches, so a switch should not be equated mechanically with "flush every cache and TLB"; nevertheless, rapid switching can reduce locality even when the explicit save/restore sequence is short.

Time sharing must balance this overhead against response time. If a round-robin quantum becomes shorter, an interactive process may receive service sooner after becoming ready, but the machine also spends a larger fraction of its time crossing scheduling

boundaries and repeatedly warming execution state. If the quantum becomes extremely long, the direct overhead falls but response begins to resemble non-preemptive FCFS. Scheduling policy therefore cannot be designed independently of context-switch mechanism and workload behaviour; the same quantum that is reasonable on one machine or workload may be poor on another.

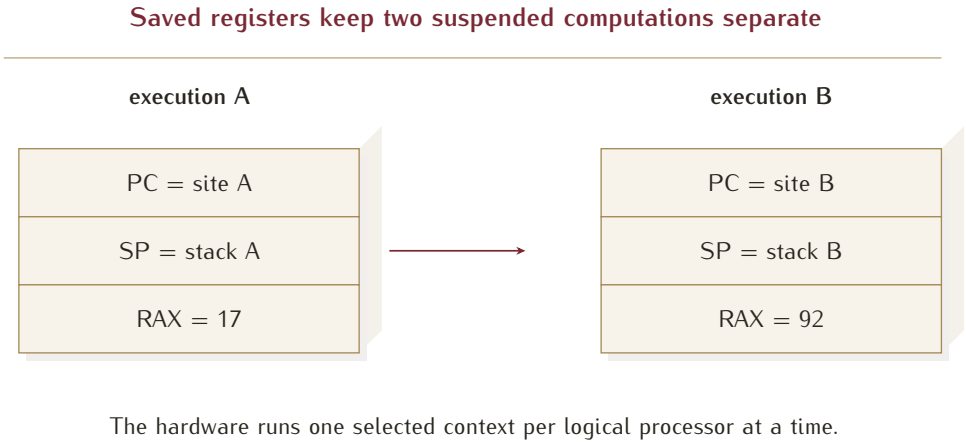


Figure 5.8: *The values belong to separate continuations even if they use the same architectural register names. Restoring a context puts its saved values back into the running hardware state.*

A step-by-step switch helps connect the abstraction to the machine. Imagine that process A is executing in user mode when a timer interrupt arrives. The interrupt-entry mechanism transfers control to privileged code and establishes an execution context from which the kernel can run safely; depending on the architecture, part of A's return state is recorded automatically and the rest is saved by the kernel's entry code. The scheduler can then account for A's consumed time and decide whether A should remain the current execution. If policy selects process B, the kernel records A's resumable thread state, changes the current scheduling entity, installs the memory-management context and kernel bookkeeping associated with B where necessary, restores B's saved kernel execution context, and lets B continue through its kernel path. When B is ready to return to user mode, the architecture-specific return path restores its user context. A kernel thread may remain entirely in kernel mode. From A's point of view, none of B's execution appears in its own instruction sequence: when A eventually runs again, it resumes from a state consistent with the interrupt and subsequent return.

Two threads in one process normally share the same memory mappings. Switching between unrelated processes normally selects a different memory context. Tagged translations can stay cached across the change, provided that each later reference uses the incoming context's mappings and permissions.

Scheduling queues provide the other half of the story. A thread that is preempted while still able to run normally returns to a ready queue or equivalent runnable structure; a thread that initiated a blocking operation is instead linked to some wait condi-

tion or kernel object from which completion can later awaken it. The completion path reverses that relationship: the event marks the blocked execution runnable and places it where the scheduler can consider it. This is why ready queues and wait queues should not be imagined as mere lists maintained for convenience. They encode two different predicates about progress: "could execute if given a logical processor" and "cannot proceed until this event changes state".

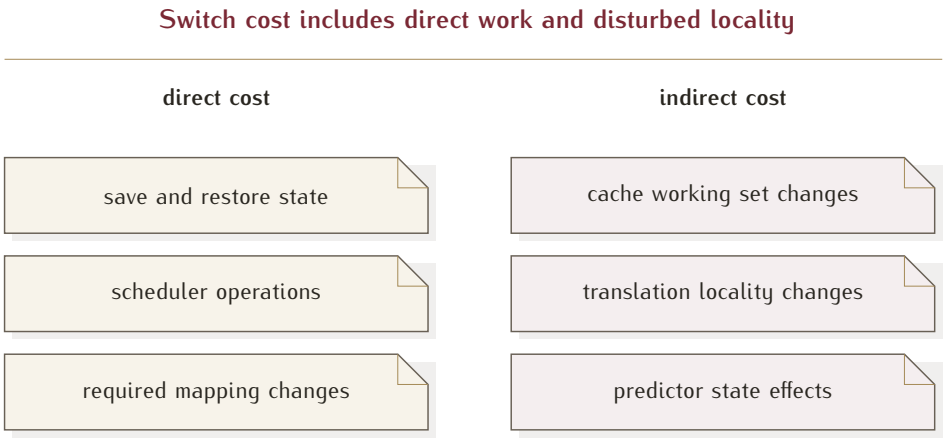


Figure 5.9: *The measured cost of switching can include later effects on ordinary instructions. Counting only the register-save instructions omits possible changes to caches and other locality-sensitive structures.*

On a multiprocessor system these transitions can occur concurrently on different logical processors. One processor may be waking a thread while another is choosing its next runnable task, so scheduler data structures themselves require synchronisation. Per-processor run queues reduce some shared contention and improve locality, but they create a balancing problem when work accumulates unevenly. Those policy details belong to Chapter 5.3; the process/thread states describe concrete execution and waiting conditions. Those structures determine whether and where an execution can next run.

5.1.4 What the kernel must remember

The persistent metadata associated with a process is commonly described in teaching as a *process control block* (PCB). Real kernels distribute this information across implementation-specific structures, including objects shared by several threads. The abstraction identifies what the kernel must retain while a process is not running.

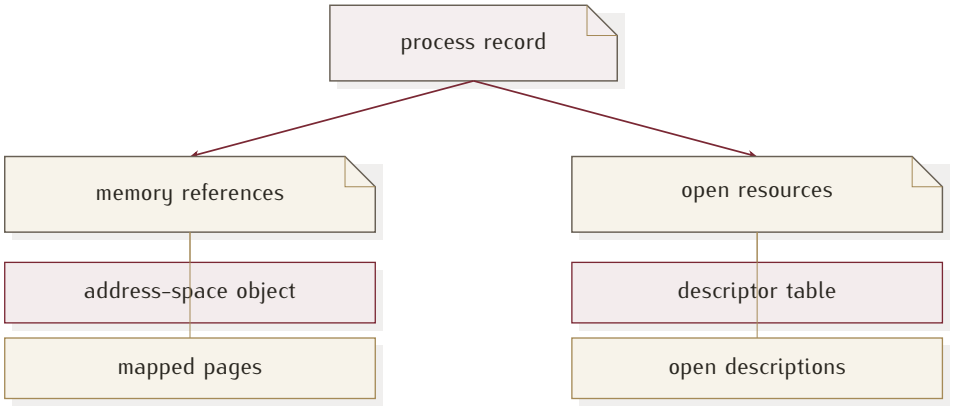
Typical information falls into several groups:

- *identity and authority*: process identifier, parent relationship, credentials, session or process-group membership;

- *saved execution state*: enough architectural state to resume an execution context, together with kernel-stack or thread-state references used by the switch path;
- *scheduling state*: ready/blocked/running status, priority or policy class, queue links, accounting and affinity information;
- *address-space state*: references to the structures that describe mappings, protection and the process's memory context;
- *resource references*: open file descriptors and the kernel objects to which they lead, signal state, timers and interprocess-communication objects;
- *relationships and lifecycle state*: parent/child information, exit status and other bookkeeping needed for creation, waiting and termination.

The PCB contains control metadata that lets the kernel locate, protect, schedule and eventually release the process's resources; its references identify memory managed elsewhere. Its fields serve different purposes: saved register state exists because execution must resume correctly, whereas a priority value exists because the kernel has chosen a scheduling policy. Keeping those roles separate will matter later, especially when a process contains several threads: some resources naturally belong to the process as a whole, while the register set, stack and scheduling state belong to each execution stream.

Kernel metadata points to resources with different lifetimes



The drawing follows references rather than copying the full resources.

Figure 5.10: *The process record provides control metadata and references to separately managed objects. Some referenced objects can be shared and outlive one process. Releasing a process therefore requires lifetime accounting across those references.*

Process creation establishes this execution context. Program replacement changes its executable image while preserving its identity; Chapter 5.2 develops both operations.

Concept check

1. Why is a logical processor, rather than a physical core, the safer unit when reasoning about schedulable hardware contexts?
2. What distinguishes ready from blocked, and why can a completion interrupt change one state without immediately causing execution?
3. Which parts of a context switch are direct kernel work and which costs arise indirectly from disturbed locality?
4. Which process-control information exists to resume computation, and which exists primarily to implement operating-system policy?

Solutions to the review questions

1. A physical core can expose more than one hardware execution context through simultaneous multithreading. The scheduler dispatches onto those logical processors. Counting only cores would miss contexts; treating them as fully independent cores would overstate their shared execution resources.
2. A ready execution can run when a processor is assigned. A blocked execution still needs an event or resource. Completion can satisfy that dependency and make it ready, but scheduling must subsequently select it for execution.
3. Direct work includes saving and restoring architectural state, updating scheduler structures and performing required address-space changes. Indirect costs include lost cache or TLB locality and changed predictor state. Those later misses are consequences of the switch even though they occur while ordinary instructions run.
4. Resumption needs the saved instruction position, registers, stack information and address-space identity. Scheduling priority, accounting, credentials, resource limits and wait relationships primarily guide policy and resource management. Some fields participate in both roles; classification follows how the kernel uses them.

5.1.5 Worked problems

State trace without a hidden dispatch

A is running, B is waiting for a network receive and C is ready. A packet arrives for B during A's quantum. Once the handler records the packet and satisfies B's wait condition, B enters the ready set alongside C. A can resume after the handler or a preemptive scheduler can choose B or C. The packet determines B's eligibility. The scheduling policy determines which thread runs next.

Context-switch overhead with coherent units

A 6 ms quantum gives the running task 6000 µs of service. A switch then occupies another 30 µs before the next task begins. The switching share of a complete cycle is $30/(6000+30) = 0.004975$, about 0.50%. Halving the quantum gives $30/(3000+30) \approx 0.99\%$. Both calculations assume a full quantum followed by exactly one switch.

PCB fields: resumption, policy and resources

The saved PC and register image reconstruct the suspended instruction stream. A reference to the address-space structures lets the kernel select the mappings that give those addresses meaning. Priority affects who receives service next, while the CPU-time counter records service already consumed. Credentials determine which requests the kernel can authorise and file references locate open objects. These fields can reside in several kernel structures linked from the task record.

Concurrency, logical processors and maximum simultaneous execution

Seven single-threaded processes are ready and three logical processors are free. The scheduler can dispatch three processes immediately. Four stay in the ready set. If a running process blocks for I/O, one of the four can take its place. All seven can make progress over time, with at most three running at any instant.

Exercise 5.1.1 — Program or process

Classify a dormant executable file and a currently running copy of it.

Worked solution

The dormant executable is a stored program representation containing code and metadata. A running instance is a process with an address space, registers, scheduling state and resources such as open files. Two processes can execute the same program file while having different register values and private data. Classifying the file and the execution instance separately explains why program identity alone does not identify a process.

Exercise 5.1.2 — Ready or blocked

A process has all input data and waits only for a processor. Which state applies?

Worked solution

The process is ready: it has everything needed to make progress except an assigned processor. A blocked process instead waits for an event such as input or resource availability. Once a scheduler dispatches this ready process, it becomes running. The distinction determines which processes are eligible candidates at the next scheduling decision.

Exercise 5.1.3 — Event completion

Does an I/O completion force the unblocked process to execute immediately?

Worked solution

An I/O completion removes the reason for blocking and normally moves the process into the ready set. The scheduler then decides when and on which processor it runs. It may dispatch immediately, preempt another process, or leave the process ready for later service according to policy and availability. Becoming runnable is therefore a state transition that precedes actual execution.

Exercise 5.1.4 — Context order

For a simplified switch path that saves the complete outgoing context on entry, order: restore selected state, save outgoing state, kernel entry event, choose runnable context.

Worked solution

Under the stated simplified path, the order is kernel entry, save outgoing state, choose a runnable context, then restore the selected state. Kernel entry provides controlled execution of the switching code. Saving preserves the interrupted computation before its registers are reused. Selection identifies the incoming process, and restoration loads the state needed to resume it. Real kernels often split state saving between entry and the task-switch routine: the scheduler can choose the incoming task before the switch routine saves the remaining kernel context. Outgoing state must be preserved before the corresponding registers are repurposed.

Exercise 5.1.5 — Core limit

Nine single-threaded ready processes share four free cores, each exposing one logical processor. What is the maximum number running simultaneously?

Worked solution

Each free core can run one of these processes at an instant under the stated model. Four cores therefore allow at most $\min(9,4) = \boxed{4}$ simultaneously running processes. The remaining five stay ready until dispatched. Time sharing can let all nine make progress over an interval, while the instantaneous running count remains limited by available execution contexts.

Summary

A process is the operating system's protected execution abstraction. State transitions describe whether it can use a processor, while the PCB preserves identity, resources and resumable processor state. Context switching creates the illusion of simultaneous progress on one processor and enables real parallel placement on several processors.

5.1.6 Exercises

Exercise 5.1.6 — Process or program

Classify each item as primarily a program or a process: an ELF file on disk; a running single-process text editor; two independently created single-process editor instances using the same executable.

Go to the solution →

Exercise 5.1.7 — State transitions

Assume a running process requests disk data that are absent from cache and must wait for device completion. State its transition, and state the transition caused by the completion interrupt.

Go to the solution →

Exercise 5.1.8 — Ready versus blocked

Explain why giving a processor to a blocked process does not solve its immediate problem.

Go to the solution →

Exercise 5.1.9 — Context record

List the minimum architectural state needed to resume an interrupted sequential computation.

Go to the solution →

Exercise 5.1.10 — Switch overhead

A 4 ms quantum is followed by a 20 microsecond context switch. What fraction of processor time is switching if every process uses its full quantum?

Go to the solution →

Exercise 5.1.11 — PCB design

A kernel PCB must support both resumption and policy. Classify saved PC, priority, open-file references, credentials and accumulated CPU time as primarily mechanism, policy, or both; then name one field that must be restored for execution to resume and one field whose value can influence a scheduling decision without being restored into a CPU register.

Go to the solution →

5.1.7 Solutions

Processes, States, and Context Switching

Solution 5.1.6

The ELF file is a program representation. The running editor is a process. The two independently created editor instances are two processes, even if they share read-only executable code pages: their identities and private execution state remain distinct. A real application may create several processes or reuse an existing one, so counting windows or launch commands alone would not establish a process count.

← Return to the exercise

Solution 5.1.7

The request moves the process from running to blocked. Completion moves it from blocked to ready; dispatch to running is a separate scheduler decision.

← Return to the exercise

Solution 5.1.8

A blocked process needs its awaited event or resource before it can use a processor. Running it cannot complete the missing disk operation, lock release or message arrival.

← Return to the exercise

Solution 5.1.9

Save the continuation PC, stack pointer, live integer registers and flags that can affect subsequent instructions. Preserve floating-point, vector and other architectural state that the interrupted computation can observe. For example, a comparison interrupted immediately before a conditional jump requires the flags as well as the jump's address. The resumed thread must also run with its valid mappings and privilege context. Hardware entry, the kernel stack and thread records can share responsibility for preserving this state.

← Return to the exercise

Solution 5.1.10

Convert the 4 ms quantum to 4000 µs. One full work-and-switch cycle takes $4000 + 20 = 4020$ µs. Its switching fraction is $20/4020 \approx 0.004975$, or about 0.4975%. This assumes every quantum is fully used and exactly one 20 µs switch follows each quantum.

← Return to the exercise

Solution 5.1.11

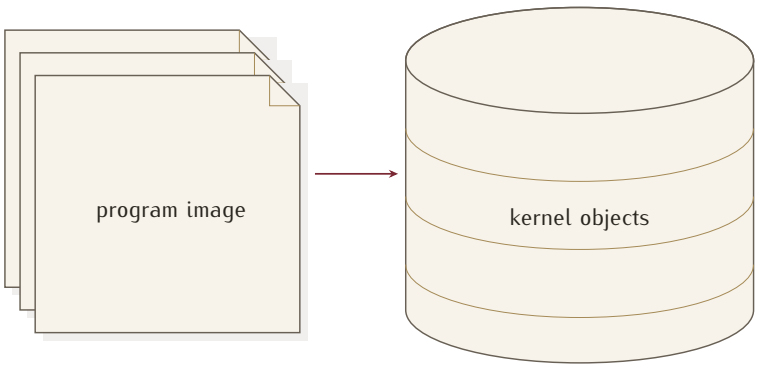
The saved PC is primarily resumption mechanism because it identifies where execution continues. Priority and accumulated CPU time are policy/accounting inputs: they may influence scheduling but are not restored as ordinary architectural registers. Open-file references are resource-management state and credentials are protection state; both participate in mechanisms whose use is governed by policy. The saved PC is the clearest example of state that must be restored for execution to resume, whereas priority is the clearest example of metadata that can change a scheduling choice without becoming part of the resumed user register state.

← Return to the exercise

5.2 PROCESS CREATION, PROGRAM REPLACEMENT, AND TERMINATION

The shell reads `producer > output.txt`. To carry out the command, it creates an execution, arranges for that execution's standard output to refer to the file, replaces its program image with `producer`, and optionally waits for completion. Unix exposes these steps through separate process-control primitives. Their order gives the shell a place to configure the child's environment before the new program begins [59, 1, 58, 32].

Program identity and process identity have different lifetimes



A process can replace its image while retaining selected kernel relationships.

Figure 5.11: *The document and stored-object symbols separate the program image from kernel-managed relationships. Fork, exec, exit and wait affect these relationships differently. The detailed lifecycle figures identify which identities and references survive each operation.*

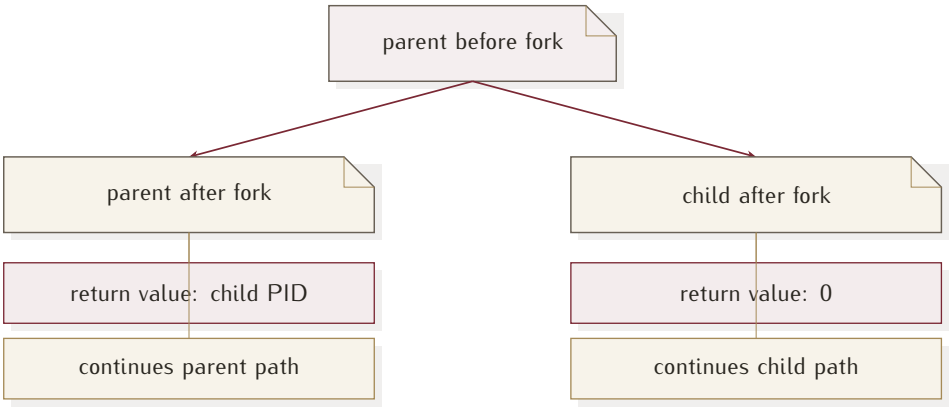
The central pattern is `fork` → setup → `exec` in the child, accompanied by `wait` or `waitpid` in the parent when lifecycle coordination is required. Understanding the pattern is easier if we follow one shell command from beginning to end, because the apparently strange fact that `fork` returns twice and `exec` does not return on success is precisely what gives the shell its control point.

5.2.1 Creation by duplication

When a process calls `fork` successfully, the kernel creates a child process whose initial user-visible state closely resembles that of the parent. Both executions resume after the same call, but the return value lets them distinguish their roles: the child receives zero, the parent receives the child's process identifier, and a failed call returns −1 only to the original caller because no child then exists.

```
pid_t pid = fork();
if (pid < 0) {
    /* creation failed: there is no child path */
} else if (pid == 0) {
    /* child path */
} else {
    /* parent path; pid identifies the child */
}
```

Fork creates a second execution history



Failure creates no child and returns an error to the parent.

Figure 5.12: *Successful fork produces two executions that continue after the call with different return values. Their later scheduling order is not determined by the diagram. Each process follows its own control path.*

For ordinary private mappings, the child begins with a logically separate virtual address space whose contents match the parent's at the instant of the fork. A write performed by the child to an ordinary global or heap object does not change the parent's object, and vice versa. Modern kernels can implement that contract efficiently with copy-on-write: parent and child initially map the same physical pages where sharing is safe, while page-table protection ensures that a later write triggers the creation of a private writable copy. Copy-on-write preserves private-mapping semantics while defer-

ring copying. Explicit shared mappings retain their sharing contract across fork. Special mapping attributes can also suppress inheritance or zero child pages; the examples below use ordinary private mappings [32].

If a variable x equals 10 before the call, both processes initially observe 10. Should the child later assign 20 to its own x, the parent can still observe 10 because the two virtual address spaces now contain different logical state. This remains true even if, for a brief interval, both virtual mappings referred to the same physical page underneath.

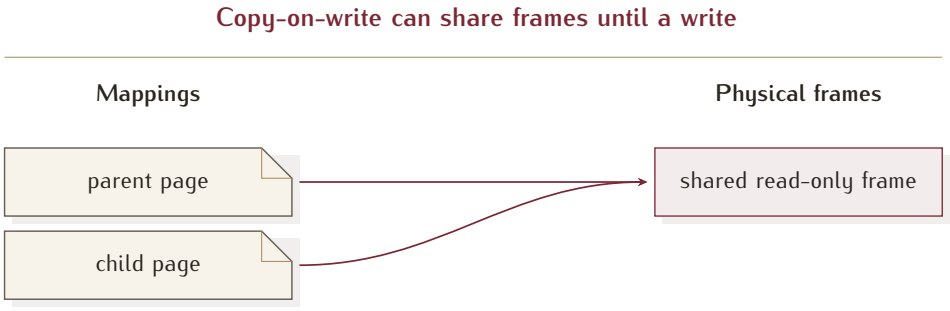


Figure 5.13: *Sharing the physical frame is compatible with distinct logical address spaces while the mappings enforce the copy-on-write protocol. A write can trigger a fault and create a private copy before the instruction is retried.*

Kernel-managed resources require a different description. After `fork`, the child receives its own descriptor table, but each inherited file descriptor initially refers to the same *open file description* as the corresponding descriptor in the parent. The distinction is observable. If both descriptors refer to a regular file, they share state associated with the same open file description, such as the current file offset and file-status flags; if they refer to a pipe, they refer to the same kernel pipe object. Yet the descriptor tables themselves are separate, so closing descriptor 1 in the child does not delete descriptor 1 from the parent's table, and redirecting the child's standard output changes only the child's descriptor mapping. Descriptor flags are per-descriptor state even though the underlying open file description is shared.

That combination—separate descriptor tables pointing to selected shared kernel objects—is exactly what a shell needs. It can fork, alter the child's descriptor table, and leave the parent's table untouched.

5.2.2 A shell command as a process-control case study

Suppose the shell has parsed

```
producer > output.txt
```

and has decided to run it as a foreground command. The following complete single-threaded POSIX program launches `./producer`, redirects its standard output to `output.txt`, reports its termination status and returns a corresponding status to its caller:

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
        if (fd < 0) {
            perror("open");
            _exit(126);
        }
        if (fd != STDOUT_FILENO) {
            if (dup2(fd, STDOUT_FILENO) < 0) {
                perror("dup2");
                _exit(126);
            }
            if (close(fd) < 0) {
                perror("close");
                _exit(126);
            }
        }
        execl("./producer", "producer", (char *)0);
        perror("execl");
        _exit(127);
    }
    int status;
    pid_t result;
    do {
        result = waitpid(pid, &status, 0);
    } while (result < 0 && errno == EINTR);
    if (result < 0) {
        perror("waitpid");
        return 1;
```

```
    }
    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        printf("child exited with status %d\n", code);
        return code;
    }
    if (WIFSIGNALED(status)) {
        int sig = WTERMSIG(status);
        printf("child terminated by signal %d\n", sig);
        return 128 + sig;
    }
    return 1;
}
```

Compile with `cc -std=c11 -Wall -Wextra launch.c -o launch`. The creation mode 0666 is filtered by the process umask. The `fd != STDOUT_FILENO` test matters when standard output was initially closed: `open` may already return descriptor 1, which must remain open. This example assumes the ordinary child-waiting contract and no other thread or signal handler reaping the child. Statuses 126 and 127 are chosen for setup and exec failures; a launched program can itself return those numbers. The $128 + \text{signal}$ convention translates a signal termination into a shell-style status. The wait interface supplies an encoded status word.

The ordering exposes the process-control mechanism. The child exists before it is running the requested executable, and this interval is useful: it still executes the shell's code, but it can reshape its own descriptor table so that descriptor 1 refers to the output file. Only after that setup does it replace its program image. The parent, meanwhile, keeps its original standard output and enough identity information to wait for the particular child it created.

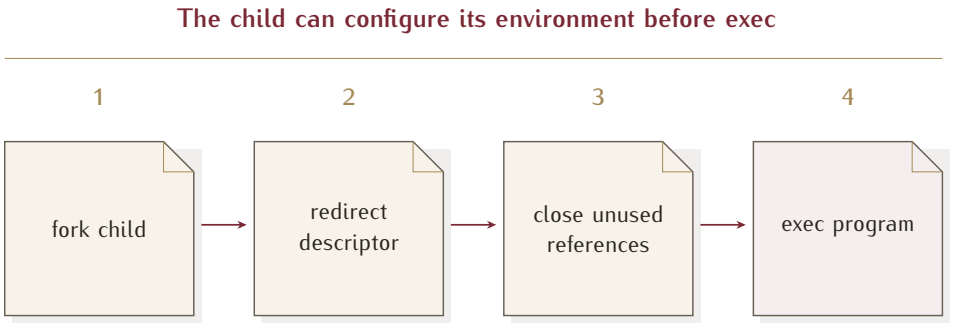


Figure 5.14: *The child initially executes the shell's code and can arrange the descriptors required by the command. Successful exec then replaces that child's program image while retaining the configured descriptors permitted by the interface.*

> **How separate process-control operations changed the shell**
>
> Ritchie's history describes early PDP-7 Unix commands replacing the shell's code, with termination loading a fresh shell. The process-control interface then evolved to separate creating an execution from replacing its program. This change fitted existing implementation mechanisms and enabled more flexible command execution [144].
> It also exposed a revealing consequence. A command that changes its own current directory cannot thereby change the parent shell's directory. Once commands ran in children, the shell needed to perform its own directory change internally. The familiar built-in `cd` thus illustrates which process owns the state being modified.
> Pipes extended composition by connecting a command's output to another command's input. The shell could arrange the connection using descriptors before starting the new program images. The sequence developed in this chapter therefore has a historical reason and an observable effect: creation provides a child to configure; program replacement preserves the configured context needed by the command.

Pipelines use the same idea. For

```
producer | consumer
```

the shell first creates a pipe, then creates children whose descriptor tables are rearranged so that the first child's standard output refers to the pipe's write end and the second child's standard input refers to the read end. Each child then executes its target program. The pipe already exists as a kernel object, and `fork` plus descriptor inheritance gives the

shell a controlled way to place references to that object in the children before their old program images disappear.

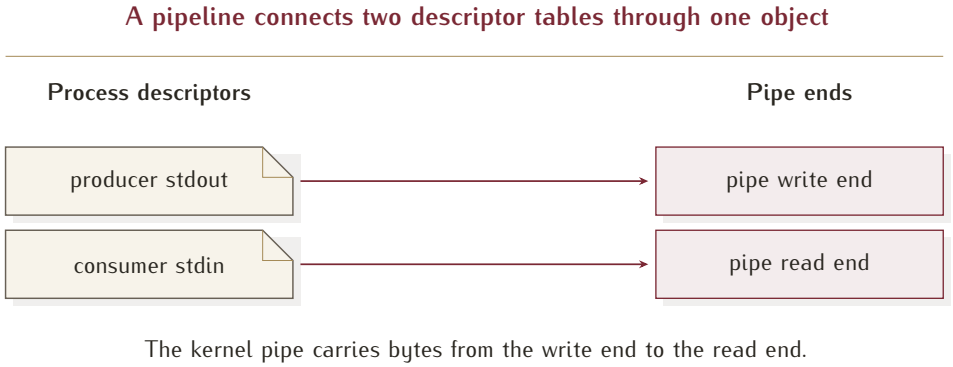


Figure 5.15: *The endpoints belong to the same pipe object, and each process has a descriptor referring to its endpoint. Unused references must be closed, especially extra write references that would delay end-of-file observation.*

This is also why descriptor lifetime matters. If every process accidentally keeps a pipe write end open, a reader may never observe end-of-file because the kernel still sees at least one live writer reference. Correct shell construction therefore closes descriptors that a process does not need, rather than merely arranging the two descriptors that it does need.

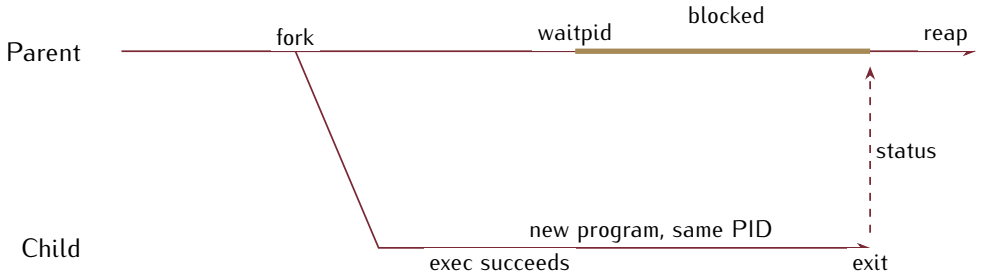


Figure 5.16: *One possible shell-style process trace. The child replaces its program with exec; the parent collects its termination status with waitpid. If the child has already terminated, the wait need not block.*

5.2.3 Program replacement without process replacement

An exec family call replaces the calling process's program image with a new executable. Code, writable data, heap and user stack are rebuilt according to the new image and its runtime environment; execution begins at the new program's entry path rather than returning to the instruction after exec. The process, however, has not been replaced by a new process in the lifecycle sense. Its process identifier remains the same, and a number of kernel relationships and attributes survive according to the interface contract.

This distinction explains a useful diagnostic question: if the shell forks a child with PID 4200 and that child successfully executes `producer`, what is the PID of `producer`? It is still 4200. `fork` created the process; `exec` changed what that process executes.

Open file descriptors normally survive successful program replacement unless the corresponding descriptor is marked close-on-exec. The close-on-exec mechanism is essential for controlling capability leakage across program boundaries. A descriptor may represent a file, socket, pipe or other kernel object that the new program should not inherit merely because the old program happened to have it open. Setting the close-on-exec descriptor flag tells the kernel to remove that descriptor as part of successful program replacement, which is often safer than relying on every future control path to remember a manual `close`.

The argument vector and environment provided to `exec` become part of the new program's initial user-space state. By contrast, ordinary variables from the previous program image cease to exist as such. The old heap is not a hidden storage area that the new executable can inspect; if data must survive across `exec`, it must be carried through an interface that the process retains or the new program is explicitly given, such as arguments, environment variables, inherited descriptors or external storage. Existing memory mappings do not survive exec. A shared-memory object can survive under its own lifetime rules, but the new program must obtain a reference and establish a new mapping [32].

A successful `exec` therefore has no normal return path. Code such as

```
execl("/bin/ls", "ls", "-l", (char *)0);
/* here only if exec failed */
_exit(127);
```

should be read literally: reaching the next statement means that the program replacement did not happen.

An exec failure normally returns -1 with `errno` set. On Linux, rare resource-exhaustion failures after the old image has already been dismantled can instead terminate the process; the caller cannot assume that every unsuccessful attempt returns [32].

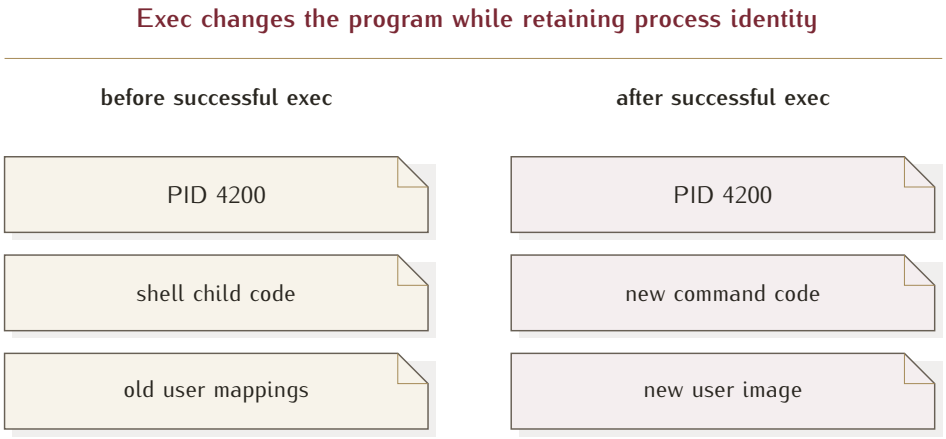


Figure 5.17: *The same process identifier belongs to a new program image after successful replacement. The old code cannot continue through a normal return from the successful call. An ordinary error return leaves the caller able to handle the failure.*

The use of `_exit` in this post-fork child path is deliberate. The child inherited user-space library state from the parent, including potentially buffered I/O. Calling the ordinary `exit` path after a failed `exec` can run cleanup logic or flush inherited buffers in ways the parent did not intend to duplicate. A low-level child that is about to become another program generally keeps the interval between `fork` and `exec` small and carefully controlled.

Creation also places the child in a relationship graph that matters to shells and supervisors. Each ordinary process has a parent identifier recording the process that created or adopted it, so repeated creation forms a process tree rather than a flat set of unrelated PIDs. The relationship is useful for lifecycle management but should not be mistaken for ownership of all resources: after `fork`, parent and child are separate processes, and either can continue or terminate independently subject to the operating system's rules. If the original parent exits while the child remains alive, the child is reparented to a designated reaper rather than being killed merely because one edge of the original tree disappeared.

Shells add two further groupings because a pipeline is often better treated as one job than as several unrelated children. A *process group* can collect the processes that belong to one pipeline or job, allowing signals to be directed to the group. A *session* groups process groups and participates in terminal/job-control rules. The exact shell protocol involves terminal foreground ownership and signals beyond the minimum process-creation model used here, but the hierarchy explains why a pipeline such as `producer | consumer` creates more structure than two children with a pipe between them: the shell must be able to stop, continue, foreground or terminate the job coherently even though the job contains several PIDs.

This also clarifies why PID identity and executable identity are separate. A child can remain in the same process group and keep the same PID while `exec` changes every ordinary instruction it will execute next. Conversely, a parent and child can run the same executable while belonging to different lifecycle roles. Process identity is a kernel relationship; program identity is part of the process image.

5.2.4 Waiting, termination and the final process record

Creation and replacement solve only the first half of the lifecycle. The kernel must also represent the fact that a child has terminated, and a parent often needs to know how it terminated. A process can finish voluntarily with an exit status, terminate because of a fatal signal or exception translated into a signal, or be terminated by an authorised external action. When the process terminates, its execution stops and its resource references are released; objects still referenced elsewhere can remain alive. Under the ordinary waiting contract, termination information remains for a parent or designated reaper to collect.

Under that contract, a terminated but uncollected child becomes a *zombie*. Explicitly ignoring `SIGCHLD` or setting `SA_NOCLDWAIT` can request automatic disposal on POSIX systems; the examples below assume neither [32]. The term does not describe a process that is still secretly running. A zombie consumes no ordinary CPU execution time and its address space has been released; what remains is the small amount of kernel bookkeeping required to preserve identity and termination status until the waiting protocol is completed. When the parent performs the appropriate `wait` or `waitpid`, it receives the status and the kernel can release the final process-table record.

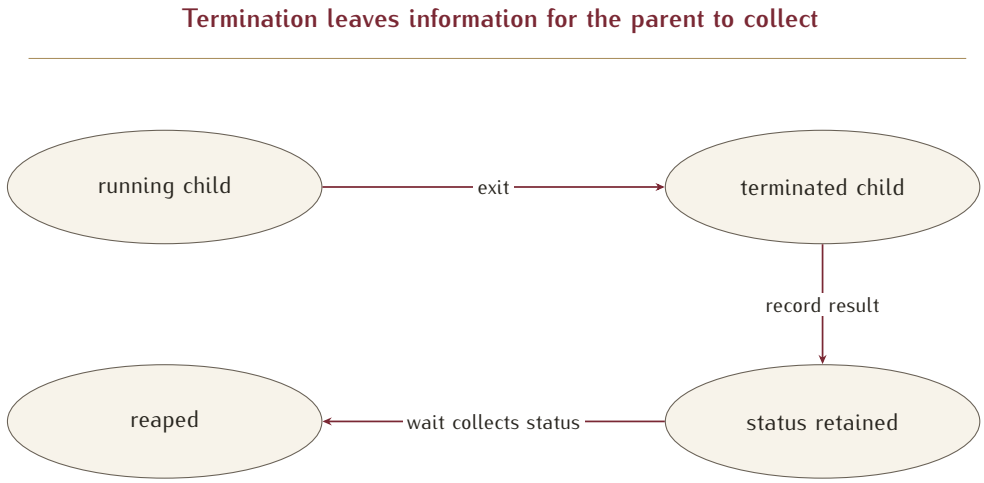


Figure 5.18: *After execution ends, the kernel can retain identity and termination information until the parent collects it. Reaping releases the retained process record under the operating system's lifecycle rules.*

An *orphan* is different. It is a process that is still alive after its original parent has terminated. Unix-like systems arrange for such a process to become the responsibility of a designated reaper so that, when the orphan eventually exits, its termination state can still be collected. A process can therefore be an orphan while actively running, whereas a zombie is already dead but not yet reaped.

Waiting also provides ordering. A foreground shell that waits for its child will normally refrain from accepting the next foreground command until that child changes into a state the shell is interested in. A supervising program may instead use `waitpid` selectively, observe stop/continue events with appropriate options, or reap many children over time. The general principle is that process termination and status collection are separate events: exit ends execution, while waiting consumes the retained lifecycle information.

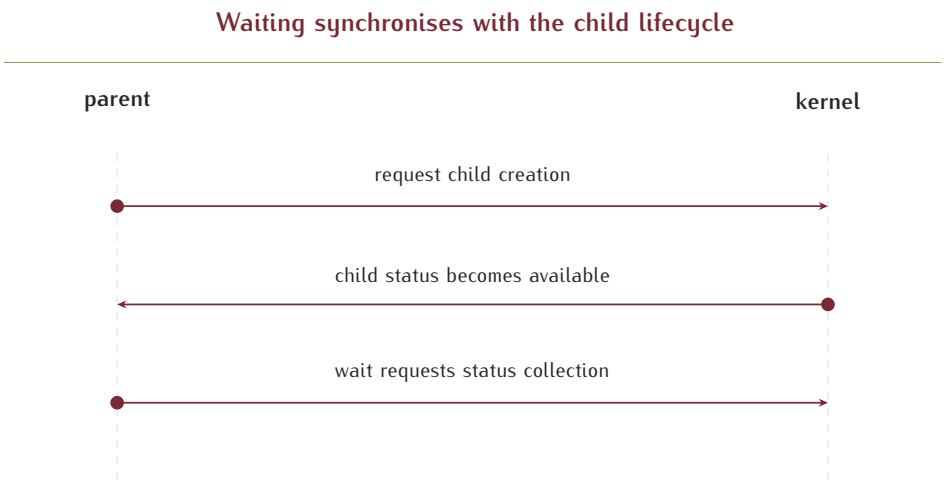


Figure 5.19: *A wait operation can block until a child reaches a reportable state or return immediately if that state is already recorded. It retrieves lifecycle information and performs the specified collection step.*

The status value returned through the waiting interface encodes how the child changed state. Portable code uses the interface's status-testing macros or equivalent mechanisms to distinguish normal exit from signal termination and other reportable events. Treating the raw status word as though it were always an ordinary exit code loses that distinction.

5.2.5 The multithreaded fork boundary

The simple shell story is easiest to reason about when the calling process has one thread. In a multithreaded process, `fork` requires greater care because the child is created from the calling thread's point of view while memory copied from the parent may describe synchronisation objects that were being used by threads that do not continue in the child. A mutex, allocator or library subsystem can therefore appear locked even though the thread that held the lock no longer exists in the child. Only the calling thread survives in the child. POSIX restricts that child to async-signal-safe operations until successful exec. At-fork handlers can coordinate specific library state, but do not make arbitrary child-side library calls safe. The earlier example explicitly assumes a single-threaded parent; a multithreaded launcher must use a path respecting these restrictions, often through `posix_spawn` [4, 58].

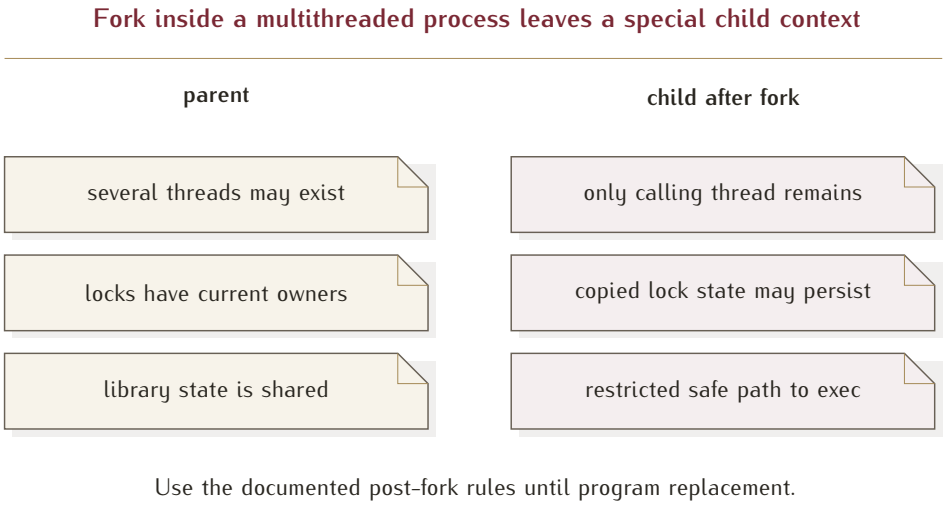


Figure 5.20: *A lock copied from another parent thread can appear held in a child where that owner thread does not exist. The safe child-side operation set and any at-fork handling must follow the platform's specified rules.*

This caveat also reinforces the conceptual separation developed in the previous chapter. A process contains resources and an address space; threads are execution streams within that container. Duplicating a process whose container holds the traces of several concurrent threads is not equivalent to duplicating every running thread. The thread model will be developed explicitly in Chapter 5.4, after scheduling has established what it means for an execution to be runnable.

Unix process control can now be read as a sequence of transformations rather than a list of unrelated calls. `fork` creates a new protected execution and a new process identity; descriptor inheritance gives parent and child references to selected common kernel objects; child-side setup reshapes the child's interface to those objects; `exec` replaces the child's program image without changing that identity; termination stops execution; and `wait/waitpid` closes the parent–child lifecycle by consuming the retained status. The next problem is policy rather than creation: once several processes exist and more than one is ready, the kernel needs a principled way to decide who should run next.

Concept check

1. Which state after `fork` is logically private, and which kernel state can still be referred to through shared open file descriptions?
2. Why does a shell perform descriptor redirection after `fork` but before `exec`?
3. Which identity property survives a successful `exec`, and why does successful program replacement have no normal return path?
4. Why are zombie and orphan different lifecycle states, and what role does `waitpid` play in reclaiming the final child record?

Solutions to the review questions

1. Parent and child have logically separate address spaces after `fork`; copy-on-write can initially share physical pages while preserving that separation. Their descriptor tables are separate, but inherited descriptors can refer to the same open file description and therefore share its file offset and status flags.
2. Redirection in the child changes the descriptors the new program will inherit while preserving the shell parent's own descriptors. Performing it before `exec` establishes the intended standard input/output for the replacement image.
3. A successful `exec` preserves the process identity, including its PID, while replacing the executable image and related program state. Execution begins in the new image; the old call site and its ordinary continuation no longer define a return path. An ordinary exec error returns to the old program with an error indication.
4. A zombie has terminated but retains the status record needed for its parent or reaper to collect it. An orphan has lost its original parent and may still be running. A successful `waitpid` for a terminated child retrieves its status and permits reclamation of the retained record.

5.2.6 Worked problems

Conditional fork trace

One process executes `pid=fork(); if (pid>0) fork();`. Assume both calls succeed. The first call returns zero to child C1, which skips the conditional body. The original process P receives C1's positive PID and makes the second call, creating C2. P, C1 and C2 all reach the following statement. Only P executed both calls.

Private variables and a shared open file description

Before fork, $x = 9$ and a regular file's offset is 100. The child assigns a new value to its private x, reads exactly 20 bytes using the inherited descriptor, and finishes before the parent inspects the state. The parent still reads 9 from its own private object. Both descriptor entries refer to the same open file description, so its file offset is now $100 + 20 = 120$. Closing the child's descriptor removes one reference to that object. The parent can continue reading from offset 120.

Exec replacement and error control flow

A child redirects standard output, marks a separate descriptor close-on-exec, and calls `execve`. On success the same PID runs the new image with its new stack. Descriptor 1 still refers to the output file, and the marked descriptor closes during replacement. On an ordinary error return, the child still executes the old program and can report the failure before calling `_exit`. Linux also specifies rare late failures that terminate the process after dismantling the old image.

Zombie, orphan and reaping

Assume ordinary child-status collection. C exits while its parent P is alive. Until P waits, C has a zombie record containing its identity and termination status. If P exits first without collecting that record, the kernel reparents it to a designated reaper, which can collect the status. A living child can likewise be reparented after its parent exits and continue running. Termination and reparenting describe separate events.

Exercise 5.2.1 — Fork failure

What child path exists when `fork` returns -1?

Worked solution

A return of -1 indicates that process creation failed. No new child process exists, so there is no child execution path to analyze. The original process must handle the error and inspect the API's error indication as appropriate. In the successful case, zero identifies the child's return path and a positive child PID identifies the parent's path. Treating -1 as a child PID would create incorrect control flow.

Exercise 5.2.2 — Copy-on-write

Does sharing a physical page before the first write make a normal global variable logically shared?

Worked solution

Copy-on-write can temporarily map the parent's and child's private virtual pages to the same physical frame. The logical address spaces remain separate. A write triggers the mechanism that gives the writer its own copy where required, so an ordinary global variable retains private-process semantics. Shared-memory objects

Worked solution (continued)

use a distinct sharing contract. Physical sharing before a write is an implementation technique and does not by itself authorize cross-process shared updates.

Exercise 5.2.3 — Exec identity

Name one property normally preserved and one region replaced by successful exec.

Worked solution

A successful `exec` preserves the process identity, including its PID, while replacing its user program image with the new executable and its associated mappings and initial user stack. Thus the same process begins executing different code. Inherited file descriptors follow rules such as close-on-exec, so preservation must be checked property by property. Successful exec transfers control into the new image and does not return to the old instruction stream.

Exercise 5.2.4 — Reaping

Which interface can wait for one specified child and return its status?

Worked solution

Use `waitpid(child_pid, &status, options)` with the desired child's PID. With ordinary blocking options, the call waits until that child reaches a reportable state and returns its status information. The parent must check the return value for errors and interpret the status using the specified macros. Collecting termination status also completes reaping of the child's retained termination information.

Exercise 5.2.5 — Shell order

Place these actions in order: child descriptor setup, fork, successful exec, parent status collection.

Worked solution

First create the child with fork. In the child's path, establish the descriptors needed by the new program, then execute the replacement image with exec. The parent later collects the child's status with a wait operation. Descriptor setup precedes exec so the new program inherits the intended streams. Parent and child may run concurrently after fork. The required order comes from dependencies between actions.

Summary

The Unix process-control pattern separates creation, program replacement and status collection. `fork` creates a child, `exec` replaces one process image, and `wait` completes the parent–child lifecycle. Correct code follows both post-fork paths and handles every failure path.

5.2.7 Exercises

Exercise 5.2.6 — Fork return values

State the return value seen by parent and child after a successful fork.

Go to the solution →

Exercise 5.2.7 — Address-space independence

After fork, parent and child each increment the same global variable once. Explain why the final value is not a shared total.

Go to the solution →

Exercise 5.2.8 — Exec identity

Which changes and which remains: executable code, PID, user stack, process identity?

Go to the solution →

Exercise 5.2.9 — Zombie diagnosis

Under the ordinary waiting contract (no automatic child disposal), a child exits and the parent remains alive but never waits. Name the state and the missing action.

Go to the solution →

Exercise 5.2.10 — Orphan diagnosis

A parent exits while the child continues. Explain what the system must do with the parent relationship.

Go to the solution →

Exercise 5.2.11 — Shell pattern

A shell must run `worker >out.txt` and then report the child's exit status. Give the ordered parent and child actions from `fork` through collection, including which side performs the descriptor redirection, what happens if `exec` fails, and why the parent must not perform the redirection in its own descriptor table.

Go to the solution →

5.2.8 Solutions

Process Creation, Program Replacement, and Termination

Solution 5.2.6

The parent receives the positive child PID. The child receives zero. A negative return denotes failure in the caller and no child path exists.

← Return to the exercise

Solution 5.2.7

Let the global start at 9 in an ordinary private mapping. Immediately after fork each process sees 9. The parent's increment changes its own copy to 10. The child's increment also changes its own copy to 10. Each reads 10 regardless of which increment ran first. Physical page sharing through copy-on-write preserves these independent logical values. Accumulating a shared total would require an explicitly shared object and a suitable update protocol.

← Return to the exercise

Solution 5.2.8 - Exec identity

Code, data and user stack are replaced. The process PID and basic process identity remain, subject to documented attribute changes.

← Return to the exercise

Solution 5.2.9

The child is a zombie. The parent must call wait or waitpid to collect the termination status.

← Return to the exercise

Solution 5.2.10

The live child is reparented to a designated reaper so that a responsible process can eventually collect its status.

← Return to the exercise

Solution 5.2.11

The shell calls `fork` and handles a negative return as a creation failure. The child opens `out.txt` for writing with create/truncate flags and checks the result. If the returned descriptor differs from standard output, it uses `dup2` to install descriptor 1 and closes the original descriptor; if it already equals 1, it leaves it open. It then calls `exec` to replace its image with `worker`. If `exec` fails, the child reports

Worked solution (continued)

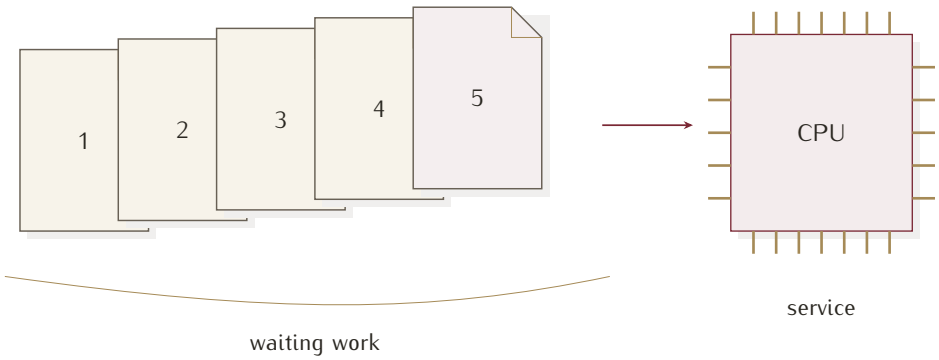
the failure and terminates with `_exit` rather than falling back into shell control flow. The parent keeps its own standard output unchanged, closes descriptors it no longer needs, and calls `waitpid` for that PID, retries only an interrupted wait (`EINTR`), handles other errors, and decodes success with `WIFEXITED/WEXITSTATUS` or signal termination with `WIFSIGNALED/WTERMSIG`. Redirection belongs in the child because after `fork` the two processes have separate descriptor tables; changing the parent's table would redirect the shell itself.

← Return to the exercise

5.3 CPU SCHEDULING AND TIME SHARING

Three executions are runnable: an interactive request, a long calculation and a background job. One logical processor can run only one of them at a time. Choosing the next execution affects response time, completed work, fairness and the cost of switching. A scheduling policy makes those trade-offs explicit; a timeline lets us calculate the consequences for a particular workload [57, 59].

Time sharing chooses who receives the next interval



The order is chosen by policy and changes when work arrives, waits or finishes.

Figure 5.21: *The queue gives a physical picture of waiting for service. The timeline diagrams later assign exact arrival, burst and quantum assumptions, making response and waiting-time differences calculable.*

A process rarely consumes CPU continuously from creation to termination. More commonly it alternates between *CPU bursts*, in which instructions execute, and periods in which it is blocked waiting for I/O, a timer, a lock, a message or another event. A CPU-bound workload tends to have long CPU bursts and relatively infrequent blocking; an I/O-bound or interactive workload often runs briefly, issues or completes some interaction, and blocks again. A policy must account for this behaviour because a policy that keeps a long CPU burst on the processor can delay many short bursts that would quickly block again and allow devices or users to make progress.

5.3.1 Events that trigger scheduling

Consider three processes observed on one logical processor. P_A arrives at time 0 and needs 7 units of CPU. P_B arrives at time 1 and needs 3 units; P_C arrives at time 2 and needs 1 unit. For this workload, each process terminates after that one CPU burst and switch costs are ignored. Even before naming an algorithm, the timeline forces us to distinguish several moments: arrival, first dispatch, intervals spent ready but not running, completion of each CPU burst, blocking, later wake-up, and final completion. Scheduling metrics are summaries of these events.

For a process i, turnaround time is

$$T_i^{turn} = T_i^{completion} - T_i^{arrival}.$$

Response time measures the delay until the process first receives service,

$$T_i^{response} = T_i^{first\ dispatch} - T_i^{arrival}.$$

Waiting time is the time spent runnable but not executing. In the simplified single-burst model used in many exercises, where the interval between arrival and completion contains only ready-queue waiting and CPU service,

$$T_i^{wait} = T_i^{turn} - T_i^{CPU}.$$

That last identity is not a universal definition. If the process blocks for I/O or sleeps during its lifetime, turnaround contains time that is neither CPU service nor ready-queue waiting, so waiting must instead be accumulated over the actual runnable intervals.

The distinction between response and turnaround is particularly important in interactive systems. A process can respond quickly, run for a short time and then remain incomplete for much longer; conversely, a batch job can have a respectable turnaround while still waiting a long time before its first instruction executes. Throughput counts completed work per unit time, processor utilisation measures the non-idle fraction of available capacity (which may include scheduling overhead), and fairness concerns how service is distributed. Improving one metric can worsen another.

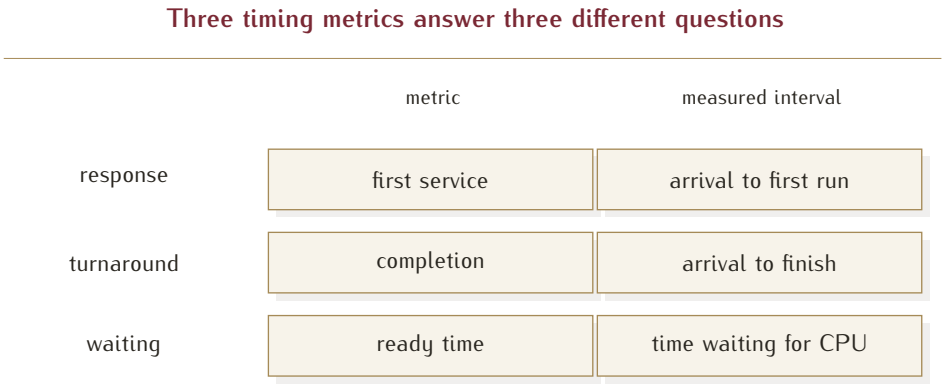


Figure 5.22: *Response measures how soon execution first receives service, while turnaround covers the entire lifecycle interval. Waiting accumulates time eligible for the CPU but not running. A policy can improve one metric while worsening another.*

The scheduler is also not invoked only when a timer expires. A scheduling decision becomes relevant whenever the set of runnable activities or the status of the current one changes: a process may block, exit, exhaust a quantum, yield voluntarily, or be joined by another process that has just arrived or become ready after an interrupt. In a non-preemptive policy, the current process continues until it blocks, exits or yields. A preemptive policy allows the kernel to stop a still-runnable process so that another may execute, which is what makes bounded time sharing possible.

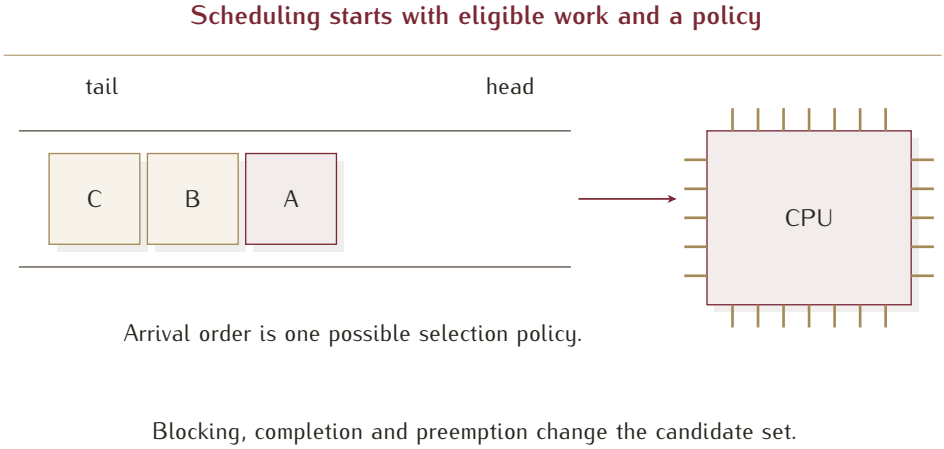


Figure 5.23: *A scheduler chooses among ready executions when a scheduling event requires a decision. The decision can use age, priority, estimated service or other policy state. An execution blocked on a resource is not eligible merely because it has high priority.*

The difference between ready waiting and blocking is easiest to see in a trace that contains I/O. Suppose a process arrives at time 0, executes for 2 units, blocks for a 5-unit device operation, becomes ready immediately when the device completes, executes for one final unit and terminates at time 8. Its turnaround is 8 and its total CPU service is 3, yet its ready-queue waiting can still be zero. Subtracting CPU service from turnaround gives 5 only because that subtraction has accidentally counted the blocked interval. In a realistic trace, a scheduler analysis should therefore mark state transitions first and accumulate waiting only over intervals in which the process is ready but not running. Formulas are shortcuts once the model has been stated; they are not substitutes for the state model.

During those five blocked units another ready task can use the processor. The device wait then overlaps useful CPU work, while the blocked task's own service remains three units.

5.3.2 From FCFS to shortest-first: what objective are we choosing?

First-come, first-served (FCFS) uses perhaps the simplest rule: runnable jobs wait in arrival order, and in the usual non-preemptive form the selected process keeps the processor until its current burst ends. The rule is easy to implement and easy to explain, but simplicity does not imply good latency. If the first process has a long CPU burst while several short jobs queue behind it, all of the short jobs inherit the long job's delay. This is the *convoy effect*.

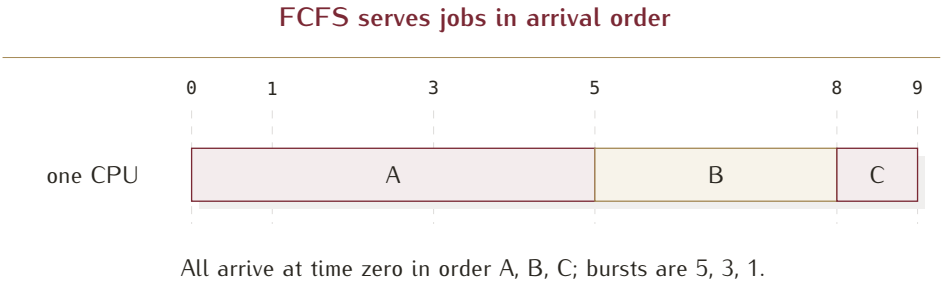


Figure 5.24: *With these assumptions, waiting times are zero for A, five for B and eight for C. The mean waiting time is 13/3 time units. FCFS grants service in arrival order.*

Suppose three processes arrive at times 0, 1 and 2 with CPU bursts 8, 4 and 1. FCFS executes P_1 from 0–8, P_2 from 8–12 and P_3 from 12–13. Their waiting times are 0, 7 and 10; their turnaround times are 8, 11 and 11. The one-unit job spends ten units waiting even though the machine could complete it in one. Nothing is "incorrect" about the schedule: FCFS has simply optimised neither average waiting nor responsiveness.

If the scheduler knew the lengths of the next CPU bursts, it could reduce average waiting by serving shorter work first. Shortest-job-first (SJF) chooses the smallest available next burst. Under the classic assumptions that a fixed set of jobs is ready together,

burst lengths are known and switching cost is ignored, SJF minimises average waiting time. The preemptive version, shortest-remaining-time-first (SRTF), extends the idea by allowing a newly arrived job with less remaining work to preempt the current one.

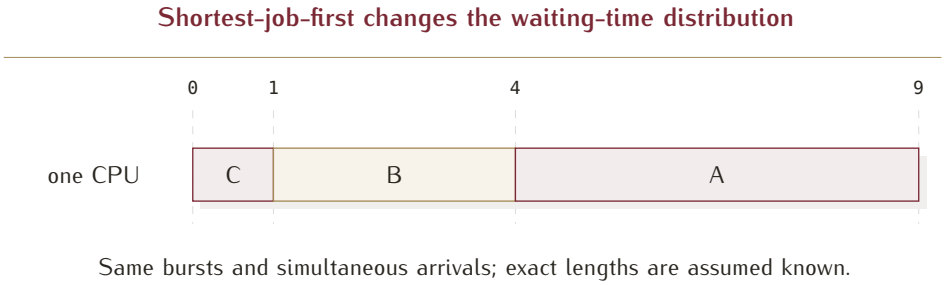


Figure 5.25: *The waiting times become zero for C, one for B and four for A, giving a mean of 5/3. The advantage depends on the stated known-length, nonpreemptive model; a real scheduler usually estimates future service.*

The assumption that makes these policies mathematically attractive is also what makes them difficult to implement literally: a general-purpose operating system does not know the future duration of each CPU burst. It can estimate. One simple predictor uses exponential averaging,

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n,$$

where $0 \leqslant \alpha \leqslant 1$, t_n is the last measured burst and τ_n the previous estimate. A larger α reacts more strongly to recent behaviour; a smaller value smooths short-term variation. The scheduler can use the estimate when choosing the next ready job.

Shortest-first policies also expose a fairness problem. If short jobs continue arriving, a long job can remain postponed even though the processor is continually busy and the scheduling rule is behaving exactly as designed. This is *starvation*: system-wide progress continues while one eligible activity fails to receive sufficient service. A policy can therefore be excellent for an average metric and still be unacceptable when bounded progress matters.

5.3.3 Round-robin and the price of preemption

Time sharing addresses a different goal. Round-robin (RR) keeps runnable processes in a queue and gives each at most a time quantum q. A process that blocks or terminates before using the quantum leaves the processor early; one that remains runnable when the quantum expires is preempted and normally returns to the tail of the queue.

For three processes that all arrive at time 0 with bursts 5, 3 and 1 and a quantum of 2, a possible schedule is

$$P_1(0\text{–}2),\ P_2(2\text{–}4),\ P_3(4\text{–}5),\ P_1(5\text{–}7),\ P_2(7\text{–}8),\ P_1(8\text{–}9).$$

The completion times are 9, 8 and 5, while first-response times are 0, 2 and 4. With exactly these three processes ready from the start, no one waits more than two complete quanta before its first dispatch; that observation depends on the queue length and workload and should not be promoted into a universal RR bound.

The quantum exposes a direct connection between scheduling policy and the context-switch costs developed in Chapter 5.1. If q becomes very large, RR approaches FCFS because each process is likely to complete a burst before preemption. If q becomes very small, service is distributed more finely, but the processor crosses scheduling boundaries more often and locality may suffer. A real choice of quantum must therefore compare human or application response requirements with direct switching overhead and indirect cache/TLB disruption. There is no context-free "best" number.

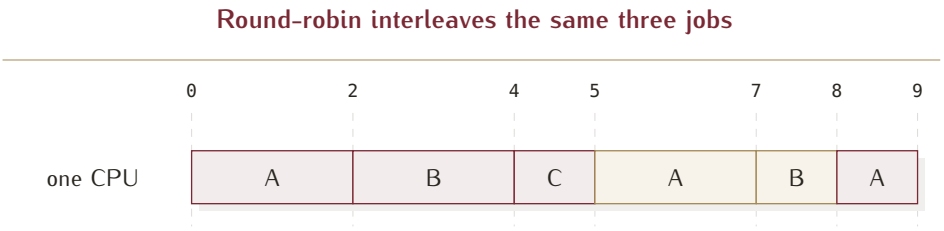


Figure 5.26: *The head job runs for at most two units before an unfinished job rejoins the queue. Completion times are five for C, eight for B and nine for A. The schedule makes the initial response of short waiting jobs visible.*

RR also illustrates why fairness is not identical to equality. Giving every runnable process an equal quantum in a simple queue is one form of proportional service, but a server handling latency-sensitive requests, a background compiler and a real-time control task may have different service objectives. General-purpose schedulers consequently combine mechanisms rather than implementing textbook RR as their entire policy.

A single workload makes the policy differences more concrete. Let P_A, P_B and P_C arrive at times 0, 1 and 2 with CPU bursts 7, 3 and 1, and suppose there is no I/O or switch cost. Under non-preemptive FCFS, P_A occupies the processor from 0 to 7, followed by P_B from 7 to 10 and P_C from 10 to 11. Waiting times are therefore 0, 6 and 8, while response times are identical because no process is preempted before its first service. The average waiting time is 14/3, and the one-unit job waits eight units before it can begin.

SRTF sees the same arrivals differently. P_A runs from 0 to 1; when P_B arrives with a three-unit burst, it has less remaining work than P_A and preempts it. P_B runs from 1 to 2, at which point the one-unit P_C arrives and becomes the shortest remaining job. P_C completes from 2 to 3, P_B finishes from 3 to 5, and P_A resumes from 5 to 11. Turnaround times are 11, 4 and 1; subtracting CPU service in this single-burst model gives waiting times 4, 1 and 0. Average waiting has fallen to 5/3, and every process received its first

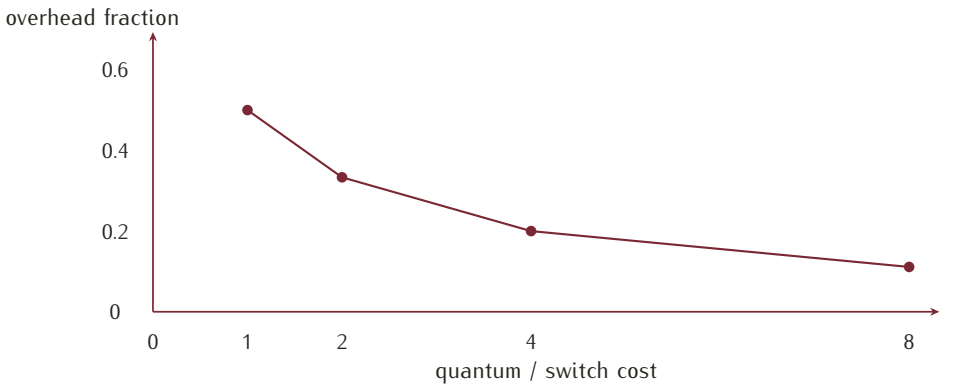
service immediately on arrival. The improvement follows from using remaining-work information that FCFS does not use.

Round-robin with $q = 2$ pursues a different objective. To fix the simultaneous-event convention, enqueue a process arriving at a quantum boundary before re-enqueuing the process whose quantum has just expired. Under this convention, the sequence is

$$P_A(0–2),\ P_B(2–4),\ P_C(4–5),\ P_A(5–7),\ P_B(7–8),\ P_A(8–10),\ P_A(10–11).$$

The completion times become 11, 8 and 5, so turnaround is 11, 7 and 3; waiting is 4, 4 and 2. Average waiting, 10/3, is worse than SRTF for this workload, but first response occurs at delays 0, 1 and 2 rather than forcing the later arrivals to wait behind the entire seven-unit burst. RR provides recurring service without predicting future burst length, at the cost of higher average waiting for this workload.

A shorter quantum increases the relative switch overhead



Simplified fraction: switch cost / (quantum + switch cost).

Figure 5.27: *The model assumes every quantum is fully used and followed by one fixed-cost switch. It isolates the overhead trade-off; real workloads include early blocking, varying switch costs and scheduler details.*

The comparison illustrates a general method for scheduling exercises. Use the same arrival/burst data, draw the exact execution timeline under each policy, derive completion and first-dispatch events from that timeline, and only then calculate metrics. Comparing policies by slogans such as "RR is fair" or "SJF is optimal" without fixing the workload and metric conceals the assumptions that make those statements meaningful.

5.3.4 Priority, feedback and inversion

Priority scheduling lets policy express that some runnable activities should be considered before others. A priority can be static, derived from administrative policy, or

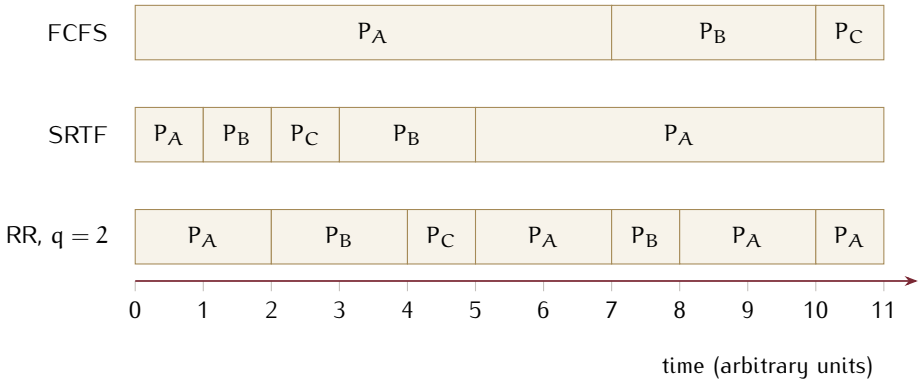


Figure 5.28: *The same workload under three policies: arrivals* $(0, 1, 2)$ *and bursts* $(7, 3, 1)$ *for* P_A, P_B, P_C*. No I/O or switch cost. For RR, an arrival at a quantum boundary joins before the expired process is re-enqueued. Consecutive* P_A *slices at the end share the same running process across a quantum boundary.*

dynamically adjusted according to behaviour such as recent CPU usage, sleep/wake patterns or waiting time. Multilevel designs separate scheduling classes into several queues. Multilevel-feedback designs additionally move activities between queues according to observed behaviour.

The danger is again starvation. If a continuous stream of higher-priority work exists, a lower-priority process can remain ready indefinitely. *Aging* addresses the problem by increasing the effective priority of a process that has waited for a long time, turning time spent without service into evidence that the policy should eventually favour it.

Priority also interacts with synchronisation. Suppose high-priority thread H requires a mutex held by low-priority thread L. If a medium-priority thread M is runnable, M may repeatedly preempt L, even though allowing L to run would release the mutex and let H continue. H is then indirectly delayed by M through a resource held by L: this is *priority inversion*. Protocols such as priority inheritance can temporarily raise the effective priority of the lock holder so that it can complete the critical section. Priority inversion shows why scheduling analysis must include the resource dependencies that determine which thread can make progress.

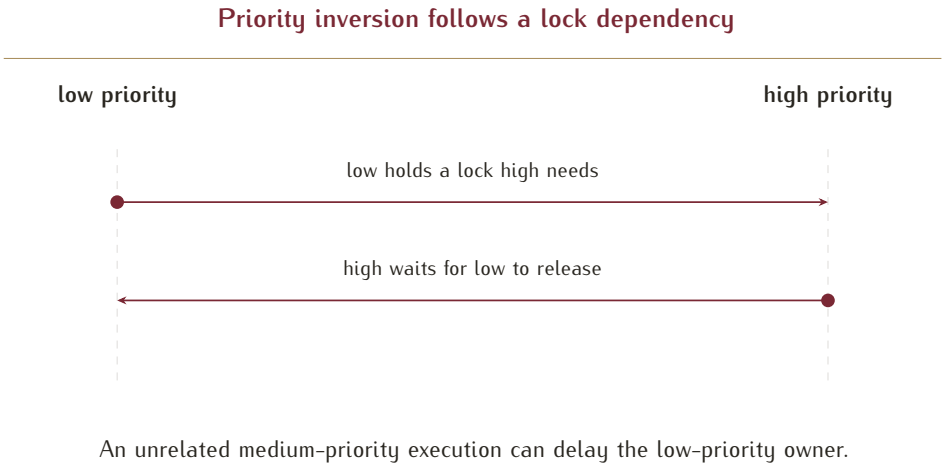


Figure 5.29: *The high-priority execution depends on progress by a lower-priority lock owner. Without an appropriate protocol, other work can delay that owner. Priority inheritance can temporarily raise the owner's priority under the mutex protocol.*

Some workloads introduce stronger requirements than ordinary interactive fairness. A real-time task may care about completing before a deadline rather than about average waiting time; a throughput-oriented batch system may willingly accept slower response if doing so reduces overhead. These objectives require explicit assumptions about admission, deadlines, periodicity or utilisation. It would therefore be misleading to call one general-purpose algorithm "optimal" without first naming the metric and workload model against which optimality is claimed.

Multilevel-feedback scheduling provides a useful conceptual bridge between the simple textbook policies and practical general-purpose behaviour. Instead of assuming that the next burst length is known, the scheduler can infer something from observed use. A thread that repeatedly consumes an entire quantum behaves differently from one that runs briefly and blocks for interaction; moving such executions among queues with different priorities or quanta lets policy respond to history without predicting an exact future burst. The design is not free: feedback rules can be gamed, priority changes need fairness safeguards, and the chosen parameters encode assumptions about what "interactive" or "CPU-intensive" behaviour should receive. The important idea is that past execution can become policy state.

5.3.5 Scheduling on several logical processors

With several schedulable hardware contexts, the kernel selects both the runnable thread and the processor on which it will run. A conceptually simple design could maintain one global ready queue, but shared queue manipulation can itself become a scalability bottleneck. Practical multiprocessor schedulers often use per-processor run queues or

related partitioning, together with mechanisms that rebalance work when some logical processors are overloaded and others are idle.

Placement has performance consequences because processors are not interchangeable in a purely temporal sense. A thread that recently ran on one core may benefit from cache contents associated with that core; moving it can lose locality. *Processor affinity* therefore expresses a preference or constraint that keeps an execution near selected hardware contexts. Affinity can improve locality, but overly rigid placement can leave one processor busy while another is idle, so load balancing and affinity pull in opposite directions. Systems may also have non-uniform memory access, heterogeneous cores or SMT siblings that share some physical resources, which further complicates the apparently simple question "where should this runnable thread execute?".

A multicore scheduler chooses among runnable threads, checks their placement constraints and accounts for the resources they would share with current work. Moving a thread to an idle core can reduce queueing yet increase memory-access time if its working set remains near the old core. The dispatcher installs the chosen thread's execution context.

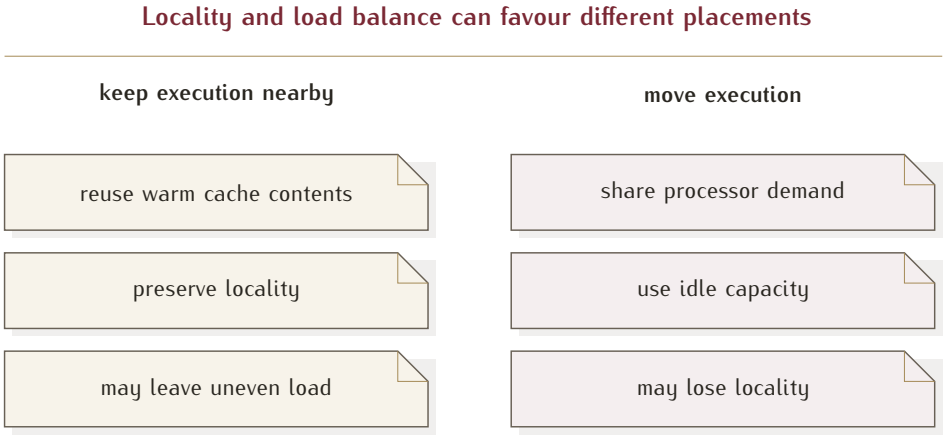


Figure 5.30: *Moving an execution can reduce queueing while increasing memory and cache costs. Multicore scheduling therefore considers more than the number of ready jobs on each processor.*

Scheduling now gives us a way to share processors among processes, but the process abstraction is sometimes too strong a boundary for cooperating work. Two activities inside one application may want to share data continuously without serialising it through pipes or other explicit IPC, while still retaining separate stacks and instruction streams that the scheduler can run independently. That motivates the next abstraction: the thread.

Concept check

1. Why is $T^{wait} = T^{turn} - T^{CPU}$ valid only under a restricted scheduling model?
2. Under which assumptions does SJF minimise average waiting, and why is prediction the practical difficulty?
3. What two competing effects appear as a round-robin quantum becomes very small?
4. How can affinity improve locality while making load balancing harder, and why is priority inversion partly a resource-allocation problem rather than only a scheduling problem?

Solutions to the review questions

1. In a model containing only CPU service and ready-queue waiting, turnaround is their sum and subtraction yields waiting time. With blocking, turnaround also contains blocked intervals; those must be removed separately. Dispatch overhead must likewise be assigned consistently before using the equation.
2. For non-preemptive jobs all ready initially on one processor, with known CPU bursts and no other blocking, increasing burst order minimises average waiting. If a longer job precedes a shorter one, swapping them reduces their total waiting by the burst-length difference without delaying later jobs further. Real workloads require estimating future bursts, which need not equal their previous observations.
3. A smaller quantum can shorten the time before an interactive execution receives another turn, but increases the frequency of dispatch and context-switch overhead. If that overhead is a substantial fraction of the quantum, useful processor time falls. The appropriate quantum depends on response goals and switching costs.
4. Affinity can retain useful cache and translation state by returning work to the same processor. Strong affinity can also leave one processor overloaded while another has capacity. Priority inversion occurs when a high-priority execution depends on a resource held by a lower-priority one; scheduler choices alone cannot remove that resource dependency without an associated protocol such as priority inheritance.

5.3.6 Worked problems

FCFS timeline with staggered arrivals

P_1, P_2 and P_3 arrive at 0, 2 and 3 with CPU bursts 5, 2 and 1.
Find the FCFS intervals and waiting times.

1. P_1 is the only job at time 0, so it runs 0–5.
2. P_2 arrived before P_3, so it runs 5–7; P_3 runs 7–8.
3. Waiting times are $0 - 0 = 0$, $5 - 2 = 3$ and $7 - 3 = 4$.

FCFS order is P_1, P_2, P_3 with waits 0, 3 and 4.
Each start time must be no earlier than arrival and the previous completion.
A short job arriving behind a long job illustrates the convoy effect even when the FIFO rule is implemented perfectly.

SJF under its actual assumptions

Three jobs are all ready at time 0 with known bursts 6, 1 and 3.
Compare FCFS order $6, 1, 3$ with non-preemptive SJF.

1. FCFS waits are 0, 6 and 7, averaging 13/3.
2. SJF orders the bursts 1, 3, 6; waits are 0, 1 and 4, averaging 5/3.
3. The improvement relies on all jobs being available together and the burst lengths being known or accurately estimated.

SJF lowers the average waiting time from 13/3 to 5/3 for this fixed ready set.
The longest burst should contribute to as few other waiting times as possible, so it belongs last.
With later arrivals, choosing the shortest currently ready job need not give the globally smallest average for all possible future arrivals.

SRTF compares remaining work

P_1 arrives at 0 with burst 7. P_2 arrives at 2 with burst 2. P_3 arrives at 5 with burst 1.
Trace the preemptions.

1. P_1 runs 0–2 and has 5 units remaining.
2. At time 2, P_2 has only 2 units, so it preempts P_1 and completes at 4.
3. P_1 runs 4–5 and has 4 units remaining. At time 5, P_3 arrives with 1 unit, preempts P_1, and completes at 6.
4. P_1 then completes 6–10.

Intervals are P_1 : 0–2, P_2 : 2–4, P_1 : 4–5, P_3 : 5–6, P_1 : 6–10.

Worked example (continued)

At every arrival, compare the new burst with the current process's remaining time, not its original burst.
If an arriving job is longer than the current remaining work, SRTF does not preempt.

Round-robin queue discipline and response

Three jobs arrive together with bursts 7, 4 and 2; the quantum is 3 and switch cost is ignored.
Find the first rotation and the response times.

1. P_1 runs 0–3 and retains 4; it goes to the queue tail.
2. P_2 runs 3–6 and retains 1; it goes to the tail.
3. P_3 runs 6–8 and completes, so it is not re-enqueued.
4. First dispatch times are 0, 3 and 6, so response times are 0, 3 and 6.

The first rotation is P_1, P_2, P_3; only unfinished jobs return to the queue.
The sum of executed slices in the first rotation is $3 + 3 + 2 = 8$ units and equals elapsed time because switch cost is ignored.
As the quantum grows beyond every burst, the same queue behaves like FCFS.

Exercise 5.3.1 — Turnaround

A job arrives at 3 and completes at 12. Compute turnaround.

Worked solution

Turnaround measures elapsed time from arrival to completion. Subtract arrival 3 from completion 12: $T = 12 - 3 = \boxed{9}$ time units. This includes execution, ready-queue waiting and any other elapsed delay in the model. It differs from response time, whose endpoint is the first dispatch, and from service time, which counts only actual execution.

Exercise 5.3.2 — Response

A job arrives at 5 and first runs at 8. Compute response time.

Worked solution

Response time measures arrival to first execution. Here the process arrives at 5 and first runs at 8, giving $R = 8 - 5 = \boxed{3}$ time units. Later preemptions and waiting

Worked solution (continued)

intervals do not alter this first-response metric. They can increase completion and turnaround, which use a different endpoint.

Exercise 5.3.3 — Quantum limit

What policy does round-robin approach as the quantum becomes very large?

Worked solution

When the quantum exceeds the remaining burst of every job in the current ready queue, each selected job completes before its quantum expires. No quantum-driven preemption occurs, so execution follows the queue order as in FCFS. For bursts 3, 5 and 2 with quantum 100, the jobs run for those full durations in order. Arrivals, blocking and other preemption rules must still be specified for a complete scheduling model.

Exercise 5.3.4 — Starvation remedy

Which mechanism gradually improves the priority of a long-waiting ready process?

Worked solution

Aging gradually raises the scheduling priority of a process that has waited for a long time. This lets a persistently ready process compete with newer or initially higher-priority work. For example, a policy can improve effective priority after each fixed waiting interval. A starvation guarantee additionally depends on the policy's bounds and admission rules; merely naming aging does not specify those details.

Exercise 5.3.5 — Inversion remedy

A high-priority thread waits for a lock held by a low-priority thread. Name one targeted remedy.

Worked solution

Priority inheritance temporarily raises the effective priority of the low-priority lock owner when a higher-priority thread is blocked on its lock. This allows the owner to run ahead of unrelated medium-priority work, finish the protected region and release the lock. The inherited priority is then removed according to the proto-

Worked solution (continued)

col. The mechanism targets the dependency causing priority inversion; it does not make the blocked high-priority thread able to enter before release.

Summary

Scheduling maps ready work to processors. FCFS is simple, shortest-first improves ideal average waiting, round-robin controls response through preemption, and priority policies express importance. Every algorithm depends on assumptions about arrivals, burst estimates, switching cost and fairness.

5.3.7 Exercises

Exercise 5.3.6 — FCFS calculation

Processes arrive at time 0 with bursts 6, 2 and 4 in that order. Compute FCFS waiting times and average waiting time.

Go to the solution →

Exercise 5.3.7 — SJF calculation

For the same simultaneous arrivals, compute the non-preemptive SJF order and average waiting time.

Go to the solution →

Exercise 5.3.8 — Response versus turnaround

Can a process have a short response time and a long turnaround time? Give a scheduling scenario.

Go to the solution →

Exercise 5.3.9 — Quantum trade-off

Round-robin runs three CPU-bound processes P1=6 ms, P2=4 ms and P3=2 ms, all arriving at time 0. Compare qualitatively a 6 ms quantum with a 1 ms quantum: state which case approaches FCFS, which improves the time before later jobs first run, and why the smaller quantum can still reduce useful throughput when context switches are not free.

Go to the solution →

Exercise 5.3.10 — SRTF preemption

P1 arrives at 0 with burst 8; P2 arrives at 2 with burst 3. Give the SRTF execution intervals.

Go to the solution →

Exercise 5.3.11 — Starvation control

Design a simple ageing rule for a priority scheduler that prevents an indefinitely ready process from being bypassed forever. State what quantity changes while the process waits, why the rule eventually forces service under finite competition, and one trade-off introduced by aggressive ageing.

Go to the solution →

5.3.8 Solutions

CPU Scheduling and Time Sharing

Solution 5.3.6

FCFS preserves the given arrival order. P1 runs during $[0, 6)$, P2 during $[6, 8)$, and P3 during $[8, 12)$. Since all arrive at time zero, waiting times equal start times: 0, 6 and 8. Their sum is 14, so mean waiting time is $14/3 \approx 4.67$ time units. Completion times are 6, 8 and 12; subtracting each burst again recovers the same waiting times.

← Return to the exercise

Solution 5.3.7

Increasing burst order is P2(2), P3(4), P1(6). They execute over $[0, 2)$, $[2, 6)$ and $[6, 12)$. All arrived at zero, so waits are 0, 2 and 6 in that order, with mean $(0 + 2 + 6)/3 = 8/3 \approx 2.67$. In original process order, the waits are P1:6, P2:0 and P3:2.

← Return to the exercise

Solution 5.3.8

For example, let A and B arrive at zero in that queue order, each with a 100-unit burst, and use RR with quantum 1 and zero switch cost. B first runs at time 1, so its response time is 1. The jobs alternate until B completes at time 200, giving turnaround 200. First service and completion measure different parts of the same trace.

← Return to the exercise

Solution 5.3.9

With a 6 ms quantum and zero switch cost, the schedule is exactly FCFS: P1 runs 0–6, P2 6–10 and P3 10–12 ms. First-response delays are 0, 6 and 10 ms. With a 1 ms quantum, the first dispatches are at 0, 1 and 2 ms, so initial response improves. If each change of process adds s ms, the latter first-response delays become 0, $1+s$ and $2+2s$. The price is many more preemptions: if each switch costs nonzero time, those switches add directly to elapsed time, and repeated displacement can also damage cache/TLB locality. For $q = 1$, the service order is $1,2,3,1,2,3,1,2,1,2,1,1$. Ten changes of process add $10s$ to the 12 ms of service. With $q = 6$ there are two changes, costing $2s$. The final consecutive slices of P1 require no change of process in this comparison.

← Return to the exercise

Solution 5.3.10

P1 executes for two units before P2 arrives. At time 2, P1 has $8-2=6$ units left and P2 needs 3, so SRTF dispatches P2. Its three units occupy $[2,5)$. P1 resumes at time 5 and uses its remaining six units over $[5,11)$. The intervals total $2+3+6=11$ units, accounting for both original bursts with zero switch cost.

← Return to the exercise

Solution 5.3.11

Use priority levels $0,\ldots,P$, with P highest, a finite quantum q, and a FIFO round-robin queue at each level. Every Δ units spent ready without service increases effective priority by one, capped at P. A task entering level P joins its tail; new arrivals and re-enqueued tasks cannot jump ahead. Assume a finite number of tasks is queued ahead at any instant, bounded switch cost, and no separate higher scheduling class that can monopolise the processor.

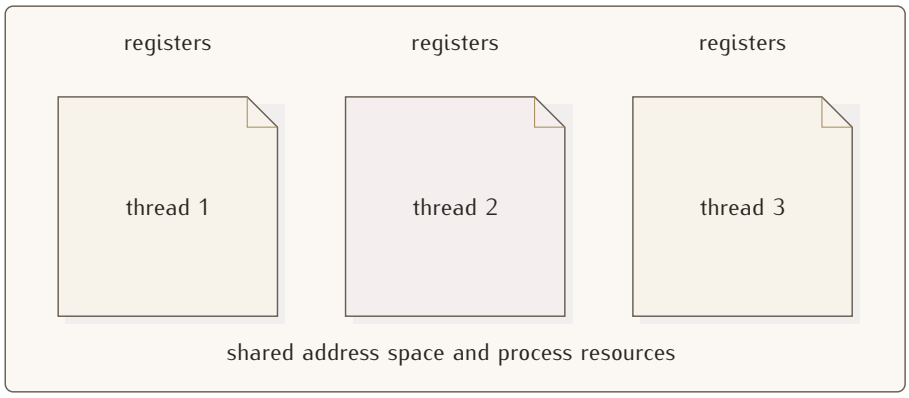
A continuously waiting task reaches level P within at most $P\Delta$. If k tasks are then ahead, each can consume at most one quantum before moving behind it or leaving. Thus it receives service after at most k quanta plus their bounded switching costs. Reaching the maximum priority alone would not suffice without this fair tie-breaking rule. After service, effective priority may reset to its base value and ageing begins again. Aggressive ageing reduces the advantage of urgent base priorities and can increase preemptions. The guarantee applies to runnable work; it cannot release a lock or complete I/O for a blocked task.

← Return to the exercise

5.4 THREADS, CONCURRENCY, AND PARALLEL EXECUTION

An application divides work among several activities that repeatedly inspect and update the same in-memory objects. Each activity needs its own instruction position and stack, while the shared objects belong to a common address space. Threads provide those separate execution streams within a process. Following the state that is private to a thread and the resources it shares explains both the convenience of cooperation and the synchronisation it requires [59, 1].

Several continuations inhabit one resource container



Each execution has its own stack and saved register state.

Figure 5.31: *The enclosing region denotes the process's shared resources. Each inner sheet denotes a thread with its own register state and stack. The enclosure groups the resources that belong to the process.*

A *thread* is a sequential execution context with its own instruction position, architectural registers and stack. Several threads can inhabit one process, sharing the process's address space and many process-wide resources while remaining independently runnable. This arrangement is powerful precisely because it removes barriers: one thread can often communicate with another by reading an ordinary object that both can address. The same property is also the source of the synchronisation problems devel-

oped in the next chapters. Sharing makes communication cheap because intermediate state is visible; correctness becomes difficult for exactly the same reason.

5.4.1 One resource container, several executions

The simplest way to understand the split is to ask what must differ if two execution streams are simultaneously inside different functions of the same program. Each needs its own next-instruction state, register set and call stack; otherwise one function call would overwrite the other's local control state. Yet the two threads may intentionally manipulate the same heap object, use the same code pages and access descriptors opened by the process.

Typically process-wide/shared	Typically per-thread
virtual address space and mappings	instruction pointer / program counter
code, globals and heap	architectural registers
open descriptor table and credentials	user stack and stack pointer
process-wide resource ownership	scheduling state and thread identifier
process-wide signal dispositions	thread-local storage and signal mask

The table is a conceptual partition rather than a complete POSIX specification. Some resources have subtler sharing rules, and a kernel may represent process-wide and thread-wide state through several linked structures. Changing a non-escaped automatic object normally changes only the owning thread's local state (which the compiler may keep in registers or on its stack), whereas changing a global or heap object changes state that every thread in the process may be able to address. A pointer bug in one thread can therefore corrupt data used by another thread or by the process as a whole; there is no process-level memory protection boundary between them.

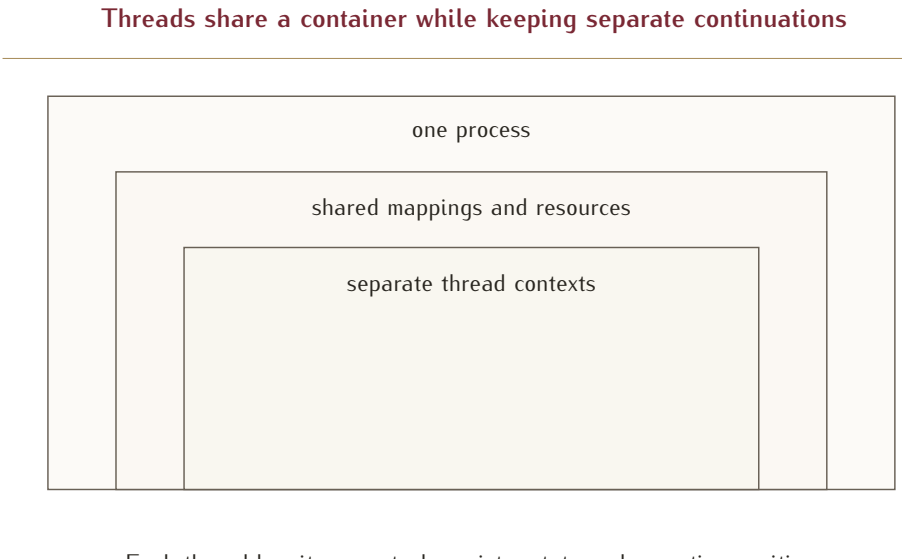


Figure 5.32: *Threads can communicate through shared objects in the same address space. Their independent execution state permits interleaving or parallel execution. Shared access creates synchronisation obligations when operations conflict.*

Thread-local storage (TLS) provides a useful middle ground. The program can name a variable through one logical declaration while arranging for each thread to access a distinct instance. TLS is not the same as an ordinary stack local: its lifetime and addressing are managed as thread-associated storage rather than as one activation record. It is useful for state such as per-thread library data, error bookkeeping or caches that should not be shared even though the code using them is common.

The scheduler-visible state also moves from process to thread once a process contains several independently runnable streams. A process can have one thread blocked in a read, another ready on a run queue and a third currently executing. Saying merely "the process is blocked" is then too coarse unless every thread has become unable to make progress. This is why operating-system discussions often shift from "process scheduling" to "thread scheduling" even though the process remains the protection and resource container.

5.4.2 Concurrency and simultaneous execution

Threads are often introduced as a way to "run things in parallel", but the same distinction established for processes still applies. Two threads are concurrent if their executions overlap in time and may interact; they are parallel only when they actually advance simultaneously on different logical processors. A single logical processor can interleave many runnable threads, producing concurrency without parallel execution. A machine

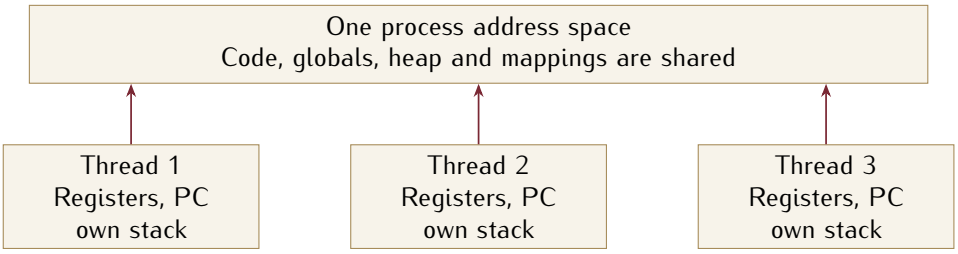


Figure 5.33: *Resource sharing and execution state are different boundaries. Each thread owns a distinct stack by convention, but stack memory is in the shared address space: another thread can access it through a valid pointer while the object remains alive. A stack is not a hardware protection boundary.*

with several logical processors can execute several ready threads at once, but creating more threads than hardware contexts does not create more physical execution capacity.

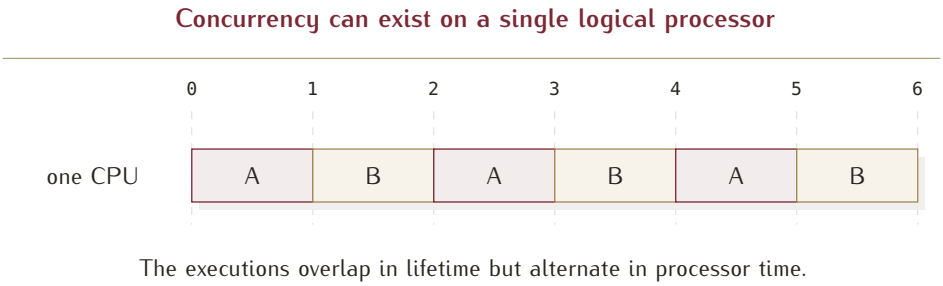


Figure 5.34: *This schedule has concurrency without simultaneous execution on that logical processor. The interleaving can still expose races because one operation can be suspended between its read and write steps.*

Even on a multicore machine, parallelism does not imply proportional speedup. If a computation contains an inherently serial portion, if threads continually wait for one another, if they compete for the same lock, or if their combined memory traffic saturates a shared bandwidth limit, additional threads can deliver diminishing returns or even reduce performance. Thread creation also has costs: stacks and kernel/runtime metadata must be allocated, work must be scheduled, and shared structures may require synchronisation. Identify activities with enough independent work to benefit from separate execution contexts, and specify how they coordinate shared state.

Threads are valuable even when no simultaneous execution occurs. An application can remain responsive while one thread waits for a slow operation; a server can structure independent requests as separate activities; a pipeline can overlap I/O preparation with computation. In these cases concurrency is partly an organisational tool. Parallelism is a possible consequence when the hardware provides additional logical processors and the workload has independent work available.

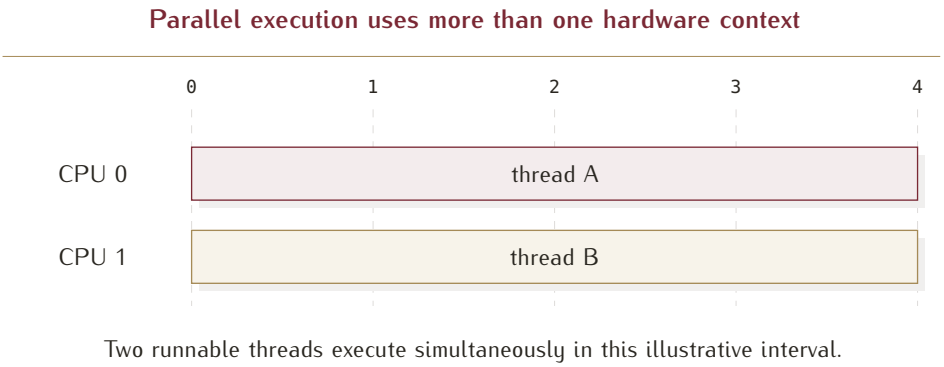


Figure 5.35: *The two rows show simultaneous execution on separate logical processors. Shared-memory ordering and synchronisation still determine which observations are permitted between the threads.*

5.4.3 Who schedules the thread?

The word *thread* describes an execution abstraction, but different systems place the scheduling boundary at different levels. In a one-to-one kernel-thread model, each application-visible thread corresponds to a kernel scheduling entity. The kernel can block one thread without blocking its siblings and can dispatch different threads of the same process on different logical processors. This gives the kernel a complete view of runnable execution contexts, at the cost of involving kernel mechanisms in thread creation, blocking and scheduling.

A user-level threading runtime can instead multiplex several user threads onto fewer kernel-visible execution contexts. If many user threads share one kernel thread, switching among them can be performed without entering the kernel and the runtime can implement application-specific scheduling policies. The limitation is structural: in a many-to-one mapping, the kernel sees only one schedulable entity, so the process cannot use several logical processors at once, and a blocking operation that blocks that kernel entity can prevent every user thread mapped to it from running.

Many-to-many or hybrid designs place several user threads on several kernel-visible contexts, attempting to combine user-space flexibility with multicore execution and blocking isolation. Such systems must coordinate two schedulers: the runtime chooses which user thread occupies a kernel execution context, while the kernel chooses where and when that context runs. The extra flexibility brings extra complexity, particularly when blocking, preemption and processor availability must be reflected between the two levels.

A thread can be scheduled independently only by a layer that represents it as an execution entity. If the kernel cannot see a user-level thread, it cannot independently

place that thread on another core; if the runtime cannot control a kernel's preemption decisions, it must tolerate those decisions when implementing its own policy.

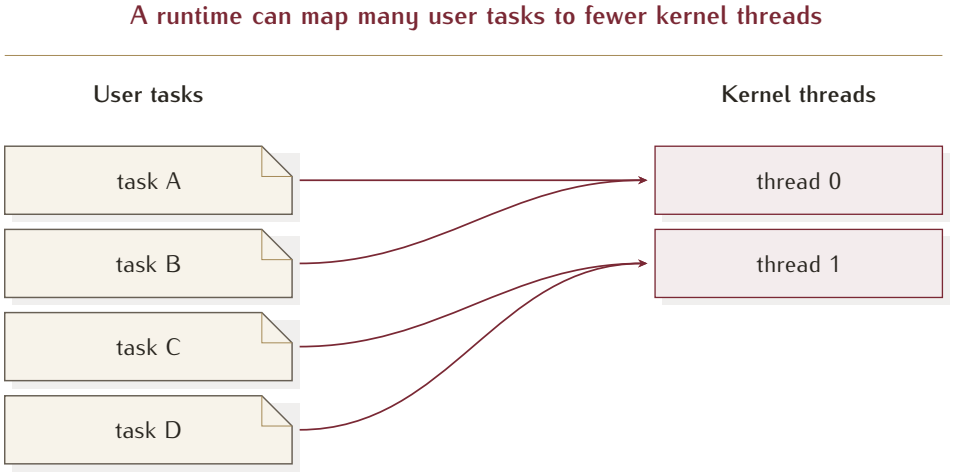


Figure 5.36: *The runtime schedules user-level work onto kernel-visible execution resources. The operating system then schedules those resources onto logical processors. Blocking behaviour depends on how the runtime and kernel cooperate.*

The number of runnable threads also interacts with scheduling. Creating more runnable threads than logical processors produces *oversubscription*: some threads must wait even if every hardware context is busy. When one thread blocks on I/O, another can use the processor. Excessive runnable concurrency can instead increase switching, cache displacement and lock contention. Compute-heavy thread pools are therefore often sized with available processor capacity in mind, whereas I/O-heavy systems may tolerate more outstanding threads because many are blocked rather than runnable at once.

A server with ten thousand live connections may need only a modest number of workers when most connections are waiting for an event. A numerical loop with continuously runnable workers may instead reach saturation near the available hardware capacity. Measuring blocking time and processor use distinguishes these cases.

This is another reason to distinguish application concurrency from hardware parallelism. A program can profit from representing hundreds of independent requests as tasks without needing hundreds of simultaneously runnable kernel threads. Runtimes, event loops and thread pools can expose large logical concurrency while controlling the number of execution contexts that actually compete for processors. The operating system scheduler sees the runnable kernel-visible entities; the application decides how much higher-level work to map onto them.

5.4.4 Thread lifecycle and ownership in POSIX programs

POSIX threads provide a concrete interface for the model. A call to `pthread_create` establishes a new thread that begins in a specified start routine, while the creating thread continues independently. A joinable thread retains enough termination state for another thread to collect it with `pthread_join`; a detached thread instead releases its join-related resources automatically when it terminates and cannot later be joined in the same way. The distinction determines responsibility for reclamation independently of scheduling priority.

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void *worker(void *arg) {
    long id = *(long *)arg;
    printf("worker %ld\n", id);
    return NULL;
}

int main(void) {
    pthread_t t[4];
    long id[4];
    int created = 0;
    int failed = 0;
    for (int i = 0; i < 4; ++i) {
        id[i] = i;
        int error = pthread_create(&t[i], NULL, worker, &id[i]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            failed = 1;
            break;
        }
        ++created;
    }
    for (int i = 0; i < created; ++i) {
        int error = pthread_join(t[i], NULL);
        if (error != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(error));
            exit(EXIT_FAILURE);
        }
    }
    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Save this as `threads.c` and compile with `cc -std=c11 -Wall -Wextra -pthread threads.c -o threads`. A successful run prints each identifier 0, 1, 2 and 3 once, in an unspecified order. If creation fails, only successfully created threads are joined. POSIX thread functions return their error number directly; the code passes that number to `strerror`.

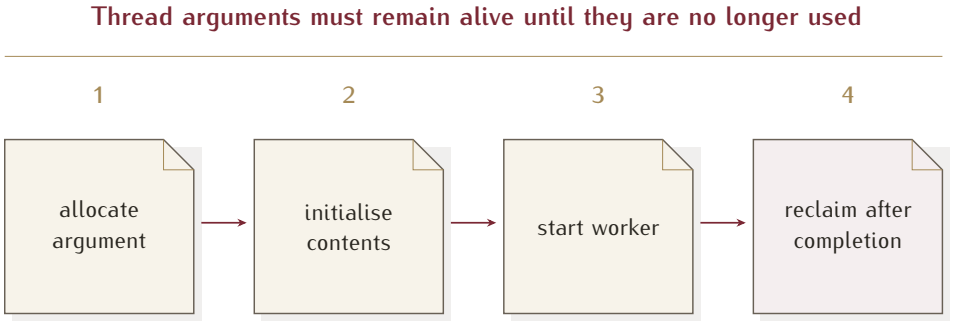


Figure 5.37: *The creator must establish a lifetime covering every worker access. Reusing a stack variable or freeing an argument too early can invalidate the worker's view even if thread creation itself succeeds.*

The example works because the `id` array remains alive until all created threads have finished using it. A common error is to pass every thread the address of the same loop variable:

```
for (long i = 0; i < 4; ++i)
    pthread_create(&t[i], 0, worker, &i);  /* unsafe design */
```

Every worker receives the same address while the creator repeatedly modifies that object. The unsynchronised read/write conflict is a C data race and makes the program's behaviour undefined; repeated or unexpected identifiers are only possible symptoms. The loop variable also goes out of scope after the loop, so a worker that uses the pointer later can access an object whose lifetime has ended. Distinct stable argument objects, kept alive until their readers finish, address both problems.

A robust program also checks thread-library return values and defines who is responsible for freeing dynamically allocated arguments or returned objects. Threads reduce the need for explicit IPC, but they do not remove resource lifetime. In fact, shared access makes lifetime mistakes easier to hide because a pointer can remain syntactically valid while the logical ownership of the object has already changed.

A thread need not correspond one-for-one with a long-lived application role. Many servers create a bounded *thread pool*: a fixed or adaptively sized set of worker threads repeatedly removes tasks from a shared queue. Separating tasks from threads avoids paying creation/destruction cost for every request and gives the application a place

to limit concurrency. If thousands of requests arrive, the queue can grow or apply backpressure while only a controlled number of workers simultaneously compete for processors, caches and downstream resources. This is an important distinction between *work* and *execution contexts*: increasing the amount of pending work does not require creating the same number of kernel threads.

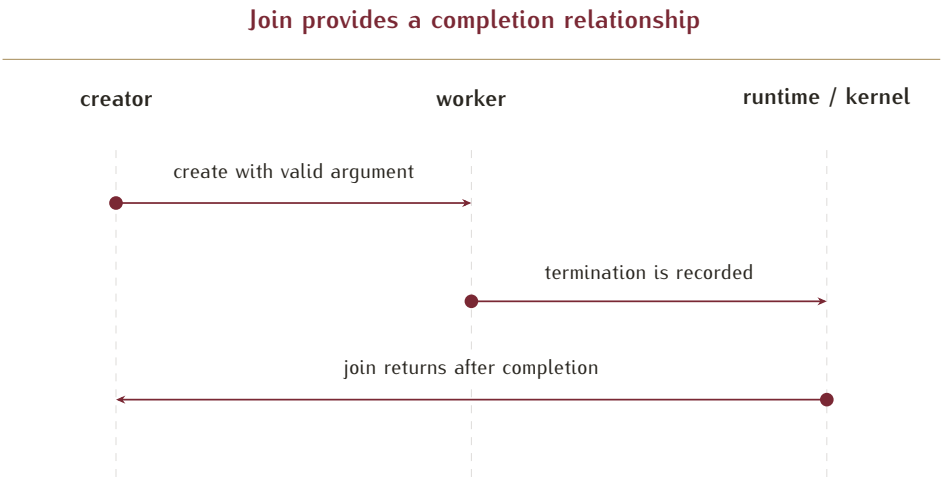


Figure 5.38: *Joining an eligible joinable thread waits for its termination and supplies the specified synchronisation relationship. It also permits the program to perform lifecycle cleanup in the correct order.*

The pool also makes ownership questions concrete. A task object may be allocated by an accepting thread, transferred to the queue, removed by a worker and eventually freed. At each stage the program needs a rule for who may access the object and when its lifetime ends. Shared addressability does not mean shared ownership. A disciplined concurrent program often reduces races by arranging that mutable objects have one logical owner at a time, using queues or explicit hand-off to transfer ownership instead of allowing every thread to mutate every reachable object.

Stacks deserve similar care. Because a thread's stack is private as execution state but resides inside the process address space, another thread can technically hold a pointer into it if the program passes such a pointer. The memory-protection mechanism does not prevent that access. Correctness must instead come from lifetime and synchronisation: a pointer to a stack object becomes invalid when the declaring activation ends, and concurrent access still requires appropriate ordering if one thread modifies the object while another observes it. A private stack belongs to one execution stream. On the POSIX systems considered here it resides in the shared process address space.

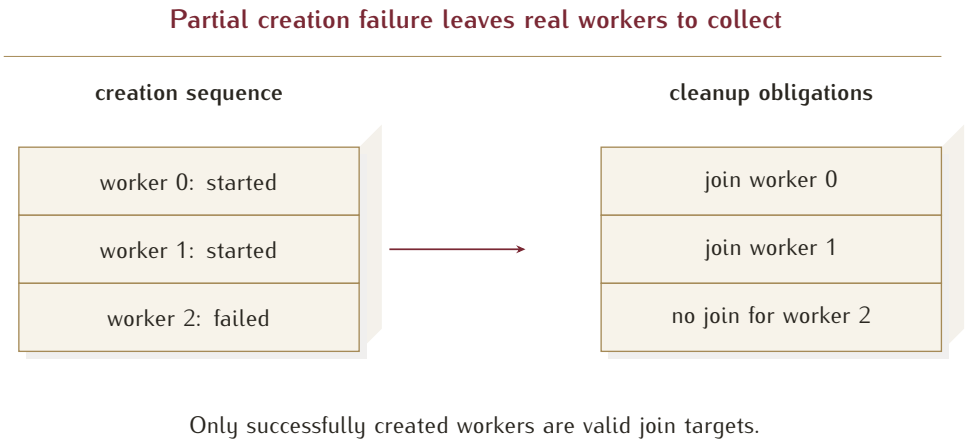


Figure 5.39: *A loop must remember how many workers were actually started. A failure at a later creation does not undo earlier workers. Cleanup joins those workers before releasing shared arguments or results they may still access.*

Signals and blocking operations further demonstrate that some state is process-wide while some is thread-specific. A process can define a shared disposition for a signal while individual threads maintain masks that influence which thread can receive it; a blocking system call normally blocks the calling kernel thread rather than every sibling in a one-to-one model. Such rules are interface-specific, but they reinforce the central abstraction: the process supplies shared resources and protection context, whereas the thread supplies an independently schedulable sequence through that context.

5.4.5 Choosing a boundary: processes, threads, or both

Processes remain attractive when isolation is itself a requirement: independent failures, separate privileges, replaceable program images or security boundaries can justify the extra cost of explicit communication. Threads are attractive when tasks belong to one trust domain and require frequent access to common in-memory state. A large service often uses both levels: multiple processes establish isolation or administrative boundaries, while each process contains a pool of worker threads that share caches, request queues or connection state.

A process boundary changes which failures and writes can propagate directly; a thread boundary changes how much state is shared by default. Performance follows from the workload and implementation, whereas the semantic distinction follows from protection and sharing.

That last point leads directly to the central difficulty of concurrent programming. If two processes communicate only through an explicit pipe, the pipe's interface mediates their interaction. If two threads can both reach `balance`, `queue_head` or a shared object graph, ordinary load and store instructions become communication. The source

program may show one assignment, yet the machine can interleave the reads, computations and writes that implement it with operations from another thread. The next chapter asks what correctness means when that interleaving is no longer controlled by sequential program order.

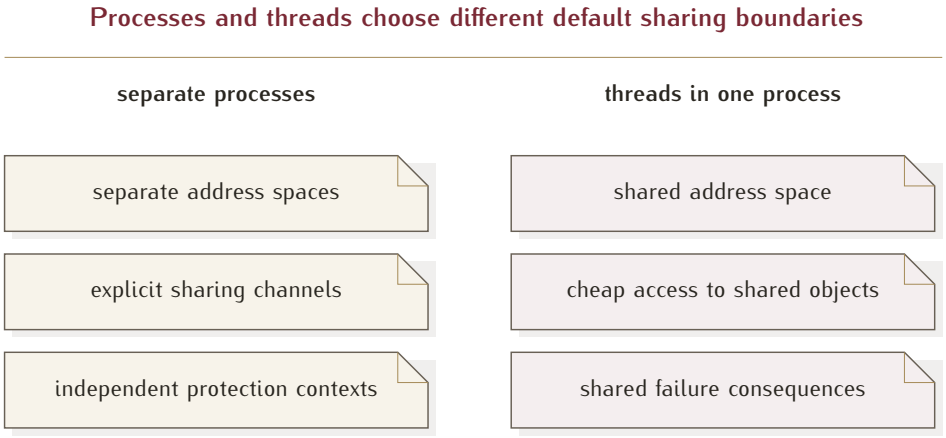


Figure 5.40: *A thread boundary permits direct sharing of process memory, while a process boundary supplies stronger default address-space separation. Either design still needs explicit protocols for shared resources and shutdown.*

Concept check

1. Which state must be private to two threads that are executing different call chains, and which state can intentionally remain process-wide?
2. Why can a many-to-one user-thread runtime provide concurrency but not simultaneous execution on two logical processors?
3. What is the difference between joinable/detached lifecycle and ready/running/blocked scheduling state?
4. Why does the ease of sharing memory make threads both convenient and dangerous?

Solutions to the review questions

1. Each independently progressing thread needs its own instruction position, registers and call stack. Address space, code, heap objects and the process's open resources can be shared. Thread-local storage provides selected data with per-thread instances.
2. All user threads in a many-to-one mapping execute through one kernel-schedulable thread. The runtime can alternate among them, providing con-

Worked solution (continued)

current progress, but the kernel can dispatch that single context on only one logical processor at a time. Simultaneous execution requires additional kernel-schedulable contexts.

3. Joinable versus detached determines how termination information and thread resources are reclaimed. Ready, running and blocked describe present scheduling eligibility and execution. A joinable thread can be in any of those states before it terminates.
4. Shared memory avoids explicit copying for many forms of communication. It also exposes objects to overlapping accesses and makes ownership, lifetime and synchronisation part of the program's correctness argument. An accessible address alone does not establish a safe protocol.

5.4.6 Worked problems

Shared and private state

Four threads execute in one process that has global data, an allocated heap object and two open files.
Which state is shared and what must remain private?

1. Code, global data, heap mappings and the process file-descriptor table belong to the process and are visible to all four threads.
2. Each thread needs a separate program counter and register image to resume a different instruction stream.
3. Each thread needs a separate stack for its own call chain and automatic variables.

The address space and process resources are shared; PC, registers and stack are per thread.
If two threads shared one stack pointer, nested calls in one would overwrite the other's activation records.
Thread-local storage provides additional per-thread objects even though they reside in one process address space.

Concurrency does not guarantee parallelism

A process has eight ready threads but can currently use two logical processors.
What is the maximum instantaneous parallelism?

1. The eight threads are concurrent because their lifetimes and progress overlap.

Worked example (continued)

2. In this model, each logical processor advances one scheduled thread context at an instant.
3. Only two schedulable hardware contexts are available, so at most two of the eight threads execute simultaneously.

Maximum instantaneous parallelism is two; the remaining six ready threads wait for dispatch.
Adding threads beyond the available logical processors can still hide blocking, but cannot by itself create additional simultaneous hardware execution capacity.
On one logical processor, all eight can remain concurrent through interleaving while only one of them advances at an instant.

Many-to-one versus kernel-visible threads

A runtime maps many user threads onto one kernel entity. One user thread performs a blocking read.
Why can the whole process stop making progress, and how does one-to-one mapping differ?

1. The kernel sees the single schedulable entity on which the runtime's user threads depend.
2. When that entity blocks, the runtime has no executing vehicle on which to run another user thread.
3. With one-to-one kernel-visible threads, the kernel can block the caller and schedule a different runnable thread, possibly on another logical processor.

The mapping determines which user threads can continue when a kernel entity blocks and how many processors can execute them.
Nonblocking I/O or a many-to-many runtime can change the outcome, so the implementation assumptions must be stated.
A page fault in a many-to-one package can have the same whole-entity blocking effect.

Stable POSIX-thread arguments

For three workers, allocate `long id[3]` in the creating thread's still-active frame. Set `id[i] = i` before passing `&id[i]` to worker `i`. Each worker reads one initialised object at a distinct address. Record each successful creation and join those workers before returning from the creating function. That order keeps all argument objects alive, including when creation of the last worker fails. Passing a pointer to the changing loop counter would instead introduce conflicting accesses and a lifetime hazard.

Exercise 5.4.1 — Heap

Is an ordinary heap allocation private to the thread that called `malloc`?

Worked solution

An ordinary heap allocation belongs to the process address space. Other threads in that process can access it if they obtain its address and the program permits the access. The allocating thread's stack and registers are separate execution state, but `malloc` ownership alone does not create memory protection. Concurrent accesses to the allocation still need the synchronization required by their operations.

Exercise 5.4.2 — Stack

How many user stacks are normally required for six active threads?

Worked solution

Six active threads normally need six user stacks, one per thread, because each execution maintains its own calls, local automatic objects and return state. They can share code and heap storage while their call chains differ. Stack storage is ordinarily mapped within the same process address space; separate stacks provide separate execution state rather than a security boundary between threads.

Exercise 5.4.3 — Isolation

Which model better isolates mutually untrusted components from arbitrary memory writes?

Worked solution

Separate processes with controlled interprocess communication provide the stronger isolation model here. Each process has its own address-space mappings and access permissions, so an arbitrary write in one process cannot normally address another's private storage. Threads share an address space and can corrupt one another's accessible objects. Shared-memory IPC must itself be designed with explicit ownership and validation rules.

Exercise 5.4.4 — Join

What two broad duties does `pthread_join` serve for a joinable thread?

Worked solution

For a joinable thread, `pthread_join` waits for the target's termination if necessary and collects the termination result while allowing its join-related resources to be reclaimed. If the target already terminated, the wait need not delay. The caller must check API errors and ensure a valid joining protocol. Join is a lifecycle operation; shared data accessed before termination still requires its own synchronization.

Exercise 5.4.5 — Address space

Five threads belong to one ordinary process. How many process address spaces do they share?

Worked solution

The five threads share one process address space. They can therefore refer to the same code, heap and global objects. Each thread separately maintains registers, a stack and scheduling-related execution state. Counting threads gives five execution contexts; counting the ordinary process virtual-address space gives $\boxed{1}$. This distinction explains both inexpensive sharing and the need to control races.

Summary

A process is a resource and protection container; a thread is an execution stream inside it. Threads reduce communication overhead and expose parallelism, but every shared object requires a clear ownership and synchronisation rule.

5.4.7 Exercises

Exercise 5.4.6 — Shared or private

Classify code, heap, registers, stack and open files as process-shared or per-thread state.

Go to the solution →

Exercise 5.4.7 — Concurrency versus parallelism

Give one example of concurrent execution without parallel execution.

Go to the solution →

Exercise 5.4.8 — User-thread limitation

Why can many-to-one user threads fail to make progress when one thread performs a blocking system call?

Go to the solution →

Exercise 5.4.9 — Argument lifetime

Why is passing the address of one loop variable to every new thread unsafe?

Go to the solution →

Exercise 5.4.10 — Join purpose

State two resources or correctness obligations addressed by joining a thread.

Go to the solution →

Exercise 5.4.11 — Model choice

A host must run two mutually untrusted plugins. Compare separate processes with threads in one process under three criteria: fault containment, communication cost and shared-state convenience. Choose a default design and state one condition under which you would reconsider it.

Go to the solution →

5.4.8 Solutions

Threads, Concurrency, and Parallel Execution

Solution 5.4.6

Threads in one process use the same address space, so code and heap objects are shared. The process also supplies the open-file descriptor table. Each thread has its own saved register context and a separate stack for its call frames. Those stacks occupy the shared address space, so another thread can access a stack object if given a valid pointer and proper lifetime/synchronisation guarantees. Thus "per-thread stack" describes ownership and normal use, rather than memory protection between threads.

← Return to the exercise

Solution 5.4.7

On one logical processor, thread A prepares an audio buffer during $[0, 1)$, thread B handles a browser event during $[1, 2)$, and A resumes during $[2, 3)$. Both threads are live during the interval and both advance. Their execution intervals on the processor are disjoint, so the trace exhibits concurrency through interleaving.

← Return to the exercise

Solution 5.4.8

The kernel sees one schedulable entity. If it blocks in the kernel, the library cannot run another user thread on that entity until the call returns.

← Return to the exercise

Solution 5.4.9

Each worker receives the same pointer while the creating thread changes the pointed-to variable. Without synchronisation, this conflicting access is a C data race with undefined behaviour. A worker can also outlive the loop variable. Store one identifier per worker in an array, initialise each before its create call, keep it unchanged, and join all successfully created workers before the array leaves scope.

← Return to the exercise

Solution 5.4.10

A successful join establishes that the target has terminated and permits reclamation of its retained joinable-thread resources. It also synchronises with the target's completed execution, so the joining thread can inspect results after the join without racing a still-running target. For example, the identifier array in the complete program may leave scope after every successful worker has been joined. Join does not repair data races that already occurred before it.

← Return to the exercise

Solution 5.4.11

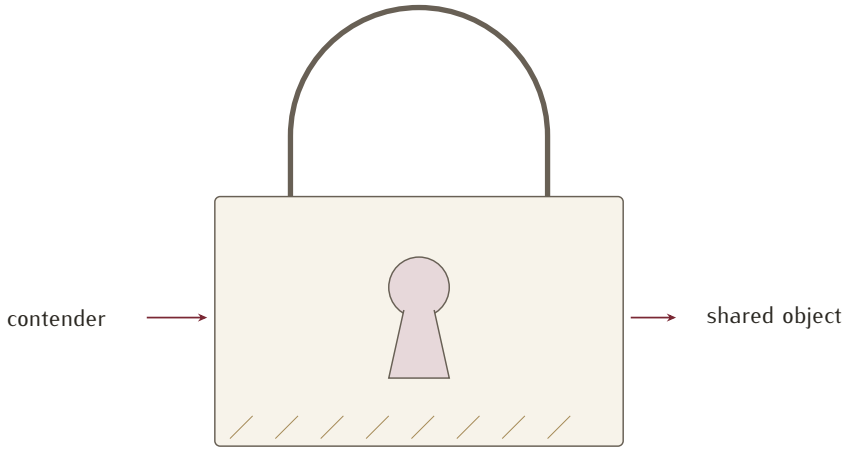
Use separate processes with restricted credentials, capabilities and IPC. Distinct address spaces provide a memory boundary: a wild pointer, allocator corruption or deliberate write in one plugin need not become a write into the other's memory. Threads would make communication and shared data structures cheaper, but that convenience also removes memory isolation and makes every shared object part of the trust boundary. A separate PID alone is insufficient for hostile-code isolation if operating-system permissions still allow process inspection, shared writable files or other powerful access. The process design therefore combines memory separation with an explicit authority policy and pays IPC/context-management costs for that containment. I would reconsider it only if the plugins were mutually trusted and the measured IPC/copying overhead dominated the workload enough to justify sharing one address space.

← Return to the exercise

5.5 SHARED STATE, CRITICAL SECTIONS, AND SEMAPHORES

Two workers finish a task and both increment the same counter. Each may read the old value before either stores a replacement. To reason about the combined result, we must state which steps can overlap and which properties must hold when an update completes. Synchronisation begins with that invariant and then supplies a protocol that preserves it across the possible executions.

An invariant needs a clear ownership rule



Ownership grants access to the complete protected operation.

Figure 5.41: *The lock symbol introduces mutual exclusion as a protocol around shared state. Correctness depends on covering the full invariant-preserving operation and coordinating all participants through the same rule.*

5.5.1 Interleavings and lost updates

Suppose an account contains 1000 units. One operation deposits 700 and another withdraws 500. Either complete sequential order leaves 1200. In a model where each operation consists of a read, local computation and write, both operations can read 1000 before either publishes a result. The deposit computes 1700; the withdrawal computes

500. If the withdrawal's store occurs last, the stored balance becomes 500 and the deposit has been lost.

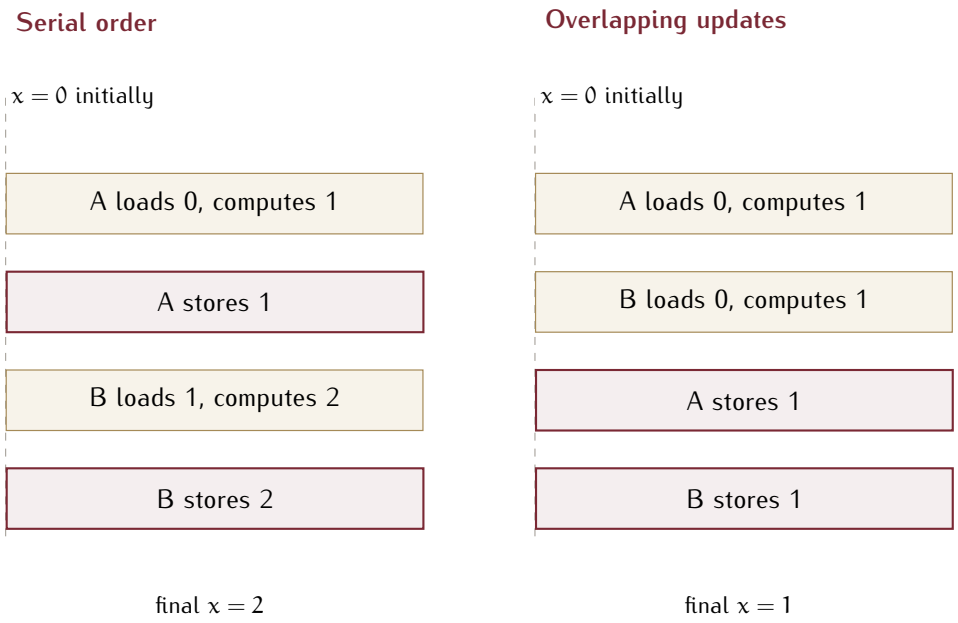


Figure 5.42: *Two increments of a counter in an abstract sequentially consistent model with individually atomic loads and stores. Read each column downward. Both workers preserve their own loaded value until their store. The overlapping execution therefore loses an increment. Conflicting unsynchronised accesses to an ordinary C object have undefined behaviour; this trace describes the declared abstract model.*

This trace describes an abstract interleaving of reads and writes. A C program that performs conflicting ordinary accesses from different threads without the required synchronisation has a *data race* and undefined behaviour. The language permits no enumeration of its possible outcomes based solely on such a trace. The abstract trace identifies the logical failure that a correct implementation must prevent [3, 1].

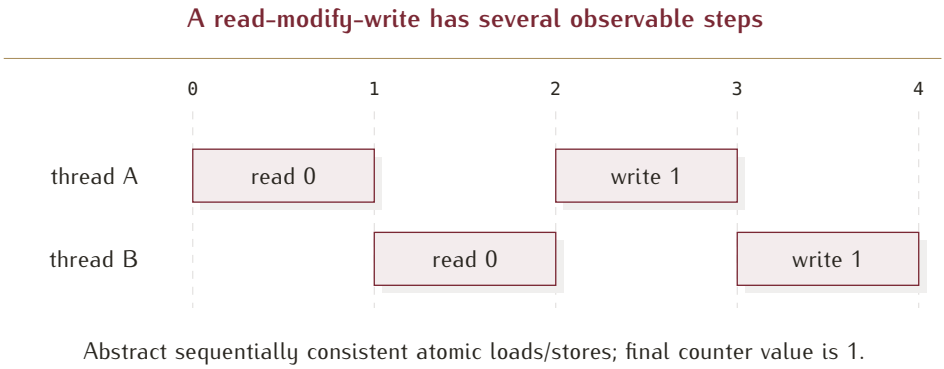


Figure 5.43: *Both executions compute their replacement from the same old value. The second store therefore loses one increment. This is an abstract atomic-load/store model; unsynchronised conflicting ordinary C accesses instead constitute a data race with undefined behaviour.*

A *race condition* is a broader protocol failure in which correctness depends on an uncontrolled ordering. Individual atomic accesses can remove a data race while leaving a logical race. For example, an atomic load of a balance followed later by an atomic store can still overwrite an intervening update. The relevant operation is the entire state transition that the application promises.

5.5.2 Choosing the critical section

For the account, the invariant relates the initial balance to all completed deposits and withdrawals. A lock must cover reading the old balance, computing the new value and publishing it:

```
acquire(lock)
    old = balance
    balance = old + delta
release(lock)
```

Acquiring the lock only for the final store leaves both reads free to observe the same old value. Acquiring it around the complete update serialises the state transitions. Once a participant releases the lock, the invariant must hold again.

The protected state can span several variables. A queue may require an array, two indices and an item count to describe one consistent state. Choosing a critical section means identifying which observations would be invalid while an update is incomplete. The same mutex must protect every conflicting access covered by the invariant.

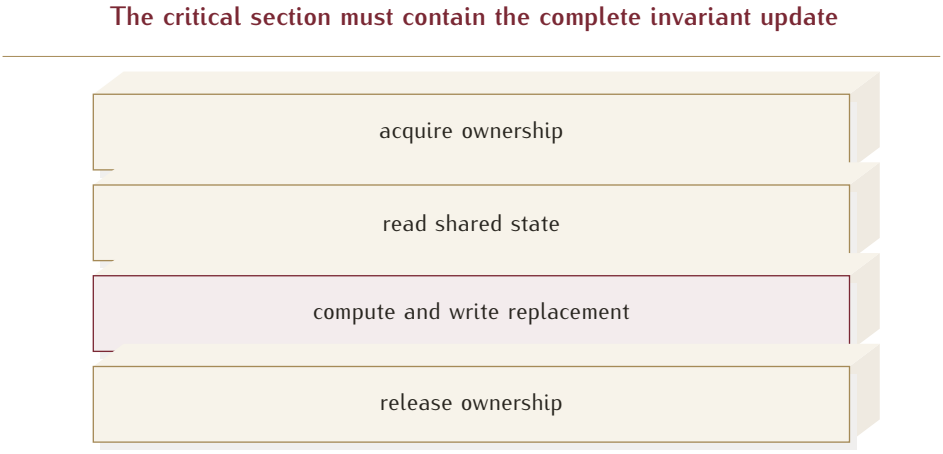


Figure 5.44: *Mutual exclusion prevents another participant from observing or modifying the protected state halfway through the operation. The chosen section must cover all steps needed to preserve the shared invariant.*

Mutual exclusion requires at most one conflicting participant in the protected region. *Progress* requires that entry remain possible when the region is available and participants are waiting. *Bounded waiting* constrains how often a requester can be bypassed. Each property needs its own argument. In strict alternation, a shared turn variable may enforce exclusion while preventing an interested thread from entering because the other thread is doing unrelated work. That execution violates progress.

5.5.3 A complete mutex-protected counter

The following POSIX C program gives two threads 100,000 increments each. Save it as `counter.c`. The shared counter is initially zero; each successful critical section increases it by exactly one.

Compiling, running and verifying a shared counter

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
static unsigned long counter;

static void checked(int error) {
```

Worked example (continued)

```
    if (error != 0) {
        fprintf(stderr, "%s\n", strerror(error));
        exit(EXIT_FAILURE);
    }
}

static void *worker(void *argument) {
    (void)argument;
    for (unsigned long i = 0; i < 100000UL; ++i) {
        checked(pthread_mutex_lock(&lock));
        ++counter;
        checked(pthread_mutex_unlock(&lock));
    }
    return NULL;
}

int main(void) {
    pthread_t threads[2];
    for (int i = 0; i < 2; ++i)
        checked(pthread_create(&threads[i], NULL, worker, NULL));
    for (int i = 0; i < 2; ++i)
        checked(pthread_join(threads[i], NULL));
    printf("%lu\n", counter);
    checked(pthread_mutex_destroy(&lock));
    return EXIT_SUCCESS;
}
```

Compile and execute on a Linux system with a C compiler and POSIX threads:

```
cc -std=c17 -O2 -Wall -Wextra -pthread counter.c -o counter
./counter
```

The expected standard output is `200000`. At the start of each critical section, the counter equals the number of increments already completed. Exclusion prevents another increment from overlapping its read–modify–write effect, so the invariant is preserved after adding one. Each worker completes 100,000 iterations, and successful joins establish that both have finished before the main thread prints the value. Error codes from the thread operations are checked; a failure terminates this example instead of continuing under an invalid assumption.
The read in `printf` occurs after both joins, when no worker can modify the counter. Moving that read into the creation loop would require a separate synchronisation argument. Removing the mutex from the worker creates a C data race; running that modified program repeatedly cannot establish a valid replacement for the lock.

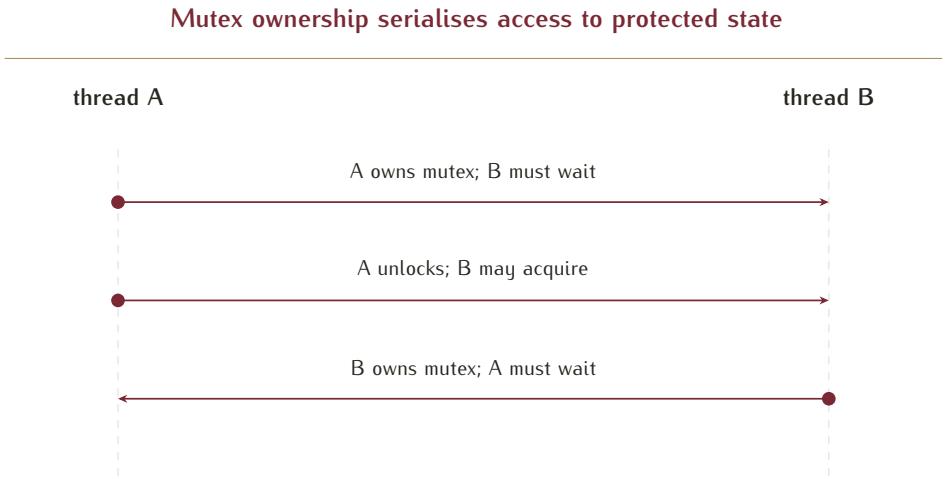


Figure 5.45: *The mutex protocol determines which participant may enter the section. Unlocking makes a future acquisition possible, but the exact next owner depends on the implementation and scheduling behaviour.*

For this isolated counter, a language-level atomic fetch-add could implement each increment as one atomic operation. A transaction involving several variables or a conditional decision may require a wider critical section. The shape of the invariant determines that boundary.

5.5.4 Atomicity, visibility and ordering

Atomicity specifies the indivisible effect of an operation. Visibility concerns which writes an execution can observe. Ordering specifies the constraints between operations and their observations. A locking API connects these properties: writes made while holding a mutex become available to a later participant that acquires the same mutex according to the API's synchronisation rules.

An ordinary polling loop over a shared C variable supplies no such protocol:

```
while (flag == 0) {
    /* wait */
}
```

If another thread modifies that ordinary variable concurrently, the program can contain a data race. Declaring it `volatile` does not establish the required inter-thread synchronisation. A correct communication protocol uses an appropriate mutex, semaphore or language-level atomic operations with the necessary ordering guarantees.

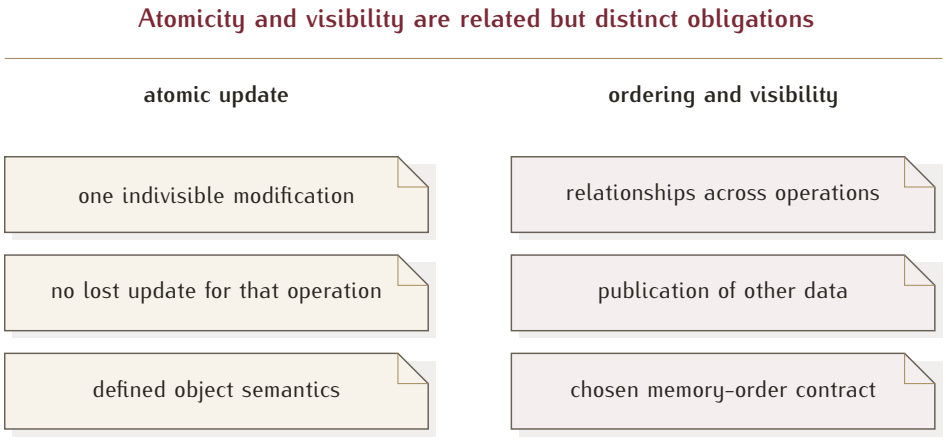


Figure 5.46: *A correct concurrent design needs both the intended atomic state transitions and the required communication order. The chosen synchronisation primitive or memory-order operation must establish each necessary relationship.*

Hardware implements these facilities using atomic instructions, coherence and ordering mechanisms. A source-level lock can also involve a wait queue and scheduler operations. Its correctness depends on the combined protocol. The implementation may use a fast path when uncontended and a different path when a requester must wait.

5.5.5 Spinning, blocking and ownership

A spinning thread repeatedly tests whether acquisition can succeed. This can be useful for a very short wait when the holder is running on another processor. A descheduled holder or a long critical section can make spinning consume substantial processor time without advancing the protected operation. Contending writes to the lock word can also move its cache line among cores. Read-mostly waiting and queued protocols address different parts of that cost.

A blocking mutex can make a waiter non-runnable until the synchronisation subsystem allows it to compete again. Blocking has scheduling and wake-up costs, but releases processor time during the wait. A wake-up changes eligibility; the scheduler still chooses when the thread actually runs.

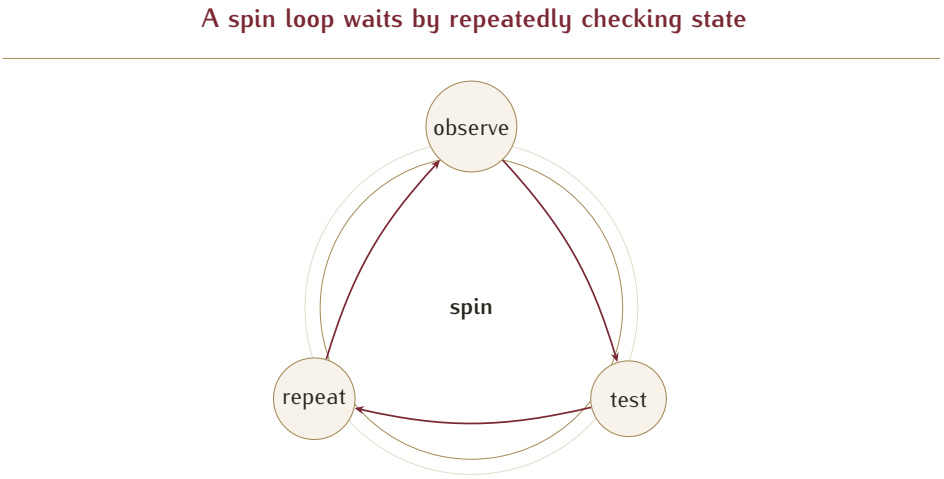


Figure 5.47: *Spinning consumes processor resources while waiting. It can be useful for short waits under suitable conditions, but scheduling and the owner's ability to run matter. A sleeping wait uses a different mechanism.*

Mutex ownership ties acquisition and release to the owning thread. A semaphore instead represents permits that can be published and consumed by different participants. That distinction allows a semaphore to express event ordering as well as resource availability. Wake-up order and fairness depend on the implementation and the protocol; they must be stated when a proof requires them.

5.5.6 Semaphore permits and event ordering

The following algorithms use abstract semaphore operations `wait` and `signal`, each assumed to complete successfully; `signal` here is not the POSIX signal-handler API. A POSIX implementation normally uses `sem_wait` and `sem_post` and must handle errors, including an interrupted wait.

A counting semaphore has an abstract non-negative permit count. A successful `wait` consumes one permit atomically. If none is available, the operation waits for the protocol to grant one. A `signal` publishes a permit or enables a waiting acquisition to complete. Internal implementations may encode waiters differently while preserving this observable behaviour.

For exclusion, initialise a semaphore to one. Every entrant consumes the sole permit before the critical section and returns it afterward. For a one-time event, initialise a semaphore to zero. A producer establishes the required state and then signals; a consumer waits before using that state. The initial value follows from what a permit means in the application.

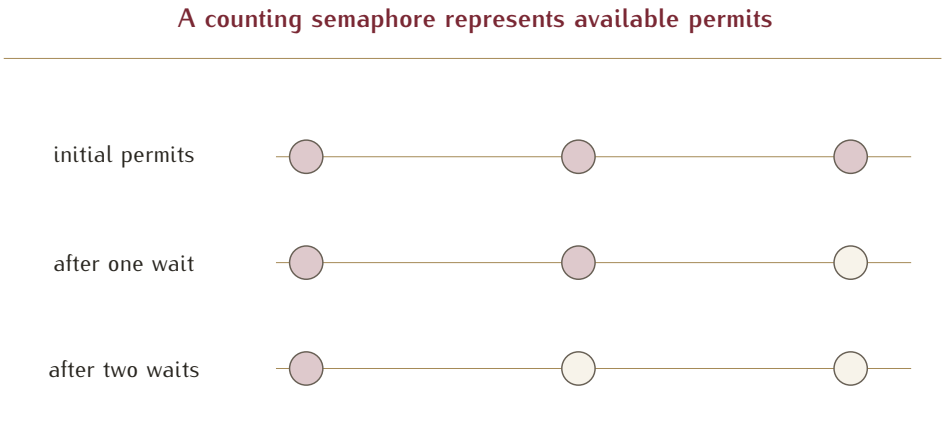


Figure 5.48: *The count represents permits under the chosen protocol, not necessarily physical items currently in a container. In-flight reservations can make those quantities differ. A zero count prevents another immediate successful wait.*

Publishing an event before or after the waiter arrives

Let `done` initially contain zero permits. If the consumer waits first, it blocks until the producer publishes a permit. If the producer signals first, the count becomes one and the consumer's later wait consumes it without blocking. Both orders satisfy the requirement that the consumer proceeds only after the event has been published. A second wait requires a second permit; the first event does not authorise an unlimited number of consumers.

For a pool of three reusable resources, initialise a semaphore to three. An acquisition consumes a permit and a release returns it. A failure path after acquisition must either release the resource or transfer ownership through a defined protocol. Losing the return path can leave later callers blocked even when no participant is using the resource. Releasing twice can admit more users than the resource pool supports.

5.5.7 Conditional updates and lock granularity

Consider a withdrawal allowed only while the balance is at least 500. An atomic load followed by a comparison and a separate atomic subtraction leaves time for another withdrawal between the check and the update. A mutex around both steps preserves the condition. A compare-and-exchange loop can also preserve it by checking that the value used for the decision is still current at the instant of the update. A failed comparison requires repeating the decision on the newly observed state.

Withdrawing from a single atomic balance

Here the balance is an unsigned number and the transaction changes only that number. The caller supplies a live, initialised atomic object.

```
#include <stdbool.h>
#include <stdatomic.h>

static bool withdraw(_Atomic unsigned *balance, unsigned cost) {
    unsigned old = atomic_load_explicit(balance, memory_order_relaxed);
    while (old >= cost) {
        if (atomic_compare_exchange_weak_explicit(
                balance, &old, old - cost,
                memory_order_relaxed, memory_order_relaxed))
            return true;
    }
    return false;
}
```

Suppose two threads each request 70 from a balance of 100. Both can initially read 100, but only one can replace 100 with 30. The other's failed comparison updates its expected value to 30. Rechecking $30 \geqslant 70$ then rejects the second withdrawal. A weak comparison can also fail spuriously, so the loop rechecks the condition after every failure. The successful comparison is the instant the withdrawal takes effect. Relaxed ordering suffices for the invariant of this single atomic object. Publishing a related receipt or updating a second account introduces additional ordering or transaction requirements.

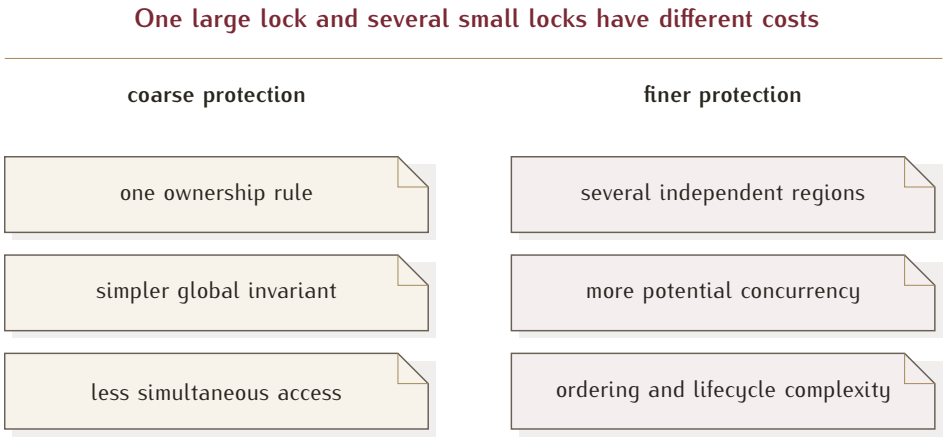


Figure 5.49: *Splitting a lock can permit independent operations to overlap. It also introduces relationships between locks when an operation spans multiple regions. Those relationships require an explicit acquisition order and invariant argument.*

A coarse lock can make a multi-object invariant straightforward to express. Finer locks permit operations on independent objects to overlap, but an operation spanning several objects must acquire the relevant locks in a consistent order. That order affects the deadlock dependencies developed in Chapter 5.7. The bounded-buffer and readers–writers problems in Chapter 5.6 apply the same method: state the invariant, identify the protected transitions, and assign each semaphore a precise meaning.

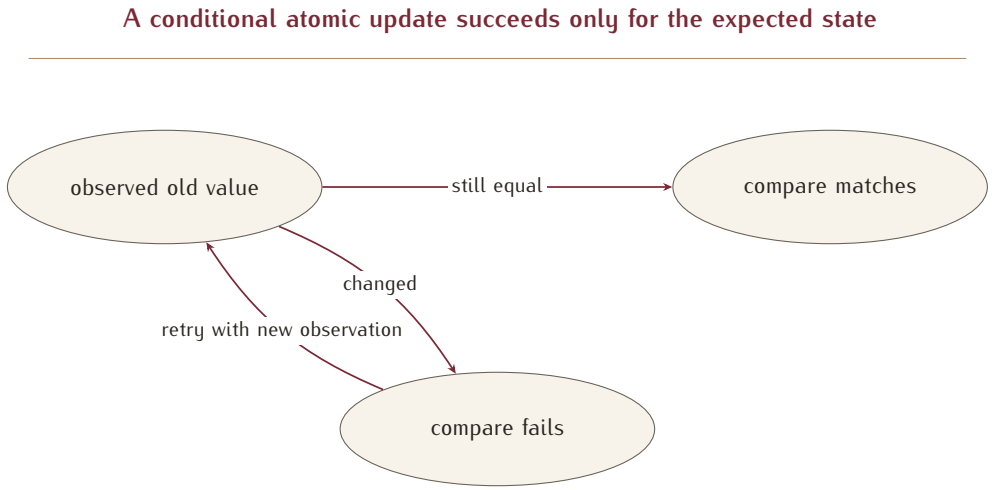


Figure 5.50: *Compare-and-exchange makes the replacement conditional on the current value matching the expectation. Failure means the caller must use the specified returned observation or retry policy. A complete algorithm must also justify progress and object lifetime.*

5.5.8 Worked problems

A lost update with atomic loads and stores

Use an abstract sequentially consistent model with individually atomic loads and stores. A shared counter starts at 10. Each of threads A and B loads it, adds 2 privately and stores the result; the full update is not indivisible.
Show an execution that ends at 12 instead of 14.

1. A reads 10; before it stores, B also reads 10.
2. A computes 12 and B independently computes 12.
3. A stores 12, then B stores 12. The second store overwrites an equal value instead of incorporating A's logical update.

The final value can be 12 because the read–modify–write sequences overlap.
Serial orders A-then-B and B-then-A both produce 14; therefore 12 is evidence of a lost update.
Protecting only the final store still permits both stale reads and does not repair the protocol.

Exclusion with no progress

Two processes use a variable `turn`. Process 0 remains in its noncritical section indefinitely while `turn==0`; process 1 wants to enter.
Which correctness property fails?

1. Strict alternation can ensure that both processes are not inside simultaneously, so mutual exclusion may hold.
2. The critical section is empty, yet process 1 is forbidden because the turn names a process that is not competing.
3. Selection therefore depends on irrelevant behaviour outside the critical section.

Progress/independence fails even though exclusion can hold.
A correct progress property would allow some waiting participant to enter when the section is free and the other participant is not competing.
A protocol can also provide progress but fail bounded waiting if one requester can be bypassed indefinitely.

Counting-semaphore trace and wake-up semantics

A semaphore starts at 2. Threads A and B complete wait; thread C then calls wait and blocks. A later signal occurs.
Track permits and state transitions.

1. A consumes one permit, leaving 1; B consumes the second, leaving 0.
2. C finds no permit and is placed in the semaphore's wait set.
3. The later signal transfers availability to a waiter, making one waiter eligible to complete wait. It does not guarantee that C runs at that instant; the scheduler controls dispatch.

Permit accounting and CPU scheduling are separate layers.
No execution may let A, B and C all consume one of only two initial permits before a signal.
Unless the API promises FIFO wake-up, a semaphore alone does not prove bounded waiting among several waiters.

Event semaphore versus ownership mutex

Producer A fills a result and consumer B must not inspect it early.
How can a binary-valued object encode the event, and why is it not an ownership mutex use?

1. Initialise `done=0` because the event has not occurred.

Worked example (continued)

2. B executes `wait(done)` before reading; A executes `signal(done)` only after publishing the result.
3. A signals an object that B waits on. This cross-activity release is valid for a semaphore event but would violate the owner-unlocks rule of an ownership mutex.

The same zero/one range can implement different contracts; ownership and event ordering must not be conflated.
Initialising `done` to one would let B pass before A publishes anything.
A resource pool with several identical units requires a counting semaphore rather than a binary event.

Exercise 5.5.1 — Critical boundary

For `x = x * 2 + 1`, which actions belong in one protected logical operation?

Worked solution

The expression contains a read, local arithmetic and a write. Protect the complete sequence: read the current x, compute $2x + 1$, then store the result before releasing exclusion. With $x = 3$, two serial updates should give $3 \to 7 \to 15$. If both threads read 3 before either writes, each computes 7 and the final value is only 7. Protecting the whole sequence makes the second thread read the first thread's completed value, restoring the required result 15.

Exercise 5.5.2 — Data race

Do two unsynchronised reads of immutable data form a data race?

Worked solution

Two reads of immutable, safely published data do not conflict: neither access changes the stored value. A C/C++ data race requires conflicting accesses with at least one write and insufficient ordering, subject to the language's atomic-access rules. The qualification "immutable" must hold throughout the reads. If a third thread writes the object concurrently, or initialization has not been safely published, a different analysis is needed. Under the stated assumptions both readers can observe the same stable value without exclusive access.

Exercise 5.5.3 — Mutex-like initial value

What initial permit count represents one free exclusive resource?

Worked solution

Initialize the semaphore to 1. The value denotes one available permit for the exclusive resource. A successful `wait` consumes it, changing availability from 1 to 0. A second participant reaching `wait` must wait until the first participant releases the permit. One corresponding `signal` makes it available again or admits a waiting participant. Initializing to 0 would block the first acquisition; initializing above 1 would allow concurrent entries. Every completed acquisition needs exactly one matching release in this use.

Exercise 5.5.4 — Event initial value

What initial value represents an event that has not yet occurred?

Worked solution

Initialize the event semaphore to 0 because no completion has yet been published. A consumer reaching `wait` before the event blocks. After producing the data, the producer calls `signal`. If a consumer is waiting, that signal permits its wait to complete; if the consumer arrives later, it consumes the stored permit. This handles both arrival orders. An initial value of 1 would allow the consumer to proceed before the producer has made the data ready.

Exercise 5.5.5 — Error path

What can happen if a thread leaves an error branch while retaining the only exclusion permit?

Worked solution

After the acquisition, the only permit has been consumed. Leaving through an error branch without releasing it leaves the semaphore with no available permit. Later callers block at `wait`, even though the protected object is idle. Follow every path from successful acquisition to function exit: each must pass through release. A single cleanup block or a scope-bound ownership guard can enforce that requirement. The repair must also leave the shared object in a consistent state before permitting another participant to enter.

Summary

A race occurs when correctness depends on uncontrolled timing. Critical sections protect invariants, and semaphores combine atomic state changes with blocking and wake-

up. The synchronisation boundary must cover every step of the logical operation that preserves the invariant.

5.5.9 Exercises

Exercise 5.5.6 — Lost update

In the abstract model with individually atomic loads and stores, construct an interleaving in which two read–compute–write increments produce only one increase.

Go to the solution →

Exercise 5.5.7 — Critical-section boundary

For balance = balance + amount, identify the complete critical section.

Go to the solution →

Exercise 5.5.8 — Strict alternation

Explain how strict alternation can block a process even when the critical section is free.

Go to the solution →

Exercise 5.5.9 — Semaphore initial value

What initial value should a binary semaphore used as a lock have, and what does each value mean?

Go to the solution →

Exercise 5.5.10 — Event ordering

Use a semaphore initially zero to ensure B consumes the data only after A finishes producing them.

Go to the solution →

Exercise 5.5.11 — Missing release

A worker takes the only permit of a binary semaphore used as a lock and then leaves through an error path without signalling it. Diagnose the resulting liveness failure, explain why the protected object being physically idle does not release the waiters, and give two repair strategies: one local control-flow repair and one API/design choice that makes this class of leak harder to create.

Go to the solution →

5.5.10 Solutions

Shared State, Critical Sections, and Semaphores

Solution 5.5.6

Use a shared counter $x = 0$ and private temporary registers a and b:

Step	Action	a	b	x
1	A reads x	0	–	0
2	B reads x	0	0	0
3	A computes $a + 1$	1	0	0
4	B computes $b + 1$	1	1	0
5	A writes a	1	1	1
6	B writes b	1	1	1

Both stores contain 1 because both computations used the original value 0. The required serial result is 2, so one update has been lost. A mutual-exclusion protocol around each complete read–modify–write makes the second read observe 1 and the second write produce 2. This table is an abstract interleaving model; ordinary unsynchronised conflicting accesses in C require a data-race analysis and cannot be justified by this table alone.

← Return to the exercise

Solution 5.5.7

Assume `amount` is private or already stable. Acquire exclusion before reading `balance`; compute the new value from that protected read; write the new balance; then release exclusion. For initial balance 100, A adding 30 and B adding 20 must produce 150 in either serial order. If A reads 100 and B also reads 100, stores of 130 and 120 can instead leave 120. Locking only the stores still permits those stale reads. Include any validity check, such as a withdrawal limit, in the same protected operation when its result controls the update.

← Return to the exercise

Solution 5.5.8

Let `turn=0`, so process 0 is entitled to enter next. Process 0 is doing unrelated work outside the critical section and never requests entry. Process 1 requests entry and repeatedly finds `turn!=1`. The section remains empty while process 1 waits. There is no competing participant whose use of the resource explains the delay. Strict alternation has coupled admission to process 0's later action of changing `turn`; this

Worked solution (continued)

violates the required progress property for a process that is the sole interested entrant.

← Return to the exercise

Solution 5.5.9

For a binary semaphore used by convention as a lock, initial value one means that one permit is available and zero means no permit is available. `wait` atomically consumes the permit or blocks; `signal` returns a permit or makes a waiter eligible to complete its wait. This convention does not by itself add mutex ownership: a generic semaphore need not require the signalling activity to be the one that acquired the permit.

← Return to the exercise

Solution 5.5.10

Use a semaphore `done` with initial value zero:

```
A: produce(data)
   signal(done)
B: wait(done)
   consume(data)
```

If B arrives first, its wait blocks at zero. A finishes producing and signals; B can then complete its wait and consume. If A finishes first, its signal leaves one available permit. B's later wait consumes that permit and proceeds immediately. In both traces the completed wait follows the signal, which follows production. The semaphore API must provide the required memory-ordering guarantees for the data. One signal serves one completed wait here; repeated production requires a protocol that accounts for multiple items and their storage.

← Return to the exercise

Solution 5.5.11

The semaphore count remains zero, so later `wait` operations can block indefinitely even though no thread is executing the protected code. The failure is a leaked synchronisation permit; depending on the surrounding dependency graph it can create an actual deadlock or an indistinguishable permanent stall. A local repair is to structure every exit path so release is guaranteed—for example a single cleanup block or language-level scope guard. An API/design repair is to pair an ownership mutex with a scope-bound guard, such as C++ `std::lock_guard`, that unlocks on ordinary scope exit and exception unwinding. A bare ownership mutex still requires explicit release; merely replacing a semaphore with `pthread_mutex_t`

Worked solution (continued)

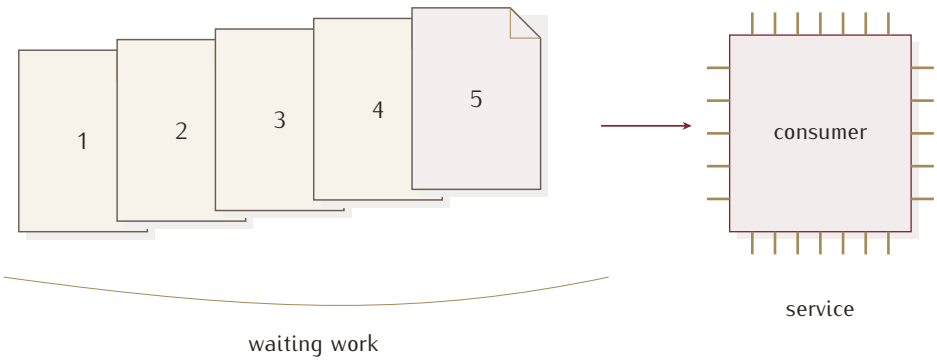
does not repair an early-return leak. C code can use an explicit cleanup path and, where cancellation is allowed, appropriate cancellation-cleanup handlers. All repairs must restore the protected invariant before releasing access.

← Return to the exercise

5.6 COUNTING SEMAPHORES AND CLASSICAL SYNCHRONISATION PROBLEMS

A producer has prepared an item, yet the queue may be full. A consumer wants an item, yet the queue may be empty. The protocol needs to reserve capacity and published items while protecting the metadata that locates them. Giving each semaphore permit a precise meaning makes its initial value and each operation derivable. The same method will explain admission policies for readers and writers [59, 1].

A bounded buffer mediates producers and consumers



The finite slots need both capacity accounting and protected metadata.

Figure 5.51: *Items occupy a finite container while producers and consumers run independently. The illustration motivates separate questions: whether a slot or item is available, and who may update the shared buffer structure at a given moment.*

We will use a small worker service as a recurring case. Producer threads receive jobs and place them in an in-memory queue; consumer threads remove jobs and process them. The queue has finite capacity because unbounded memory growth would merely move overload elsewhere. Several producers and consumers may operate concurrently, so the protocol must prevent both overflow/underflow and corruption of the queue's head, tail and slot state.

5.6.1 Deriving the bounded buffer from its invariants

Suppose a circular buffer contains N slots. At a stable point where no thread has reserved a slot or item and paused halfway through the protocol, every slot is either free or contains a published item. If F denotes the number of published items and E the number of free slots, then

$$F + E = N.$$

This equation is a property of the logical buffer state at that boundary. It immediately suggests two counting resources. A producer must consume one unit of free capacity before it inserts; a consumer must consume one published item before it removes. Therefore a semaphore representing free-slot permits begins at N, whereas one representing published-item permits begins at zero.

Those counts do not yet protect the queue metadata. Two producers that both possess a free-slot permit could still read the same tail index, write the same slot and advance the index incorrectly. Similarly, two consumers could corrupt a shared head index. Unless the queue uses a separately proven lock-free or partitioned algorithm, the metadata update needs mutual exclusion. A third synchronisation object therefore provides one permit for the metadata critical section.

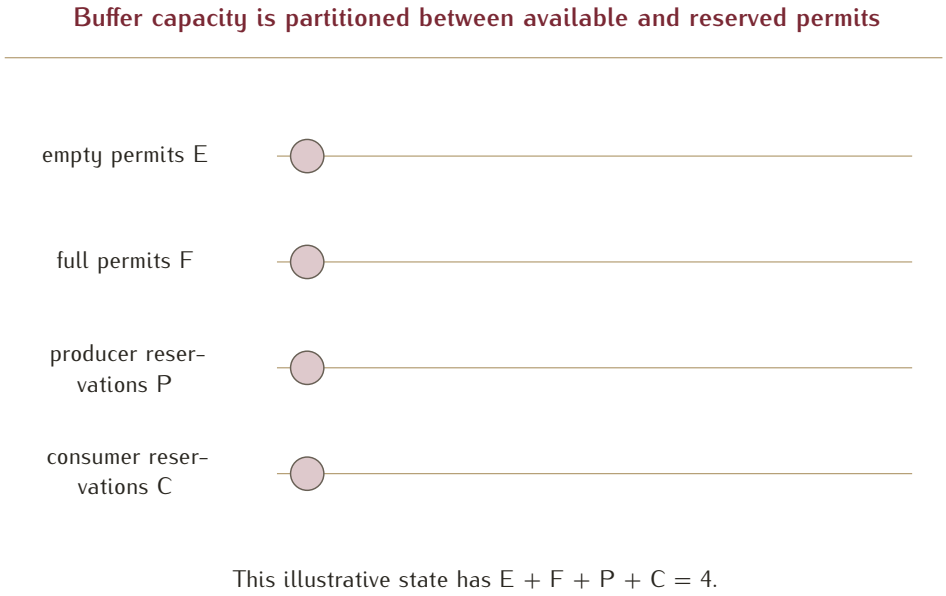


Figure 5.52: *Each unit of capacity belongs to one category in the permit-accounting model. Physical occupancy is tracked separately because insertion, removal and publication occur at different points in the protocol.*

The resulting initial state is not arbitrary:

Semaphore	Initial value	Permit meaning
`empty`	N	one unreserved free slot
`full`	0	one published, unreserved item
`mutex`	1	permission to modify shared queue metadata

A producer can now follow

```
produce item locally
wait(empty)          # reserve capacity
wait(mutex)
    insert item and update shared metadata
signal(mutex)
signal(full)         # publish one available item
```

while a consumer follows

```
wait(full)           # reserve one published item
wait(mutex)
    remove item and update shared metadata
signal(mutex)
signal(empty)        # return one free slot
consume item locally
```

The comments reveal the protocol more clearly than the semaphore names. The producer does not signal `full` merely because it "comes after" `mutex`; it signals only after the queue contains an item whose metadata has been made consistent and which may safely be claimed by a consumer. The consumer returns `empty` only after the slot has ceased to belong to the queue's occupied state.

The reservation steps also explain a subtle invariant. If a producer has completed `wait(empty)` but has not yet inserted its item, one capacity permit is no longer available even though the physical slot is still visually empty. Likewise, after a consumer completes `wait(full)` but before it removes the item, the item has been reserved and should not be counted as available to a second consumer. Consequently, the instantaneous numeric values of `full` and `empty` need not always equal the number of physically occupied and visibly empty slots while operations are in flight. Let P count producers between successful `wait(empty)` and `signal(full)`, and let C count consumers between successful `wait(full)` and `signal(empty)`. At boundaries between these abstract atomic operations, the conservation law is

$$E + F + P + C = N.$$

Each successful wait transfers one available permit into a reservation; each final signal transfers a reservation into the opposite available count. Insertion and removal change physical occupancy without changing this sum. Adding physical occupancy

again would double-count state already represented by a permit or reservation. The familiar $\texttt{full} + \texttt{empty} = N$ equation is exact at quiescent boundaries where no reservation is outstanding; it should not be used as a blind assertion at every machine instruction.

For example, if an eight-slot buffer contains three published items and no operation is in flight, `full=3`, `empty=5` and `mutex=1`. If a producer then successfully waits on `empty` but is preempted before acquiring `mutex`, the semaphore counts may read 3 and 4 even though the buffer still visibly contains three items. The "missing" slot is not lost: it is reserved by the producer. Thinking in terms of ownership of permits makes the state comprehensible.

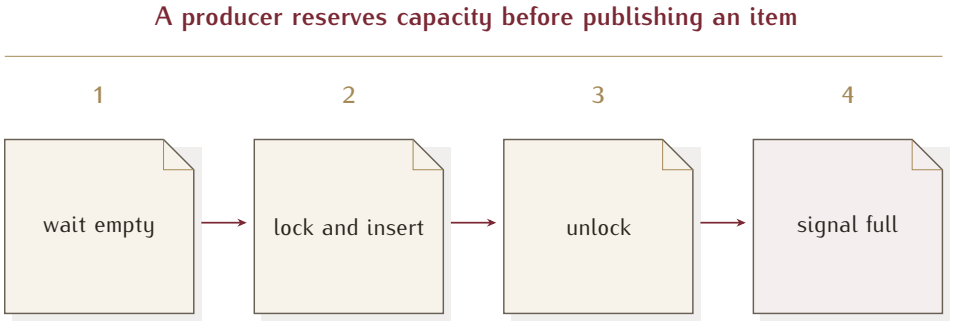


Figure 5.53: *The producer first reserves a slot, then changes the buffer under its metadata lock. Signalling full publishes an available item to consumers. The ordering maintains the capacity and availability invariants.*

An occupied slot already belongs to a consumer

Pause Figure 5.54 immediately after the consumer's `wait(full)`. Can a second consumer reserve an item? Can a second producer reserve capacity?

Solution. The state is $E = 1, F = 0, P = 0, C = 1$, with $O = 1$. The existing item belongs to the first consumer's reservation, so a second consumer must wait for another published permit. A producer can successfully wait on `empty`; the state then becomes $E = 0, F = 0, P = 1, C = 1$ and occupancy remains one until a protected insertion or removal occurs. Both sums equal two. The mutex subsequently orders metadata changes; the permit counts alone do not select a queue index.

A finite capacity introduces *backpressure*. If consumers cannot keep up with producers, an unbounded queue allows pending work and memory use to grow without limit, delaying the eventual failure rather than controlling overload. A bounded queue forces the producer side to wait once the system has accepted as much outstanding work as the chosen capacity permits. Capacity therefore becomes part of system policy: too small

Two slots; one producer and one consumer

completed step	E	F	P	C	O
initial state	2	0	0	0	0
producer: wait(empty)	1	0	1	0	0
producer: insert	1	0	1	0	1
producer: signal(full)	1	1	0	0	1
consumer: wait(full)	1	0	0	1	1
consumer: remove	1	0	0	1	0
consumer: signal(empty)	2	0	0	0	0

$E + F + P + C = 2$ in every row; O counts physical occupancy.

Figure 5.54: *A capacity permit moves between availability and reservation.* P *and* C *count in-flight producer and consumer reservations. Insertion and removal occur under the metadata mutex; its acquisition and release are omitted from these snapshots because they do not change the displayed counts. Occupancy* O *is shown separately and is excluded from the permit sum.*

a buffer may leave resources idle during ordinary bursts, while too large a buffer can increase latency and memory pressure before backpressure reaches the source.

With several producers and consumers, the semaphore counts scale naturally because they represent quantities rather than identities. Two producers may simultaneously hold two different `empty` reservations while waiting their turn for `mutex`; two consumers may similarly reserve distinct available items. What must remain impossible is for two participants to mutate the same queue metadata concurrently unless the queue algorithm was explicitly designed to permit it. The counting semaphores enable concurrency outside the metadata critical section, while the mutex serialises only the portion whose invariant requires exclusive modification.

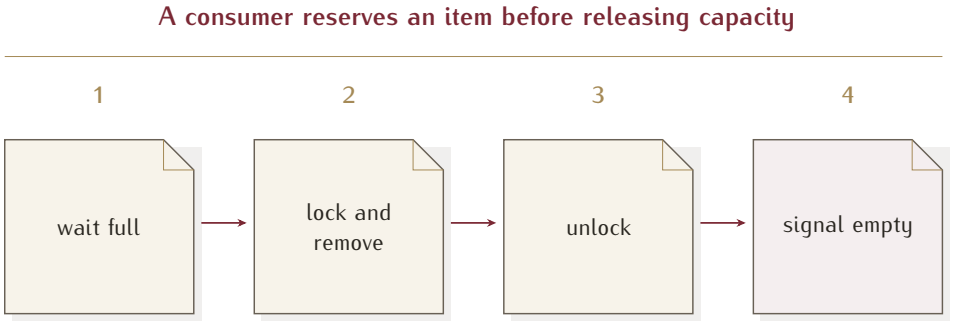


Figure 5.55: *The consumer obtains permission to remove one item, updates shared buffer state under the lock and finally returns an empty-capacity permit. This complements the producer's protocol.*

The protocol also suggests how to reason about cancellation or failure. If a producer consumes an `empty` permit and then abandons the operation without either publishing an item or returning the permit, the system has leaked capacity. If a consumer reserves a `full` permit and fails before completing/remediating the removal, it can similarly strand logical state. Robust implementations therefore need cleanup rules around reservations just as they need release rules around mutexes. Semaphore counts represent obligations as well as opportunities: once a permit has been consumed, some path must complete or undo the corresponding state transition.

5.6.2 Why acquisition order is part of the proof

It is tempting to regard the order of the two waits as a coding detail because both must eventually succeed before a producer can modify the buffer. The order is in fact a liveness property.

Suppose a producer executes

```
wait(mutex)
```

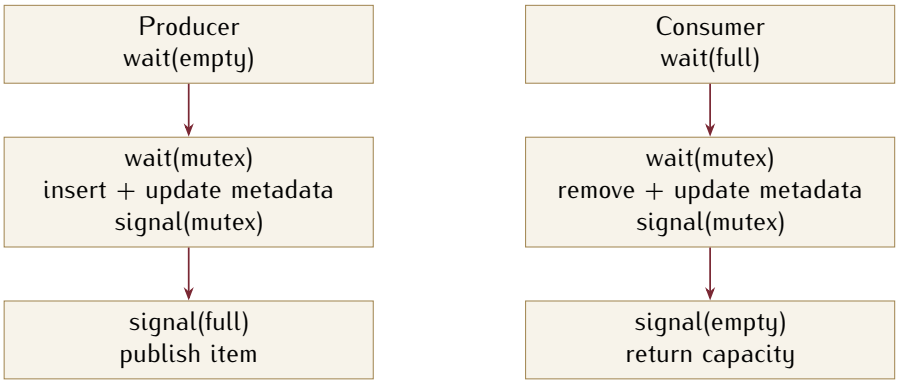


Figure 5.56: *Reserve a condition permit before entering the metadata critical section. The two middle boxes use the same mutex and cannot overlap. Initially: empty=N, full=0, mutex=1. Permits reserved by in-flight operations must be included when checking capacity conservation.*

```
wait(empty)
```

and the buffer is already full. The producer obtains `mutex`, discovers that no `empty` permit exists, and blocks while still holding exclusive access to the queue metadata. A consumer could create an empty slot by removing an item, but it also needs `mutex` to perform that removal. The consumer therefore waits for the producer, while the producer waits for a condition that only a consumer can create. The same primitives with the wrong acquisition order have formed a deadlock.

The standard protocol avoids that dependency by waiting for the condition permit *before* entering the metadata critical section. A producer that finds the buffer full blocks without holding `mutex`, so a consumer remains able to enter the queue, remove an item and return an `empty` permit. Similarly, a consumer waits for `full` before taking the metadata lock.

Waiting for capacity while holding the metadata lock can close a cycle



Each arrow points to a dependency needed for progress; space depends on consumer C.

Figure 5.57: *The graph depicts dependencies in a full-buffer situation with an incorrect acquisition order. The producer waits for capacity while holding the lock needed by the consumer. Reserving capacity before acquiring that lock avoids this particular cycle.*

This rule should not be reduced to "always take counting semaphores before mutexes". Synchronisation protocols can involve many resources, and the correct ordering follows from the dependency graph. The relevant question is whether a blocked participant holds a resource that another participant needs in order to make the awaited condition true. Chapter 5.7 will generalise this reasoning into deadlock conditions and global resource ordering.

5.6.3 Waiting for a predicate under a mutex

The worker queue maintains two availability counts and a distinct lock for its metadata. `full` and `empty` represent whether a logical condition can be satisfied: an item exists to be consumed, or capacity exists to be reserved. `mutex` represents exclusive permission to manipulate metadata whose invariant could be corrupted by overlapping updates. These are different statements.

A program could use a perfectly accurate `full` count and still be wrong if two consumers remove from the same head position. Conversely, a mutex could serialize every queue operation perfectly and still be insufficient if a consumer acquires the mutex when the buffer is empty and then spins or blocks while preventing producers from entering. Correct synchronisation often requires both *condition synchronisation* and *mutual exclusion*, with an order that allows the condition to change.

A condition variable lets the program wait on a predicate over protected state. For a buffer, that predicate can be expressed directly in terms of the item count.

The same bounded-buffer predicate can be expressed with a mutex and condition variables rather than counting semaphores. In that style, the shared state itself—for example `count`—is the source of truth. A producer locks the mutex, waits while `count==N`, inserts an item, changes `count`, signals that the non-empty condition may now hold, and unlocks. A consumer waits while `count==0` and signals non-full after removing an item. The wait operation must release the mutex atomically with blocking and reacquire it before returning; otherwise a thread could sleep while still preventing the participant that changes the predicate from entering.

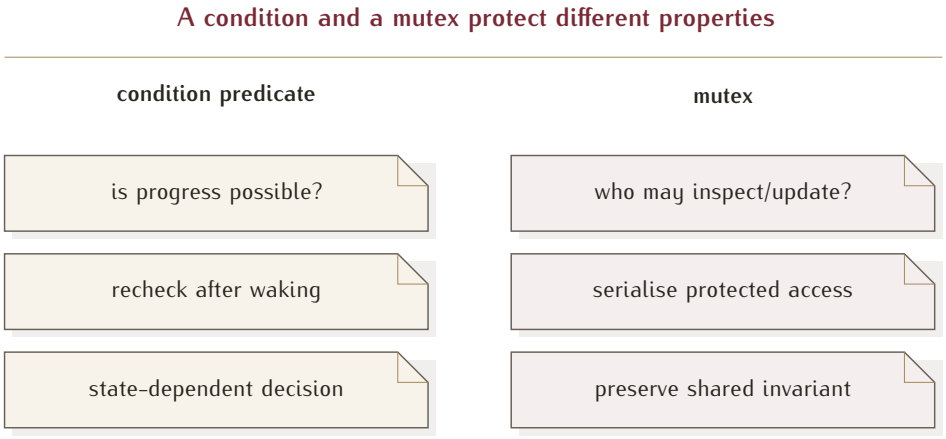


Figure 5.58: *Mutual exclusion controls access to the shared state. Condition synchronisation arranges waiting until a state-dependent predicate permits progress. Correct use coordinates both without assuming every wake-up guarantees the predicate.*

Condition waits are normally written in a loop rather than as a one-time `if`. After waking and reacquiring the mutex, the waiter must test the current predicate. Another thread may already have consumed the available item. Another thread can run first and change the state again, and some interfaces permit wake-ups for reasons that do not correspond to the exact application predicate. Rechecking under the mutex therefore turns a notification into a proof based on current shared state. This is conceptually close to the reservation logic of semaphores, but the state accounting lives in a different place.

Choosing between semaphores and mutex/condition-variable protocols is not primarily a matter of speed. Semaphores are natural when the count itself has an independent permit meaning; condition variables are natural when waiting is best expressed as a predicate over protected shared state. Both can be used incorrectly if the invariant is unclear.

5.6.4 Readers and writers: safety does not choose a policy

The readers–writers problem changes the shape of the invariant. Several readers may safely observe an object at the same time provided no writer is modifying it, while a writer requires exclusive access with respect to both readers and other writers. The basic safety rule is therefore:

- zero or more readers may coexist;
- at most one writer may be active;
- readers and a writer may not be active together.

What the rule does *not* specify is who should be admitted when readers and writers are both waiting. That is a policy question.

A classic reader-preference solution keeps a reader count protected by a mutex. The first arriving reader acquires the resource on behalf of the reader group; subsequent readers join while the group remains active; the last reader releases the resource. Writers acquire the resource directly.

```
initially: read_count = 0, mutex = 1, resource = 1

reader:
    wait(mutex)
    read_count = read_count + 1
    if read_count == 1:
        wait(resource)
    signal(mutex)

    read shared object

    wait(mutex)
    read_count = read_count - 1
    if read_count == 0:
        signal(resource)
    signal(mutex)
```

```
writer:
    wait(resource)
    write shared object
    signal(resource)
```

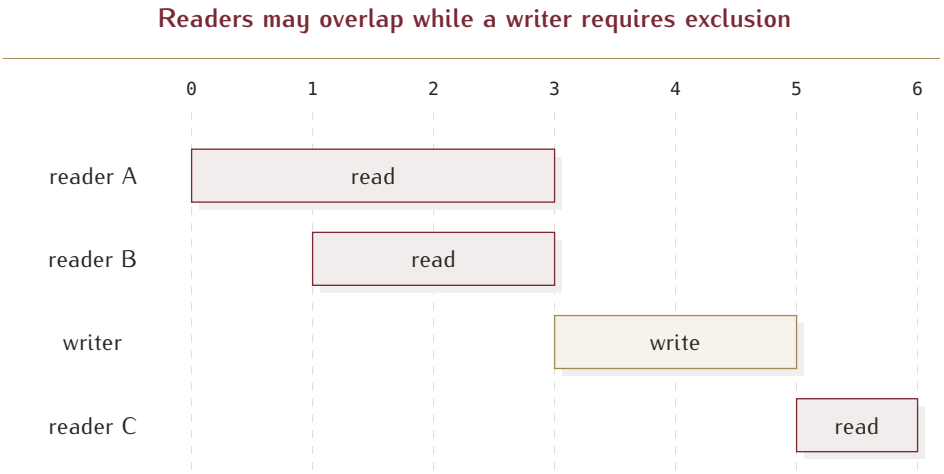


Figure 5.59: *Overlapping readers preserve read-only access, while the writer occupies an interval with no concurrent reader or writer. Safety alone does not decide which waiting participant should enter next.*

Here `mutex` protects `read_count`, while `resource` excludes writers during an admitted read phase. The first reader acquires `resource` and the last reader releases it. Since two different threads can perform those operations, use a semaphore or another primitive that supports group ownership. During entry, the first reader may increment the count to one and then block behind an active writer while retaining `mutex`. Thus a positive count alone does not prove that a reader is already accessing the object. After that first acquisition completes, the group holds the resource until its last departure.

The protocol is safe under its assumptions, but it can starve a writer. If a stream of readers continues to arrive before the active reader count falls to zero, `resource` may remain continuously owned by the reader group. A writer can therefore wait indefinitely while the system as a whole continues to make progress. This is not a deadlock; it is a policy-induced liveness failure.

A writer-preference design changes admission so that once a writer is waiting, newly arriving readers do not indefinitely extend the current reader epoch. A fair design can queue arrival order or use a turnstile-like mechanism to bound bypassing. Each policy trades something: maximum read concurrency, writer latency, reader latency, implementation complexity or fairness guarantees. There is no single readers–writers algorithm that is "the solution" independently of the workload and required progress properties.

State	May proceed concurrently	Must wait
reader phase	one or more admitted readers	every writer
writer phase	one writer	all readers and other writers
idle	one writer or a reader group	other queued contenders

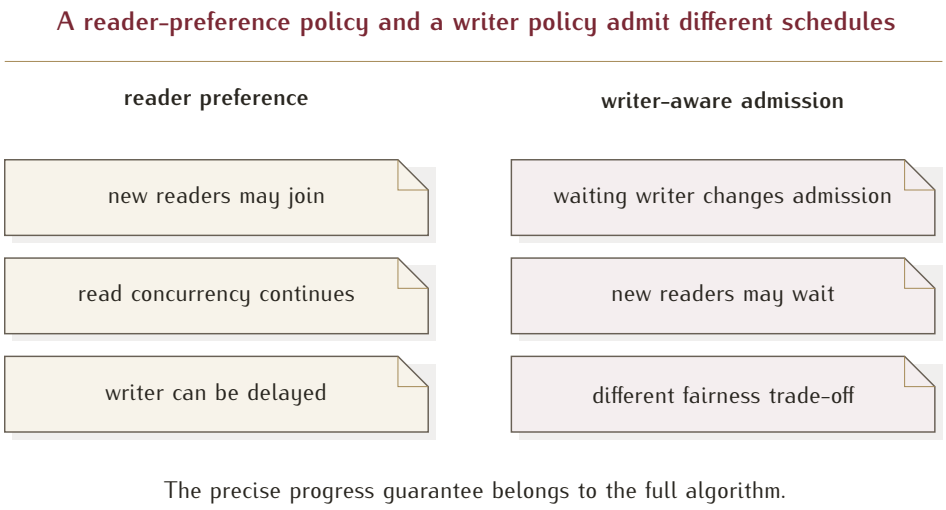


Figure 5.60: *Admission policy determines whether a stream of readers can postpone a writer indefinitely. The mutual-exclusion invariant remains necessary in either design, while fairness requires a separate argument.*

Fairness policies can also be stated in terms of admission epochs. A reader-preference design allows arriving readers to join an active reader phase even while a writer waits; a writer-preference design eventually closes the reader gate once a writer is queued; a fair design places both classes behind a common ordering mechanism so that neither can be bypassed indefinitely. The safety invariant—no reader overlaps a writer and at most one writer runs—is unchanged. What changes is the liveness contract. This is a recurring theme in concurrent programming: once safety has been proved, there may still be several legal policies with very different latency behaviour.

In practice, the right policy depends on workload and semantics. A read-mostly configuration database may value broad reader concurrency, while a writer that performs configuration updates may require a bounded time before its change becomes visible. A queue used in an interactive service may prefer approximate arrival fairness, whereas a real-time subsystem may need priority-aware admission. Treating readers–writers as one memorised algorithm hides these decisions.

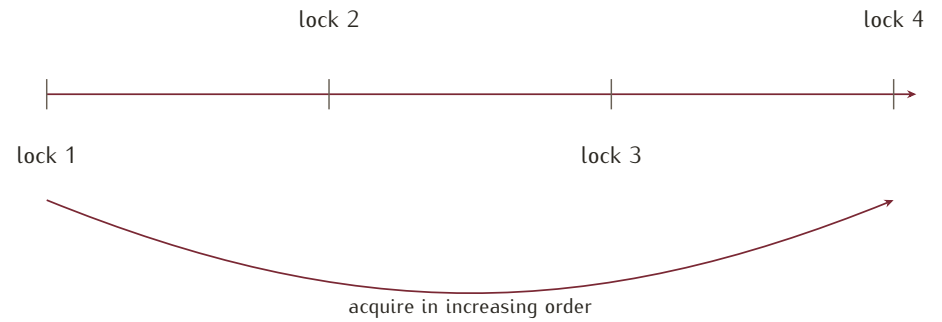
5.6.5 From local invariants to global resource order

A reliable way to design either classical problem is to work from meaning rather than code. First state the shared invariant and what each synchronisation object represents. Then identify which updates must be mutually exclusive and which predicates can force waiting. Derive initial permit counts from the initial state. After writing the protocol, examine both safety and liveness: can any interleaving expose an inconsistent state, can a blocked participant eventually be awakened, can one participant starve, and can circular resource dependencies arise?

When one operation needs several resources, its waits can depend on other holders. Dining Philosophers gives a finite example in which every local acquisition is valid and the combined state prevents all participants from completing.

A concrete version of the Dining Philosophers pattern makes the circular dependency visible. Number five resource locks from 0 to 4 and let participant i need locks i and $(i+1) \bmod 5$. If every participant acquires its first lock before requesting its second, an execution can reach a state in which all five first locks are held and every participant waits for the next. No amount of scheduler fairness can repair the state: each blocked thread requires a release operation that another blocked thread can execute only after its own wait completes. The failure is therefore a resource dependency rather than a shortage of processor time.

A global resource order removes circular acquisition



Every participant must follow the same order whenever it acquires multiple locks.

Figure 5.61: *Strictly increasing acquisition order prevents a dependency cycle composed of those ordered acquisitions. The proof depends on universal compliance and does not automatically settle unrelated waits or resource types.*

One remedy is to impose a global ordering on resource acquisition, so that every participant requests lower-ranked resources before higher-ranked ones. Another is to limit admission or redesign the operation so that resources are acquired as a set. The next chapter develops why such rules work by analysing the structural conditions for

deadlock, and it will also separate deadlock from the starvation already seen in readers–writers and from livelock, where participants remain active yet continually defeat one another's progress.

Concept check

1. How do the meanings of `full`, `empty` and `mutex` differ, and why can no one of them replace the other two in a conventional bounded buffer?
2. Why can $\texttt{full} + \texttt{empty} = \mathrm{N}$ fail as a literal instantaneous count while reservations are in flight even though no capacity has been lost?
3. How does acquiring `mutex` before waiting for `empty` create a circular dependency when the buffer is full?
4. Why can a reader-preference solution be safe yet still fail a liveness/fairness requirement for writers?

Solutions to the review questions

1. `full` counts published items available for consumption; `empty` counts slots available for reservation; `mutex` protects the shared queue updates. Availability does not serialize an update, and exclusion alone cannot create an item or a free slot. The three roles require separate state in this conventional protocol.
2. After a producer consumes an empty-slot permit, it can hold a reservation before publishing a full-item permit. A consumer can similarly hold an item reservation before returning the empty permit. The complete capacity invariant counts available permits together with these in-flight reservations; omitting the latter can make the instantaneous sum appear smaller than N.
3. With a full buffer, a producer that first acquires `mutex` and then waits for `empty` holds the mutex while waiting for a consumer to free a slot. The consumer needs that same mutex to remove an item. Each participant waits for progress that the other cannot make, creating the cycle.
4. A reader-preference protocol can exclude writers correctly while allowing new readers to join an existing reader group indefinitely. A waiting writer can then starve even though every active reader finishes. A fairness rule must limit such bypassing if writer progress is required.

5.6.6 Worked problems

Bounded-buffer values and an in-flight reservation

A capacity-6 buffer contains two items. No operation is in flight, then one producer completes `wait(empty)` but has not inserted yet.
Track `full`, `empty` and physical occupancy.

1. At the quiescent point, `full=2` and `empty=4`; their sum is the capacity.
2. The producer reserves a slot, so `empty` becomes 3 while `full` remains 2.
3. Physical occupancy is still two until insertion; consequently `full+empty=5` during this phase.

The simple equality with capacity describes quiescent boundaries, while the full protocol invariant includes reservations in flight.
Count occupied items, unreserved empty slots and the one reserved producer slot: $2+3+1=6$.
A consumer reservation similarly decreases `full` before physical removal.

Why condition-before-mutex order matters

A producer executes `wait(mutex)` followed by `wait(empty)` on a full buffer.
Construct the deadlock.

1. The producer acquires `mutex`; because `empty=0`, it then blocks while retaining `mutex`.
2. A consumer could create an empty slot only by removing an item.
3. The consumer must acquire `mutex` before removal, but the sleeping producer holds it. Neither side can reach the signal needed by the other.

Waiting on a condition while holding the lock required to change that condition creates a circular dependency.
Reorder the producer to `wait(empty)` then `wait(mutex)`; on a full buffer it sleeps without obstructing the consumer.
The same reasoning applies symmetrically if a consumer holds the mutex while waiting on `full`.

First and last reader protocol

Three readers enter a reader-preference implementation while no writer is active.
How many times is the resource semaphore acquired and released by the reader group?

1. The first reader changes `read_count` from 0 to 1 and acquires `resource`.

Worked example (continued)

2. The second and third readers increment the protected count but do not acquire separate resource permits.
3. Departures decrement the count. Only the last departure, which changes 1 to 0, signals `resource`.

The reader group acquires once at the first entry and releases once at the last exit. From the first reader's completed resource acquisition until the last reader's release, writers cannot enter. A positive count during the first reader's pending acquisition is an intermediate entry state, as described above.
The small `mutex` protects only `read_count`; holding it across the actual read would unnecessarily serialize readers.

Safety versus writer fairness

A writer is waiting while a continuous stream of readers arrives under reader preference.
Why can safety hold while liveness fails, and what policy can repair it?

1. Every active read phase still excludes writers, so no unsafe read/write overlap occurs.
2. New readers can join before the count reaches zero, preventing the waiting writer from ever acquiring `resource`.
3. A fair turnstile or queued admission gate can stop new readers from bypassing an already waiting writer indefinitely.

Mutual-exclusion safety does not imply starvation freedom; admission policy supplies fairness.
A proposed fix that merely renames the semaphore but leaves new readers able to bypass the writer has not addressed starvation.
Writer-preference may instead delay readers; policy must match the desired latency and fairness contract.

Exercise 5.6.1 — Initial values

Give `full`, `empty` and `mutex` for an empty capacity-9 buffer.

Worked solution

An empty buffer has zero consumable items, so initialize `full=0`. All nine slots are free, so `empty=9`. One participant may modify protected metadata at a time, so `mutex=1`. The capacity accounting starts with $\texttt{full} + \texttt{empty} = 9$. In a detailed

Worked solution (continued)

concurrent trace, acquired but not yet published reservations must also be counted when checking this invariant.

Exercise 5.6.2 — Publish order

Should a producer signal `full` before or after publishing the inserted item?

Worked solution

The producer must store the item and make the associated queue metadata consistent before signalling `full`. A completed consumer wait on `full` must correspond to a readable published item. Signalling early could wake a consumer while the selected slot still contains old data or while the tail/count fields are inconsistent. The synchronization API must also provide the required visibility ordering for the data.

Exercise 5.6.3 — Multiple producers

Does `empty` alone make a shared tail index safe for several producers?

Worked solution

The `empty` semaphore counts available capacity. With capacity greater than one, several producers can obtain permits concurrently. They could then read the same tail index and overwrite one slot unless metadata updates are protected. Use the specified mutex or an appropriate atomic queue algorithm around those shared updates. Capacity reservation and exclusive metadata modification enforce different conditions, and both must hold.

Exercise 5.6.4 — Writer rule

Can a reader coexist with an active writer in the classical readers–writers specification?

Worked solution

The classical specification permits multiple readers together only while no writer is active. A writer must exclude both readers and other writers so that readers see a consistent version of the shared object. Allowing a reader during a multi-step writer update could expose a mixture of old and new fields. More permissive algorithms require a different contract, such as versioned snapshots.

Exercise 5.6.5 — Fair admission

Name one mechanism that prevents an endless stream of new readers from bypassing a waiting writer.

Worked solution

Use a fair admission queue or turnstile that prevents newly arriving readers from continually overtaking a waiting writer. Once the writer is queued, later entrants must respect the admission policy. Existing readers finish, the writer gains exclusive access, and queued readers follow. The starvation claim depends on the queue's fairness guarantees and finite critical sections; an unspecified semaphore wake order alone is insufficient.

Summary

Counting semaphores encode quantities, binary semaphores protect indivisible updates, and classical problems combine both. Correctness follows from explicit invariants and acquisition order; performance and fairness require a separate policy analysis.

5.6.7 Exercises

Exercise 5.6.6 — Buffer values

A bounded buffer has capacity 10 and contains 4 items. Give full, empty and mutex values when no operation is in flight.

Go to the solution →

Exercise 5.6.7 — Producer order

Why must a producer wait for empty before acquiring mutex in the standard solution?

Go to the solution →

Exercise 5.6.8 — Consumer trace

Trace the semaphore values when one consumer removes an item from a two-item, capacity-five buffer.

Go to the solution →

Exercise 5.6.9 — Invariant

For a bounded buffer of capacity N, derive the relationship among published items, free slots and reservations. Explain why the familiar `full + empty = N` equality

Exercise 5.6.9 — Invariant (continued)

is exact only at quiescent points, and show how a producer that has already completed `wait(empty)` but not yet published its item changes the accounting.

Go to the solution →

Exercise 5.6.10 — Readers-writers safety

Why can two readers enter together but a reader and writer cannot?

Go to the solution →

Exercise 5.6.11 — Writer starvation

A reader-preference readers–writers implementation allows new readers to join while at least one reader is active. Explain the starvation trace for a waiting writer, then propose a gate/turnstile change that prevents new readers from indefinitely bypassing a queued writer. State what fairness property the change adds without claiming that it changes the basic safety invariant.

Go to the solution →

5.6.8 Solutions

Counting Semaphores and Classical Synchronisation Problems

Solution 5.6.6

With no operation in flight, `full` counts stored items and `empty` counts unoccupied slots. Thus `full=4` and `empty=10-4=6`. The exclusion semaphore is available, so `mutex=1`. The invariant $\texttt{full} + \texttt{empty} = 10$ holds in this quiescent state. During an operation, a thread may have reserved a permit before updating the buffer, so the visible semaphore values need interpretation together with outstanding reservations.

← Return to the exercise

Solution 5.6.7

Take a full one-slot buffer with $\texttt{empty} = 0$, $\texttt{full} = 1$ and $\texttt{mutex} = 1$. A producer that takes the mutex first leaves $\texttt{mutex} = 0$ and then sleeps on empty. A consumer can reserve the published item, but its wait on the mutex blocks removal. The producer now needs a slot that the consumer cannot free. With the prescribed order, the producer waits on empty while leaving the mutex available. The consumer can remove the item, release the mutex and signal empty, allowing the producer to proceed.

← Return to the exercise

Solution 5.6.8

Assume no other operation is in flight. Track (F, E, M), where M is the mutex permit, and physical occupancy O:

Event	(F, E, M)	O
Initial	$(2, 3, 1)$	2
`wait(full)`	$(1, 3, 1)$	2
`wait(mutex)`	$(1, 3, 0)$	2
Remove item	$(1, 3, 0)$	1
`signal(mutex)`	$(1, 3, 1)$	1
`signal(empty)`	$(1, 4, 1)$	1

Between the first and last semaphore operations, $C = 1$ records the consumer reservation. Hence $F + E + C = 1 + 3 + 1 = 5$ even before and after physical removal. At completion $C = 0$ and $F + E = 5$ again.

← Return to the exercise

Solution 5.6.9

Let F and E be the currently available full and empty permits. Let P count producer reservations from successful `wait(empty)` through the corresponding `signal(full)`, and C consumer reservations from successful `wait(full)` through `signal(empty)`. Capacity conservation is

$$F + E + P + C = N.$$

A producer wait changes (E, P) to $(E - 1, P + 1)$; its final publication changes (P, F) to $(P - 1, F + 1)$. A consumer wait changes (F, C) to $(F - 1, C + 1)$; its final release changes (C, E) to $(C - 1, E + 1)$. Each transition preserves the sum. Queue insertion and removal leave these four abstract counts unchanged.

For $N = 10$ with four items and no in-flight operation, $(F, E, P, C) = (4, 6, 0, 0)$. A producer reservation gives $(4, 5, 1, 0)$ while occupancy is still four. Insertion raises occupancy to five; publication then gives $(5, 5, 0, 0)$. At quiescence $P = C = 0$, so the familiar $F + E = N$ follows. The counters represent available capacity and outstanding obligations; physical occupancy must not be added as a fifth term.

← Return to the exercise

Solution 5.6.10

Suppose the protected object consists of a length and a matching payload. Two readers can inspect that stable pair at the same time. During a writer's update, a reader could observe the new length with the old payload and interpret an inconsistent record. Exclusive writer admission makes the update indivisible to admitted readers. The reader count permits an entire read phase to share one resource reservation.

← Return to the exercise

Solution 5.6.11

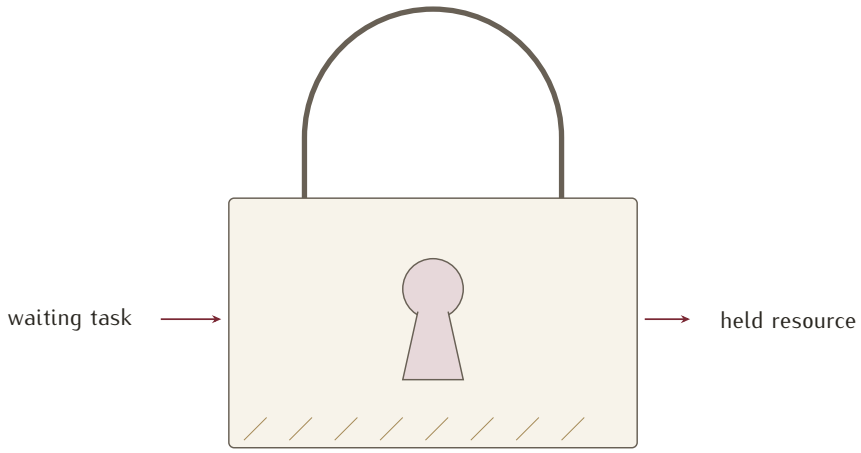
In a reader-preference design, reader R1 can hold the shared-read phase while R2 arrives; before the reader count reaches zero, R3 can join, then R4, and so on. The writer remains queued because the exclusive resource lock is released only when the last reader leaves. A common repair adds a service queue or turnstile that both readers and writers must pass: once a writer reaches the gate, later readers wait behind it rather than extending the current read phase indefinitely. Mutual exclusion between writers and readers was already the safety property; the gate adds bounded bypass if it enforces FIFO or an equivalent fair admission rule, critical sections terminate, and admitted threads eventually receive CPU time. An unspecified semaphore used as a gate would not establish that guarantee by itself.

← Return to the exercise

5.7 DEADLOCK, STARVATION, AND LIVELOCK

A producer holds the queue's mutex while waiting for free capacity. A consumer could return capacity, but first needs that mutex. Each waits for an action that the other cannot now perform. Following these dependencies exposes deadlock even when every individual lock operation is valid. We will use the same approach to distinguish a circular wait from starvation and from activity that repeatedly fails to make progress.

A held resource can become part of a dependency cycle



Progress depends on who can perform the release that another task requires.

Figure 5.62: *A lock alone does not explain deadlock. The crucial issue is the chain of dependencies among participants that hold resources and wait for others. The later allocation graphs make that chain explicit.*

That example belongs to a broader family of liveness failures that must be distinguished carefully. In *deadlock*, a set of participants is permanently waiting on dependencies that the same set cannot resolve. In *starvation*, one participant can wait indefinitely even though other participants continue to make progress; reader-preference readers–writers can exhibit this when writers are continually bypassed. In *livelock*, participants remain active and repeatedly change state, but their reactions to one another prevent useful progress—for example two agents that continually back off and retry in synchrony. Similar symptoms at the user interface can therefore require very different remedies [59, 1, 57].

5.7.1 Deadlock as a dependency structure

Consider two threads and two non-shareable locks:

```
thread T1:                  thread T2:
    wait(A)                     wait(B)
    wait(B)                     wait(A)
    critical work               critical work
    signal(B)                   signal(A)
    signal(A)                   signal(B)
```

If T1 acquires A while T2 acquires B, T1 waits for B and T2 waits for A. Neither can reach the release operation that would satisfy the other's wait. The resource dependencies contain a cycle that prevents either thread from making progress.

A *resource-allocation graph* distinguishes participants from resource types and represents allocation/request relationships explicitly. With blocking requests, non-preemptable resources, no alternative completion path and one instance per resource type, a directed cycle is sufficient for deadlock: following the cycle returns to a resource that cannot be released until the cycle has already been broken. With multiple instances of a resource type, a cycle is not by itself sufficient. Another instance may become available through a participant outside the apparent cycle, allowing one blocked request to complete and the dependency chain to unwind. In that setting we need more information about counts, allocations and remaining demands.

A resource-allocation cycle can express deadlock



One instance per resource: R1 held by P1, R2 held by P2; each requests the other.

Figure 5.63: *Assignment edges run from resource to process, and request edges run from process to resource. With one instance of each resource and both waits unresolved, neither participant can perform the release needed by the other.*

To construct the graph from a trace, record the owner of each resource and the outstanding request of each blocked participant. Follow the edges from a blocked process until they reach a possible release or return to a previously visited process.

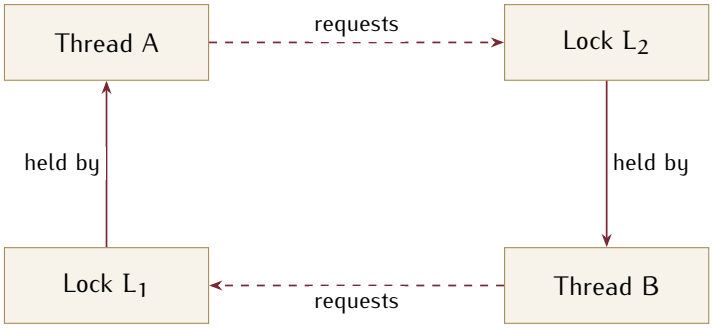


Figure 5.64: *A resource-allocation cycle with one instance of each lock. A holds* L_1 *and waits for* L_2*; B holds* L_2 *and waits for* L_1*. With blocking waits, no forced release and no alternative completion path, neither can advance. For resource types with multiple instances, a cycle alone is not sufficient proof.*

5.7.2 The Coffman conditions and what prevention means

A classical resource deadlock requires four structural conditions to hold simultaneously:

1. *mutual exclusion*: at least one required resource cannot be used by all participants simultaneously;
2. *hold and wait*: a participant can retain resources while requesting additional resources;
3. *no preemption*: a held resource remains with its owner until the owner releases it;
4. *circular wait*: participants form a cycle in which each waits for a resource held by the next.

These conditions are jointly necessary for the class of deadlocks they describe. Their presence does not imply that every execution will deadlock; it means that the structural possibility has not been ruled out. Deadlock *prevention* works by designing the system so that at least one condition cannot arise.

Breaking mutual exclusion is possible only for resources whose use can genuinely be shared or virtualised. A read-only object may be replicated; a printer physically producing one indivisible job cannot necessarily be shared in the same way, although spooling can transform direct printer ownership into ownership of a queue entry. Breaking hold-and-wait can require participants to request all needed resources together or release current resources before asking for more, but either rule can reduce utilisation or make dynamic programs awkward. Allowing preemption works for state that can be saved and restored or recomputed, but forcibly taking a mutex-protected data structure from a thread halfway through an update is not generally meaningful.

Circular wait is often the most convenient condition to attack for locks. Assign locks a stable strict total order and require each participant to acquire them only in increasing order. A class hierarchy needs a secondary order if several instances of the same class may be held together; reacquiring a non-recursive lock already held by the same thread must also be excluded. If A has rank 10 and B rank 20, code may acquire A then B but may not acquire B and later request A. Suppose a wait cycle nevertheless existed. Moving from each held resource to the next requested resource around the cycle would require the ranks to increase strictly at every step; returning to the starting point would then require a rank to be both greater than and equal to itself, which is impossible. The ordering rule removes the possibility of a cycle rather than attempting to detect one after threads are blocked.

The argument also reveals the practical difficulty. A global order must cover every relevant acquisition path, including error handling and rarely executed code. Dynamically created objects need a stable ordering rule, perhaps based on object identifiers, address order under carefully defined conditions, or hierarchy. A documented rule that half the program ignores is not a prevention strategy.

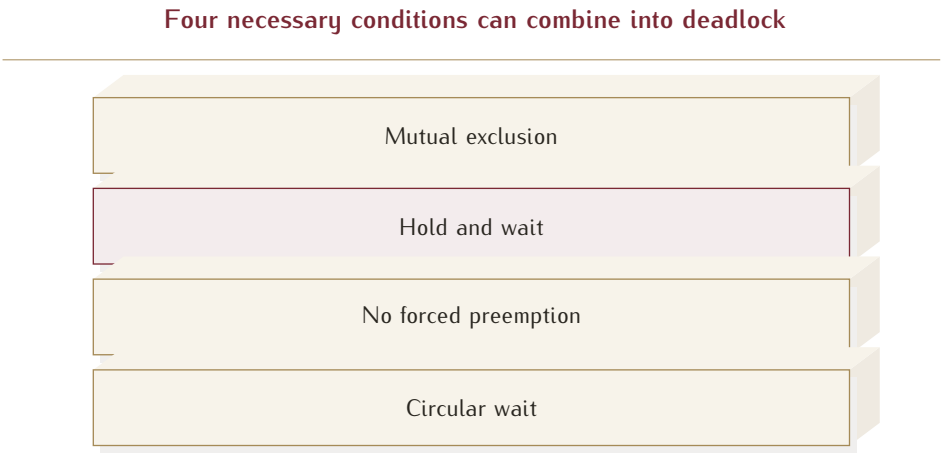


Figure 5.65: *The conditions describe a dependency structure, not four independent predictions of failure. A prevention policy changes the resource protocol so the complete combination cannot arise within its scope.*

Prevention rules also interact with software structure. A common convention is to associate the lock order with abstraction layers: a high-level object may acquire lower-level implementation locks, but low-level code may not call back into an operation that tries to acquire a higher-level lock. This is one reason callbacks while holding internal locks are dangerous. The callback may enter code outside the lock designer's control and acquire resources in an order that closes a cycle invisible in the local function. Re-

leasing the lock before invoking extensible code, documenting permitted call directions, or moving work to a later phase can make the dependency structure easier to prove.

Try-lock protocols provide another option when holding one resource while blocking for another is undesirable. A thread can acquire A, attempt B without sleeping, and if B is unavailable release A before retrying. This breaks permanent hold-and-wait, but it does not automatically guarantee useful progress: several threads can repeatedly collide, release and retry, turning a potential deadlock into livelock or starvation. Backoff and fairness policy may be needed, and the algorithm still requires a proof that partial work can be safely abandoned before retrying.

Resource preemption is similarly semantics-dependent. CPU time is naturally preemptable because architectural context can be saved and resumed. A memory page may sometimes be reclaimed after its contents are written elsewhere. A mutex-protected critical section generally cannot be "preempted" in the Coffman sense merely because the scheduler preempts the thread: the thread still owns the lock while descheduled. Distinguishing processor preemption from resource preemption prevents a subtle terminology error. Scheduling a different thread does not take the held resource away from its owner.

5.7.3 Avoidance and safe completion sequences

Prevention restricts the protocol before requests occur. *Avoidance* instead considers requests dynamically and grants one only if the resulting allocation remains in a *safe state*. A state is safe when there exists at least one completion order in which every participant can obtain its declared remaining maximum demand, finish and release its resources. An unsafe state is not necessarily deadlocked now; it is a state from which the manager can no longer guarantee that all declared maximum demands can be satisfied in some order.

The distinction is easiest to see numerically. Suppose there are two resource types, A and B, and the currently available vector is

$$\mathit{Available} = (1, 1).$$

Three processes have the following allocations and declared maximum claims:

Process	Allocation (A, B)	Maximum (A, B)	Remaining need (A, B)
P_0	$(1, 0)$	$(2, 2)$	$(1, 2)$
P_1	$(1, 1)$	$(2, 1)$	$(1, 0)$
P_2	$(1, 0)$	$(1, 2)$	$(0, 2)$

The state is safe. P_1 needs only $(1, 0)$, which the available vector can satisfy. Granting that need temporarily changes availability to $(0, 1)$ and the allocation of P_1 to $(2, 1)$. Its completion returns those units, making availability $(2, 2)$. The net effect of granting the remaining need and then completing is to add the original allocation $(1, 1)$ to the

initial available vector. At that point either P_0 or P_2 can obtain its remaining need and finish. One safe sequence is therefore

$$P_1 \to P_0 \to P_2.$$

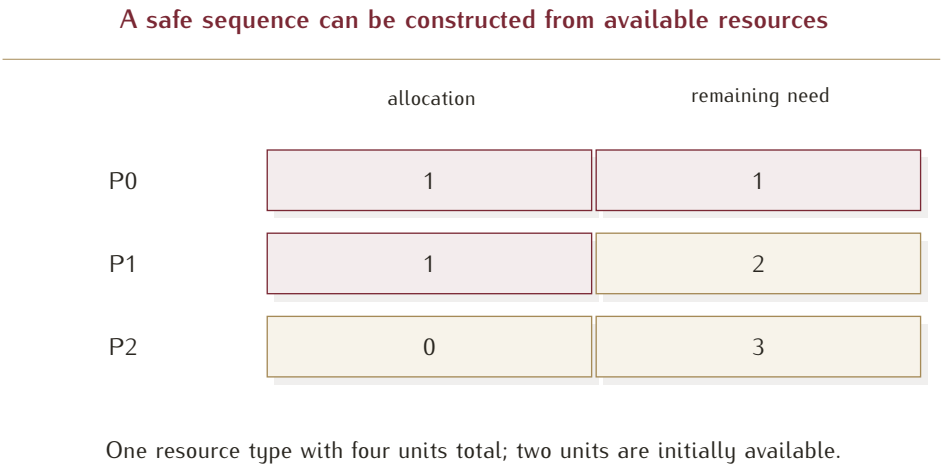


Figure 5.66: *With two available units, Po can finish and release its one allocated unit, raising available work to three. Then P1 and P2 can finish. This sequence witnesses safety under the declared maximum remaining demands.*

Now suppose P_0 asks for one additional unit of A. The request $(1, 0)$ is no larger than its remaining maximum claim and is physically available, so a purely local check could grant it. If granted, however, availability becomes $(0, 1)$ and P_0's remaining need becomes $(0, 2)$. At that moment P_0 still needs two B units, P_1 needs one A unit and P_2 needs two B units. None of the three remaining needs is bounded by $(0, 1)$, so no process is guaranteed able to complete and release resources. The new state is unsafe even though the request itself was satisfiable and no circular wait has necessarily materialised yet.

The Banker's algorithm formalises this style of reasoning by tentatively granting a request and running a safety check using current allocations, available resources and declared maximum claims. Its conceptual price is substantial: the resource manager must know meaningful upper bounds on future demands, and workloads must tolerate requests being delayed even when the requested resource is currently free. General-purpose programs often cannot state such maxima conveniently, which is one reason avoidance is more important as a reasoning model than as a universal operating-system policy.

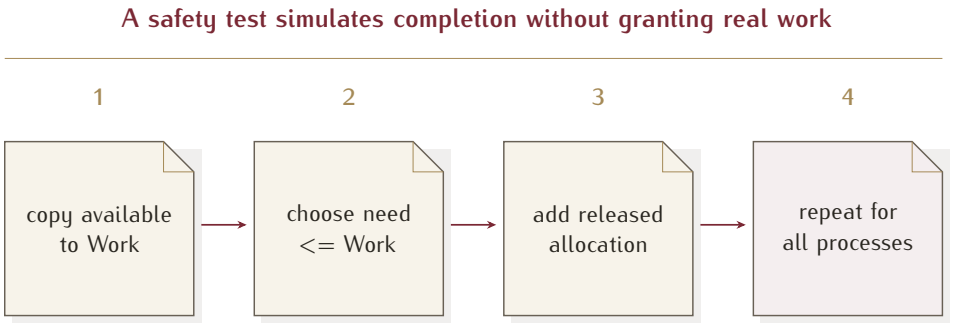


Figure 5.67: *The work vector is a reasoning device. The test assumes a selected process can finish within its declared remaining demand, then credits the resources it releases. It does not predict the actual future schedule.*

The safety test can be viewed as a constructive proof. Start with the currently available resource vector and search for a process whose remaining need can be satisfied. Using a scratch vector $Work$, simulate completion by adding the process's current allocation: $Work \leftarrow Work + Allocation_i$. This combines the hypothetical grant and return, since $(Work - Need_i) + (Allocation_i + Need_i) = Work + Allocation_i$. Repeat for the remaining processes. If every process can be removed in this way, the order found by the algorithm is a witness that the state is safe. If no remaining process fits Work, none can be the first member of a safe continuation. Every simulated completion increases or preserves Work component by component, so choosing a process that fits cannot eliminate a possible continuation. Failure at this point establishes that the state is unsafe under the declared maxima. This interpretation is useful because it separates the arithmetic from the concept: the algorithm is not predicting which process *will* run next, only proving that at least one complete ordering remains possible if future demands stay within their declarations.

It also explains why an unsafe state is not synonymous with an already deadlocked one. In the unsafe state produced by granting P_0 the extra A unit above, some process might in reality request less than its declared maximum, release a resource early or terminate, after which the system could recover naturally. Avoidance refuses the request because it cannot rely on that favourable behaviour. Safety is a guarantee under worst-case declared future demand; deadlock is a current inability of the relevant participants to progress.

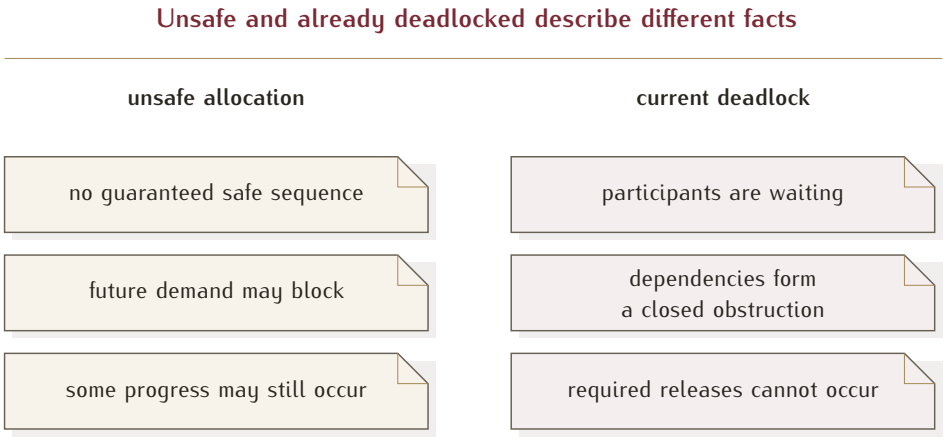


Figure 5.68: *An unsafe state can still complete under favourable future requests. A current deadlock is an actual progress obstruction in the relevant model. Confusing the two can lead to incorrect conclusions from a safety test.*

The maximum-claim requirement is therefore both the strength and the limitation of Banker's reasoning. It makes future demand conservative enough to test, but many software resources are discovered dynamically: a program may traverse a graph and learn only later which locks or buffers it will need. Declaring a useful maximum can be difficult or so pessimistic that the algorithm rejects too much work. Static lock ordering often fits software locks better precisely because it constrains the form of requests without requiring exact future counts.

5.7.4 Detection and recovery

A system can instead permit resource allocation without preventing every possible cycle and then attempt to *detect* deadlock. With single-instance resources, cycle detection on an appropriate wait-for graph can identify circular dependencies. With multiple instances, detection requires allocation and outstanding-request counts analogous to some of the information used in avoidance, although the question is now whether current waits can complete rather than whether all declared future maxima remain safe.

Detection is useful only if recovery is possible. The system may terminate one or more participants, roll back work to a recoverable checkpoint, preempt a resource whose state can safely be saved/restored, or ask a higher-level supervisor to restart a component. Every option has cost. Killing a database transaction may be acceptable if transactional recovery restores consistency; forcibly removing a non-preemptable lock from arbitrary code can leave data corrupted. A recovery policy must account for the semantics of the protected resource as well as the detected cycle.

Some systems deliberately choose to *ignore* broad classes of deadlock at a low level when the probability and cost of universal prevention exceed the cost of application-level timeout, restart or operator intervention. This choice leaves deadlock possible and assigns detection and recovery responsibility to the application or operator.

Recovery also raises a victim-selection problem. If terminating one participant can break a cycle, which participant should be sacrificed? A system may consider how much work the process has already completed, how many resources it holds, whether its state can be reconstructed, its priority, or how many other participants depend on it. Repeatedly choosing the cheapest-looking victim can itself starve one job if that job is always selected for rollback. A complete recovery policy therefore needs to reason about fairness across recovery episodes as well as about the immediate cycle.

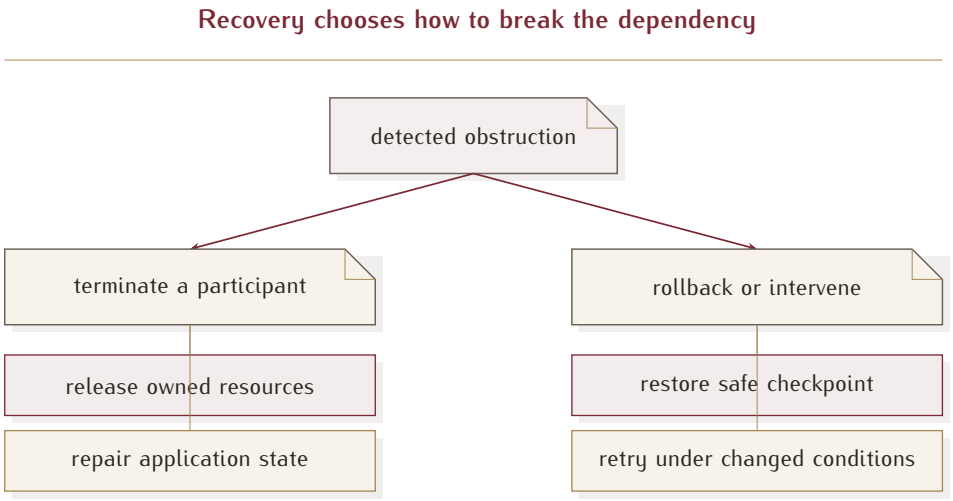


Figure 5.69: *Recovery has consequences beyond the dependency graph. A process may have partially changed shared state, so termination or rollback must account for consistency. Victim selection also affects lost work and fairness.*

Timeouts are useful operational tools but weak proofs. A timeout can prevent a caller from waiting forever and can trigger logging, cancellation or restart, yet it does not establish that the underlying dependency is absent. If two services hold resources while making synchronous calls to one another, shorter timeouts may merely convert a permanent deadlock into repeated failed transactions. Timeouts belong to failure handling; resource-order design belongs to prevention.

Distributed systems make this distinction even sharper because communication delay and failure can be hard to distinguish from waiting. An ownership-mutex implementation can associate the wait with a thread owner (though a generic futex or semaphore need not expose that ownership to the kernel), whereas a remote request may be delayed by the network, a slow peer or a failed peer. Distributed deadlock detection and

failure detectors require additional assumptions. For the local models developed here, before declaring "deadlock", identify the dependency that would have to change for progress to resume.

5.7.5 Starvation and livelock require different proofs

A program can be free of deadlock and still fail to provide useful progress. In starvation, the resource system may always have a legal next move while policy repeatedly chooses someone else. The writer in a reader-preference readers–writers protocol can wait forever if readers keep arriving. Ageing, FIFO admission, fair locks and service quotas address starvation by limiting how often a waiting participant can be bypassed.

In livelock, participants actively respond to contention yet their responses keep recreating it. Imagine two threads using a polite try-lock protocol: each acquires one resource, notices that the second is unavailable, releases the first, waits the same deterministic interval and retries. If their timing remains synchronised, both can repeatedly acquire and release without entering the useful critical section. Randomised or asymmetric backoff can reduce the chance of repeated synchrony, and assigning different roles can break the symmetry, but a probabilistic delay is not by itself a deterministic proof that every execution will make progress.

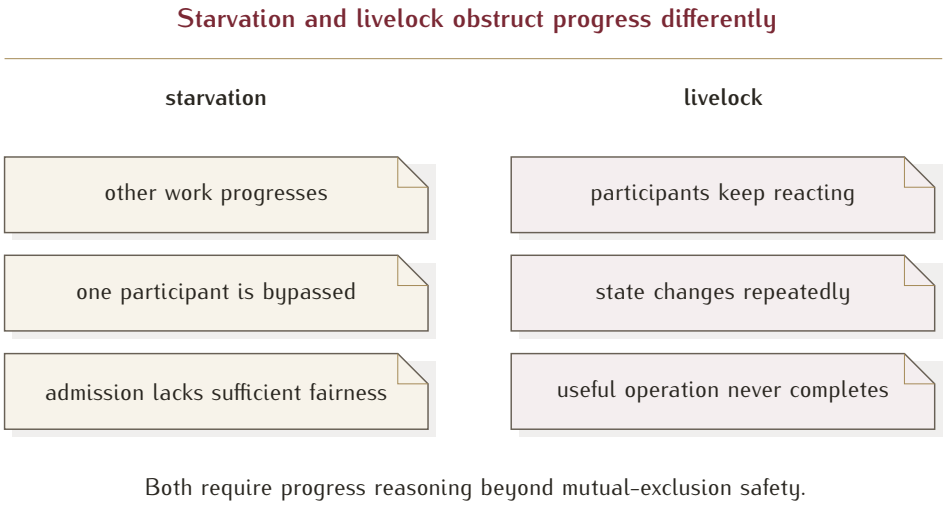


Figure 5.70: *Starvation concerns an execution that remains unserved while others advance. Livelock involves continued activity that fails to complete the intended work. Neither is diagnosed solely by a circular resource wait.*

These distinctions matter when diagnosing production systems. A deadlocked set appears stuck because no member can perform the resource-releasing action required by another. A starved thread appears stuck while other work continues successfully. A livelocked subsystem may consume substantial CPU while completing almost nothing. Sim-

ilar user-visible symptoms—"the operation never finishes"—can therefore correspond to very different scheduler and synchronisation traces.

A progress proof should therefore name its guarantee. *Deadlock-free* means the system cannot enter a state where a circular dependency permanently blocks the relevant set, but one thread may still starve. *Starvation-free* or bounded-waiting claims are stronger and normally depend on fairness assumptions about queues or scheduling. *Lock-free* and *wait-free*, terms used for non-blocking algorithms, have still different formal meanings: lock-free algorithms guarantee that some operation completes as executing threads continue taking steps, even if individual threads are delayed; a wait-free operation completes within a bounded number of its own steps, independently of other threads' delays. Neither definition promises a wall-clock bound for a thread that the scheduler never runs. Those non-blocking designs are beyond the present implementation scope, but the vocabulary illustrates why "thread-safe" is too vague to serve as a progress specification.

The full concurrency arc can now be seen as a progression of obligations. Threads make shared memory convenient, which creates interleavings that can violate invariants. Mutual exclusion and condition synchronisation restore local safety, but acquiring several resources introduces dependencies that can destroy global progress. Deadlock prevention, avoidance or recovery then addresses those dependencies, while fairness mechanisms handle starvation and progress protocols handle livelock. Correct concurrent execution requires both state invariants and explicit progress guarantees.

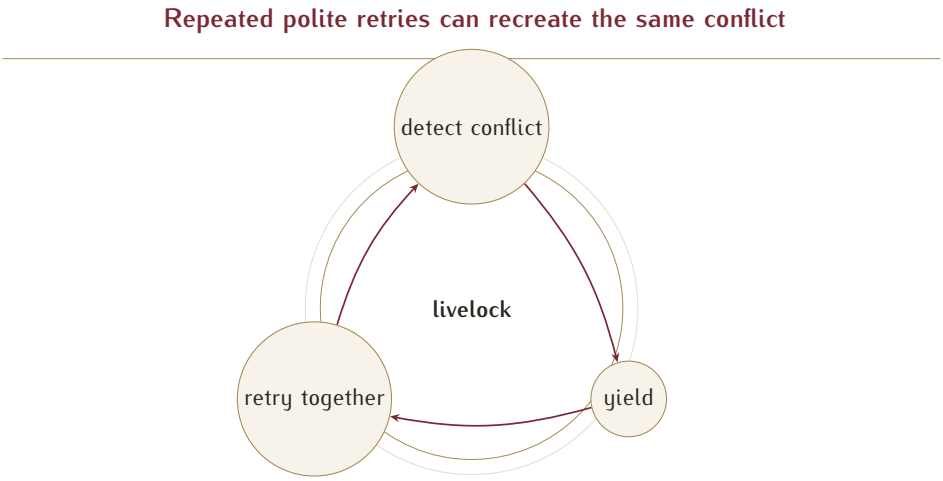


Figure 5.71: *The cycle shows why activity is insufficient evidence of progress. Randomised backoff or asymmetric roles can reduce repeated conflict, but the algorithm's exact progress guarantee still needs its own argument.*

Concept check

1. Why is a resource-allocation cycle sufficient for deadlock with single-instance resource types but not necessarily with multiple instances?
2. Which Coffman condition does a global lock-order rule eliminate, and why does the rank argument rule out a cycle?
3. How can a request be immediately satisfiable yet move the system from a safe state to an unsafe one?
4. What observable difference would you expect between deadlock, starvation and livelock in a scheduler/resource trace?

Solutions to the review questions

1. With one instance per resource type, each resource in a cycle is held by the next participant whose progress depends on another member of that cycle. No free alternative instance can satisfy the requests. With several instances, an execution outside the cycle may release an instance that allows one member to finish, so the cycle alone is insufficient.
2. A global increasing lock order eliminates circular wait. Along a hypothetical waiting cycle, every next lock would have to have a strictly greater rank than the preceding held lock. Returning to the first lock would require its rank to exceed itself, which is impossible.
3. Consider three identical resource instances. A holds one and may need three in total; B holds one and may need two. With one instance available, B can obtain it, finish and release its two instances, allowing A to finish. Granting the free instance to A is immediately possible but leaves A holding two, B holding one and neither able to obtain its final possible need. The resulting state has no safe completion sequence under the declared maxima.
4. In deadlock, the participants cannot advance because their required events depend cyclically on one another. In starvation, a particular requester keeps being bypassed while others progress. In livelock, participants continue changing state, retrying or yielding without completing the useful operation. A trace must record both state changes and completed work to distinguish these cases.

5.7.6 Worked problems

Classifying three different liveness failures

(A) A ready low-priority thread is perpetually bypassed. (B) Two threads repeatedly release and reacquire locks in synchrony but never enter. (C) Each of two blocked threads holds one lock and waits for the other.
Classify A, B and C.

1. A is ready and others continue to finish, so the failure is starvation caused by policy.
2. B remains active and changes state but makes no useful progress, so it is livelock.
3. C contains a closed wait dependency among blocked participants, so it is deadlock.

A: starvation; B: livelock; C: deadlock.
A normal disk wait that completes has a known external progress event and is none of these by itself.
Different classifications require different remedies: fairness, asymmetry/backoff, or resource-order/avoidance/recovery.

Resource-allocation graph directions

Let A and B each have one instance. A is assigned to P_1 and B to P_2. Each holder now requests the other's resource. Starting from A and alternating assignment with request edges gives the complete directed cycle

$$A \longrightarrow P_1 \longrightarrow B \longrightarrow P_2 \longrightarrow A.$$

P_1 needs P_2 to release B, while P_2 needs P_1 to release A. With blocking acquisition and release only after obtaining both locks, each potential release lies beyond an unsatisfied wait. Extra interchangeable instances or an authorised rollback would change the argument.

Global ordering prevents circular wait

Lock ranks are X=10, Y=20 and Z=30; every path must acquire in increasing order. Why is the path Z then X forbidden, and which Coffman condition is broken?

1. After holding Z at rank 30, requesting X at rank 10 decreases the rank and violates the global rule.
2. If every request strictly increases rank, following wait edges around a cycle would require ranks to increase and eventually return to the starting rank.

Worked example (continued)

3. That is impossible, so circular wait cannot arise.

Z then X is forbidden; global ordering attacks circular wait.
X then Z is legal because ranks increase, even though intermediate Y is not required.
Dynamic objects can use a stable total order such as immutable identifiers, provided every code path obeys it.

Safe sequence with one resource type

A resource has seven instances. Two are available. P_1 holds two and declares maximum five; P_2 holds three and declares maximum four.
Find a safe first process and a complete safe sequence.

1. P_1 may still need $5-2=3$, which exceeds the two available instances.
2. P_2 may still need $4-3=1$, so it can receive one, finish and release its four instances.
3. Availability becomes $2-1+4=5$, enough to grant P_1 its three remaining units. P_1 then returns all five units it holds, leaving $5-3+5=7$. The sequence is P_2, P_1.

The state is safe because a complete sequence $P_2 \to P_1$ exists.
Granting available resources merely because a current request fits is not enough; simulate completion and release for every participant.
An unsafe state is not necessarily already deadlocked; it means no completion sequence is guaranteed for all declared maxima.

Exercise 5.7.1 — Temporary block

A timer sleep expires and the process resumes normally. Is that alone deadlock?

Worked solution

A timer sleep creates a finite wait for an external completion event. If the timer expires and the process resumes, that behavior alone is not deadlock. Deadlock requires a set of activities whose waiting dependencies prevent the required progress under the model. The presence of a blocked state is therefore insufficient evidence; identify what event can release the wait and whether that event can occur.

Exercise 5.7.2 — Coffman count

How many conditions are jointly necessary for classical resource deadlock?

Worked solution

The classical conditions are mutual exclusion, hold and wait, no preemption, and circular wait: four conditions. Mutual exclusion makes a resource unavailable to another holder; hold and wait retains one resource while requesting another; no preemption prevents forcible recovery; circular wait closes the dependency chain. A deadlock-prevention policy removes at least one condition. The conditions characterize the stated resource model and do not replace examination of the actual allocation state.

Exercise 5.7.3 — Edge direction

What direction represents a process requesting resource R?

Worked solution

A request edge points from the process to the resource: $P \rightarrow R$. It means that P is waiting to acquire an instance of R. An assignment edge points the other way, from a resource instance to the process holding it. Following alternating request and assignment edges reveals who waits for whom. Reversing these meanings would invalidate the wait-cycle analysis.

Exercise 5.7.4 — All-at-once acquisition

Which Coffman condition is attacked by refusing to hold a partial set while waiting for the rest?

Worked solution

Requiring all needed resources to be acquired together removes hold and wait. A process that cannot obtain the full set begins or continues with none of that set held, so it does not retain a partial allocation while waiting for the remainder. The trade-off can include lower utilization and a need to know requirements in advance. The policy must be implemented atomically enough to prevent partial acquisition from reintroducing the condition.

Exercise 5.7.5 — Recovery

Name one cost of terminating a victim after deadlock detection.

Worked solution

Terminating a victim can discard completed computation and release resources only after the system performs the necessary cleanup. External effects, such as an already sent message or partially updated file, may require recovery and may be irreversible. Repeatedly selecting the same activity can also cause starvation. Victim selection should therefore account for restart cost, consistency obligations and fairness in addition to the immediate resource release.

Summary

Deadlock is a circular dependency among blocked participants, starvation is indefinite policy-driven postponement, and livelock is activity without progress. Prevention, avoidance, detection and recovery make different assumptions and impose different costs; no strategy is free.

5.7.7 Exercises

Exercise 5.7.6 — Classify liveness failure

Classify: a low-priority thread never scheduled; two threads indefinitely back off in response to each other and never enter; two threads each hold one lock and wait for the other.

Go to the solution →

Exercise 5.7.7 — Coffman conditions

List the four necessary conditions for resource deadlock.

Go to the solution →

Exercise 5.7.8 — Wait-for graph

Under blocking single-instance lock waits with no timeout, forced release or alternative completion, P_1 waits for P_2, P_2 for P_3, and P_3 for P_1. What structural feature is present?

Go to the solution →

Exercise 5.7.9 — Lock ordering

All code is required to acquire locks in strictly increasing rank order A=1, B=2, C=3. Is acquiring C and then A permitted? Beyond answering yes or no, give the short proof that a globally respected increasing order eliminates circular wait.

Go to the solution →

Exercise 5.7.10 — Safe state

A system has one resource type with 10 instances. Current allocations are P1=4, P2=2 and P3=2; maximum claims are P1=7, P2=4 and P3=5. Determine whether the current state is safe by exhibiting a safe completion sequence, then decide whether granting one additional instance to P3 immediately would leave the state safe.

Go to the solution →

Exercise 5.7.11 — Recovery choice

A detector finds a deadlock cycle among three services after they have already performed externally visible work. Choose one recovery action (termination, rollback, or resource preemption), state the condition that makes your choice feasible, and identify at least two costs or risks that a production policy should consider when selecting the victim.

Go to the solution →

5.7.8 Solutions

Deadlock, Starvation, and Livelock

Solution 5.7.6

Low-priority indefinite delay is starvation. Indefinite mutual backoff without useful progress is livelock. Holding one lock while waiting for the other in a cycle is deadlock.

← Return to the exercise

Solution 5.7.7

The four necessary conditions are: mutual exclusion, where at least one resource is exclusively held; hold and wait, where a participant keeps resources while requesting another; no preemption, where allocated resources remain with their holders until release; and circular wait, where each member of a cycle waits for a resource held by the next. A prevention policy breaks at least one condition. Their mere availability does not prove a particular execution is deadlocked; the actual allocation and waits must also be examined.

← Return to the exercise

Solution 5.7.8

Follow the wait-for edges $P_1 \to P_2 \to P_3 \to P_1$. This is a directed cycle. Each process needs the next to release its single-instance lock, yet the next process is itself blocked. With the stated absence of timeout, forced release or other completion path, none can perform the release that another member needs. The three processes form a deadlocked set.

← Return to the exercise

Solution 5.7.9

No. C then A violates the increasing-order rule. To see why the rule prevents circular wait, assume for contradiction that a deadlock cycle exists. Along each wait edge, a participant holds a lower-ranked lock and waits for a strictly higher-ranked one, so ranks must strictly increase as the cycle is followed. Returning to the first participant would require the rank to be both greater than and equal to its starting value, which is impossible. The proof depends on every acquisition path respecting the same global order.

← Return to the exercise

Solution 5.7.10

Initially 8 instances are allocated, so 2 are available. Remaining needs are P1=3, P2=2 and P3=3. P2 can finish first: grant its need of 2, leaving 0 available, then reclaim its full holding of 4, leaving 4 available. P1 can receive 3, leaving 1 available, then finish and return its full holding of 7, leaving 8 available. P3 can then receive 3 and return its full holding of 5, leaving all 10 available. Equivalently, the safety-test work vector increases by each original allocation: $2 \to 4 \to 8 \to 10$. One safe sequence is therefore P2, P1, P3. If one additional instance is granted to P3 immediately, availability falls to 1 while remaining needs become P1=3, P2=2 and P3=2. No participant can then obtain its remaining claim, so that hypothetical grant produces an unsafe state even though the requested instance was physically available.

← Return to the exercise

Solution 5.7.11

One defensible policy terminates a victim only if its external effects can be retried idempotently or compensated, and the resource manager can reclaim or recover the resources that form the cycle. Killing a thread does not automatically unlock an ordinary POSIX mutex; a robust-mutex recovery path or another explicit recovery mechanism may be necessary to restore consistency and release ownership. The policy must account for lost computation, partially completed I/O or transactions,

Worked solution (continued)

restart cost and fairness: repeatedly choosing the cheapest victim can starve the same service. Rollback is preferable when a trustworthy checkpoint exists; resource preemption is possible only for resources whose state can be taken and restored safely. Resource semantics determine which recovery actions can safely break the detected wait-for cycle.

← Return to the exercise

6

MEMORY, FILESYSTEMS, AND PROTECTION

Addresses connect a program's data structures to the memory system; names connect persistent objects to the filesystem. This Part follows those connections through cache locality, virtual-memory translation, page tables, file allocation, recovery and protection. Each model declares its units, state and access rules before using them in calculations or code.

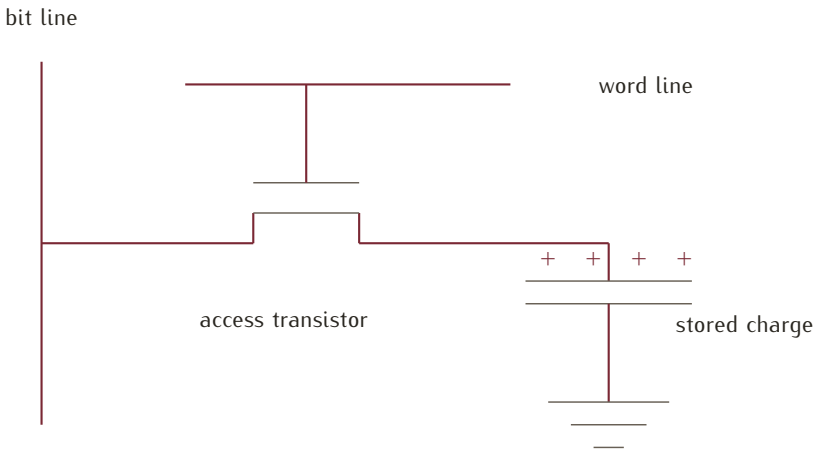
The first four chapters explain how locality, mapping structures and cached translations connect a program's addresses to available memory. The final three chapters examine persistent objects: how names reach stored data, how updates survive interruption, and how access rules constrain operations. Numerical traces and executable simulations expose the state changes behind these abstractions [1, 59, 50].

6.1 MEMORY HIERARCHIES AND LOCALITY

6.1.1 Why memories form a hierarchy

A loop visits one array element, then its neighbour, then the next. Each instruction names only the bytes it needs, while a memory transfer can bring an entire block close to the processor. The following iterations may find their operands already there. This reuse is the opportunity that a *memory hierarchy* exploits.

Different storage mechanisms support different levels



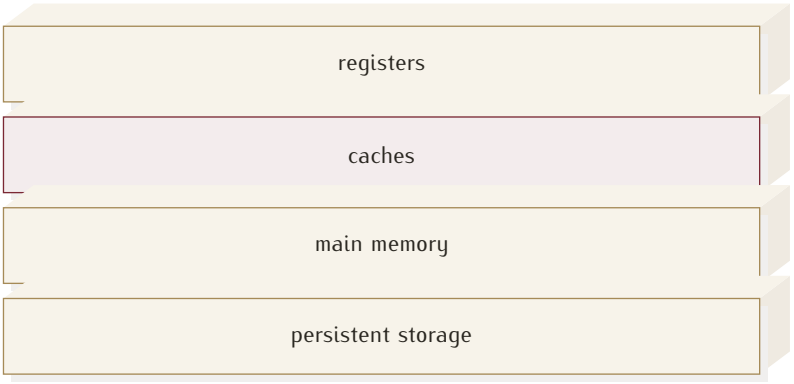
Stored charge illustrates a physical maintenance cost behind the memory hierarchy.

Figure 6.1: *The capacitor-based cell supplies a concrete example of how retaining information has physical requirements. SRAM uses feedback instead. Such differences contribute to density, speed and maintenance choices across the hierarchy.*

Registers hold operands explicitly named by instructions. Caches retain copies of useful memory blocks; main memory holds resident process and kernel data; persistent storage retains files across power loss. Following one array access through these levels reveals both the benefit of a nearby copy and the cost of fetching a missing block.

The hierarchy also reflects the physical work required to retain each bit. A small fast store and a large dense store make different choices about circuit area, stored charge and maintenance. The box below connects those choices to the roles in the table. Numerical access times still depend on the device and the complete memory subsystem.

The hierarchy places small fast stores near execution



Capacity, cost and access time differ; the drawing is not a quantitative scale.

Figure 6.2: *Each level serves a different role while retaining or supplying data used by levels above it. A lower-level access may be avoided when a suitable copy is already available nearby.*

Level	Typical role	Unit visible in this model
Registers	Current operands	Register value
L1/L2/L3 caches	Copies of active data and instructions	Cache line
DRAM	Resident memory pages	Memory access / page frame
SSD or HDD	Persistent files and backing store	Storage block

A cache line and a virtual-memory page are different units. A 4 KiB page can contain 64 lines of 64 bytes each. Keeping these units separate prevents errors when counting cache misses, page faults and transferred bytes.

Keeping a bit alive: SRAM and DRAM

A powered SRAM cell uses feedback to maintain one of two stable logical states. A typical DRAM cell holds charge on a capacitor selected by a transistor. Charge leakage requires periodic refresh, while the simpler cell supports greater density. These mechanisms help explain the use of SRAM for caches and DRAM for main memory; both require power to retain ordinary working data [141, 42].

For a capacitor, $Q = CV$. Under a deliberately simple constant-leakage model, losing charge $I\Delta t$ changes its voltage by $\Delta V = I\Delta t/C$. This model isolates a physical dependency; real refresh limits are device specifications that account for cell variation and operating conditions.

Physical mechanism (continued)

Worked check. Take a hypothetical cell with $C = 20$ fF, constant leakage $I = 1$ fA and an allowed drop of 0.2 V. Its modelled interval is

$$\Delta t = \frac{C \Delta V}{I} = \frac{20 \cdot 10^{-15} \cdot 0.2}{10^{-15}} = 4 \text{ s}.$$

Doubling leakage halves that interval to 2 s. These invented values illustrate the equation; they are unsuitable for selecting a real refresh rate. Restoring charge consumes memory-system work even while the application leaves those bytes untouched.

6.1.2 Locality and the state of a cache

Temporal locality is reuse of the same information within a short interval. Repeated updates of a loop counter are an example. *Spatial locality* is use of nearby addresses; an array scan commonly exhibits it. A cache fill can bring several future operands into the faster level when they share a line. Workloads that stride past most bytes of every line can waste that transfer capacity.

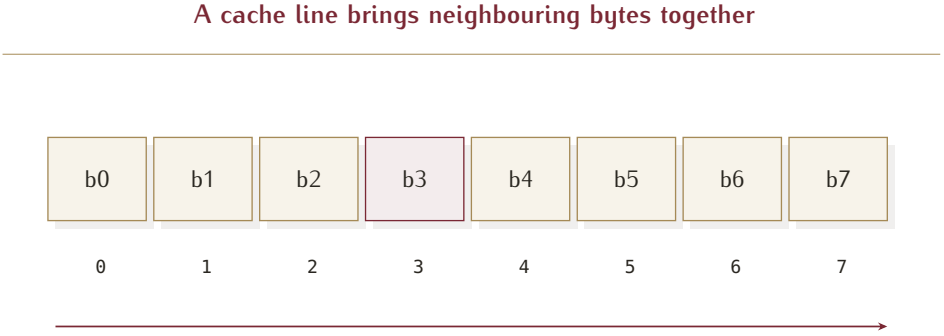


Figure 6.3: *Spatial locality makes later accesses to nearby bytes useful after the block transfer. The illustrated eight-byte line is deliberately small; actual line size belongs to the stated machine model.*

With line size B bytes, a byte address a belongs to block $b = \lfloor a/B \rfloor$ at byte offset $a \bmod B$. A *hit* occurs when the requested block is already resident with a usable copy. Otherwise the request is a *miss*. In the read-allocating cache model used in the following traces, the cache fetches the missing block. Store misses require a separately specified allocation policy. If the permitted destination is full, a replacement policy selects a victim. Mapping and associativity determine which resident blocks compete for that

destination; replacement chooses among those eligible blocks. A dirty victim in a write-back cache needs its modified data preserved before the space is reused.

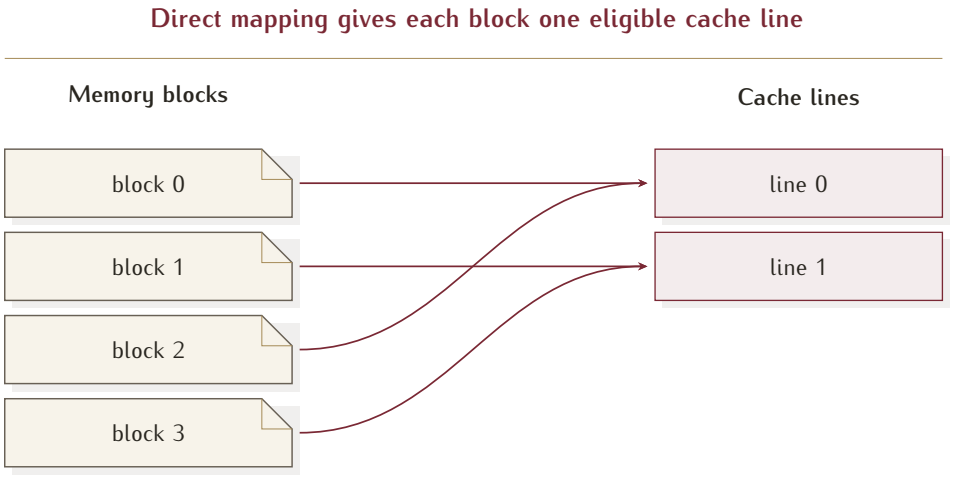


Figure 6.4: *Blocks zero and two compete for the same line even if the other line is unused. This structural competition explains conflict misses separately from the total amount of data in the cache.*

A direct-mapped cache permits one destination line for each block; with S lines, a common mapping is $b \bmod S$. A set-associative cache first selects a set, then permits the block to occupy any of its ways. A fully associative cache permits any line. Least-recently-used (LRU) replacement evicts the eligible resident block whose most recent access is oldest; each hit updates that recency order.

Separating conflict misses from capacity misses

Start with an empty two-line direct-mapped cache and read blocks $0, 2, 0$. Blocks 0 and 2 both map to line 0. Their first references are compulsory misses. The third reference misses again because block 2 displaced block 0, even though line 1 is free. A fully associative cache of the same capacity could retain both blocks, so this third miss is a conflict miss. Now give the fully associative two-line cache the sequence $0, 1, 2, 0$ with LRU replacement. The first three references introduce distinct blocks. Installing block 2 evicts 0, so the last reference also misses in the fully associative cache. This is a capacity miss under the stated comparison model.

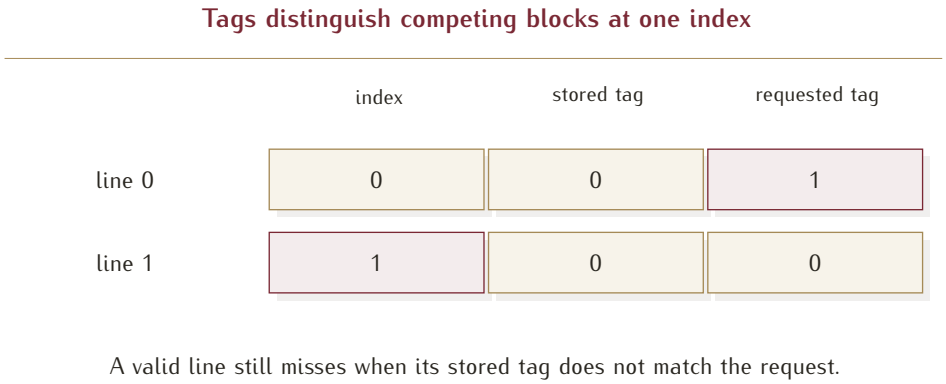


Figure 6.5: *Indexing selects the eligible line or set. The tag comparison then determines whether the requested block is actually present. A hit also requires the relevant validity and access conditions.*

A write-through cache forwards each store to the next level, potentially through a write buffer. A write-back cache updates its resident line and marks it dirty, deferring propagation until required by eviction or another hierarchy operation. On a store miss, write allocation fetches or allocates a line according to the protocol; no-write-allocation forwards the write without retaining a new line at that level. The exercises state which policy is relevant.

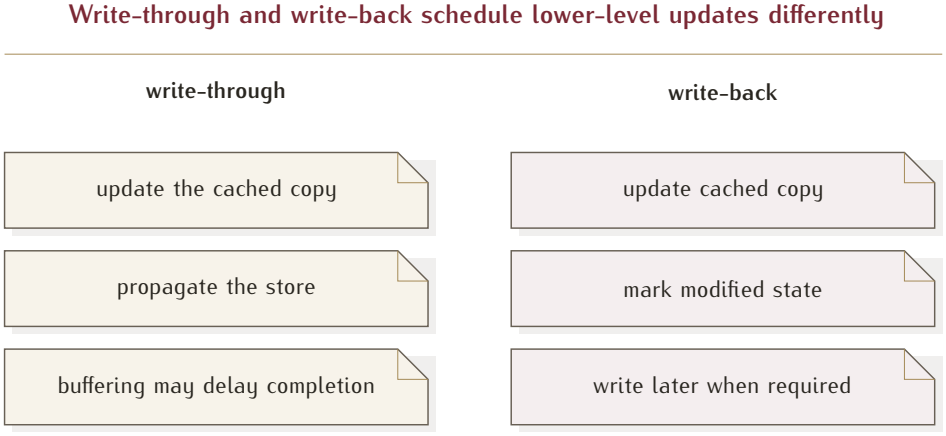


Figure 6.6: *The policies determine when a modified cached value is propagated towards the next level. Write-back requires tracking modified state and preserving it when the line is replaced or otherwise written back.*

To trace a cache, write down the initial resident blocks, examine one request, classify it using the current state, then apply any insertion and eviction before examining the

next request. A statement such as “no evictions” is part of the problem’s model. Under that assumption a first miss may turn a later reference into a hit. Without a stated capacity and policy, an eviction pattern cannot be inferred.

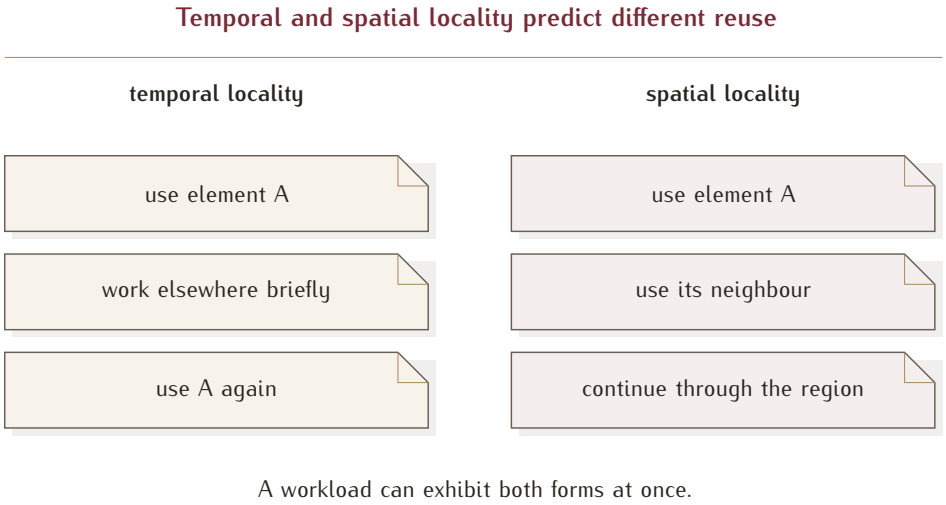


Figure 6.7: *Temporal locality reuses the same information, while spatial locality uses nearby information. The effectiveness of caching depends on whether the useful data remains available and how transfers and replacement operate.*

Figure 6.8 makes the state update explicit. Read a row, replace the cache state with the state drawn on its right, then proceed to the next row.

Predicting the next cache access

Continue Figure 6.8 with address 68. Its block is $\lfloor 68/64 \rfloor = 1$, its offset is 4, and its direct-mapped line is $1 \bmod 2 = 1$. That line still holds block 1, so the access hits. The preceding evictions occurred in line 0 and left line 1 intact. If the next address is 128, block 2 competes for line 0 again and evicts block 0. The two outcomes follow from the final state of the figure.



Each row shows the cache *after* the request. Blocks 0 and 2 compete for line 0.

Figure 6.8: *Start with both lines invalid. The second access reuses the first block. Accessing block 2 evicts block 0, so the final access misses despite having occurred earlier. The trace assumes reads, allocation on a miss and no prefetching or other accesses.*

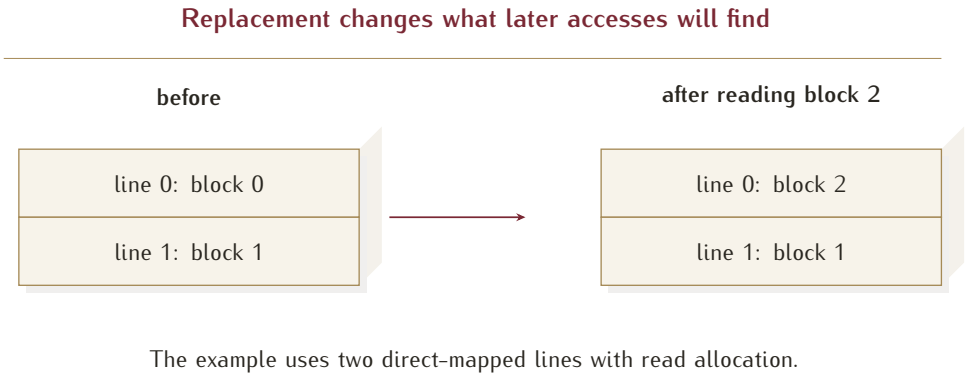


Figure 6.9: *Block two replaces block zero because both map to line zero. A subsequent read of block zero therefore misses, while a read of block one can still hit if nothing else has changed.*

A boundary changes the block count

Read 80 consecutive bytes beginning at byte address 60, with 64-byte lines. The first block is $\lfloor 60/64 \rfloor = 0$. The last byte is $60 + 80 - 1 = 139$, in block $\lfloor 139/64 \rfloor = 2$. Thus three blocks, numbered 0, 1 and 2, are touched. For a nonempty interval of length L, the general count is $\lfloor (a + L - 1)/B \rfloor - \lfloor a/B \rfloor + 1$. The formula accounts for the starting alignment.

6.1.3 Average access time and its assumptions

Let t_h be the lookup/hit time, m the miss fraction and p the *additional* penalty after a miss. Every access pays t_h; fraction m also pays p. Therefore

$$\mathrm{AMAT} = t_h + mp.$$

If the supplied miss time t_m already includes the lookup, use $(1 - m)t_h + mt_m$. Both formulas agree when $t_m = t_h + p$. Rates must be fractions in these equations: $3\% = 0.03$. Times must use a common unit. For multiple levels, a local miss rate is conditional on reaching that level; multiplying local rates gives the fraction reaching a lower level under the stated serial model.

Two levels with conditional misses

L1 lookup takes 2 cycles and misses on 10% of requests. Each L1 miss pays an 8-cycle L2 lookup; 25% of those L2 lookups add 100 cycles for memory. Hence $\mathrm{AMAT} = 2 + 0.10(8 + 0.25 \cdot 100) = 5.3$ cycles. Out of 1000 original requests, 100 reach L2 and 25 reach memory. Their total cost is $2000 + 800 + 2500 = 5300$ cycles, confirming the mean.

AMAT describes the chosen access model. Processor execution time also depends on overlap, instruction dependencies, prefetching and how much of the workload waits for memory. Use Chapter 2.1's execution-time model before equating a change in memory latency with the same change in whole-program speed.

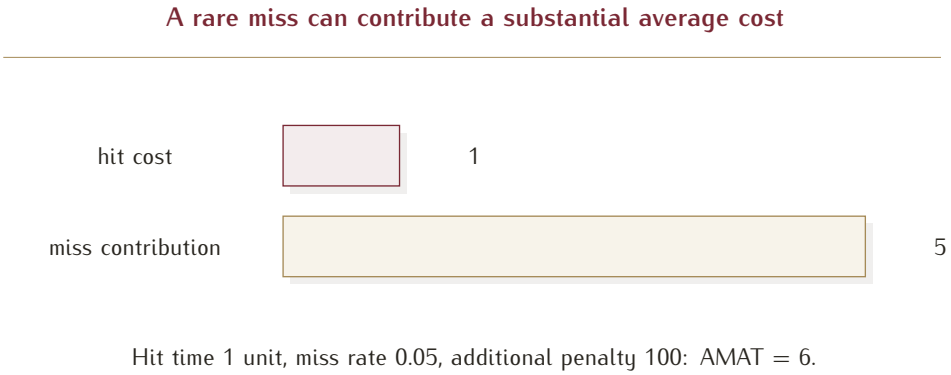


Figure 6.10: *The miss contribution is probability times additional penalty, giving five units. Adding the one-unit hit cost produces six. This formula assumes the penalty is additional rather than the total time of a miss.*

6.1.4 Exercises

Exercise 6.1.1 — Selecting a cell technology

A small CPU cache needs fast volatile storage without periodic refresh. Choose one description: A: SRAM using powered bistable cells; B: DRAM refreshed periodically; C: an HDD platter; D: NAND flash with erase blocks; E: an optical disc; F: DRAM that retains bits after power loss.

Worked solution

Exercise 6.1.2 — Refresh and density

A design compares a small SRAM cache with a much larger DRAM main memory. Explain the refresh requirement, the reason for using DRAM at the larger capacity, and which contents survive loss of power.

Worked solution

Exercise 6.1.3 — Locating the persistent copy

A document exists in a DRAM editing buffer and in an older version on an SSD. The latest changes have never been saved. State what survives a complete loss of

Exercise 6.1.3 — Locating the persistent copy (continued)

power under this model, and explain why the presence of an SSD is insufficient to preserve the changes.

Worked solution

Exercise 6.1.4 — A hierarchy under a budget

Why would a system combine a small SRAM cache, a larger DRAM memory and persistent storage? Explain the benefit and one condition under which the small cache helps little.

Worked solution

Exercise 6.1.5 — Classifying two loops

Loop A repeatedly reads one table entry. Loop B reads all elements of a contiguous array once in increasing order. Identify the locality each exhibits and describe what a cache-line fill provides in each case.

Worked solution

Exercise 6.1.6 — An aligned interval

Cache lines are 64 bytes. How many distinct lines are touched by reading 192 consecutive bytes beginning at byte address 128? Give their block numbers.

Worked solution

Exercise 6.1.7 — An unaligned interval

Repeat the preceding calculation for 192 bytes beginning at address 130. Explain the difference.

Worked solution

Exercise 6.1.8 — Reuse count

For block references $3, 8, 3, 9, 8, 8, 10$, count references to a block that has already appeared. Does that count alone establish the number of hits in a finite cache?

Worked solution

Exercise 6.1.9 — A state trace without evictions

Initial resident blocks are $\{2, 6\}$. Requests are $6, 4, 4, 2, 7, 6$. A miss inserts its block and nothing is evicted. Return H/M in order and count hits.

Worked solution

Exercise 6.1.10 — Block number and offset

With 128-byte cache lines, split address `0x3456` into a decimal block number and byte offset.

Worked solution

Exercise 6.1.11 — Counting misses after insertion

Initial blocks are $\{10, 15\}$. Access $15, 12, 13, 12, 10, 13, 14$ without eviction. Count misses and state the final contents.

Worked solution

Exercise 6.1.12 — Replacement changes the trace

A fully associative two-line cache is initially empty and uses LRU replacement. Trace $1, 2, 1, 3, 2$. Show resident blocks from least to most recently used.

Worked solution

Exercise 6.1.13 — A rate with a denominator

Out of 500 accesses, 465 hit. Give the hit rate, miss rate and number of misses.

Worked solution

Exercise 6.1.14 — Additional miss penalty

Hit time is 3 cycles, miss rate 4%, and each miss adds 70 cycles. Compute AMAT and total modelled cost for 1000 accesses with exactly that miss count.

Worked solution

Exercise 6.1.15 — Total miss time

Hits take 6 ns; a miss takes 106 ns in total. The hit fraction is 98%. Compute the mean and express the same calculation using an additional penalty.

Worked solution

Exercise 6.1.16 — Comparing cache designs

A has hit time 2 cycles and miss rate 6%; B has hit time 4 cycles and miss rate 2%. Both add 80 cycles per miss. Which has the smaller AMAT? Choose A: A at 2; B: A at 6.8; C: B at 5.6; D: B at 4; E: equal at 8; F: insufficient information.

Worked solution

Exercise 6.1.17 — Ordering the levels

Put registers, DRAM, L1, SSD and L2/L3 in their usual proximity order from the processor, and explain what locality allows the nearest small levels to achieve.

Worked solution

Exercise 6.1.18 — Useful bytes per line

A loop reads one four-byte value from each successive 64-byte line and never reuses a line. What fraction of each fetched line carries requested data? Compare a contiguous scan of sixteen four-byte values beginning at a line boundary.

Worked solution

Exercise 6.1.19 — A dirty victim

A full write-back cache must insert a new line. Its chosen victim has been modified since its lower-level copy was made. Explain the two pieces of work required and why simply overwriting the victim is incorrect.

Worked solution

Exercise 6.1.20 — An integrated scan

An initially empty cache can retain all four 64-byte lines of a 256-byte aligned array. A program scans its 64 four-byte elements twice. Each element causes one access, no prefetching or eviction occurs, hit time is 2 cycles and an additional miss costs 50 cycles. Find hits, misses, hit rate and total cost.

Worked solution

6.1.5 Solutions

Solution 6.1.1

Answer A. Powered SRAM cells retain their state without periodic refresh and provide the appropriate low-latency storage. B describes ordinary main-memory DRAM and includes the refresh requirement excluded by the question. C and E are persistent media with a different access mechanism and latency class. D is nonvolatile storage with programming and erase constraints. F assigns nonvolatility to ordinary DRAM incorrectly. SRAM still loses its contents when its sustaining power is removed.

Return to the problem

Solution 6.1.2

DRAM stores charge that leaks, so its cells need periodic refresh to preserve the encoded values. SRAM uses powered bistable circuits and needs no such refresh cycle. DRAM's compact cells provide greater density and lower cost per bit in the comparison, making a large main memory practical. Both memories are volatile: neither supplies persistent storage after power is lost. A design needing durable data adds an appropriate nonvolatile medium and a protocol that actually transfers the data to it.

Return to the problem

Solution 6.1.3

The older SSD version survives; the unsaved DRAM changes are lost. Persistence belongs to the copy actually stored on the nonvolatile medium. Merely having an SSD connected does not transfer the buffer's bytes to it. Saving requires software and the storage stack to issue writes and satisfy the applicable durability conditions. This problem assumes that the older SSD copy was already durable and that there is no battery-backed memory or other recovery mechanism.

Return to the problem

Solution 6.1.4

The cache serves frequently reused data with low latency. DRAM provides a much larger volatile capacity at lower cost per bit, and persistent storage retains data across shutdowns. Locality allows a small fast subset to serve many requests. A stream that touches each cache line once and never reuses it can gain little from retaining old lines, especially when its access stride wastes most bytes brought by each fill. The benefit therefore depends on the workload as well as capacities and access costs.

Worked solution (continued)

Return to the problem

Solution 6.1.5

Loop A exhibits temporal locality: after the first fill, repeated reads can reuse the resident entry while its line remains cached. Loop B exhibits spatial locality: one fill can supply several adjacent elements that the scan will soon use. A single-pass scan has little temporal reuse of old elements, although successive accesses may share the same line. Both classifications concern address behaviour; actual hit counts also require the initial state, line size and eviction behaviour.

Return to the problem

Solution 6.1.6

The first block is $128/64 = 2$. The last byte is $128 + 192 - 1 = 319$, in block $\lfloor 319/64 \rfloor = 4$. The touched blocks are 2, 3 and 4, so the count is three. Each contributes 64 bytes, giving $3 \cdot 64 = 192$ as a check. Alignment makes simple division sufficient here; the interval formula also works for unaligned starting addresses.

Return to the problem

Solution 6.1.7

The first block remains $\lfloor 130/64 \rfloor = 2$. The last byte is 321, in block 5. Therefore blocks 2, 3, 4 and 5 are touched: four blocks. The first contributes 62 bytes, the middle two contribute 128 bytes, and the final one contributes two, totaling 192. A two-byte shift in the start can add an entire fetched line because transfers occur at line granularity.

Return to the problem

Solution 6.1.8

The third reference reuses block 3; the fifth and sixth reuse block 8. Thus three references reuse previously seen blocks. The first references to 3, 8, 9 and 10 introduce four distinct blocks, and $7 - 4 = 3$ confirms the count. A reuse is a hit only if the block is still resident. A finite cache may have evicted it, so capacity, mapping, initial state and replacement policy are needed to deduce hits.

Return to the problem

Solution 6.1.9

The first request finds 6, so it is H. Block 4 is absent, giving M and new set $\{2, 4, 6\}$. The next 4 and then 2 are H. Block 7 gives M and expands the set to $\{2, 4, 6, 7\}$. The final 6 is H. The sequence is `H, M, H, H, M, H`, with four hits and two misses. State changes are applied immediately after each miss.

Return to the problem

Solution 6.1.10

The address is `0x3456` $= 13{,}398$. Division by 128 gives $13{,}398 = 104 \cdot 128 + 86$. Hence the block number is 104 and the offset is 86. The offset is smaller than 128, as required. Equivalently, a seven-bit mask gives `0x3456` & `0x7F` $=$ `0x56` $= 86$, and shifting right seven bits gives 104.

Return to the problem

Solution 6.1.11

References to 12, 13 and 14 are their first appearances in an initially absent state, so each misses once. Their later references hit because no block is evicted. The trace is `H, M, M, H, H, H, M`: three misses and four hits. The final resident set is $\{10, 12, 13, 14, 15\}$. Counting each occurrence of an initially absent block as a miss would incorrectly count the later 12 and 13 again.

Return to the problem

Solution 6.1.12

Reference 1 misses and leaves $[1]$. Reference 2 misses and leaves $[1, 2]$. Reference 1 hits and changes recency to $[2, 1]$. Reference 3 misses, evicts least-recent block 2 and leaves $[1, 3]$. Reference 2 now misses, evicts 1 and leaves $[3, 2]$. The trace is `M, M, H, M, M`. Reuse of block 2 at the end is a miss because its earlier copy was evicted.

Return to the problem

Solution 6.1.13

There are $500 - 465 = 35$ misses. Hit rate is $465/500 = 0.93 = 93\%$; miss rate is $35/500 = 0.07 = 7\%$. The two fractions sum to one. The denominator is all 500 accesses in both calculations. Dividing hits by misses would produce a ratio of outcomes, which is a different quantity.

Return to the problem

Solution 6.1.14

Every access pays three cycles, and fraction 0.04 pays 70 extra cycles. Thus AMAT $= 3 + 0.04 \cdot 70 = 5.8$ cycles. In 1000 accesses there are 40 misses, so the total is $1000 \cdot 3 + 40 \cdot 70 = 5800$ cycles. Dividing by 1000 recovers the mean. The phrase "adds 70" makes the penalty additional to the lookup.

Return to the problem

Solution 6.1.15

The weighted mean is $0.98 \cdot 6 + 0.02 \cdot 106 = 5.88 + 2.12 = 8$ ns. Since the miss time includes the lookup, its additional penalty is $106 - 6 = 100$ ns. The equivalent expression is $6 + 0.02 \cdot 100 = 8$ ns. Adding the full 106 ns to every miss after already charging the lookup would double-count six nanoseconds on those accesses.

Return to the problem

Solution 6.1.16

Answer C. Cache A has $2 + 0.06 \cdot 80 = 6.8$ cycles; B has $4 + 0.02 \cdot 80 = 5.6$ cycles. B's lower miss cost contribution offsets its slower lookup. Options A and D omit misses. B reports A's correct mean but selects the slower design. E does not follow either calculation. F is unnecessary because the stated model supplies every required quantity. B has the lower mean access time for the specified request distribution. Application speedup would also depend on the time spent waiting for memory.

Return to the problem

Solution 6.1.17

The usual order is registers, L1, L2/L3, DRAM, SSD. The ordering describes the conventional hierarchy studied here. Frequently reused operands and nearby data can remain in the nearer levels, so many requests are satisfied without reaching the large slower levels. Registers are named directly by instructions, while cache residency is generally managed by hardware. Capacity and access latency are properties of a specific design; the hierarchy explains the intended roles.

Return to the problem

Solution 6.1.18

The strided loop uses $4/64 = 1/16 = 6.25\%$ of each fetched line. The other 60 bytes bring no benefit to this stated workload. Sixteen contiguous four-byte values total 64 bytes; if they begin at a line boundary, one line supplies all of them, giving 100%

Worked solution (continued)

useful payload for that line. This comparison isolates spatial locality and assumes that every requested line must be fetched once.

Return to the problem

Solution 6.1.19

The cache must preserve the victim's current data by writing it back according to the hierarchy's protocol, then make space for the new line and fetch that line. The dirty state indicates that the lower level lacks the newest bytes. Overwriting the victim without preserving them would lose completed stores. A clean victim already has an up-to-date lower-level copy in this model and can be discarded without the same write-back.

Return to the problem

Solution 6.1.20

Each line contains 16 elements. During the first scan, the first reference to each of four lines misses; the other $64 - 4 = 60$ references hit. The second scan has 64 hits because all lines remain resident. Across 128 accesses there are 124 hits and four misses. Hit rate is $124/128 = 96.875\%$. Total cost is $128 \cdot 2 + 4 \cdot 50 = 456$ cycles, with mean $456/128 = 3.5625$ cycles. The trace, capacity assumption and timing model each contribute a distinct part of the result.

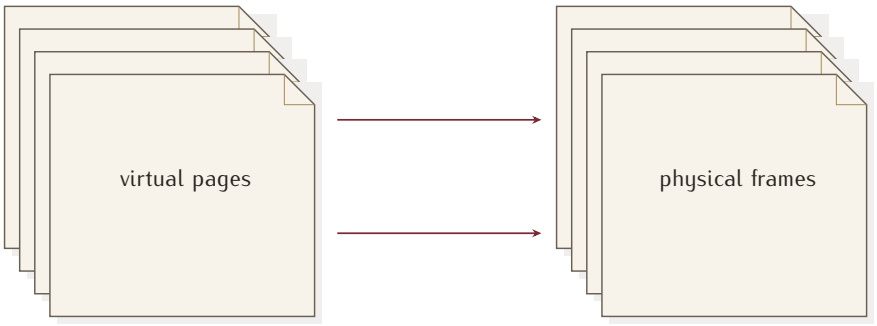
Return to the problem

6.2 VIRTUAL MEMORY AND DEMAND PAGING

6.2.1 Address spaces, pages and frames

Two processes can both issue a load from `0x2345` and reach different bytes. The number belongs to a process's *virtual address space*; the active mapping determines the physical location. The memory-management unit (MMU) translates the address and checks permissions using mapping state established by the operating system. Deliberately giving both processes a mapping to the same physical frame enables controlled sharing. Following the page number and the unchanged byte offset makes both isolation and sharing visible. A virtual address (VA) names a byte in that virtual space; a physical address (PA) identifies its location in the physical-memory model.

Address spaces arrange logical pages over physical storage



A mapping relates page-sized intervals while preserving the byte offset.

Figure 6.11: *The page stacks represent two naming levels. Translation relates virtual pages to physical frames while preserving positions within each page. The mapping can also express isolation and controlled sharing.*

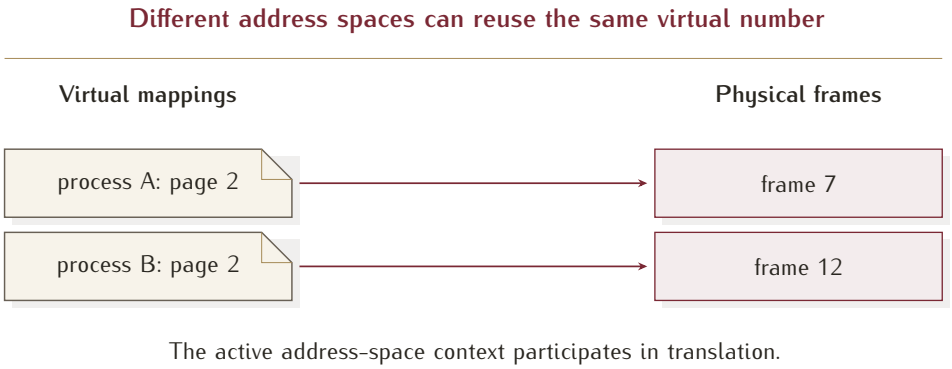


Figure 6.12: *The virtual page number is interpreted within an address space. Equal virtual addresses in two processes can therefore identify different physical storage, with distinct permissions and contents.*

Paging divides the virtual space into fixed-size *pages* and physical memory into equally sized *frames*. For page size $P = 2^b$ bytes,

$$VA = VPN \cdot P + d, \qquad VPN = \lfloor VA/P \rfloor, \qquad 0 \leqslant d < P.$$

The virtual page number VPN selects a page-table entry (PTE). That entry supplies a physical frame number PFN and state such as presence and permission bits. For an allowed resident mapping,

$$PA = PFN \cdot P + d.$$

The offset d survives translation because corresponding positions within a page and a frame have the same byte displacement. The entry address, the entry's encoded value, the frame number and the resulting data address are four separate quantities.

Two processes, one virtual address

With 4 KiB pages, $VA = \texttt{0x2345}$ has $VPN = 2$ and offset `0x345`. Process A maps page 2 to frame 7, yielding `0x7345`. Process B maps page 2 to frame 12, yielding `0xC345`. If both tables instead map page 2 to frame 7, their translations reach the same bytes. Permissions can still differ between the two mappings.

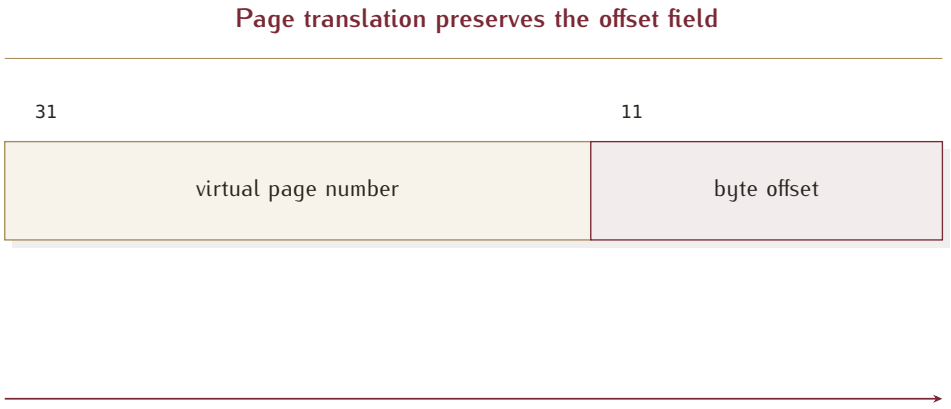


Figure 6.13: *The low twelve bits select a byte within a page. Translation replaces the virtual page number with a physical frame number while preserving those twelve offset bits. Physical address width can differ from virtual address width.*

Figure 6.14 retains those private mappings and adds a shared read-only frame reached through different virtual page numbers.

Two virtual names for a shared byte

Use the shared mappings in Figure 6.14 with 4096-byte pages. Find a virtual address in each process that reads byte `0x20` of the shared frame.

Solution. A uses `0x8020` and B uses `0x6020`. Both translate to `0x14020`, since PFN 20 is decimal. Their virtual page numbers differ, while their page offsets and physical result agree. A store through either mapping violates its read-only permission. Thus sharing specifies both which bytes coincide and which operations each mapping permits.

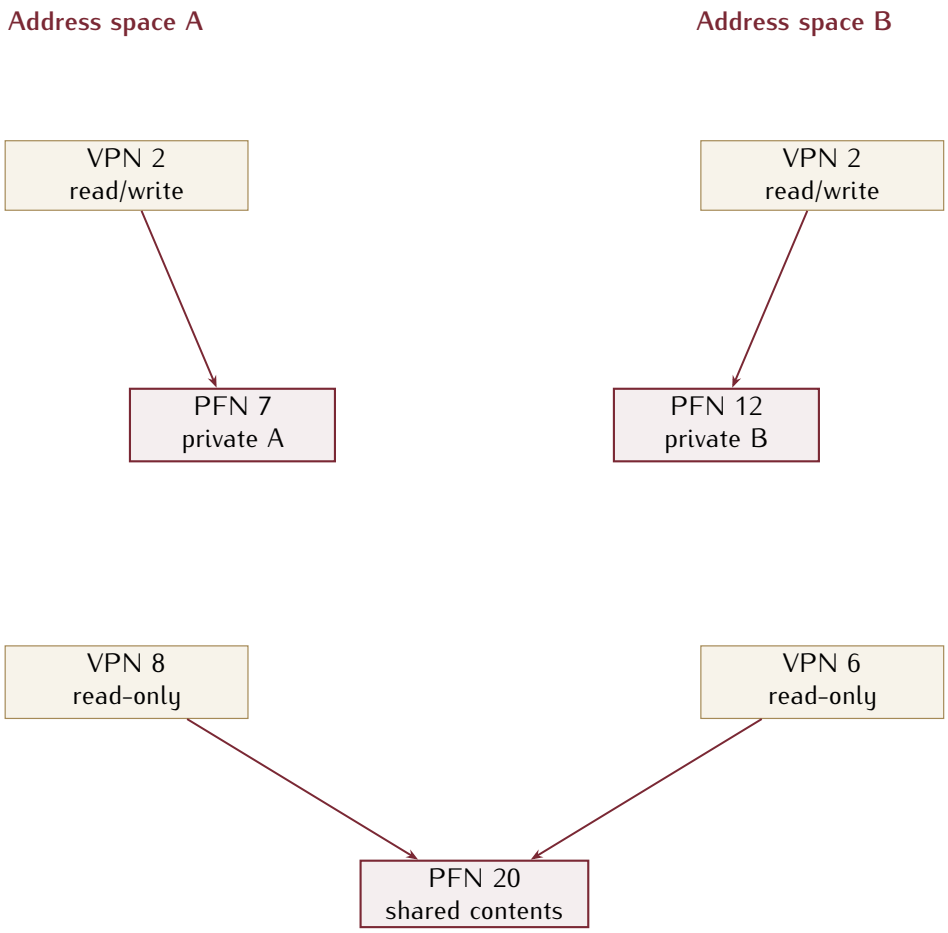


Figure 6.14: *Separate mapping contexts can give the same virtual page different physical frames and give different virtual pages the same shared frame. Offsets are preserved on every arrow. Permissions belong to each mapping; the shared example grants read access in both contexts.*

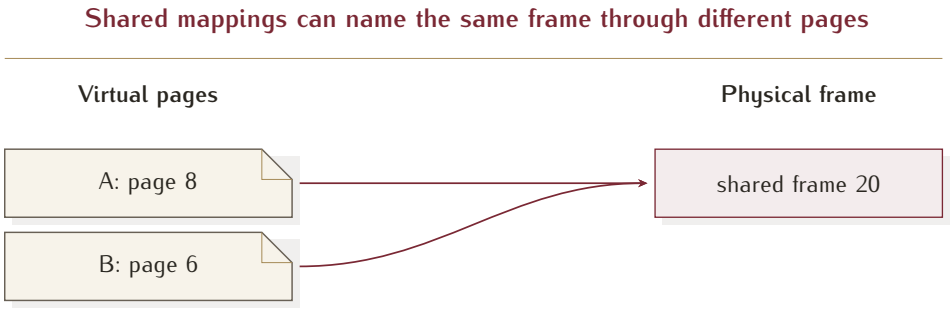


Figure 6.15: *Both mappings lead to the same frame while retaining their own virtual page numbers. Byte offset 32 in either page reaches byte offset 32 in that shared frame, assuming both accesses are permitted.*

6.2.2 Residency, exceptions and demand paging

This chapter uses `valid` to mean that the simplified entry permits a resident translation. In real designs, presence, address-space allocation and read/write/execute permissions have distinct representations. A page-table format assigns specific bit positions to those fields. Chapters 6.3–6.4 define array-based formats explicitly, allowing the address calculations and validity tests to be derived from the stated layouts.

A failed translation or permission check transfers control to an exception handler. The operating system determines whether the address belongs to a permitted region, whether a page must be brought into memory, whether copy-on-write work is appropriate, or whether the access must be rejected. A page fault therefore requires interpretation. The hardware exception alone does not establish that storage I/O will occur.

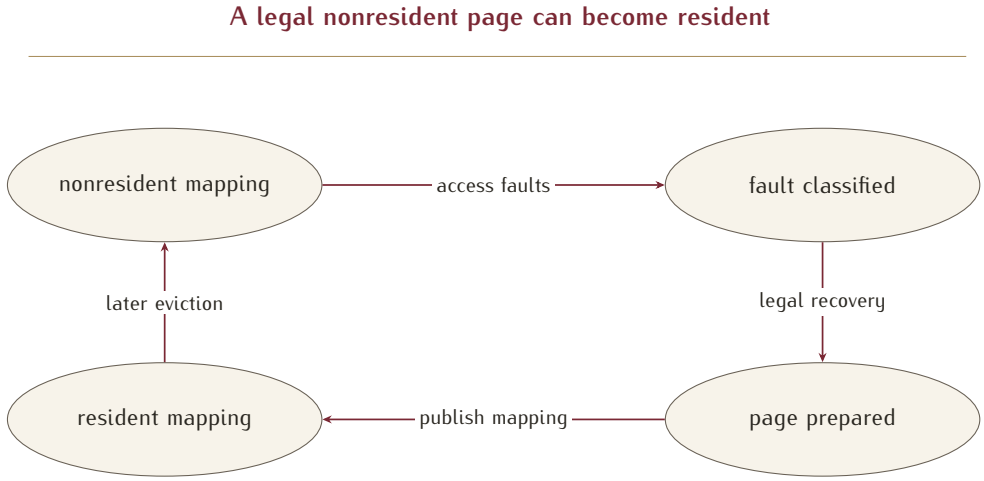


Figure 6.16: *The operating system distinguishes a recoverable residency problem from an invalid or forbidden access. Recovery may fetch contents, zero a page or make a copy depending on the mapping and cause.*

Demand paging populates a resident mapping when an access needs it. For a legal access to a nonresident page, the handler obtains a free frame or selects a victim, revokes the victim's old mappings and cached translations so they cannot access a reused frame, preserves necessary victim data, obtains the requested content, updates the mapping and translation state, then resumes the faulting instruction. A *clean* victim has a reusable backing copy and can be discarded in the model. A *dirty* victim has changed since that copy was made and requires preservation before its frame can be reused. Anonymous pages and implementation-specific backing policies require the same reasoning about where the only current data lives.

Phase	Required result
Fault classification	Access is legal and recovery is possible
Frame selection	A frame is available or a victim has been chosen
Victim revocation	Old references cannot access the frame after reuse
Victim preservation	Modified content is safely retained if required
Page population	The requested page's bytes occupy the frame
Mapping update	PTE and cached translation state agree
Restart	The original instruction can retry its access

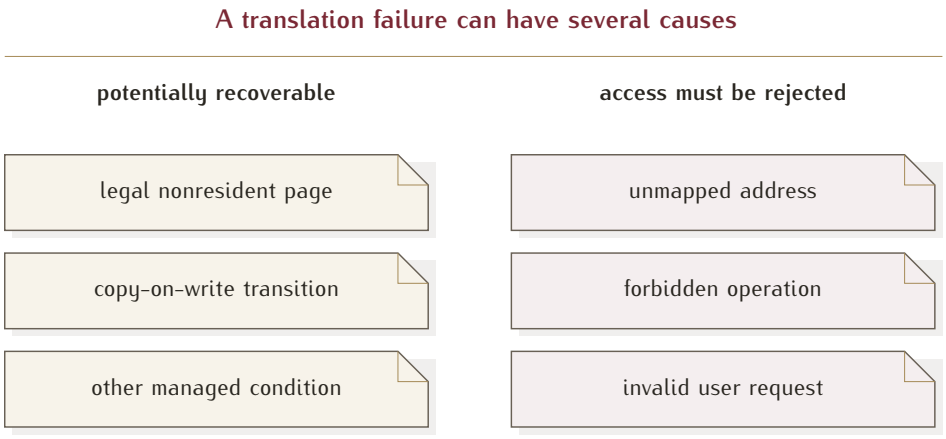


Figure 6.17: *The existence of a fault does not by itself identify the recovery action. The operating system checks the mapping and cause before allocating storage, changing permissions or reporting a failure.*

Figure 6.18 follows one recoverable fault through the participating components. The operating system must preserve the faulting instruction's execution state while other work uses the processor.

What happens to the destination register?

A load into `rax` encounters a legal nonresident page. The fault handler completes the required read and installs a permitted mapping. The interrupted load must then retry: its destination receives the requested value only when the retried instruction completes. Advancing directly to the following instruction would omit the program's read. With a free frame and a zero-filled anonymous page, the handler can initialise the frame locally; the retry requirement remains, while the storage-transfer lane disappears.

6.2.3 Working sets and replacement

For a chosen reference window, a process's *working set* is the set of distinct pages referenced in that window. Its size counts pages, including each page once. Multiplying by page size estimates the resident bytes needed for that set, before table and kernel overhead. The window matters: a set measured over an entire run can hide changes between computational phases.

A legal page fault that requires a storage read

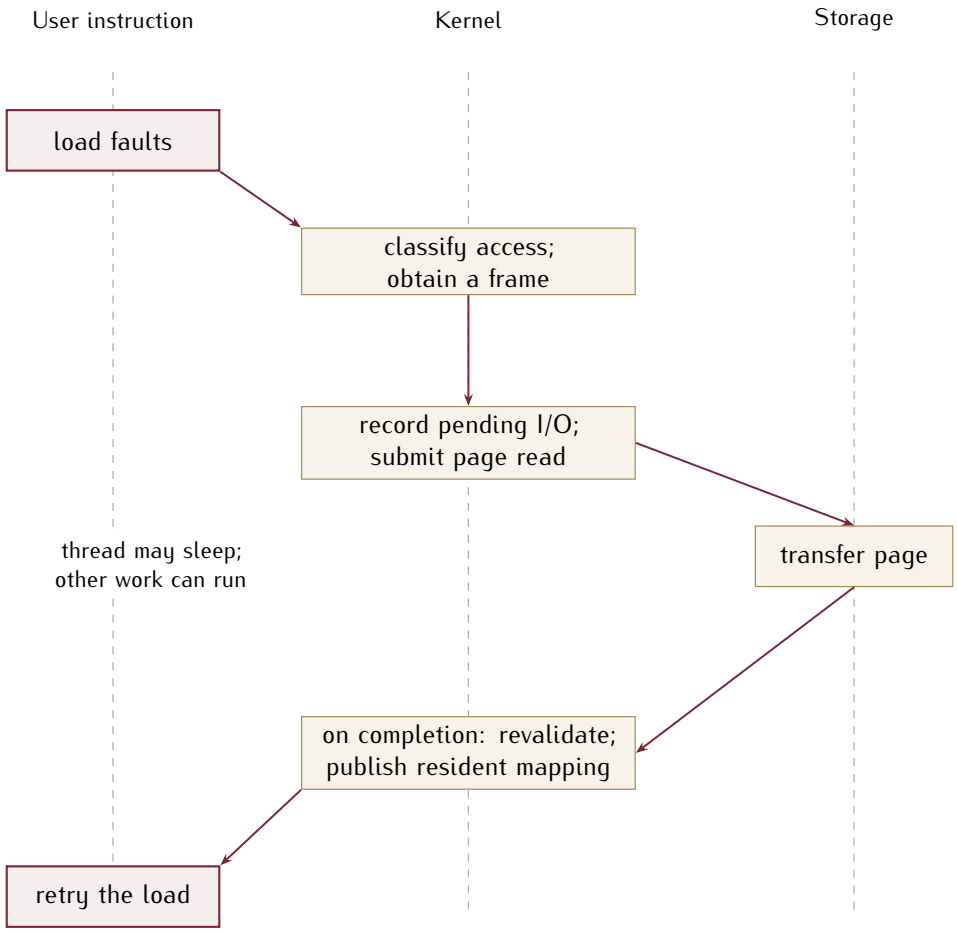


Figure 6.18: *Time runs down the page. Completion tracking is established before the request can complete; the sleeping protocol must avoid a lost wake-up. This path assumes a free frame and a legal file-backed page requiring I/O. Zero-fill and copy-on-write faults can follow different paths without a storage read. The instruction completes only after its retry succeeds.*

A moving window changes the working-set membership

	recent four references	set	size
window A	2,1,3,4	{1,2,3,4}	4
window B	3,4,5,5	{3,4,5}	3

This teaching model uses reference-count windows.

Figure 6.19: *The set contains distinct pages used within the chosen window. Repeated references count once in its membership. The original time-window formulation and this discrete teaching model share the need to state the window explicitly.*

When active processes collectively need substantially more frames than can remain resident, recently evicted pages may be requested again almost immediately. Repeated faults and transfers consume most of the available time; this behaviour is called *thrashing*. Reducing concurrent resident demand can help. A replacement policy uses evidence such as recency and reference bits to choose victims, while an allocation policy determines how many frames each process may use. A faster instruction pipeline cannot remove the storage transfers created by an inadequate resident set.

1968: observing the memory a computation actually uses

Peter Denning's working-set paper approached memory allocation through program behaviour. A running computation creates a demand for processor time and a demand for the information it currently uses. Treating these demands together offered a way to reason about multiprogramming and the repeated page movement that can overwhelm useful execution [25].

Demand changes during a run: a program can finish scanning one region and begin consulting another. Denning expressed recency through a window of process execution time. The discrete trace below instead uses a fixed number of references. In either model, the stated window determines which pages contribute to the estimate.

Consider the reference sequence $1, 2, 1, 3, 4, 3$. A window containing the last four references contains pages $\{1, 3, 4\}$: three distinct pages. After references $5, 5$, the last four are $4, 3, 5, 5$ and the set becomes $\{3, 4, 5\}$. The size remains three while its membership changes. This small trace captures the reason a count alone cannot identify which pages should stay resident.

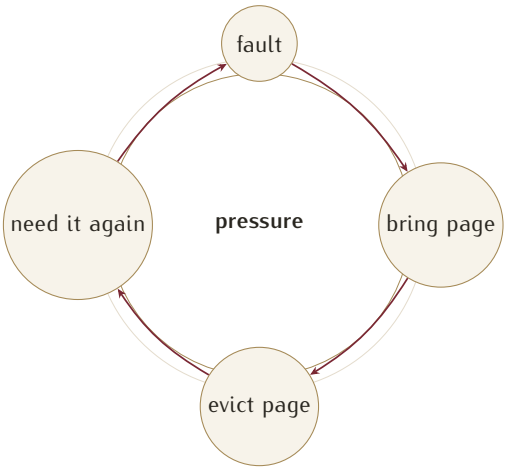


Figure 6.20: *The cycle illustrates thrashing pressure: useful execution repeatedly waits for pages recently displaced. Reducing concurrent demand or changing allocation can help, depending on the workload and replacement policy.*

6.2.4 Accounting for page-fault cost

Let p be the probability that a logical access faults, t_r the total normal-access time and t_f the *total* time for a faulting access, including the specified recovery and retry work. Then

$$\mathrm{EAT} = (1-p)t_r + pt_f.$$

If the fault cost is instead described as an *additional* penalty, use $\mathrm{EAT} = t_r + p \cdot \mathrm{penalty}$. State which convention applies before substituting numbers. Even a small fault probability can dominate because recovery can be orders of magnitude slower than a resident access.

A small probability with a large cost

Take $t_r = 80$ ns, total $t_f = 2{,}000{,}000$ ns and $p = 10^{-4}$. Then $\mathrm{EAT} = 0.9999 \cdot 80 + 0.0001 \cdot 2{,}000{,}000 = 279.992$ ns. Only one access in ten thousand faults on average, yet the mean is about 3.5 times the resident time. The total-fault-time convention prevents counting the normal 80 ns twice on those accesses.

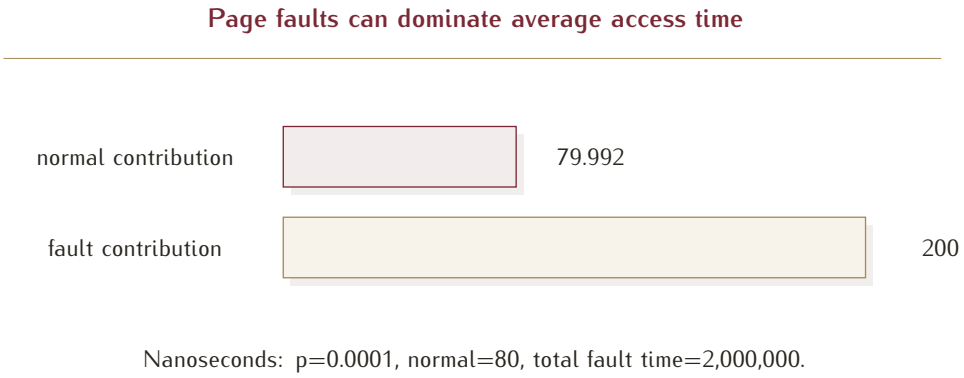


Figure 6.21: *The weighted contributions sum to 279.992 ns. The fault term here is the total cost of a faulting access, so the ordinary access cost is weighted by one minus p rather than counted again on faulting accesses.*

6.2.5 Exercises

Exercise 6.2.1 — Virtual and physical names

Define VA and PA for a load issued by a user process. State which component translates the address and which component establishes the mapping.

Worked solution

Exercise 6.2.2 — Same address in two processes

With 4 KiB pages, both P and Q load from VA `0x3456`. P maps VPN 3 to PFN 8; Q maps VPN 3 to PFN 15. Both accesses are allowed. Give the two physical addresses.

Worked solution

Exercise 6.2.3 — Controlled sharing

P maps VPN 2 and Q maps VPN 9 to the same physical frame 5. Pages are 4096 bytes. Which virtual address in each process reaches offset 100 of that shared frame?

Worked solution

Exercise 6.2.4 — Page and frame terminology

Choose the correct statement: A: pages are fixed physical disk sectors; B: frames are variable-length filenames; C: a virtual page maps to an equally sized physical

Exercise 6.2.4 — Page and frame terminology (continued)

frame; D: a VPN is a byte offset; E: paging requires identical VA and PA values; F: the offset is chosen independently during each translation.

Worked solution

Exercise 6.2.5 — Four-kibibyte pages

For $VA =$ `0x4B9A` and page size 4096 bytes, give VPN and offset as decimal integers. Show the decomposition.

Worked solution

Exercise 6.2.6 — One-kibibyte pages

For $VA =$ `0x2E7D` and page size 1024 bytes, give VPN and offset as decimal integers. Show the decomposition.

Worked solution

Exercise 6.2.7 — Two-kibibyte pages

For $VA =$ `0x6FED` and page size 2048 bytes, give VPN and offset as decimal integers. Show the decomposition.

Worked solution

Exercise 6.2.8 — Eight-kibibyte pages

For $VA =$ `0x17ABC` and page size 8192 bytes, give VPN and offset as decimal integers. Show the decomposition.

Worked solution

Exercise 6.2.9 — A resident mapping

Pages are 4096 bytes. VA `0x6789` maps through a valid entry with PFN 11. Compute the PA and identify the VPN.

Worked solution

Exercise 6.2.10 — A failed validity check

A simplified PTE has valid=0. Which immediate action is correct? A: use VPN as PFN; B: use the stale PFN anyway; C: return zero as file data; D: raise an exception for OS handling; E: erase the process's files; F: assume the page is on disk and complete the load.

Worked solution

Exercise 6.2.11 — A smaller page mapping

With 1024-byte pages, translate VA `0x2DA5` through a valid entry containing PFN 7.

Worked solution

Exercise 6.2.12 — What an entry must express

Explain why a page table needs mapping and access state as well as a frame number. Give a case where the frame-number field alone is insufficient.

Worked solution

Exercise 6.2.13 — Ordering clean-victim recovery

A legal nonresident access faults, no frame is free, and the selected victim is clean with a valid backing copy. Order these tokens: UPDATE, LOAD, RESTART, VICTIM, FAULT. Here VICTIM includes selecting the frame, revoking the victim's resident mappings and completing the required translation invalidations before reuse. UPDATE publishes the incoming page's mapping. Explain each dependency.

Worked solution

Exercise 6.2.14 — Inserting dirty write-back

Repeat the preceding recovery sequence when the selected victim is dirty. Insert token WRITEBACK at the appropriate point and explain why it belongs there.

Worked solution

Exercise 6.2.15 — Discarding a clean victim

A clean victim's frame is reused, and the process requests that page again later. Explain where the page's bytes come from and why clean eviction did not itself lose data.

Worked solution

Exercise 6.2.16 — Retrying the instruction

A load faults on a legal nonresident page. Recovery succeeds. Should execution skip the load or retry it? Explain using a load that is meant to place a value in a register.

Worked solution

Exercise 6.2.17 — Working-set cardinality

Within one reference window, pages are $1, 4, 4, 6, 1, 8, 6, 9$. Find the working set, its cardinality and its size in bytes for 4 KiB pages.

Worked solution

Exercise 6.2.18 — Identifying thrashing

Three active processes each repeatedly use eight pages. Only 18 frames are available for their data, and their sets are disjoint. Explain the capacity pressure and one operating-system response.

Worked solution

Exercise 6.2.19 — Fault-weighted access time

Resident access takes 120 ns. A faulting access takes 1,500,000 ns in total. Fault probability is 0.0002. Compute EAT and the slowdown relative to a resident access.

Worked solution

Exercise 6.2.20 — A frame budget decision

A and B have disjoint working sets of 7 and 9 pages; C needs 6. There are 18 available frames. Which pairs can remain fully resident, and what does that tell you about admitting all three?

Worked solution

6.2.6 Solutions

Solution 6.2.1

The VA names a byte in the process's virtual address space. The PA identifies the corresponding byte in the physical-memory model. The MMU performs ordinary translation using the selected translation structures and cached mappings. The operating system establishes and changes the mappings, including permissions and residency. Separating these roles explains how ordinary loads proceed in hardware while faults and mapping changes require operating-system work.

Return to the problem

Solution 6.2.2

Split the VA as `0x3456` $= 3 \cdot 4096 +$ `0x456`. The offset is unchanged. P obtains $(8 \ll 12)|$`0x456` $=$ `0x8456`; Q obtains $(15 \ll 12)|$`0x456` $=$ `0xF456`. The same virtual number therefore reaches different physical bytes because the two translation contexts select different frame mappings. If both entries instead selected the same frame, this VA would refer to shared storage.

Return to the problem

Solution 6.2.3

P uses $2 \cdot 4096 + 100 = 8292 =$ `0x2064`. Q uses $9 \cdot 4096 + 100 = 36{,}964 =$ `0x9064`. Both translate to $5 \cdot 4096 + 100 = 20{,}580 =$ `0x5064`. Sharing concerns the physical frame, so the virtual page numbers need not match. Appropriate permissions and synchronisation are still required for the intended use of the shared bytes.

Return to the problem

Solution 6.2.4

Answer C. Paging maps a fixed-size virtual page to a frame of the same size while preserving the position within it. A confuses virtual-memory units with storage sectors. B assigns a naming concept to physical memory. D confuses page number with the within-page displacement. E would eliminate the mapping freedom demonstrated by separate process tables. F contradicts offset preservation: changing the displacement would select a different byte of the page.

Return to the problem

Solution 6.2.5

A 4 KiB page occupies 2^{12} bytes. The lowest three hexadecimal digits of `0x4B9A` are therefore the offset, `0xB9A` $= 11 \cdot 256 + 9 \cdot 16 + 10 = 2970$. The leading digit gives VPN 4. Recombining them yields $4 \cdot 4096 + 2970 = 19354$, the decimal value of the original address. The ordered fields are `4, 2970`.

Return to the problem

Solution 6.2.6

A 1024-byte page begins at a multiple of `0x400`. Page 11 begins at $11 \cdot$ `0x400` $=$ `0x2C00`. Subtracting that base from `0x2E7D` leaves `0x27D` $= 637$, below the 1024-byte bound. The ordered fields are `11, 637`. In decimal, $11 \cdot 1024 + 637 = 11901$.

Return to the problem

Solution 6.2.7

With 2048-byte pages, the offset mask is $2048 - 1 =$ `0x7FF`. Masking `0x6FED` gives `0x7ED` $= 2029$. Removing that offset leaves `0x6800`, which equals $13 \cdot 2048$. The ordered fields are `13, 2029`. The byte lies just 19 bytes below the start of page 14, consistent with $2048 - 2029 = 19$.

Return to the problem

Solution 6.2.8

Divide the address `0x17ABC` $= 96956$ by 8192. Eleven whole pages account for $11 \cdot 8192 = 90112$ bytes, leaving $96956 - 90112 = 6844$. This remainder fits in the 13-bit offset field. The ordered fields are `11, 6844`. As a hexadecimal check, the page begins at `0x16000` and the byte is `0x1ABC` beyond it.

Return to the problem

Solution 6.2.9

The low 12 bits give offset `0x789`, or 1929, and the remaining bits give VPN 6. PFN 11 begins at $11 \cdot 4096 = 45{,}056 =$ `0xB000`. Add the offset to get $PA = 45{,}056 + 1929 = 46{,}985 =$ `0xB789`. The virtual page number selects the entry; the entry's frame number determines the physical base. The offset remains `0x789`.

Return to the problem

Solution 6.2.10

Answer D. The translation cannot complete under the simplified validity rule, so hardware raises an exception. The OS then determines whether recovery is legal and possible. A and B bypass the failed mapping. C substitutes invented data. E is unrelated to translation. F assumes both a cause and successful recovery before the OS has classified the fault. The access might refer to a legitimate nonresident page or to an illegal address.

Return to the problem

Solution 6.2.11

A 1024-byte page has ten offset bits. Division gives `0x2DA5` $= 11 \cdot 1024 + 421$, so VPN is 11 and offset is 421, or `0x1A5`. Frame 7 begins at $7 \cdot 1024 = 7168 =$ `0x1C00`. Therefore PA $= 7168 + 421 = 7589 =$ `0x1DA5`. Copying the last three hexadecimal digits as an offset would be wrong for a ten-bit field because three complete hex digits contain twelve bits.

Return to the problem

Solution 6.2.12

The frame number says where resident bytes would be found, while state determines whether and how the access may use that mapping. Presence or validity can forbid using a stale frame field. Permission bits can permit reads while forbidding writes or execution. For example, a write to a resident read-only page must be checked against the policy even when its PFN is known. The format studied in an exercise determines which state bits are represented.

Return to the problem

Solution 6.2.13

The sequence is `FAULT, VICTIM, LOAD, UPDATE, RESTART`. The fault begins recovery. VICTIM selects the frame and prevents old mappings or cached translations from reaching it after reuse; only then may LOAD overwrite it with the incoming page. After the bytes are present, the PTE and required translation state are updated. Restart then retries the instruction using a usable mapping. Since the victim is clean and has a reusable backing copy, this model needs no victim write-back phase.

Return to the problem

Solution 6.2.14

The sequence becomes `FAULT, VICTIM, WRITEBACK, LOAD, UPDATE, RESTART`. VICTIM selects the frame and prevents old users from accessing it during reuse. Its newest bytes must then be preserved before LOAD overwrites the frame with the incoming page. WRITEBACK therefore precedes LOAD in this serial model. UPDATE publishes the completed mapping after the new page is in place, and RESTART follows that update. Omitting preservation would lose modifications held only by the dirty victim.

Return to the problem

Solution 6.2.15

The clean page has an up-to-date reusable backing copy under the stated model. Eviction removes its resident copy and changes the mapping state; it does not destroy that backing copy. A later legal access can fault and reload the bytes into an available frame, possibly a different one. Its VPN remains part of the process's address space, while the chosen PFN can change. A page without any preserved current copy would require a different eviction decision.

Return to the problem

Solution 6.2.16

The faulting load must be retried according to the architecture's restart semantics. Before recovery it could not obtain the requested value; skipping it would leave the destination without the intended result. After the page is populated and the mapping is made usable, restarting the load allows the ordinary access to complete. The handler must preserve the process context needed for that restart. An unrecoverable fault follows a different error path.

Return to the problem

Solution 6.2.17

Removing repeated page numbers gives $W = \{1, 4, 6, 8, 9\}$. Its cardinality is five, although the window contains eight references. Five 4 KiB pages require $5 \cdot 4096 = 20{,}480$ bytes, or 20 KiB, for their resident contents. Page tables and other system memory are additional. A different reference window may produce a different working set.

Return to the problem

Solution 6.2.18

Their combined active demand is $3 \cdot 8 = 24$ frames, six more than the 18 available. If replacement repeatedly removes pages soon needed again, fault traffic can dominate execution: thrashing. Temporarily suspending one process reduces the two active working sets to 16 frames, which fit under the stated assumptions. This is a capacity argument; exact fault counts still depend on the interleaving and replacement policy.

Return to the problem

Solution 6.2.19

Use the total-fault-time formula: $\mathrm{EAT} = (1 - 0.0002)120 + 0.0002 \cdot 1{,}500{,}000 = 119.976 + 300 = 419.976$ ns. The ratio to resident time is $419.976/120 = 3.4998$. The small fault fraction contributes 300 ns to the mean because each fault is expensive. The stated total already includes the faulting access's work, so another 120 ns is not added on that branch.

Return to the problem

Solution 6.2.20

The pair sizes are $A + B = 16$, $A + C = 13$ and $B + C = 15$, so every pair fits within 18 frames. All three require $7 + 9 + 6 = 22$, exceeding the budget by four. The working-set policy can therefore keep any chosen pair fully resident while delaying another process, subject to other system costs and scheduling goals. The arithmetic identifies feasible sets; it does not by itself choose which process should receive priority.

Return to the problem

6.3 ADDRESS TRANSLATION IN CODE

6.3.1 Representing a mapping in a program

A load names a byte, yet reaching that byte requires a mapping. Imagine following the address in `rdi`: its upper field selects a virtual page, while its low bits locate a byte within that page. Translation replaces the page number and preserves the position inside it. An array can make each step visible: select an entry, read its encoded state, establish that the mapping is usable, then compose the result.

Translation code reads real tables to compute a modelled address

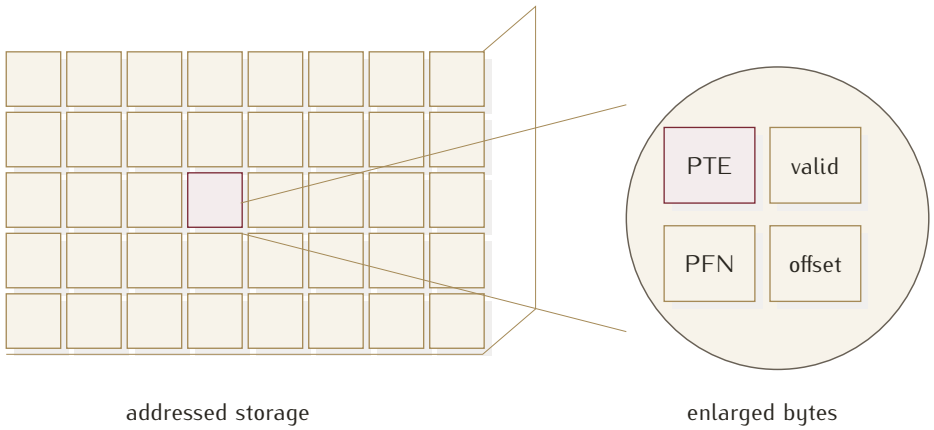


Table storage belongs to the host process; the computed PA belongs to the simulation.

Figure 6.22: *The enlargement distinguishes encoded entry fields from the memory storing them. The routine first reads a valid host-array location, then computes an address value within its declared model. It does not dereference that simulated physical result.*

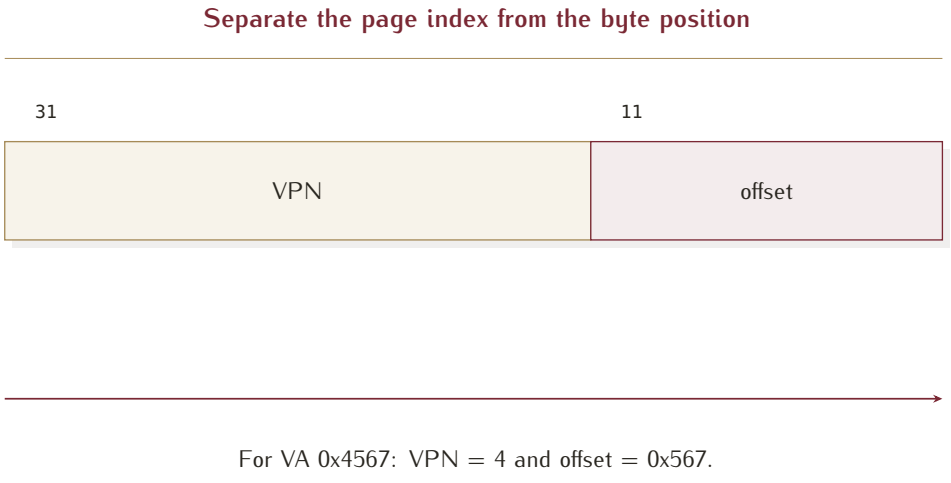


Figure 6.23: *Shifting the address right by twelve yields the page number, while masking with 4095 yields the offset. Recombining those original fields reconstructs the virtual address and provides an independent split check.*

The routines here read their arrays from ordinary process memory and calculate a *simulated* physical-address value. They return that value or a specified failure result; they never dereference the simulated result. Separate paper calculations explicitly identify any table placed in the simulated physical address space. Keeping the host pointer and the simulated address distinct is essential when turning the model into executable code.

For b offset bits, a page holds 2^b bytes. Define $\mathit{mask} = 2^b - 1$. Then

$$\mathrm{VPN} = \mathrm{VA} \gg b, \qquad d = \mathrm{VA} \,\&\, \mathit{mask}.$$

If entries occupy s bytes and the table starts at byte address T, entry VPN begins at $T + s \cdot \mathrm{VPN}$. A load reads the encoded entry. Decoding it yields a physical frame number PFN, and an allowed translation produces

$$\mathrm{PA} = (\mathrm{PFN} \ll b) \mid d.$$

Provided the frame base fits the output width, the shifted frame and offset have disjoint bit positions, so addition produces the same value. For a w-bit output, require $\mathrm{PFN} < 2^{w-b}$; the exercise bounds below satisfy this condition. The routine must also establish that its selected table entry is readable and that the decoded mapping permits the operation.

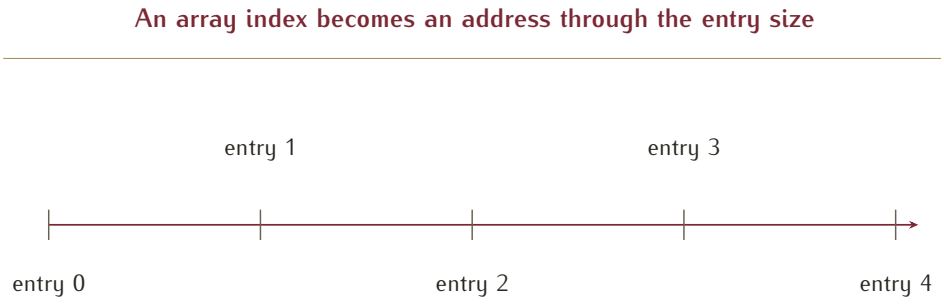


Figure 6.24: *The page-size shift computes the index; the entry-size scale computes its location in the table. Those are different units. With T equal to 0x100000, entry four begins at host address 0x100020.*

Quantity	Interpretation
VPN	Index of a virtual page
$T + sVPN$	Byte address of an entry in the host array
PTE	Encoded value read from that entry
PFN	Number of the simulated physical frame
$PFN \cdot 2^b$	Simulated byte address of the frame's beginning
PA	Simulated address of the requested byte

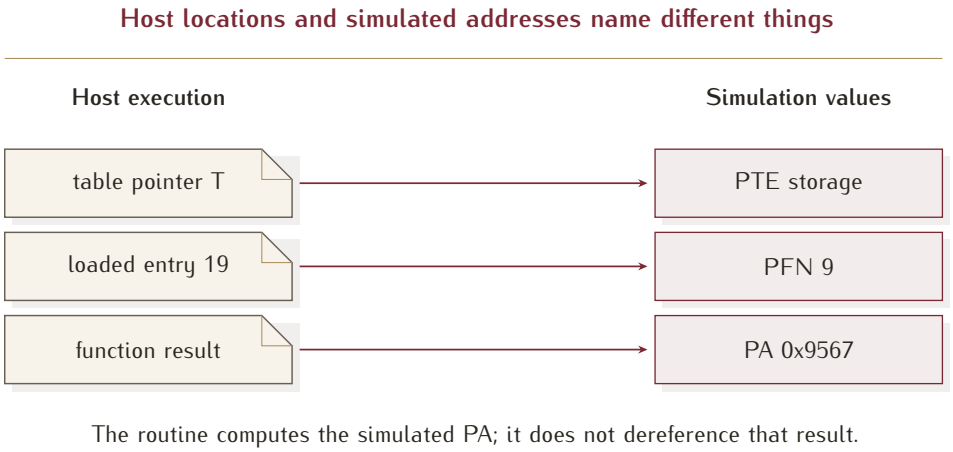


Figure 6.25: *The host pointer must refer to a readable array in the executing process. The calculated physical address is an output value in the simulation. Confusing those domains would turn arithmetic into an unintended memory access.*

6.3.2 Entry formats and validity

The entry format is part of the model. The simplest array stores the PFN directly and assumes every selected entry is usable. A second format stores an eight-byte value

$$\mathrm{PTE} = (\mathrm{PFN} \ll 1) \mid \mathit{valid}, \qquad \mathit{valid} = \mathrm{PTE} \,\&\, 1.$$

Bit zero determines validity; shifting right by one extracts the frame number. These bit positions define the simulated data structure. Hardware page-table formats include architecture-specific permissions and other fields, which must be decoded using their own specification [26, 27].

Following an address through a valid entry

Take 4096-byte pages, $VA = \texttt{0x4567}$, eight-byte entries and table base $T = \texttt{0x100000}$. The split gives $\mathrm{VPN} = 4$ and $d = \texttt{0x567}$. Entry 4 begins at $\texttt{0x100000} + 4 \cdot 8 = \texttt{0x100020}$. If its value is 19, bit zero is one and $19 \gg 1 = 9$ gives the frame number. Therefore $PA = (9 \ll 12) | \texttt{0x567} = \texttt{0x9567}$. The host array location `0x100020` and the simulated result `0x9567` identify different objects in the model.

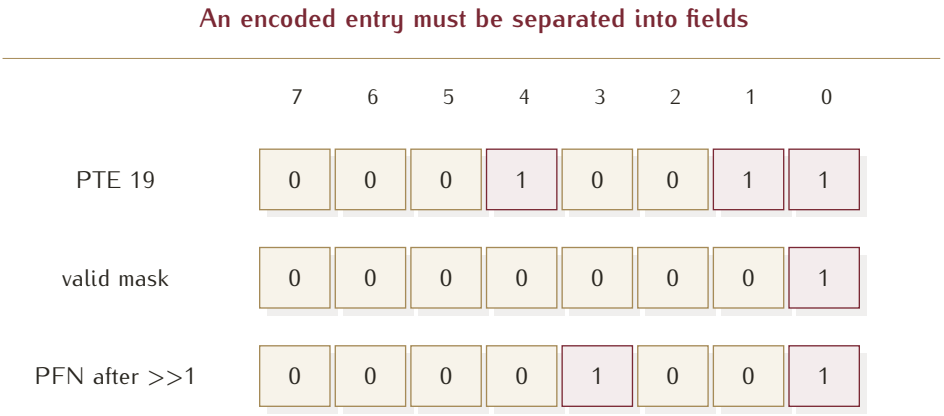


Figure 6.26: *This is the chapter's invented valid-bit encoding, with PFN in the upper bits and validity in bit zero. It is not a hardware page-table format. Bounds must be checked before reading the entry.*

Figure 6.27 follows the same numbers through the four operations. At each panel, identify the kind of quantity before doing arithmetic: an index selects an entry, an address locates its bytes, and an encoded value supplies the mapping.

Rejecting an invalid entry with nonzero frame bits

With 1024-byte pages, entry value 26 has validity $26 \,\&\, 1 = 0$. Its upper bits decode to 13, but that field cannot authorise a translation. The routine returns its failure result. Changing the entry to 27 makes the same frame valid. For offset 1023, the accepted result is $13 \cdot 1024 + 1023 = 14{,}335$.

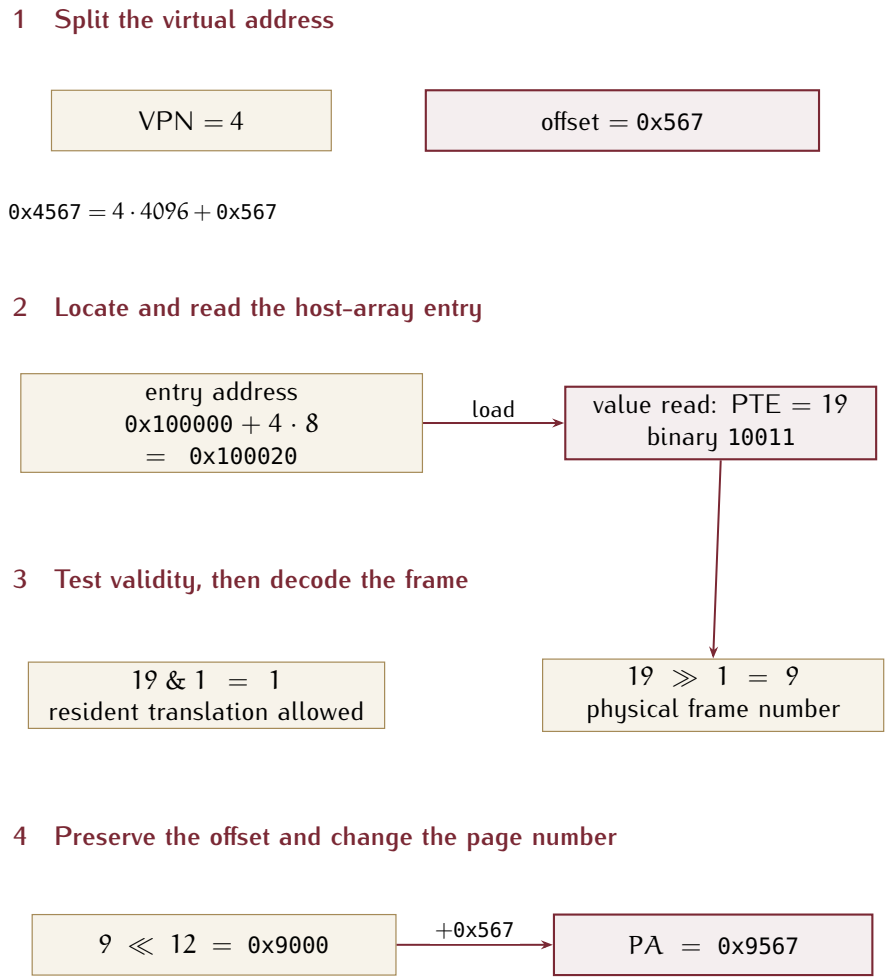


Figure 6.27: *Read the four panels from top to bottom. The host-array entry address identifies the bytes read by the simulation; the final value names a simulated physical byte. The offset survives unchanged. A zero validity bit stops the trace before frame composition.*

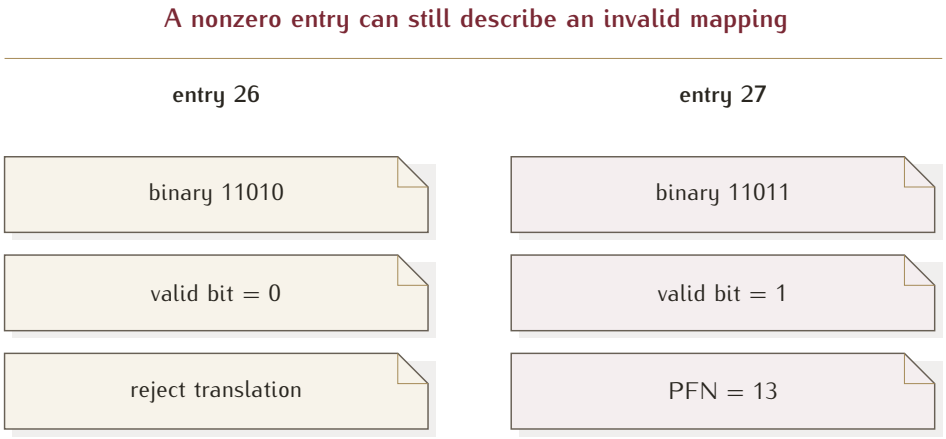


Figure 6.28: *The validity rule inspects its specified bit. A frame-number field with nonzero bits cannot override a zero valid bit. Only an allowed mapping permits the later physical-address composition.*

Checking whether the entire entry is zero would miss the invalid entry 26. The test must select the validity bit. Bounds and validity also answer different questions: bounds establish that reading the entry itself is safe, while validity determines whether its contents describe an allowed mapping.

6.3.3 Implementing the arithmetic

The assembly examples use unsigned, zero-extended address values and logical shifts. In the following frame-composition routine, `rdi` holds VA, `rsi` holds a valid PFN, and `rax` receives PA. The offset has 12 bits; `rcx` and the condition flags are scratch state.

```
mov rax, rsi
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The mask discards the virtual page number before the offset is combined with the physical frame. Writing `ecx` clears the upper half of `rcx`, which is appropriate for this 12-bit field. Without the mask, virtual-page bits could alter the physical-frame field.

A hardware page-table-base register (PTBR in the abstract model) identifies the physical root or base of the active translation structure. A host-array pointer in an assembly simulation plays an analogous indexing role but remains an ordinary virtual pointer within that process.

For a table lookup, first obtain VPN, then use an eight-byte scale to load the selected entry. The page-size shift and entry-size scale serve separate purposes. `lea` computes an entry address; `mov` with a memory operand reads that entry's contents.

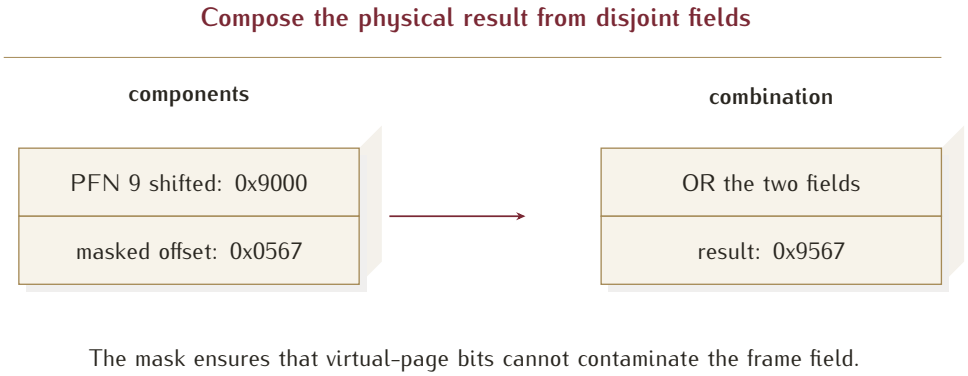


Figure 6.29: *The shifted frame leaves the offset positions zero. The masked offset fills only those positions, so bitwise OR and addition produce the same result when the frame fits the output width.*

For the valid-bit format, `test rax,1` examines the validity bit without destroying the entry. A zero result selects the failure path. On success, shifting right by one removes the encoded validity bit; shifting left by b converts the resulting frame number to a byte address. These two shifts are separate steps in the derivation even when an implementation later combines them.

6.3.4 Failure results, bounds and executable functions

The assembly exercises use `0xFFFFFFFFFFFFFFFF` as a failure result and constrain valid physical addresses to a smaller range. The sentinel therefore cannot be confused with a successful result. `mov rax,-1` sets all 64 bits. `mov eax,-1` instead yields `0x00000000FFFFFFFF` after zero extension and does not satisfy this convention.

When a problem supplies a readable entry for every allowed VPN, that premise permits the array access. A reusable routine accepting arbitrary input needs an explicit table length and must reject an out-of-range index before the load. Appendix C contains a complete assembly function, a C caller, compilation commands and tests for the valid, invalid and out-of-bounds paths. The same appendix explains how the calculation fragments in the exercises become callable functions.

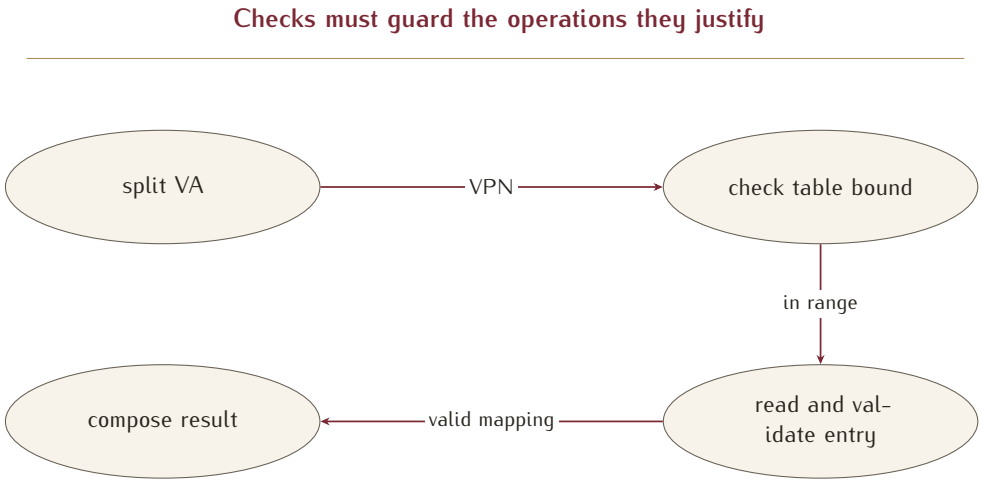


Figure 6.30: *The table-bound check precedes the table load because it justifies that host-memory access. The validity check precedes composition because it establishes that the encoded mapping is usable.*

A useful trace records VA, VPN, offset, entry address, raw entry, validity, decoded PFN and final result. Boundary tests use offset zero, the last offset in a page, adjacent pages, PFN zero and invalid entries with nonzero upper bits. A load that crosses a page boundary needs both pages' mappings and permissions; translating the starting byte alone cannot establish the entire access.

The cost model adds another dependency. With one page-table level, no TLB, no data caches and all pages resident, the entry must be read before the data address is known. The logical access therefore requires one memory read for translation followed by one for data. Chapter 6.4 extends the structure to multiple levels and examines the role of cached translations.

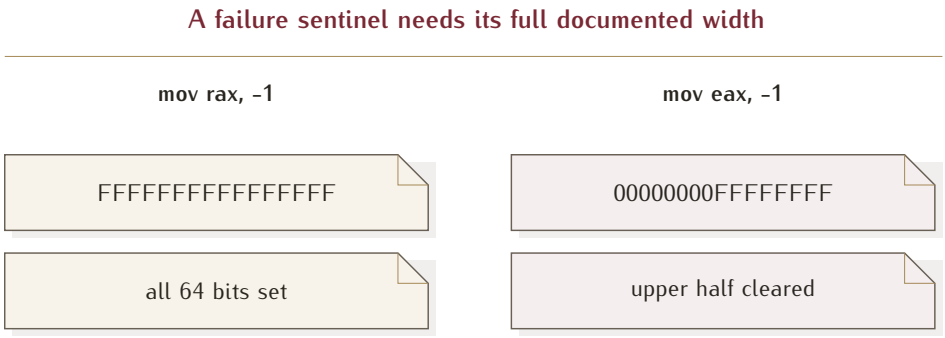


Figure 6.31: *The instruction destination width changes the final RAX value. A sentinel is unambiguous only when the valid-output range excludes it and the implementation returns the exact documented bit pattern.*

6.3.5 Exercises

Exercise 6.3.1 — Split a 4096-byte page address

For VA `0x34567` with 12 offset bits, calculate VPN and offset in decimal. Reconstruct the original address to verify them.

Worked solution

Exercise 6.3.2 — Recovering an address from its fields

A translation trace records VPN 10 and offset 943 with 1024-byte pages. Recover the virtual address in decimal and hexadecimal. A second record gives the same VPN and offset 1024. Determine whether that second pair is a valid decomposition, and normalise it if the two numbers were intended to describe a byte position.

Worked solution

Exercise 6.3.3 — Split a 2048-byte page address

For VA `0x7CDE` with 11 offset bits, calculate VPN and offset in decimal. Reconstruct the original address to verify them.

Worked solution

Exercise 6.3.4 — An access spanning two pages

An eight-byte load begins at VA `0x1FFC`; pages are 4096 bytes. Virtual page 1 maps to physical frame 9 and page 2 maps to frame 20. Both mappings are readable. Identify the virtual and physical byte ranges used by the load. Explain why translating the first byte does not establish a contiguous eight-byte physical interval.

Worked solution

Exercise 6.3.5 — Construct a physical address

Write a general assembly fragment for `rdi`=VA and `rsi`=valid PFN, with 12 offset bits. Return PA in `rax`; `rcx` and flags may change. VA is unsigned 32-bit and $0 \leqslant \text{PFN} < 2^{14}$. Trace VA `0x9025`, PFN 6.

Worked solution

Exercise 6.3.6 — Testing both ends of a physical frame

Construct PA from VA in `rdi` and PFN in `rsi` for 1024-byte pages, using `rax` for the result and `rcx` as scratch. VA is unsigned 32-bit and $0 \leqslant \text{PFN} < 2^{14}$. Derive tests for PFN 6 at offset zero and the last offset. Also trace VA `0x2425`. Explain which defect those boundary tests would reveal.

Worked solution

Exercise 6.3.7 — Diagnosing virtual-page contamination

A fragment for 16 KiB pages shifts PFN left by 14 and ORs it with the entire VA. With VA `0x24025` and PFN 6, diagnose the error and write the corrected fragment. Inputs are in `rdi`/`rsi`, output in `rax`, and `rcx` is scratch. VA is unsigned 32-bit and $0 \leqslant \text{PFN} < 2^{14}$. Explain why a mask for 4 KiB pages would also be unsuitable.

Worked solution

Exercise 6.3.8 — Proving the output range

A routine accepts unsigned 32-bit VA and $0 \leqslant \text{PFN} < 2^{14}$ for 2048-byte pages. Derive the largest possible PA and determine whether a 32-bit unsigned result could represent it. Then implement the composition with VA in `rdi`, PFN in `rsi`, result in `rax` and scratch `rcx`. Trace VA `0x4825`, PFN 6.

Worked solution

Exercise 6.3.9 — Load the indexed PTE

The readable table at `rsi` contains eight-byte entries whose entire value is a PFN. VA is in `rdi`, with 12 offset bits. Return the selected PFN in `rax`. For the trace use table base `0x200000`, VA `0x7013`, and entry 7 equal to 23. Other entries may differ.

Worked solution

Exercise 6.3.10 — Diagnosing an address returned as data

A lookup for 1024-byte pages computes VPN in `rax`, then executes `lea rax,[rsi+rax*8]`. The table at `rsi` stores PFNs in eight-byte entries. For table base `0x200000`, VA `0x1C13` and entry 7 equal to 23, explain the observed result and give a complete corrected lookup fragment with VA initially in `rdi`.

Worked solution

Exercise 6.3.11 — Separating page size from entry size

A lookup uses 2048-byte pages and eight-byte table entries that store PFNs directly. A programmer proposes multiplying VPN by 2048 to find its table entry. Explain which size governs the shift and which governs the array scale. Write the correct fragment for VA in `rdi`, table base in `rsi` and PFN result in `rax`. Trace base `0x200000`, VA `0x3813` and entry 7 equal to 23.

Worked solution

Exercise 6.3.12 — Following the boundary between adjacent entries

Use 8192-byte pages and eight-byte table entries storing PFNs directly. Write a lookup fragment with VA in `rdi`, table base in `rsi` and result in `rax`. Derive the last VA selecting entry 7 and the first selecting entry 8. For table base `0x200000`, trace VA `0xE013` when entry 7 contains 23, and give the two entry addresses at the page boundary.

Worked solution

Exercise 6.3.13 — Invalid is a control decision

A PTE is encoded as $(\mathrm{PFN} \ll 1) | \mathtt{valid}$. Its stored value is 42. A proposed implementation accepts every nonzero PTE. Explain the defect and give the correct decision.

Worked solution

Exercise 6.3.14 — The PTE address

A physical linear table begins at `0x180000`, uses eight-byte PTEs, and the requested VPN is `0x36`. Compute the PTE's physical address in hexadecimal and decimal.

Worked solution

Exercise 6.3.15 — Two dependent RAM accesses

In a one-level translation model without TLB or caches, a RAM access takes 75 ns. The PTE and requested data are both resident. Compute translation plus data access time and explain the dependency.

Worked solution

Exercise 6.3.16 — Changing the active table

P and Q have different page tables. Explain what the abstract PTBR must identify when Q starts running, and why cached translations require attention too.

Worked solution

Exercise 6.3.17 — A complete walker with 12 offset bits

Use `rdi`=VA and `rsi`=base of a readable eight-byte-PTE array. Encode each entry as $(\text{PFN} \ll 1)|\mathit{valid}$, with $\text{PFN} < 2^{14}$. Return PA in `rax` if valid, otherwise `0xFFFFFFFFFFFFFFFF`. `rcx` and flags may change. Implement the general fragment, then trace VA `0x3033` with raw PTE 35 and with raw PTE 34.

Worked solution

Exercise 6.3.18 — A complete walker with 10 offset bits

Use `rdi`=VA and `rsi`=base of a readable eight-byte-PTE array. Encode each entry as $(\text{PFN} \ll 1)|\mathit{valid}$, with $\text{PFN} < 2^{14}$. Return PA in `rax` if valid, otherwise `0xFFFFFFFFFFFFFFFF`. `rcx` and flags may change. Implement the general fragment, then trace VA `0xC33` with raw PTE 35 and with raw PTE 34. What does the routine return for a raw PTE of 1 at the same VA?

Worked solution

Exercise 6.3.19 — A complete walker with 11 offset bits

Use `rdi`=VA and `rsi`=base of a readable eight-byte-PTE array. Encode each entry as $(\text{PFN} \ll 1)|\mathit{valid}$, with $\text{PFN} < 2^{14}$. Return PA in `rax` if valid, otherwise `0xFFFFFFFFFFFFFFFF`. `rcx` and flags may change. Implement the general fragment, then trace VA `0x1833` with raw PTE 35 and with raw PTE 34. Which precondition fails if the caller supplies only three entries?

Exercise 6.3.19 — A complete walker with 11 offset bits (continued)

Worked solution

Exercise 6.3.20 — A complete walker with 13 offset bits

Use `rdi`=VA and `rsi`=base of a readable eight-byte-PTE array. Encode each entry as $(\text{PFN} \ll 1)|valid$, with $\text{PFN} < 2^{14}$. Return PA in `rax` if valid, otherwise `0xFFFFFFFFFFFFFFFF`. `rcx` and flags may change. Implement the general fragment, then trace VA `0x6033` with raw PTE 35 and with raw PTE 34. Explain the effect of replacing `mov rax,-1` with `mov eax,-1`.

Worked solution

6.3.6 Solutions

Solution 6.3.1

The page size is $2^{12} = 4096$ bytes and its offset mask is `0xFFF`. Thus VPN $=$ `0x34567` $\gg 12 = 52$ and d $=$ `0x34567` & `0xFFF` $= 1383$. The resulting pair is `52, 1383`. Reconstruction gives $52 \cdot 4096 + 1383 = 214375 =$ `0x34567`. The low field is a byte displacement, so it must be smaller than the page size.

Return to the problem

Solution 6.3.2

The first address is $10 \cdot 1024 + 943 = 11{,}183 =$ `0x2BAF`. Its offset satisfies $0 \leqslant 943 < 1024$. The second pair violates the offset bound because 1024 is one complete page. Its arithmetic value is $10 \cdot 1024 + 1024 = 11 \cdot 1024 = 11{,}264 =$ `0x2C00`. The normalised fields are therefore VPN 11 and offset 0. A field decomposition must satisfy both the reconstruction equation and the range of each field.

Return to the problem

Solution 6.3.3

The page size is $2^{11} = 2048$ bytes and its offset mask is `0x7FF`. Thus VPN $=$ `0x7CDE` $\gg 11 = 15$ and d $=$ `0x7CDE` & `0x7FF` $= 1246$. The resulting pair is `15, 1246`. Reconstruction gives $15 \cdot 2048 + 1246 = 31966 =$ `0x7CDE`. The low field is a byte displacement, so it must be smaller than the page size.

Return to the problem

Solution 6.3.4

The virtual interval is `0x1FFC` through `0x2003`. Its first four bytes have VPN 1 and offsets `0xFFC`–`0xFFF`, so they map to `0x9FFC`–`0x9FFF`. Its remaining four bytes have VPN 2 and offsets 0–3, so they map to `0x14000`–`0x14003`, because frame 20 begins at $20 \cdot 4096 =$ `0x14000`. The ranges contain $4 + 4 = 8$ bytes but are physically discontiguous. The processor must obtain and check the second mapping as well; a valid first mapping alone says nothing about the second page. Byte order determines how those bytes form the loaded value after access is permitted.

Return to the problem

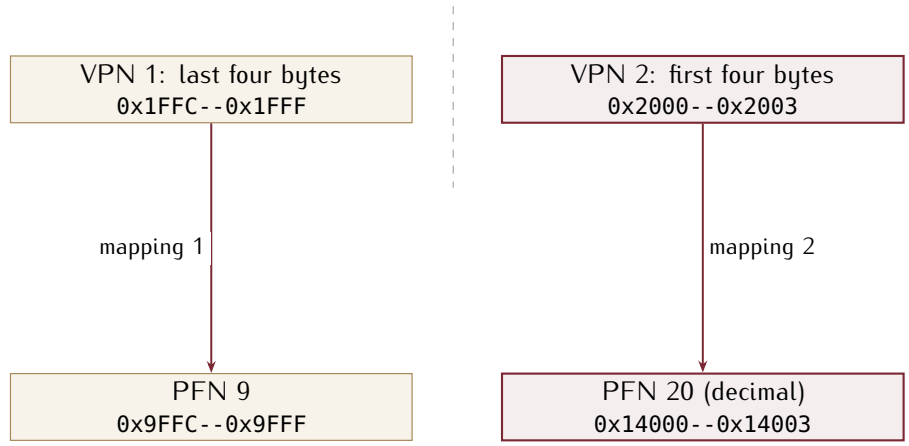


Figure 6.32: *A page boundary splits an access even though its virtual addresses are consecutive. The two physical ranges lie in different frames. Their drawn positions represent correspondence, rather than physical spacing.*

Solution 6.3.5

```
mov rax, rsi
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The shift converts PFN 6 into frame base $6 \cdot 4096 = 24576$. The mask retains offset 37 from the VA, producing $PA = 24576 + 37 = 24613 =$ `0x6025`. Writing `ecx` clears its high bits. The two fields occupy disjoint bit positions, so OR combines them without changing either. The code depends on the inputs and works for every allowed PFN and offset.

Return to the problem

Solution 6.3.6

```
mov rax, rsi
shl rax, 10
mov rcx, rdi
and ecx, 1023
or rax, rcx
```

The shift converts PFN 6 into frame base $6 \cdot 1024 = 6144$. The mask retains offset 37 from the VA, producing $PA = 6144 + 37 = 6181 =$ `0x1825`. Writing `ecx` clears its high bits. For this PFN, offset zero gives 6144; offset 1023 gives 7167. Both remain in frame 6. These boundaries test the mask independently of the virtual page number.

Return to the problem

Solution 6.3.7

```
mov rax, rsi
shl rax, 14
mov rcx, rdi
and ecx, 16383
or rax, rcx
```

The shift converts PFN 6 into frame base $6 \cdot 16384 = 98304$. The mask retains offset 37 from the VA, producing $PA = 98304 + 37 = 98341 =$ `0x18025`. Writing `ecx` clears its high bits. OR-ing the unmasked VA produces `0x18000|0x24025` = `0x3C025`, which selects frame 15 instead of frame 6. Masking with 16383 first removes every

Worked solution (continued)

VPN bit. A mask for 12-bit pages would also discard two legitimate offset bits in this 14-bit format.

Return to the problem

Solution 6.3.8

```
mov rax, rsi
shl rax, 11
mov rcx, rdi
and ecx, 2047
or rax, rcx
```

The shift converts PFN 6 into frame base $6 \cdot 2048 = 12288$. The mask retains offset 37 from the VA, producing PA $= 12288 + 37 = 12325 =$ `0x3025`. Writing `ecx` clears its high bits. The largest permitted PFN is $2^{14} - 1$ and the largest offset is $2^{11} - 1$, giving maximum PA $2^{25} - 1 = 33554431$. The declared inputs therefore fit a 32-bit unsigned result as well as the 64-bit output register.

Return to the problem

Solution 6.3.9

```
mov rax, rdi
shr rax, 12
mov rax, [rsi + rax*8]
```

Shifting VA right by 12 yields VPN 7 and discards offset 19. Entry 7 is $7 \cdot 8 = 56 =$ `0x38` bytes after the base, at `0x200038`. The qword load returns 23 in `rax`. The final instruction performs an eight-byte memory read. There is one indexed load after the VPN has been derived.

Return to the problem

Solution 6.3.10

```
mov rax, rdi
shr rax, 10
mov rax, [rsi + rax*8]
```

Shifting VA right by 10 yields VPN 7 and discards offset 19. Entry 7 is $7 \cdot 8 = 56 =$ `0x38` bytes after the base, at `0x200038`. The qword load returns 23 in `rax`.

Worked solution (continued)

Replacing the final `mov` by `lea` would return `0x200038`, the PTE's address. The requested output is 23, the contents stored at that address.

Return to the problem

Solution 6.3.11

```
mov rax, rdi
shr rax, 11
mov rax, [rsi + rax*8]
```

Shifting VA right by 11 yields VPN 7 and discards offset 19. Entry 7 is $7 \cdot 8 = 56 =$ `0x38` bytes after the base, at `0x200038`. The qword load returns 23 in `rax`. The scale is 8 because each table element is eight bytes. The page size determines the shift, while the PTE size determines the scale. Interchanging those sizes would index outside the intended entry.

Return to the problem

Solution 6.3.12

```
mov rax, rdi
shr rax, 13
mov rax, [rsi + rax*8]
```

Shifting VA right by 13 yields VPN 7 and discards offset 19. Entry 7 is $7 \cdot 8 = 56 =$ `0x38` bytes after the base, at `0x200038`. The qword load returns 23 in `rax`. The last VA in page 7 is $8 \cdot 8192 - 1 = 65535$ and still uses entry 7. The next VA, 65536, uses entry 8 at `0x200040`. The offset never contributes to the table index.

Return to the problem

Solution 6.3.13

The validity bit is the low bit, so $42 \,\&\, 1 = 0$. This mapping is invalid even though the full entry is nonzero. The upper field would decode to $42 \gg 1 = 21$, but the contract forbids using it on the invalid path. Test bit zero with `test rax,1` and branch on zero to the failure result. A full-value zero test would incorrectly accept many invalid entries carrying stale frame bits.

Return to the problem

Solution 6.3.14

VPN `0x36` is 54. Its byte displacement is $54 \cdot 8 = 432 =$ `0x1B0`. Add it to the base: `0x180000+0x1B0=0x1801B0`. In decimal, the base is 1,572,864 and the sum is 1,573,296. This locates the table entry; a subsequent read and decoding step are needed to derive the data address.

Return to the problem

Solution 6.3.15

The first 75 ns reads the PTE using the table base and VPN. Its result supplies the physical frame, so the data address cannot be completed until that read finishes. The second 75 ns reads the requested data. Total modelled time is $75 + 75 = 150$ ns. There is no page-fault service in this case; residency was given. The calculation assumes serial accesses and excludes other overhead.

Return to the problem

Solution 6.3.16

PTBR must select the root or base of Q's table, so future walks use Q's mapping. Leaving P's base would translate Q's virtual references using P's context. Existing TLB entries also require a valid context distinction or the appropriate invalidation; otherwise a cached P translation could bypass the new table selection. The abstract rule is that both walked and cached translations must belong to the running address space.

Return to the problem

Solution 6.3.17

```
mov rax, rdi
shr rax, 12
mov rax, [rsi + rax*8]
test rax, 1
jz .Linvalid
shr rax, 1
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
jmp .Ldone
.Linvalid:
mov rax, -1
.Ldone:
```

Worked solution (continued)

The VPN is 3 and the offset is 51, so the selected entry is at base+24. With PTE 35, bit zero is one and $35 \gg 1 = 17$ gives the PFN. Frame base is $17 \cdot 4096 = 69632$; the PA is 69683, or `0x11033`. With PTE 34, bit zero is zero, and the branch returns the 64-bit all-ones sentinel before composing any PA. PFN zero with valid=1 remains a successful mapping. The code preserves `rdi` and `rsi` and uses no stack or syscall.

Return to the problem

Solution 6.3.18

```
mov rax, rdi
shr rax, 10
mov rax, [rsi + rax*8]
test rax, 1
jz .Linvalid
shr rax, 1
shl rax, 10
mov rcx, rdi
and ecx, 1023
or rax, rcx
jmp .Ldone
.Linvalid:
mov rax, -1
.Ldone:
```

The VPN is 3 and the offset is 51, so the selected entry is at base+24. With PTE 35, bit zero is one and $35 \gg 1 = 17$ gives the PFN. Frame base is $17 \cdot 1024 = 17408$; the PA is 17459, or `0x4433`. With PTE 34, bit zero is zero, and the branch returns the 64-bit all-ones sentinel before composing any PA. PFN zero with valid=1 remains a successful mapping. The code preserves `rdi` and `rsi` and uses no stack or syscall. PTE 1 has valid bit one and decodes to PFN zero. Its frame base is zero, so the successful result is the offset, $51 =$ `0x33`. This test distinguishes a valid mapping into frame zero from the all-ones failure result.

Return to the problem

Solution 6.3.19

```
mov rax, rdi
shr rax, 11
mov rax, [rsi + rax*8]
test rax, 1
jz .Linvalid
```

Worked solution (continued)

```
shr rax, 1
shl rax, 11
mov rcx, rdi
and ecx, 2047
or rax, rcx
jmp .Ldone
.Linvalid:
mov rax, -1
.Ldone:
```

The VPN is 3 and the offset is 51, so the selected entry is at base+24. With PTE 35, bit zero is one and $35 \gg 1 = 17$ gives the PFN. Frame base is $17 \cdot 2048 = 34816$; the PA is 34867, or `0x8833`. With PTE 34, bit zero is zero, and the branch returns the 64-bit all-ones sentinel before composing any PA. PFN zero with valid=1 remains a successful mapping. The code preserves `rdi` and `rsi` and uses no stack or syscall. A three-entry array has valid indices 0, 1 and 2. The trace selects VPN 3, so its load at base+24 would begin one byte position beyond the array's final byte (offset 23). A length-aware wrapper must reject index 3 before loading the PTE, even if memory beyond the array happens to contain an odd value.

Return to the problem

Solution 6.3.20

```
mov rax, rdi
shr rax, 13
mov rax, [rsi + rax*8]
test rax, 1
jz .Linvalid
shr rax, 1
shl rax, 13
mov rcx, rdi
and ecx, 8191
or rax, rcx
jmp .Ldone
.Linvalid:
mov rax, -1
.Ldone:
```

The VPN is 3 and the offset is 51, so the selected entry is at base+24. With PTE 35, bit zero is one and $35 \gg 1 = 17$ gives the PFN. Frame base is $17 \cdot 8192 = 139264$; the PA is 139315, or `0x22033`. With PTE 34, bit zero is zero, and the branch returns the 64-bit all-ones sentinel before composing any PA. PFN zero with valid=1 remains a successful mapping. The code preserves `rdi` and `rsi` and uses no stack or syscall.

Worked solution (continued)

The 32-bit write clears the upper half of RAX. It would return `0x00000000FFFFFFFF`, whereas the specified failure value is `0xFFFFFFFFFFFFFFFF`. A caller comparing the full 64-bit result would miss the failure.

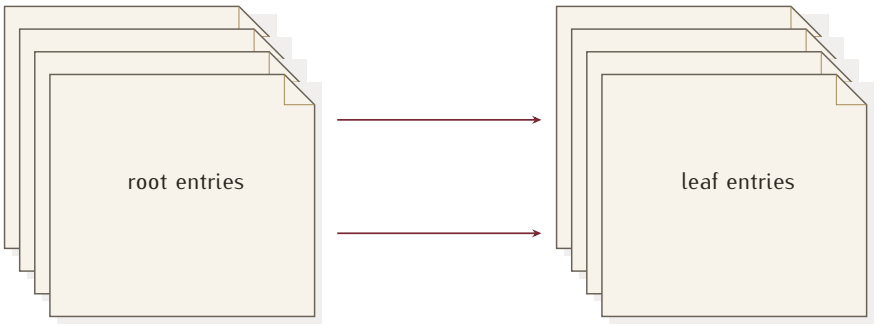
Return to the problem

6.4 TLBS AND MULTILEVEL PAGE TABLES

6.4.1 Caching translations

The array loop has already translated an address on its current page. A nearby access can reuse that translation if it remains in the *translation lookaside buffer* (TLB), together with the relevant access attributes. Otherwise, the mapping must be obtained through the page-table mechanism. A successful resident walk and a page-fault exception lead to different work, so we will trace them separately. A cached translation can also reject an access whose permissions are insufficient.

A page-table hierarchy links several sets of entries



One loaded entry selects the table used by the following lookup.

Figure 6.33: *The two stacks represent levels in a translation hierarchy. The first selection supplies the next table's base; the later selection supplies a final mapping. A cached translation can bypass this dependent walk under its validity rules.*



Figure 6.34: *The same virtual page number can require a different result in another address space. TLB organisation and context-management rules must prevent one process from using another process's stale translation.*

Many adjacent byte references fall within the same page, and repeated visits to that page reuse its translation. This gives spatial and temporal locality a role at page granularity as well as cache-line granularity. A real TLB matches tags in associative hardware. The linear search exercises below use four records in an ordinary array to make the matching rule visible. All simulated frame numbers in this chapter satisfy $0 \leqslant \text{PFN} < 2^{14}$, so they are distinct from the all-ones search-failure result and their shifted physical addresses fit in 64 bits. Their loop time is unrelated to a real TLB's associative lookup latency.

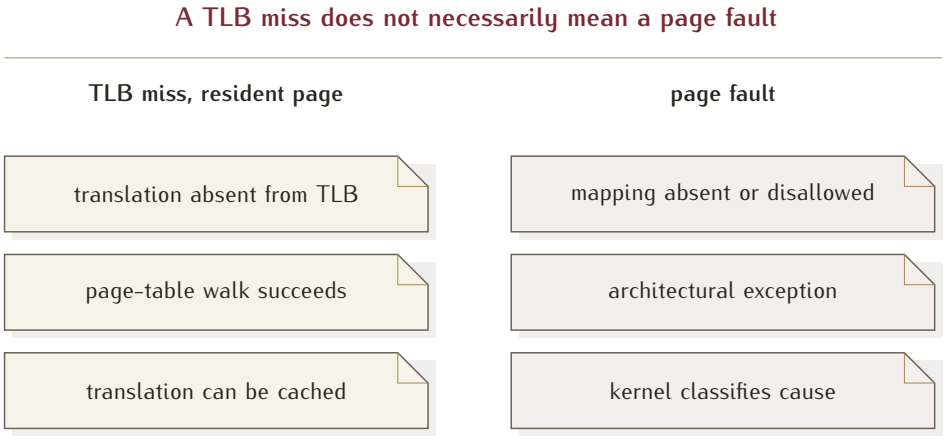


Figure 6.35: *The TLB is a translation cache. Missing from that cache says nothing by itself about residency or permission in the page tables. The later lookup and access checks determine the outcome.*

Translation identity includes the address-space context. After a context switch, old entries must either be distinguished by an appropriate context tag or invalidated according to the architecture's rules. After changing a mapping, software must make the required cached translation state consistent before relying on the new mapping. The simplified trace exercises assume one fixed context and no concurrent mapping changes.

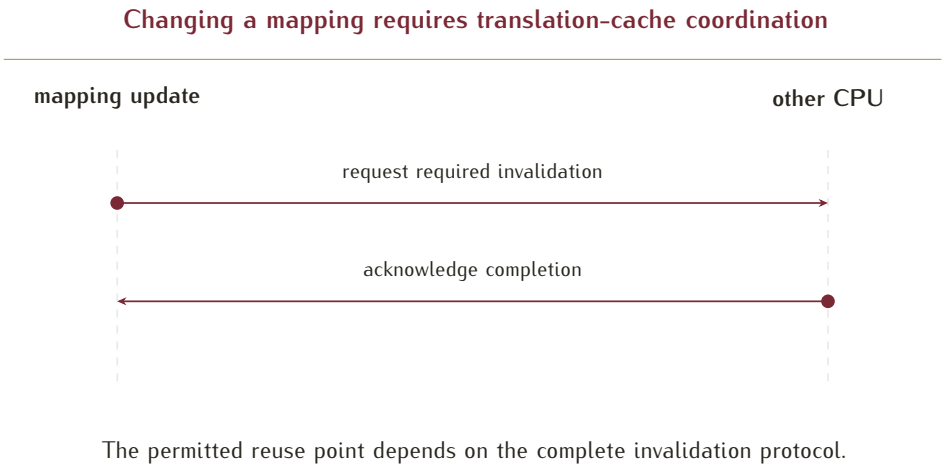


Figure 6.36: *An old cached translation can outlive a changed page-table entry unless the architecture's coordination requirements are met. Safe frame reuse therefore includes the necessary local and remote translation-cache handling.*

6.4.2 Why a linear table becomes large

An n-bit virtual space with 2^b-byte pages has 2^{n-b} virtual pages. A complete linear table with s bytes per entry therefore occupies

$$S = 2^{n-b} s \text{ bytes.}$$

Most processes use only selected portions of a large virtual space. A multilevel table can omit lower tables for unused regions. The top-level index selects a pointer to a lower-level table; that table's index selects another entry. The final leaf supplies the frame mapping. Each pointer load depends on the preceding index and table selection.

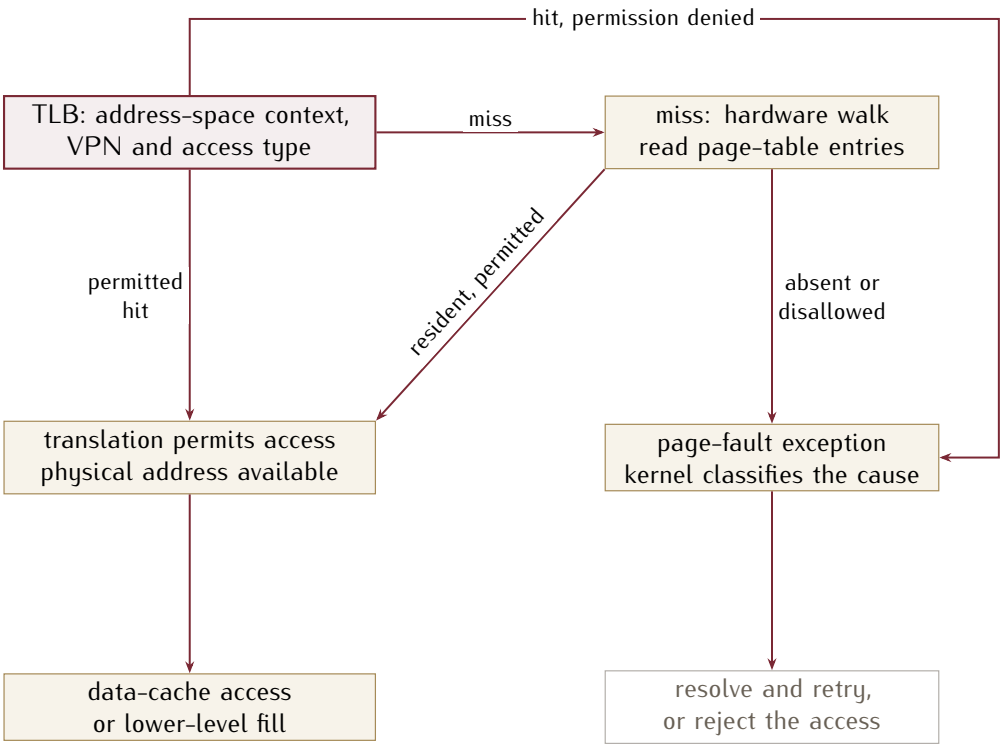


Figure 6.37: *Three principal translation paths in a hardware-walk design: a permitted TLB hit, a miss resolved by resident page tables, and a fault requiring kernel action. Permission failure can arise on a hit or during a walk. A successful walk can populate the TLB. Logical dependencies are shown; implementations may overlap translation with a virtually indexed cache lookup.*

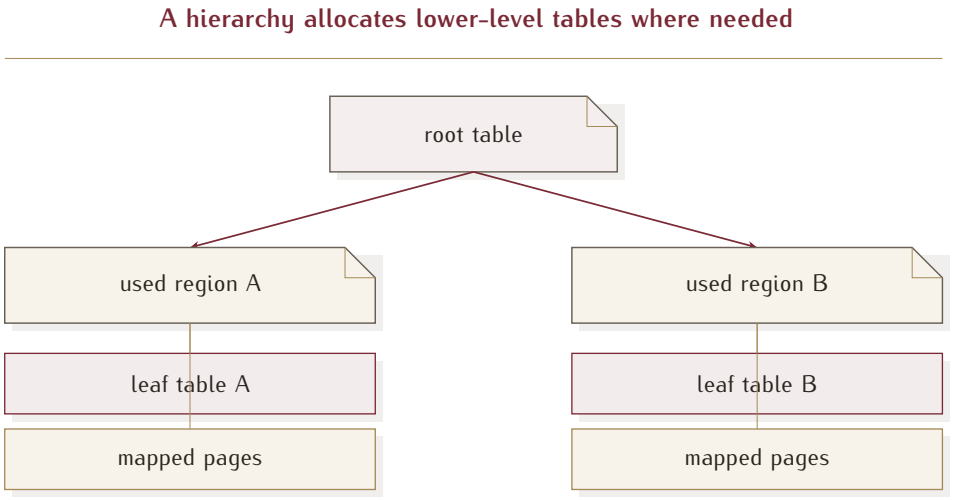


Figure 6.38: *The hierarchy trades additional dependent lookup steps for more selective allocation of table storage. Its benefit depends on how the virtual address space is populated and on the table geometry.*

For a two-level format with p_1 upper-index bits, p_2 lower-index bits and b offset bits, $n = p_1 + p_2 + b$ and

$$i_1 = VA \gg (p_2 + b), \quad i_2 = (VA \gg b) \,\&\, (2^{p_2} - 1), \quad d = VA \,\&\, (2^b - 1).$$

These formulas assume that bits above the specified n-bit VA are zero. A routine receiving a wider value must follow its input contract. Reject nonzero excess bits if they make the address invalid. Mask them only when the contract explicitly defines truncation. Silent truncation could otherwise turn an invalid address into a different valid one.

Counting allocated tables

Take a 32-bit address space split as 10/10/12, with four-byte entries. Each table has $2^{10} = 1024$ entries and occupies 4096 bytes. If only three distinct upper-level regions contain mappings, allocate one root and three lower tables: $4 \cdot 4096 = 16$ KiB. The complete linear table would occupy $2^{20} \cdot 4 = 4$ MiB. If all 1024 regions need lower tables, the two-level structure instead occupies 1025 pages and includes root overhead. Sparsity determines the saving.

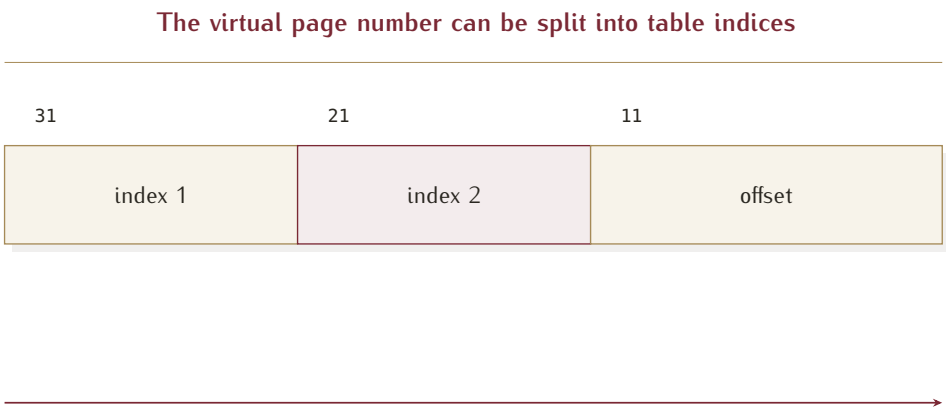


Figure 6.39: *The first index selects a root entry, the second selects an entry in the selected lower-level table, and the offset remains within the final page. Actual architectures specify their own field widths and entry formats.*

6.4.3 A precise simulated two-level walk

The exercises use eight-byte entries, a root pointer in `rsi`, and a zero-extended 32-bit VA in `rdi`. A root entry contains a directly usable pointer to its L2 array. A leaf entry contains $(\mathrm{PFN} \ll 1)|1$. Both levels are allocated and valid for the requested address. The program reads

$$T_2 = M_8[T_1 + 8i_1], \quad \mathrm{PTE} = M_8[T_2 + 8i_2], \quad \mathrm{PA} = ((\mathrm{PTE} \gg 1) \ll 12)|d.$$

Here $M_8[a]$ means an eight-byte load at byte address a. Unlike a root entry in many hardware formats, the simulated pointer has no flag bits to remove. A 10/10/12 split with eight-byte entries makes each table 8 KiB in this exercise; the tables need not fit in one 4 KiB page. Treating it as the four-byte teaching example above would halve the entry addresses incorrectly.

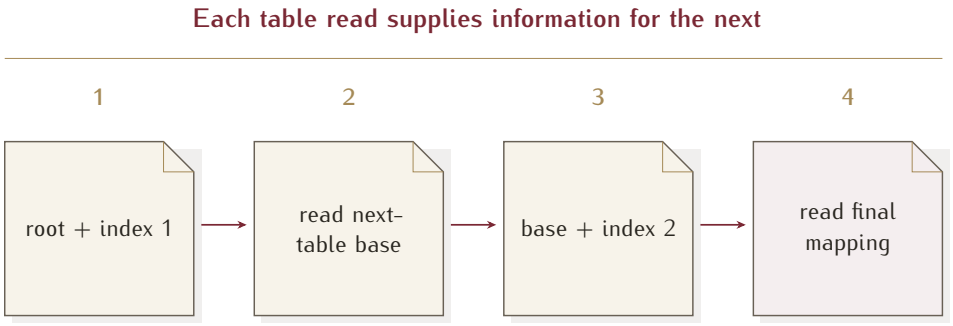


Figure 6.40: *A multilevel walk follows a pointer dependency through the hierarchy. The offset does not participate in these table indices; it is preserved until the final frame number is known.*

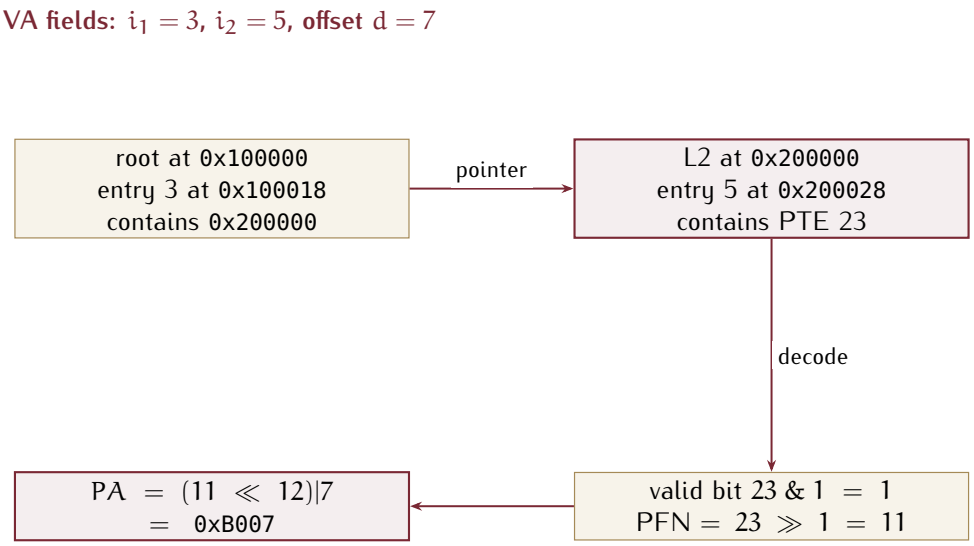
Inspecting the two dependent loads

Let $i_1 = 3$, $i_2 = 5$, offset 7 and root base `0x100000`. The root entry is at `0x100018`. If it contains `0x200000`, the leaf entry is at `0x200028`. A leaf value 23 decodes to PFN 11, giving PA = `0xB007`. The second load must use `0x200000` as its base. Reusing the root base would consult a different object.

Changing only the offset

Keep the tables of Figure 6.41, but change the offset from 7 to 9 without crossing the page boundary. Which loads change?

Solution. Both indices remain $i_1 = 3, i_2 = 5$. The program still reads the root entry at `0x100018` and the leaf at `0x200028`; the decoded frame remains 11. Only the final composition changes, giving `0xB009`. This repeated mapping is what a TLB can cache. In this exercise the root and leaf remain allocated and valid throughout.



The first load supplies the base used by the second load.
Both tables use eight-byte entries in this simulation.

Figure 6.41: *Two dependent host-memory reads implement the simulated table walk. Root pointers are directly usable host addresses; the final value is a simulated physical address. The offset bypasses both indexing operations.*

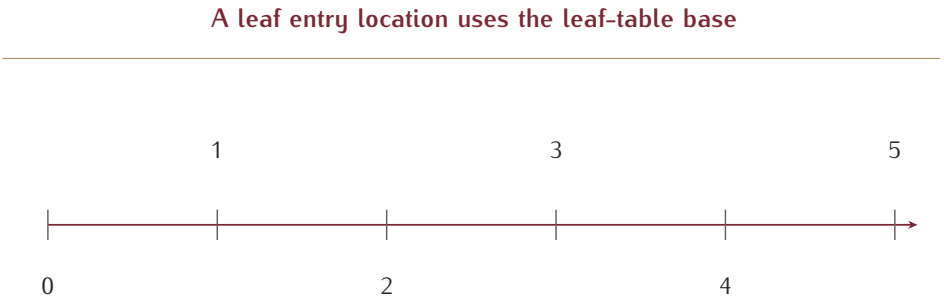


Figure 6.42: *The scale belongs to the entry size, while the base comes from the previous lookup. Reusing the root base for this second index would access a different object in the simulation.*

6.4.4 Translation cost with stated timing assumptions

For a serial TLB lookup of duration t_T, RAM access t_M, TLB hit fraction h, a one-level page table and no page faults or cache hits, a TLB hit costs $t_T + t_M$; a miss costs $t_T + 2t_M$. Thus

$$\mathrm{EAT} = h(t_T + t_M) + (1 - h)(t_T + 2t_M) = t_T + (2 - h)t_M.$$

For k levels all read from RAM on a miss, replace the miss path by $t_T + (k + 1)t_M$, giving $\mathrm{EAT} = t_T + [1 + k(1 - h)]t_M$. Page-walk caches, cached PTEs, parallel work and larger-page mappings can change this cost. This formula predicts the mean for the serial access paths specified above.

Changing page size affects both table size and the number of distinct pages touched. Larger pages reduce the number of entries and can increase TLB reach, defined approximately as entry count times page size for uniform entries. They can also increase unused space within allocations. A complete comparison states the workload and which costs are being measured.

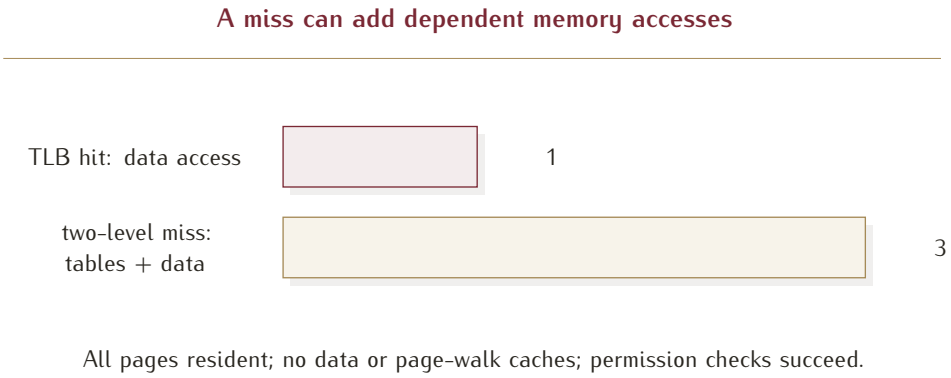


Figure 6.43: *The simplified count includes two table reads followed by the data read on a two-level miss. A TLB hit avoids those table reads. Real timing can overlap work or use cached walk information, so the assumptions are essential.*

6.4.5 Exercises

Exercise 6.4.1 — Tracing cached translations

TLB contains VPNs $\{2, 5\}$. Requests are $5, 7, 5, 2, 7, 8$. Misses insert translations and no eviction occurs. Return H/M and count hits.

Worked solution

Exercise 6.4.2 — An initially empty TLB

Trace VPNs $9, 9, 10, 9, 10, 11$ in an initially empty TLB with insertion and no eviction. How many distinct translations are installed?

Worked solution

Exercise 6.4.3 — Bytes, lines and pages

A scan reads 8192 bytes beginning at a 4096-byte page boundary. Pages are 4096 bytes and cache lines are 64 bytes. With an empty sufficiently large TLB, how many distinct page translations and cache lines does the scan touch?

Worked solution

Exercise 6.4.4 — Distinguishing a miss from a fault

A TLB lookup misses, but a page-table walk finds a present, readable mapping. State the next steps. Then state what changes if the walk finds the requested page absent.

Worked solution

Exercise 6.4.5 — A bounded TLB simulation

`rdi` is a requested VPN. `rsi` points to four records, each [VPN:8 bytes][PFN:8 bytes]. Exactly one record matches. Return its PFN in `rax`; `rcx` and flags may change. Write a general search that also safely returns all ones if no match exists.

Worked solution

Exercise 6.4.6 — Trace all record fields

Use records $(\mathrm{VPN}, \mathrm{PFN}) = (3,40), (8,12), (6,19), (10,7)$ and requested VPN 6. Trace the search and give the byte offset of the loaded PFN.

Worked solution

Exercise 6.4.7 — Diagnose the record stride

A programmer scans the same four records by adding 8 after every failed VPN comparison. Explain the defect and give a complete corrected fragment.

Worked solution

Exercise 6.4.8 — Failure and last-entry tests

For the bounded search, give tests that exercise the last record and no match. Use records $(1,9), (2,8), (3,7), (4,6)$ and specify the outputs and final comparisons.

Worked solution

Exercise 6.4.9 — Extract the 10/10/12 fields

A zero-extended 32-bit VA has 10 upper-index bits, 10 lower-index bits and 12 offset bits. For VA `0x12345ABC`, return the three fields as decimal tokens in that order.

Worked solution

Exercise 6.4.10 — Extract the 8/12/12 fields

A zero-extended 32-bit VA has 8 upper-index bits, 12 lower-index bits and 12 offset bits. For VA `0x67DEF123`, return the three fields as decimal tokens in that order.

Worked solution

Exercise 6.4.11 — Extract the 12/8/12 fields

A zero-extended 32-bit VA has 12 upper-index bits, 8 lower-index bits and 12 offset bits. For VA `0xABCDE789`, return the three fields as decimal tokens in that order.

Worked solution

Exercise 6.4.12 — Extract the 9/11/12 fields

A zero-extended 32-bit VA has 9 upper-index bits, 11 lower-index bits and 12 offset bits. For VA `0x45678234`, return the three fields as decimal tokens in that order.

Worked solution

Exercise 6.4.13 — Walk a 10/10/12 table

`rdi` is a zero-extended 32-bit VA split 10/10/12. `rsi` points to L1. Every entry is eight bytes. L1[i_1] holds a directly usable L2 pointer; L2[i_2] contains (PFN $\ll$ 1)|1. The relevant entries are valid and readable. Return PA in `rax`; `rcx`, `rdx` and flags may change. Trace VA `0xC0504D`, L1 base `0x100000`, selected L2 base `0x300000`, and leaf PTE 27.

Worked solution

Exercise 6.4.14 — Walk an 8/12/12 table

`rdi` is a zero-extended 32-bit VA split 8/12/12. `rsi` points to L1. Every entry is eight bytes. L1[i_1] holds a directly usable L2 pointer; L2[i_2] contains (PFN $\ll$ 1)|1. The relevant entries are valid and readable. Return PA in `rax`; `rcx`, `rdx` and flags may change. Trace VA `0x300504D`, L1 base `0x100000`, selected L2 base `0x300000`, and leaf PTE 27. How large are the root and one complete L2 table in this layout?

Worked solution

Exercise 6.4.15 — Walk a 12/8/12 table

`rdi` is a zero-extended 32-bit VA split 12/8/12. `rsi` points to L1. Every entry is eight bytes. L1[i_1] holds a directly usable L2 pointer; L2[i_2] contains (PFN $\ll$ 1)|1. The relevant entries are valid and readable. Return PA in `rax`; `rcx`, `rdx` and flags may change. Trace VA `0x30504D`, L1 base `0x100000`, selected L2 base `0x300000`, and leaf PTE 27. Which virtual-address increment changes only the lower index from 5 to 6 in this trace?

Worked solution

Exercise 6.4.16 — Walk a 9/11/12 table

`rdi` is a zero-extended 32-bit VA split 9/11/12. `rsi` points to L1. Every entry is eight bytes. L1[i_1] holds a directly usable L2 pointer; L2[i_2] contains (PFN $\ll$ 1)|1. The relevant entries are valid and readable. Return PA in `rax`; `rcx`, `rdx` and flags may change. Trace VA `0x180504D`, L1 base `0x100000`, selected L2 base `0x300000`, and leaf PTE 27. What changes when a VA crosses from lower index 2047 at offset 4095 to the next byte?

Worked solution

Exercise 6.4.17 — A large linear table

A complete 48-bit virtual space uses 4096-byte pages and eight-byte PTEs. Calculate the linear table size in bytes and GiB.

Worked solution

Exercise 6.4.18 — Changing page size

For a complete 32-bit virtual space and eight-byte PTEs, compare linear table sizes for 4 KiB and 8 KiB pages.

Worked solution

Exercise 6.4.19 — Serial one-level timing

TLB lookup takes 4 ns, RAM takes 90 ns, and TLB hit rate is 96%. The page table has one level. Assume serial accesses, no caches and no page faults. Compute mean translation plus data time.

Worked solution

Exercise 6.4.20 — Adding a second walk level

Repeat the preceding model with a two-level page table and the same 96% TLB hit rate. What time is added per miss and to the mean?

Worked solution

6.4.6 Solutions

Solution 6.4.1

The trace is `H, M, H, H, H, M`. VPN 5 is initially present. The first 7 misses and is inserted, so its later reference hits. VPN 2 also hits; VPN 8 misses at the end. Thus there are four hits and two misses. The final set is $\{2, 5, 7, 8\}$. These are translation-cache outcomes; they say nothing by themselves about whether the requested data hits in a data cache.

Return to the problem

Solution 6.4.2

The first 9 misses, the second hits, the first 10 misses, the following 9 and 10 hit, and the first 11 misses. The sequence is `M, H, M, H, H, M`, with three installations for distinct VPNs 9, 10 and 11. There are three hits and three misses. A page can remain resident throughout all of this; a TLB miss alone does not imply a page fault.

Return to the problem

Solution 6.4.3

The aligned interval spans $8192/4096 = 2$ pages, hence two distinct translations. It spans $8192/64 = 128$ cache lines. The first reference to each page can install one translation, which subsequent accesses within that page reuse. A cache line and a page have different sizes, so the 128 line references cannot be used as the translation count.

Return to the problem

Solution 6.4.4

With a usable resident mapping, the walk supplies the frame and access attributes, the translation can be inserted into the TLB, and the original access proceeds to the data. If the walk finds an absent mapping, hardware raises the appropriate exception; the operating system classifies and possibly repairs it. A TLB miss is a cache event, while the absence discovered during the walk is a mapping-state event. Their recovery paths have different costs.

Return to the problem

Solution 6.4.5

```
xor ecx, ecx
.Lsearch:
cmp rdi, [rsi + rcx]
je .Lfound
add rcx, 16
cmp rcx, 64
jb .Lsearch
mov rax, -1
jmp .Ldone
.Lfound:
mov rax, [rsi + rcx + 8]
.Ldone:
```

The byte offsets tested are 0, 16, 32 and 48. Each comparison reads the VPN at the record start; a match loads its PFN eight bytes later. After a mismatch at offset 48, adding 16 reaches 64 and the unsigned bound test ends the loop, avoiding a fifth record. The problem's unique-match premise selects the successful path. The explicit failure path also gives defined behaviour for the extended no-match test.

Return to the problem

Solution 6.4.6

At record offset 0 the compared VPN is 3, so the search advances. At offset 16 the VPN is 8, so it advances again. At offset 32 the VPN is 6, so the search succeeds. The associated PFN is at offset $32 + 8 = 40$ from the array base and has value 19. The fourth record is never examined. Using the record's first qword as the answer would incorrectly return the matching VPN.

Return to the problem

Solution 6.4.7

```
xor ecx, ecx
.Lsearch:
cmp rdi, [rsi + rcx]
je .Lfound
add rcx, 16
cmp rcx, 64
jb .Lsearch
mov rax, -1
jmp .Ldone
.Lfound:
```

Worked solution (continued)

```
  mov rax, [rsi + rcx + 8]
  .Ldone:
```

A record contains two eight-byte fields, so its total size is 16 bytes. An eight-byte stride alternates between VPN and PFN fields, allowing a frame number to be mistaken for a tag and missing the intended record boundaries. The corrected search advances by 16, compares only record starts, and loads the frame from offset 8 within the matching record. The bound 64 is four times the complete record size.

Return to the problem

Solution 6.4.8

Requesting VPN 4 compares tags 1, 2 and 3 unsuccessfully, then matches tag 4 at offset 48 and loads PFN 6 at offset 56. Requesting VPN 5 fails at all four offsets. After the final failure the counter becomes 64, so the loop ends and returns `0xFFFFFFFFFFFFFFFF`. Neither test reads at offset 64. These cases expose early-exit and off-by-one defects that a match in the first record would hide.

Return to the problem

Solution 6.4.9

Remove the lower 22 bits to get $i_1 = VA \gg 22 = 72$. Remove the offset and mask the next 10 bits: $i_2 = (VA \gg 12)\,\&\,1023 = 837$. The low twelve bits give $d = VA\,\&\,4095 = 2748$. The answer is `72, 837, 2748`. Reconstructing $(72 \ll 22)|(837 \ll 12)|2748$ gives `0x12345ABC`. Each index fits its declared field width.

Return to the problem

Solution 6.4.10

Remove the lower 24 bits to get $i_1 = VA \gg 24 = 103$. Remove the offset and mask the next 12 bits: $i_2 = (VA \gg 12)\,\&\,4095 = 3567$. The low twelve bits give $d = VA\,\&\,4095 = 291$. The answer is `103, 3567, 291`. Reconstructing $(103 \ll 24)|(3567 \ll 12)|291$ gives `0x67DEF123`. Each index fits its declared field width.

Return to the problem

Solution 6.4.11

Remove the lower 20 bits to get $i_1 = VA \gg 20 = 2748$. Remove the offset and mask the next 8 bits: $i_2 = (VA \gg 12)\,\&\,255 = 222$. The low twelve bits give $d = VA\,\&\,4095 = 1929$. The answer is `2748, 222, 1929`. Reconstructing $(2748 \ll 20)|(222 \ll 12)|1929$ gives `0xABCDE789`. Each index fits its declared field width.

Worked solution (continued)

Return to the problem

Solution 6.4.12

Remove the lower 23 bits to get $i_1 = VA \gg 23 = 138$. Remove the offset and mask the next 11 bits: $i_2 = (VA \gg 12)\,\&\,2047 = 1656$. The low twelve bits give $d = VA\,\&\,4095 = 564$. The answer is `138, 1656, 564`. Reconstructing $(138 \ll 23)|(1656 \ll 12)|564$ gives `0x45678234`. Each index fits its declared field width.

Return to the problem

Solution 6.4.13

```
mov rax, rdi
shr rax, 22
mov rdx, [rsi + rax*8]
mov rcx, rdi
shr rcx, 12
and ecx, 1023
mov rax, [rdx + rcx*8]
shr rax, 1
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The upper index is 3, the lower index 5, and the offset 77. The first load uses `0x100000`+$3 \cdot 8$=`0x100018` and obtains `0x300000`. The second uses that new base plus $5 \cdot 8$, giving `0x300028`. PTE 27 decodes to PFN 13, so $PA = (13 \ll 12)|77 = 53{,}325 =$ `0xD04D`. The lower-index mask is 1023 for this layout; using a mask from another split can select the wrong leaf. Validity and pointer availability were preconditions, so this fragment has no fault path.

Return to the problem

Solution 6.4.14

```
mov rax, rdi
shr rax, 24
mov rdx, [rsi + rax*8]
mov rcx, rdi
shr rcx, 12
and ecx, 4095
```

Worked solution (continued)

```
mov rax, [rdx + rcx*8]
shr rax, 1
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The upper index is 3, the lower index 5, and the offset 77. The first load uses `0x100000`+3 · 8=`0x100018` and obtains `0x300000`. The second uses that new base plus 5 · 8, giving `0x300028`. PTE 27 decodes to PFN 13, so PA = (13 ≪ 12)|77 = 53,325 = `0xD04D`. The lower-index mask is 4095 for this layout; using a mask from another split can select the wrong leaf. Validity and pointer availability were preconditions, so this fragment has no fault path.
The eight-bit root index selects $2^8 = 256$ entries, so the root occupies $256 \cdot 8 = 2048$ bytes. Twelve lower-index bits select 4096 entries in each complete L2 table, or $4096 \cdot 8 = 32768$ bytes. Each pointer in the relatively small root can cover a large region.

Return to the problem

Solution 6.4.15

```
mov rax, rdi
shr rax, 20
mov rdx, [rsi + rax*8]
mov rcx, rdi
shr rcx, 12
and ecx, 255
mov rax, [rdx + rcx*8]
shr rax, 1
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The upper index is 3, the lower index 5, and the offset 77. The first load uses `0x100000`+3 · 8=`0x100018` and obtains `0x300000`. The second uses that new base plus 5 · 8, giving `0x300028`. PTE 27 decodes to PFN 13, so PA = (13 ≪ 12)|77 = 53,325 = `0xD04D`. The lower-index mask is 255 for this layout; using a mask from another split can select the wrong leaf. Validity and pointer availability were preconditions, so this fragment has no fault path.
Add one page, $2^{12} = 4096$ bytes. The VA changes from `0x30504D` to `0x30604D`. The root index remains 3 and the byte offset remains `0x04D`. The leaf address moves one

Worked solution (continued)

eight-byte entry from `0x300028` to `0x300030`. The new physical address depends on the contents of that entry.

Return to the problem

Solution 6.4.16

```
mov rax, rdi
shr rax, 23
mov rdx, [rsi + rax*8]
mov rcx, rdi
shr rcx, 12
and ecx, 2047
mov rax, [rdx + rcx*8]
shr rax, 1
shl rax, 12
mov rcx, rdi
and ecx, 4095
or rax, rcx
```

The upper index is 3, the lower index 5, and the offset 77. The first load uses `0x100000`+$3 \cdot 8$=`0x100018` and obtains `0x300000`. The second uses that new base plus $5 \cdot 8$, giving `0x300028`. PTE 27 decodes to PFN 13, so PA $= (13 \ll 12) | 77 =$ 53,325 $=$ `0xD04D`. The lower-index mask is 2047 for this layout; using a mask from another split can select the wrong leaf. Validity and pointer availability were preconditions, so this fragment has no fault path.

The offset wraps to zero, the eleven-bit lower index wraps from 2047 to zero and the upper index increases by one. If that upper index was 3, the last VA in its region is $(4 \ll 23) - 1 =$ `0x1FFFFFF`, followed by `0x2000000`. The walk now consults root entry 4, which can point to a different lower table.

Return to the problem

Solution 6.4.17

The VPN has $48 - 12 = 36$ bits, giving 2^{36} entries. Multiplying by eight bytes gives $2^{36} \cdot 2^3 = 2^{39} =$ 549,755,813,888 bytes. One GiB is 2^{30} bytes, so the size is $2^{39}/2^{30} = 2^9 =$ 512 GiB. This is a complete linear table; a sparse multilevel representation allocates only needed lower tables.

Return to the problem

Solution 6.4.18

For 4 KiB, $b = 12$ and there are $2^{32-12} = 2^{20}$ entries. Their size is $2^{20} \cdot 8 = 2^{23}$ bytes, or 8 MiB. For 8 KiB, $b = 13$, giving 2^{19} entries and 2^{22} bytes, or 4 MiB. Doubling page size halves the number of pages and table entries. It can also change allocation fragmentation and TLB reach, which this storage calculation does not quantify.

Return to the problem

Solution 6.4.19

A TLB hit costs $4 + 90 = 94$ ns. A miss costs $4 + 90 + 90 = 184$ ns: lookup, PTE read, then data read. The weighted mean is $0.96 \cdot 94 + 0.04 \cdot 184 = 90.24 + 7.36 = 97.6$ ns. Equivalently, $4 + (2 - 0.96)90 = 97.6$. The final data access is paid on both paths.

Return to the problem

Solution 6.4.20

A hit remains 94 ns. A miss now reads two table entries before the data, costing $4 + 3 \cdot 90 = 274$ ns. The mean is $0.96 \cdot 94 + 0.04 \cdot 274 = 90.24 + 10.96 = 101.2$ ns. The additional level adds 90 ns per miss and $0.04 \cdot 90 = 3.6$ ns to the mean, matching $101.2 - 97.6$. This assumes both table reads reach RAM and that no walk caching or overlap shortens the path.

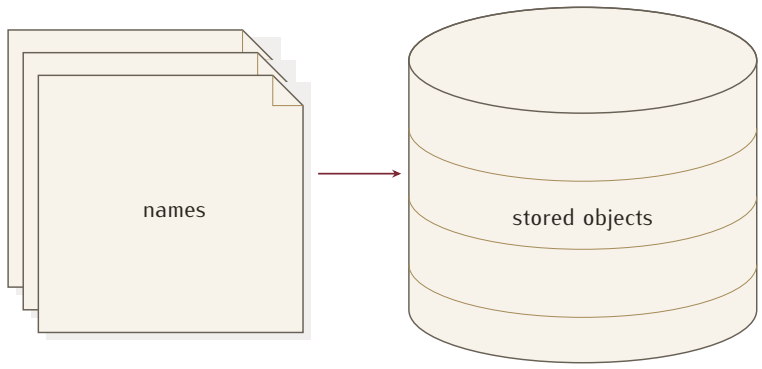
Return to the problem

6.5 FILES, NAMES, AND ALLOCATION

6.5.1 Files, metadata and directory names

A program opens a pathname and receives a descriptor. Later it reads from an offset, while another process may rename or remove a name for the same underlying object. To explain what each operation affects, follow the chain from directory entry to file metadata, open reference and stored contents. Each link has its own lifetime and naming rules. The allocation structures then connect a logical byte offset to the blocks that contain the data.

Names provide routes to separately managed objects



Names lead to objects; the storage representation has its own structure.

Figure 6.44: *The sheets stand for directory names and the store for filesystem objects. Several names can lead to one object, and an open reference can survive name removal. The drawing separates naming from storage and lifetime.*

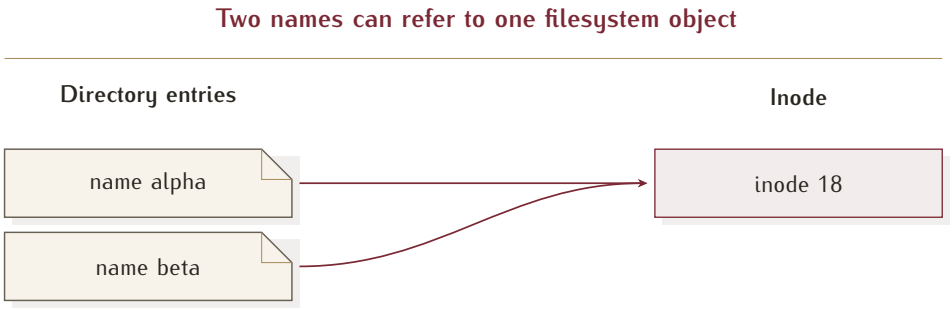


Figure 6.45: *The names are directory entries, while the inode identifies the shared filesystem object and its metadata. Removing one name does not remove the other link or necessarily end the object's lifetime.*

This chapter models a regular file on persistent storage as an object with contents and metadata; virtual files and special device interfaces have different contracts. In the byte-stream model, its contents are an ordered sequence of bytes. Metadata describes properties such as length, owner, permissions, timestamps and storage mapping. A filename is resolved through directories; a directory associates a name with an internal file identifier. In the Unix-like model used here, that identifier is an inode number. The inode describes the object independently of the particular directory name used to reach it.

Sequential access consumes bytes in order using a current position. Direct access allows a program to move to a specified byte offset, for example with a seek operation on a seekable file, and read or write there. The abstract file offset differs from a disk block number. The filesystem maps the offset through its allocation structures to the required blocks, and the storage stack performs the I/O.

6.5.2 Resolving a pathname

An absolute pathname starts from the root directory. A relative pathname starts from the process's current directory or another explicitly supplied directory context. Resolve one component at a time: check search/traversal permission on the current directory, look up the component there, obtain the next inode and continue. Final-object operations may require additional permissions. The last component identifies the requested object. Intermediate components must resolve to directories under the stated model.

A pathname is a sequence of lookups

Root inode 2 maps `project` to inode 7. Inode 7 maps `labs` to 13; inode 13 maps `notes` to 24. Resolving `/project/labs/notes` visits inodes $2, 7, 13, 24$. The final result is inode 24. Starting the relative path `labs/notes` in inode 7 reaches the same object. Each lookup uses the directory reached by the preceding lookup.

A hard link is another directory entry referring to the same inode in the applicable filesystem. Removing one name decrements the link count; other names continue to refer to the object. An open file can remain usable after its last name is removed, until the remaining references are released. File-backed memory mappings can also retain references after their original descriptors close. The lifetime exercises below exclude such additional references unless stated [32]. A symbolic link is a separate object containing a pathname. Following it performs pathname resolution of its stored target. It can reach another filesystem and can become dangling if its target path no longer resolves. A relative symlink target is interpreted relative to the directory containing the link in the ordinary Unix-like model.

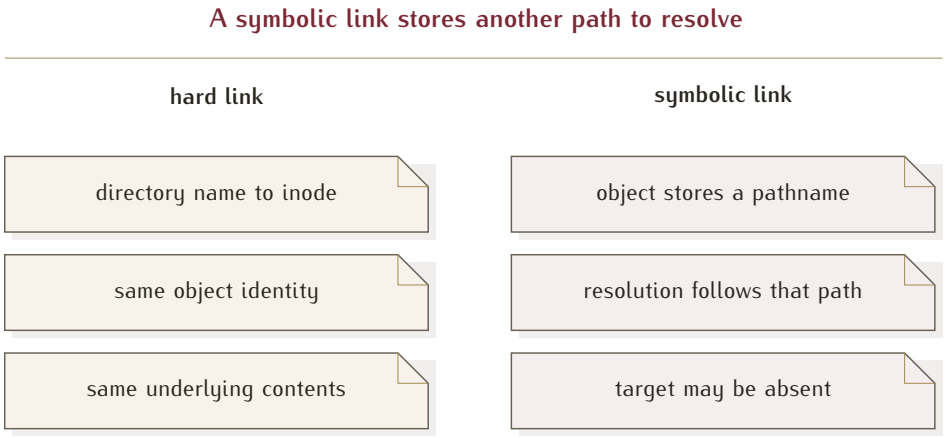


Figure 6.46: *The link types create different relationships. A symbolic link can become dangling because its stored path no longer resolves, while a hard link directly preserves a name for the same inode within the applicable filesystem.*

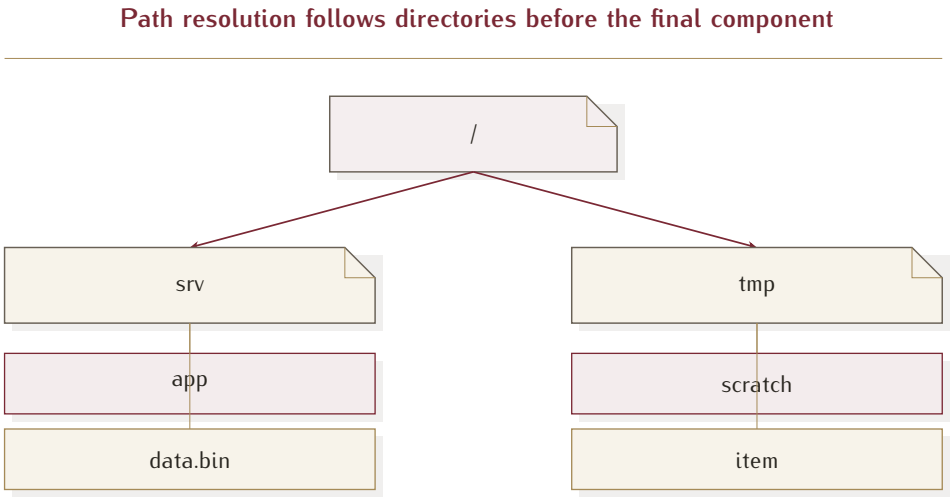


Figure 6.47: *Each intermediate component must resolve to a traversable directory under the relevant access rules. The final operation applies its own rules to the last component and object.*

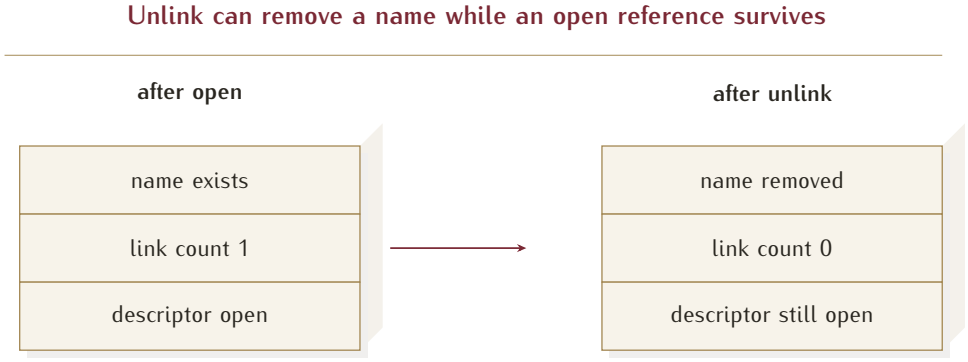


Figure 6.48: *The open reference can keep the object usable after its final directory name is removed. Reclamation becomes possible only after the relevant remaining references are released under the filesystem's lifetime rules.*

Removing a name while keeping the object open

In Figure 6.49, the process unlinks `draft` after opening it as descriptor 3. It then attempts to reopen `draft` and to read through descriptor 3. Assume the directory entry is not recreated and access otherwise succeeds.

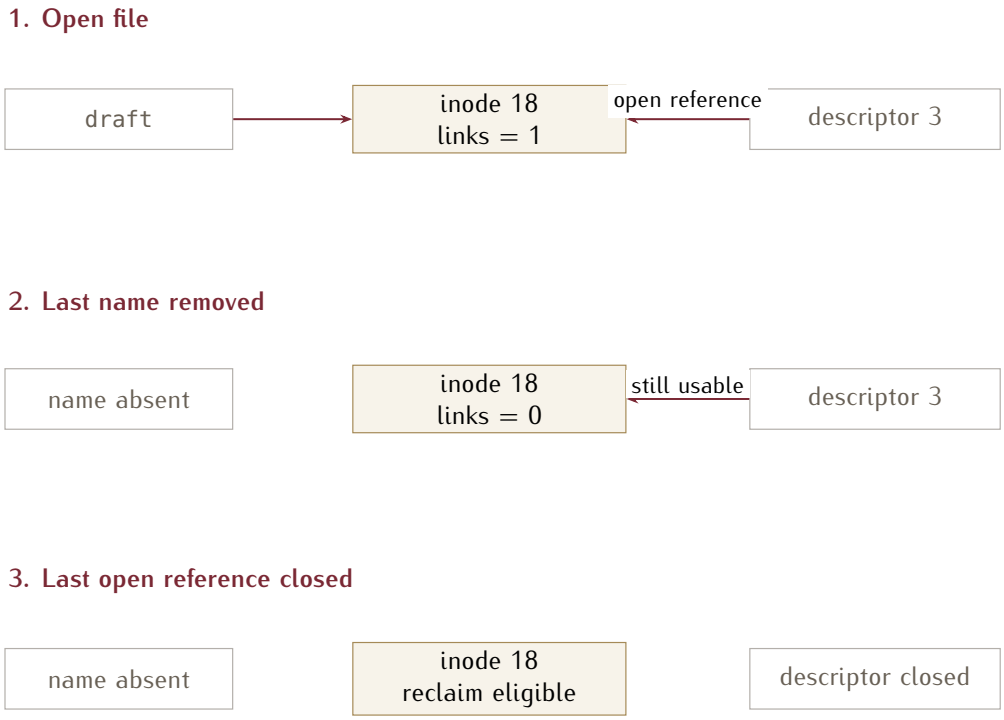


Figure 6.49: *Name reachability and object lifetime change at different events. For clarity, the arrow from the descriptor abbreviates its open file description and filesystem reference. This model excludes duplicated or inherited open references, memory mappings and other lifetime holders. Reclamation becomes possible only after the final name and final open reference are gone.*

Worked example (continued)

Solution. The pathname lookup fails because the name has been removed. Descriptor 3 still holds an open reference to inode 18 and can read its contents at the current offset, subject to the usual end-of-file result. The inode's zero link count records the absence of directory names. Closing the last open reference then permits reclamation under the figure's assumptions; it does not retroactively invalidate earlier reads.

6.5.3 Allocating blocks

A file of $L > 0$ bytes stored in whole blocks of B bytes needs $\lceil L/B \rceil$ data blocks in the fully allocated model. The unused bytes of the last block are internal fragmentation. Index blocks and inode storage add metadata overhead. Sparse files can represent holes without allocating every intervening data block; use that model only when the problem specifies it.

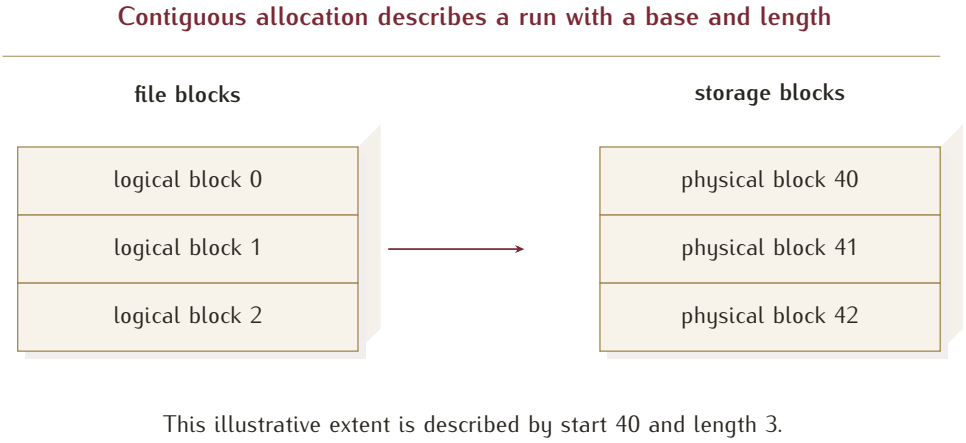


Figure 6.50: *The offset within the extent determines the storage block by addition. Contiguous runs can make large sequential access efficient, while growth and fragmentation require allocation policy.*

Contiguous allocation records a starting position and an extent length, making sequential access straightforward. Growth can require finding or relocating contiguous space. Linked allocation stores a successor for each file block; reaching a late block generally requires traversing earlier links. A file allocation table (FAT) stores those successor relationships in a table indexed by allocation-unit number; real FAT volumes use clusters. The exercises call their allocation units blocks. Indexed allocation stores references in an index structure, with direct and indirect pointers providing access at different file sizes. This is one inode mapping design; an inode can instead describe extents or another filesystem-specific structure.

Scheme	Mapping data	Question to check
Contiguous	Start and extent length	Can the extent grow?
Linked / FAT	Successor per allocated block	How is the chain traversed?
Indexed / inode	Direct or indirect block references	Which index level holds the entry?

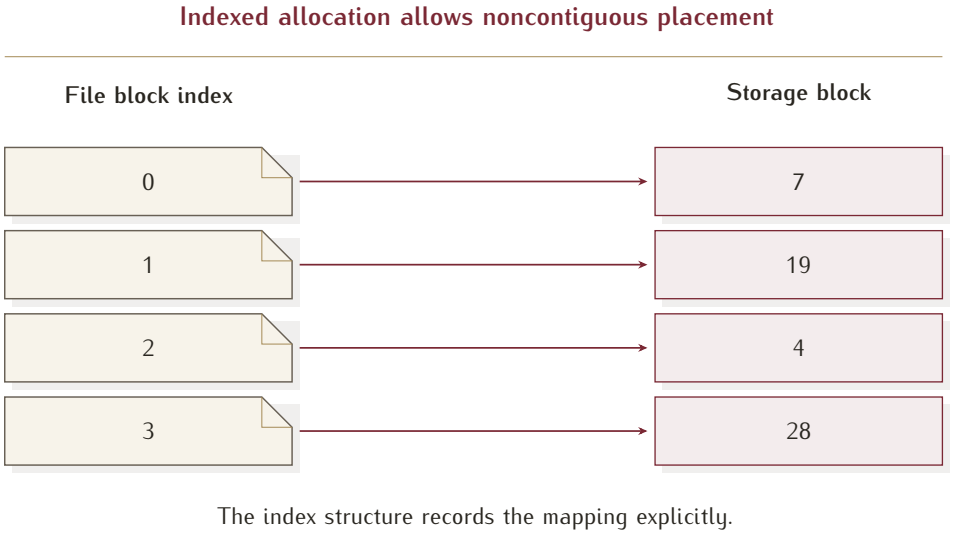


Figure 6.51: *Logical file order is independent of physical block-number order. Reading consecutive file blocks follows the allocation metadata, which can reference nonadjacent storage blocks.*

6.5.4 Free space and allocation accounting

A bitmap uses one bit per managed block. For N blocks it needs $\lceil N/8 \rceil$ bytes. State the bit convention: one may mean free in one problem and allocated in another. Counting set bits is meaningful only after that convention is known. A free-list block can hold several free block numbers; reserve the documented slot for its successor pointer before counting the remaining capacity.

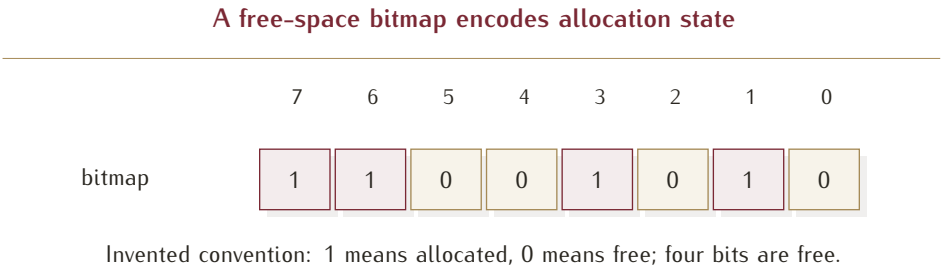


Figure 6.52: *The convention must be stated because either bit polarity is possible. Under this one, four blocks are allocated and four are free. Updating the bitmap must stay consistent with object allocation metadata.*

Data, index and padding are separate costs

An 18,000-byte file uses 4096-byte data blocks and one separate index block. The data-block count is $\lceil 18{,}000/4096 \rceil = 5$, occupying 20,480 bytes. The final data block has $20{,}480 - 18{,}000 = 2480$ unused bytes. Including the index block, storage is six blocks, or 24,576 bytes. Thus useful content occupies 18,000 bytes, allocated data capacity is 20,480 bytes and total allocated storage in this model is 24,576 bytes; the 2480-byte tail is padding within the data allocation.

To follow a FAT chain, begin with the file's first block, append that block to the result, then read its successor until the end marker. A cycle indicates an invalid chain in this simple model. Directory names, inode numbers and data-block numbers belong to different namespaces; a numerical coincidence between them does not establish a relationship.

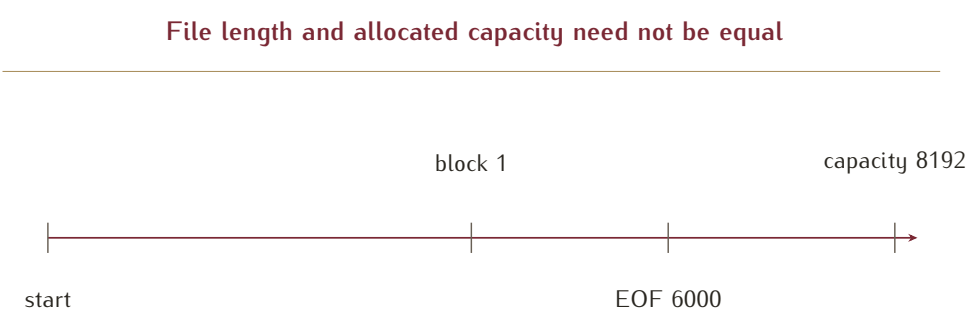


Figure 6.53: *A 6000-byte file occupies only part of the second block in this simple non-sparse example. Its logical end of file is at byte offset 6000; unused capacity after that point is not additional file content.*

6.5.5 Exercises

Exercise 6.5.1 — Contents and metadata

A file contains the bytes of a poem, is owned by user Ada, is 1800 bytes long and grants its group read permission. Classify these four facts as contents or metadata.

Worked solution

Exercise 6.5.2 — The directory association

Choose the primary association stored in the directory model: A: CPU register to cache line; B: filename to inode number; C: VPN to PFN; D: every file byte to a username; E: clock frequency to file size; F: process ID to instruction opcode.

Worked solution

Exercise 6.5.3 — Direct access

A seekable byte-stream file has 100-byte fixed-size records starting at offset zero. What offset begins record 37 when records are numbered from zero? Describe how a program can read it without consuming all preceding records.

Worked solution

Exercise 6.5.4 — One object, two names

Directory entries `draft` and `copy` both name inode 18. A write through `draft` changes the file's first byte. What byte does a subsequent read at file offset zero through `copy` see, assuming the write completed and no intervening changes occurred?

Worked solution

Exercise 6.5.5 — Absolute lookup

Root inode 2 contains `srv`→7 and `tmp`→9. Inode 7 contains `project`→12; inode 12 contains `book`→25. Resolve `/srv/project/book`, listing visited inodes.

Worked solution

Exercise 6.5.6 — Relative lookup

The current directory is inode 12, containing `notes`→31 and `old`→40. Inode 31 contains `week1`→52. Resolve `notes/week1` and explain where lookup starts.

Worked solution

Exercise 6.5.7 — Recognising an absolute path

Which path is absolute in the Unix-like model: A: `home/ada`; B: `../ada`; C: `./notes`; D: `/home/ada`; E: `notes/report`; F: `../../tmp`? Explain the other forms.

Worked solution

Exercise 6.5.8 — A failed intermediate component

Root maps `a` to directory inode 4. Inode 4 maps `b` to a regular file inode 8. Explain why resolving `/a/b/c` fails in the stated model.

Worked solution

Exercise 6.5.9 — Hard-link lifetime

A regular file has four hard links and no open descriptors. Three names are removed. Give the remaining link count and explain whether its data is now eligible for reclamation.

Worked solution

Exercise 6.5.10 — An open but unnamed file

A process has an open descriptor for a file with one hard link. The name is removed. Can the process continue using the descriptor, and what happens when the last open reference closes?

Worked solution

Exercise 6.5.11 — A symbolic link across filesystems

A symbolic link stores `/mnt/archive/report`. The link and target are on different filesystems. Explain why the link can work and what occurs if the target path is later removed.

Worked solution

Exercise 6.5.12 — Relative symlink context

A link at `/srv/project/latest` stores the relative target `draft/report`. Under ordinary Unix-like resolution, which pathname is followed? Explain the base directory.

Worked solution

Exercise 6.5.13 — Bitmap storage

A volume manages 10,000 blocks with one bitmap bit per block. How many bytes are needed, and how many bits of the final byte are unused?

Worked solution

Exercise 6.5.14 — A free-list block

A 4096-byte block stores four-byte block numbers. One slot is reserved for the next free-list block pointer. How many free block numbers fit?

Worked solution

Exercise 6.5.15 — Contiguous growth

A file occupies blocks 20–24 contiguously. Block 25 belongs to another file and the first file must grow by two blocks. Explain the difficulty and two possible allocation-design responses.

Worked solution

Exercise 6.5.16 — An index-block capacity

A single 4096-byte index block contains only eight-byte data-block pointers. Data blocks are 4096 bytes. How many data bytes can this one index block address, excluding its own storage?

Worked solution

Exercise 6.5.17 — Whole data blocks

A fully allocated file contains 13,000 bytes and uses 4096-byte data blocks. Find its data-block count and unused tail capacity.

Worked solution

Exercise 6.5.18 — Following a FAT chain

A FAT contains $4 \to 11$, $11 \to 7$, $7 \to \text{EOF}$, $5 \to 9$, $9 \to \text{EOF}$. A file starts at block 4. Give its ordered block chain and its allocated capacity for 2048-byte blocks.

Worked solution

Exercise 6.5.19 — Counting free bits

A bitmap uses 1 for free. Bits for blocks 0–9 are `1 0 0 1 1 0 1 0 0 1`. List free blocks, count them, then show the bitmap after allocating block 4.

Worked solution

Exercise 6.5.20 — From a name to a data byte

Directory `/lab` maps `results` to inode 30. That inode's direct block list is $[8, 14, 21]$. Blocks contain 1024 bytes. For file offset 2500, identify the logical block index, physical block and byte displacement, assuming the file is long enough.

Worked solution

6.5.6 Solutions

Solution 6.5.1

The poem's encoded bytes are contents. Owner, byte length and permission information are metadata describing the object and its use. The length describes how many content bytes belong to the file, while the permission controls access under the policy. A filename belongs to the directory association used to locate the object. Keeping these roles separate is useful when the same file has several names.

Return to the problem

Solution 6.5.2

Answer B. A directory maps names to internal object identifiers, represented here by inode numbers. A belongs to a different hardware relationship. C describes virtual-memory translation. D confuses access policy with byte storage. E and F do not describe directory lookup. Once the inode is found, its metadata and storage mapping allow the filesystem to operate on the file.

Return to the problem

Solution 6.5.3

Record i begins at byte offset $100i$, so record 37 begins at 3700. A program can seek to offset 3700 and request 100 bytes, checking the actual read result and the file's length. This is direct access through the file abstraction. The filesystem still translates the byte offset into its allocation structures; the offset 3700 is not a physical disk address.

Return to the problem

Solution 6.5.4

Both names resolve to the same inode and therefore the same underlying file contents. The read through `copy` sees the changed byte. Creating another hard link does not duplicate the file's data. An independent copied file would have its own object identity and contents; the given directory entries explicitly share identity.

Return to the problem

Solution 6.5.5

The initial slash selects root inode 2. Looking up `srv` there reaches 7; looking up `project` in 7 reaches 12; looking up `book` in 12 reaches 25. The visited sequence is $2, 7, 12, 25$, and the final file is inode 25. The unrelated `tmp` entry is never consulted. Each component is looked up in the directory reached by the previous component.

Worked solution (continued)

Return to the problem

Solution 6.5.6

The path has no leading slash, so lookup starts at the current directory inode 12. Its `notes` entry reaches 31; that directory's `week1` entry reaches 52. The sequence is $12, 31, 52$, with final inode 52. Starting at root would apply a different context and could reach a different object or fail. The current directory is part of the meaning of a relative pathname.

Return to the problem

Solution 6.5.7

Answer D. The leading slash anchors resolution at the root directory. A and E begin with ordinary components relative to the current directory. C explicitly starts from the current directory. B and F first traverse one or two parent components relative to that context. Parent components do not make a path absolute; the initial root marker does.

Return to the problem

Solution 6.5.8

Resolution reaches inode 4 for `a`, then inode 8 for `b`. Another component, `c`, remains, so `b` must provide a directory in which that name can be looked up. It is a regular file, so traversal cannot continue. The failure occurs at the intermediate component even if the file's contents happen to include the character `c`. File bytes are not directory entries in this model.

Return to the problem

Solution 6.5.9

The link count decreases from four to $4 - 3 = 1$. One directory name still refers to the inode, so the file remains reachable and its contents must be retained. Removing the final name would reduce the count to zero; with no remaining open references under the stated model, the object could then be reclaimed. Counting removed names alone is insufficient without the initial link count.

Return to the problem

Solution 6.5.10

In the Unix-like lifetime model, removing the name reduces the hard-link count to zero but leaves the open object reference alive. The process can continue using its descriptor subject to the ordinary descriptor semantics. After the final open reference closes, the unnamed object can be reclaimed. The directory name and the open reference are separate ways the system keeps track of the object.

Return to the problem

Solution 6.5.11

The symbolic link stores a pathname that is resolved when the link is followed. Resolution can traverse a mount point and reach another filesystem, so the target need not share the link's filesystem. If the stored path stops resolving, the link object still exists but is dangling. Its existence does not preserve the target inode's lifetime as a hard link would within the applicable filesystem.

Return to the problem

Solution 6.5.12

The relative target is interpreted from the directory containing the symlink, `/srv/project`. The resulting path is `/srv/project/draft/report`. It is not interpreted from the invoking process's current directory. This makes the link's stored target meaningful relative to its own location, although moving the link or its containing directories can affect what the path reaches.

Return to the problem

Solution 6.5.13

The storage requirement is $\lceil 10{,}000/8 \rceil = 1250$ bytes. Since 10,000 is exactly divisible by eight, the final byte has no unused bits. The bitmap represents allocation state rather than storing the blocks' contents, so block size does not enter this calculation. A count of 10,001 blocks would require 1251 bytes with seven unused bits.

Return to the problem

Solution 6.5.14

Each four-byte block number consumes one slot. The block contains $4096/4 = 1024$ slots. The next-list pointer takes one whole slot, leaving 1023 free block numbers. Equivalently, reserve four bytes for the pointer and divide the remaining 4092 bytes by four. Both methods account for the pointer at its full width.

Return to the problem

Solution 6.5.15

The existing five-block extent cannot extend directly because its immediate successor is occupied. A design requiring a single contiguous extent may relocate the file into a larger free extent, copying its contents and updating metadata. A design supporting multiple extents may allocate a second extent elsewhere and record it. Both require free space, yet their metadata and I/O costs differ. Total free capacity alone does not guarantee a suitably large contiguous run.

Return to the problem

Solution 6.5.16

The index has $4096/8 = 512$ pointer slots. Each selects one 4096-byte data block, so addressed data capacity is $512 \cdot 4096 = 2{,}097{,}152$ bytes, or 2 MiB. The index block's own 4096 bytes are metadata overhead and were excluded. The result assumes every slot is available for a pointer and every referenced block is fully used for file data.

Return to the problem

Solution 6.5.17

Three blocks hold $3 \cdot 4096 = 12{,}288$ bytes, which is insufficient. Four hold 16,384 bytes, so $\lceil 13{,}000/4096 \rceil = 4$ blocks are required. Unused capacity in the last block is $16{,}384 - 13{,}000 = 3384$ bytes. Metadata overhead is separate, and no sparse-hole optimisation is part of this problem.

Return to the problem

Solution 6.5.18

Start at 4, whose successor is 11. Entry 11 gives successor 7; entry 7 marks the end. The chain is `4, 11, 7`. It contains three blocks and allocates $3 \cdot 2048 = 6144$ data bytes. The chain beginning at 5 belongs to a different starting point. The actual file length can be smaller than allocated capacity because its final block may be partially used.

Return to the problem

Solution 6.5.19

The one bits identify blocks $\{0, 3, 4, 6, 9\}$, so five blocks are free. Allocating block 4 changes its bit from one to zero. The new bitmap is `1 0 0 1 0 0 1 0 0 1`, with four free blocks. The meaning of one was supplied explicitly; reversing that convention would reverse every allocation decision.

Worked solution (continued)

Return to the problem

Solution 6.5.20

After resolving the name to inode 30, divide the offset: $2500 = 2 \cdot 1024 + 452$. The zero-based logical block index is 2, selecting the third pointer, physical block 21. The byte displacement within that block is 452. Thus the access follows directory name, inode, direct pointer 2, block 21, then byte 452. The logical block index 2 must not be mistaken for the physical block number.

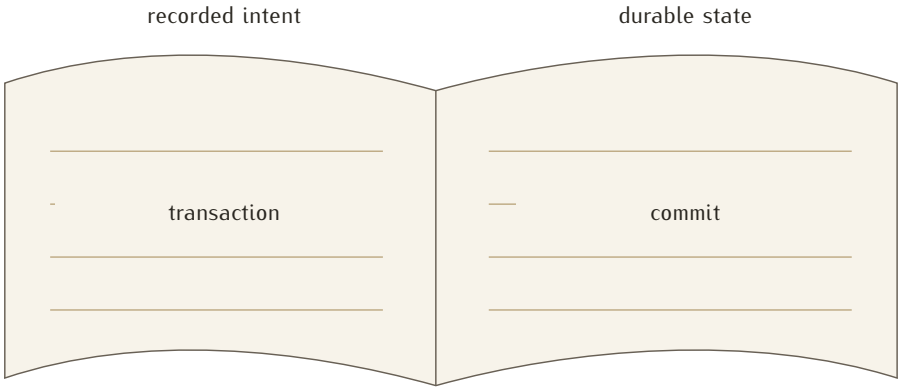
Return to the problem

6.6 CONSISTENCY AND CRASH RECOVERY

6.6.1 Why persistence needs a consistency model

A file grows by one block. The contents, allocation bitmap and inode pointer can reach durable storage at different moments. If power fails between them, reboot exposes a partial update. Following exactly which values survived turns crash recovery into a state-reconstruction problem. Consistency requires the surviving references and allocation records to agree; the recovery protocol must establish that agreement from durable evidence.

A durable record can make an interrupted update recoverable



The ordering of durable records determines what recovery can establish.

Figure 6.54: *The journal preserves enough information to classify and complete transactions after a failure. Its usefulness depends on the order in which records, commit information and home updates become durable. The open-book form is a visual analogy for that recorded evidence.*

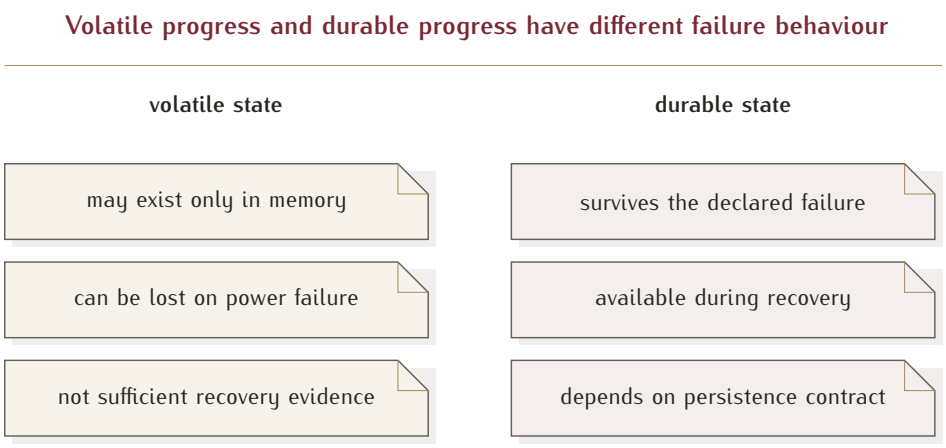


Figure 6.55: *A returned write, an in-memory cache update and durable storage are distinct events in many systems. Recovery reasoning begins by stating which events survive the failure being considered.*

This chapter uses a simple filesystem with no shared data extents, snapshots or deduplication. For each ordinary data block b, let $U(b)$ count references from allocated files and let $F(b)$ indicate free-space membership. A consistent block has either $(U, F) = (1, 0)$ or $(0, 1)$. Reserved metadata blocks are accounted for separately. A missing block has $(0, 0)$; a block referenced more than once has $U > 1$; a block both used and free has $U \geqslant 1, F = 1$. The assumptions matter: intentionally shared extents in another filesystem require reference-counting rules.

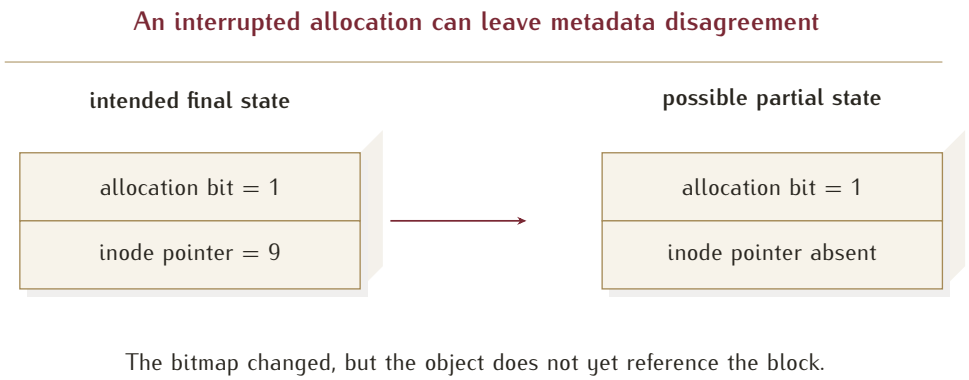


Figure 6.56: *The partial state can lose track of usable capacity even though neither individual write is malformed. Consistency reasoning concerns relationships between persistent structures and the order in which their updates become durable.*

6.6.2 Reconstruction and repair

A consistency checker reconstructs use from file metadata, independently reads free-space information, and compares the two views. It can also traverse directories and compare reconstructed hard-link counts with counts stored in inodes. This work may require scanning many structures, so its cost grows with filesystem size and layout. Repairing a structural inconsistency does not establish that every byte still expresses the user's intended content. Ambiguous ownership or lost data may require reporting damage and consulting backups.

Comparing independent allocation views

Files reference blocks $\{2, 4, 7, 9\}$, while the bitmap marks $\{2, 3, 7, 9\}$ allocated. Block 3 is marked occupied without a file reference; block 4 is referenced while marked free. The symmetric difference is $\{3, 4\}$, containing two discrepancies. The second is dangerous because the allocator might give block 4 to another file. Reconstructing the bitmap from trustworthy metadata can restore allocation consistency under this exercise's assumptions.

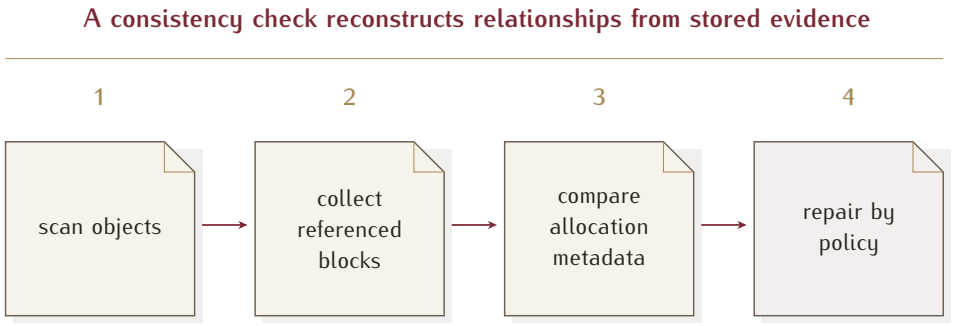


Figure 6.57: *The checker derives ownership information from persistent metadata and compares it with allocation records. A repair policy resolves discrepancies, but missing or ambiguous information can limit what the checker can reconstruct.*

6.6.3 A redo journal and the durable commit boundary

A journal records an update before applying it to its permanent, or *home*, locations. This chapter uses a redo model: a transaction contains BEGIN, a complete description of its changes, and COMMIT. Its log records and commit must become durable in the required order before home updates are allowed. Assume recovery can distinguish complete valid records from torn or incomplete writes; a concrete design needs suitable

atomicity, sequence and integrity checks to establish that premise. After a crash, a complete committed transaction can be replayed; an uncommitted transaction is discarded under this model because its changes were not installed at home.

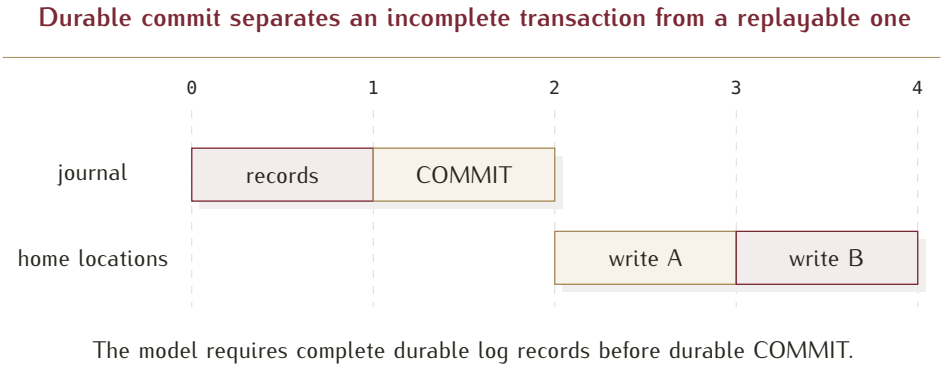


Figure 6.58: *Home-location updates begin only after the committed log supplies enough durable information for recovery. The timeline expresses the ordering constraint, not equal physical durations.*

Durable boundary	Recovery interpretation
No complete commit	Transaction is incomplete; discard its redo records
Complete commit, homes incomplete	Replay committed changes
Complete commit, homes durable	Reclaim/checkpoint journal space safely

The write order is BEGIN, change records, durable COMMIT, home writes, durable completion/checkpoint, then journal-space reuse. "Written" must mean durable at the relevant boundary; issuing a request to a volatile cache is insufficient. Device flushes, barriers and filesystem protocols implement these requirements in concrete systems. The exercises reason about the declared durable states.

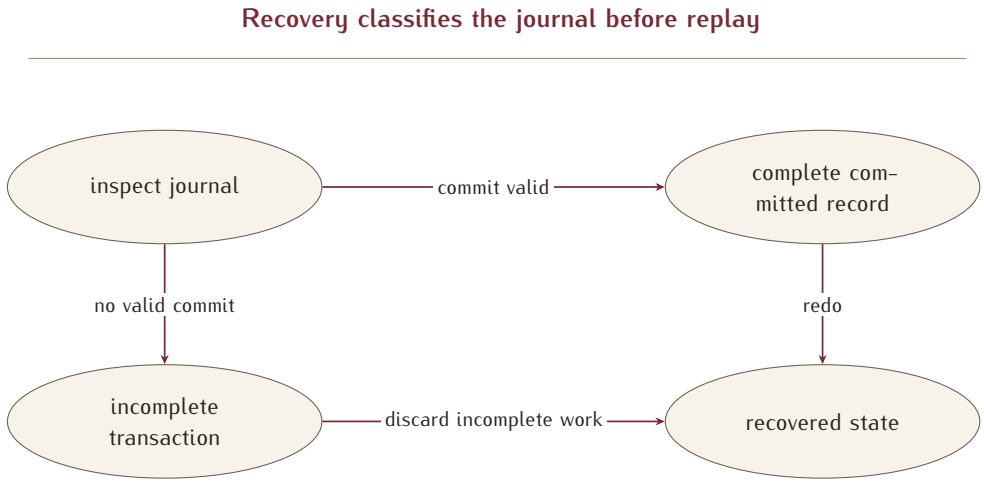


Figure 6.59: *The recovery procedure trusts only a transaction satisfying the journal protocol's commit conditions. Redo installs the recorded final values. Incomplete work is handled according to the specified logging model.*

Replay must be safe to repeat, including if recovery itself crashes. A redo record such as "set bitmap bit 9 to one" is idempotent: repeating it has the same final effect. "Increment this counter" is generally not idempotent. Logging the intended final value or using a protocol that detects an already-applied operation avoids repeated increments. With several transactions affecting the same block, replay and home installation must also respect their transaction order: replaying an older final value after a newer one would undo later work. The examples assume an ordered redo protocol and no concurrent writers during recovery.

Two committed updates to the same block

The home block initially contains 4. Transaction A logs the final value 7 and commits. Transaction B then logs 9 for the same block and commits. A crash leaves 7 at home and both complete transactions in the journal. Replaying A then B installs 7 and then 9, preserving the committed order. Reversing the order would leave 7 and lose B's later change. Idempotence makes each assignment safe to repeat. Transaction order determines which assignment must be last.

A crash after some home writes

Transaction T sets an allocation bit and installs a new inode pointer. Both new values and COMMIT are durable in the journal, but only the allocation bit reached

Worked example (continued)

its home location. Recovery writes both recorded values to their homes. Rewriting the already-correct bit has no additional effect; installing the pointer finishes the transaction. Journal space can be reused after both home values are durable. Clearing the journal before that point would remove the information needed after another crash.

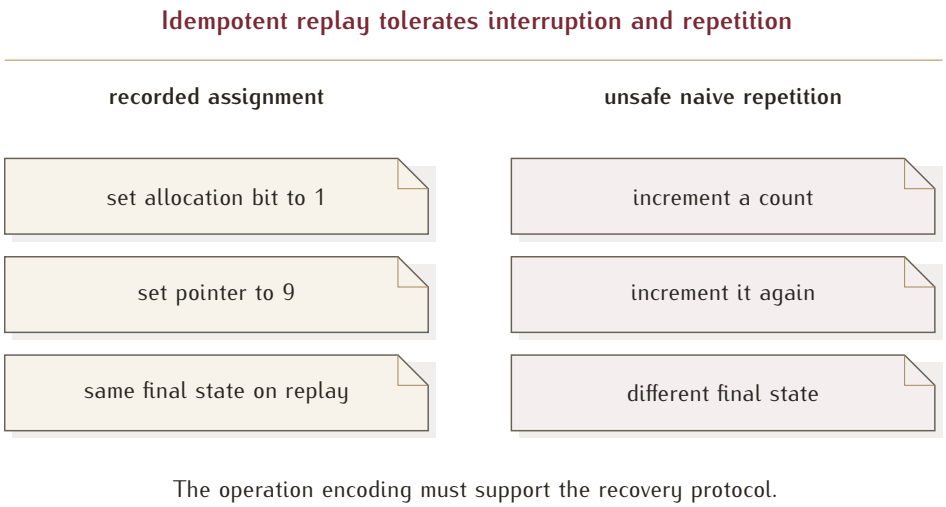


Figure 6.60: *Replaying final-value assignments can produce the same state after repeated interruptions. A raw increment is not idempotent, so a system using such an operation needs additional protocol state to prevent duplicate application.*

A second crash during replay

Start from the third row of Figure 6.61. Recovery writes the allocation bit to one, then crashes again before installing the inode pointer. What does the next recovery do?

Solution. The committed journal still describes both final values. The next recovery writes bit 9 to one and the pointer to 9. The first assignment leaves the already-set bit unchanged. Once both home writes and the required completion record are durable, the transaction's journal space can be reused. Reclamation before that boundary would discard the evidence still needed for the missing pointer. The result follows from logged assignments being safe to repeat and from preserving the committed log during recovery.

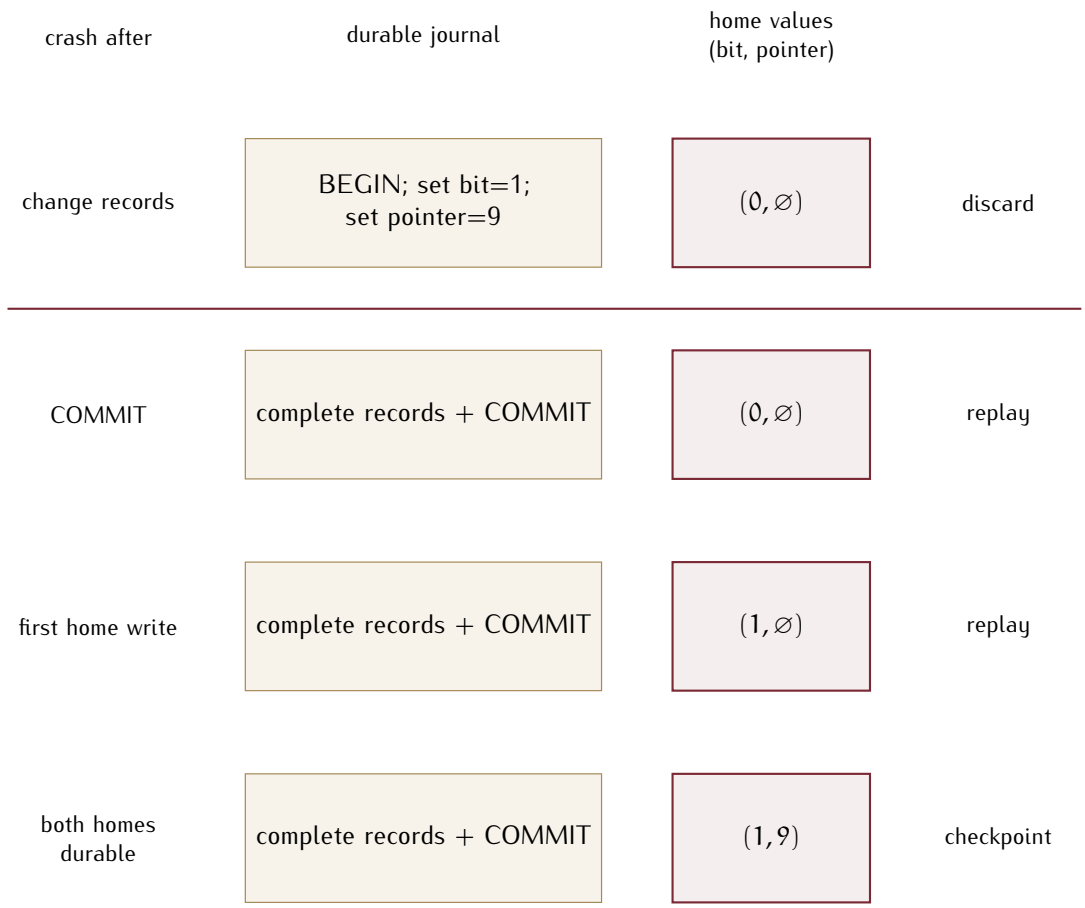


Figure 6.61: *Durable snapshots in a redo journal. The horizontal rule marks the transition to a committed transaction. Home writes begin only below that boundary. Replaying sets both recorded final values, so repeating recovery also produces* $(1, 9)$*. Journal reuse follows durable home completion and the required checkpoint protocol. Payload-data ordering is a separate policy.*

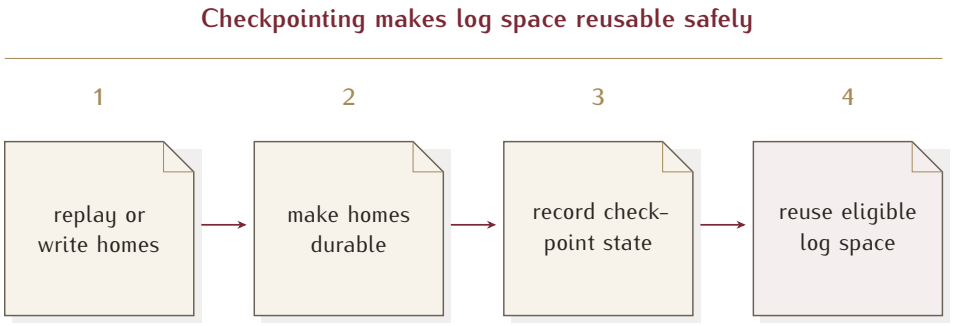


Figure 6.62: *Log records cannot be discarded merely because a write was issued to its home location. The protocol must establish the required durable state before reusing records that recovery might still need.*

6.6.4 Data journaling, metadata journaling and guarantees

Full data journaling writes changed data and metadata into the journal and later to their homes. Ignoring control records and batching, k modified blocks can therefore cause $2k$ block writes. Metadata journaling reduces this overhead by journaling structural updates while writing file data to its final location. Its guarantees about the relationship between data and committed metadata depend on ordering rules and the filesystem mode. Metadata consistency alone does not establish the correctness of recovered payload bytes. Data-ordering, initialisation and confidentiality guarantees require explicit rules; applications also need a durability protocol for their own multi-file updates [35].

Crash consistency, application-level atomicity and backup are separate concerns. A journal can restore coherent filesystem structures after interrupted writes; backups retain earlier recoverable versions, including after an authorised accidental deletion. The exercises below use explicit models so that recovery decisions can be derived from recorded evidence.

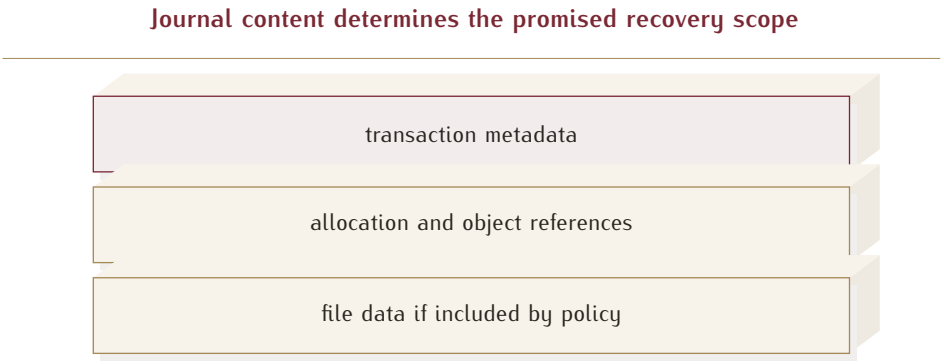


Figure 6.63: *A metadata-only journal can restore structural relationships without necessarily preserving every latest user-data byte. Data ordering rules and synchronisation interfaces determine stronger guarantees.*

6.6.5 Exercises

Exercise 6.6.1 — One update, several blocks

Extending a file requires writing new contents, allocating a block in a bitmap and adding an inode pointer. Explain how a crash can leave a structurally inconsistent state even if each completed block write is individually intact.

Worked solution

Exercise 6.6.2 — An allocated orphan

A bitmap marks block 17 allocated. A crash occurred before any inode was updated to reference it. Assume it is an ordinary data block, with no other owner. Classify the inconsistency.

Worked solution

Exercise 6.6.3 — Duplicate block ownership

Files A, B and C reference $\{2, 6, 10\}$, $\{4, 6, 12\}$ and $\{8, 10, 14\}$ respectively. Shared extents are forbidden. Which blocks have multiple owners, and how many distinct duplicate blocks exist?

Worked solution

Exercise 6.6.4 — Classify two snapshots

Snapshot S1 has a block absent from both the free list and all inode references. S2 has one block referenced by two files. Return tokens MISSING or DOUBLE_USED for S1 and S2, in order.

Worked solution

Exercise 6.6.5 — The checker's evidence

Choose the main consistency-checking action: A: increase CPU frequency; B: compare reconstructed file/block use with allocation metadata; C: accept every bitmap bit as authoritative; D: encrypt every filename; E: delete all open files; F: infer file ownership from block numbers alone.

Worked solution

Exercise 6.6.6 — Reconstructing distinct use

Inodes reference $\{1, 3, 8\}$ and $\{5, 9\}$. Reserved metadata blocks are handled separately. Give the reconstructed used set and count.

Worked solution

Exercise 6.6.7 — Comparing two allocation views

Inodes imply used set $U = \{1, 4, 6, 9\}$. The bitmap marks $B = \{1, 3, 6, 10\}$ used. List blocks with disagreement and classify each side of the difference.

Worked solution

Exercise 6.6.8 — Why a full scan costs time

A checker reads all allocated inodes and traverses their block mappings before comparing free-space records. Explain why its work can be substantial even when only one interrupted update caused the inconsistency.

Worked solution

Exercise 6.6.9 — Order the redo protocol

Order LOG_BEGIN, LOG_CHANGES, LOG_COMMIT, HOME_WRITE and REMOVE_LOG. Treat COMMIT and completed home writes as durable at their respective boundaries.

Worked solution

Exercise 6.6.10 — Why logging comes first

A proposed redo implementation writes an inode to its home location before recording the new allocation bitmap value in the journal. Explain the crash risk under this chapter's model.

Worked solution

Exercise 6.6.11 — Records without commit

After a crash the journal has BEGIN and all change records for T, but no durable COMMIT. No home changes were permitted before commit. Should recovery replay T?

Worked solution

Exercise 6.6.12 — The earliest home write

Which event permits the first home write in the stated protocol: A: calling write on BEGIN; B: computing new data in RAM; C: a complete durable log including COMMIT; D: seeing a free bitmap bit; E: opening the target file; F: reserving a buffer?

Worked solution

Exercise 6.6.13 — Partial home installation

T has a complete durable COMMIT. Its three home updates are H1, H2 and H3, but only H1 reached storage before the crash. Give a safe recovery action assuming redo records contain idempotent final values.

Worked solution

Exercise 6.6.14 — Two transaction decisions

T1 has BEGIN, records and COMMIT; T2 has BEGIN and records only. T1's homes are incomplete. Return REPLAY or DISCARD for T1,T2.

Worked solution

Exercise 6.6.15 — A repeatable redo operation

Compare "set allocation bit 12 to one" with "increment free-block counter". Which is idempotent as written, and how can the counter update be represented for safe repeated redo?

Worked solution

Exercise 6.6.16 — Reclaiming journal space

All home writes have been issued, but some may still be in volatile device cache. Can the journal records immediately be reused? Explain the required boundary.

Worked solution

Exercise 6.6.17 — Full-journal write count

A transaction changes four metadata blocks and three data blocks. Count writes of these seven blocks under full data journaling, ignoring control records, coalescing and retries.

Worked solution

Exercise 6.6.18 — Metadata-only accounting

For the same transaction, journal only the four metadata blocks and write the three data blocks directly home. Ignore control records. Count writes and name a guarantee that still requires an explicit ordering policy.

Worked solution

Exercise 6.6.19 — Classifying journal contents

Classify an inode block, a file's payload block, a free-space bitmap block and a directory block as METADATA or DATA in the filesystem model.

Worked solution

Exercise 6.6.20 — Three recovery states

T1 is committed and all homes are known durable. T2 is committed with homes incomplete. T3 lacks commit and has no home updates. Return CLEAN, REPLAY or DISCARD for T1,T2,T3 and explain all three.

Worked solution

6.6.6 Solutions

Solution 6.6.1

The logical update spans distinct persistent blocks. If only the inode pointer reaches storage, it can refer to a block still marked free. If only the allocation bit reaches storage, the block can appear occupied without a file reference. Each completed write can be internally correct while the relationship between structures is incomplete. A consistency protocol must address the group of updates and their durable ordering.

Return to the problem

Solution 6.6.2

The reconstructed use count is $U(17) = 0$, while free membership is $F(17) = 0$ because the bitmap marks it allocated. The block is missing from both the referenced and free sets: an allocated orphan that wastes space. Under the simple model, a checker can return it to free space once the absence of a legitimate reference is established. Its allocation bit alone does not identify a file owner.

Return to the problem

Solution 6.6.3

Block 6 appears in A and B, and block 10 appears in A and C. Every other listed block appears once. Thus the duplicate-block set is $\{6, 10\}$ and its cardinality is two. Counting both references to each duplicate would count references rather than distinct bad blocks. The conclusion relies on the stated prohibition on intentional shared extents.

Return to the problem

Solution 6.6.4

S1 is `MISSING`: neither view accounts for the block, so capacity is lost. S2 is `DOUBLE_USED`: two file mappings claim the same ordinary data block. The sequence is `MISSING, DOUBLE_USED`. The second state can cause cross-file corruption if one file changes or frees the shared block. Each snapshot is classified independently.

Return to the problem

Solution 6.6.5

Answer B. Independent views of the same relationships reveal discrepancies such as used-and-free blocks or inconsistent link counts. A is a performance change. C

Worked solution (continued)

removes the independent comparison needed to detect bitmap errors. D concerns a different protection mechanism. E is not a consistency rule. F lacks the directory and inode evidence that establishes ownership. Repair then follows the filesystem's model and the reliability of the available evidence.

Return to the problem

Solution 6.6.6

Take the union of the two lists: $\{1, 3, 5, 8, 9\}$. It contains five distinct blocks. No number appears twice, so the ordinary data blocks listed here have no duplicate references. The resulting set can be compared with the allocation bitmap. Reserved metadata must remain excluded from this comparison or be added through its own accounting rule.

Return to the problem

Solution 6.6.7

The symmetric difference is $U \triangle B = \{3, 4, 9, 10\}$, so four block numbers disagree. Blocks 4 and 9 are referenced but marked free, risking reallocation of live data. Blocks 3 and 10 are marked used without references, wasting space in this model. The common blocks 1 and 6 agree and need no correction from this particular comparison.

Return to the problem

Solution 6.6.8

The checker must first discover which relationships are inconsistent, and the ordinary metadata may not identify the interrupted update. Reconstructing ownership can require reading many inodes and indirect blocks across the volume. The amount of evidence scanned can therefore depend on filesystem size and layout, rather than the size of the original interrupted operation. A journal supplies a bounded record of recent transactions that can focus normal crash recovery.

Return to the problem

Solution 6.6.9

The sequence is `LOG_BEGIN`, `LOG_CHANGES`, `LOG_COMMIT`, `HOME_WRITE`, `REMOVE_LOG`. Begin and all change records establish a complete transaction description. A durable commit makes that description eligible for replay. Home writes then install the recorded values. Only after those values are durable can

Worked solution (continued)

journal space be reclaimed. Removing the log earlier can destroy the only recovery description for incomplete home updates.

Return to the problem

Solution 6.6.10

A crash after the inode write can leave a persistent pointer to a block whose allocation update has no durable redo record. Recovery lacks the complete transaction description needed to make the structures agree. The redo protocol requires the full log and durable commit before any home update is installed. That ordering ensures that a partially installed committed transaction can be completed from the surviving records.

Return to the problem

Solution 6.6.11

Recovery discards T as incomplete under this redo model. The missing commit means the transaction never crossed the durable acceptance boundary. Since the protocol forbade its home updates before commit, ignoring its records preserves the previous home state. A system allowing uncommitted home writes would require a different recovery protocol, including the information needed to undo or otherwise handle them.

Return to the problem

Solution 6.6.12

Answer C. Recovery needs a complete durable committed transaction before any partial home installation can be tolerated. A only issues one record and does not establish durability or completeness. B and F describe volatile preparation. D is allocation information, and E establishes a file handle. None of those events alone provides the persistent redo evidence required by the protocol.

Return to the problem

Solution 6.6.13

Replay H1, H2 and H3 from the journal. Rewriting H1 installs its already-correct final value again; H2 and H3 complete the missing changes. Idempotence makes the repeated H1 safe and also permits recovery to be repeated after another crash. After all home values are durable, T can be checkpointed and its log space reclaimed.

Return to the problem

Solution 6.6.14

The ordered result is `REPLAY, DISCARD`. T1 crossed the durable commit boundary, so recovery completes its home updates. T2 did not, so its uncommitted redo records are ignored under the protocol. The fact that both have change records is insufficient to make their recovery actions the same; commit status is decisive.

Return to the problem

Solution 6.6.15

Setting the bit to one is idempotent: repeating it leaves the same value one. Incrementing a counter repeatedly changes its value again on every replay. For a simple final-value redo record, store "set the counter to N", where N is the transaction's intended final value and the transaction protocol defines its ordering. Repeating that assignment has the same final result. More elaborate concurrent protocols can use additional operation identifiers and recovery rules.

Return to the problem

Solution 6.6.16

The protocol must establish that the committed home updates are durable before reusing the journal records. Issuing writes does not by itself guarantee survival after power loss. If the journal were overwritten while home data remained volatile, a crash could lose both the final updates and their recovery description. Concrete implementations use their documented flush, ordering and checkpoint mechanisms to establish the boundary.

Return to the problem

Solution 6.6.17

There are $4 + 3 = 7$ modified blocks. Each is written once to the journal and once to its home location, so the count is $2 \cdot 7 = 14$ block writes. Commit and other control records were explicitly excluded. This is a model of duplicated block traffic, rather than a complete device-level write-amplification measurement.

Return to the problem

Solution 6.6.18

The four metadata blocks are each written twice, contributing eight writes. The three data blocks are written once, contributing three. Total is $8 + 3 = 11$, compared with 14 for full data journaling. The ordering between data writes and committed metadata still needs a defined policy, including how references to newly allocated

Worked solution (continued)

data avoid exposing inappropriate stale contents. A lower write count alone does not establish the durability of application contents.

Return to the problem

Solution 6.6.19

The ordered sequence is `METADATA, DATA, METADATA, METADATA`. An inode describes the file and its mapping. A payload block holds the file's contents. The bitmap describes allocation state. A directory maps names to object identifiers and is filesystem metadata, even though its implementation stores bytes in blocks. The classification concerns the role of the bytes in the filesystem.

Return to the problem

Solution 6.6.20

Return `CLEAN, REPLAY, DISCARD`. T1's durable homes already contain its committed state, so its journal space can be reclaimed through checkpointing. T2 needs replay to finish installing the committed values. T3 never committed and can be discarded under the no-uncommitted-home-write rule. The word "known" in T1 is significant: if recovery cannot establish that homes are complete, idempotent replay may still be required.

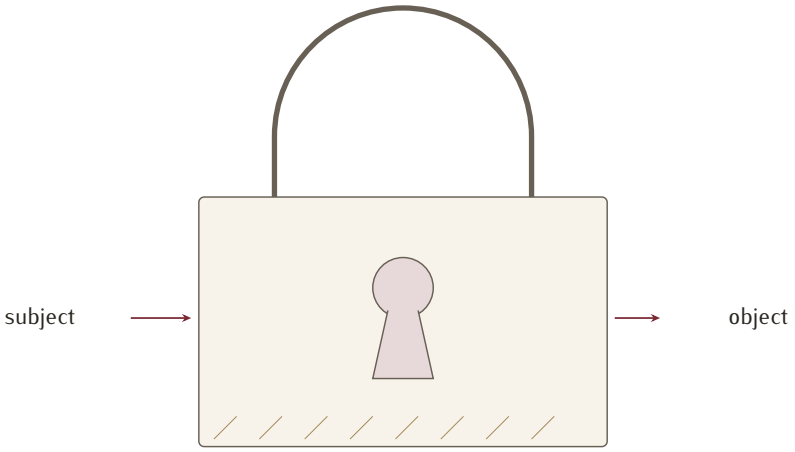
Return to the problem

6.7 PROTECTION, ENCRYPTION, AND AUDITING

6.7.1 What an access decision protects

A request arrives to read a file, modify a record or recover encrypted contents. The system needs evidence about the requester, a rule governing the operation and a way to account for the result. Following the request through these decisions separates access control, cryptographic protection and audit records. Each mechanism answers a specific question, and a complete design must preserve the assumptions on which its answer depends.

Protection is a relationship between a subject and an object



Identity, requested operation and policy determine the access decision.

Figure 6.64: *The lock symbol represents an enforced decision rather than a universal security guarantee. The chapter distinguishes permission, confidentiality, integrity and audit evidence so each claim can be matched to its mechanism and threat model.*

Confidentiality restricts disclosure to authorised parties. Integrity concerns permitted modification and the trustworthiness of stored state under the chosen policy. Availability concerns timely access to data and services by authorised users. A mechanism can contribute to more than one goal; identify the concrete threat before choosing a control.

Confidentiality, integrity and availability ask different questions

	goal	question
confidentiality	disclosure	who can read?
integrity	modification	who can change?
availability	service	can authorised work proceed?

A system can satisfy one goal while failing another.

Figure 6.65: *The three goals distinguish kinds of harm and protection. Encryption can address selected disclosure risks, while access control, validation, redundancy and recovery may address other parts of the system's requirements.*

An abstract access request contains a subject, an object and an operation. Authentication establishes an identity or credentials; authorisation evaluates whether that subject may perform the operation on that object under the applicable policy. An access-control list (ACL) associates permissions with users or groups for an object. The exercises use an explicit allow-set model with default denial: a listed operation is allowed, an absent operation is denied. Group permissions are combined only as stated. Concrete systems can also have ordering, masks, deny entries and inheritance semantics, which must be read from their own specifications.

An access matrix read one request at a time

For object `draft`, let Ada have {READ,WRITE}, Ben have {READ}, and Cy have the empty set. Ben's WRITE request is denied because WRITE is absent from his set. Ada's READ request is allowed. Cy's READ is denied. The answer sequence is `DENY, ALLOW, DENY`. The order of requests determines the order of tokens in the answer.

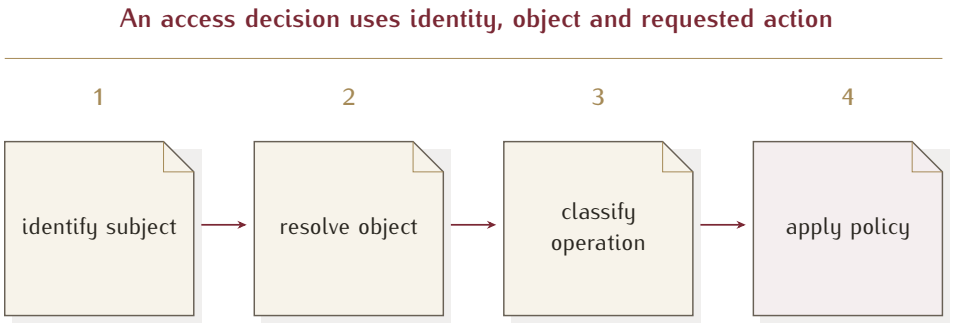


Figure 6.66: *Permission is a relationship between a subject, an object and an operation under a policy. A successful earlier check does not automatically authorise a later operation on a changed object or context.*

The owner or an appropriately privileged administrator can manage policy in the model. Inheritable directory rules can provide defaults for newly created objects. Inheritance must specify when rules are copied or consulted, which operations they cover, and how later changes affect existing objects. The word “inheritance” alone does not answer those questions.

6.7.2 Directory checks and object checks

Resolving a name requires traversal through its directories. A subject may have permission to read a final file yet lack the traversal permission needed to reach it by a particular path. Listing a directory and traversing it are distinct operations in the abstract model. Listing reveals names; traversal allows resolution of a known component. In ordinary Unix terminology these are closely related to read and search/execute permission on a directory. Creating or removing names involves additional checks on the containing directory, and restrictions such as sticky-directory rules can apply.

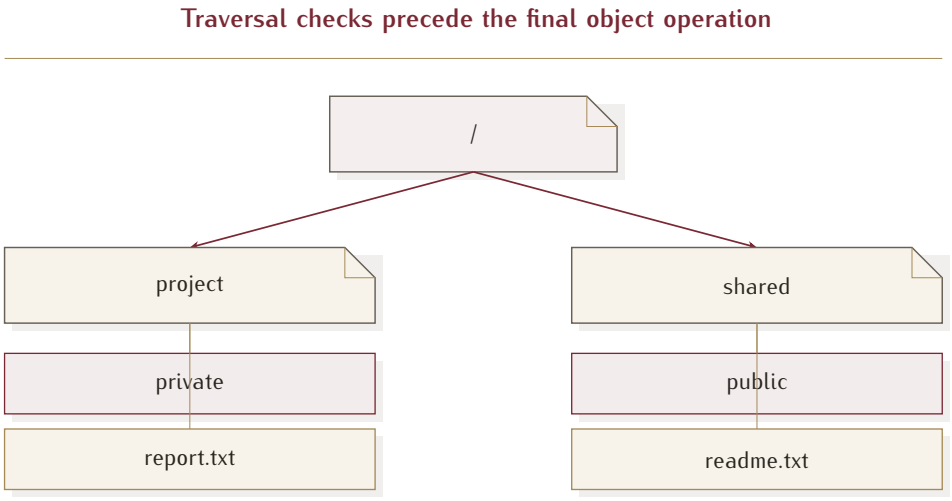


Figure 6.67: *Reaching a file can require permission to traverse every directory in the path in addition to permission for the final requested operation. Directory listing permission and traversal permission answer different questions.*

To decide a request, identify the subject and object, identify the operation, gather the applicable rules for the path and object, then evaluate the rule conditions. A useful solution states each failed check. An allow-or-deny decision follows from that complete sequence of checks.

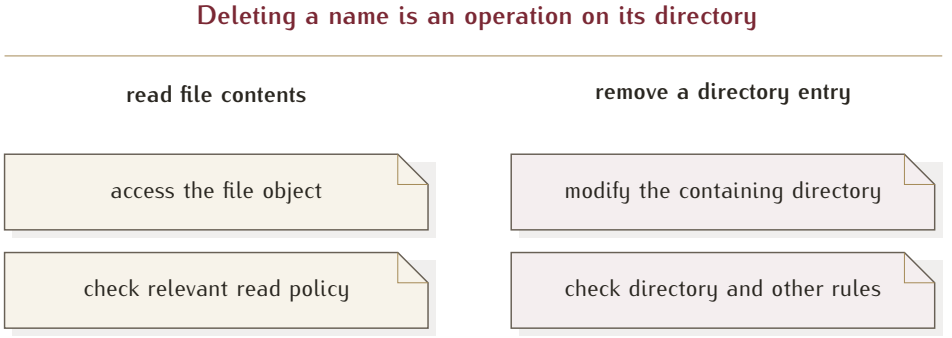


Figure 6.68: *File read permission and permission to remove one of its names are different checks. The directory operation affects a name-to-object relationship; it does not necessarily require permission to modify the file's bytes.*

6.7.3 Encryption and the location of the adversary

Operating-system permission checks govern access through the running system. Encryption at rest addresses reading stored blocks without the required key, for example after an offline theft of a drive. A transparent encrypted filesystem receives plaintext from an authorised application, encrypts the persistent representation, and decrypts it when delivering authorised reads. Plaintext can still exist in application memory and other permitted working locations. Protecting the key and the unlocked system is therefore part of the design.

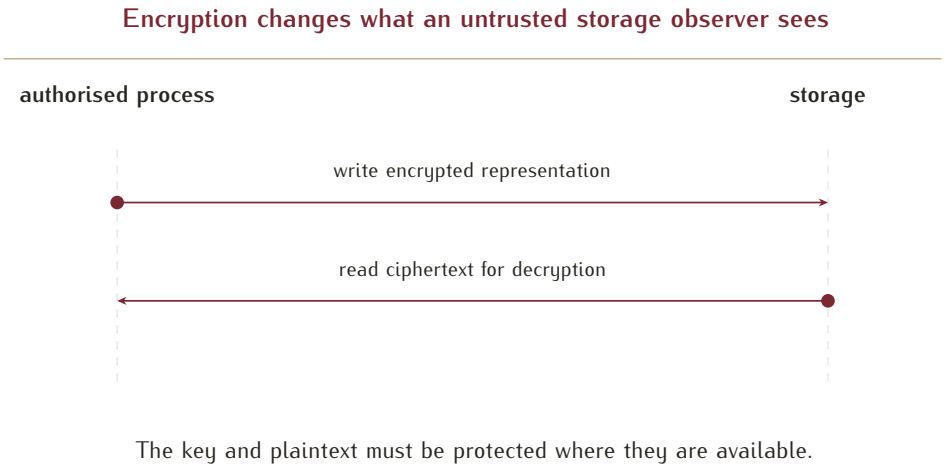


Figure 6.69: *Encryption can conceal stored contents from an observer lacking the key under the chosen threat model. An authorised running endpoint may still hold plaintext and keys, so its compromise raises a different problem.*

Threat	Main control in the exercise model
Unauthorised request through the OS	Access-control policy and enforcement
Offline reading of a removed drive	Encryption with a separately protected key
Both access paths	Both controls, with key protection
Accidental authorised deletion	Recoverable versions or backups

Encryption does not by itself establish all integrity or availability guarantees. Whether stored changes can be detected depends on the cryptographic mode and authentication design. Losing the only usable key can make correct ciphertext inaccessible. The exercises ask for the control addressing a specified threat, so avoid attributing unrelated guarantees to it.

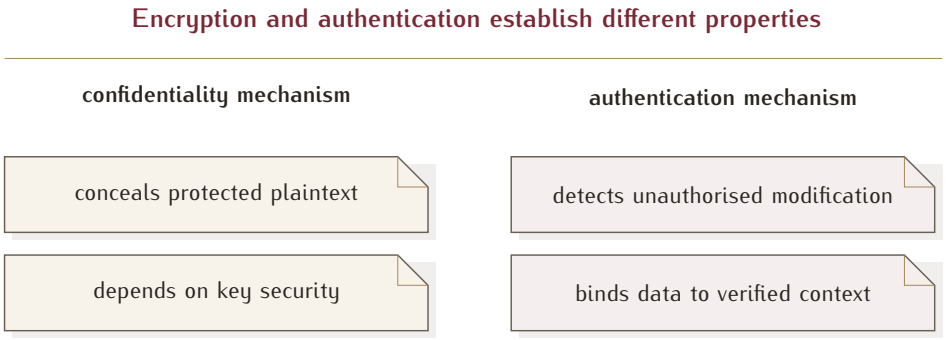


Figure 6.70: *An encrypted-looking byte sequence is not automatically protected against undetected modification. The selected cryptographic construction and nonce, key and verification rules determine which properties are provided.*

6.7.4 Audit evidence and its interpretation

Auditing records events so they can be reconstructed and attributed. A useful record includes time, subject, object, operation and result; identifiers and context may be needed to distinguish concurrent sessions. The result is essential: a denied WRITE is evidence of an attempted operation, while a successful WRITE is evidence that the access was accepted. In this chapter's exercise logs, `WRITE ... OK` specifically records a completed content-changing write. An authorisation or open-for-writing decision alone would not establish that event. It does not imply durable storage. Under that log contract, finding the latest successful modification requires filtering by object, modifying operation and success before ordering by time. Real logs may also need sequence identifiers to resolve equal timestamps or clock adjustments.

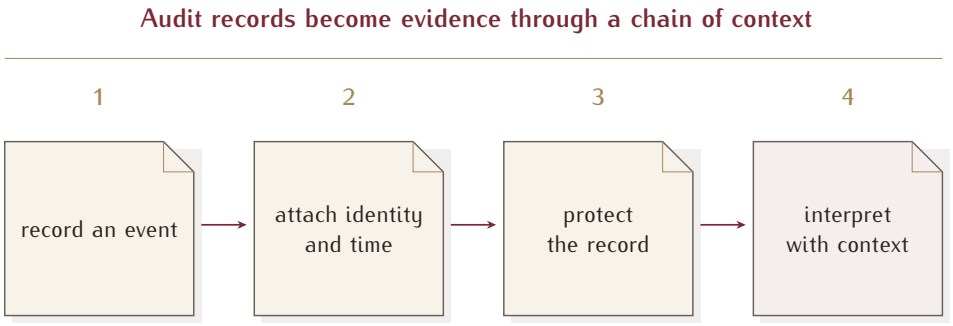


Figure 6.71: *The evidential value of a record depends on what generated it, what it actually reports and whether it could be altered or lost. Interpretation must preserve the distinction between an attempted action and a completed one.*

Logs need protection from tampering and an appropriate retention policy. A log's account identifier establishes what identity the system recorded; further evidence may be needed to attribute the action to a person or process. Auditing supports investigation. The ability to record an authorised deletion does not prevent that deletion or recover its contents.

Filtering before selecting the latest event

A log contains 09:10 Ada WRITE draft OK; 09:12 Ben WRITE draft DENY; 09:14 Cy READ draft OK; 09:15 Ben WRITE notes OK. The latest successful modification of `draft` is Ada's 09:10 WRITE. The later entries are excluded respectively by failure, a read-only operation, and a different object. The requested triple is `ADA, WRITE, OK`.

An audit trace distinguishes attempt, decision and outcome

	event	evidence supplied
request	operation attempted	subject and target
decision	policy accepted or denied	authorisation result
completion	operation succeeded or failed	actual outcome

These records can occur at different times and in different components.

Figure 6.72: *An accepted access decision does not prove that later I/O succeeded. Conversely, an attempted operation can be denied before any object modification. A useful audit correlates the relevant records without collapsing their meanings.*

6.7.5 Exercises

Exercise 6.7.1 — The three protection goals

Classify: S1 restrict disclosure of a research file; S2 prevent unauthorised changes to its measurements; S3 keep it accessible to authorised researchers. Give CONFIDENTIALITY, INTEGRITY or AVAILABILITY in order. Explain which property each requirement protects.

Worked solution

Exercise 6.7.2 — An integrity incident

An unauthorised user changes a stored exam score without reading the rest of the file. Which protection goal is directly violated? Explain why the absence of a broad read does not remove the violation.

Worked solution

Exercise 6.7.3 — Availability and a lost key

An encrypted archive remains intact, but its only decryption key is permanently lost. Which goal fails for its authorised owner, and why does intact ciphertext not suffice?

Worked solution

Exercise 6.7.4 — Elements of a request

Choose the tuple needed for an abstract authorisation decision: A: subject, object, operation, applicable rule; B: CPU, clock, opcode, cache; C: filename only; D: password only; E: timestamp only; F: storage capacity only.

Worked solution

Exercise 6.7.5 — Evaluate an ACL

For `report`, Ada has {READ,WRITE}, Ben has {READ}, and Cy has no permissions. With default denial, evaluate Ben:WRITE, Ada:READ and Cy:READ. Return ALLOW/DENY in order.

Worked solution

Exercise 6.7.6 — Group permissions

Group research grants READ and group editors grants WRITE. Ada belongs only to research; Ben belongs to both. Effective permissions are the union of the listed group grants, with default denial. Evaluate Ada:WRITE, Ben:READ, Ben:WRITE.

Worked solution

Exercise 6.7.7 — Who changes policy?

In the stated ownership model, an ordinary reader asks to add WRITE to their own ACL entry. The owner and an administrator have policy-management authority, and the reader has neither role. Is the change allowed? Explain the distinction between using and editing a permission.

Worked solution

Exercise 6.7.8 — An inheritance rule

A directory’s creation rule gives new files group READ. It copies that entry at creation and does not propagate later changes. File F is created, then the directory default is changed to group READ,WRITE. What group rights does F retain, and what does a new file G receive?

Worked solution

Exercise 6.7.9 — Traversal before a final read

A user may READ `/a/b/file`, may traverse `/a`, but may not traverse `/a/b`. Assume root traversal is allowed and no usable descriptor has already been opened. The path has no alternative links in this problem. Can the requested pathname access succeed?

Exercise 6.7.9 — Traversal before a final read (continued)

Worked solution

Exercise 6.7.10 — List and traverse separately

Directory D grants Ada {LIST,TRAVERSE}, Ben {TRAVERSE}, and Cy no rights. Evaluate Ada:LIST, Ben:LIST, Ben:TRAVERSE, Cy:TRAVERSE.

Worked solution

Exercise 6.7.11 — Decision order

Order IDENTIFY, OPERATION, RULES and DECIDE in this chapter's abstract access-checking procedure. Explain what must be known before the final decision.

Worked solution

Exercise 6.7.12 — Protecting names

Explain why a directory needs protection even when it contains only name-to-inode associations. Give two consequences of unauthorised changes to those associations.

Worked solution

Exercise 6.7.13 — Offline storage theft

Correct ACLs protect a running system. A thief removes its powered-off SSD and reads blocks directly. Choose the additional control addressing disclosure in this scenario: A: longer pathnames; B: a larger TLB; C: encryption with a key unavailable to the thief; D: extra directory entries; E: faster DRAM; F: a permission check that the offline reader bypasses.

Worked solution

Exercise 6.7.14 — Where plaintext exists

A transparent encrypted filesystem receives bytes from an authorised application, stores their encrypted blocks and later returns the read result to the application. Give PLAINTEXT or CIPHERTEXT at application SAVE input, persistent DISK storage and application READ output.

Worked solution

Exercise 6.7.15 — Match control to threat

For S1 an authenticated user requests an unauthorised read through the OS; S2 a removed encrypted drive is read offline; S3 both risks must be addressed. Return ACL, CRYPTO or BOTH in order.

Worked solution

Exercise 6.7.16 — Encryption and key exposure

The stored file is encrypted, but a readable plaintext copy of its decryption key is kept on the same stolen device. Explain which assumption of the offline-protection argument is broken.

Worked solution

Exercise 6.7.17 — A useful audit record

A log stores only time and object name for each access. What fields should be added to attribute an event, distinguish READ from WRITE and distinguish a successful operation from a denied attempt?

Worked solution

Exercise 6.7.18 — An authorised mistake

A user is permitted to delete a file and deletes it accidentally. Explain what ACL enforcement and auditing can each establish, and what mechanism is needed to recover an older copy.

Worked solution

Exercise 6.7.19 — Filtering an incident log

Events are: 10:00 ADA WRITE report OK; 10:03 BEN WRITE report DENY; 10:04 CY READ report OK; 10:05 DAN WRITE notes OK; 10:06 CY WRITE report OK; 10:07 BEN WRITE report DENY. Return SUBJECT,OPERATION,RESULT for the latest successful modification of `report`.

Worked solution

Exercise 6.7.20 — Limits of attribution

A protected log records account ADA performing a successful write at 14:20. What does this establish directly, and what additional question remains before attributing the action to a particular human?

Worked solution

6.7.6 Solutions

Solution 6.7.1

The ordered answers are `CONFIDENTIALITY`, `INTEGRITY` and `AVAILABILITY`. S1 limits who can learn the research data. S2 protects the measurements against unauthorised change. S3 requires that authorised researchers can actually obtain the data when needed. A locked archive can prevent disclosure while still failing S3 if every authorised reader has lost the key.

Return to the problem

Solution 6.7.2

Integrity is directly violated because the stored score was modified without permission. Confidentiality concerns disclosure, which is a different question; the scenario does not need a broad read to establish the unauthorised modification. Availability would concern whether authorised parties can obtain the service or data. A single incident can affect several goals, but the evidence given here directly establishes the integrity failure.

Return to the problem

Solution 6.7.3

Availability fails for the owner because the archive can no longer be recovered in usable form with the available key material. Intact ciphertext preserves the encoded bytes, yet authorised access also requires the ability to decrypt them. The example shows why encryption design needs a deliberate key-recovery policy consistent with the threat model. It does not imply that the ciphertext's confidentiality has failed.

Return to the problem

Solution 6.7.4

Answer A. The decision needs the requesting identity, the resource, the requested action and the rule that relates them. B names hardware state. C omits requester, operation and policy. D can contribute to authentication but does not identify the authorised operation on a particular object. E is contextual information, and F is a capacity measure. None of C–F alone can determine whether a specific access is permitted.

Return to the problem

Solution 6.7.5

Ben's set lacks WRITE, so the first request is DENY. Ada's set contains READ, so the second is ALLOW. Cy's empty set contains no READ permission, so the third is DENY. The sequence is `DENY, ALLOW, DENY`. Each request is evaluated independently against the same object's ACL, preserving the question's order.

Return to the problem

Solution 6.7.6

Ada receives only {READ}, so her WRITE is denied. Ben receives $\{\mathrm{READ}\} \cup \{\mathit{WRITE}\} = \{\mathrm{READ}, \mathit{WRITE}\}$, so both his READ and WRITE requests are allowed. The ordered result is `DENY, ALLOW, ALLOW`. The union rule is an explicit assumption of this exercise; another ACL system can have deny entries or masks that change group combination.

Return to the problem

Solution 6.7.7

The change is denied. READ authorises a content operation; it does not grant authority to alter the object's access policy. Under the supplied model, the owner or the appropriately privileged administrator must make the ACL change. If every reader could add arbitrary rights to their own entry, the ACL would fail to enforce the intended separation between permissions and policy management.

Return to the problem

Solution 6.7.8

F retains the copied READ entry because the problem explicitly excludes propagation to existing objects. G, created under the new default, receives READ and WRITE. The rule therefore determines permissions at creation time. Assuming that F automatically gains WRITE would introduce propagation semantics not present in the contract. Both files can still have other permissions if separately specified.

Return to the problem

Solution 6.7.9

It is denied at directory `/a/b`. Reaching the final object through this pathname requires traversal of every intermediate directory. The final file's READ permission cannot satisfy a missing directory traversal check. The solution identifies the first blocking condition along the given path; an already open descriptor or another permitted path would be a different access scenario.

Worked solution (continued)

Return to the problem

Solution 6.7.10

Ada has LIST, so her request is ALLOW. Ben lacks LIST, so listing is DENY, although his independent TRAVERSE right allows resolution of a known component. Cy lacks TRAVERSE, so the final request is DENY. The sequence is `ALLOW, DENY, ALLOW, DENY`. Listing names and traversing a known name are separate permissions in this model.

Return to the problem

Solution 6.7.11

Use `IDENTIFY, OPERATION, RULES, DECIDE`. Establish the subject and target object, identify the action requested, gather the applicable directory and object rules, then evaluate their conditions. A final decision made before the operation is known could confuse READ with WRITE; one made without applicable rules lacks the policy evidence. Concrete implementations may combine steps internally while preserving these dependencies.

Return to the problem

Solution 6.7.12

Directory changes can remove a name that users rely on or redirect a name to a different object. They can also create misleading names or expose information through listing. These effects matter even when the file's payload bytes are stored elsewhere. The directory governs how objects are named and reached, so its traversal, listing and modification policies are part of protecting the namespace.

Return to the problem

Solution 6.7.13

Answer C. Encryption makes the stored representation require the protected key for recovery under the cryptographic assumptions. A and D change naming, B and E change hardware performance, and F describes the same bypassed enforcement path. The key must remain unavailable through the stolen material and other accessible sources. ACLs remain useful for requests through the authorised running system.

Return to the problem

Solution 6.7.14

The sequence is `PLAINTEXT, CIPHERTEXT, PLAINTEXT`. The application supplies usable content; encryption changes its persistent representation; decryption restores usable content on an authorised read. This does not claim that plaintext never exists elsewhere in the running machine. Application memory and other permitted working buffers can contain plaintext while the system is unlocked.

Return to the problem

Solution 6.7.15

The result is `ACL, CRYPTO, BOTH`. S1 requires authorisation enforcement on the running access path. S2 requires encryption whose key is unavailable to the offline reader. S3 needs both because the two threats reach the data through different paths. The classification does not make either mechanism a substitute for key protection, backups or controls against an already compromised privileged environment.

Return to the problem

Solution 6.7.16

The argument required the thief to lack the key. A usable plaintext key on the stolen device gives the reader the information needed to decrypt the stored ciphertext, so the intended separation between encrypted data and protected key is lost. Encrypting the file remains a mathematical transformation, but its confidentiality benefit against this particular theft depends on key protection.

Return to the problem

Solution 6.7.17

Add subject identity, operation and result. Together with time and object, these fields answer who requested what action on which resource, when, and with what outcome. A denied WRITE must remain distinguishable from a successful WRITE when investigating modifications. Session or process identifiers and protected ordering information can further improve attribution where concurrent activity would otherwise be ambiguous.

Return to the problem

Solution 6.7.18

ACL enforcement can establish that the request was permitted under the policy; it cannot infer the user's intended target from that permission alone. Auditing can record the identity, object, operation and result for investigation. Recovering the contents requires an available earlier version, snapshot or backup under an appropriate recovery policy. A record that a deletion occurred does not contain the deleted file's bytes automatically.

Return to the problem

Solution 6.7.19

Filter to object `report`, operation WRITE and result OK. The surviving events are Ada at 10:00 and Cy at 10:06. The later one yields `CY, WRITE, OK`. Ben's later attempts are denied, Cy's earlier read is not a modification, and Dan's write concerns another object. Filtering before selecting the latest time prevents each of those distractors from changing the answer.

Return to the problem

Solution 6.7.20

The log directly attributes a completed write at 14:20 to the authenticated account ADA under the stated event contract. To identify a human, an investigator must establish who controlled that session. Correlating its session identifier with authentication records and the process that issued the write can distinguish interactive use from an automated job or misuse of a credential. The quality of that attribution depends on the integrity and completeness of the records.

Return to the problem

A

CONSULTING ARCHITECTURAL SPECIFICATIONS

Architectural specifications, binary interfaces and operating-system documentation describe different boundaries. The guide below identifies the source that governs each kind of question and explains how to read its scope. The book develops the required concepts; these references support verification and further study.

A.1 FOUR SOURCE LAYERS

Question	Primary source	What it guarantees	What it does not guarantee
What does an instruction do?	Intel SDM or AMD APM instruction entry	operands, operation, flags, exceptions, mode and encoding rules	latency, throughput or one fixed internal circuit
How do compiled functions cooperate?	System V AMD64 psABI	argument and return classification, stack alignment, preserved registers, ELF conventions	service numbers or Linux kernel implementation
How does Linux implement the interface?	Linux documentation and source tree	the implementation in the inspected revision	permanent symbol names or behaviour on another operating system
How fast is it on one processor?	Intel/AMD optimisation and processor-specific guides; measured counters	documented performance characteristics for named families	ISA semantics or universal timing

A.2 INTEL MANUAL MAP

The combined Intel 64 and IA-32 Software Developer's Manual is large because it serves several audiences [26].

volume 1: basic architecture. Programmer-visible execution environment, data types, registers, operating modes, floating point, SIMD and general architectural concepts.

volume 2: instruction set reference. The normative place for a mnemonic: valid forms, operation pseudocode, flags, exceptions and encoding tables.

volume 3: system programming guide. Protection, paging, interrupts, exceptions, system instructions, APIC, memory ordering, performance monitoring and operating-system mechanisms.

volume 4: model-specific registers. MSRs exposed by processor families. Availability must be checked with CPUID and model-specific documentation.

The optimisation manual is a separate microarchitectural source. Use it for front-end, execution, cache and tuning questions, never to redefine the architectural result of an instruction [28].

A.3 AMD MANUAL MAP

The AMD64 Architecture Programmer's Manual is organised differently and documents AMD's architectural contract, including vendor-specific facilities [27].

volume 1.	Application programming, registers, data types, memory model and the programmer-visible environment.
volume 2.	System programming: modes, protection, paging, exceptions, interrupts, system calls and system resources.
volume 3.	General-purpose and system instruction reference.
volumes 4 and 5.	Vector/media and x87/MMX-related instruction families.

Processor Programming References and Zen-family optimisation guides add model-specific registers, performance events and microarchitectural guidance. They are narrower and more revision-sensitive than the architecture manual [29].

A.4 SYSTEM V AMD64 PSABI MAP

The canonical x86-64 psABI project is a living specification maintained in a public repository [30]. Read it when the question contains any of the following phrases:

- function arguments or return values across separately compiled files;
- stack alignment, red zone, register preservation or variadic calls;
- structure classification, relocations, TLS, PLT/GOT or position-independent code;
- process initial stack, auxiliary vector or AMD64-specific ELF details.

The psABI specifies how user-space binary components cooperate. Its Linux appendix also documents the differences in register use at syscall entry. For the service numbers themselves, consult the target kernel's system-call table and exported headers.

A.5 LINUX SOURCE MAP

The source tree is easier to navigate when the architectural boundary is identified first [35].

Path	Typical question
`arch/x86/entry/`	How do syscalls, exceptions and interrupts enter and leave the kernel?
`arch/x86/kernel/`	How are x86 CPU facilities, APIC, traps, process state and feature enablement managed?
`arch/x86/mm/` and `mm/`	How are page tables, faults, mappings and general virtual memory implemented?
`fs/binfmt_elf.c`	How does `execve` load an ELF process image and construct its initial stack?
`fs/`	How do VFS and filesystems implement file operations?
`kernel/irq/`	How does the generic interrupt subsystem represent and dispatch IRQs?
`block/`	How are block requests queued and completed?
`drivers/nvme/host/`	How does the host driver operate NVMe queues and PCI transport?
`kernel/dma/` and `drivers/iommu/`	How are DMA mappings and I/O address translation implemented?

A.6 A REPEATABLE READING METHOD

For a low-level phenomenon, use the following order.

1. Write the observable question precisely: result bits, flags, register convention, fault, mapping or device transaction.
2. Read the architecture-manual or ABI rule that defines the contract.
3. Identify the Linux entry point or subsystem only if an operating-system implementation is involved.
4. Build the smallest reproducible program, assembly fragment or trace.
5. Use `objdump`, `readelf`, `gdb`, `strace`, `perf` or kernel tracing according to the layer being tested.
6. Record processor model, kernel version, compiler version and command line before drawing a performance conclusion.

> **Caution**
>
> Page numbers and internal function names age quickly. Cite manual volume, chapter or instruction mnemonic; cite a Linux source path, symbol and commit or release tag. A link to a moving `master` branch is useful for exploration but insufficient for a reproducible technical claim.

A.7 SIX GUIDED SOURCE-READING EXERCISES

The Linux traces below refer to release tag `v7.1`, so that paths and symbols identify a reproducible implementation [35].

A.1 — Arithmetic flags

Read the `ADD` entry in the Intel and AMD manuals. Identify its defined arithmetic flags and construct two 8-bit examples separating `CF` and `OF`.

Worked solution

The defined flags are `CF`, `OF`, `SF`, `ZF`, `AF` and `PF` [26, 27]. Addition of `0xFF` and `0x01` produces `0x00` after truncation: the unsigned sum 256 requires a ninth bit, so CF=1; the signed sum $-1 + 1 = 0$ is representable, so OF=0. ZF=1, SF=0, AF=1 and PF=1. AF records carry from bit 3, while PF records even parity of the low result byte, including the zero byte.
Addition of `0x7F` and `0x01` produces `0x80`. The unsigned sum 128 fits in eight bits, giving CF=0. The signed sum $127 + 1$ exceeds 127, giving OF=1. Here ZF=0, SF=1, AF=1 and PF=0 because `0x80` has one set bit. Thus CF tests the unsigned range and OF the signed range for the same result bits. In a debugger, execute each `add al,1` from the stated initial value and inspect flags immediately, before another flag-writing instruction.

A.2 — Aggregate arguments

Under the System V AMD64 LP64 ABI, classify a structure containing two `double` members and a structure containing a `double` followed by 32 `char` elements. Confirm how ordinary functions receive these arguments.

Worked solution

With the ordinary ABI layout, `Pair` occupies 16 bytes, aligned to 8. Its two eightbytes are both classified SSE and are passed in `xmm0` and `xmm1` when those argument registers are available. `Big` occupies 40 bytes, aligned to 8. Its five-eightbyte aggregate does not satisfy the wide-vector exception and has class MEMORY; its value is passed on the stack [30].
Save this complete program as `aggregate.c`:

```
#include <stdio.h>
struct Pair { double x, y; };
```

Worked solution (continued)

```
struct Big { double x; char bytes[32]; };
double pair_sum(struct Pair p) { return p.x + p.y; }
double big_first(struct Big b) { return b.x; }
int main(void) {
    struct Pair p = {2.0, 4.0};
    struct Big b = {3.0, {0}};
    printf("%.0f %.0f\n", pair_sum(p), big_first(b));
    return 0;
}
```

```
gcc -std=c17 -O2 -fno-inline aggregate.c -o aggregate
./aggregate
objdump -d -Mintel aggregate
```

The output is `6 3`. In the externally callable `pair_sum`, a scalar double addition uses the two XMM argument values and returns in `xmm0`. At entry to `big_first`, the first member is at `[rsp+8]`, above the return address; the caller allocated and copied the aggregate into the outgoing stack area. Inspect these named functions, since optimisers may additionally create private specialised versions with different internal conventions. Stack argument passing here transmits a value; it does not make the C parameter a pointer.

A.3 — System-call entry

Follow `entry_SYSCALL_64` to `do_syscall_64`. Separate the state saved by hardware from the state saved by Linux software.

Worked solution

In 64-bit mode, `syscall` places the return RIP in `rcx`, saves flags in `r11` with the architecturally specified RF handling, and loads the kernel entry state from configured MSRs. It masks active flags as configured. Hardware neither pushes a stack frame nor changes `rsp` on this entry path.
In `arch/x86/entry/entry_64.S`, Linux executes `swapgs`, saves user `rsp` in per-CPU scratch storage, switches address-space state where configured, and selects the kernel stack. Software then constructs `pt_regs`, including user stack state, saved flags, return address, syscall number and general registers. It passes the register-frame pointer and syscall number to `do_syscall_64` in `arch/x86/entry/syscall_64.c`.
That function performs entry work, dispatches a valid native number through `x64_sys_call`, records the result in the saved accumulator and performs exit work. Its return value selects whether the checked user state permits the `sysretq` path;

Worked solution (continued)

other cases use the slower return path. The register frame resembles an interrupt-return frame because Linux constructs it in software. Its presence does not imply that `syscall` pushed it.

A.4 — ELF loading

Connect `PT_LOAD`, `PT_INTERP` and the auxiliary vector in `fs/binfmt_elf.c` to the output of `readelf -lW` for a dynamically linked executable.

Worked solution

Run `readelf -lW ./aggregate` on the executable from A.2. Each LOAD row describes a mapped segment: file offset, virtual address, file size, memory size, flags and alignment. `load_elf_binary` processes these program headers. When memory size exceeds file size, the extra mapped bytes are zero-initialised, as required for uninitialised storage. For a PIE, runtime addresses additionally include the chosen load bias.

The INTERP row names the dynamic loader. Linux opens that file and maps its loadable segments through `load_elf_interp`; initial user execution begins at the interpreter's relocated entry point. The loader later transfers control to the executable after performing its startup work.

`create_elf_tables` builds auxiliary-vector entries on the initial stack: `AT_PHDR` gives the executable's program-header address, `AT_PHNUM` its count, `AT_PHENT` the entry size, `AT_BASE` the interpreter's base, and `AT_ENTRY` the executable's relocated entry. `readelf` reports file metadata; it cannot report the actual ASLR-selected addresses of a future run. In GDB, stop at startup and use `info auxv` to compare the runtime values. The executable's `AT_ENTRY` and the initial interpreter PC therefore serve different purposes.

A.5 — NVMe completion

Follow an ordinary successful PCI NVMe I/O completion into the Linux block layer. Identify the device-written entry, the matched request and the completion operations.

Worked solution

In `drivers/nvme/host/pci.c`, the device writes a `struct nvme_completion` into the host-memory completion queue. `nvme_irq` calls `nvme_poll_cq`; the latter checks the phase bit through `nvme_cqe_pending` and executes `dma_rmb()` before consuming the rest of the entry. `nvme_handle_cqe` obtains the command identifier and uses the

Worked solution (continued)

queue's tag set to find the associated `struct request`. An asynchronous controller event follows a separate special case.
For the ordinary unbatched completion path, `nvme_pci_complete_rq` unmaps the DMA request and calls `nvme_complete_rq`. In `drivers/nvme/host/core.c`, disposition handling chooses completion, retry or another recovery path. Successful completion reaches `nvme_end_req` and `blk_mq_end_request`. Batched completions take the corresponding batch helpers. The PCI code advances the completion head and updates its doorbell as required by the queue protocol.
The hardware entry is a compact protocol record; the kernel request holds software I/O state. Matching them by command identifier connects device completion to the waiting software operation. For a write, that completion establishes only the command's specified persistence semantics.

A.6 — True and name dependencies

Compare a five-stage load-use hazard with register renaming for these instructions, assuming distinct valid memory and no exceptions:

```
mov rax, [rdi]       # I1
add rcx, rax         # I2
mov rax, rdx         # I3
```

Identify RAW, WAR and WAW dependencies and explain which constraints renaming can remove.

Worked solution

I2 requires the value loaded by I1: I1 to I2 is a true RAW dependency on `rax`. In the five-stage PIPE timing used in this book, the load result arrives after the following instruction would need it in Execute. A load-use interlock delays that instruction for one cycle and forwarding then supplies the result.
I2 must read the earlier `rax` before I3's logical overwrite: this is a WAR name dependency. I1 and I3 both write `rax`, producing a WAW name dependency. With renaming, assign I1's destination physical register P7 and bind I2's source to P7; assign I3's destination a different register P8. I3 can produce P8 without destroying P7 or waiting for I2 to read it. Architectural retirement preserves program order and the final mapping of `rax`.
I2 still waits for P7 to become ready. Renaming creates separate storage for different versions of a name; it cannot supply a value before its producer computes or loads it. This example contains a load followed by register-only operations, so no memory-alias dependency is involved.

B

TECHNICAL REFERENCE

Use the command tables to inspect a program, the register tables to reconstruct a call, and the timing formulas to check a numerical trace. The integrated cases follow one operation across the hardware and operating-system interfaces.

B.1 LABORATORY AND COMMAND REFERENCE

These commands support the laboratories and examples in this book. A shell or C manual supplies their complete option and language definitions. The emphasis is on reproducible observation: inspect an artefact, state a prediction, run the command, and explain any discrepancy.

Navigating and inspecting files.

Command	Purpose
`pwd`	Print the current working directory.
`ls -la path`	List entries, including hidden names, with metadata.
`cd path`	Change the shell's current directory.
`mkdir name`	Create a directory.
`cp source target`	Copy a file.
`mv source target`	Move or rename a directory entry.
`rm name`	Remove a directory entry; use cautiously.
`ln source hardname`	Create a hard link.
`ln -s target linkname`	Create a symbolic link containing a target path.
`stat file`	Display file type, size, permissions and timestamps.
`file file`	Classify content from metadata and signatures.

Viewing bytes and text.

Command	Purpose
`cat file`	Copy file bytes to standard output; suitable for ordinary text, not arbitrary binary data.
`xxd -g 1 file`	Hexadecimal dump grouped by individual bytes.
`od -An -tx1 file`	Print bytes as hexadecimal without addresses when `-An` is used.
`wc -c file`	Count bytes.
`wc -l file`	Count newline characters/lines by the utility's convention.
`printf '...'`	Produce controlled bytes without the implementation-dependent conveniences of `echo`.

Compilation stages.

For a source file `main.c`:

```
gcc -E main.c -o main.i # preprocess
gcc -S -O0 main.c -o main.s # compile to assembly
gcc -c main.s -o main.o # assemble to relocatable object
gcc main.o -o main # link executable
```

Useful inspection commands include `readelf -h main`, `readelf -s main.o`, `objdump -d main` and `nm main.o`. The object and executable headers record different stages of the build. Use them here to confirm that text source, assembly, object files and executables are different representations.

Process inspection.

Command	Purpose
`ps -ef`	Snapshot of processes.
`ps -o pid,ppid,state,comm`	Select identifiers, parent, state and command columns.
`jobs`, `fg`, `bg`	Shell job-control commands.
`kill PID`	Send a termination signal by default.
`cat /proc/PID/maps`	On Linux, display a process's virtual-memory mappings.

Recommended compiler diagnostics.

For small exercises, compile with:

```
gcc -std=c17 -Wall -Wextra -Wconversion -Wsign-conversion \
 -Wshadow -pedantic -O2 program.c -o program
```

Warnings frequently reveal width, signedness and precedence mistakes. Their coverage is incomplete, so a warning-free build still requires reasoning about correctness. During debugging, sanitizers can detect many runtime errors:

```
gcc -std=c17 -Wall -Wextra -O1 -g \
 -fsanitize=address,undefined program.c -o program
```

A clean sanitizer run covers the paths and inputs exercised in that run. Bounds and lifetime arguments extend the analysis to other cases.

A minimal byte-printing helper.

```
#include <stddef.h>
#include <stdio.h>

static void print_bytes(const void *object, size_t size) {
 const unsigned char *p = object;
 for (size_t i = 0; i < size; ++i)
 printf("%02x%s", (unsigned)p[i], i + 1 == size ? "\n": " ");
}
```

Reading an object's representation through `unsigned char` is the standard C technique for byte inspection. The displayed byte order is a property of the implementation; the abstract value of the object remains governed by its type.

B.2 LINUX ASSEMBLY TOOLCHAIN REFERENCE

This appendix gives a minimal reproducible workflow for the accompanying examples. Commands assume GNU tools on an x86-64 Linux system.

From C to compiler assembly.

```
gcc -O0 -fno-omit-frame-pointer -S -masm=intel example.c -o example_O0.s
gcc -O2 -S -masm=intel example.c -o example_O2.s
```

The `-S` option stops before assembly. Compare optimisation levels, but do not assume that the source variable names survive.

Relocatable objects and executables.

```
gcc -c -O0 example.c -o example.o
gcc example.o -o example
objdump -dr -Mintel example.o
objdump -d -Mintel example
readelf -h -S -s -r example.o
readelf -W -l -s example
```

Use `-r` with object disassembly to show relocation entries. Program headers from `readelf -l` describe loadable segments and permissions.

Hand-written GNU assembler.

A source beginning with `.intel_syntax noprefix` can be assembled through GCC:

```
gcc -c raw.s -o raw.o
gcc -nostdlib -no-pie raw.o -o raw
```

For ordinary functions linked with C, omit `-nostdlib`; provide global symbols and follow the System V ABI.

NASM alternative.

```
nasm -f elf64 raw.asm -o raw.o
ld raw.o -o raw
```

NASM and GNU assembler Intel syntax are similar but not identical. Directives, symbol-address syntax and comments differ. The complete assembly examples in Appendix C use GNU assembler with Intel syntax and identify the required directives.

Debugger workflow.

```
gdb -q ./example
(gdb) set disassembly-flavor intel
(gdb) break main
(gdb) run
(gdb) disassemble /r main
(gdb) info registers
(gdb) x/16gx $rsp
(gdb) si
```

`disassemble /r` shows instruction bytes. The `x` command uses the form `x/NFU address`: count, format and unit size. Examples include `x/16bx ptr` for bytes and `x/8gx $rsp` for eight qwords.

Observe system calls.

```
strace -f ./program
strace -e trace=read,write,openat,close,exit_group ./program
```

The trace shows system calls at the kernel boundary and their decoded arguments. It does not replace disassembly: a library wrapper may issue additional calls or retry interrupted operations.

Inspect raw bytes.

```
objcopy -O binary --only-section=.text raw.o text.bin
xxd -g1 text.bin
objdump -d -Mintel raw.o
```

This workflow checks the bytes produced by an encoding whose allowed values are restricted. Always inspect the assembled bytes; constants and register choices can change

instruction length and encoding. A relocatable object may still contain unresolved relocation fields: inspect them with `readelf -r raw.o`, and link before extracting bytes that require final symbol addresses.

Useful compiler diagnostics.

```
gcc -Wall -Wextra -Wconversion -Wpadded -O2 source.c
gcc -fsanitize=address,undefined -g source.c -o checked
```

Warnings and sanitizers expose many width, layout and memory-lifetime mistakes during development. Their results complement the reasoning about bounds and object lifetime.

B.3 x86-64 REGISTER AND CONVENTION REFERENCE

General-purpose registers and subregisters.

64-bit name	Smaller names	ABI role or architectural note
`rax`	`eax`, `ax`, `al`	function return value; syscall number and raw result
`rbx`	`ebx`, `bx`, `bl`	callee-saved
`rcx`	`ecx`, `cx`, `cl`	function argument 4; caller-saved; destroyed by `syscall`
`rdx`	`edx`, `dx`, `dl`	argument 3; high half for multiply and divide
`rsi`	`esi`, `si`, `sil`	argument 2
`rdi`	`edi`, `di`, `dil`	argument 1
`rbp`	`ebp`, `bp`, `bpl`	callee-saved; optional frame pointer
`rsp`	`esp`, `sp`, `spl`	stack pointer
`r8`	`r8d`, `r8w`, `r8b`	function and syscall argument 5
`r9`	`r9d`, `r9w`, `r9b`	function and syscall argument 6
`r10`	`r10d`, `r10w`, `r10b`	syscall argument 4; caller-saved
`r11`	`r11d`, `r11w`, `r11b`	caller-saved; destroyed by `syscall`
`r12`–`r15`	corresponding `d`, `w`, and `b` forms	callee-saved

A write to a 32-bit general-purpose subregister clears the high 32 bits of its 64-bit parent. Writes to 8- or 16-bit subregisters preserve the other bits. The legacy high-byte names `ah`, `bh`, `ch` and `dh` cannot be encoded with a REX prefix and should be avoided in new 64-bit code.

System V AMD64 function convention.

- For scalar integer/pointer arguments, without hidden arguments or aggregates, the first six use `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`.
- Integer/pointer return: `rax`; a second integer return component may use `rdx`.
- Caller-saved: `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`.
- Callee-saved: `rbx`, `rbp`, `r12`–`r15`; the callee also restores the caller's stack position on return.
- The baseline stack alignment before `call` is 16 bytes; after the pushed return address, a callee enters with `rsp` mod $16 = 8$. Stack-passed wide vectors and over-aligned arguments can require stronger alignment under the psABI.
- A leaf function may use the 128-byte red zone below `rsp` when the environment follows the ABI and asynchronous kernel code preserves it.

Linux x86-64 syscall convention.

number	arg 1	arg 2	arg 3	arg 4	arg 5	arg 6
`rax`	`rdi`	`rsi`	`rdx`	`r10`	`r8`	`r9`

The raw result is in `rax`. `rcx` and `r11` are destroyed by `syscall`. System-call numbers are operating-system ABI data and should be taken from the target system's headers or documentation rather than guessed.

Condition-code quick reference.

Meaning	Jump	Set form	Condition after CMP a,b
Equal	`je`/`jz`	`sete`	$a = b$
Not equal	`jne`/`jnz`	`setne`	$a \neq b$
Unsigned above	`ja`	`seta`	$a > b$ unsigned
Unsigned below	`jb`	`setb`	$a < b$ unsigned
Signed greater	`jg`	`setg`	$a > b$ signed
Signed less	`jl`	`setl`	$a < b$ signed
Greater/equal signed	`jge`	`setge`	$a \geqslant b$ signed
Less/equal unsigned	`jbe`	`setbe`	$a \leqslant b$ unsigned

B.4 y86-64 quick reference

Programmer-visible state.

The teaching ISA exposes fifteen 64-bit registers, a 64-bit program counter, condition codes ZF, SF, OF, byte-addressed memory and a status code. Register id `F` means "no register".

id	register	id	register	id	register	id	register
0	`rax`	1	`rcx`	2	`rdx`	3	`rbx`
4	`rsp`	5	`rbp`	6	`rsi`	7	`rdi`
8	`r8`	9	`r9`	A	`r10`	B	`r11`
C	`r12`	D	`r13`	E	`r14`	F	RNONE

Instruction formats.

Instruction class	icode:ifun	register byte	constant	length
`halt`	0:0	–	–	1
`nop`	1:0	–	–	1
`rrmovq/cmovXX`	2:0–6	rA:rB	–	2
`irmovq`	3:0	F:rB	valC	10
`rmmovq`	4:0	rA:rB	displacement	10
`mrmovq`	5:0	rA:rB	displacement	10
`OPq`	6:0–3	rA:rB	–	2
`jXX`	7:0–6	–	destination	9
`call`	8:0	–	destination	9
`ret`	9:0	–	–	1
`pushq`	A:0	rA:F	–	2
`popq`	B:0	rA:F	–	2

All eight-byte constants are little-endian. The next sequential address is `valP=PC+length`.

ALU and condition functions.

ifun	OPq	ifun	condition
0	add	0	always
1	sub	1	le
2	and	2	l
3	xor	3	e
		4	ne
		5	ge
		6	g

Stage summary.

Stage	Questions to answer
Fetch	What are `icode/ifun`? Is there a register byte? Is there `valC`? What is `valP`? Is the instruction valid?
Decode	Which registers are `srcA/srcB`? What values are read? Which registers are `dstE/dstM`?
Execute	What are `aluA/aluB`? Which ALU function applies? Is `valE` an arithmetic result, effective address or stack update? Are condition codes changed?
Memory	Is memory read or written? Which address and write data are used? Did an address error occur?
Write-back	Which destination receives `valE`? Which receives `valM`?
PC	Does the next PC come from `valP`, `valC`, `valM`, or a condition-dependent choice?

Status values.

Status	Meaning	Consequence
AOK	normal operation	continue
HLT	halt encountered	stop cleanly
ADR	invalid instruction/data address	stop with address-error status
INS	invalid instruction encoding	stop with encoding-error status

B.5 DATAPATH AND PIPELINE CONTROL REFERENCE

Common SEQ source and destination rules.

Use these selectors to check the equations derived from each instruction's semantics.

Instruction	srcA	srcB	dstE	dstM
rrmovq/cmov	rA	–	rB if condition	–
irmovq	–	–	rB	–
rmmovq	rA	rB	–	–
mrmovq	–	rB	–	rA
OPq	rA	rB	rB	–
call	–	rsp	rsp	–
ret	rsp	rsp	rsp	–
pushq	rA	rsp	rsp	–
popq	rsp	rsp	rsp	rA

Memory controls.

Instruction	Read?	Write?	Address/data
rmmovq	no	yes	address valE, data valA
mrmovq	yes	no	address valE
call	no	yes	address valE, data valP
ret	yes	no	address valA
pushq	no	yes	address valE, data valA
popq	yes	no	address valA

Pipeline-register contents.

f	predicted PC.
d	status, icode, ifun, rA, rB, valC and valP.
e	instruction fields (icode and ifun), status, valC, operand values valA/valB, source identifiers srcA/srcB and destination identifiers dstE/dstM.
m	status, icode, condition result, valE, valA, dstE and dstM.
w	status, icode, valE, valM, dstE and dstM.

Hazard recognition.

Load-use.

A load-use hazard exists when the instruction in Execute is `mrmovq` or `popq` and its `E_dstM` equals either source of the instruction in Decode. Standard action: stall F and D, bubble E.

Conditional-branch misprediction.

A conditional jump in Execute whose predicted direction differs from `e_Cnd` invalidates younger instructions. Standard action for the predict-taken design: bubble D and E, and select the saved fall-through PC.

Return.

A return in D, E or M makes the fetch address uncertain until memory supplies the popped PC. Standard action: stall Fetch and inject a bubble into Decode, except when a simultaneous load-use condition requires Decode to stall.

Forwarding checklist.

When Decode needs a source value, search for the newest producer in this conceptual order:

1. value newly computed by Execute, matched against its effective destination (`RNONE` for a failed conditional move);
2. value newly read by Memory;
3. ALU value stored in M;
4. memory value in W, then its ALU value (memory-result priority resolves `popq %rsp`);
5. register-file value.

A special case supplies `D_valP` as `valA` for calls and jumps when that value must travel down the pipeline.

Trace template.

Cycle	F	D	E	M	W
1					
2					
3					
4					
5					
6					

For a rigorous trace, write both the instruction and the fields relevant to the hazard: sources, destinations, valE/valM and branch condition.

B.6 EXCEPTION AND SYSTEM-CALL REFERENCE

Exception classes.

Event/class	Origin	Reported continuation	Typical consequence
External interrupt	asynchronous source	architectural boundary after older work	service source, then resume or schedule
Fault	synchronous exception	normally the faulting instruction	repair and retry, or terminate
Trap	synchronous exception	normally after the triggering instruction	report/debug, then continue when appropriate
Abort	severe processor exception	reliable restart not generally available	contain failure or stop affected execution

Selected x86 architectural vectors.

Vector	Conventional name	Class/use
0	Divide error (`#DE`)	fault
1	Debug (`#DB`)	fault/trap depending on cause
3	Breakpoint (`#BP`)	trap
6	Invalid opcode (`#UD`)	fault
8	Double fault (`#DF`)	abort
13	General protection (`#GP`)	fault
14	Page fault (`#PF`)	fault
18	Machine check (`#MC`)	abort
32 and above	OS-selected external vectors	interrupts

The table is deliberately selective. Fault, trap and abort are processor-exception classes; an external interrupt is a separate asynchronous source of protected control transfer. Vector allocation for external interrupts is operating-system and platform policy, and the detailed class of a processor exception can depend on its cause. Native x86-64 `syscall` is a dedicated protected-entry instruction rather than an Intel fault, trap or abort exception.

IDT arithmetic.

For descriptor size d and vector v,

$$\text{descriptor address} = \text{IDT base} + vd.$$

In x86-64 long mode, an IDT gate descriptor occupies 16 bytes, so a full 256-entry table occupies 4096 bytes.

Linux x86-64 raw system-call convention.

Purpose	Register
Service number	`rax`
Argument 1	`rdi`
Argument 2	`rsi`
Argument 3	`rdx`
Argument 4	`r10`
Argument 5	`r8`
Argument 6	`r9`
Return value / raw error	`rax`
User return RIP saved by `syscall`	`rcx`
User flags saved by `syscall`	`r11`

These registers implement the native Linux x86-64 kernel entry convention. Service numbers are platform-specific. Applications should normally use the system's library wrappers.

A trace checklist.

When analysing an entry path, answer in order:

1. What event or instruction initiates entry?
2. Is it synchronous or asynchronous?
3. Which vector or fixed entry point is selected?
4. Which privilege level and stack are used?
5. Which state is saved by hardware and by software?
6. Which table or dispatcher selects the service routine?
7. What validates user-controlled values?
8. Where is the result placed?
9. Which instruction restores user execution?

B.7 I/O AND STORAGE CALCULATION REFERENCE

Polling and interrupt cost.

For check cost c cycles, CPU frequency f and polling period P:

$$U_{poll} = \frac{c}{fP}, \qquad E[L_{poll}] \approx \frac{P}{2}.$$

For event rate λ, fixed interrupt cost C_i seconds and batch size b:

$$U_{int,fixed} \approx \frac{\lambda C_i}{b}.$$

The polling latency assumes an event phase uniformly distributed within a feasible period ($c/f \leqslant P$). The interrupt estimate assumes a stable event rate and a mean batch size b. Both utilisation formulas omit the actual work performed per event.

Bus and link bandwidth.

Parallel peak bandwidth:

$$B_{peak} = \frac{w}{8} t f,$$

where w is bits per transfer, t transfers per clock and f clocks per second. Useful bandwidth can be modelled as

$$B_{useful} = B_{raw} \eta u,$$

where η is payload efficiency and u is busy fraction.

Magnetic-disk access.

$$T_{rev} = \frac{60}{RPM}, \qquad E[T_{rot}] = \frac{T_{rev}}{2},$$

$$T_{access} = T_{seek} + T_{rot} + \frac{n}{B} + T_{controller}.$$

The mean rotational latency assumes a uniformly distributed angular position. For one outstanding independent request, a rough IOPS estimate is $1/E[T_{access}]$. Queueing and reordering can change observed throughput and latency.

RAID capacity and small writes.

For equal drives of capacity C, ignoring metadata and spares:

$$C_0 = NC, \quad C_1 = C \ (N = 2), \quad C_{10} = NC/2 \ (N \geqslant 4 \text{ even}),$$

$$C_5 = (N-1)C \ (N \geqslant 3), \quad C_6 = (N-2)C \ (N \geqslant 4).$$

A classic RAID 5 read-modify-write small update performs two reads and two writes. Full-stripe writes avoid reading old data and parity.

Flash write amplification.

$$WA = \frac{\text{physical bytes programmed}}{\text{host bytes written}}.$$

$WA \geqslant 1$ when each host byte must eventually be programmed, without compression or deduplication, and the accounting interval includes deferred writes. Garbage collection, metadata and relocation raise the numerator; compression or a short observation window can invalidate that lower bound.

PIO–DMA crossover.

For

$$C_{PIO} = S_p + nc_p, \qquad C_{DMA} = S_d + nc_d,$$

the equal-cost size is

$$n^* = \frac{S_d - S_p}{c_p - c_d}.$$

In the case considered here, DMA has higher setup cost ($S_d > S_p$) and lower cost per byte ($c_d < c_p$), so its CPU cost becomes smaller above the positive crossover. If $S_d \leqslant S_p$ and $c_d < c_p$, DMA is already no more expensive at size zero and cheaper at positive sizes in this model. DMA may still be chosen below the crossover for overlap or avoided above it when mapping cost or latency dominates.

DMA-visible addresses.

A CPU pointer is a virtual address in a CPU address space. The operating system can translate that address to physical memory, while a device is given a DMA or device-visible address returned by the platform's DMA-mapping interface. With an IOMMU the device-visible address can differ from the CPU physical address as well as from the original virtual address. Drivers must therefore use the address produced by the DMA API rather than deriving a device address from a user or kernel pointer.

Unit discipline.

- Convert RPM to revolutions per second before using time formulas.
- Distinguish bits/s from bytes/s.
- State whether MB means 10^6 bytes and MiB means 2^{20} bytes.
- Add latencies only after converting them to the same unit.
- Compare the advertised peak with the useful payload rate measured for the workload.

B.8 INTEGRATED I/O TRACES

The individual mechanisms become fully intelligible when they are recombined into end-to-end traces. The traces in this appendix follow complete operations across ap-

plication code, the kernel, controller queues, memory and hardware completion. Each trace deliberately names the boundary at which responsibility changes.

B.8.1 Case study I: from a key press to a blocked read

Assume an application is reading a local virtual terminal, with an interrupt-driven PS/2-style keyboard controller. Its `read` is blocked because the terminal input condition is unsatisfied. A key press now supplies the input needed by that condition: a newline in canonical mode, or sufficient bytes in the selected noncanonical mode. USB keyboards add host-controller polling and USB transfers before the input event reaches this path.

Event timeline.

1. The keyboard detects the physical event and emits a device-specific code.
2. A controller places the code in a data register or queue and marks the condition ready.
3. The controller raises an interrupt request. The interrupt-routing hardware selects a destination core and vector.
4. At an instruction boundary, the core records return state, enters privileged execution and transfers through the IDT.
5. The low-level interrupt stub saves the registers required by its calling convention and invokes the keyboard driver.
6. The driver reads the controller status and data, acknowledges the source and translates the device code into the operating system's input representation.
7. The terminal subsystem appends the input to a kernel buffer. Depending on terminal mode, it may wait for a complete line or make a byte immediately available.
8. Once the input condition is satisfied, the blocked reader is marked runnable. After interrupt handling, execution may resume in the interrupted task or pass through the scheduler.
9. When the scheduler selects the reader, its `read` system call continues in the kernel, copies available bytes to the user buffer and returns a count.

The input event satisfies the terminal wait condition. Scheduling and the resumed read path determine when the application receives the bytes.

Worked example: response-time decomposition

Suppose interrupt routing and entry take 2 µs, driver and terminal processing take 8 µs, and the awakened process waits 140 µs before being scheduled. Copying and return take 3 µs. The user-visible latency is approximately

$$2 + 8 + 140 + 3 = 153\ \mu s.$$

Worked example (continued)

Optimising the handler by one microsecond changes the total little because scheduler delay dominates this instance.

Acknowledge at the correct layer

The device, interrupt controller and operating system may each require a distinct completion action. Clearing only the controller's end-of-interrupt state while leaving the device condition active can cause immediate retriggering. Clearing the device too early can discard unread data.

B.8.2 Case study II: a file write that must become durable

An application writes a database record and then requests durability. The path contains two different completions: data accepted by the kernel and data committed to persistent media.

Application and page-cache phase.

The application calls `write`. The kernel service path checks the descriptor, pointer and length. In a buffered file system, the kernel usually copies data into page-cache pages, marks them dirty and returns a byte count. The returned count can be shorter than requested: the application must handle errors and short writes until the intended record has been accepted. It may then reuse its buffer, although a power failure may still lose the update.

A later `fsync`-like operation asks the kernel to write dirty data and required metadata in an order that makes the file-system update recoverable. The file system creates block requests. The block layer may merge adjacent requests and submit them to a storage driver.

Controller and DMA phase.

The driver maps buffers for DMA and builds descriptors using the resulting device-visible addresses, direction, logical block addresses and lengths. It synchronises and publishes descriptors, rings a doorbell and may block the caller. The controller fetches the descriptors, reads data from memory and sends it to the storage device. Completion entries are written into memory and an interrupt or polled queue notifies the driver.

A device with a volatile write cache can report ordinary write completion before the medium is safe. Durability therefore may require a flush or force-unit-access command. A successful synchronisation call returns after the relevant persistence contract is satisfied; failure must be reported and handled. Persisting a newly created directory entry can additionally require synchronising its containing directory. These operations do not by themselves make an arbitrary multiblock database record crash-atomic.

Observed event	What it proves
`write` returns a positive count	that many bytes were accepted under the API; often copied into kernel memory
DMA completion	controller consumed or produced the described memory buffers
Device write completion	command completed according to device semantics; cache may still be volatile
Successful flush completion	earlier writes reached the persistence domain promised by the device

Worked example: throughput with batched persistence

A device takes 0.25 ms to transfer each 64 KiB record and 2 ms for a cache flush. Flushing after every record gives 2.25 ms per record, about 444 records/s. Writing eight records and flushing once takes $8 \cdot 0.25 + 2 = 4$ ms, or 2000 records/s. Batching improves throughput but increases the amount of recent work at risk before the flush and can increase individual commit latency.

B.8.3 Case study III: a page fault that requires storage I/O

A process references a valid virtual page whose contents are part of an executable file but are not resident in memory.

1. The memory-management unit cannot complete translation and raises a page fault.
2. The processor enters the page-fault handler with the faulting instruction address preserved.
3. The kernel finds the virtual-memory area and confirms that the access is permitted.
4. No suitable cached page exists, so the kernel allocates a physical frame and creates the requests needed for the file data.
5. The kernel arranges completion tracking and the driver submits the DMA request. The wait protocol checks completion while registering the waiter, so an early completion cannot be lost.
6. If I/O is still pending, the faulting task sleeps and another task can run. The device transfers the data into the frame and reports completion.
7. Completion processing records the result and makes the waiter runnable. When the fault handler resumes, it checks the result and revalidates the mapping before installing the page-table entry.
8. On successful I/O and mapping installation, the kernel returns from the exception. The original instruction executes again and now succeeds.

The device performs the transfer while another task can use the processor. The faulting load resumes only after the page is resident, its mapping is revalidated and the scheduler restores the waiting task.

Worked example: major-fault latency

Assume fault entry and VM lookup take 8 µs, queue waiting is 1.5 ms, device service is 9 ms, completion processing is 20 µs, and rescheduling adds 0.4 ms. Total is about

$$0.008 + 1.5 + 9 + 0.020 + 0.4 = 10.928 \text{ ms}.$$

At 3 GHz this spans roughly 32.8 million cycles. Blocking the task is essential; busy waiting would waste enormous CPU capacity.

B.8.4 Case study IV: an NVMe-style queue submission

Modern high-performance storage commonly presents submission and completion queues in host memory. Details vary by specification, but the general pattern illustrates the Part IV mechanisms clearly.

1. The driver chooses a submission-queue slot and fills a command containing an operation, logical block range, identifier and pointers to data descriptions.
2. A memory-ordering operation publishes the command before the driver updates a doorbell register through MMIO.
3. The controller fetches the command by DMA, reads or writes the data buffers and executes the storage operation.
4. The controller writes a completion entry into host memory. A phase or ownership indicator distinguishes new entries after ring wrap-around.
5. With interrupts enabled for the queue, it can raise an MSI-X interrupt according to the notification policy; otherwise the host discovers the entry by polling.
6. The driver consumes completion entries, unmaps or recycles buffers, advances the completion head and finishes the operating-system request.

Multiple queue pairs let cores submit with less shared locking. Queue affinity can align an application, software completion path and interrupt on the same core or NUMA node. This improves locality but must be balanced against uneven workloads.

Worked example: queue-depth throughput bound

A device has average command latency 100 µs. With one outstanding request, Little's-law reasoning bounds throughput near $1/100\,\mu\text{s} = 10{,}000$ operations/s. With queue depth 32 and enough internal parallelism, the latency-bound ceiling becomes roughly 320,000 operations/s. The device may hit bandwidth or con-

Worked example (continued)

troller limits first, and queueing can increase latency, but concurrency is necessary to expose parallel service.

B.8.5 Case study V: choosing between PIO, DMA, interrupts and polling

Consider three transfers.

four-byte configuration read. DMA setup is far larger than the payload. A direct MMIO or port read is appropriate, possibly with polling if the register becomes ready quickly.

one 4 kib storage request during light load. DMA moves the data; an interrupt provides low idle CPU cost. The fixed setup is justified by payload size and overlap.

continuous high-rate network receive. DMA descriptor rings move packets. One interrupt per packet is too expensive, so interrupt moderation, batching or a hybrid interrupt-to-poll mode is appropriate.

The mechanisms are not competing at one level. DMA answers *who moves bytes*; polling and interrupts answer *how completion is noticed*; the bus protocol answers *how transactions are carried*; the system call answers *how an application requests the operation*.

B.8.6 Failure analysis checklist

For an end-to-end trace, ask what happens if:

- the user pointer is invalid;
- the service number or descriptor is out of range;
- the device never completes;
- the interrupt is lost or continuously asserted;
- the DMA length is wrong;
- a memory barrier is missing;
- the drive fails during a RAID update;
- power fails after a buffered write but before a persistence barrier;
- the same completion is processed twice.

A complete design specifies timeouts, cancellation, reset, idempotence or recovery for the failures relevant to its model.

B.8.7 Integrated exercises with solutions

T.1 - Key press and sleeping process

Why does the interrupted process not necessarily receive the key, and why can the process that receives it return from `read` much later?

Worked solution

The interrupt can arrive while any process is running. The handler records input in kernel state and wakes the process waiting on the terminal. That process runs only when the scheduler selects it, then resumes its blocked system call and copies the input to user memory.

T.2 - Fault and retry

During a major page fault, why must the saved address identify the original load rather than the following instruction?

Worked solution

The load has not produced its architectural result. After storage I/O installs the page, the processor must execute the load again. Continuing after it would leave the destination register without the required loaded value.

T.3 - Completion ambiguity

A driver receives a DMA completion for a write. May it immediately promise application-level persistence?

Worked solution

Not in general. DMA completion can mean that the controller consumed the host buffer, while data remain in volatile controller or device caches. The storage and file-system contracts determine whether a flush or other persistence command is required.

T.4 - Queue publication

A driver updates an MMIO doorbell before all descriptor stores are globally visible. Give the possible failure and the remedy.

Worked solution

The controller may fetch an old or partially initialised descriptor, producing the wrong address, length or operation. Obtain valid DMA mappings and fill the descriptor using their addresses. Apply the DMA API's ownership and cache-synchronisation operations where required, then the prescribed ordering barrier, and finally ring the doorbell with the MMIO accessor. A barrier alone does not clean a dirty cache line on a noncoherent platform.

T.5 - End-to-end bottleneck

A 4 KiB random disk read spends 8 ms seeking, 4 ms rotating, 0.03 ms transferring, 0.02 ms in interrupt handling and 0.2 ms waiting to be scheduled. Which optimisation can at most save 0.02 ms, and why is it secondary?

Worked solution

Adding the terms gives $8 + 4 + 0.03 + 0.02 + 0.2 = 12.25$ ms. Removing all interrupt-handler time would leave 12.23 ms, a saving of $0.02/12.25 \approx 0.16\%$. Seek and rotation together occupy 12 ms, about 98% of this request. Avoiding one positioning operation offers much more scope for improvement.

B.9 PROCESS AND CONCURRENCY QUICK REFERENCE

Concept	Operational meaning
Ready	Eligible to run, waiting only for a processor.
Blocked	Waiting for an event, resource or condition; not eligible to run.
`fork`	Create a child process with a logically copied address space and two return paths.
`exec`	Replace the calling process image; does not create a new PID.
`wait/waitpid`	Collect child state and termination status.
Binary semaphore	Mutual-exclusion or event gate with at most one available permit; it is not automatically an ownership-checking mutex.
Counting semaphore	Atomic count of available events or resource units.
Deadlock prevention	Constrain requests so at least one Coffman condition cannot hold.

Scheduling formulas.

For a simple single-CPU burst model,

$$T^{turn} = T^{completion} - T^{arrival}, \qquad T^{wait} = T^{turn} - T^{CPU}.$$

Response time ends at the first dispatch, not at completion.

Bounded-buffer initialisation.

For capacity N: `empty`=N, `full`=0, `mutex`=1. Reserve the required condition before acquiring `mutex`; never block on `empty` or `full` while holding the buffer mutex.

Coffman conditions.

Mutual exclusion; hold and wait; no preemption; circular wait. Breaking any one prevents resource deadlock of this form.

B.10 COMPACT GLOSSARY

The glossary is grouped by Part. Each definition uses the abstraction established in that Part; the corresponding chapter supplies qualifications, derivations and exceptions.

Part I: Representation and Program Execution

ABI (application binary interface)

Binary-level conventions for calls, registers, object layout and linking that allow separately compiled components to cooperate.

Address

A numerical identifier for a location in an address space; memory is commonly byte-addressed.

Application

Software that performs a user-facing task using services supplied by system software.

ASCII

A 7-bit character code containing English letters, digits, punctuation and control characters.

Bias

A fixed offset used to store signed exponents as unsigned fields in floating-point formats.

Bit

A binary digit with abstract value 0 or 1.

Bit mask

A bit vector whose 1 positions select fields or flags for testing or modification.

Byte

The smallest addressable unit in the systems considered here; 8 bits.

Byte order / endianness

The order in which the bytes of a multibyte scalar value are stored at increasing addresses.

Carry

A bit propagated beyond a position during addition; final carry is relevant to unsigned overflow but is not the signed-overflow condition.

Code point

An integer in the Unicode codespace, from U+0000 through U+10FFFF. Some code points are unassigned or reserved, including the surrogate range.

Combinational circuit

A circuit whose outputs are functions only of current inputs, apart from physical propagation delay.

Compiler

A program that translates a source representation into a lower-level representation, commonly assembly or machine-oriented intermediate code.

Context switch

Saving one thread's execution context and restoring another's so the CPU can change which thread runs, within one process or across processes.

Encoding

A rule mapping abstract objects or coded values to a representation and defining the inverse interpretation.

File

In the Unix programming model, a byte sequence accessed through a uniform I/O interface; devices and pipes participate in the same abstraction.

Filesystem

The operating-system subsystem and on-storage structures that organise files into namespaces, directories and metadata.

Fixed width

A representation that retains a predetermined number of bits; values outside its range require truncation, wrapping, saturation or rejection according to the contract.

Floating point

A finite representation storing sign, significand information and exponent so that precision scales approximately with magnitude.

Gate

A physical or abstract component implementing a small Boolean function.

Hexadecimal

Base-16 positional notation; one digit corresponds exactly to four bits.

IEEE 754

The principal standard defining binary and decimal floating-point formats, operations, rounding and special values.

Kernel

The privileged core of an operating system that manages hardware resources and implements protected services.

Linker

A program that combines object files, resolves symbols and relocates references to form an executable or shared object.

NaN

IEEE floating-point "not a number" value used for invalid numerical results and propagated through many operations; unordered in comparisons.

Nibble

Four bits, corresponding to one hexadecimal digit.

Object file

A binary container holding machine code/data plus symbols and relocation metadata; a relocatable object is not normally directly executable.

Overflow

A condition in which the mathematical result lies outside a representation's range. Unsigned C arithmetic wraps by definition; signed C overflow is undefined behaviour.

Process

The operating system's abstraction of a running program, including execution context, virtual address space and resource references.

Radix / base

The number of digit values in a positional numeral system.

Sign extension

Widening a two's-complement value by replicating its sign bit, preserving its signed value.

Subnormal

A nonzero finite binary floating-point value whose magnitude is smaller than the smallest positive normal value. In binary32 and binary64 its exponent field is zero and its fraction field is nonzero.

System call

A controlled request by a process for a kernel service.

Two's complement

The dominant fixed-width signed-integer encoding, representing $-x$ as $2^w - x$ modulo 2^w.

Unicode

A universal character repertoire assigning code points independently of a particular byte encoding.

UTF-8

A variable-length Unicode encoding using one to four bytes and preserving ASCII byte values.

Virtual memory

The abstraction giving each process a protected virtual address space whose pages are mapped to physical memory or backing storage.

Part II: Machine-Level Programming

ABI (Application Binary Interface)

Machine-level contract for calling conventions, data layout, object formats and interaction among separately compiled components.

Address

Integer-like value that identifies a byte location in a process virtual address space.

Addressing mode

Rule used by an instruction to compute an operand address, commonly base plus index times scale plus displacement.

Alias

Two expressions or pointers that can refer to overlapping storage.

ASLR

Randomisation of virtual-memory mapping locations to make addresses less predictable.

Basic block

Straight-line instruction sequence with one entry and no branch except at its end.

Bad byte

Byte value excluded by a representation or transmission format, requiring an allowed encoding for the intended data.

Branch misprediction

Incorrect processor prediction of control flow, causing speculative work to be discarded.

Cache

Small fast storage that retains copies of recently or nearby used memory blocks.

Calling convention

Rules for arguments, returns, register preservation, stack alignment and calls.

Canonical address

x86-64 virtual address whose unused high bits have the architecturally required sign extension.

Condition codes

Flag bits such as CF, OF, ZF and SF that record selected properties of an arithmetic or logical result.

CPI

Average clock cycles per retired machine instruction over a workload.

Disassembly

Decoding machine-code bytes into assembly mnemonics and operands.

Effective address

Address computed by an instruction before a memory access, or returned as arithmetic by `lea`.

Endianness

Order in which bytes of a multibyte value are stored. x86-64 uses little-endian order.

File descriptor

Small process-local integer naming an open kernel object through a descriptor table.

Frame pointer

Optional stable base register, commonly `rbp`, used to reference stack-frame locations.

Immediate

Constant encoded as part of an instruction.

ISA

Instruction-set architecture: programmer-visible machine state and instruction semantics.

Leaf function

Function that makes no calls to other functions.

Little-endian

Storage convention in which the least significant byte occupies the lowest address.

Microarchitecture

Concrete processor design implementing an ISA.

NX / DEP

Policy that prevents instruction execution from pages lacking execute permission.

Object file

Relocatable container holding machine code, data, symbols and relocation records.

Operand width

Number of bits read or written by an instruction operand.

PIE

Position-independent executable, enabling relocation of the main program image.

Red zone

128-byte area below `rsp` available to System V AMD64 leaf functions under ABI conditions.

Relocation

Linker record describing an address-dependent field that must be adjusted when symbol locations become known.

ROP

Return-oriented programming, a control-flow reuse technique that chains existing instruction sequences.

Stack canary
Secret value checked before return to detect many sequential stack-frame overwrites.

Syscall
Controlled request by user code for a kernel service.

Two's complement
Standard fixed-width signed-integer encoding used by x86-64.

Virtual address
Process-visible address translated and protected by the memory-management system.

Zero extension
Filling newly introduced high bits with zero when increasing an unsigned value's width.

Part III: Processor Organisation

Architectural state
State visible to machine-language programs: registers, PC, condition codes, memory and status.

Bubble
An intentionally empty pipeline entry that performs no architectural work.

Bypass / forwarding
Directly routing a produced value to a consumer before register-file write-back.

Clock skew
Difference in clock-arrival time at different state elements.

Combinational circuit
Circuit whose outputs are functions only of current inputs.

Commit
Make an instruction's result part of architectural state.

Condition code
One-bit summary of an arithmetic result, such as zero, sign or overflow.

Control hazard
Uncertainty about the next instruction address.

Critical path
Longest relevant combinational delay between state elements.

Data hazard

Timing conflict caused by an instruction needing a value produced by another.

Datapath

Hardware blocks and connections that store, move and transform data.

Decode

Stage that interprets register fields and selects operand sources/destinations.

Edge-triggered

Capturing state on a clock transition rather than throughout a level.

Execute

Stage containing ALU work, effective-address calculation and condition evaluation.

Fan-out

Number of gate inputs driven by one output; high fan-out can increase delay.

Fetch

Stage that reads instruction bytes and computes the sequential next address.

Flip-flop

Edge-triggered one-bit storage element.

Glitch

Short unwanted output transition caused by unequal combinational path delays.

Hazard

Condition that makes naive pipeline overlap incorrect or impossible.

HCL

A compact pedagogical notation for Boolean and word-level control functions in the Y86-64 datapaths.

Hold time

Interval after the clock edge during which a sampled input must remain stable.

Instruction-set architecture

Programmer-visible contract defining instructions and state transitions.

Interlock

Hardware detection and stalling of a hazard.

Latency

Time from starting an operation until its result is complete.

Latch

Level-sensitive storage element.

Memory stage

Pipeline stage that reads or writes data memory.

Metastability

Analogue state that can arise when a flip-flop's timing requirements are violated.

Microarchitecture

Concrete processor organisation implementing an ISA.

Minterm

An AND of one literal for each input variable. It evaluates to one for precisely one input row. The canonical sum of products selects the minterms of the rows where the function is one.

Multiplexer

Circuit selecting one data input according to control bits.

Pipeline register

Clocked boundary that carries one instruction's intermediate values to the next stage.

Precise exception

Exception state in which all older instructions and no younger instructions have committed.

Propagation delay

Time for an input change to affect a circuit output.

RAW dependency

Read-after-write dependence: a consumer reads a value produced by an older instruction.

Register file

Storage structure that supplies register operands; a teaching datapath may model the architectural register set directly, while a production microarchitecture can use a larger physical register file.

Sequential circuit

Circuit whose behaviour depends on current input and stored state.

Setup time

Interval before a clock edge during which a sampled input must be stable.

Speculation

Performing work before it is certain to belong to the correct architectural path.

Stall

Hold a pipeline register unchanged for a cycle.

Structural hazard

Conflict caused by simultaneous demand for one hardware resource.

Throughput

Number of completed operations per unit time in steady state.

Write-back

Stage that updates the register file.

Part IV: Exceptional Control Flow, I/O, and Storage

Abort

Processor exception for which a precise, reliable restart is not generally available; a machine check need not identify a single causative instruction.

APIC

Programmable interrupt architecture with I/O routing and per-logical-processor local controllers.

Arbitration

Policy that chooses which requesting bus master receives access.

Block device

Device interface based on independently addressable fixed-size blocks.

Bus

Interconnection and protocol carrying address, data and control transactions.

Bus master

Component permitted to initiate transactions rather than only respond.

Busy waiting

Repeatedly testing a condition while performing no other useful work.

Character device

Stream- or record-oriented device interface.

Coherence

Property ensuring participants observe compatible values for shared memory locations.

Controller

Hardware that translates between a device protocol and CPU-visible registers or queues.

CPL

Current Privilege Level in the x86 protection model.

Cycle stealing

DMA strategy that acquires individual bus opportunities between other masters' accesses.

Descriptor

Memory or table record describing a handler, buffer or I/O operation.

Device driver

Software that implements an operating-system interface to a controller; it may run in the kernel or through a controlled user-space device interface.

DMA

Direct Memory Access: a device or DMA engine transfers memory data as a bus master.

Doorbell

Controller register written to notify a device that new descriptors are available.

Error code

Value pushed or returned to describe the cause of an exception or failed service.

Exception

Architecturally reported event associated with instruction execution or processor state; on x86, faults, traps and aborts are exception classes. Some machine checks are asynchronous to the causative operation.

Fault

Synchronous exception normally reported at the faulting instruction, permitting retry when the condition can be repaired.

FTL

Flash Translation Layer mapping logical block addresses to physical flash pages.

Garbage collection

SSD process that copies live pages and erases blocks to recover free space.

Handler

Privileged routine entered in response to a processor exception, an external interrupt or another protected-entry mechanism.

IDT

Interrupt Descriptor Table mapping x86 vectors to gate descriptors.

IOMMU

Memory-management unit that translates and protects device DMA accesses.

IOPS

Input/output operations per second; meaningful only with workload details.

IRQ

Interrupt request identifier or line associated with a device event.

IST

Interrupt Stack Table mechanism selecting dedicated x86-64 handler stacks.

Interrupt

Asynchronous external event that requests processor attention at an architecturally permitted boundary.

Interrupt affinity

Policy assigning an interrupt vector to one or more logical processors.

Interrupt moderation

Delaying or batching notifications to reduce entry overhead.

Kernel mode

Privileged execution mode used by the operating-system kernel.

Latency

Time from request or event to response or completion.

Logical block address

Linear block number presented by a storage device.

Memory-mapped I/O

Controller registers placed in the processor's physical address map.

MMIO barrier

Ordering operation used to enforce a device protocol across memory and register accesses.

MSI/MSI-X

Message-signalled interrupt mechanisms used by PCI Express devices.

Over-provisioning
Flash capacity reserved for mapping, replacement and garbage collection.

Page
Unit of flash reading/programming and, separately, unit of virtual-memory mapping.

Parity
Redundant value, often XOR-based, used to reconstruct missing array data.

PIO
Programmed I/O in which CPU instructions move the payload.

Polling
Periodic or continuous reading of device status to detect completion.

Posted write
Write accepted by an intermediate component before reaching the final target.

Precise exception
Exception whose visible state corresponds to a well-defined instruction boundary.

Privilege level
Hardware-enforced authority level controlling instructions and memory access.

RAID
Organisation of multiple drives for performance and/or redundancy.

Read-ahead
Prefetching likely future storage data before explicit application requests.

Ring buffer
Circular descriptor or data queue shared by producer and consumer.

Rotational latency
Waiting for the required disk sector to rotate under the head.

Scatter–gather
DMA over a list of non-contiguous memory segments.

Seek time
Time for a disk actuator to position the head over the required track.

Short count
Successful I/O result transferring fewer bytes than requested.

System call

Controlled request from an application to a kernel service.

System-call ABI

Register and entry convention used by binaries to invoke kernel services.

Throughput

Work or data completed per unit time.

Trap

Synchronous processor exception normally reported after the triggering instruction, allowing continuation at the following instruction when the architecture and handler permit it.

TRIM/discard

Notification that specified SSD logical blocks no longer hold live data.

User mode

Restricted execution mode for ordinary applications.

Vector

Small integer selecting an exception or interrupt descriptor.

Wear levelling

SSD policy distributing erase cycles across flash blocks.

Write amplification

Ratio of physical flash programming to host-requested writes.

Write-back cache

Buffer that acknowledges writes before their final persistent placement.

Part V: Processes and Concurrency

Context switch

The save-and-restore operation that suspends one execution context and activates another.

Deadlock

A state in which each member of a set waits for an event that only another member of the set can cause.

Data race

Potentially concurrent conflicting accesses to the same memory location, at least one non-atomic and at least one a write, with neither ordered before the other by the language memory model's happens-before relation.

Livelock

Continued state changes without useful progress.

Process

A protected execution context containing processor state, an address space, resources and identity.

Process control block (PCB)

Kernel metadata used to identify, schedule, protect, suspend and resume a process.

Race condition

A correctness failure in which the result depends on an uncontrolled concurrent interleaving.

Semaphore

An atomic synchronisation object used to count permits, block waiters and wake participants.

Starvation

Indefinite postponement while other participants continue to make progress.

Thread

An execution sequence with its own program counter, registers and stack inside a process; depending on the implementation, it is scheduled by the kernel or by a user-level runtime.

Part VI: Memory, Filesystems and Protection

Access-control list (ACL)

Permissions associated with an object. Its evaluation rules determine which subjects can perform each operation.

AMAT

Average memory access time. With a hit cost paid on every request and an additional miss penalty, the mean is hit cost plus miss probability times penalty.

Audit record

Recorded evidence of an event, including its subject, object, operation and outcome under the stated logging contract.

Cache line

An aligned block of memory retained and transferred as a unit by a cache.

Checkpoint

In the redo model, a durable boundary establishing which logged changes have reached their home locations, permitting corresponding log space to be reused.

Dirty page

A resident page with changes that must be preserved before its frame can be reused, unless those contents may be discarded.

Extent

A contiguous run of allocation units represented by a starting address and a length.

Frame

A unit of physical memory that can hold one virtual page of the corresponding size.

Hard link

A directory entry naming an existing filesystem object. Several names can refer to the same inode.

Inode

A filesystem record identifying an object and describing its metadata and data mapping.

Locality

A tendency to reuse recently accessed information or to access nearby locations.

Page-table entry (PTE)

An encoded record used in translation, interpreted according to its level, format, state and permissions.

Redo

Recovery that reinstalls committed updates from durable records, following the protocol's prescribed order.

Symbolic link

An object containing a pathname that is resolved when the link is followed.

Thrashing

Repeated displacement and refetching of actively needed pages, leaving little time for useful computation.

TLB

Translation lookaside buffer. It caches address translations and associated attributes in the applicable address-space context.

Working set

The distinct pages referenced during a specified window of execution.

C

BUILDING AND RUNNING THE EXAMPLES

This appendix gives complete programs for the object-layout and translation models used in the book. The target is Linux on x86-64, with a C compiler, GNU assembler and the System V AMD64 calling convention. The commands use `cc`; a GNU GCC installation can also use `gcc`. Python examples elsewhere in the book specify Python 3. Each source file below can be transcribed directly from the listing; the editable edition also contains the files under `examples/`.

C.1 FROM A CALCULATION FRAGMENT TO A FUNCTION

An assembly calculation specifies initial registers, readable memory, scratch registers and result registers. A callable function adds entry and return conventions. For the System V AMD64 ABI, the first three integer or pointer arguments arrive in `rdi`, `rsi` and `rdx`; a scalar integer result is returned in `rax`. A caller's `call` places a return address on the stack, and `ret` consumes it.

A leaf function that uses only caller-saved registers and makes no nested calls can often leave the stack untouched. If a calculation changes a callee-saved register, the surrounding function must preserve and restore it. Stack alignment, output representation and memory bounds remain part of the function's obligations. Chapter 2.6 develops these conventions in detail.

The complete source below selects GNU assembler's Intel syntax with `.intel_syntax noprefix`. In this syntax, `#` begins a line comment; NASM-style semicolon comments in a calculation should be removed or changed to `#` when transcribing it into a GNU

source. NASM directives and GNU directives are different. The source below already contains every directive needed for its stated build command.

C.2 A CHECKED TRANSLATION FUNCTION

Use 4096-byte pages and an array of eight-byte entries. A valid entry is $(\mathrm{PFN} \ll 1)|1$ with $0 \leqslant \mathrm{PFN} < 2^{40}$; bit zero clear means invalid. A successful physical-address value is below 2^{52}. The routine returns the all-ones 64-bit value on an out-of-bounds VPN, an invalid entry or an out-of-range frame. The table argument must point to `count` readable entries when `count` is nonzero. A zero count permits a null pointer because the routine rejects the index before loading memory.

Save this assembly source as `translate.S`:

```
.intel_syntax noprefix
.text
.globl translate
.type translate, @function
translate:
    mov rax, rdi
    shr rax, 12
    cmp rax, rdx
    jae .Lfail
    mov rax, [rsi + rax*8]
    test rax, 1
    jz .Lfail
    shr rax, 1
    mov r8, rax
    shr r8, 40
    jnz .Lfail
    shl rax, 12
    mov rcx, rdi
    and ecx, 4095
    or rax, rcx
    ret
.Lfail:
    mov rax, -1
    ret
.size translate, .-translate
.section .note.GNU-stack,"",@progbits
```

The first shift computes VPN. The unsigned comparison rejects VPN $\geqslant$ `count`, so the following load reads an entry within the promised allocation. The validity test occurs

before frame extraction. Shifting a copy of PFN right by 40 detects any bit outside its allowed range. The remaining shifts and mask construct the physical-address value. Both exits execute `ret`, and the function changes only caller-saved registers.

C.3 A CALLER AND ITS EXPECTED RESULTS

Save the following as `translation_demo.c`:

```c
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

extern uint64_t translate(uint64_t va, const uint64_t *table,
                          uint64_t count);

int main(void) {
    const uint64_t table[] = {7, 26, 19};
    const struct {
        uint64_t va;
        uint64_t expected;
    } cases[] = {
        {0x25,   0x3025},
        {0x1007, UINT64_MAX},
        {0x2fff, 0x9fff},
        {0x3000, UINT64_MAX},
        {0x2000, 0x9000}
    };
    for (unsigned i = 0; i < sizeof cases / sizeof cases[0]; ++i) {
        uint64_t actual = translate(cases[i].va, table, 3);
        printf("%016" PRIx64 "\n", actual);
        if (actual != cases[i].expected) return 1;
    }
    if (translate(0, NULL, 0) != UINT64_MAX) return 1;
    const uint64_t oversized[] = {(UINT64_C(1) << 41) | 1};
    if (translate(0, oversized, 1) != UINT64_MAX) return 1;
    return 0;
}
```

Compile and run from the directory containing both files:

```
cc -std=c17 -O2 -Wall -Wextra translation_demo.c translate.S -o translation_demo
./translation_demo
```

The expected output is:

```
0000000000003025
ffffffffffffffff
0000000000009fff
ffffffffffffffff
0000000000009000
```

Entry 0 equals 7, encoding valid frame 3; VA `0x25` therefore becomes `0x3025`. Entry 1 equals 26 and has a clear validity bit, so VA `0x1007` fails. Entry 2 equals 19, encoding valid frame 9; its last and first byte offsets produce `0x9FFF` and `0x9000`. VA `0x3000` selects index 3, outside a three-entry table, and is rejected before a load. The final two silent checks verify an empty table and a frame outside the declared range. Exit status zero means all listed comparisons passed.

This test establishes these paths for the supplied cases. A broader test can generate valid frames and offsets, compare against $(\text{PFN} \ll 12) | (VA \,\&\, 4095)$, and separately exercise invalid entries and index boundaries. Memory readability remains a caller obligation for the array extent passed to the function.

C.4 CHECKING STRUCTURE LAYOUT

The structure example in Chapter 2.3 assumes the System V AMD64 layout. The following program obtains the implementation's actual offsets. Save it as `layout.c`:

```c
#include <stddef.h>
#include <stdio.h>

struct record {
    char tag;
    int count;
    short code;
    void *next;
};

int main(void) {
    printf("size=%zu align=%zu\n", sizeof(struct record),
           _Alignof(struct record));
    printf("tag=%zu count=%zu code=%zu next=%zu\n",
           offsetof(struct record, tag),
           offsetof(struct record, count),
           offsetof(struct record, code),
           offsetof(struct record, next));
    return 0;
}
```

Compile with `cc -std=c17 -Wall -Wextra layout.c -o layout`, then run `./layout`. On the stated target the output is:

```
size=24 align=8
tag=0 count=4 code=8 next=16
```

Compare each offset with the alignment calculation before comparing the total size. A different target ABI can produce a different layout; the program reports that layout directly. It does not define a portable file format.

C.5 THREADS, INSPECTION AND MODEL TRACES

Chapter 5.5 supplies the complete `counter.c` program and its compilation command. The expected result follows from exclusion around each increment and joining both workers before the final read. Timing that program would answer an additional performance question and would require a stated workload and measurement method.

For assembly inspection, `objdump -d -Mintel translation_demo` shows the linked instructions. A debugger can stop at `translate`, inspect its arguments and step through the bounds and validity branches. The expected values above provide checkpoints independent of the debugger's display format.

Y86-64, SEQ, PIPE, Boolean circuits and filesystem crash models are defined through state and transition rules in their chapters. Their worked traces can be reproduced by applying one transition at a time to the stated initial state. The initial state and transition rules specify every input needed to reproduce the trace.

REFERENCES AND FURTHER READING

Source policy. Specifications and architecture manuals define the contracts on which the technical examples rely. The historical notes follow a documented source hierarchy: original manuals, standards and contemporary papers are preferred; institutional archives maintained by manufacturers, universities, museums and public agencies are used for chronology and artefacts; peer-reviewed historical research is used to resolve attribution; textbooks supply synthesis but do not replace primary evidence. Dates attached to evolving manuals and web documentation are consultation dates, not publication dates unless explicitly stated.

[1] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, 2016. Pedagogical reference for the programmer-centred treatment of representation, machine code, processor organisation, performance, linking, exceptional control flow, virtual memory, I/O and concurrency; also the acknowledged provenance of the Y86-64, SEQ and PIPE teaching models.

[2] ISO/IEC JTC1/SC22/WG14, *Programming Languages – C*, Committee Draft N1570, 2011, https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf.

[3] ISO/IEC JTC1/SC22/WG14, *Programming Languages – C*, Committee Draft N3096, 2023, https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf; see also ISO/IEC 9899:2024.

[4] IEEE and The Open Group, *POSIX.1-2024 / The Open Group Base Specifications, Issue 8*, 2024, https://pubs.opengroup.org/onlinepubs/9799919799/.

[5] The Unicode Consortium, *The Unicode Standard, Version 17.0.0*, 2025, https://www.unicode.org/versions/Unicode17.0.0/.

[6] F. Yergeau, *UTF-8, a Transformation Format of ISO 10646*, RFC 3629, IETF, 2003.

[7] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019, https://standards.ieee.org/ieee/754/6210/.

[8] National Institute of Standards and Technology, *Recommendation for the Entropy Sources Used for Random Bit Generation*, NIST SP 800-90B, 2018, https://doi.org/10.6028/NIST.SP.800-90B.

[9] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai and O. Mutlu, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," *Proceedings of the 41st International Symposium on Computer Architecture*, 2014, pp. 361–372, doi:10.1109/ISCA.2014.6853210.

[10] R. Pike and K. Thompson, "Hello World" (multilingual title), *Proceedings of the Winter 1993 USENIX Conference*, 1993, pp. 43–50.

[11] Free Software Foundation, *Debugging with GDB*, GDB 17.2 documentation, 2026, https://sourceware.org/gdb/current/onlinedocs/gdb.html.

[12] Free Software Foundation, *GNU Binutils Documentation*, Version 2.46, 2026, https://sourceware.org/binutils/docs/.

[13] Aleph One, "Smashing the Stack for Fun and Profit," *Phrack*, 7(49), Article 14, 1996.

[14] C. Cowan, C. Pu, D. Maier, H. Hinton, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle and Q. Zhang, "StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks," *Proceedings of the 7th USENIX Security Symposium*, 1998.

[15] M. V. Wilkes, "The Best Way to Design an Automatic Calculating Machine," *Proceedings of the Manchester University Computer Inaugural Conference*, 1951, pp. 16–18.

[16] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse and R. Panigrahy, "Design Tradeoffs for SSD Performance," *Proceedings of the USENIX Annual Technical Conference*, 2008, pp. 57–70.

[17] D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, 23(1), 1991, pp. 5–48.

[18] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988.

[19] D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, 17(7), 1974, pp. 365–375.

[20] G. Boole, *An Investigation of the Laws of Thought*, Walton and Maberly, 1854.

[21] C. E. Shannon, “A Symbolic Analysis of Relay and Switching Circuits,” *Transactions of the AIEE*, 57(12), 1938, pp. 713–723. https://doi.org/10.1109/T-AIEE.1938.5057767.

[22] J. von Neumann, *First Draft of a Report on the EDVAC*, 1945.

[23] G. E. Moore, “Cramming More Components onto Integrated Circuits,” *Electronics*, 38(8), 1965.

[24] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” AFIPS Spring Joint Computer Conference, 1967.

[25] P. J. Denning, “The Working Set Model for Program Behavior,” *Communications of the ACM*, 11(5), 1968, pp. 323–333. https://denninginstitute.com/pjd/PUBS/WSModel_1968.pdf.

[26] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer’s Manual*, Combined Volumes 1, 2A–2D, 3A–3D and 4, official combined online edition consulted 13 July 2026, https://cdrdv2-public.intel.com/825743/325462-sdm-vol-1-2abcd-3abcd-4.pdf.

[27] Advanced Micro Devices, *AMD64 Architecture Programmer’s Manual*, Vols. 1–5, Publication 40332, Revision 4.09, 9 March 2026, https://docs.amd.com/v/u/en-US/40332_4.09_APM_PUB.

[28] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Volume 1, Order No. 248966-050, online edition consulted 13 July 2026, https://cdrdv2-public.intel.com/821612/248966-Optimization-Reference-Manual-V1-050.pdf.

[29] Advanced Micro Devices, *Software Optimization Guide for the AMD Zen 5 Microarchitecture*, Publication 58455, Revision 1.00, 15 August 2024, https://docs.amd.com/v/u/en-US/58455_1.00.

[30] x86 psABIs project, *System V Application Binary Interface: AMD64 Architecture Processor Supplement*, living specification, canonical repository consulted 13 July 2026, https://gitlab.com/x86-psABIs/x86-64-ABI.

[31] TIS Committee, *Tool Interface Standard: Executable and Linking Format (ELF), Version 1.2*, 1995; generic ABI reference consulted 11 July 2026, https://gabi.xinuos.com/.

[32] Linux man-pages project, documentation of Linux kernel and C-library interfaces, online edition consulted 11 July 2026, https://www.kernel.org/doc/man-pages/.

[33] Linux kernel documentation, "Kernel Entries" for x86-64, current online documentation consulted 13 July 2026, https://docs.kernel.org/arch/x86/entry_64.html.

[34] Linux kernel documentation, "Entry/exit handling for exceptions, interrupts, syscalls and KVM," current online documentation consulted 13 July 2026, https://docs.kernel.org/core-api/entry.html.

[35] L. Torvalds and Linux kernel contributors, *Linux kernel source tree*, release tag v7.1, especially `arch/x86/entry/syscall_64.c`, `arch/x86/entry/entry_64.S`, `arch/x86/entry/syscalls/syscall_64.tbl`, `fs/binfmt_elf.c`, `drivers/nvme/host/pci.c` and `drivers/nvme/host/core.c`; named paths rechecked 5 November 2026, https://github.com/torvalds/linux/tree/v7.1.

[36] Linux kernel contributors, *VFIO – "Virtual Function I/O"*, kernel documentation for controlled user-space device access and IOMMU isolation, consulted 13 July 2026, https://docs.kernel.org/driver-api/vfio.html.

[37] Linux kernel documentation, "Control-flow Enforcement Technology (CET) Shadow Stack," current x86 documentation consulted 13 July 2026, https://docs.kernel.org/arch/x86/shstk.html.

[38] Linux kernel documentation, "Dynamic DMA mapping using the generic device," current online documentation consulted 13 July 2026, https://docs.kernel.org/core-api/dma-api.html.

[39] Linux kernel documentation, "Dynamic DMA mapping Guide," current online documentation consulted 13 July 2026, https://docs.kernel.org/core-api/dma-api-howto.html.

[40] Linux kernel documentation, *Linux Kernel Memory Barriers*, current online documentation consulted 13 July 2026, https://docs.kernel.org/core-api/wrappers/memory-barriers.html.

[41] J. M. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Prentice Hall, 2003.

[42] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Addison-Wesley, 2011.

[43] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed., Cambridge University Press, 2015.

[44] S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices*, 3rd ed., Wiley, 2006.

[45] H. W. Johnson and M. Graham, *High-Speed Digital Design: A Handbook of Black Magic*, Prentice Hall, 1993.

[46] S. H. Hall and H. L. Heck, *Advanced Signal Integrity for High-Speed Digital Designs*, Wiley, 2009.

[47] B. Jacob, S. Ng and D. Wang, *Memory Systems: Cache, DRAM, Disk*, Morgan Kaufmann, 2008.

[48] R. Micheloni, L. Crippa and A. Marelli, *Inside NAND Flash Memories*, Springer, 2010.

[49] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed., Morgan Kaufmann, 2020.

[50] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.

[51] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, 1st ed., Morgan Kaufmann, 2021.

[52] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed., Pearson, 2018.

[53] D. A. Patterson and D. R. Ditzel, "The Case for the Reduced Instruction Set Computer," *ACM SIGARCH Computer Architecture News*, 8(6), 1980, pp. 25–33.

[54] J. Cocke and V. Markstein, "The Evolution of RISC Technology at IBM," *IBM Journal of Research and Development*, 34(1), 1990, pp. 4–11.

[55] D. A. Patterson, G. Gibson and R. H. Katz, "A Case for Redundant Arrays of Inexpensive Disks (RAID)," ACM SIGMOD Conference, 1988, pp. 109–116.

[56] NVM Express, Inc., *NVM Express Base Specification*, Revision 2.3, 2025, https://nvmexpress.org/wp-content/uploads/NVM-Express-Base-Specification-Revision-2.3-2025.08.01-Ratified.pdf.

[57] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Version 1.10, Arpaci-Dusseau Books, 2023, https://pages.cs.wisc.edu/~remzi/OSTEP/.

[58] M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.

[59] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed., Pearson, 2023. Pedagogical reference for operating-system abstractions, processes and scheduling, memory policy, filesystems, I/O layers, deadlocks, virtualisation, multiprocessors, protection and system design.

[60] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, 63(9), 1975, pp. 1278–1308.

[61] V. G. Cerf, *ASCII Format for Network Interchange*, RFC 20, October 1969, DOI: 10.17487/RFC0020, https://www.rfc-editor.org/info/rfc20/.

[62] International Electrotechnical Commission, *IEC 80000-13:2025, Quantities and Units – Part 13: Information Science and Technology*, 2nd ed., 2025, https://webstore.iec.ch/en/publication/87379.

[63] G. A. Blaauw and F. P. Brooks Jr., "The Structure of SYSTEM/360, Part I: Outline of the Logical Structure," *IBM Systems Journal*, 3(2), 1964, pp. 119–135.

[64] H. Shacham, "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)," *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007, pp. 552–561, DOI: 10.1145/1315245.1315313.

[65] U.S. General Accounting Office, *Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia*, Report IMTEC-92-26, 1992, https://www.gao.gov/products/imtec-92-26.

[66] Ariane 501 Inquiry Board, *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*, European Space Agency and CNES, 1996, https://sci.esa.int/web/cluster/-/38889-ariane-501-report-by-board-of-inquiry.

[67] P. E. Ceruzzi, *Computing: A Concise History*, MIT Press, 2012.

[68] M. Campbell-Kelly, W. Aspray, N. Ensmenger and J. R. Yost, *Computer: A History of the Information Machine*, 3rd ed., Westview Press, 2013.

[69] Computer History Museum, "The Stored Program," *Revolution: The First 2000 Years of Computing*, online historical exhibit consulted 13 July 2026, https://www.computerhistory.org/revolution/birth-of-the-computer/4/87.

[70] Computer History Museum, "EDSAC," historical exhibit and timeline materials consulted 13 July 2026, https://www.computerhistory.org/revolution/birth-of-the-computer/4/95.

[71] Computer History Museum, "Microprocessor Integrates CPU Function onto a Single Chip," *The Silicon Engine*, online historical exhibit consulted 13 July 2026, official exhibit.

[72] Computer History Museum, *Memory and Storage Timeline*, online historical exhibit consulted 13 July 2026, https://www.computerhistory.org/timeline/memory-storage/.

[73] Computer History Museum, "MOS Dynamic RAM Competes with Magnetic Core Memory on Price," *The Silicon Engine*, online historical exhibit consulted 13 July 2026, official exhibit.

[74] IBM, “The IBM System/360,” IBM History, online historical account consulted 13 July 2026, https://www.ibm.com/history/system-360.

[75] IBM, “RAMAC,” IBM History, online historical account consulted 13 July 2026, https://www.ibm.com/history/ramac.

[76] IBM, “The IBM 7030, aka Stretch,” IBM History, online historical account consulted 13 July 2026, https://www.ibm.com/history/stretch.

[77] IEEE, *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985, 1985; superseded by IEEE Std 754-2019, https://standards.ieee.org/ieee/754/993/.

[78] The Unicode Consortium, *Unicode History Corner*, online historical materials consulted 13 July 2026, https://www.unicode.org/history/.

[79] Computer History Museum, “David John Wheeler,” pioneer profile consulted 13 July 2026, https://computerhistory.org/profile/david-john-wheeler/.

[80] PCI-SIG, *PCI Express Base Specification, Revision 2.1*, revision history, 4 March 2009, specification hosted by Intel.

[81] PCI-SIG, *PCI Express Base Specification Revision 7.0*, released 11 June 2025, official specification page consulted 13 July 2026, https://pcisig.com/PCIExpress/Spec/Base/_7.0.

[82] Compute Express Link Consortium, *Compute Express Link Specification, Revision 4.0*, announced 18 November 2025, official release notice consulted 5 November 2026, release announcement.

[83] NVM Express, Inc., *NVMe Set of Specifications*, including Base Specification Revision 2.4 ratified 31 July 2026, official specification index consulted 5 November 2026, https://nvmexpress.org/specifications/.

[84] Seagate Technology, “Seagate Delivers Industry’s Highest Capacity Hard Drives with Next-Generation Mozaic 4+,” 2026, official announcement consulted 13 July 2026, official announcement.

[85] G. W. Leibniz, “Explication de l’arithmétique binaire,” *Mémoires de l’Académie Royale des Sciences*, 1703, pp. 85–89; digitised primary text hosted by the Università degli Studi di Bari, facsimile.

[86] Computer History Museum, *The Babbage Engine: History and Analytical Engine*, institutional exhibit consulted 13 July 2026, official exhibit.

[87] Computer History Museum, “ENIAC,” *Revolution: The First 2000 Years of Computing*, institutional exhibit consulted 13 July 2026, official exhibit.

[88] Computer History Museum, “Konrad Zuse finishes the Z3 computer,” *Timeline of Computer History: 1941*, consulted 13 July 2026, official timeline.

[89] University of Manchester, “The Manchester Small Scale Experimental Machine – The Baby,” institutional archive consulted 13 July 2026, official archive.

[90] University of Manchester, “The Manchester Mark 1,” institutional archive consulted 13 July 2026, official archive.

[91] University of Cambridge Computer Laboratory, “EDSAC and the Cambridge Computer Laboratory,” institutional history consulted 13 July 2026, official archive.

[92] University of Cambridge Computer Laboratory, “Microprogram control matrix, EDSAC II,” Relics Project object CL-70/99, consulted 5 November 2026, institutional collection record.

[93] Computer History Museum, “Sir Maurice V. Wilkes,” pioneer profile consulted 13 July 2026, official profile.

[94] University of Manchester, *The Atlas Computer*, institutional archive consulted 13 July 2026, official archive.

[95] Computer History Museum, “EDSAC Computer Employs Delay-Line Storage,” *The Storage Engine*, consulted 13 July 2026, official exhibit.

[96] Computer History Museum, “Williams Demonstrates CRT Storage,” *The Storage Engine*, consulted 13 July 2026, official exhibit.

[97] Computer History Museum, “Whirlwind Computer Debuts Core Memory,” *The Storage Engine*, consulted 13 July 2026, official exhibit.

[98] Computer History Museum, *The Storage Engine: Timeline*, institutional exhibit consulted 13 July 2026, official timeline.

[99] Computer History Museum, “Storage Subsystems Emerge,” *The Storage Engine*, institutional exhibit consulted 13 July 2026, official exhibit.

[100] Computer History Museum, “The Integrated Circuit,” *Revolution*, institutional exhibit consulted 13 July 2026, official exhibit.

[101] Computer History Museum, “Practical Monolithic Integrated Circuit Concept Patented,” *The Silicon Engine*, consulted 13 July 2026, official exhibit.

[102] Texas Instruments, *SN54LS181 / SN74LS181 Arithmetic Logic Units / Function Generators*, manufacturer data sheet, undated online edition consulted 13 July 2026, official data sheet.

[103] Intel Corporation, “The First Programmable Microprocessor: The 4004,” Intel History, consulted 13 July 2026, official timeline.

[104] Intel Corporation, “The Beginning of a Legend: The 8086,” Intel History, consulted 13 July 2026, official timeline.

[105] Intel Corporation, *1980 Annual Report*, contemporary corporate report documenting the 8087 numeric data processor, official archive.

[106] Intel Corporation, *Microprocessor Quick Reference Guide*, historical introduction dates and processor summaries, consulted 13 July 2026, official archive.

[107] Advanced Micro Devices, “AMD Ushers in Era of Cinematic Computing with the AMD Athlon 64 FX Processor,” launch announcement, 23 September 2003, official filing archive.

[108] Intel Corporation, *Intel Control-flow Enforcement Technology Answers Call to Protect Against Common Malware Threats*, 2020; notes publication of the CET specification in 2016, official paper.

[109] International Business Machines Corporation, *IBM System/360 Principles of Operation*, Publication A22-6821-1, 1964; digitised by the Computer History Museum, manual.

[110] International Business Machines Corporation, *IBM 704 Electronic Data-Processing Machine: Manual of Operation*, historical manual catalogue record, undated catalogue entry consulted 13 July 2026, Computer History Museum, institutional record.

[111] International Business Machines Corporation, *IBM 709 Data Processing System Reference Manual*, Publication A22-6501-1, 1958; catalogue record, Computer History Museum, institutional record.

[112] IBM, “The IBM 650,” IBM History, consulted 13 July 2026, official history.

[113] IBM, “The IBM System/370,” IBM History, consulted 13 July 2026, official history.

[114] IBM, “How Unicode Relates to Prior Standards Such as ASCII and EBCDIC,” IBM Documentation, consulted 13 July 2026, official documentation.

[115] IBM, “Magnetic Tape,” IBM History, consulted 13 July 2026, official history.

[116] IBM, “Dynamic Random-Access Memory,” IBM History, consulted 13 July 2026, official history.

[117] IBM, “Time-sharing,” IBM History, consulted 13 July 2026, official history.

[118] Digital Equipment Corporation, *PDP-5 Handbook*, 1964, digitised by the Computer History Museum, manual.

[119] Computer History Museum, "CDC 6600," institutional timeline and collection materials, consulted 13 July 2026, official exhibit.

[120] J. E. Thornton, *Design of a Computer: The Control Data 6600*, Scott, Foresman, 1970; digitised by the Computer History Museum, book.

[121] Computer History Museum, "Card Capacitor Read-Only Storage," *Revolution: Inside System/360*, consulted 13 July 2026, official exhibit.

[122] R. M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," *IBM Journal of Research and Development*, 11(1), 1967, pp. 25–33, doi:10.1147/rd.111.0025.

[123] National Museum of American History, Smithsonian Institution, "Log Book With Computer Bug," object NMAH 334663, consulted 13 July 2026, institutional catalogue.

[124] A. L. Leiner, *System Specifications for the DYSEAC*, National Bureau of Standards Report 1951, issued 1952, doi:10.6028/NBS.RPT.1951; see also NIST, "Computer Development at the National Bureau of Standards," official publication record.

[125] Intel Corporation, *8237A High Performance Programmable DMA Controller*, manufacturer data sheet, late-1970s design; archival copy hosted by MIT PDOS, data sheet.

[126] IBM, "The IBM PC," IBM History, including the 12 August 1981 launch, consulted 13 July 2026, official history.

[127] PCI-SIG, *MSI-X Engineering Change Notice*, dated 10 June 2003, affecting PCI Local Bus Specifications 2.3 and 3.0, official specification record.

[128] J. Hadi Salim, R. Olsson and A. Kuznetsov, "Beyond Softnet," *Proceedings of the 5th Annual Linux Showcase and Conference*, 2001, conference paper.

[129] Linux kernel development community, *NAPI*, current networking documentation consulted 13 July 2026, `https://docs.kernel.org/networking/napi.html`.

[130] Linux kernel development community, *Linux Tracing Technologies Guide*, current documentation consulted 13 July 2026, `https://docs.kernel.org/trace/index.html`.

[131] Linux kernel development community, *BPF Documentation*, current documentation consulted 13 July 2026, `https://docs.kernel.org/bpf/`.

[132] Linux kernel development community, *x86 IOMMU Support*, current documentation consulted 13 July 2026, https://docs.kernel.org/arch/x86/iommu.html.

[133] Linux kernel development community, *IOMMUFD*, current userspace-API documentation consulted 13 July 2026, https://docs.kernel.org/userspace-api/iommufd.html.

[134] Linux kernel development community, *Kernel Self-Protection*, current security documentation consulted 13 July 2026, https://docs.kernel.org/security/self-protection.html.

[135] Compute Express Link Consortium, *CXL Specification Revision 4.0*, Version 1.0 evaluation copy dated 26 February 2026, official evaluation copy.

[136] Intel. *Serial ATA II Server, Networked Storage Features To Extend Serial ATA 1.0*. 25 February 2002. Records the August 2001 release of Serial ATA 1.0. https://www.intel.com/pressroom/archive/releases/2002/20020225net_b.htm.

[137] NVM Express. *NVMe/TCP: What You Need to Know About the Specification*. March 2019. Records initial RDMA/FC bindings and the November 2018 TCP binding. https://nvmexpress.org/wp-content/uploads/March-2019-NVMe-TCP-What-You-Need-to-Know-About-the-Specification.pdf.

[138] Texas Instruments, *CMOS Power Consumption and Cpd Calculation*, SCAA035B, June 1997, https://www.ti.com/lit/an/scaa035b/scaa035b.pdf.

[139] C. Wellheuser, *Metastability Performance of Clocked FIFOs*, Texas Instruments, SCZA004A, 1996, https://www.ti.com/lit/an/scza004a/scza004a.pdf.

[140] Stephen M. Nolan, Jose M. Soltero and Shreyas Rao, *Understanding and Interpreting Standard-Logic Data Sheets*, Texas Instruments application report SZZA036C, December 2002, revised June 2016. https://www.ti.com/lit/an/szza036c/szza036c.pdf.

[141] Samsung Semiconductor, "DRAM," manufacturer explanation of cell storage, refresh and memory roles, consulted 5 November 2026, https://semiconductor.samsung.com/dram/.

[142] KIOXIA, "What is Multi-level Cell Technology Realizing Larger Capacity Flash Memory?" manufacturer technical explanation, consulted 5 November 2026, https://www.kioxia.com/en-jp/rd/technology/multi-level-cell.html.

[143] KIOXIA, "What is NAND Flash Memory?" manufacturer technical explanation, consulted 5 November 2026, https://www.kioxia.com/en-jp/rd/technology/nand-flash.html.

[144] D. M. Ritchie, "The Evolution of the Unix Time-sharing System," *AT&T Bell Laboratories Technical Journal*, 63(8), part 2, October 1984, pp. 1577–1593, doi:10.1002/j.1538-7305.1984.tb00054.x; author's online reprint, https://www.nokia.com/bell-labs/about/dennis-m-ritchie/hist.html.

[145] D. M. Ritchie, "The Development of the C Language," Second History of Programming Languages conference, 1993; author's online reprint, https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html.

[146] Texas Instruments, "CPU in a 24-pin package: TI's new SN54/74181 arithmetic logic unit," advertisement, *Electronic Design*, 26 April 1970, p. 27. Archived original issue: https://www.worldradiohistory.com/Archive-Electronic-Design/1970/Electronic-Design-V18-N09-1970-0426-Minicomputers.pdf.

[147] R. E. Bryant and D. R. O'Hallaron, *HCL Descriptions of Y86-64 Processors*, CS:APP3e Web Aside ARCH:HCL, 29 December 2014. https://csapp.cs.cmu.edu/3e/waside/waside-hcl.pdf.